OPEN-TEXT EDITION · SECOND EDITION

A hands-on introduction to discrete-event & Monte Carlo simulation

Simulation Modeling

using the **Kotlin Simulation Library** (KSL)

Manuel D. Rossetti

University of Arkansas

Open Educational Resource

Free & open source

Simulation Modeling using the Kotlin Simulation Library (KSL)

2nd Edition, Version 2.5

Manuel D. Rossetti

January 2023

Table of contents

Preface **1**
Release History . . . 3
KSL Project Page . . . 5
Book Support Files . . . 5
Acknowledgments . . . 6
Intended Audience . . . 6
Organization of the Book . . . 8

About the Author **11**

1. Simulation Modeling **12**
1.1. Simulation Modeling . . . 13
1.2. Why Simulate? . . . 14
1.3. Types of Systems and Simulation Models . . . 17
1.4. Simulation: Descriptive or Prescriptive Modeling? . . . 23
1.5. Randomness in Simulation . . . 24
1.6. Simulation Languages . . . 25
1.7. Simulation Methodology . . . 27
1.8. Overview of the Kotlin Simulation Library . . . 38
1.9. Simulation as a Profession . . . 40
1.10. Exercises . . . 43

2. Modeling Randomness **46**
2.1. Random Number Generator . . . 47
2.1.1. Random Package . . . 48
2.1.2. Creating and Using Streams . . . 53

2.1.3. Common Random Numbers 58
2.1.4. Creating and Using Antithetic Streams 60
2.1.5. Frequently Asked Questions about Random Numbers 63
2.2. Random Variate Generation 65
2.2.1. Basic Random Variate Generation 65
2.2.2. Continuous and Discrete Random Variables 68
2.2.3. Creating and Using Random Variables 72
2.2.4. Functions of Random Variables 88
2.3. Probability Distribution Models 94
2.4. Distribution Fitting Using the KSL 100
2.4.1. Estimating Distribution Parameters 100
2.4.2. Continuous Distribution Recommendation Framework 108
2.4.3. Discrete Distribution Framework 128
2.4.4. Illustrative Examples from Appendix B 132
2.5. Summary 157
2.6. Exercises 159

3. Monte Carlo Methods 165
3.1. Collecting Statistics 166
3.1.1. Creating and Using a Statistic 168
3.1.2. Histograms and Frequencies 174
3.1.3. Batch Statistics 186
3.1.4. Statistics Summary 192
3.2. Simple Monte Carlo Integration 194
3.3. Review of Statistical Concepts 198
3.3.1. Point Estimates and Confidence Intervals 199
3.3.2. Sample Size Determination 202
3.3.3. Determining the Sample Size for a Monte Carlo Simulation Experiment 206
3.4. Simulating the Game of Craps 215
3.5. The News Vendor Problem 218
3.6. A Simple Inspection Process 222
3.7. Stochastic Activity Networks 225
3.8. Monte-Carlo Experiments 230

3.9. Summary 240
3.10. Exercises 242

4. Introduction to Discrete Event Modeling 258
4.1. Discrete-Event Dynamic Systems 260
4.2. How the Discrete-Event Clock Works 262
4.3. Simulating a Queueing System By Hand 268
4.4. Modeling DEDS in the KSL 283
4.4.1. Event Scheduling 284
4.4.2. Simple Event Scheduling Examples 289
4.4.3. Up and Down Component Example 300
4.4.4. Modeling a Simple Queueing System 307
4.4.5. More Details About the Pharmacy Model Implementation 322
4.5. Enhancing the Drive Through Pharmacy Model 328
4.5.1. Modeling a Simple Resource 329
4.5.2. Modeling a Queue with Statistical Collection 331
4.5.3. Modeling a Repeating Event Pattern 335
4.5.4. Collecting More Detailed Statistics 338
4.5.5. Implementing the Enhanced Pharmacy Model 338
4.6. More Drive Through Fun 347
4.6.1. Modeling a Resource with a Waiting Line 350
4.6.2. Modeling the Tandem Queue of Example 4.6 355
4.6.3. Modeling with the Station Package 367
4.7. Summary 372
4.8. Exercises 374

5. Analyzing and Accessing Simulation Output 388
5.1. Types of Statistical Variables 390
5.2. Types of Simulation With Respect To Output Analysis . . . 395
5.3. Analysis of Finite Horizon Simulations 398
5.4. Capturing Output for a Simple Finite Horizon Simulation . 400
5.4.1. KSL Functionality for Capturing Statistical Results 409
5.4.2. Additional Remarks 427

5.4.3. Querying the KSL Database 429
5.5. Sequential Sampling for Finite Horizon Simulations 431
5.6. Analysis of Infinite Horizon Simulations 436
5.6.1. Assessing the Effect of Initial Conditions 447
5.6.2. Performing the Method of Replication-Deletion . . . 455
5.6.3. The Method of Batch Means 464
5.6.4. Performing the Method of Batch Means 470
5.7. Comparing System Configurations 475
5.7.1. Comparing Two Systems 476
5.7.2. Concepts for Comparing Systems 494
5.7.3. Multiple Comparison with the Best Procedures (MCB) 498
5.7.4. Screening Procedures 509
5.7.5. Comparing Different Models 512
5.8. Summary 516
5.9. Exercises 519

6. Process View Modeling Using the KSL 527
6.1. What are Entities? 528
6.2. The Process View 531
6.3. Understanding KSL Processes and Entities 546
6.4. Examples of Process Modeling 557
6.4.1. Holding Entities in a Queue: `HoldQueue` Class . . . 557
6.4.2. Signaling Entities 559
6.4.3. Understanding Blocking Queues 564
6.4.4. Allowing Entities to Wait for a Process 571
6.4.5. Process Interaction 575
6.5. Modeling a STEM Career Mixer 591
6.5.1. Conceptualizing the System 593
6.5.2. Implementing the STEM Mixer Model 604
6.6. The Tie-Dye T-Shirt Model 611
6.6.1. Implementing the Tie-Dye T-Shirt Model 612
6.7. Summary 623
6.8. Exercises 625

7. Advanced Event and Process View Modeling 654
7.1. Modeling with Processes and Resources 656
7.1.1. Modeling Space with Resources 656
7.1.2. Resource Pools 667
7.1.3. Computer Test and Repair Shop Example 677
7.2. Modeling Non-Stationary Systems 690
7.2.1. Non-Stationary Arrival Processes 690
7.2.2. Modeling Resources Under Non-Stationary Conditions 704
7.3. Collecting Statistics Over Periods of Time 715
7.4. Enhancing the STEM Career Mixer Model 735
7.4.1. Non-Stationary STEM Career Fair Mixer 735
7.4.2. Modeling Walking Time 739
7.4.3. Collecting Period Statistics for the Mixer Model . . 741
7.4.4. Implementing the Revised STEM Mixer Model . . . 743
7.4.5. Running the Revised STEM Mixer Model 757
7.5. Examples of Advanced Event Models 784
7.5.1. Modeling Balking and Reneging 784
7.5.2. Modeling a Reorder Point, Reorder Quantity Inventory Policy 791
7.6. Applying Queueing Theory Results to Verify and Validate a Simulation 817
7.6.1. Analyzing the Preparation Station 821
7.6.2. Analyzing the Build Lines 822
7.6.3. Analyzing the Packaging Station 823
7.6.4. Analyzing the Palletizing Station 824
7.6.5. Analyzing the Total System Time 825
7.6.6. Other Issues for Verification and Validation 828
7.7. Summary 831
7.8. Exercises 832

8. Modeling Entity Movement 855
8.1. Resource Constrained Transfer 857
8.1.1. Test and Repair with Resource Constrained Transfer 857

8.2. Constrained Transfer with Movable Resources 866
8.2.1. Spatial Models 867
8.2.2. Movable Resources 874
8.2.3. Tandem Queue Model With Movement 879
8.3. Modeling the Test and Repair System with Movable Resources 891
8.4. Modeling Systems with Conveyors 899
8.4.1. KSL Conveyor Constructs 907
8.4.2. Tandem Queue System with Conveyors 914
8.4.3. Test and Repair via Conveyors 929
8.4.4. Miscellaneous Concepts in Conveyor Modeling . . . 937
8.5. Modeling Guided Path Transporters 946
8.5.1. Guided Path Networks 949
8.5.2. KSL Guided Path Constructs 951
8.5.3. A Simple AGV System 961
8.5.4. Deadlock, Spurs, and Idle Vehicles 968
8.5.5. Test and Repair with Guided Transporters 971
8.5.6. Further Topics in Guided Path Modeling 979
8.6. Summary 980
8.7. Exercises 983

9. Advanced Monte Carlo Methods 996
9.1. Bootstrap Methods 997
9.1.1. Bootstrapping Using the KSL 1008
9.2. Variance Reduction Techniques 1028
9.2.1. Common Random Numbers (CRN) 1031
9.2.2. Antithetic Variates (AV) 1038
9.2.3. Indirect Estimation 1045
9.2.4. Control Variates (CV) 1046
9.2.5. Stratified and Post Stratified Sampling 1057
9.2.6. Conditional Expectation (CE) 1061
9.2.7. Importance Sampling (IS) 1065
9.3. Generating Multi-Variate and Correlated Random Variates 1071
9.3.1. Generating from a Bivariate Normal Distribution . . 1072
9.3.2. Copulas and Multi-variate Generation Methods . . . 1076

9.3.3. Autocorrelated Generation 1086
9.3.4. Introduction to Markov Chain Monte Carlo Methods 1089
9.4. Summary 1111
9.5. Exercises 1113

10. Simulating Many Scenarios and Performing Experiments 1125
10.1. Simulating Many Scenarios 1127
10.1.1. Control Annotations 1127
10.1.2. Random Variable Parameters 1135
10.1.3. Building Models with `ModelBuilderIfc` 1138
10.1.4. Setting Up and Running Multiple Scenarios 1141
10.1.5. Executing Scenarios Concurrently: `ConcurrentScenarioRunner`1147
10.2. Experimental Design Methods 1152
10.2.1. Defining Factors 1153
10.2.2. Defining Experimental Designs 1155
10.2.3. Executing a Designed Experiment 1163
10.2.4. Analyzing a Designed Experiment 1170
10.2.5. Central Composite Designs and Response Surface Methods 1179
10.3. Summary 1184
10.4. Exercises 1185

11. Simulation Optimization Methods 1193
11.1. Simulation Optimization Methods 1194
11.2. Defining Simulation Optimization Problems 1198
11.2.1. Specifying Penalty Functions 1203
11.2.2. Defining Input Variables 1209
11.2.3. Linear and Functional Constraints 1212
11.2.4. Response Constraints 1215
11.2.5. Setting Up a Problem Definition 1217
11.3. Representing the Simulation Oracle 1221
11.3.1. Requesting Simulation Evaluations 1223
11.3.2. Representing Model Inputs 1224
11.3.3. Representing Model Outputs 1228

11.4. Characterizing Simulation Optimization Algorithms 1235
11.4.1. Structure of the Iterative Process 1238
11.4.2. Solver Plug-in Behavior 1242
11.4.3. Miscellaneous Concepts in the `Solver` Class 1245
11.5. Implemented Solvers 1254
11.5.1. Simulated Annealing 1255
11.5.2. Cross Entropy 1261
11.5.3. R-SPLINE 1271
11.5.4. Randomized Restart Solvers 1276
11.6. Using a Solver on a Problem 1279
11.6.1. Illustrating the Cross-Entropy Solver 1280
11.6.2. Illustrating Simulated Annealing with Random Restarts 1291
11.7. Additional Solvers 1300
11.8. Summary 1302
11.9. Exercises 1304

Appendices **1311**

A. Generating Pseudo-Random Numbers and Random Variates **1311**
A.1. Pseudo Random Numbers 1312
A.1.1. Random Number Generators 1314
A.2. Generating Random Variates from Distributions 1324
A.2.1. Inverse Transform Method 1325
A.2.2. Convolution 1340
A.2.3. Acceptance/Rejection 1344
A.2.4. Mixture Distributions, Truncated Distributions, and Shifted Random Variables 1350
A.3. Summary 1357
A.4. Exercises 1357

B. Probability Distribution Modeling **1371**
B.1. Random Variables and Probability Distributions 1374

B.2. Modeling with Discrete Distributions 1382
B.3. Fitting Discrete Distributions 1383
B.3.1. Fitting a Poisson Distribution 1384
B.3.2. Visualizing the Data 1385
B.3.3. Estimating the Rate Parameter for the Poisson Distribution 1396
B.3.4. Chi-Squared Goodness of Fit Test for Poisson Distribution 1400
B.3.5. Chi-Squared Goodness of Fit Test 1402
B.3.6. Using the fitdistrplus R Package on Discrete Data . 1407
B.3.7. Fitting a Discrete Empirical Distribution 1411
B.4. Modeling with Continuous Distributions 1414
B.5. Fitting Continuous Distributions 1418
B.5.1. Visualizing the Data 1419
B.5.2. Statistically Summarize the Data 1421
B.5.3. Hypothesizing and Testing a Distribution 1425
B.5.4. Kolmogorov-Smirnov Test 1434
B.5.5. Visualizing the Fit 1438
B.6. Testing Uniform (0,1) Pseudo-Random Numbers 1444
B.6.1. Chi-Squared Goodness of Fit Tests for Pseudo-Random Numbers 1447
B.6.2. Higher Dimensional Chi-Squared Test 1450
B.6.3. Kolmogorov-Smirnov Test for Pseudo-Random Numbers 1454
B.6.4. Testing for Independence and Patterns in Pseudo-Random Numbers 1457
B.7. Additional Distribution Modeling Concepts 1459
B.8. Summary 1463
B.9. Exercises 1464

C. Queueing Theory **1471**
C.1. Single Line Queueing Stations 1472
C.1.1. Queueing Notation 1475
C.1.2. Little's Formula 1479

C.1.3. Deriving Formulas for Markovian Single Queue Systems 1484
C.2. Examples and Applications of Queueing Analysis 1494
C.2.1. Infinite Queue Examples 1495
C.2.2. Finite Queue Examples 1504
C.3. Non-Markovian Queues and Approximations 1514
C.4. Summary of Queueing Formulas 1517
C.4.1. M/M/1 Queue 1518
C.4.2. M/M/c Queue 1519
C.4.3. M/M/c/k Queue 1520
C.4.4. M/G/c/c Queue 1521
C.4.5. M/M/1/k Queue 1521
C.4.6. M/M/c/k Queue 1522
C.4.7. M/M/1/k/k Queue 1523
C.4.8. M/M/c/k/k Queue 1523
C.5. Exercises 1524

D. Distributions **1534**
D.1. Discrete Distrbutions 1534
D.2. Continuous Distrbutions 1536

E. KSL Utility Packages **1541**
E.1. The `OutputDirectory` Class and `KSL` Object 1542
E.2. Logging Options 1545
E.3. The `KSLFileUtil` Object 1546
E.4. CSV, Excel, and Tabular Data Files 1548
E.5. The `DataFrameUtil` Object 1561
E.6. KSL Database Utilities 1564
E.7. Array Utilities 1574
E.8. KSL Plotting Utilities 1579
E.9. Miscellaneous Utilities 1589

F. Statistical Tables **1592**

References **1599**

Preface

This book is intended as an introductory textbook for a first course in discrete-event simulation modeling and analysis for upper-level undergraduate students as well as graduate students. While the text is focused towards engineering students (primarily industrial engineering) it could also be utilized by computer science and data science majors. Practitioners interested in learning simulation and the KSL could also use this book independently of a course.

The online version of this book is licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

When citing this book, please use the following format:

Rossetti, M.D. (2023). Simulation Modeling using the Kotlin Simulation Library (KSL), On-line and Open Text Edition. Retrieved from https://rossetti.github.io/KSLBook/ licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

The purpose of this book is to provide an overview of the Kotlin Simulation Library (KSL). The KSL facilitates simulation modeling by providing Kotlin language libraries that ease the development of simulation models. The KSL has a substantial amount of functionality implemented in the following packages:

- `ksl.utilities`: a variety of support utilities for performing discrete-event and Monte Carlo experiments including:

  - probability distribution models
  - random number generation
  - random variate generation
  - statistical collection (observation based and time weighted summary statistics, histograms, frequency tabulation, bootstrapping, box plot summary, etc.)
  - statistical comparison
  - Markov chain Monte Carlo
  - Extension functions for Arrays (sampling, filling, statistics, input, output)
  - file input and output utilities (CSV processing, Markdown tables, Excel, tabular files)
  - database - supports the creation, connection, and usage of databases
    * export databases to CSV, text files, Excel, DataFrames
    * import data to tables from Excel worksheets
    * capture simulation results to well-structured database for post processing

- `ksl.calendar`: linked list, priority queue, skew heap, and tree set based event calendars
- `ksl.simulation`: model development, experiment execution, reporting, batching
- `ksl.observers`: variable tracing, replication data collection, Welch plotting, data file collection
- `ksl.modeling`
  - event generation, schedules, random elements
  - non-homogeneous Poisson process generation
  - queues with automated statistical collection
  - process view implementation based on coroutines
  - response variable statistical collection
  - resource capacity schedules
  - responses collected by periods of time
  - aggregate statistical collection

- `ksl.simopt`: provides a framework for performing simulation optimization
- `ksl.controls.experiments`: provides a framework for performing designed experiments

This book discusses a large portion of this functionality. The KSL Github project page discusses how to access the code and examples discussed within this textbook.

Software oriented guides to the previous packages are also available. The guides indicate the basics for getting started with the various packages associated with the KSL and some of the underlying technical details of the implementation. In addition to the base KSL library, the web-site has applications that enable GUI and text-oriented interfaces for the essential aspects of working with simulation models. The list of applications and their guides can be found here.

Portions of this book also appear within my other textbook on simulation:

Rossetti, M.D. (2021). Simulation Modeling and Arena, 3rd and Open Text Edition. Retrieved from https://rossetti.github.io/RossettiArenaBook/ licensed under the Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

Release History

You are reading the on-line edition of *Simulation Modeling using the Kotlin Simulation Library (KSL)* by Dr. Manuel D. Rossetti. Because of its on-line nature, updated versions of the book will be released, as needed, to correct issues and add new material. This section summarizes noteworthy updates.

- 1st Edition, Version 1.0, released January 2023
  - first main release of text book

Release History

- 2nd Edition Version 2.0, released January 2024
  - Added distribution fitting Section 2.4 to Chapter 2
  - Added Chapter 9 on advanced Monte Carlo methods
- 2nd Edition Version 2.1, released July 2024
  - Added numbered examples to each chapter corresponding to code in KSLExamples project
  - Added examples on generation from mixture, truncated, and shifted distributions to Section 2.2.3 of Chapter 2. Also added how to generate using the acceptance-rejection technique to Section 2.2.3.
  - Added two new examples to Section 3.6 and Section 3.7 to Chapter 3.
  - Added Section 4.6 on station modeling to Chapter 4.
  - Added Section 5.7.4 for coverage of screening techniques
  - Added Section E.8 Appendix E for coverage of plotting utilities.
- 2nd Edition, Version 2.2, released February 2025
  - Added Chapter 8 for modeling entity movement
- 2nd Edition, Version 2.3, released August 2025
  - Updated chapters to be consistent with release 1.2.2 of the KSL
- 2nd Edition, Version 2.4, released September 2025
  - Added Chapter 10 on simulation optimization
- 2nd Edition, Version 2.5, released July 2026
  - Split former chapter 10 into two chapters: 10 and 11
  - Chapter 10 now contains the scenario material of Chapter 5 and the experiment material of chapter 10
  - Chapter 11 is now solely dedicated to simulation optimization.

If you find typographical errors or other issues related to the text or supporting files, then please use the book's repository's issue tracking system to create a new issue. You should first check if the same or similar issue has already been submitted. The issue tracking system is for filing issues about the correctness of the text or files. It is **not** about general questions about simulation concepts, solutions to homework, how to do something in the KSL, etc. Such issues will not be considered and will be deleted as needed.

KSL Project Page

The KSL project is based on a git repository

The KSL is a multi-project gradle project with key projects:

- `KSLCore`: the main project with core development functionality
- `KSLExamples`: a project that has the examples related to this textbook as well as additional illustrative examples
- `KSLProjectTemplate`: a gradle based project that is configured to use the `KSLCore` functionality as a dependency. Use this project as a starter template for projects using the KSL.

Additional modules exist to support KSL based applications. See the repository for the details.

The KSL documentation is generated by KDocs.

Artifacts for using the KSL have been made available on Maven central.

Book Support Files

By cloning the repository or downloading the zip archive, you can have a local copy of the entire book. Thus, if you do not have regular access to

internet services, you can still read and utilize the materials. I encourage students within a class setting to clone the book repository using a program such as Github desktop.

Acknowledgments

Special thanks to the Open Educational Resources team at the University of Arkansas. This work was supported in part by a grant from the OER Course Materials Conversion Faculty Funding program at the University of Arkansas.

I would like to thank John Wiley and Sons, Inc. for their flexibility in returning to me my copyrights from previous editions of my textbooks. This allowed this open textbook to exist. I would also like to thank the students in my classes who tested versions of my work and provided feedback, suggestions, and comments.

Lastly, I would like to thank my children Joseph, and Maria, who gave me their support and understanding and my wife, Amy, who not only gave me support, but also helped with creating figures, diagrams, and with proof-reading. Thanks so much!

Intended Audience

Discrete-event simulation is an important tool for the modeling of complex systems. It is used to represent manufacturing, transportation, and service systems in a computer program for the purpose of performing experiments. The representation of the system via a computer program enables the testing of engineering design changes without disruption to the system being modeled.

Intended Audience

Simulation modeling involves elements of system modeling, computer programming, probability and statistics, and engineering design. Because simulation modeling involves these individually challenging topics, the teaching and learning of simulation modeling can be difficult for both instructors and students. Instructors are faced with the task of presenting computer programming concepts, probability modeling, and statistical analysis all within the context of teaching how to model complex systems such as factories and supply chains. In addition, because of the complexity associated with simulation modeling, specialized computer languages are needed and thus must be taught to students for use during the model building process. This book is intended to help instructors with this daunting task.

Traditionally, there have been two primary types of simulation textbooks 1) those that emphasize the theoretical (and mostly statistical) aspects of simulation, and 2) those that emphasize the simulation language or package. The intention of this book is to blend these two aspects of simulation textbooks together while adding and emphasizing the art of model building. Thus the book contains chapters on modeling and chapters that emphasize the statistical aspects of simulation. However, the coverage of statistical analysis is integrated with the modeling in such a way to emphasize the importance of both topics.

This book utilizes the Kotlin Simulation Library as the primary modeling tool for teaching simulation. The KSL is open source and freely available. Users familiar with commercial simulation languages should note that the KSL has many of the features found in those languages.

I feel strongly that simulation is best learned by doing. The book is structured to enable and encourage students to get engaged in the material. The overall approach to presenting the material is based on a hands-on concept for student learning. The style of writing is informal, tutorial, and centered around examples that students can implement while reading the chapters. The book assumes a basic knowledge of probability and statistics, and an introductory knowledge of computer programming. Even

though these topics are assumed, the book provides integrated material that should refresh students on the basics of these topics. Thus, instructors who use this book should not have to formally cover this material, and can be assured that students who read the book will be aware of these concepts within the context of simulation.

Organization of the Book

Chapter 1 is an introduction to the field of simulation modeling. After Chapter 1 the student should know what simulation is and be able to put the different types of simulation into context. Chapter 2 introduces the basics of random number generation and random variate generation within the context of the KSL library.

Chapter 3 introduces problem solving and statistical concepts related to Monte Carlo simulation experiments. Chapter 4 introduces the important concept of how a discrete-event clock "ticks" and sets the stage for process modeling using activity diagramming. Finally, simple (but comprehensive) examples of KSL event modeling are presented.

Chapter 5 presents important concepts of statistical analysis that occur within discrete-event simulation modeling. This chapter should provide a refresher for students on statistical concepts. Chapter 6 dives deeper into process-oriented modeling. Important concepts within process-oriented modeling (e.g. entities, attributes, activities, state variables, etc.) are emphasized within the context of a number of examples. In addition, a deeper understanding of the KSL is developed including flow of control and input/output. After finishing Chapter 6, students should be able to model interesting systems from a process viewpoint using the KSL. Chapter 7 presents more advanced concepts within simulation and especially how the KSL facilitates the modeling. In particular, non-stationary arrivals and resource staffing are introduced in Chapter 7, as well as constructs for more advanced modeling with resources. Chapter 8 addresses how

to model the movement of entities within constrained and unconstrained contexts. Chapter 9 presents more advanced techniques used within Monte Carlo methods. Chapter 10 presents an introduction to using experimental design techniques within the context of simulation modeling. In addition, Chapter 11 provides and introduction to the important area of simulation optimization methods and how the KSL facilitates the application of these techniques.

The Appendix A and Appendix B are extremely useful for understanding the concepts of random variate generation and distribution modeling. For undergraduate students, I recommend starting with Appendix A and Appendix B. Appendix C provides an overview of queueing theory, which can be useful when verifying and validating the results of simulation models involving queues. Section D.1, Section D.2, and Appendix F provide information on probability distributions and statistical tables. Of particular note is Appendix E, which covers useful utility functionality in support of KSL modeling.

Future chapters are planned for when new KSL functionality is developed.

0. Preface
1. Chapter 1 Simulation Modeling
2. Chapter 2 Modeling Randomness
3. Chapter 3 Monte Carlo Methods
4. Chapter 4 Introduction to Discrete Event Modeling
5. Chapter 5 Analyzing Simulation Output
6. Chapter 6 Process View Modeling
7. Chapter 7 Advanced Event and Process View Modeling
8. Chapter 8 Modeling Entity Movement
9. Chapter 9 Advanced Monte Carlo Methods
10. Chapter 10 Simulating Many Scenarios and Performing Experiments
11. Chapter 11 Simulation Optimization Methods
12. Appendices

- Appendix A Generating Pseudo-Random Numbers and Random Variates
- Appendix B Probability Distribution Modeling
- Appendix C Queueing Theory
- Appendix E KSL Utility Packages
- Section D.1 Discrete Distributions
- Section D.2 Continuous Distributions
- Appendix F Statistical Tables

12. References

Depending on the level of programming skill of the students, instructors should be able to cover Chapter 1 through Chapter 6 within a semester course.

About the Author

Dr. Manuel Rossetti, P.E. Dr. Rossetti is a University Professor of Industrial Engineering at the University of Arkansas. He served as the inaugural Director for the University of Arkansas Data Science program from 2019-2024. In addition, Dr. Rossetti served as the Director for the NSF I/UCRC Center for Excellence in Logistics and Distribution (CELDi) from 2013-2020. Dr. Rossetti has published over 125 journal and conference articles in the areas of simulation, logistics/inventory, and healthcare and has been the PI or Co-PI on funded research projects totaling over 6.4 million dollars. He was selected as a Lilly Teaching Fellow in 1997/98 and was voted Best IE Teacher by IE students in 2007, 2009, and 2017. He won the IE Department Outstanding Teacher Award in 2001-02, 2007-08, and 2010-11. He received the College of Engineering Imhoff Teaching Award in 2012 and was elected an IIE Fellow. In 2013, the UA Alumni Association awarded Dr. Rossetti the Charles and Nadine Baum Faculty Teaching Award, the highest award for teaching at the university. In 2015, Dr. Rossetti served as Program Chair for the Winter Simulation Conference. In 2024, Dr. Rossetti served as the General Chair for the Winter Simulation Conference. He is also the author of the book, Simulation Modeling and Arena, published by John Wiley & Sons.

Dr. Rossetti grew up in Canton, Ohio and is an ardent Cleveland sports fan. He received his Ph.D. and MSIE degrees in Industrial and Systems Engineering from The Ohio State University and his BSIE degree from the University of Cincinnati. He is a registered professional engineer in the State of Arkansas.

1. Simulation Modeling

Learning Objectives

- To be able to describe what computer simulation is
- To be able to discuss why simulation is an important analysis tool
- To be able to list and describe the various types of computer simulations
- To be able to describe a simulation methodology

In this book, you will learn how to model systems within a computer environment in order to analyze system design configurations. The models that you will build and exercise are called simulation models. When developing a simulation model, the modeler attempts to represent the system in such a way that the representation assumes or mimics the pertinent outward qualities of the system. This representation is called a simulation model. When you execute the simulation model, you are performing a simulation. In other words, simulation is an instantiation of the act of simulating. A simulation is often the next best thing to observing the real system. If you have confidence in your simulation, you can use it to infer how the real system will operate. You can then use your inference to understand and improve the system's performance.

1.1. Simulation Modeling

In general, simulations can take on many forms. Almost everyone is familiar with the board game Life. In this game, the players imitate life by going to college, getting a job, getting married, etc. and finally retiring. This board game is a simulation of life. As another example, the military performs war game exercises which are simulations of battlefield conditions. Both of these simulations involve a physical representation of the thing being simulated. The board game, the rules, and the players represent the simulation model. The battlefield, the rules of engagement, and the combatants are also physical representations. No wishful thinking will make the simulations that you develop in this book real. This is the first rule to remember about simulation. A simulation is only a model (representation) of the real thing. You can make your simulations as realistic as time and technology allows, but they are not the real thing. As you would never confuse a toy airplane with a real airplane, you should never confuse a simulation of a system with the real system. You may laugh at this analogy, but as you apply simulation to the real world you will see analysts who forget this rule. Don't be one.

All the previous examples involved a physical representation or model (real things simulating other real things). In this book, you will develop computer models that simulate real systems. Ravindran et al. (1987) define computer simulation as: "A numerical technique for conducting experiments on a digital computer which involves logical and mathematical relationships that interact to describe the behavior of a system over time." Computer simulations provide an extra layer of abstraction from reality that allows fuller control of the progression of and the interaction with the simulation. In addition, even though computer simulations are one step removed from reality, they are often capable of providing constructs which cannot be incorporated into physical simulations. For example, an airplane flight simulator can have emergency conditions for which it would be too dangerous or costly to provide in a physical based simulation training

scenario. This representational power of computer modeling is one of the main reasons why computer simulation is used.

1.2. Why Simulate?

Imagine trying to analyze the following situation. Patients arrive at an emergency room. The arrival of the patients to the emergency department occurs randomly and may vary with the day of the week and even the hour of the day. The hospital has a triage station, where the arriving patient's condition is monitored. If the patient's condition warrants immediate attention, the patient is expedited to an emergency room bed to be attended by a doctor and a nurse. In this case, the patient's admitting information may be obtained from a relative. If the patient does not require immediate attention, the patient goes through the admitting process, where the patient's information is obtained. The patient is then directed to the waiting room, to wait for allocation to a room, a doctor, and a nurse. The doctors and nurses within the emergency department must monitor the health of the patients by performing tests and diagnosing the patient's symptoms. This occurs on a periodic basis. As the patient receives care, the patient may be moved to and require other facilities (MRI, X-ray, etc.). Eventually, the patient is either discharged after receiving care or admitted to the main hospital. The hospital is interested in conducting a study of the emergency department in order to improve the care of the patients while better utilizing the available resources. To investigate this situation, you might need to understand the behavior of certain measures of performance:

- The average number of patients that are waiting.
- The average waiting time of the patients and their average total time in the emergency department.
- The average number rooms required per hour.

- The average utilization of the doctors and nurses (and other equipment).

Because of the importance of emergency department operations, the hospital has historical records available on the operation of the department through its patient tracking system. With these records, you might be able to estimate the current performance of the emergency department. Despite the availability of this information, when conducting a study of the emergency department you might want to propose changes to how the department will operate (e.g. staffing levels) in the future. Thus, you are faced with trying to predict the future behavior of the system and its performance when making changes to the system. In this situation, you cannot realistically experiment with the actual system without possibly endangering the lives or care of the patients. Thus, it would be better to model the system and to test the effect of changes on the model. If the model has acceptable fidelity, then you can infer how the changes will affect the real system. This is where simulation techniques can be utilized.

If you are familiar with operations research and industrial engineering techniques, you may be thinking that the emergency department can be analyzed by using queueing models. Later chapters of this book will present more about queueing models; however, for the present situation, the application of queueing models will most likely be inadequate due to the complex policies for allocating nurses, doctors, and beds to the patients. In addition, the dynamic nature of this system (the non-stationary arrivals, changing staffing levels, etc.) cannot be well modeled with current analytical queueing models. Queueing models might be used to analyze portions of the system, but a total analysis of the dynamic behavior of the entire system is beyond the capability of these types of models. But, a total analysis of the system is not beyond simulation modeling.

Simulation may be the preferred modeling methodology if the understanding gained from developing and using a simulation model is worth the time and cost associated with developing and using the model. Good uses of simulation include:

- Understanding how complex interactions in the system affect performance.
- Understanding how randomness affects performance.
- Comparing a fixed set of design alternatives to determine which design meets the performance goals under which conditions
- Training people to prepare them for dealing with events that may be disruptive to the actual system.
- The model will be used repeatedly for decision making.
- When the decision associated with the problem has a high cost so that the cost of building the model and evaluating the design is worth its development.
- When the current system does not yet exist and you need to ensure that the chosen design will meet specifications.

Simulation modeling activities encapsulate all three major modeling methods of data analytics: descriptive, predictive, and prescriptive. Descriptive modeling uses historical data to describe what happened in order to understand past behavior of a system. Predictive modeling uses historical data to develop models that help us understand future behavior in order the answer what *may* happen. Descriptive modeling summarizes past data for understanding. Predictive modeling uses past data to predict future behavior. Prescriptive modeling indicates what should be done and is integral to answering questions involving system design.

A simulation model is both a descriptive and predictive model. In addition, when coupled with stochastic optimization methods or a rigorous design process that evaluates and recommends designs, a simulation model becomes an integral part of the prescriptive modeling process. A simulation model *describes* how a system works by encapsulating that description within the operating runs/constructs of the model. A simulation model uses descriptive models (input models, summary statistics). A simulation

model *predicts* future system response. A simulation model can be used to predict future behavior through running what-if scenarios. Simulation is inherently a predictive modeling methodology. Unlike other predictive modeling techniques found in data analytics, such as regression, neural networks, random forests, etc., simulation explicitly incorporates *domain* knowledge into the modeling activity by incorporating system operating behavior. The behavior is modeled through the physical and logical rules that apply to the relationships between the components of the system. Unlike pure statistical predictive models, simulation has the advantage of explicitly representing relationships rather than just relying on discovering relationships.

A key advantage of simulation modeling is that it has the capability of modeling the entire system and its complex inter-relationships. The representational power of simulation provides the flexible modeling that is required for capturing complex processes. As a result, all the important interactions among the different components of the system can be accounted for within the model. The modeling of these interactions is inherent in simulation modeling because simulation imitates the behavior of the real system (as closely as necessary). The prediction of the future behavior of the system is then achieved by monitoring the behavior of different modeling scenarios as a function of simulated time.

Real world systems are often too complex for analytical models and often too expensive to experiment with directly. Simulation models allow the modeling of this complexity and enable low cost experimentation to make inferences about how the actual system might behave.

1.3. Types of Systems and Simulation Models

The main purpose of a simulation model is to allow observations about a particular system to be collected as a function of time. So far the word *system* has been used in much of the discussion, without formally

discussing what a system is. According to (Blanchard and Fabrycky 1990) a system is a set of inter-related components working together towards a common objective. The standard for systems engineering provides a deeper definition:

> "A system is a composite of people, products, and processes that provide a capability to satisfy stated needs. A complete system includes the facilities, equipment (hardware and software), materials, services, data, skilled personnel, and techniques required to achieve, provide, and sustain system effectiveness." (Command 1991)

Figure 1.1 illustrates the fact that a system is embedded within an environment and that typically a system requires inputs and produces output using internal components. How you model a particular system will depend upon the intended use of the model and how you perceive the system. The modeler's view of the system colors how they conceptualize it. For example, for the emergency room situation, "What are the system boundaries? Should the ambulance dispatching and delivery process be modeled? Should the details of the operating room be modeled?" Clearly, the emergency room has these components, but your conceptualization of it as a system may or may not include these items, and thus, your decisions regarding how to conceptualize the system will drive the level of abstraction within your modeling. An important point to remember is that two perfectly logical and rational people can look at the same thing and conceptualize that thing as two entirely different systems based on their "Weltanschauung" or world view.

Because how you conceptualize a system drives your modeling, it is useful to discuss some general system classifications. Systems might be classified by whether or not they are man-made (e.g. manufacturing system) or whether they are natural (e.g. solar system). A system can be physical (e.g. an airport) or conceptual (e.g. a system of equations). If stochastic or random behavior is an important component of the system then the system is said to be stochastic, if not then it is considered deterministic.

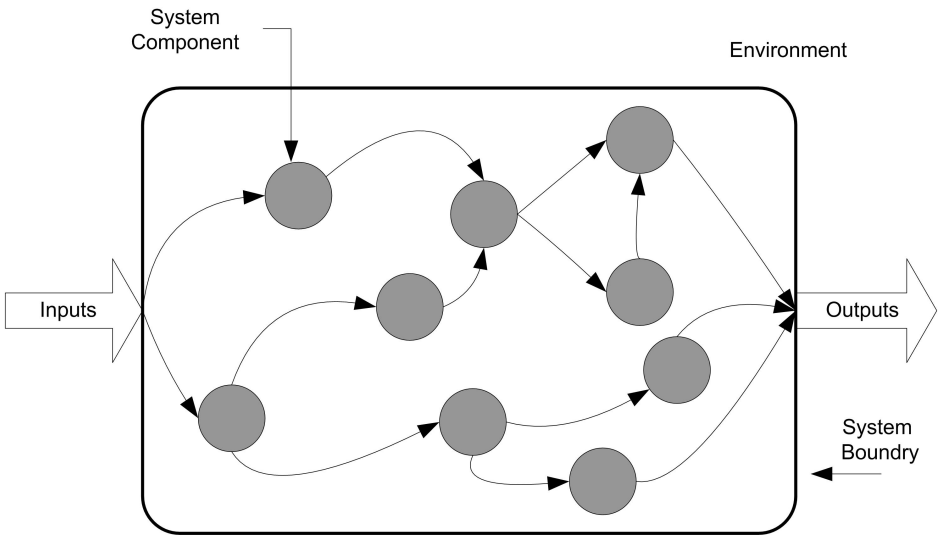


Figure 1.1.: Conceptualization of a System

One of the more useful ways to look at a system is whether it changes with respect to time. If a system does not change significantly with respect to time it is said to be static, else it is called dynamic. If a system is dynamic, you might want to consider how it evolves with respect to time. A dynamic system is said to be discrete if the state of the system changes at discrete points in time. A dynamic system is said to be continuous if the state of the system changes continuously with time. This dichotomy is purely a function of your level of abstraction. If conceptualizing a system as discrete, serves our purposes then you can call the system discrete. Figure 1.2 illustrates this classification of systems. This book primarily examines stochastic, dynamic, discrete systems.

The main purpose of a simulation model is to allow observations about a particular system to be gathered as a function of time. From that standpoint, there are two distinct types of simulation: 1) discrete event and 2) continuous.

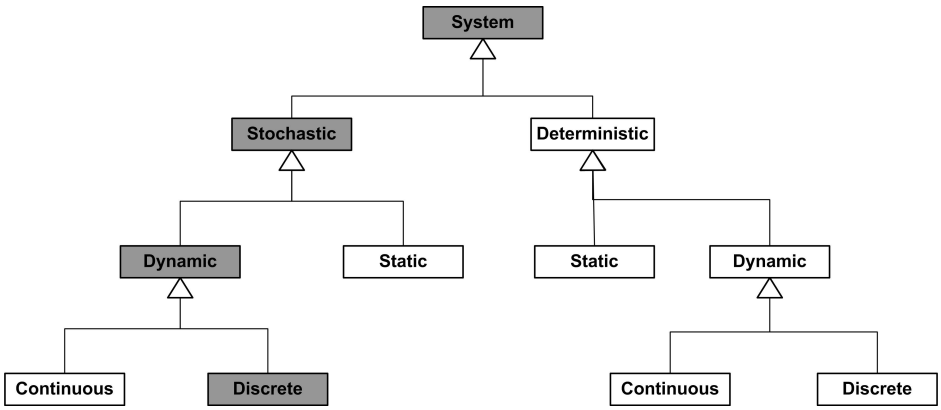


Figure 1.2.: General Types of Systems

Just as discrete systems change at discrete points in time, in discrete event simulation observations are gathered at selected points in time when certain changes take place in the system. These selected points in time are called events. On the other hand, continuous simulation requires that observations be collected continuously at every point in time (or at least that the system is described for all points in time). The types of models to be examined in this book are called discrete-event simulation models.

To illustrate the difference between the two types of simulation, contrast a fast food service counter with that of oil loading facility that is filling tankers. In the fast food service counter system, changes in the status of the system occur when a customer either arrives to place an order or when the customer receives their food. At these two events, measures such as queue length and waiting time will be affected. At all the other points in time, these measures remain either unchanged (e.g. queue length) or not yet ready for observation (e.g. waiting time of the customer). For this reason, the system does not need to be observed on a continuous basis. The system need only be observed at selected discrete points in time, resulting in the applicability of a discrete-event simulation model.

In the case of the oil tanker loading example, one of the measures of performance is the amount of oil in each tanker. Because the oil is a liquid, it cannot be readily divided into discrete components. That is, it flows continuously into the tanker. It is not necessary (or practical) to track each molecule of oil individually, when you only care about the level of the oil in the tanker. In this case, a model of the system must describe the rate of flow over time and the output of the model is presented as a function of time. Systems such as these are often modeled using differential equations. The solution of these equations involves numerical methods that integrate the state of the modeled system over time. This, in essence, involves dividing time into small equal intervals and stepping through time.

Often both the discrete and continuous viewpoints are relevant in modeling a system. For example, if oil tanker arrives at the port to be filled, we have an arrival event that changes the state of the system. This type of modeling situation is called combined continuous discrete modeling.

A system and our resulting model depends on how we characterize the state of the system. The **state** of a system is the set of properties/variables that describe the system at any time $\{x_1(t), x_2(t), ...\}$ where $x_1(t)$ is a variable that represents a system property value at time t. Variables that take on a countable set of values are **discrete**. Variables that take on an uncountable set of values are said to be **continuous**. For discrete variables, we can define a mapping from the set of integers to each possible value. Even in the discrete case, there may be an infinite number of values. Continuous variables are represented by the set of real numbers. That is, there are an uncountably infinite number of possible values that the variable can take on.

A discrete system has all discrete variables in its state. A continuous system has all continuous variables in its state. A combined continuous-discrete system has both types of variables in defining its state. Consider an airplane:

- If we are interested in the number of parts operating to specification at any time t, then the state is $\{N_1(t), N_2(t), ...\}$ where $N_1(t) = 1$ if part 1 is operating and 0 if part 1 is not operating to specification. The state vector consists of all discrete variables. This is a discrete system.
- If we are interested in the temperature of each part at time t, then the state is $\{T_1(t), T_2(t), ...\}$, where $T_1(t)$ is the temperature in Celsius of part 1 at time t, etc. The state vector consists of all continuous variables. This is continuous system.
- If we are interested in the velocity of a plane at time t and the number of wheels deployed at time t, then the state is $\{v(t), n(t)\}$ where $v(t)$ is the velocity of the plane in meters/second and $n(t)$ is $\{0, 1, 2, 3, 4\}$ wheels. Then the state vector consists of both continuous and discrete variables that change with time. This is a combined continuous/discrete system.

Static systems are systems for which time is not a significant factor. In other words, that the state does not evolve over time. Dynamic systems are systems for which system state changes with respect to time. In a deterministic system, the variables are not governed by underlying random processes. In a stochastic system, some of the variables are governed by underlying random processes. Time can change continuously or at discrete points. When time changes only at discrete points in time, we call these points, events.

Some simulation languages have modeling constructs for both continuous and discrete modeling; however, this book does not cover the modeling of continuous or combined continuous discrete systems. There are many useful references on this topic. We will be modeling discrete-event dynamic stochastic systems in this textbook.

1.4. Simulation: Descriptive or Prescriptive Modeling?

A descriptive model describes how a system behaves. Simulation is at its heart a descriptive modeling technique. Simulation is used to depict the behaviors or characteristics of existing or proposed systems. However, a key use of simulation is to convey the *required* behaviors or properties of a proposed system. In this situation, simulation is used to prescribe a solution. A prescriptive model tells us what to do. In other words, simulation can also be used for prescriptive modeling. Figure 1.3 illustrates the concept of using simulation to recommend a solution.

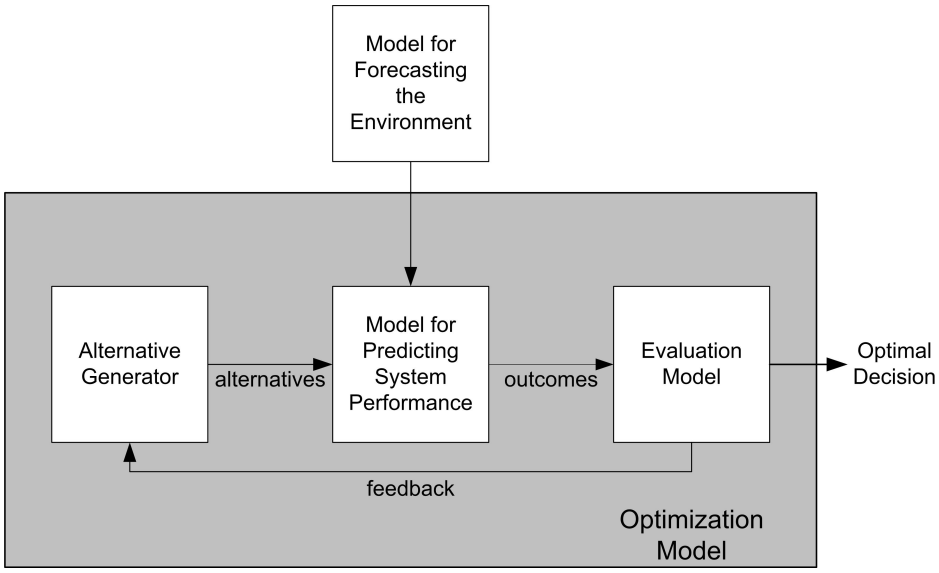


Figure 1.3.: Using Simulation for Prescriptive Analysis

In the figure, a simulation model is used for predicting the behavior of the system. Input models are used to characterize the system and its environment. An evaluative model is used to evaluate the output of the

simulation model to understand how the output compares to desired goals. The alternative generator is used to generate different scenarios to be feed into the simulation model for evaluation. Through a feedback mechanism the inputs can be changed based on the evaluation of the outputs and eventually a recommended solution can be achieved.

For example, in the modeling a drive up pharmacy, suppose that the probability of a customer waiting longer than 3 minutes in line had to be less than 10%. To form design alternatives, the inputs (e.g. number of pharmacists, possibly the service process) can be varied. Each alternative can then be evaluated to see if the waiting time criteria is met. In this simple situation, you might act as your own alternative generator and the evaluative model is as simple as meeting a criteria; however, in more complex models, there will often be hundreds of inputs to vary and multiple competing objectives. In such situations, simulation optimization and heuristic search methods are often used. This is an active and important area of research within simulation.

1.5. Randomness in Simulation

In most real-life situations, the arrival process and the service process occur in a random fashion. Even though the processes may be random, it does not mean that you cannot describe or model the randomness. To have any hope of simulating the situation, you must be able to model the randomness. One of the ways to model this randomness is to describe the phenomenon as a random variable governed by a particular probability distribution. For example, if the arrivals to the bank occur according to a Poisson process, then from probability theory it is known that the distribution of inter-arrival times is an exponential distribution. In general, information about how the customers arrive must be secured either through direct observation of the system or by using historical data. If neither source of information is available, then some plausible assumptions must be made to describe the random process by a probability model.

If historical data is available, there are two basic choices for how to handle the modeling. The first choice is to develop a probability model given the data. The second choice is to try to drive the simulation directly from the historical data. The latter approach is not recommended. First of all, it is extremely unlikely that the captured data will be in a directly usable form. Secondly, it is even more unlikely that the data will be able to adequately represent all the modeling scenarios that you will need through the course of experimenting with the model. For example, suppose that you only have 1 day's worth of arrival data, but you need to simulate a month's worth of system operation. If you simply re-drive your simulation using the 1 day's worth of data, you are not simulating different days! It is much more advisable to develop probability models either from historical data or from data that you capture in developing your model. Appendix B discusses some of the tools and techniques for modeling probability distributions.

Once a probability model has been developed, statistical theory provides the means for obtaining random samples based on the use of uniformly distributed random numbers on the interval (0,1). These random samples are then used to map the future occurrence of an event on the time scale. For example, if the inter-arrival time is exponential then a random sample drawn from that distribution would represent the time interval until the occurrence of the next arrival. The process of generating random numbers and random variables within simulation is presented in Appendix A.

1.6. Simulation Languages

Discrete event simulation normally involves a tremendous volume of computation. Consequently, the use of computers to carry out these computations is essential; however, the volume of computations is not the only obstacle in simulation. If you consider the bank teller example discussed in the previous sections, you will discover that it involves a complex logical structure that requires special expertise before it can be translated into a computer model. Attempting to implement the simulation model, from

scratch, in a general purpose language such as FORTRAN, Visual Basic, C/C++, Java, Kotlin will require above average programming skills. In the absence of specialized libraries for these languages that try to relieve the user from some of the burden, simulation as a tool would be relegated to "elite" programmers. Luckily, the repetitive nature of computations in simulation allows the development of computer libraries that are applicable to simulation modeling situations. For example, libraries or packages must be available for ordering and processing events chronologically, as well as generating random numbers and automatically collecting statistics. Such a library for simulating discrete-event systems in Java is available from the author, see (Rossetti 2008).

The computational power and storage capacity of computers has motivated the development of specialized simulation languages. Some languages have been developed for continuous or discrete simulations. Others can be used for combined continuous and discrete modeling. All simulation languages provide certain standard programming facilities and will differ in how the user will take advantage of these facilities. There is normally some trade-off between how flexible the language is in representing certain modeling situations. Usually, languages that are highly flexible in representing complex situations require more work (and care) by the user to account for how the model logic is developed. Some languages are more programming oriented (e.g. SIMSCRIPT) and others are more "drag and drop" (e.g. ProModel, Arena, etc.).

The choice of a simulation language is a difficult one. There are many competing languages, each having their own advantages and disadvantages. The Institute for Operations Research and Management Science (INFORMS) often has a yearly product review covering commercial simulation languages, see for example (http://lionhrtpub.com/orms/). In addition to this useful comparison, you should examine the Winter Simulation Conference (www.wintersim.org). The conference has hands on exhibits of simulation software and the conference proceedings often have tutorials for the various software packages. Past proceedings have been made available

electronically through the generous support of the INFORMS Society for Simulation (http://www.informs-sim.org/wscpapers.html).

Kotlin was chosen for this textbook because of the author's experience utilizing the software, its ease of use, and the availability of open source libraries. While all languages have flaws, using a language with extensions or libraries to support simulation is essential in performing high performance simulation studies. Once you learn one simulation language well, it is much easier to switch to other languages and to understand which languages will be more appropriate for certain modeling situations.

The library extensions for Kotlin to support simulation are called the KSL, for Kotlin Simulation Library. The KSL supports both the event-view and the process-view of simulation. That is, when using the process-view library extensions , the modeler describes the process that an "entity" experiences while flowing through or using the elements of the system. You will learn about how the KSL facilitates both the event-view and the process modeling view throughout this textbook.

1.7. Simulation Methodology

This section presents a brief overview of the steps of simulation modeling by discussing the process in the context of a methodology. A methodology is simply a series of steps to follow. Since simulation involves systems modeling, a simulation methodology based on the general precepts of solving a problem through systems analysis is presented here. A general methodology for solving problems can be stated as follows:

1. Define the problem
2. Establish measures of performance for evaluation
3. Generate alternative solutions
4. Rank alternative solutions

5. Evaluate and Iterate during process
6. Execute and evaluate the solution

This methodology can be referred to by using the first letter of each step. The DEGREE methodology for problem solving represents a series of steps that can be used during the problem solving process. The first step helps to ensure that you are solving the right problem. The second step helps to ensure that you are solving the problem for the right reason, i.e. your metrics must be coherent with your problem. Steps 3 and 4 ensure that the analyst looks at and evaluates multiple solutions to the problem. In other words, these steps help to ensure that you develop the right solution to the problem. A good methodology recognizes that the analyst needs to evaluate how well the methodology is doing. In step 5, the analyst evaluates how the process is proceeding and allows for iteration. Iteration is an important concept that is foreign to many modelers. The concept of iteration recognizes that the problem solving process can be repeated until the desired degree of modeling fidelity has been achieved. Start the modeling at a level that allows it to be initiated and do not try to address the entire situation in each of the steps. Start with small models that work and build them up until you have reached your desired goals. It is important to get started and get something established on each step and continually go back in order to ensure that the model is representing reality in the way that you intended. The final step is often overlooked. Simulation is often used to recommend a solution to a problem. Step 6 indicates that if you have the opportunity you should execute the solution by implementing the decisions. Finally, you should always follow up to ensure that the projected benefits of the solution were obtained.

The DEGREE problem solving methodology should serve you well; however, simulation involves certain unique actions that must be performed during the general overall problem solving process. When applying DEGREE to a problem that may require simulation, the general DEGREE approach needs to be modified to explicitly consider how simulation will interact with the overall problem solving process.

Figure 1.4 represents a refined general methodology for applying simulation to problem solving.

1. Problem Formulation
    1. Define the problem
    2. Define the system
    3. Establish performance metrics
    4. Build conceptual model
    5. Document model assumptions
2. Simulation Model Building
    1. Model translation
    2. Input data modeling
    3. Verification
    4. Validation
3. Experimental Design and Analysis
    1. Preliminary Runs
    2. Final experiments
    3. Analysis of results
4. Evaluate and Iterate
    1. Documentation
    2. Model manual
    3. User manual
5. Implementation

The first phase, problem formulation, captures the essence of the first two steps in the DEGREE process. The second phase, model building, captures the essence of step 3 of the DEGREE process. When building models, you are either explicitly or implicitly developing certain design alternatives. The third phase, experimental design and analysis, encapsulates some of steps 3 and 4 of the DEGREE process. In designing experiments, design alternatives are specified and when analyzing experiments their worth is being evaluated with respect to problem objectives. The fourth phase, evaluate and iterate, captures the notion of iteration and includes the documentation of the model. Finally, the fifth phase, implementation, completes the simulation process. Documentation is essential when trying to ensure the ongoing and future use of the simulation model, and implementation recognizes that simulation projects often fail if there is no follow through on the recommended solutions.

The problem formulation phase of the study consists of five primary activities:

1. Defining the problem
2. Defining the system
3. Establishing performance metrics.
4. Building conceptual models
5. Documenting modeling assumptions

A problem starts with a perceived need. These activities are useful in developing an appreciation for and an understanding of what needs to be solved. The basic output of the problem definition activity is a problem definition statement. A problem definition statement is a narrative discussion of the problem. A problem definition statement is necessary to accurately and concisely represent the problem for the analyst and for the problem stakeholders. This should include all the required assumptions made during the modeling process. It is important to document your assumptions so that you can examine their effect on the model during the

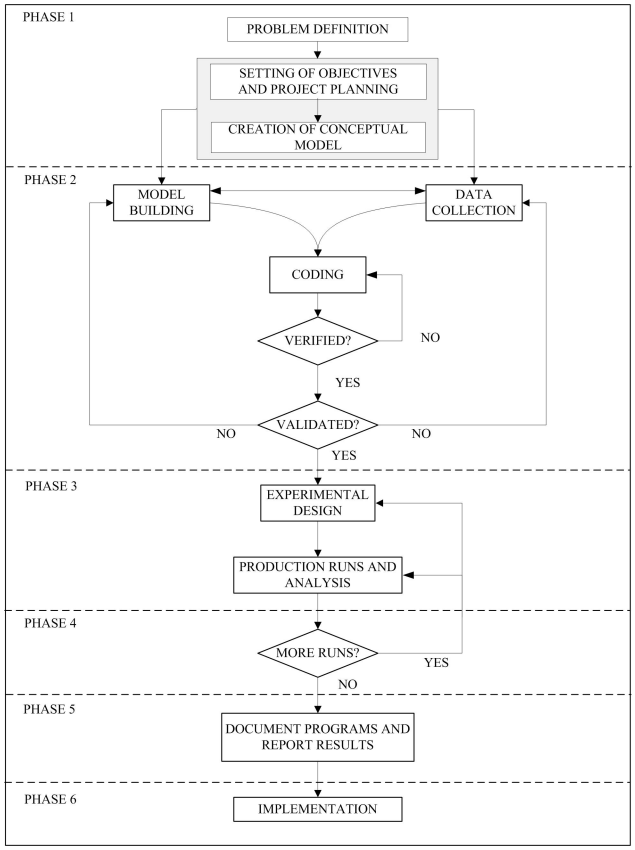


Figure 1.4.: General Simulation Methodology

verification, validation, and experimental analysis steps of the methodology. This ensures that the problem is well understood and that all parties agree upon the nature of the problem and the goals of the study. The general goals of a simulation study often include:

- Comparison: To compare system alternatives and their performance measures across various factors (decision variables) with respect to some objectives
- Optimization: This is a special case of comparison in which you try to find the system configuration that optimizes performance subject to constraints.
- Prediction: To predict the behavior of the system at some future point in time.
- Investigation: To learn about and gain insight into the behavior of the system given various inputs.

These general goals will need to be specialized to the problem under study. The problem definition should include a detailed description of the objectives of the study, the desired outputs from the model, and the types of scenarios to be examined or decisions to be made.

The second activity of this phase produces a definition of the system. A system definition statement is necessary to accurately and concisely define the system, particularly its boundaries. The system definition statement is a narrative, which often contains a pictorial representation of the major elements of the system. This ensures that the simulation study is focused on the appropriate areas of interest to the stakeholders and that the scope of the project is well understood.

When defining the problem and the system, one should naturally begin to develop an understanding of how to measure system performance. The third activity of problem formulation makes this explicit by encouraging the analyst to define the required performance measures for the model. To meaningfully compare alternative scenarios, objective and measurable

metrics describing the performance of the system are necessary. The performance metrics should include quantitative statistical measures from any models used in the analysis (e.g. simulation models), quantitative measures from the systems analysis, (e.g. cost/benefits), and qualitative assessments (e.g. technical feasibility, human, operational feasibility). The focus should be placed on the performance measures that are considered to be the most important to system decision-makers and tied directly to the objectives of the simulation study. Evaluation of alternatives can then proceed in an objective and unbiased manner to determine which system scenario performs the best according to the decision maker's preferences.

The problem definition statement, the system definition statement, and explicit performance metrics set the stage for more detailed modeling. These activities should be captured in a written form. Within this text, you will develop models of certain "ready-made" book problems. One way to accomplish the problem formulation phase of a simulation study is to consider writing yourself a "book problem". You will need enough detail in these documents that a simulation analyst (you) can develop a model in any simulation language for the given situation.

With a good understanding of the problem and of the system under study, you should be ready to begin your detailed model formulations. Model formulation does not mean a computer program. You should instead use conceptual modeling tools: conceptual diagrams, flow charts, etc. prior to any use of software to implement a model. The purpose of conceptual modeling tools is to convey a more detailed system description so that the model may be translated into a computer representation. General descriptions help to highlight the areas and processes of the system that the model will simulate. Detailed descriptions assist in simulation model development and coding efforts. Some relevant diagramming constructs include:

1. Context Diagrams: A context diagram assists in conveying the general system description. The diagram is a pictorial representation of the system that often includes flow patterns typically encountered.

Context diagrams are often part of the system description document. There are no rules for developing context diagrams. If you have an artistic side, here is your opportunity to shine!

2. Activity Diagrams: An activity diagram is a pictorial representation of the process for an entity and its interaction with resources while within the system. If the entity is a temporary entity (i.e. it flows through the system) the activity diagram is called an activity flow diagram. If the entity is permanent (i.e. it remains in the system throughout its life) the activity diagram is called an activity cycle diagram. Activity diagrams will be used extensively within this text.

3. Software engineering diagrams: Because simulation entails software development, the wide-variety of software engineering diagramming techniques can be utilized to provide information for the model builder. Diagrams such as flow charts, database diagrams, IDEF (ICAM DEFinition language) diagrams, UML (unified modeling language) diagrams, state charts, etc are all useful in documenting complex modeling situations. These techniques assist development and coding efforts by focusing attention on describing, and thus understanding, the elements within the system. Within this text, activity diagrams will be augmented with some simple flow chart symbols and some simple state diagrams will be used to illustrate a variety of concepts.

In your modeling, you should start with an easy conceptual model that captures the basic aspects and behaviors of the system. Then, you should begin to add details, considering additional functionality. Finally, you should always remember that the complexity of the model has to remain proportional to the quality of the available data and the degree of validity necessary to meet the objectives of the study. In other words, don't try to model the world!

After developing a solid conceptual model of the situation, simulation model building can begin. During the simulation model building phase, alternative

system design configurations are developed based on the previously developed conceptual models. Additional project planning is also performed to yield specifications for the equipment, resources, and timing required for the development of the simulation models. The simulation models used to evaluate the alternative solutions are then developed, verified, validated, and prepared for analysis. Within the context of a simulation project this process includes:

- Input Data Preparation: Input data is analyzed to determine the nature of the data and to determine further data collection needs. Necessary data is also classified into several areas. This classification establishes different aspects of the model that are used in model development.
- Model Translation: Description of the procedure for coding the model, including timing and general procedures and the translation of the conceptual models into computer simulation program representations.
- Verification: Verification of the computer simulation model is performed to determine whether or not the program performs as intended. To perform model verification, model debugging is performed to locate any errors in the simulation code. Errors of particular importance include improper flow control or entity creation, failure to release resources, and logical/arithmetic errors or incorrectly observed statistics. Model debugging also includes scenario repetition utilizing identical random number seeds, "stressing" the model through a sensitivity analysis (varying factors and their levels) to ensure compliance with anticipated behavior, and testing of individual modules within the simulation code.
- Validation: Validation of the simulation model is performed to determine whether or not the simulation model adequately represents the real system. The simulation model is shown to personnel (of various levels) associated with the system in question. Their input concerning the realism of the model is critical in establishing the

validity of the simulation. In addition, further observations of the system are performed to ensure model validity with respect to actual system performance. A simple technique is to statistically compare the output of the simulation model to the output from the real system and to analyze whether there is a significant (and practical) difference between the two.

Model translation will be a large component of each chapter as you learn how to develop simulation models. Verification and validation techniques will not be a major component of this text, primarily because the models will be examples made for educational purposes. This does not mean that you should ignore this important topic. You are encouraged to examine many of the useful references on validation, see for example (Balci 1997) and (Balci 1998).

After you are confident that your model has been verified and validated to suit your purposes, you can begin to use the model to perform experiments that investigate the goals and objectives of the project. Preliminary simulation experiments should be performed to set the statistical parameters associated with the main experimental study. The experimental method should use the simulation model to generate benchmark statistics of current system operations. The simulation model is then altered to conform to a potential scenario and is re-run to generate comparative statistics. This process is continued, cycling through suggested scenarios and generating comparative statistics to allow evaluation of alternative solutions. In this manner, objective assessments of alternative scenarios can be made.

For a small set of alternatives, this "one at a time" approach is reasonable; however, often there are a significant number of design factors that can affect the performance of the model. In this situation, the analyst should consider utilizing formal experimental design techniques. This step should include a detailed specification of the experimental design (e.g. factorial, etc) and any advanced output analysis techniques (e.g. batching, initialization bias prevention, variance reduction techniques, multiple comparison procedures, etc.) that may be required during the execution of the experiments. During

this step of the process, any quantitative models developed during the previous steps are exercised. Within the context of a simulation project, the computer simulation model is exercised at each of the design points within the stipulated experimental design.

Utilizing the criteria specified by system decision-makers, and utilizing the simulation model's statistical results, alternative scenarios should then be analyzed and ranked. A methodology should be used to allow the comparison of the scenarios that have multiple performance measures that trade-off against each other.

If you are satisfied that the simulation has achieved your objectives then you should document and implement the recommended solutions. If not, you can iterate as necessary and determine if any additional data, models, experimentation, or analysis is needed to achieve your modeling objectives. Good documentation should consist of at least two parts: a technical manual, which can be used by the same analyst or by other analysts, and a user manual. A good technical manual is very useful when the project has to be modified, and it can be a very important contribution to software reusability and portability. The approach to documenting the example models in this text can be used as an example for how to document your models. In addition to good model development documentation, often the simulation model will be used by non-analysts. In this situation, a good user manual for how to use and exercise the model is imperative. The user manual is a product for the user who may not be an expert in programming or simulation issues; therefore clearness and simplicity should be its main characteristics. If within the scope of the project, the analyst should also develop implementation plans and follow through with the installation and integration of the proposed solutions. After implementation, the project should be evaluated as to whether or not the proposed solution met the intended objectives.

In this textbook, we will use an open source library for performing stochastic discrete event simulation called the Kotlin Simulation Library (KSL). The next section provides a brief overview.

1.8. Overview of the Kotlin Simulation Library

The KSL (a Kotlin Simulation Library) is a simulation library for Kotlin. The KSL's current version has packages that support random number generation, statistical collection, basic reporting, and discrete-event simulation modeling via both the event-view and the process-view.

The purpose of the KSL is to support education and research within simulation. Current simulation languages hide the implementation details of the workings of a simulation. As such, students exposed to a simulation language such as (ProModel, Arena, and AutoMod, etc.) are able to learn the practice of simulation without having a detailed understanding of the fundamental mechanisms that enable the technology. This has both advantages and disadvantages. The advantage is that more engineers are capable of using and applying simulation technology to improve systems. The disadvantage is that their modeling abilities depend heavily on a particular software package and simulation modeling has the potential to become a black-box technology. The use of commercial simulation software programs results in proprietary models that have limited interoperability. Many users of commercial simulation software programs can build complicated models, but may have limited understanding of how the simulation actually works.

When teaching simulation, especially at the undergraduate level, simulation languages enable students with introductory programming skills to model systems and perform experiments. Within a typical introductory semester course on simulation it is possible to cover the basics of simulation (random numbers, modeling, and statistical analysis) along with the coverage of a tool (Arena, ProModel, AutoMod, etc.) It is difficult to teach students how simulation works based on a general purpose programming language due to the reduced emphasis on more advanced programming skills especially for non-computer science majors. The KSL assists in bridging this gap by providing a standard library in Kotlin for simulation modeling. One of the fundamental design goals of the KSL is to clearly demonstrate how

one can implement the fundamental mechanisms that enable simulation technology. By studying the implementation details students will gain a deeper understanding of how simulation works. They will become better simulation modelers, practitioners, and users of commercial simulation languages.

The KSL also supports research within simulation. Naturally, the KSL can be used as a modeling tool to simulate complex systems. Simulators, such as a Supply Chain Simulator, can be built based on the KSL framework. In addition, the design of the KSL provides a framework for the testing of simulation artifacts through a well-defined class and interface hierarchy. The structure of the KSL permits the easy switching of various components within a simulation, the event calendar, random number generator, statistics, etc. For example, the efficiency of different event calendars can be tested by providing an event calendar to the KSL that implements the `CalendarIfc` interface. Different algorithms can be "plugged into" the framework for testing.

This book presents an overview of a simulation library for Kotlin (KSL). The library is divided into packages (calendar, controls, modeling, observers, simulation, and utilities). As necessary, these packages may contain sub-packages, which implement various aspects of the library. The KSL is organized as two open source projects: `KSLCore` and `KSLExamples.` Each of these projects is further organized into packages.

`KSLCore`: ksl is the main package that holds all sub-packages

- calendar: The calendar package implements classes that provide event calendar processing
- simulation: The simulation package implements the main classes involved in constructing and running simulation models.
- modeling: The modeling package is the heart of the KSL. In the modeling package, supporting model elements such as queues, resources, variables, responses, etc. are implemented. This package will be discussed in detail in this document.

  - observers: The observers package provides support classes for observing model elements during the simulation run. The KSL is designed to allow each model element to have observers attached to it thereby implementing the classic Observer pattern. Observers can be used to collect statistics, write output to files, display model element state, etc.

  - utilities: The utilities package provides support classes that are used by the KSL. The random, statistics, reporting, and database related sub-packages will be discussed in this document.

`KSLExamples`: examples is the main package that holds all sub-packages

- book: packages containing the examples of this book
- general: examples of the use of various KSL constructs that are not necessarily discussed within the book.

The purpose of the KSL is to provide support for the development of discrete-event simulation programs within Kotlin. This book provides an overview of the functionality and use of the classes found within the KSL. Additional information is available through the documentation API. We begin our discussion in the next chapter with the *utilities* package within the `KSLCore.` These support packages can be used independently of building discrete-event simulation models.

1.9. Simulation as a Profession

To wrap up this introductory chapter, I would like to encourage you to consider simulation as a career. As such, your learning is only getting started with this textbook.

There are a number of topics in simulation that will not be given full discussion within this text. However, based on the material in this text,

you should be well prepared to apply simulation to problems in engineering and business. To continue your education in simulation, you should consider reading (Chung 2004) and (Banks 1998). Chapter 13 of (Chung 2004) provides a number of examples of the application simulation, and Chapters 14-21 of (Banks 1998) illustrate simulation in areas such as health care, the military, logistics, and manufacturing. In addition, Chapters 22 and 23 of (Banks 1998) provide excellent practical advice on how to perform and manage a successful simulation project.

Finally, you should get involved in professional societies and organizations that support simulation modeling and analysis, such as:

- American Statistical Association (ASA)
- Association for Computing Machinery: Special Interest Group on Simulation (ACM/SIGSIM)
- Institute of Electrical and Electronics Engineers: Systems, Man, and Cybernetics Society (IEEE/SMC)
- Institute for Operations Research and the Management Sciences: Simulation Society (INFORMS-SIM)
- Institute of Industrial Engineers (IIE)
- National Institute of Standards and Technology (NIST)
- The Society for Modeling and Simulation International (SCS)

An excellent opportunity to get involved is through the Winter Simulation Conference. You can find information concerning these organizations and the Winter Simulation Conference easily through the internet.

As a final thought you should try to adhere to the following rules during your practice of simulation:

1. To only perform simulation studies that have clearly defined objectives and performance metrics.

2. To never perform a simulation study when the problem can be solved through simpler and more direct techniques.
3. To always carefully plan, collect, and analyze the data necessary to develop good input distributions for simulation models.
4. To never accept simulation results or decisions based on a sample size of one.
5. To always use statistically valid methods to determine the sample size for simulation experiments.
6. To always check that statistical assumptions (e.g. independence, normality, etc.) are reasonably valid before reporting simulation results.
7. To always provide simulation results by reporting the sample size, the sample average, and a measure of sample variation (e.g. sample variance, sample standard deviation, standard error, sample half-width for a specified confidence level, etc.)
8. To always perform simulation experiments with a random number generator that has been tested and proven to be reliable.
9. To take steps to mitigate the effects of initialization bias when performing simulations involving steady-state performance.
10. To always verify simulation models through such techniques as debugging, event tracing, animation, etc.
11. To always validate simulation models through common sense analytical thinking techniques, discussions with domain experts, real-data, and sensitivity analysis techniques.
12. To always carefully plan, perform, and analyze simulation experiments using appropriate experimental design techniques.

By following these rules, you should have a successful and professional simulation career. I hope that you enjoy the book.

1.10. Exercises

Exercise 1.1. Using the resources at (http://www.informs-sim.org/wsc-papers.html) find an application of simulation to a real system and discuss why simulation was important to the analysis.

Exercise 1.2. Customers arrive to a gas station with two pumps. Each pump can reasonably accommodate a total of two cars. If all the space for the cars is full, potential customers will balk (leave without getting gas). What measures of performance will be useful in evaluating the effectiveness of the gas station? Describe how you would collect the inter-arrival and service times of the customers necessary to simulate this system.

Exercise 1.3. Classify the systems as either being discrete or continuous:

- Electrical Capacitor (You are interested in modeling the amount of current in a capacitor at any time t).
- On-line gaming system. (You are interested in modeling the number of people playing Halo 4 at any time t.)
- An airport. (You are interested in modeling the percentage of flights that depart late on any given day).

Exercise 1.4. Classify the systems as either being discrete or continuous:

- Parking lot
- Level of gas in Fayetteville shale deposit
- Printed circuit board manufacturing facility

Exercise 1.5. Classify the systems as either being discrete or continuous:

- Elevator system (You are interested in modeling the number of people waiting on each floor and traveling within the elevators.)
- Judicial system (You are interested in modeling the number of cases waiting for trial.)
- The in-air flight path of an airplane as it moves from an origin to a destination.

Exercise 1.6. What is model conceptualization? Give an example of something that might be produced during model conceptualization.

Exercise 1.7. The act of implementing the model in computer code, including timing and general procedures and the representation of the conceptual model into a computer simulation program is called: _____.

Exercise 1.8. Which of the following does the problem formulation phase of simulation not include? - Define the system - Establish performance metrics - Verification - Build conceptual models

---

Exercise 1.9. *Fill in the blank* The general goals of a simulation include the _____ of system alternatives and their performance measures across various factors (decision variables) with respect to some objectives.

Exercise 1.10. *Fill in the blank* The general goals of a simulation include the _____ of system behavior at some future point in time.

Exercise 1.11. *True* or *False* Verification of the simulation model is performed to determine whether the simulation model adequately represents the real system.

---

2. Modeling Randomness

Learning Objectives

- To be able to generate random numbers using the Kotlin Simulation Library (KSL)
- To be able to control random number streams within the KSL
- To be able to generate random variates using the KSL
- To be able to use the KSL for basic probability computations

This chapter overviews the functionality of the KSL for modeling randomness within simulation models. The focus of this chapter is on getting started using the basic classes and functionality of the KSL. The theory and methods related to random number generation and random variate generation are provided in Appendix A. In that appendix, the underlying theory of the inverse transform method, convolution, acceptance rejection, and particular distribution modeling concepts are reviewed. In addition, the concepts of pseudo-random number generation are discussed. This chapter assumes that the reader has some familiarity with the general concepts presented in Appendix A.

> **Note**
>
> This chapter provides a series of example Kotlin code that illustrates the use of KSL constructs for generating random numbers and random variates. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found

here.

2.1. Random Number Generator

This section discusses how random number generation is implemented within the KSL. The purpose is to present how these concepts can be put into practice.

The random number generator used within the KSL is described in L'Ecuyer et al. (2002) and has excellent statistical properties. It is based on the combination of two multiple recursive generators resulting in a period of approximately 3.1×10^{57}. This is the same generator that is now used in many commercial simulation packages. The generator used in the KSL is defined by the following equations.

$$
\begin{aligned}
R_{1,i} &= (1,403,580R_{1,i-2} - 810,728R_{1,i-3}) \bmod (2^{32} - 209) \\
R_{2,i} &= (527,612R_{2,i-1} - 1,370,589R_{2,i-3}) \bmod (2^{32} - 22,853) \\
Y_i &= (R_{1,i} - R_{2,i}) \bmod (2^{32} - 209) \\
U_i &= \frac{Y_i}{2^{32} - 209}
\end{aligned}
$$

To illustrate how this generator works, consider generating an initial sequence of pseudo-random numbers from the generator. The generator takes as its initial seed a vector of six initial values $(R_{1,0}, R_{1,1}, R_{1,2}, R_{2,0}, R_{2,1}, R_{2,2})$. The first initially generated value, U_i, will start at index 3. To produce five pseudo random numbers using this generator we need an initial seed vector, such as:

$$\{R_{1,0}, R_{1,1}, R_{1,2}, R_{2,0}, R_{2,1}, R_{2,2}\} = \{12345, 12345, 12345, 12345, 12345, 12345\}$$

Using the recursive equations, the resulting random numbers are as follows:

	i=3	i=4	i=5	i=6	i=7
$Z_{1,i-3} =$	12345	12345	12345	3023790853	3023790853
$Z_{1,i-2} =$	12345	12345	3023790853	3023790853	3385359573
$Z_{1,i-1} =$	12345	3023790853	3023790853	3385359573	1322208174
$Z_{2,i-3} =$	12345	12345	12345	2478282264	1655725443
$Z_{2,i-2} =$	12345	12345	2478282264	1655725443	2057415812
$Z_{2,i-1} =$	12345	2478282264	1655725443	2057415812	2070190165
$Z_{1,i} =$	3023790853	3023790853	3385359573	1322208174	2930192941
$Z_{2,i} =$	2478282264	1655725443	2057415812	2070190165	1978299747
$Y_i =$	545508589	1368065410	1327943761	3546985096	951893194
$U_i =$	0.127011122076	0.318527565471	0.309186015655	0.82584686312	0.221629915834

While it is beyond the scope of this book to explore the theoretical underpinnings of this generator, it is important to note that the generator allows multiple independent streams to be defined along with sub-streams.

The fantastic thing about this generator is the sheer size of the period. Based on their analysis, L'Ecuyer et al. (2002) state that it will be "approximately 219 years into the future before average desktop computers will have the capability to exhaust the cycle of the (generator) in a year of continuous computing." In addition to the period length, the generator has an enormous number of streams, approximately 1.8×10^{19} with stream lengths of 1.7×10^{38} and sub-streams of length 7.6×10^{22} numbering at 2.3×10^{15} per stream. Clearly, with these properties, you do not have to worry about overlapping random numbers when performing simulation experiments. The generator was subjected to a rigorous battery of statistical tests and is known to have excellent statistical properties.

2.1.1. Random Package

The concepts within L'Ecuyer et al. (2002) have been implemented within the `ksl.utilities.random.rng` package in the KSL. A key organizing

principle for the `random` package is the use of interfaces. An interface allows classes to act like other classes. It is a mechanism by which a class can promise to have certain behaviors (i.e. methods). The KSL utilizes interfaces to separate random number generation concepts from stream control concepts.

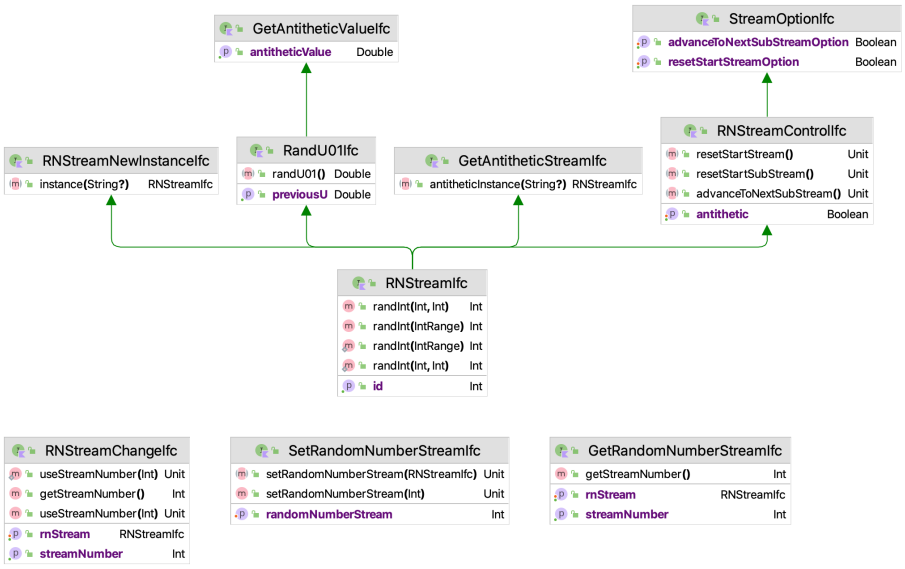


Figure 2.1.: Random Number Stream Interfaces

Figure 2.1 shows the important interfaces within the `ksl.utilities.random.rng` package. The `RandU01Ifc` defines the methods for getting the next pseudo-random number and the previous pseudo-random number via `randU01()` and `previousU`. The `randInt(i: Int, j: Int)` method can be used to generate a random integer uniformly over the range from i to j. The `GetAntitheticStreamIfc` and `RNStreamNewInstanceIfc` interfaces allow a new object instance to be created from the stream. In the case of the `GetAntitheticStreamIfc` interface the created stream will produce antithetic variates from the stream. If U is a pseudo-random number, then

$1 - U$ is the antithetic variate of U.

The `RNStreamControlIfc` defines methods for controlling the underlying stream of pseudo-random numbers.

- `resetStartStream()`: positions the random number stream at the beginning of its sequence. This is the same location in the stream as assigned when the random number stream was created and initialized.
- `resetStartSubstream()`: resets the position of the random number stream to the start of the current substream. If the random number stream has advanced into the substream, then this method resets to the beginning of the substream.
- `advanceToNextSubStream()`: positions the random number stream at the beginning of its next substream. This method move through the current substream and positions the stream at the beginning of the next substream.
- `antithetic` indicates to the stream to start producing antithetic variates. If the option is true, the stream should start producing antithetic variates with the next call to `randU01()`. If the option is false, the stream should stop producing antithetic variates.

The `StreamOptionIfc` defines methods for automating the control of the stream during simulation runs.

- `advanceToNextSubStreamOption`: Indicates that the stream will be advanced to the next substream for the beginning of the next simulation replication.
- `resetStartStreamOption`: Indicates that the underlying stream will be reset to its starting point for the beginning of the next simulation replication.

The `RNStreamIfc` interface assumes that the underlying pseudo-random number generator can produce multiple streams that can be further divided into substreams. The reset methods allow the user to move within the

streams. Classes that implement the `RNStreamControlIfc` can manipulate the streams in a well-defined manner.

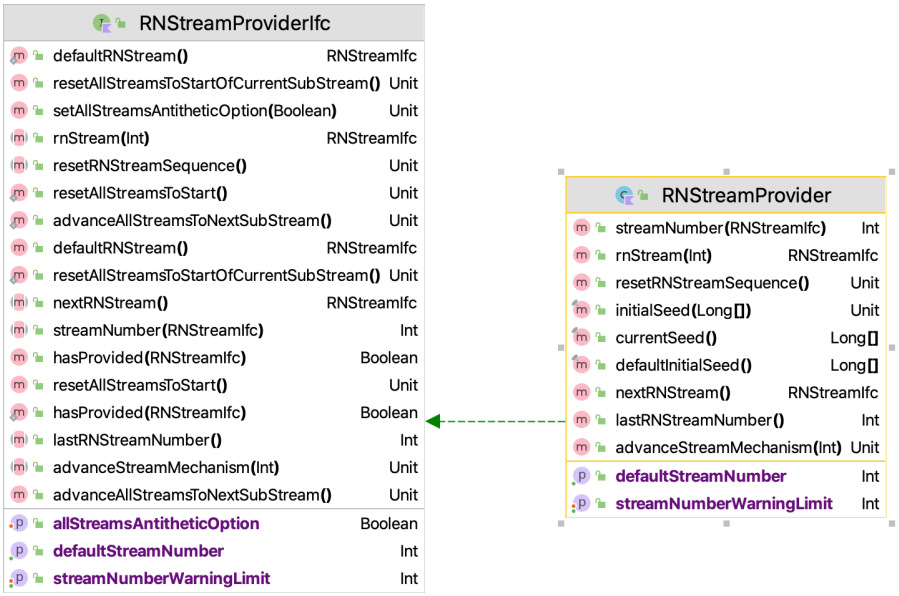


Figure 2.2.: RNStreamProviderIfc Interface

To create a concrete instance of a stream, we must have a random number stream provider. This functionality is defined by the `RNStreamProviderIfc` interface and its default implementation, `RNStreamProvider`. Figure 2.2 illustrates the functionality available for creating random number streams. This interface conceptualizes the creation of random number streams as a process of making a sequence of streams numbered 1, 2, 3, ...

A random number stream provider must define a default stream, which can be retrieved via the `defaultRNStream()` method. By default, for the KSL, the default stream is the first stream created and is labeled with

the sequence number 1. The sequence number of a stream can be used to retrieve a particular stream from the provider. The following methods allow for creation and access to streams.

- `nextRNStream()`: returns the next random number stream associated with the provider. Each call to `nextRNStream()` makes a new stream in the sequence of streams.
- `lastRNStreamNumber()`: returns the number of the stream that was last made. This indicates how many streams have been made. If 0 is returned, then no streams have been made by the provider.
- `rnStream(k : Int)`: returns the k^{th} stream. If k is greater than `lastRNStreamNumber()` then `lastRNStreamNumber()` is advanced according to the additional number of streams by creating any intermediate streams. For example, if `lastRNStreamNumber()` = 10 and k = 15, then streams 11, 12, 13, 14, 15 are assumed provided and stream 15 is returned and `lastRNStreamNumber()` now equals 15. If k is less than or equal to `lastRNStreamNumber()`, then no new streams are created and `lastRNStreamNumber()`stays at its current value and the k^{th} stream is returned.
- `streamNumber(RNStreamIfc stream)`: returns the stream number of the instance of a stream.
- `advanceStreamMechanism(int n)`: advances the underlying stream mechanism by the specified number of streams, without actually creating the streams. The value of `lastRNStreamNumber()` remains the same after advancing through the streams. In other words, this method should act as if `nextRNStream()` was not called but advance the underlying stream mechanism as if n streams had been provided.
- `resetRNStreamSequence()`: Causes the random number stream provider to act as if has never created any streams. Thus, the next call to `nextRNStream()` will return the 1^{st} stream.

The random number stream provider also facilitates the control of all streams that have been created. This functionality is similar to how the position within an individual stream can be manipulated, except the

provider performs the functionality on *all streams* that it has provided The following methods perform this functionality.

- `resetAllStreamsToStart()`: resets all created streams to the start of their stream.
- `resetAllStreamsToStartOfCurrentSubStream()`: resets all created streams to the start of their current sub-stream.
- `advanceAllStreamsToNextSubstream()`: advances all created streams to the start of their next sub-stream.
- `setAllStreamsAntitheticOption(option: Boolean)`: changes all created streams to have their antithetic option either off = false or on = true.

Many random number generators require the specification of a seed to start the generated sequence. Even though the generator within the KSL uses seeds, there really is no need to specify the seeds because of the well defined methods for moving within the streams. However, if you really want to get into the details and specify seeds, the you should review the `RNStreamFactory` class, which directly implements the random number generator discussed in L'Ecuyer et al. (2002).

Now, let's illustrate how to create and manipulate streams.

2.1.2. Creating and Using Streams

To create a random number stream, the user must utilize an instance of `RNStreamProvider.` This process is illustrated in the following code. This code creates two instances of `RNStreamProvider` and gets the first stream from each instance. The instances of `RNStreamProvider` use the exact same underlying default seeds. Thus, they produce *exactly the same* sequence of streams.

Example 2.1 (RNStreamProvider)**.** This example illustrates how to create a `RNStreamProvider`, create streams, and generate pseudo-random numbers from the stream.

```
fun main() {
    // make a provider for creating streams
    val p1 = RNStreamProvider()
    // get the first stream from the provider
    val p1s1 = p1.rnStream(streamNum = 1)
    // make another provider, the providers are identical
    val p2 = RNStreamProvider()
    // Thus, the first streams returned are identical
    val p2s1 = p2.rnStream(streamNum = 1)
    print(String.format("%3s %15s %15s %n", "n", "p1s1",
 ↪  "p2s1"))
    for (i in 1..5) {
        print(String.format("%3d %15f %15f %n", i,
 ↪  p1s1.randU01(), p2s1.randU01()))
    }
}
```

Thus, in the following code output, the randomly generated values are exactly the same for the two streams.

```
  n            p1s1            p2s1
  1        0.127011        0.127011
  2        0.318528        0.318528
  3        0.309186        0.309186
  4        0.825847        0.825847
  5        0.221630        0.221630
```

There is really very little need for the general programmer to create a `RNStreamProvider` because the KSL supplies a default provider that can

be used to provide a virtually infinite number of streams. The need for directly accessing the functionality of `RNStreamProvider` is for explicit control of stream creation in such situations like running code on different computers in parallel. While the providers produce the same streams, you can force one provider to be different from another provider by manipulating the seeds. The function, `advanceStreamMechanism(n: Int),` essentially manipulates the underlying seed vector by advancing the seed values as if `n` streams have been created. If this function is used prior to creating any streams, you have essentially set the starting seed value, causing the created streams to be *different* from the default starting values. In addition, the provider can control all streams that it produces. So, unless you are trying to do some advanced work that involves coordinating multiple streams, you should not need to create multiple instances of `RNStreamProvider.`

Because the most common use case is to just have a single provider of streams, the KSL facilitates this through the `KSLRandom` *object.* The `KSLRandom` object has a wide range of methods to facilitate random variate generation.The most important methods include:

- `nextRNStream()`: calls the underlying default `RNStreamProvider` to create a new random number stream
- `rnStream(int k)`: returns the k^{th} stream from the default `RNStreamProvider`
- `defaultRNStream()`: calls the underlying default `RNStreamProvider` for its default stream

In the following code example, these methods are used to create streams that are used to generate random numbers. The first line of the code uses the static method `defaultRNStream()` of `KSLRandom` to get the default stream and then generates three random numbers. The stream is then advanced and three new random numbers are generated. Then,the stream is reset to its starting (initial seed) and it then repeats the original values. Finally, the a new stream is created via `KSLRandom.nextRNStream()` and then used to generate new random numbers. From a conceptual standpoint, each stream contains an independent sequence of random numbers from

any other stream (unless of course they are made from different providers). They are conceptually infinite and independent due to their enormous periods.

Example 2.2 (KSLRandom)**.** This example illustrates how to use the `KSLRandom` class to create streams, advance the stream, and reset the stream.

```
fun main() {
    val s1 = KSLRandom.defaultRNStream()
    println("Default stream is stream 1")
    println("Generate 3 random numbers")
    for (i in 1..3) {
        println("u = " + s1.randU01())
    }
    s1.advanceToNextSubStream()
    println("Advance to next sub-stream and get some more
↪   random numbers")
    for (i in 1..3) {
        println("u = " + s1.randU01())
    }
    println("Notice that they are different from the first
↪   3.")
    s1.resetStartStream()
    println("Reset the stream to the beginning of its
↪   sequence")
    for (i in 1..3) {
        println("u = " + s1.randU01())
    }
    println("Notice that they are the same as the first 3.")
    println("Get another random number stream")
    val s2 = KSLRandom.nextRNStream()
    println("2nd stream")
    for (i in 1..3) {
```

```
        println("u = " + s2.randU01())
    }
    println("Notice that they are different from the first
 ↪  3.")
}
```

The resulting output from this code is as follows. Again, the methods of the `RNStreamControlIfc` interface that permit movement within a stream are extremely useful for controlling the randomness associated with a simulation.

```
Default stream is stream 1
Generate 3 random numbers
u = 0.12701112204657714
u = 0.3185275653967945
u = 0.3091860155832701
Advance to next sub-stream and get some more random numbers
u = 0.07939898979733463
u = 0.4803395047575741
u = 0.8583222470551328
Notice that they are different from the first 3.
Reset the stream to the beginning of its sequence
u = 0.12701112204657714
u = 0.3185275653967945
u = 0.3091860155832701
Notice that they are the same as the first 3.
Get another random number stream
2nd stream
u = 0.7595818622487196
u = 0.9783105732613708
u = 0.6851358081931826
Notice that they are different from the first 3.
```

2.1.3. Common Random Numbers

Common random numbers (CRN) is a Monte Carlo method that has different experiments utilize the same random numbers. CRN is a variance reduction technique, discussed more fully in Chapter 9, that allows the experimenter to block out the effect of the random numbers used in the experiment. To facilitate the use of common random numbers the KSL has the aforementioned stream control mechanisms. One way to implement common random numbers is to use two instances of `RNStreamProvider` as was previously illustrated. In that case, the two providers produce the same sequence of streams and thus those streams can be used on the different experiments. An alternative method that does not require the use of two providers is to create a copy of the stream directly from the stream instance. The following code clones the stream instance.

Example 2.3 (Cloning for CRN)**.** This example clones an instance of a stream and uses it to generate common random numbers.

```
fun main() {
    // get the default stream
    val s = KSLRandom.defaultRNStream()
    // make a clone of the stream
    val clone = s.crnInstance()
    print(String.format("%3s %15s %15s %n", "n", "U", "U
 ↪  again"))
    for (i in 1..3) {
        print(String.format("%3d %15f %15f %n", i,
 ↪  s.randU01(), clone.randU01()))
    }
}
```

Since the instances have the same underlying state when the clone is created, they produce the same random numbers. Please note that the cloned stream

instance is **not provided** by the underlying `RNStreamProvider` and thus it is not part of the set of streams managed or controlled by the provider.

```
  n                U          U again
  1         0.127011         0.127011
  2         0.318528         0.318528
  3         0.309186         0.309186
```

An alternative method is to just use the `resetStartStream()` method of the stream to reset the stream to the desired location in its sequence and then reproduce the random numbers. This is illustrated in the following code.

Example 2.4 (Resetting the stream for CRN)**.** This example resets the stream to generate common random numbers.

```
fun main() {
    val s = KSLRandom.defaultRNStream()
    // generate regular
    print(String.format("%3s %15s %n", "n", "U"))
    for (i in 1..3) {
        val u = s.randU01()
        print(String.format("%3d %15f %n", i, u))
    }
    // reset the stream and generate again
    s.resetStartStream()
    println()
    print(String.format("%3s %15s %n", "n", "U again"))
    for (i in 1..3) {
        val u = s.randU01()
        print(String.format("%3d %15f %n", i, u))
    }
}
```

Notice that the generated numbers are the same.

```
n               U
1        0.127011
2        0.318528
3        0.309186

n         U again
1        0.127011
2        0.318528
3        0.309186
```

Thus, an experiment can be executed, then the random numbers reset to the desired location. Then, by changing the experimental conditions and re-running the simulation, the same random numbers are used. If many streams are used, then by accessing the `RNStreamProvider` you can reset all of the controlled streams with one call and then perform the next experiment.

2.1.4. Creating and Using Antithetic Streams

Recall that if a pseudo-random number is called U then its antithetic value is $1 - U$. There are a number of methods to access antithetic values. The simplest is to create an antithetic instance from a given stream. This is illustrated in the following code. Please note that the antithetic stream instance is not provided by the underlying `RNStreamProvider` and thus it is not part of the set of streams managed or controlled by the provider. The new instance functionality directly creates the new stream based on the current stream so that it has the same underling state and it is set to produce antithetic values.

Example 2.5 (Generating Antithetic Numbers)**.** This example illustrates how to create an antithetic instance of a stream and generate antithetic random numbers.

```
fun main() {
    // get the default stream
    val s = KSLRandom.defaultRNStream()
    // make its antithetic version
    val ans = s.antitheticInstance()
    print(String.format("%3s %15s %15s %15s %n", "n", "U",
 ↪  "1-U", "sum"))
    for (i in 1..5) {
        val u = s.randU01()
        val au = ans.randU01()
        print(String.format("%3d %15f %15f %15f %n", i, u,
 ↪  au, u + au))
    }
}
```

Notice that the generated values sum to 1.0.

n	U	1-U	sum
1	0.127011	0.872989	1.000000
2	0.318528	0.681472	1.000000
3	0.309186	0.690814	1.000000
4	0.825847	0.174153	1.000000
5	0.221630	0.778370	1.000000

An alternate method that does not require the creation of another stream involves using the `resetStartStream()` method and the `antithetic` property of the current stream. If you have a stream, you can use the `antithetic` property to cause the stream to start producing antithetic values. If you use the `resetStartStream()` method and then set the

antithetic option to true, the stream will be set to its initial starting point and then produce antithetic values.

Example 2.6 (Resetting for Antithetic Numbers)**.** This example illustrates how to create an antithetic instance of a stream and generate antithetic random numbers.

```
fun main() {
    val s = KSLRandom.defaultRNStream()
    s.resetStartStream()
    // generate regular
    print(String.format("%3s %15s %n", "n", "U"))
    for (i in 1..5) {
        val u = s.randU01()
        System.out.printf("%3d %15f %n", i, u)
    }
    // generate antithetic
    s.resetStartStream()
    s.antithetic = true
    println()
    print(String.format("%3s %15s %n", "n", "1-U"))
    for (i in 1..5) {
        val u = s.randU01()
        print(String.format("%3d %15f %n", i, u))
    }
}
```

Notice that the second set of random numbers is the antithetic complement of the first set in this output. Of course, you can also create multiple instances of `RNStreamProvider`, and then create streams and set one of the streams to produce antithetic values.

```
  n               U
```

```
1         0.127011
2         0.318528
3         0.309186
4         0.825847
5         0.221630

n              1-U
1         0.872989
2         0.681472
3         0.690814
4         0.174153
5         0.778370
```

2.1.5. Frequently Asked Questions about Random Numbers

1. **What are pseudo-random numbers?** Numbers generated through an algorithm that appear to be random, when in fact, they are created by a deterministic process.

2. **Why do we want to control randomness within simulation models?** By controlling randomness, we can better understand if changes in simulation responses are due to factors of interest or due to underlying statistical variation caused by sampling. Do you think that it is better to compare two systems using the same inputs or different inputs? Suppose we have a work process that we have redesigned. We have the old process and the new process. Would it be better to test the difference in the process by using two different workers or the same worker? Most people agree that using the same worker is better. This same logic applies to randomness. Since we can control which pseudo-random numbers we use, it is better to test the difference between two model alternatives by using the same pseudo-random numbers. We use seeds and streams to do this.

3. **What are seeds and streams?** A random number stream is a sub-sequence of pseudo-random numbers that start at particular place within a larger sequence of pseudo-random numbers. The starting point of a sequence of pseudo-random numbers is called the seed. A seed allows us to pick a particular stream. Having multiple streams is useful to assign different streams to different sources of randomness within a model. This facilitates the control of the use of pseudo-random numbers when performing experiments.

4. **How come my simulation results are always the same?** Random number generators in computer simulation languages come with a default set of streams that divide the "circle" up into independent sets of random numbers. The streams are only independent if you do not use up all the random numbers within the subsequence. These streams allow the randomness associated with a simulation to be controlled. During the simulation, you can associate a specific stream with specific random processes in the model. This has the advantage of allowing you to check if the random numbers are causing significant differences in the outputs. In addition, this allows the random numbers used across alternative simulations to be better synchronized. Now a common question in simulation can be answered. That is, "If the simulation is using random numbers, why to I get the same results each time I run my program?" The corollary to this question is, "If I want to get different random results each time I run my program, how do I do it?" The answer to the first question is that the underlying random number generator is starting with the same seed each time you run your program. Thus, your program will use the same pseudo random numbers today as it did yesterday and the day before, etc. The answer to the corollary question is that you must tell the random number generator to use a different seed (or alternatively a different stream) if you want different invocations of the program to produce different results. The latter is not necessarily a desirable goal. For example, when developing your simulation programs, it is desirable to have repeatable results so that you can know that your

program is working correctly.

5. **How come my simulation results are unexpectedly different?** Sometimes by changing the order of method calls you change the sequence of random numbers that are assigned to various things that happen in the model (e.g. attribute, generated service times, paths taken, etc.). Please see the item (4) "How come my results are always the same?". Now, the result can sometimes be radically different if different random numbers are used for different purposes. By using specific streams assigned to specific random processes, you reduce this possibility and increase the likelihood that two models that have different configurations will have differences due to the change and not due to the random numbers used.

2.2. Random Variate Generation

This section will overview how to generate random variates using the KSL. A random variate is an observation of a random variable from some probability distribution. Appendix A describes four basic methods for generating random variates:

- inverse transform
- convolution
- acceptance/rejection
- mixture distributions

The KSL provides classes that facilitate these methods (and others).

2.2.1. Basic Random Variate Generation

In this section, we provide a couple of examples of generating random variates from "first principles". The examples follow closely those presented in Appendix A.

Example 2.7 (Generating Exponential Random Variates)**.** Consider a random variable, X, that represents the time until failure for a machine tool. Suppose X is exponentially distributed with an expected value of $1.\overline{33}$. Generate a random variate for the time until the first failure using a uniformly distributed value of $u = 0.7$.

```
fun main() {
    val u = 0.7
    val mean = 1.333333
    val x = rExpo(mean, u)
    println("Generated X = $x")
}

fun rExpo(mean: Double, u: Double) : Double {
    return -mean*ln(1.0 - u)
}
```

The function `rExpo` implements the inverse CDF function for the exponential distribution for any mean and any value of u. By calling the function with the appropriate parameters, we can easily generate exponential random variates. Just as in Example A.2 of Appendix A the result is 1.6053. In this example, the value of u was given; however, it could have easily been generated using an instance of the `RNStreamIfc` interface and the `randU01()` function.

This next example illustrates how to generate negative binomial random variates via convolution.

Example 2.8 (Negative Binomial Variates Via Convolution)**.** Use the following pseudo-random numbers $u_1 = 0.35$, $u_2 = 0.64$, $u_3 = 0.14$, generate a random variate from a shifted negative binomial distribution having parameters $r = 3$ and $p = 0.3$.

```
fun main() {
    val u1 = 0.35
    val u2 = 0.64
    val u3 = 0.14
    val p = 0.3
    val x1 = rGeom(p, u1)
    val x2 = rGeom(p, u2)
    val x3 = rGeom(p, u3)
    val x = x1 + x2 + x3
    println("Generated X = $x")
}

fun rGeom(p: Double, u: Double): Double {
    val n = ln(1.0 - u)
    val d = ln(1.0 - p)
    return 1.0 + floor(n / d)
}
```

Recall from Appendix A that we refer to the geometric distribution as the shifted geometric distribution, when $X_i \sim Shifted\ Geometric(p)$ with range $\{1, 2, 3, ...\}$, and X_i can be generated via inverse transform with:

$$X_i = 1 + \left\lfloor \frac{ln(1 - U_i)}{ln(1 - p)} \right\rfloor$$

Note also that the shifted negative binomial distribution is just the sum of shifted geometric random variables. The code implements a function to generate a shifted geometric random variate. Because we need a shifted negative binomial with $r = 3$ and $p = 0.3$, the function is used three times to generate three shifted geometric random variates with $p = 0.3$. The observed values are summed to generate the shifted negative binomial. Just as in Example A.6 of Appendix A the result is 6.0.

As we will see in the next section, the concepts of Appendix A have been generalized and implemented within the `ksl.utilities.random` package for various common types of random variables.

2.2.2. Continuous and Discrete Random Variables

The KSL has the capability to generate random variates from both discrete and continuous distributions. The `ksl.utilities.random.rvariable` package supports this functionality. The package has a set of interfaces that define the behavior associated with random variables. Concrete sub-classes of specific random variables are created by sub-classing `RVariable.` As shown in Figure 2.3, every random variable has access to an object that implements the `RNStreamIfc` interface. This gives it the ability to generate pseudo-random numbers and to control the streams. The `GetValueIfc` interface is the key interface because in this context it returns a random value from the random variable. For example, if `d` is a reference to an instance of a sub-class of type `RVariable`, then `d.value` generates a random value.

The names and parameters (based on common naming conventions) associated with the continuous random variables are as follows:

- `BetaRV(alpha1, alpha2)`
- `ChiSquaredRV(degreesOfFreedom)`
- `ExponentialRV(mean)`
- `GammaRV(shape, scale)`
- `GeneralizedBetaRV(alpha1, alpha2, min, max)`
- `JohnsonBRV(alpha1, alpha2, min, max)`
- `LaplaceRV(mean scale)`
- `LogLogisticRV(shape, scale)`
- `LognormalRV(mean, variance)`
- `NormalRV(mean, variance)`
- `PearsonType5RV(shape, scale)`
- `PearsonType6RV(alpha1, alpha2, beta)`

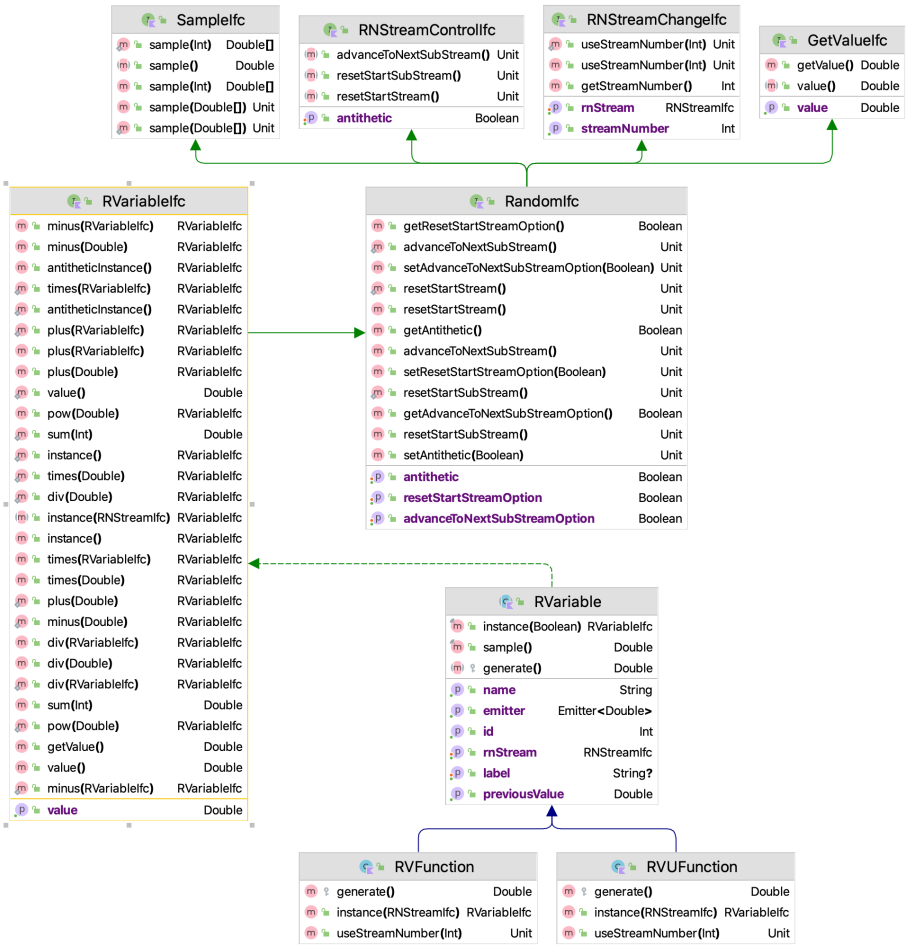


Figure 2.3.: Random Variable Interfaces

- `StudentTRV(degreesOfFreedom)`
- `TriangularRV(min, mode, max)`
- `UniformRV(min, max)`
- `WeibullRV(shape, scale)`

> **Note**
>
> Be careful to note that the normal and lognormal random variables are specified with the mean and the **variance** (not the standard deviation). Because Kotlin has named parameters, a recommended good practice is to use the parameter names when creating the object instances. This reduces the chances that you have confused the parameters.

The names of the discrete random variables are as follows:

- `BernoulliRV(probOfSuccess)`
- `BinomialRV(pSucces, numTrials)`
- `ConstantRV(constVal)` a degenerate probability mass on a single value that cannot be changed
- `DEmpiricalRV(values, cdf)` values is an array of numbers and cdf is an array representing the cumulative distribution function over the values
- `DUniformRV(min, max)`
- `GeometricRV(probOfSucces)` with range is 0 to infinity
- `NegativeBinomialRV(probOfSuccess, numSuccess)` with range is 0 to infinity. The number of failures before the r^{th} success.
- `PoissonRV(mean)`
- `ShiftedGeometricRV(probOfSucces)` range is 1 to infinity
- `VConstantRV(constVal)` a degenerate probability mass on a single value that can be changed

All classes that represent random variables also have optional parameters to specify the stream number and the random number stream provider.

As noted for the `NormalRV` constructor, we see that the third parameter, called `streamNum,` is used to specify the random number stream. The default stream provider is the provider associated with `KSLRandom.`

```
class NormalRV @JvmOverloads constructor(
    val mean: Double = 0.0,
    val variance: Double = 1.0,
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc =
     ↪  KSLRandom.DefaultRNStreamProvider,
    name: String? = null
) : ParameterizedRV(streamNum, streamProvider, name)
```

If the stream number is not provided (i.e. the default of 0), then the next stream from the default provider is allocated to the new instance of the random variable. Thus, unless the programming specifically assigns streams, all random variables are automatically constructed such that they use different underlying streams.

As you can see, the name of the distribution followed by the letters RV designate the class names. Implementations of these classes extend the `RVarable` class, which implements the `RVariableIfc` interface. Users simply create an instance of the class and then use it to get a sequence of values that have the named probability distribution. In order to implement a new random variable (i.e. some random variable that is not already implemented) you can extend the class `RVariable.` This provides a basic template for what is expected in the implementation. However, it implies that you need to implement all of the required interfaces. The key method to implement is the protected `generate()` method, which should return the generated random value according to some algorithm.

In almost all cases, the KSL utilizes the inverse transform algorithm for generating random variates. Thus, there is a one to one mapping of the underlying pseudo-random number and the resulting random variate.

Even in the case of distributions that do not have closed form inverse cumulative distribution functions, the KSL utilizes numerical methods to approximate the function whenever feasible. For example, the KSL uses a rational function approximation, see (Cody 1969), to implement the inverse cumulative distribution function for the standard normal distribution. The inversion for the gamma distribution is based on Algorithm AS 91 for inverting the chi-squared distribution and exploiting its relationship with the gamma distribution. The beta distribution also uses numerical methods to compute the cumulative distribution function as well as bi-section search to determine the inverse for the cumulative distribution function.

The KSL implements the `BernoulliRV, DUniformRV, GeometricRV, NegativeBinomialRV,` and `ShiftedGeometricRV` classes using the methods described in Chapter 2 of Rossetti (2015). While more efficient methods may be available, the `PoissonRV` and `BinomialRV` distributions are implemented by searching the probability mass functions. Both search methods use an approximation to get close to the value of the inverse and then search up or down through the cumulative distribution function. Because of this both distributions use numerically stable methods to compute the cumulative distribution function values. The `DEmpiricalRV` class also searches through the cumulative distribution function.

The following sections will overview the generation algorithms and provide examples for using some of these distributions.

2.2.3. Creating and Using Random Variables

This section presents how to create and use random variables via KSL constructs. The basic approach is to create an instance of a specific type of random variable and use the instance to generate observations.

Example 2.9 (Generating Normal Random Variates)**.** The following example code illustrates how to create a normal random variable and how to generate values using the `value` property. It is important to note that

the `value` property will return a newly generated value each time that it is accessed.

```
fun main() {
    // create a normal mean = 20.0, variance = 4.0 random
     ↪  variable,
    // using stream number 1
    val n = NormalRV(20.0, 4.0, streamNum = 1)
    print(String.format("%3s %15s %n", "n", "Values"))
    // generate some values
    for (i in 1..5) {
        // the value property returns a generated value
        val x = n.value
        print(String.format("%3d %15f %n", i, x))
    }
}
```

The resulting output is what you would expect.

```
  n          Values
  1       17.718732
  2       19.056360
  3       19.003682
  4       21.875759
  5       18.466600
```

Alternatively, the user can use the `sample()` method to generate an array of values that can be later processed. The `SampleIfc` interface has the following useful functions:

- `sample()` returns a new value
- `sample(sampleSize: Int)` returns an array of values
- `sampleInto(values: DoubleArray)` fills an array with values

- `sampleAsRows(sampleSize: Int, nRows: Int = 1)` makes a 2-D array with samples in the rows
- `sampleAsColumns(sampleSize: Int, nRows: Int = 1)` makes a 2-D array with samples in the columns
- `sampleInto(matrix: Array<DoubleArray>)` fills a 2-D array with samples

> **Note**
>
> Notice that the previous examples created the random variable **outside** of the loop that is used to generate values. This implements the best practice of creating an object once and using it *many* times. The `NormalRV` class was used outside of the loop to create the object and then used many times within the loop to get new random variates. It may not be an error to create random variables within loops, but it is generally bad practice and inefficient in terms of object creation. If you see yourself creating many instances of an object and then using it only one time before it is discarded, you should reconsider what you are doing.

The following code illustrates how to use the `sample()` function with a triangular distribution.

Example 2.10 (Using the sample() function)**.** The following example code illustrates how to create a triangular random variable and how to generate values using the `sample()` function.

```
fun main() {
    // create a triangular random variable with min = 2.0,
     ↪  mode = 5.0, max = 10.0
    val t = TriangularRV(2.0, 5.0, 10.0, streamNum = 1)
    // sample 5 values
    val sample = t.sample(5)
    print(String.format("%3s %15s %n", "n", "Values"))
```

```
    for (i in sample.indices) {
        print(String.format("%3d %15f %n", i + 1, sample[i]))
    }
}
```

Again, the output is what we would expect.

```
  n           Values
  1         3.745929
  2         4.764898
  3         4.724053
  4         7.360658
  5         4.306321
```

> **Note**
>
> The `SampleIfc` interface provides functionality to generate arrays of observations. The KSL provides extensive functionality for working with arrays within its utilities. This functionality is briefly discussed in Section E.7 of Appendix E.

It is important to note that the full range of functionality related to stream control is also available for random variables. That is, the underlying stream can be reset to its start, can be advanced to the next substream, can generate antithetic variates, etc. By default, if a random variable is created without specifying a stream number, the new instance of the random variable is supplied with its own *unique* stream. Since the underlying random number generator has an enormous number of streams, approximately 1.8×10^{19}, it is very unlikely that the user will create so many streams as to start reusing them. However, the streams that are used by random variable instances can be supplied directly so that they may be shared by specifying a stream number.

Example 2.11 (Specifying a Stream)**.** The following code example illustrates how to assign a specific stream by passing a specific stream number into the constructor of the random variable.

```
fun main() {
    // create a normal mean = 20.0, variance = 4.0, with the
     ↪  stream
    val n = NormalRV(20.0, 4.0, streamNum = 3)
    print(String.format("%3s %15s %n", "n", "Values"))
    for (i in 1..5) {
        // value property returns generated values
        val x = n.value
        print(String.format("%3d %15f %n", i, x))
    }
}
```

Within the underlying code, the previous example uses the default provider of the `KSLRandom` class to get stream 3 when constructing the instance of the normal random variable.

A discrete empirical random variable requires the specification of the values and their probabilities. This requires a little more setup. The user must supply the set of values that can be generated as well as an array holding the *cumulative distribution probability* across the values. The following code illustrates how to do this.

Example 2.12 (Using a DEmpirical Random Variable)**.** The values are provided via an array and the probabilities must be specified in the form of the cumulative probabilities, such that the last element is 1.0.

```
fun main() {
    // values is the set of possible values
    val values = doubleArrayOf(1.0, 2.0, 3.0, 4.0)
```

```
    // cdf is the cumulative distribution function over the
     ↪  values
    val cdf = doubleArrayOf(1.0 / 6.0, 3.0 / 6.0, 5.0 / 6.0,
     ↪  1.0)
    //create a discrete empirical random variable
    val n1 = DEmpiricalRV(values, cdf, streamNum = 1)
    println(n1)
    print(String.format("%3s %15s %n", "n", "Values"))
    for (i in 1..5) {
        print(String.format("%3d %15f %n", i, n1.value))
    }

    println()
    println(n1.probabilityPoints)
}
```

The results are as follows:

```
DEmpiricalRV(values=[1.0, 2.0, 3.0, 4.0], cdf=[0.16666666666666666, 0.5, 0.833333333
  n          Values
  1        1.000000
  2        2.000000
  3        2.000000
  4        3.000000
  5        2.000000
```

Section A.2.4 of Appendix A defines mixture and truncated distributions. The following provides examples of how to create and use random variables from these distributions. The examples follow those presented in Section A.2.4.

Example 2.13 (Mixture Distribution)**.** Suppose the time that it takes to pay with a credit card, X_1, is exponentially distributed with a mean of 1.5

minutes and the time that it takes to pay with cash, X_2, is exponentially distributed with a mean of 1.1 minutes. In addition, suppose that the chance that a person pays with credit is 70%. Then, the overall distribution representing the payment service time, X, has an hyper-exponential distribution with parameters $\omega_1 = 0.7$, $\omega_2 = 0.3$, $\lambda_1 = 1/1.5$, and $\lambda_2 = 1/1.1$. Generate 5 random variates from this distribution.

```
fun main() {
    // rvs is the list of random variables for the mixture
    val rvs = listOf(ExponentialRV(1.5), ExponentialRV(1.1))
    // cdf is the cumulative distribution function over the
     ↪  random variables
    val cdf = doubleArrayOf(0.7, 1.0)
    //create a mixture random variable
    val he = MixtureRV(rvs, cdf, streamNum = 1)
    print(String.format("%3s %15s %n", "n", "Values"))
    for (i in 1..5) {
        print(String.format("%3d %15f %n", i, he.value))
    }
}
```

```
  n          Values
  1        0.575249
  2        2.621730
  3        1.143410
  4        0.659060
  5        2.114972
```

The `MixtureRV` class requires a list of random variables and a specification of the probability associated with each random variable in the form of a cumulative distribution function. In the provided code sample, a list is created holding the two exponential random variables. Since the distribution

associated with the credit card payment is first in the list, the probability of 0.7 is specified first in the `cdf` array.

The next example illustrates how to generate from a truncated distribution. A *truncated distribution* is a distribution derived from another distribution for which the range of the random variable is restricted. Suppose we have a random variable, X with PDF, $f(x)$ and CDF $F(x)$. Suppose that we want to constrain $f(x)$ over interval $[a, b]$, where $a < b$ and the interval $[a, b]$ is a subset of the original support of $f(x)$. Note that it is possible that $a = -\infty$ or $b = +\infty$. This leads to a random variable from a truncated distribution $F^*(x)$ having CDF:

$$F^*(x) = \begin{cases} 0 & \text{if } x < a \\ \frac{F(x)-F(a)}{F(b)-F(a)} & a \leq x \leq b \\ 0 & \text{if } b < x \end{cases}$$

This leads to a straight forward algorithm for generating from $F^*(x)$ as follows:

1. Generate $u \sim U(0, 1)$
2. $W = F(a) + (F(b) - F(a))u$
3. $X = F^{-1}(W)$
4. return X

To implement this algorithm, we need the original CDF $F(x)$, its range, and the truncated range $[a, b]$.

Example 2.14 (Truncated Distribution)**.** Suppose X represents the distance between two cracks in highway. Suppose that X has an exponential distribution with a mean of 10 meters. Generate 5 random distance values restricted between 3 and 6 meters.

```
fun main() {
    val cdf = Exponential(mean = 10.0)
    val rv = TruncatedRV(cdf, Interval(0.0,
     ↪ Double.POSITIVE_INFINITY),
        Interval(3.0, 6.0), streamNum = 1)
    print(String.format("%3s %15s %n", "n", "Values"))
    for (i in 1..5) {
        print(String.format("%3d %15f %n", i, rv.value))
    }
}
```

Note that the first line defines a *distribution* and not a random variable. The exponential distribution has range $[0, +\infty]$. In this situation, we are truncating the distribution over the range $[a = 3, b = 6]$. The `TruncatedRV` class implements the aforementioned algorithm. The class requires an invertible CDF, the original range of the CDF, and the truncated range. In the code, an instance of an exponential distribution (see Section 2.3) is created to provide the cumulative distribution function, $F(x)$. This allows the implementation to compute $F(a)$ and $F(b)$. Then, instances of the `Interval` class are used to define the original range and the truncated range. As we can see from the following results, the values are limited to the range of $[3, 6]$.

```
  n          Values
  1        3.334730
  2        3.861643
  3        3.835288
  4        5.408551
  5        3.591583
```

Appendix A also defines a shifted random variable. Suppose X has a given distribution $f(x)$, then the distribution of $X + \delta$ is termed the shifted

distribution and is specified by $g(x) = f(x - \delta)$. It is easy to generate from a shifted distribution, simply generate X according to $F(x)$ and then add δ. The KSL implements this idea via the `ShiftedRV` class. The following example from Appendix A illustrates how simple it is to use a shifted random variable.

Example 2.15 (Shifted Random Variable)**.** Suppose X represents the time to setup a machine for production. From past time studies, we know that it cannot take any less than 5.5 minutes to prepare for the setup and that the time after the 5.5 minutes is random with a Weibull distribution with shape parameter $\alpha = 3$ and scale parameter $\beta = 5$. Generate 5 random observations of the setup time.

```
fun main() {
    val w = WeibullRV(shape = 3.0, scale = 5.0)
    val rv = ShiftedRV(5.0, w, streamNum = 1)
    print(String.format("%3s %15s %n", "n", "Values"))
    for (i in 1..5) {
        print(String.format("%3d %15f %n", i, rv.value))
    }
}
```

Notice that the `ShiftedRV` class requires the shift parameter and the random variable that is to be shifted.

This next example is a bit more complex. Recall from Appendix A that the acceptance/rejection algorithm is a general purpose method for generating from any probability distribution, $f(x)$. The method requires a majorizing function, $g(x)$, such that $g(x) \geq f(x)$ for $-\infty < x < +\infty$. From the majorizing function, we compute its area:

$$c = \int_{-\infty}^{+\infty} g(x)dx$$

We then define c as the majorizing constant. Using the majorizing constant and majorizing function, we can define a proposal distribution, $w(x)$, where $w(x)$ is defined as $w(x) = g(x)/c$. To summarize, the acceptance/rejection algorithm requires three things:

- $f(x)$ The PDF from which random variates need to be generated.
- $w(x)$ The proposal distribution to generate possible variates for acceptance or rejection.
- c The majorizing constant (or area under the majorizing function) to be used in the acceptance/rejection test.

The `AcceptanceRejectionRV` class implements the acceptance/rejection algorithm given these three components. Let's explore these concepts via an example.

Example 2.16 (Acceptance-Rejection Random Variable)**.** Consider the following PDF over the range $[-1, 1]$. Generate 1000 random variates from this distribution.

$$f(x) = \begin{cases} \frac{3}{4}(1 - x^2) & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

As discussed in Example A.8 of Appendix A, we can use $g(x) = 3/4$ as the majorizing function, which results in $c = 3/2$, and $w(x)$

$$w(x) = \begin{cases} \frac{1}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Notice that, $w(x)$ results in random variables, W, where $W \sim U(-1, 1)$. Thus, the proposal distribution is simply a uniform distribution over the range from -1 to 1. The following KSL code implements these concepts. The proposal distribution, $w(x)$ is provided via an instance of the `Uniform` distribution class. Probability distributions are discussed in Section 2.3.

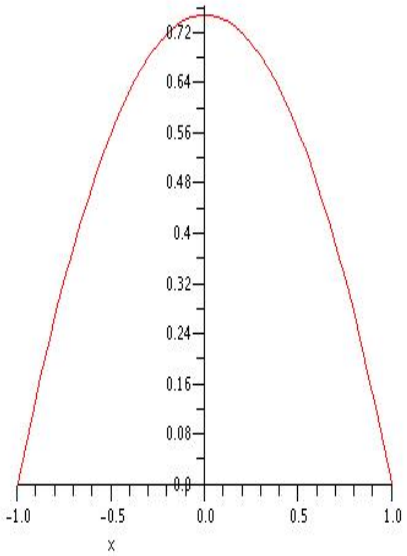


Figure 2.4.: Plot of f(x)

The `AcceptanceRejection` class then is created and used to generate 1000 observations.

```
fun main() {
    // proposal distribution
    val wx = Uniform(-1.0, 1.0)
    // majorizing constant, if g(x) is majorizing function,
     ↪  then g(x) = w(x)*c
    val c = 3.0 / 2.0
    val rv = AcceptanceRejectionRV(wx, c, fOfx, streamNum =
     ↪  1)
    val h = Histogram.create(-1.0, 1.0, 100)
    for (i in 1..1000) {
        h.collect(rv.value)
    }
    println(h)
    val hp = h.histogramPlot()
    hp.showInBrowser()
    val dp = DensityPlot(h) { x -> fOfx.pdf(x) }
    dp.showInBrowser()
}

object fOfx : PDFIfc {
    override fun pdf(x: Double): Double {
        if ((x < -1.0) || (x > 1))
            return 0.0
        return (0.75 * (1.0 - x * x))
    }

    override fun domain(): Interval {
        return Interval(-1.0, 1.0)
    }

}
```

Notice that the implementation requires an implementation of the PDF from which random variates are to be generated, $f(x)$. The `fOfx` *object* is used to implement the function and its domain. The code also illustrates how to create a histogram of the observations. Histograms are discussed further in Section 3.1.2 of Chapter 3.

In what follows, we briefly describe some additional functionality of the KSL for generating random variates and for applying randomness to arrays and lists.

The preferred method for generating random values from random variables is to create an instance of the appropriate random variable class; however, the KSL also provides a set of functions for generating random values within the `KSLRandom` object. For all the previously listed random variables, there is a corresponding function that will generate a random value. For example, the function `rNormal()` of the `KSLRandom` object will generate a normally distributed value. Each function is named with an "r" in front of the distribution name. By using an import of `KSLRandom` functions the user can more conveniently call these methods. The following code example illustrates how to do this.

Example 2.17 (Using KSLRandom Functions)**.** The functions require the specification of the parameters of the distribution for each invocation.

```
fun main() {
    val v = rUniform(10.0, 15.0) // generate a U(10, 15)
     ↪ value
    val x = rNormal(5.0, 2.0) // generate a Normal(mu=5.0,
     ↪ var= 2.0) value
    val n = rPoisson(4.0).toDouble() //generate from a
     ↪ Poisson(mu=4.0) value
    print(String.format("v = %f, x = %f, n = %f %n", v, x,
 ↪ n))
}
```

In addition to random values through these functions, the `KSLRandom` object provides a set of methods for randomly selecting from arrays and lists and for creating permutations of arrays and lists. In addition, there is a set of methods for sampling from arrays and lists without replacement. The following code provides examples of using these methods.

Example 2.18 (Random Lists)**.** The `randomlySelect()` function can be used to randomly select, with equal probability from a Kotlin list. An extension function has been defined to allow this to be done directly from the list.

```
fun main() {
    // create a list
    val strings = listOf("A", "B", "C", "D")
    // randomly pick from the list, with equal probability
    for (i in 1..5) {
        println(KSLRandom.randomlySelect(strings))
    }
    println()
    // use the extension function
    for (i in 1..5) {
        println(strings.randomlySelect())
    }
}
```

It is also possible to specify a distribution in the form of a CDF array over the items in the list to permit sampling that is not equally likely. There are also extension functions declared on arrays for directly performing this form of random selection.

This next example illustrates how to define a population of values via the `DPopulation` class and to use it to perform sampling operations such as random samples and permutations. Similar functionality is also demonstrated by directly using the functions of the `KSLRandom` object

Example 2.19 (Random Permuation)**.** This example defines a population over the integer from 1 to 10 and permutes the population. It also illustrates how to sample from the population and permute a mutable list.

```
fun main() {
    // create an array to hold a population of values
    val y = DoubleArray(10)
    for (i in 0..9) {
        y[i] = (i + 1).toDouble()
    }

    // create the population
    val p = DPopulation(y, streamNum = 1)
    println(p.contentToString())

    println("Print the permuted population")
    // permute the population
    p.permute()
    println(p.contentToString())

    // directly permute the array using KSLRandom
    println("Permuting y")
    KSLRandom.permute(y)
    println(y.contentToString())

    // sample from the population
    val x = p.sample(5)
    println("Sampling 5 from the population")
    println(x.contentToString())

    // create a string list and permute it
    val strList: MutableList<String> = ArrayList()
    strList.add("a")
    strList.add("b")
```

```
    strList.add("c")
    strList.add("d")
    println("The mutable list")
    println(strList)
    KSLRandom.permute(strList)
    println("The permuted list")
    println(strList)
    println("Permute using extension function")
    strList.permute()
    println(strList)
}
```

Output from the permutation examples.

```
[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0, 10.0]
Print the permuted population
[2.0, 4.0, 5.0, 9.0, 6.0, 8.0, 3.0, 1.0, 7.0, 10.0]
Permuting y
[6.0, 5.0, 2.0, 3.0, 8.0, 10.0, 4.0, 7.0, 1.0, 9.0]
Sampling 5 from the population
[1.0, 4.0, 2.0, 5.0, 3.0]
The mutable list
[a, b, c, d]
The permuted list
[a, c, b, d]
Permute using extension function
[d, a, c, b]
```

2.2.4. Functions of Random Variables

The KSL also contains an algebra for working with random variables. A well-known property of random variables is that a function of a random

variable is also a random variable. That is, let $f(\cdot)$ be an arbitrary function and let X be a random variable. Then, the quantity $Y = f(X)$ is also a random variable. Various properties of Y such as expectation, $E[Y]$ and $Var[Y]$ may be of interest. A classic example of this is the relationship between the normal random variable and the lognormal random variable. If $X \sim N(\mu, \sigma^2)$ then the random variable $Y = e^X$ will be lognormally distributed $LN(\mu_l, \sigma_l^2)$, where

$$E[Y] = \mu_l = e^{\mu + \sigma^2/2}$$

$$Var[Y] = \sigma_l^2 = e^{2\mu + \sigma^2}\left(e^{\sigma^2} - 1\right)$$

Thus, one can define new random variables simply as functions of other random variables.

The interface `RVariableIfc` and base class `RVariable` provides the ability to construct new random variables that are functions of other random variables by overriding the $(+, -, \times, \div)$ operators and providing extension functions for various math functions. For example, we can defined two random variables and then a third that is the sum of the first two random variables. The random variable that is defined as the sum will generate random variates that represent the sum. Functions, such as `sin(), cos()` as well as many other standard math functions can be applied to random variables to create new random variables. That is, the KSL provides the ability to create arbitrarily complex random variables that are defined as *functions* of other random variables. This capability will be illustrated in this section with a couple of examples.

Example 2.20 (Erlang Via Convolution)**.** Suppose we have k independent random variables, X_i each exponentially distributed with mean θ. Then, the random variable:

$$Y = \sum_{i=i}^{k} X_i$$

will be an Erlang(k, θ) where k is the shape parameter and θ is the scale parameter. Set up a KSL model to generate 1000 Erlang random variables with $k = 5$ and $\theta = 10$.

A possible solution to this problem is to use the KSL to define a new random variable that is the sum of 5 exponential random variables.

```
fun main(){
    var erlang: RVariableIfc = ExponentialRV(10.0, streamNum
     ↪  = 1)
    for(i in 1..4) {
        erlang = erlang + ExponentialRV(10.0)
    }
    println("stream number = ${erlang.streamNumber}")
    val sample = erlang.sample(1000)
    val stats = Statistic(sample)
    println()
    print(stats)
    sample.writeToFile("erlang.txt")
}
```

The first line of this code creates and stores an instance of an exponential random variable with mean 10. The for loop is **not** generating any random variates. It is defining a new random variable that is the sum of 4 additional exponential random variables. The defined random variable is used to

generate a sample of size 1000 and using the `Statistic` class (discussed in the next chapter) a basic statistical summary is computed. Also, using the `writeToFile` KSL extension function for double arrays, the sample is written to a file. The results as a histogram are also presented. The statistical results are as follows.

```
stream number = 1

ID 31
Name Statistic_1
Number 1000.0
Average 50.22504832196012
Standard Deviation 22.130436282067713
Standard Error 0.699825842645625
Half-width 1.3732971036698247
Confidence Level 0.95
Confidence Interval [48.85175121829029, 51.598345425629944]
Minimum 8.517383088139447
Maximum 147.81714627300153
Sum 50225.048321960116
Variance 489.75621003465903
Deviation Sum of Squares 489266.45382462436
Last value collected 32.293283161673294
Kurtosis 0.9888978451474602
Skewness 0.8816305761311749
Lag 1 Covariance -19.73716912800237
Lag 1 Correlation -0.040340327798310656
Von Neumann Lag 1 Test Statistic -1.2518042413131365
Number of missing observations 0.0
Lead-Digit Rule(1) -1
```

Notice that the histogram looks like an Erlang distribution and the estimated results are what we would expect for an Erlang distribution with $k = 5$ and $\theta = 10$.

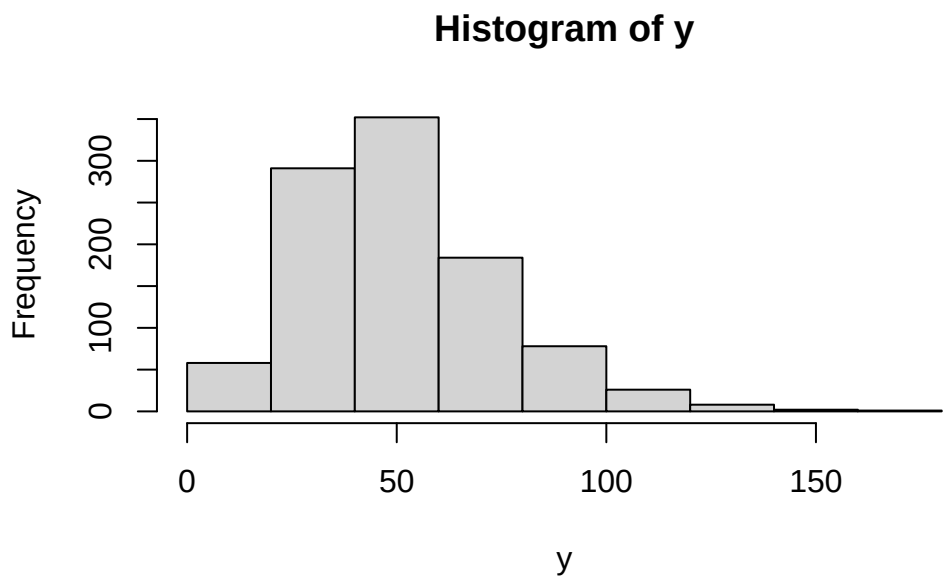


Figure 2.5.: Histogram for Erlang Generated Data

To illustrate a couple of other examples consider the following code. In this code, the previously noted relationship between normal random variables and lognormal random variables is demonstrated in the first 6 lines of the code.

Example 2.21 (Functions of Random Variables)**.** This example uses the fact that if $X \sim N(\mu, \sigma^2)$, then $Y = e^X \sim \text{LN}(\mu_l, \sigma_l^2)$. In addition, the code illustrates how to generate a beta random variable via its relationship with gamma random variables. The assignment of streams is a bit more complex for functions of random variables. Notice that in the following code, the random variable (normal in the first case) is assigned stream number 1. This stream is then used by all of the transformations. In the example of generating a beta random variable, the first random variable, `y1` has stream 2. The random variable `y2` has the next stream number (3); however, because the denominator has `y1 + y2` and `y1` appears first in the functional transformation, the resulting random variable `y1 + y2` will have stream 2. Then, when `y1` is divided by `y1 + y2`, the resulting random variable `betaRV` will use stream 2. Thus, the order of operation is important in the determination of the resulting assigned stream.

```
fun main(){
    // define a lognormal random variable, y
    val x = NormalRV(2.0, 5.0, streamNum = 1)
    val y = exp(x)
    // generate from y
    println("stream number = ${y.streamNumber}")
    val ySample = y.sample(1000)
    println(ySample.statistics())
    // define a beta random variable in terms of gamma
    val alpha1 = 2.0
    val alpha2 = 5.0
    val y1 = GammaRV(alpha1, 1.0, streamNum = 2)
    val y2 = GammaRV(alpha2, 1.0)
    val betaRV = y1/(y1+y2)
```

```
    val betaSample = betaRV.sample(500)
    println("stream number = ${betaRV.streamNumber}")
    println(betaSample.statistics())
}
```

One method for generating Beta random variables exploits its relationship with the Gamma distribution. If $Y_1 \sim Gamma(\alpha_1, 1)$ and $Y_2 \sim Gamma(\alpha_2, 1)$, then $X = Y_1/(Y_1 + Y_2)$ has a Beta(α_1, α_2) distribution. In the previous KSL code, we defined two gamma random variables and define the beta random variable using the algebraic expression. This code *defines* and *constructs* a new random variable that is function of the previously defined random variables. Through this pattern you can define complex random variables and use those random variables anywhere that a random variable is needed.

2.3. Probability Distribution Models

The `ksl.utilities.random.rvariable` package is the key package for generating random variables; however, it does not facilitate performing calculations involving the underlying probability distributions. To perform calculations involving probability distributions, you should use the `ksl.utilities.distributions` package. This package has almost all the same distributions represented within the `ksl.utilities.random.rvariable` package.

Figure 2.6 illustrates the interfaces used to define probability distributions. First, the interface, `CDFIfc` serves as the basis for discrete distributions via the `DiscreteDistributionIfc` interface, for continuous distributions via the `ContinuousDistributionIfc` interface and the general `DistributionIfc` interface. The discrete distributions such as the geometric, binomial, etc. implement the `DiscreteDistributionIfc` and `PMFIfc` interfaces. Similarly, continuous distributions like the normal,

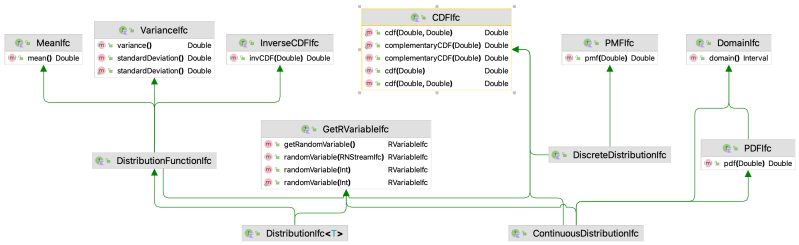


Figure 2.6.: Distribution Interfaces

uniform, etc. implement the `ContinuousDistributionIfc` and `PDFIfc` interfaces. All concrete implementations of distributions extend from the abstract `Distribution` class, which implements the `DistributionIfc` interface. Thus, all distributions have the following capabilities:

- `cdf(b: Double)`: computes the cumulative probability, $F(b) = P(X \leq b)$
- `cdf(a: Double, b: Double)`: computes the cumulative probability, $P(a \leq X \leq b)$
- `complementaryCDF(b: Double)`: computes the cumulative probability, $1 - F(b) = P(X > b)$
- `mean()`: returns the expected value (mean) of the distribution
- `variance()`: returns the variance of the distribution
- `standardDeviation()`: returns the standard deviation of the distribution
- `invCDF(p: Double)`: returns the inverse of the cumulative distribution function $F^{-1}(p)$. This is performed by numerical search if necessary

Discrete distributions have a method called `pmf(k: Double)` that returns the probability associated with the value k. Continuous distributions have a probability density function, $f(x)$, implemented in the method, `pdf(x`

`: Double)`. Finally, all distributions know how to create random variables through the `GetRVariableIfc` interface that provides the following methods.

- `RVariableIfc randomVariable(stream: RNStreamIfc)`: returns a new instance of a random variable based on the current values of the distribution's parameters that uses the supplied stream
- `RVariableIfc randomVariable(streamNum: Int)`: returns a new instance of a random variable based on the current values of the distribution's parameters that uses the supplied stream number
- `RVariableIfc randomVariable()`: returns a new instance of a random variable based on the current values of the distribution's parameters that uses a newly created stream

As an example, the following code illustrates some calculations for the binomial distribution.

Example 2.22 (Computing with a Binomial Distribution)**.** This example code illustrates how to create a binomial distribution and to use some of its functions to compute the mean, variance, and perform some basic calculations involving probabilities. Notice that the parameters of a distribution can be changes and that distributions can create random variables for generating variates.

```
fun main() {
    // make and use a Binomial(p, n) distribution
    val n = 10
    val p = 0.8
    println("n = $n")
    println("p = $p")
    val bnDF = Binomial(p, n)
    println("mean = " + bnDF.mean())
    println("variance = " + bnDF.variance())
    // compute some values
```

```
    print(String.format("%3s %15s %15s %n", "k", "p(k)",
↪   "cdf(k)"))
    for (i in 0..10) {
        print(String.format("%3d %15.10f %15.10f %n", i,
↪   bnDF.pmf(i), bnDF.cdf(i)))
    }
    println()
    // change the probability and number of trials
    bnDF.probOfSuccess = 0.5
    bnDF.numTrials = 20
    println("mean = " + bnDF.mean())
    println("variance = " + bnDF.variance())
    // make random variables based on the distributions
    val brv = bnDF.randomVariable(1)
    print(String.format("%3s %15s %n", "n", "Values"))
    // generate some values
    for (i in 1..5) {
        // value property returns generated values
        val x = brv.value.toInt()
        print(String.format("%3d %15d %n", i, x))
    }
}
```

The output shows the mean, variance, and basic probability calculations.

```
n = 10
p = 0.8
mean = 8.0
variance = 1.5999999999999996
  k            p(k)          cdf(k)
  0    0.0000001024    0.0000001024
  1    0.0000040960    0.0000041984
  2    0.0000737280    0.0000779264
```

```
  3    0.0007864320    0.0008643584
  4    0.0055050240    0.0063693824
  5    0.0264241152    0.0327934976
  6    0.0880803840    0.1208738816
  7    0.2013265920    0.3222004736
  8    0.3019898880    0.6241903616
  9    0.2684354560    0.8926258176
 10    0.1073741824    1.0000000000
```

The `ksl.utilities.random.rvariable` package creates instances of random variables that are immutable. That is, once you create a random variable, its parameters cannot be changed. However, distributions permit their parameters to be changed and they also facilitate the creation of random variables. The example code uses the properties `probOfSuccess` and `numTrials` to change the parameters of the previously created binomial distribution and then creates a random variable based on the mutated distribution.

```
mean = 10.0
variance = 5.0
  n          Values
  1               7
  2               9
  3               9
  4              12
  5               8
```

The results are as we would expect. Similar calculations can be made for continuous distributions. In most cases, the concrete implementations of the various distributions have specialized methods beyond those generic methods described here. Please refer to the documentation for further details.

There are a number of useful companion object methods defined for the binomial, normal, gamma, and Student-T distributions. Specifically, for the binomial distribution, has the following methods

- `binomialPMF(j: Int, n: Int, p: Double)`: directly computes the probability for the value j
- `binomialCDF(j: Int, n: Int, p: Double)`: directly computes the cumulative distribution function for the value j
- `binomialCCDF(j: Int, n: Int, p: Double)`- directly computes the complementary cumulative distribution function for the value of j
- `binomialInvCDF(x: Double, n: Int, p: Double)`: directly computes the inverse cumulative distribution function

These methods are designed to perform their calculations in a numerically stable manner to ensure numerical accuracy. The normal distribution has the following companion object methods for computations involving the standard normal distribution.

- `stdNormalCDF(z: Double)`: the cumulative probability for a Z $N(0,1)$ random variable, i.e. $F(z) = P(Z \leq z)$
- `stdNormalComplementaryCDF(z: Double)`: returns $1 - P(Z \leq z)$
- `stdNormalInvCDF(p: Double)`: returns $z = F^{-1}(p)$ the inverse of the cumulative distribution function

The Student-T distribution also has two convenience methods to facilitate computations.

- `cdf(dof: Double, x: Double)`: computes the cumulative distribution function for x given the degrees of freedom
- `invCDF(dof: Double, p: Double)`: computes the inverse cumulative distribution function or t-value for the supplied probability given the degrees of freedom.

Within the `Gamma` class's companion object there are some convenience methods for computing the gamma function, the natural logarithm of the gamma function, the incomplete gamma function, and the digamma function (derivative of the natural logarithm of the gamma function).

2.4. Distribution Fitting Using the KSL

This section describes how to apply the methods discussed in Appendix B to fit probability distributions using the KSL. The package `ksl.utilities.distributions.fitting` contains classes that facilitate the estimation of the parameters of discrete and continuous probability distributions. In addition, the quality of the estimated parameters can be examined based on a number of metrics that measure the goodness of fit of the probability model to the data. Finally, there is functionality that will automatically fit a set of possible distributions to a data set and recommend the best distribution based on a combination of metrics. This section will first describe the organization of the functionality related to parameter estimation. Then, the scoring models for measuring the quality of fit will be presented. Finally, a number of examples will illustrate the use of the classes on the data from the examples discussed in Appendix B.

2.4.1. Estimating Distribution Parameters

As discussed in Appendix B a key step in the distribution fitting process is estimating the parameters of the hypothesized probability model. For example, if we want to fit a normal distribution to the data, we must estimate the mean (μ) and the variance (σ^2) of the associated distribution. Each distribution may be specified by different parameters. For example, the gamma distribution is specified by the shape (α) and scale (β) parameters. Section D.1 and Section D.2 provide a listing of many common distributions and their parameters.

The estimation of the parameters of a distribution requires the execution of an algorithm. As mentioned in Appendix B, the maximum likelihood estimation method is commonly used because it often produces estimates that have desirable statistical properties. However, the maximum likelihood method may require the solving of a non-linear optimization problem. The method of moments is also a common algorithm that is used to estimate the parameters of various distributions. In the method of moments, the theoretical moment equations are matched with the statistically computed moments resulting in a set of equations that must be solved to form the parameter estimators. In some cases, the method of moments results in simple equations, but for some distributions it may result in the need to solve a system of non-linear equations. Many other methods of parameter estimation exist, such as the percentile method, which may be general or targeted to specific distributions. The KSL facilitates the development of routines to estimate parameters by providing a well-defined parameter estimation interface. By implementing the `ParameterEstimatorIfc` interface, KSL users can incorporate additional distributions and estimation methods into the estimation framework.

Figure 2.7 illustrates some of the key classes and interfaces involved in the parameter estimation framework. The functionality starts with the `ParameterEstimatorIfc` interface, which requires the implementation of the estimation routine and a property (`checkRange`) to indicate if the data should be checked for shifting onto the domain of the distribution. The `ParameterEstimatorIfc` interface defines a general function that uses a 1-dimensional data array, and estimates the parameters of an hypothesized uni-variate distribution via some estimation algorithm. It is important to note that a distribution may have more than one method for estimating its parameters. In the figure, the maximum likelihood estimator for the gamma distribution is illustrated; however, for example, there is also a method of moments estimator for the gamma distribution implemented within the KSL distribution fitting framework.

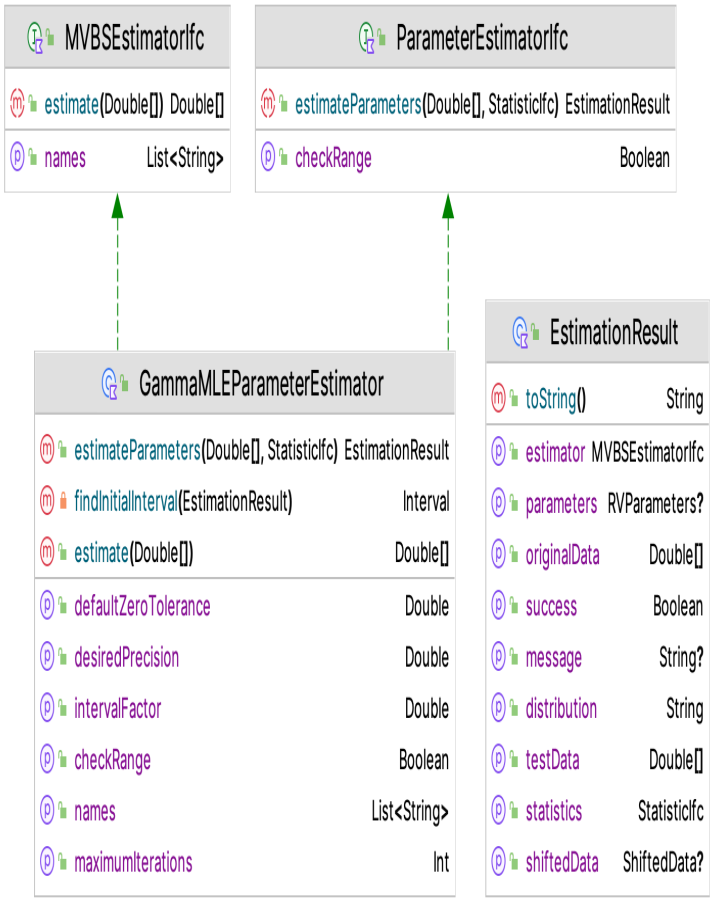


Figure 2.7.: Important Classes for Parameter Estimation

```
interface ParameterEstimatorIfc {

    /**
     *  Indicates if the estimator requires that the range of
 ↪  the data be checked for a shift
     *  before the estimation process.
     */
    val checkRange: Boolean

    /**
     *  Estimates the parameters associated with some
 ↪  distribution. The returned [EstimationResult]
     *  needs to be consistent with the intent of the desired
 ↪  distribution.
     *  Note the meaning of the fields associated with
 ↪  [EstimationResult]
     */
    fun estimateParameters(data: DoubleArray, statistics:
     ↪  StatisticIfc = Statistic(data)): EstimationResult
}
```

The basic contract for the estimation process is that the returned `EstimationResult` is consistent with the required parameter estimation. The data class `EstimationResult` stores information about the estimation process to be returned as a result of its application on the supplied data. The key property is the `success` property, which indicates whether the parameter estimation process was successful. Given that many estimation processes may require advanced optimization methods, the estimation process might not converge or some other problem might have occurred. If the `success` property is true, then the results should be meaningful. The `EstimationResult` also returns the estimator that was used (`MVBSestimatorIfc`) in a form that can be used within KSL bootstrapping constructs, the estimated parameters as an instance of

`RVParameters`, a message concerning the estimation process, a text (string) representation of the distribution and its parameters, the original data (`originalData`), the shifted data if shifted, the statistics associated with the original data, and data that is suitable for testing for goodness of fit (`testData`). The `MVBSestimatorIfc` interface defines a method that takes in an array of data and produces a (double) array that holds the estimated parameters. In addition, the interface defines a list of the names to use for the parameters. This form for the estimation results facilitates performing a bootstrapping process that can provide confidence intervals for the estimated parameters.

Example 2.23 (Parameter Estimation)**.** The following code illustrates the estimation of the parameters for a normal distribution.

```
fun main(){
    // define a normal random variable,
    val x = NormalRV(2.0, 5.0, streamNum = 1)
    // generate some data
    val data = x.sample(100)
    // create the estimator
    val ne = NormalMLEParameterEstimator
    // estimate the parameters
    val result = ne.estimateParameters(data)
    println(result)
    val bss = BootstrapSampler(data, ne)
    val list = bss.bootStrapEstimates(400)
    for (element in list) {
        println(element.toString())
    }
}
```

The results of this estimation process are show here. Notice that the estimation process was successful and the summary statistics were reported.

The data was not shifted for the estimation process and the estimated parameters were $\mu = 2.4138851463671918$ and $\sigma^2 = 5.571943618235626$. The bootstrapping process, see Section 9.1 of Chapter 9 was used to provide confidence intervals on the estimated parameters. This functionality is available for any of the distributions and the estimation routines that have been defined within the KSL.

```
Estimation Results:
The estimation was a SUCCESS!
Estimation message:
The normal parameters were estimated successfully using a MLE technique

Statistics for the data:
ID 4
Name Statistic_1
Number 100.0
Average 2.0892468525989156
Standard Deviation 2.123930458872048
Standard Error 0.2123930458872048
Half-width 0.4214338787318177
Confidence Level 0.95
Confidence Interval [1.667812973867098, 2.5106807313307336]
Minimum -4.430752220590554
Maximum 6.493367339631967
Sum 208.92468525989156
Variance 4.5110805941244285
Deviation Sum of Squares 446.5969788183184
Last value collected 3.5738686354135556
Kurtosis -0.2314557519668217
Skewness -0.2890094690738773
Lag 1 Covariance -0.40973073091033607
Lag 1 Correlation -0.09174507449523522
Von Neumann Lag 1 Test Statistic -0.8229876116312043
```

```
Number of missing observations 0.0
Lead-Digit Rule(1) -1


Shift estimation results:
null

Estimated parameters:
RV Type = Normal
Double Parameters {mean=2.0892468525989156, variance=4.5110805941244285}

-----------------------------------------------------
Bootstrap statistical results:
-----------------------------------------------------
statistic name = mean
number of bootstrap samples = 400
size of original sample = 100
original estimate = 2.0892468525989156
bias estimate = -0.01587415275757076
across bootstrap average = 2.073372699841345
bootstrap std. err. estimate = 0.21341609857688634
default c.i. level = 0.95
norm c.i. = [1.670958985329941, 2.5075347198678903]
basic c.i. = [1.6728521695280434, 2.506080452483194]
percentile c.i. = [1.6724132527146374, 2.505641535669788]

-----------------------------------------------------
Bootstrap statistical results:
-----------------------------------------------------
statistic name = variance
number of bootstrap samples = 400
size of original sample = 100
original estimate = 4.5110805941244285
bias estimate = -0.013680373758844233
```

```
across bootstrap average = 4.497400220365584
bootstrap std. err. estimate = 0.6474490755210855
default c.i. level = 0.95
norm c.i. = [3.2421037232562866, 5.78005746499257]
basic c.i. = [3.163271658068438, 5.717662725018807]
percentile c.i. = [3.3044984632300505, 5.858889530180419]
```

The parameters of the following distributions can be estimated from data using the KSL.

- Beta: via `BetaMOMParameterEstimator`
- Binomial: via `BinomialMaxParameterEstimator` or `BinomialMOMParameterEstimator`
- Exponential: via `ExponentialMLEEstimator`
- Gamma: via `GammaMLEParameterEstimator` or `GammaMOMParameterEstimator`
- Generalized Beta: via `GeneralizedBetaMOMParameterEstimator`
- Laplace: via `LaplaceMLEParameterEstimator`
- Logistic: via `LogisticMOMParameterEstimator`
- Lognormal: via `LognormalMLEParameterEstimator`
- Negative Binomial: via `NegBinomialMOMParameterEstimator`
- Normal: via `NormalMLEParameterEstimator`
- Pearson Type 5: via `PearsonType5MLEParameterEstimator`
- Poisson: via `PoissonMLEParameterEstimator`
- Triangular: via `TriangularParameterEstimator`
- Uniform: via `UniformParameterEstimator`
- Weibull: via `WeibullMLEParameterEstimator` or `WeibullPercentileParameterEstimator`

Again, by implementing the concepts illustrated in Figure 2.7, a KSL user can implement additional distribution parameter estimation methods. If you need to fit a distribution that is not modeled within the KSL, then you need to implement the distribution's parameter estimation procedure. In addition, you will also need to implement a class representing the distribution. Depending on the type of distribution, this may involve implementing the requirements of the `ContinuousDistributionIfc` interface

or the `DiscreteDistributionIfc` interface for the underlying probability model.

2.4.2. Continuous Distribution Recommendation Framework

As described in Appendix B, the distribution modeling process may require that the parameters of many distributions be estimated and the quality of those probability models examined to recommend an overall distribution. This process normally involves statistical tests or metrics (e.g. square error criterion) to assess how well the probability model represents the data.

Rather than relying on statistical tests, the KSL's distribution recommendation framework defines a set of criteria for assessing the quality of the probability model's representation. These criteria are called *scoring models*. Figure 2.8 illustrates the scoring models that the KSL uses within its automated fitting process. To implement your own scoring model, you need to sub-class from the abstract base class `PDFScoringModel.`

The primary method that needs to be implemented is the abstract `score()` function.

```
abstract fun score(data: DoubleArray, cdf:
 ↪  ContinuousDistributionIfc) : Score
```

The contract for this function is that it will return a numerical value that measures the quality of the distribution fit given the observed data and a hypothesized continuous distribution function. In addition, the function, `badScore(),` should return the worse possible score for the metric. The `badScore()` function should be used if there is some error or issue that prevents the scoring procedure from determining a score for the fit. A `Score` is a data class that indicates whether the score is `valid` or not and provides the metric that determined the score. You can think of the metric as the units of measure. The metric defines the domain (or set of legal

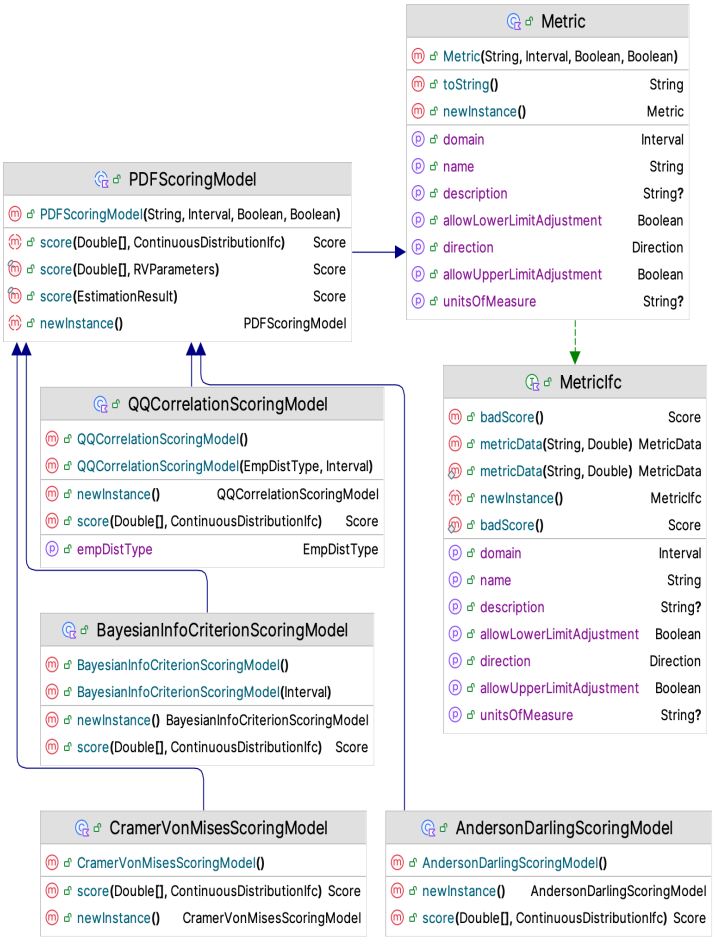


Figure 2.8.: Important Classes for PDF Scoring

values) for the metric and its direction. By direction, we mean whether bigger scores are better than smaller scores or vice versa. This information is used when a set of scores are combined into an overall score.

The KSL has many pre-defined scoring models. The user can specify the criteria to use for the scoring evaluation process or accept the defaults. Two of the scoring models are based on a histogram summary of the data. This involves dividing the range of observations via break points b_j and tabulating frequencies or proportions of the data falling with the defined intervals or bins. The KSL does not use arbitrary break points from a histogram binning process. Instead the KSL attempts to define the break points for the intervals such that each interval has the *same* probability of occurrence. This ensures that the expected number of observations within each interval is the same. The theoretical basis for this approach can be found in (Williams 1950).

The scoring criteria based on a binning process are the sum of squared errors (SSE), Chi-Squared (CSQ), and the Mallows L2 (ML2) criterion. For these models, let c_j be the observed count of the x values contained in the j^{th} interval $\left(b_{j-1}, b_j\right]$, let $h_j = c_j/n$ be the relative frequency for the j^{th} interval, and let p_j be defined as:

$$p_j = P\{b_{j-1} \leq X \leq b_j\} = \int_{b_{j-1}}^{b_j} f(x)\mathrm{d}x$$

Thus, we can define SSE, CSQ, and ML2 as follows:

- Sum of Squared Error (SSE) criterion - The squared error is defined as the sum over the intervals of the squared difference between the relative frequency and the theoretical probability associated with each interval:

$$\text{SSE} = \sum_{j=1}^{k} (h_j - p_j)^2$$

- Chi-Squared criterion (CSQ) - The chi-squared criterion is the χ^2 test statistic value that compares the observed counts c_j to the expected counts np_j over the intervals.

$$\chi_0^2 = \sum_{j=1}^{k} \frac{\left(c_j - np_j\right)^2}{np_j}$$

- Mallows L2 Criterion: (Levina and Bickel 2001) provide the definition of the Mallows L2 distance, also known as the earthmovers distance. In terms of p_j and h_j, this is:

$$M_{L_2} = \left(\frac{1}{k} \sum_{j=1}^{k} (h_j - p_j)^2 \right)^{\frac{1}{2}}$$

Note that under the specification of the breakpoints resulting in intervals with equal probabilities, we have $p_j = p$ and $p = k/n$. This allows the SSE criterion to be simplified as follows:

$$\text{SSE} = \sum_{j=1}^{k} (h_j - p_j)^2 = \sum_{j=1}^{k} (h_j - p)^2$$

In addition, the Mallows criterion can be written in terms of the SSE as follows:

$$M_{L_2} = \left(\frac{1}{k} \sum_{j=1}^{k} (h_j - p_j)^2 \right)^{\frac{1}{2}} = \left(\frac{\text{SSE}}{k} \right)^{\frac{1}{2}}$$

Thus, we have $M_{L_2}^2 = \frac{\text{SSE}}{k}$. Finally, notice that the chi-squared criterion (CSQ) under the assumption that $p_j = p$ can be simplified as:

$$\chi_0^2 = \sum_{j=1}^{k} \frac{\left(c_j - np_j\right)^2}{np_j} = n \sum_{j=1}^{k} \frac{\left(h_j - p_j\right)^2}{p_j} = \frac{n}{p} \sum_{j=1}^{k} \left(h_j - p\right)^2 = \frac{n}{p}\text{SSE}$$

Therefore, under the assumption that $p_j = p$, that is equally probable intervals, we have that:

$$\chi_0^2 = \frac{n^2}{k}\text{SSE} = n^2 M_{L_2}^2$$

The relationships between CSQ, SSE, and ML2 are important to note because during the scoring process, the metrics are converted to a common scale. If a linear transformation is chosen for the transformation (which is the default), then the resulting values for CSQ, SSE, and ML2 will be the same in terms of their *scaled* values. Thus, under a linear transformation, only one of CSQ, SSE, and ML2 should be included in the evaluation process.

The metrics that do not depend on a histogram binning process include the Kolmogorov-Smirnov (KS) criterion, Cramer Von Mises (CVM) criterion, Anderson-Darling (AD) criterion, P-P plot sum of squared errors (PPSSE), P-P plot correlation (PPC), Q-Q plot sum of squared errors (QQSSE), Q-Q plot correlation (QQC), Bayesian Information Criterion (BIC), and the Akaike Information Criterion (AIC). These scoring metrics are defined as follows:

- Kolmogorov-Smirnov criterion - The Kolmogorov-Smirnov (KS) criterion is based on the K-S test statistic, where, $D_n = \max\{D_n^+, D_n^-\}$, represents the largest vertical distance between the hypothesized distribution and the empirical distribution over the range of the distribution.

$$D_n^+ = \max_{1 \le i \le n} \left\{ \tilde{F}_n \left(x_{(i)} \right) - \hat{F}(x_{(i)}) \right\}$$
$$= \max_{1 \le i \le n} \left\{ \frac{i}{n} - \hat{F}(x_{(i)}) \right\}$$

$$D_n^- = \max_{1 \le i \le n} \left\{ \hat{F}(x_{(i)}) - \tilde{F}_n \left(x_{(i-1)} \right) \right\}$$
$$= \max_{1 \le i \le n} \left\{ \hat{F}(x_{(i)}) - \frac{i-1}{n} \right\}$$

- Cramer Von Mises criterion (CVM) - The Cramer Von Mises criterion is a distance measure used to compare the goodness of fit of a cumulative distribution function, $F(x)$ to the empirical distribution, $F_n(x)$. The distance is defined as:

$$\omega^2 = \int_{-\infty}^{+\infty} \left[F_n(x) - F(x) \right]^2 dF(x)$$

The CVM metric can be computed from data sorted in ascending order $(x_1, x_2, \cdots, x_n)$, i.e. the order statistics, as:

$$T = n\omega^2 = \frac{1}{12n} + \sum_{i=1}^{n} \left[\frac{2i-1}{2n} - F(x_i) \right]^2$$

- Anderson Darling criterion (AD) - The Anderson-Darling criterion is similar in spirit to the Cramer Von Mises test statistic except that it is designed to detect discrepancies in the tails of the distribution.

$$A^2 = n \int_{-\infty}^{+\infty} \frac{\left[F_n(x) - F(x) \right]^2}{F(x)(1 - F(x))} dF(x)$$

The AD metric can be computed from data sorted in ascending order $(x_1, x_2, \cdots, x_n)$, i.e. the order statistics, as:

$$A^2 = -n - \sum_{i=1}^{n} \frac{2i-1}{n}\Big[\ln(F(x_i)) + \ln(1 - F(x_{n+1-i}))\Big]$$

- P-P Plot Sum of Squared Errors (PPSSE) - Based on the concepts found in (Gan and Koehler 1990), the sum of squared error related to the P-P plot of the theoretical distribution versus the empirical distribution was developed as a scoring criterion. Let $(x_{(1)}, x_{(2)}, ... x_{(n)})$ be the order statistics. Let the theoretical distribution be represented with $\widehat{F}(x_{(i)})$ for $i = 1, 2, ... n$ where $\widehat{F}$ is the CDF of the hypothesized distribution. Define the empirical distribution as

$$\tilde{F}_n(x_{(i)}) = \frac{i - 0.5}{n}$$

Then, the P-P Plot sum of squared error (PPSSE) criterion is defined as:

$$\text{PP Squared Error} = \sum_{i=1}^{n} (\tilde{F}_n(x_{(i)}) - \hat{F}(x_{(i)}))^2$$

Notice that the Cramer-Von-Mises criterion and the PPSSE criterion are related as follows:

$$T = \frac{1}{12n} + \text{PPSSE}$$

Therefore, under a linear transformation, it is not recommended to include both PPSSE and CVM in the evaluation process.

- P-P Correlation (PPC) - Based on the concepts found in (Gan and Koehler 1990), the P-P correlation scoring model computes the Pearson correlation associated with a P-P plot. That is, the scoring model computes the correlation between $(\tilde{F}_n(x_{(i)}), \hat{F}(x_{(i)}))$ for $i = 1, 2, ... n$.

- Q-Q Plot Sum of Squared Error (QQSSE) - Again, based on the concepts found in (Gan and Koehler 1990), the sum of squared error related to the Q-Q plot of the theoretical quantiles versus the empirical quantiles was developed as a scoring criterion. For a Q-Q plot define $q_i = (i - 0.5)/n$, for $i = 1, 2, ... n$ and $x_{q_i} = \hat{F}^{-1}(q_i)$ where $\hat{F}^{-1}(\cdot)$ is the inverse CDF of the hypothesized distribution. Finally, let $(x_{(1}, x_{(2)}, ... x_{(n)})$ be the order statistics. Then, the QQSSE is the sum of the squared error as follows:

$$\text{QQSSE} = \sum_{i=1}^{n} (x_{(i)} - x_{q_i})^2$$

- Q-Q Plot Correlation (QQC) - Similar to PPC, the Q-Q plot correlation criterion is the Pearson correlation of $(x_{(i)}, x_{q_i})$ for $i = 1, 2, ... n$, with values closer to 1.0 indicating a good fit.

- Bayesian Information Criterion (BIC) - Developed by (Schwarz 1978), the BIC is used for model selection where models with lower BIC are preferred. The BIC is based on the log-likelihood function. Let $f(x; \vec{\theta})$ be the theoretical probability density function with parameters $\vec{\theta}$. The likelihood function of a sample of $\vec{x} = (x_1, x_2, \cdots, x_n)$ is:

$$L(\vec{\theta}; \vec{x}) = \prod_{i=1}^{n} f(x_i; \vec{\theta})$$

The likelihood function of the sample, measures the likelihood that the sample came from the distribution with the given parameters. The log-likelihood function for an observation x_i is $\ln(f(x_i; \vec{\theta})$. Thus, the log-likelihood function for the sample is:

$$\text{LL}(\vec{\theta}; \vec{x}) = \sum_{i=1}^{n} ln(f(x_i; \vec{\theta}))$$

The BIC is defined in terms of the value of the log-likelihood of the sample as follows:

$$\text{BIC} = k\ln(n) - 2\text{LL}(\hat{\vec{\theta}}; \vec{x})$$

where n is the sample size, k is the number of parameters estimated for the model, $\hat{\vec{\theta}}$ are the estimated parameter values.

- Akaike Information Criterion (AIC) - Developed by (Akaike 1974), the AIC is used for model selection where models with lower AIC are preferred. Like the BIC, the AIC is defined in terms of the log-likelihood function of the sample. The KSL uses the formulation described in (Vose 2010) as follows:

$$\text{AIC} = \left(\frac{n - 2k + 2}{n - k + 1}\right) - 2\text{LL}(\hat{\vec{\theta}}; \vec{x})$$

These scoring models avoid summarizing the data based on a histogram, which requires a specification of the bin sizes (or widths) and tabulation of frequencies or proportions associated with the bins.

As illustrated in Figure 2.8, the scoring models are implemented by sub-classing from the `PDFScoringModel` abstract base class. Users of the KSL distribution recommendation framework can select which scoring models to include during the evaluation process. The default set of scoring models are BIC, AD, CVM, and QQC. During testing, the BIC criterion performed very well across all but very low sample sizes. The AD and CVM criteria tend to perform better for lower sample sizes, while the QQC criterion tends to perform better for higher sample sizes and for particular distribution families.

The quality of a parametric fit for a specific distribution can be evaluated by one or more scoring models. Since distribution quality metrics have been

designed to measure different aspects of the fit, the KSL allows the scoring results to be combined into an overall score. Suppose distribution F_k is evaluated by the scoring models, each resulting in score S_i for $i = 1, 2, \cdots, m$, where m is the number of scoring models. In general, the scores may not have the same scales and the same direction of *goodness*. The KSL scoring system translates and scales each score S_i in to a value measure, M_i, that has a common scale and direction (a bigger score is better). These value measures are then combined into an overall value (V_k) for the distribution using weights (w_i) across the scoring criteria:

$$V_k = \sum_{i=1}^{m} w_i \times M_i$$

The distribution that has the overall largest value, V_k, is then recommended as the best fitting probability model.

This methodology is based on the commonly used multi-objective decision analysis (MODA) method for choosing among a set of alternatives. In the KSL's default application of this methodology, each scoring model is weighted equally and a linearly additive model is assumed. The framework is designed to allow the user to define and apply their own scoring models and to set the preference weights among the metrics. In addition, sensitivity analysis of the weights can easily be accomplished. Once the best distribution has been recommended, then the KSL facilitates performing statistical tests with resulting p-values.

Figure 2.9 presents the main class (`PDFModeler`) for performing the continuous distribution fitting task. The companion object provides functionality to estimate the shift parameter, shift the data to the left, compute confidence intervals for the minimum and maximum of the data via bootstrapping, estimate the range of possible values for the distribution, create distributions from estimated parameters, create the default evaluation model, and work with histograms.

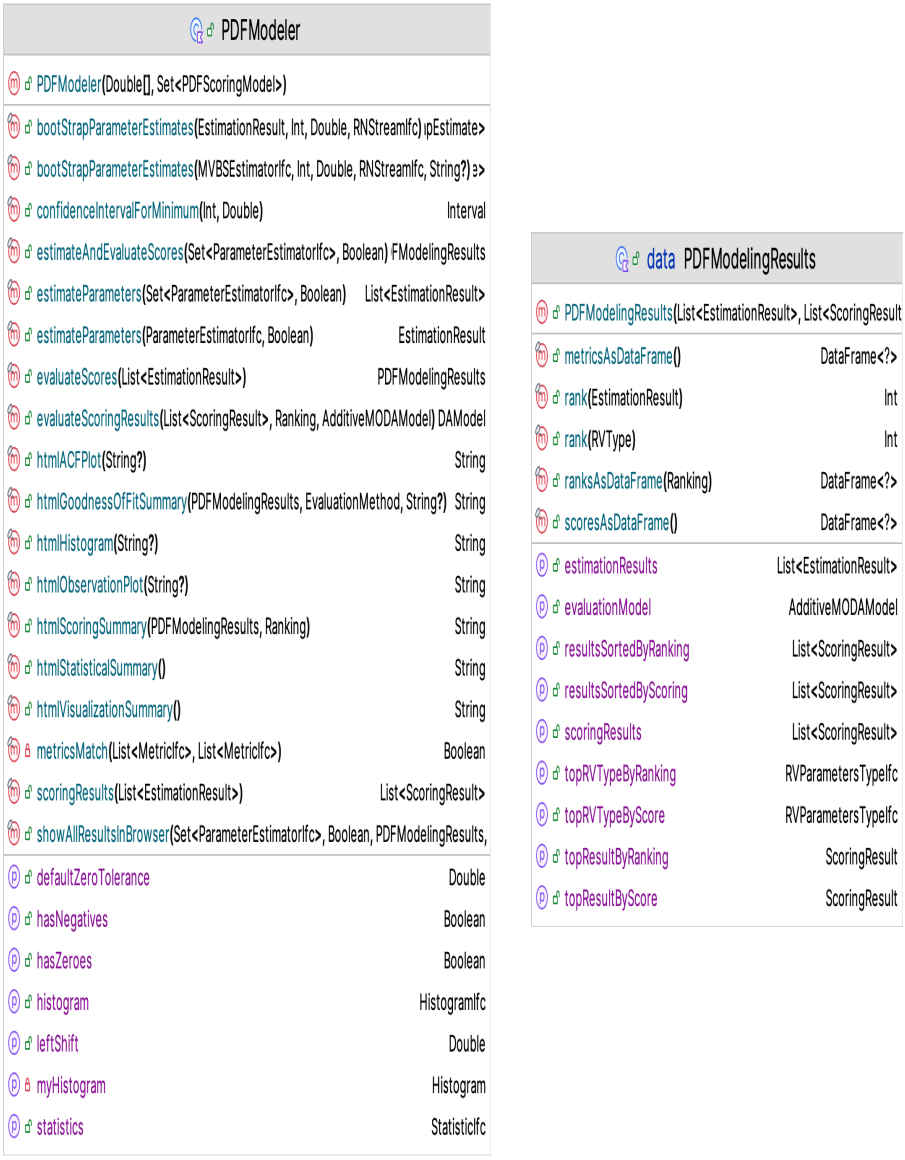


Figure 2.9.: PDF Modeling Framework

The modeling process starts with creating an instance of `PDFModeler` by supplying the data to be modeled and the set of scoring models to be used within the evaluation process. Then, functions can be used to perform the estimation and scoring tasks.

- `estimateParameters(estimators: Set<ParameterEstimatorIfc>, automaticShifting: Boolean = true)`: This function estimates the parameters for all estimators represented by the supplied set of estimators. The parameter, `automaticShifting` controls whether the data will be automatically shifted. If the automatic shift parameter is true (the default), then a confidence interval for the minimum of the data is estimated from the data. If the upper limit of the estimated confidence interval is greater than the value specified by the default zero tolerance property, then the data is shifted to the left and used in the estimation process. The estimated shift will be recorded in the result. Automated shift estimation will occur only if the automatic shifting parameter is true, if the estimator requires that its range be checked, and if the data actually requires a shift. If the automatic shifting parameter is false, then no shifting will occur. In this case it is up to the user to ensure that the supplied data is representative of the set of estimators to be estimated. The returned list will contain the results for each estimator.
- `evaluateScores(estimationResults: List<EstimationResult>)`: This function applies the supplied scoring models to the results from the parameter estimation process. Each distribution with estimated parameters will be scored by each of the supplied models and the results tabulated as an instance of the `PDFModelingResults` class.
- `estimateAndEvaluateScores(estimators: Set<ParameterEstimatorIfc> = allEstimators, automaticShifting: Boolean = true, scoringModels: Set<PDFScoringModel> = allScoringModels)`: As its name implies, this function combines the functions of `estimateParameters()` and `evaluateScores()` into a single function for the convenience of the modeler. The returned instance

of the `PDFModelingResults` class has the estimation results and the scoring results.

- `showAllResultsInBrowser(fileName: String = "pdfModelingResults")`: This function makes a histogram, observations plot, auto-correlation plot, performs the fitting and scoring process, and performs goodness of fit tests on the top scoring distribution and displays all the results by opening a browser window. The generated `html` file is stored in the `KSL.plotDir` directory using the supplied name as the pre-fix for a temporary file. This one function performs all the necessary fitting steps and returns the results, including plots, within an `html` page.
- `showAllGoodnessofFitSummariesInBrowser(pdfModelingResults: PDFModelingResults)`: This function shows the goodness of fit summaries for every distribution used within the fitting process. This function shows the raw scores, the scaled scores, the ranked scores, and the PDF summary results for each distribution. The PDF summary results includes the `FitDistPlot` plots as well as the results of the Chi-Squared, K-S, Anderson Darling, and Cramer Von Mises, statistical tests. This function is very useful when comparing the quality of the fitted distributions.

The `PDFModeling` class encapsulates the estimation and scoring process. The purpose of this class is to serve as the general location for implementing the estimation of distribution parameters across many distributions. The general use involves the following:

```
val d = PDFModeler(data)
val estimationResults: List<EstimationResult> =
 ↪  d.estimateParameters(d.allEstimators)
val scoringResults = d.scoringResults(estimationResults)
val model = d.evaluateScoringResults(scoringResults)
scoringResults.forEach( ::println)
```

The scoring results will be updated with the evaluation information and will contain the evaluation scores. The scoring results can be sorted to find the recommended distribution based on the evaluation score. Alternatively, the single function can be used:

```
val d = PDFModeler(data)
val results  = d.estimateAndEvaluateScores()
```

This function returns an instance of `PDFModelingResults,` which will have the results of the entire fitting process. The advantage of using the individual functions may permit some further customization of the estimation process. The `PDFModelingResults` class in Figure 2.9 is a data class that holds all of the results from the fitting and scoring process. In addition, the class holds an instance of the scoring model (`AdditiveMODAModel`), which can be used to perform additional analysis of the scores. The property `resultsSortedByScoring` provides the list of results sorted such that the top performer is the first element of the list, and `topResultByScore` returns that first element.

To summarize, the KSL continuous distribution modeling framework allows the modeler to:

- Define and select from a set of distributions to evaluate
- Define and select from a set of parameter estimation methods
- Compute bootstrap estimates of the confidence intervals for the estimated parameters
- Define and select from a set of distribution fit quality metrics (scoring models)
- Combine scoring models into an overall measure
- Recommend the best distribution based on weighted preference among evaluation metrics
- Make observations plots, histograms, autocorrelation plots, P-P plots, Q-Q plots, and empirical distribution plots.

All of this functionality is encapsulated in the `PDFModeler` class. This functionality will be illustrated in Section 2.4.4.

After performing the estimation and scoring process, the modeler may want to perform statistical tests for the top performing distribution (or others). The `ContinuousCDFGoodnessOfFit` class facilitates the performing of the following goodness of fit tests:

- Chi-Squared
- Kolmogorov-Smirnov
- Anderson-Darling
- Cramer-Von Mises

Figure 2.10 shows the framework of classes and interfaces for performing goodness of fit tests within the KSL. The main class is the `ContinuousCDFGoodnessOfFit` class. This class will compute the test statistics and their P-values.

The chi-squared test is the most challenging test to execute in an automated fashion. The KSL follows the recommendation found in Chapter 6 of (Law 2007) for setting up the chi-squared test. The KSL does not use arbitrary break points from a histogram generation process. Instead the KSL attempts to define the break points for the chi-squared intervals such that each interval has the *same* probability of occurrence. This also ensures that the expected number of observations within each interval is approximately the same. The theoretical basis for this approach can be found in (Williams 1950). (Williams 1950) considered the testing of $U(0, 1)$ random variates. In the case of the $U(0, 1)$ distribution, the choice of the number of intervals determines the break points because each interval is equally likely. (Williams 1950) recommended choosing the class limits such that the expected number in the interval was n/k, where n is the number of observations and k is the number of class intervals. Based on the ability to ensure that the resulting chi-squared test statistic actually has a chi-squared distribution, (Williams 1950) recommended that the number of class intervals be:

Figure 2.10.: Continuous Distribution Goodness of Fit Framework

$$k = \left\lceil 4 \sqrt[5]{\frac{2(n-1)^2}{z_{1-\alpha}}} \right\rceil$$

This approach produces equally distant break points between $(0, 1)$. Let's call those break points u_i for $i = 1, \cdots, k$. We then define the break points for the chi-squared test for the distribution, $F(x)$, as $b_i = F^{-1}(u_i)$, which will result in non-equally spaced break points for $F(x)$, but the probability p_i associated with the intervals will all be the same. For the resulting recommended break points, the procedure attempts to ensure that the number of intervals is at least 3 and that the expected number within the intervals is at least 5. If the number of observations of the sample, n, is less than or equal to 15, this may not be possible. The procedure ensures that there are at least 3 intervals and if the expected count is less than 5 for any interval the user is warned. This same procedure is used in determining the break points for the squared error criteria. This approach reduces the sensitivity of the chi-squared fitting process and the squared error criteria to the choice of the intervals. This functionality is found in the `suggestBreakPoints()` function of the companion object of the `ContinuousCDFGoodnessOfFit` class.

As an example, the following code generates an exponentially distributed sample and applies the goodness of fit tests to the data.

Example 2.24 (Goodness of Fit Testing)**.**

```
fun main() {
    val d = Exponential(10.0)
    val e = d.randomVariable(3)
    e.advanceToNextSubStream()
    val n = 1000
    val data = e.sample(n)
    val gof = ContinuousCDFGoodnessOfFit(data, d)
    println(gof)
```

```
}
```

The results, as expected, indicate that the data is exponentially distributed. Notice how the binning process for the data results in the bin probabilities and expected number per bin being all equal. The observed counts look pretty consistent for the intervals. Thus, the chi-squared test does not reject the hypothesis that the data is exponential with a mean of 10. In addition, the Anderson-Darling, K-S test, and Cramer Von Mises P-values indicate that we should not reject the hypothesis that the data is exponentially distributed. The `showAllResultsInBrowser()` function of the `PDFModeler` class automatically performs these goodness of fit tests for the recommended distribution from the PDF modeling process.

```
GOF Results for Distribution: Exponential(mean=10.0)
-----------------------------------------------------

Chi-Squared Test Results:
Bin Label                 P(Bin)       Observed   Expected
  1 [ 0.00, 0.17)         0.016949           20    16.9492
  2 [ 0.17, 0.34)         0.016949           14    16.9492
  3 [ 0.34, 0.52)         0.016949           23    16.9492
  4 [ 0.52, 0.70)         0.016949           14    16.9492
  5 [ 0.70, 0.89)         0.016949           21    16.9492
  6 [ 0.89, 1.07)         0.016949           21    16.9492
  7 [ 1.07, 1.26)         0.016949           22    16.9492
  8 [ 1.26, 1.46)         0.016949           10    16.9492
  9 [ 1.46, 1.66)         0.016949           18    16.9492
 10 [ 1.66, 1.86)         0.016949           12    16.9492
 11 [ 1.86, 2.06)         0.016949           16    16.9492
 12 [ 2.06, 2.27)         0.016949           12    16.9492
 13 [ 2.27, 2.49)         0.016949           23    16.9492
 14 [ 2.49, 2.71)         0.016949           15    16.9492
```

```
15 [ 2.71, 2.93)        0.016949          17    16.9492
16 [ 2.93, 3.16)        0.016949          14    16.9492
17 [ 3.16, 3.40)        0.016949          21    16.9492
18 [ 3.40, 3.64)        0.016949          13    16.9492
19 [ 3.64, 3.89)        0.016949          11    16.9492
20 [ 3.89, 4.14)        0.016949          20    16.9492
21 [ 4.14, 4.40)        0.016949          18    16.9492
22 [ 4.40, 4.67)        0.016949          19    16.9492
23 [ 4.67, 4.94)        0.016949          23    16.9492
24 [ 4.94, 5.22)        0.016949          19    16.9492
25 [ 5.22, 5.51)        0.016949          17    16.9492
26 [ 5.51, 5.81)        0.016949          18    16.9492
27 [ 5.81, 6.12)        0.016949          12    16.9492
28 [ 6.12, 6.44)        0.016949          17    16.9492
29 [ 6.44, 6.76)        0.016949          16    16.9492
30 [ 6.76, 7.10)        0.016949          13    16.9492
31 [ 7.10, 7.45)        0.016949          18    16.9492
32 [ 7.45, 7.82)        0.016949          17    16.9492
33 [ 7.82, 8.19)        0.016949          21    16.9492
34 [ 8.19, 8.59)        0.016949          17    16.9492
35 [ 8.59, 8.99)        0.016949          25    16.9492
36 [ 8.99, 9.42)        0.016949           9    16.9492
37 [ 9.42, 9.86)        0.016949          14    16.9492
38 [ 9.86,10.33)        0.016949          15    16.9492
39 [10.33,10.82)        0.016949          13    16.9492
40 [10.82,11.33)        0.016949          12    16.9492
41 [11.33,11.87)        0.016949          18    16.9492
42 [11.87,12.44)        0.016949          23    16.9492
43 [12.44,13.05)        0.016949          20    16.9492
44 [13.05,13.69)        0.016949           9    16.9492
45 [13.69,14.38)        0.016949          14    16.9492
46 [14.38,15.13)        0.016949          16    16.9492
47 [15.13,15.93)        0.016949          13    16.9492
48 [15.93,16.80)        0.016949          22    16.9492
```

```
 49 [16.80,17.75)         0.016949           15    16.9492
 50 [17.75,18.80)         0.016949           18    16.9492
 51 [18.80,19.98)         0.016949           23    16.9492
 52 [19.98,21.32)         0.016949           23    16.9492
 53 [21.32,22.86)         0.016949           25    16.9492
 54 [22.86,24.68)         0.016949            9    16.9492
 55 [24.68,26.91)         0.016949           14    16.9492
 56 [26.91,29.79)         0.016949           16    16.9492
 57 [29.79,33.84)         0.016949           15    16.9492
 58 [33.84,40.78)         0.016949           15    16.9492
 59 [40.78,155.00)        0.016949           22    16.9490

Number of estimated parameters = 1
Number of intervals = 59
Degrees of Freedom = 57
Chi-Squared Test Statistic = 61.52812706136465
P-value = 0.31723313805973075
Hypothesis test at 0.05 level:
The p-value = 0.31723313805973075 is >= 0.05 : Do not reject hypothesis.

Goodness of Fit Test Results:
K-S test statistic = 0.017687069238841113
K-S test p-value = 0.9075526348536717

Anderson-Darling test statistic = 0.3610051171259556
Anderson-Darling test p-value = 0.8863125779601514

Cramer-Von-Mises test statistic = 0.043345445388941854
Cramer-Von-Mises test p-value = 0.9152815646329959
```

2.4.3. Discrete Distribution Framework

For discrete distributions, the KSL provides the `PMFModeler` class. Since there are less discrete distributions within the KSL than continuous distributions, the `PMFModeler` class does not provide the full range of functionality provided by the `PDFModeler` class. Instead the `PMFModeler` class focuses on estimating the parameters of discrete distributions. Similar to how the `PDFModeler` class functions, the `PMFModeler` has an `estimateParameters()` function that uses a set of objects that implement the `ParameterEstimatorIfc` interface. The default set of discrete distributions includes the Poisson, negative binomial, and the binomial distributions. Of course users can define additional discrete distributions and implement parameter estimators for those distributions.

Figure 2.11 presents the PMF estimation framework. As indicated the key function is the `estimateParameters()` function. Note also that the companion object for the `PMFModeler` class has the function `equalizedPMFBreakPoints()`, which can be useful during the goodness of fit testing of the distribution. The purpose of the function is to attempt to form intervals that have similar probabilities.

The application of the `estimateParameters()` function results in the creation of a list that contains instances of `EstimationResult`, one for each of the estimation methods.

Given the limited number of discrete distributions, the KSL does not provide a scoring model approach for selecting the best distribution. Instead, the `DiscretePMFGoodnessOfFit` class, illustrated in Figure 2.12 can be used to check the goodness of fit for the discrete distribution. The primary method for testing the goodness of it is the chi-squared goodness of fit test. Similar to how the continuous distribution defines break points that result in approximately equal probabilities for the bins and expected counts, the KSL attempts to form intervals for the chi-squared test that have approximately equal probabilities. The following code illustrates how to fit a negative binomial distribution to some randomly generated data.

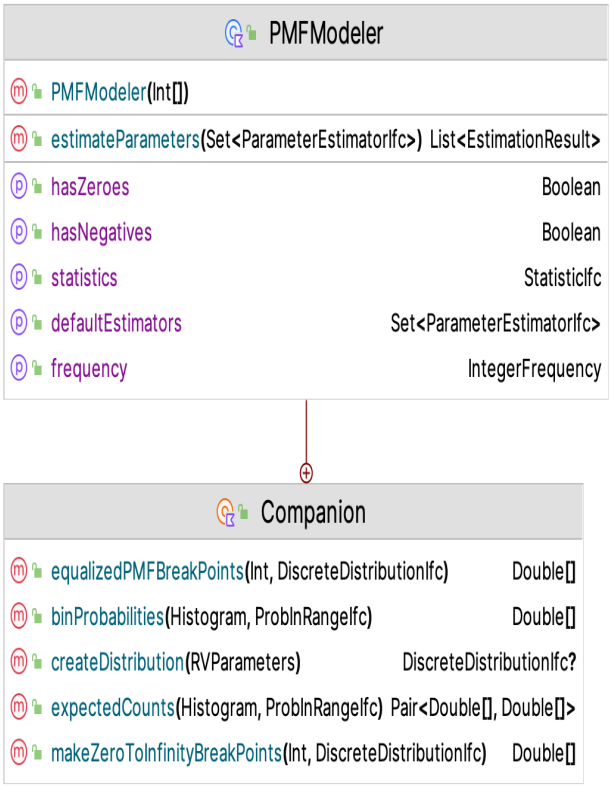


Figure 2.11.: PMF Estimation Framework

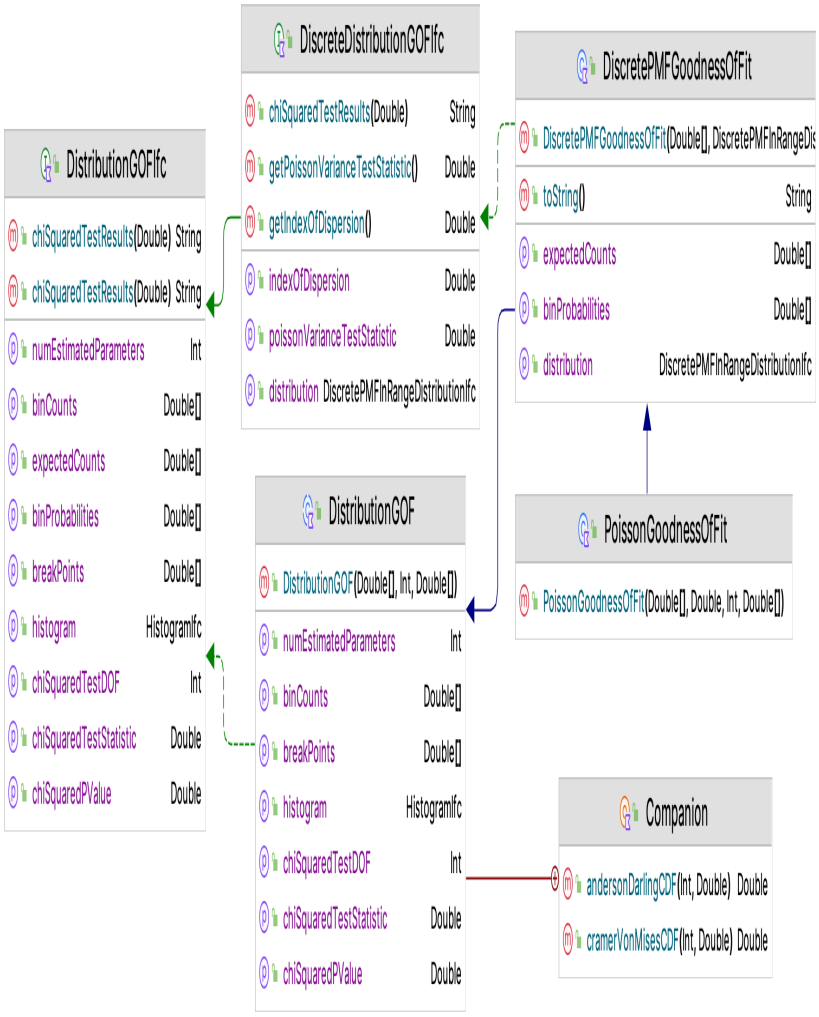


Figure 2.12.: Discrete Distribution Goodness of Fit Framework

Example 2.25 (Discrete Goodness of Fit Testing)**.**

```
fun main() {
    val dist = NegativeBinomial(0.2, theNumSuccesses = 4.0)
    val rv = dist.randomVariable(streamNumber = 3)
    rv.advanceToNextSubStream()
    val data = rv.sample(200)
    val breakPoints =
     ↪  PMFModeler.makeZeroToInfinityBreakPoints(data.size,
     ↪  dist)
    val pf = DiscretePMFGoodnessOfFit(data, dist, breakPoints
     ↪  = breakPoints)
    println(pf.chiSquaredTestResults())
}
```

The results indicate the challenge of trying to make bins with equal probability for discrete distributions. In general, it may not be possible to do so. Thus, the modeler is encouraged to update the break points as needed during the distribution assessment process. As expected the results indicate that we should not reject the hypothesis of a negative binomial distribution for this situation.

```
Chi-Squared Test Results:
Bin Label                 P(Bin)       Observed   Expected
  1 [ 0.00, 3.00)         0.016960            4    3.39200   *** Warning: expected <
  2 [ 3.00, 5.00)         0.039322            7    7.86432
  3 [ 5.00, 6.00)         0.029360           10    5.87203
  4 [ 6.00, 7.00)         0.035232            7    7.04643
  5 [ 7.00, 8.00)         0.040265            7    8.05306
  6 [ 8.00, 9.00)         0.044292            5    8.85837
  7 [ 9.00,10.00)         0.047245           10    9.44893
  8 [10.00,11.00)         0.049134            9    9.82689
  9 [11.00,12.00)         0.050028            6    10.0056
```

```
 10 [12.00,13.00)         0.050028           11    10.0056
 11 [13.00,14.00)         0.049258            6    9.85162
 12 [14.00,15.00)         0.047851           12    9.57015
 13 [15.00,16.00)         0.045937            7    9.18734
 14 [16.00,17.00)         0.043640           17    8.72798
 15 [17.00,18.00)         0.041073            4    8.21457
 16 [18.00,19.00)         0.038335            7    7.66693
 17 [19.00,20.00)         0.035510            7    7.10200
 18 [20.00,21.00)         0.032669            9    6.53384
 19 [21.00,22.00)         0.029869            5    5.97379
 20 [22.00,23.00)         0.027154            3    5.43072
 21 [23.00,24.00)         0.024556            4    4.91126   *** Warning: expected <
 22 [24.00,26.00)         0.041903           13    8.38058
 23 [26.00,28.00)         0.033376            9    6.67520
 24 [28.00,31.00)         0.036998            6    7.39969
 25 [31.00,36.00)         0.036799            7    7.35973
 26 [36.00,Infinity)      0.033207            8    6.64147

Number of estimated parameters = 2
Number of intervals = 26
Degrees of Freedom = 23
Chi-Squared Test Statistic = 25.69659399548871
P-value = 0.31539706650371313
Hypothesis test at 0.05 level:
The p-value = 0.31539706650371313 is >= 0.05 : Do not reject hypothesis.
```

2.4.4. Illustrative Examples from Appendix B

This section illustrates how to use the KSL probability distribution modeling frameworks by applying the previously discussed constructs to two examples from Appendix B. We will start with the fitting of Poisson distribution to the data from Example B.1, which is repeated here for convenience. The data associated with the examples of this section can be

found in the chapter files with in the `KSLExamples` project associated with the KSL source code repository.

> **Note**
>
> Distribution fitting often requires visualizing the data. The KSL provides support for making plots via the lets-plot library. Section E.8 of Appendix E illustrates the basics of KSL plotting functionality. This section illustrates some plots that are important for distribution modeling.

Example 2.26 (Fitting a Poisson Distribution)**.** Suppose that we are interested in modeling the demand for a computer laboratory during the morning hours between 9 am to 11 am on normal weekdays. During this time a team of undergraduate students has collected the number of students arriving to the computer lab during 15 minute intervals over a period of 4 weeks.

Since there are four 15 minute intervals in each hour for each two hour time period, there are 8 observations per day. Since there are 5 days per week, we have 40 observations per week for a total of $40 \times 4 = 160$ observations. A sample of observations per 15 minute interval are presented in Table 2.1. The full data set is available with the chapter files. Check whether a Poisson distribution is an appropriate model for this data.

Table 2.1.: Computer Laboratory Arrival Counts by Week, Period, and Day

Week	Period	M	T	W	TH	F
1	9:00-9:15 am	8	5	16	7	7

Week	Period	M	T	W	TH	F
1	9:15-9:30 am	8	4	9	8	6
1	9:30-9:45 am	9	5	6	6	5
1	9:45-10:00 am	10	11	12	10	12
1	10:00-10:15 am	6	7	14	9	3
1	10:15-10:30 am	11	8	7	7	11
1	10:30-10:45 am	12	7	8	3	6
1	10:45-11:00 am	8	9	9	8	6
2	9:00-9:15 am	10	13	7	7	7
3	9:00-9:15 am	5	7	14	8	8
4	9:00-9:15 am	7	11	8	5	4
4	10:45-11:00 am	8	9	7	9	6

Since the testing of dependence on the day of the week or the period of observation was already performed in Appendix B, we focus on the distribution fitting process in this demonstration of the KSL constructs. Here, we need to fit a Poisson distribution to the data. The first step is reading the data from the CSV file. This can be readily accomplished by using the CSV file reading capabilities of Kotlin's `DataFrame` library. The following code uses the KSL's `KSLFileUtil` object to open a file choosing dialog for the user to select the file, which then has its contents read in using the CSV file processing function for the `DataFrame` object. This data is converted into an integer array and returned. A discussion of CSV file processing and data frames can be found in Section E.4 and Section E.5 of Appendix E.

```
fun readCountData(): IntArray {
    // choose file:
     ↪ KSL/KSLExamples/chapterFiles/Appendix-Distribution
     ↪ Fitting/PoissonCountData.csv
```

```
    val file = KSLFileUtil.chooseFile()
    if (file != null) {
        val df = DataFrame.readCSV(
            file,
            colTypes = mapOf(
                "week" to ColType.Int,
                "period" to ColType.Int,
                "day" to ColType.String,
                "count" to ColType.Int
            )
        )
        val count by column<Int>()
        val countData: DataColumn<Int> = df[count]
        return countData.toIntArray()
    }
    return IntArray(0)
}
```

Once the data is an array, the KSL's discrete distribution framework can be easily applied. In the following code, the data is used to visualize the count data in the form of an integer frequency plot, which we can use like a histogram to view the distribution of the data and see if it has a shape that would support the hypothesis of a Poisson distribution.

```
val data = readCountData()
val f = IntegerFrequency(data)
val fp = f.frequencyPlot()
fp.showInBrowser()
fp.saveToFile("Lab_Count_Freq_Plot")
```

Figure 2.13 clearly indicates that the Poisson distribution is a candidate model.

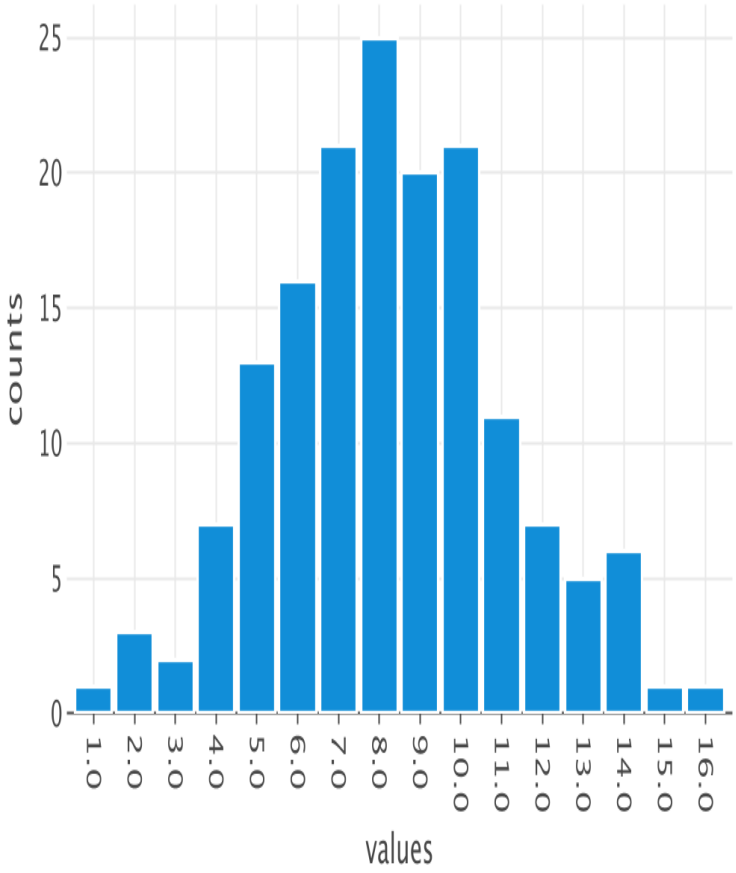


Figure 2.13.: Integer Frequency Plot of Lab Count Data

The following code will create a time series observation plot of the data and an auto correlation plot of the data. The use of both of these plots for analysis of data is discussed in Appendix B. This code will also save the plots to files and display the plots within a browser window.

```
val op = ObservationsPlot(data)
op.saveToFile("Lab_Count_Obs_Plot")
op.showInBrowser()
val acf = ACFPlot(data.toDoubles())
acf.saveToFile("Lab_Count_ACF_Plot")
acf.showInBrowser()
println(f)
```

Just as in Appendix B, the time series plot does not indicate a trend or pattern with respect to time as shown in Figure 2.14.

Again, the ACF plot, see Figure 2.15, does not indicate that the data has any significant concerns with respect to autocorrelation.

Because the plots indicate that the data is likely to be independent and identically distributed, it appears that it should be safe to proceed with the distribution modeling task. The following code will estimate the parameters, perform a goodness of fit test, and display a plot that compares the theoretical PMF to the empirical PMF.

```
val pmfModeler = PMFModeler(data)
val results = pmfModeler.estimateParameters(setOf(␣
↪ PoissonMLEParameterEstimator))
val e = results.first()
println(e)
val mean = e.parameters!!.doubleParameter("mean")
val pf = PoissonGoodnessOfFit(data.toDoubles(), mean = mean)
println(pf)
val plot = PMFComparisonPlot(data, pf.distribution)
```

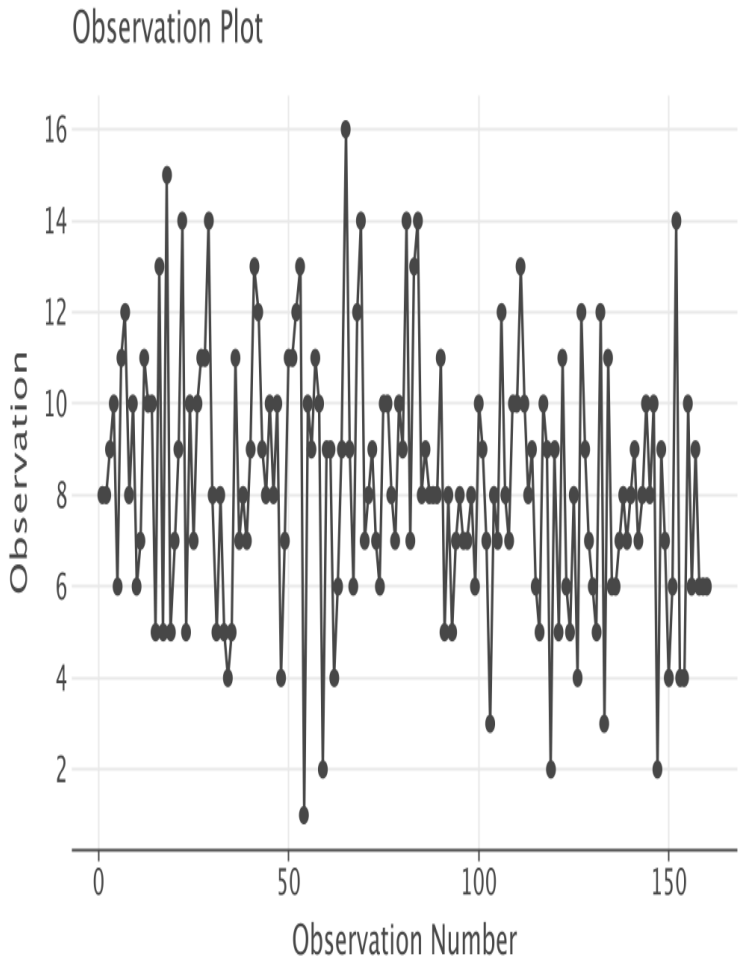


Figure 2.14.: Time Series Plot of Lab Count Data

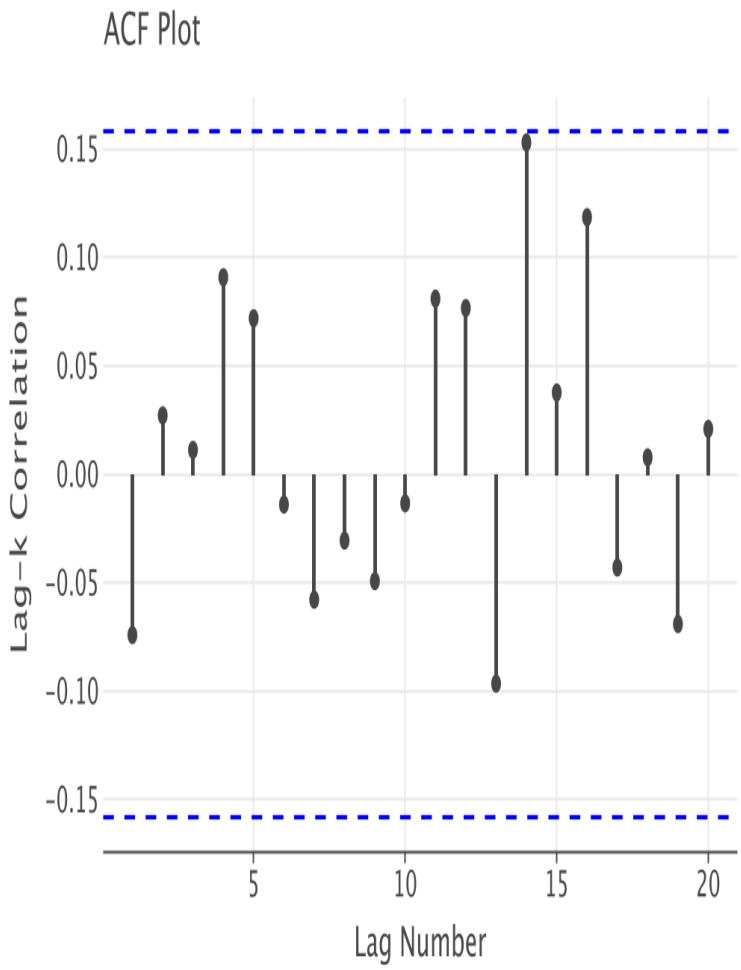


Figure 2.15.: Autocorrelation Plot of Lab Count Data

```
plot.saveToFile("Lab_Count_PMF_Plot")
plot.showInBrowser()
```

The code first creates an instance of the `PMFModeler` class to perform the estimation. In this situation, only the estimator for the Poisson distribution is specified as input to the `estimateParameters()` function. The estimated mean of the distribution is extracted from the estimated parameters and used for the `PoissonGoodnessOfFit` class. The `PoissonGoodnessOfFit` class is a straight-forward sub-class of `DiscretePMFGoodnessOfFit` class. The results of the fit are printed and then the PMF comparison plot is created.

```
Shift estimation results:
null

Estimated parameters:
RV Type = Poisson
Double Parameters {mean=8.275000000000002}
Integer Parameters {}
Double Array Parameters {}

Chi-Squared Test Results:
Bin Label                 P(Bin)       Observed   Expected
  1 [ 0.00, 4.00)         0.035151            6    5.62421
  2 [ 4.00, 5.00)         0.049782            7    7.96515
  3 [ 5.00, 6.00)         0.082390           13    13.1823
  4 [ 6.00, 7.00)         0.113629           16    18.1806
  5 [ 7.00, 8.00)         0.134326           21    21.4921
  6 [ 8.00, 9.00)         0.138943           25    22.2309
  7 [ 9.00,10.00)         0.127750           20    20.4401
  8 [10.00,11.00)         0.105713           21    16.9141
  9 [11.00,12.00)         0.079525           11    12.7241
```

```
 10 [12.00,13.00)         0.054839            7    8.77429
 11 [13.00,14.00)         0.034907            5    5.58518
 12 [14.00,Infinity)      0.043044            8    6.88700

Number of estimated parameters = 1
Number of intervals = 12
Degrees of Freedom = 10
Chi-Squared Test Statistic = 2.5923612067415056
P-value = 0.9894603477732713
Hypothesis test at 0.05 level:
The p-value = 0.9894603477732713 is >= 0.05 : Do not reject hypothesis.
```

The results of the goodness of fit test and the PMF comparison plot, Figure 2.16, clearly indicate that the Poisson distribution should not be rejected as a model for this data.

The following example analyzes the pharmacy service time data using KSL constructs.

Example 2.27 (Analyzing the Pharmacy Data using the KSL)**.** One hundred observations of the service time were collected using a portable digital assistant and are shown in Table 2.2 where the first observation is in row 1 column 1, the second observation is in row 2 column 1, the 21^{st} observation is in row 1 column 2, and so forth.

Table 2.2.: Pharmacy Service Times

61	278.73	194.68	55.33	398.39
59.09	70.55	151.65	58.45	86.88
374.89	782.22	185.45	640.59	137.64

195.45	46.23	120.42	409.49	171.39
185.76	126.49	367.76	87.19	135.6
268.61	110.05	146.81	59	291.63
257.5	294.19	73.79	71.64	187.02
475.51	433.89	440.7	121.69	174.11
77.3	211.38	330.09	96.96	911.19
88.71	266.5	97.99	301.43	201.53
108.17	71.77	53.46	68.98	149.96
94.68	65.52	279.9	276.55	163.27
244.09	71.61	122.81	497.87	677.92
230.68	155.5	42.93	232.75	255.64
371.02	83.51	515.66	52.2	396.21
160.39	148.43	56.11	144.24	181.76
104.98	46.23	74.79	86.43	554.05
102.98	77.65	188.15	106.6	123.22
140.19	104.15	278.06	183.82	89.12
193.65	351.78	95.53	219.18	546.57

Recall the basic process described in Appendix B. We will plot the data using a histogram, make a time series observation plot, and make an ACF plot. The following code performs those tasks. The plots can be shown in a browser window or saved to files.

```
val data = KSLFileUtil.scanToArray(myFile.toPath())
val d = PDFModeler(data)
println(d.histogram)
println()
val hPlot = d.histogram.histogramPlot()
hPlot.showInBrowser()
val op = ObservationsPlot(data)
```

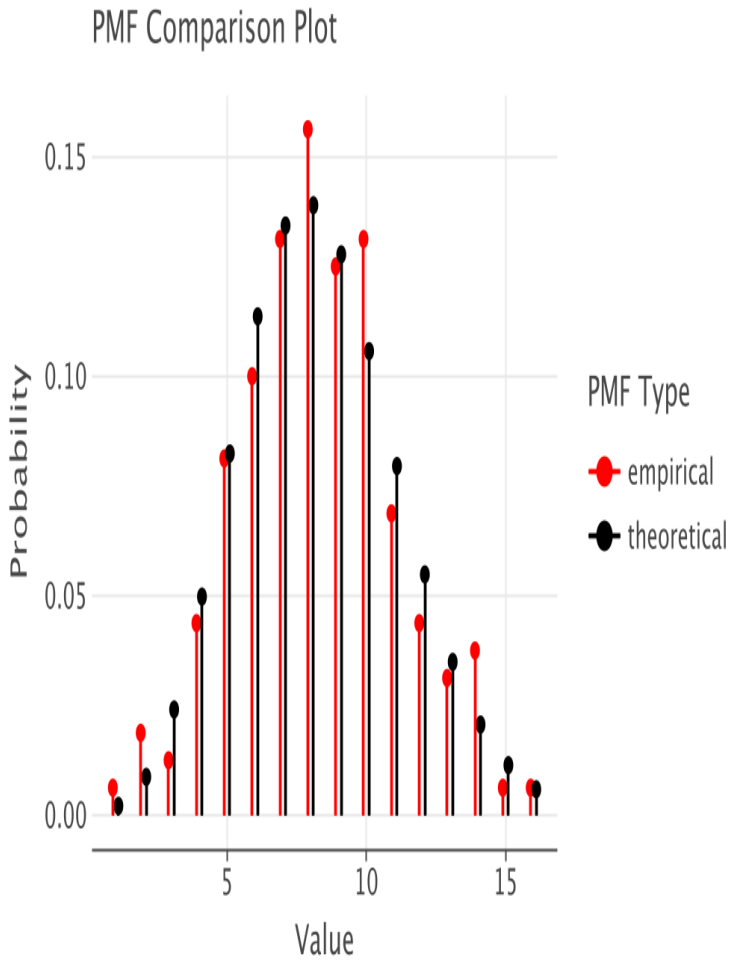


Figure 2.16.: PMF Comparison Plot of Lab Count Data

```
op.showInBrowser()
val acf = ACFPlot(data)
acf.showInBrowser()
```

This results in the following histogram summary, which clearly indicates that the data is shifted from zero to about 36.

```
-------------------------------------
Number of bins = 6
First bin starts at = 36.0
Last bin ends at = 786.0
Under flow count = 0.0
Over flow count = 0.0
Total bin count = 100.0
Total count = 100.0
-------------------------------------
Bin Range        Count CumTot Frac  CumFrac
  1 [36.00,161.00)  = 60  60.0 0.600000 0.600000
  2 [161.00,286.00)  = 22  82.0 0.220000 0.820000
  3 [286.00,411.00)  = 10  92.0 0.100000 0.920000
  4 [411.00,536.00)  = 6  98.0 0.060000 0.980000
  5 [536.00,661.00)  = 1  99.0 0.010000 0.990000
  6 [661.00,786.00)  = 1 100.0 0.010000 1.000000
-------------------------------------
```

Figure 2.17 clearly shows an exponential shape for the distribution.

Similar to the analysis in Appendix B, the time series plot, Figure 2.18, and the ACF plot, Figure 2.19 do not indicate any issues with non-stationary behavior or with dependence within the observations.

Given this analysis, we can proceed with modeling the distribution and checking for the goodness of fit. The following KSL code will estimate the

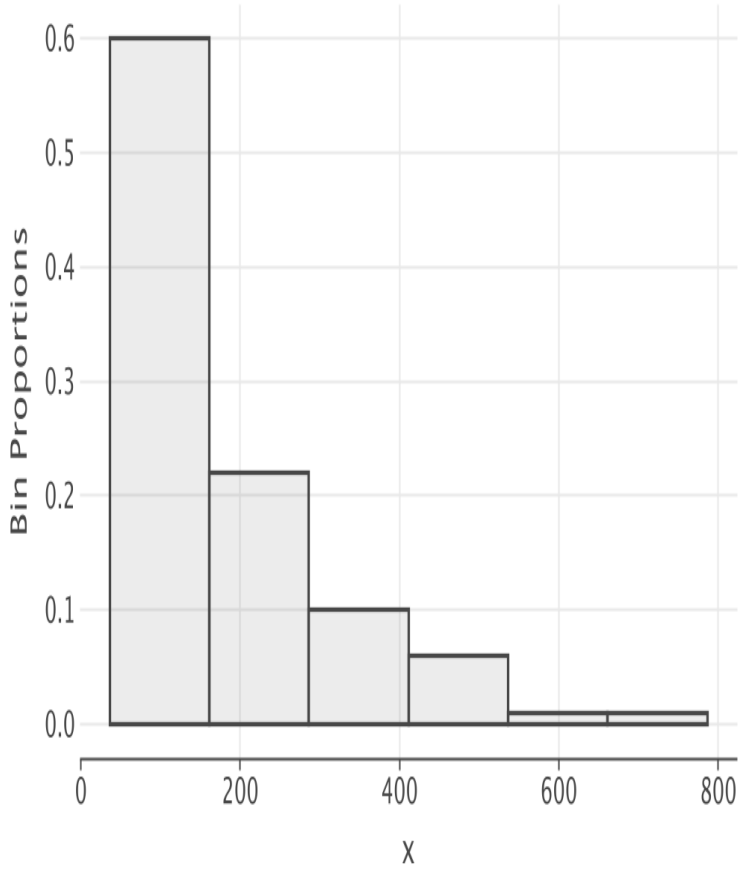


Figure 2.17.: Histogram of the Pharmacy Data

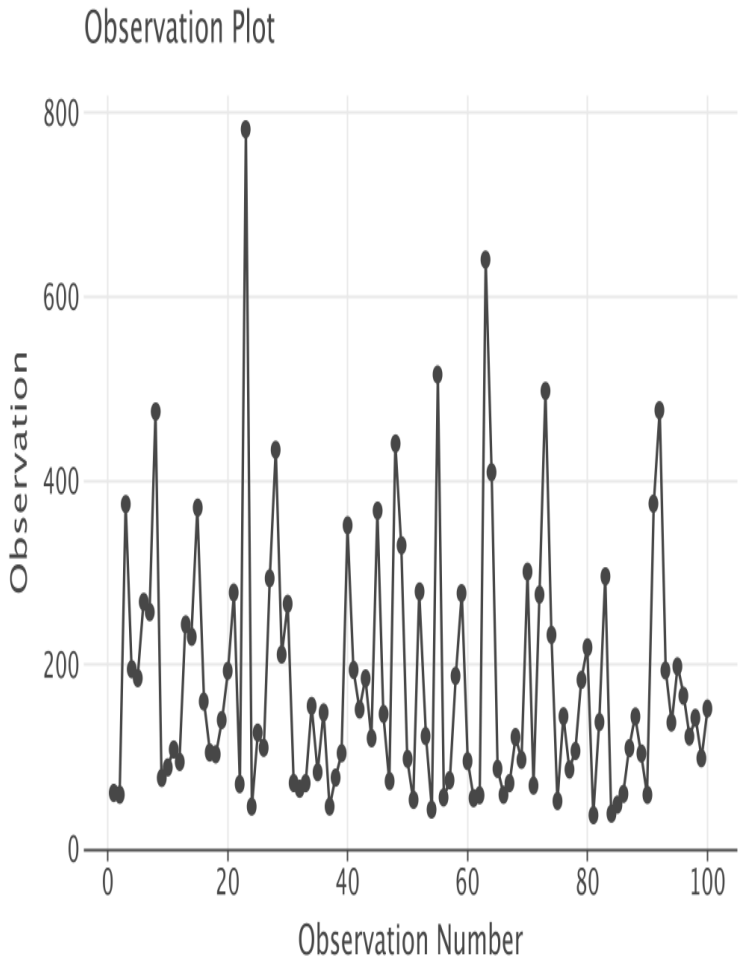


Figure 2.18.: Time Series Observation Plot for Pharmacy Data

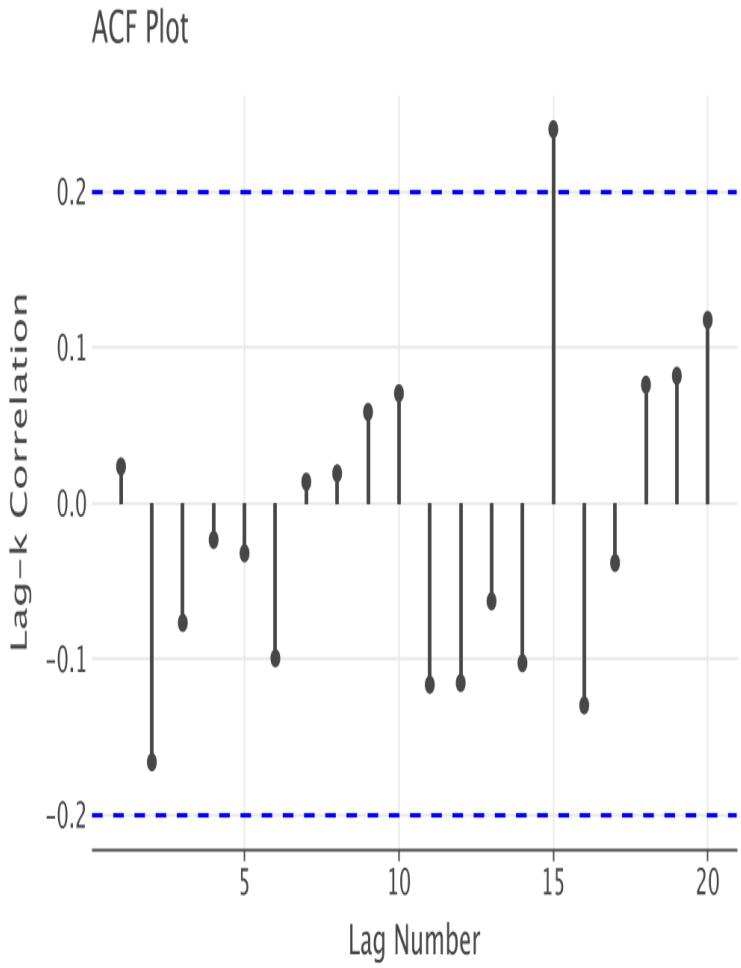


Figure 2.19.: ACF Plot for Pharmacy Data

parameters of the default distributions within the KSL and recommend a candidate distribution. Notice that the shift parameter will be *automatically* estimated for this situation.

```
val results  = d.estimateAndEvaluateScores()
println("PDF Estimation Results for each Distribution:")
println(
 ↪  "-------------------------------------------------------")
results.resultsSortedByScoring.forEach(::println)
val topResult = results.topResultByScore
val scores = results.evaluationModel.
 ↪  alternativeScoresAsDataFrame("Distributions")
println(scores)
println()
val values = results.evaluationModel.
 ↪  alternativeValuesAsDataFrame("Distributions")
println(values)
println()
val distPlot = topResult.distributionFitPlot()
distPlot.showInBrowser("Recommended Distribution
 ↪  ${topResult.name}")
println()
println("** Recommended Distribution** ${topResult.name}")
println()
```

The following table displays the distribution fitting scores for the Bayesian Information Criterion (BIC), Anderson-Darling (AD), Cramer Von Mises (CVM), and Q-Q plot correlation criteria.

By applying the value scoring model and sorting, we can see in Figure 2.21 that the top performing distribution is the exponential distribution with an overall value of 0.9484545. Thus, the recommended distribution is: 36.83628655364795 + Exponential(mean=145.94151344635202). Notice that the shift parameter was automatically estimated for this situation.

Distributions	BIC	AD	CVM	QQC
Uniform(minimum=29.3109090909090892, maximum=789.749090909090909)	1335.989305	58.795437	11.191166	0.884991
Triangular(minimum=29.3109090909090892, mode=29.3109090909090892, maximum=789.749090909090909)	1259.789561	18.410381	3.781308	0.957060
Normal(mean=182.777799999999996, variance=20053.731154770709)	1282.615098	4.857111	0.846334	0.912257
GeneralizedBeta(min=29.3109090909090892, max=789.749090909090909, alpha=0.735615348882615 8, beta=2.909405379094531 3)	1241.735791	1.812657	0.218971	0.989492
36.8362865536479 5 + Exponential(mean=145.941513446352 02)	1201.246360	0.248596	0.025284	0.997476
36.8362865536479 5 + Lognormal(mean=250.464254486551713, variance=536239.52039695836)	1252.401833	2.692427	0.376754	0.965462
36.8362865536479 5 + Gamma(shape=0.9634544188311034, scale=148.39682516222811)	1205.833520	0.267519	0.027009	0.997418
36.8362865536479 5 + Weibull(shape=1.0037140379335936, scale=151.68525974375 24)	1205.986196	0.274876	0.035600	0.997513
36.8362865536479 5 + PearsonType5(shape=0.1434019932147736 5, scale=0.0527682990861145)	1502.380969	31.139945	6.531523	0.427737

Figure 2.20.: Scoring Model Results

Distributions	BIC	AD	CVM	QQC	Overall Value
36.836286553647 95 + Exponential(mean=145.941513446352 02)	0.900000	0.996240	0.997391	0.899948	0.948545
36.836286553647 95 + Gamma(shape=0.983454418831 1034, scale=148.396825162281 1)	0.887814	0.995954	0.997854	0.899867	0.945372
36.836286553647 95 + Weibull(shape=1.008714037355 936, scale=151.685259743752 4)	0.887408	0.995842	0.997172	0.900000	0.945106
GeneralizedBeta(min=29.310909090909092, max=789.7490909090909, alpha=0.735615348882 6538, beta=2.909405379094 5313)	0.792435	0.972583	0.982603	0.888724	0.909086
36.836286553647 95 + Lognormal(mean=250.464254486 51713, variance=536239.52033965836)	0.764089	0.969276	0.970068	0.714592	0.852009
Normal(mean=182.777799999999996, variance=20053.731154707 09)	0.683834	0.926534	0.932761	0.780267	0.830849
Triangular(minimum=29.310909090909092, mode=29.310909090909092, maximum=789.7490909090909)	0.744473	0.721535	0.699584	0.843202	0.752198
Uniform(minimum=29.310909090909092, maximum=789.7490909090909)	0.542039	0.110693	0.110888	0.742012	0.376408
36.836286553647 95 + PearsonTypeI(shape=0.143401993214 77365, scale=0.052766832990 861145)	0.100000	0.528995	0.473141	0.100000	0.300534

Figure 2.21.: Overall Scoring Results

The KSL distribution recommendation framework also evaluates each distribution based on the rank obtained by each metric. The distribution that had the most first place ranks can be considered the top performing distribution. As shown in Figure 2.22, we again see that the exponential distribution is the top distribution based on rankings.

If you are unsure of the recommended distribution or you want to better understand the sensitivity of the recommendation, the `PDFModeler` companion object provides support for bootstrapping the distribution family recommendation. Assuming that the observations are in an array called `data`, the following code will repeatedly sample from the data using a bootstrapping approach to tabulate the frequency that each possible distribution is recommended by the framework.

```
val df = PDFModeler.bootstrapFamilyFrequencyAsDataFrame(data)
println(df)
```

Running the bootstrapping family frequency tabulation process on the pharamacy model data yields the following results.

```
      Distribution Ranked First cum_count proportion cumProportion
 0      Triangular          1.0       1.0     0.0025      0.002500
 1 GeneralizedBeta          2.0       3.0     0.0050      0.007500
 2     Exponential        321.0     324.0     0.8025      0.810000
 3           Gamma         76.0     400.0     0.1900      1.000000
```

Clearly, the exponential distribution is the most likely distribution to be recommended using the average scoring criterion. This process can be used before going through the PDF modeling process to suggest distributions to consider, or, as done here, it can be performed after the distribution recommendation process to provide some confidence in the recommended distribution.

Distributions	BIC_Rank	AD_Rank	CVM_Rank	QQC_Rank	First Rank Counts	Avg Rank
36.836286553647795 + Exponential(mean=145.94151344652502)	1.0	1.0	1.0	2.0	3	1.25
36.836286553647795 + Gamma(shape=0.9834544188311034, scale=148.39692516228811)	2.0	2.0	2.0	3.0	0	2.25
36.836286553647795 + Weibull(shape=1.0037140379435936, scale=151.68525971437524)	3.0	3.0	3.0	1.0	1	2.50
GeneralizedBeta(min=29.310936936936936, max=789.7490990990991, alpha=0.7356153488826158, beta=2.9094053709945313)	4.0	4.0	4.0	4.0	0	4.00
36.836286553647795 + Lognormal(mean=250.46425446851713, variance=536239.5203995936)	5.0	5.0	5.0	8.0	0	5.75
Triangular(minimum=29.310936936936936, mode=29.310936936936936, maximum=789.7490990990991)	6.0	7.0	7.0	5.0	0	6.25
Normal(mean=182.77779999999998, variance=20053.731154707709)	7.0	6.0	6.0	6.0	0	6.25
Uniform(minimum=29.310936936936936, maximum=789.7490990990991)	8.0	9.0	9.0	7.0	0	8.25
36.836286553647795 + PearsonType5(shape=0.14340199821477365, scale=0.05278822980861145)	9.0	8.0	8.0	9.0	0	8.50

Figure 2.22.: Overall Ranking Results

After we are comfortable with the recommended distribution, we can use the following code to perform a goodness of fit analysis.

```
val gof = ContinuousCDFGoodnessOfFit(topResult.
 ↪  estimationResult.testData,
  topResult.distribution,
  numEstimatedParameters = topResult.numberOfParameters
)
println(gof)
```

The results of the goodness of fit analysis clearly indicate that we should not reject the exponential distribution. Notice that the results are for the shifted distribution.

```
GOF Results for Distribution: Exponential(mean=145.94151344635202)
-----------------------------------------------------------

Chi-Squared Test Results:
Bin Label                 P(Bin)       Observed   Expected
  1 [ 0.00, 7.49)         0.050000            3    5.00000
  2 [ 7.49,15.38)         0.050000            4    5.00000
  3 [15.38,23.72)         0.050000            8    5.00000
  4 [23.72,32.57)         0.050000            3    5.00000
  5 [32.57,41.98)         0.050000            8    5.00000
  6 [41.98,52.05)         0.050000            4    5.00000
  7 [52.05,62.87)         0.050000            5    5.00000
  8 [62.87,74.55)         0.050000            8    5.00000
  9 [74.55,87.25)         0.050000            4    5.00000
 10 [87.25,101.16)        0.050000            2    5.00000
 11 [101.16,116.54)       0.050000            9    5.00000
 12 [116.54,133.72)       0.050000            3    5.00000
 13 [133.72,153.21)       0.050000            4    5.00000
 14 [153.21,175.71)       0.050000            6    5.00000
```

```
 15 [175.71,202.32)       0.050000            3    5.00000
 16 [202.32,234.88)       0.050000            4    5.00000
 17 [234.88,276.87)       0.050000            7    5.00000
 18 [276.87,336.04)       0.050000            4    5.00000
 19 [336.04,437.20)       0.050000            5    5.00000
 20 [437.20,1596.00)      0.049982            6    4.99822   *** Warning: expected <

Number of estimated parameters = 1
Number of intervals = 20
Degrees of Freedom = 18
Chi-Squared Test Statistic = 16.000784446070348
P-value = 0.5924925929222188
Hypothesis test at 0.05 level:
The p-value = 0.5924925929222188 is >= 0.05 : Do not reject hypothesis.

Goodness of Fit Test Results:
K-S test statistic = 0.0399213095618332
K-S test p-value = 0.9954253138552503

Anderson-Darling test statistic = 0.24859556182926212
Anderson-Darling test p-value = 0.9710833730000956

Cramer-Von-Mises test statistic = 0.025283543923658194
Cramer-Von-Mises test p-value = 0.9893094979248238
```

Finally, we can review the distribution plot, Figure 2.23, and see that according to the P-P plot, Q-Q plot, histogram overlay, and empirical cumulative distribution overlay plot that the exponential distribution is an excellent probability model for this data.

All the previously illustrated analysis can be performed with a few lines of code:

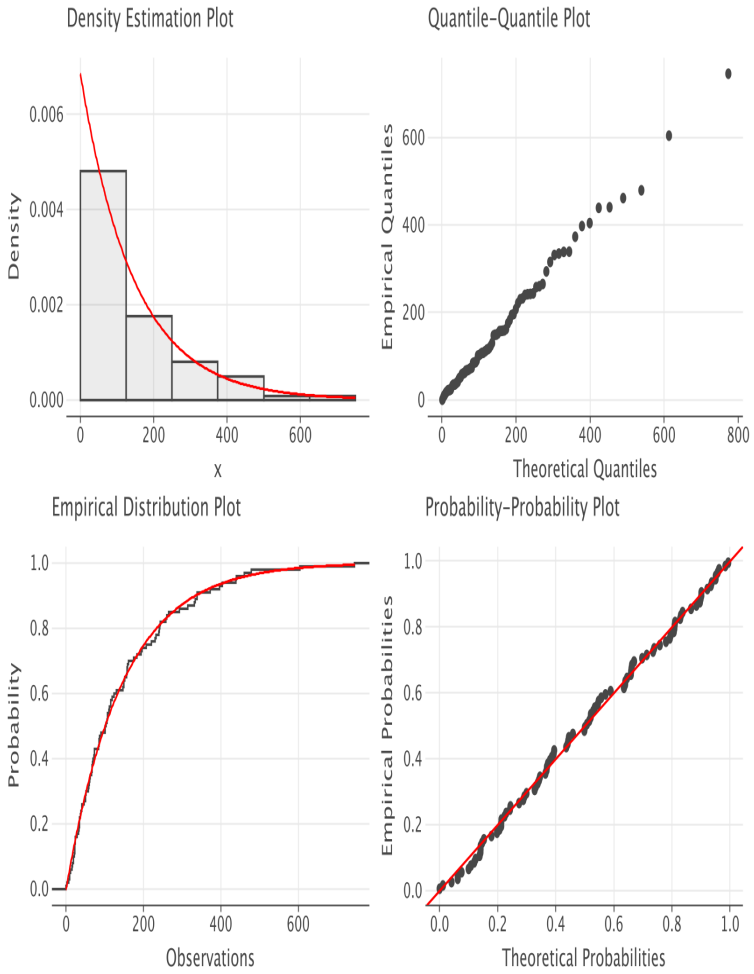


Figure 2.23.: Distribution Plot for Pharmacy Data

```
val data = KSLFileUtil.scanToArray(myFile.toPath())
val d = PDFModeler(data)
d.showAllResultsInBrowser()
```

Finally, if you want to analyze a specific distribution, then you can access the results from the returned instance of the `PDFModelingResults` class. For example, if we want to analyze the second best distribution we can access the results for the second best distribution and display its results. Assuming that the data to be analyzed is in an array called *data*, the following code illustrates how to accomplish this task.

```
val d = PDFModeler(data)
// capture the PDFModelingResults
val results  = d.estimateAndEvaluateScores()
val sortedResults: List<ScoringResult> =
 ↪  results.resultsSortedByScoring
// the array is 0 based, so the element at 1 is the 2nd best
val result: ScoringResult = sortedResults[1]
KSLFileUtil.openInBrowser(
   fileName = "PDF_Goodness_Of_Fit_Results",
   d.htmlGoodnessOfFitSummary(result)
)
```

To see the results for every distribution, just use the `showAllGoodnessOfFitSummariesInBrowser` function.

```
val d = PDFModeler(data)
val results  = d.estimateAndEvaluateScores()
d.showAllGoodnessOfFitSummariesInBrowser(results)
```

The functionally for distribution modeling within the KSL is on par with what you can find within commercial software.

2.5. Summary

The KSL contains packages that support the generation of random numbers, random variates, and the modeling of probability distributions that are commonly found in practice. These constructs facilitate the incorporation of randomness within simulation modeling. In addition, the KSL provides a framework for estimating the parameters of probability distributions and for recommending a distribution that fits a set of data.

The following new KSL constructs were discussed in this chapter:

`RNStreamIfc`: An interface that defines a stream of pseudo-random numbers, including the ability to reset the stream and to produce antithetic numbers.

`RNStreamProvider`: A class that creates and manages the random number streams used within a model.

`KSLRandom`: An object that provides functions for generating random variates from many common distributions and for randomly sampling from arrays and lists.

`RVariableIfc`: The interface implemented by all random variables, which provides the `value` property and the `sample()` function.

`NormalRV`, `ExponentialRV`, etc.: Classes that implement random variables for the common discrete and continuous distributions.

`DEmpiricalRV`: A random variable that is defined by a discrete empirical distribution.

`MixtureRV`: A random variable that is expressed as a mixture distribution. That is, a distribution that is a weighted mixture of other distributions.

`TruncatedRV`: A random variable whose distribution is truncated to a specified interval.

`ShiftedRV`: A random variable that has its domain shifted to the right.

`AcceptanceRejectionRV`: A random variable that implements the acceptance and rejection algorithm for generating random variates in a general manner.

`InverseCDFRV`: A random variable that implements the inverse transform method via bisection search of the CDF.

`RatioOfUniformsRV`: A random variable that implements the ratio of uniforms method for generating random variates.

`DistributionIfc`, `ContinuousDistributionIfc`, `DiscreteDistributionIfc`: Interfaces that define the probability distributions (e.g. computing the CDF, the PDF or PMF, and the moments) that underlie the random variables.

`ParameterEstimatorIfc`: An interface for estimating the parameters of a distribution from data.

`PDFModeler`: A class that fits and recommends continuous distributions for a set of data.

`PMFModeler`: A class that fits discrete distributions to a set of data.

`ContinuousCDFGoodnessOfFit`: A class that performs goodness of fit tests for a fitted continuous distribution.

`PoissonGoodnessOfFit`: A class that performs goodness of fit tests for a fitted Poisson distribution.

`MVSampleIfc`: An interface that defines the generation of random arrays of data.

`MVRVariableIfc`: The multi-variate analog of the `RVariableIfc` interface.

`MVRVariable`: An abstract base class for defining multi-variate random variables.

`MVIndependentMarginals`: A concrete implementation for generating independent vectors that have specified random variates for each coordinate of the vector.

The next chapter shows how to use these random variates together with the KSL's statistical collection classes to perform Monte Carlo experiments.

2.6. Exercises

Exercise 2.1. Complete the crossword puzzle found here. Turn in your completed puzzle.

Exercise 2.2. Consider the following discrete distribution of the random variable X whose probability mass function is $p(x)$.

x	0	1	2	3	4
$p(x)$	0.3	0.2	0.2	0.1	0.2

Write a KSL program to generate 4 random variates from this distribution using stream 1.

Exercise 2.3. Suppose that customers arrive at an ATM via a Poisson process with mean 7 per hour. Write a KSL program that outputs the arrival time of the first 6 customers using stream 1.

Exercise 2.4. The service time for an automated storage and retrieval system has a shifted exponential distribution. It is known that it takes a minimum of 15 seconds for any retrieval. The time beyond the minimum is exponentially distributed with a mean of 45 seconds. Write a KSL program to generate 10 observations from this distribution using stream 1.

Exercise 2.5. The time to failure for a computer printer fan has a Weibull distribution with shape parameter $\alpha = 2$ and scale parameter $\beta = 3$. Testing has indicated that the distribution is limited to the range from 1.5 to 4.5. Write a KSL program to generate 10 observations from this distribution using stream 1.

Exercise 2.6. The interest rate for a capital project is unknown. An accountant has estimated that the minimum interest rate will between 2% and 5% within the next year. The accountant believes that any interest rate in this range is equally likely. You are tasked with generating interest rates for a cash flow analysis of the project. Write a KSL program to generate 10 observations from this distribution using stream 1.

Exercise 2.7. Consider the following probability density function:

$$f(x) = \begin{cases} \dfrac{3x^2}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Write a KSL program to generate 10 observations from this distribution using stream 1 using the inverse transform technique.

Exercise 2.8. Consider the following probability density function:

$$f(x) = \begin{cases} 0.5x - 1 & 2 \leq x \leq 4 \\ 0 & \text{otherwise} \end{cases}$$

Write a KSL program to generate 10 observations from this distribution using stream 1 using the inverse transform technique.

Exercise 2.9. Consider the following probability density function:

$$f(x) = \begin{cases} \dfrac{2x}{25} & 0 \leq x \leq 5 \\ 0 & \text{otherwise} \end{cases}$$

Write a KSL program to generate 10 observations from this distribution using stream 1 using the inverse transform technique.

Exercise 2.10. Consider the following probability density function:

$$f(x) = \begin{cases} \dfrac{2}{x^3} & x > 1 \\ 0 & x \leq 1 \end{cases}$$

Write a KSL program to generate 10 observations from this distribution using stream 1 using the inverse transform technique.

Exercise 2.11. The times to failure for an automated production process have been found to be randomly distributed according to a Rayleigh distribution:

$$f(x) = \begin{cases} 2\beta^{-2} x e^{(-(x/\beta)^2)} & x > 0 \\ 0 & \text{otherwise} \end{cases}$$

Write a KSL program to generate 10 observations from this distribution with $\beta = 2.0$ using stream 1 and the inverse transform technique.

Exercise 2.12. Suppose that the processing time for a job consists of two distributions. There is a 30% chance that the processing time is lognormally distributed with a mean of 20 minutes and a standard deviation of 2 minutes, and a 70% chance that the time is uniformly distributed between 10 and 20 minutes.

Write a KSL program to generate 10 observations from this distribution using stream 1.

Exercise 2.13. Suppose that the service time for a patient consists of two distributions. There is a 25% chance that the service time is uniformly distributed with minimum of 20 minutes and a maximum of 25 minutes,

and a 75% chance that the time is distributed according to a Weibull distribution with shape of 2 and a scale of 4.5.

Write a KSL program to generate 10 observations from this distribution using stream 1.

Exercise 2.14. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{3x^2}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Write a KSL program to generate 10 observations from this distribution using the acceptance-rejection technique. Use stream 1 in your implementation.

Exercise 2.15. Consider the following function:

$$f(x) = \begin{cases} cx^2 & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

a. Determine the value of c that will turn $f(x)$ into a probability density function. The resulting probability density function is called a parabolic distribution.
b. Denote the probability density function found in part (a), $f(x)$. Let X be a random variable from $f(x)$. Derive the inverse cumulative distribution function for $f(x)$.

c. Write a KSL program to generate 10 observations from this distribution over the range of $a = 4$ and $b = 12$ using your work from part (a) and (b). Use stream 1 in your implementation.

Exercise 2.16. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{3(c-x)^2}{c^3} & 0 \leq x \leq c \\ 0 & \text{otherwise} \end{cases}$$

Derive an inverse cumulative distribution algorithm for generating from $f(x)$. Write a KSL program to generate 10 observations from this distribution for $c = 5$. Use stream 1 in your implementation.

---

3. Monte Carlo Methods

Learning Objectives

- To be able to collect statistics using classes within the KSL
- To be able to perform the basic statistical computations supported by the KSL
- To be able to perform simple Monte Carlo simulation experiments using the KSL
- To be able to apply basic statistical concepts to the analysis of simple Monte Carlo simulation experiments
- To be able to generate and collect statistics for Monte Carlo simulation experiments

This chapter illustrates how to use the KSL for simple Monte-Carlo simulation experiments. The term Monte Carlo generally refers to the set of methods and techniques that estimate quantities by repeatedly sampling from models/equations represented in a computer. As such, this terminology is somewhat synonymous with computer simulation itself. The term Monte Carlo gets its origin from the Monte Carlo casino in the Principality of Monaco, where gambling and games of chance are well known. There is no one Monte Carlo method. Rather there is a collection of algorithms and techniques. In fact, the ideas of random number generation and random variate generation previously discussed form the foundation of Monte Carlo methods.

For the purposes of this chapter, we limit the term Monte Carlo methods to those techniques for generating and estimating the expected values of random variables, especially in regards to static simulation. In static

simulation, the notion of time is relatively straightforward with respect to system dynamics. For a static simulation, time 'ticks' in a regular pattern and at each 'tick' the state of the system changes (new observations are produced).

A key requirement in performing a Monte Carlo simulation is the ability to collect and report statistics on observations generated by the simulation. The KSL supports the collection of statistics within the `ksl.utilities.statistics` package. Before proceeding with examples illustrating Monte Carlo methods, the following section overviews some of the statistical and reporting capabilities of the KSL.

> **Note**
>
> This chapter provides a series of example Kotlin code that illustrates the use of KSL constructs for Monte Carlo methods. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

3.1. Collecting Statistics

The KSL has a wide variety of classes that support statistic computations. A main theme in understanding the usage of the classes within the `ksl.utilities.statistics` package is the concept of collection. This concept is encapsulated within the `CollectorIfc` interface. The methods of the `CollectorIfc` interface are illustrated in Figure 3.1.

Something is a collector, if it implements the `CollectorIfc` interface. The implication is that those values presented to the various `collect` methods will be observed and tabulated into various quantities based on the presented values. The `collect` method has been overloaded to facilitate collection of double values, arrays of double values, and boolean values.

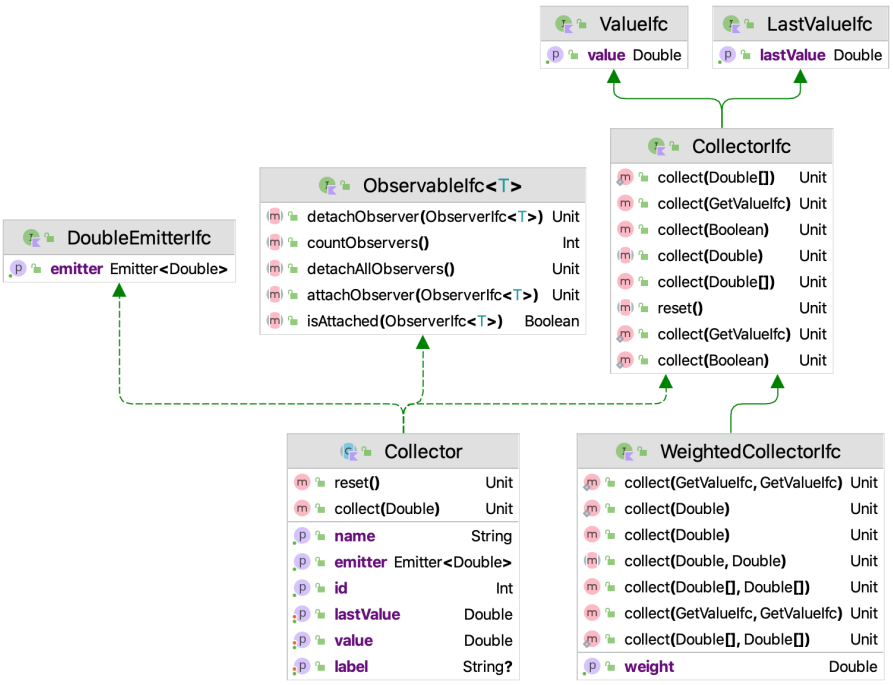


Figure 3.1.: CollectorIfc Interface

A collector can be reset. Resetting a collector should set the state of the collector as if no values had been observed. Thus, resetting a collector should clear all previous collection results. The collector may or may not store any of the observations related to the collection process. By default, the standard approach taken is not to store the observations, but rather to summarize the observations in some manner. To facilitate the further use of the observed values, the base class, `Collector` is observable and will also emit observed values. Thus, other classes can be connected to a base observation process.

Figure 3.2 presents the major classes and interfaces within the statistics package. The `CollectIfc` interface is implemented within the abstract base class `Collector`, which serves as the basis for various concrete implementations of statistical collectors. There are two major kinds of statistics, one of which assumes that the values presented must be weighted, the `WeightedStatisticIfc` interface and the `WeightedStatistic` class. The other branch of classes, derived from `AbstractStatistic` do not necessarily have to be weighted. The main classes to be discussed in this chapter are `Statistic` and `Histogram`.

3.1.1. Creating and Using a Statistic

The `Statistic` class has a great deal of functionality. It accumulates summary statistics on the values presented to it via its `collect` methods. Recall also that since the `Statistic` class implements the `CollectorIfc` interface, you can use the `reset()` method to clear all accumulated statistics and reuse the `Statistic` instance. The major statistical quantities are found in the `StatisticIfc` interface.

As can be seen in Figure 3.3, the `Statistic` class not only computes the standard statistical quantities such as the count, average, and variance, it also has functionality to compute confidence intervals, skewness, kurtosis, the minimum, the maximum, and lag 1 covariance and correlation. The

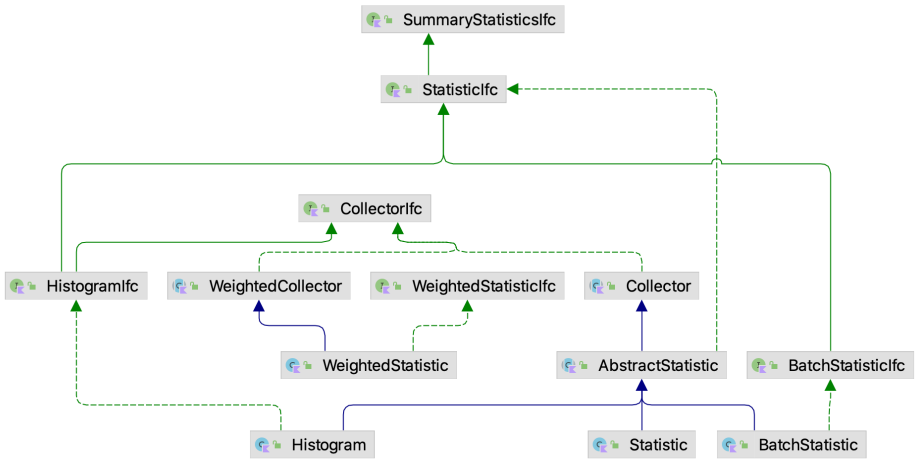


Figure 3.2.: Major Classes and Interfaces in the Statistics Package

computed confidence intervals are based on the assumption that the observed data is normally distributed or that the sample size is large enough to justify using the central limit theorem to assume that the sampling distribution is normal. Thus, we can assume that the confidence intervals are approximate. The summary statistics are computed via efficient one pass algorithms that do not require any observed data to be stored. The algorithms are designed to minimize issues related to numerical precision within the calculated results. The `toString()` method of the `Statistic` class has been overridden to contain all of the computed values. Let's illustrate the usage of the `Statistic` class with some code. In this code, first we create a normal random variable to be able to generate some data. Then, two statistics are created. The first statistic directly collects the generated values. The second statistic is designed to collect $P(X \geq 20.0)$ by observing whether the generated value meets this criteria as defined by the boolean expression `x >= 20.0`.

Example 3.1 (Creating and Using Statistics)**.** This example illustrates how

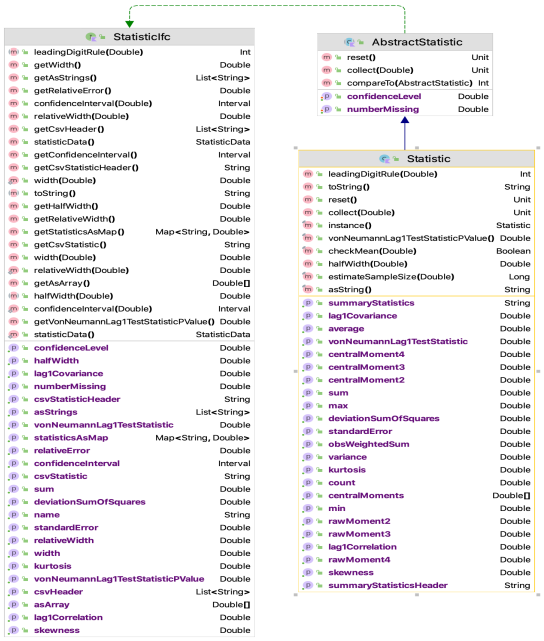


Figure 3.3.: Major Accumulated Statistical Quantities

to create instances of the `Statistic` class to collect randomly generated observations from a normal distribution.

```
fun main() {
    // create a normal mean = 20.0, variance = 4.0 random
     ↪  variable
    val n = NormalRV(20.0, 4.0, streamNum = 3)
    // create a Statistic to observe the values
    val stat = Statistic("Normal Stats")
    val pGT20 = Statistic("P(X>=20")
    // generate 100 values
    for (i in 1..100) {
        // getValue() method returns generated values
        val x = n.value
        stat.collect(x)
        pGT20.collect(x >= 20.0)
    }
    println(stat)
    println(pGT20)
    val reporter = StatisticReporter(mutableListOf(stat,
     ↪  pGT20))
    println(reporter.halfWidthSummaryReport())
}
```

The results for the statistics collected directly on the observations from the `toString()` method are as follows.

```
ID 1
Name Normal Stats
Number 100.0
Average 20.370190128861807
Standard Deviation 2.111292233346322
Standard Error 0.2111292233346322
```

```
Half-width 0.4189261806189412
Confidence Level 0.95
Confidence Interval [19.951263948242865, 20.78911630948075]
Minimum 15.020744984423821
Maximum 25.33588436770212
Sum 2037.0190128861807
Variance 4.457554894588499
Weighted Average 20.370190128861797
Weighted Sum 2037.0190128861796
Sum of Weights 100.0
Weighted Sum of Squares 41935.76252316213
Deviation Sum of Squares 441.2979345642614
Last value collected 21.110736402119805
Last weighted collected 1.0
Kurtosis -0.534855387072145
Skewness 0.20030433873223502
Lag 1 Covariance -0.973414579833684
Lag 1 Correlation -0.22057990840016864
Von Neumann Lag 1 Test Statistic -2.2136062395518343
Number of missing observations 0.0
Lead-Digit Rule(1) -1
```

Of course, this is probably more output than what you need, but you can use the properties and methods illustrated in Figure 3.3 to access specific desired quantities.

Notice that in the code example that the $P(X \geq 20.0)$ is also collected. This is done by using the boolean expression `x >= 20.0` within the `collect()` method. This expression evaluates to either true or false. The true values are presented as 1.0 and the false values as 0.0. Thus, this expression acts as an indicator variable and facilitates the estimation of probabilities. The results from the statistics can be pretty printed by using the `StatisticReporter` class, which takes a list of objects that implement the

`StatisticIfc` interface and facilitates the writing and printing of various statistical summary reports.

```
    val reporter = StatisticReporter(mutableListOf(stat,
     ↪  pGT20))
    println(reporter.halfWidthSummaryReport())
```

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                                                    Count          Average      Half-
Width
-------------------------------------------------------------------------------------------
Normal Stats                                              100          20.3702           0
P(X>=20                                                   100           0.5100           0
-------------------------------------------------------------------------------------------
```

The `KSLArrays` object has a number of very useful methods that work on arrays and compute various statistical quantities.

- `indexOfMin(x: DoubleArray): Int`: returns the index of the element that is smallest. If there are ties, the first found is returned.
- `min(x: DoubleArray) : Double`: returns the element that is smallest. If there are ties, the first found is returned.
- `indexOfMax(x: DoubleArray) : Int`: returns the index of the element that is largest If there are ties, the first found is returned.
- `max(x: DoubleArray) : Double`: returns the element that is largest. If there are ties, the first found is returned.
- `countLessEqualTo(data: DoubleArray, x: Double) : Int`: returns the count of the elements that are less than or equal to x
- `countLessThan(data: DoubleArray, x: Double) : Int`: returns the count of the elements that are less than x
- `countGreaterEqualTo(data: DoubleArray, x: Double) : Int`: returns the count of the elements that are greater than or equal to x

- `countGreaterThan(data: DoubleArray, x: Double) : Int`: returns the count of the elements that are greater than x
- `orderStatistics(data: DoubleArray) : DoubleArray`: returns a sorted copy of the supplied array ordered from smallest to largest

These functions have also been implemented as extension functions of the `DoubleArray` class. The `Statistic` class's companion object also implements a few useful statistical summary functions.

- `estimateSampleSize(desiredHW: Double, stdDev: Double, level: Double) : Long`: returns the approximate sample size necessary to reach the desired half-width at the specified confidence level given the estimate of the sample standard deviation.
- `median(data: DoubleArray) : Double`: the statistical median of the supplied array
- `percentile(data: DoubleArray, p: Double) : Double`: the p^{th} percentile of the data
- `quantile(data: DoubleArray, q: Double) : Double`: the q^{th} percentile of the data based on definition 7 of quantiles as per the R definitions.

As one can see, there is much functionality available for the collection and reporting of summary statistics. In the next section, we present an overview of distribution summaries involving histograms and frequencies.

3.1.2. Histograms and Frequencies

A histogram tabulates counts and frequencies of observed data over a set of contiguous intervals. Let $b_0, b_1, \cdots, b_k$ be the breakpoints (end points) of the class intervals such that $(b_0, b_1], (b_1, b_2], \cdots, (b_{k-1}, b_k]$ form k disjoint and adjacent intervals. The intervals do not have to be of equal width. Also, b_0 can be equal to $-\infty$ resulting in interval $(-\infty, b_1]$ and b_k can be equal to $+\infty$ resulting in interval $(b_{k-1}, +\infty)$. Define $\Delta b_j = b_j - b_{j-1}$ and if all the intervals have the same width (except perhaps for the end

intervals), $\Delta b = \Delta b_j$. To count the number of observations that fall in each interval, we can use the count function:

$$c(\vec{x} \leq b) = \#\{x_i \leq b\}\ i = 1, \ldots, n$$

$c(\vec{x} \leq b)$ counts the number of observations less than or equal to x. Let c_j be the observed count of the x values contained in the j^{th} interval $\left(b_{j-1}, b_j\right]$. Then, we can determine c_j via the following equation:

$$c_j = c(\vec{x} \leq b_j) - c(\vec{x} \leq b_{j-1})$$

The key parameters of a histogram are the break points associated with the bins. Since knowledge of the data is often necessary to specify good break points, the `Histogram` class's companion object provides a number of methods for creating break points and recommending break points based on sample data. For example, the function, `createBreakPoints` which has input parameters:

- The first bin lower limit (b_0): This is the starting point of the range over which the data will be tabulated.
- The number of bins (k))
- The width of the bins, (Δb)

Figure 3.4 presents the methods of the `Histogram` class. The `Histogram` class is utilized in a very similar manner as the `Statistic` class by collecting observations. The observations are then tabulated into the bins. The `Histogram` class allows the user to tabulate the bin contents via the collect() methods inherited from the `AbstractStatistic` base class. Since data may fall below the first bin and after the last bin, the implementation also provides counts for those occurrences. Since a `Histogram` is a sub-class of `AbstractStatistic`, it also implements the `StatisticIfc` to provide summary statistics on the data tabulated within the bins.

In some cases, the client may not know in advance the appropriate settings for the number of bins or the width of the bins. In this situation, one can use the `recommendBreakPoints` function, which uses an array of data

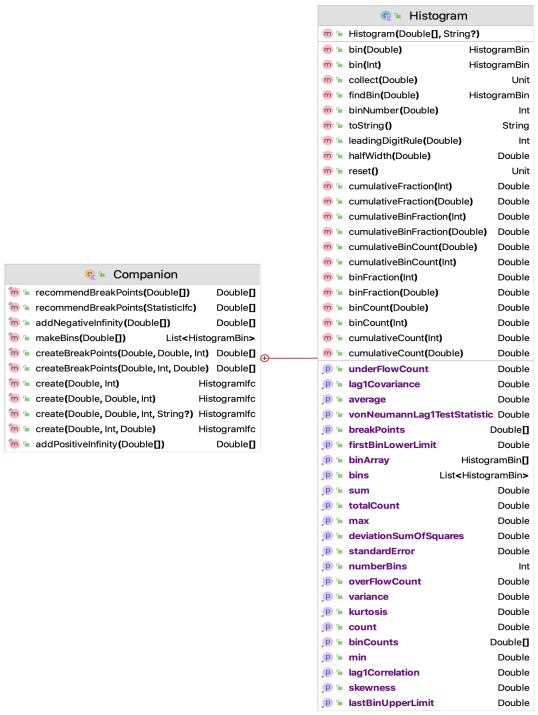


Figure 3.4.: Histogram Class

or statistics computed on the data. The function `recommendBreakPoints` computes a reasonable lower limit, number of bins, and bin width based on the statistics.

Example 3.2 (Histograms and Breakpoints)**.** This example illustrates how to configure the breakpoints for a histogram and to use the histogram to observe observations from a random sample.

```
fun main() {
    val d = ExponentialRV(2.0, streamNum = 1)
    val data = d.sample(1000)
    var bp = Histogram.recommendBreakPoints(data)
    bp = Histogram.addPositiveInfinity(bp)
    val h = Histogram(breakPoints = bp)
    for (x in data) {
        h.collect(x)
    }
    println(h)
    val plot = h.histogramPlot()
    plot.showInBrowser("Exponentially Distributed Data")
}
```

```
Histogram: ID_4
---------------------------------------
Number of bins = 25
First bin starts at = 0.0
Last bin ends at = Infinity
Under flow count = 0.0
Over flow count = 0.0
Total bin count = 1000.0
Total count = 1000.0
---------------------------------------
Bin Range        Count CumTot Frac  CumFrac
```

```
  1 [ 0.00, 0.75)  = 333 333.0 0.333000 0.333000
  2 [ 0.75, 1.50)  = 198 531.0 0.198000 0.531000
  3 [ 1.50, 2.25)  = 150 681.0 0.150000 0.681000
  4 [ 2.25, 3.00)  = 112 793.0 0.112000 0.793000
  5 [ 3.00, 3.75)  = 75 868.0 0.075000 0.868000
  6 [ 3.75, 4.50)  = 48 916.0 0.048000 0.916000
  7 [ 4.50, 5.25)  = 28 944.0 0.028000 0.944000
  8 [ 5.25, 6.00)  = 19 963.0 0.019000 0.963000
  9 [ 6.00, 6.75)  = 16 979.0 0.016000 0.979000
 10 [ 6.75, 7.50)  = 1 980.0 0.001000 0.980000
 11 [ 7.50, 8.25)  = 5 985.0 0.005000 0.985000
 12 [ 8.25, 9.00)  = 2 987.0 0.002000 0.987000
 13 [ 9.00, 9.75)  = 4 991.0 0.004000 0.991000
 14 [ 9.75,10.50)  = 1 992.0 0.001000 0.992000
 15 [10.50,11.25)  = 2 994.0 0.002000 0.994000
 16 [11.25,12.00)  = 1 995.0 0.001000 0.995000
 17 [12.00,12.75)  = 3 998.0 0.003000 0.998000
 18 [12.75,13.50)  = 0 998.0 0.000000 0.998000
 19 [13.50,14.25)  = 0 998.0 0.000000 0.998000
 20 [14.25,15.00)  = 0 998.0 0.000000 0.998000
 21 [15.00,15.75)  = 0 998.0 0.000000 0.998000
 22 [15.75,16.50)  = 1 999.0 0.001000 0.999000
 23 [16.50,17.25)  = 0 999.0 0.000000 0.999000
 24 [17.25,18.00)  = 1 1000.0 0.001000 1.000000
 25 [18.00,Infinity)  = 0 1000.0 0.000000 1.000000
-------------------------------------
Statistics on data collected within bins:
-------------------------------------
ID 5
Name ID_4_Statistics
Number 1000.0
Average 1.9216605939547684
Standard Deviation 1.968265908806584
Standard Error 0.06224203312690073
```

```
Half-width 0.12214010773960152
Confidence Level 0.95
Confidence Interval [1.7995204862151668, 2.04380070169437]
Minimum 9.561427092951219E-4
Maximum 17.40419059595123
Sum 1921.6605939547685
Variance 3.8740706877702076
Deviation Sum of Squares 3870.1966170824376
Last value collected 8.548639846481285
Kurtosis 10.67406304701379
Skewness 2.5031667132444806
Lag 1 Covariance -0.12649071302321163
Lag 1 Correlation -0.032683278277103954
Von Neumann Lag 1 Test Statistic -0.8438384394483673
Number of missing observations 0.0
Lead-Digit Rule(1) -2
-------------------------------------
```

Because it may be challenging to specify the breakpoints for a histogram, the KSL also provides the `CachedHistogram` class, which stores the observed data within a data cache (array) until sufficient data has been observed to provide reasonable breakpoints. The default size of the cache is 512 observations. If less than the cache size is observed, the KSL attempts to find the best breakpoints based on the recommendations found here. The following code illustrates the used of the `CachedHistogram` class.

```
fun main() {
    val d = ExponentialRV(2.0, streamNum = 1)
    val data = d.sample(1000)
    val ch = CachedHistogram()
    for (x in data) {
        ch.collect(x)
    }
```

```
    println(ch)
    val plot = ch.histogramPlot()
    plot.showInBrowser("Exponentially Distributed Data")
}
```

The KSL will also tabulate count frequencies when the values are only integers. This is accomplished with the `IntegerFrequency` class. Figure 3.5 indicates the methods of the `IntegerFrequency` class. The object can return information on the counts and proportions. It can even create a `DEmpiricalCDF` distribution based on the observed data.

In the following code example, an instance of the `IntegerFrequency` class is created. Then, an instance of a binomial random variable is used to generate a sample of 10,000 observations. The sample is then collected by the `IntegerFrequency` class's `collect()` function. Notice the `collect()` function can collect observations contained within an array.

Example 3.3 (Integer Frequency Tabulation)**.** In this example, an instance of the `IntegerFrequency` class is used to tabulate observations from a binomial random variable. A bar chart representation of the frequency is also shown within a browser window.

```
fun main() {
    val f = IntegerFrequency(name = "Frequency Demo")
    val bn = BinomialRV(0.5, 100, streamNum = 2)
    val sample = bn.sample(10000)
    f.collect(sample)
    println(f)
    val plot = f.frequencyPlot()
    plot.showInBrowser("Frequency Demo Plot")
}
```

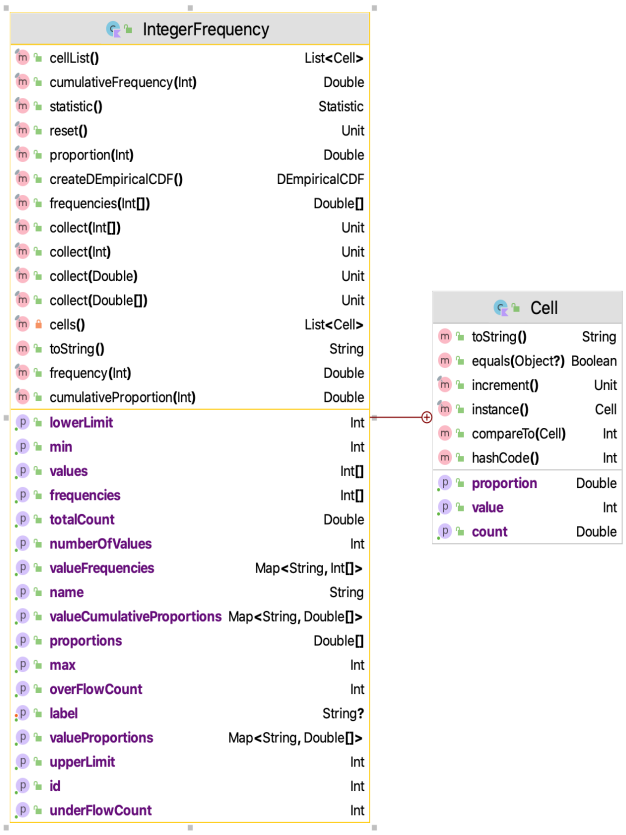


Figure 3.5.: IntegerFrequency Class

As can be noted in the output, only those integers that are actually observed are tabulated in terms of the count of the number of times the integer is observed and its proportion. The user does not have to specify the range of possible integers; however, instances of `IntegerFrequency` can be created that specify a lower and upper limit on the tabulated values. The overflow and underflow counts then tabulate when observations fall outside of the specified range.

```
Frequency Tabulation Frequency Demo
---------------------------------------
Number of cells = 37
Lower limit = -2147483648
Upper limit = 2147483647
Under flow count = 0
Over flow count = 0
Total count = 10000.0
Minimum value observed = 30
Maximum value observed = 69
---------------------------------------
Value    Count   Proportion
30   1.0     1.0E-4
33   3.0     3.0E-4
34   7.0     7.0E-4
35   10.0    0.001
36   17.0    0.0017
37   23.0    0.0023
38   48.0    0.0048
39   86.0    0.0086
40   101.0       0.0101
41   146.0       0.0146
42   221.0       0.0221
43   319.0       0.0319
44   371.0       0.0371
```

```
45   470.0       0.047
46   568.0       0.0568
47   684.0       0.0684
48   742.0       0.0742
49   768.0       0.0768
50   769.0       0.0769
51   813.0       0.0813
52   696.0       0.0696
53   680.0       0.068
54   599.0       0.0599
55   505.0       0.0505
56   386.0       0.0386
57   318.0       0.0318
58   237.0       0.0237
59   142.0       0.0142
60   107.0       0.0107
61   68.0    0.0068
62   38.0    0.0038
63   23.0    0.0023
64   13.0    0.0013
65   14.0    0.0014
66   4.0     4.0E-4
67   2.0     2.0E-4
69   1.0     1.0E-4
----------------------------------------
```

Finally, the KSL provides the ability to define labeled states and to tabulate frequencies and proportions related to the visitation and transition between the states. This functionality is available in the `StateFrequency` class. The following code example creates an instance of `StateFrequency` by providing the number of states. The states are returned in a `List` and then 10,000 states are randomly selected from the list with equal probability using the `KSLRandom` functionality to randomly select from lists. The

randomly selected state is then observed via the `collect()` method. State frequency tabulation can be useful within the context of a Markov Chain analysis.

Example 3.4 (State Frequency Tabulation)**.** This example creates six states via the constructor parameter to the `StateFrequency` class. Then, the `KSLRandom` functionality to randomly select from a list is used to generate 10000 observations of the next state. Finally, a bar chart representation of the state frequency is also shown within a browser window.

```
fun main() {
    val sf = StateFrequency(6)
    val states = sf.states
    for (i in 1..10000) {
        val state = KSLRandom.randomlySelect(states)
        sf.collect(state)
    }
    println(sf)
    val plot = sf.frequencyPlot(proportions = true)
    plot.showInBrowser("State Frequency Demo Plot")
}
```

The output is what you would expect based on selecting the states with equal probability. Notice that the `StateFrequency` class not only tabulates the visits to the states, similar to `IntegerFrequency`, it also counts and tabulates the transitions between states. These detailed tabulations are available via the various methods of the class. See the documentation for further details.

```
State Frequency Tabulation for: Identity#1
State Labels
State{id=1, number=0, name='State:0'}
State{id=2, number=1, name='State:1'}
```

```
State{id=3, number=2, name='State:2'}
State{id=4, number=3, name='State:3'}
State{id=5, number=4, name='State:4'}
State{id=6, number=5, name='State:5'}
State transition counts
[288, 272, 264, 282, 265, 286]
[283, 278, 283, 286, 296, 266]
[286, 298, 263, 264, 247, 282]
[271, 263, 275, 279, 280, 294]
[274, 305, 273, 281, 296, 268]
[254, 277, 282, 270, 313, 255]
State transition proportions
[0.17380808690404345, 0.16415208207604104, 0.15932407966203982, 0.17018708509354255,
[0.16725768321513002, 0.16430260047281323, 0.16725768321513002, 0.1690307328605201,
[0.174390243902439, 0.18170731707317073, 0.1603658536585366, 0.16097560975609757, 0.
[0.16305655836341756, 0.1582430806257521, 0.1654632972322503, 0.16787003610108303, 0
[0.1614614024749558, 0.17972893341190335, 0.16087212728344136, 0.16558632881555688,
[0.15384615384615385, 0.16777710478497881, 0.17080557238037553, 0.16353725015142337,

Frequency Tabulation Identity#1
---------------------------------------
Number of cells = 6
Lower limit = 0
Upper limit = 5
Under flow count = 0
Over flow count = 0
Total count = 10000
---------------------------------------
Value    Count   Proportion
0    1657    0.1657
1    1693    0.1693
2    1640    0.164
3    1662    0.1662
4    1697    0.1697
```

```
5    1651    0.1651
----------------------------------------
```

3.1.3. Batch Statistics

In simulation, we often collect data that is correlated. That is, the data has a dependence structure. This causes difficulty in developing valid confidence intervals for the estimators as well as invalidated a number of other statistical procedures that require independent observations. Grouping the data into batches and computing the average of each batch is one methodology for mitigating the effect of dependence within the data on statistical inference procedures. The idea is that the average associated with each batch will tend to be less dependent, especially the larger the batch size. The method of batch means provides a mechanism for developing an estimator for $Var[\bar{X}]$.

The method of batch means is based on observations $(X_1, X_2, X_3, \dots, X_n)$. The idea is to group the output into batches of size, b, such that the averages of the data within a batch are more nearly independent and possibly normally distributed.

$$\underbrace{X_1, X_2, \dots, X_b}_{batch1} \cdots \underbrace{X_{b+1}, X_{b+2}, \dots, X_{2b}}_{batch2} \cdots$$
$$\underbrace{X_{(j-1)b+1}, X_{(j-1)b+2}, \dots, X_{jb}}_{batchj} \cdots \underbrace{X_{(k-1)b+1}, X_{(k-1)b+2}, \dots, X_{kb}}_{batchk}$$

Let k be the number of batches each of size b, where, $b = \lfloor \frac{n}{k} \rfloor$. Define the j^{th} batch mean (average) as:

$$\bar{X}_j(b) = \frac{1}{b} \sum_{i=1}^{b} X_{(j-1)b+i}$$

Each of the batch means are treated like observations in the batch means series. For example, if the batch means are re-labeled as $Y_j = \bar{X}_j(b)$, the batching process simply produces another series of data, $(Y_1, Y_2, Y_3, \ldots, Y_k)$ which may be more like a random sample. Why should they be more independent? Typically, in auto-correlated processes the lag-k auto-correlations decay rapidly as k increases. Since, the batch means are formed from batches of size b, provided that b is large enough the data within a batch is *conceptually* far from the data in other batches. Thus, larger batch sizes are good for ensuring independence; however, as the batch size increases the number of batches decreases and thus variance of the estimator will increase.

To form a $(1-\alpha)\%$ confidence interval, we simply treat this new series like a random sample and compute approximate confidence intervals using the sample average and sample variance of the batch means series:

$$\bar{Y}(k) = \frac{1}{k}\sum_{j=1}^{k} Y_j$$

The sample variance of the batch process is based on the k batches:

$$S_b^2(k) = \frac{1}{k-1}\sum_{j=1}^{k}(Y_j - \bar{Y}^2)$$

Finally, if the batch process can be considered independent and identically distributed the $1-\alpha$ level confidence interval can be written as follows:

$$\bar{Y}(k) \pm t_{\alpha/2,k-1}\frac{S_b(k)}{\sqrt{k}}$$

The `BatchStatistic` class within the statistic package implements a basic batching process. The `BatchStatistic` class works with data as it is presented to its collect method. Since we do not know in advance how much data we have, the `BatchStatistic` class has rules about the minimum number of batches and the size of batches that can be formed. Theory

indicates that we do not need to have a large number of batches and that it is better to have a relatively small number of batches that are large in size.

Three properties of the `BatchStatistic` class that are important are:

- `minNumBatches`: This represents the minimum number of batches required. The default value for this attribute is determined by `BatchStatistic.MIN_NUM_BATCHES`, which is set to 20.
- `minBatchSize`: This represents the minimum size for forming initial batches. The default value for this attribute is determined by `BatchStatistic.MIN_NUM_OBS_PER_BATCH`, which is set to 16.
- `maxNumBatchesMultiple`: This represents a multiple of minimum number of batches which is used to determine the upper limit (maximum) number of batches. For example, if `maxNumBatchesMultiple = 2` and the `minNumBatches = 20`, then the maximum number of batches we can have is 40 (2*20). The default value for this property is determined by `BatchStatistic.MAX_BATCH_MULTIPLE`, which is set to 2.

The `BatchStatistic` class uses instances of the `Statistic` class to do its calculations. The bulk of the processing is done in two methods, `collect()` and `collectBatch()`. The `collect()` method simply uses an instance of the `Statistic` class (`myStatistic`) to collect statistics. When the amount of data collected (`myStatistic.count`) equals the current batch size (`currentBatchSize`) then the `collectBatch()` method is called to form a batch.

```
    override fun collect(obs: Double) {
        super.collect(obs)
        myTotNumObs = myTotNumObs + 1.0
        myValue = obs
        myStatistic.collect(myValue)
        if (myStatistic.count == currentBatchSize.toDouble())
         ↪ {
```

```
            collectBatch()
        }
    }
```

Referring to the collectBatch() method in the following code, the batches that are formed are recorded in an array called `bm`. After recording the batch average, the statistic is reset for collecting the next batch of data. The number of batches is recorded and if this has reached the maximum number of batches (as determined by the batch multiple calculation), we rebatch the batches back down to the minimum number of batches by combining adjacent batches according to the batch multiple.

```
    private fun collectBatch() {
        // increment the current number of batches
        numBatches = numBatches + 1
        // record the average of the batch
        bm[numBatches] = myStatistic.average
        // collect running statistics on the batches
        myBMStatistic.collect(bm[numBatches])
        // reset the within batch statistic for next batch
        myStatistic.reset()
        // if the number of batches has reached the maximum
        ↪  then rebatch down to
        // min number of batches
        if (numBatches == maxNumBatches) {
            numRebatches++
            currentBatchSize = currentBatchSize *
↪   minNumBatchesMultiple
            var j = 0 // within batch counter
            var k = 0 // batch counter
            myBMStatistic.reset() // clear for collection
↪   across new batches
            // loop through all the batches
```

```
            for (i in 1..numBatches) {
                myStatistic.collect(bm[i]) // collect across
↪  batches old batches
                j++
                if (j == minNumBatchesMultiple) { // have
                 ↪  enough for a batch
                    //collect new batch average

↪  myBMStatistic.collect(myStatistic.average)
                    k++ //count the batches
                    bm[k] = myStatistic.average // save the
↪  new batch average
                    myStatistic.reset() // reset for next
↪  batch
                    j = 0
                }
            }
            numBatches = k // k should be minNumBatches
            myStatistic.reset() //reset for use with new data
        }
    }
```

There are a variety of procedures that have been developed that will automatically batch the data as it is collected. The KSL has a batching algorithm based on the procedure implemented within the Arena simulation language. When a sufficient amount of data has been collected batches are formed. As more data is collected, additional batches are formed until $k = 40$ batches are collected. When 40 batches are formed, the algorithm collapses the number of batches back to 20, by averaging each pair of batches. This has the net effect of doubling the batch size. This process is repeated as more data is collected, thereby ensuring that the number of batches is between 20 and 39. In addition, the procedure also computes the lag-1 correlation so that independence of the batches can be tested.

The `BatchStatistic` class also provides a public `reformBatches()` method to allow the user to rebatch the batches to a user supplied number of batches. Since the `BatchStatistic` class implements the `StatisticalAccessorIfc` interface, it can return the sample average, sample variance, minimum, maximum, etc. of the batches. Within the discrete-event modeling constructs of the KSL, batching can be turned on to collect batch statistics during a replication. The use of these constructs will be discussed when the discrete-event modeling elements of the KSL are presented.

The following code illustrates how to create and use a `BatchStatistic`.

Example 3.5 (Batch Statistic Collection)**.** This example code illustrates how to configure an instance of the `BatchStatistic` class to collect and report statistics associated with the batching process.

```
fun main() {
    val d = ExponentialRV(2.0)

    // number of observations
    val n = 1000

    // minimum number of batches permitted
    // there will not be less than this number of batches
    val minNumBatches = 40

    // minimum batch size permitted
    // the batch size can be no smaller than this amount
    val minBatchSize = 25

    // maximum number of batch multiple
    //  The multiple of the minimum number of batches
    //  that determines the maximum number of batches
    //  e.g. if the min. number of batches is 20
```

```
    //  and the max number batches multiple is 2,
    //  then we can have at most 40 batches
    val maxNBMultiple = 2

    // In this example, since 40*25 = 1000, the batch
     ↪  multiple does not matter
    val bm = BatchStatistic(minNumBatches, minBatchSize,
     ↪  maxNBMultiple)
    for (i in 1..n) {
        bm.collect(d.value)
    }
    println(bm)
    val bma = bm.batchMeans
    var i = 0
    for (x in bma) {
        println("bm($i) = $x")
        i++
    }

    // this re-batches the 40 down to 10
    val reformed = bm.reformBatches(10)
    println(Statistic(reformed))
}
```

3.1.4. Statistics Summary

The `ksl.utilities.statistic` package defines a lot of functionality. Here is a summary of some of the useful classes and interfaces.

1. `CollectorIfc` defines a set of collect() methods for collecting data. The method is overridden to permit the collection of a wide variety of data types. The collect() method is designed to collect values and a weight associated with the value. This allows the collection of

weighted statistics. `Collector` is an abstract base class for building concrete sub-classes.

2. `DoubleArraySaver` defines methods for saving observed data to arrays.
3. `WeightedStatisticIfc` defines statistics that are computed on weighted data values. `WeightedStatistic` is a concrete implementation of the interface.
4. `AbstractStatistic` is an abstract base class for defining statistics. Sub-classes of `AbstractStatistic` compute summary statistics of some kind.
5. `Histogram` defines a class to collect statistics and tabulate data into bins.
6. `Statistic` is a concrete implementation of `AbstractStatistic` allowing for a multitude of summary statistics.
7. `BatchStatistic` is also a concrete implementation of `AbstractStatistic` that provides for summarizing data via a batching process.
8. `IntegerFrequency` tabulates integer values into a frequencies by observed values, similar to a histogram.
9. `StateFrequency` facilitates defining labeled states and tabulating visitation and transition statistics.
10. `StatisticXY` collects statistics on (x, y) pairs computing statistics on the x and y values separately, as well as the covariance and correlation between the observations within a pair.
11. `BoxPlotSummary` produces the summary statistics necessary for the construction of a box plot.
12. `OLSRegression, RegressionResultsIfc,` and `RegressionData` facilitate performing multiple variable regression. The basic use of these classes is illustrated in Section 10.2.4 of Chapter 10.
13. `BootstrapSampler` facilitates the bootstrap statistical analysis technique. Its use is describe in Section 9.1 of Chapter 9.

The most important class within the statistics package is probably the `Statistic` class. This class summarizes the observed data into summary statistics such as: minimum, maximum, average, variance, standard devia-

tion, lag-1 correlation, and count. In addition, confidence intervals can be formed on the observations based on the student-t distribution. Finally, there are useful companion object methods for computing statistics on arrays and for estimating sample sizes. The reader is encourage to review the KSL documentation for all of the functionality, including the ability to write nicely printed statistical results.

In the remaining sections of this chapter, we will illustrate the collection of statistics on simple Monte Carlo models. This begins in the next section by estimating the area of a simple one-dimensional function.

3.2. Simple Monte Carlo Integration

In this example, we illustrate one of the fundamental uses of Monte Carlo methods: estimating the area of a function. Suppose we have some function, $g(x)$, defined over the range $a \leq x \leq b$ and we want to evaluate the integral:

$$\theta = \int_a^b g(x)\mathrm{d}x$$

Monte Carlo methods allow us to evaluate this integral by considering the problem as an estimation problem. It turns out that the problem can be translated into estimating the expected value of a well-chosen random variable. While a number of different choices for the random variable exist, we will pick one of the simplest for illustrative purposes. Define Y as follows with $X \sim U(a, b)$:

$$Y = (b - a)\, g(X) \tag{3.1}$$

Notice that Y is defined in terms of $g(X)$, which is also a random variable. Because a function of a random variable is also a random variable, this makes Y a random variable . Thus, the expectation of Y can be computed as follows:

$$E[Y] = (b-a)\,E[g(X)] \tag{3.2}$$

Now, let us derive,$E[g(X)]$. By definition,

$$E_X[g(x)] = \int_a^b g(x) f_X(x) \mathrm{d}x$$

And, the probability density function for a $X \sim U(a,b)$ random variable is:

$$f_X(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

Therefore,

$$E_X[g(x)] = \int_a^b g(x) f_X(x) \mathrm{d}x = \int_a^b g(x) \frac{1}{b-a} \mathrm{d}x$$

Substituting into Equation 3.2, yields,

$$\begin{aligned} E[Y] &= E[(b-a)\,g(X)] = (b-a)\,E[g(X)] \\ &= (b-a)\int_a^b g(x)\frac{1}{b-a}\mathrm{d}x \\ &= \int_a^b g(x)\mathrm{d}x = \theta \end{aligned}$$

Therefore, by estimating the expected value of Y, we can estimate the desired integral. From basic statistics, we know that a good estimator for $E[Y]$ is the sample average of observations of Y. Let $Y_1, Y_2, ...Y_n$ be a random sample of observations of Y. Let X_i be the i^{th} observation of X. Substituting each X_i into Equation 3.1 yields the i^{th} observation of Y,

$$Y_i = (b-a)\,g(X_i)$$

Then, the sample average of is:

$$\begin{aligned} \bar{Y}(n) &= \frac{1}{n}\sum_{i=1}^{n} Y_i = (b-a)\,\frac{1}{n}\sum_{i=1}^{n}(b-a)\,g(X_i) \\ &= (b-a)\,\frac{1}{n}\sum_{i=1}^{n} g(X_i) \end{aligned}$$

where $X_i \sim U(a,b)$. Thus, by simply generating $X_i \sim U(a,b)$, plugging the X_i into the function of interest, $g(x)$, taking the average over the values and multiplying by $(b-a)$, we can estimate the integral. This works for any integral and it works for multi-dimensional integrals. While this discussion is based on a single valued function, the theory scales to multi-dimensional integration through the use of multi-variate distributions.

Example 3.6 (Area Estimation)**.** Suppose that we want to estimate the area under $f(x) = x^{\frac{1}{2}}$ over the range from 1 to 4. That is, we want to evaluate the integral:

$$\theta = \int_1^4 x^{\frac{1}{2}} \mathrm{d}x = \frac{14}{3} = 4.6\bar{6}$$

According to the previously presented theory, we need to generate $X_i \sim U(1, 4)$ and then compute $\bar{Y}$, where $Y_i = (4-1)\sqrt{X_i} = 3\sqrt{X_i}$. In addition, for this simple example, we can easily check if our Monte Carlo approach is working because we know the true area.

```
fun main() {
    val a = 1.0
    val b = 4.0
    val ucdf = UniformRV(a, b, streamNum = 3)
    val stat = Statistic("Area Estimator")
    val n = 100 // sample size
    for (i in 1..n) {
        val x = ucdf.value
        val gx = sqrt(x)
        val y = (b - a) * gx
        stat.collect(y)
    }
    System.out.printf("True Area = %10.3f %n", 14.0 / 3.0)
    System.out.printf("Area estimate = %10.3f %n",
 ↪  stat.average)
    println("Confidence Interval")
    println(stat.confidenceInterval)
}
```

```
True Area =      4.667
```

```
Area estimate =      4.781
Confidence Interval
[4.608646560421988, 4.952515649272401]
```

> **Note**
>
> Notice that the previous examples created the random variable **outside** of the loop that is used to generate values. It may not be an error to create random variables within loops, but it is generally bad practice and inefficient in terms of object creation. If you see yourself creating many instances of an object and then using it only one time before it is discarded, you should reconsider what you are doing.

Because confidence intervals may form the basis for decision making, you can use the confidence interval half-width in determining the sample size. A review of these and other statistical concepts will be the focus of the next section.

3.3. Review of Statistical Concepts

The simulation models that have been illustrated have estimated the quantity of interest with a point estimate (i.e. $\bar{X}$). For example, in the previous section, we can easily compute the true area under the curve as $4.6\bar{6}$; however, the point estimate returned by the simulation was a value of 4.781, based on 100 samples. Thus, there is sampling error in our estimate. The key question examined in this section is how to control the sampling error in a simulation experiment. The approach that we will take is to determine the number of samples so that we can have high confidence in our point estimate. In order to address these issues, we need to review some basic statistical concepts in order to understand the meaning of sampling error.

3.3.1. Point Estimates and Confidence Intervals

Let x_i represent the i^{th} observations in a sample, $x_1, x_2, ...x_n$ of size n. Represent the random variables in the sample as $X_1, X_2, ...X_n$. The random variables form a random sample, if 1) the X_i are independent random variables and 2) every X_i has the same probability distribution. Let's assume that these two assumptions have been established. Denote the unknown cumulative distribution function of X as $F(x)$ and define the unknown expected value and variance of X with $E[X] = \mu$ and $Var[X] = \sigma^2$, respectively.

A statistic is any function of the random variables in a sample. Any statistic that is used to estimate an unknown quantity based on the sample is called an estimator. What would be a good estimator for the quantity $E[X] = \mu$? Without going into the details of the meaning of statistical goodness, one should remember that the sample average is generally a good estimator for $E[X] = \mu$. Define the sample average as follows:

$$\bar{X} = \frac{1}{n} \sum_{i=1}^{n} X_i \tag{3.3}$$

Notice that $\bar{X}$ is a function of the random variables in the sample and therefore it is also a random variable. This makes $\bar{X}$ a statistic, since it is a function of the random variables in the sample.

Any random variable has a corresponding probability distribution. The probability distribution associated with a statistic is called its sampling distribution. The sampling distribution of a statistic can be used to form a confidence interval on the point estimate associated with the statistic. The point estimate is simply the value obtained from the statistic once the data has been realized. The point estimate for the sample average is computed from the sample:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

A confidence interval expresses a degree of certainty associated with a point estimate. A specific confidence interval does not imply that the parameter μ is inside the interval. In fact, the true parameter is either in the interval or it is not within the interval. Instead you should think about the confidence level $1-\alpha$ as an assurance about the procedure used to compute the interval. That is, a confidence interval procedure ensures that if a large number of confidence intervals are computed each based on n samples, then the proportion of the confidence intervals that actually contain the true value of μ should be close to $1-\alpha$. The value α represents risk that the confidence interval procedure will produce a specific interval that does not contain the true parameter value. Any one particular confidence interval will either contain the true parameter of interest or it will not. Since you do not know the true value, you can use the confidence interval to assess the risk of making a bad decision based on the point estimate. You can be confident in your decision making or conversely know that you are taking a risk of α of making a bad decision. Think of α as the risk that using the confidence interval procedure will get you fired. If we know the sampling distribution of the statistic, then we can form a confidence interval procedure.

Under the assumption that the sample size is large enough such that the distribution of $\bar{X}$ is normally distributed then you can form an approximate confidence interval on the point estimate, $\bar{x}$. Assuming that the sample variance:

$$S^2(n) = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2 \tag{3.4}$$

is a good estimator for $Var[X] = \sigma^2$, then a $(1-\alpha)100\%$ two-sided confidence interval estimate for μ is:

$$\bar{x} \pm t_{1-(\alpha/2),n-1} \frac{s}{\sqrt{n}} \tag{3.5}$$

where $t_{p,\nu}$ is the $100p$ percentage point of the Student t-distribution with ν degrees of freedom. The spreadsheet function T.INV(p, degrees freedom) computes the *left-tailed* Student-t distribution value for a given probability and degrees of freedom. That is, $t_{p,\nu} = T.INV(p, \nu)$. The spreadsheet tab labeled *Student-t* in the spreadsheet that accompanies this chapter illustrates functions related to the Student-t distribution. Thus, in order to compute $t_{1-(\alpha/2),n-1}$, use the following formula:

$$t_{1-(\alpha/2),n-1} = T.INV(1-(\alpha/2), n-1)$$

The T.INV.2T(α, ν) spreadsheet function directly computes the *upper two-tailed* value. That is,

$$t_{1-(\alpha/2),n-1} = T.INV.2T(\alpha, n-1)$$

Within the statistical package R, this computation is straight-forward by using the $t_{p,n} = qt(p, n)$ function.

$$t_{p,n} = qt(p, n)$$

```
#'qt(0.975, 5)
qt(0.975,5)
```

```
[1] 2.570582
```

```
#' set alpha
alpha = 0.05
#'qt(1-(alpha/2, 5)
qt(1-(alpha/2),5)
```

```
[1] 2.570582
```

Using the KSL, we can compute the quantile of the Student-T distribution via the following:

```
fun main(){
    val level = 0.95
    val dof = 5.0
    val alpha = 1.0 - level
    val p = 1.0 - alpha / 2.0
    val t: Double = StudentT.invCDF(dof, p)
    println("p = $p dof = $dof t-value = $t")
}
```

With results:

```
p = 0.975 dof = 5.0 t-value = 2.5705818445939186
```

3.3.2. Sample Size Determination

The confidence interval for a point estimator can serve as the basis for determining how many observations to have in the sample. From Equation 3.5, the quantity:

$$h = t_{1-(\alpha/2),n-1}\frac{s}{\sqrt{n}} \tag{3.6}$$

is called the half-width of the confidence interval. You can place a bound, ϵ, on the half-width by picking a sample size that satisfies:

$$h = t_{1-(\alpha/2),n-1}\frac{s}{\sqrt{n}} \leq \epsilon \tag{3.7}$$

We call ϵ the margin of error for the bound. Unfortunately, $t_{1-(\alpha/2),n-1}$ depends on n, and thus Equation 3.7 is an iterative equation. That is, you must try different values of n until the condition is satisfied. Within this text, we call this method for determining the sample size the *Student-T iterative method*.

Alternatively, the required sample size can be approximated using the normal distribution. Solving Equation 3.7 for n yields:

$$n \geq \left(\frac{t_{1-(\alpha/2),n-1}\ s}{\epsilon} \right)^2$$

As n gets large, $t_{1-(\alpha/2),n-1}$ converges to the $100(1-(\alpha/2))$ percentage point of the standard normal distribution $z_{1-(\alpha/2)}$. This yields the following approximation:

$$n \geq \left(\frac{z_{1-(\alpha/2)} s}{\epsilon} \right)^2 \tag{3.8}$$

Within this text, we refer to this method for determining the sample size as the *normal approximation* method. This approximation generally works well for large n, say $n > 50$. Both of these methods require an initial value for the standard deviation. This computation is implemented in the function, `estimateSampleSize` of the `Statistic` class's companion object.

```
       /**
        * Estimate the sample size based on a normal
↪  approximation
        *
        * @param desiredHW the desired half-width (must be
↪  bigger than 0)
        * @param stdDev    the standard deviation (must be
↪  bigger than or equal to 0)
```

```
         * @param level     the confidence level (must be
↪  between 0 and 1)
         * @return the estimated sample size
         */
        fun estimateSampleSize(desiredHW: Double, stdDev:
         ↪  Double, level: Double): Long {
            require(desiredHW > 0.0) { "The desired
↪  half-width must be > 0" }
            require(stdDev >= 0.0) { "The desired std. dev.
↪  must be >= 0" }
            require(!(level <= 0.0 || level >= 1.0)) {
↪  "Confidence Level must be (0,1)" }
            val a = 1.0 - level
            val a2 = a / 2.0
            val z = Normal.stdNormalInvCDF(1.0 - a2)
            val m = z * stdDev / desiredHW * (z * stdDev /
             ↪  desiredHW)
            return (m + .5).roundToLong()
        }
```

In order to use either the *normal approximation* method or the *Student-T iterative* method, you must have an initial value for, s, the sample standard deviation. The simplest way to get an initial estimate of s is to make a small initial pilot sample (e.g. $n_0 = 5$). Given a value for s you can then set a desired bound and use the formulas. The bound is problem and performance measure dependent and is under your subjective control. You must determine what bound is reasonable for your given situation. One thing to remember is that the bound is squared in the denominator for evaluating n. Thus, very small values of ϵ can result in very large sample sizes.

Example 3.7 (Determining the Sample Size)**.** Suppose we are required to estimate the output from a simulation so that we are 99% confidence that we are within ± 0.1 of the true population mean. After taking a pilot sample of size $n = 10$, we have estimated $s = 6$. What is the required sample size?

We will use Equation 3.8 to determine the sample size requirement. For a 99% confidence interval, we have $\alpha = 0.01$ and $\alpha/2 = 0.005$. Thus, $z_{1-0.005} = z_{0.995} = 2.5758293064439264$. Because the margin of error, ϵ is 0.1, we have that,

$$n \geq \left(\frac{z_{1-(\alpha/2)}\, s}{\epsilon} \right)^2 = \left(\frac{2.5758293064439264 \times 6}{0.1} \right)^2 = 23885.63 \approx 23886$$

The following KSL code will easily compute the sample size.

```
fun main() {
    val desiredHW = 0.1
    val s0 = 6.0
    val level = 0.99
    val n = Statistic.estimateSampleSize(
        desiredHW = desiredHW,
        stdDev = s0,
        level = level
    )
    println("Sample Size Determination")
    println("desiredHW = $desiredHW")
    println("stdDev = $s0")
    println("Level = $level")
    println("recommended sample size = $n")
}
```

```
Sample Size Determination
desiredHW = 0.1
stdDev = 6.0
Level = 0.99
recommended sample size = 23886
```

If the quantity of interest is a proportion, then a different method can be used. In particular, a $100 \times (1 - \alpha)\%$ large sample two sided confidence interval for a proportion, p, has the following form:

$$\widehat{p} \pm z_{1-(\alpha/2)} \sqrt{\frac{\widehat{p}(1-\widehat{p})}{n}} \tag{3.9}$$

where $\widehat{p}$ is the estimate for p. From this, you can determine the sample size via the following equation:

$$n = \left(\frac{z_{1-(\alpha/2)}}{\epsilon} \right)^2 \widehat{p}(1-\widehat{p}) \tag{3.10}$$

Again, a pilot run is necessary for obtaining an initial estimate of $\widehat{p}$, for use in determining the sample size. If no pilot run is available, then $\widehat{p} = 0.5$ is often assumed as a worse case approximation.

If you have more than one performance measure of interest, you can use these sample size techniques for each of your performance measures and then use the maximum sample size required across the performance measures. Now, let's illustrate these methods based on a small simulation.

3.3.3. Determining the Sample Size for a Monte Carlo Simulation Experiment

To facilitate some of the calculations related to determining the sample size for a simulation experiment, I have constructed a spreadsheet called

SampleSizeDetermination.xlsx, which is found in the book support files for this chapter. You may want to utilize that spreadsheet as you go through this and subsequent sections.

Using a simple example, we will illustrate how to determine the sample size necessary to estimate a quantity of interest with a high level of confidence.

Example 3.8 (Sample Size Example)**.** Suppose that we want to simulate a normally distributed random variable, X, with $E[X] = 10$ and $Var[X] = 16$. From the simulation, we want to estimate the true population mean with 99 percent confidence for a half-width ± 0.50 margin of error. In addition to estimating the population mean, we want to estimate the probability that the random variable exceeds 8. That is, we want to estimate, $p = P(X > 8)$. We want to be 95% confident that our estimate of $P(X > 8)$ has a margin of error of ± 0.05. What are the sample size requirements needed to meet the desired margins of error?

Using simulation for this example is for illustrative purposes. Naturally, we actually know the true population mean and we can easily compute the $p = P(X > 8)$. for this situation. However, the example will illustrate sample size determination, which is an important planning requirement for simulation experiments.

Let X represent the unknown random variable of interest. Then, we are interested in estimating $E[X] = \mu$ and $p = P(X > 8)$. To estimate these quantities, we generate a random sample, $(X_1, X_2, ..., X_n)$. $E[X]$ can be estimated using $\bar{X}$. To estimate a probability it is useful to define an indicator variable. An indicator variable has the value 1 if the condition associated with the probability is true and has the value 0 if it is false. To estimate p, define the following indicator variable:

$$Y_i = \begin{cases} 1 & X_i > 8 \\ 0 & X_i \leq 8 \end{cases}$$

This definition of the indicator variable Y_i allows $\bar{Y}$ to be used to estimate p. Recall the definition of the sample average $\bar{Y}$. Since Y_i is a 1 only if $X_i > 8$, then the $\sum_{i=1}^{n} Y_i$ simply adds up the number of 1's in the sample. Thus, $\sum_{i=1}^{n} Y_i$ represents the count of the number of times the event $X_i > 8$ occurred in the sample. Call this $\#\{X_i > 8\}$. The ratio of the number of times an event occurred to the total number of possible occurrences represents the proportion. Thus, an estimator for p is:

$$\hat{p} = \bar{Y} = \frac{1}{n}\sum_{i=1}^{n} Y_i = \frac{\#\{X_i > 8\}}{n}$$

Therefore, computing the average of an indicator variable will estimate the desired probability. The code for this situation is quite simple:

```
fun main() {
    val rv = NormalRV(10.0, 16.0, streamNum = 3)
    val estimateX = Statistic("Estimated X")
    val estOfProb = Statistic("Pr(X>8)")
    val r = StatisticReporter(mutableListOf(estOfProb,
     ↪  estimateX))
    val n0 = 20 // sample size
    for (i in 1..n0) {
        val x = rv.value
        estimateX.collect(x)
        estOfProb.collect(x > 8)
    }
    println(r.halfWidthSummaryReport())
}
```

In the code, an instance of Statistic is used to observe values of the quantity $(x > 8)$. This quantity is actually a logical condition which will evaluate

to true (1) or false (0) given the value of x. Thus, the Statistic will be recording observations of 1's and 0's. Because the Statistic computes the average of the values that it "collects", we will get an estimate of the probability. Thus, the quantity, $(x > 8)$ is an indicator variable for the desired event. Using a Statistic to estimate a probability in this manner is quite effective.

Now, in more realistic simulation situations, we would not know the true population mean and variance. Thus, in solving this problem, we will ignore that information. To apply the previously discussed sample size determination methods, we need estimates of $\sigma = \sqrt{Var[X]}$ and $p = P(X > 8)$. In order to get estimates of these quantities, we need to run our simulation experiment; however, in order to run our simulation experiment, we need to know how many observations to make. This is quite a catch-22 situation!

To proceed, we need to run our simulation model for a pilot experiment. From a small number of samples (called a pilot experiment, pilot run or pilot study), we will estimate σ and $p = P(X > 8)$ and then use those estimates to determine how many samples we need to meet the desired margins of error. We will then re-run the simulation using the recommended sample size.

The natural question to ask is how big should your pilot sample be? In general, this depends on how much it costs for you to collect the sample. Within a simulation experiment context, generally, cost translates into how long your simulation model takes to execute. For this small problem, the execution time is trivial, but for large and complex simulation models the execution time may be in hours or even days. A general rule of thumb is between 10 and 30 observations. For this example, we will use an initial pilot sample size, $n_0 = 20$.

Running the model for the 20 replications yields the following results.

Table 3.1.: Simulation Results for $n_0 = 20$ Replications

Performance Measures	Average	95% Half-Width
Pr(X>8)	0.80	0.1921
Estimated X	12.2484	2.2248

Notice that the confidence interval for the true mean is $(10.0236, 14.4732)$ with a half-width of 2.2248, which exceeds the desired margin of error, $\epsilon = 0.5$. Notice also that this particular confidence interval happens to not contain the true mean $E[X] = 10$. To determine the sample size so that we get a half-width that is less than $\epsilon = 0.1$ with 99% confidence, we can use Equation 3.8. In order to do this, we must first have an estimate of the sample standard deviation, s. Since Table 3.1 reports a half-width for a 95% confidence interval, we need to use Equation 3.6 and solve for s in terms of h. Rearranging Equation 3.6 yields,

$$s = \frac{h\sqrt{n}}{t_{1-(\alpha/2),n-1}} \tag{3.11}$$

Using the results from Table 3.1 and $\alpha = 0.05$ in Equation 3.11 yields,

$$s_0 = \frac{h\sqrt{n_0}}{t_{1-(\alpha/2),n_0-1}} = \frac{2.2248\sqrt{20}}{t_{0.975,19}} = \frac{2.2248\sqrt{20}}{2.093024} = 4.7536998$$

Now, we can use Equation 3.8 to determine the sample size requirement. For a 99% confidence interval, we have $\alpha = 0.01$ and $\alpha/2 = 0.005$. Thus, $z_{0.995} = 2.576$. Because the margin of error, ϵ is 0.5, we have that,

$$n \geq \left(\frac{z_{1-(\alpha/2)}\ s}{\epsilon}\right)^2 = \left(\frac{2.5758293064439264 \times 4.7536998}{0.5}\right)^2 = 599.73 \approx 600$$

To determine the sample size for estimating $p = P(X > 8)$ with 95% confidence to ± 0.1, we can use Equation 3.10

$$n = \left(\frac{z_{1-(\alpha/2)}}{\epsilon}\right)^2 \hat{p}(1-\hat{p}) = \left(\frac{z_{0.975}}{0.05}\right)^2 (0.80)(1-0.80) = \left(\frac{1.96}{0.05}\right)^2 (0.80)(0.20) = 245.86 \approx 246$$

All of these calculations can be easily accomplished using Kotlin code as follows:

```
fun main() {
    val rv = NormalRV(10.0, 16.0, streamNum = 3)
    val estimateX = Statistic("Estimated X")
    val estOfProb = Statistic("Pr(X>8)")
    val r = StatisticReporter(mutableListOf(estOfProb,
     ↪  estimateX))
    val n0 = 20 // sample size
    for (i in 1..n0) {
        val x = rv.value
        estimateX.collect(x)
        estOfProb.collect(x > 8)
    }
    println(r.halfWidthSummaryReport())
    val desiredHW = 0.5
    val s0 = estimateX.standardDeviation
    val level = 0.99
    val n = Statistic.estimateSampleSize(
        desiredHW = desiredHW,
        stdDev = s0,
        level = level
    )
    println("Sample Size Determination")
    println("desiredHW = $desiredHW")
    println("stdDev = $s0")
```

```
    println("Level = $level")
    println("recommended sample size = $n")
    println()
    val m = Statistic.estimateProportionSampleSize(
        desiredHW = 0.05,
        pEst = estOfProb.average,
        level = 0.95
    )
    println("Recommended sample size for proportion = $m")
}
```

The results match the values computed from the equations.

```
Sample Size Determination
desiredHW = 0.5
stdDev = 4.753707020199372
Level = 0.99
recommended sample size = 600
Recommended sample size for proportion = 246
```

By using the maximum of $\max(246, 600) = 600$, we can re-run the simulation for this number of replications. Doing so, yields,

Table 3.2.: Simulation Results for $n = 600$ Replications

Performance Measures	Average	95% Half-Width
Pr(X>8)	0.7117	0.0363
Estimated X	10.2712	0.3284

As can be seen in Table 3.2, the half-width values meet the desire margins of error. It may be possible that the margins of error might not be met. This

suggests that more than $n = 600$ observations is needed to meet the margin of error criteria. Equation 3.8 and Equation 3.10 are only *approximations* and based on a pilot sample. Thus, if there was considerable sampling error associated with the pilot sample, the approximations may be inadequate.

As can be noted from this example in order to apply the normal approximation method for determining the sample size based on the pilot run, we need to compute the initial sample standard deviation, s_0, from the initial reported half-width, h. This requires the use of Equation 3.11 to first compute the value of s from h. We can avoid this calculation by using the *half-width ratio* method for determining the sample size.

Let h_0 be the initial value for the half-width from the pilot run of n_0 replications. Then, rewriting Equation 3.6 in terms of the pilot data yields:

$$h_0 = t_{1-(\alpha/2),n_0-1}\frac{s_0}{\sqrt{n_0}}$$

Solving for n_0 yields:

$$n_0 = t^2_{1-(\alpha/2),n_0-1}\frac{s_0^2}{h_0^2} \tag{3.12}$$

Similarly for any n, we have:

$$n = t^2_{1-(\alpha/2),n-1}\frac{s^2}{h^2} \tag{3.13}$$

Taking the ratio of n_0 to n (Equation 3.12 and Equation 3.13) and assuming that $t_{1-(\alpha/2),n-1}$ is approximately equal to $t_{1-(\alpha/2),n_0-1}$ and s^2 is approximately equal to s_0^2, yields,

$$n \cong n_0\frac{h_0^2}{h^2} = n_0\left(\frac{h_0}{h}\right)^2$$

This is the half-width ratio equation.

$$n \geq n_0 \left(\frac{h_0}{h}\right)^2 \tag{3.14}$$

The issue that we face when applying Equation 3.14 is that Table 3.1 only reports the 95% confidence interval half-width. Thus, to apply Equation 3.14 the value of h_0 will be based on an $\alpha = 0.05$. If the desired half-width, h, is required for a different confidence level, e.g. a 99% confidence interval, then you must first translate h_0 to be based on the desired confidence level or set the confidence level for the `StatisticReporter` to be 99%. To compute the 95% half-width from Table 3.1, you can compute s from Equation 3.11 and then recomputing the actual half-width, h, for the desired confidence level.

Using the results from Table 3.1 and $\alpha = 0.05$ in Equation 3.11 we already know that $s_0 = 4.7536998$ for a 95% confidence interval. Thus, for a 99% confidence interval, where $\alpha = 0.01$ and $\alpha/2 = 0.005$, we have:

$$h_0 = t_{1-(\alpha/2),n_0-1}\frac{s_0}{\sqrt{n_0}} = t_{0.995,19}\frac{4.7536998}{\sqrt{20}} = 2.861\frac{4.7536998}{\sqrt{20}} = 3.041127$$

Now, we can apply Equation 3.14 to this problem with the desire half-width bound $\epsilon = 0.5 = h$:

$$n \geq n_0 \left(\frac{h_0}{h}\right)^2 = 20\left(\frac{3.041127}{h}\right)^2 = 20\left(\frac{3.041127}{0.5}\right)^2 = 739.87 \cong 740$$

We see that for this pilot sample, the half-width ratio method recommends a substantially larger sample size, 740 versus 600. In practice, the half-width ratio method tends to be conservative by recommending a larger sample size. We will see another example of these methods within the next section.

Based on these examples, you should have a basic understanding of how a simulation experiment can be performed to meet a desired margin of error. In general, simulation models are much more interesting than this simple example.

3.4. Simulating the Game of Craps

This section presents an application of Monte Carlo methods to the estimation of the probability of winning the game of "craps" as played in Las Vegas. The following example will illustrate how to generate random variates and collect statistics in a useful manner.

Example 3.9 (Simulating Craps)**.** The basic rules of the game are as follows: one player, the "shooter", rolls a pair of dice. If the outcome of that roll is a 2, 3, or 12, the shooter immediately loses; if it is a 7 or an 11, the shooter wins. In all other cases, the number the shooter rolls on the first toss becomes the "point", which the shooter must try to duplicate on subsequent rolls. If the shooter manages to roll the point before rolling a 7, the shooter wins; otherwise the shooter loses. It may take several rolls to determine whether the shooter wins or loses. After the first roll, only a 7 or the point have any significance until the win or loss is decided. Using the KSL random and statistic packages give answers and corresponding estimates to the following questions. Be sure to report your estimates in the form of confidence intervals.

a) Before the first roll of the dice, what is the probability the shooter will ultimately win?
b) What is the expected number of rolls required to decide the win or loss?

The solution to this problem involves the use of the `Statistic` class to collect the probability that the shooter will win and for the expected number

of rolls. The following code illustrates the approach. The `DUniformRV` class is used to represent the two dice. Two statistics are created to estimate the probability of winning and to estimate the expected number of rolls required to decide a win or a loss. A for loop is use to simulate 5000 games. For each game, the logic of winning, losing or matching the first point is executed. When the game is ended the instances of `Statistic` are used to report the statistical results.

```
fun main() {
    val d1 = DUniformRV(1, 6, streamNum = 3)
    val d2 = DUniformRV(1, 6, streamNum = 4)
    val probOfWinning = Statistic("Prob of winning")
    val numTosses = Statistic("Number of Toss Statistics")
    val numGames = 5000
    for (k in 1..numGames) {
        var winner = false
        val point = d1.value.toInt() + d2.value.toInt()
        var numberoftoss = 1
        if (point == 7 || point == 11) {
            // automatic winner
            winner = true
        } else if (point == 2 || point == 3 || point == 12) {
            // automatic loser
            winner = false
        } else { // now must roll to get point
            var continueRolling = true
            while (continueRolling) {
                // increment number of tosses
                numberoftoss++
                // make next roll
                val nextRoll = d1.value.toInt() +
                ↪ d2.value.toInt()
                if (nextRoll == point) {
                    // hit the point, stop rolling
```

```
                    winner = true
                    continueRolling = false
                } else if (nextRoll == 7) {
                    // crapped out, stop rolling
                    winner = false
                    continueRolling = false
                }
            }
        }
        probOfWinning.collect(winner)
        numTosses.collect(numberoftoss.toDouble())
    }
    val reporter = StatisticReporter()
    reporter.addStatistic(probOfWinning)
    reporter.addStatistic(numTosses)
    println(reporter.halfWidthSummaryReport())
}
```

As we can see from the results of the statistics reporter, the probability of winning is just a little less than 50 percent.

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                              Count         Average      Half-
Width
-------------------------------------------------------------------------------
Prob of winning                    5000          0.4960          0.0139
Number of Toss Statistics          5000          3.4342          0.0856
-------------------------------------------------------------------------------
```

Now let's look at a slightly more complex static simulation.

3.5. The News Vendor Problem

The news vendor model is a classic inventory model that allows for the modeling of how much to order for a decision maker facing uncertain demand for an upcoming period of time. It takes on the news vendor moniker because of the context of selling newspapers at a newsstand. The vendor must anticipate how many papers to have on hand at the beginning of a day so as to not run short or have papers left over. This idea can be generalized to other more interesting situations (e.g. air plane seats). Discussion of the news vendor model is often presented in elementary textbooks on inventory theory. The reader is referred to (Muckstadt and Sapras 2010) for a discussion of this model.

The basic model is considered a single period model; however, it can be extended to consider an analysis covering multiple (or infinite) future periods of demand. The representation of the costs within the modeling offers a number of variations.

Let D be a random variable representing the demand for the period. Let $F(d) = P[D \leq d]$ be the cumulative distribution function for the demand. Define $G(q, D)$ as the profit at the end of the period when q units are ordered at the start of the period with D units of demand. The parameter q is the decision variable associated with this model. Depending on the value of q chosen by the news vendor, a profit or loss will occur.

There are two cases to consider $D \geq q$ or $D < q$. That is, when demand is greater than or equal to the amount ordered at the start of the period and when demand is less than the amount ordered at the start of the period. If $D \geq q$ the news vendor runs out of items, and if $D < q$ the news vendor will have items left over.

In order to develop a profit model for this situation, we need to define some model parameters. In addition, we need to determine the amount that we might be short and the amount we might have left over in order to determine the revenue or loss associated with a choice of q. Let c be

the purchase cost. This is the cost that the news vendor pays the supplier for the item. Let s be the selling price. This is the amount that the news vendor charges customers. Let u be the salvage value (i.e. the amount per unit that can be received for the item after the planning period). For example, the salvage value can be the amount that the news vendor gets when selling a left over paper to a recycling center. We assume that selling price $>$ purchase cost $>$ salvage value.

Consider the context of a news vendor planning for the day. What is the possible amount sold? If D is the demand for the day and we start with q items, then the most that can be sold is $\min(D, q)$. That is, if D is bigger than q, you can only sell q. If D is smaller than q, you can only sell D.

What is the possible amount left over (available for salvage)? If D is the demand and we start with q, then there are two cases: 1) demand is less than the amount ordered ($D < q$) or 2) demand is greater than or equal to the amount ordered ($D \geq q$). In case 1, we will have $q - D$ items left over. In case 2, we will have 0 left over. Thus, the amount left over at the end of the day is $\max(0, q - D)$. Since the news vendor buys q items for c, the cost of the items for the day is $c \times q$. Thus, the profit has the following form:

$$G(D, q) = s \times \min(D, q) + u \times \max(0, q - D) - c \times q \tag{3.15}$$

In words, the profit is equal to the sales revenue plus the salvage revenue minus the ordering cost. The sales revenue is the selling price times the amount sold. The salvage revenue is the salvage value times the amount left over. The ordering cost is the cost of the items times the amount ordered. To find the optimal value of q, we should try to maximize the expected profit. Since D is a random variable, $G(D, q)$ is also a random variable. Taking the expected value of both sides of Equation 3.15, yields:

$$g(q) = E[G(D, q)] = sE[\min(D, q)] + uE[\max(0, q - D)] - cq$$

Whether or not this equation can be optimized depends on the form of the distribution of D; however, simulation can be used to estimate this expected value for any given q. Let's look at an example.

Example 3.10 (Sly's BBQ Wings)**.** Sly's convenience store sells BBQ wings for 25 cents a piece. They cost 15 cents a piece to make. The BBQ wings that are not sold on a given day are purchased by a local food pantry for 2 cents each. Assuming that Sly decides to make 30 wings a day, what is the expected revenue for the wings provided that the demand distribution is as show in Table 3.3.

Table 3.3.: Distribution of BBQ Wing Demand

d_i	5	10	40	45	50	55	60
$f(d_i)$	0.1	0.2	0.3	0.2	0.1	0.05	0.05
$F(d_i)$	0.1	0.3	0.6	0.8	0.9	0.95	1.0

The code for the news vendor problem will require the generation of the demand from BBQ Wing demand distribution. In order to generate from this distribution, the `DEmpiricalRV` class should be used. Since the cumulative distribution has already been computed, it is straight forward to write the inputs for the `DEmpiricalRV` class.

```
fun main() {
    val q = 30.0 // order qty
    val s = 0.25 //sales price
    val c = 0.15 // unit cost
```

```
    val u = 0.02 //salvage value
    val values = doubleArrayOf(5.0, 10.0, 40.0, 45.0, 50.0,
     ↪  55.0, 60.0)
    val cdf = doubleArrayOf(0.1, 0.3, 0.6, 0.8, 0.9, 0.95,
     ↪  1.0)
    val dCDF = DEmpiricalRV(values, cdf, streamNum = 3)
    val stat = Statistic("Profit")
    val n = 100.0 // sample size
    var i = 1
    while (i <= n) {
        val d = dCDF.value
        val amtSold = minOf(d, q)
        val amtLeft = maxOf(0.0, q - d)
        val g = s * amtSold + u * amtLeft - c * q
        stat.collect(g)
        i++
    }
    System.out.printf("%s \t %f %n", "Count = ", stat.count)
    System.out.printf("%s \t %f %n", "Average = ",
 ↪  stat.average)
    System.out.printf("%s \t %f %n", "Std. Dev. = ",
 ↪  stat.standardDeviation)
    System.out.printf("%s \t %f %n", "Half-width = ",
 ↪  stat.halfWidth)
    println((stat.confidenceLevel * 100).toString() + "% CI =
 ↪  " + stat.confidenceInterval)
}
```

Running the model for 100 days results in the following output.

```
Count =      100.000000
Average =    1.677500
Std. Dev. =      2.201324
```

```
Half-width =     0.436790
95.0% CI = [1.2407095787617952, 2.1142904212382048]
```

3.6. A Simple Inspection Process

The following example illustrates how to model a simple inspection process that has individual items being defective and includes the random selection of items for inspection.

Example 3.11 (Simple Inspection Process)**.** A manufacturing process has a simple inspection process. The process produces items and places them in boxes with four items per box. The manufacturing process has a defect rate of 15%. That is, the probability of an individual item being defective is 0.15. Assume that the chance of an item being defective is independent of any other item being defective. An inspector tests each box by randomly sampling one item from the box. If the selected item is good, the box is passed; if the selected item is defective, the box is rejected. We are interested in estimating the expected number of boxes that the inspector will look at before the first box is rejected.

This situation could be modeled using the concepts of probability theory; however, we will simulate the process to illustrate the use of KSL constructs. The probability that an item is defective can be modeled with a Bernoulli random variable with probability of success equal 0.15. In the following code, we model the inspection process with a function that counts the number of boxes that are inspected until the first rejected box.

```
fun main() {
    val itemRV = BernoulliRV(probOfSuccess = 0.15, streamNum
     ↪  = 3)
    val itemsPerBox = 4
    val stat = Statistic("Num Until Rejection")
    val sampleSize = 100
    for (i in 1..sampleSize) {
        val countBoxes = numUntilFirstRejection(itemRV,
         ↪  itemsPerBox)
        stat.collect(countBoxes)
    }
    print(String.format("%s \t %f %n", "Count = ",
 ↪  stat.count))
    print(String.format("%s \t %f %n", "Average = ",
 ↪  stat.average))
    print(String.format("%s \t %f %n", "Std. Dev. = ",
 ↪  stat.standardDeviation))
    print(String.format("%s \t %f %n", "Half-width = ",
 ↪  stat.halfWidth))
    println((stat.confidenceLevel * 100).toString() + "% CI =
 ↪  " + stat.confidenceInterval)
}

/**
 *  This function counts the number of boxes inspected until
 ↪  the first
 *  box is found that has a randomly selected item that is
 ↪  defective.
 */
fun numUntilFirstRejection(itemRV: BernoulliRV, itemsPerBox:
 ↪  Int): Double {
    require(itemsPerBox >= 1) { "There must be at least 1
 ↪  item per box" }
```

```
    var count = 0.0
    do {
        count++
        // randomly generate the box
        val box = itemRV.sample(itemsPerBox)
        // randomly sample from the box
        val inspection = KSLRandom.randomlySelect(box)
        // stops if bad = 1.0 is found, continues if good =
         ↪  0.0 is found
    } while (inspection != 1.0)
    return count
}
```

Notice that first a sample from the Bernoulli random variable is obtained. The size of this sample is the number of items per box (equal to 4 in this example). The returned sample array contains elements with values 1.0 or 0.0, with 1.0 indicating a defective item. Then, the `KSLRandom` functionality is used to randomly select from the sample "box" an element (item). If the item is good, the sampling continues with a new box. If the selected item is bad, then the sampling stops and returns the number of sample boxes inspected until the first bad box is found.

```
Count =      100.000000
Average =    5.710000
Std. Dev. =      4.688897
Half-width =     0.930379
95.0% CI = [4.779621063279056, 6.640378936720944]
```

The results indicate that on average about 5.71 boxes are inspected until the first box is rejected.

3.7. Stochastic Activity Networks

A stochastic activity network (SAN) is a common modeling paradigm often used to represent project networks and other forms of network modeling. An activity network consists of a set of activities that have an ordered precedence indicating the dependence between the activities. For example, an activity C might not be able to start until both activities A and B are completed. Thus, (A, B) precede C in the network. A common question in such networks is how to determine the critical path or the longest path in the network. The following example will illustrate how to simulate a simple SAN to estimate the expected length of the critical path and estimate probabilities associated with time to complete the network.

Example 3.12 (Project Network)**.** Consider a project consisting of nine jobs (A, B, ···, I) with the precedence relationships and activity times. The table provides the minimum, mode, and maximum for each activity. We will assume that the activity times can be modeled using a triangular distribution with the appropriate parameters.

Job	Predecessors	Min	Mode	Max
A	-	2	5	8
B	A	6	9	12
C	A	6	7	8
D	B,C	1	4	7
E	A	7	8	9
F	D,E	5	14	17
G	C	3	12	21
H	F,G	3	6	9
I	H	5	8	11

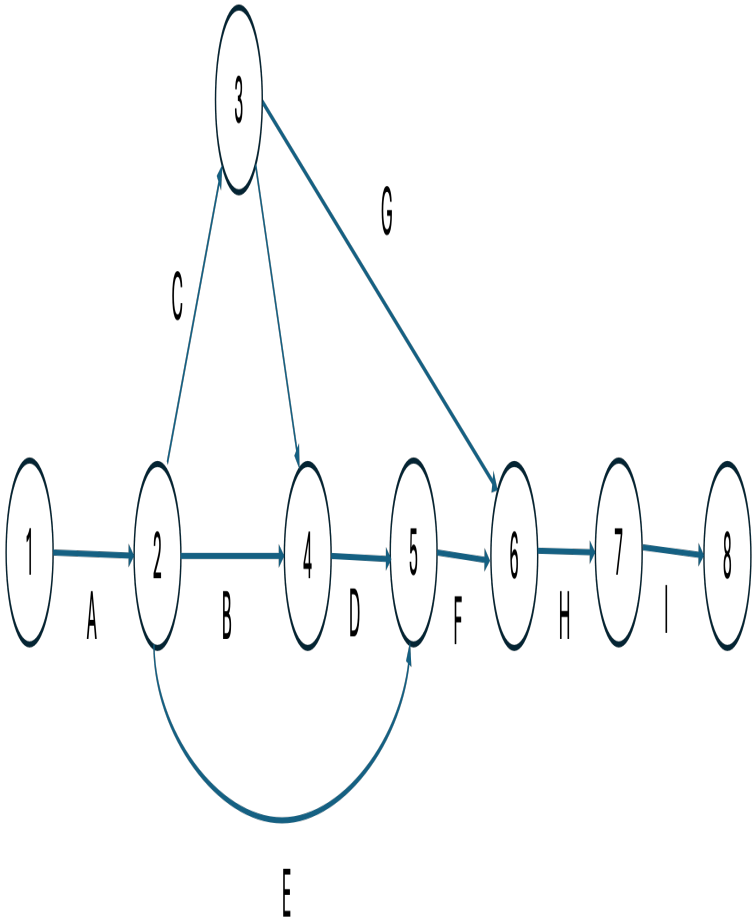


Figure 3.6.: Project Network Diagram

Develop a simulation model that will estimate the expected time to completion for the project. In addition, estimate the probability that the project will be completed within 50 days. Finally, estimate the probability that the project will be completed between 42 and 48 days.

The key to modeling this situation is representing the data associated with the network. The longest path in the network will depend upon the random activity times. A simple approach to the determining the longest path in the network is to enumerate the paths. Then, we can randomly generate the time associated with each activity. From the random times, we can determine the length of each path. If we know the length of each path, then we can compute the path that is the longest.

In the following code, we represent the activities and the associated activity time (triangular) distributions using a map.

```
val activityRVs = mapOf<String, TriangularRV>(
    "A" to TriangularRV(2.0, 5.0, 8.0, streamNum = 2),
    "B" to TriangularRV(6.0, 9.0, 12.0, streamNum = 3),
    "C" to TriangularRV(6.0, 7.0, 8.0, streamNum = 4),
    "D" to TriangularRV(1.0, 4.0, 7.0, streamNum = 5),
    "E" to TriangularRV(7.0, 8.0, 9.0, streamNum = 6),
    "F" to TriangularRV(5.0, 14.0, 17.0, streamNum = 7),
    "G" to TriangularRV(3.0, 12.0, 21.0, streamNum = 8),
    "H" to TriangularRV(3.0, 6.0, 9.0, streamNum = 9),
    "I" to TriangularRV(5.0, 8.0, 11.0, streamNum = 10),
)
```

If you look closely at Figure 3.6, you can discern that there are four unique paths through the network. Since the paths are clear for this example, we can represent the paths with a list of lists.

```
    val paths = mutableListOf<List<String>>(
        listOf("A", "B", "D", "F", "H", "I"),
        listOf("A", "E", "F", "H", "I"),
        listOf("A", "C", "D", "F", "H", "I"),
        listOf("A", "B", "C", "G", "H", "I"),
    )
```

This data structure will allow each path to be enumerated to determine the length of the network once the random activity times have been realized. In general, for large networks, this enumeration approach will not be the best approach and specialized algorithms will be required to find the longest path.

Now that we have the data structures for holding the activity times and the paths, we can write functions that can take advantage of the data structures. The first function that we will write will randomly generate the activity times for the network. That is, in essence, randomly generate the time for each arc (activity) of the network. This can be accomplished by iterating through the activity map.

```
fun generateActivityTimes(activities: Map<String,
 ↪  TriangularRV>) : Map<String, Double> {
    val map = mutableMapOf<String, Double>()
    for ((name, rv) in activities){
        map[name] = rv.value
    }
    return map
}
```

In this code, the activities map is iterated for each activity and the associated random variable is called to generate the associated activity time. The realized activity time is stored in an outgoing map that holds the activity name and its activity time. This map holds a realized network.

From a realized network, we can enumerate the paths to determine the longest path and thus the completion time. This idea is implemented in the following function.

```
fun timeToCompletion(activityTimes: Map<String, Double>,
 ↪  paths: List<List<String>>): Double {
    val times = mutableListOf<Double>()
    for (path in paths) {
        var pathTime = 0.0
        for (activity in path) {
            pathTime = pathTime + activityTimes[activity]!!
        }
        times.add(pathTime)
    }
    return times.max()
}
```

In this code, each path in the provided list of paths is enumerated. The time to complete the path is the sum of the individual activity times on the path. Once we have all the total path lengths computed, we can easily use the `max()` function of the Kotlin collection library to determine the maximum time, which is returned as the time to complete the realization of the sample network. Now, we are ready to simulate many instances of the network to collect statistics associated with the expected performance and probability of completion. The following code puts these ideas into action.

```
    val timeToCompleteStat = Statistic("Time to Completion")
    val probLT50Days = Statistic("P(T<=50)")
    val probWith42and48Days = Statistic("P(42<=T<=48)")
    val interval = Interval(42, 48)
    val sampleSize = 1000
    for (i in 1..sampleSize) {
```

```
        val a = generateActivityTimes(activityRVs)
        val t = timeToCompletion(a, paths)
        timeToCompleteStat.collect(t)
        probLT50Days.collect(t <= 50)
        probWith42and48Days.collect(interval.contains(t))
    }
    val sr =
     ↪ StatisticReporter(mutableListOf(timeToCompleteStat,
     ↪ probLT50Days, probWith42and48Days))
    println(sr.halfWidthSummaryReport())
```

In this code, we randomly generate 1000 networks and thus 1000 completion times. As in previous examples, instances of the `Statistic` class are used to collect and report statistics for the simulation.

Statistical Summary Report

Name	Count	Average	Half-Width
Time to Completion	1000.0	47.78885659544101	0.23712135354990163
P(T<=50)	1000.0	0.7089999999999989	0.028200797908293447
P(42<=T<=48)	1000.0	0.4499999999999999	0.030887260143834475

The results indicate that on average it takes about 47.8 days to complete the network. In the following section, we explore generalizing the execution of Monte Carlo experiments.

3.8. Monte-Carlo Experiments

Because running Monte-Carlo experiments is very common the KSL has provided some classes to support simple experimentation on Monte-Carlo

problems within the `ksl.utilities.mcintegration` package. Figure 3.7 illustrates the classes within the package.

A Monte-Carlo experiment is conceptualized as a two-step process: 1) an initial pilot sample, and 2) the sampling to meet the desired half-width criteria. The initial pilot sample determines based on the equations of Section 3.3.2 how many samples are needed to meet the criteria. Then, the remaining samples are executed until a maximum number of samples is reached or the half-width criteria is met.

The sampling is performed in the form of micro and macro replications. A micro replication observes the desired quantity for a pre-determined sample size. Then, the micro replications are repeated for a pre-determined overall number of macro replications. Let's use some notation to represent these concepts.

Let r be the number of micro replications and let n be the number of macro replications. Let Y_{ij} be the i^{th} observation of the r micro replications and let j be the j^{th} observation of the n macro replications. Let $\bar{Y}_{\cdot j} = \frac{1}{r}\sum_{i=1}^{r} Y_{ij}$ be the sample average for the j^{th} macro replication. Let k be the sample size of the initial set of macro replications. Thus, they form a random sample of size k, as follows:

$$(\bar{Y}_{\cdot 1}, \bar{Y}_{\cdot 2}, \ldots, \bar{Y}_{\cdot k})$$

From this sample, the number of macro replications needed to meet the criteria is determined using Equation 3.8. The reason that this sample is formed from the r micro replications is to help ensure that the observations $\bar{Y}_{\cdot j}$ are approximately normally distributed. According to the Central Limit Theorem, sample averages tend to be normally distributed as the number of observations within the sample increase towards infinity.

The simulation is performed in two loops: an outer loop called the macro replications and an inner loop called the micro replications. The user specifies a desired (half-width) error bound (ϵ), an initial sample size (k),

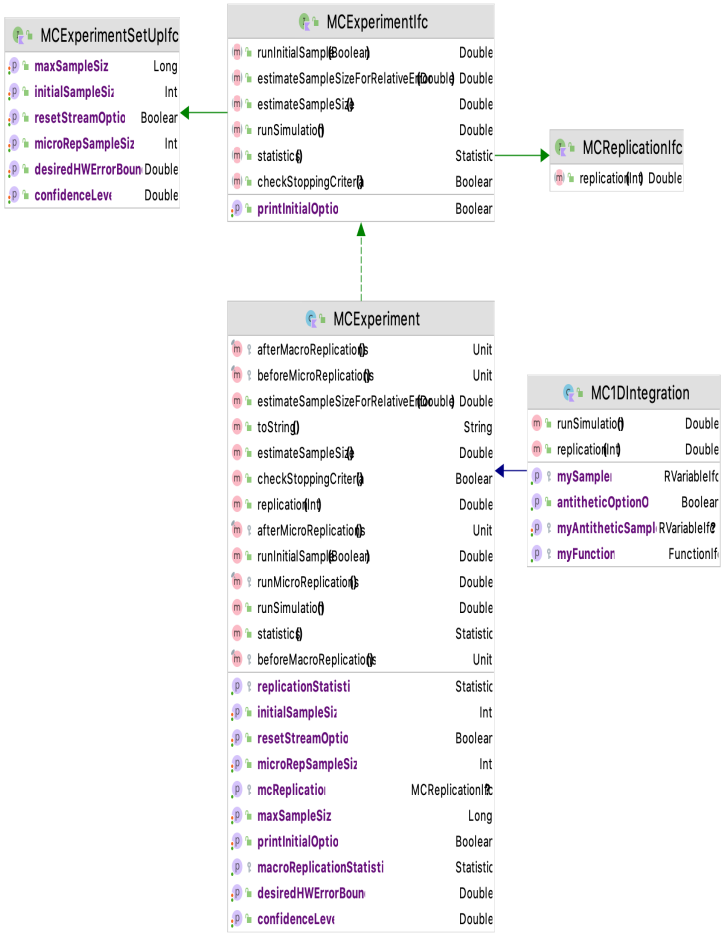


Figure 3.7.: Classes in ksl.utilities.mcintegration

and a maximum sample size limit (M) for the macro replications. The initial sample size is used to generate a pilot sample from which an estimate of the number of samples needed to meet the half-width criteria. Let's call the estimated sample size, m. If $m > k$, then an additional $(m - k)$ samples will be taken or until the error criteria is met or the maximum number of samples M is reached. Thus, if $m > M$, and the error criterion is not met during the macro replications a total of M observations will be observed. Thus, the total number of macro replications will not exceed M. If the error criteria is met before M is reached, the number of macro replications n will be somewhere between k and M. For each of the n, macro replications, a set of micro replications will be executed. Let r be the number of micro replications. The micro replications represent the evaluation of r observations of the Monte-Carlo evaluation. Thus, the total number of observations will be $n \times r$.

By default, the number of macro replications should be relatively small and the number of micro replications large. Specific settings will be problem dependent. The default initial sample size, k is 30, with a maximum number of macro replications of $M = 10000$. The default half-width error bound is 0.001. The default setting of the number of micro replications, r, is 100. Again, these are all adjustable by the user via the properties of the `MCExperiment` class. The initial pilot sample is executed automatically as part of the overall sampling; however, using the `runInitialSample()` method, the user can just execute the initial sample and review the results before setting up the overall sampling. In the following code, we re-implement the news vendor problem using the `MCExperiment` class.

To use the `MCExperiment` class the user can either subclass `MCExperiment` or provide a function that represents a replication of an individual observation. This can be done by using the `MCReplicationIfc` functional interface.

```
fun interface MCReplicationIfc {
    /**
```

```
     * @param j the current replication number. Could be used
 ↪  to implement more advanced
     * sampling that needs the current replication number.
 ↪  For example, antithetic sampling.
     */
    fun replication(j: Int): Double
}
```

A Kotlin functional interface allows the function to be supplied easily as a parameter of a function. An interface with only one abstract method is called a functional interface, or a Single Abstract Method (SAM) interface. Using the functional programming constructs of Kotlin the user can supply a lambda expression as the function, implement the interface, or reference a class or object method that has the same functional signature. In the example presented here, for simplicity and clarity, we implement the interface.

```
class NewsVendor(var demand: RVariableIfc) : MCReplicationIfc
 ↪  {
    var orderQty = 30.0 // order qty
        set(value) {
            require(value > 0)
            field = value
        }
    var salesPrice = 0.25 //sales price
        set(value) {
            require(value > 0)
            field = value
        }
    var unitCost = 0.15 // unit cost
        set(value) {
            require(value > 0)
            field = value
```

```
        }
    var salvageValue = 0.02 //salvage value
        set(value) {
            require(value > 0)
            field = value
        }

    override fun replication(j: Int): Double {
        val d = demand.value
        val amtSold = minOf(d, orderQty)
        val amtLeft = maxOf(0.0, orderQty - d)
        return salesPrice * amtSold + salvageValue * amtLeft
         ↪  - unitCost * orderQty
    }

}
```

Notice the function, `replication` which implements computation of the profit equation for the news vendor problem. The key input parameters of the problem have also been implemented as properties of the class so that different problem instances can be easily created. The user of the class must provide a random variable to represent the demand. The running of the experiment is implemented in the following code.

Example 3.13 (News Vendor Via MCExperiment)**.** This example illustrates how to use the `MCExperiment` class to run a Monte-Carlo experiment for the news vendor problem.

```
fun main() {
    val values = doubleArrayOf(5.0, 10.0, 40.0, 45.0, 50.0,
     ↪  55.0, 60.0)
    val cdf = doubleArrayOf(0.1, 0.3, 0.6, 0.8, 0.9, 0.95,
     ↪  1.0)
    val dCDF = DEmpiricalRV(values, cdf, streamNum = 3)
    val nv = NewsVendor(dCDF)
    val exp = MCExperiment(nv)
    exp.desiredHWErrorBound = 0.01
    exp.runSimulation()
    println(exp)
}
```

This code creates an instance of the news vendor problem and supplies the same demand distribution used in the previous section. The instance of `NewsVendor`, which implements the `MCReplicationIfc` interface is supplied to the `MCExperiment` as a constructor parameter. Then the desired half-width bound is set and the simulation executed.

```
Monte Carlo Simulation Results
initial Sample Size = 30
max Sample Size = 10000
reset Stream OptionOn = false
Estimated sample size needed to meet criteria = 2045.0
desired half-width error bound = 0.01
actual half-width = 0.00999980532316225
error gap (hw - bound) = -1.946768377510122E-7
------------------------------------------------------------
The half-width criteria was met!
------------------------------------------------------------
```

```
**** Sampling results ****
Number of macro replications executed = 2047.0
Number of micro replications per macro replication = 100
Total number of observations = 204700.0
ID 2
Name Statistic_1
Number 2047.0
Average 1.5049719101123593
Standard Deviation 0.23069885711869198
Standard Error 0.005099017725643279
Half-width 0.00999980532316225
Confidence Level 0.95
Confidence Interval [1.494972104789197, 1.5149717154355216]
```

As we can see from the results, the desired half-width criteria of 0.01 was met based on 2047 macro replications.

The KSL also provides a class called `MC1DIntegration` that is a subclass of `MCExperiment` which facilitates the evaluation of 1-dimensional integrals as previously illustrated in Section 3.2. For that situation a function and a sampling distribution can be provided.

- Let $f(x)$ be the probability distribution for the random variable supplied by the sampler.
- Let $g(x)$ be the function that needs to be integrated.
- Let $h(x)$ be a factorization of $g(x)$ such that $g(x) = h(x) * f(x)$, that is $h(x) = g(x)/f(x)$

The following code illustrates the use of the `MC1DIntegration` class to evaluate the following integral:

$$\theta = \int_0^{\pi} \sin(x) \mathrm{d}x$$

With $g(x) = \sin(x)$, we need to specify a sampling distribution, $f(x)$ over the range of $[0, \pi]$. By choosing, $f(x)$ as the uniform distribution over $[0, \pi]$, we have that

$$f_X(x) = \begin{cases} \frac{1}{\pi} & 0 \leq x \leq \pi \\ 0 & \text{otherwise} \end{cases}$$

Thus,

$$h(x) = \begin{cases} \pi \sin(x) & 0 \leq x \leq \pi \\ 0 & \text{otherwise} \end{cases}$$

In the following code, notice that we multiply by π in the functional evaluation to account for the range of sampling.

Example 3.14 (1-D Integration via MCExperiment)**.** This example illustrates how to use the `MCExperiment` class to run a Monte-Carlo experiment on a one-dimensional integration problem.

```
fun main() {
    class SinFunc : FunctionIfc {
        override fun f(x: Double): Double {
            return Math.PI * sin(x)
        }
    }

    val f = SinFunc()
```

```
    val mc = MC1DIntegration(f, UniformRV(0.0, Math.PI,
     ↪  streamNum = 3))
    println()
    mc.runSimulation()
    println(mc)
}
```

The results based on the default settings do not meet the desired half-width criteria. We leave it as an exercise for the reader to further explore this problem.

```
Monte Carlo Simulation Results
initial Sample Size = 30
max Sample Size = 10000
reset Stream OptionOn = false
Estimated sample size needed to meet criteria = 63129.0
desired half-width error bound = 0.001
actual half-width = 0.0025130156865938546
error gap (hw - bound) = 0.0015130156865938546
------------------------------------------------------------
The half-width criteria was not met!
The user should consider one of the following:
1. increase the desired error bound using desiredHWErrorBound
2. increase the number of macro replications using maxSampleSize
2. increase the number of micro replications using microRepSampleSize
------------------------------------------------------------
**** Sampling results ****
Number of macro replications executed = 10000.0
Number of micro replications per macro replication = 100
Total number of observations = 1000000.0
```

```
ID 2
Name Statistic_1
Number 10000.0
Average 1.9998788102129432
Standard Deviation 0.09754281939519215
Standard Error 9.754281939519214E-4
Half-width 0.0025130156865938546
Confidence Level 0.99
Confidence Interval [1.9973657945263494, 2.002391825899537]
```

3.9. Summary

In this chapter, we learned how to use KSL constructs to collect statistics within Monte Carlo models. We use the KSL's random variate generation methods and the functionality for collecting and reporting statistics to easily setup and perform Monte Carlo experiments. The quantities to be estimated from the Monte Carlo experiments are random variables. Thus, the Monte Carlo procedure generates a random sample from the experiment. The random sample from the experiment provides estimates such as the sample average and other statistical quantities. For example, to estimate a probability, we used an indicator variable and computed statistics on the observations of the indicator variable.

When executing a Monte Carlo experiment, we should report the sampling error or a confidence interval on the quantities estimated from the Monte Carlo experiment. In addition, because we are performing an experiment, we should plan the sampling by understanding the sampling error and if possible, pre-plan the required sample size. Besides the random variate generation and statistical classes within the KSL, we can use the KSL's `MCExperiment` class to execute Monte Carlo experiments.

The following new KSL constructs were discussed in this chapter:

`Statistic`: A class that collects summary statistics (e.g. the average, variance, minimum, maximum, and confidence interval) on observations.

`Histogram`: A class that tabulates observations into the intervals defined by a set of break points.

`IntegerFrequency`: A class that tabulates the frequency of integer valued observations.

`StateFrequency`: A class that tabulates the frequency of visits to, and transitions between, a set of states.

`BatchStatistic`: A class that collects statistics on observations by forming batches and computing statistics on the batch means.

`StatisticReporter`: A class that produces summary reports, such as half-width summary tables, for a set of statistics.

`MCExperiment`: A class that performs a Monte Carlo experiment, including the determination of the number of samples needed to meet a desired error bound.

`MCReplicationIfc`: An interface that defines the function that produces one replication (sample) of a Monte Carlo experiment.

`MC1DIntegration`: A class that estimates the integral of a one dimensional function via Monte Carlo sampling.

In addition, we explored how to develop models for which time is not a significant factor. In the case of the news vendor problem, where we simulated each day's demand, time advanced at regular intervals. In the case of the area estimation problem, time was not a factor in the simulation. These types of simulation experiments are often termed static. In the next chapter, we begin our exploration of simulation experiments where time is an integral component in driving the behavior of the system. In addition, we will see that time will not necessarily advance at regular intervals (e.g. hour 1, hour 2, etc.). This will be the focus of the rest of the book.

3.10. Exercises

Exercise 3.1. Compute the required sample size necessary to ensure a 95% confidence interval with a half-width of no larger than 30 minutes for the total time to produce a part. Given the half-width of the system time after 10 runs is 47.25, compute the sample size using the half-width ratio method. Round the answer to the next highest integer.

Exercise 3.2. Compute the required sample size necessary to ensure a 95% confidence interval with a half-width of no larger than 30 minutes for the total time to produce a part. Given the half-width of the system time after 10 runs is 47.25. Find the approximate number of replications needed in order to have a 99% confidence interval that is within plus or minus 2 minutes of the true mean system time using the half-width ratio method.

Exercise 3.3. Assume that the following results represent the summary statistics for a pilot run of 10 replications from a simulation for the system time in minutes. $\overline{x} = 78.2658 \pm 9.39$ with confidence 95%. Find the approximate number of *additional* replications in order to have a 99% confidence interval that is within plus or minus 2 minutes of the true mean system time.

Exercise 3.4. Suppose $n = 10$ observations were collected on the time spent in a manufacturing system for a part. The analysis determined a 95% confidence interval for the mean system time of $[18.595, 32.421]$.

a. Find the approximate number of samples needed to have a 95% confidence interval that is within plus or minus 2 minutes of the true mean system time.
b. Find the approximate number of samples needed to have a 99% confidence interval that is within plus or minus 1 minute of the true mean system time.

Exercise 3.5. Suppose a pilot run of a simulation model estimated that the average waiting time for a customer during the day was 11.485 minutes based on an initial sample size of 15 replications with a 95% confidence interval half-width of 1.04. Using the half-width ratio sample size determination techniques, recommend a sample size to be 95% confident that you are within $\pm$ 0.10 of the true mean waiting time in the queue. The half-width ratio method requires a sample size of what amount?

Exercise 3.6. Assume that the following table represents the summary statistics for a pilot run of 10 replications from a simulation.

Simulation Statistics	NPV	P(NPV $<$ 0)
Sample Average	83.54	0.4
Standard Deviation	39.6	
Count	10	10

a. Find the approximate number of additional replications to execute in order to have a 99% confidence interval that is within plus or minus 20 dollars of the true mean net present value using the normal approximation method.

b. Find the number of replications necessary to be 99% confident that you have an interval within plus or minus 2% of the true probability of negative present value.

Exercise 3.7. The service time for an automated storage and retrieval system has a shifted exponential distribution. It is known that it takes a minimum of 15 seconds for any retrieval. The time beyond the minimum is exponentially distributed with a mean of 45 seconds. Setup a model that will generate 20 observations of the retrieval times. Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations. Use stream 1 for your sampling.

Exercise 3.8. The time to failure for a computer printer fan has a Weibull distribution with shape parameter $\alpha = 2$ and scale parameter $\beta = 3$. Setup a model that will generate 50 observations of the failure times. Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations. Use stream 1 for your sampling.

Exercise 3.9. The time to failure for a computer printer fan has a Weibull distribution with shape parameter $\alpha = 2$ and scale parameter $\beta = 3$. Testing has indicated that the distribution is limited to the range from 1.5 to 4.5. Set up a model to generate 100 observations from this truncated distribution. Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations. Use stream 1 for your sampling.

Exercise 3.10. The interest rate for a capital project is unknown. An accountant has estimated that the minimum interest rate will between 2% and 5% within the next year. The accountant believes that any interest rate in this range is equally likely. You are tasked with generating interest rates for a cash flow analysis of the project. Set up a model to generate 100 observations of the interest rate values for the capital project analysis. Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations. Use stream 1 for your sampling.

Exercise 3.11. Develop a model to generate 30 observations from the following probability density function:

$$f(x) = \begin{cases} \dfrac{3x^2}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations. Use stream 1 for your sampling.

Exercise 3.12. Suppose that the service time for a patient consists of two distributions. There is a 25% chance that the service time is uniformly distributed with minimum of 20 minutes and a maximum of 25 minutes, and a 75% chance that the time is distributed according to a Weibull distribution with shape of 2 and a scale of 4.5. Use stream 1 for your sampling.

Setup a model to generate 100 observations of the service time. Compute the theoretical expected value of the distribution. Estimate the expected value of the distribution and compute a 95% confidence interval on the expected value. Did your confidence interval contain the theoretical expected value of the distribution?

Exercise 3.13. Suppose that X is a random variable with a $N(\mu = 2, \sigma = 1.5)$ normal distribution. Generate 100 observations of X using a simulation model. Use stream 1 for your sampling.

- Estimate the mean from your observations. Report a 95% confidence interval for your point estimate.
- Estimate the variance from your observations. Report a 95% confidence interval for your point estimate.
- Estimate the $P(X > 3)$ from your observations. Report a 95% confidence interval for your point estimate.

Exercise 3.14. Samples of 20 parts from a metal grinding process are selected every hour. Typically 2% of the parts need rework. Let X denote the number of parts in the sample of 20 that require rework. A process problem is suspected if X exceeds its mean by more than 1 standard deviation. Simulate 1000 hours of the process, i.e. 1000 samples of size 20, and estimate the chance that X exceeds its expected value by more than 1 standard deviation. Use stream 1 for your sampling.

Exercise 3.15. Consider the following discrete distribution of the random variable X whose probability mass function is $p(x)$. Setup a model to generate 30 observations of the random variable X. Use stream 1 for your sampling. Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations.

x	0	1	2	3	4
$p(x)$	0.3	0.2	0.2	0.1	0.2

Exercise 3.16. The demand for parts at a repair bench per day can be described by the following discrete probability mass function. Setup a model to generate 30 observations of the demand. Use stream 1 for your sampling. Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations.

Demand	0	1	2
Probability	0.3	0.2	0.5

Exercise 3.17. Use the Monte Carlo method estimate the following integral with 95% confidence to within ± 0.01. Use stream 1 for your sampling.

$$\int_1^4 \left(\sqrt{x} + \frac{1}{2\sqrt{x}}\right) \mathrm{d}x$$

Exercise 3.18. Use the Monte Carlo method estimate the following integral with 99% confidence to within ± 0.01. Use stream 1 for your sampling.

$$\int_0^{\pi} (\sin(x) - 8x^2)\,\mathrm{d}x$$

Exercise 3.19. Use the Monte Carlo method estimate the following integral with 99% confidence to within ± 0.01. Use stream 1 for your sampling.

$$\theta = \int_0^1 \int_0^1 (4x^2 y + y^2)\,\mathrm{d}x\mathrm{d}y$$

Exercise 3.20. A firm is trying to decide whether or not it should purchase a new scale to check a package filling line in the plant. The scale would allow for better control over the filling operation and result in less overfilling. It is known for certain that the scale costs \$800 initially. The annual cost has been estimated to be normally distributed with a mean of \$100 and a standard deviation of \$10 (stream 1). The extra savings associated with better control of the filling process has been estimated to be normally distributed with a mean of \$300 and a standard deviation of \$50 (stream 2). The salvage value has been estimated to be uniformly distributed between \$90 and \$100 (stream 3). The useful life of the scale varies according to the amount of usage of the scale. The manufacturing has estimated that the useful life can vary between 4 to 7 years with the chances given in the following table (stream 4).

years	4	5	6	7
f(years)	0.3	0.4	0.1	0.2
F(years)	0.3	0.7	0.8	1.0

The interest rate has been varying recently and the firm is unsure of the rate for performing the analysis. To be safe, they have decided that the interest rate should be modeled as a beta random variable over the range from 6 to 9 percent with alpha = 5.0 and beta = 1.5 (stream 5). Given all the uncertain elements in the situation, they have decided to perform a simulation analysis in order to assess the expected present value of the decision and the chance that the decision has a negative return.

We desire to be 95% confident that our estimate of the true expected present value is within ± 10 dollars. Develop a model for this situation.

Exercise 3.21. A firm is trying to decide whether or not to invest in two proposals A and B that have the net cash flows shown in the following table, where $N(\mu, \sigma)$ represents that the cash flow value comes from a normal distribution with the provided mean and standard deviation.

End of Year	0	1	2	3	4
A	$N(-150, 10)$	$N(475, 10)$	$N(275, 10)$	$N(275, 20)$	$N(250, 40)$
B	$N(-100, 5)$	$N(350, 10)$	$N(350, 10)$	$N(275, 20)$	$N(275, 30)$

The interest rate has been varying recently and the firm is unsure of the rate for performing the analysis. To be safe, they have decided that the interest rate should be modeled as a beta random variable over the range from 2 to 7 percent with $\alpha_1 = 4.0$ and $\alpha_2 = 1.2$. Given all the uncertain elements in the situation, they have decided to perform a simulation analysis in order to assess the situation. Use the KSL to answer the following questions:

- Compare the expected present worth of the two alternatives. Estimate the probability that alternative A has a higher present worth than alternative B.
- Determine the number of samples needed to be 95% confidence that you have estimated the $P[PW(A) > PW(B)]$ to within $\pm$ 0.10.

Use stream 1 for the cash flows and stream 2 for the interest rate.

Exercise 3.22. A U-shaped component is to be formed from the three parts A, B, and C. The picture is shown in the figure below. The length of A is lognormally distributed with a mean of 20 millimeters and a standard deviation of 0.2 millimeter (stream 1). The thickness of parts B (stream 2) and C (stream 3) is uniformly distributed with a minimum of 4.98 millimeters and a maximum of 5.02 millimeters. Assume all dimensions are independent.

Develop a model to estimate the probability that the gap D is less than 10.1 millimeters with 95% confidence to within plus or minus 0.01.

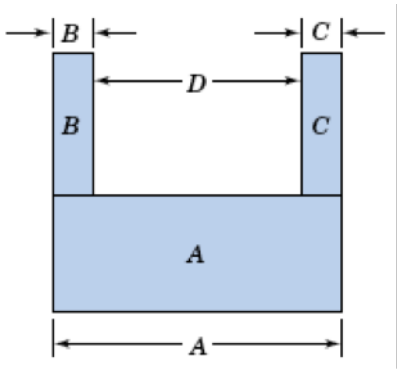


Figure 3.8.: U-Shaped Component

Exercise 3.23. Shipments can be transported by rail or trucks between New York and Los Angeles. Both modes of transport go through the city of St. Louis. The mean travel time and standard deviations between the major cities for each mode of transportation are shown in the following figure.

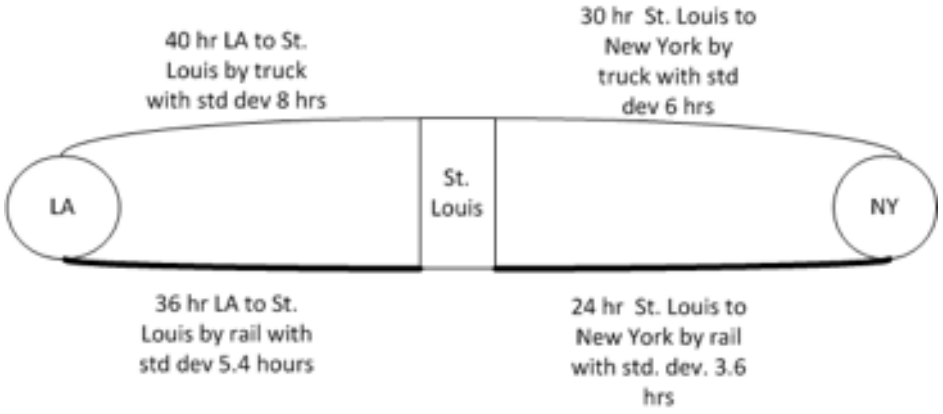


Figure 3.9.: Truck Paths

Assume that the travel times (in either direction) are lognormally distributed as shown in the figure. For example, the time from NY to St. Louis (or St. Louis to NY) by truck is 30 hours with a standard deviation of 6 hours. In addition, assume that the transfer time in hours in St. Louis is triangularly distributed with parameters (8, 10, 12) for trucks (truck to truck). The transfer time in hours involving rail is triangularly distributed with parameters (13, 15, 17) for rail (rail to rail, rail to truck, truck to rail). We are interested in determining the shortest total shipment time combination from NY to LA. Develop a simulation for this problem. Use stream 1 for the truck time and stream 2 for the rail time between New York and St. Louis, stream 3 for the truck time and stream 4 for the rail time between St. Louis and Los Angeles, stream 5 for the truck-to-truck

transfer time and stream 6 for the transfer times involving rail. Base your analysis on 10,000 samples.

a. How many shipment combinations are there?

b. Write an expression for the total shipment time of the truck only combination.

c. We are interested in estimating the average shipment time for each shipment combination and the probability that the shipment combination will be able to deliver the shipment within 85 hours.

d. Estimate the probability that a shipping combination will be the shortest.

Exercise 3.24. A firm produces YBox gaming stations for the consumer market. Their profit function is:

$$\text{Profit} = (\text{unit price} - \text{unit cost}) \times (\text{quantity sold}) - \text{fixed costs}$$

Suppose that the unit price is \$200 per gaming station, and that the other variables have the following probability distributions:

Unit Cost	80	90	100	110
Probability	0.20	0.40	0.30	0.10
Quantity Sold	1000	2000	3000	
Probability	0.10	0.60	0.30	
Fixed Cost	150000	200000	250000	
Probability	0.40	0.30	0.30	

Use a simulation model to generate 1000 observations of the profit. Use stream 1 for the unit cost distribution, stream 2 for the quantity sold distribution, and stream 3 for the fixed cost distribution.

Estimate the mean profit from your sample and compute a 95% confidence interval for the mean profit. Estimate the probability that the profit will be positive.

Exercise 3.25. T. Wilson operates a sports magazine stand before each game. He can buy each magazine for 33 cents and can sell each magazine for 50 cents. Magazines not sold at the end of the game are sold for scrap for 5 cents each. Magazines can only be purchased in bundles of 10. Thus, he can buy 10, 20, and so on magazines prior to the game to stock his stand. The lost revenue for not meeting demand is 17 cents for each magazine demanded that could not be provided. Mr. Wilson's profit is as follows:

$$\begin{aligned}\text{Profit} = &(\text{revenue from sales}) - (\text{cost of magazines}) \\ &- (\text{lost profit from excess demand}) \\ &+ (\text{salvage value from sale of scrap magazines})\end{aligned}$$

Not all game days are the same in terms of potential demand. The type of day depends on a number of factors including the current standings, the opponent, and whether or not there are other special events planned for the game day weekend. There are three types of game days demand: high, medium, low. The type of day has a probability distribution associated with it.

Type of Day	High	Medium	Low
Probability	0.35	0.45	0.20

The amount of demand for magazines then depends on the type of day according to the following distributions:

Demand	PMF	CDF	PMF	CDF	PMF	CDF
40	0.03	0.03	0.1	0.1	0.44	0.44
50	0.05	0.08	0.18	0.28	0.22	0.66
60	0.15	0.23	0.4	0.68	0.16	0.82
70	0.2	0.43	0.2	0.88	0.12	0.94
80	0.35	0.78	0.08	0.96	0.06	1.0
90	0.15	0.93	0.04	1.0		
100	0.07	1.0				

Let Q be the number of units of magazines purchased (quantity on hand) to setup the stand. Let D represent the demand for the game day. If $D > Q$, Mr. Wilson sells only Q and will have lost sales of $D - Q$. If $D < Q$, Mr. Wilson sells only D and will have scrap of $Q - D$. Assume that he has determined that $Q = 50$.

Make sure that you can estimate the average profit and the probability that the profit is greater than zero for Mr. Wilson. Develop a model to estimate the average profit based on 100 observations. Use stream 1 for the type of day distribution and streams 2, 3, 4 for the demand for each type of day.

Exercise 3.26. The time for an automated storage and retrieval system in a warehouse to locate a part consists of three movements. Let X be the time to travel to the correct aisle. Let Y be the time to travel to the correct location along the aisle. And let Z be the time to travel up to the correct location on the shelves. Assume that the distributions of X, Y, and Z are as follows:

$X \sim$ lognormal with mean 20 and standard deviation 10 seconds (stream 1)

$Y \sim$ uniform with minimum 10 and maximum 15 seconds (stream 2)

$Z \sim$ uniform with minimum of 5 and a maximum of 10 seconds (stream 3)

Develop a model that can estimate the average total time that it takes to locate a part and can estimate the probability that the time to locate a part exceeds 60 seconds. Base your analysis on 1000 observations.

Exercise 3.27. Lead-time demand may occur in an inventory system when the lead-time is other than instantaneous. The lead-time is the time from the placement of an order until the order is received. The lead-time is a random variable. During the lead-time, demand also occurs at random. Lead-time demand is thus a random variable defined as the sum of the demands during the lead-time, or LDT $= \sum_{i=1}^{T} D_i$ where i is the time period of the lead-time and T is the lead-time. The distribution of lead-time demand is determined by simulating many cycles of lead-time and the demands that occur during the lead-time to get many realizations of the random variable LDT. Notice that LDT is the *convolution* of a random number of random demands. Suppose that the daily demand for an item is given by the following probability mass function (stream 1):

Daily Demand (items)	4	5	6	7	8
Probability	0.10	0.30	0.35	0.10	0.15

The lead-time is the number of days from placing an order until the firm receives the order from the supplier.

Assume that the lead-time is a constant 10 days. Develop a model to simulate 1000 instances of LDT. Report the summary statistics for the 1000 observations. Estimate the chance that LDT is greater than or equal to 60. Report a 95% confidence interval on your estimate.

Assume that the lead-time has a shifted geometric distribution with probability parameter equal to 0.2 (stream 2). Use a model to simulate 1000 instances of LDT. Report the summary statistics for the 1000 observations. Estimate the chance that LDT is greater than or equal to 60. Report a 95% confidence interval on your estimate.

Exercise 3.28. If $Z \sim N(0,1)$, and $Y = \sum_{i=1}^{k} Z_i^2$ then $Y \sim \chi_k^2$, where χ_k^2 is a chi-squared random variable with k degrees of freedom. Setup a model to generate 50 χ_5^2 random variates. Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations. Use stream 1 for your sampling.

Exercise 3.29. Setup a model that will generate 30 observations from the following probability density function using the Acceptance-Rejection algorithm for generating random variates. Use stream 1 for your sampling.

$$f(x) = \begin{cases} \dfrac{3x^2}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Exercise 3.30. This procedure is due to (Box and Muller 1958). Let U_1 and U_2 be two independent uniform (0,1) random variables and define:

$$X_1 = \cos(2\pi U_2)\sqrt{-2ln(U_1)}$$
$$X_2 = \sin(2\pi U_2)\sqrt{-2ln(U_1)}$$

It can be shown that X_1 and X_2 will be independent standard normal random variables, i.e. $N(0,1)$. Use this to implement the Box and Muller algorithm for generating normal random variables. Generate 1000 $N(\mu = 2, \sigma = 0.75)$ random variables via this method. Report the minimum, maximum, sample average, and 95% confidence interval half-width of the observations. Assign stream 1 to U_1 and stream 2 U_2.

Exercise 3.31. In the popular English game of Hazard, a player must first determine which of the five numbers from 5-9 will be the "main" point. The player does this by rolling two dice until such time as the point sum equals one of these five numbers. The player then rolls again. He/she wins if the point sum of this roll corresponds with the "main" point as follows: main 5 corresponds with a point sum of 5, main 6 corresponds with a point sum of 6 or 7, main 7 corresponds with sum 7 or 11, main 8 corresponds with sum 8 or 12, and main 9 corresponds with sum 9. The player loses if, having taken on a main point of 5 or 9, he/she then rolls a sum of 11 or 12, or by rolling a sum of 11 against a main of 6 or 8, or by rolling a sum of 12 against a main of 7. In every other situation, the sum thrown becomes the player's "chance" point. From here on the player rolls two dice until either the "chance" point (player wins) or the "main" point (player loses) reappears.

Using the KSL verify via simulation that the probability of the player winning is equal to 0.5228, where the main and the chance points contribute 0.1910 and 0.3318, respectively, to the probability of winning. Estimate the probabilities to within 4 decimal places of accuracy. Use stream 1 for your sampling.

4. Introduction to Discrete Event Modeling

Learning Objectives

- To be able to recognize and define the characteristics of a discrete-event dynamic system (DEDS)
- To be able to explain how time evolves in a DEDS
- To be able to develop and read an activity flow diagram
- To be able to create, run, and examine the results of a KSL model of a simple DEDS

In Chapter 3, we explored how to develop models in the KSL for which time is not a significant factor. In the case of the news vendor problem, where we simulated each day's demand, time advanced at regular intervals. In the case of the area estimation problem, time was not a factor in the simulation. These types of simulations are often termed static simulations. In the next section, we begin our exploration of simulation where time is an integral component in driving the behavior of the system. In addition, we will see that time will not necessarily advance at regular intervals (e.g. hour 1, hour 2, etc.). This will be the focus of the rest of the textbook.

In this chapter, we explore the KSL simulation software platform for developing and executing simulation models using the event-view. We will begin our study of the major emphasis of this textbook: modeling discrete-event dynamic systems. Recall that a discrete-event dynamic

system (DEDS) is a system that evolves dynamically through time. This chapter will introduce how time evolves for DEDSs and illustrate how the KSL can be used to develop models for simple discrete-event systems.

We can think of a system as a set of inter-related objects that work together to accomplish a common objective. The objects within a system are the concepts, abstractions, things, and processes with definable boundaries and unique identity given our modeling perspective. Within the traditional simulation parlance, objects of particular interest within a system have often been called entities; however, this text takes a broader view to include other modeling elements of interest within the system (e.g. resources).

A discrete event system is a system in which the state of the system changes only at discrete points in time. There are essentially two fundamental viewpoints for modeling a discrete event system within simulation: the event view and the process view. These views are simply different representations for the same system. In the event view, the system and its elements are conceptualized as reacting to events. In the process view, the movement of entities through their processes implies the events within the system in the system. This chapter focuses on the event view.

The objects within the system change state because events occur within the system. In addition, the changes in state for an object may cause additional events to occur, affecting other objects within the system. Through the propagation of events and object state changes, the entire system state evolves through time. The state of the system is essentially the collection of states associated with all the objects in the system. The notion of modeling what happens to a system at particular events in time is the basis of discrete event modeling.

> **Note**
>
> This chapter provides a series of example Kotlin code that illustrates the use of KSL constructs for discrete-event dynamic systems. The full source code of the examples can be found in the accompanying

`KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

4.1. Discrete-Event Dynamic Systems

(Nelson 1995) states that the event-view "defines system events by describing what happens to the system as it encounters entities". In addition, (Nelson 1995) states that the process view "implies system events by describing what happens to an entity as it encounters the system". One can consider the event-view as taking the perspective of the system and the process-view as taking the perspective of the entity. The process-view describes the life cycles of objects within the system. Because of the way in which the process-view has been implemented through a network transaction flow approach within many simulation packages, the term entity has taken on the connotation of objects that flow or move within the system. To avoid confusion, the more general term, object, will be used to encompass non-flowing objects as well as flowing objects within the system.

To understand the event-view, the concepts of event, activity, state, and process must be clearly defined. (Rumbaugh et al. 1991) defines state and its relationship to events and activities as "an abstraction of the attribute values and links of an object. Sets of values are grouped together into a state according to properties that affect the gross behavior of the object. A state corresponds to the interval between two events. A state has duration; it occupies an interval of time. A state is often associated with an activity that takes time to complete or with the value of an object satisfying some condition. In defining states, we ignore those attributes that do not affect the behavior of the object and we lump together in a single state all combinations of attribute values and links that have the same response to events." Thus, the state of an object should entail only those aspects that are required for the modeling.

An event is something that happens at an instant in time that corresponds to a change in object state. An event is a one-way transmission of information from one object to another that causes an action resulting in a change in state. An event is said to be a scheduled event (or timed, determined, bounded) if its occurrence can be expressed as a function of system time and can thus be scheduled in time. For example, suppose you have a doctor's appointment at 11 am. The beginning of the appointment represents a scheduled event. An event is said to be a conditional event if it is dependent upon the outcome of certain conditions that cannot be predicted with certainty in advance (e.g. the availability of a server). Conditional events are sometimes called unbounded or contingent. If a scheduled event ends an activity, then the activity is said to be a scheduled or timed activity; otherwise, it is said to be a conditional activity.

Associated with an event is an action that is an instantaneous operation, i.e. it takes zero simulated time to complete. In contrast, an activity is an operation that takes simulated time to complete. An activity can be associated with the state of an object over an interval. Activities are defined by the occurrence of two events that represent the activity's beginning time and ending time and mark the entrance and exit of the state associated with the activity. In the simulation literature, it is common to refer to activities that take a specified duration of time as simply activities. To be more specific, it is useful to classify activities of a specified duration as timed activities. Getting your haircut is an example of a timed activity. There is a clear beginning and a clear ending for the activity. An activity that encompasses an interval of time that is unspecified or of indefinite length is called a conditional activity. An example of a conditional activity is a customer waiting in a queue for the barber to become available. The length of the activity depends on when the barber becomes available.

In the next section, we begin our exploration of simulation where time is an integral component in driving the behavior of the system. In addition, we will see that time will not necessarily advance at regular intervals (e.g. hour 1, hour 2, etc.). As discussed, an event occurs at a specific point in time, thus we need to understand how the simulation time clock works.

4.2. How the Discrete-Event Clock Works

This section introduces the concept of how time evolves in a discrete event simulation. This topic will also be revisited in future chapters after you have developed a deeper understanding for many of the underlying elements of simulation modeling.

In discrete-event dynamic systems, an event is something that happens at an instant in time which corresponds to a change in system state. An event can be conceptualized as a transmission of information that causes an action resulting in a change in system state. Let's consider a simple bank which has two tellers that serve customers from a single waiting line. In this situation, the system of interest is the bank tellers (whether they are idle or busy) and any customers waiting in the line. Assume that the bank opens at 9 am, which can be used to designate time zero for the simulation. It should be clear that if the bank does not have any customers arrive during the day that the state of the bank will not change. In addition, when a customer arrives, the number of customers in the bank will increase by one. In other words, an arrival of a customer will change the state of the bank. Thus, the arrival of a customer can be considered an event.

Table 4.1.: Customer Time of Arrivals

Customer	Time of arrival	Time between arrival
1	2	2
2	5	3
3	7	2
4	15	8

Figure 4.1 illustrates a time line of customer arrival events. The values for the arrival times in Table 4.1 have been rounded to whole numbers, but in general the arrival times can be any real valued numbers greater than zero. According to the figure, the first customer arrives at time two and

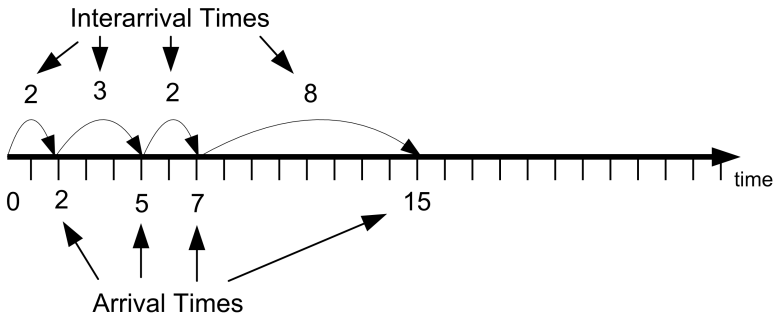


Figure 4.1.: Customer Arrival Process

there is an elapse of three minutes before customer 2 arrives at time five. From the discrete-event perspective, *nothing* is happening in the bank from time $[0, 2)$; however, at time 2, an arrival event occurs and the subsequent actions associated with this event need to be accounted for with respect to the state of the system. An event denotes an instance of time that changes the state of the system. Since an event causes actions that result in a change of system state, it is natural to ask: What are the actions that occur when a customer arrives to the bank?

- The customer enters the waiting line.
- If there is an available teller, the customer will immediately exit the line and the available teller will begin to provide service.
- If there are no tellers available, the customer will remain waiting in the line until a teller becomes available.

Now, consider the arrival of the first customer. Since the bank opens at 9 am with no customers and all the tellers idle, the first customer will

enter and immediately exit the queue at time 9:02 am (or time 2) and then begin service. After the customer completes service, the customer will exit the bank. When a customer completes service (and departs the bank) the number of customers in the bank will decrease by one. It should be clear that some actions need to occur when a customer completes service. These actions correspond to the second event associated with this system: the customer service completion event. What are the actions that occur at this event?

- The customer departs the bank.
- If there are waiting customers, the teller indicates to the next customer that he/she will serve the customer. The customer will exit the waiting line and will begin service with the teller.
- If there are no waiting customers, then the teller will become idle.

Table 4.2 contains the service times for each of the four customers that arrived in Figure 4.1. Examine the figure with respect to customer 1. Based on the figure, customer 1 can enter service at time two because there were no other customers present in the bank. Suppose that it is now 9:02 am (time 2) and that the service time of customer 1 is known *in advance* to be 8 minutes as indicated in Table 4.2. From this information, the time that customer 1 is going to complete service can be pre-computed. From the information in the figure, customer 1 will complete service at time 10 (current time + service time = 2 + 8 = 10). Thus, it should be apparent, that for you to recreate the bank's behavior over a time period that you must have knowledge of the customer's service times. The service time of each customer coupled with the knowledge of when the customer began service allows the time that the customer will complete service and depart the bank to be computed in advance. A careful inspection of Figure 4.2 and knowledge of how banks operate should indicate that a customer will begin service either immediately upon arrival (when a teller is available) or coinciding with when another customer departs the bank after being served. This latter situation occurs when the queue is not empty after a

customer completes service. The times that service completions occur and the times that arrivals occur constitute the pertinent events for simulating this banking system.

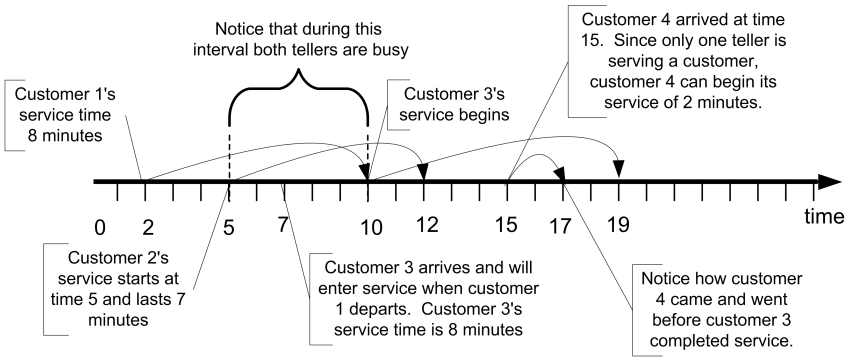


Figure 4.2.: Customer Service Process

Table 4.2.: Service Time for First Four Customers

Customer	Service Time Started	Service Time	Service Time Completed
1	2	8	10
2	5	7	12
3	10	9	19
4	15	2	17

If the arrival and the service completion events are combined, then the time ordered sequence of events for the system can be determined. Figure 4.3 illustrates the events ordered by time from smallest event time to largest event time. Suppose you are standing at time two. Looking forward, the next event to occur will be at time 5 when the second customer arrives. When you simulate a system, you must be able to generate a sequence of

events so that, at the occurrence of each event, the appropriate actions are invoked that change the state of the system.

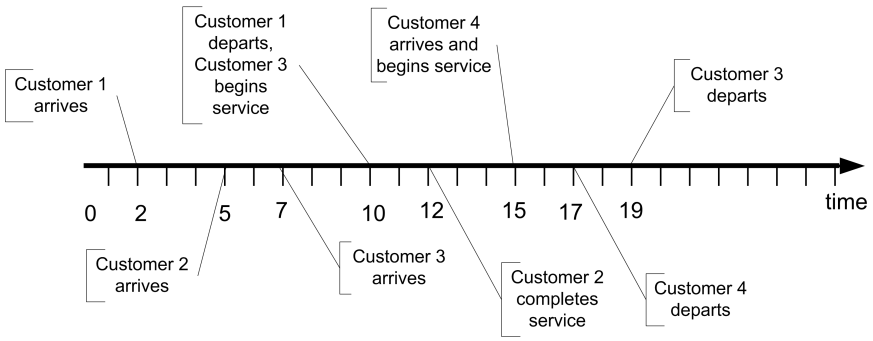


Figure 4.3.: Events Ordered by Time Process

Time	Event	Comment
0	Bank Opens	
2	Arrival	Customer 1 arrives, enters service for 8 minutes, one teller becomes bus
5	Arrival	Customer 2 arrives, enters service for 7 minutes, the second teller becor
7	Arrival	Customer 3 arrives, waits in queue
10	Service Completion	Customer 1 completes service, customer 3 exits the queue and enters se
12	Service Completion	Customer 2 completes service, no customers are in the queue so a teller
15	Arrival	Customer 4 arrives, enters service for 2 minutes, one teller becomes bus
17	Service Completion	Customer 4 completes service, a teller becomes idle
19	Service Completion	Customer 3 completes service

The real system will simply evolve over time; however, a simulation of the system must recreate the events. In simulation, events are created

by adding additional logic to the normal state changing actions. This additional logic is responsible for scheduling future events that are implied by the actions of the current event. For example, when a customer arrives, the time to the next arrival can be generated and scheduled to occur at some future time. This can be done by generating the time until the next arrival and adding it to the current time to determine the actual arrival time. Thus, all the arrival times do not need to be pre-computed prior to the start of the simulation. For example, at time two, customer 1 arrived. Customer 2 arrives at time 5. Thus, the time between arrivals (3 minutes) is added to the current time and customer 2's arrival at time 5 is scheduled to occur. Every time an arrival occurs this additional logic is invoked to ensure that more arrivals will continue within the simulation.

Adding additional scheduling logic also occurs for service completion events. For example, since customer 1 immediately enters service at time 2, the service completion of customer 1 can be scheduled to occur at time 10 (current time + service time = 2 + 8 = 10). Notice that other events may have already been scheduled for times less than time 10. In addition, other events may be inserted before the service completion at time 10 actually occurs. From this, you should begin to get a feeling for how a computer program can implement some of these ideas.

Based on this intuitive notion for how a computer simulation may execute, you should realize that computer logic processing need only occur at the event times. That is, the state of the system does not change between events. Thus, it is *not necessary* to step incrementally through time checking to see if something happens at time 0.1, 0.2, 0.3, etc. (or whatever time scale you envision that is fine enough to not miss any events). Thus, in a discrete-event dynamic system simulation the clock does not "tick" at regular intervals. Instead, the simulation clock jumps from event time to event time. As the simulation moves from the current event to the next event, the current simulation time is updated to the time of the next event and any changes to the system associated with the next event are executed. This allows the simulation to evolve over time.

4.3. Simulating a Queueing System By Hand

This section builds on the concepts discussed in the previous section in order to provide insights into how discrete event simulations operate. In order to do this, we will simulate a simple queueing system by hand. That is, we will process each of the events that occur as well as the state changes in order to trace the operation of the system through time.

Consider again the simple banking system described in the previous section. To simplify the situation, we assume that there is only one teller that is available to provide service to the arriving customers. Customers that arrive while the teller is already helping a customer form a single waiting line, which is handled in a first come, first served manner. Let's assume that the bank opens at 9 am with no customers present and the teller idle. The time of arrival of the first eight customers is provided in the following table.

Customer Number	Time of Arrival	Service Time
1	3	4
2	11	4
3	13	4
4	14	3
5	17	2
6	19	4
7	21	3
8	27	2

We are going to process these customers in order to recreate the behavior of this system over from time 0 to 31 minutes. To do this, we need to be able to perform the "bookkeeping" that a computer simulation model would normally perform for us. Thus, we will need to define some variables associated with the system and some attributes associated with the customers. Consider the following system variables.

System Variables

- Let t represent the current simulation clock time.
- Let $N(t)$ represent the number of customers in the system (bank) at any time t.
- Let $Q(t)$ represent the number of customers waiting in line for the teller at any time t.
- Let $B(t)$ represent whether or not the teller is busy (1) or idle (0) at any time t.

Because we know the number of tellers available, we know that the following relationship holds between the variables:

$$N\left(t\right) = Q\left(t\right) + B(t)$$

Note also that, knowledge of the value of $N(t)$ is sufficient to determine the values of $Q(t)$ and $B(t)$ at any time t. For example, if we know that there are 3 customers in the bank, then $N\left(t\right) = 3$, and since the teller must be serving 1 of those customers, $B\left(t\right) = 1$ and there must be 2 customers waiting, $Q\left(t\right) = 2$. In this situation, since $N\left(t\right)$, is sufficient to determine all the other system variables, we can say that $N\left(t\right)$ is the system's *state variable*. The state variable(s) are the minimal set of system variables that are necessary to represent the system's behavior over time. We will keep track of the values of all of the system variables during our processing in order to collect statistics across time. Quantities such as $N(t), Q(t)$, and $B(t)$ are variables that persist over time. In simulation, we call these variables *time-persistent* variables and we will need to collect specially defined statistics called *time-persistent statistics* on these quantities. This computation will be illustrated later in this section. Within the KSL, we will use the `TWResponse` class to collect time-persistent statistics.

Attributes are properties of things that move through the system. In the parlance of simulation, we call the things that move through the system

entity instances or entities. In this situation, the entities are the customers, and we can define a number of attributes for each customer. Here, customer is a type of entity or entity type. If we have other things that flow through the system, we identify the different types (or entity types). The attributes of the customer entity type are as follows. Notice that each attribute is subscripted by i, which indicates the i^{th} customer instance.

Entity Attributes

- Let ID_i be the identity number of the customer. ID_i is a unique number assigned to each customer that identifies the customer from other customers in the system.
- Let A_i be the arrival time of the i^{th} customer.
- Let S_i be the time the i^{th} customer started service.
- Let D_i be the departure time of the i^{th} customer.
- Let ST_i be the service time of the i^{th} customer.
- Let T_i be the total time spent in the system of the i^{th} customer.
- Let W_i be the time spent waiting in the queue for the i^{th} customer.

Quantities such as $A_i, S_i, D_i, \text{ST}_i, T_i$ and W_i all represent quantities that can be observed at specific event times. These quantities are often called *observation-based* or *tally based* variables. As we will see in subsequent sections of this chapter, we will use the KSL construct, `Response`, to capture statistics on these types of variables.

Clearly, there are also relationships between these attributes. We can compute the total time in the system for each customer from their arrival and departure times:

$$T_i = D_i - A_i$$

In addition, we can compute the time spent waiting in the queue for each customer as:

$$W_i = T_i - \mathrm{ST}_i = S_i - A_i$$

As in the previous section, there are two types of events: arrivals and departures. Let $E(t)$ be (A) for an arrival event at time t and be (D) for a departure event at time t. As we process each event, we will keep track of when the event occurs and the type of event. We will also keep track of the state of the system after each event has been processed. To make this easier, we will keep track of the system variables and the entity attributes within a table as follows.

Table 4.3.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA

In the table, we see that the initial state of the system is empty and idle. Since there are no customers in the bank, there are no tabulated attribute values within the table. Reviewing the provided information, we see that customer 1 arrives at $t = 3$ and has a service time of 4 minutes. Thus, $\mathrm{ID}_1 = 1$, $A_1 = 3$, and $\mathrm{ST}_1 = 4$. We can enter this information directly into our bookkeeping table. In addition, because the bank was empty and idle when the customer arrived, we can note that the time that customer 1 starts service is the same as the time of their arrival and that they did not spend any time waiting in the queue. Thus, $S_1 = 3$ and $W_1 = 0$. The table has been updated as follows.

Table 4.4.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA
3	A	1	1	0	1	3	4	3	NA	NA	0	E(7) = D(1), E(11) = A(2

Because customer 1 arrived to an empty system, they immediately started service at time 3. Since we know the service time of the customer, $\mathrm{ST}_1 = 4$, and the current time, $t = 3$, we can determine that customer 1, will depart from the system at time 7 ($t = 3 + 4 = 7$). That means we will have a departure event for customer 1 at time 7. According to the provided data, the next customer, customer 2, will arrive at time 11. Thus, we have two pending events, a departure of customer 1 at time 7 and the arrival of customer 2 at time 11. This fact is noted in the column labeled "Pending E(t)" for pending events. Here E(7) = D(1), E(11) = A(2) indicates that customer 1 will depart, D_1, at time 7, $E\,(7)$ and customer 2 will arrive, A_2, at the event at time 11, $E(11)$. Clearly, the next event time will be the minimum of 7 and 11, which will be the departure of the first customer. Thus, our bookkeeping table can be updated as follows.

Table 4.5.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA
3	A	1	1	0	1	3	4	3	NA	NA	0	E(7) = D(1), E(11) = A(2
7	D	0	0	0	1	3	4	3	7	4	0	E(11) = A(2)

Since there are no customers in the bank and only the one pending event, the next event will be the arrival of customer 2 at time 11. The table can be updated as follows.

Table 4.6.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA
3	A	1	1	0	1	3	4	3	NA	NA	0	E(7) = D(1), E(11) = A(
7	D	0	0	0	1	3	4	3	7	4	0	E(11) = A(2)
11	A	1	1	0	2	11	4	11	NA	NA	0	E(13) = A(3), E(15) = D

Since the pending event set is E(13) = A(3), E(15) = D(2) the next event will be the arrival of the third customer at time 13 before the departure of the second customer at time 15. We will now have a queue form. Updating our bookkeeping table, yields:

Table 4.7.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA
3	A	1	1	0	1	3	4	3	NA	NA	0	E(7) = D(1), E(11) = A(
7	D	0	0	0	1	3	4	3	7	4	0	E(11) = A(2)
11	A	1	1	0	2	11	4	11	NA	NA	0	E(13) = A(3), E(15) = D
13	A	2	1	1	3	13	4	15	NA	NA	2	E(14) = A(4), E(15) = D

Notice that in the last table update, we did not update S_i and W_i because customer 3 had to wait in queue and did not start service. Customer 3 will start service, when customer 2 departs. Reviewing the pending event set, we see that the next event will be the arrival of customer 4 at time 14, which yields the following bookkeeping table.

Table 4.8.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA
3	A	1	1	0	1	3	4	3	NA	NA	0	E(7) = D(1), E(11) = A(
7	D	0	0	0	1	3	4	3	7	4	0	E(11) = A(2)
11	A	1	1	0	2	11	4	11	NA	NA	0	E(13) = A(3), E(15) = D
13	A	2	1	1	3	13	4	15	NA	NA	2	E(14) = A(4), E(15) = D
14	A	3	1	2	4	14	3	19	NA	NA	5	E(15) = D(2), E(17) = A

Notice that we now have 3 customers in the system, 1 in service and 2 waiting in the queue. Reviewing the pending event set, we see that customer 2 will finally complete service and depart at time 15.

Table 4.9.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA
3	A	1	1	0	1	3	4	3	NA	NA	0	E(7) = D(1), E(11) = A(
7	D	0	0	0	1	3	4	3	7	4	0	E(11) = A(2)
11	A	1	1	0	2	11	4	11	NA	NA	0	E(13) = A(3), E(15) = D
13	A	2	1	1	3	13	4	15	NA	NA	2	E(14) = A(4), E(15) = D
14	A	3	1	2	4	14	3	19	NA	NA	5	E(15) = D(2), E(17) = A
15	D	2	1	1	2	11	4	11	15	4	0	E(17) = A(5), E(19) = D

Because customer 2 completes service at time 15 and customer 3 is waiting in the line, we see that customer 3's attributes for S_i and W_i were updated within the table. Since customer 3 has started service and we know their service time of 4 minutes, we know that they will depart at time 19. The

pending events set has been updated accordingly and indicates that the next event will be the arrival of customer 5 at time 17.

Table 4.10.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA
3	A	1	1	0	1	3	4	3	NA	NA	0	E(7) = D(1), E(11) = A(
7	D	0	0	0	1	3	4	3	7	4	0	E(11) = A(2)
11	A	1	1	0	2	11	4	11	NA	NA	0	E(13) = A(3), E(15) = D
13	A	2	1	1	3	13	4	15	NA	NA	2	E(14) = A(4), E(15) = D
14	A	3	1	2	4	14	3	19	NA	NA	5	E(15) = D(2), E(17) = A
15	D	2	1	1	2	11	4	11	15	4	0	E(17) = A(5), E(19) = D
17	A	3	1	2	5	17	2	22	NA	NA	5	E(19) = D(3), E(19) = A

Now, we have a very interesting situation with the pending event set. We have two events that are scheduled to occur at the same time, the departure of customer 3 at time 19 and the arrival of customer 6 at time 19. In general, the order in which events are processed that occur at the same time may affect how future events are processed. That is, the order of event processing may change the behavior of the system over time and thus the order of processing is, in general, important. However, in this particular simple system, the order of processing will not change what happens in the future. We will simply have an update of the state variables at the same time. In this instance, we will process the departure event first and then the arrival event. If you are not convinced that this will not make a difference, then I encourage you to change the ordering and continue the processing. In more complex system simulations, a priority indicator is attached to the events so that the events can be processed in a consistent manner. Rather than continuing the step-by-step processing of the events through time 31, we will present the completed table. We leave it as an

exercise for the reader to continue the processing of the customers. The completed bookkeeping table at time 31 is as follows.

Table 4.11.: Hand Bank Simulation Bookkeeping Table.

System Variables					Entity Attributes							
t	E(t)	N(t)	B(t)	Q(t)	ID(i)	A(i)	ST(i)	S(i)	D(i)	T(i)	W(i)	Pending E(t)
0	NA	0	0	0	NA	NA	NA	NA	NA	NA	NA	NA
3	A	1	1	0	1	3	4	3	NA	NA	0	E(7) = D(1), E(11) = A(
7	D	0	0	0	1	3	4	3	7	4	0	E(11) = A(2)
11	A	1	1	0	2	11	4	11	NA	NA	0	E(13) = A(3), E(15) = D
13	A	2	1	1	3	13	4	15	NA	NA	2	E(14) = A(4), E(15) = D
14	A	3	1	2	4	14	3	19	NA	NA	5	E(15) = D(2), E(17) = A
15	D	2	1	1	2	11	4	11	15	4	0	E(17) = A(5), E(19) = D
17	A	3	1	2	5	17	2	22	NA	NA	5	E(19) = D(3), E(19) = A
19	D	2	1	1	3	13	4	15	19	6	2	E(19) = A(6)
19	A	3	1	2	6	19	4	24	NA	NA	5	E(21) = A(7), E(22) = D
21	A	4	1	3	7	21	3	28	NA	NA	7	E(22) = D(4), E(24) = D
22	D	3	1	2	4	14	3	19	22	8	5	E(24) = D(5), E(27) = A
24	D	2	1	1	5	17	2	22	24	7	5	E(27) = A(8), E(28) = D
27	A	3	1	2	8	27	2	31	NA	NA	4	E(28) = D(6), E(31) = D
28	D	2	1	1	6	19	4	24	28	9	5	E(31) = D(7)
31	D	1	1	0	7	21	3	28	31	10	7	E(33) = D(8)

Given that we have simulated the bank over the time frame from 0 to 31 minutes, we can now compute performance statistics for the bank and measure how well it is operating. Two statistics that we can easily compute are the average time spent waiting in line and the average time spent in the system.

Let n be the number of customers observed to depart during the simulation. In this simulation, we had $n = 7$ customers depart during the time frame of 0 to 31 minutes. The time frame over which we analyze the performance

of the system is called the simulation time horizon. In this case, the simulation time horizon is fixed and known in advance. When we perform a computer simulation experiment, the time horizon is often referred to as the *simulation run length* (or simulation *replication length*). To estimate the average time spent waiting in line and the average time spent in the system, we can simply compute the sample averages ($\overline{T}$ and $\bar{W}$) of the observed quantities (T_i and W_i)for each departed customer.

$$\overline{T} = \frac{1}{7}\sum_{i=1}^{7} T_i = \frac{4+4+6+8+7+9+10}{7} = \frac{48}{7} \cong 6.8571$$

$$\overline{W} = \frac{1}{7}\sum_{i=1}^{7} W_i = \frac{0+0+2+5+5+5+7}{7} = \frac{24}{7} \cong 3.4286$$

These statistical quantities are based on observations and we use our standard approach to computing averages for these observations. Besides the average time spent in the system and the average time spent waiting, we might also want to compute the average number of customers in the system, the average number of customers in the queue, and the average number of busy tellers. You might be tempted to simply average the values in the $N(t)$, $B(t)$, and $Q(t)$ columns of the bookkeeping table. Unfortunately, that will result in an incorrect estimate of these values because a simple average will not take into account how long each of the variables persisted with its values over time. In this situation, we are really interested in computing a time average. This is because the variables, $N(t)$, $B(t)$, and $Q(t)$ are called time-persistent variables. This type of variable is always encountered in discrete event simulations.

To make the discussion concrete, let's focus on the number of customers in the queue, $Q(t)$. Notice the number of customers in the queue, $Q(t)$ takes on constant values during intervals of time corresponding to when the queue has a certain number of customers. $Q(t) = \{0, 1, 2, 3, ...\}$. The values of $Q(t)$ form a step function in this particular case. The following figure

illustrates the step function nature of this type of variable. A realization of the values of variable is called a sample path.

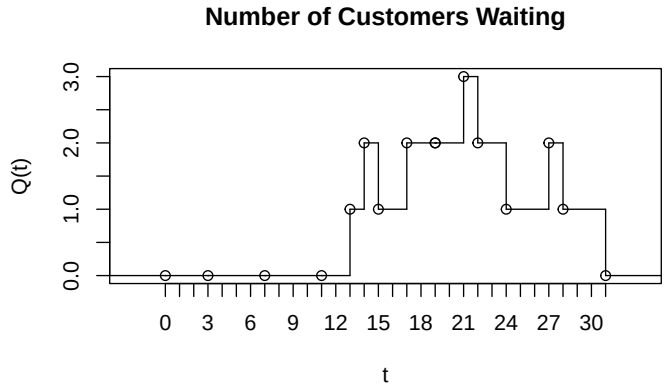


Figure 4.4.: Number of Customers Waiting in Queue for the Bank Simulation

That is, for a given (realized) sample path, $Q(t)$ is a function that returns the number of customers in the queue at time t. The mean value theorem of calculus for integrals states that given a function, $f(\bullet)$, continuous on an interval (a, b), there exists a constant, c, such that

$$\int_a^b f(x)\,\mathrm{dx} = f(c)(b-a)$$

$$f(c) = \frac{\int_a^b f(x)\,\mathrm{dx}}{(b-a)}$$

The value, $f(c)$, is called the mean value of the function. A similar function can be defined for $Q(t)$ This function is called the time-average (and represents the *mean value* of the $Q(t)$ function):

$$\overline{Q} = \frac{\int_{t_0}^{t_n} Q\,(t)\,\mathrm{dt}}{t_n - t_0}$$

This function represents the *average with respect to time* of the given state variable. This type of statistical variable is called time-persistent because $Q(t)$ is a function of time.

In the particular case where $Q(t)$ represents the number of customers in the queue, $Q(t)$ will take on constant values during intervals of time corresponding to when the queue has a certain number of customers. Let $Q\,(t) = \ q_k$ for $t_{k-1} \leq t \leq t_k$. Then, the time-average can be rewritten as follows:

$$\overline{Q} = \frac{\int_{t_0}^{t_n} Q\,(t)\,\mathrm{dt}}{t_n - t_0} = \sum_{k=1}^{n} \frac{q_k(t_k - t_{k-1})}{t_n - t_0}$$

Note that $q_k(t_k - t_{k-1})$ is the area under the curve, $Q\,(t)$ over the interval $t_{k-1} \leq t \leq t_k$ and because

$$t_n - t_0 = (t_1 - t_0) + (t_2 - t_1) + (t_3 - t_2) + \ \cdots + (t_{n-1} - t_{n-2}) + \ (t_n - t_{n-1})$$

we can write,

$$t_n - t_0 = \sum_{k=1}^{n} t_k - t_{k-1}$$

The quantity $t_n - t_0$ represents the total time over which the variable is observed. Thus, the time average is simply the area under the curve divided by the amount of time over which the curve is observed. From this equation, it should be noted that each value of q_k is weighted by the length of time that the variable has the value. If we define, $w_k = (t_k - t_{k-1})$, then we can re-write the time average as:

$$\overline{Q} = \frac{\int_{t_0}^{t_n} Q\left(t\right) \mathrm{dt}}{t_n - t_0} = \sum_{k=1}^{n} \frac{q_k(t_k - t_{k-1})}{t_n - t_0} = \frac{\sum_{k=1}^{n} q_k w_k}{\sum_{i=1}^{n} w_k}$$

This form of the equation clearly shows that each value of q_k is weighted by:

$$\frac{w_k}{\sum_{i=1}^{n} w_k} = \frac{w_k}{t_n - t_0} = \frac{(t_k - t_{k-1})}{t_n - t_0}$$

This is why the time average is often called the time-weighted average. If $w_k = 1$, then the time-weighted average is the same as the sample average.

Now we can compute the time average for $Q\left(t\right),\ N\left(t\right)$ and $B(t)$. Using the following formula and noting that $t_n - t_0 = 31$

$$\overline{Q} = \sum_{k=1}^{n} \frac{q_k(t_k - t_{k-1})}{t_n - t_0}$$

We have that the numerator computes as follows:

$$\sum_{k=1}^{n} q_k \left(t_k - t_{k-1}\right) = 0\left(13 - 0\right) + 1\left(14 - 13\right) + 2\left(15 - 14\right) + 1\left(17 - 15\right) + 2\left(19 - 17\right)$$

$$+1\left(19 - 19\right) + \ 2\left(21 - 19\right) + \ 3\left(22 - 21\right) + \ 2\left(24 - 22\right) +$$

$$1\left(27 - 24\right) + \ 2\left(28 - 27\right) + \ 1\left(31 - 28\right) = 28$$

And, the final time-weighted average number in the queue is:

$$\overline{Q} = \frac{28}{31} \cong 0.903$$

The average number in the system and the average number of busy tellers can also be computed in a similar manner, resulting in:

$$\overline{N} = \frac{52}{31} \cong 1.677$$

$$\overline{B} = \frac{24}{31} \cong 0.7742$$

The value of $\overline{B}$ is most interesting for this situation. Because there is only 1 teller, the fraction of the tellers that are busy is 0.7742. This quantity represents the *utilization* of the teller. The utilization of a resource represents the proportion of time that the resource is busy. Let c represent the number of units of a resource that are available. Then the utilization of the resource is defined as:

$$\overline{U} = \frac{\overline{B}}{c} = \frac{\int_{t_0}^{t_n} B\left(t\right) \mathrm{dt}}{c(t_n - t_0)}$$

Notice that the numerator of this equation is simply the total time that the resource is busy. So, we are computing the total time that the resource is busy divided by the total time that the resource could be busy, $c(t_n - t_0)$, which is considered the utilization.

In this simple example, when an arrival occurs, you must determine whether or not the customer will enter service or wait in the queue. When a customer departs the system, whether or not the server will become idle must be determined, by checking the queue. If the queue is empty, then the server becomes idle; otherwise the next customer in the queue begins service. In order to develop a computer simulation model of this situation, the actions

that occur at an event must be represented within code. For example, the pseudo-code for the arrival event would be:

Pseudo-code for Arrival Event

1. schedule arrival time of next customer
    1. AT = generate time to next arrival according to inter-arrival distribution
    2. schedule arrival event at, t + AT
2. check status of the servers (idle or busy)
    1. if idle, do
        1. allocate a server to the customer
        2. ST = generate service time according to the service time distribution
        3. schedule departure event at, t + ST
    2. if busy, do
        1. increase number in queue by 1 (place customer in queue)

Pseudo-code for Departure Event

1. if queue is not empty
    1. remove next customer from the queue
    2. allocate the server to the next customer in the queue
    3. ST = generate service time according to the service time distribution
    4. schedule departure event at, t + ST
2. else the queue is empty

1. make the customer's server idle

Notice how the arrival event schedules the next arrival and the departure event may schedule the next departure event (provided the queue is not empty). Within the KSL, this pseudo-code must be represented in a class method that will be called at the appropriate event time.

To summarize, discrete-event modeling requires two key abilities:

- The ability to represent the actions associated with an event in computer code (i.e. in a class method).
- The ability to schedule events so that they will be called at the appropriate event time, i.e. the ability to have the event's logic called at the appropriate event time.

Thus, the scheduling and execution of events is critical to discrete-event modeling. The scheduling of additional events within an event is what makes the system progress through time (jumping from event to event). The KSL supports the scheduling and execution of events within its calendar and model packages. The next section overviews the libraries available within the KSL for discrete-event modeling.

4.4. Modeling DEDS in the KSL

Discrete event modeling within the KSL is facilitated by two packages: 1) the `ksl.simulation` package and 2) the `ksl.calendar` package. The `ksl.simulation` package has classes that implement behavior associated with simulation events and models. The `ksl.calendar` package has classes that facilitate the scheduling and processing of events.

4.4.1. Event Scheduling

The following classes within the `ksl.simulation` package work together to provide for the scheduling and execution of events:

- `Model`: An instance of `Model` holds the model and facilitates the running of the simulation according to run parameters such as simulation run length and number of replications. An instance of a Model is required to serve as the parent to any model elements within a simulation model. It is the top-level container for model elements.
- `Executive`: The Executive controls the execution of the events and works with the calendar package to ensure that events are executed and the appropriate model logic is called at the appropriate event time. This class is responsible for placing the events on a calendar, allowing events to be canceled, and executing the events in the correct order.
- `KSLEvent`: This class represents a simulation event. The attributes of `KSLEvent` provide information about the name, time, priority, and type of the event. The user can also check whether or not the event is canceled or if it has been scheduled. In addition, a generic attribute of type `<T>` can be associated with an event and can be used to pass information along with the event.
- `ModelElement`: This class serves as a base class for all classes that are within a simulation model. It provides access to the scheduler and ensures that model elements participate in common simulation actions (e.g. warm up, initialization, etc.).

Figure 4.5 illustrates the relationships between the classes `Model`, `ModelElement`, and `Executive`. The `ModelElement` class represents the primary building block for KSL models. A `ModelElement` represents something (an element) that can be placed within an instance of a `Model`. The `Model` class subclasses `ModelElement`. Every `ModelElement` can contain many other instances of `ModelElement`. As such, an instance of a

`Model` can also contain model elements. There can only be one instance of the `Model` class within the simulation. It acts as the parent (container) for all other model elements. Model elements in turn also hold other model elements. Instances arranged in this pattern form an object hierarchy that represents the simulation model. The instance of a `Model` holds (references) the instance of the `Executive` class. The instance of the `Executive` uses an instance of a class that implements the `CalendarIfc` interface. The simulation also references an instance of the `Experiment` class. The `Experiment` class allows the specification and control of the run parameters for the simulation. Every instance of a `ModelElement` must be a child of another `ModelElement` or the `Model`. This implies that instances of `ModelElement` have access to the main model, which has access to an instance of `Model` and thus the instance of the `Executive`. Therefore sub-classes of `ModelElement` have access to the `Executive` and can schedule events.

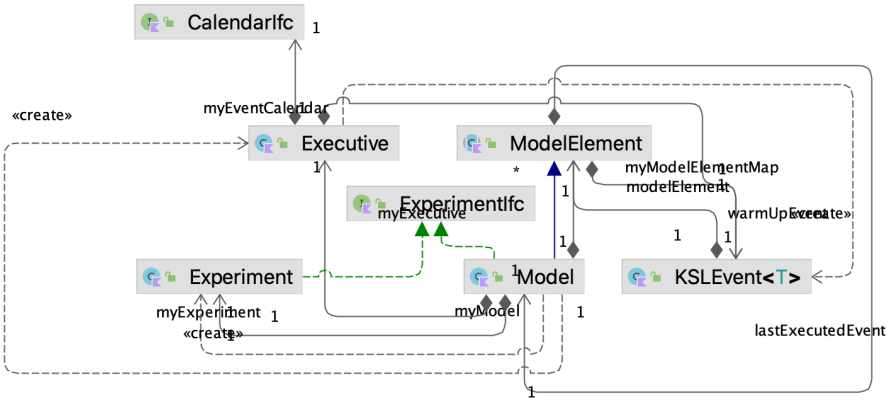


Figure 4.5.: ksl.simulation Package and Relationships

`ModelElement` is an abstract base class for building other classes that can exist within a model. Sub-classes of the `ModelElement` class will have a set of methods that facilitate the scheduling of events.

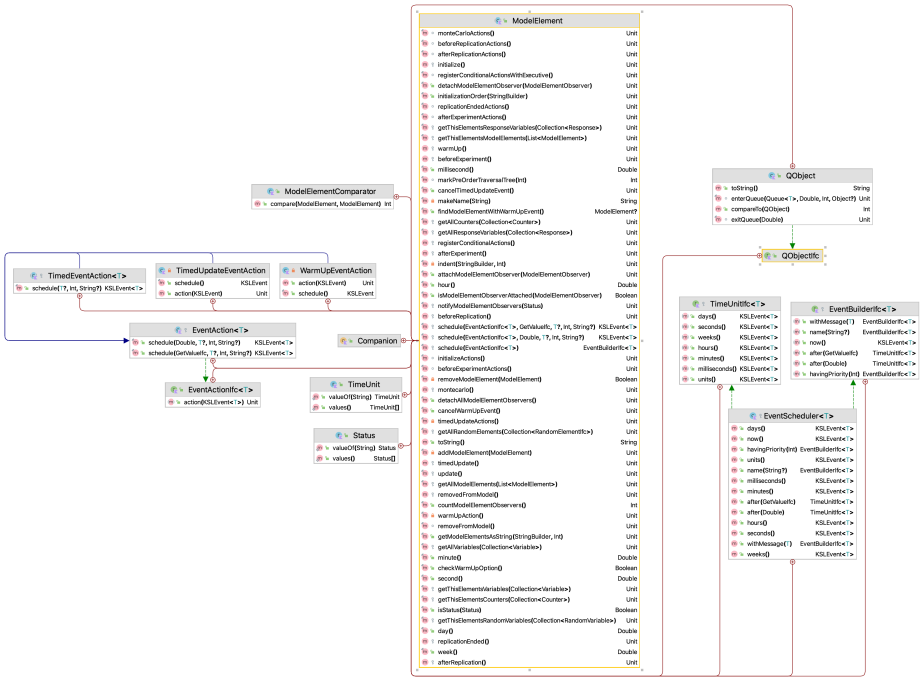


Figure 4.6.: ModelElement and Its Inner Classes

Figure 4.6 illustrates the protected methods of the `ModelElement` class. Since these methods are protected, sub-classes will have them available through inheritance. The scheduling of new events results in the creation of a new `KSLEvent` instance and the placement of the event on the event calendar via the `Executive.`

The following code listing illustrates the key method for scheduling events within the class `ModelElement` Notice that the instance of the `Executive` is called via `executive` property. In addition, the user can supply information for the creation of an event such as its time, name, and priority. The user can also provide an instance of classes that implement the `EventActionIfc` interface. This interface promises to have an `action()` method. Within the `action()` method the user can provide the code that is necessary to represent the state changes associated with the event.

```
    /**
     * Allows event actions to be scheduled by model elements
     * @param eventAction the event action to schedule
     * @param timeToEvent the time to the next action
     * @param priority the priority of the action
     * @param message a general object to attach to the
↪   action
     * @param name a name to associate with the event for the
↪   action
     * @return the scheduled event
     */
    protected fun <T> schedule(
        eventAction: EventActionIfc<T>,
        timeToEvent: Double,
        message: T? = null,
        priority: Int = KSLEvent.DEFAULT_PRIORITY,
        name: String? = null
    ): KSLEvent<T> {
        return executive.scheduleEvent(this, eventAction,
         ↪  timeToEvent, message, priority, name)
```

```
    }
```

There are two ways that the user can provide event action code: 1) provide a class that implements the `EventActionIfc` interface and supply it when scheduling the event or 2) by treating the `EventActionIfc` as a functional interface and using Kotlin's functional method representation.

Notice that besides the `schedule()` method of the `ModelElement` class, the `EventAction` class definition also facilitates the scheduling of the event action.

```
    protected abstract inner class EventAction<T> :
     ↪ EventActionIfc<T> {

        /**
         * Allows event actions to more conveniently schedule
↪   themselves
         * @param timeToEvent the time to the next action
         * @param priority the priority of the action
         * @param message a general object to attach to the
↪   action
         * @param name a name to associate with the event for
↪   the action
         * @return the scheduled event
         */
        fun schedule(
            timeToEvent: Double,
            message: T? = null,
            priority: Int = KSLEvent.DEFAULT_PRIORITY,
            name: String? = null
        ): KSLEvent<T> {
            return schedule(this, timeToEvent, message,
             ↪ priority, name)
```

```
        }

        fun schedule(
            timeToEvent: GetValueIfc,
            message: T? = null,
            priority: Int = KSLEvent.DEFAULT_PRIORITY,
            name: String? = null
        ): KSLEvent<T> {
            return schedule(timeToEvent.value, message,
             ↪  priority, name)
        }
    }
```

These scheduling methods have also been overloaded to allow the specification of time via instances that implement the `GetValueIfc`. This allows instances of random variables to be used directly with the time until the event ultimately being randomly drawn from the distribution.

4.4.2. Simple Event Scheduling Examples

This section presents two simple examples to illustrate event scheduling. The first example illustrates the scheduling of events using the `EventActionIfc` interface. The second example shows how to simulate a Poisson process and collect simple statistics.

4.4.2.1. Implementing Event Actions Using the EventActionIfc Interface

In the first example, there will be two events scheduled with actions. The time between the events will all be deterministic. The specifics of the events are as follows:

1. Event Action One: This event first occurs at time 10.0. It prints out a message. Its later occurrences are scheduled by Event Action Two.
2. Event Action Two: This event prints out a message. Then, it schedules an action one event 15 time units into the future. It also reschedules itself to reoccur in 20 minutes.

The following code listing provides the code for this simple event example. Let's walk carefully through the construction and execution of this code.

First, the class sub-classes from `ModelElement.` This enables the class to have access to all the scheduling methods within `ModelElement` and provides one method that needs to be overridden: `initialize().` Every `ModelElement` has an `initialize()` method. The `initialize()` method does nothing within the class `ModelElement.` However, the `initialize()` method is *critical* to properly modeling using instances of `ModelElement` within the KSL architecture. The purpose of the `initialize()` method is to provide code that can occur once at the beginning of each replication of the simulation, prior to the execution of any events. Thus, the `initialize()` method is the perfect location to schedule initial events onto the event calendar so that when the replications associated with the simulation are executed initial events will be on the calendar ready for execution. Notice that in this example the `initialize()` method does two things:

1. schedules the first event action one event at time 10.0 via the call: `schedule(myEventActionOne, 10.0)`
2. schedules the first action two event at time 20.0 via the call: `schedule(myEventActionTwo, 20.0)`

Example 4.1 (Scheduling Events)**.** This example code illustrates how to create event actions and to schedule their occurrence at specific event times.

```
class SchedulingEventExamples (parent: ModelElement, name:
 ↪  String? = null) :
    ModelElement(parent, name) {
    private val myEventActionOne: EventActionOne =
     ↪  EventActionOne()
    private val myEventActionTwo: EventActionTwo =
     ↪  EventActionTwo()

    override fun initialize() {
        // schedule a type 1 event at time 10.0
        schedule(myEventActionOne, 10.0)
        // schedule an event that uses myEventAction for time
         ↪  20.0
        schedule(myEventActionTwo, 20.0)
    }

    private inner class EventActionOne :
     ↪  EventAction<Nothing>() {
        override fun action(event: KSLEvent<Nothing>) {
            println("EventActionOne at time : $time")
        }
    }

    private inner class EventActionTwo :
     ↪  EventAction<Nothing>() {
        override fun action(event: KSLEvent<Nothing>) {
            println("EventActionTwo at time : $time")
            // schedule a type 1 event for time t + 15
            schedule(myEventActionOne, 15.0)
            // reschedule the EventAction event for t + 20
            schedule(myEventActionTwo, 20.0)
        }
    }
```

```
}
```

The function call `schedule(myEventActionTwo, 20.0)` schedules an event 20 time units into the future where the event will be handled via the instance of the class `EventActionTwo`, which implements the `EventActionIfc` interface. The reference `myEventActionTwo` refers to an object of type `EventActionTwo`, which is an instance of the inner classes defined within `SchedulingEventExamples`. This variable is defined as a class property on line 4 and an instance is created via the constructor. To summarize, the `initialize()` method is used to schedule the initial occurrences of the two types of events. The `initialize()` method occurs right before time 0.0. That is, it occurs right before the simulation clock starts.

Now, let us examine the actions that occur for the two types of events. Within the `action()` method of `EventActionOne`, we see the following code:

```
println("EventActionOne at time : $time")
```

Here a simple message is printed that includes the simulation time via the inherited `time` property of the `ModelElement` class. Thus, by implementing the `action()` method of the `EventActionIfc` interface, you can supply the logic that occurs when the event is executed by the simulation executive. In the implemented `EventActionTwo` class, a simple message is printed and event action one is scheduled. In addition, the `schedule()` method is used to reschedule the `action()` method. The following code illustrates how to setup and run the simulation.

```
fun main() {
    val m = Model("Scheduling Example")
    SchedulingEventExamples(m.model)
    m.lengthOfReplication = 100.0
```

```
    m.simulate()
}
```

The main method associated with the `SchedulingEventExamples` class indicates how to create and run a simulation model. The first line of the main method creates an instance of a `Model.` The next line makes an instance of `SchedulingEventExamples` and attaches it to the simulation model. The property called `m.model` returns an instance of the `Model` class that is associated with the instance of the simulation. The next line sets up the simulation to run for 100 time units and the last line tells the simulation to begin executing via the `simulate()` method. The output of the code is as follows:

```
EventActionOne at time : 10.0
EventActionTwo at time : 20.0
EventActionOne at time : 35.0
EventActionTwo at time : 40.0
EventActionOne at time : 55.0
EventActionTwo at time : 60.0
EventActionOne at time : 75.0
EventActionTwo at time : 80.0
EventActionOne at time : 95.0
EventActionTwo at time : 100.0
```

Notice that event action one output occurs at time 10.0. This is due to the event that was scheduled within the `initialize()` method. Event action two occurs for the first time at time 20.0 and then every 20 time units. Notice that event action one occurs at time 35.0. This is due to the event being scheduled in the action method of event action two.

4.4.2.2. Overview of Simulation Run Context

When the simulation runs, much underlying code is executed. At this stage it is not critically important to understand how this code works; however, it is useful to understand, in a general sense, what is happening. The following outlines the basic processes that are occurring when `m.simulate()` occurs:

1. Setup the simulation experiment
2. For each replication of the simulation:
    a. Initialize the replication
    b. Initialize the executive and calendar
    c. Initialize the model and all model elements
    d. While there are events on the event calendar or the simulation is not stopped
        i. Determine the next event to execute
        ii. Update the current simulation time to the time of the next event
        iii. Call the action() method of the instance of `EventActionIfc` that was attached to the next event.
        iv. Execute the actions associated with the next event
    e. Execute end of replication model logic
3. Execute end of simulation experiment logic

Step 2(b) initializes the executive and calendar and ensures that there are no events at the beginning of the simulation. It also resets the simulation time to 0.0. Then, step 2(c) initializes the model. In this step, the initialize() methods of all of the model elements are executed. This is why it was important to implement the `initialize()` method in the example and have it schedule the initial events. Then, step 2(d) begins the execution of the events that were placed on the calendar. In looking at the code listings, it is not possible to ascertain how the `action()` methods are actually invoked unless you understand that during step 2(d) each scheduled event is removed from the calendar and its associated action called. In the case of the event action one and two events in the example, these actions are specified in the `action()` method of `EventActionOne` and `EventActionTwo`. After all the events in the calendar are executed or the simulation is not otherwise stopped, the replication is ended. Any clean up logic (such as statistical collection) is executed at the end of the replication. Finally, after all replications have been executed, any logic associated with ending the simulation experiment is invoked. Thus, even though the code does not directly call the event logic it is still invoked by the simulation executive because the events are scheduled. Thus, if you schedule events, you can be assured that the logic associated with the events will be executed.

> **!** Important
>
> In upcoming examples, we will provide names for the model elements. The most important point to remember is that the name of a model element must be unique. If you do not provide a name for a model element, then a unique name will be created for you. The name of a model element should not contain the period "." character. If it does, the period will be replaced with an underscore "_" character. If you happen to specify the same name for more than one model element, an error will be reported. **Bottom line: Provide unique names for model elements.**

4.4.2.3. Simulating a Poisson Process

The second simple example illustrates how to simulate a Poisson process. Recall that a Poisson process models the number of events that occur within some time interval. For a Poisson process the time between events is exponentially distributed with a mean that is the reciprocal of the rate of occurrence for the events. For simplicity, this example simulates a Poisson process with rate 1 arrival per unit time. Thus, the mean time between events is 1.0 time unit. In this case the action is very simple, increment a counter that is tracking the number of events that have occurred.

The code for this example is as follows.

Example 4.2 (Simulating a Poisson Process)**.** This code example illustrates how to generate a Poisson process by scheduling events that have an inter-event time that is exponentially distributed. The code also illustrates how to use a `Counter` to collect statistics.

```
class SimplePoissonProcess (parent: ModelElement, name:
 ↪  String? = null) :
    ModelElement(parent, name) {
    private val myTBE: RandomVariable = RandomVariable(parent
     ↪  = this, rSource = ExponentialRV(1.0, streamNum = 1))
    private val myCount: Counter = Counter(parent = this,
     ↪  name = "Counts events")
    private val myEventHandler: EventHandler = EventHandler()

    override fun initialize() {
        super.initialize()
        schedule(myEventHandler, myTBE.value)
    }

    private inner class EventHandler : EventAction<Nothing>()
     ↪  {
```

```
        override fun action(event: KSLEvent<Nothing>) {
            myCount.increment()
            schedule(myEventHandler, myTBE.value)
        }
    }
}
```

There are a few new elements of this code to note. First, this example uses two new KSL model elements: `RandomVariable` and `Counter`. A `RandomVariable` is a sub-class of `ModelElement` that is used to represent random variables within a simulation model. The `RandomVariable` class must be supplied an instance of a class that implements the `RVariableIfc` interface. Recall that implementations of the `RVariableIfc` interface have a `value` property that returns a random value and permit random number stream control. The supplied stream control is important when utilized advanced simulation statistical methods. For example, stream control is used to advance the state of the underlying stream to the next substream at the end of every replication of the model. This helps in synchronizing the use of random numbers in certain types of experimental setups.

A `Counter` is also a sub-class of `ModelElement` which facilitates the incrementing and decrementing of a variable and the statistical collection of the variable across replications. The value of the variable associated with the instance of a `Counter` is automatically reset to 0.0 at the beginning of each replication. Lines 2 and 3 within the constructor create the instances of the `RandomVariable` and the `Counter`.

Since we are modeling a Poisson process, the `initialize()` method is used to schedule the first event using the random variable that represents the time between events. This occurs on the only line of the `initialize()` method. The event logic, found in the inner class `EventHandler`, causes the counter to be incremented. Then, the next arrival is scheduled to occur. Thus, it is very easy to model an arrival process using this pattern.

```
fun main(){
    val s = Model("Simple PP")
    SimplePoissonProcess(s.model)
    s.lengthOfReplication = 20.0
    s.numberOfReplications = 50
    s.simulate()
    println()
    val r = s.simulationReporter
    r.printAcrossReplicationSummaryStatistics()
    println("Done!")
}
```

The last items to note are in the `main()` method of the class, where the simulation is created and run. In setting up the simulation, the run length is set to 20 time units and the number of replications associated with the simulation is set to 50.

A replication represents a sample path of the simulation that starts and ends under the same conditions. Thus, statistics collected on each replication represent independent and identically distributed observations of the simulation model's execution. In this example, there will be 50 observations of the counter observed. Since we have a Poisson process with rate 1 event per time unit and we are observing the process for 20 time units, we should expect that about 20 events should occur on average.

Right after the `simulate()` method is called, an instance of a `SimulationReporter` is created for the simulation. A `SimulationReporter` has the ability to write out statistical output associated with the simulation. The code uses the `printAcrossReplicationSummaryStatistics()` method to write out a simple summary report across the 50 replications for the `Counter`. Note that using the `Counter` to count the events provided for automatically collected statistics across the replications for the counter. As you can see from the output, the average number of events is close to the theoretically expected amount.

```
Across Replication Statistical Summary Report
Tue Nov 29 16:41:25 CST 2022
Simulation Results for Model: MainModel


Number of Replications: 50
Length of Warm up period: 0.0
Length of Replications: 20.0

--------------------------------------------------------------------------------
Counters
--------------------------------------------------------------------------------
Name                                    Average       Std. Dev.     Count
--------------------------------------------------------------------------------
Counts events                         20.060000        3.235076       50.000000
--------------------------------------------------------------------------------
```

> **Note**
>
> In the previous model, the base time unit was conceptualized as *time units*. We can think of the time units as being in seconds, minutes, days, etc. The key thing to remember is that the underlying time value, reported by a model element's `time` property is in some base time unit that you conceptualize. If you think of time as in minutes, then you need to ensure that randomly generated time values have the appropriate time unit. For example, make sure that the run length is in minutes if you conceptualize time in minutes and convert all random quantities to minutes when fitting distributions. While there are functions that facilitate time conversion within the `ModelElement` class, they are not discussed in this text to avoid overly complicating the presentation.

4.4.3. Up and Down Component Example

This section further illustrates DEDS modeling with a component of a system that is subject to random failures.

Example 4.3 (Up and Down Component)**.** Consider a component subject to random breakdowns that has two states UP and DOWN. The time until failure is random and governed by an exponential distribution with a mean of 1.0 time units. This represents the time that the component is in the UP state. Once the component fails, the component goes into the DOWN state. The time that the component spends in the DOWN state is governed by an exponential distribution with a mean of 2.0 time units. In this model, we are interested in estimating the proportion of time that the component is in the UP state and tracking the number of failures over the running time of the simulation. In addition, we are interested in measuring the cycle length of the component. The cycle length is the time between entering the UP state. The cycle length should be equal to the sum of the time spent in the up and down states.

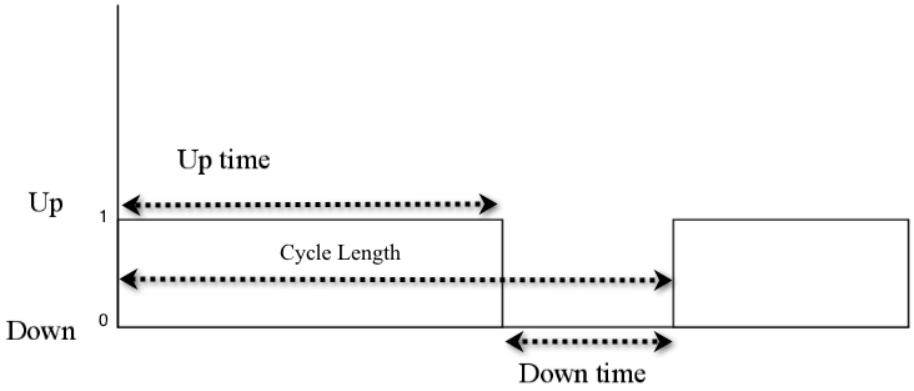


Figure 4.7.: Up and Down Component Process

Figure 4.7 illustrates the dynamics of the component over time.

The following steps are useful in developing this model:

1. Conceptualize the system/objects and their states
2. Determine the events and the actions associated with the events
3. Determine how to represent the system and objects as ModelElements
4. Determine how to initialize and run the model

The first step is to conceptualize how to model the system and the state of the component. A model element, `UpDownComponent,` will be used to model the component. To track the state of the component, it is necessary to know whether or not the component is UP or DOWN. A variable can be used to represent this state. However, since we need to estimate the proportion of time that the component is in the UP state, a `TWResponse` variable will be used. `TWResponse` is a sub-class of `ModelElement` that facilitates the observation and statistical collection of time-based variables. Time-bases variables, which are discussed further in the next Chapter, are a type of variable that changes values at particular instants of time. Time-based variables must have time-weighted statistics collected. Time-weighted statistics weight the value of the variable by the proportion of time that the variable is set to a value. To collect statistics on the cycle length we can use a `Response. Response` is a sub-class of `ModelElement` that can take on values within a simulation model and allows easy statistical observation of its values during the simulation run. This class provides observation-based statistical collection. Further discussion of observation-based statistics will be presented in subsequent sections.

Because this system is so simple the required performance measures can be easily computed theoretically. According to renewal theory, the probability of being in the UP state in the long run should be equal to:

$$P(UP) = \frac{\theta_u}{\theta_u + \theta_d} = \frac{1.0}{1.0 + 2.0} = 0.\overline{33}$$

where θ_u is the mean of the up-time distribution and θ_d is the mean of the down-time distribution. In addition, the expected cycle length should be

$\theta_u + \theta_d = 3.0$.

The `UpDownComponent` class extends the `ModelElement` class and has object references to instances of `RandomVariable, TWResponse, Response,` and `Counter` classes. Within the constructor of `UpDownComponent,` we need to create the instances of these objects for use within the class, as shown in the following code fragment.

```
class UpDownComponent (parent: ModelElement, name: String? =
↪  null) : ModelElement(parent, name) {
    companion object {
        const val UP = 1.0
        const val DOWN = 0.0
    }
    private val myUpTime: RandomVariable
    private val myDownTime: RandomVariable
    private val myState: TWResponse
    private val myCycleLength: Response
    private val myCountFailures: Counter
    private val myUpChangeAction = UpChangeAction()
    private val myDownChangeAction = DownChangeAction()
    private var myTimeLastUp = 0.0

    init {
        myUpTime = RandomVariable(parent = this, rSource =
↪  ExponentialRV(1.0, streamNum = 1), name = "up time")
        myDownTime = RandomVariable(parent = this, rSource =
↪  ExponentialRV(2.0, streamNum = 2), name = "down time")
        myState = TWResponse(this, name = "state")
        myCycleLength = Response(this, name = "cycle length")
        myCountFailures = Counter(this, name = "count
↪  failures")
    }
```

Lines 3 and 4 define two constants to represent the up and down states within the companion object. Lines 5-9 declare additional references needed to represent the up and down time random variables and the variables that need statistical collection (`myState, myCycleLength,` and `myCountFailures`). Notice how **unique** names have been provided for the random variables and for the response variables.

Lines 10 and 11 define and create the event actions associated with the end of the up-time and the end of the down time. The variable `myTimeLastUp` is used to keep track of the time that the component last changed into the UP state, which allows the cycle length to be collected. In lines 20-23, the random variables for the up and downtime are constructed using exponential distributions.

As show in the following code listing, the `initialize()` method sets up the component.The variable `myTimeLastUp` is set to 0.0 in order to assume that the last time the component was in the UP state started at time 0.0. Thus, we are assuming that the component starts the simulation in the UP state. Finally, in line 7 the initial event is scheduled to cause the component to go down according to the uptime distribution. This is the first event and then the component can start its regular up and down pattern. In the action associated with the change to the UP state, line 18 sets the state to UP. Line 22 schedules the time until the component goes down. Line 16 causes statistics to be collected on the value of the cycle length. The code `time - myTimeLastUp` represents the elapsed time since the value of `myTimeLastUp` was set (in line 20), which represents the cycle length. The `DownChangeAction` is very similar. Line 31 counts the number of failures (times that the component has gone down). Line 32 sets the state of the component to DOWN and line 34 schedules when the component should next transition into the UP state.

```
    override fun initialize() {
        // assume that the component starts in the UP state
         ↪  at time 0.0
```

```
        myTimeLastUp = 0.0
        myState.value = UP
        // schedule the time that it goes down
        schedule(myDownChangeAction, myUpTime.value)
    }

    private inner class UpChangeAction :
     ↪ EventAction<Nothing>() {
        override fun action(event: KSLEvent<Nothing>) {
            // this event action represents what happens when
             ↪ the component goes up
            // record the cycle length, the time btw up
             ↪ states
            myCycleLength.value = time - myTimeLastUp
            // component has just gone up, change its state
             ↪ value
            myState.value = UP
            // record the time it went up
            myTimeLastUp = time
            // schedule the down state change after the
             ↪ uptime
            schedule(myDownChangeAction, myUpTime.value)
        }
    }

    private inner class DownChangeAction :
     ↪ EventAction<Nothing>() {
        override fun action(event: KSLEvent<Nothing>) {
            // component has just gone down, change its state
             ↪ value
            myCountFailures.increment()
            myState.value = DOWN
            // schedule when it goes up after the downtime
```

```
            schedule(myUpChangeAction, myDownTime.value)
        }
    }
```

The following listing presents the code to construct and execute the simulation. This code is very similar to previously presented code for running a simulation. In line 3, the simulation is constructed. Then, in line 5, the model associated with the simulation is accessed. This model is then used to construct an instance of the `UpDownComponent` in line 5. Finally, lines 7-12 represent setting up the replication parameters of the simulation, running the simulation, and causing output to be written to the console via a `SimulationReporter.`

```
fun main() {
    // create the simulation model
    val m = Model("UpDownComponent")
    // create the model element and attach it to the model
    val tv = UpDownComponent(m)
    // set the running parameters of the simulation
    m.numberOfReplications = 5
    m.lengthOfReplication = 5000.0
    // tell the simulation model to run
    m.simulate()

    m.simulationReporter.
 ↪  printAcrossReplicationSummaryStatistics()
}
```

The results for the average time in the up state and the cycle length are consistent with the theoretically computed results.

```
Across Replication Statistical Summary Report
Sat Oct 03 09:18:39 CDT 2026
Simulation Results for Model: UpDownComponent

Number of Replications: 5
Length of Warm up period: 0.0
Length of Replications: 5000.0
--------------------------------------------------------------------------------
Response Variables
--------------------------------------------------------------------------------
Name                                    Average       Std. Dev.       Count
--------------------------------------------------------------------------------
state                                  0.329133       0.003009    5.000000
cycle length                           2.994698       0.060782    5.000000
--------------------------------------------------------------------------------


--------------------------------------------------------------------------------
Counters
--------------------------------------------------------------------------------
Name                                    Average       Std. Dev.       Count
--------------------------------------------------------------------------------
count failures                     1670.200000      33.980877    5.000000
--------------------------------------------------------------------------------
```

This example only scratches the surface of what is possible. Imagine if there were 20 components. We could easily create 20 instances of the `UpDownComponent` class and add them to the model. Even more interesting would be to define the state of the system based on which components were in the up state. This would be the beginning of modeling the reliability of a complex system. This type of modeling can be achieved by making the individual model elements (e.g. `UpDownComponent`) more reusable and allow the modeling objects to interact in complex ways. More complex modeling will be the focus of the next chapter.

4.4.4. Modeling a Simple Queueing System

This section presents a simple queueing system consisting of one server with a single waiting line. The logic of this system is essentially the same as that presented in Section 4.3 for the example where we simulated the queue by hand.

Example 4.4 (Drive Through Pharmacy)**.** This example considers a small pharmacy that has a single line for waiting customers and only one pharmacist. Assume that customers arrive at a drive through pharmacy window according to a Poisson distribution with a mean of 10 per hour. The time that it takes the pharmacist to serve the customer is random and data has indicated that the time is well modeled with an exponential distribution with a mean of 3 minutes. Customers who arrive to the pharmacy are served in the order of arrival and enough space is available within the parking area of the adjacent grocery store to accommodate any waiting customers.

The drive through pharmacy system can be conceptualized as a single server waiting line system, where the server is the pharmacist. An idealized representation of this system is shown in Figure 4.8. If the pharmacist is busy serving a customer, then additional customers will wait in line. In such a situation, management might be interested in how long customers wait in line, before being served by the pharmacist. In addition, management might want to predict if the number of waiting cars will be large. Finally, they might want to estimate the utilization of the pharmacist in order to ensure that he or she is not too busy.

When modeling the system first question to ask is: *What is the system*? In this situation, the system is the pharmacist and the potential customers as idealized in Figure 4.8. Now you should consider the entities of the system. An entity is a conceptual thing of importance that flows through a system potentially using the resources of the system. Therefore, one of the first questions to ask when developing a model is: *What are the entities*? In this

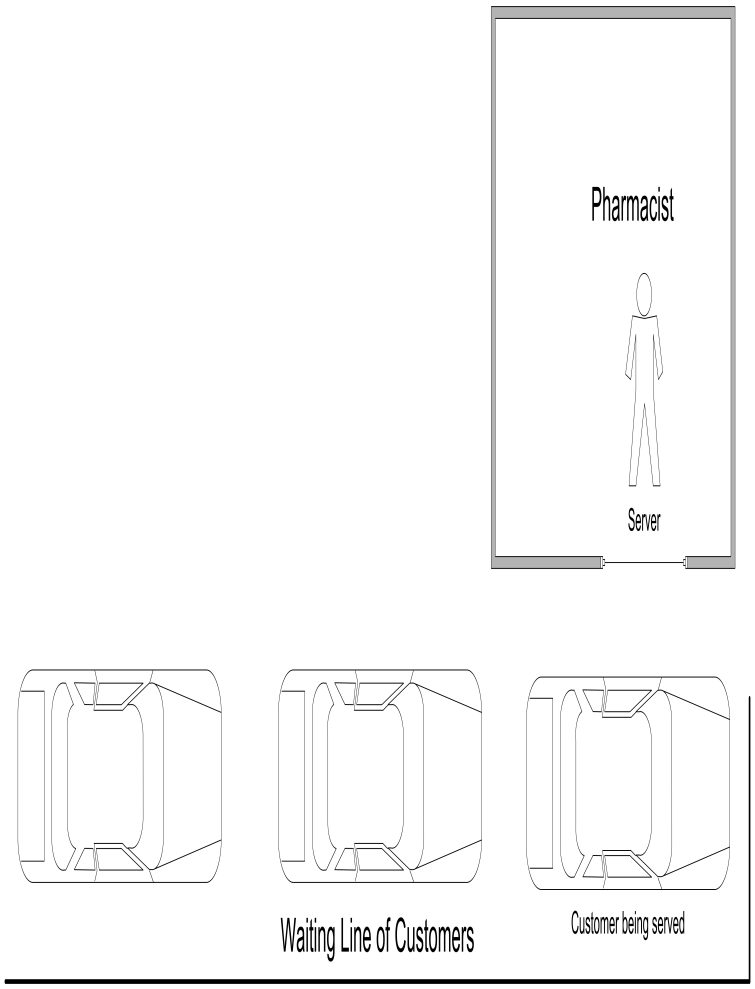


Figure 4.8.: Drive Through Pharmacy

situation, the entities are the customers that need to use the pharmacy. This is because customers are discrete things that enter the system, flow through the system, and then depart the system.

Since entities often use things as they flow through the system, a natural question is to ask: *What are the resources that are used by the entities*? A resource is something that is used by the entities and that may constrain the flow of the entities within the system. Another way to think of resources is to think of the things that provide service in the system. In this situation, the entities "use" the pharmacist in order to get their medicine. Thus, the pharmacist is a resource.

Another useful conceptual modeling tool is the *activity diagram*. An *activity* is an operation that takes time to complete. An activity is associated with the state of an object over an interval of time. Activities are defined by the occurrence of two events which represent the activity's beginning time and ending time and mark the entrance and exit of the state associated with the activity. An activity diagram is a pictorial representation of the process (steps of activities) for an entity and its interaction with resources while within the system. If the entity is a temporary entity (i.e. it flows through the system) the activity diagram is called an activity flow diagram. If the entity is permanent (i.e. it remains in the system throughout its life) the activity diagram is called an activity cycle diagram. The notation of an activity diagram is very simple, and can be augmented as needed to explain additional concepts:

- Queues: shown as a circle with queue labeled inside
- Activities: shown as a rectangle with appropriate label inside
- Resources: shown as small circles with resource labeled inside
- Lines/arcs: indicating flow (precedence ordering) for engagement of entities in activities or for obtaining resources. Dotted lines are used to indicate the seizing and releasing of resources.
- zigzag lines: indicate the creation or destruction of entities

Activity diagrams are especially useful for illustrating how entities interact with resources. In addition, activity diagrams help in finding the events and in identifying some the state changes that must be modeled. Activity diagrams are easy to build by hand and serve as a useful communication mechanism. Since they have a simple set of symbols, it is easy to use an activity diagram to communicate with people who have little simulation background. Activity diagrams are an excellent mechanism to document a conceptual model of the system before building the model.

Figure 4.9 shows the activity diagram for the pharmacy situation. The diagram describes the life of an entity within the system. The zigzag lines at the top of the diagram indicate the creation of an entity. Consider following the life of the customer through the pharmacy. Following the direction of the arrows, the customers are first created and then enter the queue. Notice that the diagram clearly shows that there is a queue for the drive-through customers. You should think of the entity flowing through the diagram. As it flows through the queue, the customer attempts to start an activity. In this case, the activity requires a resource. The pharmacist is shown as a resource (circle) next to the rectangle that represents the service activity.

The customer requires the resource in order to start its service activity. This is indicated by the dashed arrow from the pharmacist (resource) to the top of the service activity rectangle. If the customer does not get the resource, they wait in the queue. Once they receive the number of units of the resource requested, they proceed with the activity. The activity represents a delay of time and in this case the resource is used throughout the delay. After the activity is completed, the customer releases the pharmacist (resource). This is indicated by another dashed arrow, with the direction indicating that the units of the resource are being put back or released. After the customer completes its service activity, the customer leaves the system. This is indicated with the zigzag lines going to no-where and indicating that the object leaves the system and is disposed The conceptual model of this system can be summarized as follows:

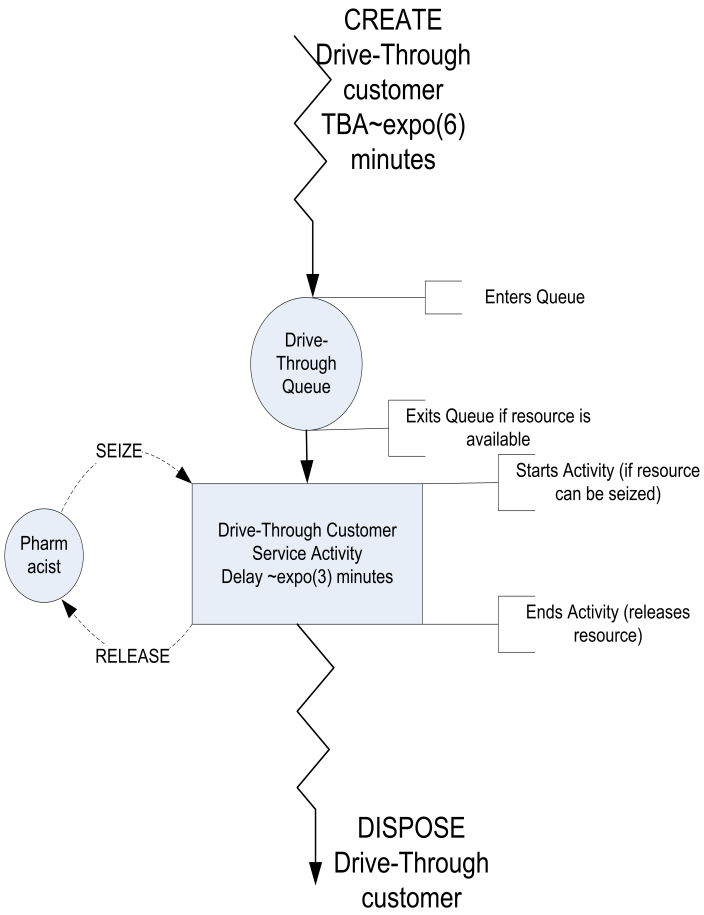


Figure 4.9.: Activity Diagram of Drive Through Pharmacy

- System: The system has a pharmacist that acts as a resource, customers that act as entities, and a queue to hold the waiting customers. The state of the system includes the number of customers in the system, in the queue, and in service.
- Events: Arrivals of customers to the system, which occur within an inter-event time that is exponentially distributed with a mean of 6 minutes.
- Activities: The service time of the customers are exponentially distributed with a mean of 3 minutes.
- Conditional delays: A conditional delay occurs when an entity has to wait for a condition to occur in order to proceed. In this system, the customer may have to wait in a queue until the pharmacist becomes available.

With an activity diagram and pseudo-code to represent a solid conceptual understanding of the system, you can begin the model development process.

In the current example, pharmacy customers arrive according to a Poisson process with a mean of $\lambda = 10$ per hour. According to probability theory, this implies that the time between arrivals is exponentially distributed with a mean of $(1/\lambda)$. Thus, for this situation, the mean time between arrivals is 6 minutes.

$$\frac{1}{\lambda} = \frac{1 \text{ hour}}{10 \text{ customers}} \times \frac{60 \text{ minutes}}{1 \text{ hour}} = \frac{6 \text{ minutes}}{\text{customers}}$$

Let's assume that the pharmacy is open 24 hours a day, 7 days a week. In other words, it is always open. In addition, assume that the arrival process does not vary with respect to time. Finally, assume that management is interested in understanding the long term behavior of this system in terms of the average waiting time of customers, the average number of customers, and the utilization of the pharmacist.

To simulate this situation over time, you must specify how long to run the model. Ideally, since management is interested in long run performance, you should run the model for an infinite amount of time to get long term performance; however, you probably don't want to wait that long! For the sake of simplicity, assume that 250 hours (15,000 minutes) of operation is long enough.

The logic of this model follows very closely the discussion of the bank teller example. Let's define the following variable:

- Let t represent the current simulation clock time.
- Let c represent the number of available pharmacists
- Let $N(t)$ represent the number of customers in the system at any time t.
- Let $Q(t)$ represent the number of customers waiting in line for the at any time t.
- Let $B(t)$ represent the number of pharmacists that are busy at any time t.
- Let TBA_i represent the time between arrivals, which we will assume is exponentially distributed with a mean of 6 minutes.
- Let ST_i represent the service time of the i^{th} customer, which we will assume is exponentially distributed with a mean of 3 minutes.
- Let E_a represent the arrival event.
- Let E_s represent the end of service event.
- Let K represent the number of customers processed

Within the KSL model, we will model $N(t)$, $Q(t)$, and $B(t)$ with instances of the `TWResponse` class The use of the `TWResponse` class will automate the collection of time averages for these variables as discussed in Section 4.3. Both TBA_i and ST_i will be modeled with instances of the `RandomVariable` class instantiated with instances of the `ExponentialRV` class. The pseudo-code for this situation is as follows.

Arrival Actions for Event E_a

```
N(t) = N(t) + 1
if (B(t) < c)
  B(t) = B(t) + 1
  schedule E_s at time t + ST_i
else
  Q(t) = Q(t) + 1
endif
schedule E_a at time t + TBA_i
```

In the arrival actions, first we increment the number of customers in the system. Then, the number of busy pharmacists is compared to the number of pharmacists that are available. If there is an available pharmacist, the number of busy pharmacists is incremented and the end of service for the arriving customer is scheduled. If all the pharmacists are busy, then the customer must wait in the queue, which is indicated by incrementing the number in the queue. To continue the arrival process, the arrival of the next customer is scheduled.

End of Service Actions for Event E_s

```
B(t) = B(t) - 1
if (Q(t) > 0)
  Q(t) = Q(t) - 1
  B(t) = B(t) + 1
  schedule E_s at time t + ST_i
endif
N(t) = N(t) - 1
K = K + 1
```

In the end of service actions, the number of busy pharmacists is decreased by one because the pharmacist has completed service for the departing customer. Then the queue is checked to see if it has customers. If the queue has customers, then a customer is removed from the queue (decreasing

the number in queue) and the number of busy pharmacists is increased by one. In addition, the end of service event is scheduled. Finally, the number of customers in the system is decremented and the count of the total customers processes is incremented.

The following code listing presents the definition of the variables and their creation within the KSL. The drive through pharmacy system is modeled via a class `DriveThroughPharmacy` that sub-classes from `ModelElement` to provide the ability to schedule events. The `RandomVariable` instances `myServiceRV` and `myArrivalRV` are used to represent the ST_i and TBA_i random variables. $B(t)$, $N(t)$, and $Q(t)$ are modeled with the objects `myNumBusy`, `myNS`, and `myQ`, respectively, all instances of the `TWResponse` class. The tabulation of the number of processed customers, K, is modeled with a KSL counter, using the `Counter` class.

```
class DriveThroughPharmacy(
    parent: ModelElement, numServers: Int = 1,
    name: String? = null
) : ModelElement(parent, name) {
    init{
        require(numServers >= 1) {"The number of pharmacists
 ↪  must be >= 1"}
    }

    var numPharmacists = numServers
        set(value) {
            require(value >= 1){"The number of pharmacists
 ↪  must be >= 1"}
            field = value
        }

    private val myServiceRV: RandomVariable = RandomVariable(
        parent = this,
        rSource = ExponentialRV(mean = 0.5, streamNum = 2),
 ↪  name = "ServiceRV"
```

```
    )
    val serviceRV: RandomVariableCIfc
        get() = myServiceRV

    private val myArrivalRV: RandomVariable = RandomVariable(
        parent = this,
        rSource = ExponentialRV(1.0, 1), name = "ArrivalRV"
    )
    val arrivalRV: RandomVariableCIfc
        get() = myArrivalRV

    private val myNumInQ: TWResponse = TWResponse(parent =
     ↪  this, name = "PharmacyQ")
    val numInQ: TWResponseCIfc
        get() = myNumInQ

    private val myNumBusy: TWResponse = TWResponse(parent =
     ↪  this, name = "NumBusy")
    private val myNS: TWResponse = TWResponse(parent = this,
     ↪  name = "# in System")
    private val myNumCustomers: Counter = Counter(parent =
     ↪  this, name = "Num Served")
    private val myTotal: AggregateTWResponse =
     ↪  AggregateTWResponse(parent = this, name = "aggregate
     ↪  # in system")
    private val myArrivalEventAction: ArrivalEventAction =
     ↪  ArrivalEventAction()
    private val myEndServiceEventAction:
     ↪  EndServiceEventAction = EndServiceEventAction()

    init {
        myTotal.observe(myNumInQ)
        myTotal.observe(myNumBusy)
```

```
    }
```

The main constructor of the `DriveThroughPharmacy` class checks for valid input parameters, instantiates the KSL model elements, and instantiates the arrival and end of service event actions.

As can be seen in the following code, the arrival event is represented by the inner class `ArrivalEventAction` extending the abstract class `EventAction` to model the arrival event action. The Kotlin code closely follows the pseudo-code. Notice the use of the `initialize()` method to schedule the first arrival event. The `initialize()` method is called just prior to the start of the replication for the simulation. The modeler can think of the `initialize()` method as being called at time $t^- = 0$.

```
    override fun initialize() {
        super.initialize()
        // start the arrivals
        schedule(myArrivalEventAction, myArrivalRV)
    }

    private inner class ArrivalEventAction :
     ↪ EventAction<Nothing>() {
        override fun action(event: KSLEvent<Nothing>) {
            myNS.increment() // new customer arrived
            if (myNumBusy.value < numPharmacists) { // server
             ↪ available
                myNumBusy.increment() // make server busy
                // schedule end of service
                schedule(myEndServiceEventAction,
↪ myServiceRV)
            } else {
                myNumInQ.increment() // customer must wait
            }
```

```
            // always schedule the next arrival
            schedule(myArrivalEventAction, myArrivalRV)
        }
    }
```

In line 4 the first arrival event is scheduled within the `initialize()` method. Within the `ArrivalEventAction` class implementation of the `action()` method, we see the number of customers in the system incremented, the status of the pharmacists checked, and customers either starting service or having to wait in the queue.

The following code fragment presents the logic associated with the end of service event.

```
    private inner class EndServiceEventAction :
     ↪ EventAction<Nothing>() {
        override fun action(event: KSLEvent<Nothing>) {
            myNumBusy.decrement() // customer is leaving
↪   server is freed
            if (myNumInQ.value > 0) { // queue is not empty
                myNumInQ.decrement() //remove the next
↪   customer
                myNumBusy.increment() // make server busy
                // schedule end of service
                schedule(myEndServiceEventAction,
↪   myServiceRV)
            }
            myNS.decrement() // customer left system
            myNumCustomers.increment()
        }
    }
```

First the number of busy servers is decremented because a pharmacist is

becoming idle. Then, the queue is checked to see if it is not empty. If the queue is not empty, then the next customer must be removed, the server made busy again and the customer scheduled into service. Finally, the `TWResponse` variable, `myNS,` indicates that a customer has departed the system and the `Counter` for the number of customers processed is incremented.

The following method can be used to run the model based on a desired number of servers.

```
fun main() {
    val model = Model("Drive Through Pharmacy",
     ↪  autoCSVReports = true)
    model.numberOfReplications = 30
    model.lengthOfReplication = 20000.0
    model.lengthOfReplicationWarmUp = 5000.0
    // add DriveThroughPharmacy to the main model
    val dtp = DriveThroughPharmacy(model, 1)
    dtp.arrivalRV.initialRandomSource = ExponentialRV(6.0, 1)
    dtp.serviceRV.initialRandomSource = ExponentialRV(3.0, 2)
    model.simulate()
    model.print()
}
```

As will be discussed in Chapter 5, this model is really an infinite horizon simulation. We will approximate this infinite horizon with a finite horizon of 250 hours (or 15,000) minutes. The setting of the length of the replication is 20000 with a warm up period (see Chapter 5) of 5000 minutes. This leaves (20,000 - 5,000) = 15,000 minutes for the observations of the simulation. Each *replication* provides 15000 minutes of observation of the system and the number of replications is specified as 30. That is, we are repeating the observations over 30 independent and identically distributed replications.

The results indicate that the utilization of the pharmacist is about 50%. This means that about 50% of the time the pharmacist was busy. For this

type of system, this is probably not a bad utilization, considering that the pharmacist probably has other in-store duties. The reports also indicate that there was less than one customer on average waiting for service.

```
Name: Drive Through Pharmacy
Last Executive stopping message:
    Scheduled end event occurred at time 20000.0
Replication Process Information:
--------------------------------
Iterative Process Name: Model: Replication Process
Beginning Execution Time: 2022-11-29T22:58:06.768592Z
End Execution Time: 2022-11-29T22:58:07.479916Z
Elapsed Execution Time: 711.324ms
Max Allowed Execution Time: Not Specified
Done Flag: true
Has Next: false
Current State: EndedState
Ending Status Indicator: COMPLETED_ALL_STEPS
Stopping Message: Completed all steps.
--------------------------------
Experiment Name: Experiment_1
Experiment ID: 2
Planned number of replications: 30
Replication initialization option: true
Antithetic option: false
Reset start stream option: false
Reset next sub-stream option: true
Number of stream advancements: 0
Planned time horizon for replication: 20000.0
Warm up time period for replication: 5000.0
Maximum allowed replication execution time not specified.
Current Replication Number: 30
--------------------------------
```

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                                                    Count          Average      Half-
Width
-------------------------------------------------------------------------------------------
PharmacyQ                                                  30           0.5025          0
NumBusy                                                    30           0.5035          0
# in System                                               30           1.0060          0
aggregate # in system                                      30           1.0060          0
Num Served                                                 30        2513.2667         17
-------------------------------------------------------------------------------------------
```

This single server waiting line system is a very common situation in practice. In fact, this exact situation has been studied mathematically through a branch of operations research called queueing theory. For specific modeling situations, formulas for the long term performance of queueing systems can be derived. This particular pharmacy model happens to be an example of an M/M/1 queueing model. The first M stands for Markov arrivals, the second M stands for Markov service times, and the 1 represents a single server. Markov was a famous mathematician who examined the exponential distribution and its properties. According to queueing theory, the expected number of customer in queue, L_q, for the M/M/1 model is:

$$L_q = \frac{\rho^2}{1-\rho}$$
$$\rho = \lambda/\mu$$
$$\lambda = \text{arrival rate to queue}$$
$$\mu = \text{service rate}$$

In addition, the expected waiting time in queue is given by $W_q = L_q/\lambda$. In the pharmacy model, $\lambda = 1/6$, i.e. 1 customer every 6 minutes on average,

and $\mu = 1/3$, i.e. 1 customer every 3 minutes on average. The quantity, ρ, is called the utilization of the server. Using these values in the formulas for L_q and W_q results in:

$$\begin{aligned} \rho &= 0.5 \\ L_q &= \frac{0.5 \times 0.5}{1 - 0.5} = 0.5 \\ W_q &= \frac{0.5}{1/6} = 3 \text{ minutes} \end{aligned}$$

In comparing these analytical results with the simulation results, you can see that they match to within statistical error. The appendix presents an analytical treatment of queues. These analytical results are available for this special case because the arrival and service distributions are exponential; however, simple analytical results are not available for many common distributions, e.g. lognormal. With simulation, you can easily estimate the above quantities as well as many other performance measures of interest for wide ranging queueing situations. For example, through simulation you can easily estimate the chance that there are 3 or more cars waiting.

4.4.5. More Details About the Pharmacy Model Implementation

There are a few additional concepts to note about the implementation of the pharmacy model. The first set of items to notice is how the random variables and statistical response variables were implemented. In the following code snippet, we see that random variables for the service time and time between arrivals were defined using Kotlin private properties. This encapsulates the random variables within the simulation model. However, it can be useful to allow users of the code to have some access to the underlying objects. For this purpose, two public properties `serviceRV` and `arrivalRV` were defined.

```
    private val myServiceRV: RandomVariable = RandomVariable(
        parent = this,
        rSource = ExponentialRV(mean = 0.5, streamNum = 2),
↪   name = "ServiceRV"
    )
    val serviceRV: RandomVariableCIfc
        get() = myServiceRV

    private val myArrivalRV: RandomVariable = RandomVariable(
        parent = this,
        rSource = ExponentialRV(1.0, 1), name = "ArrivalRV"
    )
    val arrivalRV: RandomVariableCIfc
        get() = myArrivalRV
```

These properties return instances of the `RandomVariableCIfc` interface. This interface limits what can be changed about the underlying random variable. This interface allows the user to change the initial random source associated with the random variable. As noted in the interface comments, this needs to be controlled in such a manner to ensure that each replication starts with the same initial conditions. A naive user may not realize the implications of these changes and thus there are some limitations imposed.

```
interface RandomVariableCIfc : StreamOptionIfc, IdentityIfc {

    /**
     * Provides a reference to the underlying source of
↪   randomness
     * to initialize each replication.
     * Controls the underlying source for the RandomVariable.
↪   This is the
     * source to which each replication will be initialized.
↪   This is only used
```

```
     * when the replication is initialized. Changing the
↪  reference has no effect
     * during a replication, since the random variable will
↪  continue to use
     * the reference returned by property randomSource.
↪  Please also see the
     * discussion in the class documentation.
     *
     * The initial random source should not be changed while
↪  the model is running.
     */
    var initialRandomSource: RVariableIfc

    fun asString(): String

}
```

Similarly for simulation response variables, there should be controlled access that limits how the properties can be accessed from outside of the model element. In the following code snippet, a private property to track the state of the queue is defined. Since the `TWResponse` class has public methods and properties that permit the user to change its values, we do not want to fully expose this internal state to outside clients.

```
    private val myNumInQ: TWResponse = TWResponse(parent =
↪  this, name = "PharmacyQ")
    val numInQ: TWResponseCIfc
        get() = myNumInQ
```

However, it is useful for outside clients to have access to immutable properties and to summary statistics related to the response variable. This is permitted by the use of the `TWResponseCIfc` interface, which limits access.

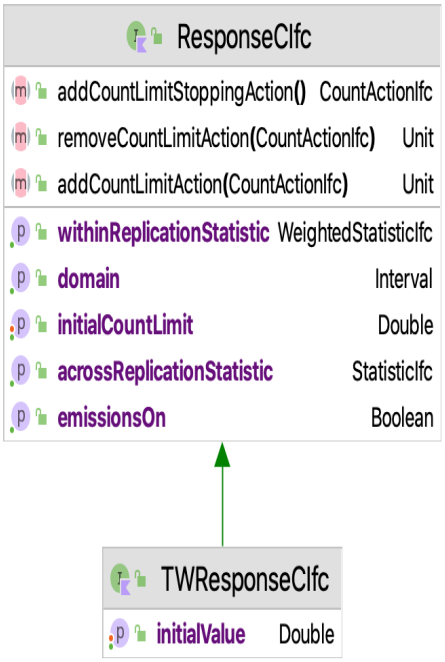


Figure 4.10.: Controlled Access to Responses

As indicated in Figure 4.10, the two interfaces `TWResponseCIfc` and `ResponseCIfc` limit access to time-weighted and tally-based response variables to access within and across replication statistics and to add count actions. Count actions permit actions to take place when the number of observations reaches a particular value. One use of count actions is to stop the simulation when a fixed number of observations have been observed.

Within the implementation, there are a couple of additional items to note. The KSL allows for the definition of aggregate responses. Aggregate responses observe other responses and permit the definition of derived responses. In the pharmacy model implementation, there is an aggregate response called `myTotal.` The relevant code is shown below.

```
    private val myNumBusy: TWResponse = TWResponse(parent =
     ↪  this, name = "NumBusy")
    private val myNS: TWResponse = TWResponse(parent = this,
     ↪  name = "# in System")
    private val myNumCustomers: Counter = Counter(parent =
     ↪  this, name = "Num Served")
    private val myTotal: AggregateTWResponse =
     ↪  AggregateTWResponse(parent = this, name = "aggregate
     ↪  # in system")
    private val myArrivalEventAction: ArrivalEventAction =
     ↪  ArrivalEventAction()
    private val myEndServiceEventAction:
     ↪  EndServiceEventAction = EndServiceEventAction()

    init {
        myTotal.observe(myNumInQ)
        myTotal.observe(myNumBusy)
    }
```

This code defines an aggregate response that observes two variables, `myNumInQ` and `myNumBusy.` Thus, whenever either of these two variables

change, the aggregate response is updated to ensure that it represents the *total* of the two variable. Thus, `myTotal` represents the total number of customers in the system at any time t. As can be noted in the output results, this response has the same statistics as the response collected directly by the variable `myNS` labeled "# in System".

Finally, the implementation has a couple of interesting new items within the main execution method. Notice that in line 1 in the model constructor that the parameter `autoCSVReports` is set to true. This will cause comma separated value files to be produced in the model's default output directory that contains all of the within and across replication statistics. In addition, note how the initial random sources for the arrival and service distributions are set in lines 8 and 9. Finally, note the use of `model.print()`, which causes the default simulation execution information and results to be printed to the console.

```
fun main() {
    val model = Model("Drive Through Pharmacy",
     ↪  autoCSVReports = true)
    model.numberOfReplications = 30
    model.lengthOfReplication = 20000.0
    model.lengthOfReplicationWarmUp = 5000.0
    // add DriveThroughPharmacy to the main model
    val dtp = DriveThroughPharmacy(model, 1)
    dtp.arrivalRV.initialRandomSource = ExponentialRV(6.0, 1)
    dtp.serviceRV.initialRandomSource = ExponentialRV(3.0, 2)
    model.simulate()
    model.print()
}
```

In the next section, we redo the pharmacy model using some new KSL constructs that facilitate the modeling of queues.

4.5. Enhancing the Drive Through Pharmacy Model

In this section, we re-implement the drive through pharmacy model to illustrate a few more KSL constructs.

Example 4.5 (Enhance Drive Through Pharmacy)**.** Consider again the drive through pharmacy situation. Assume that customers arrive at a drive through pharmacy window according to a Poisson distribution with a mean of 10 per hour. The time that it takes the pharmacist to serve the customer is random and data has indicated that the time is well modeled with an exponential distribution with a mean of 3 minutes. Customers who arrive to the pharmacy are served in the order of arrival and enough space is available within the parking area of the adjacent grocery store to accommodate any waiting customers. For this situation, we are interested in estimating the following performance measures:

a. Expected time spent waiting in the queue.
b. Probability of having 0, 1, 2, 3, etc. in the queue upon arrival.
c. The utilization of the pharmacist.
d. The number of times that the pharmacist attempted to serve a customer.
e. A histogram for the total time spent in the system.

The solution to this situation will involve six new KSL constructs.

- `SResource`: To model the pharmacist as a resource and automatically collect resource statistics.
- `Queue`: To model the customer waiting line.
- `QObject`: To model the customers and facilitate statistical collection on waiting time in the queue.
- `IntegerFrequencyResponse`: To collect statistics on the number of customers waiting when another customer arrives.
- `HistogramResponse`: To collect a histogram on the system times of the customers.

- `EventGenerator`: To model the arrival pattern of the customers.

The purpose is to cover the basics of these classes for future modeling. The `SResource` class represents a *simple* resource that has a unchangeable capacity. The `Queue` and `QObject` classes facilitate the holding of entities within waiting lines or queues, while the `EventGenerator` class codifies the basics of generating events according to a repetitive pattern. We will start by expanding on the concept of a resource.

4.5.1. Modeling a Simple Resource

As we saw in the drive through pharmacy example, when the customer arrives they need the pharmacist in order to proceed. As noted in the activity diagram depicted in Figure 4.9, we denoted the pharmacist as a resource (circle) in the diagram. A resource is something that is needed by the objects *(or entities)* that experience the system's activities. In the pharmacy example, the resource (pharmacist) was required to start the service activity, denoted with a arrow with the word "seize" in the activity diagram, pointing from the resource to the start of the activity. After the activity is completed, there is a corresponding arrow labeled "release" pointing from the end of the activity back to the resource. This arrow denotes the returning of the used resource units back to the pool of available units.

For the purposes of this situation, we are going to represent the concept of a simple resource with the aptly named `SResource` class (for simple resource). Let's take a look at how the KSL represents a simple resource.

A resource has a capacity that represents the maximum number of units that it can have available at any time. When a resource is seized some amount of units become busy (or allocated). When the units are no longer needed, they are released. If we let $A(t)$ be the number of available units, $B(t)$ be the number of busy units and c be the capacity of the resource, we have that $c = A(t) + B(t)$ or $A(t) = c - B(t)$. A resource is considered

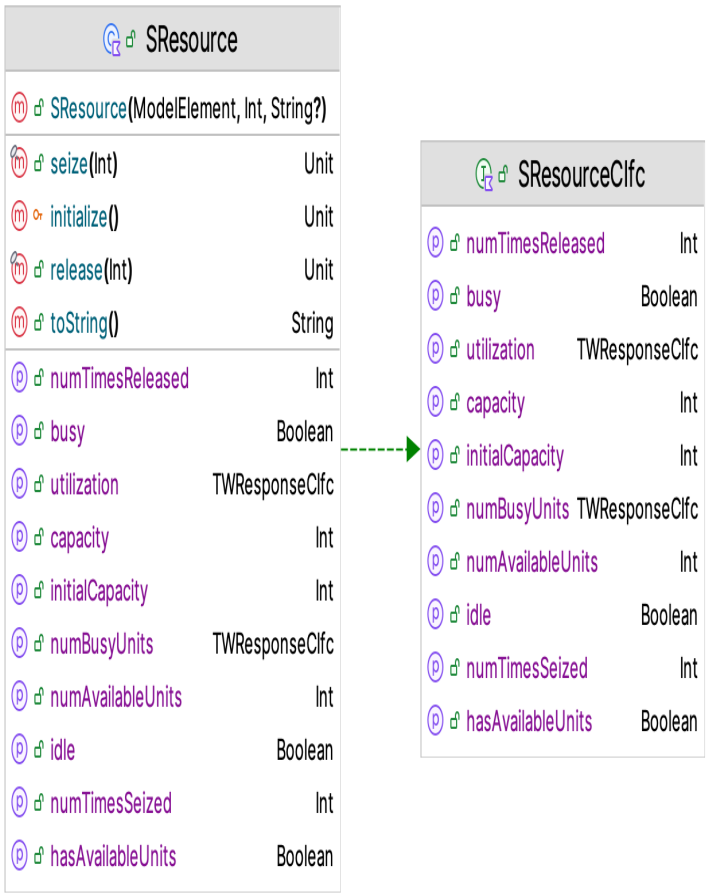


Figure 4.11.: A Simple Resource Class

busy if $B(t) > 0$. That is, a resource is busy if some units are allocated. A resource is considered idle if no units are busy. That is, $B(t) = 0$, which implies that $A(t) = c$.

If the number of available units of a resource are unable to meet the number of units required by a "customer", then we need to decide what to do. To simplify this modeling, we are going to assume two things 1) customers only request 1 unit of the resource at a time, and 2) if the request cannot be immediately supplied the customer will wait in an associated queue. These latter two assumptions are essentially what we have previously assumed in the modeling of the pharmacy situation. Modeling often requires the use of simplifying assumptions.

The `SResource` class of Figure 4.11 has functions `seize()` and `release()` which take (seize) and return (release) units of the resource. It also has properties (`busy` and `idle`) that indicate if the resource is busy or idle, respectively. In addition, the property `numBusyUnits` represents $B(t)$ and the property `numAvailableUnits` represents $A(t)$. For convenience, the `hasAvailableUnits` property indicates if the resource has units that can be seized. Finally, the number of times the resource is seized and released are tabulated. The main performance measures are time weighted response variables for tabulating the time-average number of busy units and the time-average instantaneous utilization of the resource. Instantaneous utilization, $U(t)$ is governed by tracking $U(t) = B(t)/c$.

4.5.2. Modeling a Queue with Statistical Collection

The `Queue` class, is used to model waiting lines. The `Queue` class is a subclass of `ModelElement` that is able to hold instances of the class `QObject` and will *automatically* collect statistics on the number in the queue and the time spent in the queue.

Figure 4.12 illustrates the classes involved when using the `Queue` and `QObject` classes. The first thing to note is that `QObject` is an inner class

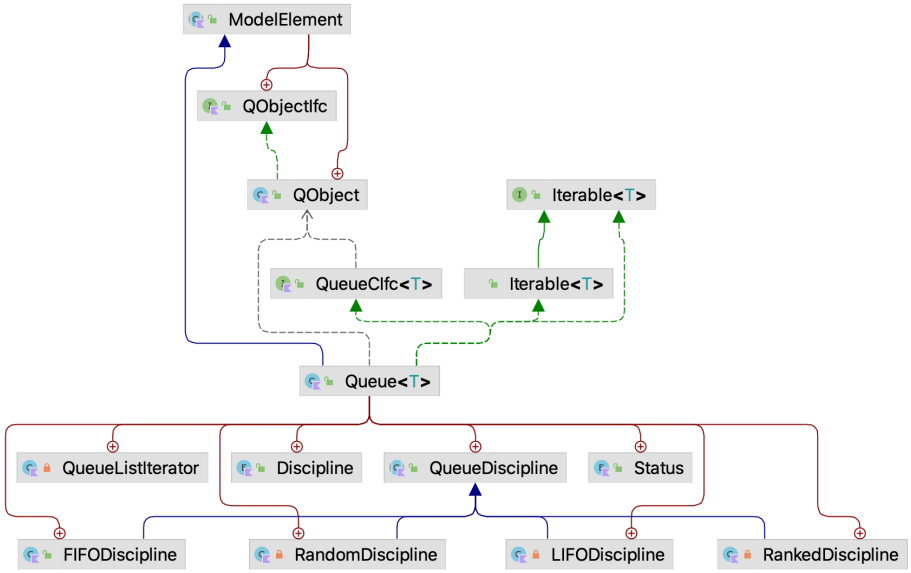


Figure 4.12.: Overview of Classes Related to Queue and QObject

of ModelElement. This permits subclasses of ModelElement to create instances of QObject that have access to all the architecture of the model but are not model elements. That is, instances of QObject are transitory and are not added to the model element hierarchy that has been previously described. Users create QObject instances, use them to model elements of interest in the system that may experience waiting and then allow the Kotlin garbage collector to deallocate the memory associated with the instances. In Figure 4.12, we also see the class Queue<T: ModelElement.QObject is a sub-class of ModelElement that is parameterized by sub-types of QObject. Thus, users can develop sub-classes of QObject and still use Queue to hold and collect statistics on those object instances. As noted in the figure, Queue also implements iterable. The last item to notice from Figure 4.12 is that queues can be governed by four basic queue disciplines: FIFO, LIFO, random, and ranked. In the case of a ranked queue, the queue is ordered by the priority property of QObject. The KSL permits the changing of the queue discipline during the simulation. The default queue discipline is FIFO.

Figure 4.13 presents the properties and methods of the Queue and QObject classes. Here we can see that the Queue class has some standard methods for inserting and removing QObject instances. For the purposes of this chapter, the most noteworthy of these are:

```
fun enqueue(qObject: T, priority: Int = qObject.priority,
 ↪  obj: Any? = qObject.attachedObject)

fun peekNext(): T?

fun removeNext(): T?
```

The enqueue method places QObject instances into the queue using the supplied priority. It also allows the user to attach an instance of Any to the QObject instance. The peekNext method provides a reference to the next QObject to be removed according the specified queue discipline and

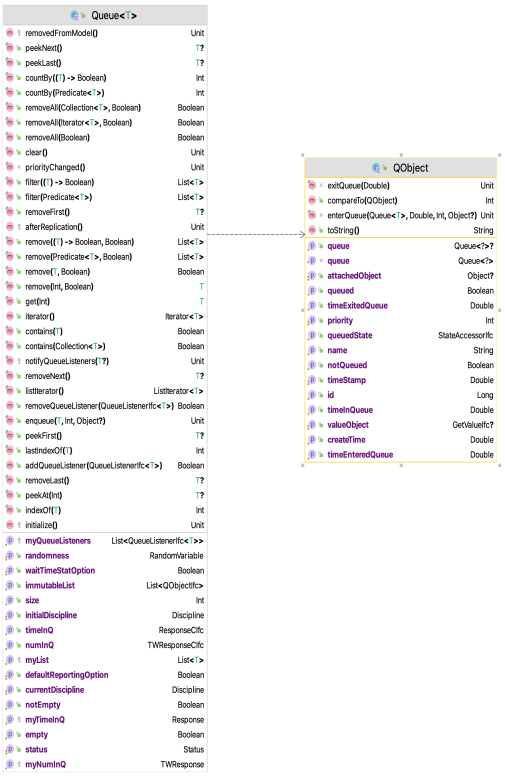


Figure 4.13.: Properties and Methods of Queue and QObject

the `removeNext` method will remove the next `QObject` instance. During the enqueue and removal processes statistics are tabulated on the number of items in the queue and how much time the items spent in the queue. These responses are available via the `timeInQ` and `numInQ` properties. We will see how to use these classes within the revised pharmacy model. Before proceeding with reviewing the implementation, let us examine the `EventGenerator` class.

4.5.3. Modeling a Repeating Event Pattern

The `EventGenerator` class allows for the periodic generation of events similar to that achieved by "CREATE" modules in other simulation languages. This class works in conjunction with the `GeneratorActionIfc` interface, which is used to listen and react to the events that are generated by this class. Users of the class can supply an instance of an `GeneratorActionIfc` to provide the actions that take place when the event occurs. Alternatively, if no `GeneratorActionIfc` is supplied, by default the `generator(event: KSLEvent)` method of this class will be called when the event occurs. Thus, sub-classes can simply override this method to provide behavior for when the event occurs. If no instance of an `GeneratorActionIfc` instance is supplied and the `generate()` method is not overridden, then the events will still occur; however, no meaningful actions will take place. The key input parameters to the `EventGenerator` include:

time until the first event This parameter is specified with an object that implements the `RandomIfc.` It should be used to represent a positive real value that represents the time after time 0.0 for the first event to occur. If this parameter is not supplied, then the first event occurs at time 0.0. This parameter is specified by the `initialTimeUntilFirstEvent` property.

time between events This parameter is specified with an object that implements the `RandomIfc.` It should be used to represent a positive real value that represents the time between events. If this parameter

is not supplied, then the time between events is positive infinity. This parameter is specified by the `initialTimeBetweenEvents` property.

time until last event This parameter is specified with an object that implements the `RandomIfc`. It should be used to represent a positive real value that represents the time that the generator should stop generating. When the generator is created, this variable is used to set the ending time of the generator. Each time an event is to be scheduled the ending time is checked. If the time of the next event is past this time, then the generator is turned off and the event will not be scheduled. The default is positive infinity. This parameter is specified by the `initialEndingTime` property.

maximum number of events A value of type long that supplies the maximum number of events to generate. Each time an event is to be scheduled, the maximum number of events is checked. If the maximum has been reached, then the generator is turned off. The default is `Long.MAX_VALUE`. This parameter cannot be `Long.MAX_VALUE` when the time until next always returns a value of 0.0. This is specified by the `initialMaximumNumberOfEvents` property.

the generator action This parameter can be used to supply an instance of `GeneratorActionIfc` interface to supply logic to occur when the event occurs.

The most common use case for an `EventGenerator` is very similar to a compound Poisson process. The `EventGenerator` is setup to run with a time between events until the simulation completes; however, there are a number of other possibilities that are facilitated through various methods associated with the `EventGenerator` class. The first possibility is to subclass the `EventGenerator` to make a custom generator for objects of a specific class. To facilitate this the user need only over ride the generate() method that is part of the `EventGenerator` class. For example, you could design classes to create customers, parts, trucks, demands, etc.

In addition to customization through sub-classes, there are a number of useful methods that are available for controlling the `EventGenerator`.

`turnOffGenerator()` This method allows an `EventGenerator` to be turned off. The next scheduled generation event will not occur. This method will cancel a previously scheduled generation event if one exists. No future events will be scheduled after turning off the generator. Once the generator has been turned off, it cannot be restarted until the next replication.

`turnOnGenerator(t: GetValueIfc)` If the generator was not started upon initialization at the beginning of a replication, then this method can be used to start the generator. The generator will be started t time units after the call. If this method is used when the generator is already started it does nothing. If this method is used after the generator is done it does nothing. If this method is used after the generator has been suspended it does nothing. In other words, if the generator is already on, this method does nothing.

`suspend()` This method suspends the event generation pattern. The generator is still on, but the generation of events is suspended. The next scheduled generation event is canceled.

`resume()` If the generator is suspended then this method causes the event generator to proceed with the event generation pattern by scheduling a new event according to the time between event distribution.

`suspended` Checks if the generator is suspended.

`done` Checks if the generator has been turned off. The generator can be turned off via the `turnOffGenerator()` method or it may turn off when it has reached its time until last event or if the maximum number of events is reached. As previously noted, once a generator has been turned off, it cannot be turned on again within the same replication.

In considering these methods, a generator can turn itself off (as an action) within or caused by the code within its `generate()`method or in the supplied `GeneratorActionIfc` interface. It might also `suspend()` itself in a similar manner. Of course, a class that has a reference to the generator may also turn it off or suspend it. To resume a suspended event generator, it is necessary to schedule an event whose action invokes the `resume()` method. Obviously, this can be within a sub-class of `EventGenerator` or within another class that has a reference to the event generator.

4.5.4. Collecting More Detailed Statistics

Example 4.5 also has requirements to collect the probability associated with the number of customers in the queue when a new customer arrives and for collecting a histogram for the time spent in the system for the customers. These requirements will be implemented using the `IntegerFrequencyResponse` and `HistogramResponse` classes. The `IntegerFrequencyResponse` and `HistogramResponse` classes are implementations of the `IntegerFrequency` and `Histogram` classes described in Section 3.1.2. The `HistogramResponse` class tabulates counts and frequencies of observed data over a set of contiguous intervals. The `IntegerFrequencyResponse` class will also tabulate count frequencies when the values are only integers. Both of these classes are designed to be used within a KSL discrete-event simulation. One item to note is that both classes will collect statistics from within a warm up period (see Section 5.6.1). In fact, these classes report statistics based on observations from every replication of the simulation.

4.5.5. Implementing the Enhanced Pharmacy Model

Now we are ready to review the revised implementation of the drive through pharmacy model which puts the previously described classes into action. Only portions of the code are illustrated here. For full details see the

example files in the `ksl.examples.book.chapter4` package. To declare an instance of the `SResource` class, the following pattern is recommended:

```
private val myPharmacists: SResource = SResource(
    parent = this,
    capacity = numServers,
    name = "${this.name}:Pharmacists"
)
val resource: SResourceCIfc
    get() = myPharmacists
```

Notice that the name of the parent model element is used as a prefix for the name of the resource to ensure that the name for the pharmacist is unique. Also note that exposure to useful components of `SResource` are made possible via the `SResourceCIfc` interface. Through this interface the initial capacity can be changed, the state of the resource can be accessed, and access to the statistical collection of the number busy and utilization are available.

To collect the histogram and integer frequency statistics, we can use the following declarations.

```
private val mySysTime: Response = Response(parent =
 ↪  this, name = "${this.name}:SystemTime")
val systemTime: ResponseCIfc
    get() = mySysTime

private val mySysTimeHistogram: HistogramResponse =
 ↪  HistogramResponse(theResponse = mySysTime)
val systemTimeHistogram: HistogramIfc
    get() = mySysTimeHistogram.histogram

private val myInQ = IntegerFrequencyResponse(parent =
 ↪  this, name = "${this.name}:NQUponArrival")
```

The `HistogramResponse` class requires an instance of the `Response` class. In this case, we supply a reference the response that is used to collect the system times for the customers. The reference is used internally to observe the response. The instance of the `IntegerFrequencyResponse` class will be used within the arrival logic to observe the number of customers in the queue when the customer arrives.

To declare an instance of the Queue class, we use the following code.

```
    private val myWaitingQ: Queue<QObject> = Queue(parent =
     ↪  this, name = "${this.name}:PharmacyQ")
    val waitingQ: QueueCIfc<QObject>
        get() = myWaitingQ
```

Notice how we also declare a public property that exposes part of the queue functionality, especially related to getting access to the statistical responses.

We can create and use an instance of `EventGenerator` with the following code. We see that the event generator uses a reference to the functional interface `GeneratorActionIfc`. This is accomplished with the `this::arrival` syntax, which provides a reference to the `arrival` function shown in the code.

In addition, the time between arrivals random variable is supplied for both the time until the first event and the time between events. The `initialize()` method of the `EventGenerator` class ensures that the first event is scheduled automatically at the start of the simulation. In addition, the `EventGenerator` class continues rescheduling the arrivals according to the time between arrival pattern. As previously noted, this process can be suspended, resumed, and turned off if needed. An event generator can also be specified not to automatically start at time 0.

```
    private val endServiceEvent = this::endOfService

    private val ad  = ExponentialRV(1.0, 1)
    private val myArrivalGenerator: EventGenerator =
     ↪  EventGenerator(
        parent = this, generateAction = this::arrival,
↪   timeUntilFirstRV = ad, timeBtwEventsRV = ad
    )
    val arrivalGenerator: EventGeneratorRVCIfc
        get() = myArrivalGenerator

    private fun arrival(generator: EventGeneratorIfc) {
        myNS.increment() // new customer arrived
        myInQ.value = myWaitingQ.numInQ.value.toInt()
        val arrivingCustomer = QObject()
        myWaitingQ.enqueue(qObject = arrivingCustomer) //
↪   enqueue the newly arriving customer
        if (myPharmacists.hasAvailableUnits) {
            myPharmacists.seize()
            val customer: QObject? = myWaitingQ.removeNext()
             ↪  //remove the next customer
            // schedule end of service, include the customer
             ↪  as the event's message
            schedule(eventAction = endServiceEvent,
↪   timeToEvent = myServiceRV, message = customer)
        }
    }
```

In the code for arrivals, we also see the use of the `Queue` class (via the variable `myWaitingQ`) and the `QObject` class. On line 7 of the code, we see `val arrivingCustomer = QObject()` which creates an instance of `QObject` that represents the arriving customer. Then, using the `enqueue` method the customer is placed within the queue. This action is performed

regardless of whether the customer has to wait to ensure that zero wait times are collected. Notice that on line 6, the `IntegerFrequencyResponse` instance is used to observe the number of customers in the queue upon arrival.

Then, in lines 9-14, we see that the number busy is checked against the number of pharmacists by using the `hasAvailableUnits` property of the `SResource` instance. If there are available pharmacists, then a unit of the pharmacist resource is seized, the next customer is removed from the queue, and the customer's end of service action is scheduled. Notice how the `schedule` method is different from the previous implementation. In this implementation, the customer is attached to the `KSLEvent` instance and is held in the calendar with the event until the event is removed from the calendar and its execution commences. Let's take a look at the revised end of service action.

```
    private fun endOfService(event: KSLEvent<QObject>) {
        myPharmacists.release()
        if (!myWaitingQ.isEmpty) { // queue is not empty
            myPharmacists.seize()
            val customer: QObject? = myWaitingQ.removeNext()
             ↪ //remove the next customer
            // schedule end of service
            schedule(eventAction = endServiceEvent,
↪   timeToEvent = myServiceRV, message = customer)
        }
        departSystem(departingCustomer = event.message!!)
    }

    private fun departSystem(departingCustomer: QObject) {
        mySysTime.value = (time -
↪   departingCustomer.createTime)
        myNS.decrement() // customer left system
        myNumCustomers.increment()
```

```
    }
```

Here we see the same basic logic as in the previous example, except in this case we can use the resource and the queue. First, a unit of the pharmacist is released. Then, the queue is checked to see if it is empty, if not, a unit of the pharmacist is seized and the next customer is removed. Then, the customer's service is scheduled. We always handle the departing customer by grabbing it from the message attached to the event via the `event.message` property. This departing customer is sent to a private method that collects statistics on the customer. Notice two items. First, it is perfectly okay to call other methods from within the event routines. In fact, this is encouraged and can help organize your code.

Secondly, we are passing along the instance of `QObject` until it is no longer needed. In the `departingSystem` method, we get the current simulation time via the `time` property that is available to all model elements. This property represents the current simulation time. We then subtract off the time that the `QObject` was created by using the `createTime` property of the departing customer. This is assigned to the value property of an instance of `Response` called `mySystemTime.` This causes the system time of every departing customer to be collected and statistics reported. The process of collecting statistics on `QObject` instances is extremely common and works if you understand how to pass the `QObject` instance along via the `KSLEvent` instances.

There is another little tidbit that is occurring in the reference coded snippet. Earlier in the arrivals code snippet, you might not have noticed the following line of code:

```
private val endServiceEvent = this::endOfService
```

For convenience, this line of code is capturing a `functional` reference to the `endOfService` method. The `EventActionIfc` interface is actually a

functional interface, which allows functional references that contain the same signature to be used without having to implement the interface. This feature of Kotlin allows the functional reference to `private fun endOfService(event: KSLEvent<QObject>)` to serve as a parameter to the `schedule()` method. This alleviates the need to implement an inner class that extends the `EventActionIfc` interface. A similar strategy was used for the implementation of `GeneratorActionIfc` for use in the event generator, except in that case we did not declare a variable to hold the reference to the function. The style that you employ can be based on your own personal preferences.

The results of running the simulation match the previously reported results.

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                                    Count          Average      Half-
Width
-------------------------------------------------------------------------------
Pharmacy:Pharmacists:NumBusy             30           0.5035           0.0060
Pharmacy:Pharmacists:Util                30           0.5035           0.0060
Pharmacy:NumInSystem                     30           1.0060           0.0271
Pharmacy:SystemTime                      30           6.0001           0.1441
Pharmacy:PharmacyQ:NumInQ                30           0.5025           0.0222
Pharmacy:PharmacyQ:TimeInQ               30           2.9961           0.1235
SysTime >= 4 minutes                     30           0.5136           0.0071
Pharmacy:Pharmacists:SeizeCount          30        2513.4000          17.6653
Pharmacy:NumServed                       30        2513.2667          17.6883
-------------------------------------------------------------------------------
```

In the results, we see the system time and the queueing time reported. The response called `Pharmacy:Pharmacists:SeizeCount` reports the number of times that the pharmacist resource was seized. Notice that its value is very close to the number of customers served. The difference is due to

the fact that seizing occurs before the service time and the number served is collected when the customer departs. Notice also that the use of the `SResource` class causes statistics for the number busy and the utilization to be reported. The statistics are exactly the same in this situation because there is only one pharmacist.

We also see a statistic called `SysTime > 4.0 minutes.` This was captured by using an `IndicatorResponse,` which is a subclass of `Response` that allows the user to specify a function that results in boolean expression and an instance of a `Response` to observe. The expression is collected as a 1.0 for true and 0.0 for false. In this example, we are observing the response called `mySysTime.`

```
    private val mySTGT4: IndicatorResponse =
     ↪ IndicatorResponse(
        predicate = { x -> x >= 4.0 },
        observedResponse = mySysTime,
        name = "SysTime >= 4 minutes"
    )
    val probSystemTimeGT4Minutes: ResponseCIfc
        get() = mySTGT4
```

This allow a probability to be estimated. In this case, we estimated the probability that a customer's system time was more than 4.0 minutes.

In addition, the call to the `print()` function will provide the histogram and integer frequency results in the console.

```
Pharmacy:SystemTime:Histogram
   binNum           binLabel binLowerLimit binUpperLimit binCount cumCount proportio
0       1   1 [ 0.00, 4.00)            0.0            4.0  48589.0  48589.0   0.48544
1       2   2 [ 4.00, 8.00)            4.0            8.0  24974.0  73563.0   0.24951
2       3   3 [ 8.00,12.00)            8.0           12.0  12711.0  86274.0   0.12699
3       4   4 [12.00,16.00)           12.0           16.0   6658.0  92932.0   0.06651
```

```
4      5   5 [16.00,20.00)          16.0          20.0   3441.0  96373.0   0.03437
5      6   6 [20.00,24.00)          20.0          24.0   1911.0  98284.0   0.01909
6      7   7 [24.00,28.00)          24.0          28.0    985.0  99269.0   0.00984
7      8   8 [28.00,32.00)          28.0          32.0    458.0  99727.0   0.00457
8      9   9 [32.00,36.00)          32.0          36.0    221.0  99948.0   0.00220
9     10  10 [36.00,40.00)          36.0          40.0    143.0 100091.0   0.00142
```

The integer frequency tabulation shows the number of customers in queue upon the arrival of a new customer. We can see that there is about a 0.75 chance that a customer arrives to an empty queue.

```
Pharmacy:NQUponArrival
    cellLabel value   count cum_count proportion cumProportion
 0  label: 0     0 74944.0   74944.0   0.747243      0.747243
 1  label: 1     1 12615.0   87559.0   0.125780      0.873023
 2  label: 2     2  6303.0   93862.0   0.062845      0.935869
 3  label: 3     3  3178.0   97040.0   0.031687      0.967555
 4  label: 4     4  1581.0   98621.0   0.015764      0.983319
 5  label: 5     5   765.0   99386.0   0.007628      0.990947
 6  label: 6     6   417.0   99803.0   0.004158      0.995104
 7  label: 7     7   235.0  100038.0   0.002343      0.997448
 8  label: 8     8   136.0  100174.0   0.001356      0.998804
 9  label: 9     9    72.0  100246.0   0.000718      0.999521
10 label: 10    10    27.0  100273.0   0.000269      0.999791
11 label: 11    11    13.0  100286.0   0.000130      0.999920
12 label: 12    12     5.0  100291.0   0.000050      0.999970
13 label: 13    13     3.0  100294.0   0.000030      1.000000
```

The `IntegerFrequencyResponse` and `HistogramResponse` also facilitate the plotting of their results. See the documentation for further details.

4.6. More Drive Through Fun

Many fast food franchises have configured their restaurants such that customers using the drive through option first place an order at an ordering station and then pickup and pay at following station. This situation is called a tandem queueing system as illustrated in Figure 4.14. This section presents KSL constructs that facilitate the modeling of simple queueing situations, like that faced by fast food drive through lines.

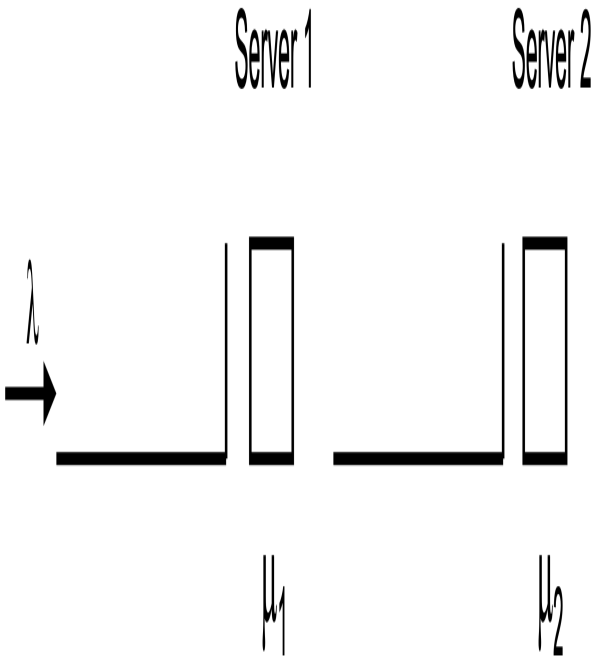


Figure 4.14.: Tandem Queue

A tandem queue is a sequence of queues that must be visited (in order) to receive service from resources. The following example presents an illustrative situation.

Example 4.6 (Tandem Queueing System)**.** Suppose a service facility consists of two stations in series (tandem), each with its own FIFO queue. Each station consists of a queue and a single server. A customer completing service at station 1 proceeds to station 2, while a customer completing service at station 2 leaves the facility. Assume that the inter-arrival times of customers to station 1 are IID exponential random variables with a mean of 6 minutes. Service times of customers at station 1 are exponential random variables with a mean of 4 minute, and at station 2 are exponential random variables with mean 3 minute. Develop an model for this system. Run the simulation for 30 replications of 20000 minutes, with a warm up period of 5000 minutes. Estimate for each station the expected average delay in queue for the customer, the expected time-average number of customers in queue, and the expected utilization. In addition, estimate the average number of customers in the system and the average time spent in the system.

The first thing to note about this situation is that the system consists of two very similar components: station 1 and station 2. The second thing to note is that the same *types* of events occur for each of the two components. That is, there are arrival and departure events for each station. The state change logic for the arrival and departure events is exactly the same as we saw for the pharmacy situation:

Arrival Actions for Event E_a

```
N(t) = N(t) + 1
if (B(t) < c)
  B(t) = B(t) + 1
  schedule E_s at time t + ST_i
else
  Q(t) = Q(t) + 1
endif
schedule E_a at time t + TBA_i
```

We increase the number in the system $N(t)$ by 1 and check if the number of busy servers $B(t)$ is less than the capacity c. If so, we start the customer into service; otherwise, we place the customer in the queue.

End of Service Actions for Event E_s

```
B(t) = B(t) - 1
if (Q(t) > 0)
  Q(t) = Q(t) - 1
  B(t) = B(t) + 1
  schedule E_s at time t + ST_i
endif
N(t) = N(t) - 1
```

The end of service actions are as previously seen. There is one less server busy and if the queue has customers, it is processed and the customer's service starts. Since the same logic will need to be implemented for each station, it makes sense from an object-oriented perspective, to conceptualize a class that encapsulates the data and behavior to represent this situation.

Figure 4.14 should give an idea of what the class should represent. In the figure, there are two stations with each station containing a queue and a server (or resource). Thus, we should build a class that models a single queue that holds objects that must wait for a server to be available. We are going to call this thing a `SingleQStation`. Notice that the input to

the first station is a customer from some arrival process and that the input to the second station is a customer departing the first station. Thus, the main difference between how these components act is where they receive customers from and where they send completed customers. Notice that a station needs to know where to send its completed customers. What else does a station need to know in order to process the customers? The station will need to know how many servers are available at the station and will need to know how to determine the processing time for the customers. For this modeling, we need to combine the concepts of a resource and a queue as a model element component. Let's put the resource together with a queue to get a station for processing in the `SingleQStation` class.

4.6.1. Modeling a Resource with a Waiting Line

The `SingleQStation` class will use an instance of the `SResource` class to represent its resource. In addition, the `SingleQStation` class will use an instance of the `Queue` class presented in the previous section to represent the waiting line for the customers that need to wait for their requested units of the resource. The customers that use the single queue station will be represented by instances of the `QObject` class. We are now ready to put most of these pieces together to construct the `SingleQStation` class. Once we have an understanding of the `SingleQStation` class, we will be ready to model Example 4.6.

Let's start with an overview of the functionality of the `SingleQStation` class and then review the code implementation. Figure 4.15 presents the constructor, functions, and properties of the `SingleQStation` class. The main item to note about the constructor is that it can take in an instance of the `SResource` class. The second thing to note is that instances can process instances of the `QObject` class via the `process()` function. The `process()` function represents the actions that should occur when something arrives to the station. The `endOfProcessing()` function represents what should

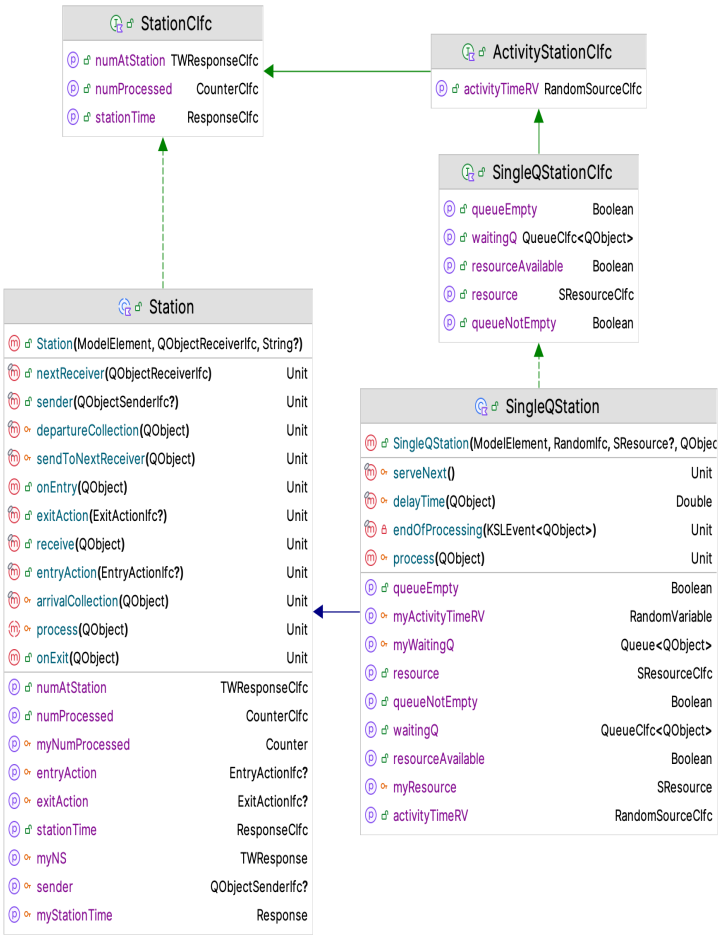


Figure 4.15.: The SingleQStation Class

happen when the processing is completed. That is, the `process()` function is the "arrival event" and the `endOfProcessing()` function is the "departure event". Let's look at the code.

The logic of the `process()` function should look very familiar. Just as was done in previous examples, the arriving customer immediately enters the queue. Then, if the resource is available, the next customer is placed into service.

```
    /**
     *  Receives the qObject instance for processing. Handle
↪   the queuing
     *  if the resource is not available and begins service
↪   for the next customer.
     */
    override fun process(arrivingQObject: QObject) {
        // enqueue the newly arriving qObject
        myWaitingQ.enqueue(arrivingQObject)
        if (isResourceAvailable) {
            serveNext()
        }
    }
```

The `serveNext()` function removes the next customer from the queue, seizes the resource, and schedules the end of processing event for the customer. Notice that the customer starting service is attached to the event. Note that the purpose of the `delayTime()` function is to determine the processing time for the customer when using the resource. There are two default options available. The delay can be supplied from the `QObject` instance via the `valueObject` property. If attached to the `QObject` instance the `valueObject` property returns something that returns a `Double` value. This value can be used for anything you want it to represent. In this case, it can be used to supply a delay time. The second option is to use the

processing time that was specified for the station if the value object is not attached to the `QObject` instance.

```
    /**
     * Called to determine which waiting QObject will be
↪   served next Determines
     * the next customer, seizes the resource, and schedules
↪   the end of the
     * service.
     */
    protected fun serveNext() {
        //remove the next customer
        val nextCustomer = myWaitingQ.removeNext()!!
        myResource.seize()
        // schedule end of service, if the customer can
         ↪  supply a value,
        // use it otherwise use the processing time RV
        schedule(this::endOfProcessing,
↪   delayTime(nextCustomer), nextCustomer)
    }

    /**
     *  Could be overridden to supply different approach for
↪   determining the service delay
     */
    protected fun delayTime(qObject: QObject) : Double {
        return qObject.valueObject?.value ?:
         ↪  myActivityTimeRV.value
    }
```

As mentioned, the `endOfProcessing()` function represents the logic that should occur after the customer completes its use of the resource for the processing time. In the following code, first the customer completing service

is grabbed from the event's message. After releasing the resource, the queue is checked and if it is not empty the next customer is started into service. Then, the leaving customer exits the station and is sent to the next location. The function `sendToNextReceiver()` will be discussed further within the context of the example.

```
    /**
     *  The end of processing event actions. Collect
↪   departing statistics and send the qObject
     *  to its next receiver. If the queue is not empty,
↪   continue processing the next qObject.
     */
    private fun endOfProcessing(event: KSLEvent<QObject>) {
        val leaving: QObject = event.message!!
        myResource.release()
        if (isQueueNotEmpty) { // queue is not empty
            serveNext()
        }
        sendToNextReceiver(leaving)
    }
```

There is a lot more happening within the `SingleQStation` class than presented here. As you can see from Figure 4.15, the `SingleQStation` class inherits from the `Station` class, which is an abstract base class. The `Station` class ensures that statistics that are common to all station types are collected. In addition, the `Station` class provides for the attachment of functions that may be executed when something arrives to the station and also when something departs. Such *entry* and *exit* actions provide users the ability to add behavior to a station without having to use inheritance via the use of Kotlin *lambda* functions. Entry and exit actions are represented by single abstract method (SAM) via the `EntryActionIfc` and `ExitActionIfc` interfaces. Lambda expressions can be attached to a station to effect useful logic.

We now have a new KSL class that can be used to model many different situations (like the drive through pharmacy) that involve the use of resources and waiting lines. Let's continue this by implementing the model for Example 4.6. This will motivate how to send and receive `QObject` instances.

4.6.2. Modeling the Tandem Queue of Example 4.6

The main concepts needed to put the pieces together to represent the tandem queue described in Example 4.6 are now modeled. The main remaining concept needed is how to send and receive customers within such systems. Since this is a very common requirement, the KSL provides basic functionality to help with this modeling task. Two new KSL constructs will be introduced and then used within the implementation of the tandem queue model. The first is an interface that promises to allow the receiving of `QObject` instances: the `QObjectReceiverIfc` interface.

```
fun interface QObjectReceiverIfc {
    fun receive(qObject: ModelElement.QObject)
}
```

Note that the `QObjectReceiverIfc` interface is SAM functional interface. Classes that implement the `QObjectReceiverIfc` interface promise to have a `receive()` function. The idea is to have a defined protocol for system components that will do *something* with the received `QObject` instance. As you may now realize, the `SingleQStation` class implements the `QObjectReceiverIfc` interface via its inheritance from the `Station` class. As shown in the following code, the `SingleQStation` class extends the `Station` class, which implements the `QObjectReceiverIfc` interface.

```
open class SingleQStation(
    parent: ModelElement,
```

```
    activityTime: RVariableIfc = ConstantRV.ZERO,
    resource: SResource? = null,
    nextReceiver: QObjectReceiverIfc =
     ↪ NotImplementedReceiver,
    name: String? = null
) : Station(parent, nextReceiver, name = name),
 ↪ SingleQStationCIfc {
...
}
```

Figure 4.16 presents the major classes and interfaces of the `ksl.modeling.station` package. Central to the functionality is the `Station` class and the `QObjectReceiverIfc` interface.

The `QObjectReceiverIfc` interface defines a protocol for receiving `QObject` instances and the `Station` class provides default functionality for receiving an arriving `QObject` instance and for *sending* a completed `QObject` instance to its next location. Reviewing the `Station` class's code is useful to understanding how this works. The `Station` class is an *abstract* class that provides the `sendToNextReceiver()` function.

```
abstract class Station(
    parent: ModelElement,
    private var nextReceiver: QObjectReceiverIfc =
     ↪ NotImplementedReceiver,
    name: String? = null
) : ModelElement(parent, name), QObjectReceiverIfc,
 ↪ StationCIfc {

    /**
     *  Sets the receiver of qObject instances from this
 ↪ station
     */
```

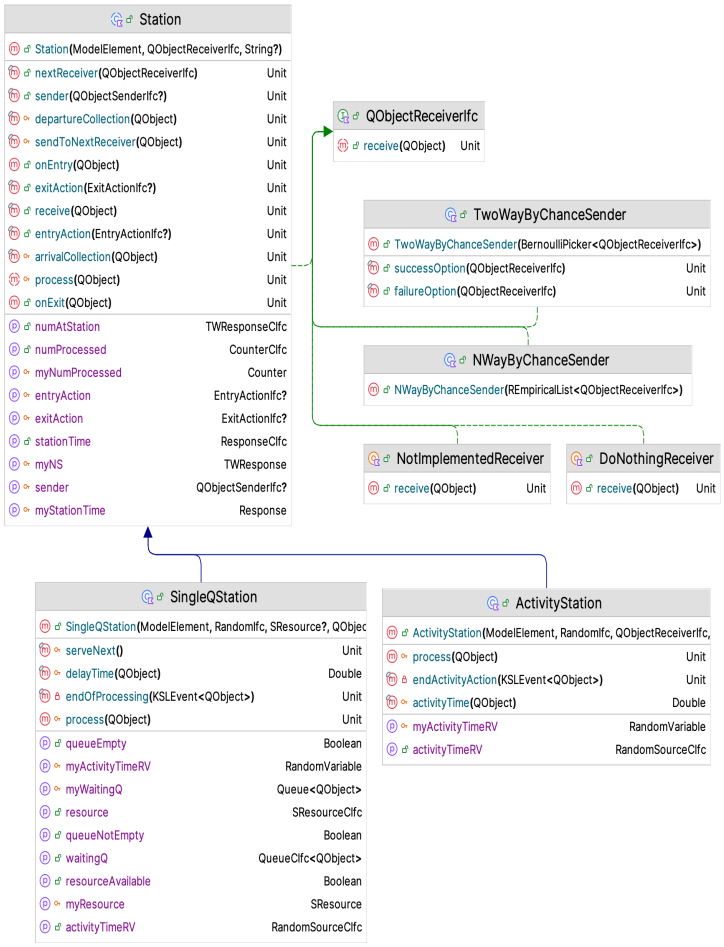


Figure 4.16.: Major Classes and Interfaces of the Station Package

```
    fun nextReceiver(receiver: QObjectReceiverIfc) {
        nextReceiver = receiver
    }
.
.
    protected fun sendToNextReceiver(completedQObject:
     ↪  QObject) {
        departureCollection(completedQObject)
        exitAction?.onExit(completedQObject)
        onExit(completedQObject)
        if (sender != null) {
            sender!!.send(completedQObject)
        } else {
            if (completedQObject.sender != null) {

↪   completedQObject.sender!!.send(completedQObject)
            } else {
                nextReceiver.receive(completedQObject)
            }
        }
    }

    /**
     *  Can be used to supply a sender that will be used
↪   instead of
     *  the default behavior. The default behavior uses a
↪   sender attached
     *  to the QObject instance and if not attached will send
↪   the QObject
     *  to the next receiver.
     */
    fun sender(sender: QObjectSenderIfc?) {
        this.sender = sender
```

```
    }
```

There are three mechanisms for determining where to send the departing `QObject` instance. The first mechanism allows the user to attach a `QObjectSenderIfc` instance via the `sender()` function. As you can see from the code, if this mechanism is supplied, then it is used to send the completed `QObject` instance somewhere (i.e. to some receiver). If this first mechanism is not supplied, then the station looks to see if the completed `QObject` instance has a specified sender. If it does, then the `QObject` instance's sender is used for sending. Every `QObject` has an (optional) property called `sender` that (if set) should return an instance of a `QObjectSenderIfc` interface.

```
/**
 *  A functional interface that promises to send. Within the
 *  context of qObjects a sender should cause a qObject
 *  to be (eventually) received by a receiver.
 */
fun interface QObjectSenderIfc {
    fun send(qObject: ModelElement.QObject)
}
```

The idea is that the sender will know how to send its related `QObject` instance to a suitable receiver. Thus, complex routing logic could be attached to the `QObject` instance. The `sendToNextReceiver()` function checks to see if the `QObject` instance has a sender to assist with its routing. If it does, the sender is used to get the next location and then sends the `QObject` instance to the location by telling the location to receive the object. If the sender is not present, then the `nextReceiver` property is used to send the `QObject` instance to the specified receiver. The receiver's behavior determines what happens to the `QObject` instance next.

The final approach supplies the destination (receiver) as part of the creation of the station instance. Notice that the `Station` class takes in a parameter called `nextReceiver` which represents an object that implements the `QObjectReceiverIfc` interface. This parameter can be used to specify where the departing `QObject` instance should be sent. That is, the next object that should receive the departing object. The default value for this parameter is the `NotImplementedReceiver` object. This object will throw a not implemented yet exception if you do not replace it with something that models the situation. You need to replace the default `NotImplementedReceiver` object only if one of the other two mechanisms are not being used by the station.

> **Note**
>
> As you will soon see in the following example, creating a station without specifying a receiver can be very convenient. This is why the `NotImplementedReceiver` object is the default. However, if you forget to set the receiver to something useful or do not use one of the other mechanisms, you will get the not implemented yet error.

The approach of specifying the receivers to visit allows for the modeling of very complex systems by simply "hooking" up the object instances in the correct order. We can now illustrate this with the implementation of the example.

The following code presents the class constructor for the `TandemQueue` class. The class takes in the random variables need for the arrival and two service processes. Then, because of the requirement to report the total system time and the total number of customers in the system, we have standard definitions for time-weighted response variables and a counter. This is very similar to how we defined the pharmacy system.

```
class TandemQueue(
    parent: ModelElement,
```

```
    name: String? = null
) : ModelElement(parent, name) {

    private val myNS: TWResponse = TWResponse(parent = this,
     ↪  name = "${this.name}:NS")
    val numInSystem: TWResponseCIfc
        get() = myNS

    private val mySysTime: Response = Response(parent =
     ↪  this, name = "${this.name}:TotalSystemTime")
    val totalSystemTime: ResponseCIfc
        get() = mySysTime

    private val myNumProcessed: Counter = Counter(parent =
     ↪  this, name = "${this.name}:TotalProcessed")
    val totalProcessed: CounterCIfc
        get() = myNumProcessed
...
```

Now, the real magic of the station package can be used. The following code represents the *rest* of the implementation of the tandem queue system. The code uses an `EventGenerator` instance to model the arrival process to the first station. Then, two instances of the `SingleQStation` class are created to represent the first and second station in the system. Notice the implementation of the `init{}` block. The `nextReceiver` property for station 1 is set to be the second station and the `nextReceiver` property for station 2 is set to an instance of the `ExitSystem` inner class. This approach relies on *replacing* the default `NotImplementedReceiver` receiver. Notice that if you did not rely on this default, you would need to create the stations (receivers) in the reverse order so that you could supply the correct receiver within the station's constructor. The `init{}` block would not be necessary with that approach.

```
    private val ad = ExponentialRV(6.0, 1)
    private val myArrivalGenerator: EventGenerator =
     ↪  EventGenerator(
        parent = this,
        generateAction = this::arrivalEvent, timeUntilFirstRV
↪   = ad, timeBtwEventsRV = ad
    )

    private val myStation1: SingleQStation = SingleQStation(
        parent = this,
        activityTime = ExponentialRV(4.0, 2),
        name = "${this.name}:Station1"
    )
    val station1: SingleQStationCIfc
        get() = myStation1

    private val myStation2: SingleQStation = SingleQStation(
        parent = this,
        activityTime = ExponentialRV(3.0, 3),
        name = "${this.name}:Station2"
    )
    val station2: SingleQStationCIfc
        get() = myStation2

    init {
        myStation1.nextReceiver(myStation2)
        myStation2.nextReceiver(ExitSystem())
    }

    private fun arrivalEvent(generator: EventGeneratorIfc) {
        val customer = QObject()
        myNS.increment()
        myStation1.receive(customer)
```

```
    }

    private inner class ExitSystem : QObjectReceiverIfc {
        override fun receive(arrivingQObject: QObject) {
            mySysTime.value = time -
↪   arrivingQObject.createTime
            myNumProcessed.increment()
            myNS.decrement()
        }
    }
}
```

The arrival process shown in the `arrivalEvent()` function creates the arriving customer, increments the number in the system, and tells station 1 to receive the customer. This will set off a series of events which will occur within the `SingleQStation` instances that eventually result in the customer being received by the `ExitSystem` instance. The `ExitSystem` instance is used to collect statistics on the departing customer. The modeling involves hooking up the system components so that they work together to process the customers. This creation and use of objects in this manner is a hallmark of object-oriented programming. The following code can be used to simulate the system.

```
fun main(){
    val sim = Model("TandemQ Model")
    sim.numberOfReplications = 30
    sim.lengthOfReplication = 20000.0
    sim.lengthOfReplicationWarmUp = 5000.0
    val tq = TandemQueue(sim, name = "TandemQ")
    sim.simulate()
    sim.print()
}
```

The results from running the following code are not very interesting except for noticing the number of statistics that are *automatically* captured within the output. Notice for example that the resources automatically report the average number of busy units and the utilization of the resource. In addition, stations automatically report the total time spent at a station, the number of objects at the station, and the number completed by the station.

Statistical Summary Report

Name	Count	Average	Half-Width
TandemQ:NS	30	3.04	0.093
TandemQ:TotalSystemTime	30	18.132	0.5
TandemQ:Station1:R:NumBusy	30	0.671	0.008
TandemQ:Station1:R:Util	30	0.671	0.008
TandemQ:Station1:NS	30	2.031	0.088
TandemQ:Station1:StationTime	30	12.107	0.483
TandemQ:Station1:Q:NumInQ	30	1.359	0.081
TandemQ:Station1:Q:TimeInQ	30	8.101	0.457
TandemQ:Station2:R:NumBusy	30	0.5	0.004
TandemQ:Station2:R:Util	30	0.5	0.004
TandemQ:Station2:NS	30	1.01	0.023
TandemQ:Station2:StationTime	30	6.029	0.135
TandemQ:Station2:Q:NumInQ	30	0.51	0.02
TandemQ:Station2:Q:TimeInQ	30	3.042	0.12
TandemQ:TotalProcessed	30	2512.667	17.41
TandemQ:Station1:NumProcessed	30	2512.7	17.535
TandemQ:Station2:NumProcessed	30	2512.667	17.41

Example 4.7 (Tandem Queueing System with Dependent Service Times)**.** Reconsider Example 4.6 with the following variation. Suppose that there

are two types of parts A and B. The time that it takes to process the parts at the first station depends on the type of part. Part A parts have an exponential service time with a mean of 2.0 minutes (stream 2). Part B parts have an exponential service time with a mean of 6.0 minutes (stream 2). Suppose that data indicates that 40 percent of the parts that arrive are of type A (stream 4). Revise the solution for Example 4.6 to account for the service time at station 1 being dependent upon the type of part.

This situation can be modeled by specifying that station 1 use the processing QObject instances to determine the activity time at the station. This can be accomplished by setting the `useQObjectForActivityTime` property of the `SingleQStation` class to `true`. This setting will cause the station to attempt to use the `valueObject` property of the `QObject` instances to determine the activity time. The `valueObject` property should hold an instance of an object that implements the `GetValueIfc` interface. Since the `RandomVariable` class implements this interface, it is a simple matter to assign the `valueObject` property when the `QObject` is created. The following code illustrates the solution.

```
    private val myStation1: SingleQStation = SingleQStation(
        parent = this,
        name = "${this.name}:Station1"
    )
    val station1: SingleQStationCIfc
        get() = myStation1

    private val myStation2: SingleQStation = SingleQStation(
        parent = this,
        activityTime = ExponentialRV(3.0, 3),
        name = "${this.name}:Station2"
    )
```

```
    val station2: SingleQStationCIfc
        get() = myStation2

    init {
        myStation1.useQObjectForActivityTime = true
        myStation1.nextReceiver(myStation2)
        myStation2.nextReceiver(ExitSystem())
    }

    private val myPartType = RandomVariable(this,
     ↪  BernoulliRV(0.4, 4))
    private val myPartTypeAServiceRV = RandomVariable(this,
     ↪  ExponentialRV(4.0, 2))
    private val myPartTypeBServiceRV = RandomVariable(this,
     ↪  ExponentialRV(6.0, 2))

    private fun arrivalEvent(generator: EventGeneratorIfc) {
        val customer = QObject()
        if (myPartType.value == 1.0) {
            customer.valueObject = myPartTypeAServiceRV
        } else {
            customer.valueObject = myPartTypeBServiceRV
        }
        myNS.increment()
        myStation1.receive(customer)
    }
```

The first thing to notice is that the declaration for station 1 does not have its activity time provided. In this case, the activity time defaults to 0.0. Next, the `init` block has been updated such that station one's `useQObjectForActivityTime` is set to true. This is followed by definitions for the random variables. Here the determination of the type of part is specified with a Bernoulli random variable. Finally, in the arrival event for

the parts, the type of part is randomly determined and the `valueObject` property of the `QObject` instance is assigned to correct random variable based on the type. The underlying logic that handles the determination of the activity time for station 1 will now randomly generate a value using the random variable that was assigned to the `valueObject` property. Thus, the service time at station 1 will depend upon the part type. Similar strategies can be implemented for any station. The tricky aspect is the reassignment of the `valueObject` property before the queue object is sent to the next station. This might be accomplished by using lambda functions assigned to the *entry* or *exit* actions for the station as mentioned in Section 4.6.1.

4.6.3. Modeling with the Station Package

The `ksl.modeling.station` package has many options that can be utilized to model complex situations. This section discusses some of the options and offers some reasons and strategies for their application.

Imagine if the tandem queue system consisted of 100 stations. How might you model that situation? You would need to make 100 instances of the `SingleQStation` class. This could easily be accomplished in a for-loop which captures the instances into a list of stations. Then, the list could be iterated to assign the `nextReceiver` property of the station. That is, iterate the list and assign each station's receiver to the next element in the list. This approach builds on the notion of directly connecting the receivers as illustrated in the tandem queue example. However, there may be better approaches.

A better approach is to use the sending option that is attached to the `QObject` instance when the instance is created and let every created `QObject` instance follow their own *sequence* through the system. This can be accomplished by using an instance of the `ReceiverSequence` class.

The `ReceiverSequence` class sub-classes from the `QObjectSender` class. As previously mentioned *senders* implement the `QObjectSenderIfc` in-

terface. That is, they know how or where to send the `QObject` instance. The main variation between senders is the selection of the next `QObjectReceiverIfc` instance to receive the `QObject` instance. The `QObjectSender` class is an abstract base class that requires the implementation of the `selectNextReceiver()` function.

```
abstract class QObjectSender() :  QObjectSenderIfc {

    /**
     *  Can be used to supply logic if the iterator does not
 ↪  have
     *  more receivers.
     */
    private var noNextReceiverHandler:
     ↪  ((ModelElement.QObject) -> Unit)? = null

    fun noNextReceiverHandler(fn: ((ModelElement.QObject) ->
     ↪  Unit)?){
        noNextReceiverHandler = fn
    }

    abstract fun selectNextReceiver(): QObjectReceiverIfc?

    final override fun send(qObject: ModelElement.QObject) {
        val selected = selectNextReceiver()
        if (selected != null) {
            beforeSendingAction?.action(selected, qObject)
            selected.receive(qObject)
            afterSendingAction?.action(selected, qObject)
        } else {
            noNextReceiverHandler?.invoke(qObject)
        }
    }
```

```
    private var beforeSendingAction: SendingActionIfc? = null

    fun beforeSendingAction(action : SendingActionIfc){
        beforeSendingAction = action
    }

    private var afterSendingAction: SendingActionIfc? = null

    fun afterSendingAction(action : SendingActionIfc){
        afterSendingAction = action
    }
}
```

The `QObjectSender` class defines how `QObject` instances will be *sent* and allows for the handling of a receiver not being selected. In addition, you can use *lambda* expressions to add actions prior to and after sending the `QObject` instance. Let's review the implementation of the `ReceiverSequence` class.

```
class ReceiverSequence(
    private val receiverItr: ListIterator<QObjectReceiverIfc>
) : QObjectSender() {

    override fun selectNextReceiver(): QObjectReceiverIfc? {
        return if (receiverItr.hasNext()){
            receiverItr.next()
        } else {
            null
        }
    }
}
```

As can be seen in the code for the `ReceiverSequence` class, a list iterator

is used and checked. If the next receiver is found, it is returned and the `QObject` instance will be sent to the receiver. So imagine that you have a list of `QObjectReceiverIfc` instances, in a list called `stations`, then the following code could be implemented to assign the sequence to the `QObject` instance when it is created.

```
    private fun arrivalEvent(generator: EventGenerator){
        val itr = stations.listIterator()
        val customer = QObject()
        customer.sender = ReceiverSequence(itr)
        customer.sender.send(customer)
    }
```

The sender will cause the customer to go to the first receiver presented by the iterator and then after that each station will automatically cause the customer to continue using the iterator until there are no more receivers. The trick is handling the last receiver, which could easily be a receiver like the `ExitSystem` receiver used in the tandem queue implementation that does not cause further use of the iterator. This approach is very flexible. For example, imagine different types of customers being assigned different iterators (based on different lists of stations). The approach would allow stations to be visited multiple times, skipped, etc. all based on the ordering of stations in a list and the resulting iterators from those lists.

As previously mentioned, there are actually 3 mechanisms (or options) for sending `QObject` instances: 1) directly specifying a receiver (illustrated in the tandem queue model), 2) attaching a sender to the `QObject` instance (discussed in this section), and 3) specifying a sender for the station. We have already discussed options 1 and 2. For option 3, while it is possible that every station may have its own unique instance of a sender (i.e. a `QObjectSenderIfc` instance), a more plausible approach would be that the stations *share* a global sender. In this approach you simply defer the sending functionality to some complex object at the system level that determines where to send the `QObject` instance next. Since system control

logic may take into account the entire state of the system in order to optimally route the workload among the stations, this approach seems extremely useful.

In reviewing Figure 4.16 you may also notice the `ActivityStation`, the `TwoWayByChanceSender`, and `NWayByChanceSender` classes. The `ActivityStation` class models a station that does not have a resource by implementing a simple scheduled delay for a specified activity time. The `TwoWayByChanceSender` class provides probabilistic routing between two specified receivers according to a Bernoulli random variable. The `NWayByChanceSender` class provides probabilistic routing from a list of receivers according to a discrete empirical distribution. The `TwoWayByChanceSender` and `NWayByChanceSender` classes are special in that they behave as both *senders* and *receivers* of `QObject` instances. That is, they implement both the `QObjectReceiverIfc` interface and the `QObjectSenderIfc` interface. Essentially, their receiving action is to immediately execute the sending action. This dual behavior allows them to be used in any of the three options for sending `QObject` instances.

As one final point of discussion, the ability to attach Kotlin *lambda* expressions within the stations package should not be under appreciated. Let's suppose you were simulating a system where parts might need to repeatedly visit a particular station and you wanted to count the number of times that the part visited the station in order to invoke control logic based on the count. How might you model this situation using stations? The first task would be to subclass `QObject` to have a sub-type that had an attribute to hold the visit count.

```
private inner class Part(var visitCount: Int = 0) : QObject()
```

Now, suppose the station for which we wanted to track the visits was the drilling station (called `drilling`). Then, when setting up the initialization logic within the `init{}` block we can attach a lambda expression to the drilling station and ensure that every time that the station is visited, the

part increments the visitation count. The code might look something like this:

```
    init {
        myArrivalGenerator.generatorAction {
↪   drilling.receive(Part()) }
        drilling.exitAction { (it as Part).visitCount++ }
        drilling.nextReceiver(grinding)
        // etc.
        grinding.nextReceiver(exit)
    }
```

Since the `GeneratorActionIfc` is a SAM functional interface, the code takes advantage of this by supplying the generation logic as a lambda expression. In addition, the exit action for the drilling station is specified as a lambda expression. In this case, the reference object of the lambda is cast to an instance of the Part class and then the visit count attribute is incremented. This code illustrates a single line lambda; however, there is no limitation on the number of lines of the lambda expression. Also, if you do not like lambda expressions, you can simply make an inner class that implements the `ExitActionIfc` interface and use the syntax which best matches your programming style.

Using these constructs, the KSL can model very large and complex queueing situations using the relatively simple framework provided by the `ksl.modeling.station` package. The next chapter will present many ways in which you can capture and use the simulation results to make decisions based on the statistical results.

4.7. Summary

This chapter introduced how to model discrete event dynamic systems using the KSL. The KSL facilitates the model building process, the model

running process, and the output analysis process.

The main model elements covered included:

`Model`: Used to hold all model elements. Used to create and control the execution of the model.

`ModelElement`: Used as an abstract base class for creating new model elements for a simulation.

`RandomVariable`: A sub-class of `ModelElement` used to model randomness within a simulation.

`Response`: A sub-class of `ModelElement` used to collect statistics on observation-based variables.

`TWResponse`: A sub-class of `Response` used to collect statistics on time-weighted variables in the model.

`Counter`: A sub-class of `ModelElement` used to count occurrences and collect statistics.

`IndicatorResponse`: A sub-class of `Response` used to collect on boolean expressions by observing another response.

`AggregateTWResponse`: A sub-class of `TWResponse` used to collect time weighted statistics by observing other `TWResponse` instances.

`SimulationReporter`: Used to gather and report statistics on a simulation model.

`KSLEvent`: Used to model different events scheduled in time during a simulation.

`EventActionIfc`: An interface used to define an action() method that represents event logic within the simulation.

`Queue`: A sub-class of `ModelElement` that holds instances of the class `QObject` and will *automatically* collect statistics on the number in the queue and the time spent in the queue.

`EventGenerator`: A subclass of `ModelElement` that facilitates the repeated generation of events.

`GeneratorActionIfc`: An interface used to implement the actions associated with event generators.

`SResource`: A simple resource that has 1 or more units that represent its capacity. The units can be seized and released. Statistics on the utilization and number of busy units are automatically reported.

`SingleQStation`: A station that has a single waiting line for customers to wait in when its associated resource does not have available units. Statistics on time spent at the station, number of customers at the station, and number of customers processed are automatically reported.

The KSL has many other facets that have yet to be touched upon. Not only does the KSL allow the modeler to build and analyze simulation models, but it also facilitates data collection, statistical analysis, and experimentation.

The next chapter will dive deeper into how to use the KSL to capture, analyze, and report the data associated with a discrete-event simulation.

4.8. Exercises

Exercise 4.1. Using the supplied data set, draw the sample path for the state variable, $Y(t)$. Assume that the value of $Y(t)$ is the value of the state variable just after time t. Compute the time average over the supplied time range.

t	0	1	6	10	15	18	20	25	30	34	39	42
$Y(t)$	1	2	1	1	1	2	2	3	2	1	0	1

Exercise 4.2. Using the supplied data set, draw the sample path for the state variable, $N(t)$. Give a formula for estimating the time average number in the system, $N(t)$, and then use the data to compute the time average number in the system over the range from 0 to 25. Assume that the value of $N(t$ is the value of the state variable just after time t.

t	0	2	4	5	7	10	12	15	20
$N(t)$	0	1	0	1	2	3	2	1	0

Exercise 4.3. Consider the banking situation described within the chapter. A simulation analyst observed the operation of the bank and recorded the information given in the following table. The information was recorded right after the bank opened during a period of time for which there was only one teller working. From this information, you would like to re-create the operation of the system.

Customer Number	Time of Arrival	Service Time
1	3	4
2	11	4
3	13	4
4	14	3
5	17	2
6	19	4
7	21	3
8	27	2
9	32	2
10	35	4

11	38	3
12	45	2
13	50	3
14	53	4
15	55	4

Complete a table similar to that used in the chapter and compute the average of the system times for the customers. What percentage of the total time was the teller idle? Compute the percentage of time that there were 0, 1, 2, and 3 customers in the queue.

Exercise 4.4. Consider the following inter-arrival and service times for the first 20 customers to a single server queueing system.

Customer Number	Inter-Arrival Time	Service Time	Time of Arrival
1	22	74	22
2	89	105	111
3	21	34	132
4	26	38	158
5	80	23	
6	81	26	
7	78	90	
8	20	26	
9	32	37	
10	13	88	
11	28	38	
12	18	73	
13	29	93	

14	19	25
15	20	93
16	23	5
17	78	37
18	20	51
19	109	28
20	78	85

We are given the inter-arrival times. Determine the time of arrival of each customer. Complete the event and state variable change table associated with this situation. Draw a sample path graph for the variable $N(t)$ which represents the number of customers in the system at any time t. Compute the average number of customers in the system over the time period from 0 to 700. Draw a sample path graph for the variable $NQ(t)$ which represents the number of customers waiting for the server at any time t. Compute the average number of customers in the queue over the time period from 0 to 700. Draw a sample path graph for the variable $B(t)$ which represents the number of servers busy at any time t. Compute the average number of busy servers in the system over the time period from 0 to 700. Compute the average time spent in the system for the customers.

Exercise 4.5. Parts arrive at a station with a single machine according to a Poisson process with the rate of 1.5 per minute. The time it takes to process the part has an exponential distribution with a mean of 30 seconds. There is no upper limit on the number of parts that wait for process. Setup an model to estimate the expected number of parts waiting in the queue and the utilization of the machine. Run your model for 1,000,000 seconds for 30 replications and report the results. Use stream 1 for the time between arrivals and stream 2 for the service times. Use M/M/1 queueing results from the chapter to verify that your simulation is working as intended.

Exercise 4.6. A large car dealer has a policy of providing cars for its customers that have car problems. When a customer brings the car in for repair, that customer has use of a dealer's car. The dealer estimates that the dealer cost for providing the service is $10 per day for as long as the customer's car is in the shop. Thus, if the customer's car was in the shop for 1.5 days, the dealer's cost would be $15. Arrivals to the shop of customers with car problems form a Poisson process with a mean rate of one every other day. There is one mechanic dedicated to the customer's car. The time that the mechanic spends on a car can be described by an exponential distribution with a mean of 1.6 days. Setup a model to estimate the expected time within the shop for the cars and the utilization of the mechanic. Run your model for 10000 days for 30 replications and report the results. Estimate the total cost per day to the dealer for this policy. Use the M/M/1 queueing results from the chapter to verify that your simulation is working as intended. Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.7. YBox video game players arrive according to a Poisson process with rate 10 per hour to a two-person station for inspection. The inspection time per YBox set is exponentially distributed with a mean of 10 minutes. On the average 82% of the sets pass inspection. The remaining 18% are routed to an adjustment station with a single operator. Adjustment time per YBox is uniformly distributed between 7 and 14 minutes. After adjustments are made, the units depart the system. The company is interested in the total time spent in the system. Run your model for 10000 minutes for 30 replications and report the results. Use stream 1 for the time between arrivals, stream 2 for inspection times, stream 3 for inspections, and stream 4 for adjustments.

Exercise 4.8. Referring to the pharmacy model discussed in Section 4.4.4, suppose that the customers arriving to the drive through pharmacy can decide to enter the store instead of entering the drive through lane. Assume a 90% chance that the arriving customer decides to use the drive through pharmacy and a 10% chance that the customer decides to use the store. Model this situation with and discuss the effect on the performance of the drive through lane. Use stream 3 for the decision process. Run your model for 30 replications of length 20000 minutes and a warm up period of 5000 minutes.

Exercise 4.9. SQL queries arrive to a database server according to a Poisson process with a rate of 1 query every minute. The time that it takes to execute the query on the server is typically between 0.6 and 0.8 minutes uniformly distributed. The server can only execute 1 query at a time. Develop a simulation model to estimate the average delay time for a query. Use stream 1 for the time between query arrivals and stream 2 for the query execution time. Run your model for 30 replications having a length of 100,000 minutes. Use stream 1 for the query arrival process and stream 2 for the time to execute the query.

Exercise 4.10. Passengers arrive to an airport security check point at a small airport for identification inspection according to a Poisson process with a rate of 30 per hour. Assume that the check point is staffed by a single security officer. The officer can check the passenger's identification with a minimum time of .75 minute, most likely value of 1.5 minutes, and a maximum of 3 minutes, triangularly distributed. Past data has shown

that 93% of the passengers immediately pass the identification inspection. Those passengers that immediately pass move on to the security area. Those that do not pass are sent to a separate area for further investigation. For the purposes of this exercise, the activities after the determination of passing identification inspection are outside the scope of this modeling. Develop a simulation model that can estimate the following quantities:

- utilization of the security officer
- average number of passengers waiting for identification inspection
- average time spent by passengers waiting for identification inspection
- the probability that a passenger must wait longer than 5 minutes for identification inspection
- the average number of passengers cleared per day
- the average number of passengers denied per day

Assume that the check point operates for 10 hours per day, starting at 8 am. Also, assume that when the check point opens, there are no passengers waiting. Finally, for simplicity, assume that we are only interested in the statistics collected during the 10-hour time span. Analyze this situation for 20 independent days of operation. Use stream 1 for the arrival process, stream 2 for the service process, and stream 3 for the inspection process.

Exercise 4.11. Consider a single pump gas station where the arrival process is Poisson with a mean time between arrivals of 10 minutes. The service time is exponentially distributed with a mean of 6 minutes.

Build a KSL model to simulate this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the probability that you have to wait for service?

b. What is the expected number of customers at the station?

c. What is the expected time waiting in the line to get a pump?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.12. Suppose an operator has been assigned to the responsibility of maintaining 3 machines. For each machine the probability distribution of the running time before a breakdown is exponentially distributed with a mean of 9 hours. The repair time also has an exponential distribution with a mean of 2 hours.

Build a KSL model to simulate this situation. Run the model for 20,000 hours with a warm up period of 5,000 hours. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the probability that the operator is idle?

b. What is the expected number of machines that are running?

c. What is the expected number of machines that are not running?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.13. Each airline passenger and his or her carry-on baggage must be checked at the security checkpoint. Suppose XNA averages 10 passengers per minute with exponential inter-arrival times. To screen passengers, the airport must have a metal detector and baggage X-ray machines. Whenever a checkpoint is in operation, two employees are required (one operates the metal detector, one operates the X-ray machine). The passenger goes through the metal detector and simultaneously their

bag goes through the X-ray machine. A checkpoint can check an average of 12 passengers per minute according to an exponential distribution.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the probability that a passenger will have to wait before being screened?

b. On average, how many passengers are waiting in line to enter the checkpoint?

c. On average, how long will a passenger spend at the checkpoint?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.14. Customers arrive at a one-window drive in bank according to a Poisson distribution with a mean of 10 per hour. The service time for each customer is exponentially distributed with a mean of 5 minutes. There are 3 spaces in front of the window including that for the car being served. Other arriving cars can wait outside these 3 spaces.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the probability that an arriving customer can enter one of the 3 spaces in front of the window?

b. What is the probability that an arriving customer will have to wait outside the 3 spaces?

c. How long is an arriving customer expected to wait before starting service?

d. How many spaces should be provided in front of the window so that an arriving customer can wait in front of the window at least 20% of the time? In other words, the probability of at least one open space must be greater than 20%.

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.15. Joe Rose is a student at Big State U. He does odd jobs to supplement his income. Job requests come every 5 days on the average, but the time between requests is exponentially distributed. The time for completing a job is also exponentially distributed with a mean of 4 days.

Build a KSL model to analyze this situation. Run the model for 20,000 days with a warm up period of 5,000 days. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the chance that Joe will not have any jobs to work on?

b. What is the average value of the waiting jobs if Joe gets about $25 per job?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.16. The manager of a bank must determine how many tellers should be available. For every minute a customer stands in line, the manager believes that a delay cost of 5 cents is incurred. An average of 15 customers per hour arrive at the bank. On the average, it takes a teller 6 minutes to complete the customer's transaction. It costs the bank $9 per hour to have a teller available. Inter-arrival and service times can be assumed to be exponentially distributed.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. What is the minimum number of tellers that should be available in order for the system to be stable (i.e. not have an infinite queue)?

b. If the system has 3 tellers, what is the probability that there will be no one in the bank?

c. What is the expected total cost of the system per hour, when there are 2 tellers?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.17. Sly's convenience store operates a two-pump gas station. The lane leading to the pumps can house at most five cars, including those being serviced. Arriving cars go elsewhere if the lane is full. The distribution of the arriving cars is Poisson with a mean of 20 per hour. The time to fill up and pay for the purchase is exponentially distributed with a mean of 6 minutes.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. What is the percentage of cars that will seek business elsewhere?

b. What is the utilization of the pumps?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.18. An airline ticket office has two ticket agents answering incoming phone calls for flight reservations. In addition, two callers can be put on hold until one of the agents is available to take the call. If all four phone lines (both agent lines and the hold lines) are busy, a potential customer gets a busy signal, and it is assumed that the call goes to another ticket office and that the business is lost. The calls and attempted calls occur randomly (i.e. according to Poisson process) at a mean rate of 15 per hour. The length of a telephone conversation has an exponential distribution with a mean of 4 minutes.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. What is probability of losing a potential customer?

b. What is the probability that an arriving phone call will not start service immediately but will be able to wait on a hold line?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.19. SuperFastCopy has three identical copying machines. When a machine is being used, the time until it breaks down has an exponential distribution with a mean of 2 weeks. A repair person is kept on call to repair the machines. The repair time for a machine has an exponential distribution with a mean of 0.5 week. The downtime cost for each copying machine is $100 per week.

Build a KSL model to analyze this situation. Run the model for 20,000 weeks with a warm up period of 5,000 weeks. Based on 30 replications of your simulation, what is the expected downtime cost per week.

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.20. NWH Cardiac Care Unit (CCU) has 5 beds, which are virtually always occupied by patients who have just undergone major heart surgery. Two registered nurses (RNs) are on duty in the CCU in each of the three 8 hour shifts. Each patient requires a nurse's attention about every two hours, following an exponential distribution. The RN will then spend an average of 30 minutes (exponentially distributed) assisting the patient and updating medical records regarding the problem and care provided.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. What is the average number of patients being attended by the nurses?

b. What is the average time that a patient spends waiting for one of the nurses to arrive?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 4.21. HJ Bunt, Transport Company maintains a large fleet of refrigerated trailers. For the purposes of this problem assume that the number of refrigerated trailers is conceptually infinite. The trailers require service on an irregular basis in the company owned and operated service shop. Assume that the arrival of trailers to the shop is approximated by a Poisson distribution with a mean rate of 3 per week. The length of time needed for servicing a trailer varies according to an exponential distribution with a mean service time of one-half week per trailer. The current policy is to utilize a centralized contracted outsourced service center whenever more than two trailers are in the company shop, so that, at most one trailer is allowed to wait. Assume that there is currently one 1 mechanic in the company shop.

Build a KSL model to analyze this situation. Run the model for 20,000 weeks with a warm up period of 5,000 weeks. Based on 30 replications of your simulation, what is the expected number of repairs that are outsourced per week?

Use stream 1 for the time between arrivals and stream 2 for the service times.

---

5. Analyzing and Accessing Simulation Output

Learning Objectives

- To be able to recognize the different types of statistical quantities used within and produced by simulation models.
- To be able to analyze finite horizon simulations via the method of replications.
- To be able to analyze infinite horizon simulations via the method of batch means and the method of replication-deletion.
- To be able to compare simulation alternatives and make valid decisions based on the statistical output of a simulation.
- To be able to access output from KSL simulation models in many different forms.

Because the inputs to the simulation are random, the outputs from the simulation are also random. You can think of a simulation model as a function that maps inputs to outputs. This chapter presents the statistical analysis of the outputs from simulation models.

In addition, a number of issues that are related to the proper execution of simulation experiments are presented. For example, the simulation outputs are dependent upon the input random variables, input parameters, and the initial conditions of the model. Initial conditions refer to the starting conditions for the model, e.g. whether or not the system starts "empty and

idle". The effect of initial conditions on steady state simulations will be discussed in this chapter.

Input parameters are related to the controllable and uncontrollable factors associated with the system. For a simulation model, *all* input parameters are controllable; however, in the system being modeled we typically have control over only a limited set of parameters. Thus, in simulation you have the unique ability to control the random inputs into your model. This chapter will discuss how to take advantage of controlling the random inputs.

Input parameters can be further classified as decision variables. That is, those parameters of interest that you want to change in order to test model configurations for decision-making. The structure of the model itself may be considered a decision variable when you are trying to optimize the performance of the system. When you change the input parameters for the simulation model and then execute the simulation, you are simulating a different design alternative.

This chapter describes how to analyze the output from a single design alternative and how to analyze the results of multiple design alternatives. The focus of this chapter is on understanding the types of data produced by a discrete-event simulation and how to analyze that data. The KSL facilitates the capture and analysis of various statistical quantities. This chapter will present the most common and useful approaches to instrumenting a model and extracting the captured data. Although the focus will be on demonstrating this functionality on simple models, you will readily see how easily the approaches can be scaled up to larger models with substantial output requirements.

> **Note**
>
> This chapter provides a series of example Kotlin code that illustrates the use of KSL constructs for working with data generated from KSL models. The full source code of the examples can be found

> in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

To begin the discussion you need to build an understanding of the types of statistical quantities that may be produced by a simulation experiment.

5.1. Types of Statistical Variables

A simulation experiment occurs when the modeler sets the input parameters to the model and executes the simulation. This causes events to occur and the simulation model to evolve over time. During the execution of the simulation, the behavior of the system is observed and various statistical quantities computed. When the simulation reaches its termination point, the statistical quantities are summarized in the form of output reports.

A simulation experiment may be for a single replication of the model or may have multiple replications. A *replication* is the generation of one sample path which represents the evolution of the system from its initial conditions to its ending conditions. If you have multiple replications within an experiment, each replication represents a different sample path, starting from the same initial conditions and being driven by the same input parameter settings. Because the randomness within the simulation can be controlled, the underlying random numbers used within each replication of the simulation can be made to be independent. Thus, as the name implies, each replication is an independently generated "repeat" of the simulation.

Within a single sample path (replication), the statistical behavior of the model can be observed.

Definition 5.1 (Within Replication Statistic)**.** The statistical quantities collected during a replication are called *within replication statistics*.

Definition 5.2 (Across Replication Statistic)**.** The statistical quantities collected across replications are called *across replication statistics*. Across replication statistics are collected based on the observation of the final values of within replication statistics.

Within replication statistics are collected based on the observation of the sample path and include observations on entities, state changes, etc. that occur during a sample path execution. The observations used to form within replication statistics are not likely to be independent and identically distributed. Since across replication statistics are formed from the final values of within replication statistics, one observation per replication is available. Since each replication is considered independent, the observations that form the sample for across replication statistics are likely to be independent and identically distributed. The statistical properties of within and across replication statistics are inherently different and require different methods of analysis. Of the two, within replication statistics are the more challenging from a statistical standpoint.

As we saw in Section 4.3 of Chapter 4, there are two primary types of observations: *tally* and *time-persistent*. Tally data represent a sequence of equally weighted data values that do not persist over time. This type of data is associated with the duration or interval of time that an object is in a particular state or how often the object is in a particular state. As such it is observed by marking (tallying) the time that the object enters the state and the time that the object exits the state. Once the state change takes place, the observation is over (it is gone, it does not persist, etc.). If we did not observe the state change, then we would have missed the observation. The time spent in queue, the count of the number of customers served, whether or not a particular customer waited longer than 10 minutes are all examples of tally data. We have used the KSL `Response` class to collect this type of data.

Time-persistent observations represent a sequence of values that persist over some specified amount of time with that value being weighted by the amount of time over which the value persists. These observations are

directly associated with the values of the state variables within the model. The value of a time-persistent observation persists in time. For example, the number of customers in the system is a common state variable. If we want to collect the average number of customers in the system *over time*, then this will be a time-persistent statistic. While the value of the number of customers in the system changes at discrete points in time, it holds (or persists with) that value over a duration of time. This is why it is called a time-persistent variable. We have used the KSL `TWResponse` class to collect this type of data.

Figure 5.1 illustrates a single sample path for the number of customers in a queue over a period of time. From this sample path, events and subsequent statistical quantities can be observed.

- Let A_i $i = 1 \ldots n$ represent the time that the i^{th} customer enters the queue
- Let D_i $i = 1 \ldots n$ represent the time that the i^{th} customer exits the queue
- Let $W_i = D_i - A_i$ $i = 1 \ldots n$ represent the time that the i^{th} customer spends in the queue

Thus, W_i $i = 1 \ldots n$ represents the sequence of wait times for the queue, each of which can be individually observed and tallied. This is tally type data because the customer enters a state (the queued state) at time A_i and exits the state at time D_i. When the customer exits the queue at time D_i, the waiting time in queue, $W_i = D_i - A_i$ can be observed or tallied. W_i is only observable at the instant D_i. This makes W_i tally based data and, once observed, its value never changes again with respect to time. Tally data is most often associated with an entity that is moving through states that are implied by the simulation model. An observation becomes available each time the entity enters and subsequently exits the state.

With tally data it is natural to compute the sample average as a measure of the central tendency of the data. Assume that you can observe n customers

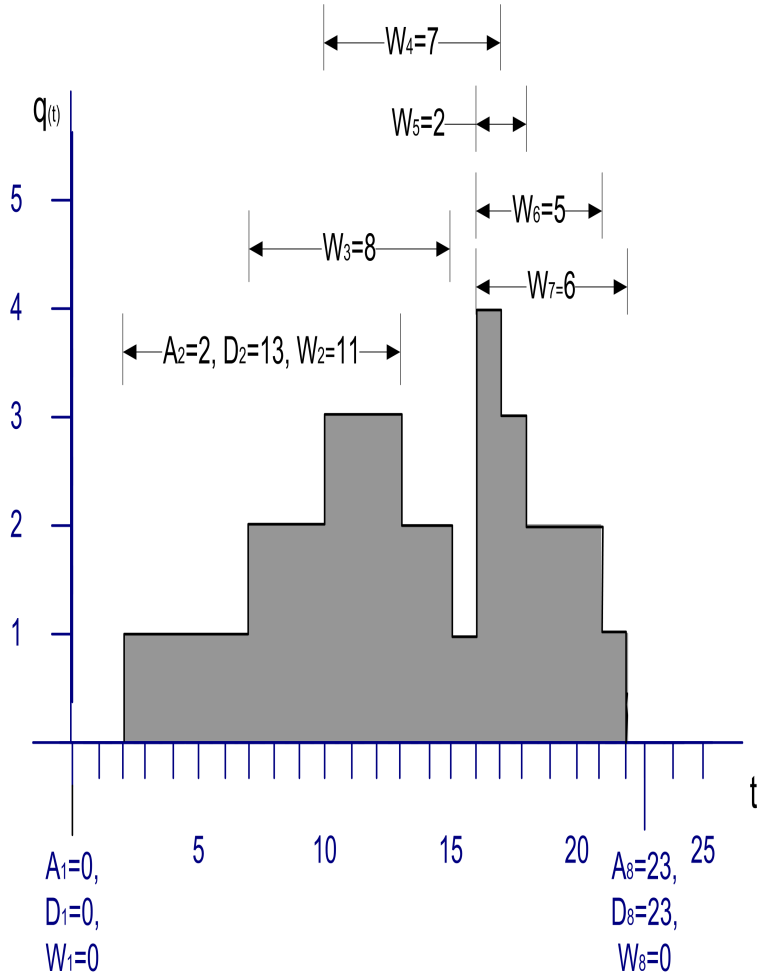


Figure 5.1.: Sample Path for Tally and Time-Persistent Data

entering and existing the queue, then the average waiting time across the n customers is given by:

$$\bar{W}(n) = \frac{1}{n}\sum_{i=1}^{n} W_i$$

Many other statistical quantities, such as the minimum, maximum, and sample variance, etc. can also be computed from these observations. Unfortunately, within replication data is often (if not always) correlated with respect to time. In other words, within replication observations like, $W_i\, i = 1 \ldots n$, are not statistically independent. In fact, they are likely to also not be identically distributed. Both of these issues will be discussed when the analysis of infinite horizon or steady state simulation models is presented.

The other type of statistical variable encountered within a replication is based on time-persistent observations. Let $q(t), t_0 < t \leq t_n$ be the number of customers in the queue at time t. Note that $q(t) \in \{0, 1, 2, ...\}$. As illustrated in Figure 5.1, $q(t)$ is a function of time (a step function in this particular case). That is, for a given (realized) sample path, $q(t)$ is a function that returns the number of customers in the queue at time t.

In simulation, we compute the time-average:

$$\bar{L}_q(n) = \frac{1}{t_n - t_0}\int_{t_0}^{t_n} q(t)\mathrm{d}t$$

This function represents the average with respect to time of the given state variable. This type of statistical variable is called time-persistent because $q(t)$ is a function of time (i.e. it persists over time).

Tally-based statistics and time-persistent statistics both are collected **during** a replication and form *within replication* statistical quantities. When we compute statistics across the replications, we call these statistics *across replication* statistics.

Now that we understand the type of data that occurs within a replication, we need to develop an understanding for the types of simulation situations that require specialized statistical analysis. The next section introduces this important topic.

5.2. Types of Simulation With Respect To Output Analysis

When modeling a system, specific measurement goals for the simulation responses are often required. The goals, coupled with how the system operates, will determine how you execute and analyze the simulation experiments. In planning the experimental analysis, it is useful to think of simulations as consisting of two main categories related to the period of time over which a decision needs to be made:

Finite horizon In a finite-horizon simulation, a well define ending time or ending condition can be specified which clearly defines the end of the simulation. Finite horizon simulations are often called *terminating* simulations, since there are clear terminating conditions.

Infinite horizon In an infinite horizon simulation, there is no well defined ending time or condition. The planning period is over the life of the system, which from a conceptual standpoint lasts forever. Infinite horizon simulations are often called *steady state* simulations because in an infinite horizon simulation you are often interested in the long-term or steady state behavior of the system.

For a finite horizon simulation, an event or condition associated with the system is present which indicates the end of each simulation replication. This event can be specified in advance or its time of occurrence can be a random variable. If it is specified in advance, it is often because you do not want information past that point in time (e.g. a 3 month planning horizon). It might be a random variable in the case of the system stopping when

a condition is met. For example, an ending condition may be specified to stop the simulation when there are no entities left to process. Finite horizon simulations are very common since most planning processes are finite. A few example systems involving a finite horizon include:

- Bank: bank doors open at 9am and close at 5pm
- Military battle: simulate until force strength reaches a critical value
- Filling a customer order: suppose a new contract is accepted to produce 100 products, you might simulate the production of the 100 products to see the cost, delivery time, etc.

For a finite horizon simulation, each replication represents a sample path of the model for one instance of the finite horizon. The length of the replication corresponds to the finite horizon of interest. For example, in modeling a bank that opens at 9 am and closes at 5 pm, the length of the replication would be 8 hours.

In contrast to a finite horizon simulation, an infinite horizon simulation has no natural ending point. Of course, when you actually simulate an infinite horizon situation, a finite replication length must be specified. Hopefully, the replication length will be long enough to satisfy the goal of observing long run performance. Examples of infinite horizon simulations include:

- A factory where you are interested in measuring the steady state throughput.
- A hospital emergency room which is open 24 hours a day, 7 days of week.
- A telecommunications system which is always operational.

Infinite horizon simulations are often tied to systems that operate continuously and for which the long-run or steady state behavior needs to be estimated.

Because infinite horizon simulations often model situations where the system is always operational, they often involve the modeling of non-stationary processes. In such situations, care must be taken in defining what is meant by long-run or steady state behavior. For example, in an emergency room that is open 24 hours a day, 365 days per year, the arrival pattern to such a system probably depends on time. Thus, the output associated with the system is also non-stationary. The concept of steady state implies that the system has been running so long that the system's behavior (in the form of performance measures) no longer depends on time; however, in the case of the emergency room since the inputs depend on time so do the outputs. In such cases it is often possible to find a period of time or cycle over which the non-stationary behavior repeats. For example, the arrival pattern to the emergency room may depend on the day of the week, such that every Monday has the same characteristics, every Tuesday has the same characteristics, and so on for each day of the week. Thus, on a weekly basis the non-stationary behavior repeats. You can then define your performance measure of interest based on the appropriate non-stationary cycle of the system. For example, you can define Y as the expected waiting time of patients *per week*. This random variable may have performance that can be described as long-term. In others, the long-run weekly performance of the system may be stationary. This type of simulation has been termed steady state cyclical parameter estimation within (Law 2007).

Of the two types of simulations, finite horizon simulations are easier to analyze. Luckily they are the more typical type of simulation found in practice. In fact, when you think that you are faced with an infinite horizon simulation, you should very carefully evaluate the goals of your study to see if they can just as well be met with a finite planning horizon. The analysis of both of these types of simulations will be discussed in this chapter through examples.

5.3. Analysis of Finite Horizon Simulations

This section illustrates how tally-based and time-persistent statistics are collected within a replication and how statistics are collected across replications. Finite horizon simulations can be analyzed by traditional statistical methodologies that assume a random sample, i.e. independent and identically distributed random variables. A simulation experiment is the collection of experimental design points (specific input parameter values) over which the behavior of the model is observed. For a particular design point, you may want to repeat the execution of the simulation multiple times to form a sample at that design point. To get a random sample, you execute the simulation starting from the same initial conditions and ensure that the random numbers used within each replication are independent. Each replication must also be terminated by the same conditions. It is very important to understand that independence is achieved across replications, i.e. the replications are independent. The data *within* a replication may or may not be independent.

The method of *independent replications* is used to analyze finite horizon simulations. Suppose that n replications of a simulation are available where each replication is terminated by some event E and begun with the same initial conditions. Let Y_{rj} be the j^{th} observation on replication r for $j = 1, 2, \cdots, m_r$ where m_r is the number of observations in the r^{th} replication, and $r = 1, 2, \cdots, n$, and define the sample average for each replication to be:

$$\bar{Y}_r = \frac{1}{m_r} \sum_{j=1}^{m_r} Y_{rj}$$

If the data is time-based then,

$$\bar{Y}_r = \frac{1}{T_E} \int_0^{T_E} Y_r(t) \mathrm{d}t$$

$\bar{Y}_r$ is the sample average based on the observation within the r^{th} replication. It is a random variable that can be observed at the end of each replication, therefore, $\bar{Y}_r$ for $r = 1, 2, \ldots, n$ forms a random sample. Thus, standard statistical analysis of the random sample can be performed.

To make this concrete, suppose that you are examining a bank that opens with no customers at 9 am and closes its doors at 5 pm to prevent further customers from entering. Let, $W_{rj} j = 1, \ldots, m_r$, represents the sequence of waiting times for the customers that entered the bank between 9 am and 5 pm on day (replication) r where m_r is the number of customers who were served between 9 am and 5 pm on day r. For simplicity, ignore the customers who entered before 5 pm but did not get served until after 5 pm. Let $N_r(t)$ be the number of customers in the system at time t for day (replication) r. Suppose that you are interested in the mean daily customer waiting time and the mean number of customers in the bank on any given 9 am to 5 pm day, i.e. you are interested in $E[W_r]$ and $E[N_r]$ for any given day. At the end of each replication, the following can be computed:

$$\bar{W}_r = \frac{1}{m_r} \sum_{j=1}^{m_r} W_{rj}$$

$$\bar{N}_r = \frac{1}{8} \int_0^8 N_r(t) \, \mathrm{d}t$$

At the end of all replications, random samples: $\bar{W}_r$ and $\bar{N}_r$ are available from which sample averages, standard deviations, confidence intervals, etc. can be computed. Both of these samples are based on observations of within replication data.

Both $\bar{W}_r$ and $\bar{N}_r$ for $r = 1, 2, \ldots, n$ are averages of many observations within the replication. Sometimes, there may only be one observation based on the entire replication. For example, suppose that you are interested in the probability that someone is still in the bank when the doors close at 5 pm, i.e. you are interested in $\theta = Pr\{N(t = 5pm) > 0\}$. In order to estimate

this probability, an indicator variable can be defined within the simulation and observed each time the condition was met or not. For this situation, an indicator variable,I_r, for each replication can be defined as follows:

$$I_r = \begin{cases} 1 & N(t = 5pm) > 0 \\ 0 & N(t = 5pm) \leq 0 \end{cases}$$

Therefore, at the end of the replication, the simulation must tabulate whether or not there are customers in the bank and record the value of this indicator variable. Since this happens only once per replication, a random sample of the I_r for $r = 1, 2, \ldots, n$ will be available after all replications have been executed. We can use the observations of the indicator variable to estimate the desired probability.

Since the analysis of the system will be based on a random sample, the key design criteria for the experiment will be the required number of replications. In other words, you need to determine the sample size.

Because confidence intervals may form the basis for decision making, you can use the confidence interval half-width in determining the sample size. For example, in estimating $E[W_r]$ for the bank example, you might want to be 95% confident that you have estimated the true waiting time to within ± 2 minutes.

Thus, all of the sample size determination methods discussed in Section 3.3.2 of Chapter 3 can be applied.

5.4. Capturing Output for a Simple Finite Horizon Simulation

In this section, we will build a model for a simple finite horizon simulation with a couple of new modeling issues to handle. However, the primary focus of this section is to illustrate how to capture and report statistical

results using the KSL. Let's start with an outline of the example system to be modeled.

Example 5.1 (Pallet Processing Work Center)**.** A truckload of pallets arrives overnight to a facility. Within the truck there are a random number of pallets. The number of pallets can be modeled with a binomial random variable with mean of 80 and a variance of 16. This translates to parameters $n = 100$ and $p = 0.8$. Each individual pallet is unloaded and transported to a work center for processing, one at a time, sequentially until all pallets are delivered. The unloading and transport time is exponentially distributed with a mean of 5 minutes. Once a pallet arrives at the work center it requires 1 of 2 workers to be processed. If a worker is available, the pallet is immediately processed by a worker. If no workers are available, the pallet waits in a FIFO line until a worker becomes available. The time to process the pallet involves breaking down and processing each package on the pallet. The time to process an entire pallet can be modeled with a triangular distribution with a minimum time of 8 minutes, a most likely time of 12 minutes, and a maximum time of 15 minutes. The work at the work center continues until all pallets are processed for the day. The facility manager is interested in how long the pallets wait at the work center and how long it takes for all pallets to be completed on a given day. In addition, the manager is interested in the probability that there is overtime. That is, the chance that the total time to process the pallets is more than 480 minutes.

This example is a finite horizon simulation because there are a finite (but random) number of pallets to be processed such that the simulation will run until all pallets are processed. The system starts with an arrival of

a random number of pallets, that are then processed until all pallets are completed. Therefore there is a well defined starting and ending point for the simulation. Although we do not know when the simulation will end, there is a well-specified condition (all pallets processed) that governs the ending of the simulation. Thus, while the time horizon may be random, it is still finite. Conceptually, this modeling situation is very similar to the pharmacy model discussed in Section 4.4.4. Thus, the implementation of the model will be very similar to the pharmacy model example, but with a couple of minor differences to handle the finite number of pallets that arrive and to capture statistics about the total processing time. The full code is available with the examples. This presentation focuses on new concepts.

As for previous modeling, we define and use random variables to represent the randomness within the model.

```
class PalletWorkCenter(
    parent: ModelElement,
    numWorkers: Int = 2,
    name: String? = null
) :
    ModelElement(parent, name = name) {

    init {
        require(numWorkers >= 1) { "The number of workers
 ↪  must be >= 1" }
    }

    @set:KSLControl(
        controlType = ControlType.INTEGER,
        lowerBound = 1.0
    )
    var numWorkers = numWorkers
        set(value) {
```

```
            require(value >= 1) { "The number of workers must
↪ be >= 1" }
            require(!model.isRunning) { "Cannot change the
↪ number of workers while the model is running!" }
            field = value
        }

    private val myProcessingTimeRV: RandomVariable =
     ↪ RandomVariable(
        parent = this,
        rSource = TriangularRV(min = 8.0, mode = 12.0, max =
↪ 15.0, streamNum = 3), name = "ProcessingTimeRV"
    )
    val processingTimeRV: RandomVariableCIfc
        get() = myProcessingTimeRV
    private val myTransportTimeRV: RandomVariable =
     ↪ RandomVariable(
        parent = parent,
        rSource = ExponentialRV(5.0, 2), name =
↪ "TransportTimeRV"
    )
    val transportTimeRV: RandomVariableCIfc
        get() = myTransportTimeRV
    private val myNumPalletsRV: RandomVariable =
     ↪ RandomVariable(
        parent = parent,
        rSource = BinomialRV(0.8, 100, 1), name =
↪ "NumPalletsRV"
    )
    val numPalletsRV: RandomVariableCIfc
        get() = myNumPalletsRV
```

Notice that we defined the random variables and assign them to rele-

vant properties within the class body. To capture the total time to process the pallets and the probability of overtime, we define two response variables. Notice that the probability of overtime is implemented as an `IndicatorResponse` that observes the total processing time.

```
    private val myNumBusy: TWResponse = TWResponse(parent =
     ↪ this, name = "NumBusyWorkers")
    val numBusyWorkers: TWResponseCIfc
        get() = myNumBusy
    private val myUtil: TWResponseFunction =
     ↪ TWResponseFunction(
        function = { x -> x/(this.numWorkers) },
        observedResponse = myNumBusy, name = "Worker
↪   Utilization"
    )
    val workerUtilization: TWResponseCIfc
        get() = myUtil
    private val myPalletQ: Queue<QObject> = Queue(parent =
     ↪ this, name = "PalletQ")
    val palletQ: QueueCIfc<QObject>
        get() = myPalletQ
    private val myNS: TWResponse = TWResponse(parent = this,
     ↪ name = "Num Pallets at WC")
    val numInSystem: TWResponseCIfc
        get() = myNS
    private val mySysTime: Response = Response(parent =
     ↪ this, name = "System Time")
    val systemTime: ResponseCIfc
        get() = mySysTime
    private val myNumProcessed: Counter = Counter(parent =
     ↪ this, name = "Num Processed")
    val numPalletsProcessed: CounterCIfc
        get() = myNumProcessed
    private val myTotalProcessingTime = Response(parent =
     ↪ this, name = "Total Processing Time")
```

```
    val totalProcessingTime: ResponseCIfc
        get() = myTotalProcessingTime
    private val myOverTime: IndicatorResponse =
     ↪  IndicatorResponse(
        predicate = { x -> x >= 480.0 },
        observedResponse = myTotalProcessingTime,
        name = "P{total time > 480 minutes}"
    )
    val probOfOverTime: ResponseCIfc
        get() = myOverTime
```

The main logic of arrivals to the work center and completions of the pallets is essentially the same as previously discussed. The main difference here is that there is a finite number of pallets that need to be created. While there are many ways to represent this situation (e.g. use an `EventGenerator`), this presentation keeps it simple. The first two lines of this code snippet, capture the functional references for the end of service and end of transport event actions. We have seen this kind of code before. Then, a variable, `numToProcess`, is defined. This variable will hold the randomly generated number of pallets to process and is assigned in the `initialize()` method. Within the `initialize()` method, the transport of the first pallet is scheduled with the provided transport time random variable.

```
    private val endServiceEvent = this::endOfService
    private val endTransportEvent = this::endTransport

    var numToProcess: Int = 0

    override fun initialize() {
        numToProcess = myNumPalletsRV.value.toInt()
        schedule(eventAction = endTransportEvent, timeToEvent
↪   = myTransportTimeRV)
    }
```

```
    private fun endTransport(event: KSLEvent<Nothing>) {
        if (numToProcess >= 1) {
            schedule(eventAction = endTransportEvent,
↪   timeToEvent = myTransportTimeRV)
            numToProcess = numToProcess - 1
        }
        val pallet = QObject()
        arrivalAtWorkCenter(pallet)
    }
```

The `endTransport` event action checks to see if there are more pallets to transport and if so schedules the end of the next transport. In addition, it creates a `QObject` that is sent to the work center for processing via the `arrivalAtWorkCenter` method. As shown in the following code, the arrival and end of service actions are essentially the same as in the previous pharmacy examples.

```
    private fun arrivalAtWorkCenter(pallet: QObject) {
        myNS.increment() // new pallet arrived
        myPalletQ.enqueue(pallet) // enqueue the newly
↪   arriving pallet
        if (myNumBusy.value < numWorkers) { // server
         ↪  available
            myNumBusy.increment() // make server busy
            val nextPallet: QObject? = myPalletQ.removeNext()
             ↪  //remove the next pallet
            // schedule end of service, include the pallet as
             ↪  the event's message
            schedule(eventAction = endServiceEvent,
↪   timeToEvent = myProcessingTimeRV, message = nextPallet)
        }
    }
```

```
    private fun endOfService(event: KSLEvent<QObject>) {
        myNumBusy.decrement() // pallet is leaving server is
↪   freed
        if (!myPalletQ.isEmpty) { // queue is not empty
            val nextPallet: QObject? = myPalletQ.removeNext()
             ↪  //remove the next pallet
            myNumBusy.increment() // make server busy
            // schedule end of service
            schedule(eventAction = endServiceEvent,
↪   timeToEvent = myProcessingTimeRV, message = nextPallet)
        }
        departSystem(completedPallet = event.message!!)
    }
```

Finally, we have the statistical collection code. The `departSystem` method captures the time in the system and the number of pallet processed. However, we see something new within a method called `replicationEnded()`

```
    private fun departSystem(completedPallet: QObject) {
        mySysTime.value = (time - completedPallet.createTime)
        myNS.decrement() // pallet left system
        myNumProcessed.increment()
    }

    override fun replicationEnded() {
        myTotalProcessingTime.value = time
    }
```

Recall the overview of how the underlying simulation code is processed from Section 4.4.2.2 of Chapter 4. Step 2(e) of that discussion notes that the actions associated with the end of replication logic is automatically

executed. Just like the `initialize()` method every model element has a `replicationEnded()` method. This method is called automatically for every model element infinitesimally before the end of the simulation replication. Thus, statistical collection can be implemented without the concern that the statistical accumulators will be cleared *after* the replication. Observations within the `replicationEnded()` method are still within the replication (i.e. they produce within replication data). In the above code snippet, the `Response` variable `myTotalProcessingTime` is assigned the current simulation time, which happens to be the time at which the processing of the pallets was completed. This occurs because after the appropriate number of transports are scheduled, no further pallets arrive to be processed at the work center. Thus, eventually, there will be no more events to process, and according to Step 2(d) of Section 4.4.2.2, this will cause the execution of the current replication to stop and proceed with any end of replication model logic. Thus, this logic captures the total time to process the pallets. In addition, because an `IndicatorResponse` was also attached to `myTotalProcessingTime` the collection of the probability of over time will also be captured. The basic results of the model are as follows:

Statistical Summary Report

Name	Count	Average	Half-Width
NumBusyWorkers	10.0	1.9174709267367938	0.04615844720276659
PalletQ:NumInQ	10.0	7.397061723049351	2.18200524048673
PalletQ:TimeInQ	10.0	44.65967086387171	12.756611150942986
Num Pallets at WC	10.0	9.314532649786145	2.2091925024424928
System Time	10.0	56.32044792861602	12.742871337228127
Total Processing Time	10.0	489.20830013021094	16.326885695752644
P{total time > 480 minutes}	10.0	0.6	0.369408717216522
Num Processed	10.0	80.4	2.5054518188054615

If you are paying attention to the presentation of previous summary statis-

tics, you will notice that the last summary report is presented in a nice tabular format. This output was actually generated by the KSL class `SimulationReporter`, which can make MarkDown output. This and additional output capturing will be discussed in the next section.

5.4.1. KSL Functionality for Capturing Statistical Results

This section presents a number of methods that can be used to capture and report the results from a KSL model. The following topics will be covered:

- Automatically capturing within and across replication results to comma separated value files
- Capturing replication data via the `ReplicationDataCollector` class
- Tracing a response variable via the `ResponseTrace` class
- Using the `SimulationReporter` class to output results in MarkDown and adjusting the confidence level of the reports
- Capturing all simulation results to a database using the `KSLDatabaseObserver` class and illustrating how to get results from the database

These topics will be discussed within the context of the pallet example of the previous section. The main method for running the pallet example was expanded to illustrate additional data collection functionality. It is unlikely that you will want to capture results both as comma separated value files and using a database, but both approaches are illustrated. If you are familiar with database technologies, then using the database will likely serve most if not all of your needs.

```
fun main() {
    val model = Model("Pallet Processing", autoCSVReports =
     ↪  true)
    model.numberOfReplications = 10
```

```
model.experimentName = "Two Workers"
// add the model element to the main model
val palletWorkCenter = PalletWorkCenter(model)

// demonstrate how to capture a trace of a response
 ↪  variable
val trace = ResponseTrace(palletWorkCenter.numInSystem)

// demonstrate capture of replication data for specific
 ↪  response variables
val repData = ReplicationDataCollector(model)
repData.addResponse(palletWorkCenter.totalProcessingTime)
repData.addResponse(palletWorkCenter.probOfOverTime)

// demonstrate capturing data to database with an
 ↪  observer
val kslDatabaseObserver = KSLDatabaseObserver(model)

// simulate the model
model.simulate()

// demonstrate that reports can have specified confidence
 ↪  level
val sr = model.simulationReporter
sr.printHalfWidthSummaryReport(confLevel = .99)

// show that report can be written to MarkDown as a table
 ↪  in the output directory
var out = model.outputDirectory.createPrintWriter(
 ↪  "hwSummary.md")
sr.writeHalfWidthSummaryReportAsMarkDown(out)
println()
```

```
    //output the collected replication data to prove it was
     ↪ captured
    println(repData)

    // use the database to create a Kotlin DataFrame
    val dataFrame = kslDatabaseObserver.db.
     ↪ acrossReplicationViewStatistics
    println(dataFrame)

    model.experimentName = "Three Workers"
    palletWorkCenter.numWorkers = 3
    model.simulate()

    out = model.outputDirectory.createPrintWriter(
     ↪ "AcrossExperimentResults.md")
    kslDatabaseObserver.db.writeTableAsMarkdown(
 ↪ "ACROSS_REP_VIEW", out)
}
```

Let's start with automatically collecting responses within comma separated value (CSV) files and how to find them in the file system.

The first thing to note is how the output from a KSL simulation is organized. The KSL model class allows the user to specify the output directory for the model results. If the user does not specify an output directory, the default directory will be within a folder called `kslOutput` that will be created within the same folder that the model was executed. That is, the current working directory for the user. Within the `kslOutput` directory all KSL related work will be stored. In particular, a unique directory derived from the name of the simulation will be created to hold all the results from a particular simulation model's execution. These locations can be changed, but the defaults are well-specified and useful.

Figure 5.2 illustrates the output directory after running the

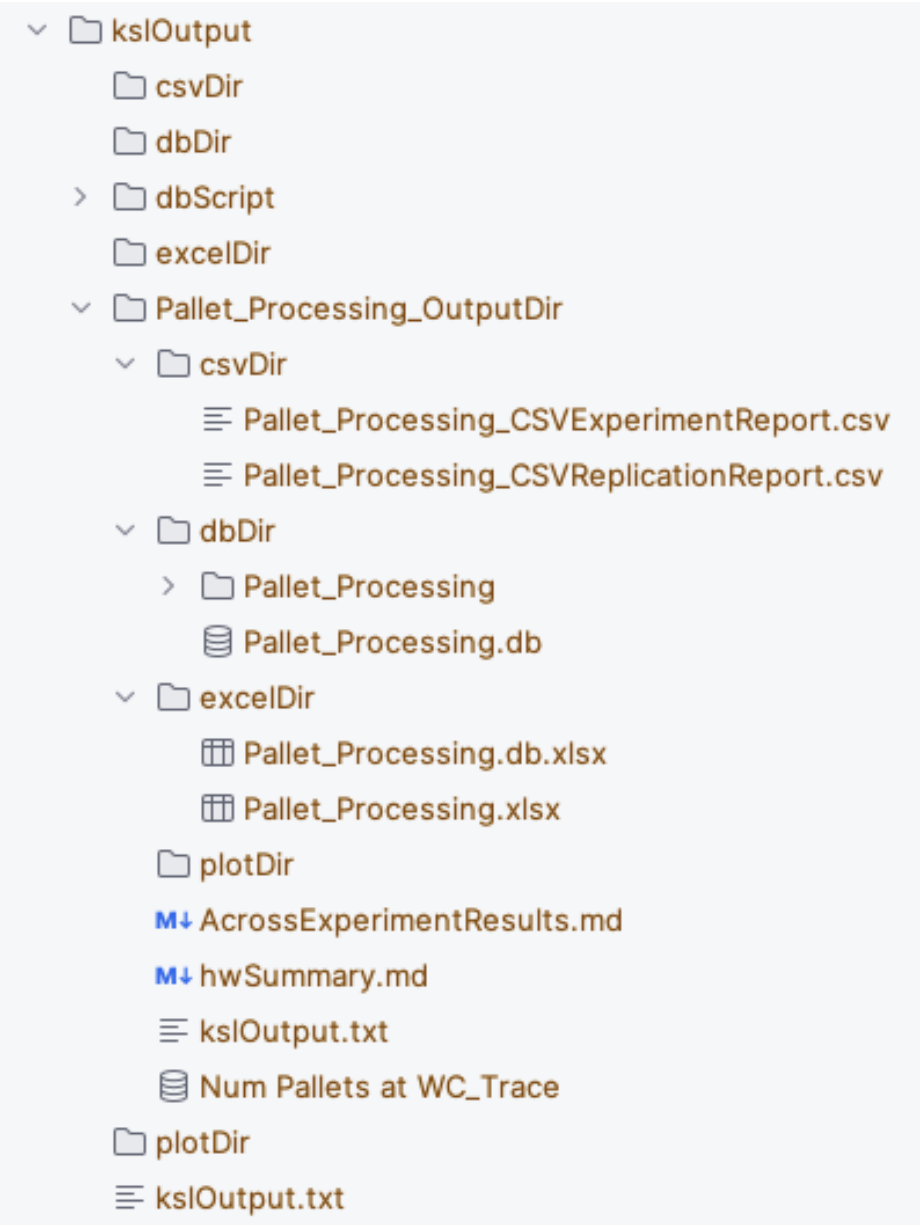


Figure 5.2.: Organization of KSL Output Directories

pallet model. You should see the `kslOutput` directory and a directory called `Pallet_Processing_OutputDir`. Within the `Pallet_Processing_OutputDir` directory there are folders called `db` and `excel`. These folders are the default directories for holding database related files and Excel related output files. Within the `db` folder there is a file called `Pallet_Processing.db`, which is an SQLite database that was created to hold the KSL simulation results. Then, there are two files called `hwSummary.md` and `kslOutput.txt`. There are also two CSV files, two labeled with `_ExperimentReport.csv` and `_ReplicationReport.csv`. This labeling scheme is the default and is derived from the context of the item. In addition, there is a file called `Num Pallets at WC_Trace`, which represents the trace data for the number of pallets at the workcenter within a database file. The setting of the `autoCSVReports` option to true when creating the model is what caused the two files labeled with `_ExperimentReport.csv` and with `_ReplicationReport.csv` to be produced. The following table is from `Pallet_Processing_CSVReplicationReport.csv`.

Table 5.1.: First 8 columns and 5 Rows of Pallet_Processing_CSVRepl cationReport.csv.

SimName	ModelName	ExpName	RepNum	ResponseType	ResponseID	ResponseName
Pallet Processing	MainModel	Two Workers	1	TWResponse	7	NumBusyWork
Pallet Processing	MainModel	Two Workers	1	TWResponse	9	PalletQ:NumIn
Pallet Processing	MainModel	Two Workers	1	Response	10	PalletQ:TimeIn
Pallet Processing	MainModel	Two Workers	1	TWResponse	11	Num Pallets at
Pallet Processing	MainModel	Two Workers	1	Response	12	System Time
Pallet Processing	MainModel	Two Workers	1	Response	14	Total Processin

As can be seen in the previous table, the replication report has information about the simulation, model, experiment, replication number, response type, and name. In total, the replication report has 19 columns and will contain every replication observation for every response variable in the

model. Notice that the experiment name is the string ("Two Workers") that was provided for the experiment's name on line 4 of the main method.

The following table illustrates columns 8 through 12 of the file.

Table 5.2.: Next 5 columns and 5 Rows of Pallet_Processing_CSVReplicationReport.csv.

Statistic.Name	Count	Average	Minimum	Maximum
NumBusyWorkers	79	1.889149	0.00000	2.00000
PalletQ:NumInQ	150	3.288616	0.00000	7.00000
PalletQ:TimeInQ	76	20.874804	0.00000	41.56455
Num Pallets at WC	152	5.177765	0.00000	9.00000
System Time	76	32.866354	12.61226	52.43014
Total Processing Time	1	482.417201	482.41720	482.41720

This data can be easily processed by other of statistical programs such as R or opened directly within Excel. The following table is from `Pallet_Processing_CSVExperimentReport.csv` and represents the across replication summary statistics for the responses and counters in the model.

Table 5.3.: Columns 7-15 and 9 Rows of Pallet_Processing_CSVE mentReport.csv.

Statistic.Name	Count	Average	Standard.Deviation	Standard.Error	Half.width	C
NumBusyWorkers	10	1.917471	0.0645251	0.0204046	0.0461584	
PalletQ:NumInQ	10	7.397062	3.0502330	0.9645684	2.1820052	
PalletQ:TimeInQ	10	44.659671	17.8325128	5.6391357	12.7566112	
Num Pallets at WC	10	9.314533	3.0882382	0.9765867	2.2091925	
System Time	10	56.320448	17.8133058	5.6330619	12.7428713	
Total Processing Time	10	489.208300	22.8234125	7.2173967	16.3268857	

P{total time > 480 minutes}	10	0.600000	0.5163978	0.1632993	0.3694087
Num Processed	10	80.400000	3.5023801	1.1075498	2.5054518
NumBusyWorkers	10	2.239980	0.1880656	0.0594716	0.1345340

As can be noted in the table, this is essentially the same information as reported in the output summary statistics. Again, these responses are automatically captured by simply setting the `autoCSVReports` option to true when creating the model.

A user may want to trace the values of specific response variables to files for post processing or display. This can be accomplished by using the `ResponseTrace` class. This code snippet, attaches an instance of the `ResponseTrace` class to the number in system response variable (`palletWorkCenter.numInSystem`) via the property that exposes the response to clients of the class. The user needs either an instance of the response variable or the exact name of the variable. This is one reason why the public property `numInSystem` was supplied.

```
// demonstrate how to capture a trace of a response
 ↪  variable
val trace = ResponseTrace(palletWorkCenter.numInSystem)
```

Attaching an instance of the `ResponseTrace` class to a response causes the trace to observe any value changes of the variable.

Table 5.4.: First 10 rows of Num Pallets at WC_Trace File.

n	t	x(t)	t(n-1)	x(t(n-1))	w	r	nr	sim	model	exp
1	0.000000	0	0.000000	0	0.0000000	1	1	Pallet Processing	MainModel	Two
2	7.126878	1	0.000000	0	7.1268782	1	2	Pallet Processing	MainModel	Two
3	19.739140	0	7.126878	1	12.6122616	1	3	Pallet Processing	MainModel	Two
4	26.281530	1	19.739140	0	6.5423903	1	4	Pallet Processing	MainModel	Two

5	32.059599	2	26.281530	1	5.7780694	1	5	Pallet Processing	MainModel	Two
6	33.697050	3	32.059599	2	1.6374507	1	6	Pallet Processing	MainModel	Two
7	34.220684	4	33.697050	3	0.5236344	1	7	Pallet Processing	MainModel	Two
8	38.888641	5	34.220684	4	4.6679570	1	8	Pallet Processing	MainModel	Two
9	40.431432	4	38.888641	5	1.5427903	1	9	Pallet Processing	MainModel	Two
10	46.112274	5	40.431432	4	5.6808427	1	10	Pallet Processing	MainModel	Two
11	46.776521	4	46.112274	5	0.6642467	1	11	Pallet Processing	MainModel	Two

In the output, $x(t)$ is the value of the variable at time t. Also, t_{n-1} is the previous time and $x(t_{n-1})$ is the value of the variable at the previous time. This facilitates plotting of the variable values over time. The column r represents the replication number and the column nr represents the number of observations in the current replication. These files can become quite large. Thus, it is recommended that you take the trace off when not needed and limit your trace coverage.

Before discussing the KSL database, note that the previously mentioned MarkDown functionality is implemented with the following code:

```
// show that report can be written to MarkDown as a table
 ↪  in the output directory
val out = model.outputDirectory.createPrintWriter(
 ↪  "hwSummary.md")
sr.writeHalfWidthSummaryReportAsMarkDown(out)
```

This code uses the model's output directory property to create a file based on a `PrintWriter` in the directory and then uses the `PrintWriter` instance via the `SimulationReporter` class. The KSL has a utility class found in the `ksl.io` package called `MarkDown` that facilitates simple construction of MarkDown text constructs, especially tables and rows within tables.

As shown in Figure 5.3, the `SimualationReporter` class has functionality to write simulation results in a number of formats. If you are a LaTeX user,

Figure 5.3.: The Functionality of the SimulationReporter Class

you may want to try the functionality to create LaTeX tables. Besides reporting functionality, there is some limited ability to create copies of the underlying statistical objects. But, to directly get raw data, you should use the `ReplicationDataCollector` class or the KSL database.

The `ReplicationDataCollector` class and the `ExperimentDataCollector` class (not discussed here) are observers that can be attached to the model that will hold replication data and across experiment data, respectively. The collection can be limited to specific responses or all responses. The classes will hold the data observed from the simulation in memory (via arrays). These arrays can be accessed to perform post processing within code as needed.

```
    // demonstrate capture of replication data for specific
     ↪  response variables
    val repData = ReplicationDataCollector(model)
    repData.addResponse(palletWorkCenter.totalProcessingTime)
    repData.addResponse(palletWorkCenter.probOfOverTime)
```

This code creates a `ReplicationDataCollector` instance and configures the instance to collect the replication data for the total processing time and probability of over time responses. A simple output of the first 10 values of the data arrays is shown here.

```
   Total Processing Time P{total time > 480 minutes}
0             482.417201                          1.0
1             461.544205                          0.0
2             521.417293                          1.0
3             476.025297                          0.0
4             534.281150                          1.0
5             485.735690                          1.0
6             477.468018                          0.0
7             482.557886                          1.0
```

```
8              499.628817                    1.0
9              471.007443                    0.0
```

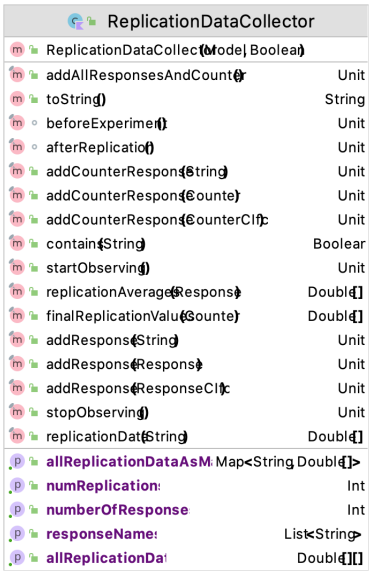


Figure 5.4.: The Functionality of the ReplicationDataCollector Class

The most notable functionality shown in Figure 5.4 is the `allReplicationDataAsMap` property. The map is indexed by the name of the response as the key and the replication data as a `DoubleArray`. Thus, any response data that you need can be readily access within memory. The `ExperimentDataCollector` class (not discussed here) has similar functionality but will capture the data across different experiments. Experiment functionality will be discussed in a later chapter.

Now, let's discuss the most useful KSL functionality for capturing simulation data, the `KSLDatabase` class. The following code creates an observer of the model in the form of a `KSLDatabaseObserver`. This class can be

attached to a model and then will collect and insert all response and counter related statistical quantities into a well-structured database. The default database is an SQLite database but other database engines will also work. There is built in functionality for creating Derby and PostgreSQL based databases.

Figure 5.5.: The Functionality of the KSLDatabase Class

Figure 5.5 presents the functionality of the `KSLDatabase` class. This class is built on top of functionality within the KSL `ksl.utilities.io.dbutil` package for creating databases via the `DatabaseIfc` interface and the `Database` class. Here our focus is on how the data from a KSL simulation is captured and stored. The capturing occurs because the user attaches an instance of a `KSLDatabaseObserver` to a model as shown in this code snippet.

```
// demonstrate capturing data to database with an
 ↪  observer
val kslDatabaseObserver = KSLDatabaseObserver(model)
```

This code creates an observer that accesses the simulation data after each replication and stores it within a database. The database will be stored within the `dbDir` folder of the simulation model's output directory. Note that if you execute **any** simulation that has the **same name** as a previously executed simulation then the previous database will be deleted and recreated. This might cause you to lose previous simulation results. This behavior is the default because generally, when you re-run your simulation you want the results of that run to be written into the database. However, it is possible to not overwrite the database and store additional experiments within the same database. To do this, get an instance of an existing KSL database before creating the `KSLDatabaseObserver.` In addition, by changing the name of the experiment associated with the model, then the results from different experiments can be stored in the *same* database. This approach is useful when comparing results across simulation configurations.

The database will have the table structure shown in Figure 5.6.

The KSL database consists of twelve tables that capture information and data concerning the execution of a simulation and resulting statistical quantities. Figure 5.6 presents the database diagram for the `KSL_DB` database schema.

- `SIMULATION_RUN`: contains information about the simulation runs that are contained within the database. Such information as the name of the simulation, model, and experiment are captured. In addition, time stamps of the start and end of the experiment, the number of replications, the replication length, the length of the warm up period and options concerning stream control.

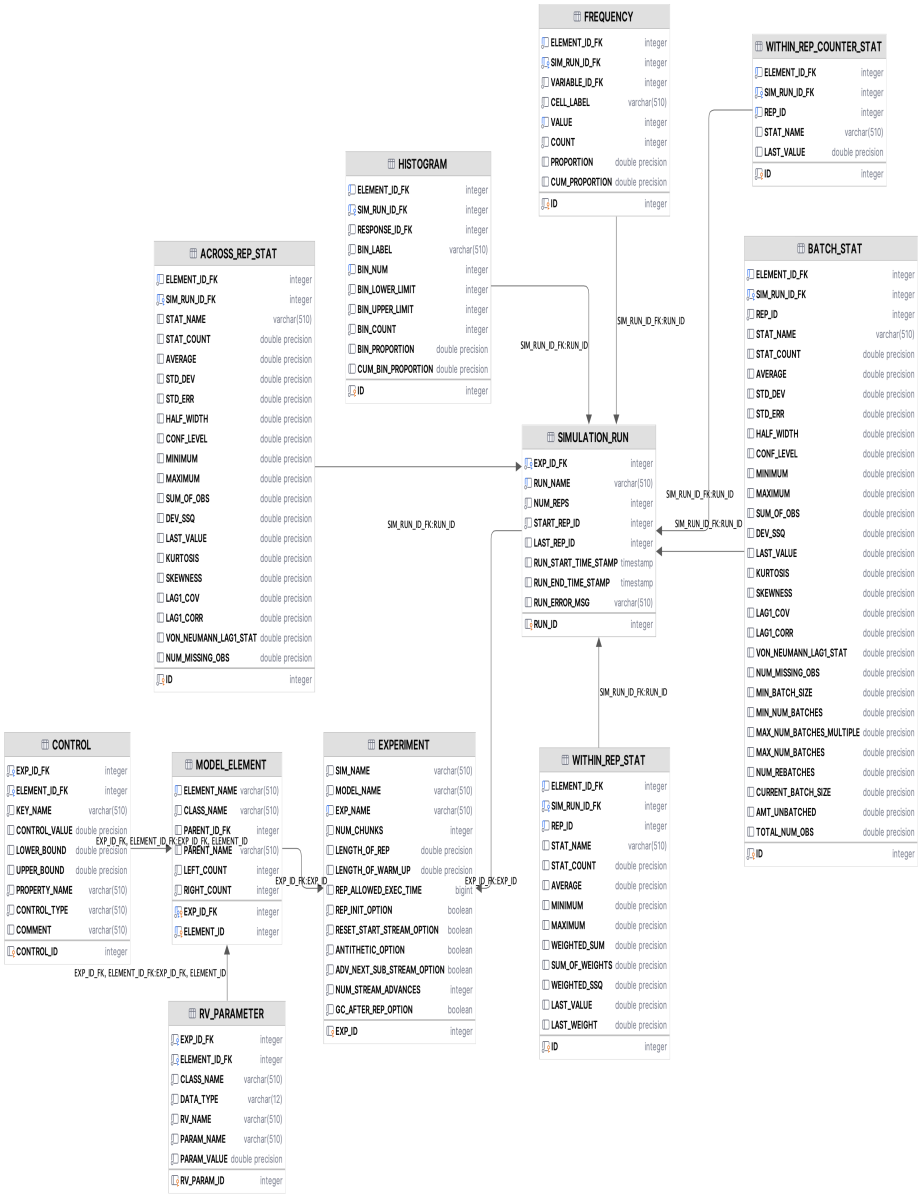


Figure 5.6.: KSL Database Tables

- `MODEL_ELEMENT` contains information about the instances of `ModelElement` that were used within the execution of the simulation run. A model element has an identifier that is considered unique to the simulation run. That is, the simulation run ID and the model element ID are the primary key of this table. The name of the model element, its class type, the name and ID of its parent element are also held for each entity in `MODEL_ELEMENT`. The parent/child relationship permits an understanding of the model element hierarchy that was present when the simulation executed.
- `WITHIN_REP_STAT` contains information about within replication statistical quantities associated with `TWResponse` and `Response` instances from each replication of a set of replications of the simulation. The name, count, average, minimum, maximum, weighted sum, sum of weights, weighted sum of squares, last observed value, and last observed weight are all captured.
- `WITHIN_REP_COUNTER_STAT` contains information about with replication observations associated with Counters used within the model. The name of the counter and the its value at the end of the replication are captured for each replication of a set of replications of the simulation.
- `ACROSS_REP_STAT` contains information about the across replication statistics associated with `TWResponse, Response,` and `Counter` instances within the model. Statistical summary information across the replications is automatically stored.
- `BATCH_STAT` contains information about the batch statistics associated with `TWResponse, Response,` and `Counter` instances within the model. Statistical summary information across the batches is automatically stored.
- `HISTOGRAM` contains the results from `HistogramResponse` instances when they are used within a model.
- `FREQUENCY` contains the results from `IntegerFrequencyResponse` instances when they are used within a model.
- `TIME_SERIES_RESPONSE` represents within replication collection of response variables based on periods of time. The average value of

the response during the period is recorded for each period for each replication of each simulation run.

- `EXPERIMENT` holds information across experimental runs. This is illustrated in Chapter 10.
- `CONTROL` holds the controls associated with a model as discussed in Section 10.1.1.
- `RV_PARAMETER` holds the random variables and their parameters as discussed in Section 10.1.2.

In addition to the base tables, the KSL database contains views of its underlying tables to facilitate simpler data extraction. Figure 5.7 presents the pre-defined views for the KSL database. The views, in essence, reduce the amount of information to the most likely used sets of data for the across replication, batch, and within replication captured statistical quantities. In addition, the `PW_DIFF_WITHIN_REP_VIEW` holds all pairwise differences for every response variable, time weighted variable, or counter from across all experiments within the database. This view reports $(A - B)$ for every within replication ending average, where A is a simulation run that has higher simulation ID than B and they represent an individual performance measure. From this view, pairwise statistics can be computed across all replications. Since simulation IDs are assigned sequentially within the database, the lower the ID the earlier the simulation was executed.

The information within the `SIMULATION_RUN`, `MODEL_ELEMENT`, `WITHIN_REP_STAT`, and `WITHIN_REP_COUNTER_STAT` tables are written to the database at the end of each replication. The `ACROSS_REP_STAT` and `BATCH_STAT` tables are filled after the entire experiment is completed. Even though the `ACROSS_REP_STAT` table could be constructed directly from the data captured within the tables holding with replication data, this is not done. Instead, the across replication statistics are directly written from the simulation after all replications of an experiment are completed.

As an illustration consider running a simulation multiple times within the same program execution but with different parameters. The following code

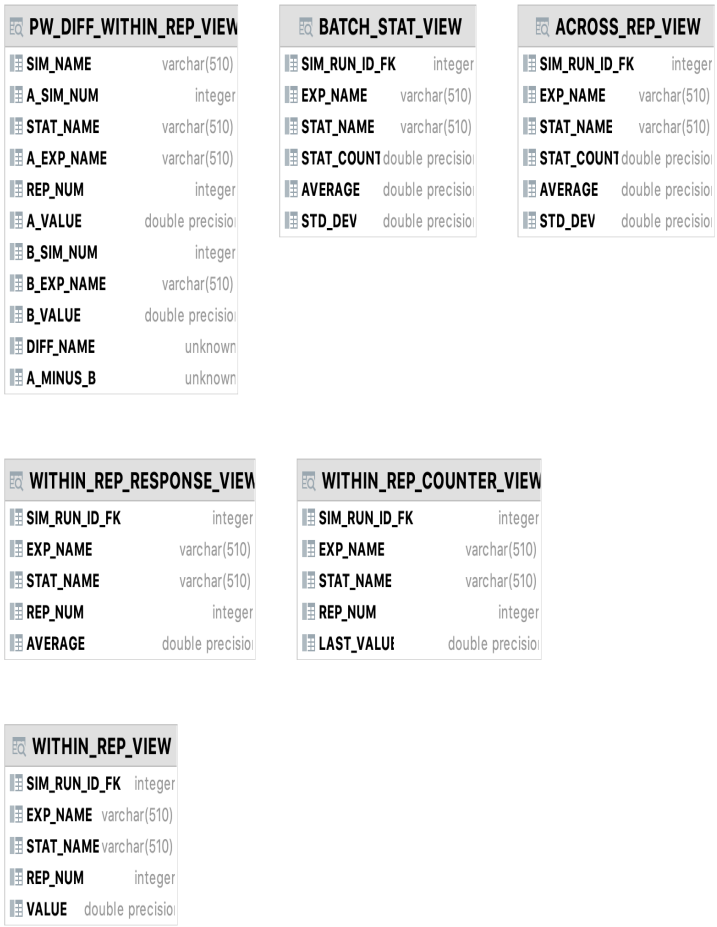


Figure 5.7.: KSL Database Views

illustrates how this might be achieved. Please note the following code at the bottom of the main execution routine.

```
    model.experimentName = "Three Workers"
    palletWorkCenter.numWorkers = 3
    model.simulate()

    out = model.outputDirectory.createPrintWriter(
     ↪  "AcrossExperimentResults.md")
    kslDatabaseObserver.db.writeTableAsMarkdown(
↪   "ACROSS_REP_VIEW", out)
```

In this code, the experiment name has been updated to "Three Workers" and the number of workers property changed. The model was simulated again. The underlying KSL database was accessed to write out the across replication view statistics. As shown in the following table, the database retained the data from the first execution and added the data from the second execution. This only occurred because the name of the experiment was changed between calls to the `simulate()` method. In the results, we see that adding a worker causes the chance of overtime to become zero.

Table: ACROSS_REP_VIEW

SIM_ID	EXP_NAME	STAT_NAME	COUNT	AVERAGE	STD_DE
1	Two Workers	Num Pallets at WC	10.0	9.3145	3.0882
2	Three Workers	Num Pallets at WC	10.0	3.2083	0.7035
1	Two Workers	Num Processed	10.0	80.4	3.5023
2	Three Workers	Num Processed	10.0	79.1	4.1217
1	Two Workers	NumBusyWorkers	10.0	1.9174	0.0645
2	Three Workers	NumBusyWorkers	10.0	2.2399	0.1881
1	Two Workers	PalletQ:NumInQ	10.0	7.3971	3.0502
2	Three Workers	PalletQ:NumInQ	10.0	0.9683	0.5725
1	Two Workers	PalletQ:TimeInQ	10.0	44.6596	17.832

SIM_ID	EXP_NAME	STAT_NAME	COUNT	AVERAGE	STD_DE
2	Three Workers	PalletQ:TimeInQ	10.0	4.9261	2.6231
1	Two Workers	P{total time > 480 minutes}	10.0	0.6	0.5163
2	Three Workers	P{total time > 480 minutes}	10.0	0.0	0.0
1	Two Workers	System Time	10.0	56.32044792861602	17.813
2	Three Workers	System Time	10.0	16.5628	2.6346
1	Two Workers	Total Processing Time	10.0	489.2083	22.823
2	Three Workers	Total Processing Time	10.0	412.6305	28.144

A `KSLDatabase` instance is constructed to hold the data from any KSL simulation. As such, a simulation execution can have many observers and thus could have any number of `KSLDatabase` instances that collect data from the execution. The most common case for multiple databases would be the use of an embedded database as well as a database that is stored on a *remote* database server.

Once you have a database that contains the schema to hold KSL based data, you can continue to write results to that database as much as you want. If your database is on a server, then you can easily collect data from different simulation executions that occur on different computers by referencing the database on the server. Therefore, if you are running multiple simulation runs in parallel on different computers or in the "cloud", you should be able to capture the data from the simulation runs into one database.

5.4.2. Additional Remarks

The functionality of the KSL database depends upon how `Response`, `TWResponse`, and `Counter` instances are named within a KSL model. A KSL model is organized into a tree of `ModelElement` instances with the instance of the `Model` at the top of the tree. The `Model` instance for the simulation model contains instances of `ModelElement`, which are referred to as children of the parent model element. Each model element instance

can have zero or more children, and those children can have children, etc. Each `ModelElement` instance must have a unique integer ID and a unique name. The unique integer ID is automatically provided when a `ModelElement` instance is created. The user can supply a name for a `ModelElement` instance when creating the instance. The name must be unique within the simulation model.

A recommended practice to ensure that model element names are unique is to use the name of the parent model element as part of the name. If the parent name is unique, then all children names will be unique relative to any other model elements. For example, in the following code `name` references the name of the current model element (an instance of `Queue`), which is serving as the parent for the children model element (responses) declared within the constructor body. Unfortunately, this approach may lead to very long names if the model element hierarchy is deep.

```
    protected val myNumInQ: TWResponse = TWResponse(this,
     ↪  name = "${name}:NumInQ")
    override val numInQ : TWResponseCIfc
        get() = myNumInQ

    protected val myTimeInQ: Response = Response(this, name =
     ↪  "${name}:TimeInQ")
    override val timeInQ : ResponseCIfc
        get() = myTimeInQ
```

The name supplied to the `TWResponse` and `Response` constructors will cause the underlying statistic to have the indicated names. The response's name cannot be changed once it is set. The statistic name is important for referencing statistical data within the KSL database.

One complicating factor involves using the KSL database to analyze the results from multiple simulation models. In order to more readily compare the results of the same performance measure between two different

simulation models, the user should try to ensure that the names of the performance measures are the same. If the above recommended naming practice is used, the names of the statistics may depend on the order in which the model element instances are created and added to the model element hierarchy. If the model structure never changes between different simulation models then this will not present an issue; however, if the structure of the model changes between two different simulation models (which can be the case), the statistic names may be affected. If this issue causes problems, you can always name the desired output responses or counters exactly what you want it to be and use the same name in other simulation models, as long as you ensure unique names.

Since the model element ID is assigned automatically based on the number of model elements created within the model, the model element numbers between two instances of the same simulation model will most likely be different. Thus, there is no guarantee that the IDs will be the same and using the model element ID as part of queries on the KSL database will have to take this into account. You can assume that the name of the underlying statistic is the same as its associated model element and since it is user definable, it is better suited for queries based on the KSL database.

5.4.3. Querying the KSL Database

The KSL database is a database and thus it can be queried from within Kotlin or from other programs. If you have an instance of the `KSLDatabase`, you can extract information about the simulation run using the methods of the `KSLDatabase` class. Since the underlying data is stored in a relational database, SQL queries can be used on the database. The discussion of writing and executing SQL queries is beyond the scope of this text. To facilitate output when using the KSL, the KSL has a few methods to be aware of, including:

- `exportAllTablesAsCSV()`: writes all the tables to separate CSV files

- `exportDbToExcelWorkbook()`: writes all the tables and views to a single Excel workbook
- `multipleComparisonAnalyzerFor(set of experiment name, response name)`: returns an instance of the `MultipleComparisonAnalyzer` class in order to perform a multiple comparison analysis of a set of experiments on a specific response name.
- a number of properties that return Kotlin data frames of the underlying database tables.

The data frame functionality was illustrated in the following code from the main execution method:

```
// use the database to create a Kotlin DataFrame
val dataFrame = kslDatabaseObserver.db.
 ↪  acrossReplicationViewStatistics
println(dataFrame)
```

> **Note**
>
> The Kotlin data frames library has excellent capabilities for processing data within a tabular format. The KSL has additional functionality for processing files via its utilities as described in Section E.4 of Appendix E. Also, you might want to explore the extensions for data frames described in Section E.5 of Appendix E.

Once you have the data frame you can employ whatever data wrangling and extraction methods that you need. Finally, because the KSL database is a *database*, it can be accessed via R or other software programs such as IntelliJ's DataGrip or DBeaver and additional analysis performed on the KSL simulation data.

Based on the discussion in this section, the KSL has very useful functionality for working with the data generated from your simulation models.

5.5. Sequential Sampling for Finite Horizon Simulations

The methods discussed in Section 3.3.2 of Chapter 3 for determining the sample size are based on pre-determining a *fixed* sample size and then making the replications. If the half-width equation is considered as an iterative function of n:

$$h(n) = t_{\alpha/2,n-1}\frac{s(n)}{\sqrt{n}} \leq E$$

Then, it becomes apparent that additional replications of the simulation can be executed until the desired half-with bound is met. This is called sequential sampling, and in this case the sample size of the experiment is not known in advance. The brute force method for implementing this approach would be to run and rerun the simulation each time increasing the number of replications until the criterion is met.

To implement this within the KSL, we need a way to stop or end a simulation when a criteria or condition is met. Because of the hierarchical nature of the model elements within a model and because there are common actions that occur when running a model the `Observer` pattern can be used here.

Figure 5.8 presents part of the `ksl.observers` package which defines a base class called `ModelElementObserver` that can be attached to instances of `ModelElement` and then will be notified if various actions take place. There are a number of actions associated with a `ModelElement` that occur during a simulation that can be listened for by a `ModelElementObserver`:

- `beforeExperiment()`: This occurs prior to the first replication and before any events are executed.

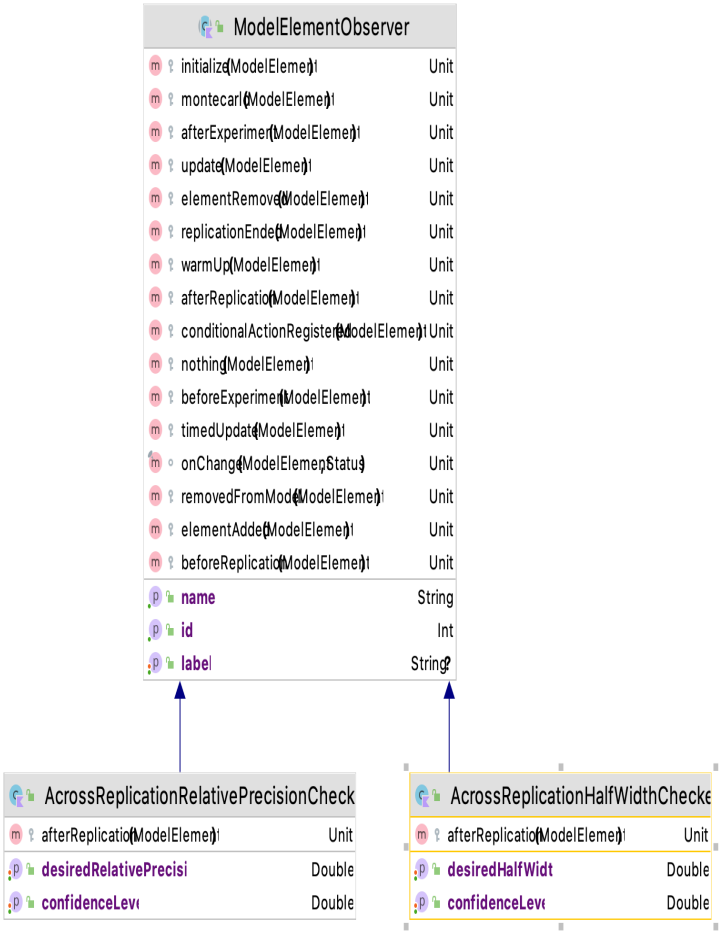


Figure 5.8.: Half-Width Observer Checking Code

- `beforeReplication()`: This occurs prior to each replication and before any events are executed. The event calendar is cleared after this action.
- `initialize()`: This occurs at the start of every replication (after beforeReplication() and after the event calendar is cleared) but before any events are executed. As we have seen, it is safe to schedule events in this method.
- `warmUp()`: This occurs during a replication if a warm up period has been specified for the model. The statistical accumulators are cleared during this action if applicable.
- `replicationEnded()`: This occurs at the end of every replication prior to the clearing of any statistical accumulators.
- `afterReplication()`: This occurs at the end of every replication after the statistical accumulators have been cleared for the replication.
- `afterExperiment()`: This occurs after all replications have been executed and prior to the end of the simulation.

`ModelElementObservers` are notified right *after* the `ModelElement` experiences the above mentioned actions. Thus, users of the `ModelElementObserver` need to understand that the state of the model element is available after the simulation actions of the model element have occurred.

The `AcrossReplicationHalfWidthChecker` class listens to the `afterReplication()` method of a `Response` instance and checks the current value of the half-width. This is illustrated in the following code listing.

```
    override fun afterReplication(modelElement: ModelElement)
     ↪ {
        if (modelElement.model.currentReplicationNumber <=
         ↪ 2){
```

```
            return
        }
        val statistic =
         ↪ myResponse.myAcrossReplicationStatistic
        val hw = statistic.halfWidth(confidenceLevel)
        if (hw <= desiredHalfWidth){
            modelElement.model.endSimulation("Half-width =
↪  ($desiredHalfWidth) condition met for response
↪  ${myResponse.name}")
        }
    }
```

Notice that a reference to the observed `Response` is used to get access to the across replication statistics. If there are more than 2 replications, then the half-width is checked against a user supplied desired half-width. If the half-width criterion is met, then the simulation is told to end using the `Model` class's `endSimulation()` method. The `endSimulation()` method causes the simulation to not execute any future replications and to halt further execution.

All that is needed is to get a reference to the response variable that needs to be checked so that the observer can be attached. This can be done easily if a reference to the response is exposed through the class's interface. In this case the property, `totalProcessingTime`, an instance of `ResponseCIfc`, is used. Alternatively if the string name representation of the response is available, it can be used to extract the response from the model via the model's functionality for getting model elements.

```
fun response(name: String): Response?
fun timeWeightedResponse(name: String): TWResponse?
fun counter(name: String): Counter?
```

Example 5.2 (Sequential Sampling)**.** The following code listing illustrates how to set up half-width checking for the pallet processing model.

```
fun main() {
    val model = Model("Pallet Processing Ex 2")
    model.numberOfReplications = 10000
    model.experimentName = "Two Workers"
    // add the model element to the main model
    val palletWorkCenter = PalletWorkCenter(model)

    val hwc = AcrossReplicationHalfWidthChecker(
     ↪ palletWorkCenter.totalProcessingTime)
    hwc.desiredHalfWidth = 5.0

    // simulate the model
    model.simulate()

    // demonstrate that reports can have specified confidence
     ↪ level
    val sr = model.simulationReporter

    sr.printHalfWidthSummaryReport()

}
```

Notice that the number of replications is set to an arbitrarily high value (10000) and the desired half-width is specified as 5.0. In the original example, the half-width for the total processing time was 16.3269 based on 10 replications. As we can see from the following output, the half-width criteria has been met with 149 replications.

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%
```

```
Name                                                    Count         Average      Half-
Width
-------------------------------------------------------------------------------------
NumBusyWorkers                                            149          1.9197          0
PalletQ:NumInQ                                            149          7.3231          0
PalletQ:TimeInQ                                           149         44.2002          2
Num Pallets at WC                                         149          9.2428          0
System Time                                               149         55.8544          2
Total Processing Time                                     149        495.6675          4
P{total time > 480 minutes}                               149          0.7114          0
Num Processed                                             149         81.5503          0
-------------------------------------------------------------------------------------
```

5.6. Analysis of Infinite Horizon Simulations

This section discusses how to plan and analyze infinite horizon simulations. When analyzing infinite horizon simulations, the primary difficulty is the nature of within replication data. In the finite horizon case, the statistical analysis is based on three basic requirements:

1. Observations are independent
2. Observations are sampled from identical distributions
3. Observations are drawn from a normal distribution (or enough observations are present to invoke the central limit theorem)

These requirements were met by performing independent replications of the simulation to generate a random sample. In a direct sense, the data within a replication do not satisfy any of these requirements; however, certain procedures can be imposed on the manner in which the observations are gathered to ensure that these statistical assumptions may be acceptable. The following will first explain why within replication data typically violates

these assumptions and then will provide some methods for mitigating the violations within the context of infinite horizon simulations.

To illustrate the challenges related to infinite horizon simulations, a simple spreadsheet simulation was developed for a M/M/1 queue.

Consider a single server queueing system as illustrated Figure 5.9.

For a single server queueing system, there is an equation that allows the computation of the waiting times of each of the customers based on knowledge of the arrival and service times. Let $X_1, X_2, ...$ represent the successive service times and $Y_1, Y_2, ...$ represent the successive inter-arrival times for each of the customers that visit the queue. Let $E[Y_i] = 1/\lambda$ be the mean of the inter-arrival times so that λ is the mean arrival rate. Let $E[Y_i] = 1/\mu$ be the mean of the service times so that μ is the mean service rate. Let W_i be the waiting time in the queue for the i^{th} customer. That is, the time between when the customer arrives until they enter service.

Lindley's equation, see (Gross and Harris 1998), relates the waiting time to the arrivals and services as follows:

$$W_{i+1} = max(0, W_i + X_i - Y_i)$$

The relationship says that the time that the $(i+1)^{st}$ customer must wait is the time the i^{th} waited, plus the i^{th} customer's service time, X_i (because that customer is in front of the i^{th} customer), less the time between arrivals of the i^{th} and $(i+1)^{st}$ customers, Y_i. If $W_i + X_i - Y_i$ is less than zero, then the $((i+1)^{st}$ customer arrived after the i^{th} finished service, and thus the waiting time for the $(i+1)^{st}$ customer is zero, because his service starts immediately.

Suppose that $X_i \sim exp(E[X_i] = 0.7)$ and $Y_i \sim exp(E[Y_i] = 1.0)$. This is a M/M/1 queue with $\lambda = 1$ and $\mu = 10/7$. Thus, based on traditional queueing theory results:

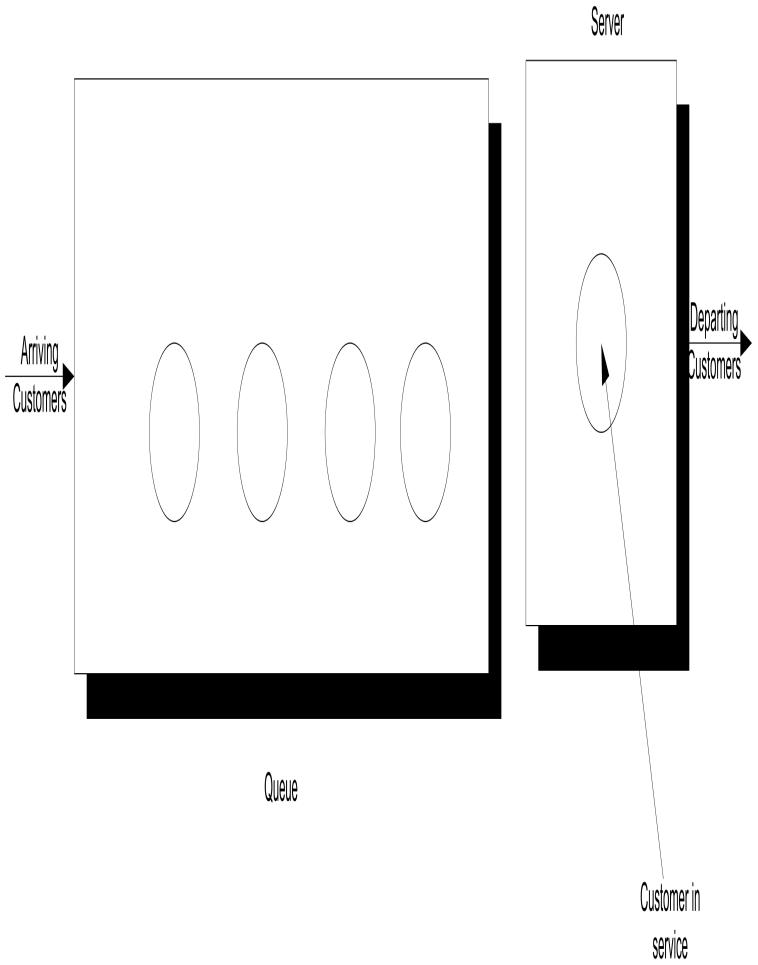


Figure 5.9.: Single Server Queueing System

$$\rho = 0.7$$

$$L_q = \frac{0.7 \times 0.7}{1 - 0.7} = 1.6\bar{3}$$

$$W_q = \frac{L_q}{\lambda} = 1.6\bar{3} \text{ minutes}$$

Example 5.3 (Lindley Equation)**.** Lindley's equation can be readily implemented in using KSL constructs as illustrated in the following code listing.

```
class LindleyEquation(
    var tba: RandomIfc = ExponentialRV(1.0),
    var st: RandomIfc = ExponentialRV(0.7),
    var numReps: Int = 30,
    var numObs: Int = 100000,
    var warmUp: Int = 10000
) {
    init {
        require(numReps > 0) { "The number of replications
 ↪  must be > 0" }
        require(numObs > 0) { "The number of replications
 ↪  must be > 0" }
        require(warmUp >= 0) { "The number of replications
 ↪  must be > 0" }
    }

    val avgw = Statistic("Across rep avg waiting time")
    val avgpw = Statistic("Across rep prob of wait")
    val wbar = Statistic("Within rep avg waiting time")
    val pw = Statistic("Within rep prob of wait")
```

```
    fun simulate(r: Int = numReps, n: Int = numObs, d: Int =
     ↪  warmUp) {
        require(r > 0) { "The number of replications must be
↪  > 0" }
        require(n > 0) { "The number of replications must be
↪  > 0" }
        require(d >= 0) { "The number of replications must be
↪  > 0" }
        for (i in 1..r) {
            var w = 0.0 // initial waiting time
            for (j in 1..n) {
                w = Math.max(0.0, w + st.value - tba.value)
                wbar.collect(w) // collect waiting time
                pw.collect(w > 0.0) // collect P(W>0)
                if (j == d) { // clear stats at warmup
                    wbar.reset()
                    pw.reset()
                }
            }
            //collect across replication statistics
            avgw.collect(wbar.average)
            avgpw.collect(pw.average)
            // clear within replication statistics for next
             ↪  rep
            wbar.reset()
            pw.reset()
        }

    }

    fun print() {
        println("Replication/Deletion Lindley Equation
↪  Example")
```

```
        println(avgw)
        println(avgpw)
        println()
        val sr = StatisticReporter(mutableListOf(avgw,
         ↪  avgpw))
        print(sr.halfWidthSummaryReport())
    }

}
```

This implementation can be readily extended to capture the data to files for display in spreadsheets or other plotting software. As part of the plotting process it is useful to display the cumulative sum and cumulative average of the data values.

$$\sum_{i=1}^{n} W_i \text{ for } n = 1, 2, ...$$

$$\frac{1}{n}\sum_{i=1}^{n} W_i \text{ for } n = 1, 2, ...$$

Figure 5.10 presents the cumulative average plot of the first 1000 customers. As seen in the plot, the cumulative average starts out low and then eventually trends towards 1.2 minutes.

The analytical results indicate that the true long-run expected waiting time in the queue is 1.633 minutes. The average over the 1000 customers in the simulation is 1.187 minutes. The results in Figure 5.11 indicated that the sample average is significantly lower than the true expected average. We will explore why this occurs shortly.

The first issue to consider with this data is independence. To do this you should analyze the 1000 observations in terms of its autocorrelation.

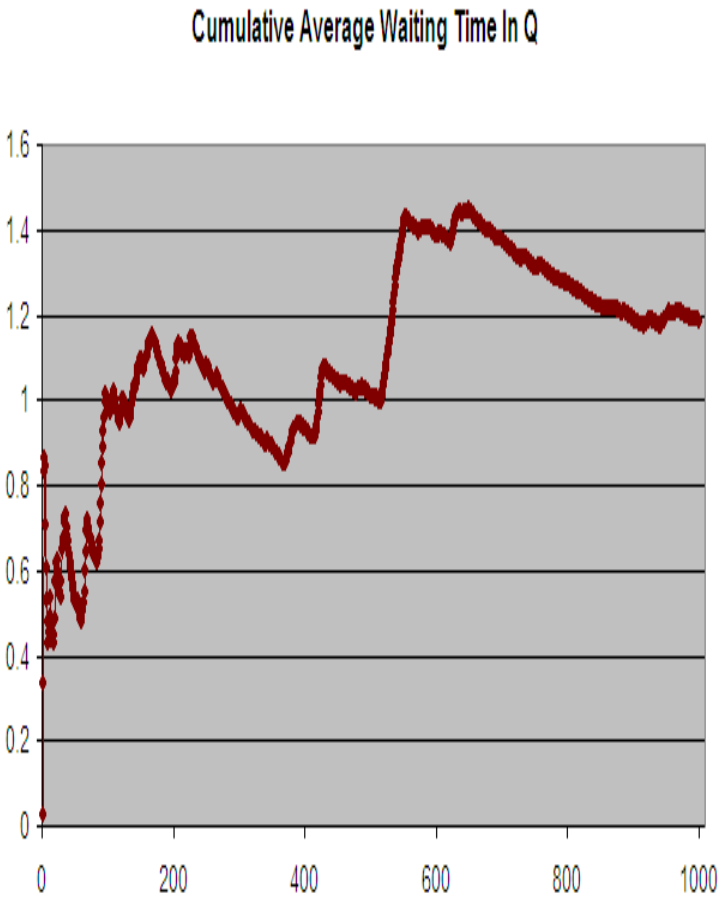


Figure 5.10.: Cumulative Average Waiting Time of 1000 Customers

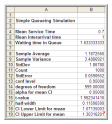


Figure 5.11.: Lindley Equation Results across 1000 Customers

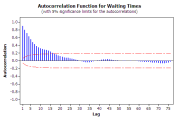


Figure 5.12.: Autocorrelation Plot for Waiting Times

From Figure 5.12, it is readily apparent that the data has strong positive correlation. The lag-1 correlation for this data is estimated to be about 0.9. Figure 5.12 clearly indicates the strong first order linear dependence between W_i and W_{i-1}. This positive dependence implies that if the previous customer waited a long time the next customer is likely to wait a long time. If the previous customer had a short wait, then the next customer is likely to have a short wait. This makes sense with respect to how a queue operates.

Strong positive correlation has serious implications when developing confidence intervals on the mean customer waiting time because the usual estimator for the sample variance:

$$S^2(n) = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2$$

is a biased estimator for the true population variance when there is correlation in the observations. This issue will be re-examined when ways to mitigate these problems are discussed.

The second issue that needs to be discussed is that of the non-stationary behavior of the data. Non-stationary data indicates some dependence on

time. More generally, non-stationary implies that the $W_1, W_2, W_3, \ldots, W_n$ are not obtained from identical distributions.

Why should the distribution of W_1 not be the same as the distribution of W_{1000}? The first customer is likely to enter the queue with no previous customers present and thus it is very likely that the first customer will experience little or no wait (the way W_0 was initialize in this example allows a chance of waiting for the first customer). However, the 1000^{th} customer may face an entirely different situation. Between the 1^{st} and the 1000^{th} customer there might likely be a line formed. In fact from the M/M/1 formula, it is known that the steady state expected number in the queue is 1.633. Clearly, the conditions that the 1^{st} customer faces are different than the 1000^{th} customer. Thus, the distributions of their waiting times are likely to be different.

This situation can be better understood by considering a model for the underlying data. A time series, $X_1, X_2, \ldots$, is said to be *covariance stationary* if:

- The mean exists and $\theta = E[X_i]$, for i = 1,2,..., n
- The variance exists and $Var[X_i] = \sigma^2 > 0$, for i = 1,2,..., n
- The lag-k autocorrelation, $\rho_k = cor(X_i, X_{i+k})$, is not a function of i, i.e. the correlation between any two points in the series does not depend upon where the points are in the series, it depends only upon the distance between them in the series.

In the case of the customer waiting times, we can conclude from the discussion that it is very likely that $\theta \neq E[X_i]$ and $Var[X_i] \neq \sigma^2$ for *each* i = 1,2,..., n for the time series.

Do you think that is it likely that the distributions of W_{9999} and W_{10000} will be similar? The argument, that the 9999^{th} customer is on average likely to experience similar conditions as the 10000^{th} customer, sure seems reasonable.

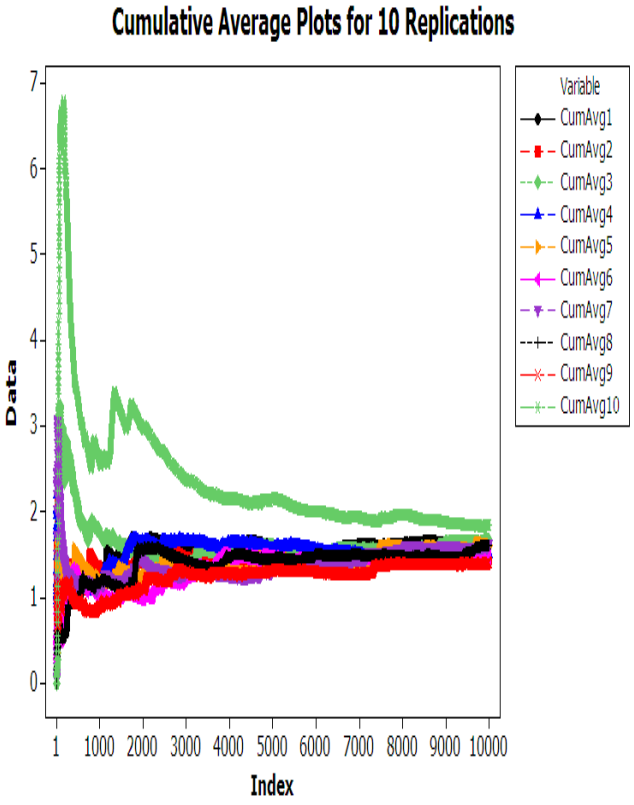


Figure 5.13.: Multiple Sample Paths of Queueing Simulation

Figure 5.13 shows 10 different replications of the cumulative average for a 10000 customer simulation. From the figure, we can see that the cumulative average plots can vary significantly over the 10000 customers with the average tracking above the true expected value, below the true expected value, and possibly towards the true expected value. For the case of 10000 customers, you should notice that the cumulative average starts to approach the expected value of the steady state mean waiting time in the queue with increasing number of customers. This is the law of large numbers in action. It appears that it takes a period of time for the performance measure to *warm up* towards the true mean. Determining the warm up time will be the basic way to mitigate the problem of this non-stationary behavior.

From this discussion, we can conclude that the second basic statistical assumption of identically distributed data is not valid for within replication data. From this, we can also conclude that it is very likely that the data is not normally distributed. In fact, for the M/M/1 it can be shown that the steady state distribution for the waiting time in the queue is not a normal distribution. Thus, all three of the basic statistical assumptions are violated for the within replication data of this example. This problem needs to be addressed in order to properly analyze infinite horizon simulations.

There are two basic methods for performing infinite horizon simulations. The first is to perform multiple replications. This approach addresses independence and normality in a similar fashion as the finite horizon case, but special procedures will be needed to address the non-stationary aspects of the data. The second basic approach is to work with one very long replication. Both of these methods depend on first addressing the problem of the non-stationary aspects of the data. The next section looks at ways to mitigate the non-stationary aspect of within-replication data for infinite horizon simulations.

5.6.1. Assessing the Effect of Initial Conditions

Consider the output stochastic process X_i of the simulation. Let $F_i(x|I)$ be the conditional cumulative distribution function of X_i where I represents the initial conditions used to start the simulation at time 0. If $F_i(x|I) \rightarrow F(x)$ when $i \rightarrow \infty$, for all initial conditions I, then $F(x)$ is called the steady state distribution of the output process. (Law 2007).

In infinite horizon simulations, estimating parameters of the steady state distribution, $F(x)$, such as the steady state mean, θ, is often the key objective. The fundamental difficulty associated with estimating steady state performance is that unless the system is initialized using the steady state distribution (which is not known), there is no way to directly observe the steady state distribution.

It is true that if the steady state distribution exists and you run the simulation long enough the estimators will tend to converge to the desired quantities. Thus, within the infinite horizon simulation context, you must decide on how long to run the simulations and how to handle the effect of the *initial conditions* on the estimates of performance. The initial conditions of a simulation represent the state of the system when the simulation is started. For example, in simulating the pharmacy system, the simulation was started with no customers in service or in the line. This is referred to as *empty and idle*. The initial conditions of the simulation affect the rate of convergence of estimators of steady state performance.

Because the distributions $F_i(x|I)$ at the start of the replication tend to depend more heavily upon the initial conditions, estimators of steady state performance such as the sample average, $\bar{X}$, will tend to be *biased*. A point estimator, $\widehat{\theta}$, is an *unbiased* estimator of the parameter of interest, θ, if $E[\widehat{\theta}] = \theta$. That is, if the expected value of the sampling distribution is equal to the parameter of interest then the estimator is said to be unbiased. If the estimator is biased then the difference, $E[\widehat{\theta}] - \theta$, is called the bias of, $\widehat{\theta}$, the estimator.

Note that any individual difference between the true parameter, θ, and a particular observation, X_i, is called error, $\epsilon_i = X_i - \theta$. If the expected value of the errors is not zero, then there is bias. A particular observation is not biased. Bias is a property of the estimator. Bias is analogous to being consistently off target when shooting at a bulls-eye. It is as if the sights on your gun are crooked. In order to estimate the bias of an estimator, you must have multiple observations of the estimator. Suppose that you are estimating the mean waiting time in the queue as per the previous example and that the estimator is based on the first 20 customers. That is, the estimator is:

$$\bar{W}_r = \frac{1}{20} \sum_{i=1}^{20} W_{ir}$$

and there are $r = 1, 2, ... 10$ replications. The following table shows the sample average waiting time for the first 20 customers for 10 different replications.

Table 5.5.: Ten Replications of 20 Customers

r	$\bar{W}_r$	$B_r = \bar{W}_r - W_q$
1	0.194114	-1.43922
2	0.514809	-1.11852
3	1.127332	-0.506
4	0.390004	-1.24333
5	1.05056	-0.58277
6	1.604883	-0.02845
7	0.445822	-1.18751
8	0.610001	-1.02333
9	0.52462	-1.10871
10	0.335311	-1.29802
	$\bar{\bar{W}} = 0.6797$	$\bar{B} = -0.9536$

In the table, B_r is an estimate of the bias for the r^{th} replication, where $W_q = 1.6\bar{3}$. Upon averaging across the replications, it can be seen that $\bar{B} = -0.9536$, which indicates that the estimator based only on the first 20 customers has significant negative bias, i.e. on average it is less than the target value.

This is the so called *initialization bias problem* in steady state simulation. Unless the initial conditions of the simulation can be generated according to $F(x)$, which is not known, you must focus on methods that detect and/or mitigate the presence of initialization bias.

One strategy for initialization bias mitigation is to find an index, d, for the output process, X_i, so that $X_i; i = d+1, ...$ will have substantially similar distributional properties as the steady state distribution $F(x)$. This is called the simulation warm up problem, where d is called the warm up point, and $i = 1, ..., d$ is called the warm up period for the simulation. Then, the estimators of steady state performance are based only on $X_i; i = d+1, ...$, which represent the data after deleting the warm up period.

For example, when estimating the steady state mean waiting time for each replication r the estimator would be:

$$\bar{W_r} = \frac{1}{n-d} \sum_{i=d+1}^{n} W_{ir}$$

For time-based performance measures, such as the average number in queue, a time T_w can be determined past which the data collection process can begin. Estimators of time-persistent performance such as the sample average are computed as:

$$\bar{Y_r} = \frac{1}{T_e - T_w} \int_{T_w}^{T_e} Y_r(t)dt$$

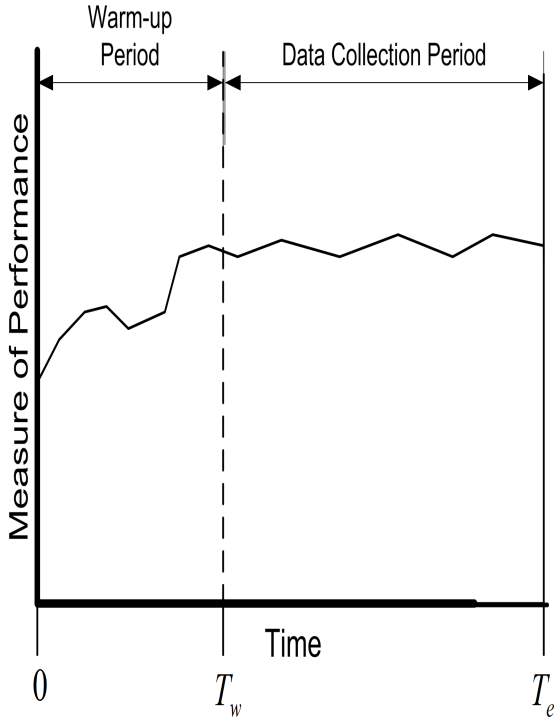


Figure 5.14.: The Concept of the Warm Up Period

Figure 5.14 shows the concept of a warm up period for a simulation replication. When you perform a simulation, you can easily specify a time-based warm up period using the `lengthOfReplicationWarmUp` property of the `Model` class. In fact, even for observation based data, it will be more convenient to specify the warm up period in terms of time. A given value of T_w implies a particular value of d and vice versa. Specifying a warm up period, causes an event to be scheduled for time T_w. At that time, all the accumulated statistical counters are cleared so that the net effect is that statistics are only collected over the period from T_w to T_e. The problem then becomes that of finding an appropriate warm up period.

Before proceeding with how to assess the length of the warm up period, the concept of steady state needs to be further examined. This subtle concept is often misunderstood or misrepresented. Often you will hear the phrase: *The system has reached steady state*. The correct interpretation of this phrase is that the distribution of the desired performance measure has reached a point where it is sufficiently similar to the desired steady state distribution. Steady state is a concept involving the performance measures generated by the system as time goes to infinity. However, sometimes this phrase is interpreted incorrectly to mean that the system *itself* has reached steady state. Let me state emphatically that the system *never* reaches steady state. If the system itself reached steady state, then by implication it would never change with respect to time. It should be clear that the system continues to evolve with respect to time; otherwise, it would be a very boring system! Thus, it is incorrect to indicate that the system has reached steady state. Because of this, do not to use the phrase: *The system has reached steady state*.

Understanding this subtle issue raises an interesting implication concerning the notion of deleting data to remove the initialization bias. Suppose that the state of the system at the end of the warm up period, T_w, is exactly the same as at $T = 0$. For example, it is certainly possible that at time T_w for a particular replication that the system was empty and idle. Since the state of the system at T_w is the same as that of the initial conditions, there will be no effect of deleting the warm up period for this replication. In fact

there will be a negative effect, in the sense that data will have been thrown away for no reason. Deletion methods are predicated on the likelihood that the state of the system seen at T_w is more representative of steady state conditions. At the end of the warm up period, the system can be in *any of the possible* states of the system. Some states will be more likely than others. If multiple replications are made, then at T_w each replication will experience a different set of conditions at T_w. Let $I^r_{T_w}$ be the initial conditions (state) at time T_w on replication r. By setting a warm up period and performing multiple replications, you are in essence sampling from the distribution governing the state of the system at time T_w. If T_w is long enough, then on average across the replications, you are more likely to start collecting data when the system is in states that are more representative over the long term (rather than just empty and idle).

Many methods and rules have been proposed to determine the warm up period. The interested reader is referred to (Wilson and Pritsker 1978), Lada et al. (2003), (Litton and Harmonosky 2002), White et al. (2000), Cash et al. (1992), and (Rossetti and Delaney 1995) for an overview of such methods. This discussion will concentrate on the visual method proposed in (Welch 1983).

The basic idea behind Welch's graphical procedure is simple:

- Make R replications. Typically, $R \geq 5$ is recommended.
- Let Y_{rj} be the j^{th} observation on replication r for $j = 1, 2, \cdots, m_r$ where m_r is the number of observations in the r^{th} replication, and $r = 1, 2, \cdots, n$,
- Compute the averages across the replications for each $j = 1, 2, \ldots, m$, where $m = min(m_r)$ for $r = 1, 2, \cdots, n$.

$$\bar{Y}_{\cdot j} = \frac{1}{n} \sum_{r=1}^{n} Y_{rj}$$

- Plot, $\bar{Y}_{\cdot j}$ for each $j = 1, 2, \ldots, m$

- Apply smoothing techniques to $\bar{Y}_{\cdot j}$ for $j = 1, 2, \ldots, m$
- Visually assess where the plot start to converge

Let's apply the Welch's procedure to the replications generated from the Lindley equation simulation. Using the 10 replications stored in a spreadsheet we can compute the average across each replication for each customer.

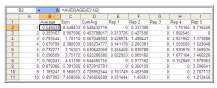


Figure 5.15.: Computing the Averages for the Welch Plot

In Figure 5.15, cell B2 represents the average across the 10 replications for the 1^{st} customer. Column D represents the cumulative average associated with column B.

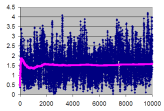


Figure 5.16.: Welch Plot with Superimposed Cumulative Average Line

Figure 5.16 is the plot of the cumulative average (column D) superimposed on the averages across replications (column B). The cumulative average is one method of smoothing the data. From the plot, you can infer that after about customer 3000 the cumulative average has started to converge. Thus, from this analysis you might infer that $d = 3000$.

When you perform an infinite horizon simulation by specifying a warm up period and making multiple replications, you are using the method of *replication-deletion.* If the method of replication-deletion with $d = 3000$ is used for the current example, a slight reduction in the bias can be achieved as indicated in the following table.

Table 5.6.: Replication-Deletion Results, d = 3000

r	$\bar{W}_r$ $(d = 0)$	$\bar{W}_r(d = 3000)$	$B_r(d = 0)$	$B_r(d = 3000)$
1	1.594843	1.592421	-0.03849	-0.04091
2	1.452237	1.447396	-0.1811	-0.18594
3	1.657355	1.768249	0.024022	0.134915
4	1.503747	1.443251	-0.12959	-0.19008
5	1.606765	1.731306	-0.02657	0.097973
6	1.464981	1.559769	-0.16835	-0.07356
7	1.621275	1.75917	-0.01206	0.125837
8	1.600563	1.67868	-0.03277	0.045347
9	1.400995	1.450852	-0.23234	-0.18248
10	1.833414	1.604855	0.20008	-0.02848
	$\bar{\bar{W}} = 1.573617$	$\bar{\bar{W}} = 1.603595$	$\bar{B} = -0.05972$	$\bar{B} = -0.02974$
	s = 0.1248	s = 0.1286	s = 0.1248	s = 0.1286
95% LL	1.4843	1.5116	-0.149023	-0.121704
95% UL	1.6629	1.6959	-0.029590	0.062228

While not definitive for this simple example, the results suggest that deleting the warm up period helps to reduce initialization bias. This model's warm up period will be further analyzed using additional tools available in the next section.

In performing the method of replication-deletion, there is a fundamental trade-off that occurs. Because data is deleted, the variability of the estimator will tend to increase while the bias will tend to decrease. This is a trade-off between a reduction in bias and an increase in variance. That is, accuracy is being traded off against precision when deleting the warm up period. In addition to this trade off, data from each replication is also being thrown away. This takes computational time that could be expended more effectively on collecting usable data. Another disadvantage of performing replication-deletion is that the techniques for assessing the warm up period (especially graphical) may require significant data storage. The Welch

plotting procedure requires the saving of data points for post processing after the simulation run. In addition, significant time by the analyst may be required to perform the technique and the technique is subjective.

When a simulation has many performance measures, you may have to perform a warm up period analysis for every performance measure. This is particularly important, since in general, the performance measures of the same model may converge towards steady state conditions at different rates. In this case, the length of the warm up period must be sufficiently long enough to cover all the performance measures. Finally, replication-deletion may simply compound the bias problem if the warm up period is insufficient relative to the length of the simulation. If you have not specified a long enough warm up period, you are potentially compounding the problem for n replications.

Despite all these disadvantages, replication-deletion is very much used in practice because of the simplicity of the analysis after the warm up period has been determined. Once you are satisfied that you have a good warm up period, the analysis of the results is the same as that of finite horizon simulations. Replication-deletion also facilitates the use of experimental design techniques that rely on replicating design points.

The next section illustrates how to perform the method of replication-deletion on this simple M/M/1 model.

5.6.2. Performing the Method of Replication-Deletion

The first step in performing the method of replication-deletion is to determine the length of the warm up period. This example illustrates how to:

- Save the values from observation and time-based data to files for post processing
- Make Welch plots based on the saved data

- Setup and run the multiple replications
- Interpret the results

When performing a warm-up period analysis, the first decision to make is the length of each replication. In general, there is very little guidance that can be offered other than to try different run lengths and check for the sensitivity of your results. Within the context of queueing simulations, the work by (Whitt 1989) offers some ideas on specifying the run length, but these results are difficult to translate to general simulations.

Since the purpose here is to determine the length of the warm up period, then the run length should be bigger than what you suspect the warm up period to be. In this analysis, it is better to be conservative. You should make the run length as long as possible given your time and data storage constraints. Banks et al. (2005) offer the rule of thumb that the run length should be at least 10 times the amount of data deleted. That is, $n \geq 10d$ or in terms of time, $T_e \geq 10T_w$. Of course, this is a "catch 22" situation because you need to specify n or equivalently T_e in order to assess T_w. Setting T_e very large is recommended when doing a preliminary assessment of T_w. Then, you can use the rule of thumb of 10 times the amount of data deleted when doing a more serious assessment of T_w (e.g. using Welch plots etc.)

A preliminary assessment of the current model has already been performed based on the previously described spreadsheet simulation. That assessment suggested a deletion point of at least $d = 3000$ customers. This can be used as a starting point in the current effort. Now, T_w needs to be determined based on d. The value of d represents the customer number for the end of the warm up period. To get T_w, you need to answer the question: How long (on average) will it take for the simulation to generate d observations. In this model, the mean number of arrivals is 1 customer per minute. Thus, the initial T_w is

$$3000 \text{ customers} \times \frac{\text{minute}}{1 \text{ customer}} = 3000 \text{ minutes}$$

and therefore the initial T_e should be 30,000 minutes. That is, you should specify 30,000 minutes for the replication length.

5.6.2.1. Determining the Warm Up Period

To perform a more rigorous analysis of the warm up period, we need to run the simulation for multiple replications and capture the data necessary to produce Welch plots for the performance measures of interest. Within the KSL, this can be accomplished by using the classes within the `ksl.observers.welch` package. There are two classes of note:

`WelchDataFileCollector` This class captures data associated with a Response within the simulation model. Every observation from every replication is written to a binary data file. In addition, a text based data file is created that contains the meta data associated with the data collection process. The meta data file records the number of replications, the number of observations for each replication, and the time between observations for each replication.

`WelchDataFileAnalyzer` This class uses the files produced by the `WelchDataFileCollector` to produces the Welch data. That is, the average across each row of observations and the cumulative average over the observations. This class produces files from which the plots can be made.

A warm up period analysis is associated with a particular response. The goal is to determine the number of observations of the response to delete at the beginning of the simulation. We can collect the relevant data by attaching an observer to the response variable.

Example 5.4 (Welch Plot Analysis)**.** This code illustrates how to capture Welch plot data and to show the Welch plot within a browser window.

```
fun main(){
    val model = Model("Drive Through Pharmacy")
    // add DriveThroughPharmacy to the main model
    val dtp = DriveThroughPharmacyWithQ(model, 1)
    dtp.arrivalRV.initialRandomSource = ExponentialRV(1.0, 1)
    dtp.serviceRV.initialRandomSource = ExponentialRV(0.7, 2)

    val rvWelch = WelchFileObserver(dtp.systemTime, 1.0)
    val twWelch = WelchFileObserver(dtp.numInSystem, 10.0)
    model.numberOfReplications = 5
    model.lengthOfReplication = 50000.0
    model.simulate()
    model.print()
    println(rvWelch)
    println(twWelch)

    val rvFileAnalyzer =
     ↪  rvWelch.createWelchDataFileAnalyzer()
    val twFileAnalyzer =
     ↪  twWelch.createWelchDataFileAnalyzer()

    rvFileAnalyzer.createCSVWelchPlotDataFile()
    twFileAnalyzer.createCSVWelchPlotDataFile()

    val wp = WelchPlot(analyzer = rvFileAnalyzer)
    wp.defaultPlotDir = model.outputDirectory.plotDir
    wp.showInBrowser()
    wp.saveToFile("SystemTimeWelchPlot")

}
```

In the code, we create two instances of `WelchFileObserver` one for the system time (`systemTime`) and one for the number in the system (`numInSyste`) both referenced from the exposed properties of the drive through pharmacy class. Since the number in the system is a time-persistent variable, it is discretized based on intervals of 10 time units. The concept of discretizing the time-persistent data is illustrated in Figure 5.18. The `WelchFileObserver` instances are used to create `WelchDataFileAnalyzer` instances, which are used to make the data for plotting. You can use any plotting program that you want to display the plot.

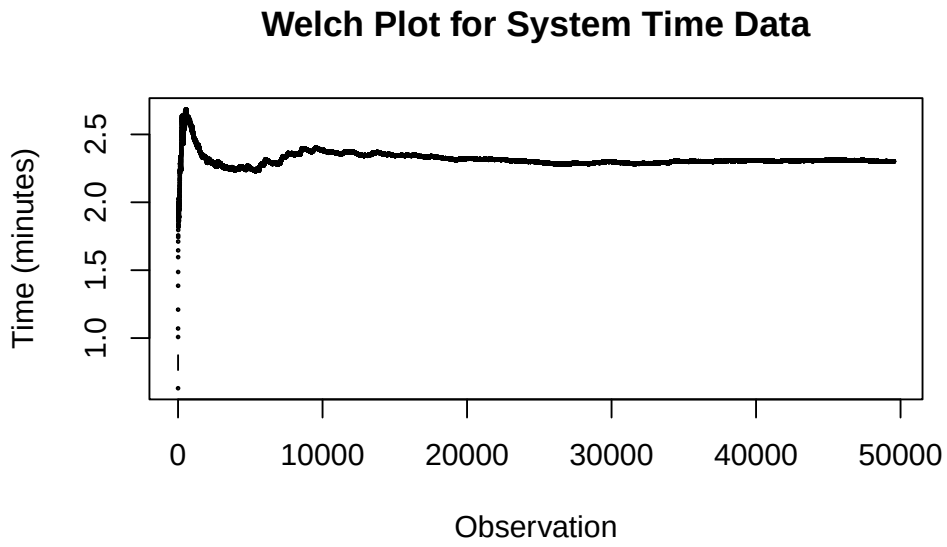


Figure 5.17.: Welch Plot for System Time Analysis

Figure 5.17 illustrates the plot of the data. From this plot, we can estimate the number of observations to delete as approximately 20000.

Time-persistent observations are saved within a file from within the model such that the time of the observation and the value of the state variable at the time of change are recorded. Thus, the observations are not equally spaced in time. In order to perform the Welch plot analysis, we need to

cut the data into discrete equally spaced intervals of time.

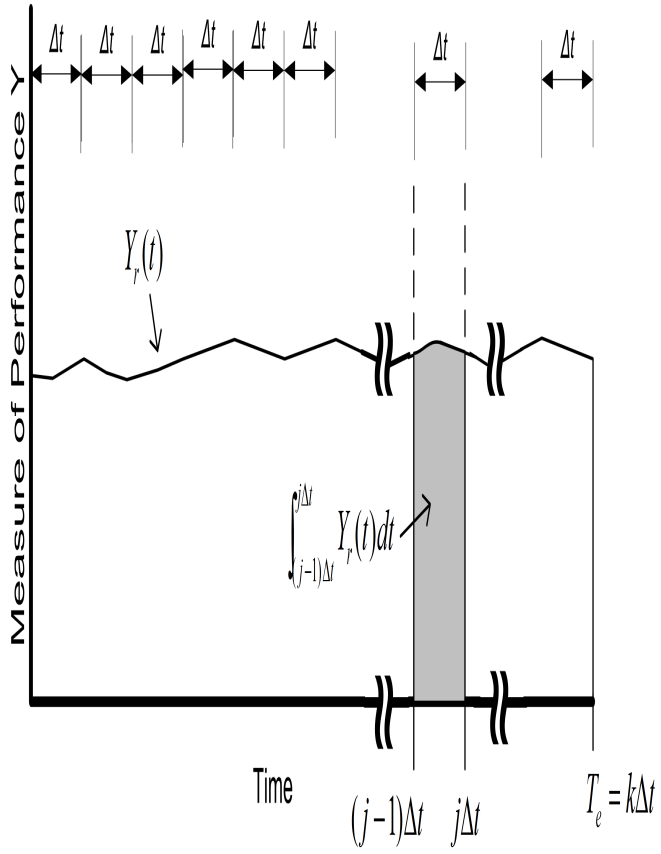


Figure 5.18.: Discretizing Time-Persistent Data

Figure 5.18 illustrates the concept of discretizing time-persistent data. Suppose that you divide T_e into k intervals of size Δt, so that $T_e = k \times \Delta t$. The time average over the j^{th} interval is given by:

$$\bar{Y}_{rj} = \frac{1}{\Delta t} \int_{j-1)\Delta t}^{j\Delta t} Y_r(t)dt$$

Thus, the overall time average can be computed from the time average associated with each interval:

$$\begin{aligned} \bar{Y}_r &= \frac{\int_0^{T_e} Y_r(t)dt}{T_e} = \frac{\int_0^{T_e} Y_r(t)dt}{k\Delta t} \\ &= \frac{\sum_{j=1}^{k} \int_{(j-1)\Delta t}^{j\Delta t} Y_r(t)dt}{k\Delta t} = \frac{\sum_{j=1}^{k} \bar{Y}_{rj}}{k} \end{aligned}$$

Each of the $\bar{Y}_{rj}$ are computed over intervals of time that are equally spaced and can be treated as if they are observation (tally) based data.

The computation of the $\bar{Y}_{rj}$ for time-persistent data can be achieved by using the `WelchDataFileCollector` class and specifying a discretization interval. Since the number in queue data is time-persistent, time based batches are selected, and the batch size is specified in terms of time. In this case, the data is being batched based on a time interval of 10 minutes. This produces a file which contains the $\bar{Y}_{rj}$ as observations. This file can then be analyzed as previously illustrated.

The resulting plot is show in Figure 5.19. This plot is sparser than the previous plot because each observation represents the average of 10 minutes. There are 5000 observations. From the plot, we can conclude that after observation 2000, we see a steady convergence. The 2000th observation represents 20000 time units (minutes) because every observation represents 10 time units.

Once you have performed the warm up analysis, you still need to use your simulation model to estimate system performance. Because the process of analyzing the warm up period involves saving the data you could use the already saved data to estimate your system performance after truncating the

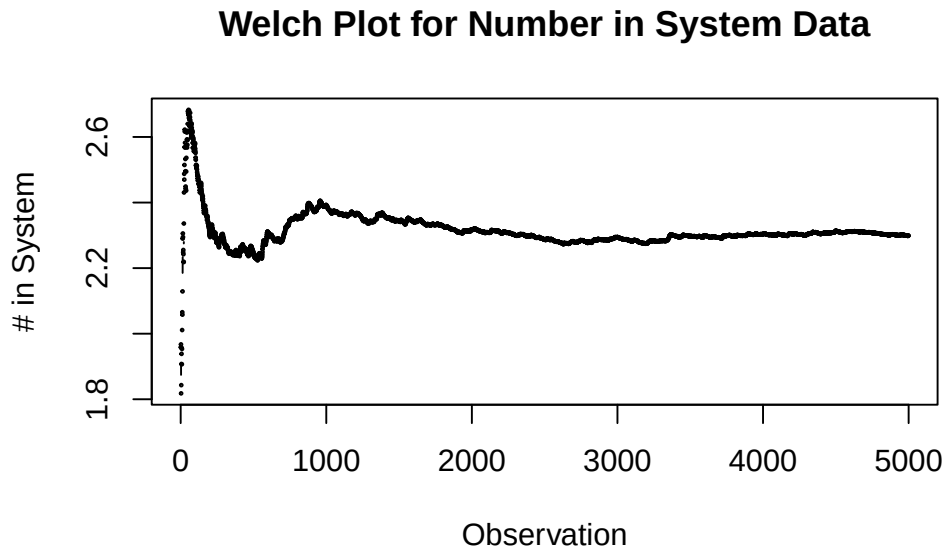


Figure 5.19.: Welch Plot of Time-Persistent Number in System Data

initial portion of the data from the data sets. If re-running the simulation is relatively inexpensive, then you can simply set the warm up period via the `Model` class and re-run the model. Following the rule of thumb that the length of the run should be at least 10 times the warm up period, the simulation was re-run with the settings (30 replications, 20000 minute warm up period, 200,000 minute replication length). The results shown in the following table indicate that there does not appear to be any significant bias with these replication settings.

Table 5.7.: Across Replication Statistics for Drive Through Pharmacy

Name	Count	Average	Half-Width
NumBusy	30.0	.7001	.0008
# in System	30.0	2.3402	.0103
System Time	30.0	2.3389	.01

Name	Count	Average	Half-Width
PharmacyQ:NumInQ	30.0	1.6401	.0098
PharmacyQ:TimeInQ	30.0	1.6392	.0097
SysTime > 4.0 minutes	30.0	.1806	.0013
Num Served	30.0	180104.9667	145.7244

The true waiting time in the queue is $1.6\bar{3}3$ and it is clear that the 95% confidence interval contains this value.

The process described here for determining the warm up period for steady state simulation is tedious and time consuming. Research into automating this process is an active area of investigation. The work by (Robinson 2005) and Rossetti and Li (2005) holds some promise in this regard; however, there remains the need to integrate these methods into computer simulation software. Even though determining the warm up period is tedious, some consideration of the warm up period should be done for infinite horizon simulations.

Once the warm up period has been found, you can set the warm up period using the `Model` class. Then, you can use the method of replication-deletion to perform your simulation experiments. Thus, all the discussion previously presented on the analysis of finite horizon simulations can be applied.

When determining the number of replications, you can apply the fixed sample size procedure after performing a pilot run. If the analysis indicates that you need to make more runs to meet your confidence interval half-width you have two alternatives: 1) increase the number of replications or 2) keep the same number of replications but increase the length of each replication. If n_0 was the initial number of replications and n is the number of replications recommended by the sample size determination procedure, then you can instead set T_e equal to $(n/n_0)T_e$ and run n_0 replications. Thus, you will still have approximately the same amount of data collected over your replications, but the longer run length may reduce the effect of initialization bias.

As previously mentioned, the method of replication-deletion causes each replication to delete the initial portion of the run. As an alternative, you can make one long run and delete the initial portion only once. When analyzing an infinite horizon simulation based on one long replication, a method is needed to address the correlation present in the within replication data. The method of batch means is often used in this case and has been automated in within the KSL. The next section discusses the statistical basis for the batch means method and addresses some of the practical issues of using it within the KSL.

5.6.3. The Method of Batch Means

In the batch means method, only one simulation run is executed. After deleting the warm up period, the remainder of the run is divided into k batches, with each batch average representing a single observation.

Figure 5.20 illustrates the concept of batching observations. The advantages of the batch means method are that it entails a long simulation run, thus dampening the effect of the initial conditions. The disadvantage is that the within replication data is correlated and unless properly formed the batches may also exhibit a strong degree of correlation.

The following presentation assumes that a warm up analysis has already been performed and that the data that has been collected occurs after the warm up period. For simplicity, the presentation assumes observation based data. The discussion also applies to time-based data that has been cut into discrete equally spaced intervals of time as previously described. Therefore, assume that a series of observations, $(X_1, X_2, X_3, \ldots, X_n)$, is available from within the one long replication after the warm up period. As shown earlier, the within replication data can be highly correlated. In that section, it was mentioned that standard confidence intervals based on

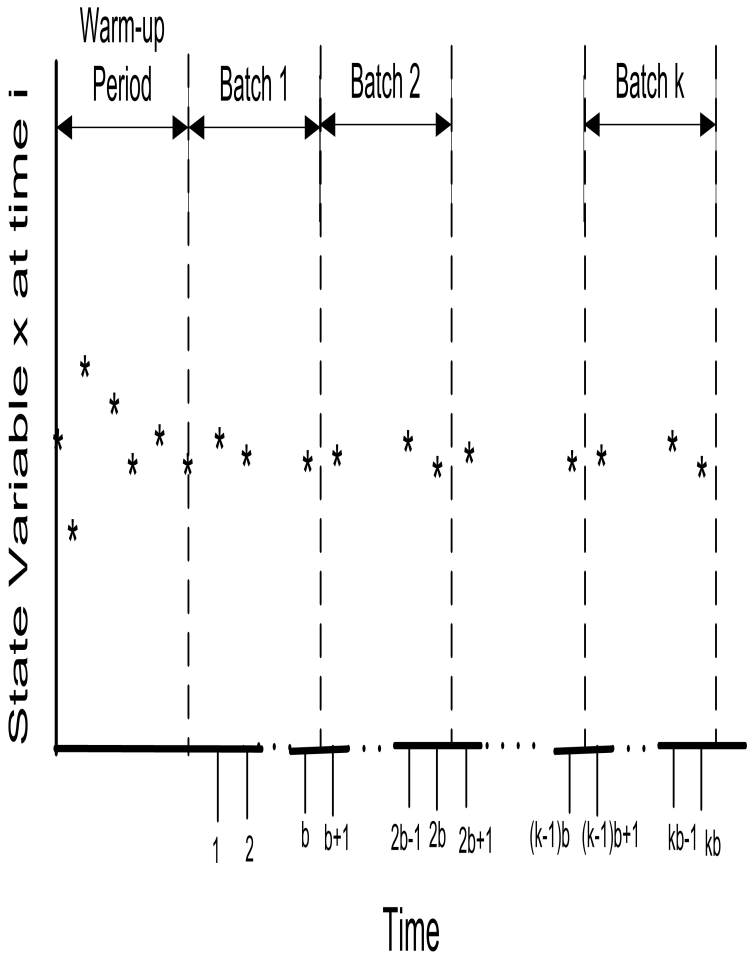


Figure 5.20.: Illustration of the Batch Means Method

$$S^2(n) = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2$$

are not appropriate for this type of data. Suppose you were to ignore the correlation, what would be the harm? In essence, a confidence interval implies a certain level of confidence in the decisions based on the confidence interval. When you use $S^2(n)$ as defined above, you will not achieve the desired level of confidence because $S^2(n)$ is a biased estimator for the variance of $\bar{X}$ when the data is correlated. Under the assumption that the data is covariance stationary, an assessment of the harm in ignoring the correlation can be made. For a series that is covariance stationary, one can show that

$$Var(\bar{X}) = \frac{\gamma_0}{n}\left[1 + 2\sum_{k=1}^{n-1}\left(1 - \frac{k}{n}\right)\rho_k\right]$$

where $\gamma_0 = Var(X_i)$, $\gamma_k = Cov(X_i, X_{i+k})$, and $\rho_k = \gamma_k/\gamma_0$ for $k = 1, 2, \ldots, n-1$.

When the data is correlated, S^2/n is a biased estimator of $Var(\bar{X})$. To show this, you need to compute the expected value of S^2/n as follows:

$$E\left[S^2/n\right] = \frac{\gamma_0}{n}\left[1 - \frac{2R}{n-1}\right]$$

where

$$R = \sum_{k=1}^{n-1}(1 - \frac{k}{n})\rho_k$$

Bias is defined as the difference between the expected value of the estimator and the quantity being estimated. In this case, the bias can be computed with some algebra as:

$$\text{Bias} = E\left[S^2/n\right] - Var(\bar{Y}) = \frac{-2\gamma_0 R}{n-1}$$

Since $\gamma_0 > 0$ and $n > 1$ the sign of the bias depends on the quantity R and thus on the correlation. There are three cases to consider: zero correlation, negative correlation, and positive correlation. Since $-1 \leq \rho_k \leq 1$, examining the limiting values for the correlation will determine the range of the bias.

For positive correlation, $0 \leq \rho_k \leq 1$, the bias will be negative, $(-\gamma_0 \leq Bias \leq 0)$. Thus, the bias is negative if the correlation is positive, and the bias is positive if the correlation is negative. In the case of positive correlation, S^2/n underestimates the $Var(\bar{X})$. Thus, using S^2/n to form confidence intervals will make the confidence intervals too short. You will have unjustified confidence in the point estimate in this case. The true confidence will not be the desired $1-\alpha$. Decisions based on positively correlated data will have a higher than planned risk of making an error based on the confidence interval.

One can easily show that for negative correlation, $-1 \leq \rho_k \leq 0$, the bias will be positive $(0 \leq Bias \leq \gamma_0)$. In the case of negatively correlated data, S^2/n over estimates the $Var(\bar{X})$. A confidence interval based on S^2/n will be too wide and the true quality of the estimate will be better than indicated. The true confidence coefficient will not be the desired $1-\alpha$; it will be greater than $1-\alpha$.

Of the two cases, the positively correlated case is the more severe in terms of its effect on the decision making process; however, both are problems. Thus, the naive use of S^2/n for dependent data is highly unwarranted. If you want to build confidence intervals on $\bar{X}$ you need to find an unbiased estimator of the $Var(\bar{X})$.

The method of batch means provides a way to develop (at least approximately) an unbiased estimator for $Var(\bar{X})$. Assuming that you have a

series of data point, the method of batch means method divides the data into subsequences of contiguous batches:

$$\underbrace{X_1, X_2, \ldots, X_b}_{batch1} \cdots \underbrace{X_{b+1}, X_{b+2}, \ldots, X_{2b}}_{batch2} \cdots$$
$$\underbrace{X_{(j-1)b+1}, X_{(j-1)b+2}, \ldots, X_{jb}}_{batchj} \cdots \underbrace{X_{(k-1)b+1}, X_{(k-1)b+2}, \ldots, X_{kb}}_{batchk}$$

and computes the sample average of the batches. Let k be the number of batches each consisting of b observations, so that $k = \lfloor n/b \rfloor$. If b is not a divisor of n then the last $(n - kb)$ data points will not be used. Define $\bar{X}_j(b)$ as the j^{th} batch mean for $j = 1, 2, \ldots, k$, where,

$$\bar{X}_j(b) = \frac{1}{b} \sum_{i=1}^{b} X_{(j-1)b+i}$$

Each of the batch means are treated like observations in the batch means series. For example, if the batch means are re-labeled as $Y_j = \bar{X}_j(b)$, the batching process simply produces another series of data, $(Y_1, Y_2, Y_3, \ldots, Y_k)$ which may be more like a random sample. To form a $(1 - \alpha)\%$ confidence interval, you simply treat this new series like a random sample and compute approximate confidence intervals using the sample average and sample variance of the batch means series:

$$\bar{Y}(k) = \frac{1}{k} \sum_{j=1}^{k} Y_j$$

$$S_b^2(k) = \frac{1}{k-1} \sum_{j=1}^{k} (Y_j - \bar{Y}^2)$$

$$\bar{Y}(k) \pm t_{\alpha/2,k-1} \frac{S_b(k)}{\sqrt{k}}$$

Since the original X's are covariance stationary, it follows that the resulting batch means are also covariance stationary. One can show, see (Alexopoulos and Seila 1998), that the correlation in the batch means reduces as both the size of the batches, b and the number of data points, n increases. In addition, one can show that $S_b^2(k)/k$ approximates $\text{Var}(\bar{X})$ with error that reduces as both b and n increase towards infinity.

The basic difficulty with the batch means method is determining the batch size or alternatively the number of batches. Larger batch sizes are good for independence but reduce the number of batches, resulting in higher variance for the estimator. (Schmeiser 1982) performed an analysis that suggests that there is little benefit if the number of batches is larger than 30 and recommended that the number of batches remain in the range from 10 to 30. However, when trying to assess whether the batches are independent, it is better to have a large number of batches (> 100) so that tests on the lag-k correlation have better statistical properties.

There are a variety of procedures that have been developed that will automatically batch the data as it is collected, see for example (Fishman and Yarberry 1997), (Steiger and Wilson 2002), and Banks et al. (2005). has its own batching algorithm. The batching algorithm is described in Kelton et al. (2004) page 311. See also (Fishman 2001) page 254 for an analysis of the effectiveness of the algorithm.

The discussion here is based on the description in Kelton et al. (2004). When the algorithm has recorded a sufficient amount of data, it begins by forming k = 20 batches. As more data is collected, additional batches are formed until k = 40 batches are collected. When 40 batches are formed, the algorithm collapses the number of batches back to 20, by averaging each pair of batches. This has the net effect of doubling the batch size. This process is repeated as more data is collected, thereby ensuring that the number of batches is between 20 and 39.

For time-persistent data, the approach requires that the data be discretized as previously discussed in the section on warm up period analysis. Then, the same batch method is applied to ensure between 20 and 39 batches.

In addition, the process also computes the lag-1 correlation so that a test can be performed to check if the correlation is significant by testing the hypothesis that the batch means are uncorrelated using the following test statistic, see (Alexopoulos and Seila 1998):

$$C = \sqrt{\frac{k^2-1}{k-2}}\left[\hat{\rho}_1 + \frac{[Y_1-\bar{Y}]^2 + [Y_k-\bar{Y}]^2}{2\sum_{j=1}^{k}(Y_j-\bar{Y})^2}\right]$$

$$\hat{\rho}_1 = \frac{\sum_{j=1}^{k-1}(Y_j-\bar{Y})(Y_{j+1}-\bar{Y})}{\sum_{j=1}^{k}(Y_j-\bar{Y})^2}$$

The hypothesis is rejected if $C > z_\alpha$ for a given confidence level α. If the batch means do not pass the test, then increasing the replication run length should permit more data to be collected and increase the size of the batches. This may permit the correlation test to pass.

5.6.4. Performing the Method of Batch Means

Performing the method of batch means in the KSL is relatively straight forward. The following assumes that a warm up period analysis has already been performed. Batches are formed during the simulation run and the confidence intervals are based on the batches. In this situation, the primary concern will be to determine the run length that will ensure a desired half-width on the confidence intervals. Both fixed sampling and sequential sampling methods can be applied.

The following code present the process to set up batching within the KSL for the Drive Through Pharmacy Model. The key class is the `StatisticalBatchingElement` class, which must be added to the `Model` (see line 9) prior to running the simulation. The `StatisticalBatchingElement` has one key parameter which represents the interval used to discretize the time weighted variables. If no interval is

supplied, then the algorithm ensures that the initial number of batches collected before applying the previously described batching algorithm is 512.

Example 5.5 (Performing a Batch Means Analysis)**.** This code illustrates how to perform a single run, batch means analysis, by adding a batching element to the model.

```
fun main(){
    val model = Model("Drive Through Pharmacy")
    // add DriveThroughPharmacy to the main model
    val dtp = DriveThroughPharmacyWithQ(model, 1)
    dtp.arrivalRV.initialRandomSource = ExponentialRV(1.0, 1)
    dtp.serviceRV.initialRandomSource = ExponentialRV(0.7, 2)
    model.numberOfReplications = 1
    model.lengthOfReplication = 1000000.0
    model.lengthOfReplicationWarmUp = 100000.0
    val batchingElement = model.statisticalBatching()
    model.simulate()
    val sr = batchingElement.statisticReporter
    println(sr.halfWidthSummaryReport())
}
```

The analysis performed to determine the warm up period should give you some information concerning how long to make this single run and how long to set it's warm up period. Assume that a warm up analysis has been performed using n_0 replications of length T_e and that the analysis has indicated a warm up period of length T_w. Then, we can use this information is setting up the run length and warm up period for the single replication experiment.

As previously discussed, the method of replication deletion spreads the risk of choosing a bad initial condition across multiple replications. The method of batch means relies on only one replication. If you were satisfied with

the warm up period analysis based on n_0 replications and you were going to perform replication deletion, then you are willing to throw away the observations contained in at least $n_0 \times T_w$ time units and you are willing to use the data collected over $n_0 \times (T_e - T_w)$ time units. Therefore, the warm up period for the single replication can be set at $n_0 \times T_w$ and the run length can be set at $n_0 \times T_e$ time units. For example, suppose your warm up analysis was based on the initial results of $n_0 = 10$, $T_e = 30000$, $T_w = 10000$. Thus, your starting run length would be $n_0 \times T_e = 10 \times 30,000 = 300,000$ and the warm-up period will be $n_0 \times T_w = 100,000$. The following table show the results based on replication deletion

Table 5.8.: Replication-Deletion Half-Width Summary report $n = 10$, $T_e = 30000$, $T_w = 10000$

Response Name	n	$\bar{x}$	hw
PharmacyQ:Num In Q	10	1.656893	0.084475
PharmacyQ:Time In Q	10	1.659100	0.082517
NumBusy	10	0.699705	0.004851
Num in System	10	2.356598	0.088234
System Time	10	2.359899	0.085361

This table shows the results based on batching one long replication.

Table 5.9.: Batch Means Summary Report $T_e = 300000$, $T_w = 100000$

Response Name	n	$\bar{x}$	hw
PharmacyQ:Num In Q	32	1.631132	0.047416
PharmacyQ:Time In Q	24	1.627059	0.055888
NumBusy	32	0.699278	0.004185
Num in System	32	2.330410	0.049853
System Time	24	2.324590	0.057446

Suppose now you want to ensure that the half-widths from a single replication are less than a given error bound. The half-widths reported by the simulation for a single replication are based on the *batch means*. You can get an approximate idea of how much to increase the length of the replication by using the half-width sample size determination formula.

$$n \cong n_0 \left(\frac{h_0}{h} \right)^2$$

In this case, you interpret n and n_0 as the number of batches. From previous results, there were 32 batches for the time-weighted variables. Based on $T_e = 300000$ and $T_w = 100000$ there was a total of $T_e - T_w = 200000$ time units of observed data. This means that each batch represents $200,000 \div 32 = 6250$ time units. Using this information in the half-width based sample size formula with $n_0 = 32$, $h_0 = 0.049$, and $h = 0.02$, for the number in the system, yields:

$$n \cong n_0 \frac{h_0^2}{h^2} = 32 \times \frac{(0.049)^2}{(0.02)^2} = 192 \text{ batches}$$

Since each batch in the run had 6250 time units, this yields the need for 1,200,000 time units of observations. Because of the warm up period, you therefore need to set T_e equal to (1,200,000 + 100,000 = 1,300,000). Rerunning the simulation yields the results shown in the following table. The results show that the half-width meets the desired criteria. This approach is approximate since you do not know how the observations will be batched when making the final run.

Table 5.10.: Batch Means Summary Report $T_e = 1,300,000$, $T_w = 100000$

Response Name	n	$\bar{x}$	hw
NumBusy	32	0.698973	0.001652
Num in System	32	2.322050	0.019413

Response Name	n	$\bar{x}$	hw
PharmacyQ:Num In Q	32	1.623077	0.018586
PharmacyQ:Time In Q	36	1.625395	0.017383
System Time	36	2.324915	0.017815

Rather than trying to fix the amount of sampling, you might instead try to use a sequential sampling technique that is based on the half-width computed during the simulation run. In order to do this, we need to create a specific observer that can stop the simulation when the half-width criteria is met for the batch means. Currently, the KSL does not facilitate this approach.

Once the warm up period has been analyzed, performing infinite horizon simulations using the batch means method is relatively straight forward. A disadvantage of the batch means method is that it will be more difficult to use classical experimental design methods. If you are faced with an infinite horizon simulation, then you can use either the replication-deletion approach or the batch means method readily within the KSL. In either case, you should investigate if there may be any problems related to initialization bias. If you use the replication-deletion approach, you should play it safe when specifying the warm up period. Making the warm up period longer than you think it should be is better than replicating a poor choice. When performing an infinite horizon simulation based on one long run, you should make sure that your run length is long enough. A long run length can help to "wash out" the effects of initial condition bias.

Ideally, in the situation where you have to make many simulation experiments using different parameter settings of the same model, you should perform a warm up analysis for **each design configuration**. In practice, this is not readily feasible when there are a large number of experiments. In this situation, you should use your common sense to pick the design configurations (and performance measures) that you feel will most likely suffer from initialization bias. If you can determine long enough warm

up periods for these configurations, the other configurations should be relatively safe from the problem by using the longest warm up period found.

There are a number of other techniques that have been developed for the analysis of infinite horizon simulations including the standardized time series method, the regenerative method, and spectral methods. An overview of these methods and others can be found in (Alexopoulos and Seila 1998) and in (Law 2007).

So far you have learned how to analyze the data from one design configuration. A key use of simulation is to be able to compare alternative system configurations and to assist in choosing which configurations are best according to the decision criteria. The next section discusses how to compare different system configurations.

5.7. Comparing System Configurations

The previous sections have concentrated on estimating the performance of a system through the execution of a single simulation model. The running of the model requires the specification of the input variables (e.g. mean time between arrivals, service distribution, etc.) and the structure of the model (e.g. FIFO queue, process flow, etc.) The specification of a set of inputs (variables and/or structure) represents a particular system configuration, which is then simulated to estimate performance. To be able to simulate design configurations, you may have to build different models or you may be able to use the same model supplied with different values of the program inputs. In either situation, you now have different design configurations that can be compared. This allows the performance of the system to be estimated under a wide-variety of controlled conditions. It is this ability to easily perform these what-if simulations that make simulation such a useful analysis tool. Figure 5.21 illustrates two design configurations.

Each configuration supplies its own inputs (X^1 or X^2) to the model and produces its own outputs (Y^1 or Y^2), which can then be compared.

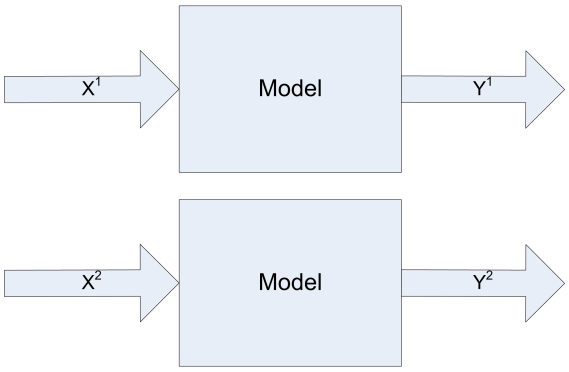


Figure 5.21.: Model Inputs and Outputs for Two Design Configurations

Naturally, when you have different design configurations, you would like to know which configurations are better than the others. Since the simulations are driven by random variables, the outputs from each configuration (e.g. Y^1, Y^2) are also random variables. The estimate of the performance of each system must be analyzed using statistical methods to ensure that the differences in performance are not simply due to sampling error. In other words, you want to be confident that one system is statistically better (or worse) than the other system.

5.7.1. Comparing Two Systems

The techniques for comparing two systems via simulation are essentially the same as that found in books that cover the statistical analysis of two samples (e.g. (Montgomery and Runger 2006)). This section begins with a review of these methods. Assume that samples from two different populations (system configurations) are available:

$X_{11}, X_{12}, \ldots, X_{1n_1}$ a sample of size n_1 from system configuration 1

$X_{21}, X_{22}, \ldots, X_{2n_2}$ a sample of size n_2 from system configuration 2

The samples represent a performance measure of the system that will be used in a decision regarding which system configuration is preferred. For example, the performance measure may be the average system throughput per day, and you want to pick the design configuration that has highest throughput.

Assume that each system configuration has an unknown population mean for the performance measure of interest, $E[X_1] = \theta_1$ and $E[X_2] = \theta_2$. Thus, the problem is to determine, with some statistical confidence, whether $\theta_1 < \theta_2$ or alternatively $\theta_1 > \theta_2$. Since the system configurations are different, an analysis of the situation of whether $\theta_1 = \theta_2$ is of less relevance in this context.

Define $\theta = \theta_1 - \theta_2$ as the mean difference in performance between the two systems. Clearly, if you can determine whether $\theta > 0$ or $\theta < 0$ you can determine whether $\theta_1 < \theta_2$ or $\theta_1 > \theta_2$. Thus, it is sufficient to concentrate on the difference in performance between the two systems.

Given samples from two different populations, there are a number of ways in which the analysis can proceed based on different assumptions concerning the samples. The first common assumption is that the observations within each sample for each configuration form a random sample. That is, the samples represent independent and identically distributed random variables. Within the context of simulation, this can be easily achieved for a given system configuration by performing replications. For example, this means that $X_{11}, X_{12}, \ldots, X_{1n_1}$ are the observations from n_1 replications of the first system configuration. A second common assumption is that both

populations are normally distributed or that the central limit theorem can be used so that sample averages are at least approximately normal.

To proceed with further analysis, assumptions concerning the population variances must be made. Many statistics textbooks present results for the case of the population variance being known. In general, this is not the case within simulation contexts, so the assumption here will be that the variances associated with the two populations are unknown. Textbooks also present cases where it is assumed that the population variances are equal. Rather than making that assumption it is better to test a hypothesis regarding equality of population variances.

The last assumption concerns whether or not the two samples can be considered independent of each other. This last assumption is very important within the context of simulation. Unless you take specific actions to ensure that the samples will be independent, they will, in fact, be dependent because of how simulations use (re-use) the same random number streams. The possible dependence between the two samples is not necessarily a bad thing. In fact, under certain circumstance it can be a good thing.

The following sections first presents the methods for analyzing the case of unknown variance with independent samples. Then, we focus on the case of dependence between the samples. Finally, how to use the KSL to do the work of the analysis will be illustrated.

5.7.1.1. Analyzing Two Independent Samples

Although the variances are unknown, the unknown variances are either equal or not equal. In the situation where the variances are equal, the observations can be pooled when developing an estimate for the variance. In fact, rather than just assuming equal or not equal variances, you can (and should) use an F-test to test for the equality of variance. The F-test can be found in most elementary probability and statistics books (see (Montgomery and Runger 2006)).

The decision regarding whether $\theta_1 < \theta_2$ can be addressed by forming confidence intervals on $\theta = \theta_1 - \theta_2$. Let $\bar{X}_1$, $\bar{X}_2$, S_1^2, and S_2^2 be the sample averages and sample variances based on the two samples (k = 1,2):

$$\bar{X}_k = \frac{1}{n_k} \sum_{j=1}^{n_k} X_{kj}$$

$$S_k^2 = \frac{1}{n_k - 1} \sum_{j=1}^{n_k} (X_{kj} - \bar{X}_k)^2$$

An estimate of $\theta = \theta_1 - \theta_2$ is desired. This can be achieved by estimating the difference with $\hat{D} = \bar{X}_1 - \bar{X}_2$. To form confidence intervals on $\hat{D} = \bar{X}_1 - \bar{X}_2$ an estimator for the variance of $\hat{D} = \bar{X}_1 - \bar{X}_2$ is required. Because the samples are independent, the computation of the variance of the difference is:

$$Var(\hat{D}) = Var(\bar{X}_1 - \bar{X}_2) = \frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}$$

where σ_1^2 and σ_2^2 are the unknown population variances. Under the assumption of equal variance, $\sigma_1^2 = \sigma_2^2 = \sigma^2$, this can be written as:

$$Var(\hat{D}) = Var(\bar{X}_1 - \bar{X}_2) = \frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2} = \sigma^2 (\frac{1}{n_1} + \frac{1}{n_2})$$

where σ^2 is the common unknown variance. A pooled estimator of σ^2 can be defined as:

$$S_p^2 = \frac{(n_1 - 1)S_1^2 + (n_2 - 1)S_2^2}{n_1 + n_2 - 2}$$

Thus, a $(1 - \alpha)\%$ confidence interval on $\theta = \theta_1 - \theta_2$ is:

$$\hat{D} \pm t_{\alpha/2,v} s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}}$$

where $v = n_1 + n_2 - 2$. For the case of unequal variances, an approximate $(1-\alpha)\%$ confidence interval on $\theta = \theta_1 - \theta_2$ is given by:

$$\hat{D} \pm t_{\alpha/2,v} \sqrt{S_1^2/n_1 + S_2^2/n_2}$$

where

$$v = \left\lfloor \frac{(S_1^2/n_1 + S_2^2/n_2)^2}{\frac{(S_1^2/n_1)^2}{n_1+1} + \frac{(S_2^2/n_2)^2}{n_2+1}} - 2 \right\rfloor$$

Let $[l, u]$ be the resulting confidence interval where l and u represent the lower and upper limits of the interval with by construction $l < u$. Thus, if $u < 0$, you can conclude with $(1-\alpha)\%$ confidence that $\theta = \theta_1 - \theta_2 < 0$ (i.e. that $\theta_1 < \theta_2$). If $l > 0$, you can conclude with $(1-\alpha)\%$ that $\theta = \theta_1 - \theta_2 > 0$ (i.e. that $\theta_1 > \theta_2$). If $[l, u]$ contains 0, then no conclusion can be made at the given sample sizes about which system is better. This does not indicate that the system performance is the same for the two systems. You *know* that the systems are different. Thus, their performance will be different. This only indicates that you have not taken enough samples to detect the true difference. If sampling is relatively cheap, then you may want to take additional samples in order to discern an ordering between the systems.

Two configurations are under consideration for the design of an airport security checkpoint. A simulation model of each design was made. The replication values of the throughput per minute for the security station for each design are provided in the following table.

	Design 1	Design 2
1	10.98	8.93

	Design 1	Design 2
2	8.87	9.82
3	10.53	9.27
4	9.40	8.50
5	10.10	9.63
6	10.28	9.55
7	8.86	9.30
8	9.00	9.31
9	9.99	9.23
10	9.57	8.88
11		8.05
12		8.74
$\bar{x}$	9.76	9.10
s	0.74	0.50
n	10	12

Assume that the two simulations were run independently of each other, using different random numbers. Recommend the better design with 95% confidence.

According to the results:

$$\hat{D} = \bar{X}_1 - \bar{X}_2 = 9.76 - 9.1 = 0.66$$

In addition, we should test if the variances of the samples are equal. This requires an F test, with $H_0 : \sigma_1^2 = \sigma_2^2$ versus $H_1 : \sigma_1^2 \neq \sigma_2^2$. Based on elementary statistics, the test statistic is: $F_0 = S_1^2/S_2^2$. The rejection criterion is to reject H_0 if $F_0 > f_{\alpha/2,n_1-1,n_2-1}$ or $F_0 < f_{1-\alpha/2,n_1-1,n_2-1}$, where $f_{p,u,v}$ is the upper percentage point of the F distribution. Assuming a 0.01 significance level for the F test, we have $F_0 = (0.74)^2/(0.50)^2 = 2.12$. Since $f_{0.005,9,11} = 5.54$ and $f_{0.995,9,11} = 0.168$, there is not enough evidence to conclude that the variances are different at the 0.01 significance level.

The value of $f_{p,u,v}$ can be determined in as F.INV.RT(p, u, v). Note also that $f_{1-p,u,v} = 1/f_{p,v,u}$. In *R*, the formula is $f_{p,u,v} = qf(1-p, u, v)$, since *R* provides the quantile function, not the upper right tail function.

Since the variances can be assumed equal, we can use the pooled variance, which is:

$$\begin{aligned} S_p^2 &= \frac{(n_1-1)S_1^2 + (n_2-1)S_2^2}{n_1+n_2-2} \\ &= \frac{(10-1)(0.74)^2 + (12-1)(0.5)^2}{12+10-2} \\ &= 0.384 \end{aligned}$$

Thus, a $(1-0.05)\%$ confidence interval on $\theta = \theta_1 - \theta_2$ is:

$$\begin{aligned} &\hat{D} \pm t_{\alpha/2,v} s_p \sqrt{\frac{1}{n_1} + \frac{1}{n_2}} \\ &0.66 \pm t_{0.025,20}(\sqrt{0.384})\sqrt{\frac{1}{10} + \frac{1}{12}} \\ &0.66 \pm (2.086)(0.6196)(0.428) \\ &0.66 \pm 0.553 \end{aligned}$$

where $v = n_1 + n_2 - 2 = 10 + 12 - 2 = 20$. Since this results in an interval $[0.10, 1.21]$ that does not contain zero, we can conclude that design 1 has the higher throughput with 95% confidence.

The confidence interval can assist in making decisions regarding relative performance of the systems from a *statistically significant* standpoint. However, if you make a conclusion about the ordering of the system, it still may not be practically significant. That is, the difference in the system performance is statistically significant but the actual difference is of no practical use. For example, suppose you compare two systems in terms of throughput with resulting output $\bar{X}_1 = 5.2$ and $\bar{X}_2 = 5.0$ with

the difference statistically significant. While the difference of 0.2 may be statistically significant, you might not be able to achieve this in the actual system. After all, you are making a decision based on a *model of the system* not on the real system. If the costs of the two systems are significantly different, you should prefer the cheaper of the two systems since there is no practical difference between the two systems. The fidelity of the difference is dependent on your modeling assumptions. Other modeling assumptions may overshadow such a small difference.

The notion of practical significance is model and performance measure dependent. One way to characterize the notion of practical significance is to conceptualize a zone of performance for which you are indifferent between the two systems.

Figure 5.22 illustrates the concept of an indifference zone around the difference between the two systems. If the difference between the two systems falls in this zone, you are indifferent between the two systems (i.e. there is no practical difference).

Using the indifference zone to model the notion of practical significance, if $u < -\Delta$, you can conclude confidence that $\theta_1 < \theta_2$, and if $l > \Delta$, you can conclude with confidence that $\theta_1 > \theta_2$. If l falls within the indifference zone and u does not (or vice versa), then there is not enough evidence to make a confident conclusion. If $[l, u]$ is totally contained within the indifference zone, then you can conclude with confidence that there is no practical difference between the two systems.

5.7.1.2. Analyzing Two Dependent Samples

In this situation, continue to assume that the observations within a sample are independent and identically distributed random variables; however, the samples themselves are not independent. That is, assume that the $(X_{11}, X_{12}, \ldots, X_{1n_1})$ and $(X_{21}, X_{22}, \ldots, X_{2n_2})$ from the two systems are dependent.

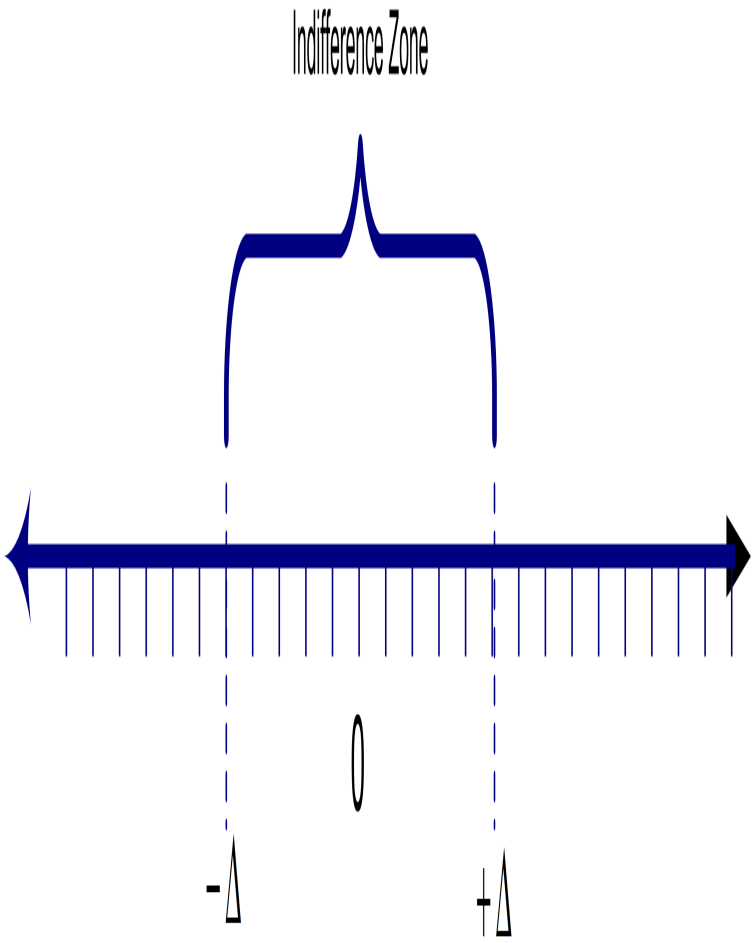


Figure 5.22.: Indifference Zone Concept

For simplicity, suppose that the difference in the configurations can be implemented using a simple parameter change within the model. For example, the mean processing time is different for the two configurations. First, run the model to produce $(X_{11}, X_{12}, \ldots, X_{1n_1})$ for configuration 1. Then, change the parameter and re-executed the model to produce $(X_{21}, X_{22}, \ldots, X_{2n_2})$ for configuration 2.

Assuming that you did nothing with respect to the random number streams, the second configuration used the same random numbers that the first configuration used. Thus, the generated responses will be correlated (dependent). In this situation, it is convenient to assume that each system is run for the same number of replications, i.e. $n_1 = n_2 =$ n. Since each replication for the two systems uses the same random number streams, the correlation between $(X_{1,j}, X_{2,j})$ will not be zero; however, each pair will still be independent *across* the replications. The basic approach to analyzing this situation is to compute the difference for each pair:

$$D_j = X_{1j} - X_{2j} \quad \text{for } j = 1, 2, \ldots, n$$

The $(D_1, D_2, \ldots, D_n)$ will form a random sample, which can be analyzed via traditional methods. Thus, a $(1-\alpha)\%$ confidence interval on $\theta = \theta_1 - \theta_2$ is:

$$\bar{D} = \frac{1}{n} \sum_{j=1}^{n} D_j$$

$$S_D^2 = \frac{1}{n-1} \sum_{j=1}^{n} (D_j - \bar{D})^2$$

$$\bar{D} \pm t_{\alpha/2, n-1} \frac{S_D}{\sqrt{n}}$$

The interpretation of the resulting confidence interval $[l, u]$ is the same as in the independent sample approach. This is the paired-t confidence interval presented in statistics textbooks.

Assume that the two simulations were run dependently using common random numbers. Recommend the better design with 95% confidence.

Assume that the two simulations were run dependently using common random numbers. Recommend the better design with 95% confidence.

Table 5.11.: Comparing Two Designs with CRN

	Design 1	Design 2	Difference
1	52.56	49.92	2.64
2	48.77	47.08	1.69
3	53.49	50.62	2.87
4	50.60	48.45	2.15
5	51.60	49.20	2.40
6	51.77	49.33	2.44
7	51.09	48.81	2.27
8	53.51	50.64	2.88
9	47.44	46.08	1.36
10	47.94	46.45	1.48
$\bar{x}$	50.88	48.66	2.22
s	2.18	1.64	0.55
n	10	10	10

According to the results:

$$\bar{D} = \bar{X}_1 - \bar{X}_2 = 50.88 - 48.66 = 2.22$$

Also, we have that $S_D^2 = (0.55)^2$. Thus, a $(1 - 0.05)\%$ confidence interval on $\theta = \theta_1 - \theta_2$ is:

$$\begin{aligned}
\hat{D} &\pm t_{\alpha/2,n-1}\frac{S_D}{\sqrt{n}} \\
2.22 &\pm t_{0.025,9}\frac{0.55}{\sqrt{10}} \\
2.22 &\pm (2.261)(0.1739) \\
2.22 &\pm 0.0.393
\end{aligned}$$

Since this results in an interval $[1.827, 2.613]$ that does not contain zero, we can conclude that design 1 has the higher throughput with 95% confidence.

Of the two approaches (independent versus dependent) samples, the latter is much more prevalent in simulation contexts. The approach is called the method of *common random numbers (CRN)* and is a natural by product of how most simulation languages handle their assignment of random number streams.

To understand why this method is the preferred method for comparing two systems, you need to understand the method's affect on the variance of the estimator. In the case of independent samples, the estimator of performance was $\hat{D} = \bar{X}_1 - \bar{X}_2$. Since

$$\begin{aligned}
\bar{D} &= \frac{1}{n}\sum_{j=1}^{n} D_j \\
&= \frac{1}{n}\sum_{j=1}^{n}(X_{1j} - X_{2j}) \\
&= \frac{1}{n}\sum_{j=1}^{n} X_{1j} - \frac{1}{n}\sum_{j=1}^{n} X_{2j} \\
&= \bar{X}_1 - \bar{X}_2 \\
&= \hat{D}
\end{aligned}$$

The two estimators are the same, when $n_1 = n_2 = n$; however, their variances are not the same. Under the assumption of independence, computing the variance of the estimator yields:

$$V_{\text{IND}} = Var(\bar{X}_1 - \bar{X}_2) = \frac{\sigma_1^2}{n} + \frac{\sigma_2^2}{n}$$

Under the assumption that the samples are not independent, the variance of the estimator is:

$$V_{\text{CRN}} = Var(\bar{X}_1 - \bar{X}_2) = \frac{\sigma_1^2}{n} + \frac{\sigma_2^2}{n} - 2\text{cov}(\bar{X}_1, \bar{X}_2)$$

If you define $\rho_{12} = corr(\bar{X}_1, \bar{X}_2)$, the variance for the common random number situation is:

$$V_{\text{CRN}} = V_{\text{IND}} - 2\sigma_1\sigma_2\rho_{12}$$

Therefore, whenever there is positive correlation $\rho_{12} > 0$ within the pairs we have that, $V_{\text{CRN}} < V_{\text{IND}}$.

If the variance of the estimator in the case of common random numbers is smaller than the variance of the estimator under independent sampling, then a *variance reduction* has been achieved. The method of common random numbers is called a variance reduction technique. If the variance reduction results in a confidence interval for θ that is tighter than the independent case, the use of common random numbers should be preferred. The variance reduction needs to be big enough to overcome any loss in the number of degrees of freedom caused by the pairing. When the number of replications is relatively large ($n > 30$) this will generally be the case since the student-t value does not vary appreciatively for large degrees of freedom. Notice that the method of common random numbers might backfire and cause a variance increase if there is negative correlation between the pairs.

An overview of the conditions under which common random numbers may work is given in (Law 2007).

This notion of pairing the outputs from each replication for the two system configurations makes common sense. When trying to discern a difference, you want the two systems to experience the same randomness so that you can more readily infer that any difference in performance is due to the inherent difference between the systems and not caused by the random numbers.

In experimental design settings, this is called blocking on a factor. For example, if you wanted to perform and experiment to determine whether a change in a work method was better than the old method, you should use the same worker to execute both methods. If instead, you had different workers execute the methods, you would not be sure if any difference was due to the workers or to the proposed change in the method. In this context, the worker is the factor that should be blocked. In the simulation context, the random numbers are being blocked when using common random numbers.

5.7.1.3. Using Common Random Numbers

The following explores how independent sampling and common random numbers can be implemented using the KSL. To illustrate this situation, we will utilize the familiar pallet processing example. For this situation, we will compare the performance of the system based on the time to process all pallets using two and three workers. The following code will demonstrate how to get the results based on not using common random number and based on using common random numbers. To facilitate this analysis we will use a `KSLDatabaseObserver` and the resulting `KSLDatabase` to capture the output from the simulations. In addition, we will use the `MultipleComparisonAnalyzer` class found in the `ksl.utilities.statistics` package to tabulate and compute the paired difference results. Based on our previous work with the pallet processing

model, the code is very straight-forward. The following function executes the model twice. In the code, we make sure to assign different names to the experiments before running them. Of course, the number of workers is also changed. The code executes both simulation experiments sequentially and the results are stored in the KSL database.

```
fun withoutCommonRandomNumbers(){
    val model = Model("Pallet Model Experiments")
    model.numberOfReplications = 30
    // add the model element to the main model
    val palletWorkCenter = PalletWorkCenter(model)
    // use a database to capture the response data
    val kslDatabaseObserver = KSLDatabaseObserver(model)

    model.experimentName = "Two Workers"
    palletWorkCenter.numWorkers = 2
    model.simulate()

    model.experimentName = "Three Workers"
    palletWorkCenter.numWorkers = 3
    model.simulate()

    val responseName =
     ↪  palletWorkCenter.totalProcessingTime.name
    val db = kslDatabaseObserver.db

    val expNames = listOf("Two Workers", "Three Workers")
    val comparisonAnalyzer =
     ↪  db.multipleComparisonAnalyzerFor(expNames,
     ↪  responseName)
    println(comparisonAnalyzer)
}
```

Notice the second to last line of the code where we use the instance of the

KSLDatabase to create an instance of the MultipleComparisonAnalyzer class based on the names of the experiments and the name of the response variable to be compared. The MultipleComparisonAnalyzer has a significant amount of output, part of which is shown here.

```
Statistical Summary Report

Difference Data
Name                                                    Count          Average        Std.
-------------------------------------------------------------------------------------------
Two Workers - Three Workers                                30          64.7723          57
-------------------------------------------------------------------------------------------
95.0% Confidence Intervals on Difference Data
Two Workers - Three Workers [43.15986335728935, 86.38476883829914]
```

We should note the sample standard deviation on the difference, $s_d = 57.8792$. Since the confidence interval does not contain zero, we can conclude with 95% confidence that the system with two workers has a longer time to complete the processing of the pallets. Of course, this result should be expected.

Because the simulation experiments were executed sequentially within the same invocation of the program, the random number streams in the three worker simulation continued from where the streams ended for the two worker simulation. Thus, the two experiments used different underlying random numbers and can be considered as independent sampling. To utilize common random numbers we must manipulate the streams. The following code illustrates how to implement common random numbers within the KSL.

Example 5.6 (Performing a CRN Analysis)**.** This code example illustrates how to perform a common random number analysis for two systems by running them within the same execution frame and setting the reset start stream option to true.

```
fun withCommonRandomNumbers(){
    val model = Model("Pallet Model Experiments")
    model.numberOfReplications = 30
    // add the model element to the main model
    val palletWorkCenter = PalletWorkCenter(model)
    // use a database to capture the response data
    val kslDatabaseObserver = KSLDatabaseObserver(model)

    model.resetStartStreamOption = true
    model.experimentName = "Two Workers"
    palletWorkCenter.numWorkers = 2
    model.simulate()

    model.experimentName = "Three Workers"
    palletWorkCenter.numWorkers = 3
    model.simulate()

    val responseName =
     ↪  palletWorkCenter.totalProcessingTime.name
    val db = kslDatabaseObserver.db

    val expNames = listOf("Two Workers", "Three Workers")
    val comparisonAnalyzer =
     ↪  db.multipleComparisonAnalyzerFor(expNames,
     ↪  responseName)
    println(comparisonAnalyzer)
}
```

The only difference between this code and the previous code is the addition of line 9:

```
    model.resetStartStreamOption = true
```

This line tells the model to automatically reset the streams of all of the random variables (and other constructs that use streams) back to their starting point in their stream prior to executing the replications of the experiment. This causes the three worker simulation to start in the streams in the same location as the two worker simulation. This facilitates the syncronization of the use of the random numbers. In addition, there are two other methods that the KSL automatically performs to assist in syncronization. First, each random variable utilizes its own dedicated stream, by default, and secondly, at the end of each replication, the random number streams are advanced (automatically) to the next substream in their main stream. This ensures that every replication starts in the same place in the streams. The effect of common random numbers on variance reduction can be dramatic. The results for this situation are as follows:

```
Statistical Summary Report

Difference Data
Name                                        Count            Average       Std. Dev.
-------------------------------------------------------------------------------------
Two Workers - Three Workers                    30            64.9215         33.0561
-------------------------------------------------------------------------------------
95.0% Confidence Intervals on Difference Data
Two Workers - Three Workers [52.57814571744155, 77.26483056807177]
```

We see the that the sample standard deviation on the difference, $s_d = 33.0561$ for the case of common random numbers has been significantly reduced (about a 43% reduction in this case). What does this mean for the simulation analyst? First the confidence interval half-width will also be shortened. Thus, we have a tighter margin of error for the same confidence level. This may be more important when the results are closer and when it takes significantly longer to execute the models. That is, we get more precision on the estimate of the difference for no extra computing.

It is important to note that setting the reset start stream option to true as indicated here is predicated on running both models in the same program execution. If we had run the two worker model today and then changed the number of workers to 3 for a run tomorrow, the simulations would have used the same random number streams (common random numbers by default). If we had saved the results in the database *and* reattached the same database to the execution, we would capture the results from both runs (just like we did in this example). Thus, we would still have used common random numbers. You need to be careful if you take this approach not to over write your earlier results by properly utilizing the KSL database functionality.

5.7.2. Concepts for Comparing Systems

The analysis of multiple systems stems from a number of objectives. First, you may want to perform a *sensitivity analysis* on the simulation model. In a sensitivity analysis, you want to measure the effect of small changes to key input parameters to the simulation. For example, you might have assumed a particular arrival rate of customers to the system and want to examine what would happen if the arrival rate decreased or increased by 10%. Sensitivity analysis can help you to understand how robust the model is to variations in assumptions. When you perform a sensitivity analysis, there may be a number of factors that need to be examined in combination with other factors. This may result in a large number of experiments to run. Besides performing a sensitivity analysis, you may want to compare the performance of multiple alternatives in order to choose the best alternative. This type of analysis is performed using multiple comparison statistical techniques. This section will also illustrate how to perform a multiple comparison with some of the constructs of the KSL.

Suppose that you are interested in analyzing k systems based on performance measures θ_i, $i = 1, 2, \ldots, k$. The goals may be to compare each of the k systems with a base case (or existing system), to develop an ordering

among the systems, or to select the best system. In any case, assume that some decision will be made based on the analysis and that the risk associated with making a bad decision needs to be controlled. In order to perform an analysis, each θ_i must be estimated, which results in sampling error for each individual estimate of θ_i. The decision will be based upon all the estimates of θ_i (for every system). The sampling error involves the sampling for each configuration. This compounds the risk associated with an overall decision. To see this more formally, the "curse" of the Bonferroni inequality needs to be understood.

The Bonferroni inequality states that given a set of (not necessarily independent) events, E_i, which occur with probability, $1-\alpha_i$, for $i = 1, 2, \ldots, k$ then a lower bound on the probability of the intersection of the events is given by:

$$P\{\cap_{i=1}^{k} E_i\} \geq 1 - \sum_{i=1}^{k} \alpha_i$$

In words, the Bonferroni inequality states that the probability of all the events occurring is at least one minus the sum of the probability of the individual events occurring. This inequality can be applied to confidence intervals, which are probability statements concerning the chance that the true parameter falls within intervals formed by the *procedure.*

Suppose that you have c confidence intervals each with confidence $1-\alpha_i$. The i^{th} confidence interval is a statement S_i that the confidence interval procedure will result in an interval that contains the parameter being estimated. The confidence interval procedure forms intervals such that S_i will be true with probability $1-\alpha_i$. If you define events, $E_i = \{S_i \text{is true}\}$, then the intersection of the events can be interpreted as the event representing all the statements being true.

$$P\{\text{all } S_i \text{ true}\} = P\{\cap_{i=1}^{k} E_i\} \geq 1 - \sum_{i=1}^{k} \alpha_i = 1 - \alpha_E$$

where $\alpha_E = \sum_{i=1}^{k} \alpha_i$. The value α_E is called the overall error probability. This statement can be restated in terms of its complement event as:

$$P\{\text{one or more } S_i \text{ are false}\} \leq \alpha_E$$

This gives an upper bound on the probability of a false conclusion based on the procedures that generated the confidence intervals.

This inequality can be applied to the use of confidence intervals when comparing multiple systems. For example, suppose that you have, $c = 10$, 90% confidence intervals to interpret. Thus, $\alpha_i = 0.10$, so that

$$\alpha_E = \sum_{i=1}^{10} \alpha_i = \sum_{i=1}^{10} (0.1) = 1.0$$

Thus, $P\{\text{all } S_i \text{ true}\} \geq 0$ or $P\{\text{one or more } S_i \text{ are false}\} \leq 1$. In words, this is implying that the chance that all the confidence intervals procedures result in confidence intervals that cover the true parameter is greater than zero and less than 1. Think of it this way: If your boss asked you how confident you were in your decision, you would have to say that your confidence is somewhere between zero and one. This would not be very reassuring to your boss (or for your job!).

To combat the "curse" of Bonferroni, you can adjust your confidence levels in the individual confidence statements in order to obtain a desired overall risk. For example, suppose that you wanted an overall confidence of 95% on making a correct decision based on the confidence intervals. That is you desire, $\alpha_E = 0.05$. You can pre-specify the α_i for each individual confidence interval to whatever values you want provided that you get an overall error probability of $\alpha_E = 0.05$. The simplest approach is to assume $\alpha_i = \alpha$. That is, use a common confidence level for all the confidence intervals. The question then becomes: What should α be to get $\alpha_E = 0.05$? Assuming that you have c confidence intervals, this yields:

$$\alpha_E = \sum_{i=1}^{c} \alpha_i = \sum_{i=1}^{c} \alpha = c\alpha$$

So that you should set $\alpha = \alpha_E/c$. For the case of $\alpha_E = 0.05$ and $c = 10$, this implies that $\alpha = 0.005$. What does this do to the width of the individual confidence intervals? Since the $\alpha_i = \alpha$ have gotten smaller, the confidence coefficient (e.g. z value or t value) used in computation of the confidence interval will be larger, resulting in a wider confidence interval. Thus, you must trade-off your overall decision error against wider (less precise) individual confidence intervals.

Because the Bonferroni inequality does not assume independent events it can be used for the case of comparing multiple systems when common random numbers are used. In the case of independent sampling for the systems, you can do better than simply bounding the error. For the case of comparing k systems based on independent sampling the overall confidence is:

$$P\{\text{all } S_i \text{ true}\} = \prod_{i=1}^{c}(1 - \alpha_i)$$

If you are comparing k systems where one of the systems is the standard (e.g. base case, existing system, etc.), you can reduce the number of confidence intervals by analyzing the difference between the other systems and the standard. That is, suppose system one is the base case, then you can form confidence intervals on $\theta_1 - \theta_i$ for $i = 2, 3, \ldots, k$. Since there are $k - 1$ differences, there are $c = k - 1$ confidence intervals to compare.

If you are interested in developing an ordering between all the systems, then one approach is to make all the pair wise comparisons between the systems. That is, construct confidence intervals on $\theta_j - \theta_i$ for $i \neq j$. The number of confidence intervals in this situation is

$$c = \binom{k}{2} = \frac{k(k-1)}{2}$$

The trade-off between overall error probability and the width of the individual confidence intervals will become severe in this case for most practical situations.

Because of this problem a number of techniques have been developed to allow the selection of the best system (or the ranking of the systems) and still guarantee an overall pre-specified confidence in the decision. Multiple comparison procedures are described in (Goldsman and Nelson 1998) and the references therein. See also (Law 2007) for how these methods relate to other ranking and selection methods.

5.7.3. Multiple Comparison with the Best Procedures (MCB)

While it is beyond the scope of this textbook to review multiple comparison with the best (MCB) procedures, it is useful to have a basic understanding of the form of the confidence interval constructed by these procedures. To see how MCB procedures avoid part of the issues with having a large number of confidence intervals, we can note that the confidence interval is based on the difference between the best and the best of the rest. Suppose we have k system configurations to compare and suppose that somehow you knew that the i^{th} system *is the best*. Now, consider a confidence interval for system's i performance metric, θ_i, of the following form:

$$\theta_i - \max_{j \neq i} \theta_j$$

This difference is the difference between the best (θ_i) and the second best. This is because if θ_i is the best, when we find the maximum of those remaining, it will be next best (i.e. second best). Now, let us suppose that system i is *not* the best. Reconsider, $\theta_i - \max_{j \neq i} \theta_j$. Then, this difference

will represent the difference between system i, which is not the best, and the best of the remaining systems. In this case, because i is not the best, the set of systems considered in $\max_{j \neq i} \theta_j$ will contain the best. Therefore, in either case, this difference:

$$\theta_i - \max_{j \neq i} \theta_j$$

tells us exactly what we want to know. This difference allows us to compare the best system with the rest of the best. MCB procedures build k confidence intervals:

$$\theta_i - \max_{j \neq i} \theta_j \text{ for } i = 1, 2, \ldots, k$$

Therefore, only k confidence intervals need to be considered to determine the best rather than, $\binom{k}{2}$.

This form of confidence interval has also been combined with the concept of an indifference zone. Indifference zone procedures use a parameter δ that indicates that the decision maker is indifferent between the performance of two systems, if the difference is less than δ. This indicates that even if there is a difference, the difference is not practically significant to the decision maker within this context. These procedures attempt to ensure that the overall probability of correct selection of the best is $1-\alpha$, whenever, $\theta_{i^*} - \max_{j \neq i^*} \theta_j > \delta$, where i^* is the best system and δ is the indifference parameter. This assumes that larger is better in the decision making context. There are a number of single stage and multiple stage procedures that have been developed to facilitate decision making in this context.

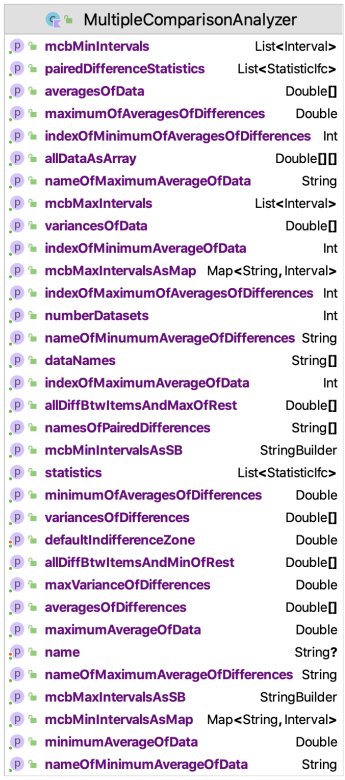
One additional item of note to consider when using these procedures. The results may not be able to distinguish between the best and others. For example, assuming that we desire the maximum (bigger is better), then let $\widehat{i}$ be the index of the system found to be the largest and let $\widehat{\theta}_i$ be the estimated performance for system i, then, these procedures allow the following:

- If $\hat{\theta}_i - \hat{\theta}_{\hat{i}} + \delta \leq 0$, then we declare that system i not the best.
- If $\hat{\theta}_i - \hat{\theta}_{\hat{i}} + \delta > 0$, then we can declare that system i is not statistically different from the best. In this case, system i, may in fact be the best.

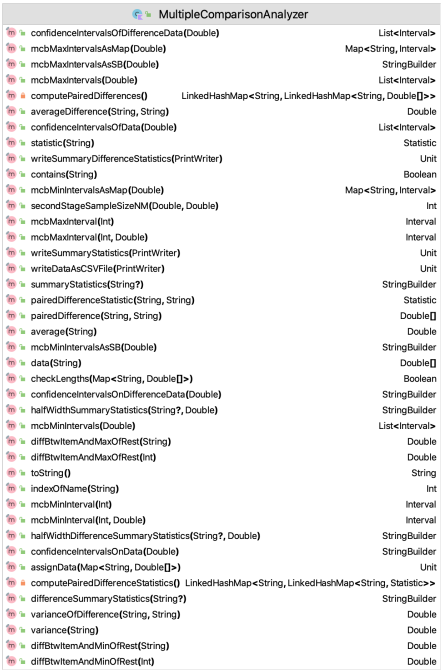
The computations necessary to perform many different multiple comparision procedures can be found in the `MultipleComparisonAnalyzer` class illustrated in Figure 5.23.

The `MultipleComparisonAnalyzer` class depends on having the data to be compared in the form of named arrays within a map `dataMap: Map<String, DoubleArray>`. Each array is associated with the name of the array via the map's key parameter. There needs to be at least 2 data arrays, and the length of each data array must be the same. As we have seen in the previous example, the KSL facilitates extracting and organizing the simulation data in this form from the KSL database. Such data can also be readily extracted from instances of a `ReplicationDataObserver` or an `ExperimentDataObserver`. The following code illustrates setting up a `MultipleComparisonAnalyzer` based on arrays.

```
fun main() {
    val data = LinkedHashMap<String, DoubleArray>()
    val d1 = doubleArrayOf(63.72, 32.24, 40.28, 36.94, 36.29,
     ↪  56.94, 34.10, 63.36, 49.29, 87.20)
    val d2 = doubleArrayOf(63.06, 31.78, 40.32, 37.71, 36.79,
     ↪  57.93, 33.39, 62.92, 47.67, 80.79)
    val d3 = doubleArrayOf(57.74, 29.65, 36.52, 35.71, 33.81,
     ↪  51.54, 31.39, 57.24, 42.63, 67.27)
    val d4 = doubleArrayOf(62.63, 31.56, 39.87, 37.35, 36.65,
     ↪  57.15, 33.30, 62.21, 47.46, 79.60)
    data["One"] = d1
    data["Two"] = d2
    data["Three"] = d3
    data["Four"] = d4
```



(a) Properties



(b) Methods

Figure 5.23.: Properties and Methods of the MultipleComparisonAnalyzer Class

```
    val mca = MultipleComparisonAnalyzer(data)
    println(mca)
}
```

The `MultipleComparisonAnalyzer` class will automatically compute the following:

- statistical summaries and confidence intervals on the individual data arrays
- statistical summaries and confidence intervals on all pair-wise differences
- the variance of the difference that is the largest
- the data set that has the smallest average and its average
- the data set that has the largest average and its average
- the pair-wise difference that is the smallest and its value
- the pair-wise difference that is the largest and its value
- multiple comparison confidence intervals based on finding the maximum value
- multiple comparison confidence intervals based on finding the minimum value
- screening intervals for best designs by constructing subsets containing the best configuration

The user can specify the indifference zone parameter to be included in the confidence interval calculations. The `MultipleComparisonAnalyzer` class does not utilize the probability of correct selection or assume some underlying distribution. The confidence intervals that are computed have the following forms. Define Y_{ij} as the i^{th} observation of system j. Thus, $\bar{Y}_{\cdot j}$ is the mean performance of system j. Let δ be the indifference zone parameter. When selecting the system with the largest $\bar{Y}_{\cdot j}$ as the best, the confidence intervals for configurations $i = 1, 2, \ldots, k$ are computed by:

$$\min\{0, \bar{Y}_{\cdot i} - \max_{j \neq i} \bar{Y}_{\cdot j} - \delta\} \leq \theta_i - \max_{j \neq i} \theta_j \leq \max\{0, \bar{Y}_{\cdot i} - \max_{j \neq i} \bar{Y}_{\cdot j} + \delta\}$$

When selecting the system with the smallest $\bar{Y}_{\cdot j}$ as the best, the confidence intervals for configurations $i = 1, 2, \ldots, k$ are computed by:

$$\min\{0, \bar{Y}_{\cdot i} - \min_{j \neq i} \bar{Y}_{\cdot j} - \delta\} \leq \theta_i - \min_{j \neq i} \theta_j \leq \max\{0, \bar{Y}_{\cdot i} - \min_{j \neq i} \bar{Y}_{\cdot j} + \delta\}$$

The results for the example code previously discussed illustrate the form of these confidence intervals.

```
MCB Maximum Intervals
Indifference delta: 0.0
Name        Interval
One     [0.0, 0.7999999999999972]
Two     [-0.7999999999999972, 0.0]
Three       [-5.686000000000007, 0.0]
Four        [-1.2579999999999956, 0.0]

MCB Minimum Intervals
Indifference delta: 0.0
Name        Interval
One     [0.0, 5.686000000000007]
Two     [0.0, 4.88600000000001]
Three       [-4.4280000000000115, 0.0]
Four        [0.0, 4.4280000000000115]
```

Notice how the form of these confidence interval equations determines how they are read. For the case of finding the maximum, a system whose interval has an upper limit of 0.0 is ruled out as the best, because its average falls at least δ below the best of the rest. A system whose interval has an upper limit greater than 0.0 cannot be ruled out. This is the decision rule $\widehat{\theta}_i - \widehat{\theta}_{\hat{i}} + \delta \leq 0$ stated earlier in this section. The reverse is true for the case of finding the minimum: a system whose interval has a lower limit of 0.0 is ruled out, and a system whose interval has a lower limit less than 0.0 cannot be ruled out. When $\delta = 0$, as in the output above, the sample best is the only system that cannot be ruled out, and its interval has a lower limit of

0.0 for the maximum and an upper limit of 0.0 for the minimum. When $\delta > 0$, more than one system may remain, and the remaining intervals may have both limits different from 0.0. Notice also that every interval contains 0.0, so whether an interval contains zero does not decide anything; only the limits that equal 0.0 do. Also, note that the confidence intervals (as defined here) do not have a distribution requirement for computing a test statistic value. Thus, their forms do not depend on the distributional assumptions. However, the probability of correctly selecting the best will depend upon some distributional assumptions. If more than one system cannot be ruled out, it is advisable to increase the sample size to see if a clear winner emerges.

Further details of such procedures can be found in (Goldsman and Nelson 1998). As a note, some of those procedures require the computation of critical values from the multivariate T distribution and Rinott's constant. Rinott's constant can be computed using the `Rinott` class within the `ksl.utilities.statistics` package and limited values for the multivariate T distribution are available via the `MultipleComparisonAnalyzer` class's companion object. Finally, the quantiles of the Tukey distribution are available via the `Tukey` class located in the `ksl.utilities.distributions` package.

To illustrate the application of MCB methods, let us consider the two-stage Bonferroni based procedure developed by (Matejcik and Nelson 1995) and also described in (Goldsman and Nelson 1998) as well as in (Banks et al. 2005). This presentation follows the one in (Banks et al. 2005). To apply the procedure, the analyst specifies a probability of correct selection (PCS) as $(1 - \alpha)$ and performs the experimentation in two-stages. The first stage is essentially a pilot set of replications from which the variability of the design configurations is estimated and used to plan the second stage of the procedure. The procedure is applicable when the data is normally distributed and can be applied if the observations are independent or if CRN has been used.

Two-Stage Bonferroni Procedure

1. Specify the indifference parameter, δ, PCS as $(1-\alpha)$, the initial sample size, n_0, and compute $t = t_{1-\alpha/(k-1),n_0-1}$
2. Execute the n_0 replications for each system and capture Y_{ij} as the i^{th} observation of system j
3. Compute $\bar{Y}_{\cdot j}$ as the mean performance of system j, for $j = 1, 2, \cdots, k$. Then for all $i \neq j$, compute the sample variance of the differences and the largest sample variance $\hat{S}^2 = max_{i \neq j} S_{ij}^2$

$$S_{ij}^2 = \frac{1}{n_0 - 1} \sum_{r=1}^{n_0} ((Y_{ri} - Y_{rj}) - (\bar{Y}_{\cdot i} - \bar{Y}_{\cdot j}))^2$$

4. Calculate the second stage sample size:

$$n = max\left\{n_0, \left\lceil \frac{t^2 \hat{S}^2}{\delta^2} \right\rceil\right\}$$

5. Make $n - n_0$ additional replications (or just make n by re-running). Capture Y_{ij} as the i^{th} observation of system j
6. Calculate the overall sample means for $j = 1, 2, \cdots, k$ for each design.

$$\bar{Y}_{\cdot j} = \frac{1}{n} \sum_{r=1}^{n} Y_{rj}$$

7. Select the system with the largest $\bar{Y}_{\cdot j}$ as the best and if maximizing form the confidence intervals:

$\min\{0, \bar{Y}_{\cdot i} - \max_{j \neq i} \bar{Y}_{\cdot j} - \delta\} \leq \theta_i - \max_{j \neq i} \theta_j \leq \max\{0, \bar{Y}_{\cdot i} - \max_{j \neq i} \bar{Y}_{\cdot j} + \delta\}$

If minimizing, form the confidence intervals:

$\min\{0, \bar{Y}_{\cdot i} - \min_{j \neq i} \bar{Y}_{\cdot j} - \delta\} \leq \theta_i - \min_{j \neq i} \theta_j \leq \max\{0, \bar{Y}_{\cdot i} - \min_{j \neq i} \bar{Y}_{\cdot j} + \delta\}$

The `MultipleComparisonAnalyzer` class automatically computes everything that is needed to apply the Two-Stage Bonferroni procedure. The `toString()` method can be used to see all the results. The second stage sample size can be computed via the function `secondStageSampleSizeNM()`, and the maximum variance can be found from property, `maxVarianceOfDifferences`. In addition, the maximum or minimum performer can be noted from the `nameOfMaximumAverageOfData` and `nameOfMinimumAverageOfData` properties. The respective performance of the maximum and minimum via the `maximumAverageOfData` and `minimumAverageOfData` properties. Finally, the MCB intervals for each case are found from the `mcbMaxIntervals(delta)` and `mcbMinIntervals(delta)` functions respectively. Note that if $n = n_0$ in step 5, then no additional replications are required and the PCS will still be met. (Matejcik and Nelson 1995) recommend that the initial number of replications in the first stage should be greater than or equal to 10 ($n_0 \geq 10$). The two-stage Bonferroni procedure is somewhat conservative in recommending the sample size requirements. If the number of replications is of concern, then the reader should consult (Banks et al. 2005) for other procedures.

The following KSL code illustrates the basic use of the `MultipleComparisonAnalyzer` class on some data.

```
fun main() {
    val data = LinkedHashMap<String, DoubleArray>()
    data["One"] = doubleArrayOf(63.72, 32.24, 40.28, 36.94,
     ↪  36.29, 56.94, 34.10, 63.36, 49.29, 87.20)
    data["Two"] = doubleArrayOf(63.06, 31.78, 40.32, 37.71,
     ↪  36.79, 57.93, 33.39, 62.92, 47.67, 80.79)
    data["Three"] = doubleArrayOf(57.74, 29.65, 36.52, 35.71,
     ↪  33.81, 51.54, 31.39, 57.24, 42.63, 67.27)
    data["Four"] = doubleArrayOf(62.63, 31.56, 39.87, 37.35,
     ↪  36.65, 57.15, 33.30, 62.21, 47.46, 79.60)
    val mca = MultipleComparisonAnalyzer(data)
```

```
    val out = PrintWriter(System.out, true)
    mca.writeDataAsCSVFile(out)
    out.println()
    println(mca)
    println("num data sets: " + mca.numberDatasets)
    println(mca.dataNames.contentToString())
    val r = mca.secondStageSampleSizeNM(2.0, 0.95)
    println("Second stage sampling recommendation R = $r")
}
```

The output from the running of the code contains everything you need for a MCB analysis.

```
Multiple Comparison Report: Statistical Summary Report

Raw Data
Name                          Count          Average       Std. Dev.
-------------------------------------------------------------------
One                           10          50.0360          17.7027
Two                           10          49.2360          16.2450
Three                         10          44.3500          13.1509
Four                          10          48.7780          15.9415
-------------------------------------------------------------------

95.0% Confidence Intervals on Data
One [37.37228240333723, 62.69971759666277]
Two [37.61502264027472, 60.85697735972529]
Three   [34.94237884397989, 53.7576211560201]
Four    [37.37412553653087, 60.18187446346914]

Statistical Summary Report
```

```
Difference Data
Name                                Count           Average       Std. Dev.
-------------------------------------------------------------------------
One - Two                           10            0.8000          2.1208
One - Three                         10            5.6860          5.3384
One - Four                          10            1.2580          2.3430
Two - Three                         10            4.8860          3.4435
Two - Four                          10            0.4580          0.3445
Three - Four                        10           -4.4280          3.1384
-------------------------------------------------------------------------

95.0% Confidence Intervals on Difference Data
One - Two   [-0.717163895232598, 2.317163895232595]
One - Three [1.8671698996711696, 9.504830100328828]
One - Four  [-0.41806967690470764, 2.934069676904709]
Two - Three [2.422703009361264, 7.349296990638738]
Two - Four  [0.21155519141551002, 0.7044448085844948]
Three - Four    [-6.673063053959199, -2.182936946040797]

Max variance = 28.498048888888913
Min performer = Three
Min performance = 44.349999999999994
Max performer = One
Max performance = 50.036
Min difference = Three - Four
Min difference value = -4.427999999999998
Max difference = One - Three
Max difference value = 5.685999999999999

MCB Maximum Intervals
Indifference delta: 0.0
Name        Interval
One     [0.0, 0.7999999999999972]
Two     [-0.7999999999999972, 0.0]
```

```
Three       [-5.686000000000007, 0.0]
Four        [-1.2579999999999956, 0.0]

MCB Minimum Intervals
Indifference delta: 0.0
Name        Interval
One     [0.0, 5.686000000000007]
Two     [0.0, 4.88600000000001]
Three       [-4.4280000000000115, 0.0]
Four        [0.0, 4.4280000000000115]

num data sets: 4
[One, Two, Three, Four]
Second stage sampling recommendation R = 45
```

5.7.4. Screening Procedures

Assuming that the data is normally distributed (with or without CRN), screening procedures can be designed to select a subset of the system designs such that the retained subset will contain the true best system with probability greater than $1 - \alpha$. Such procedures are useful when there are a large number of designs or during automated search procedures. Once a reliable subset has been found then MCB procedures can be used to select the best from the designated subset. The following screening procedure is based on (Nelson et al. 2001).

- Step 1: Specify the PCS as $(1 - \alpha)$, a common sample size, $n \geq 2$, and compute $t = t_{1-\alpha/(k-1),n-1}$
- Step 2: Execute the n replications for each system and capture Y_{ij} as the i^{th} observation of system j
- Step 3: Compute $\bar{Y}_{\cdot j}$ as the mean performance of system j, for $j = 1, 2, \cdots, k$. Then for all $i \neq j$, compute the sample variance of the differences:

$$S_{ij}^2 = \frac{1}{n_0 - 1} \sum_{r=1}^{n_0} ((Y_{ri} - Y_{rj}) - (\bar{Y}_{\cdot i} - \bar{Y}_{\cdot j}))^2$$

- Step 4: If bigger is better, then place system i in the selection subset if for all $j \neq i$:

  $$\bar{Y}_{\cdot i} \geq \bar{Y}_{\cdot j} - t\frac{S_{ij}}{\sqrt{n}}$$

  If smaller is better, then place system i in the selection subset if for all $j \neq i$:

  $$\bar{Y}_{\cdot i} \leq \bar{Y}_{\cdot j} + t\frac{S_{ij}}{\sqrt{n}}$$

Suppose we want to apply the screening procedure to the pairwise comparison data and assume that smaller is better. The screening procedure essentially says to add the system to the subset of remaining alternatives if its sample average is smaller than all the other sample means that have been adjusted by a positive quantity that accounts for sampling error. In the pairwise difference data, we have $\bar{Y}_{\cdot i}$ and S_{ij}.

Let's take a look at a simplified example. Note that $\hat{se}_{ij} = S_{ij}/\sqrt{n}$. If smaller is better, then place system i in the selection subset if for all $j \neq i$, $\bar{Y}_{\cdot i} \leq \bar{Y}_{\cdot j} + t * \hat{se}_{ij}$. In this example suppose $k = 4$, $\alpha = 0.05$, $n = 10$, $t = t_{1-\alpha/(k-1),n-1} = 2.5096$. In the following the sample average for system 1, $\bar{Y}_{\cdot 1} = 50.036$ is compared to the computed upper limits for all the other configurations.

$$\begin{aligned}
\bar{Y}_{\cdot 1} = 50.036 &\overset{?}{\leq} \bar{Y}_{\cdot 2} + t * \hat{se}_{12} = 49.236 + 2.5096 \times 0.6707 = 50.9191 \\
&\overset{?}{\leq} \bar{Y}_{\cdot 3} + t * \hat{se}_{13} = 44.35 + 2.5096 \times 1.6881 = 48.5865 \\
&\overset{?}{\leq} \bar{Y}_{\cdot 4} + t * \hat{se}_{14} = 48.778 + 2.5096 \times 0.7409 = 50.6374
\end{aligned}$$

Note that system 1 fails to be less than **all** the limits. Thus, system 1 is eliminated. This calculation is continued for all the systems to form the subset holding the remaining systems that were not eliminated during the screening process.

To apply the `MultipleComparisonAnalyzer` class on a KSL simulation, we can just attach a `KSLDatabaseObserver` instance and ask for the `MultipleComparisonAnalyzer` instance as illustrated in the following code for the pallet model.

Example 5.7 (Performing a MCB Analysis)**.** In this KSL code example, three configurations are compared using CRN and MCB.

```
fun main() {
    val model = Model("Pallet Model MCB")

    // add the model element to the main model
    val palletWorkCenter = PalletWorkCenter(model)
    // use a database to capture the response data
    val kslDatabaseObserver = KSLDatabaseObserver(model)
    // simulate the model
    model.experimentName = "One Worker"
    palletWorkCenter.numWorkers = 1
    model.resetStartStreamOption = true
    model.numberOfReplications = 30
    model.simulate()

    model.experimentName = "Two Workers"
    palletWorkCenter.numWorkers = 2
    model.simulate()

    model.experimentName = "Three Workers"
    palletWorkCenter.numWorkers = 3
    model.simulate()
```

```
    val responseName =
     ↪ palletWorkCenter.totalProcessingTime.name
    val db = kslDatabaseObserver.db

    val expNames = listOf("One Worker","Two Workers", "Three
     ↪ Workers")
    val comparisonAnalyzer =
     ↪ db.multipleComparisonAnalyzerFor(expNames,
     ↪ responseName)
    println(comparisonAnalyzer)
}
```

It should be obvious that 3 workers results in the smallest total processing time. Notice in the screening output that the 3 worker case is the only system that passes all the screening tests.

```
MCB Minimum Intervals
Indifference delta: 0.0
Name        Interval
One Worker      [0.0, 517.7287463179446]
Two Workers     [0.0, 64.92148814275674]
Three Workers       [-64.92148814275674, 0.0]

Min Screening Intervals
One Worker  average = 943.8223691355911     Two Workers [-
Infinity, 499.44125093353716]
One Worker  average = 943.8223691355911     Three Workers   [-
Infinity, 444.40016711694113]
Two Workers average = 491.0151109604032     One Worker  [-
Infinity, 952.2485091087251]
Two Workers average = 491.0151109604032     Three Workers   [-
Infinity, 438.43696524296155]
```

```
Three Workers   average = 426.09362281764646        One Worker  [-
Infinity, 962.1289134348857]
Three Workers   average = 426.09362281764646        Two Workers [-
Infinity, 503.3584533857183]

Alternatives remaining after screening for minimum:
[Three Workers]
```

5.7.5. Comparing Different Models

In Example 5.7, the three alternatives were configurations of one model: each experiment changed the number of workers and simulated the same `Model` instance again. Often the alternatives differ in their logic, not just in their inputs. For example, the drive-through pharmacy might use one waiting line that feeds two pharmacists, or a separate waiting line for each pharmacist. Each design needs its own model, and no input setting turns one model into the other. The MCB analysis itself does not change. What changes is how the results of both models reach one database, so that the `multipleComparisonAnalyzerFor()` function can find them.

The simplest approach is to create one `KSLDatabase` instance and attach a `KSLDatabaseObserver` instance to each model, passing the shared database to the observer's constructor. The following example shows this approach.

Example 5.8 (Comparing Two Different Models with MCB)**.** In this KSL code example, the classes `SingleLinePharmacy` and `TwoLinePharmacy` stand for the two models of the pharmacy; they are not part of the KSL examples. Both record the time that a customer spends in the system with a response named "System Time".

```
fun main() {
    // one database holds the results of both models
    val db = KSLDatabase("PharmacyDesigns.db")
```

```
    val modelA = Model("Single Line Model")
    SingleLinePharmacy(modelA, name = "Pharmacy")
    modelA.experimentName = "Single Line"
    modelA.numberOfReplications = 30
    modelA.lengthOfReplication = 20000.0
    modelA.lengthOfReplicationWarmUp = 5000.0
    KSLDatabaseObserver(modelA, db)

    val modelB = Model("Two Line Model")
    TwoLinePharmacy(modelB, name = "Pharmacy")
    modelB.experimentName = "Two Lines"
    modelB.numberOfReplications = 30
    modelB.lengthOfReplication = 20000.0
    modelB.lengthOfReplicationWarmUp = 5000.0
    KSLDatabaseObserver(modelB, db)

    modelA.simulate()
    modelB.simulate()

    val expNames = listOf("Single Line", "Two Lines")
    val comparisonAnalyzer =
     ↪  db.multipleComparisonAnalyzerFor(expNames, "System
     ↪  Time")
    println(comparisonAnalyzer)
}
```

The code creates the database first. Each model is then built, given its own experiment name and the same run parameters, and observed by a `KSLDatabaseObserver` instance that writes to the shared database rather than to a database of its own. After both models are simulated, the call to the `multipleComparisonAnalyzerFor()` function is the same as in Example 5.7: it takes the list of experiment names and the name of the

response, and the output has the same form.

In Example 5.7, three requirements were met without any effort, because every experiment used the same model. When the models differ, the following requirements must be met by the modeler:

- **The compared response must have the same name in both models.** The `multipleComparisonAnalyzerFor()` function looks up one response name in every experiment. If a model has no response with that name, the function stops with an error about the number of observations in the data arrays, which does not mention the response name. Many KSL examples prefix a response's name with the name of its model element, as in `"${this.name}:TimeInSystem"`. For such models, give the two elements the same name, as the example does with `"Pharmacy"`, or name the responses explicitly.
- **Each experiment must have a distinct experiment name.** The database identifies an experiment by its name. Simulating a second model with an experiment name that is already in the database causes an error, so the comparison can never mix the results of two experiments under one name.
- **Each experiment must use the same number of replications.** The MCB procedure works with the pairwise differences between the replications of each alternative, so the alternatives must have the same number of replications.

The example in Example 5.7 set `resetStartStreamOption` to true. One `Model` instance owns one set of random number streams, so without the reset the second and third experiments would have continued the streams where the previous experiment stopped, and the comparison would not have used common random numbers. Two different models each own their own streams, and each starts at the beginning of its streams. Thus, the option is not needed here. Common random numbers only reduce the variance to the extent that the streams are synchronized, however. The random variables that play the same role in the two models, such as the

time between arrivals and the service times, should be assigned the same stream numbers in both models.

The second approach is to place the models in scenarios and run them with the `ScenarioRunner` class, which Section 10.1.4 covers in detail. The runner creates one database, attaches an observer to each scenario's model, and names each experiment after its scenario. The following code shows the comparison with the two models of Example 5.8, where `modelA` and `modelB` are built as in the example, without the observers.

```
val scenarios = listOf(
    Scenario(model = modelA, name = "Single Line"),
    Scenario(model = modelB, name = "Two Lines")
)
val runner = ScenarioRunner("Pharmacy Designs",
 ↪  scenarios)
runner.simulate()
val comparisonAnalyzer =
 ↪  runner.kslDb?.multipleComparisonAnalyzerFor(
    expNames = runner.scenarioList.map { it.name },
    responseName = "System Time"
)
println(comparisonAnalyzer)
```

The runner meets the distinct-name requirement for us, but the same response name and the same number of replications are still the modeler's responsibility. Because the runner scales to any number of alternatives and also supports running them concurrently, it is the better choice once there are more than a few alternatives or the comparison will be repeated.

To learn more about how to simulate many scenarios, perform experiments, and optimally search for the best configuration, see Chapter 10 and Chapter 11.

5.8. Summary

This chapter described many of the statistical aspects of simulation that you will typically encounter in performing a simulation study. An important aspect of performing a correct simulation analysis is to understand the type of data associated with your performance measures (time-based versus observation-based) and how to collect/analyze such data. Then in your modeling you will be faced with specifying the time horizon of your simulation. Most situations involve finite-horizons, which are fortunately easy to analyze via the method of replications. This allows a random sample to be formed across replications and to analyze the simulation output via traditional statistical techniques.

In the case of infinite horizon simulations, things are more complicated. You must first analyze the effect of any warm up period on the performance measures and decide whether you should use the method of replication-deletion or the method of batch means.

Since you often want to use simulation to make a recommendation concerning a design configuration, an analysis across system configurations must be carefully planned. When performing your analysis, you should consider how and when to use the method of common random numbers and you should consider the impact of common random numbers on how you analyze the simulation results.

The following new KSL constructs were discussed in this chapter:

`replicationEnded()`: A function of model elements that can be overridden to provide logic and actions that occur at the end of a replication. An example illustrated how to use the function to collect statistics.

`autoCSVReports`: A parameter of the `Model` class that can be used to indicate whether within replication statistics and across replication statistics are written to a comma separated value files.

`ReplicationDataCollector`: A class that can be used to observe and collect across replication statistical values within arrays for further analysis.

`ResponseTrace`: A class that can be used to capture observation based and time-persistent observations to a file, recording the value of the variable and the time that the variable changed.

`SimulationReporter`: This class is similar to the `StatisticReporter` class and can be used to write out statistical summary reports containing KSL model outputs.

`KSLDatabaseObserver`: This class can be attached to a KSL model instance to create a database for holding within replication and across replication statistical outputs.

`KSLDatabase`: This class allows access to a database containing within replication and across replication statistical outputs from a KSL model run.

`AcrossReplicationHalfWidthChecker`: This class can be attached to a response within a KSL model and permits sequential sampling until a desired half-width criteria is met.

`WelchDataFileCollector`: This class captures data associated with simulation responses for the purpose of creating a Welch plot.

`WelchDataFileAnalyzer`: This class produces data that can be used to make a Welch plot.

`WelchPlot`: One of the KSL plots and facilitates the analysis of Welch data for the analysis of a warm up period for an infinite horizon simulation.

`BatchingElement`: A model element that can be used to facilitate the application of the batch means method for infinite horizon simulation analysis.

`resetStartStreamOption`: A property of the KSL `Model` class that can be used to cause the resetting of all random number streams to their starting point prior to a simulation run.

`MultipleComparisonAnalyzer`: This class facilitates the analysis and comparison of multiple simulation experiments by computer pairwise statistics and computing MCB confidence intervals. It also facilitates applying screening methods to multiple simulation runs.

Now that you have a solid understanding of how to program and model using the KSL and how to analyze your results, you are ready to explore the application of the KSL to additional modeling situations involving more complicated systems. The next chapter describes how to use the KSL for process view modeling.

5.9. Exercises

Exercise 5.1. Indicate whether or not the statistical quantity should be classified as tally-based or time-persistent.

Classification	Description
	The number of jobs waiting to be processed by a machine
	The number of jobs completed during a week
	The number in customers in queue:
	The time that the resource spends serving a customer
	The number of items in sitting on a shelf waiting to be sold
	The waiting time in a queue
	The number of patients processed during the first hour of the day
	The time that it takes a bus to complete its entire route

Classification	Description
	The number of passengers departing the bus at Dickson street The amount of miles that a truck travel empty during a week

---

Exercise 5.2. *True* or *False* A replication is the generation of one sample path which represents the evolution of the system from its initial conditions to its ending conditions.

Exercise 5.3. In a _____ horizon simulation, a well defined ending time or ending condition can be specified which clearly demarks the end of the simulation.

Exercise 5.4. Which of the following are finite horizon situations? Select all that apply.

a. Bank: bank doors open at 9 am and close at 5 pm
b. Military battle: simulate until force strength reaches a critical value
c. A factory where we are interested in measuring the steady state throughput
d. A hospital emergency room which is open 24 hours a day, 7 days a week

---

Exercise 5.5. Consider a manufacturing system comprising two different machines and two operators. Each operator is assigned to run a single machine. Parts arrive with an exponentially distributed inter-arrival time with a mean of 4 minutes. The arriving parts are one of two types. Sixty percent of the arriving parts are Type 1 and are processed on Machine 1. These parts require the assigned operator for a one-minute setup operation. The remaining 40 percent of the parts are Type 2 parts and are processed on Machine 2. These parts require the assigned operator for a 1.5-minute (constant) setup operation. The service times (excluding the setup time) are lognormally distributed with a mean of 4.5 minutes and a variance of 1 minute for Type 1 parts and a mean of 7.5 minutes and a variance of $(1.5)^2$ minutes for Type 2 parts. The operator of the machine is required to both setup and operate the machine.

Run your model for 20000 minutes, with 10 replications. Use stream 1 for the arrival process, stream 2 for machine 1 processing, stream 3 for machine 2 processing, and stream 4 for the part type determination.

(a) Report the utilization of the operators. In addition, report the total time spent in the system for each type of part.
(b) Perform a warm up analysis on this system.

Exercise 5.6. YBox video game players arrive according to a Poisson process with rate 10 per hour to a two-person station for inspection. The inspection time per YBox set is exponentially distributed with a mean of 10 minutes. On the average 82% of the sets pass inspection. The remaining 18% are routed to an adjustment station with a single operator. Adjustment time per YBox is uniformly distributed between 7 and 14 minutes. After adjustments are made, the units are routed back to the inspection station to be retested. Assume that a part can be adjusted as many times as needed until it passes inspection. Build an simulation model of this system. Use a replication length of 30,000 minutes. Use stream 1

for the arrival process, stream 2 for the inspection time, stream 3 for the adjustment time, and stream 4 to determine if the part needs adjustment.

a. Perform a warm up analysis of the total time a set spends in the system and estimate the system time to within 2 minutes with 95% confidence.

b. Collect statistics to estimate the average number of times a given job is adjusted.

c. Suppose that any one job is not allowed more than two adjustments, after which time the job must be discarded. Modify your simulation model and estimate the number of discarded jobs.

Exercise 5.7. Create a model to simulate observations from a $N(\mu, \sigma^2)$ random variable. Use your simulation to generate two independent samples of size $n_1 = 20$ and $n_2 = 30$ from normal distributions having $\mu_1 = 2$, $\sigma_1^2 = 0.64$ and $\mu_2 = 2.2, \sigma_2^2 = 0.64$. Assume that you don't know the true means and variances. Use the method of independent samples to test whether $\mu_2 > \mu_1$. Use stream 1 for the first sample and stream 2 for the second sample.

Exercise 5.8. Create a model to simulate observations from a $N(\mu, \sigma^2)$ random variable. Use your simulation to generate two independent samples of size $n_1 = 20$ and $n_2 = 30$ from normal distributions having $\mu_1 = 2$, $\sigma_1^2 = 0.64$, and $\mu_2 = 2.2$, $\sigma_2^2 = 0.36$. Assume that you don't know the true means and variances. Use the method of independent samples to test whether $\mu_2 > \mu_1$. Use stream 1 for the first sample and stream 2 for the second sample.

Exercise 5.9. Create a model to simulate observations from a $N(\mu, \sigma^2)$ random variable. Use your simulation to generate two independent samples of size $n_1 = 30$ and $n_2 = 30$ from normal distributions having $\mu_1 = 2$, $\sigma_1^2 = 0.64$ and $\mu_2 = 2.2$, $\sigma_2^2 = 0.36$. Assume that you don't know the true means and variances. Use the paired-t method to test whether $\mu_2 > \mu_1$. Use stream 1 for the first sample and stream 2 for the second sample.

Exercise 5.10. Create a model to simulate observations from a $N(\mu, \sigma^2)$ random variable. Use your simulation to generate two *dependent* samples of size $n_1 = 30$ and $n_2 = 30$ from normal distributions having $\mu_1 = 2$, $\sigma_1^2 = 0.64$ and $\mu_2 = 2.2$, $\sigma_2^2 = 0.36$. Use the method of common random numbers. Assume that you don't know the true means and variances. Use the paired-t method to test whether $\mu_2 > \mu_1$. Use stream 1 for both samples, resetting it to the start of the stream before generating the second sample.

Exercise 5.11. Consider a M/M/1 queueing system with arrival rate of 1 customer per minute and a service rate of 1.2 customers per minute. Use stream 1 for the arrival process and stream 2 for the service process and perform the following:

a. Develop a 95% confidence interval for your estimate of the mean waiting time based on the data from 1 replication of 1000 customers. Discuss why this is inappropriate. How does your simulation estimate compare to the theoretical value?

b. How does your running average track the theoretical value? What would happen if you increased the number of customers?

c. Construct a Welch plot using 5 replications of the 1000 customers. Determine a warm up point for this simulation. Do you think that 1000 customers are enough?

d. Make an autocorrelation plot of the first 1000 customer wait times. What are the assumptions for forming the confidence interval in part (a). Is this data independent and identically distributed? What is the implication of your answer for your confidence interval in part (a)?

e. Use your warm up period from part (c) and generate an additional 1000 customers after the warm up point. Use the method of batch means to batch the 1000 observations into 40 batches of size 25. Make an autocorrelation plot of the 40 batch means. Compute a 95% confidence interval for the mean waiting time using the 40 batches.

f. Use the method of replication deletion to develop a 95% confidence interval for the mean waiting time. Use your warm-up period from part (c). Compare the result with that of (a) and (e) and discuss. What do you recommend for the warmup period?

Exercise 5.12. Reconsider Exercise 4.10 of Chapter 4, with stream 1 for the arrival process, stream 2 for the service process, and stream 3 for the inspection process. Suppose that after the passenger identification inspection, we now need to model a simplified version of the baggage screening process.

Part (a): Once passengers clear the identification check, they proceed to the X-ray baggage screening. Those that were denied, exit the system. For those proceeding to X-ray screening, at a minimum, it takes 1.5 minutes

per passenger for the X-ray process to complete. Typically, this process takes 2.5 minutes. At the most, this process can last 7 minutes. There are two X-ray machines. Use stream 4 for the X-ray screening time.

- What is the maximum time that a passenger had to wait in line for the X-ray machine?
- What is the utilization of the X-ray machines?

Part (b): To increase security measures, a more extensive security check of passengers is performed after the baggage scan. Every 15th passenger will go through a full-body scan and manual baggage review. This inspection usually takes 5 minutes. At the least, it will take 3 minutes and at the most 10 minutes. Assume there is unlimited availability of resources to model this extra security check. Use stream 5 for the full-body scan and manual baggage review inspection.

- What is the overall average cycle time of passengers (from the time they enter the system until they are through all security points) who are selected for this check?

Exercise 5.13. You have been hired to analyze the needs for loading dock facilities at a trucking terminal. The present terminal has 4 docks on the main building. Any trucks that arrive when all docks are full are assigned to a secondary terminal, which is a short distance away from the main terminal. Assume that the arrival process is Poisson with a rate of 5 trucks each hour. There is no available space at the main terminal for trucks to wait for a dock. At the present time nearly two-thirds of the arriving trucks are diverted to the secondary terminal. The average service time per truck is two hours on the main terminal and 3 hours on the secondary terminal, both exponentially distributed. Two proposals are being considered. The first proposal is to expand the main terminal by adding docks so that at least 80% of the arriving trucks can be served

there with the remainder being diverted to the secondary terminal. The second proposal is to expand the space that can accommodate up to 8 trucks. Then, only when the holding area is full will the trucks be diverted to secondary terminal.

Build KSL models to analyze this situation. Run the models for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. For the first proposal, what is the required number of docks so that at least 80% of the arriving trucks can be served?

b. For the second proposal, will the specified number of spaces result in at least 80% of the arriving trucks being served at the main facility?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 5.14. Two machines are being considered for processing a job within a factory. Jobs arrive at the machine according to a Poisson process with a rate of 4.5 jobs per hour. The first machine has an exponentially distributed processing time with a mean of 10 minutes. For the second machine the vendor has indicated that the mean processing time is 10 minutes but with a standard deviation of 6 minutes with a lognormal distribution.

Build KSL models to analyze this situation. Run the models for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, which machine is better in terms of the average waiting time of the jobs?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 5.15. SuperFastCopy wants to install self-service copiers, but cannot decide whether to put in one or two machines. They predict that arrivals will be Poisson with a rate of 30 per hour, and the time spent copying is exponentially distributed with a mean of 1.75 minutes. Because the shop is small they want the probability of 5 or more customers in the shop to be small, say less than 7%.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, make a statistically valid recommendation concerning the number of machines needed to meet the customer service requirement.

Use stream 1 for the time between arrivals and stream 2 for the service times.

---

6. Process View Modeling Using the KSL

Learning Objectives

- To be able to explain the process view perspective and how it compares to the event view
- To be able to model discrete-event dynamic systems using the process view
- To be able to build process view models using the KSL

There are essentially two fundamental viewpoints for modeling a discrete event system within simulation: the event view and the process view. Chapter 4 presented the event view. This chapter will present the process view. These views are simply different representations for the same system. In the event view, the system and its elements are conceptualized as reacting to events. In the process view, the movement of entities through their processes implies the events within the system in the system.

In discrete-event dynamic systems, an event is something that happens at an instant in time which corresponds to a change in system state. An event can be conceptualized as a transmission of information that causes an action resulting in a change in system state. In the event view, we *directly* modeled the system state by identifying the events and specifying the state changes. In the process view, the events will be *implied* by the action of entities within the system. An *entity* is an object of interest that moves through the system. Entities experience processes as they experience the

system. In the simplest sense, a process can be thought of as a sequence of activities, where an activity is an element of the system that takes an interval of time to complete. In the pharmacy example, the service of the customer by the pharmacist was an activity. The representation of the dynamic behavior of the system by describing the process flows of the entities moving through the system is called process-oriented modeling. The modeling perspective based on identifying and describing the system's processes is called the process-view.

> **Note**
>
> This chapter provides a series of example Kotlin code that illustrates the use of KSL constructs for implementing process view simulation models. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

6.1. What are Entities?

When modeling a system, there are often many entity types. For example, consider a retail store. Besides customers, the products might also be considered as entity types. The products (instances of the entity type) are received by the store and wait on the shelves until customers select them for purchase. Entities (entity instances) may come in groups and then are processed individually or they might start out as individual units that are formed into groups. For example, a truck arriving to the store may be an entity that consists of many pallets that contain products. The customers select the products from the shelves and during the check out process the products are placed in bags. The customers then carry their bags to their cars. Entities are uniquely identifiable within the system. If there are two customers in the store, they can be distinguished by the *values* of their attributes. For example, considering a product as an entity type, it

may have attributes *serial number*, *weight*, *category*, and *price*. The set of attributes for a type of entity is called its *attribute set*. While all products might have these attributes, they do not necessarily have the same values for each attribute. For example, consider the following two products:

- (serial number = 12345, weight = 8 ounces, category = green beans, price = \$0.87)
- (serial number = 98765, weight = 8 ounces, category = corn, price = \$1.12)

The products carry or retain these attributes and their values as they move through the system. In other words, attributes are attached to or associated with entity types. The values of the attributes for particular entity instances might change during the operation of the system. For example, a mark down on the price of green beans might occur after some period of time. Attributes can be thought of as variables that are attached to entity types.

Not all information in a system is local to the entity types. For example, the number of customers in the store, the number of carts, and the number of check out lanes are all characteristics of the system. These types of data are called *system attributes*. In simulation models, this information can be modeled with global *variables* or other data modules (e.g. resources) to which all entity instances can have access. By making these quantities visible at the system level, the information can be shared between the entity instances within the model and between the components of the system.

Figure 6.1 illustrates the difference between global (system) variables and entities with their attributes in the context of a warehouse. In the figure, the trucks are entities with attributes: arrival time, type of product, amount of product, and load tracking number. Notice that both of the trucks have these attributes, but each truck has different *values* for their attributes. The figure also illustrates examples of global variables, such as, number of trucks loading, number of trucks unloading, number of busy forklifts, etc. This type of information belongs to the whole system.

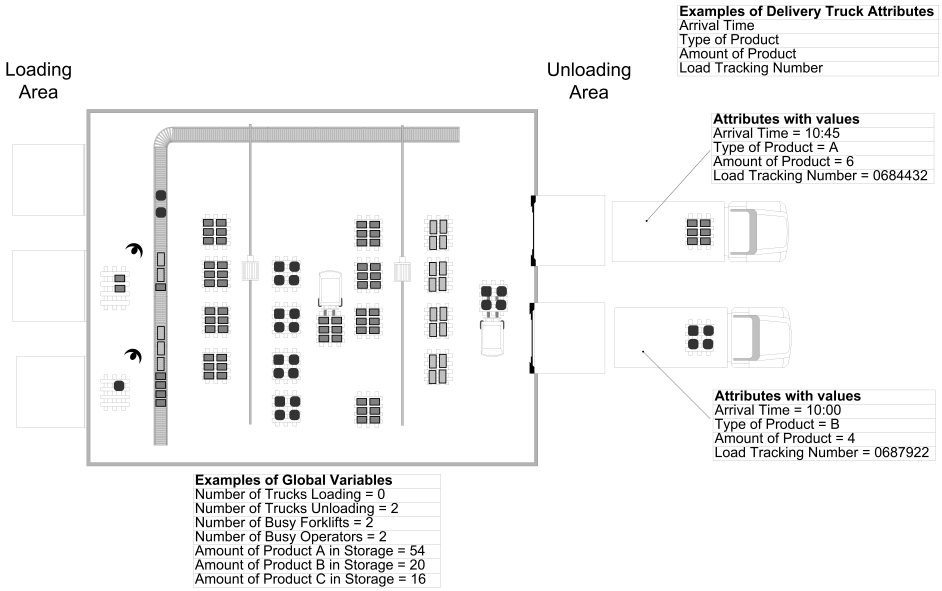


Figure 6.1.: Global Variables and Attributes within a System

Modeling a system based on entities is the basis of object-oriented programming methods. In object-oriented programming, we identify the classes that describe how the object instances interact. In the process-view, we start with the entities (classes) and create (object) instances of the entities. We can have many different types of classes (entity types) that work together to describe the behavior of the overall system.

Once a basic understanding of the system is accomplished through understanding system variables, the entities (classes), and their attributes, you must start to understand the processes within the system. Developing the process description for the various types of entities within a system and connecting the flow of entities to the changes in state variables of the system is the essence of process-oriented modeling.

6.2. The Process View

Before introducing the many of the technical details of processing modeling within the KSL, we will start with a simple example. The example will again be the drive through pharmacy model but in this section it will be implemented using the process modeling constructs available within the KSL.

Recall that we have a small pharmacy that has a single line for waiting customers and only one pharmacist. Assume that customers arrive at a drive through pharmacy window according to a Poisson distribution with a mean of 10 per hour. The time that it takes the pharmacist to serve the customer is random and data has indicated that the time is well modeled with an exponential distribution with a mean of 3 minutes. Customers who arrive to the pharmacy are served in the order of arrival and enough space is available within the parking area of the adjacent grocery store to accommodate any waiting customers.

The drive through pharmacy system can be conceptualized as a single server waiting line system, where the server is the pharmacist. An idealized

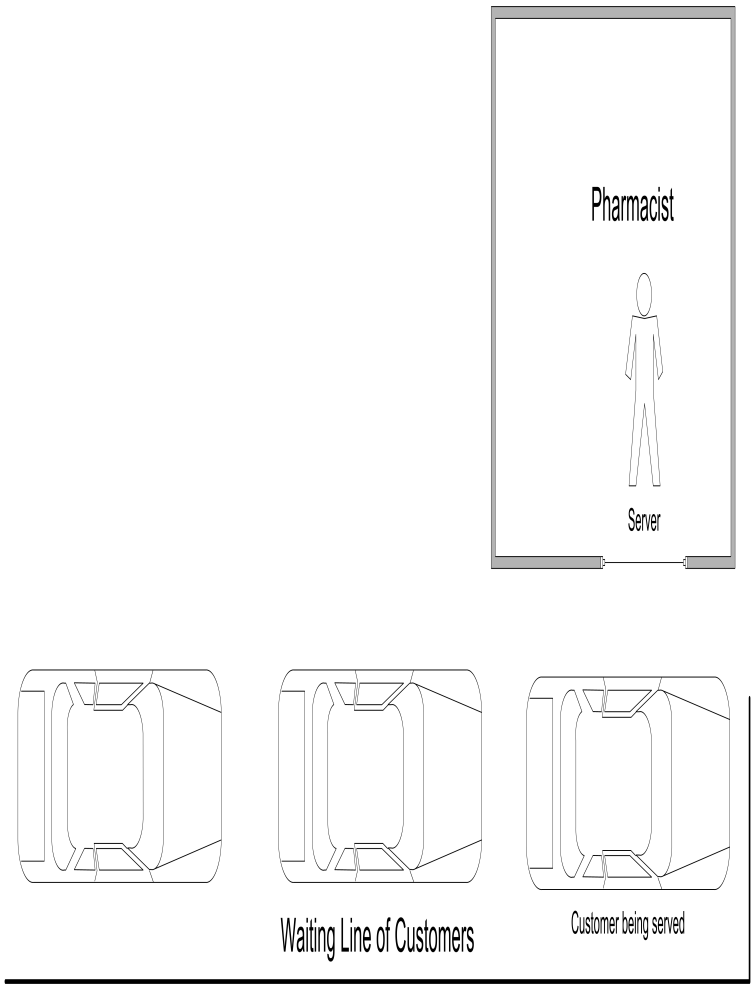


Figure 6.2.: Drive Through Pharmacy

representation of this system is shown in Figure 6.2. In Section 4.4.4 of Chapter 4, we presented an activity diagram for the pharmacy.

The activity diagram is a basic description of the process experienced by the customer. In Chapter 4, we used the activity diagram to identify the arrival and end of service events. In this chapter, we will almost directly translate the activity diagram to KSL code.

Here is the portion of the KSL code to model the process for the customers of the drive through pharmacy:

```
    private inner class Customer : Entity() {
        val pharmacyProcess: KSLProcess = process {
            wip.increment()
            timeStamp = time
            val a = seize(resource = pharmacists)
            delay(delayDuration = serviceTime)
            release(allocation = a)
            timeInSystem.value = time - timeStamp
            wip.decrement()
            numCustomers.increment()
        }
    }
```

The code defines a class called `Customer` which is a subclass of `Entity`. The `Entity` class is a base class that encapsulates the ability to experience processes. Now, notice the definition and initialization of the variable `pharmacyProcess` within the `Customer` class. This is a very special function builder that allows for the definition of a *coroutine* that defines the process for the entity. Coroutines are software constructs that permit the suspension and resumption of programming statements. Kotlin supports the use of coroutines for asynchronous programming. In this situation, the KSL leverages Kotlin's coroutine library to implement the process view.

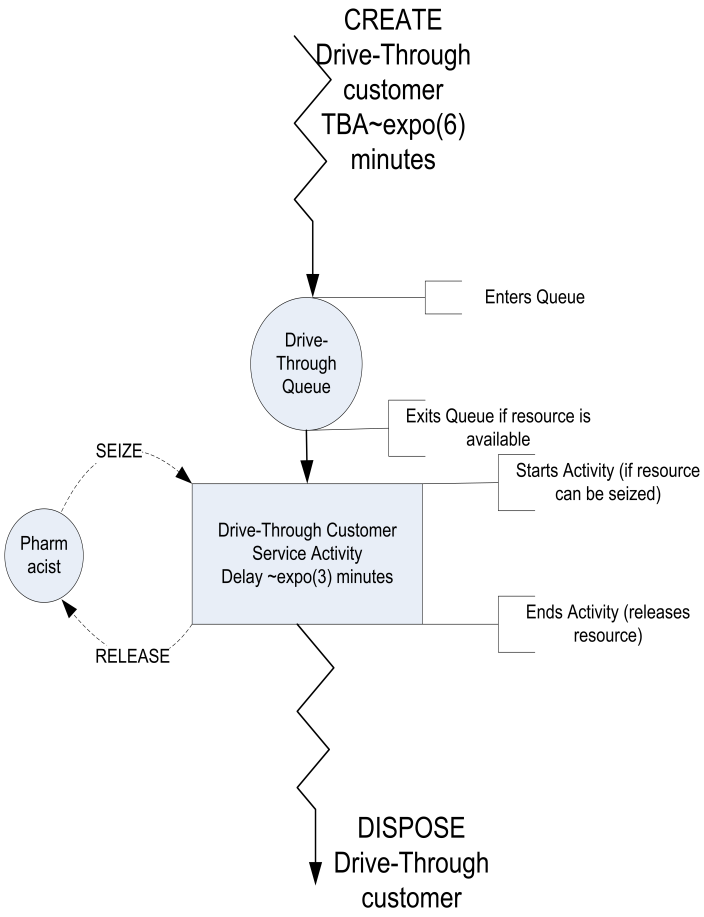


Figure 6.3.: Activity Diagram of Drive Through Pharmacy

Example 6.1 (Process View Model for Drive Through Pharmacy System)**.** This example illustrates how to represent the previously presented drive through pharmacy model of Example 4.4 in Section 4.4.4 using KSL process view constructs.

```
class DriveThroughPharmacy(
    parent: ModelElement,
    numPharmacists: Int = 1,
    name: String? = null
) : ProcessModel(parent, name) {
    init {
        require(numPharmacists > 0) { "The number of
 ↪  pharmacists must be >= 1" }
    }

    private val pharmacists: ResourceWithQ = ResourceWithQ(
        parent = this,
        name = "Pharmacists",
        capacity = numPharmacists
    )

    private val serviceTime: RandomVariable =
     ↪  RandomVariable(parent = this, rSource =
     ↪  ExponentialRV(0.5, 2))
    val serviceRV: RandomVariableCIfc
        get() = serviceTime
    private val timeBetweenArrivals: RandomVariable =
     ↪  RandomVariable(
        parent = parent,
        rSource = ExponentialRV(1.0, 1)
    )
```

```
    val arrivalRV: RandomVariableCIfc
        get() = timeBetweenArrivals
    private val wip: TWResponse = TWResponse(parent = this,
     ↪  name = "${this.name}:NumInSystem")
    val numInSystem: TWResponseCIfc
        get() = wip
    private val timeInSystem: Response = Response(parent =
     ↪  this, name = "${this.name}:TimeInSystem")
    val systemTime: ResponseCIfc
        get() = timeInSystem
    private val numCustomers: Counter = Counter(parent =
     ↪  this, name = "${this.name}:NumServed")
    val numCustomersServed: CounterCIfc
        get() = numCustomers
    private val mySTGT4: IndicatorResponse =
     ↪  IndicatorResponse(
        predicate = { x -> x >= 4.0 },
        observedResponse = timeInSystem,
        name = "SysTime > 4.0 minutes"
    )
    val probSystemTimeGT4Minutes: ResponseCIfc
        get() = mySTGT4

    override fun initialize() {
        schedule(eventAction = this::arrival, timeToEvent =
↪   timeBetweenArrivals)
    }

    private fun arrival(event: KSLEvent<Nothing>) {
        val c = Customer()
        activate(c.pharmacyProcess)
        schedule(eventAction = this::arrival, timeToEvent =
↪   timeBetweenArrivals)
```

```
    }

    private inner class Customer : Entity() {
        val pharmacyProcess: KSLProcess = process {
            wip.increment()
            timeStamp = time
            val a = seize(resource = pharmacists)
            delay(delayDuration = serviceTime)
            release(allocation = a)
            timeInSystem.value = time - timeStamp
            wip.decrement()
            numCustomers.increment()
        }
    }
}
```

Most of this code should look very familiar. It includes the definition of the random variables to model the time between arrivals and service time. It also defines the responses to collect statistics on the performance of the system. In addition, it uses the functional notation for referencing functional interfaces to schedule the arrival events. Within the arrival event, the customer is created and it is told to `activate()` its process.

A key item to notice is that the `DriveThroughPharmacy` class now sub-classes from `ProcessModel` rather than the `ModelElement` class. The `ProcessModel` class is a sub-class of the `ModelElement` class which provides for the ability to model using the process view. While there is much functionality within the `ProcessModel` class, for the purposes of process view modeling, the key item of note is a protected inner class called `Entity`. The `Entity` class provides the ability to define processes for entities. This is done via the `process()` function, which is a special function that enables

the use of suspending functions within process routines. In the drive through pharmacy code, we see the use of the `process` function to define the `pharmacyProcess` property.

The process called `pharmacyProcess` has a nice linear flow and avoids the event call back approach of the event-view. The first line `wip.increment()` simply increments the number of customers in the system. The next line involving `timeStamp` assigns the arrival time of the customer. Then, the customer attempts to seize the pharmacist. If the pharmacist is available, it is allocated to the customer. The variable "a" holds the allocation information. If a pharmacist is not available, the customer is held in a queue related to the invocation of the `seize()` function. The `seize()` function is a *suspending* function. This is the magic of coroutines. The execution of the coroutine literally stops within the `seize()` function if the pharmacist is not available. When the pharmacist becomes available, the customer (and suspended code) resumes moving through the process. The `delay()` function is another *suspending* function. The delay function essentially schedules an event to represent the service time and when the event occurs the coroutine is resumed. The `release()` deallocates the resource from the customer. The final three lines simply collect statistical quantities.

> **Note**
>
> When using the `seize()` function to seize a resource, you are actually making a request for the resource. If the request is not immediately filled, then the request waits in a queue. Since we used a `ResourceWithQ` class to model the resource, the request will wait in the queue associated with the resource. The `seize()` function returns an instance of the `Allocation` class. The returned allocation has properties that indicate which queue and which resource was involved with the request. This is why the `release()` function takes in the allocation. This allows the correct resource to be deallocated and the appropriate queue to be checked when the release occurs.

It is critical to understand that 1) there are many entities created via the `arrival` method, 2) they all experience the same process description, 3) each entity has its own instance of the process, and 4) they may all be at different points of their process instances at different times. Since many customers are active at the same time (in a pseudo-parallelism) they compete for the pharmacist. This causes queueing. This process view depends on shared state. In this example, the primary shared state is via the resource. This is a new construct and was defined with the following line.

```
    private val pharmacists: ResourceWithQ = ResourceWithQ(
        parent = this,
        name = "Pharmacists",
        capacity = numPharmacists
    )
```

A `ResourceWithQ` is a construct that can be used in `ProcessModel` instances. These resources track the number of entities that are allocated and can hold them in a common queue while they wait. The ability to describe processes in this manner is what makes the process view very popular and often serves as the basis for commercial software. The KSL provides this view in open-source code.

The main code to run the model is as follows.

```
fun main(){
    val m = Model()
    val dtp = DriveThroughPharmacy(m, name = "DriveThrough")
    dtp.arrivalRV.initialRandomSource = ExponentialRV(6.0, 1)
    dtp.serviceRV.initialRandomSource = ExponentialRV(3.0, 2)
    m.numberOfReplications = 30
    m.lengthOfReplication = 20000.0
    m.lengthOfReplicationWarmUp = 5000.0
```

```
    m.simulate()
    m.print()
}
```

Notice how the specification of the arrival and service random variables have been provided by setting the initial random sources of the underlying random variables. If we run the code, we get the following results.

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                                        Count        Average      Half-
Width
-------------------------------------------------------------------------------
NumBusy                                        30         0.5035         0.0060
# in System                                    30         1.0060         0.0271
System Time                                    30         6.0001         0.1441
PharmacyQ:NumInQ                               30         0.5025         0.0222
PharmacyQ:TimeInQ                              30         2.9961         0.1235
SysTime > 4.0 minutes                          30         0.5136         0.0071
Num Served                                     30      2513.2667        17.6883
-------------------------------------------------------------------------------
```

If you look back at the previous results, you will see that these results are exactly the same! Thus, we can model the pharmacy with either the event view or the process view with confidence.

Before discussing additional functionality enabled within the `KSLProcessBuilder`, we present some functionality that facilitates the creation and activation of entities. Notice that within the pharmacy model that after an customer is created to start its process we must schedule its activation.

```
    private fun arrival(event: KSLEvent<Nothing>) {
        val c = Customer()
        activate(c.pharmacyProcess)
        schedule(eventAction = this::arrival, timeToEvent =
↪   timeBetweenArrivals)
    }
```

This is so common the KSL provides a class called `EntityGenerator` that automates this process. The `EntityGenerator` class subclasses from `EventGenerator` and allows for a creation pattern to be specified. Figure 6.4 illustrates this relationship.

Figure 6.4.: EntityGenerator and EventGenerator Classes

The `EntityGenerator` class is defined as an inner class of `ProcessModel`. Reviewing its code is useful.

```
    protected inner class EntityGenerator<T : Entity>(
        private val entityCreator: () -> T,
        timeUntilTheFirstEntity: RVariableIfc,
        timeBtwEvents: RVariableIfc,
        maxNumberOfEvents: Long = Long.MAX_VALUE,
        timeOfTheLastEvent: Double =
         ↪  Double.POSITIVE_INFINITY,
        var activationPriority: Int =
         ↪  KSLEvent.DEFAULT_PRIORITY + 1,
        name: String? = null
    ) : EventGenerator(
        this@ProcessModel, null, timeUntilTheFirstEntity,
        timeBtwEvents, maxNumberOfEvents, timeOfTheLastEvent,
         ↪  name
    ) {

        private val myEntityGeneratorAction =
         ↪  GeneratorActionIfc { generate() }

        override fun generate() {
            val entity = entityCreator()
            require(entity.defaultProcess != null) { "There
↪   was no default process specified for the entity. Ensure
↪   that the `defaultProcess` property is set via the
↪   process() function or directly." }
            activate(entity.defaultProcess!!, priority =
↪   activationPriority)
        }

    }
```

As shown in the code, the key additional functionality is that the `EntityGenerator` takes in a *function* that knows how to create a subclass

of type `Entity`. The simplest way to provide such a function is to provide a reference to the constructor function of the subclass. This is illustrated in the following simplified re-do of the pharmacy model.

Example 6.2 (Demonstrating an Entity Generator)**.** This example illustrates how to construct an instance of the `EntityGenerator` class and use it to create entity instances according to a time between event pattern.

```
class EntityGeneratorExample(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {

    private val worker: ResourceWithQ = ResourceWithQ(parent
     ↪  = this, name = "${this.name}:Worker")
    private val st = RandomVariable(parent = this, rSource =
     ↪  ExponentialRV(3.0, 2))
    private val wip = TWResponse(parent = this, name =
     ↪  "${this.name}:WIP")
    private val tip = Response(parent = this, name =
     ↪  "${this.name}:TimeInSystem")
    private val tba = ExponentialRV(6.0, 1)
    private val generator = EntityGenerator(
        entityCreator = ::Customer,
        timeUntilTheFirstEntity = tba,
        timeBtwEvents = tba
    )
    private val counter = Counter(parent = this, name =
     ↪  "${this.name}:NumServed")

    private inner class Customer : Entity() {
```

```
        val pharmacyProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            wip.increment()
            timeStamp = time
            val a = seize(worker)
            delay(st)
            release(a)
            tip.value = time - timeStamp
            wip.decrement()
            counter.increment()
        }
    }
}
```

Notice the following line which takes in a reference to the `Customer` class constructor using the functional syntax `::Customer.` In addition, notice the specification of the `isDefaultProcess` argument for the `process()` function. Setting the `isDefaultProcess` argument to true indicates that the defined process routine will be the default process to be activated when the `EntityGenerator` causes the generation event to occur.

```
    private val generator = EntityGenerator(
        entityCreator = ::Customer,
        timeUntilTheFirstEntity = tba,
        timeBtwEvents = tba
    )
```

There is no arrival method necessary because that logic is within the entity generator's `generate()` method:

```
        override fun generate() {
            val entity = entityCreator()
            require(entity.defaultProcess != null) {"There
↪   was no initial process specified for the entity"}
            activate(entity.defaultProcess!!, priority =
↪   activationPriority)
        }
```

The entity is created using the passed in constructor function and the entity's default process is started. By default, the `process()` function of the `KSLProcessBuilder` class automatically adds each newly defined *named* process to the entity's map of named processes. If the `name` argument is not supplied, then the defined process is not added to the process map. The map of named processes for an entity can be accessed via the `processes` property of the `Entity` class.

To specify the default process, provide the `isDefaultProcess` argument of the `process()` function to be true. Thus, by using an instance of an `EntityGenerator` associated with a particular subclass of `Entity`, we can automatically create the instances of the subclass and activate their processes. Of course, the creation of the subclass (e.g. `Customer`) might be much more complex; however, the `EntityGenerator` takes in a *function* that creates the instance of the subclass. Thus, it can be any function, not just the constructor function. Therefore, instances can be configured in complex ways before they are activated by supplying an appropriate function.

> **!** Important
>
> Note that an `EntityGenerator` relies on the entity having a default process defined via the `process()` method. An entity generator will create the entity and start the process that is defined as the default process. Rather than use the `process()` function, you can directly

> specify the default process via the `defaultProcess` property of the `Entity` class. If you try to use an `EntityGenerator` without a defined default process, then an illegal state exception will occur.

In the next section, we will take a closer look at how the KSL makes the process view possible. This will help you to better use the process modeling capabilities found in the KSL.

6.3. Understanding KSL Processes and Entities

Entities can experience many processes. Thus, there needs to be a mechanism to activate the processes and to cleanup after the processes have completed. The `ProcessModel` class facilitates the modeling of entities experiencing processes. A `ProcessModel` has inner classes (`Entity`, `EntityGenerator`, etc.) that can be used to describe entities and the processes that they experience.

As noted in Figure 6.5 a `ProcessModel` can activate KSL processes, can start entity process sequences, can dispose of entities, and will perform some after replication cleanup. One of the activities that a process model must perform is to terminate any processes that are still suspended after the replication is completed. In addition, it ensures that no entity terminates its process while still having allocations to a resource. Thus, a `ProcessModel` is needed to manage the entities and processes that it manages. This is why in the pharmacy code example, the pharmacy model is a subclass of `ProcessModel`.

```
class DriveThroughPharmacy(
    parent: ModelElement,
    numPharmacists: Int = 1,
    name: String? = null
) : ProcessModel(parent, name)
```

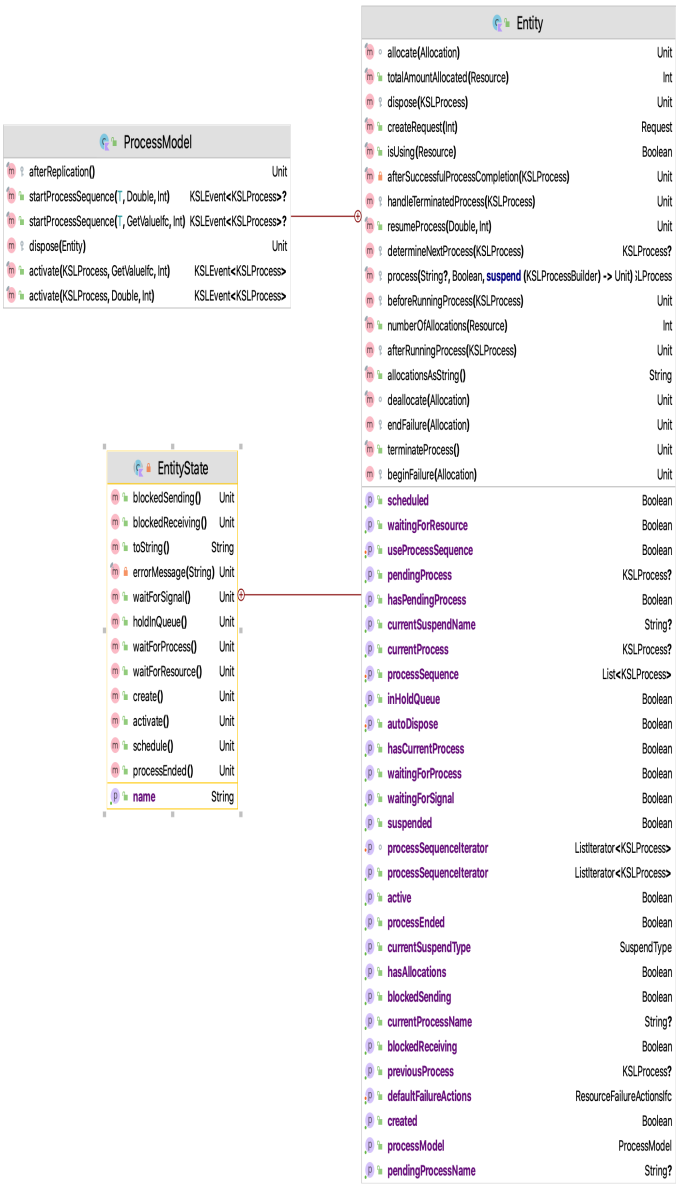


Figure 6.5.: Overview of the ProcessModel Class

This provides the modeler with access to the inner classes e.g. `Entity` that are inherited by the subclass for use in process modeling.

The key inner class is `Entity,` which has a function `process()` that uses a builder to describe the entity's process in the form of a *coroutine.* An entity can have many processes described and it may follow specified processes based on different modeling logic. A process model facilitates the running of a sequence of processes that are stored in an entity's `processSequence` property. An entity can **experience only one process at a time**. After completing the process, the entity will try to use its sequence to run the next process (if available). Individual processes can be activated for specific entities. But, again, an entity instance may only be activated to experience *one* process at a time, even if it has many defined processes. The entity experiences processes *sequentially.* An error will occur if you try to activate and run a process for a given entity instance if the entity is already executing a process . Note that this does not in anyway limit the creation and activation of *different* entity instances and their processes.

An `Entity` instance is something that can experience processes and as such may wait in queues. `Entity` is a subclass of `QObject.` Thus, statistics can be collected automatically on entities, if they experience waiting. The general approach to defining a process for an entity is to use the `process()` function to define a process that a subclass of Entity can follow. Entity instances may use resources, signals, hold queues, etc. as shared mutable state.

Entities may follow a process sequence if defined. An entity can have many properties that define different processes that it might experience. The user can store the processes in data structures. In fact, there is a `processSequence` property for this purpose that defines a list of processes that the entity will follow. However, the user is responsible for adding processes to the sequence. In addition, if you provide a name for the process when defining it via the `process()` function, then the processes are stored in the entity instance's `processes` map, which can be accessed via the provided name.

The following code defines a process and assigns the function to the property `pharmacyProcess`. This property is of type `KSLProcess`. Because the `isDefaultProcess` argument was supplied to the `process()` function, the process is defined as the default process to activate by entity generators. Each process can also be provided a string name via an argument of the `process()` function. The name of a process may also be useful in tracing and debugging process code.

```
    private inner class Customer : Entity() {
        val pharmacyProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
        ...
        }
```

`KSLProcess` is an interface that provides a limited view of a KSL process instance. The interface allows the user to check the state of the process, get its identification (id, and name) and access the entity that is associated with the process. Underneath, the instance of a `KSLProcess` is mapped onto a specially constructed Kotlin coroutine.

```
interface KSLProcess {
    val id: Int
    val name: String
    val isCreated: Boolean
    val isSuspended: Boolean
    val isTerminated: Boolean
    val isCompleted: Boolean
    val isRunning: Boolean
    val isActivated: Boolean
    val currentStateName: String
    val entity: ProcessModel.Entity
    /**
     *  The simulation time that the process first started
 ↪  (activated) after being created.
```

```
     *  If the process was never started, this returns
↪  Double.NaN
     */
    val processStartTime: Double

    /**
     *  The simulation time that the process completed.
     *  If the process was never completed, this returns
↪  Double.NaN
     */
    val processCompletionTime: Double

    /**
     *  The elapsed time from start to completion.
     */
    val processElapseTime: Double
        get() = processCompletionTime - processStartTime
}
```

The `process()` function is a special builder function related to a `KSLProcessBuilder`. A `KSLProcessBuilder` provides the functionality for describing a process. A process is an instance of a coroutine that can be suspended and resumed. The methods of the `KSLProcessBuilder` are the suspending functions that are allowed within the process modeling paradigm of the KSL. The various suspending functions have an optional string name parameter to identify the name of the suspension point. While not required for basic modeling, identifying the suspension point can be useful for more advanced modeling involving the cancellation or interrupting of a process. A unique name can be used to determine which suspension point is suspending the process when the process has many suspension points. We will examine the functionality of a `KSLProcessBuilder` later in this section.

First and foremost, process modeling starts with understanding and using instances of the `Entity` class.

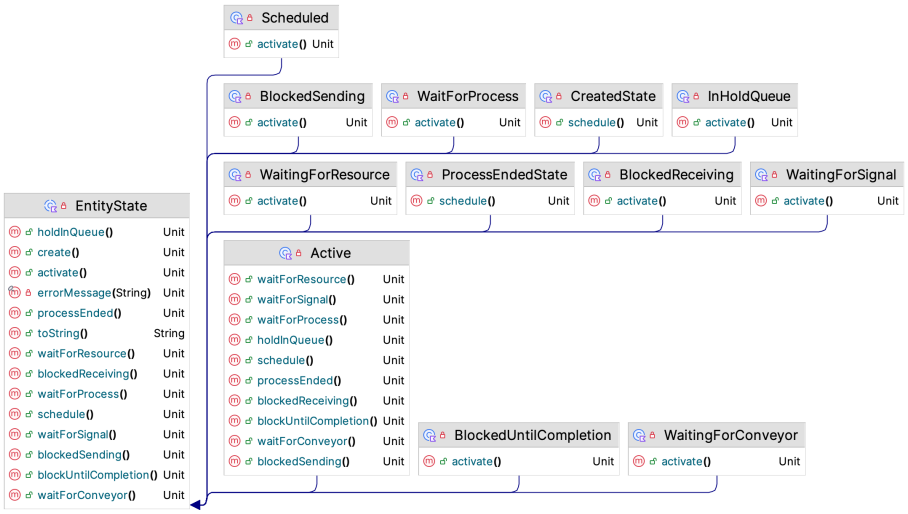


Figure 6.6.: Defined Entity States

Figure 6.5 indicates that there is an inner class called `EntityState` within an Entity. As seen in Figure 6.6, `EntityState` serves as the base class for defining the legal states for an entity and the legal state transitions using the state pattern as show in Figure 6.7.

- `CreatedState`: The entity is placed into this state when it is created. From the created state, the entity can be scheduled to be active.
- `Scheduled`: The scheduled state indicates that the entity is associated with an event that is pending to occur in the event calendar. The entity is scheduled to be active at some future time. The underlying process associated with the entity is suspended.
- `Active`: This is the state that indicates that the entity is executing an underlying process. It is executing non-suspending code within its process. It is the *active* entity. From the active state, the entity

can be suspended for a number of reasons, each mapped to various states.

- `WaitingForResource`: This state indicates that the entity's process is suspended because the entity is waiting for units of a resource.
- `WaitingForSignal`: This state indicates that the entity's process is suspended because the entity is waiting on an arbitrary signal to be sent.
- `BlockedSending`: This state indicates that the entity's process is suspended because the entity is trying to send an item via a shared blocking queue and there is not space for the item in the queue. Blocking queues can be used to communicate between processes.
- `BlockedReceiving`: This state indicates that the entity's process is suspended because the entity is trying to receive items from a blocking queue and there are no items to receive. This is the other end of the communication channel formed by a blocking queue.
- `InHoldQueue`: This state indicates that the entity's process is suspended because the entity is an arbitrary queue that holds entities until they are removed and re-activated.
- `WaitForProcess`: An entity may activate another process. This state indicates that the entity's process is suspended because the entity is waiting for the process to complete before proceeding.
- `BlockedUntilCompletion`: An entity may activate another process that has blockages. This state indicates that the entity's process is suspended because the entity is waiting for the blockage to complete.

The entity states are mapped onto the lower level coroutine via an internal inner class (`ProcessCoroutine`). This class defines the legal states of the coroutine shown in Figure 6.8.

Again, these states define the legal states of the underlying process coroutine. In general, users will not care about these internal details, but a basic understanding of what are legal transitions, shown in Figure 6.9 can be helpful. The following are the process coroutine states:

- Created: The process coroutine is placed in this state when it is

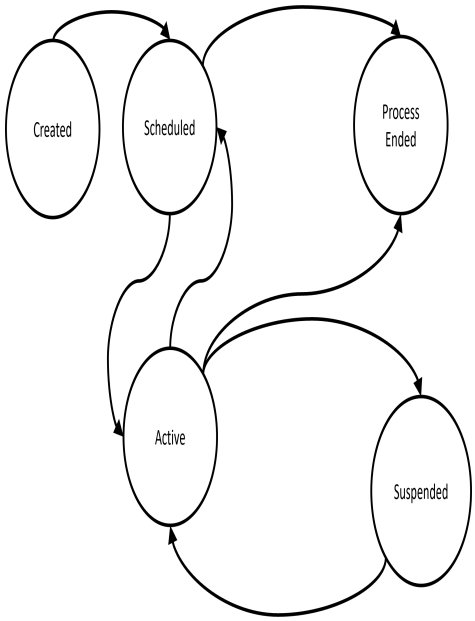


Figure 6.7.: Legal Entity State Transitions

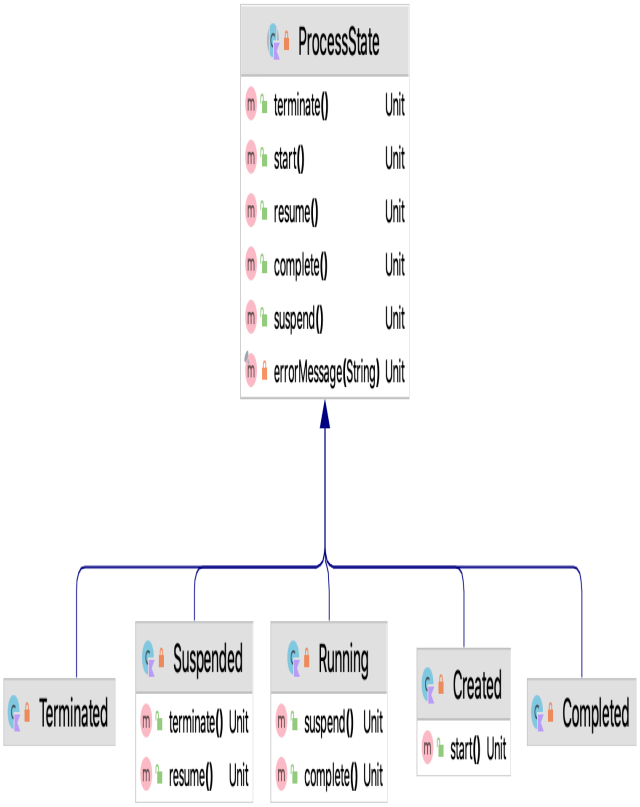


Figure 6.8.: Defined Process States

instantiated.

- Running: The coroutine is running after it is started. This occurs when the entity activates the associated process. Processes are started by scheduling an event that invokes the coroutine code at the appropriate simulated time. The entity moves through its process and when the entity is between suspension points the process is considered to be in the running state.
- Suspended: The underlying process coroutine is suspended using Kotlin's coroutine suspension functionality. Suspension is mapped to the various suspension states associated with an entity.
- Completed: The process coroutine has exited normally from the process routine or reached the end of the process routine. Once completed the coroutine is finished.
- Terminated: The process coroutine has exited abnormally via an error or exception or the user has directly terminated the process. Once terminated the coroutine is finished.

When a process coroutine exits normally the process coroutine is placed in the completed state. Then, the `ProcessModel` checks if the entity has a next process to start. This check occurs via the `determineNextProcess()` function of the entity. If the entity has been specified to use its process sequence via its `useProcessSequence` property, then the process sequence will be used to determine the next process to activate. If the entity is not using its process sequence, then the entity will not automatically start the next process. The user can also override the `determineNextProcess()` function and directly determine the next process to start. The default behavior is to not follow a process sequence such that no new process will be started (automatically).

If the entity is executing a process and the process is suspended, then the process routine may be terminated. This causes the currently suspended process to exit, essentially with an error condition. No further programming statements within the process coroutine will execute. The process ends (placed in the terminated state). All resources that the entity has allocated

will be deallocated. If the entity was waiting in a queue, the entity is removed from the queue and no statistics are collected on its queueing. If the entity is experiencing a delay, then the event associated with the delay is canceled. If the entity has additional processes in its process sequence they are not automatically executed. If the user requires specific behavior to occur for the entity after termination, then the user should override the Entity's `handleTerminatedProcess()` function to supply specific logic. Termination happens immediately, with no time delay.

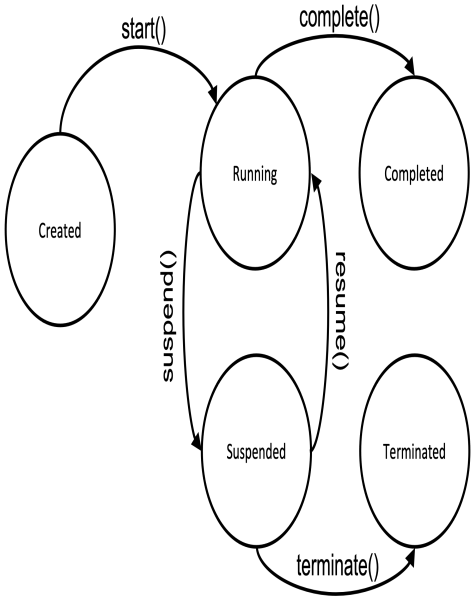


Figure 6.9.: Legal Process States Transitions

The following section will illustrate through some simple models some of the functionality enabled by `KSLProcessBuilder.`

6.4. Examples of Process Modeling

The following example illustrates how to use a hold queue via the `HoldQueue` class. A hold queue holds an entity within a queue until it is removed. It is important to note that the entity that goes into the hold queue cannot remove itself. Thus, as you will see in the following code, we schedule an event that causes the entities to be removed at a specific time. The hold queue is created and an event action defined to represent the event. The entity process is simple, when the entity enters the process it immediately enters the hold queue. The process will be suspended. After being removed and resumed, the entity continues through a couple of delays.

6.4.1. Holding Entities in a Queue: `HoldQueue` Class

Example 6.3 (Illustrating a HoldQueue)**.** This example illustrates how to create an instance of the `HoldQueue` class and how to use it to hold entities until they can be released. An event is scheduled to cause held entities to be removed and to resume their processing.

```
class HoldQExample(parent: ModelElement) :
↪  ProcessModel(parent, null)  {
    private val myHoldQueue: HoldQueue = HoldQueue(this,
    ↪  "hold")
    private val myEventActionOne: EventActionOne =
    ↪  EventActionOne()

    private inner class Customer: Entity() {
        val holdProcess : KSLProcess = process() {
            println("time = $time : before being held
↪  customer = ${this@Customer.name}")
```

```
            hold(myHoldQueue)
            println("time = $time : after being held customer
↪   = ${this@Customer.name}")
            delay(10.0)
            println("time = $time after the first delay for
↪   customer = ${this@Customer.name}")
            delay(20.0)
            println("time = $time after the second delay for
↪   customer = ${this@Customer.name}")
        }
    }

    override fun initialize() {
        val e = Customer()
        activate(e.holdProcess)
        val c = Customer()
        activate(c.holdProcess, 1.0)
        schedule(myEventActionOne, 5.0)
    }

    private inner class EventActionOne :
     ↪  EventAction<Nothing>() {
        override fun action(event: KSLEvent<Nothing>) {
            println("Removing and resuming held entities at
↪   time : $time")
            myHoldQueue.removeAllAndResume()
        }
    }
}
```

The `initialize()` method creates a couple of entities and activates their

hold process. In addition, the event representing the release from the hold queue is scheduled for time 5.0. In the event logic, we see that the hold queue instance is used to call its `removeAllAndResume()` function. The `HoldQueue` class is a subclass of the `Queue` class. Thus, statistics are collected when it is used. In addition, it can be iterated and searched. If a reference to a particular entity is available, then the `removeAndResume()` function can be used to remove and resume a specific entity. These two methods automatically resume the entity's process. Other methods inherited from the `Queue` class allows for the entities to be removed *without* resuming their processes. It is then the responsibility of the user to properly resume the suspended processes by directly using the entity instance. There are also methods for terminating the held entity's processes. The output from this simple simulation is as follows:

```
time = 0.0 : before being held customer = ID_1
time = 1.0 : before being held customer = ID_2
Removing and resuming held entities at time : 5.0
time = 5.0 : after being held customer = ID_1
time = 5.0 : after being held customer = ID_2
time = 15.0 after the first delay for customer = ID_1
time = 15.0 after the first delay for customer = ID_2
time = 35.0 after the second delay for customer = ID_1
time = 35.0 after the second delay for customer = ID_2
```

We see that the two customers are held in the queue right after activation. Then, the event at time 5.0 occurs which removes and resumes the held entities. The rest of the output indicates the entities continue their processes.

6.4.2. Signaling Entities

The next example illustrates the use of the `Signal` class, which builds off of the `HoldQueue` class, as shown in Figure 6.10.

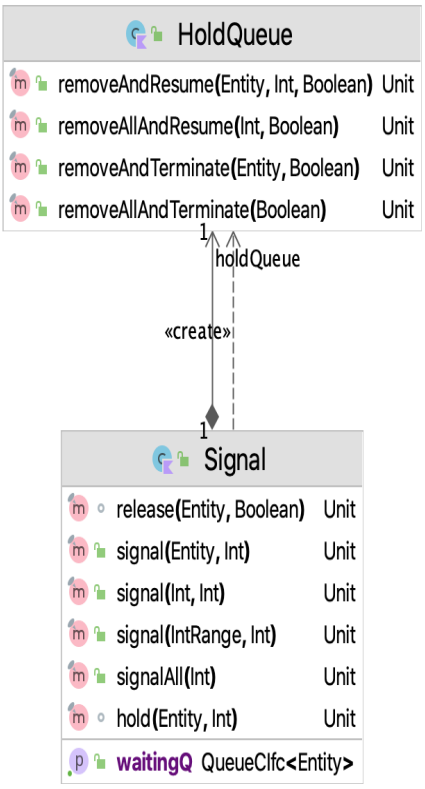


Figure 6.10.: HoldQueue and Signal Classes

The `Signal` class uses an instance of the `HoldQueue` class to hold entities until they are notified to move via the index of their rank in the queue. If you want the first entity to be signaled, then you call `signal(0)`. The entity is notified that its suspension is over and it removes itself from the hold queue. Thus, contrary to the `HoldQueue` class the user does not have to remove and resume the corresponding entity.

Here is some example code. Notice that the code subclasses from the `ProcessModel` class. All implementations that use the process modeling constructs must subclass from the `ProcessModel` class. Then, the instance of the `Signal` class is created. An inner class implements the process description and represents the entity's process that uses the signal. In the process, the entity immediately waits for the signal. After the signal, the entity has a simple delay and then the process ends.

Example 6.4 (Illustrating how to Hold and Signal Entities)**.** This example illustrates how to use the `Signal` class to hold entities until a signal is sent. Once the signal is received the entity continues its processing.

```
fun main(){
    val m = Model()
    SignalExample(m)
    m.numberOfReplications = 1
    m.lengthOfReplication = 50.0
    m.simulate()
    m.print()
}

class SignalExample(parent: ModelElement, name: String? =
 ↪  null) : ProcessModel(parent, name) {
```

```
    private val signal = Signal(this, "SignalExample")

    private inner class SignaledEntity : Entity() {
        val waitForSignalProcess: KSLProcess = process {
            println("$time > before waiting for the signal:
↪   ${this@SignaledEntity.name}")
            waitFor(signal)
            println("$time > after getting the signal:
↪   ${this@SignaledEntity.name}")
            delay(5.0)
            println("$time > after the second delay for
↪   entity: ${this@SignaledEntity.name}")
            println("$time > exiting the process of entity:
↪   ${this@SignaledEntity.name}")
        }
    }

    override fun initialize() {
        for (i in 1..10){
            activate(SignaledEntity().waitForSignalProcess)
        }
        schedule(this::signalEvent, 3.0)
    }

    private fun signalEvent(event: KSLEvent<Nothing>){
        signal.signal(0..4)
    }
}
```

The `initialize()` method creates 10 instances of the `SignalEntity` subclass of the `Entity` class and activates each entity's `waitForSignalProcess`

process. It also schedules an event to cause the signal to occur at time 3.0. In the signal event, the reference to the `Signal` class instance, `signal,` is used to call the `signal()` method of the `Signal` class. The first 5 waiting entities are signaled using a Kotlin range. The output from the process is as follows.

```
0.0 > before waiting for the signal: ID_1
0.0 > before waiting for the signal: ID_2
0.0 > before waiting for the signal: ID_3
0.0 > before waiting for the signal: ID_4
0.0 > before waiting for the signal: ID_5
0.0 > before waiting for the signal: ID_6
0.0 > before waiting for the signal: ID_7
0.0 > before waiting for the signal: ID_8
0.0 > before waiting for the signal: ID_9
0.0 > before waiting for the signal: ID_10
3.0 > signaling the entities in range 0..4
3.0 > after getting the signal: ID_1
3.0 > after getting the signal: ID_2
3.0 > after getting the signal: ID_3
3.0 > after getting the signal: ID_4
3.0 > after getting the signal: ID_5
8.0 > after the second delay for entity: ID_1
8.0 > exiting the process of entity: ID_1
8.0 > after the second delay for entity: ID_2
8.0 > exiting the process of entity: ID_2
8.0 > after the second delay for entity: ID_3
8.0 > exiting the process of entity: ID_3
8.0 > after the second delay for entity: ID_4
8.0 > exiting the process of entity: ID_4
8.0 > after the second delay for entity: ID_5
8.0 > exiting the process of entity: ID_5
```

We see that at time 0.0, the 10 entities are created and their process

activated so that they wait for the signal. Then, at time 3.0, the signal occurs and each of the signaled entities (in turn) resume their processes. Eventually, they complete their process after the 5.0 time unit delay. Notice that since the simulation run length was 50.0 time units, the simulation continues until that time. However, since there are no more signals, the last 5 entities remain waiting (suspended) at the end of the simulation. As previously mentioned, the `ProcessModel` class is responsible for removing these entities that are suspended after the replication has completed. Thus, when the next replication starts, there will not be 5 entities still waiting. The KSL takes care of these common clean up actions automatically.

6.4.3. Understanding Blocking Queues

This next example illustrates the use of a blocking queue. Blocking queues are often used in asynchronous programs to communicated between different threads. In the case of KSL process models, we can use the concept of blocking queues to assist with communication between two processes.

Blocking queues can block on sending or on receiving. The typical use is to block when trying to dequeue an item from the queue when the queue is empty or if you try to enqueue an item and the queue is full. A process trying to dequeue from an empty queue is blocked until some other process inserts an item into the queue. A process trying to enqueue an item in a full queue is blocked until some other process makes space in the queue, either by dequeuing one or more items or clearing the queue completely. The KSL provides a class called `BlockingQueue` that facilitates this kind of modeling. In the case of the KSL, depending on the configuration of the queue, both the sender and the receiver may block.

There are actually three queues used in the implementation of the `BlockingQueue` class. One queue called the `senderQ` will hold entities that place items into the blocking queue if the queue is full. Another queue, called the `receiverQ,` will hold entities that are waiting for items to be placed in the queue to be removed. Lastly, is the blocking queue, itself,

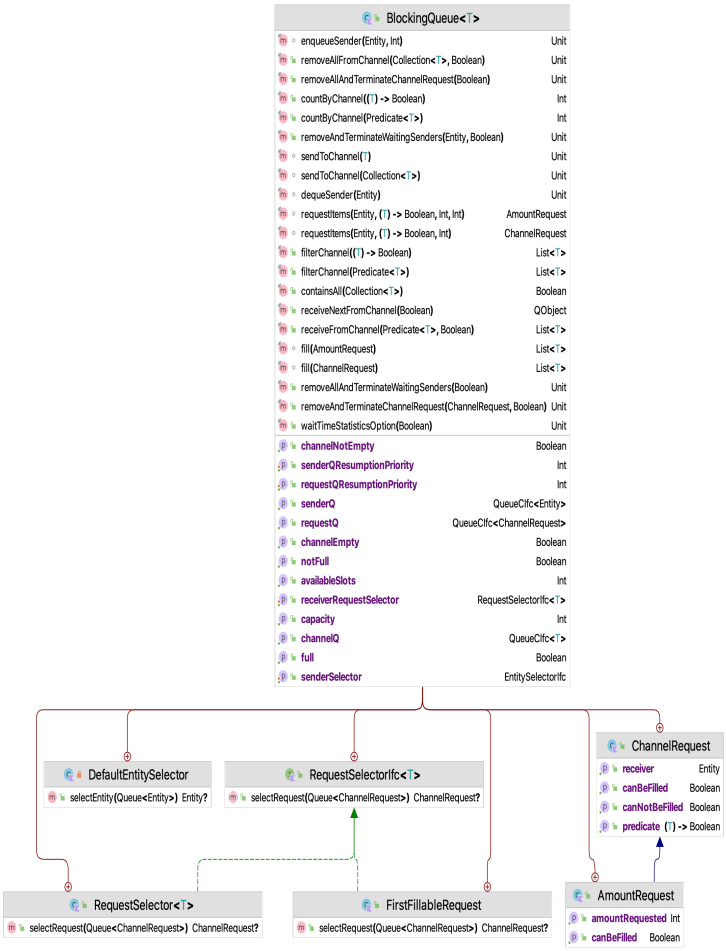


Figure 6.11.: The BlockingQueue Class

which is better conceptualized as a channel between the sender and the receiver. This queue is called the `channelQ` and holds items that are placed into it for eventual removal. Statistics can be collected (or not) on any of these queues. The sender and receiver queues are essentially instances of the KSL `Queue` class. As such, they have no capacity limitation. The channel queue can have a capacity. If a capacity is not provided, it is `Int.MAX_VALUE`, which is essentially infinite.

As noted in Figure 6.11 we see that the items within the queue are requested from the receiver. These requests can have a specific amount. The receiver will need to wait until the specific amount of their request becomes available in the queue (channel). Users can provide a selection rule for the requests to determine which requests will be selected for filling when new items arrive within the channel. The default request selection rule is to select the next request. The following code illustrates the basic creation and use of a blocking queue.

Example 6.5 (Illustrating a Blocking Queue)**.** This example illustrates how to use a blocking queue. A blocking queue can cause an entity to wait until an item is available in the channel or cause an entity to wait until there is space in the channel.

```
class BlockingQExample(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {

//    val blockingQ: BlockingQueue<QObject> =
 ↪  BlockingQueue(this)
    val blockingQ: BlockingQueue<QObject> =
     ↪  BlockingQueue(this, capacity = 10)

    init {
//        blockingQ.waitTimeStatisticsOption(false)
```

```
    }
```

You can create a blocking queue with a capacity (or not) and you can specify whether the statistics are collected (or not) as noted by the commented code of the example. In this example, the queue (channel) has a capacity of 10 items.

In the following code, we implement the process for receiving items from the blocking queue. Notice that there is a loop within the process. This illustrates that all normal Kotlin control structures are available within process routines. The entity (receiver) will loop through this process 15 time and essentially wait for single item to be placed in the blocking queue. If the entity hits the `waitForItems()` suspending function call and there is an item in the queue, then it immediately receives the item and continues with its process. If an item is not available in the blocking queue when the entity reaches the `waitForItems()` suspending function, then the requesting entity will wait until an item becomes available and will not proceed until it receives the requested amount of items. After receiving its requested items, the entity continues with its process.

```
    private inner class Receiver: Entity() {
        val receiving : KSLProcess = process("receiving") {
            for (i in 1..15) {
                println("$time > before the first delay for
↪   entity: ${this@Receiver.name}")
                delay(1.0)
                println("$time > trying to get item for
↪   entity: ${this@Receiver.name}")
                waitForItems(blockingQ, 1)
                println("$time > after getting item for
↪   entity: ${this@Receiver.name}")
                delay(5.0)
                println("$time > after the second delay in
↪   ${this@Receiver.name}")
```

```
            }
            println("$time > exiting the process in
↪   ${this@Receiver.name}")
        }
    }
```

In this code snippet, the entity (sender), also loops 15 times. Each time within the loop, the entity creates an instance of a `QObject` and places it in the blocking queue via the `send()` method. Since the blocking queue has capacity 10, and the time between requests by the receive is a little longer than the time between sends, the blocking queue will reach its capacity. If it does reach its capacity, then the sender will have to wait (blocking) at the `send()` suspending function until space becomes available in the channel.

```
    private inner class Sender: Entity() {
        val sending : KSLProcess = process("sending") {
            for (i in 1..15){
                delay(5.0)
                println("$time > after the first delay for
↪   sender ${this@Sender.name}")
                val item = QObject()
                println("$time > before sending an item from
↪   sender ${this@Sender.name}")
                send(item, blockingQ)
                println("$time > after sending an item from
↪   sender ${this@Sender.name}")
            }
            println("$time > exiting the process for sender
↪   ${this@Sender.name}")
        }
    }
```

```
    override fun initialize() {
        val r = Receiver()
        activate(r.receiving)
        val s = Sender()
        activate(s.sending)
    }

}
```

In the `intialize()` method, single instances of the receiver and sender are created and their processes activated. The model is setup to run for 100 time units

```
fun main(){
    val m = Model()
    val test = BlockingQExample(m)

    m.lengthOfReplication = 100.0
    m.numberOfReplications = 1
    m.simulate()
    m.print()
}
```

The output of the simulation is illustrative of the coordination that occurs between the receiver and the sender. We see in this output that the receiving entity blocks at time 1.0. Finally, at time 5.0 the sender sends an item and it is received by the receiving entity. Both processes continue.

```
0.0 > before the first delay for receiving entity: ID_1
1.0 > trying to get item for receiving entity: ID_1
```

```
5.0 > after the first delay for sender ID_2
5.0 > before sending an item from sender ID_2
5.0 > after sending an item from sender ID_2
5.0 > after getting item for receiving entity: ID_1
10.0 > after the first delay for sender ID_2
10.0 > before sending an item from sender ID_2
10.0 > after sending an item from sender ID_2
10.0 > after the second delay for receiving entity: ID_1
10.0 > before the first delay for receiving entity: ID_1
.
.
.
```

The queueing statistics indicate that the sender never blocked and the receiver had a small amount of blocking.

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                                                  Count          Average      Half-
Width
-------------------------------------------------------------------------------------
BlockingQueue_4:SenderQ:NumInQ                            1           0.0000
BlockingQueue_4:SenderQ:TimeInQ                           1           0.0000
BlockingQueue_4:RequestQ:NumInQ                           1           0.2100
BlockingQueue_4:RequestQ:TimeInQ                          1           0.3077
BlockingQueue_4:ChannelQ:NumInQ                           1           0.3300
BlockingQueue_4:ChannelQ:TimeInQ                          1           2.2000
-------------------------------------------------------------------------------------
```

As illustrated in this example, a blocking queue can facilitate the passing of information between two processes. These types of constructs can serve as the basis for communicating between agents which can invoke different procedures for different messages and wait until receiving and or sending

messages. We will see another example of using a blocking queue later in this chapter.

6.4.4. Allowing Entities to Wait for a Process

In the previous three examples, we saw how we can use a hold queue, a signal, and a blocking queue within a process description. In the case of the blocking queue, we saw how two processes communicated. In this next simple example, we also see how two processes can coordinate their flow via the use of the `waitFor(process: KSLProcess)` suspending function. The purpose of the `waitFor(process: KSLProcess)` suspending function is to allow one entity to start another process and have the entity that starts the process wait until the newly activated process is completed. The following code indicates the signature of the `waitFor(process: KSLProcess)` suspending function.

```
    /** Causes the current process to suspend until the
     ↪  specified process has run to completion.
     *  This is like run blocking.  It activates the
↪  specified process and then waits for it
     *  to complete before proceeding.
     *
     * @param process the process to start for an entity
     * @param timeUntilActivation the time until the start
↪  the process
     * @param priority the priority associated with the event
↪  to start the process
     */
    suspend fun waitFor(
        process: KSLProcess,
        timeUntilActivation: Double = 0.0,
        priority: Int = KSLEvent.DEFAULT_PRIORITY,
        suspensionName: String? = null
```

```
    )
```

The `waitFor(process: KSLProcess)` suspending function activates the named process and suspends the current process until the activated process completes. Then, the suspending process is resumed. Let's take a look at a simple example. Most of this class is simply to define the two interacting processes.

Example 6.6 (Illustrating Waiting for a Process)**.** This example illustrates how you can use one process to start another process. In addition, the process that starts the secondary process will wait until the secondary process completes before continuing.

```
class WaitForProcessExample(parent: ModelElement) :
↪  ProcessModel(parent, null) {
    private val worker: ResourceWithQ = ResourceWithQ(this,
    ↪  "worker", 1)
    private val tba = RandomVariable(this, ExponentialRV(6.0,
    ↪  1), "Arrival RV")
    private val st = RandomVariable(this, ExponentialRV(3.0,
    ↪  2), "Service RV")
    private val wip = TWResponse(this, "${name}:WIP")
    private val tip = Response(this, "${name}:TimeInSystem")
    private val arrivals = Arrivals()
    private val total = 1
    private var n = 1

    private inner class Customer : Entity() {
        val simpleProcess: KSLProcess = process {
            println("\t $time > starting simple process for
↪  entity: ${this@Customer.name}")
```

```
            wip.increment()
            timeStamp = time
            use(worker, delayDuration = st)
            tip.value = time - timeStamp
            wip.decrement()
            println("\t $time > completed simple process for
↪   entity: ${this@Customer.name}")
        }

        val wfp = process {
            val c = Customer()
            println("$time > before waitFor simple process
↪   for entity: ${this@Customer.name}")
            waitFor(c.simpleProcess)
            println("$time > after waitFor simple process for
↪   entity: ${this@Customer.name}")
        }
    }

    override fun initialize() {
        arrivals.schedule(tba)
    }

    private inner class Arrivals : EventAction<Nothing>() {
        override fun action(event: KSLEvent<Nothing>) {
            if (n <= total) {
                val c = Customer()
                println("$time > activating the waitFor
↪   process for entity: ${c.name}")
                activate(c.wfp)
                schedule(tba)
                n++
            }
```

```
        }
    }
}
```

The `Customer` class is very similar to previous examples of a simple queueing situation. However, notice the use of the `use()` function. Because the combination of seize-delay-release is so common, the KSL provides the `use()` function to combine these into a convenient suspending function. It is illustrative to see the implementation:

```
    suspend fun use(
        resource: ResourceWithQ,
        amountNeeded: Int = 1,
        seizePriority: Int = PRIORITY,
        delayDuration: GetValueIfc,
        delayPriority: Int = PRIORITY,
    ) {
        val a = seize(resource, amountNeeded, seizePriority)
        delay(delayDuration, delayPriority)
        release(a)
    }
```

Getting back to Example 6.6, the code defines a process called `WaitForAnotherProcess`. The sole purpose of this process is to create an instance of the customer, activate its simple process, and wait for it to complete. The output from activating one instance of the wait for another process is as follows:

```
0.8149947795247992 > activating the waitFor process for entity: ID_1
```

```
0.8149947795247992 > before waitFor simple process for entity: ID_1
     0.8149947795247992 > starting simple process for entity: ID_2
     5.091121672376351 > completed simple process for entity: ID_2
5.091121672376351 > after waitFor simple process for entity: ID_1
```

We see that the activation of entity ID_1 occurs at time 0.81, when it then subsequently activates entity ID_2's simple process. Entity ID_1 then suspends while entity ID_2 executes its simple process, which completes at time 5.09. Then, entity ID_1's process is allowed to complete. Thus, we see that it is easy to activate separate processes and to coordinate their completion.

6.4.5. Process Interaction

In this section, we will explore how processes associated with two (or more) different entities can interact during their execution by directly suspending and resuming each other. The previous process constructs are built upon the general lower level functionality of suspending the process and then resuming the process. For example, for the wait and signal construct, the entity will wait on a particular signal. During the wait, the entity's process is *suspended.* The signal causes the waiting entities to resume their process from the wait for signal suspension point. The KSL implements the previously described suspension functions by using the low-level `suspend()` function. Once an entity's process is suspended, it needs to be resumed. The entity's `resumeProcess()` function provides this capability. The following example illustrates how to facilitate the direct suspension and resumption of processes.

Example 6.7 (Direct Process Interaction)**.** Consider modeling the interaction between a soccer mom and her daughter. The soccer mom has a

daughter who plays forward on a soccer team. The game day proceeds as follows. First, the mom drives the daughter to the game. The drive takes 30 minutes. After arriving to the field, the daughter exits the mini-van, which takes 2 minutes. After the daughter exits the van, the mom departs to run some errands, meanwhile the daughter plays soccer. The mom's errands take approximately 45 minutes. The soccer game takes approximately 60 minutes. If the mom returns from the errands before the game ends, the mom waits patiently to pick up her daughter to go home. If the game ends, before the mom returns, the daughter waits patiently for the mom to return. Once the mother and daughter are together, the daughter loads up her soccer gear and enters the van. This loading time takes 2 minutes. After all is loaded, the happy mom and daughter drive home, which takes 30 minutes.

We can build a process model for this situation and use instances of the `Suspension` class within the implementation. The key to implementing this situation is to capture the suspensions and to share the references of the entities. Let's start with the mom's process. The following code shows the mom's process with extensive print statements added to illustrate the interaction.

```
    private inner class Mother : Entity() {

        val daughterExiting: Suspension = Suspension(name =
         ↪  "Suspend for daughter to exit van")
        val daughterPlaying: Suspension = Suspension(name =
         ↪  "Suspend for daughter playing")
        val daughterLoading: Suspension = Suspension(name =
         ↪  "Suspend for daughter entering van")

        val momProcess = process {
```

```
            println("$time> starting mom =
↪  ${this@Mother.name}")
            println("$time> mom = ${this@Mother.name} driving
↪  to game")
            delay(30.0)
            println("$time> mom = ${this@Mother.name} arrived
↪  at game")
            val daughter = Daughter(this@Mother)
            activate(daughter.daughterProcess)
            println("$time> mom = ${this@Mother.name}
↪  suspending for daughter to exit van")
            //suspend mom's process
            suspendFor(daughterExiting)
            println("$time> mom = ${this@Mother.name} running
↪  errands...")
            delay(45.0)
            println("$time> mom = ${this@Mother.name}
↪  completed errands")
            if (daughter.motherShopping.isSuspended){
                println("$time> mom, ${this@Mother.name}, mom
↪  resuming daughter done playing after errands")
                resume(daughter.motherShopping)
            } else {
                println("$time> mom, ${this@Mother.name}, mom
↪  suspending because daughter is still playing")
                suspendFor(daughterPlaying)
            }
            suspendFor(daughterLoading)
            println("$time> mom = ${this@Mother.name} driving
↪  home")
            delay(30.0)
            println("$time> mom = ${this@Mother.name} arrived
↪  home")
```

```
        }
    }
```

In this modeling, it is important to realize the reason for suspensions. The soccer mom will need three suspensions: 1) suspending while the daughter exits the vehicle, 2) suspending if the daughter is still playing, or 3) suspending while the daughter is entering the van. The three instances of the `Suspension` class represent these three possibilities and also provide shared state information.

The next concept to grasp is that the mom's process activates the daughter's process. The mom's process creates an instance of the daughter and then activates the daughter's process. The mom's process captures a reference to the `Daughter` in the `daughter` variable. The activation of a process schedules an event to occur. In this case, the event is scheduled for the current time. Note that the daughter's process starts after the mom completes the delay for driving to the field.

Then, the mom suspends using the `suspendFor()` function, specifying the appropriate `Suspension` instance. The `suspendFor()` function takes in the suspension that labels the suspension point. Now, let's look at the daughter's process.

When the mom process activates the daughter, the daughter starts the delay to exit the van. After exiting the van, the daughter tells the mom process via the `resume(mother.daughterExiting)` function to resume. Recall, that the mom process immediately suspended while the daughter exited the van. The critical item to note is that a reference to the `Mom` entity is required to create the daughter instance and the suspensions are public properties of the mother class. Thus, the daughter has a reference to the mom instance within the daughter's process routine.

In the following code, the daughter has her own suspension possibility via the `motherShopping` Suspension.

```
    private inner class Daughter(val mother: Mother) :
     ↪  Entity() {

        val motherShopping: Suspension = Suspension("Suspend
         ↪  for mother shopping")

        val daughterProcess = process {
            println("$time> starting daughter
↪  ${this@Daughter.name}")
            println("$time> daughter, ${this@Daughter.name},
↪  exiting the van")
            delay(2.0)
            println("$time> daughter, ${this@Daughter.name},
↪  exited the van")
            // resume mom process
            println("$time> daughter, ${this@Daughter.name},
↪  resuming mom")
            resume(mother.daughterExiting)
            println("$time> daughter, ${this@Daughter.name},
↪  starting playing")
//            delay(30.0)
            delay(60.0)
            println("$time> daughter, ${this@Daughter.name},
↪  finished playing")
            // suspend if mom isn't here
            if (mother.daughterPlaying.isSuspended){
                // mom's errand was completed and mom
                 ↪  suspended because daughter was playing
                resume(mother.daughterPlaying)
            } else {
                println("$time> daughter,
↪  ${this@Daughter.name}, mom errands not completed
↪  suspending")
```

```
                suspendFor(motherShopping)
            }
            println("$time> daughter, ${this@Daughter.name},
↪  entering van")
            delay(2.0)
            println("$time> daughter, ${this@Daughter.name},
↪  entered van")
            // resume mom process
            println("$time> daughter, ${this@Daughter.name},
↪  entered van, resuming mom")
            resume(mother.daughterLoading)
        }
    }
```

After playing the daughter uses the `daughterPlaying` Suspension of the mother to determine if the mother is suspended because her errands were completed before the daughter completed playing soccer. If the mother is not suspended for the daughter's playing, then the daughter suspends while the mother completes shopping.

Now, let's look at the rest of the mom's process. We see that after suspending to let the daughter exit the van, the mom will run her errands. Since the mom could return early, she checks if the daughter is suspended waiting for her to complete shopping, and if so, the mother resumes the daughter (so that the daughter can start loading). If the daughter is not suspended for the mom's shopping, she is still playing. So, the mom will suspend until the daughter completes playing.

```
        val momProcess = process {
            println("$time> starting mom =
↪  ${this@Mother.name}")
            println("$time> mom = ${this@Mother.name} driving
↪  to game")
```

```
            delay(30.0)
            println("$time> mom = ${this@Mother.name} arrived
↪  at game")
            val daughter = Daughter(this@Mother)
            activate(daughter.daughterProcess)
            println("$time> mom = ${this@Mother.name}
↪  suspending for daughter to exit van")
            //suspend mom's process
            suspendFor(daughterExiting)
            println("$time> mom = ${this@Mother.name} running
↪  errands...")
            delay(45.0)
            println("$time> mom = ${this@Mother.name}
↪  completed errands")
            if (daughter.motherShopping.isSuspended){
                println("$time> mom, ${this@Mother.name}, mom
↪  resuming daughter done playing after errands")
                resume(daughter.motherShopping)
            } else {
                println("$time> mom, ${this@Mother.name}, mom
↪  suspending because daughter is still playing")
                suspendFor(daughterPlaying)
            }
            suspendFor(daughterLoading)
            println("$time> mom = ${this@Mother.name} driving
↪  home")
            delay(30.0)
            println("$time> mom = ${this@Mother.name} arrived
↪  home")
        }
```

The mother will always suspend for the daughter loading. After being resumed by the daughter to indicate that the loading is complete, the mom

delays for the drive home. The following output illustrates what happens for the case of the mother's errand time being less than the playing time of the daughter.

```
0.0> starting mom = ID_1
0.0> mom = ID_1 driving to game
30.0> mom = ID_1 arrived at game
30.0> mom = ID_1 suspending for daughter to exit van
30.0> starting daughter ID_2
30.0> daughter, ID_2, exiting the van
32.0> daughter, ID_2, exited the van
32.0> daughter, ID_2, resuming mom
32.0> daughter, ID_2, starting playing
32.0> mom = ID_1 running errands...
77.0> mom = ID_1 completed errands
77.0> mom, ID_1, mom suspending because daughter is still playing
92.0> daughter, ID_2, finished playing
92.0> daughter, ID_2, entering van
94.0> daughter, ID_2, entered van
94.0> daughter, ID_2, entered van, resuming mom
94.0> mom = ID_1 driving home
124.0> mom = ID_1 arrived home
```

We see in this output that the mom suspends at time 30, while the daughter exits the van. The daughter starts playing and the mom starts her errands at time 32. The mom completes her errands at time 77 and suspends until the daughter is done playing at time 92. At which time, the daughter enters the van at time 94. They both drive off "together" starting at time 94.

We encourage the interested reader to re-run the code after changing the playing time to a smaller value, for example, 30 minutes. Then, you will see that the daughter suspends until the mother completes her errands. Thus, by carefully suspending and resuming processes, we can coordinate

their interaction as they proceed through time. This is the essence of the process interaction approach to simulation model representation. However, this low level coordination requires special attention to shared state and a deep understanding of how the processes interact, which can be error prone.

This same interaction can be achieved using other KSL constructs. Specifically, the `HoldQueue` and `Signal` constructs can also be used. Notice that when using the `Suspension` class, there were no waiting statistics collected when the processes wait for each other to complete. If you need waiting statistics, then using the `HoldQueue` or the `Signal` class can be useful.

The following code illustrates the use of the `HoldQueue` class. Notice that there are four hold queues defined. These represent the suspension points from the previous example. Just as before, the mother activates the daughter and then suspends via the `hold()` function. The mother uses the shared instances of the hold queues to check if the daughter is waiting in the `waitForMomToShopQ` hold queue. If so, the mother removes the daughter from the queue and then resumes the daughter's process. the mother will suspend (if necessary) for the daughter to complete playing via the `hold(waitForDaughterToPlayQ)` call. In addition, the mother will hold for the daughter to enter the van.

```
class SoccerMomViaHoldQ(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {

    private val waitForDaughterToExitQ = HoldQueue(this, name
     ↪  = "WaitForDaughterToExitQ")
    private val waitForDaughterToPlayQ = HoldQueue(this, name
     ↪  = "WaitForDaughterToPlayQ")
    private val waitForDaughterToLoadQ = HoldQueue(this, name
     ↪  = "WaitForDaughterToLoadQ")
```

```
    private val waitForMomToShopQ = HoldQueue(this, name =
     ↪  "WaitForMomToShopQ")

    override fun initialize() {
        val m = Mother()
        activate(m.momProcess)
    }

    private inner class Mother : Entity() {
        val momProcess = process {
            println("$time> starting mom =
↪  ${this@Mother.name}")
            println("$time> mom = ${this@Mother.name} driving
↪  to game")
            delay(30.0)
            println("$time> mom = ${this@Mother.name} arrived
↪  at game")
            val daughter = Daughter(this@Mother)
            activate(daughter.daughterProcess)
            println("$time> mom = ${this@Mother.name}
↪  suspending for daughter to exit van")
            //suspend mom's process
            hold(waitForDaughterToExitQ)
            println("$time> mom = ${this@Mother.name} running
↪  errands...")
            delay(45.0)
            println("$time> mom = ${this@Mother.name}
↪  completed errands")
            if (waitForMomToShopQ.contains(daughter)){
                println("$time> mom, ${this@Mother.name}, mom
↪  resuming daughter done playing after errands")
                waitForMomToShopQ.removeAndResume(daughter)
            } else {
```

```
                println("$time> mom, ${this@Mother.name}, mom
↪  suspending because daughter is still playing")
                hold(waitForDaughterToPlayQ)
            }
            hold(waitForDaughterToLoadQ)
            println("$time> mom = ${this@Mother.name} driving
↪  home")
            delay(30.0)
            println("$time> mom = ${this@Mother.name} arrived
↪  home")
        }
    }
```

The daughter's process is very similar to the previous example code. Notice that the shared references to the hold queues are used to check if the mother is waiting.

```
    private inner class Daughter(val mother: Mother) :
     ↪  Entity() {

        val daughterProcess = process {
            println("$time> starting daughter
↪  ${this@Daughter.name}")
            println("$time> daughter, ${this@Daughter.name},
↪  exiting the van")
            delay(2.0)
            println("$time> daughter, ${this@Daughter.name},
↪  exited the van")
            // resume mom process
            println("$time> daughter, ${this@Daughter.name},
↪  resuming mom")
            waitForDaughterToExitQ.removeAndResume(mother)
            println("$time> daughter, ${this@Daughter.name},
↪  starting playing")
```

```
            delay(30.0)
//            delay(60.0)
            println("$time> daughter, ${this@Daughter.name},
 ↪  finished playing")
            // suspend if mom isn't here
            if (waitForDaughterToPlayQ.contains(mother)){
                // mom's errand was completed and mom
                 ↪  suspended because daughter was playing

 ↪  waitForDaughterToPlayQ.removeAndResume(mother)
            } else {
                println("$time> daughter,
 ↪  ${this@Daughter.name}, mom errands not completed
 ↪  suspending")
                hold(waitForMomToShopQ)
            }
            println("$time> daughter, ${this@Daughter.name},
 ↪  entering van")
            delay(2.0)
            println("$time> daughter, ${this@Daughter.name},
 ↪  entered van")
            // resume mom process
            println("$time> daughter, ${this@Daughter.name},
 ↪  entered van, resuming mom")
            waitForDaughterToLoadQ.removeAndResume(mother)
        }
    }
```

Since the implementation using the `Signal` class is very similar, its use is left as an exercise for the reader.

The KSL has one other concept that facilitates process interaction: *blockages*. A blockage is like a semaphore or lock on a portion of a suspending

function. Think of a blockage as a gate that prevents another process from proceeding with its own process until another process has proceeded through the marked portion of code.

A blockage can be used to block (suspend) other entities while they wait for the blockage to be cleared. The user can mark process code with the start of a blockage and a subsequent end of the blockage. While the entity that creates the blockage is within the blocking code, other entities can be made to wait until the blockage is cleared. Only the entity that creates the blockage can start and clear it. A started blockage must be cleared before the end of the process routine that contains it; otherwise, an exception will occur. Thus, blockages that are started must always be cleared. The primary purpose of this construct is to facilitate process interaction between entities. Using blockages should be preferred over the use of `Suspension` instances. The following code illustrates the soccer mom and daughter interaction using blockages.

For the purposes of clarity of presentation, the print statements have been removed from the example code. The exercises ask the reader to show via print statements that the implementation using blockages results in the same interaction. Let's start with the mother's process. Since the daughter might have to wait for the mom to finish shopping, we will define a blockage to cover this activity.

```
    private inner class Mom : Entity() {
        val shopping: Blockage = Blockage("shopping")

        val momProcess = process {
            delay(30.0)
            val daughter = Daughter(this@Mom)
            activate(daughter.daughterProcess)
            waitFor(daughter.unloading)
            startBlockage(shopping)
            delay(45.0)
```

```
            clearBlockage(shopping)
            waitFor(daughter.playing)
            waitFor(daughter.loading)
            delay(30.0)
            println("$time> mom = ${this@Mom.name} arrived
↪   home")
        }
    }
```

Again, the mother creates and activates the daughter. The mother uses the `waitFor(blockage: Blockage)` suspending function to wait for the daughter to complete unloading. To see how this works, let's review the daughter's process.

The daughter has three potential blockages to represent unloading, playing, and loading. In the following code, notice how the unloading delay is between the start of a blockage and the clearing of the unloading blockage. This matching `startBlockage` and `clearBlockage` is like putting up a fence. The `waitFor(daughter.unloading)` call within the mother's process, will check if the blockage is started or cleared. If the blockage is cleared, then the entity immediately continues (does not suspend); however, if the blockage is started, the entity executing the `waitFor()` will suspend until the blockage is cleared. Thus, the mom will wait until the daughter is unloaded.

```
    private inner class Daughter(val mom: Mom) : Entity() {
        val unloading: Blockage = Blockage("unloading")
        val playing: Blockage = Blockage("playing")
        val loading: Blockage = Blockage("loading")

        val daughterProcess = process {
            startBlockage(unloading)
            delay(2.0)
```

```
            clearBlockage(unloading)
            startBlockage(playing)
            // delay(30.0)
            delay(60.0)
            clearBlockage(playing)
            waitFor(mom.shopping)
            startBlockage(loading)
            delay(2.0)
            clearBlockage(loading)
        }
    }
```

Notice the use of the `shopping` blockage within the mother's process. This blockage surrounds the delay for shopping. Now, in the daughter's process we see the use of the `waitFor(mom.shopping)` function call. Again, if the mom is still shopping, the daughter will suspend; however, if the mom has cleared the shopping blockage, the daughter proceeds to the unloading activity. If you need queue statistics, when using blockages, you can still use the `Queue` class and enqueue the entity prior to the `waitFor` call and remove the entity after the `waitFor` call.

> **!** Important
>
> A blockage can only be started and cleared by the entity that instantiates it. In addition, a blockage that has been started must be cleared before the end of the process within which it appears. A common error in the usage of blockages would be to not clear the blockage.

As illustrated in the previous example, using a blockage around an activity is a common use case. Because of this, the KSL provides a special type of blockage just for activities. Here is the code that uses `BlockingActivity` instances.

```
    private inner class Mom : Entity() {
        val shopping: BlockingActivity =
         ↪ BlockingActivity(45.0)
        val momProcess = process {
            delay(30.0)
            val daughter = Daughter(this@Mom)
            activate(daughter.daughterProcess)
            waitFor(daughter.unloading)
            perform(shopping)
            waitFor(daughter.playing)
            waitFor(daughter.loading)
            delay(30.0)
        }
    }
```

Notice the use of the new `perform()` suspending function. The `perform()` function essentially wraps a delay with a blockage. This ensures that you do not forget to clear the blockage. The daughter's process is also simplified as follows.

```
    private inner class Daughter(val mom: Mom) : Entity() {
        val unloading: BlockingActivity =
         ↪ BlockingActivity(2.0)
        val playing: BlockingActivity =
         ↪ BlockingActivity(60.0)
        val loading: BlockingActivity = BlockingActivity(2.0)

        val daughterProcess = process {
            perform(unloading)
            perform(playing)
            waitFor(mom.shopping)
            perform(loading)
        }
```

```
    }
```

Besides blockages for activities, the KSL provides blockages for the following common situations:

- `BlockingResourceUsage`: Wraps a blockage around the `use()` suspending function when using a resource or resource with queue during an activity.
- `BlockingResourcePoolUsage`: Wraps a blockage around the `use()` suspending function when using a resource pool or resource pool with queue during an activity.
- `BlockingMovement`: Wraps a blockage around the use of the `move()` suspending function. The movement of entities will be discussed in a subsequent chapter.

In the next section, we will develop a more realistically sized process model for a STEM Career Mixer involving students and recruiters.

6.5. Modeling a STEM Career Mixer

In this section, we model the operation of a STEM Career Fair mixer during a six-hour time period. The purpose of the example is to illustrate the following concepts:

- Probabilistic flow of entities
- Collecting statistics on observational (tally) and time-persistent data using the KSL responses
- Using the `seize()`, `delay()`, and `release()` functions of the `KSLProcessBuilder` class

Example 6.8 (STEM Career Mixer System)**.** Students arrive to a STEM career mixer event according to a Poisson process at a rate of 0.5 student per minute. The students first go to the name tag station, where it takes between 15 and 45 seconds uniformly distributed to write their name and affix the tag to themselves. We assume that there is plenty of space at the tag station, as well has plenty of tags and markers, such that a queue never forms.

After getting a name tag, 50% of the students wander aimlessly around, chatting and laughing with their friends until they get tired of wandering. The time for aimless students to wander around is triangularly distributed with a minimum of 15 minutes, a most likely value of 20 minutes, and a maximum value of 45 minutes. After wandering aimlessly, 90% decide to buckle down and visit companies and the remaining 10% just are too tired or timid and just leave. Those that decide to buckle down visit the MalWart station and then the JHBunt station as described next.

The remaining 50% of the original, newly arriving students (call them non-aimless students), first visit the MalWart company station where there are 2 recruiters taking resumes and chatting with applicants. At the MalWart station, the student waits in a single line (first come first served) for 1 of the 2 recruiters. After getting 1 of the 2 recruiters, the student and recruiter chat about the opportunities at MalWart. The time that the student and recruiter interact is exponentially distributed with a mean of 3 minutes. After visiting the MalWart station, the student moves to the JHBunt company station, which is staffed by 3 recruiters. Again, the students form a single line and pick the next available recruiter. The time that the student and recruiter interact at the JHBunt station is also exponentially distribution, but with a mean of 6 minutes. After visiting the JHBunt station, the student departs the mixer.

The organizer of the mixer is interested in collecting statistics on the following quantities within the model:

- number of students attending the mixer at any time t
- number of students wandering at any time t
- utilization of the recruiters at the MalWart station
- utilization of the recruiters at the JHBunt station
- number of students waiting at the MalWart station
- number of students waiting at the JHBunt station
- the waiting time of students waiting at the MalWart station
- the waiting time of students waiting at the JHBunt station
- total time students spend at the mixer broken down in the following manner
  - all students regardless of what they do and when they leave
  - students that wander and then visit recruiters
  - students that do not wander

The STEM mixer organizer is interested in estimating the average time students spend at the mixer (regardless of what they do).

6.5.1. Conceptualizing the System

When developing a simulation model, whether you are an experienced analyst or a novice, you should follow a modeling recipe. I recommend developing answers to the following questions:

- *What is the system?*
  - *What are the elements of the system?*
  - *What information is known by the system?*

- *What are the required performance measures?*
- *What are the entity types?*
  - *What information must be recorded or remembered for each entity instance?*
  - *How are entities (entity instances) introduced into the system?*
- *What are the resources that are used by the entity types?*
  - *Which entity types use which resources and how?*
- *What are the process flows? Sketch the process or make an activity flow diagram*
- *Develop pseudo-code for the situation*
- *Implement the model*

We will apply each of these questions to the STEM mixer example. The first set of questions: *What is the system? What information is known by the system?* are used to understand what should be included in the modeling and what should not be included. In addition, understanding the system to be modeled is essential for validating your simulation model. If you have not clearly defined what you are modeling (i.e. the system), you will have a very difficult time validating your simulation model.

The first thing that I try to do when attempting to understand the system is to draw a picture. A hand drawn picture or sketch of the system to be modeled is useful for a variety of reasons. First, a drawing attempts to put down "on paper" what is in your head. This aids in making the modeling more concrete and it aids in communicating with system stakeholders. In fact, a group stakeholder meeting where the group draws the system on a shared whiteboard helps to identify important system elements to include in the model, fosters a shared understanding, and facilitates model acceptance by the potential end-users and decision makers.

You might be thinking that the system is too complex to sketch or that it is impossible to capture all the elements of the system within a drawing. Well, you are probably correct, but drawing an idealized version of the system helps modelers to understand *that you do not have to model reality* to get good answers. That is, drawing helps to *abstract* out the important and essential elements that need to be modeled. In addition, you might think, why not take some photographs of the system instead of drawing? I say, go ahead, and take photographs. I say, find some blueprints or other helpful artifacts that represent the system and its components. These kinds of things can be very helpful if the system or process already exists. However, do not stop at photographs, drawing facilitates free flowing ideas, and it is an active/engaging process. Do not be afraid to draw. The art of abstraction is essential to good modeling practice.

Alright, have I convinced you to make a drawing? So, your next question is, what should my drawing look like and what are the rules for making a drawing? Really? My response to those kinds of questions is that you have not really bought into the benefits of drawing and are looking for reasons to not do it. There are no concrete rules for drawing a picture of the system. I like to suggest that the drawing can be like one of your famous kindergarten pictures you used to share with your grandparents. In fact, if your grandparents could look at the drawing and be able to describe back to you what you will be simulating, you will be on the right track! There are not really any rules.

Well, I will take that back. I have one rule. The drawing should not look like an engineer drew it. Try not to use engineering shapes, geometric shapes, finely drawn arrows, etc. You are not trying to make a blueprint. You are not trying to reproduce the reality of the system by drawing it to perfect scale. You are attempting to conceptualize the system in a form that facilitates communication. Don't be afraid to put some labels and text on the drawing. Make the drawing a picture that is rich in the elements that are in the system. Also, the drawing does not have to be perfect, and it does not have to be complete. Embrace the fact that you may have to circle back and iterate on the drawing as new modeling issues arise.

You have made an excellent drawing if another simulation analyst could take your drawing and write a useful narrative description of what you are modeling.

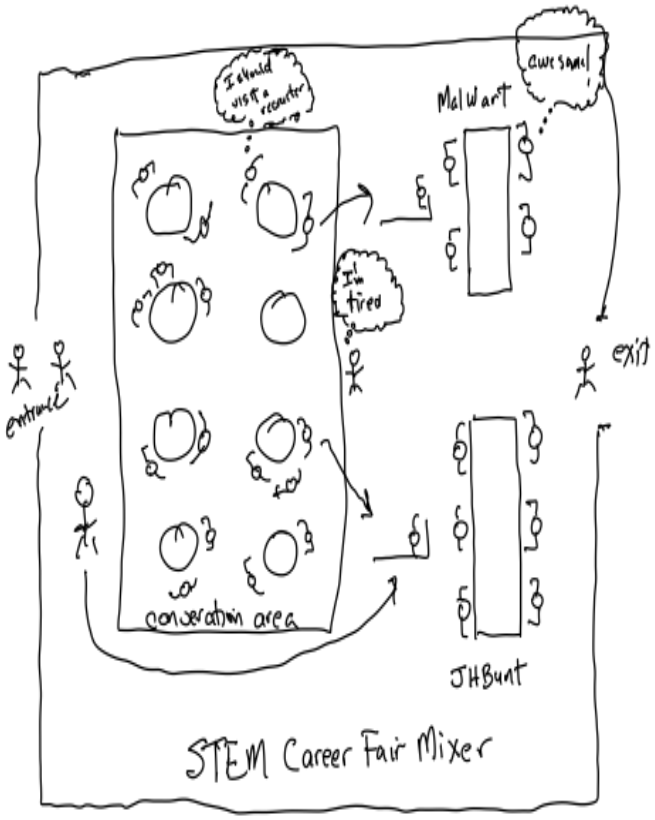


Figure 6.12.: Rich Picture System Drawing of STEM Career Mixer

Figure 6.12 illustrates a drawing for the STEM mixer problem. As you can see in the drawing, we have students arriving to a room that contains some tables for conversation between attendees. In addition, the two company stations are denoted with the recruiters and the waiting students. We

also see the wandering students and the timid students' paths through the system. At this point, we have a narrative description of the system (via the problem statement) and a useful drawing of the system. We are ready to answer the following questions:

- *What are the elements of the system?*
- *What information is known by the system?*

When answering the question "what are the elements?", your focus should be on two things: 1) the concrete structural things/objects that are required for the system to operate and 2) the things that are operated on by the system (i.e. the things that flow through the system).

What are the elements?

- students (non-wanderers, wanderers, timid)
- recruiters: 2 for MalWart and 3 for JHBunt
- waiting lines: one line for MalWart recruiters, one line for JHBunt recruiters
- company stations: MalWart and JHBunt
- The paths that students may take.

Notice that the answers to this question are mostly things that we can point at within our picture (or the real system).

When answering the question "what information is known at the system level?", your focus should be identifying facts, input parameters, and environmental parameters. Facts tend to relate to specific elements identified by the previous question. Input parameters are key variables, distributions, etc. that are typically under the control of the analyst and might likely be varied during the use of the model. Environmental parameters are also a kind of input parameter, but they are less likely to be varied by the modeler because they represent the operating environment of the system that is

most likely not under control of the analyst to change. However, do not get hung up on the distinction between input parameters and environmental parameters, because their classification may change based on the objectives of the simulation modeling effort. Sometimes innovative solutions come from questioning whether or not an environmental parameter can really be changed.

What information is known at the system level?

- students arrive according to a Poisson process with mean rate 0.5 students per minute or equivalently the time between arrivals is exponentially distributed with a mean of 2 minutes
- time to affix a name tag ~ uniform(15, 45) seconds
- 50% of students are go-getters and 50% are wanderers
- time to wander ~ triangular(15, 20, 45) minutes
- 10% of wanderers become timid/tired, 90% visit recruiters
- number of MalWart recruiters = 2
- time spent chatting with MalWart recruiter ~ exponential(3) minutes
- number of JHBunt recruiters = 3
- time spent chatting with JHBunt recruiter ~ exponential(6) minutes

A key characteristic of the answers to this question is that the information is "global". That is, it is known at the system level. We will need to figure out how to represent the storage of this information when implementing the model within software.

The next question to address is "*What are the required performance measures?*" For this situation, we have been given as part of the problem a list of possible performance measures, mostly related to time spent in the system and other standard performance measures related to queueing systems. In general, you will not be given the performance measures. Instead,

you will need to interact with the stakeholders and potential users of the model to identify the metrics that they need in order to make decisions based on the model. Identifying the metrics is a key step in the modeling effort. If you are building a model of an existing system, then you can start with the metrics that stakeholders typically use to character the operating performance of the system. Typical metrics include cost, waiting time, utilization, work in process, throughput, and probability of meeting design criteria.

Once you have a list of performance measures, it is useful to think about how you would collect those statistics *if you were an observer standing within the system.* Let's pretend that we need to collect the total time that a student spends at the career fair and we are standing somewhere at the actual mixer. How would you physically perform this data collection task? In order to record the total time spent by each student, you would need to note when each student arrived and when each student departed. Let A_i be the arrival time of the i^{th} student to arrive and let D_i be the departure time of the i^{th} student. Then, the system time (T) of the i^{th} student is $T_i = D_i - A_i$. How could we keep track of A_i for each student? One simple method would be to write the time that the student arrived on a sticky note and stick the note on the student's back. Hey, this is pretend, right? Then, when the student leaves the mixer, you remove the sticky note from their back and look at your watch to get D_i and thus compute, T_i. Easy! We will essentially do just that when implementing the collection of this statistic within the simulation model. Now, consider how you would keep track of the number of students attending the mixer. Again, standing where you can see the entrance and the exit, you would increment a counter every time a student entered and decrement the counter every time a student left. We will essentially do this within the simulation model.

Thus, identifying the performance measures to collect and how you will collect them will answer the 2^{nd} modeling question. You should think about and document how you would do this for every key performance measure. However, you will soon realize that simulation modeling languages, like Arena, have specific constructs that automatically collect common

statistical quantities such as resource utilization, queue waiting time, queue size, etc. Therefore, you should concentrate your thinking on those metrics that will not automatically be collected for you by the simulation software environment. In addition to identifying and describing the performance measures to be collected, you should attempt to classify the underlying data needed to compute the statistics as either observation-based (tally) data or time-persistent (time-weighted) data. This classification will be essential in determining the most appropriate constructs to use to capture the statistics within the simulation model. The following table summarizing the type of each performance measure requested for the STEM mixer simulation model.

Performance Measure	**Type**
Average number of students attending the mixer at any time t	Time persistent
Average number of students wandering within the mixer at any time t	Time persistent
Average utilization of the recruiters at the MalWart station	Time persistent
Average utilization of the recruiters at the MalWart station	Time persistent
Average utilization of the recruiters at the JHBunt station	Time persistent
Average number of students waiting at the MalWart station	Time persistent
Average number of students waiting at the JHBunt station	Time persistent
Average waiting time of students waiting at the MalWart station	Tally
Average waiting time of students waiting at the JHBunt station	Tally
Average system time for students regardless of what they do	Tally
Average system time for students that wander and then visit recruiters	Tally
Average system time for students that do not wander	Tally

Now, we are ready to answer the rest of the questions. When performing a process-oriented simulation, it is important to identify the entity types and the characteristics of their instances. An entity type is a classification or indicator that distinguishes between different entity instances. If you are familiar with object-oriented programming, then an entity type is a class, and an entity instance is an object of that class. Thus, an entity type describes the entity instances (entities) that are in its class. An entity

instance (or just entity) is something that flows through the processes within the system. Entity instances (or just entities) are realizations of objects within the entity type (class). Different entity types may experience different processes within the system. The next set of questions are about understanding entity types and their instances: *What are the entity types? What information must be recorded or remembered for each entity (entity instance) of each entity type? How are entities (entity instances) for each entity type introduced into the system?*

Clearly, the students are a type of entity. We could think of having different sub-types of the student entity type. That is, we can conceptualize three types of students (non-wanderers, wanderers, and timid/tired). However, rather than defining three different classes or types of students, we will just characterize one general type of entity called a Student and denote the different types with an attribute that indicates the type of student. An attribute is a named property of an entity type and can have different values for different instances. We will use the attributes to indicate whether the student wanders or not and whether the wandering student leaves without visiting recruiters. Then, we will these attributes to help in determining the path that a student takes within the system and when collecting the required statistics.

We are basically told how the students (instances of the student entity type) are introduced to the system. That is, we are told that the arrival process for students is Poisson with a mean rate of 0.5 students per minute. Thus, we have that the time between arrivals is exponential with a mean of 2 minutes. *What information do we need to record or be remembered for each student?* The answer to this question identifies the attributes of the entity. Recall that an attribute is a named property of an entity type which can take on different values for different entity instances. Based on our conceptualization of how to collect the total time in the system for the students, it should be clear that each entity (student) needs to remember the time that the student arrived. That is our sticky note on their backs. In addition, since we need to collect statistics related to how long the different types of students spend at the STEM fair, we need each

student (entity) to remember what type of student they are (non-wanderer, wanderer, and timid/tired).

After identifying the entity types, you should think about identifying the resources. Resources are system elements that entity instances need in order to proceed through the system. The lack of a resource causes an entity to wait (or block) until the resource can be obtained. By the way, if you need help identifying entity types, you should identify the things that are waiting in your system. It should be clear from the picture that students wait in either of two lines for recruiters. Thus, we have two resources: MalWart Recruiters and JHBunt Recruiters. We should also note the capacity of the resources. The capacity of a resource is the total number of units of the resource that may be used. Conceptually, the 2 MalWart recruiters are identical. There is no difference between them (as far as noted in the problem statement). Thus, the capacity of the MalWart recruiter resource is 2 units. Similarly, the JHBunt recruiter resource has a capacity of 3 units.

In order to answer the "and how" part of the question, I like to summarize the entities, resources and activities experienced by the entity types in a table. Recall that an activity is an interval of time bounded by two events. Entities experience activities as they move through the system. The following table summarizes the entity types, the activities, and the resources. This will facilitate the drawing of an activity diagram for the entity types.

Entity Type	**Activity**	**Resource Used**
Student (all)	time to affix a name tag ~ UNIF(15, 45) seconds	None
Student (wanderer)	Wandering time ~ TRIA(15, 15, 45) minutes	None
Student (not timid)	Talk with MalWart ~ EXPO(3) minutes	1 of the 2 MalWart recruite
Student (not timid)	Talk with JHBunt ~ EXPO(6) minutes	1 of the 3 JHBunt recruiter

Now we are ready to document the processes experienced by the entities. A good way to do this is through an activity flow diagram. As previously

described, an activity diagram will have boxes that show the activities, circles to show the queues and resources, and arrows, to show the paths taken. Figure 6.13 presents the activity diagram for this situation.

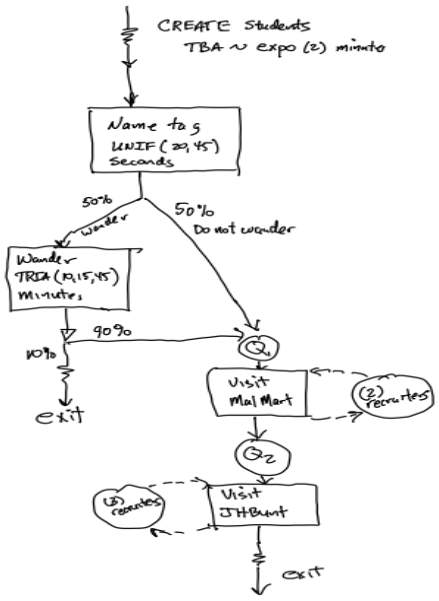


Figure 6.13.: Activity Diagram for STEM Career Mixer

In Figure 6.13, we see that the students are created according to a time between arrival (TBA) process that is exponentially distributed with a mean of 2 minutes and they immediately experience the activity associated with affixing the name tag. Note that the activity does not require any resources and thus has no queue in front of it. Then, the students follow one of two paths, with 50% becoming "non-wanderers" and 50% becoming "wanderers". The wandering students, wander aimlessly, for period of time, which is represented by another activity that does not require any resources. Then, the 90% of the students follow the non-wanderer path and the remaining 10% of the students, the timid/tired students, leave

the system. The students that decide to visit the recruiting stations, first visit the MalWart station, where in the diagram we see that there is an activity box for their talking time with the recruiter. During this talking time, they require one of the recruiters and thus we have a resource circle indicating the usage of the resource. In addition, we have a circle with a queue denoted before the activity to indicate that the students visiting the station may need to wait. The JHBunt station is represented in the same fashion. After visiting the JHBunt station, the students depart the mixer. Now, we are ready to represent to start representing this system in KSL code.

6.5.2. Implementing the STEM Mixer Model

The first thing that should be done is to prepare to use a process model by defining the model element that will contain the system. So, we start the modeling by defining a class that is a subclass of `ProcessModel`:

```
class StemFairMixer(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {
```

We will place the model elements needed within this class. We have already identified the need for many random variables. So, let's add those next.

```
class StemFairMixer(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {
    private val myTBArrivals: RVariableIfc =
     ↪  ExponentialRV(2.0, 1)
    private val myNameTagTimeRV = RandomVariable(this,
     ↪  UniformRV((15.0/60.0), (45.0/60.0), 2))
```

```
    private val myWanderingTimeRV = RandomVariable(this,
     ↪  TriangularRV(15.0, 20.0, 45.0, 3),
        name = "WanderingT")
    private val myTalkWithJHBunt = RandomVariable(this,
     ↪  ExponentialRV(6.0, 4))
    private val myTalkWithMalMart = RandomVariable(this,
     ↪  ExponentialRV(3.0, 5))
    private val myDecideToWander = RandomVariable(this,
     ↪  BernoulliRV(0.5, 6))
    private val myDecideToLeave = RandomVariable(this,
     ↪  BernoulliRV(0.1, 7))
```

Notice that I have converted the time for performing the name tag activity to minutes and that all other time units are in minutes. Also, the random variables have specific stream numbers specified.

We know that we need to collect statistics. So, using the KSL `Response` and `TWResponse` classes these KSL constructs can be added next.

```
class StemFairMixer(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {
    private val myTBArrivals: RVariableIfc =
     ↪  ExponentialRV(2.0, 1)
    private val myNameTagTimeRV = RandomVariable(this,
     ↪  UniformRV((15.0 / 60.0), (45.0 / 60.0), 2))
    private val myWanderingTimeRV = RandomVariable(this,
     ↪  TriangularRV(15.0, 20.0, 45.0, 3),
        name = "WanderingT")
    private val myTalkWithJHBunt = RandomVariable(this,
     ↪  ExponentialRV(6.0, 4))
    private val myTalkWithMalMart = RandomVariable(this,
     ↪  ExponentialRV(3.0, 5))
```

```
    private val myDecideToWander = RandomVariable(this,
     ↪  BernoulliRV(0.5, 6))
    private val myDecideToLeave = RandomVariable(this,
     ↪  BernoulliRV(0.1, 7))

    private val myOverallSystemTime = Response(this,
     ↪  "OverallSystemTime")
    private val mySystemTimeNW = Response(this,
     ↪  "NonWanderSystemTime")
    private val mySystemTimeW = Response(this,
     ↪  "WanderSystemTime")
    private val mySystemTimeL = Response(this,
     ↪  "LeaverSystemTime")
    private val myNumInSystem = TWResponse(this,
     ↪  "NumInSystem")
```

Finally, we are ready to add the elements necessary for the process model.

```
class StemFairMixer(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {

    private val myTBArrivals: RVariableIfc =
     ↪  ExponentialRV(2.0, 1)
    private val myNameTagTimeRV = RandomVariable(this,
     ↪  UniformRV((15.0/60.0), (45.0/60.0), 2))
    private val myWanderingTimeRV = RandomVariable(this,
     ↪  TriangularRV(15.0, 20.0, 45.0, 3),
        name = "WanderingT")
    private val myTalkWithJHBunt = RandomVariable(this,
     ↪  ExponentialRV(6.0, 4))
    private val myTalkWithMalMart = RandomVariable(this,
     ↪  ExponentialRV(3.0, 5))
```

```
    private val myDecideToWander = RandomVariable(this,
     ↪  BernoulliRV(0.5, 6))
    private val myDecideToLeave = RandomVariable(this,
     ↪  BernoulliRV(0.1, 7))

    private val myOverallSystemTime = Response(this,
     ↪  "OverallSystemTime")
    private val mySystemTimeNW = Response(this,
     ↪  "NonWanderSystemTime")
    private val mySystemTimeW = Response(this,
     ↪  "WanderSystemTime")
    private val mySystemTimeL = Response(this,
     ↪  "LeaverSystemTime")
    private val myNumInSystem = TWResponse(this,
     ↪  "NumInSystem")

    private val myJHBuntRecruiters: ResourceWithQ =
     ↪  ResourceWithQ(this,
      capacity = 3, name = "JHBuntR")
    val jhBuntRecruiters : ResourceWithQCIfc
        get() = myJHBuntRecruiters

    private val myMalWartRecruiters: ResourceWithQ =
     ↪  ResourceWithQ(this,
      capacity = 2, name = "MalWartR")
    val malWartRecruiters : ResourceWithQCIfc
        get() = myMalWartRecruiters

    private val generator = EntityGenerator(::Student,
     ↪  myTBArrivals, myTBArrivals)
```

Well, actually, we cannot really add the `EntityGenerator` until we have defined the class `Student`. This is done as an inner class of `StemFairMixer`.

Because it is an inner class of `StemFairMixer`, it will have access to all the previously defined random variables, resources, and statistical responses.

In the following code, two attributes, `isWanderer` and `isLeaver` are defined as properties of `Student`. Notice how the values of these properties are assigned upon creation using the random variables and how the values of 1.0 or 0.0 are converted to boolean values. This is performed by using an extension function found in the `ksl.utilities.random.rvariable` package within the `KSLRandom` class file. It is very convenient for use with if statements. It is important to note that individual instances of the `Student` class will get different values for their `isWanderer` and `isLeaver` properties. When the object instance is created, the assignment to the property occurs. At that time, a new random value is generated, converted from 1.0 or 0.0 to true or false and then assigned to the created student object.

```
    private inner class Student : Entity() {
        private val isWanderer =
         ↪ myDecideToWander.value.toBoolean()
        private val isLeaver =
         ↪ myDecideToLeave.value.toBoolean()

        val stemFairProcess = process(isDefaultProcess =
         ↪ true) {
            myNumInSystem.increment()
            delay(myNameTagTimeRV)
            if (isWanderer) {
                delay(myWanderingTimeRV)
                if (isLeaver) {
                    departMixer(this@Student)
                    return@process
                }
            }
            val mw = seize(myMalWartRecruiters)
```

```
            delay(myTalkWithMalMart)
            release(mw)
            val jhb = seize(myJHBuntRecruiters)
            delay(myTalkWithJHBunt)
            release(jhb)
            departMixer(this@Student)
        }
```

The process followed by each student is defined in the process property called `stemFairProcess.` Notice how we first increment the number in the system and start the delay for the name tag activity. If the student is a wandering student, we experience the delay for wandering. And, if the student is a wandering student that leaves early, then the `departingMixer()` function is called. As we will see in a moment, this function will be used to collect statistics on departing students.

Now we have something new, we have a `return` statement within a process. As previously noted, Kotlin flow of control statements are available within coroutines and `return` is a flow of control statement. Because the return statement is within a process builder, we need to be more specific about the return label. This can be specified by the name of the builder function. In this case `process.` Kotlin also allows you to explicitly label the return. This return statement will cause the normal exit from the process routine for those students that leave without visiting recruiters.

The students that visit the recruiter *(seize-delay-release)* the related resources and then depart. The following code shows the `departMixer()` function. In this function, we decrement the number in the system and collect the system time statistics. Notice how we use the attributes within the boolean conditions of the if statements to get the correct system time response variables.

```
        private fun departMixer(departingStudent: Student) {
            myNumInSystem.decrement()
            val st = time - departingStudent.createTime
            myOverallSystemTime.value = st
            if (isWanderer) {
                mySystemTimeW.value = st
                if (isLeaver) {
                    mySystemTimeL.value = st
                }
            } else {
                mySystemTimeNW.value = st
            }
        }
    }
```

The results of this simulation indicate that the JHBunt recruiters have the highest utilization.

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                                    Count        Average      Half-
Width
-------------------------------------------------------------------------------
OverallSystemTime                         400        34.3985       0.8019
NonWanderSystemTime                       400        22.3367       0.8123
WanderSystemTime                          400        47.1614       0.7897
LeaverSystemTime                          400        27.0096       0.2321
NumInSystem                               400        16.9105       0.4610
JHBuntR:InstantaneousUtil                 400         0.8411       0.0065
JHBuntR:NumBusyUnits                      400         2.5232       0.0195
JHBuntR:ScheduledUtil                     400         0.8411       0.0065
JHBuntR:WIP                               400         7.5014       0.3772
JHBuntR:Q:NumInQ                          400         4.9783       0.3648
```

```
JHBuntR:Q:TimeInQ                400          10.6658        0.7411
MalWartR:InstantaneousUtil       400           0.6766        0.0071
MalWartR:NumBusyUnits            400           1.3531        0.0142
MalWartR:ScheduledUtil           400           0.6766        0.0071
MalWartR:WIP                     400           2.6757        0.1209
MalWartR:Q:NumInQ                400           1.3226        0.1105
MalWartR:Q:TimeInQ               400           2.7871        0.2147
JHBuntR:SeizeCount               400         154.4125        1.0191
MalWartR:SeizeCount              400         164.1375        1.2031
-------------------------------------------------------------------------------
```

The results produced by the KSL are within statistical variation of the same system modeled with a commercial simulation language discussed in this book.

6.6. The Tie-Dye T-Shirt Model

This section presents another process modeling situation using the KSL. In this modeling situation the key feature to be illustrated is the use of a `BlockingQueue` to communicate and coordinate between two processes. We will also see that a process can spawn another process.

Example 6.9 (Tie Dye T-Shirts System)**.** Suppose production orders for tie-dye T-shirts arrive to a production facility according to a Poisson process with a mean rate of 1 per hour. There are two basic psychedelic designs involving either red or blue dye. For some reason the blue shirts are a little more popular than the red shirts so that when an order arrives about 70% of the time it is for the blue dye designs. In addition, there are two different package sizes for the shirts, 3 and 5 units. There is a 25%

chance that the order will be for a package size of 5 and a 75% chance that the order will be for a package size of 3. Each of the shirts must be individually hand made to the customer's order design specifications. The time to produce a shirt (of either color) is uniformly distributed within the range of 15 to 25 minutes. There are currently two workers who are setup to make either shirt. When an order arrives to the facility, its type (red or blue) is determined and the pack size is determined. Then, the appropriate number of white (un-dyed) shirts are sent to the shirt makers with a note pinned to the shirt indicating the customer order, its basic design, and the pack size for the order. Meanwhile, the paperwork for the order is processed and a customized packaging letter and box is prepared to hold the order. It takes another worker between 8 to 10 minutes to make the box and print a custom thank you note. After the packaging is made it waits prior to final inspection for the shirts associated with the order. After the shirts are combined with the packaging, they are inspected by the packaging worker which is distributed according to a triangular distribution with a minimum of 5 minutes, a most likely value of 10 minutes, and a maximum value of 15 minutes. Finally, the boxed customer order is sent to shipping.

6.6.1. Implementing the Tie-Dye T-Shirt Model

Before proceeding you might want to jot down your answers to the modeling recipe questions and then you can compare how you are doing with respect to what is presented in this section. The modeling recipe questions are:

- What is the system? What information is known by the system?
- What are the required performance measures?

- What are the entities? What information must be recorded or remembered for each entity? How are entities introduced into the system?
- What are the resources that are used by the entities? Which entities use which resources and how?
- What are the process flows? Sketch the process or make an activity flow diagram
- Develop pseudo-code for the situation
- Implement the model

The entities can be conceptualized as the arriving orders. Since the shirts are processed individually, they should also be considered entities. In addition, the type of order (red or blue) and the size of the order (3 or 5) must be tracked. Since the type of the order and the size of the order are properties of the order, attributes can be used to model this information. The resources are the two shirt makers and the packager. The flow is described in the scenario statement: orders arrive, shirts are made, meanwhile packaging is made. Then, orders are assembled, inspected, and finally shipped. It should be clear that a an `EntityGenerator` can be used to generate Poisson arrivals can create the orders, but if shirts are entities, how should they be created?

To create the shirts, the order process start the shirt making process based on the size of the order. After this, there will be two types of entities in the model, the orders and the shirts. The shirts can be made and meanwhile the order can be processed. After the shirts for an order are made, they need to be combined together and then matched for the order. This implies that a method is required to uniquely identify the order and coordinating between the processes. This is another piece of information that both the order and the shirt require. Thus, an attribute will be used to note the order number.

The activity diagram for this situation is given in Figure 6.14. After the order is created, the process separates into the order making process and the shirt making process. Notice that the orders and shirts must be synchronized together after each of these processes. In addition, the order making process and the final packaging process share the packager as a resource.

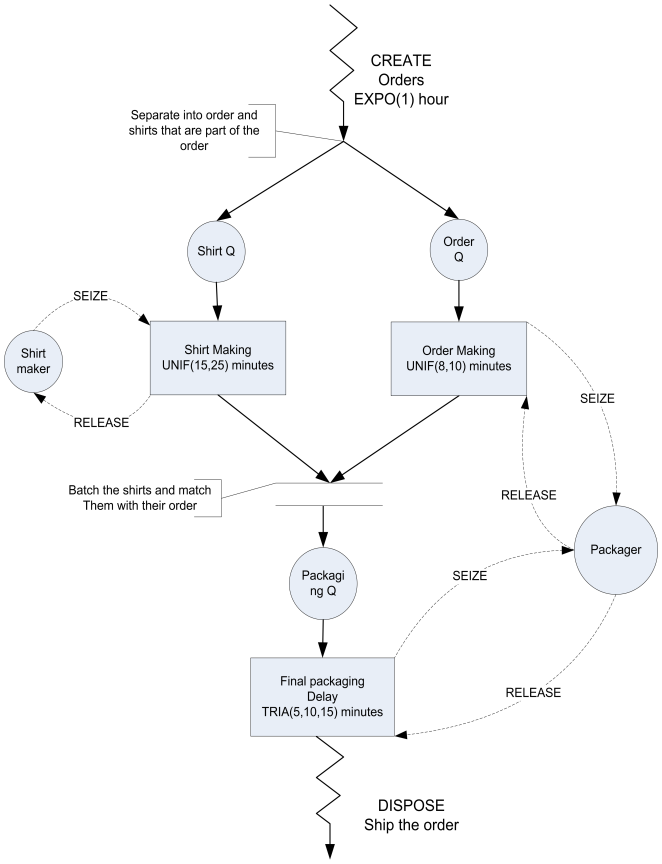


Figure 6.14.: Activity Diagram for Tie Dye T-Shirts Example

Just as in the previous example, you should start by subclassing from

`ProcessModel` and add the necessary random variables and responses. This is illustrated in the following code. In this situation, we use a discrete empirical distribution to model the type and size of the order. The other random variables are straight forward applications of the `RandomVariable` class. We define two responses to track the total time in the system and the total number of orders in the system.

```
class TieDyeTShirts(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {

    private val myTBOrders: RVariableIfc =
     ↪ ExponentialRV(60.0)
    private val myType: RVariableIfc =
     ↪ DEmpiricalRV(doubleArrayOf(1.0, 2.0),
     ↪ doubleArrayOf(0.7, 1.0))
    private val mySize: RVariableIfc =
     ↪ DEmpiricalRV(doubleArrayOf(3.0, 5.0),
     ↪ doubleArrayOf(0.75, 1.0))
    private val myOrderSize = RandomVariable(this, mySize)
    private val myOrderType = RandomVariable(this, myType)
    private val myShirtMakingTime = RandomVariable(this,
     ↪ UniformRV(15.0, 25.0))
    private val myPaperWorkTime = RandomVariable(this,
     ↪ UniformRV(8.0, 10.0))
    private val myPackagingTime = RandomVariable(this,
     ↪ TriangularRV(5.0, 10.0, 15.0))

    private val mySystemTime = Response(this, "System Time")
    private val myNumInSystem = TWResponse(this, "Num in
     ↪ System")
```

Now we can develop the process modeling constructs. We will use an

`EntityGenerator`, resources to represent the shirt makers and the packager. Notice that we use a `RequestQ` to represent the queue for the orders. A `RequestQ` is a subclass of the Queue class that is specifically designed to work with seize requests for resources. In general, the `seize()` suspend function allows both the specification of the resource being seized and the queue that will hold the entity if the seize request is not immediately filled. As noted in the activity diagram, there are actually two queues involving the use of the packager. The queue that holds the orders and a queue that holds the completed orders for final packaging. We will see the use of these constructs in the process description. Finally, we use an instance of a `BlockingQueue` to hold completed shirts and communicate that they are ready.

```
private val myShirtMakers: ResourceWithQ =
 ↪  ResourceWithQ(this, capacity = 2, name =
 ↪  "ShirtMakers_R")
private val myOrderQ : RequestQ = RequestQ(this, name =
 ↪  "OrderQ")
private val myPackager: ResourceWithQ =
 ↪  ResourceWithQ(this, "Packager_R")
private val generator = EntityGenerator(::Order,
 ↪  myTBOrders, myTBOrders)
private val completedShirtQ: BlockingQueue<Shirt> =
 ↪  BlockingQueue(this, name = "Completed Shirt Q")
```

The order process follows the basic outline of the activity diagram. As we have previously seen, we design a class to represent the orders and the order making process by creating a subclass of the `Entity` class. Similar to the previous examples, this code uses two properties to hold the type and size of the order and assigns their values based on the random variable instances of the outer class. As note in the code, the `type` property is never used in the rest of the model; however, it could be useful if we wanted to count the type of shirts produced or for other statistics by type. We

also have a list to hold the complete orders. Again, as noted in the code, the list is not really used in a meaningful manner. In this case, it is used to capture the list of items from the `waitForItems()` function, but not subsequently used. If a future process required the created shirts, then the list could be useful.

```
    private inner class Order: Entity() {
        val type: Int = myOrderType.value.toInt() // in the
        ↪ problem, but not really used
        val size: Int = myOrderSize.value.toInt()
        var completedShirts : List<Shirt> = emptyList() //
        ↪ not really necessary

        val orderMaking: KSLProcess =
        ↪ process(isDefaultProcess = true) {
            myNumInSystem.increment()
            for (i in 1..size) {
                val shirt = Shirt(this@Order.id)
                activate(shirt.shirtMaking)
            }
            var a = seize(myPackager, queue = myOrderQ)
            delay(myPaperWorkTime)
            release(a)
            // wait for shirts
            completedShirts = waitForItems(completedShirtQ,
↪   size, { it.orderNum == this@Order.id })
            a = seize(myPackager)
            delay(myPackagingTime)
            release(a)
            myNumInSystem.decrement()
            mySystemTime.value = time - this@Order.createTime
        }
    }
```

When the order making process is activated, there is a for-loop that makes the shirts and activates the shirt making process. This activates the process at the current time. It is important to note that the activated shirt making processes are scheduled to be activated at the current time. Since those events are on the event calendar, they will not be executed until the current event finishes. The current event is essentially the code in the process before the next suspension point. This provides the notion of pseudo-parallelism. The shirt making processes are really pending activation.

Meanwhile, the order continues with its process by using the packager in the classic *(seize-delay-release)* pattern. However, note the signature of the `seize()` method, which specifies the queue for waiting orders.

```
    var a = seize(myPackager, queue = myOrderQ)
```

Thus, this use of the packager causes the entity (the order) to wait in the queue for orders. The later seize of the packager cause the order to wait in the pre-defined queue associated with the `ResourceWithQ` class defined for the packager.

After the paper work is done, the order is ready to start final packaging if it has the shirts associated with the order. This is where the blocking queue is used. The `waitForItems()` call will be explained more fully shortly. However, at this point, we can think of the order waiting for the correct number of shirts to arrive that are associated with this particular order. After the shirts arrive, the order process continues and again seizes the packager for the packaging time. In this usage of the `seize()` method, we need only specify an instance of a `ResourceWithQ`. A `ResourceWithQ` has a pre-defined `RequestQ` that holds the requests for the resource. We see that it is simple to share a resource between two different activities. The `seize()` method also takes in an optional argument that specifies the priority of the seize request. If there are more that one suspending `seize()` functions that compete for the same resource, you can used the priority parameter to specify which seize request has the priority if more

than one is waiting at the same time. Finally, the statistics on the order are collected before its process ends. Now, let's look at the shirt making process.

```
    private inner class Shirt(val orderNum: Long): Entity() {
        val shirtMaking: KSLProcess = process( "Shirt
         ↪ Making"){
            val a = seize(myShirtMakers)
            delay(myShirtMakingTime)
            release(a)
            // send to orders
            send(this@Shirt, completedShirtQ)
        }
    }
```

The shirt making process is constructed based on a different entity that represents what happens to a shirt. There are a couple of interesting things to note. First a property `orderNum` is defined using Kotlin's concise syntax for declaring properties in the default constructor. The property `orderNum` is used to identify, for the shirt, which order created it. Then, the shirt uses the shirt makers resource to make the shirt via the *(seize-delay-release)* code.

Finally, the blocking queue that was defined as part of the process model is used to send a reference to the shirt to the channel queue that connects the shirt process with the ordering process. A Kotlin qualified this reference must be used to identify the shirt within the `KSLProcessBuilder.` As the shirts are made, they are sent to the channel. When the correct number of shirts for the order are made the waiting order can pull them from the channel and continue with its process. Now, let's take a closer look at the blocking code statement in the order process:

```
completedShirts = waitForItems(completedShirtQ, size,
 ↪  {it.orderNum == this@Order.id})
```

To understand this code fragment, we need to see some of the implementation of the `BlockingQueue` class and the suspend functions that are using it. The signature of this method is as follows:

```
    /**
     * This method will block (suspend) until the required
 ↪  number of items that meet the criteria
     * become available within the blocking queue.
     *
     * @param blockingQ the blocking queue channel that has
 ↪  the items
     * @param amount the number of items needed from the
 ↪  blocking queue that match the criteria
     * @param predicate a functional predicate that tests
 ↪  items in the queue for the criteria
     * @param blockingPriority the priority associated with
 ↪  the entity if it has to wait to receive
     * @param suspensionName the name of the suspension
 ↪  point. can be used to identify which receive suspension
 ↪  point
     * the entity is experiencing if there are more than one
 ↪  receive suspension points within the process.
     * The user is responsible for uniqueness.
     */
    suspend fun <T : ModelElement.QObject> waitForItems(
        blockingQ: BlockingQueue<T>,
        amount: Int = 1,
        predicate: (T) -> Boolean = alwaysTrue,
        blockingPriority: Int = KSLEvent.DEFAULT_PRIORITY,
        suspensionName: String? = null
```

```
    ): List<T>
```

The first thing to note is the parameter `amount`. This parameter specifies how many items are being requested from the blocking queue. The next parameter is the `predicate`. This parameter represents a function that takes in the type being held in the queue and returns true or false depending on some condition. In this case, the type is `Shirt`. This is a common functional idiom found in functional programming languages such as Kotlin. The predicate is defined as lambda expression that checks if the order number of the shirt `it.orderNum` is equal to the order number of the current order (`this@Order.id`). If so, the shirt belongs to the order. Once the specified amount is found within the channel the suspension will end and the ordering process will continue.

```
completedShirts = waitForItems(completedShirtQ, size,
 ↪  {it.orderNum == this@Order.id})
```

Whenever an item is sent to a blocking queue, the blocking queue will check to see if there are any receivers waiting for items. If there are receivers waiting for items, then the blocking queue will use the blocking queue's request selector to select the next request to be filled. The request is examined to see if it can be filled. That is, the blocking queue checks to see if all the items requested that meet the request criteria are in the queue. In this case, the request criteria is specified by the previously mentioned lambda expression. If the request can be filled, the items are given to the waiting receiver and the receiver's process can continue.

As discussed in the previous section introducing blocking queues, the default request selector simply looks only at the next waiting request. Other rules for selecting requests can also be provided when configuring the blocking queue. For example, you can provide an instance of a `FirstFillableRequest` selector as shown in the following code. This class

is available as in inner class of `BlockingQueue`. As can be seen here, this selector will scan the waiting requests and return the first request that can be filled or null if no requests can be filled.

```
    /**
     * Allows the next request that can be filled to be
↪   selected regardless of its
     * location within the request queue.
     */
    inner class FirstFillableRequest() :
     ↪  RequestSelectorIfc<T> {
        override fun selectRequest(queue:
         ↪  Queue<ChannelRequest>): ChannelRequest? {
            for (request in queue) {
                if (request.canBeFilled) {
                    return request
                }
            }
            return null
        }
    }
```

The results of running the Tie-Dye T-Shirt model indicate that there is substantial waiting done by the order until the shirts are completed.

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                                                Count           Average      Half-
Width
-------------------------------------------------------------------------------------
System Time                                            30           69.9237       8.1833
Num in System                                          30            1.2040       0.2395
ShirtMakers_R:InstantaneousUtil                        30            0.5578       0.0713
```

ShirtMakers_R:NumBusyUnits	30	1.1156	0.1427
ShirtMakers_R:ScheduledUtil	30	0.5578	0.0713
ShirtMakers_R:Q:NumInQ	30	1.8824	0.6407
ShirtMakers_R:Q:TimeInQ	30	26.9951	7.4232
ShirtMakers_R:WIP	30	2.9980	0.7570
OrderQ:NumInQ	30	0.0354	0.0102
OrderQ:TimeInQ	30	1.8121	0.4476
Packager_R:InstantaneousUtil	30	0.3038	0.0366
Packager_R:NumBusyUnits	30	0.3038	0.0366
Packager_R:ScheduledUtil	30	0.3038	0.0366
Packager_R:Q:NumInQ	30	0.0104	0.0040
Packager_R:Q:TimeInQ	30	0.5975	0.2438
Packager_R:WIP	30	0.3142	0.0391
Completed Shirt Q:RequestQ:NumInQ	30	0.8544	0.2034
Completed Shirt Q:RequestQ:TimeInQ	30	49.4049	7.7277
Completed Shirt Q:ChannelQ:NumInQ	30	0.7806	0.1202
Completed Shirt Q:ChannelQ:TimeInQ	30	14.1351	0.7906
ShirtMakers_R:SeizeCount	30	27.4333	3.4022
Packager_R:SeizeCount	30	15.6000	1.7640

6.7. Summary

This chapter introduced the concepts involved in process view modeling. The process view perspective enables the modeler to imagine that they are the entity, experiencing different activities while using resources. The model development approach focuses on what happens to an entity and results in a process description. A process is a set of activities that change the state of the system. The execution of a process by an entity causes events and state changes. As such, the underlying mechanisms for implementing the process view are still predicated on understanding the event view. However, the process view allows for a more natural modeling of the flow of events.

The following new KSL constructs were discussed in this chapter:

`Entity`: An inner class of the `ProcessModel` class that facilitates the definition of processes for the process view.

`ProcessModel`: A sub-class of the `ModelElement` class that contains the architecture for defining and running coroutines that form the basis for the process view.

`KSLProcess`: The class that wraps the Kotlin coroutine functionality to enable the process view. Instances of this class are created by the `process()` function builder.

`ResourceWithQ`: This KSL construct provides a resource with a queue that can be seized and release by entities within a process.

`EntityGenerator`: This KSL class provides functionality to create entities according to a pattern represented by a time between creation and time between creations. Once the entity is created it will be automatically activated.

`HoldQueue`: This class will hold an entity in a queue until the entity is removed. It provides a `hold()` function that suspend the process until the entity is removed and resumed.

`Signal`: This class allows the entity to wait for a signal to occur before proceeding within a process.

`BlockingQueue`: This class will hold entities in a channel for receivers and senders of entities. Receivers may block waiting for an entity to appear in the channel and senders may block if space is not available in the channel.

`Suspension`: This class facilitates the suspension and resumption of entities between processes.

`Blockage`: This class represents a general mechanism for controlling the interaction between processes by allowing one process to mark sections of a process that cause the suspension of other processes while the marked section of a process completes.

`activate()`: This function is used to schedule the start of an entity's process.

`seize()`: This suspending function allows an entity to request a resource and if not available suspend.

`release()`: This function causes an allocation of a resource to be returned.

`delay()`: This suspending function suspends the process until a time period has elapsed.

`use()`: This suspending function combines the seize, delay, release functions.

`waitFor()`: This suspending function allows a process to suspend until another process completes or to suspend until a signal occurs or a blockage is cleared.

`suspendFor()`: This suspending function works with the `Suspension` class to allow an entity to suspend for a particular suspension point.

`perform()`: This suspending function works with common use cases of blockages.

Based on the constructs discussed so far, very complex systems can be modeled using the KSL. In the next chapter, we explore additional advanced modeling constructs.

6.8. Exercises

Exercise 6.1. Suppose that a customer arriving to the drive through pharmacy will decide to balk if the number of cars waiting in line is 4 or more. A customer is said to *balk* if he or she refuses to enter the system and simply departs without receiving service. Model this situation using and estimate the probability that a customer will balk because the line is

too long. Run your model for 1 year, with 20 replications. Use stream 1 for the arrival process and stream 2 for the service process.

Exercise 6.2. Samples of 20 parts from a metal grinding process are selected every hour. Typically 2% of the parts need rework. Let X denote the number of parts in the sample of 20 that require rework. A process problem is suspected if X exceeds its mean by more than 1 standard deviation. Simulate 30 hours of the process, i.e. 30 samples of size 20, and estimate the chance that X exceeds its expected value by more than 1 standard deviation. Use stream 1.

Exercise 6.3. Samples of 20 parts from a metal grinding process are selected every hour. Typically 2% of the parts need rework. Let X denote the number of parts in the sample of 20 that require rework. A process problem is suspected if X exceeds its mean by more than 1 standard deviations. Each time X exceeds its mean by more than 1 standard deviations all X of the parts requiring rework are sent to a rework station. Use stream 1 to determine if a sample requires rework.

Each part consists of two subcomponents, which are split off and repaired separately. The splitting process takes 1 worker and lasts U(10, 20) minutes per part (stream 2). After the subcomponents have been split, they are repaired in different processes. Subcomponent 1 takes U(5, 10) minutes (stream 3) to repair with 1 worker at its repair process and subcomponent 2 takes expo(7.5) minutes (stream 4) to repair with 1 worker at its repair process. Once both of the subcomponents have been repaired, they are joined back together to form the original part. The joining process takes 5 minutes with 1 worker. The part is then sent back to the main production area, which is outside the scope of this problem. Simulate 8 hours of

production and estimate the average time that it takes a part to be repaired. Use 20 replications.

Exercise 6.4. TV sets arrive at a two-inspector station for testing. The time between arrivals is exponential with a mean of 15 minutes (stream 1). The inspection time per TV set is exponential with a mean of 10 minutes (stream 2). On the average, 82 percent of the sets pass inspection (stream 3). The remaining 18% are routed to an adjustment station with a single operator. Adjustment time per TV set is uniform between 7 and 14 minutes (stream 4). After adjustments are made, sets are routed back to the inspection station to be retested. We are interested in estimating the total time a TV set spends in the system before it is released.

Develop a model for this situation. Report the average system time for the TV sets based on 20 replications of 4800 minutes. Also report statistics for the average number of times a given TV is adjusted.

Exercise 6.5. A simple manufacturing system is staffed by 3 operators. Parts arrive according to a Poisson process with a mean rate of 15 per hour (stream 1) to a workstation for a drilling process at one of three identical drill presses. The parts wait in a single queue until a drill press is available. Each part has a particular number of holes that need to be drilled. Each hole takes a Lognormal time to be drilled with an approximate mean of 1 minute and a standard deviation of 30 seconds (stream 2). Once the holes are drilled, the part goes to the grinding operation. At the grinding operation, one of the 3 available operators grinds out the burrs on the part. This activity takes approximately 5 minutes plus or minus 30 seconds (stream 3). After the grinding operation the part leaves the system. The drill presses operate without an operator. Assume that the parts have a

discrete uniform number of holes to drill over the range from 2 to 4 (stream 4).

Develop a model for this situation. Report the average system time for the parts based on 20 replications of 4800 minutes.

Exercise 6.6. Consider a single pump gas station where the arrival process is Poisson with a mean time between arrivals of 10 minutes. The service time is exponentially distributed with a mean of 6 minutes.

Build a KSL model to simulate this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the probability that you have to wait for service?

b. What is the mean number of customer at the station?

c. What is the expected time waiting in the line to get a pump?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.7. Suppose an operator has been assigned to the responsibility of maintaining 3 machines. For each machine the probability distribution of the running time before a breakdown is exponentially distributed with a mean of 9 hours. The repair time also has an exponential distribution with a mean of 2 hours.

Build a KSL model to simulate this situation. Run the model for 20,000 hours with a warm up period of 5,000 hours. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the probability that the operator is idle?

b. What is the expected number of machines that are running?

c. What is the expected number of machines that are not running?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.8. SuperFastCopy wants to install self-service copiers, but cannot decide whether to put in one or two machines. They predict that arrivals will be Poisson with a rate of 30 per hour, and the time spent copying is exponentially distributed with a mean of 1.75 minutes. Because the shop is small they want the probability of 5 or more customers in the shop to be small, say less than 7%.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, estimate the probability of 5 or more customers in the shop with one copier and with two copiers, and recommend the number of copiers.

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.9. Each airline passenger and his or her carry-on baggage must be checked at the security checkpoint. Suppose XNA averages 10 passengers per minute with exponential inter-arrival times. To screen passengers, the airport must have a metal detector and baggage X-ray machines. Whenever a checkpoint is in operation, two employees are required (one operates the metal detector, one operates the X-ray machine). The passenger goes

through the metal detector and simultaneously their bag goes through the X-ray machine. A checkpoint can check an average of 12 passengers per minute according to an exponential distribution.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the probability that a passenger will have to wait before being screened?

b. On average, how many passengers are waiting in line to enter the checkpoint?

c. On average, how long will a passenger spend at the checkpoint?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.10. Two machines are being considered for processing a job within a factory. Jobs arrive at the machine according to a Poisson process with a rate of 4.5 jobs per hour. The first machine has an exponentially distributed processing time with a mean of 10 minutes. For the second machine the vendor has indicated that the mean processing time is 10 minutes but with a standard deviation of 6 minutes with a lognormal distribution.

Build KSL models to analyze this situation. Run the models for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, which machine is better in terms of the average waiting time of the jobs?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.11. Customers arrive at a one-window drive in bank according to a Poisson distribution with a mean of 10 per hour. The service time for each customer is exponentially distributed with a mean of 5 minutes. There are 3 spaces in front of the window including that for the car being served. Other arriving cars can wait outside these 3 spaces.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the probability that an arriving customer can enter one of the 3 spaces in front of the window?

b. What is the probability that an arriving customer will have to wait outside the 3 spaces?

c. How long is an arriving customer expected to wait before starting service?

d. How many spaces should be provided in front of the window so that an arriving customer can wait in front of the window at least 20% of the time? In other words, the probability of at least one open space must be greater than 20%.

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.12. Joe Rose is a student at Big State U. He does odd jobs to supplement his income. Job requests come every 5 days on the average, but the time between requests is exponentially distributed. The time for completing a job is also exponentially distributed with a mean of 4 days.

Build a KSL model to analyze this situation. Run the model for 20,000 days with a warm up period of 5,000 days. Based on 30 replications of your simulation, estimate the following performance measures.

a. What is the chance that Joe will not have any jobs to work on?

b. What is the average value of the waiting jobs if Joe gets about $25 per job?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.13. The manager of a bank must determine how many tellers should be available. For every minute a customer stands in line, the manager believes that a delay cost of 5 cents is incurred. An average of 15 customers per hour arrive at the bank. On the average, it takes a teller 6 minutes to complete the customer's transaction. It costs the bank $9 per hour to have a teller available. Inter-arrival and service times can be assumed to be exponentially distributed.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. What is the minimum number of tellers that should be available in order for the system to be stable (i.e. not have an infinite queue)?

b. If the system has 3 tellers, what is the probability that there will be no one in the bank?

c. What is the expected total cost of the system per hour, when there are 2 tellers?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.14. You have been hired to analyze the needs for loading dock facilities at a trucking terminal. The present terminal has 4 docks on the main building. Any trucks that arrive when all docks are full are assigned to a secondary terminal, which is a short distance away from the main terminal. Assume that the arrival process is Poisson with a rate of 5 trucks each hour. There is no available space at the main terminal for trucks to wait for a dock. At the present time nearly two-thirds of the arriving trucks are diverted to the secondary terminal. The average service time per truck is two hours on the main terminal and 3 hours on the secondary terminal, both exponentially distributed. Two proposals are being considered. The first proposal is to expand the main terminal by adding docks so that at least 80% of the arriving trucks can be served there with the remainder being diverted to the secondary terminal. The second proposal is to expand the space that can accommodate up to 8 trucks. Then, only when the holding area is full will the trucks be diverted to secondary terminal.

Build KSL models to analyze this situation. Run the models for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. For the first proposal, what is the required number of docks so that at least 80% of the arriving trucks can be served?

b. For the second proposal, will the specified number of spaces have least 80% of the arriving trucks can be served?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.15. Sly's convenience store operates a two-pump gas station. The lane leading to the pumps can house at most five cars, including those being serviced. Arriving cars go elsewhere if the lane is full. The distribution of the arriving cars is Poisson with a mean of 20 per hour. The time to fill up and pay for the purchase is exponentially distributed with a mean of 6 minutes.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. What is the percentage of cars that will seek business elsewhere?

b. What is the utilization of the pumps?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.16. An airline ticket office has two ticket agents answering incoming phone calls for flight reservations. In addition, two callers can be put on hold until one of the agents is available to take the call. If all four phone lines (both agent lines and the hold lines) are busy, a potential customer gets a busy signal, and it is assumed that the call goes to another ticket office and that the business is lost. The calls and attempted calls occur randomly (i.e. according to Poisson process) at a mean rate of 15 per hour. The length of a telephone conversation has an exponential distribution with a mean of 4 minutes.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. What is probability of losing a potential customer?

b. What is the probability that an arriving phone call will not start service immediately but will be able to wait on a hold line?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.17. SuperFastCopy has three identical copying machines. When a machine is being used, the time until it breaks down has an exponential distribution with a mean of 2 weeks. A repair person is kept on call to repair the machines. The repair time for a machine has an exponential distribution with a mean of 0.5 week. The downtime cost for each copying machine is $100 per week.

Build a KSL model to analyze this situation. Run the model for 20,000 weeks with a warm up period of 5,000 weeks. Based on 30 replications of your simulation, what is the expected downtime cost per week.

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.18. NWH Cardiac Care Unit (CCU) has 5 beds, which are virtually always occupied by patients who have just undergone major heart surgery. Two registered nurses (RNs) are on duty in the CCU in each of the three 8 hour shifts. Each patient requires a nurse's attention about every two hours, following an exponential distribution. The RN will then spend an average of 30 minutes (exponentially distributed) assisting the patient and updating medical records regarding the problem and care provided.

Build a KSL model to analyze this situation. Run the model for 20,000 minutes with a warm up period of 5,000 minutes. Based on 30 replications of your simulation, answer the following questions.

a. What is the average number of patients being attended by the nurses?

b. What is the average time that a patient spends waiting for one of the nurses to arrive?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.19. HJ Bunt, Transport Company maintains a large fleet of refrigerated trailers. For the purposes of this problem assume that the number of refrigerated trailers is conceptually infinite. The trailers require service on an irregular basis in the company owned and operated service shop. Assume that the arrival of trailers to the shop is approximated by a Poisson distribution with a mean rate of 3 per week. The length of time needed for servicing a trailer varies according to an exponential distribution with a mean service time of one-half week per trailer. The current policy is to utilize a centralized contracted outsourced service center whenever more than two trailers are in the company shop, so that, at most one trailer is allowed to wait. Assume that there is currently one 1 mechanic in the company shop.

Build a KSL model to analyze this situation. Run the model for 20,000 weeks with a warm up period of 5,000 weeks. Based on 30 replications of your simulation, what is the expected number of repairs that are outsourced per week?

Use stream 1 for the time between arrivals and stream 2 for the service times.

Exercise 6.20. The Hog BBQ Joint is interested in understanding the flow of customers for diner (5 pm to 9 pm). Customers arrive in parties of 2, 3, 4, or 5 with probabilities 0.4, 0.3, 0.2, 0.1, respectively (stream 1). The time between arrivals is exponentially distributed with a mean of 1.4 minutes (stream 2). Customers must arrive prior to 9 pm in order to be seated. The dining area has 50 tables. Each table can seat 2 people. For parties, with more than 2 customers, the tables are moved together. Each arriving group gets in line to be seated. If there are already 6 parties in line, the arriving group will leave and go to another restaurant. The time that it takes to be served is triangularly distributed with parameters (14, 19, 24) in minutes (stream 3). The time that it takes to eat is lognormally distributed with a mean of 24 minutes and a standard deviation of 5 minutes (stream 4). When customers are finished eating, they go to the cashier to pay their bill. The time that it takes the cashier to process the customers is gamma distributed with a mean of 1.5 minutes and a standard deviation of 0.5 minutes (stream 5).

Develop a model for this situation. Simulate 30 days of operation. Make a table like the following to summarize your results.

	Average	Half-width
Number of customers served		
Number of busy tables		
Number of waiting parties		
Number of parties that depart without eating		
Utilization of cashier		
Customer System Time (in minutes)		
Probability of waiting to be seated > 5 minutes		

Exercise 6.21. In the Tie-Dye T-Shirt model, the owner is expecting the business to grow during the summer season. The owner is interested in estimating the average time to produce an order and the utilization of the workers if the arrival rate for orders increases. Re-run the model for 30 eight hour days with the arrival rate increased by 20, 40, 60, and 80 percent. Will the system have trouble meeting the demand? Use the statistics to justify your conclusions.

Use the following stream allocations:

- Stream 1, time between orders
- Stream 2, order type
- Stream 3, order size
- Stream 4, shirt making time
- Stream 5, paper work time
- Stream 6, packaging time

Exercise 6.22. Suppose that the inspection and packaging process has been split into two processes for the Tie-Dye T-Shirt system and assume that there an additional worker to perform inspection. The inspection process is uniformly distributed between 2 and 5 minutes. After inspection there is a 4 percent chance that the whole order will have to be scrapped (and redone). If the order fails inspection, the scrapped order should be counted and a new order should be initiated into the system. If the order passes inspection, it goes to packaging where the packaging time is distributed according to a triangular distribution with parameters (2, 4, 10) all in minutes. Re-run the model for 30, 8-hour days, with the arrival rate increased by 20, 40, 60, and 80%. Will the system have trouble meeting the demand? In other words, how does the throughput (number of shirts produced per day) change in response to the increasing demand rate?

Use the following stream allocations:

- Stream 1, time between orders
- Stream 2, order type
- Stream 3, order size
- Stream 4, shirt making time
- Stream 5, paper work time
- Stream 6, packaging time
- Stream 7, inspection time
- Stream 8, scrapping process

Exercise 6.23. Hungry customers arrive to a Mickey R's drive through restaurant at a mean rate of 10 per hour according to a Poisson process (stream 1). Management is interested in improving the total time spent within the system (i.e. from arrival to departure with their food).

Management is considering a proposed system that splits the order taking, payment activity and the order delivery processes. The first worker will take the orders from an order-taking speaker. This takes on average 1 minute plus or minus 20 seconds uniformly distributed (stream 2). When the order taking activity is completed, the making of the order will start. It takes approximately 3 minutes (plus or minus 20 seconds) to make the customer's order, uniformly distributed (stream 3). Assume that the kitchen has enough cooks to make orders as soon as they are taken. Meanwhile, the customer will be instructed to drive to the first window to pay for the order. Assume that the time that it takes the customer to move forward is negligible. The first worker accepts the payment from the customer. This takes on average 45 seconds plus or minus 20 seconds uniformly distributed (stream 4). After paying for the order the customer is instructed to pull forward to the second window, where a second worker delivers the order. Assume that the time that it takes the customer to move forward is negligible.

If the order is not completed by the time the customer reaches the second window, then the customer must wait for the order to be completed. If the order is completed before the customer arrives to the 2nd window, then the order must wait for the customer. After both the order and the customer are at the 2nd window, the 2nd worker packages the customer's order and gives it to the customer. This takes approximately 30 seconds with a standard deviation of 10 seconds, lognormally distributed (stream 5). After the customer receives their order they depart.

Simulate this system for the period from 10 am to 2 pm. Report the total time spent in the system for the customers based on 30 days.

Exercise 6.24. The city is considering improving its hazardous waste and bulk item drop off area to improve service. Cars arrive to the drop off area at a rate of 10 per hour according to a Poisson process (stream 1). Each car contains items for drop off. There is a 10% chance that the car will contain 1 item, a 50% chance that the car will contain 2 items, and a 40% chance that the car will contain 3 items (stream 2). There is an 80% chance that an item will be hazardous (e.g. chemicals, light bulbs, electronic equipment, etc.) and a 20% chance that the item will be a bulk item, which cannot be picked up in the curbside recycling program (stream 3). Of the 80% of items that have hazardous waste, about 10% are for electronic equipment that must be inspected and taken apart (stream 4).

A single worker assists the citizen in taking the material out of their car and moving the material to the recycling center. This typically takes between 0.5 to 1.5 minutes per item (uniformly distributed (stream 5)) if the item is not a bulk item. If the item is a bulk item, then the time takes a minimum of 1 minute, most likely 2.5 minutes, with a maximum of 4 minutes per item triangularly distributed (stream 6). The worker finishes all items in a car before processing the next car.

Another worker will begin sorting the items immediately after the item is unloaded. This process takes 1-2 minutes per item uniformly distributed (stream 7). If the item is electronic equipment, the items are placed in front of a special disassembly station to be taken apart.

The same worker that performs sorting also performs the disassembly of the electronic parts. Items that require sorting take priority over items that require disassembly. Each electronic item takes between 8 to 16 minutes uniformly distributed to disassemble (stream 8).

The hazardous waste recycling center is open for 7 hours per day, 5 days per week. Simulate 12 weeks of performance and estimate the following quantities:

- Utilization of the workers
- Average waiting time for items waiting to be unloaded
- Average number of items waiting to be unloaded
- Average number of items waiting to be sorted
- Average waiting time of items to be sorted
- Average number of items waiting to be disassembled
- Average waiting time for items waiting to be disassembled.

Exercise 6.25. Orders for street lighting poles require the production of the tapered pole, the base assembly, and the wiring/lighting assembly package. Orders are released to the shop floor with an exponential time between arrival of 20 minutes (stream 1). Assume that all the materials for the order are already available within the shop floor.

Once the order arrives, the production of the pole begins. Pole production requires that the sheet metal be cut to a trapezoidal shape. This process

takes place on a cutting shear. After cutting, the pole is rolled using a press brake machine. This machine rolls the sheet to an almost closed form. After rolling, the pole is sealed on an automated welding machine. Each of these processes are uniformly distributed with ranges $[3, 5]$, $[6, 10]$, and $[4, 8]$ minutes, (streams 2, 3, 4) respectively.

While the pole is being produced, the base is being prepared. The base is a square metal plate with four holes drilled for bolting the place to the mounting piece and a large circular hole for attaching the pole to the base. The base plates are in stock so that only the holes need to be cut. This is done on a water jet cutting machine. This process takes approximately 20 minutes plus or minus 2 minutes, triangularly distributed (stream 5). After the holes are cut, the plate goes to a grinding/deburring station, which takes a mean of 10 minutes, exponentially distributed (stream 6).

Once the plate and the pole are completed, they are transported to the inspection station. Inspection takes 20 minutes, exponentially distributed (stream 7) with 1 operator. There could be a quality problem with the pole or with the base (or both). The chance that the problem is with the base is 0.02 (stream 8) and the chance that the problem is with the pole is 0.01 (stream 9). If either or both have a quality issue, the pole and base go to a rework station for rework. Rework is performed by a single operator and typically takes on average 100 minutes, exponentially distributed (stream 10). After rework, the pole and base are sent to final assembly. If no problems occur with the pole or the base, the pole and base are sent directly to final assembly.

At the assembly station, the pole is fixed to the base plate and the wiring assembly is placed within the pole. This process takes 1 operator approximately 30 minutes with a standard deviation of 4 minutes according to a lognormal distribution (stream 11). After assembly, the pole is sent to the shipping area for final delivery.

The shop is interested in taking on additional orders which would essentially double the arrival rate. Estimate the utilization of each resource and the average system time to produce an order for a lighting pole. Assume that

the system runs 5 days per week, with two, eight hours shifts per day. Any production that is not completed within 5 days is continued on the next available shift. Run the model for 10 years assuming 52 weeks per year to report your results.

Exercise 6.26. Patients arrive at an emergency room where they are treated and then depart. Arrivals are exponentially distributed with a mean time between arrivals of 0.3 hours (stream 1). Upon arrival, patients are assigned a rating of 1 to 5, depending on the severity of their ailments. Use stream 2 to determine the category. Patients in Category 1 are the most severe, and they are immediately sent to a bed where they await medical attention. All other patients must first wait in the receiving room until a basic registration form and medical record are completed. They then proceed to a bed.

The emergency room has three beds, one registration nurse, and two doctors. In all cases, the priority for allocating these resources is based on the severity of the ailment. Hint: Read the documentation for the `Queue` class and rank the queue by a severity attribute represented by the `QObject` priority property.

The registration time for patients in Categories 2 through 5 is Uniform (0.1, 0.2) hours (stream 3). The treatment time for all patients is triangularly distributed with the minimum, most likely, and maximum values differing according to the patient's category. The distribution of patients by category and the corresponding minimum, most likely, and maximum treatment times are summarized below. Use streams 4-8 for the treatment times of the categories of patients.

Category	1	2	3	4	5
Percent	6	8	18	33	35

Category	1	2	3	4	5
Minimum	0.8	0.7	0.4	0.2	0.1
Most Likely	1.2	0.95	0.6	0.45	0.35
Maximum	1.6	1.1	0.75	0.6	0.45

The required responses for this simulation include:

- Average number of patients waiting for registration
- Utilization of beds
- System time of each type of patient and overall (across patient types)

Using a run length of 30 days, develop a model to estimate the required responses. Report the responses based on estimating the system time of a patient regardless of type based on 50 replications.

Exercise 6.27. Customers enter a fast-food restaurant according to an exponential inter-arrival time with a mean of 0.7 minutes (use stream 1). Customers have a choice of ordering one of three kinds of meals: (1) a soft drink, (2) fries, or (3) soft drink, fries, and a burger. Upon arriving at the restaurant, the customer enters a single queue, awaits the availability of a cashier, gives the order to the cashier, then the customer pays the cashier. After the order is placed, the cooking can begin. The customer then waits until their order is ready. After receiving the order, the customer exits. A cashier may not take any additional orders until the current customer has paid. In this system, there are two cooks and two cashiers. The time to order and pay is represented by a triangular distribution with parameters (0.4, 0.8, 1.2) minutes and (0.2, 0.4, 0.6) minutes, respectively. The cooking time depends on the order as follows:

Type	Percentage	Cooking Time
1	30%	Uniform(0.3,0.8)
2	15%	Uniform(0.8,1.1)
3	55%	Uniform(1.0, 1.4)

Use stream 2 for the ordering distribution, stream 3 for the paying distribution, streams 4, 5 and 6, for the type 1, 2, and 3 cooking distributions, respectively. Use stream 7 for the distribution across the order types.

Model the system for 8 hours of operation with 30 replications. Make a table like the following to summarize your answers for your replications.

	Average	Half-width
Type 1 Throughput		
Type 2 Throughput		
Type 3 Throughput		
Utilization of cashiers		
Utilization of cooks		
Customer System Time (in minutes)		
Customer Waiting Time (in minutes)		
Probability of wait $>$ 5 minutes		

Exercise 6.28. Jobs arrive in batches of ten items each. The inter-arrival time is exponentially distributed with a mean of 2 hours (stream 1). The machine shop contains 2 milling machines and one drill press. About 30% of the items require drilling before being processed on the milling machine (stream 2). Drilling time per item is uniformly distributed between (10, 15) minutes (stream 3). The milling time is exponentially distributed (stream 4) with a mean of 15 minutes for items that do not require drilling, and

uniformly distributed (stream 5) between (15,20) minutes for items that do. Assume that the shop has two 8-hour shifts each day and that you are only interested in the *first* shift's performance. Any jobs left over at the end of the first shift are left to be processed by the second shift. Estimate the average number of jobs left for the second shift to complete at the end of the first shift to within plus or minus 5 jobs with 95% confidence. What is your replication length? Number of replications? Determine the utilization of the drill press and the milling machines as well as the average time an item spends in the system.

Exercise 6.29. A repair and inspection facility consists of two stations, a repair station with two technicians, and an inspection station with 1 inspector. Each repair technician works at a rate of 7.5 items per hour (stream 2), while the inspector can inspect 15 items per hour (stream 3) each exponentially distributed. Approximately 10% (stream 5) of all items fail inspection and are sent back to the repair station (this percentage holds even for items that have been repaired two to three times). If an item fails inspection three times then it is scrapped. When an item is scrapped, the item is sent to a disassembly station to recover the usable parts.

At the disassembly station, the items wait in a queue until a technician is available. The disassembly time is quite complex and is distributed according to a Lognormal distribution with a mean of 200 minutes and a standard deviation of 10 minutes (stream 4). Assume that items arrive according to a Poisson arrival process with a rate of 12 per hour (stream 1).

The weekly performance of the system is the key objective of this simulation analysis. Assume that the system starts empty and idle on Monday mornings and runs continuously for 2 shifts per day for 5 days. Each shift has 420 minutes of actual work time available. Any items not completed by the end of 2^{nd} shift are carried over to the 1^{st} shift of the next day. Any

items left over at the end of the week are handled by a separate weekend staff that is not of concern to the current study.

Estimate the following:

- The average system time of items that pass inspection on the first attempt. Measure this quantity such that you are 95% confident to within +/- 3 minutes.
- The average number of items in the system at any time.
- The system time (for completed items) regardless of the number of times the item failed inspection.
- The probability that an item will be scrapped.
- The average number of items completed per week.
- The average number of items scrapped per week.
- The average number of items left for the weekend staff.

a. Sketch an activity diagram for this situation.

b. Assume that there are 2 technicians at the repair station, 1 inspector at the inspection station, and 1 technician at the disassembly station. Develop a model for this situation.

Exercise 6.30. Reconsider Exercise 6.29 Assume that there are 2 technicians at the repair station and 1 inspector at the inspection station. The disassembly station is also staffed by the 2 technicians that are assigned to the repair station. That is, the disassembly and repair stations share their technicians. The repair station should have priority over work at the disassembly station. Develop a model for this situation. Compare your results to Exercise 6.29, which design do you recommend?

Exercise 6.31. As part of a diabetes prevention program, a clinic is considering setting up a screening service in a local mall. They are considering two designs: Design A: After waiting in a single line, each walk-in patient is served by one of three available nurses. Each nurse has their own booth, where the patient is first asked some medical health questions, then the patient's blood pressure and vitals are taken, finally, a glucose test is performed to check for diabetes. In this design, each nurse performs the tasks in sequence for the patient. If the glucose test indicates a chance of diabetes, the patient is sent to a separate clerk to schedule a follow-up at the clinic. If the test is not positive, then the patient departs.

Design B: After waiting in a single line, each walk-in is served in order by a clerk who takes the patient's health information, a nurse who takes the patient's blood pressure and vitals, and another nurse who performs the diabetes test. If the glucose test indicates a chance of diabetes, the patient is sent to a separate clerk to schedule a follow-up at the clinic. If the test is not positive, then the patient departs. In this configuration, there is no room for the patient to wait between the tasks; therefore, a patient who has had their health information taken cannot move ahead unless the nurse taking the vital signs is available. Also, a patient having their glucose tested must leave that station before the patient in blood pressure and vital checking can move ahead.

Patients arrive to the system according to a Poisson arrival process at a rate of 9.5 per hour (stream 1). Assume that there is a 5% chance (stream 2) that the glucose test will be positive. For design A, the time that it takes to have the paperwork completed, the vitals taken, and the glucose tested are all log-normally distributed with means of 6.5, 6.0, and 5.5 minutes respectively (streams 3, 4, 5). They all have a standard deviation of approximately 0.5 minutes. For design B, because of the specialization of the tasks, it is expected that the mean of the task times will decrease by 10%.

Assume that the clinic is open from 10 am to 8 pm (10 hours each day) and that any patients in the clinic before 8 pm are still served. The distribution

used to model the time that it takes to schedule a follow up visit is a Weibull distribution with shape 2.6 and scale 7.3 using stream 6.

Make a statistically valid recommendation as to the best design based on the average system time of the patients. We want to be 95% confident of our recommendation to within 2 minutes.

Exercise 6.32. A copy center has one fast copier and one slow copier. The copy time per page for the fast copier is thought to be lognormally distributed with a mean of 1.6 seconds and a standard deviation of 0.3 seconds (stream 2). A co-op Industrial Engineering student has collected some time study data on the time to copy a page for the slow copier. The times, in seconds, are given in the data set associated with Exercise B.13. Use stream 3 for the fitted distribution.

The copy times for the slow and fast copiers are given on a per page basis. Thus, the total time to perform a copy job of N pages is the sum of the copy times for the N individual pages. Each individual page's time is random.

Customers arrive to the copy center according to a Poisson process with a mean rate of 1 customer every 40 seconds (stream 1). The number of copies requested by each customer is equally likely over the range of 10 and 50 copies (stream 4). The customer is responsible for filling out a form that indicates the number of copies to be made. This results in a copy job which is processed by the copying machines in the copy center. The copying machines work on the entire job at one time.

The policy for selecting a copier is as follows: If the number of copies requested is less than or equal to 30, the slow copier will be used. If the number of copies exceeds 30, the fast copier will be used, with one exception: If no jobs are in queue on the slow copier and the number of jobs waiting for the fast copier is at least two, then the customer will be served by the slow copier. After the customer gives the originals for copying, the

customer proceeds to the service counter to pay for the copying. Assume that giving the originals for copying requires no time and thus does not require action by the copy center personnel. In addition, assume that one cashier handles the payment counter only so that sufficient workers are available to run the copy machines. The time to complete the payment transaction is lognormally distributed with a mean of 20 seconds and a standard deviation of 10 seconds (stream 5). As soon as both the payment and the copying job are finished, the customer takes the copies and departs the copying center. The copy center starts out a day with no customers and is open for 10 hours per day.

Management has requested that the co-op Industrial Engineer develop a model because they are concerned that customers have to wait too long for copies. Recently, several customers complained about long waits. Their standard is that the probability that a customer waits longer than 4 minutes should be no more than 10%. They define a customer's waiting time as the time interval from when the customer enters the store to the time the customer leaves the store with their completed copy job. If the waiting time criteria is not met, several options are available: The policy for allocating jobs to the fast copier could be modified or the company could purchase an additional copier which could be either a slow copier or a fast copier.

Develop a model for this problem. Based on 25 replications, report in table form, the appropriate statistics on the waiting time of customers, the daily throughput of the copy center, and the utilization of the payment clerk. In addition estimate the probability that a customer spends in the system is longer than 4 minutes.

Exercise 6.33. Passengers arrive at an airline terminal according to an exponential distribution for the time between arrivals with a mean of 1.5 minutes (stream 1). Of the arriving passengers 7% are frequent flyers (stream 2). The time that it takes the passenger to walk from the main

entrance to the check-in counter is uniform between 2.5 and 3.5 minutes (stream 3). Once at the counter the travelers must wait in a single line until one of four agents is available to serve them. The check-in time (in minutes) is Gamma distributed with a mean of 5 minutes and a standard deviation of 4 minutes (stream 4). When their check-in is completed, passengers with carry-on only go directly to security. Those with a bag to check, walk to another counter to drop their bag. The time to walk to bag check is uniform(1,2) minutes (stream 9). Since the majority of the flyers are business, only 25% of the travelers have a bag to check (stream 5). At the baggage check, there is a single line served by one agent. The time to drop off the bag at the bag check stations is lognormally distributed with a mean of 2 minutes and a standard deviation of 1 minute (stream 6). After dropping off their bags, the traveler goes to security. The time to walk to security (either after bag check or directly from the check in counter) is exponentially distributed with a mean of 8 minutes (stream 10). At the security check point, there is a single line, served by two TSA agents. The TSA agents check the boarding passes of the passengers. The time that it takes to check the boarding pass is triangularly distributed with parameters (2, 3, 4) minutes (stream 7). After getting their identity checked, the travelers go through the screening process. We are not interested in the screening process. However, the time that it takes to get through screening is distributed according to a triangular distribution with parameters (5, 7, 9) minutes (stream 8). After screening, the walking time to the passenger's gate is exponentially distributed with a mean of 5 minutes (stream 11).

We are interested in estimating the average time it takes from arriving at the terminal until a passenger gets to their gate. This should be measured overall and by type (frequent flyer versus non-frequent flyer).

Assume that the system should be studied for 16 hours per day.

a. Report the average and 95% confidence interval half-width on the following based on the simulation of 10 days of operation. Report all time units in minutes.

Statistic	Average	Half-Width
Utilization of the check-in agents		
Utilization of the TSA agents		
Utilization of the Bag Check agent		
Frequent Flyer Total time to Gate		
Non-Frequent Flyer Total time to Gate		
Total time to Gate regardless of type		
Number of travelers in the system		
Number of travelers waiting for check-in		
Number of travelers waiting for security		
Time spent waiting for check-in		
Time spent waiting for security		

b. Based on the results of part (a) determine the number of replications necessary to estimate the total time to reach their gate regardless of type to within $\pm$ 1 minute with 95% confidence.

Exercise 6.34. Consider the soccer mom and daughter example of Section 6.4.5. Use the `Signal` class to implement the example. Use print statements to prove that the interaction is the same as the implementation with the `Suspension` class.

Exercise 6.35. Consider the soccer mom and daughter example of Section 6.4.5. Update the `Blockage` example to implement the example. Use print statements to prove that the interaction is the same as the implementation with the `Suspension` class.

Exercise 6.36. Consider the soccer mom and daughter example of Section 6.4.5. Update the `BlockingActivity` example to implement the example. Use print statements to prove that the interaction is the same as the implementation with the `Suspension` class.

Exercise 6.37. Consider the Tie-Dye T-Shirt Example 6.9.

a. Redo the example using the `HoldQueue` class to model the process interaction.
b. Redo the example using blockages to model the process interaction.

---

7. Advanced Event and Process View Modeling

Learning Objectives

- To be able to model complex systems effectively with processes and resources.
- To be able to model non-stationary arrivals using arrival schedules.
- To be able to model the staffing/scheduling of resources using resource capacity schedules.
- To be able to capture statistics over specific periods of time.
- To be able to model balking and reneging within queueing situations.
- To be able to model complex systems effectively with events.
- To be able to apply basic verification and validation methods within simulation.

The primary purpose of this chapter is to dive deeper into more complex modeling situations that are commonly experienced within discrete-event dynamic systems. First we will explore how to use the KSL process view constructs to model the common situation of a station where work is performed. Because the KSL is built upon and can take advantage of the flow of control and data structures available within Kotlin, we will see that fairly complex systems can be readily modeled using the primitive elements discussed in the previous chapter. In addition, when modeling a system,

there may be situations where there is limited waiting space for the entities to be processed or we need to maintain some distance between entities that wait (such as social distance requirements) This situation presents a number of interesting modeling possibilities. We will look at a way to use resources to handle this situation.

Many realistic modeling situations have non-stationary characteristics that require the generation of arrivals where the mean rate of arrival depends on the time, such as lunch hour at your favorite fast food restaurant. We will see how the KSL provides constructs for this situation. In addition, if the arrivals to a system vary with time, we may want to vary the capacity of the resources with respect to time. The KSL also provides some basic ways to address this situation. In addition, because the system performance can be non-stationary, we may want to collect statistics during specific intervals of time. The KSL has the ability to specify the collect of responses during specific intervals according to a schedule.

Lastly, the chapter will look at a few miscellaneous modeling situations that should help you to develop ideas for effective modeling of a variety of situations. We will start by looking more closely at processes and resources.

> **Note**
>
> This chapter provides a series of example Kotlin code that illustrates the use of KSL constructs for implementing process and event view simulation models. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

7.1. Modeling with Processes and Resources

In this section, we explore some of the nuances of modeling systems via process and resources. Specifically, we will look more carefully at the KSL constructs for modeling resources. In general, a resource is something that is required or used by the entities within a process. In real systems, whenever there is requirement for an entity to wait, it is likely that a resource is needed. Within the KSL, a resource has number of identical units which represent its capacity. For the purposes of usage, the units of the resource are indistinguishable. That is, there can be no requirement to use one of the units over some other of the units. For the entity, it does not matter which unit is supplied for a request. It only matters that it gets the requested number of units from the resource. Resource units are allocated and deallocated to entities that request them. If a request for a specific number of units of the resource cannot be allocated immediately upon request, then the entity (generally) must wait for the units (or not proceed with the request). If there is a preference between types of resources, then a pool of resources should be used. Pools of resources will also be discussed in this section. We begin our study of resources by illustrating how resources can be used to model space.

7.1.1. Modeling Space with Resources

We start the modeling of space with resources by looking at a simple system involving a tandem queue. A tandem queue is a sequence of queues in order that must be visited to receive service from resources. Let's setup the situation by first modeling the system without a space requirement between the queues. The following example presents the specifics of the situation.

Example 7.1 (Tandem Queueing System)**.** Suppose a service facility consists of two stations in series (tandem), each with its own FIFO queue. Each station consists of a queue and a single server. A customer completing service at station 1 proceeds to station 2, while a customer completing service at station 2 leaves the facility. Assume that the inter-arrival times of customers to station 1 are IID exponential random variables with a mean of 1 minute. Service times of customers at station 1 are exponential random variables with a mean of 0.7 minute, and at station 2 are exponential random variables with mean 0.9 minute. Develop a model for this system. Run the simulation for exactly 20000 minutes and estimate for each station the expected average delay in queue for the customer, the expected time-average number of customers in queue, and the expected utilization. In addition, estimate the average number of customers in the system and the average time spent in the system.

To model this situation, we need to use two resources, one for each server at the two stations.

Figure 7.1 illustrates this situation. This is a perfect application of what we have previously studied about modeling processes. The following code sets up the KSL constructs that are needed within the process modeling.

```
class TandemQueue(parent: ModelElement, name: String? = null)
 ↪  : ProcessModel(parent, name)  {

    private val worker1: ResourceWithQ = ResourceWithQ(this,
     ↪  "worker1")
    private val worker2: ResourceWithQ = ResourceWithQ(this,
     ↪  "worker2")

    private val tba = ExponentialRV(2.0, 1)
```

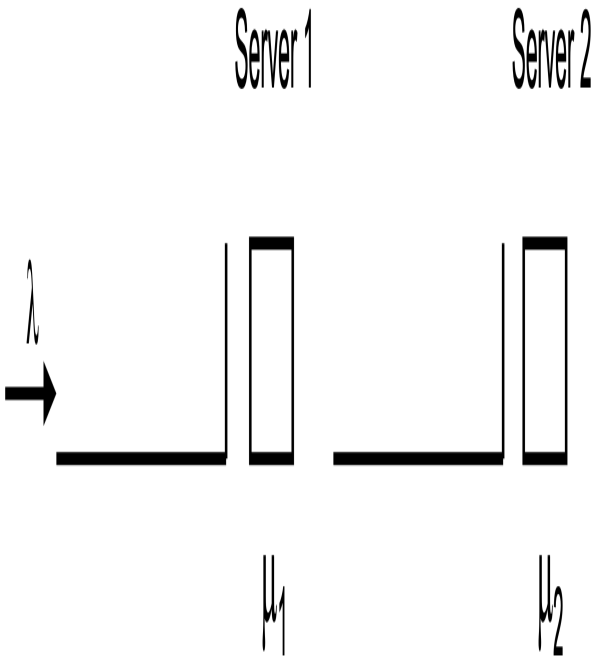


Figure 7.1.: Tandem Queue

```
    private val st1 = RandomVariable(this, ExponentialRV(0.7,
     ↪  2))
    val service1RV: RandomSourceCIfc
        get() = st1
    private val st2 = RandomVariable(this, ExponentialRV(0.9,
     ↪  3))
    val service2RV: RandomSourceCIfc
        get() = st2
    private val myArrivalGenerator =
     ↪  EntityGenerator(::Customer, tba, tba)
    val generator: EventGeneratorCIfc
        get() = myArrivalGenerator

    private val wip: TWResponse = TWResponse(this,
     ↪  "${this.name}:NumInSystem")
    val numInSystem: TWResponseCIfc
        get() = wip
    private val timeInSystem: Response = Response(this,
     ↪  "${this.name}:TimeInSystem")
    val systemTime: ResponseCIfc
        get() = timeInSystem
```

Since the resources will need a queue, we declare the two resources by using the `ResourceWithQ` class. Then, we specify the random variables that will be used to represent the time between arrivals and the service time at the two stations. Finally, we define the response variables for collecting the number in the system and the time in the system. The process modeling is a straight-forward application of the *(seize-delay-release)* pattern that was presented in the previous chapter.

```
    private inner class Customer : Entity(){
        val tandemQProcess : KSLProcess =
         ↪  process(isDefaultProcess = true) {
            wip.increment()
            timeStamp = time
            seize(worker1)
            delay(st1)
            release(worker1)
            seize(worker2)
            delay(st2)
            release(worker2)
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

Notice that this code does not save the allocations that are returned by the call to the `seize` method. This is possible because of two reasons. First, the KSL overloads the `release` method to allow for the specification of the resource to be released. This overloaded method will release **all** previously returned allocations that the entity holds of the named resource. Secondly, this situation is a perfect use case for this, because the entity only has seized the worker *once* before the specified release. Thus, we are simply releasing the last allocation held by the entity via the `release()` function. It really is user preference whether to save the allocation and then release the allocation or to use the approach of specifying the name of the resource. Of course, this works because releasing **all** previous allocations is the same as releasing the last one in this situation. In fact, this situation is so common that the KSL provides an additional short cut as illustrated in the following code.

```
private inner class Customer : Entity(){
    val tandemQProcess : KSLProcess =
     ↪  process(isDefaultProcess = true) {
        wip.increment()
        timeStamp = time
        use(worker1, delayDuration = st1)
        use(worker2, delayDuration = st2)
        timeInSystem.value = time - timeStamp
        wip.decrement()
    }
}
```

The `use` method combines the *(seize-delay-release)* pattern into one method call. The results are not very remarkable.

Name	Count	Average	Half-Width
worker1:InstantaneousUtil	30	0.351	0.002
worker1:NumBusyUnits	30	0.351	0.002
worker1:ScheduledUtil	30	0.351	0.002
worker1:WIP	30	0.54	0.007
worker1:Q:NumInQ	30	0.189	0.005
worker1:Q:TimeInQ	30	0.379	0.008
worker2:InstantaneousUtil	30	0.45	0.003
worker2:NumBusyUnits	30	0.45	0.003
worker2:ScheduledUtil	30	0.45	0.003
worker2:WIP	30	0.82	0.01
worker2:Q:NumInQ	30	0.37	0.008
worker2:Q:TimeInQ	30	0.741	0.014
TandemQModel:NumInSystem	30	1.36	0.014
TandemQModel:TimeInSystem	30	2.723	0.02
worker1:SeizeCount	30	7492.333	32.526
worker2:SeizeCount	30	7492.533	32.356

Now we are ready to study the situation of modeling finite space between the two stations.

Example 7.2 (Tandem Queueing System With Blocking)**.** Imagine that at the first station there is a chair for the customer to sit in while receiving service from the first worker. Any customers that arrive while a customer is receiving service at the first station must wait for the server to be free (i.e. the chair to be available). We assume that there is (at least conceptually) an infinite amount of space for the waiting customers at the first station. Now, at the second station, there are two chairs. The customer arriving to the second station will sit in the first chair when receiving service from the server. The second chair is provided for one waiting customer and there is no space for any other customers to wait at the second station. Thus, a customer finishing service at the first station cannot move into (use) the second station if there is a customer waiting at the second station. If a customer at the first station cannot move into the waiting line (2nd chair) at the second station, then the customer is considered blocked. What does this customer do? Well, they are selfish and do not give up their current chair until they can get a chair at the second station. Thus, waiting at the second station may cause waiting to occur at the first station. This situation is called a tandem queue with blocking, as illustrated in Figure 7.2.

Let's see how to model this situation using KSL constructs. The key is to model the chair that represents the waiting line at the second station with a resource. This is essentially modeling the waiting space with a resource. In the above scenario, there was one space for waiting.

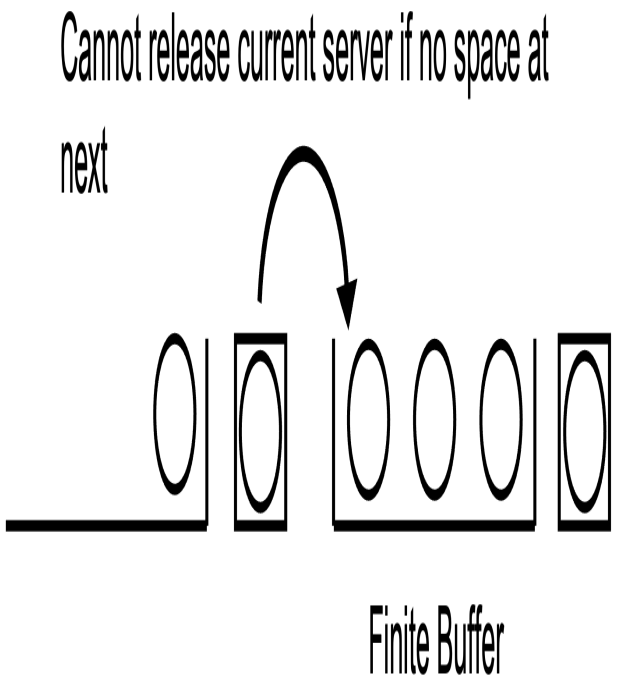


Figure 7.2.: Tandem Queue with Blocking

```
class TandemQueueWithBlocking(parent: ModelElement, name:
 ↪  String? = null) : ProcessModel(parent, name) {

    private val buffer: ResourceWithQ = ResourceWithQ(this,
     ↪  "buffer", capacity = 1)
    private val worker1: ResourceWithQ = ResourceWithQ(this,
     ↪  "worker1")
    private val worker2: ResourceWithQ = ResourceWithQ(this,
     ↪  "worker2")
```

In this code, we defined a resource called `buffer` with capacity 1 to represent the chair designated for waiting. By changing the capacity of this resource, we can study the effect of the limited space at the second station on system performance.

Exploring the effect of the buffer size is left as an exercise for the reader. The process modeling needs to be adjusted to account for this space. In the following code, notice the overlapping nature of the seize and release statements.

```
    private inner class Customer : Entity() {
        val tandemQProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            wip.increment()
            timeStamp = time
            val a1 = seize(worker1)
            delay(st1)
            val b = seize(buffer)
            release(a1)
            val a2 = seize(worker2)
            release(b)
            delay(st2)
            release(a2)
```

```
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

After receiving service at the first station, the customer attempts to seize the buffer (chair for waiting) at the second station. If there is space in the buffer, **then** the customer releases the first worker. Don't give up your chair until you get the next chair! After moving into the chair (buffer), the customer attempts to seize the second worker. If the second worker is not available the customer waits; otherwise, the customer delays for service at the second station, releases the second worker, and then departs that system. Again the key is to have this overlapping seize and release statements.

Figure 7.3 provides the activity diagram for this situation. Notice how the arrows for the seize and release of the resources overlap.

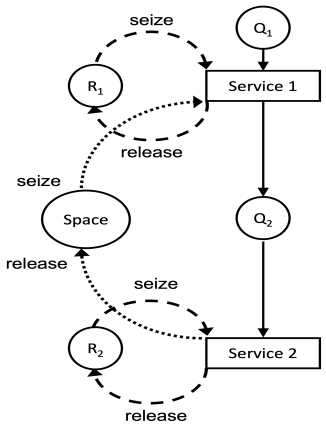


Figure 7.3.: Activity Diagram Tandem Queue with Blocking

For the given arrival rate and service parameters, the results indicate that the effect of blocking is not too significant.

Name	Count	Average	Half-Width
buffer:InstantaneousUtil	30	0.196	0.003
buffer:NumBusyUnits	30	0.196	0.003
buffer:ScheduledUtil	30	0.196	0.003
buffer:WIP	30	0.267	0.004
buffer:Q:NumInQ	30	0.071	0.002
buffer:Q:TimeInQ	30	0.143	0.003
worker1:InstantaneousUtil	30	0.422	0.003
worker1:NumBusyUnits	30	0.422	0.003
worker1:ScheduledUtil	30	0.422	0.003
worker1:WIP	30	0.761	0.012
worker1:Q:NumInQ	30	0.339	0.009
worker1:Q:TimeInQ	30	0.678	0.016
worker2:InstantaneousUtil	30	0.45	0.003
worker2:NumBusyUnits	30	0.45	0.003
worker2:ScheduledUtil	30	0.45	0.003
worker2:WIP	30	0.646	0.005
worker2:Q:NumInQ	30	0.196	0.003
worker2:Q:TimeInQ	30	0.392	0.004
TandemQModelWithBlocking:NumInSystem	30	1.406	0.016
TandemQModelWithBlocking:TimeInSystem	30	2.815	0.023
buffer:SeizeCount	30	7492.467	32.384
worker1:SeizeCount	30	7492.333	32.406
worker2:SeizeCount	30	7492.5	32.367

However, the exercises ask the reader to explore what happens if the arrival rate is increased. A tandem queueing system is just a series of stations. The concept of having stations where work is performed is very useful. A later section illustrates how to generalize these ideas, but first we explore

how to organize resources into sets or pools from which resources can be selected.

7.1.2. Resource Pools

A resource pool is a generalization of the concept of a resource that permits individual instances of the `Resource` class to be combined together into a larger *pool* of units. Resource pools facilitate the sharing of instances of resources across processes. The important concepts involved in using resource pools are 1) how to select resources to satisfy a request, and 2) how to allocate a request for units across the pool of resources. For example, if a request for units of the pool was for 2 units, and the pool contained 3 individual resources all of capacity 1, which of the 3 resources should be selected to provide the requested units? Also, suppose, for example, the request was for 2 units, and there were 3 individual resources with capacity (1, 2, 3) units, respectively. Should the resource with capacity 2 be used? Should the resource with capacity 3 be used? Should the resource with capacity 1 be used in combination with one of the other resources? As you can see, there may be many possible ways to allocate units to requests when there is a pool of resources. The KSL provides a structure for users to supply selection and allocation rules when using pools of resources through some interfaces.

The `ResourceSelectionRuleIfc` interface provides a mechanism for selecting resources from the pool that can satisfy the request. If not empty, list returned from the `ResourceSelectionRuleIfc` interface should have sufficient units available to satisfy the amount needed. In fact, it may have more than what is needed.

```
/**
 * Provides for a method to select resources from a list such
 ↪  that
 * the returned list will contain resources that can fully
 ↪  fill the amount needed
```

```
 * or the list will be empty.
 */
fun interface ResourceSelectionRuleIfc {
    /**
     * @param amountNeeded the amount needed from resources
     * @param list of resources to consider selecting from
     * @return the selected list of resources. It may be
 ↪  empty
     */
    fun selectResources(amountNeeded: Int, list:
     ↪  List<Resource>): MutableList<Resource>
}
```

The `AllocationRuleIfc` interface provides a mechanism for selecting resources from a list of resources that can satisfy the request. In essence, the `ResourceSelectionRuleIfc` interface selects the possible resources to allocated units from and supplies them to the allocation rule for allocation. As noted in the documentation, the supplied list must have sufficient units to meet the allocation request.

```

/**
 *  Function to determine how to allocate requirement for
 ↪  units across
 *  a list of resources that have sufficient available units
 ↪  to meet
 *  the amount needed.
 */
fun interface AllocationRuleIfc {

    /** The method assumes that the provided list of
     ↪  resources has
     *  enough units available to satisfy the needs of the
 ↪  request.
```

```
     *
     * @param amountNeeded the amount needed from resources
     * @param resourceList list of resources to be allocated
 ↪  from
     * @return the amount to allocate from each resource as a
 ↪  map
     */
    fun makeAllocations(amountNeeded: Int, resourceList:
     ↪  List<Resource>): Map<Resource, Int>
}
```

These two interfaces can be used in combination to form various selection and allocation possibilities for a variety of resource pool situations. The `ResourcePool` and `ResourcePoolWithQ` classes use default implementations of these functions. The KSL provides two implementations of the `ResourceSelectionRuleIfc` interface.

- `FirstFullyAvailableResource` selects the first resource from a supplied list that can fully meet the request. The returned list will either be empty or have one resource (that can meet the request).
- `ResourceSelectionRule` selects a list of resources that (in total) have enough available units to fully meet the request.

It is important to note that the `ResourceSelectionRuleIfc` interface may return an empty list if the request cannot be met. This is used to determine if the entity must wait.

The KSL provides a default instance of the `AllocationRuleIfc` interface called the `AllocateInOrderListedRule` rule. This rule takes in a list of resources that in total has enough units available and allocates from each listed resource (in the order listed) until the entire amount requested is filled. Thus, in both the selection rule and the allocation rule, the order of the resources within the pool are important. Again, if you want or need to have different rules, then you can implement these interfaces and supply

your instances to the `ResourcePool` and `ResourcePoolWithQ` classes to use instead of the default implementations.

The following allocation rules are available:

- `AllocateInOrderListedRule` This rule allocates all available units from each resource until amount needed is met based on the order in which the resources are listed from the selection rule. This is the default allocation rule.
- `RandomAllocationRule` This rule first randomly permutes the list from the selection rule and then allocates in the order of the permutation. In essence, this approach randomly picks from the list.
- `ResourceAllocationRule` The `ResourceAllocationRule` class facilitates the construction of other rules based on a supplied resource comparator. The following rules are based on using different resource comparators.
  - `LeastUtilizedAllocationRule` This rule sorts the resources such that list is ordered from least to most utilized and then allocates in the order listed.
  - `LeastSeizedAllocationRule` This rule sorts the resources such that this is ordered from least seized to most seized and then allocates in the order listed.
  - `MostAvailableAllocationRule` When the resources have capacity greater than one, then the resources are sorted from most capacity available to least capacity available, and then allocated in the order listed.

The KSL also provides a number of functions that can be useful when comparing or searching for resources. These functions can be found in the `ResourceRules` file along with the previously mentioned interfaces.

Let's take a look at an example situation involving resource pools.

Example 7.3 (Resource Pools)**.** In this example, there are two pools. The first pool will have 3 resources (john, paul, and george) and the second pool will have 2 resources (ringo and george). One of the resources (george) is shared (in common) between the two pools. The following code creates the four resources, adds them to lists, and then supplies the lists to instances of the `ResourcePoolWithQ` class.

```
class ResourcePoolExample(parent: ModelElement) :
 ↪  ProcessModel(parent, null) {

    private val john = Resource(this, name = "John")
    private val paul = Resource(this, name = "Paul")
    private val george = Resource(this, name = "George")
    private val ringo = Resource(this, name = "Ringo")
    private val list1 = listOf(john, paul, george)
    private val list2 = listOf(ringo, george)
    private val pool1: ResourcePoolWithQ =
     ↪  ResourcePoolWithQ(this, list1, name = "pool1")
    private val pool2: ResourcePoolWithQ =
     ↪  ResourcePoolWithQ(this, list2, name = "pool2")
    private val tba = RandomVariable(this, ExponentialRV(1.0,
     ↪  1), "Arrival RV")
    private val st = RandomVariable(this, ExponentialRV(3.0,
     ↪  2), "Service RV")
    private val decideProcess = RandomVariable(this,
     ↪  BernoulliRV(0.7, 3))
    private val wip1 = TWResponse(this, "${name}:WIP1")
    private val tip1 = Response(this,
     ↪  "${name}:TimeInSystem1")
    private val wip2 = TWResponse(this, "${name}:WIP2")
    private val tip2 = Response(this,
     ↪  "${name}:TimeInSystem2")
```

```
    private val generator = EventGenerator(this,
     ↪  this::arrivals, tba, tba)
```

The two pools are shared between two processes using straightforward *(seize-delay-release)* logic.

```
    private inner class Customer: Entity() {
        val usePool1: KSLProcess = process("Pool 1 Process")
         ↪  {
            wip1.increment()
            timeStamp = time
            val a  = seize(pool1, 1)
            delay(st)
            release(a)
            tip1.value = time - timeStamp
            wip1.decrement()
        }

        val usePool2: KSLProcess = process("Pool 2 Process")
         ↪  {
            wip2.increment()
            timeStamp = time
            val a  = seize(pool2, 1)
            delay(st)
            release(a)
            tip2.value = time - timeStamp
            wip2.decrement()
        }
    }
```

In this example, we randomly activate the two processes based on a distribution.

```
private fun arrivals(generator: EventGenerator){
    val c = Customer()
    if (decideProcess.value.toBoolean()){
        activate(c.usePool1)
    } else {
        activate(c.usePool2)
    }
}
```

Since there are four resources and two pools, the performance reports on all the individual resources usage as well as the overall performance of the pool.

Resource Pool Example Statistical Summary Report

Name	Count	Average	Half-Width
John:InstantaneousUtil	30	0.663	0.003
John:NumBusyUnits	30	0.663	0.003
John:ScheduledUtil	30	0.663	0.003
Paul:InstantaneousUtil	30	0.663	0.003
Paul:NumBusyUnits	30	0.663	0.003
Paul:ScheduledUtil	30	0.663	0.003
George:InstantaneousUtil	30	0.834	0.003
George:NumBusyUnits	30	0.834	0.003
George:ScheduledUtil	30	0.834	0.003
Ringo:InstantaneousUtil	30	0.833	0.003
Ringo:NumBusyUnits	30	0.833	0.003
Ringo:ScheduledUtil	30	0.833	0.003
pool1:NumBusy	30	2.16	0.007
pool1:FractionBusy	30	0.72	0.002
pool1:Q:NumInQ	30	0.851	0.023
pool1:Q:TimeInQ	30	1.708	0.044
pool2:NumBusy	30	1.666	0.007

Name	Count	Average	Half-Width
pool2:FractionBusy	30	0.833	0.003
pool2:Q:NumInQ	30	2.631	0.101
pool2:Q:TimeInQ	30	5.253	0.193
ResourcePoolExample_3:WIP1	30	2.34	0.028
ResourcePoolExample_3:TimeInSystem1	30	4.698	0.05
ResourcePoolExample_3:WIP2	30	4.134	0.108
ResourcePoolExample_3:TimeInSystem2	30	8.255	0.201
John:SeizeCount	30	3318.533	19.247
Paul:SeizeCount	30	3335.833	24.126
George:SeizeCount	30	4174.967	24.294
Ringo:SeizeCount	30	4151.9	23.655

In the output, we see that resource `george` has the highest utilization. This is likely due to the fact that `george` is shared between the two pools. Resource pools can be helpful when modeling the sharing of resources between activities. The example created four instances of the `Resource` class. This resource does not have a queue automatically associated with it. Then, two resource pools were created to hold the resource instances. Notice that the pool were instances of the `ResourcePoolWithQ` class. This creates a pool and a queue to hold requests made to the pool. The queue associated with the pool is also registered as being associated with the resources in the pool. Requests for the pool will wait in the pool's queue until the request can be allocated. The default selection and allocation rules were used in the example. To change the rule, the user should supply the initial resource selection and allocation rules as shown in the following code.

```
    var initialDefaultResourceSelectionRule:
     ↪ ResourceSelectionRuleIfc = ResourceSelectionRule()
        set(value) {
            require(model.isNotRunning) {"Changing the
↪ initial resource selection rule during a replication will
↪ cause replications to not have the same starting
↪ conditions"}
```

```
            field = value
        }

    var defaultResourceSelectionRule:
     ↪  ResourceSelectionRuleIfc =
     ↪  initialDefaultResourceSelectionRule
        set(value) {
            field = value
            if (model.isRunning){
                Model.logger.warn { "Changing the initial
↪  resource selection rule during a replication will only
↪  effect the current replication." }
            }
        }

    var initialDefaultResourceAllocationRule:
     ↪  ResourceAllocationRuleIfc =
     ↪  AllocateInOrderListedRule()
        set(value) {
            require(model.isNotRunning) {"Changing the
↪  initial resource allocation rule during a replication
↪  will cause replications to not have the same starting
↪  conditions"}
            field = value
        }

    var defaultResourceAllocationRule:
     ↪  ResourceAllocationRuleIfc =
     ↪  initialDefaultResourceAllocationRule
        set(value) {
            field = value
            if (model.isRunning){
                Model.logger.warn { "Changing the initial
↪  resource allocation rule during a replication will only
↪  effect the current replication." }
```

```
            }
        }
```

It is useful to note that the rules can be changed during the replication, but that they will be returned to the initial setting at the beginning of each replication. This is done to ensure that every replication starts with the same initial conditions. The `ResourcePoolCIfc` interface provides a public interface for resource pools.

```
interface ResourcePoolCIfc {
    val numBusyUnits: TWResponseCIfc
    val fractionBusyUnits: ResponseCIfc
    val resources: List<ResourceCIfc>
    val numAvailableUnits: Int
    val hasAvailableUnits: Boolean
    val capacity: Int
    val numBusy: Int
    val fractionBusy: Double
    var initialDefaultResourceSelectionRule:
     ↪ ResourceSelectionRuleIfc
    var initialDefaultResourceAllocationRule:
     ↪ ResourceAllocationRuleIfc
}
```

Note

When using the `seize()` function to seize a resource pool, you are actually making a request for the resources within the pool. If the request is not immediately filled, then the request waits in a queue. Since we used a `ResourcePoolWithQ` class to model the resource pool, the request will wait in the queue associated with the resource pool. The `seize()` function returns an instance of the

`ResourcePoolAllocation` class. The returned resource allocation has properties that indicate which queue and which resources were involved with the request. For a `ResourcePoolAllocation` there is a property called allocations, which represents a list of the allocations made from the pool. The list of allocations can be used to access the allocations, with each allocation representing an individual allocation of a resource from the pool. When seizing a resource, you can specify the number of units needed. The request for 1 unit from a resource pool results in the allocation from a single resource. Thus, the list of allocations will have 1 and only 1 allocation. The request for more than 1 unit from a resource pool, may result in allocations from more than one resource. The allocations list holds the allocations of any resources that were allocated.

The section introduced the concept of resource pools with a simple example. Later in the book, we will revisit the use of pools, especially within the context of being able to change the capacity of resources. In the next section, we discuss a more complex situation involving a flow shop.

7.1.3. Computer Test and Repair Shop Example

This section presents a common modeling situation in which entities follow a processing plan until they are completed. The KSL makes this type of modeling easy because it can leverage the full functionality of the Kotlin language.

Example 7.4 (Computer Test and Repair Shop)**.** Consider a test and repair shop for computer parts (e.g. circuit boards, hard drives, etc.) The system consists of an initial diagnostic station through which all newly arriving parts must be processed. Currently, newly arriving parts arrive

according to a Poisson arrival process with a mean rate of 3 per hour. The diagnostic station consists of 2 diagnostic machines that are fed the arriving parts from a single queue. Data indicates that the diagnostic time is quite variable and follows an exponential distribution with a mean of 30 minutes. Based on the results of the diagnostics, a testing plan is formulated for the parts. There are currently three testing stations 1. 2, 3 which consist of one machine each. The testing plan consists of an ordered sequence of testing stations that must be visited by the part prior to proceeding to a repair station. Because the diagnosis often involves similar problems, there are common sequences that occur for the parts. The company collected extensive data on the visit sequences for the parts and found that the sequences in Table 7.1 constituted the vast majority of test plans for the parts.

Table 7.1.: Test Plan Sequences

Test Plan	% of Parts	Sequence
1	25%	2,3,2,1
2	12.5%	3,1
3	37.5%	1,3,1
4	25%	2,3

For example, 25% of the newly arriving parts follow test plan 1, which consists of visiting test stations 2, 3, 2, and 1 prior to proceeding to the repair station.

The testing of the parts at each station takes time that may depend upon the sequence that the part follows. That is, while parts that follow test plan's 1 and 3 both visit test station 1, data shows that the time spent processing at the station is not necessarily the same. Data on the testing times indicate that the distribution is well modeled with a lognormal

distribution with mean, μ, and standard deviation, σ in minutes. Table 7.2 presents the mean and standard deviation for each of the testing time distributions for each station in the test plan.

Table 7.2.: Testing and Repair Distributions

Test Plan	Testing Time Parameters	Repair Time Parameters
1	(20,4.1), (12,4.2), (18,4.3), (16,4.0)	(30,60,80)
2	(12,4), (15,4)	(45,55,70)
3	(18,4.2), (14,4.4), (12,4.3)	(30,40,60)
4	(24,4), (30,4)	(35,65,75)

For example, the first pair of parameters, (20, 4.1), for test plan 1 indicates that the testing time at test station 2 has a lognormal distribution with mean,$\mu = 20$, and standard deviation,$\sigma = 4.1$ minutes.

The repair station has 3 workers that attempt to complete the repairs based on the tests. The repair time also depends on the test plan that the part has been following. Data indicates that the repair time can be characterized by a triangular distribution with the minimum, mode, and maximum as specified in the previous table. After the repairs, the parts leave the system. When the parts move between stations assume that there is always a worker available and that the transfer time takes between 2 to 4 minutes uniformly distributed. Figure 7.4 illustrates the arrangement of the stations and the flow of the parts following Plan 2 in the test and repair shop.

The company is considering accepting a new contract that will increase the overall arrival rate of jobs to the system by 10%. They are interested in understanding where the potential bottlenecks are in the system and in developing alternatives to mitigate those bottlenecks so that they can still handle the contract. The new contract stipulates that 80% of the time the testing and repairs should be completed within 480 minutes. The company runs 2 shifts each day for each 5 day work week. Any jobs not

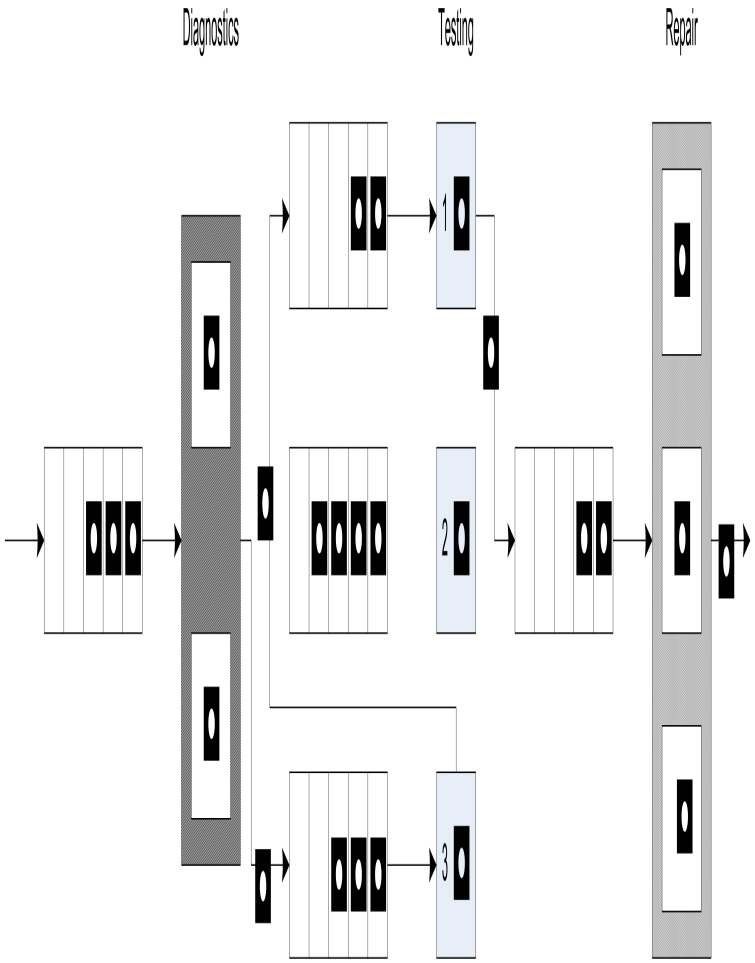


Figure 7.4.: Overview of the Test and Repair Shop

completed at the end of the second shift are carried over to first shift of the next working day. Assume that the contract is going to last for 1 year (52 weeks). Build a simulation model that can assist the company in assessing the risks associated with the new contract.

7.1.3.1. Implementing the Test and Repair Model

Before implementing the model, you should prepare by conceptualizing the process flow. Figure 7.5 illustrates the activity diagram for the test and repair system. Parts are created and flow first to the diagnostic station where they seize a diagnostic machine while the diagnostic activity occurs. Then, the test plan is assigned. The flow for the visitation of the parts to the test station is shown with a loop back to the transfer time between the stations. It should be clear that the activity diagram is representing any of the three test stations. After the final test station in the test plan has been visited, the part goes to the repair station, where 1 of 3 repair workers is seized for the repair activity. After the repair activity, the part leaves the system.

In many modeling contexts, entities will follow a specific path through the system. In a manufacturing job shop, this is often called the process plan. In a bus system, this is called a bus route. In the test and repair system, this is referred to as the test plan. To model a specify path through the system, we need an approach to specify a sequence of stations. A sequence consists of an ordered list of *steps*. Each step must indicate the resources and other information associated with the step. Thus, a sequence is built by simply providing the list of steps that must be visited. We will use Kotlin lists to hold this information. Let's take a look at the implementation.

Because of the requirement that different parts follow different sequences and have different processing times based on what resource they are using and where they are in the sequence, there are many random variables that need to be defined for this model.

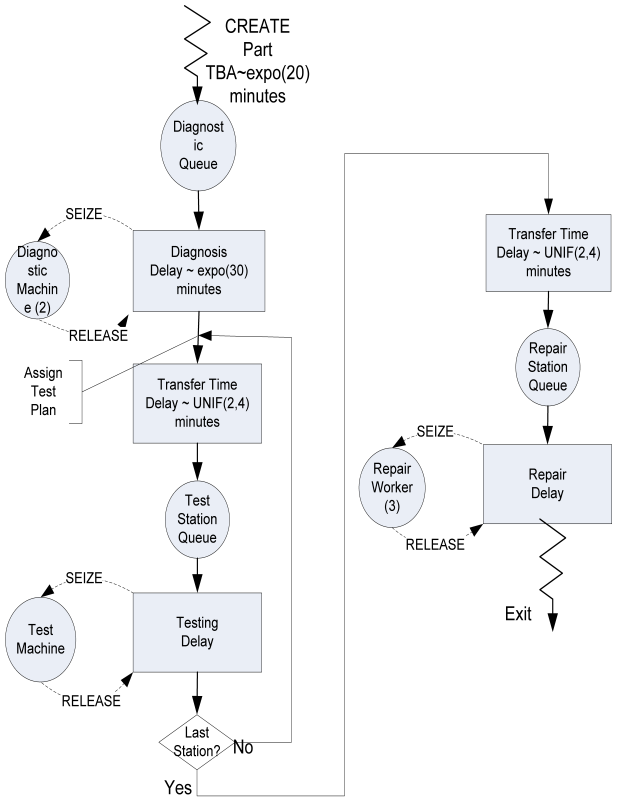


Figure 7.5.: Activity Diagram for Test and Repair System

```
class TestAndRepairShop(parent: ModelElement, name: String?
 ↪  = null) : ProcessModel(parent, name) {

    // test plan 1, distribution j
    private val t11 = RandomVariable(this, LognormalRV(20.0,
     ↪  4.1 * 4.1))
    private val t12 = RandomVariable(this, LognormalRV(12.0,
     ↪  4.2 * 4.2))
    private val t13 = RandomVariable(this, LognormalRV(18.0,
     ↪  4.3 * 4.3))
    private val t14 = RandomVariable(this, LognormalRV(16.0,
     ↪  4.0 * 4.0))
    // test plan 2, distribution j
    private val t21 = RandomVariable(this, LognormalRV(12.0,
     ↪  4.0 * 4.0))
    private val t22 = RandomVariable(this, LognormalRV(15.0,
     ↪  4.0 * 4.0))
    // test plan 3, distribution j
    private val t31 = RandomVariable(this, LognormalRV(18.0,
     ↪  4.2 * 4.2))
    private val t32 = RandomVariable(this, LognormalRV(14.0,
     ↪  4.4 * 4.4))
    private val t33 = RandomVariable(this, LognormalRV(12.0,
     ↪  4.3 * 4.3))
    // test plan 4, distribution j
    private val t41 = RandomVariable(this, LognormalRV(24.0,
     ↪  4.0 * 4.0))
    private val t42 = RandomVariable(this, LognormalRV(30.0,
     ↪  4.0 * 4.0))

    private val r1 = RandomVariable(this, TriangularRV(30.0,
     ↪  60.0, 80.0))
    private val r2 = RandomVariable(this, TriangularRV(45.0,
     ↪  55.0, 70.0))
```

```
    private val r3 = RandomVariable(this, TriangularRV(30.0,
     ↪  40.0, 60.0))
    private val r4 = RandomVariable(this, TriangularRV(35.0,
     ↪  65.0, 75.0))
    private val diagnosticTime = RandomVariable(this,
     ↪  ExponentialRV(30.0))
    private val moveTime = RandomVariable(this,
     ↪  UniformRV(2.0, 4.0))
    private val tba = ExponentialRV(20.0)
    private val myArrivalGenerator = EntityGenerator(::Part,
     ↪  tba, tba)
    val generator: EventGeneratorRVCIfc
        get() = myArrivalGenerator
```

The code uses a naming convention to keep track of which random variable is using on which sequence. For example, `t21` is the random variable required for the second step of the first test plan and `r1` is the repair time random variable for the first test plan. For each step of the test plan, we need to know the required resource and the processing time. Thus, we define each of the resources as follows.

```
    // define the resources
    private val myDiagnostics: ResourceWithQ =
     ↪  ResourceWithQ(this, "Diagnostics", capacity = 2)
    private val myTest1: ResourceWithQ = ResourceWithQ(this,
     ↪  "Test1")
    private val myTest2: ResourceWithQ = ResourceWithQ(this,
     ↪  "Test2")
    private val myTest3: ResourceWithQ = ResourceWithQ(this,
     ↪  "Test3")
    private val myRepair: ResourceWithQ = ResourceWithQ(this,
     ↪  "Repair", capacity = 3)
```

Then, we define a class to hold the information for each step and the lists to represent each of the test plans.

```
    inner class TestPlanStep(val resource: ResourceWithQ, val
     ↪  processTime: RandomIfc)

    // make all the plans
    private val testPlan1 = listOf(
        TestPlanStep(myTest2, t11), TestPlanStep(myTest3,
         ↪  t21),
        TestPlanStep(myTest2, t31), TestPlanStep(myTest1,
↪   t41), TestPlanStep(myRepair, r1)
    )
    private val testPlan2 = listOf(
        TestPlanStep(myTest3, t12),
        TestPlanStep(myTest1, t22), TestPlanStep(myRepair,
↪   r2)
    )
    private val testPlan3 = listOf(
        TestPlanStep(myTest1, t13), TestPlanStep(myTest3,
         ↪  t23),
        TestPlanStep(myTest1, t33), TestPlanStep(myRepair,
↪   r3)
    )
    private val testPlan4 = listOf(
        TestPlanStep(myTest2, t14),
        TestPlanStep(myTest3, t24), TestPlanStep(myRepair,
↪   r4)
    )
```

Now that the test plans are defined, we can develop a method for determining which plan is assigned to each part. The situation description provides a distribution associated with the test plans as provided in Table 7.2. We can use a random list (`REmpiricalList`) to model this situation.

```
    // set up the sequences and the random selection of the
     ↪ plan
    private val sequences = listOf(testPlan1, testPlan2,
     ↪ testPlan3, testPlan4)
    private val planCDf = doubleArrayOf(0.25, 0.375, 0.7,
     ↪ 1.0)
    private val planList =
     ↪ REmpiricalList<List<TestPlanStep>>(this, sequences,
     ↪ planCDf)
```

The test plans, which are lists, are added to another list called `sequences`, which will be used to randomly select the test plan according to the discrete empirical distribution as provided by the CDF across the test plans.

To capture the performance of the system, we can use `Response` and `TWResponse` instances.

```
    private val wip: TWResponse = TWResponse(this,
     ↪ "${this.name}:NumInSystem")
    val numInSystem: TWResponseCIfc
        get() = wip
    private val timeInSystem: Response = Response(this,
     ↪ "${this.name}:TimeInSystem")
    val systemTime: ResponseCIfc
        get() = timeInSystem
    private val myContractLimit: IndicatorResponse =
     ↪ IndicatorResponse({ x -> x <= 480.0 }, timeInSystem,
     ↪ "ProbWithinLimit")
    val probWithinLimit: ResponseCIfc
        get() = myContractLimit
```

Notice the use of an `IndicatorResponse` to capture the probability of completing the job within the contract limit. Finally, we can specify the

process description for this situation.

```
    private inner class Part : Entity() {
        val testAndRepairProcess: KSLProcess =
         ↪ process(isDefaultProcess = true) {
            wip.increment()
            timeStamp = time
            //every part goes to diagnostics
            use(myDiagnostics, delayDuration =
↪ diagnosticTime)
            // determine the test plan
            val plan: List<TestPlanStep> = planList.element
            // get the iterator
            val itr = plan.iterator()
            // iterate through the plan
            while (itr.hasNext()) {
                val tp = itr.next()
                use(tp.resource, delayDuration =
↪ tp.processTime)
                if (tp.resource != myRepair) {
                    delay(moveTime)
                }
            }
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

Notice how the KSL process constructs and the Kotlin language combine to implement a fairly complex situation within a short and compact process description. The critical item to note is the use of an instance of an iterator within the process. The variable `plan` is actually randomly generated using the `planList` discrete empirical list. Then, an iterator for this list is retrieved. The iterator is then used to march through the steps of the

assigned test plan within a `while` loop. Notice that we check if we are not at the repair machine. If we are not, we incur the delay to move to the next machine in the test plan. The process also includes the code to collect statistics on the part as it moves through the process. To setup to execute this code, we specify the run length and the number of replications.

```
fun main() {
    val m = Model()
    val tq = TestAndRepairShop(m, name = "TestAndRepair")
    m.numberOfReplications = 10
    m.lengthOfReplication = 52.0* 5.0*2.0*480.0
    m.simulate()
    m.print()
    val r = m.simulationReporter
    r.writeHalfWidthSummaryReportAsMarkDown(KSL.out, df =
 ↪  MarkDown.D3FORMAT)
}
```

The output is quite lengthy so the code captures it as a Markdown table.

Name	Count	Average	Half-Width
Diagnostics:InstantaneousUtil	10	0.744	0.007
Diagnostics:NumBusyUnits	10	1.489	0.013
Diagnostics:ScheduledUtil	10	0.744	0.007
Diagnostics:WIP	10	3.332	0.123
Diagnostics:Q:NumInQ	10	1.844	0.11
Diagnostics:Q:TimeInQ	10	36.964	2.043
Test1:InstantaneousUtil	10	0.855	0.005
Test1:NumBusyUnits	10	0.855	0.005
Test1:ScheduledUtil	10	0.855	0.005
Test1:WIP	10	4.1	0.283
Test1:Q:NumInQ	10	3.245	0.279

Name	Count	Average	Half-Width
Test1:Q:TimeInQ	10	57.692	4.736
Test2:InstantaneousUtil	10	0.773	0.006
Test2:NumBusyUnits	10	0.773	0.006
Test2:ScheduledUtil	10	0.773	0.006
Test2:WIP	10	2.319	0.132
Test2:Q:NumInQ	10	1.546	0.126
Test2:Q:TimeInQ	10	41.279	3.062
Test3:InstantaneousUtil	10	0.858	0.006
Test3:NumBusyUnits	10	0.858	0.006
Test3:ScheduledUtil	10	0.858	0.006
Test3:WIP	10	3.417	0.273
Test3:Q:NumInQ	10	2.559	0.269
Test3:Q:TimeInQ	10	51.318	5.176
Repair:InstantaneousUtil	10	0.864	0.006
Repair:NumBusyUnits	10	2.591	0.017
Repair:ScheduledUtil	10	0.864	0.006
Repair:WIP	10	3.824	0.135
Repair:Q:NumInQ	10	1.233	0.12
Repair:Q:TimeInQ	10	24.733	2.273
TestAndRepair:NumInSystem	10	17.572	0.803
TestAndRepair:TimeInSystem	10	352.487	14.262
ProbWithinLimit	10	0.817	0.028
Diagnostics:SeizeCount	10	12443.4	75.921
Test1:SeizeCount	10	14027	76.189
Test2:SeizeCount	10	9337.3	77.976
Test3:SeizeCount	10	12436.4	76.419
Repair:SeizeCount	10	12430.7	75.528

It looks like test machine 3 and the repair station have the highest estimated utilization values. An increase in the amount of work to the test and repair shop may have issues at those stations. The reader is asked to explore the

performance of this situation in the exercises.

In the next section, we examine an even more complex situation involving systems that have parameters or distributions that depend on time.

7.2. Modeling Non-Stationary Systems

This section tackles a number of miscellaneous topics that can enhance your modeling capabilities. The section first examines the modeling of non-stationary arrival processes. This will motivate the exploration of additional concepts in resource modeling, including how to incorporate time varying resource staffing schedules. This will enable you to better model and tabulate the effects of realistic staff changes within systems. Because non-stationary arrivals affect how statistics should be interpreted, we will also study how to collect statistics over specific periods of time.

The section also covers some useful modeling situation involving how entities interact. The first situation that we will see is that of reneging from a queue. This will introduce how to remove entities that are in a queue and terminate their processes. Let's get started by looking at how time dependent arrivals can be generated.

7.2.1. Non-Stationary Arrival Processes

If a process does not depend on time, it is said to be stationary. When a process depends on time, it is said to be non-stationary. The more formal definitions of stationary/non-stationary are avoided in the discussion that follows.

There are many ways in which you can model non-stationary (time varying) processes in your simulation models. Many of these ways depend entirely on the system being studied and through that study appropriate modeling methods will become apparent. For example, suppose that workers

performing a task learn how to more efficiently perform the task every time that they repeat the task. The task time will depend on the time (number of previously performed tasks). For this situation, you might use a learning curve model as a basis for changing the task times as the number of repetitions of the task increases.

As another example, suppose the worker's availability depends on a schedule, such as having a 30-minute break after accumulating so much time. In this situation, the system's resources are dependent on time. Modeling this situation is described later in this section.

Let's suppose that the following situation needs to be modeled. Workers are assigned a shift of 8 hours and have a quota to fill. Do you think that it is possible that their service times would, on average, be less during the latter part of the shift? Sure. They might work faster in order to meet their quota during the latter part of the shift. If you did not suspect this phenomenon and performed a time study on the workers in the morning and fit a distribution to these data, you would be developing a distribution for the service times during the morning. If you applied this distribution to the entire shift, then you may have a problem matching your simulation throughput numbers to the real throughput.

As you can see, if you suspect that a non-stationary process is involved, then it is critical that you also record the time that the observation was made. The time series plot that was discussed for input modeling helps in assessing non-stationary behavior; however, it only plots the order in which the data was collected. If you collect 25 observations of the service time every morning, you might not observe non-stationary behavior. To observe non-stationary behavior it is important to collect observations across periods of time.

One way to better plan your observations is to randomize when you will take the observations. Work sampling methods often require this. The day is divided into intervals of time and the intervals randomly selected in which to record an activity. By comparing the behavior of the data across the intervals, you can begin to assess whether time matters in the

modeling as demonstrated in Section B.3.1. Even if you do not divide time into logical intervals, it is good practice to record the date and time for each of the observations that you make so that the observations can later be placed into logical intervals. As you may now be thinking, modeling a non-stationary process will take a lot of care and more observations than a stationary process.

A very common non-stationary process that you have probably experienced is a non- stationary arrival process. In a non-stationary arrival process, the arrival process to the system will vary by time (e.g., by hour of the day). Because the arrival process is a key input to the system, the system will thus become non-stationary. For example, in the drive-through pharmacy example, the arrival of customers occurred according to a Poisson process with mean rate λ per hour.

As a reminder, λ represents the mean arrival rate or the expected number of customers per unit time. Suppose that the expected arrival rate is five per hour. This does not mean that the pharmacy will get five customers every hour, but rather that on average five customers will arrive every hour. Some hours will get more than five customers and other hours will get less. The implication for the pharmacy is that they should expect about five per hour (every hour of the day) regardless of the time of day. Is this realistic? It could be, but it is more likely that the mean number of arriving customers will vary by time of day. For example, would you expect to get five customers per hour from 4 a.m. to 5 a.m., from 12 noon to 1 p.m., from 4 p.m. to 6 p.m.? Probably not! A system like the pharmacy will have peaks and valleys associated with the arrival of customers. Thus, the mean arrival rate of the customers may vary with the time of day. That is, λ is really a function of time, $\lambda(t)$.

To more realistically model the arrival process to the pharmacy, you need to model a non-stationary arrival process. Let's think about how data might be collected in order to model a non-stationary arrival process. First, as in the previous service time example, you need to think about dividing the day into logical periods of time. A pilot study may help in assessing

how to divide time, or you might simply pick a convenient time division such as hours. You know that event generation logic requires a distribution that represents the time between arrivals. Ideally, you should collect the time of every arrival, T_i, throughout the day. Then, you can take the difference among consecutive arrivals, $T_i - T_{i-1}$ and fit a distribution to the inter-arrival times. Looking at the T_i over time may help to assess whether you need different distributions for different time periods during the day. Suppose that you do collect the observations and find that three different distributions are reasonable models given the following data:

- Exponential, $E[X] = 60$, for midnight to 8 a.m.
- Lognormal, $E[X] = 12$, $Var[X] = 4$, for 8 a.m. to 4 p.m.
- Triangular, min = 10; mode = 16; max = 20, for 4 p.m. to midnight

One simple method for implementing this situation is to switch to the appropriate distribution for the current time. In other words, schedule an event at midnight so that the time between arrival distribution can be set to exponential, schedule an event at 8 a.m. to switch to lognormal, and schedule an event at 4 p.m. to switch to triangular. This can easily be accomplished by holding the distributions within a list and using variable to determine which period of time is active and indexing into the list accordingly. Clearly, this varies the arrival process in a time-varying manner. There is nothing wrong with this modeling approach if it works well for the situation. In taking such an approach, you need a lot of data. Not only do you need to record the time of every arrival, but you need enough arrivals in order to adequately fit distributions to the time between events for various time periods. This may or may not be feasible in practice.

Often in modeling arrival processes, you cannot readily collect the actual times of the arrivals. If you can, you should try, but sometimes you just cannot. Instead, you are limited to a count of the number of arrivals in intervals of time. For example, a computer system records the number of people arriving every hour but not their individual arrival times. This is

not uncommon since it is much easier to store summary counts than it is to store all individual arrival times. Suppose that you had count data as shown in Table 7.3.

Table 7.3.: Example Arrival Counts

Interval	Mon	Tue	Wed	Thurs	Fri	Sat	Sun
12 am – 6 am	3	2	1	2	4	2	1
6 am - 9 am	6	4	6	7	8	7	4
9 am - 12 pm	10	6	4	8	10	9	5
12 pm - 2 pm	24	25	22	19	26	27	20
2 pm - 5 pm	10	16	14	16	13	16	15
5 pm - 8 pm	30	36	26	35	33	32	18
8 pm - 12 am	14	12	8	13	12	15	10

Of course, how the data is grouped into intervals will affect how the data can be modeled. Grouping is part of the modeling process. Let's accept these groups as appropriate for this situation. Looking at the intervals, you see that more arrivals tend to occur from 12 pm to 2 pm and from 5 pm to 8 pm. As discussed in Section B.3.1, we could test for this non-stationary behavior and check if the counts depend on the time of day. It is pretty clear that this is the case for this situation. Perhaps this corresponds to when people who visit the pharmacy can get off of work. Clearly, this is a non-stationary situation. What kinds of models can be used for this type of count data?

A simple and often reasonable approach to this situation is to check whether the number of arrivals in *each* interval occurs according to a Poisson distribution. If so, then you know that the time between arrivals in an interval is exponential. When you can divide time so that the number of events in the intervals is Poisson (with different mean rates), then you can use a non-stationary or non-homogeneous Poisson process as the arrival process.

Consider the first interval to be 12 am to 6 am; to check if the counts are Poisson in this interval, there are seven data points (one for every day of the week for the given period). This is not a lot of data to use to perform a chi-square goodness-of-fit test for the Poisson distribution. So, to do a better job at modeling the situation, many more days of data are needed. The next question is: Are all the days the same? It looks like Sunday may be a little different than the rest of the days. In particular, the counts on Sunday appear to be a little less on average than the other days of the week. In this situation, you should also check whether the day of the week matters to the counts. As we illustrated in Section B.3.1, one method of doing this is to perform a contingency table test.

Let's assume for simplicity that there is not enough evidence to suggest that the days are different and that the count data in each interval is well modeled with a Poisson distribution. In this situation, you can proceed with using a non-stationary Poisson process. From the data you can calculate the average arrival rate for each interval and the rate per hour for each interval as shown in Table 7.4. In the table, 2.14 is the average of the counts across the days for the interval 12 am to 6 pm. In the rate per hour column, the interval rate has been converted to an hourly basis by dividing the rate for the interval by the number of hours in the interval. Figure 7.6 illustrates the time-varying behavior of the mean arrival rates over the course of a day.

Table 7.4.: Mean Arrival Rates for Each Time Interval

Interval	Avg.	Length (hrs)	Rate per hour
12 am – 6 am	2.14	6	0.36
6 am - 9 am	6.0	3	2.0
9 am - 12 pm	7.43	3	2.48
12 pm - 2 pm	23.29	2	11.645
2 pm - 5 pm	14.29	3	4.76
5 pm - 8 pm	30.0	3	10
8 pm - 12 am	12.0	4	3.0

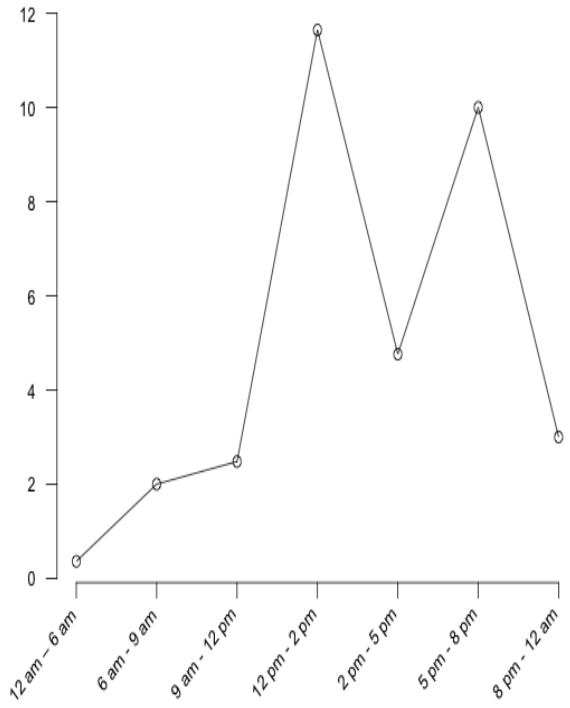


Figure 7.6.: Arrival Rate per Hour for Time Intervals

The two major methods by which a non-stationary Poisson process can be generated are thinning and rate inversion. Both methods will be briefly discussed; however, the interested reader is referred to (Leemis and Park 2006) or (Ross 1997) for more of the theory underlying these methods. Before these methods are discussed, it is important to point out that the naive method of using different Poisson processes for each interval and subsequently different exponential distributions to represent

the inter-arrival times could be used to model this situation by switching the distributions as previously discussed. However, the switching method is not technically correct for generating a non-stationary Poisson process. This is because it will not generate a non-stationary Poisson process with the correct probabilistic underpinnings. Thus, if you are going to model a process with a non-stationary Poisson process, you should use either thinning or rate inversion to be technically correct.

7.2.1.1. Thinning Method

For the thinning method, a stationary Poisson process with a constant rate,λ^*, and arrival times, t_i^*, is first generated, and then potential arrivals, t_i^*, are rejected with probability:

$$p_r = 1 - \frac{\lambda(t_i^*)}{\lambda^*}$$

where $\lambda(t)$ is the mean arrival rate as a function of time. Often, λ^* is set as the maximum arrival rate over the time horizon, such as:

$$\lambda^* = \max_t \{\lambda(t)\}$$

In the example, λ^* would be the mean of the 5 pm to 8 pm interval, $\lambda^* = 11.645$.

Example 7.5 (Thinning Algorithm Implementation)**.** The following code for implementing the thinning method creates event times at the maximum rate and thins them according to the current time.

```
fun nhppThinning(rateFunc: RateFunctionIfc, maxTime: Double)
 ↪  : DoubleArray {
    val list = mutableListOf<Double>()
    val rv = ExponentialRV(1.0/rateFunc.maximumRate)
    val u = UniformRV(0.0, rateFunc.maximumRate)
    var t = 0.0
    while (t < maxTime){
        var s = t
        do{
            s = s + rv.value
        } while((s < maxTime) && (u.value >
         ↪  rateFunc.rate(s)))
        t = s
        if (t < maxTime){
            list.add(t)
        }
    }
    return list.toDoubleArray()
}
```

The function returns the time of the events. To be able to use such numbers in an event generator, we need to generate the time between random events for the non-homogeneous Poisson process. The KSL provides such a random variable via the `NHPPTimeBtwEventRV` class found in the `ksl.modeling.nhpp` package. The above code, which can be found in the examples for this chapter, uses an instance that implements the `RateFunctionIfc` interface. The `ksl.modeling.nhpp` package provides implementations for piecewise constant and piecewise linear rate functions. The following code illustrates how to create and simulate a piecewise constant rate function using the thinning method.

```
fun main() {
    val d = doubleArrayOf(15.0, 20.0, 15.0)
    val ar = doubleArrayOf(1.0, 2.0, 1.0)
    val f = PiecewiseConstantRateFunction(d, ar)
    println("-----")
    println("intervals")
    println(f)
    val times = nhppThinning(f, 50.0)
    println(times.contentToString())
}
```

This code simulates the following piecewise rate function:

$$f(x) = \begin{cases} 1.0 & 0.0 \leq x \leq 15.0 \\ 2.0 & 15.0 \leq x \leq 35.0 \\ 1.0 & 35.0 \leq x \leq 50.0 \end{cases}$$

Notice that the rate and the duration values are provided as arrays. The theoretical basis for why thinning works can be found in (Ross 1997).

7.2.1.2. Rate Inversion Method

The second method for generating a non-stationary Poisson process is through the rate inversion algorithm. In this method, a $\lambda = 1$ Poisson process is generated, and the inverse of the mean arrival rate function is used to re-scale the times of arrival to the appropriate scale. Define $m(t)$ as the cumulative mean rate function such that:

$$m(t) = \int_0^t \lambda(s)ds \quad \text{if } t \leq 0 < \tau$$

where $\lambda(\cdot)$ is the instantaneous mean rate function defined over a range from 0 to τ. Define $m^{-1}(t)$ as the inverse of $m(t)$. Then, the following algorithm will generate a non-homogeneous Poisson process. Define $a_0 = 0$, $y_0 = 0.0$, and $n = 0$.

while $(a_n < \tau)$ {

$y_{n+1} = y_n +$ `ExponentialRV(mean=1.0)`

$a_{n+1} = m^{-1}(y_{n+1})$

$n = n + 1$

}

The resulting array, a_i holds the generated event times. This algorithm is discussed further in (Ross 1997). The algorithm generates a rate 1.0 Poisson process and transforms the resulting event times to the times on the $m^{-1}(t)$ axis. The resulting event times will be from a non-homogeneous Poisson process.

The KSL provides classes for piecewise constant and piecewise linear rate functions and their inverse cumulative rate functions.

Example 7.6 (Piecewise Constant Rate Function)**.** The following example illustrates how to setup a piecewise linear rate function and generate from it using a `NHPPEventGenerator` instance.

```
class NHPPPWLinearExample(parent: ModelElement, f:
 ↪  PiecewiseRateFunction, name: String? = null)
: ModelElement(parent, name) {

    private val myNHPPGenerator: NHPPEventGenerator =
     ↪  NHPPEventGenerator(this, f, this::arrivals, streamNum
     ↪  = 1)
```

```
    private val myCountersFC: MutableList<Counter> =
     ↪ mutableListOf()
    private val myPWRF: PiecewiseRateFunction = f

    init {
        val n: Int = f.numberSegments()
        for (i in 0 until n) {
            val c = Counter(this, "Interval FC $i")
            myCountersFC.add(c)
        }
    }

    private fun arrivals(generator: EventGenerator){
        val t: Double = time
        val i: Int = myPWRF.findTimeInterval(t)
        myCountersFC[i].increment()
    }
}
```

In the code, a list of counters is defined to collect the counts within the intervals of the function. The end points of the segments are the rates at the beginning and end of the segments. The rate at the beginning of the segment can be the same as the rate at the end of the segment. The example has two arrays, one for the rate values and one for the duration values. In the example, the first segment has beginning rate $\lambda_0 = 0.5$, duration 200.0, and ending rate, $\lambda_1 = 0.5$. The second segment begins with $\lambda_1 = 0.5$, has duration 400.0, and ending rate $\lambda_2 = 0.9$. Thus, there will be one more specified rate value than there will be duration values. To specify a piecewise linear rate function, you must have at least one segment.

```
fun main() {
    val ar = doubleArrayOf(0.5, 0.5, 0.9, 0.9, 1.2, 0.9, 0.5)
    val dd = doubleArrayOf(200.0, 400.0, 400.0, 200.0, 300.0,
     ↪  500.0)

    val f = PiecewiseLinearRateFunction(dd, ar)
    // create the experiment to run the model
    val s = Model()
    println("-----")
    println("intervals")
    System.out.println(f)
    NHPPPWLinearExample(s, f)

    // set the parameters of the experiment
    s.numberOfReplications = 1000
    s.lengthOfReplication = 2000.0

    // tell the simulation to run
    s.simulate()
    val r = s.simulationReporter
    r.printAcrossReplicationSummaryStatistics()
}
```

The output is what would be expected for this process. The interested reader is referred to (Leemis and Park 2006) for why the slope of the segments play such an important role.

```
intervals
 [0.5,0.5] slope = 0.0 [0.0,200.0] width = 200.0 [0.0,100.0] cr width = 100.0
 [0.5,0.9] slope = 0.001 [200.0,600.0] width = 400.0 [100.0,380.0] cr width = 280.0
 [0.9,0.9] slope = 0.0 [600.0,1000.0] width = 400.0 [380.0,740.0] cr width = 360.0
 [0.9,1.2] slope = 0.0014999999999999996 [1000.0,1200.0] width = 200.0 [740.0,950.0]
 [1.2,0.9] slope = -9.999999999999998E-4 [1200.0,1500.0] width = 300.0 [950.0,1265.0
```

```
 [0.9,0.5] slope = -8.0E-4 [1500.0,2000.0] width = 500.0 [1265.0,1615.0] cr width =

-------------------------------------------------------------------------------

Across Replication Statistical Summary Report
Sat Dec 31 18:03:58 CST 2022
Simulation Results for Model: MainModel


Number of Replications: 1000
Length of Warm up period: 0.0
Length of Replications: 2000.0

-------------------------------------------------------------------------------
Counters
-------------------------------------------------------------------------------
Name                                  Average       Std. Dev.    Count
-------------------------------------------------------------------------------
Interval FC 0                          99.899000       10.144837     1000.000000
Interval FC 1                         279.526000       16.741673     1000.000000
Interval FC 2                         359.733000       18.398538     1000.000000
Interval FC 3                         209.719000       14.505532     1000.000000
Interval FC 4                         315.231000       17.753502     1000.000000
Interval FC 5                         349.919000       18.208899     1000.000000
-------------------------------------------------------------------------------
```

The advantage of the rate inversion method is that there is a one for one mapping from the underlying pseudo-random numbers and the generated variates. Since the thinning method uses a form of acceptance-rejection, the one-for-one mapping can not be preserved. The disadvantage of the rate inversion method is that the mean cumulative rate function, $m(t)$ must be inverted, which may require numerical methods. The following section discusses the issues involved when using resources under non-stationary conditions.

7.2.2. Modeling Resources Under Non-Stationary Conditions

From previous modeling, a resource can be something that an entity uses as it flows through the system. If the required units of the resource are available for the entity, then the units are allocated to the entity. If the required units are unavailable for allocation, the entity's progress through the model stops until the required units become available. So far, the only way in which the units could become unavailable was for the units to be seized by an entity. The seizing of at least one unit of a resource causes the resource to be considered busy. A resource is considered to be in the idle state when all units are idle (no units are seized). Thus, in previous modeling a resource could be in one of two states: Busy or Idle.

When specifying a resource, its capacity must be given. The capacity represents the maximum number of units of the resource that can be seized at any time. In previous modeling, the capacity of the resource was *fixed*. That is, the capacity did not vary as a function of time. When the capacity of a resource was set, the resource had that same capacity for all of the replications; however, in many situations, the capacity of a resource can vary with time. For example, in the pharmacy model, you might want to have two or more pharmacists staffing the pharmacy during peak business hours. This type of capacity change is predictable and can be managed via a staffing schedule. Alternatively, the changes in capacity of a resource might be unpredictable or random. For example, a machine being used to produce products on a manufacturing line may experience random failures that require repair. During the repair time, the machine is not available for production. The KSL provides constructs for modeling scheduled capacity changes.

The `Resource` class implementation permits the capacity of the resource to change via a capacity change schedule. There are significant modeling issues that need to be considered if the capacity is permitted to change. These issues will be discussed in this section. An important issue will be how to handle resource requests that are waiting in queues when the

capacity changes. To handle this issue the modeler must register any queues that may be served by a resource with the resource so that the resource may notify the queue of the change in capacity. Another important issue will be what to do when the resource is busy and a capacity change occurs. This situation will be handled via the specification of rules for the capacity change. At present, the KSL does not provide additional constructs for modeling resource failures; however, general resource failure situations can be modeled using both the event-view and the process-view standard KSL constructs.

There are two key resource variables for understanding resources and their states.

- $c(t)$: Resource capacity, $c(t)$, returns the number of capacity units currently defined for the specified resource. This value can be changed by the user via a capacity schedule or through the use capacity change notices.
- $b(t)$: Number of busy resource units at any time t. Each time an entity seizes a resource, $b(t)$ changes accordingly.
- $a(t) = c(t) - b(t)$: Number of available resource units at any time t. Each time an entity seizes a resource or a capacity change occurs, $a(t)$ changes accordingly.

These variables may change over time for a resource. The changing of these variables defines the possible states of a resource. These states are:

Idle A resource is in the idle state when all units are idle. That is, a resource is idle if there are no busy units, i.e. $b(t) = 0$.

Busy A resource is in the busy state when it has one or more busy (seized) units. That is, when $b(t) > 0$.

Inactive A resource is in the inactive state when it has zero capacity and it is not busy. That is, when $b(t) = 0$ and $c(t) = 0$. Because of how capacity changes can be invoked, it is possible for the capacity of the

resource to be $c(t) = 0$ while resources are still busy. This will be discussed further in what follows.

A capacity schedule governs how capacity changes. The `CapacitySchedule` class specifies the amount of capacity for various durations of time. For example, in the following code, we specify a resource with a default initial capacity of 2 units, which is overridden by specifying the use of a capacity schedule.

```
    private val resource: ResourceWithQ =
     ↪  ResourceWithQ(this, name = "Resource", capacity = 1)
    private val schedule: CapacitySchedule

    init {
        schedule= CapacitySchedule(this, 0.0)
        schedule.addItem(capacity = 0, duration = 15.0)
        schedule.addItem(capacity = 1, duration = 35.0)
        schedule.addItem(capacity = 0, duration = 20.0)
        schedule.addItem(capacity = 2, duration = 30.0)
        resource.useSchedule(schedule, changeRule =
↪   CapacityChangeRule.IGNORE)
    }
```

A capacity schedule provides a start time for the schedule. That is, the time after the start of the replication for which the schedule starts and the items on the schedule get processed. In the example, the starting time of the schedule is specified in the constructor of `CapacitySchedule` as 0.0. Then, the `addItem` method is used to add capacity values and duration values (pairs). In the example, notice that the capacity can be 0 for different segments of time and that a capacity change rule is specified when indicating that the resource uses the schedule.

If the capacity is increased over its current value and there are no pending changes, then the capacity is immediately increased and requests that are

waiting for the resource will be processed to receive allocations from the resource. The requests that are waiting for the resource are only considered for allocations if the queue that they are waiting within is registered with the resource. If the resource is an instance of the `ResourceWithQ` class or if it has been added to a `ResourcePoolWithQ` instance, then the registration of the queue instances will automatically occur. However, if the modeler uses a `seize()` function with an arbitrary request queue instance supplied, then that queue will only be notified if the queue is registered with the resource. The `registerCapacityChangeQueue()` function of the `Resource` class can be used to perform the registration. There is a corresponding function to undo the registration.

Because there can be one or more queues registered on a resource to react to the capacity increase, a rule can be specified to indicate the order of notification. This is accomplished with an instance that implements the `RequestQueueNotificationRuleIfc` interface. A `RequestQueueNotificationRuleIfc` promises to supply an iterator over the registered queues. The default request queue notification rule is to notify the queues in the order in which they were registered. The user may want to change this via a specification of the `initialRequestQueueNotificationRule` property of the `Resource` class. For example, different rules may want to consider queues for the allocation based on some prioritization scheme. The requests in the queues are processed until there are none to consider or if the capacity increase is used up.

If the capacity is decreased from its current value, then the amount of the decrease is first filled from idle units. If there are not enough idle units to complete the decrease, then the change is processed according to the capacity change rule. Registered queues are also notified if the decrease in capacity causes the resource to become inactive. In this situation, the entity associated with the request is notified to possibly take action because its request would be in a queue for the upcoming inactive period.

The following discusses the two capacity change rule options available

within the KSL.

Ignore starts the time duration of the schedule change immediately, but allows the busy resource to finish processing the current entity before causing the capacity change.

Wait waits until the busy resource has finished processing the current entity before changing the resource capacity and starting the time duration of the schedule change.

It is important to note that the capacity change rule is invoked only when the resource is busy and there is a requested decrease in capacity. The rules govern when the change occurs. Basically, the ignore rule indicates that the change starts immediately. That is, the resource acts as if the capacity has been removed by putting up a sign indicating that the change has immediately reduced the capacity. The variable, $a(t)$, may in fact become less than zero in this case. That is, the ignore rule may cause a deficit in capacity. Since the resource is also busy, there will not be any available units and any new seize requests will wait and any currently waiting requests will continue to wait. The requested units will be taken away and not be allowed to be allocated as the required number is released from the resource. Let's consider some cases of this situation.

Let's suppose that your simulation professor has office hours throughout the day, except from 12-12:30 pm for lunch. What happens for each rule if you arrive at 11:55 am with a question?

Ignore Case 1:

- You arrive at 11:55 am with a 10 minute question and begin service.
- At Noon, the professor gets up and hangs a Lunch in Progress sign.
- Your professor continues to answer your question for the remaining service time of 5 minutes.
- You leave at 12:05 pm.

- Whether or not there are any students waiting in the hallway, the professor still starts lunch.
- At 12:30 pm the professor finishes lunch and takes down the lunch in progress sign. If there were any students waiting in the hallway, they can begin service.

The net effect of case 1 is that the professor lost 5 minutes of lunch time. During the 30 minute scheduled break, the professor was busy for 5 minutes and inactive for 25 minutes.

Ignore Case 2:

- You arrive at 11:55 am with a 45 minute question and begin service.
- At Noon, the professor gets up and hangs a Lunch in Progress sign.
- Your professor continues to answer your question for the remaining service time of 40 minutes.
- At 12:30 pm the professor gets up and takes down the lunch in progress sign and continues to answer your question.
- You leave at 12:40 pm

The net effect of case 2 is that the professor did not get to eat lunch that day. During the 30 minute scheduled break, the professor was busy for 30 minutes.

This rule is called Ignore since the scheduled break may be *ignored* by the resource if the resource is busy when the break occurs. Technically, the scheduled break actually occurs and the time that was scheduled may be considered as unscheduled (inactive) time.

Let's assume that the professor is at work for 8 hours each day (including the lunch break). But because of the scheduled lunch break, the total time available for useful work is 450 minutes. In case 1, the professor worked for 5 minutes more than they were scheduled. In case 2, the professor

worked for 30 minutes more than they were scheduled. As will be indicated shortly, this extra work time, must be factored into how the utilization of the resource (professor) is computed.

The situation is a bit more complex because another scheduled change may be required to occur (either positive or negative) while a current scheduled change is in progress. For the ignore case, if a positive change occurs before the end of the currently scheduled change, its duration must be long enough for the change to last longer than the current change's end time. That is, the next change cannot be scheduled to end before the current change ends. If we have a legal change notice, then if the change is a positive change, it happens immediately. In essence the current negative change is canceled and the positive change occurs. If the change is a negative change, then its end is scheduled, but it waits until the current negative change completes before proceeded with the new negative change. Thus, for ignore, negative changes can pile up if they occur sequentially.

Now, let's examine what happens if the rule is Wait.

Wait Case 1:

- You arrive at 11:55 am with a 10 minute question and begin service.
- At Noon, the professor's lunch reminder rings on his or her computer. The professor recognizes the reminder but doesn't act on it, yet.
- Your professor continues to answer your question for the remaining service time of 5 minutes.
- You leave at 12:05 pm. The professor recalls the lunch reminder and hangs a Lunch in Progress sign. Whether or not there are any students waiting in the hallway, the professor still hangs the sign and starts a 30 minute lunch.
- At 12:35 pm the professor finishes lunch and takes down the lunch in progress sign. If there were any students waiting in the hallway, they can begin service.

Wait Case 2:

- You arrive at 11:55 am with a 45 minute question and begin service.
- At Noon, the professor's lunch reminder rings on his or her computer. The professor recognizes the reminder but doesn't act on it, yet.
- Your professor continues to answer your question for the remaining service time of 40 minutes.
- You leave at 12:40 pm. The professor recalls the lunch reminder and hangs a Lunch in Progress sign. Whether or not there are any students waiting in the hallway, the professor still hangs the sign and starts a 30 minute lunch.
- At 1:10 pm the professor finishes lunch and takes down the lunch in progress sign. If there were any students waiting in the hallway, they can begin service.

The net effect of both these cases is that the professor does not miss lunch (unless the student's question takes the rest of the afternoon!). Thus, in this case the resource will experience the scheduled break after *waiting* to complete the entity in progress. Again, in this case, the tabulation of the amount of busy time may be affected by when the rule is invoked

The example involving the professor involved a resource with 1 unit of capacity. But what happens if the resource has a capacity of more than 1 and what happens if the capacity change is more than 1. The rules work essentially the same. If the scheduled change (decrease) is less than or equal to the current number of idle units, then the rules are not invoked. If the scheduled change will require busy units, then any idle units are first taken away and then the rules are invoked. In the case of the ignore rule, the units continue serving, the inactive sign goes up, and whichever unit is released first becomes inactive first.

In the case of the Ignore rule, a positive change happens immediately. In the case of the wait rule, the positive change will be delayed until after the

full duration of the negative change. Thus, for the wait rule, the changes are shifted in time, but will always occur (provided that the simulations' replication length is long enough). Thus, for the wait rule, once a capacity change is delayed because of being invoked when the resource is busy, all subsequent changes will also be delayed. In essence, the capacity changes are handled in a first-come, first served manner. Both rules ensure that an entity that is using the resource at the time of change will fully complete its current usage. Once the entity releases the resource, the rule governs how the resource's released units are allocated to the change. In the case of a positive change, the units are given back to the resource at the time of the release. In the case of a negative change, the units are taken away from the current capacity, $c(t)$. Since the rules affect the allocation of capacity, they can affect how the utilization of the resource is computed.

The calculation of instantaneous and scheduled utilization depends upon the two variables $b(t)$ and $c(t)$ for a resource. The instantaneous utilization is defined as the time weighted average of the ratio of these variables. Let $b(t)$ be the number of busy resource units at time t. Then, the time average number of busy resources is:

$$\overline{B} = \frac{1}{T} \int_0^T b(t) \mathrm{d}t$$

Let $c(t)$ be the capacity of the resource at time t. Then, the time average capacity of the resource is:

$$\overline{C} = \frac{1}{T} \int_0^T c(t) \mathrm{d}t$$

Now, we can define the instantaneous utilization at time t as:

$$IU(t) = \begin{cases} 0 & c(t) = 0 \\ 1 & b(t) \geq c(t) \\ b(t)/c(t) & \text{otherwise} \end{cases}$$

Thus, the time average instantaneous utilization is:

$$\overline{IU} = \frac{1}{T} \int_0^T IU(t) \mathrm{d}t$$

The scheduled utilization is the time average number of busy resources divided by the time average number scheduled. As can be seen in Equation 7.1, this is the same as the total time spent busy divided by the total time available for all resource units.

$$\overline{SU} = \frac{\overline{B}}{\overline{C}} = \frac{\frac{1}{T}\int_0^T b(t)dt}{\frac{1}{T}\int_0^T c(t)dt} = \frac{\int_0^T b(t)dt}{\int_0^T c(t)dt} \tag{7.1}$$

If $c(t)$ is constant, then $\overline{IU} = \overline{SU}$. Caution should be used in interpreting $\overline{IU}$ when $c(t)$ varies with time.

Now let's return to the example of the professor holding office hours. Let's suppose that the ignore option is used and consider case 2 and the 45 minute question. Let's also assume for simplicity that the professor had a take home exam due the next day and was therefore busy all day long. What would be the average instantaneous utilization and the scheduled utilization of the professor? Table 7.5 illustrates the calculations.

Table 7.5.: Example Calculation for Professor Utilization

Time interval	Busy time	Scheduled time	$b(t)$	$c(t)$	$IU(t)$
8 am - noon	240	240	1.0	1.0	1.0
12 - 12:30 pm	30	0	1.0	0	1.0
12:30 - 4 pm	210	210	1.0	1.0	1.0
	480	450			

From Table 7.5 we can compute $\overline{B}$ and $\overline{C}$ as:

$$\overline{B} = \frac{1}{480}\int_0^{480} 1.0\mathrm{d}t = \frac{480}{480} = 1.0$$

$$\overline{C} = \frac{1}{480}\int_0^{480} c(t)\mathrm{d}t = \frac{1}{480}\left\{\int_0^{240} 1.0\mathrm{d}t + \int_{270}^{480} 1.0\mathrm{d}t\right\} = 450/480 = 0.9375$$

$$\overline{IU} = \frac{1}{480}\int_0^{480} 1.0\mathrm{d}t = \frac{480}{480} = 1.0$$

$$\overline{SU} = \frac{\overline{NR}}{\overline{MR}} = \frac{1.0}{0.9375} = 1.0\bar{6}$$

Table 7.5 indicates that $\overline{IU} = 1.0$ (or 100%) and $\overline{SU} = 1.06$ (or 106%). Who says that professors don't give their all! Thus, with scheduled utilization, a schedule can result in the resource having its scheduled utilization higher than 100%. There is nothing wrong with this result, and if it is the case that the resource is busy for more time than it is scheduled, you would definitely want to know.

The choice of which rule to use comes down to what is given priority. The Ignore rule gives priority to the entity by cutting short the scheduled change. The Wait rule gives priority to the resource by ensuring that the

scheduled change always occurs. If the resources are people, then using the Wait rule seems more appropriate. Of course, if these rules do not meet the requirements of your modeling situation, you are always free to implement difference rules.

In the next section, we examine how to collect statistics over specific intervals of time. Then, in Section 7.4, the arrival schedules, the capacity schedules, and the collection of statistics by period are all put to use in an enhanced version of the STEM Career Mixer model.

7.3. Collecting Statistics Over Periods of Time

For some modeling situations (especially non-stationary systems), we need to be able to collect statistics during specific intervals of time. To be able to do this, we need to be able to observe what happens during those intervals. In general, to collect statistics during a particular interval of time will require the scheduling of events to observe the system (and its statistics) at the start of the period and then again at the end of the period in order to record what happened during the period. Let's define some variables to clarify the concepts. We will start with a discussion of collecting statistics for tally-based data.

Let $\left(x_1,\ x_2, x_3, \cdots, x_{n(t)}\right)$ be a sequence of observations up to and including time t.

Let $n(t)$ be the number of observations up to and including time t.

Let $s\,(t)$ be the cumulative sum of the observations up to and including time t. That is,

$$s\,(t) = \sum_{i=1}^{n(t)} x_i$$

Let $\overline{x}(t)$ be the cumulative average of the observations up to and including time t. That is,

$$\overline{x}(t) = \frac{1}{n(t)} \sum_{i=1}^{n(t)} x_i = \frac{s(t)}{n(t)}$$

We should note that we have $s(t) = \overline{x}(t) \times n(t)$. Let t_b be the time at the beginning of a period (interval) of interest and let t_e be the time at the end of a period (interval) of interest such that $t_b \leq t_e$. Define $s(t_b, t_e]$ as the sum of the observations during the interval, $(t_b, t_e]$. Clearly, we have that,

$$s(t_b, t_e] = s(t_e) - s(t_b)$$

Define $n(t_b, t_e]$ as the count of the observations during the interval, $(t_b, t_e]$. Clearly, we have that,

$$n(t_b, t_e] = n(t_e) - n(t_b)$$

Finally, we have that the average during the interval, $(t_b, t_e]$ as

$$\overline{x}(t_b, t_e] = \frac{s(t_b, t_e]}{n(t_b, t_e]}$$

Thus, if during the simulation we can observe, $s(t_b, t_e]$ and $n(t_b, t_e]$, we can compute the average during a specific time interval. To do this in a simulation, we can schedule an event for time, t_b, and observe, $s(t_b)$ and $n(t_b)$. Then, we can schedule an event for time, t_e, and observe, $s(t_e)$ and $n(t_e)$. Once these values are captured, we can compute $\overline{x}(t_b, t_e]$. If we observe this quantity across multiple replications, we will have the average that occurs during the defined period.

So far, the discussion has been about tally-based data. The process is essentially the same for time-persistent data. Recall that a time-persistent variable typically takes on the form of a step function. For example, let $y(t)$ represents the value of some state variable at any time t. Here $y(t)$ will take on constant values during intervals of time corresponding to when the state variable changes, for example. $y(t) = \{0, 1, 2, 3, \ldots\}$. $y(t)$ is a curve (a step function in this particular case) and we compute the time average over the interval $(t_b, t_e]$.as follows.

$$\overline{y}\,(t_b, t_e) = \frac{\int_{t_b}^{t_e} y\,(t)\,\mathrm{dt}}{t_e - t_b}$$

Similar to the tally-based case, we can define the following notation. Let $a\,(t)$ be the cumulative area under the state variable curve.

$$a\,(t) = \int_0^t y\,(t)\,\mathrm{dt}$$

Define $\overline{y}(t)$ as the cumulative average up to and including time t, such that:

$$\overline{y}\,(t) = \frac{\int_0^t y\,(t)\,\mathrm{dt}}{t} = \frac{a\,(t)}{t}$$

Thus, $a\,(t) = t \times \overline{y}\,(t)$. So, if we have a function to compute $\overline{y}\,(t)$ we have the ability to compute,

$$\overline{y}\,(t_b, t_e) = \frac{\int_{t_b}^{t_e} y\,(t)\,\mathrm{dt}}{t_e - t_b} = \frac{a\,(t_e) - a\,(t_b)}{t_e - t_b}$$

Again, a strategy of scheduling events for t_b and t_e can allow you to observe the required quantities at the beginning and end of the period and then observe the desired average.

Figure 7.7 illustrates these quantities for a time-persistent variable. The shaded region is the area that accumulates during the interval, $a(t_e) - a(t_b)$, and the dashed line is the resulting time average. In the figure, the area is 8 and the interval is 4 time units in length, so the time average during the interval is 2.

The two derivations just presented look different. One divides by a count of observations and the other divides by elapsed time. Within the KSL, both are implemented by a single mechanism. Every `Response` and `TWResponse` instance maintains a within replication weighted statistic that accumulates a *weighted sum* and a *sum of weights*. The type of the response determines what the weights represent.

- For a `Response` instance, which collects tally-based data, each observation is collected with a weight of 1.0. Thus, the weighted sum is $s(t)$ and the sum of the weights is $n(t)$.
- For a `TWResponse` instance, which collects time-persistent data, each value is collected with a weight equal to the amount of time that the value persisted. Thus, the weighted sum is the area, $a(t)$, and the sum of the weights is the elapsed time.

This is why observing a small number of quantities at t_b and again at t_e is sufficient to compute either average. The following code, which is executed at the end of every observed interval, performs the computation.

```
val sum: Double = w.weightedSum - data.mySumAtStart
val denom: Double = w.sumOfWeights -
 ↪  data.mySumOfWeightsAtStart
val numObs: Double = w.count - data.myNumObsAtStart
val avg = sum / denom
```

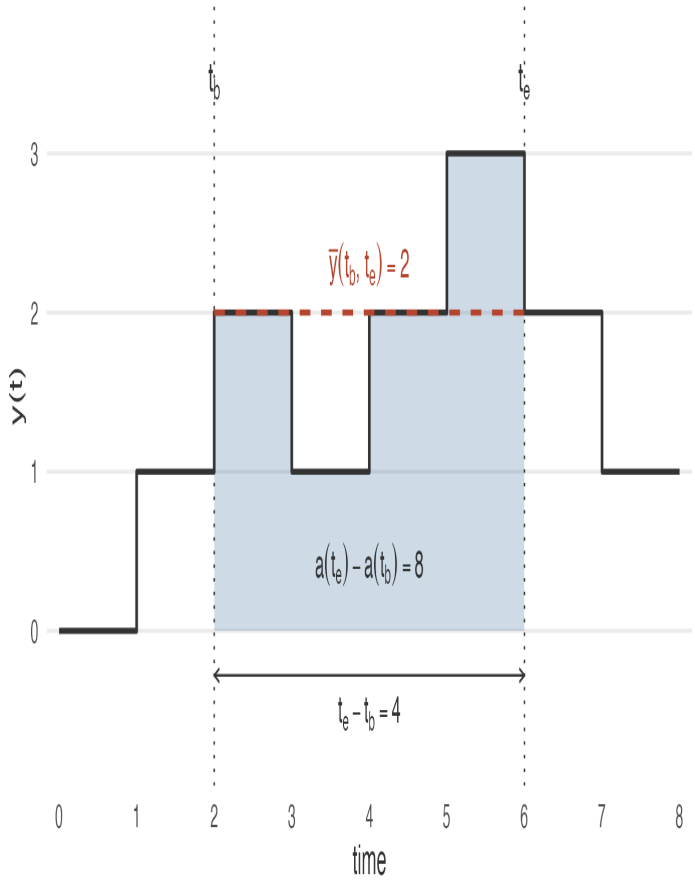


Figure 7.7.: The Time Average of a Time-Persistent Variable over the Interval $(t_b, t_e]$

In the code, `w` is the within replication weighted statistic of the response and `data` holds the values that were captured at the start of the interval. Table 7.6 maps each line of the computation to the notation that was developed above.

Table 7.6.: Mapping the Interval Computation to the Derived Formulas

Code	Notation	For a `Response`	For a `TWResponse`
`sum`	numerator	$s(t_e) - s(t_b)$	$a(t_e) - a(t_b)$
`denom`	denominator	$n(t_e) - n(t_b)$	$t_e - t_b$
`avg`	the average	$\overline{x}(t_b, t_e]$	$\overline{y}(t_b, t_e]$

Notice that the value of `numObs` is not used within the division. It answers a different question. Were there any observations at all during the interval? For a `Response` instance, if `numObs` is zero, then there is nothing to average and no observation is recorded. For a `TWResponse` instance, a `numObs` of zero simply means that the variable did not change during the interval, in which case its average is its (constant) value. The consequences of this distinction are discussed in what follows.

Every mechanism presented in this section uses this same computation. The mechanisms differ only in when the intervals start and stop and in how the resulting observations are reported.

The key to collecting the statistics on observation-based over an interval of time is the usage of the `ResponseInterval` class within the KSL. Using the previously defined notation, we can capture $\overline{x}\,(t)$, $\overline{y}\,(t)$, and $n(t)$ via the within replication statistics of response variables.

The `ResponseInterval` class represents an interval of time over which statistical collection should be performed. An interval is specified by providing an interval start time and a duration. The duration must be finite and greater than zero.

Simulation responses in the form of instances of `Response`, `TWResponse`, and `Counter` instances can be added to the interval for observation. New responses are created and associated with each of the supplied responses. The new responses collect observations associated with the supplied responses only during the specified interval. In the case of response variables or time weighted variables, the average response during the interval is observed. In the case of counters, the total count during the interval is observed.

If the interval is not associated with a `ResponseSchedule`, the interval may be repeated. In which case, the statistics are collected across the intervals. A repeated interval starts immediately after the previous duration. Note that for `Response` instances that are observed, if there are no observations during the interval then the average response during the interval is undefined (and thus not observed). Therefore, interval statistics for `Response` instances are conditional on the occurrence of at least one observation. This is most relevant when the interval is repeated because intervals with no observations are not tabulated. For example, suppose that there are no students that use the MalMart recruiters during the first hour of operation, then we cannot compute an average for this interval. Suppose we observe 10 days of operation of the mixer and on 2 of the days, there were no students that used the MalMart recruiters during the first hour of operation. Then, instead of getting a sample average across 10 days, we will only get a sample average across 8 days. We cannot compute statistics over observations that do not exist.

The KSL can measure how often this situation occurs. When a response is added to a `ResponseInterval`, an optional argument requests an additional statistic that records, for each observation of the interval, whether or not the interval was empty.

```
responseInterval.addResponseToInterval(myResponse,
intervalEmptyStatOption = true)
```

This causes a response to be created whose name ends with `P(Empty)` and whose value estimates the probability that the interval contained no observations. Continuing the example, a `P(Empty)` of 0.2 indicates that 2 of the 10 days produced no observation for the interval, which is exactly the information needed to judge whether the reported interval average should be trusted.

A `ResponseInterval` permits collection of statistics over a specific interval, which might repeat. Often we want to collect statistics across many intervals according to some timed pattern. The `ResponseSchedule` class facilitates the modeling of this situation. Figure 7.8 presents the `ResponseInterval` class. Note that functions exist to add counters and responses to an interval.

The `ResponseSchedule` class allows the creation of a schedule that represents a list of intervals of time. The starting length of a schedule is 0.0. The length of a schedule depends upon the intervals added to it. The schedule's length encompasses the furthest interval added. If no intervals are added, then the schedule only has its start time and no response collection will occur.

The user adds intervals and responses for which statistics need to be collected during the intervals. The intervals within the cycle may overlap in time. The start time of an interval is specified relative to the beginning of the cycle. The length of any interval must be finite.

The schedule can be started any time after the start of the simulation. The default starting time of the schedule is time 0.0. The schedule will start automatically at the designated start time.

The schedule can be repeated after the cycle length of the schedule is reached. The default is for the schedule to automatically repeat. Note that depending on the simulation run length only a portion of the scheduled intervals may be executed. Figure 7.9 presents the `ResponseSchedule` and the `ResponseScheduleItem` classes. Response schedule items are added

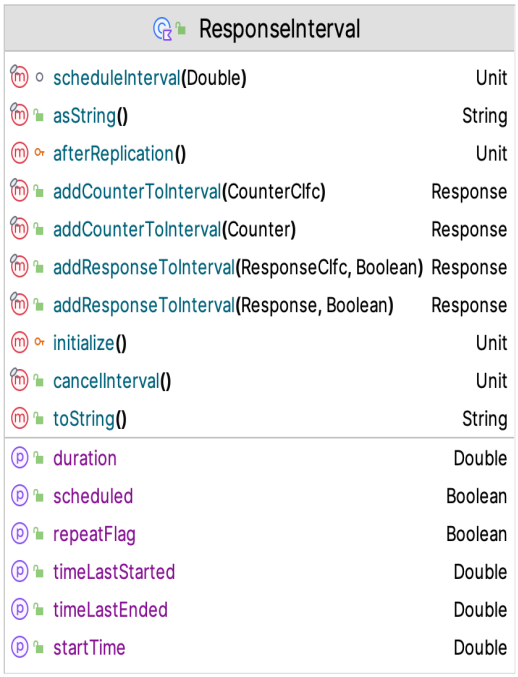


Figure 7.8.: ResponseInterval Class

to a response schedule. In addition, patterns of intervals can be added to the schedule.

The classic use case of this class is to collect statistics for particular hours of the day. For example, the user can use the `addIntervals()` method to add 24 intervals of 1 hour duration. Then, responses (response variables, time weighted variables, and counters) can be added to the schedule. In which case, they will be added to each interval. Thus, interval statistics for each of the 24 intervals will be collected for every one of the added responses. If more than one day is simulated and the schedule is allowed to repeat, then statistics are collected across the days (replications). That is, the statistics of hour 1 on day 1 are averaged with the statistics of hour 1 on all subsequent days.

Because the responses that are associated with intervals are created automatically, it is worth knowing how they are named, since the names are what appear within the statistical reports. Table 7.7 summarizes the conventions.

Table 7.7.: Naming of Automatically Created Interval Responses

Generated name	Created for
`<response>:IntervalAvg:<label>`	every response added to an interval
`<response>:ValueAtStart:<label>`	`TWResponse` instances only
`<response>:<label>:P(Empty)`	only when the empty interval option is requested
`<counter>:<label>`	counters added to an interval

When the interval belongs to a schedule, the `<label>` is itself composed of the name of the schedule, a two digit sequence number, and the label of the interval. An `addIntervals()` function call that is not supplied a label will generate one from the start and end times of the interval. Thus, the

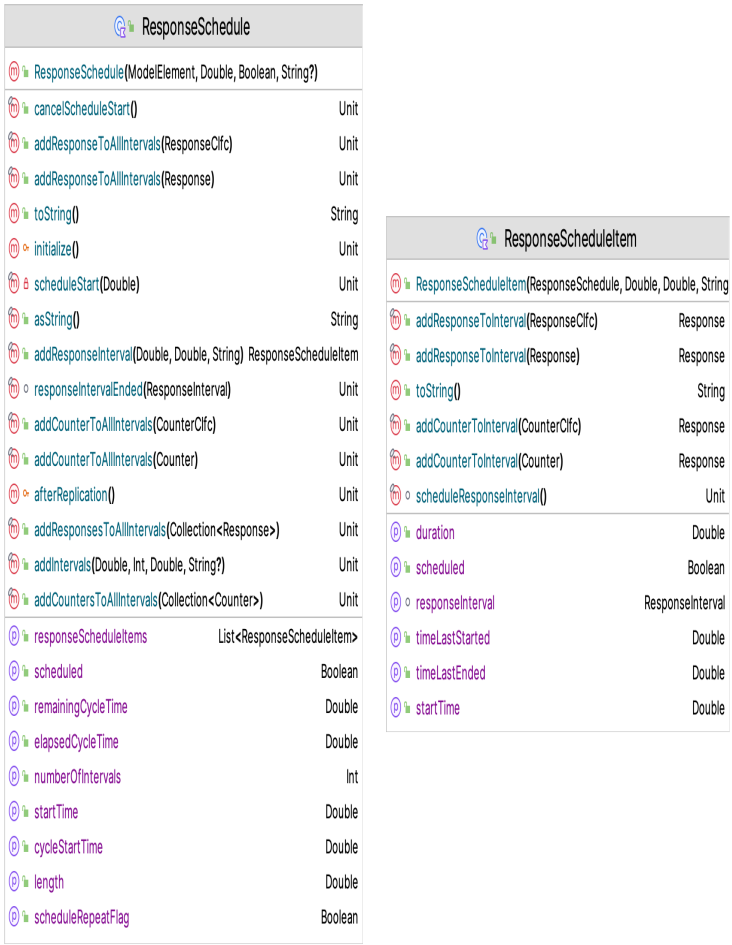


Figure 7.9.: ResponseSchedule and ResponseScheduleItem Classes

first hourly interval of a schedule named `Hourly` produces, for the number of busy units response of the JHBunt recruiters, a statistic named:

```
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:01:[0_0,60_0]
```

The `ValueAtStart` response deserves comment. It is created automatically for every `TWResponse` instance that is added to an interval and it records the value of the variable at the moment that the interval began. For example, it captures the number of students that were already waiting in a queue when the third hour of operation started. An interval average can be misleading without this information, because a high average during an interval may reflect congestion that was inherited from the previous interval rather than conditions that were created during the interval itself.

An additional approach to collecting statistics by period is to use the `TimeSeriesResponse` class. The `TimeSeriesResponse` class addresses the problem of collecting simulation responses by period. A typical use case involves the reporting of statistics over fixed time periods, such as hourly, daily, monthly, yearly, etc. A time series response divides the simulated time horizon into sequential periods, with the periods marking the time frame over which performance is observed. Users specify which responses will be tabulated by providing `Response, TWResponse,` or `Counter` instances. Then, for each period during the simulation horizon, the response over the period is recorded into a series of records, constituting a time series for the response. For `Response` and `TWResponse` instances, the recorded response represents the average of the variable during the period. For `Counter` instances, the total count during the period is recorded. Essentially, the data collected is the same as via the use of a `ResponseSchedule` with intervals representing the time periods. This data is recorded for every response, for every period, for each replication. The response or counter information is recorded at the end of each completed period. The number of periods to collect must be supplied.

The periods are contiguous and of equal length. Let t_0 denote the time at which the first period begins and let Δ denote the length of a period. Then the j^{th} period covers the interval

$$P_j = (\tau_{j-1}, \tau_j] \quad \text{where} \quad \tau_j = t_0 + j\Delta \quad \text{for} \quad j = 1, 2, \ldots, m$$

where m is the number of periods to be collected. The starting time, t_0, is specified by the `defaultStartTime` property, and the period length, Δ, and the number of periods, m, are supplied when the time series response is defined. Because the periods are contiguous, of equal length, and non-overlapping, they form a partition of the observed portion of the time horizon. This is the essential structural difference from a `ResponseSchedule`, whose intervals are placed individually and may overlap.

What is recorded for period P_j depends on the type of the response, as summarized in Table 7.8. The first two rows are exactly the interval averages that were derived at the beginning of this section, applied with $t_b = \tau_{j-1}$ and $t_e = \tau_j$. There is no new interval mathematics here. What is new is that the intervals are laid down as a partition and that the resulting values are indexed by the period number.

Table 7.8.: What Is Recorded for Each Period, by Response Type

Response type	Value recorded for period P_j	Undefined when
`Response`	$\overline{x}(\tau_{j-1}, \tau_j]$	there are no observations during the period
`TWResponse`	$\overline{y}(\tau_{j-1}, \tau_j]$	never
`Counter`	$k(\tau_j) - k(\tau_{j-1})$	never

In the table, $k(t)$ denotes the cumulative count of the counter up to and including time t, so that the value recorded for a counter is simply the

number of counted events that occurred during the period. A counter and a time-persistent variable always have a well-defined value for every period. Only a `Response` instance can fail to produce a value, and only when nothing was observed during the period.

The `TimeSeriesResponse` class does not react to warm up events. That is, periods observed prior to the warm up event will contain data that was observed during the warm up period. The standard usage for this class is likely not within an infinite horizon (steady-state) context. However, if you do not want data collected during warm up periods, then specify the default start time for the time series to be greater than or equal to the specified warm up period length using the `defaultStartTime` property. The default starting time of the first period is at time 0.0. In addition, the collected responses are not automatically shown in console output. However, the data can be accessed via a reference to the class by using functions to access the collected data or by using a `KSLDatabase` instance. The `TimeSeriesResponse` class has the benefit of indicating the periods within the collected data (and within the database). Figure 7.10 illustrates the functions and properties of the `TimeSeriesResponse` class. To specify a time series response the length of the period and the number of periods is required. Sets of responses or counters can be supplied.

In addition to supplying sets of responses and counters, convenience constructors accept a single response, a single counter, or one of each. The number of periods to collect is available through the `numPeriodsToCollect` property. Because that property is annotated as a KSL control, the number of periods can be treated as a factor within a designed experiment, as discussed in Chapter 10. If collection needs to be stopped before the specified number of periods has been observed, the `cancelCollection()` function cancels any scheduled or in-progress period collection. The `periodCounter` property reports the number of periods that have been completed.

Figure 7.10.: TimeSeriesResponse Classes

> **Warning**
>
> Across replication statistics are **not** collected by default. The `acrossRepStatisticsOption` property defaults to false. When it is false, the `acrossReplicationStatisticsByPeriod()` and `allAcrossReplicationStatisticsByPeriodAsDataFrame()` functions return an empty map and an empty data frame, respectively, without any warning. Set the option to true before simulating if you want statistics summarized across replications for each period.

An important difference between a `TimeSeriesResponse` and a `ResponseSchedule` concerns what happens when nothing is observed during a period. Recall that the average of a `Response` instance is undefined if there were no observations during the interval. A `ResponseSchedule` handles this by not tabulating the interval, with the consequence that the resulting statistics are collected over a smaller number of replications than might be expected. A `TimeSeriesResponse` instead records a row for the period with a missing value. Thus, the collected data is *rectangular*. There is one record for every response, for every period, for every replication, whether or not anything was observed. Periods that have nothing to report are explicitly marked as missing rather than silently omitted.

A `TWResponse` instance is treated differently from a `Response` instance in this situation. If a time-persistent variable does not change during a period, its value is still well-defined, because the average of a constant is that constant. Therefore, the value is recorded rather than marked as missing. Only a `Response` instance that has no observations produces a missing value.

It is useful to make this precise, because it determines how many replications actually contribute to each reported period statistic. Let n be the number of replications and let Y_{rj} denote the value recorded for period P_j on replication r. Define the indicator

$$\delta_{rj} = \begin{cases} 1 & \text{if a value was recorded for period } P_j \text{ on replication } r \\ 0 & \text{otherwise} \end{cases}$$

The collected data is the $n \times m$ array $\{Y_{rj}\}$ together with the indicators $\{\delta_{rj}\}$. Every cell of the array exists. The indicator records whether the cell holds a value. This is what is meant by saying that the data is rectangular.

The number of replications that contribute to period P_j is

$$n_j = \sum_{r=1}^{n} \delta_{rj}$$

and the across replication average for the period is

$$\overline{Y}_{\cdot j} = \frac{1}{n_j} \sum_{r=1}^{n} \delta_{rj} Y_{rj}$$

with the sample variance and the confidence interval half-width computed from the n_j contributing observations in the usual manner. In particular, the half-width reported for period P_j is based on n_j, not on n. If some periods were empty on some replications, then different periods are estimated with different amounts of data and the reported half-widths will differ accordingly.

The value of n_j is reported by the `count` property of the statistical summary that is returned by the `allAcrossReplicationStatisticsByPeriodAsList()` function. A count that is less than the number of replications indicates that the period was empty on some of them. This is the natural place to check whether the empty period issue has affected a particular analysis.

It is worth being clear about what $\overline{Y}_{\cdot j}$ estimates. Suppose that the response is time-persistent with a time-varying mean, $\mu(t) = E[Y(t)]$. This is

precisely the situation that arises in the non-stationary models of this chapter. Then, because the value recorded for a period is the time average of the response over that period,

$$E\left[\overline{Y}_{\cdot j}\right] = \frac{1}{\Delta}\int_{\tau_{j-1}}^{\tau_j} \mu\left(t\right) \mathrm{dt}$$

That is, the period average estimates the *average of the mean function over the period*. It does not estimate the value of the mean at any particular instant within the period, and it does not estimate a steady-state mean. For a tally-based response the corresponding statement is only approximate, because the period average is a ratio in which the number of observations during the period is itself random.

Consequently, the sequence

$$\overline{Y}_{\cdot 1}, \overline{Y}_{\cdot 2}, \ldots, \overline{Y}_{\cdot m}$$

is a piecewise summary of $\mu(t)$ at a resolution that is determined by the period length, Δ. A shorter period resolves more of the time-varying behavior, at the cost of fewer observations within each period. This is why plotting the sequence, rather than reading the individual values, is usually the most informative way to examine the results.

One caution applies when interpreting the sequence. Across replications, the values $Y_{1j}, Y_{2j}, \ldots, Y_{nj}$ for a given period are independent, because the replications are independent. Therefore the confidence interval that is reported for each period is valid. Within a replication, however, the values $Y_{r1}, Y_{r2}, \ldots, Y_{rm}$ across the periods are generally correlated, because the state of the system at the end of one period is the state of the system at the beginning of the next. The sequence of period averages should not be treated as a set of m independent observations. The methods discussed in Chapter 5 apply.

Figure 7.11 compares how the three mechanisms place their observation intervals over the simulated time horizon. In the figure, the darker rectangles are the intervals as originally defined and the lighter rectangles are their repetitions. Notice that only the `ResponseSchedule` permits intervals that overlap in time.

Table 7.9 summarizes the differences between the two main mechanisms for collecting statistics by period.

Table 7.9.: Comparing the `ResponseSchedule` and `TimeSeriesResponse` Classes

	`ResponseSchedule`	`TimeSeriesResponse`
Appears automatically in the summary reports	yes	no
Intervals may overlap	yes	no, contiguous only
Repeats over cycles	yes	no, a fixed number of periods
Different responses on different intervals	yes	no, all responses on all periods
How the period is identified in the data	encoded within the response name	an explicit period column
Rectangular data with explicit missing values	no	yes
Across replication statistics by period	automatic	optional, see `acrossRepStatisticsOption`

In summary, use a `ResponseSchedule` when you want the interval statistics to appear within the standard reports alongside every other response, when the intervals form a repeating daily or weekly pattern, or when the intervals

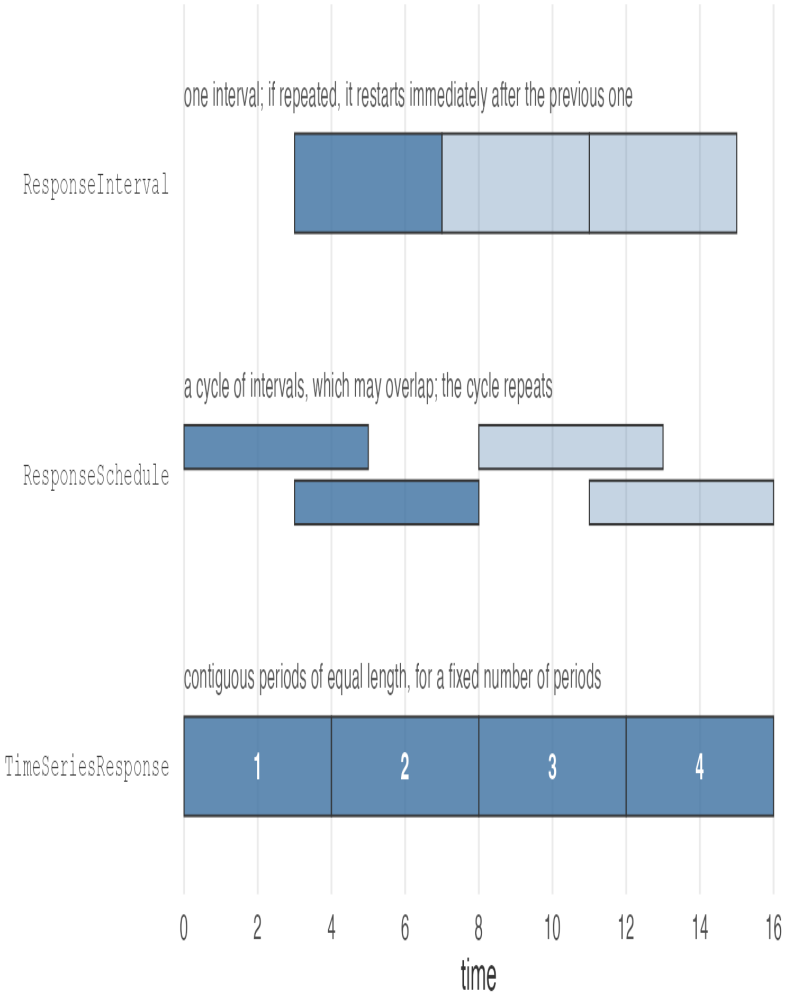


Figure 7.11.: How Each Mechanism Places Observation Intervals over the Simulated Horizon

overlap. Use a `TimeSeriesResponse` when you intend to analyze or plot the results as a series, because the explicit period numbering and the rectangular, missing-aware structure of the data are designed for exactly that purpose.

The next section applies these constructs, together with the arrival and capacity schedules of the previous sections, to an enhanced version of the STEM Career Mixer model.

7.4. Enhancing the STEM Career Mixer Model

In Chapter 6, we discussed the implementation of a model for the STEM Career Fair Mixer of Example 6.8. In this section, we put the non-stationary modeling concepts of the previous sections into practice by exploring an enhanced version of the STEM Career Fair Mixer system.

7.4.1. Non-Stationary STEM Career Fair Mixer

In this example, we embellish the STEM Career Fair mixer model to add a few modeling issues that will make the model a bit more realistic in order to illustrate the non-stationary modeling constructs provided by the KSL. The following issues will be addressed in the new model.

- In reality, those students that wander around before deciding to visit the recruiters actually visit the conversation area associated with the mixer. The organizer of the event has asked alumni to volunteer to be available within the conversation area to chat with students and discuss their career development. After all, it is a mixer. The organizer of the mixer would like to plan for the size of the conversation area.

- The STEM Fair is scheduled for 6 hours; however, currently, the simulation simply ends "abruptly" at 6 hours. How should we handle those students that are attending the mixer when the mixer ends? In this section, we will model a more realistic approach to ending the mixer and handling the students in progress.

Example 7.7. After discussing with the STEM Fair organizer, the following additional facts were discovered. Generally, near the end of the scheduled mixer time, an announcement is made to everyone in the facility through the public address (PA) system that the mixer will be ending soon. For the purposes of this modeling enhancement, we can assume that the announcement goes out 15 minutes before the end of the mixer. At that time, we can assume the following:

- The doors to the mixture are closed to new arrivals. In other words, a closed sign goes up so that no new students are admitted to the mixer.
- Any students that were chatting within the conversation area finish up their conversations and then head for the exit, without visiting the recruiting stations.
- Any student walking to the name tag area, the conversation area or to the recruiting area proceed to the exit.
- Any students already at the recruiting stations, either in line waiting, or talking with the recruiter are allowed to finish up their visit and then depart the mixer.

Based on additional observations and discussion with the STEM mixer organizer, the following changes can be assumed:

- Because the distances between the stations in the hall are now important an enhanced drawing of the system has been made that better illustrates the standard layout that has been used in the past for the mixer. The distances shown in the drawing are rough estimates.
- New data suggest that 60% of the arriving students do not visit the conversation area, but instead go directly to the recruiting area to pick one of the two recruiting stations to visit, with equal probability.
- The remaining 40% of students will first visit the conversation area. The time spent within the conversation area is a little less than previously noted to exclude the time spent walking. The conversation time is triangularly distribution with a minimum of 10 minutes, a most likely value of 15 minutes, and a maximum value of 30 minutes. After having their conversations, 90% of the students decide to visit one of the two recruiting stations. The other 10% are too tired or timid and decide to leave.
- The speed of people walking can vary greatly, but prior data suggests that for short distances within and around buildings, people walk between 1 mile per hour and 3 miles per hour, with a most likely time of 2 miles per hour, triangularly distributed.

After further conversations with students who have attended past mixers, it was discovered that there really isn't a preference for visiting one of the recruiters first. Instead, after arriving to the recruiting area, students decide to visit the recruiter that has the least number of people (waiting and interacting with the recruiters). If both recruiter locations have the same total number of students, then the students show no preference between the two recruiters. After visiting both, they depart.

Finally, suppose that new data on the arrival behavior of the students have become available. Over the course of the 6 hours that the mixer is open, the new data indicates that the rate varies significantly by the time of day. Let's assume that the Mixer opens at 2 pm and lasts until 8 pm. Data was collected over 30 minute intervals during the 6 hours of the mixer

and it showed that there was a larger arrival rate during the middle hours of the operating hours than during the beginning or ending hours of the data. The Table 7.10 summarizes the arrival rate for each 30 minute period throughout the 6 hour period.

Table 7.10.: Student Hourly Arrival Rates by 30 Minute Periods

Period	Time Frame	Duration	Mean Arrival Rate per Hour
1	2 - 2:30 pm	30	5
2	2:30 – 3 pm	30	10
3	3 - 3:30 pm	30	15
4	3:30 – 4 pm	30	25
5	4 - 4:30 pm	30	40
6	4:30 – 4 pm	30	50
7	5 - 5:30 pm	30	55
8	5:30 – 6 pm	30	60
9	6 - 6:30 pm	30	60
10	6:30 – 7 pm	30	30
11	7 - 7:30 pm	30	5
12	7:30 – 8 pm	30	5

Because of the time varying nature of the arrival of students, the STEM mixer organizers have noticed that during the peak period of the arrivals there are substantially longer lines of students waiting to speak to a recruiter and often there are not enough tables within the conversation area for students. Thus, the mixer organizers would like to advise the recruiters as to how to staff their recruiting stations during the event. As part of the analysis, they want to ensure that 90 percent of the students experience an average system time of 35 minutes or less during the 4:30-6:30 pm time frame. They would also like to measure the average number of students waiting for recruiters during this time frame as well as the average number of busy recruiters at the MalWart and JHBunt recruiting stations. You should first perform an analysis based on the current staffing

recommendations from the previous modeling and then compare those results to a staffing plan that allows the recruiters to change their assigned number of recruiters by hour.

Because of these changes to the modeling assumptions, the STEM organizer would like to better understand the following aspects of potential system performance.

- When the PA announcement occurs, how many students on average are:
  - In the conversation area
  - Attending the MalWart recruiting station
  - Attending the JHBunt recruiting station
- How long after the 6-hour scheduled time does the mixer actually end? That is, since all students are permitted to finish up what they are currently doing when the PA announcement occurs, how long does it take for the hall to be completely empty?
- How many students, on average, are in the conversation area?

7.4.2. Modeling Walking Time

To model the walking within the mixer, we need to translate the distance traveled into time. Since we are told the typical velocity for walking within a building for a person, we can randomly generate the velocity associated with a walking trip. Then, if we know the distance to walk, we can determine the time via the following relationship, where v is the speed (velocity), d is the distance, and t is the time.

$$v = \frac{d}{t}$$

Thus, we can randomly generate the value of v, and use the relationship to determine the time taken.

$$t = \frac{d}{v}$$

We know that the speed of people walking is triangularly distributed with a minimum of 1 mile per hour, a mode of 2 miles per hour, and a maximum of 3 miles per hour. This translates into a minimum of 88 feet per minute, a mode of 176 feet per minute, and a maximum of 264 feet per minute. Thus, we represent the speed as a random variable, Speed ~ TRIA(88, 176, 264) feet per minute and if we know the distance to travel, we can compute the time.

We have the following distances between the major locations of the system:

- Entrance to Name Tags, 20 feet
- Name Tags to Conversation Area, 30 feet
- Name Tags to Recruiting Area (either JHBunt or MalMart), 80 feet
- Name Tags to Exit, 140 feet
- Conversation Area to Recruiting Area (either JHBunt or MalMart), 50 feet
- Conversation Area to Exit, 110 feet
- Recruiting Area (either JHBunt or MalMart) to Exit, 60 feet

For simplicity, we assume that the distance between the JHBunt and MalWart recruiting locations within the recruiting area can be ignored.

7.4.3. Collecting Period Statistics for the Mixer Model

The following code illustrates the definition of a schedule to collect hourly responses for the STEM Fair Mixer situation.

```
    private val hourlyResponseSchedule =
     ↪  ResponseSchedule(this, 0.0, name = "Hourly")
    private val peakResponseInterval: ResponseInterval =
     ↪  ResponseInterval(this, 120.0,
     ↪  "PeakPeriod:[150.0,270.0]")

    init {
        hourlyResponseSchedule.scheduleRepeatFlag = false
        hourlyResponseSchedule.addIntervals(0.0, 6, 60.0)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myJHBuntRecruiters.numBusyUnits)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myMalWartRecruiters.numBusyUnits)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myJHBuntRecruiters.waitingQ.timeInQ)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myMalWartRecruiters.waitingQ.timeInQ)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myJHBuntRecruiters.timeAvgInstantaneousUtil)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myMalWartRecruiters.timeAvgInstantaneousUtil)
        peakResponseInterval.startTime = 150.0
        peakResponseInterval.addResponseToInterval(
↪   myTotalAtRecruiters)
        peakResponseInterval.addResponseToInterval(
↪   myJHBuntRecruiters.timeAvgInstantaneousUtil)
        peakResponseInterval.addResponseToInterval(
↪   myMalWartRecruiters.timeAvgInstantaneousUtil)
    }
```

In the code, an hourly response schedule is constructed and responses added to all the intervals. Access to specific intervals can be obtained and specific responses only added to specific intervals if needed. The code also illustrates the construction of a separate `ResponseInterval` and the adding of responses to it. Using a `ResponseSchedule` will result in many statistical quantities being defined. All the statistical responses will be automatically added to the summary statistical reports and captured within the KSL database (if used). The names of the statistics that are produced by this schedule follow the conventions summarized in Table 7.7.

The following code illustrates how to use the `TimeSeriesResponse` class within the context of the STEM Career Fair model.

```
    val timeSeriesResponse = TimeSeriesResponse(this, 60.0,
     ↪ 6,
        setOf(myJHBuntRecruiters.numBusyUnits,
↪ myMalWartRecruiters.numBusyUnits))
```

In this code, the time series response is set up to collect the number of busy units statistics for the JHBunt and MalWart recruiters. Setting up the `TimeSeriesResponse` class is simpler than using the `ResponseSchedule` class; however, the pattern of collection is limited to each period. In addition, the results are not automatically shown in console output. The following code illustrates how to extract the results from the time series response.

```
    val m = Model("STEM Fair Model")
    val mixer = StemFairMixerEnhancedSched(m, "Stem Fair
     ↪ Scheduled")
    mixer.timeSeriesResponse.acrossRepStatisticsOption = true
    val kslDb = KSLDatabaseObserver(m)
```

```
    mixer.warningTime = 30.0
    m.numberOfReplications = 400
    m.simulate()
    m.print()
    val r = m.simulationReporter
    r.writeHalfWidthSummaryReportAsMarkDown(KSL.out, df =
↪   MarkDown.D3FORMAT)
    kslDb.db.timeSeriesResponseViewData.print()
    println()
    mixer.timeSeriesResponse.
↪   allAcrossReplicationStatisticsByPeriodAsDataFrame().
↪   print()
```

In the code, the reference to the time series response is made public to access the results. In addition, a `KSLDatabaseObserver` and resulting `KSLDatabase` are used to extract the results to data frames for display purposes.

The next section will overview the revisions to the STEM Mixer model to handle the non-stationary situation.

7.4.4. Implementing the Revised STEM Mixer Model

Since the STEM mixer model has been presented in a previous chapter, this section will focus on the aspects associated with the enhancements required for this chapter. Since the mixer can close with a warning time, we need variables to handle this situation.

```
class StemFairMixerEnhanced(parent: ModelElement, name:
↪   String? = null) : ProcessModel(parent, name) {

    var lengthOfMixer = 360.0
```

```
        set(value) {
            require(value > 0.0) { "The length of the mixer
↪   must be > 0.0" }
            field = value
        }
    var warningTime = 15.0
        set(value) {
            require(value > 0.0) { "The warning limit must be
↪   > 0.0" }
            field = value
        }

    val doorClosingTime
        get() = lengthOfMixer - warningTime

    var isClosed: Boolean = false
        private set
```

The length of the mixer can be varied and the length of the warning changed. The property `isClosed` will be used to indicate that students need to proceed to the exit. This situation involves the walking of the students. Thus, we need to specify the distances, velocity, and walking times. The following code defines the random variables needed for this modeling.

```
    private val myNameTagTimeRV = RandomVariable(this,
     ↪  UniformRV((15.0 / 60.0), (45.0 / 60.0), 2))
    private val myDecideToMix = RandomVariable(this,
     ↪  BernoulliRV(0.4, 3))
    private val myDecideToLeave = RandomVariable(this,
     ↪  BernoulliRV(0.1, 4))
    private val myInteractionTimeRV = RandomVariable(this,
     ↪  TriangularRV(10.0, 15.0, 30.0, 5))
```

```
    private val myDecideRecruiter = RandomVariable(this,
     ↪  BernoulliRV(0.5, 6))
    private val myTalkWithJHBunt = RandomVariable(this,
     ↪  ExponentialRV(6.0, 7))
    private val myTalkWithMalMart = RandomVariable(this,
     ↪  ExponentialRV(3.0, 8))
    private val myWalkingSpeedRV = TriangularRV(88.0, 176.0,
     ↪  264.0, 9)

    private val walkToNameTags = RandomVariable(this, 20.0 /
     ↪  myWalkingSpeedRV)
    private val walkFromNameTagsToConversationArea =
     ↪  RandomVariable(this, 30.0 / myWalkingSpeedRV)
    private val walkFromNameTagsToRecruiting =
     ↪  RandomVariable(this, 80.0 / myWalkingSpeedRV)
    private val walkFromNameTagsToExit = RandomVariable(this,
     ↪  140.0 / myWalkingSpeedRV)
    private val walkFromConversationAreaToRecruiting =
     ↪  RandomVariable(this, 50.0 / myWalkingSpeedRV)
    private val walkFromConversationAreaToExit =
     ↪  RandomVariable(this, 110.0 / myWalkingSpeedRV)
    private val walkFromRecruitingToExit =
     ↪  RandomVariable(this, 60.0 / myWalkingSpeedRV)
```

Notice that a random variable is used to defined the walking speed and that this random variable is subsequently used to define the time to walk random variable. Because the walking speed random variable, `myWalkingSpeedRV` implements the `RVariableIfc` interface, the expression `20.0 / myWalkingSpeedRV` actually produces another instance of a `RVariableIfc` and it can be used within the `RandomVariable` instance of the model.

There are many base responses to be defined. Besides the overall system

time, we can collect the system time based on the type of student and use an indicator response to capture the probability of completing the visit within 30 minutes.

```
    private val myOverallSystemTime = Response(this,
     ↪  "OverallSystemTime")
    private val myRecruitingOnlySystemTime = Response(this,
     ↪  "RecruitingOnlySystemTime")

    init {
        myRecruitingOnlySystemTime.attachIndicator({ x -> x
↪   <= 30.0 }, "P(Recruiting Only < 30 minutes)")
    }

    private val myMixingStudentSystemTime = Response(this,
     ↪  "MixingStudentSystemTime")
    private val myMixingAndLeavingSystemTime = Response(this,
     ↪  "MixingStudentThatLeavesSystemTime")
    private val myNumInSystem = TWResponse(this,
     ↪  "NumInSystem")
    private val myNumInConversationArea = TWResponse(this,
     ↪  "NumInConversationArea")

    private val myNumAtJHBuntAtClosing = Response(this,
     ↪  "NumAtJHBuntAtClosing")
    private val myNumAtMalWartAtClosing = Response(this,
     ↪  "NumAtMalWartAtClosing")
    private val myNumAtConversationAtClosing = Response(this,
     ↪  "NumAtConversationAtClosing")
    private val myNumInSystemAtClosing = Response(this,
     ↪  "NumInSystemAtClosing")

    private val myJHBuntRecruiters: ResourceWithQ =
     ↪  ResourceWithQ(this, capacity = 3, name = "JHBuntR")
```

```
    val JHBuntRecruiters: ResourceCIfc
        get() = myJHBuntRecruiters
    private val myMalWartRecruiters: ResourceWithQ =
     ↪  ResourceWithQ(this, capacity = 2, name = "MalWartR")
    val MalWartRecruiters: ResourceCIfc
        get() = myMalWartRecruiters

    private val myTotalAtRecruiters: AggregateTWResponse =
     ↪  AggregateTWResponse(this, "StudentsAtRecruiters")
    init {
        myTotalAtRecruiters.observe(myJHBuntRecruiters.
↪   numBusyUnits)
        myTotalAtRecruiters.observe(myJHBuntRecruiters.
↪   waitingQ.numInQ)
        myTotalAtRecruiters.observe(myMalWartRecruiters.
↪   numBusyUnits)
        myTotalAtRecruiters.observe(myMalWartRecruiters.
↪   waitingQ.numInQ)
    }
```

We also need to define new responses to capture what is going on at the close of the mixer. That is, when the warning occurs that the mixer is closing. Notice also that we use an aggregate response to collect the total number of students at both of the recruiters.

To model the non-stationary arrivals, we use a piece wise constant rate function to define a non-homogeneous Poisson process and an event generator to generate the students.

```
    private val rateFunction: PiecewiseConstantRateFunction

    fun adjustRates(factor: Double){
        require(factor > 0.0) {"the adjustment factor must be
↪   >= 0.0"}
```

```
        generator.adjustRates(factor)
    }

    init {
        // set up the generator
        val durations = doubleArrayOf(
            30.0, 30.0, 30.0, 30.0, 30.0, 30.0,
            30.0, 30.0, 30.0, 30.0, 30.0, 30.0
        )
        val hourlyRates = doubleArrayOf(
            5.0, 10.0, 15.0, 25.0, 40.0, 50.0,
            55.0, 55.0, 60.0, 30.0, 5.0, 5.0
        )
        val ratesPerMinute = hourlyRates.divideConstant(60.0)
        rateFunction =
↪   PiecewiseConstantRateFunction(durations, ratesPerMinute)
    }

    private val generator =
     ↪  NHPPPiecewiseRateFunctionEventGenerator(this,
     ↪  this::createStudents,
        rateFunction = rateFunction, streamNum = 1)
```

The interval responses are defined as previously discussed. We also define a response to capture when the mixer ends. Notice that the `ResponseSchedule` is told not to repeat. This is essential because we will not specify a replication run length for this model.

```
    private val hourlyResponseSchedule =
     ↪  ResponseSchedule(this, 0.0, name = "Hourly")
    private val peakResponseInterval: ResponseInterval =
     ↪  ResponseInterval(this, 120.0,
     ↪  "PeakPeriod:[150.0,270.0]")
```

```
    val timeSeriesResponse = TimeSeriesResponse(this, 60.0,
     ↪  6,
        setOf(myJHBuntRecruiters.numBusyUnits,
↪   myMalWartRecruiters.numBusyUnits))

    init {
        hourlyResponseSchedule.scheduleRepeatFlag = false
        hourlyResponseSchedule.addIntervals(0.0, 6, 60.0)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myJHBuntRecruiters.numBusyUnits)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myMalWartRecruiters.numBusyUnits)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myJHBuntRecruiters.waitingQ.timeInQ)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myMalWartRecruiters.waitingQ.timeInQ)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myJHBuntRecruiters.timeAvgInstantaneousUtil)
        hourlyResponseSchedule.addResponseToAllIntervals(
↪   myMalWartRecruiters.timeAvgInstantaneousUtil)
        peakResponseInterval.startTime = 150.0
        peakResponseInterval.addResponseToInterval(
↪   myTotalAtRecruiters)
        peakResponseInterval.addResponseToInterval(
↪   myJHBuntRecruiters.timeAvgInstantaneousUtil)
        peakResponseInterval.addResponseToInterval(
↪   myMalWartRecruiters.timeAvgInstantaneousUtil)
    }

    private val myEndTime = Response(this, "Mixer Ending
     ↪  Time")
    init{
        myEndTime.attachIndicator({ x -> x > lengthOfMixer },
↪   "Prob(EndTime>$lengthOfMixer)")
```

```
    }
```

At the start of the replication, we need to schedule the closing of the mixer at the appropriate time and activate the processes for the students.

```
    override fun initialize() {
        isClosed = false
        schedule(this::closeMixer, doorClosingTime)
    }

    override fun replicationEnded() {
        myEndTime.value = time
    }

    private fun createStudents(eventGenerator:
    ↪ EventGenerator) {
        val student = Student()
        if (student.isMixer) {
            activate(student.mixingStudentProcess)
        } else {
            activate(student.recruitingOnlyStudentProcess)
        }
    }

    private fun closeMixer(event: KSLEvent<Nothing>) {
        isClosed = true
        // turn off the generator
        generator.turnOffGenerator()
        // collect statistics
        myNumAtJHBuntAtClosing.value =
↪   myJHBuntRecruiters.waitingQ.numInQ.value +
↪   myJHBuntRecruiters.numBusy
        myNumAtMalWartAtClosing.value =
↪   myMalWartRecruiters.waitingQ.numInQ.value +
↪   myMalWartRecruiters.numBusy
```

```
        myNumAtConversationAtClosing.value =
↪   myNumInConversationArea.value
        myNumInSystemAtClosing.value = myNumInSystem.value
    }
```

The `initialize()` method is used to schedule the closing of the mixer. In the event action associated with the closing, we turn off the student generation process and we capture the required statistics about the state of the mixer at that time. Notice the action associated with the event generator. In this case, we are not using an entity generator so that we can activate the process associated with whether the student visits the conversation area or not.

Let's first take a look at the process for those students that visit the conversation area. It is quite a lengthy process routine. In the following code, first we define two properties that capture whether the student mixes (`isMixer`) and whether they leave early (`isLeaver`). Then, the mixing student process is defined. In the definition, note that we do not add the process to the entity's process sequence. This is because we will have another process (for non-mixing students) and we do not want the processes to automatically start. We controlled the starting of the processes within the event generator action.

```
    private inner class Student : Entity() {
        val isMixer = myDecideToMix.value.toBoolean()
        val isLeaver = myDecideToLeave.value.toBoolean()

        val mixingStudentProcess = process {
            myNumInSystem.increment()
            delay(walkToNameTags)
            // at name tag station
            if (isClosed) {
                // mixer closed during walking
```

```
                delay(walkFromNameTagsToExit)
                departMixer(this@Student)
            } else {
                // get name tags
                delay(myNameTagTimeRV)
                if (isClosed) {
                    // mixer closed during name tag
                    delay(walkFromNameTagsToExit)
                    departMixer(this@Student)
                } else {
                    // goto the conversation area
                    delay(walkFromNameTagsToConversationArea)
                    if (isClosed) {
                        // closed during walking, must leave
                        delay(walkFromConversationAreaToExit)
                        departMixer(this@Student)
                    } else {
                        // start the conversation
                        myNumInConversationArea.increment()
                        delay(myInteractionTimeRV)
                        myNumInConversationArea.decrement()
                        if (isClosed) {
                            // closed during conversation,
                             ↪ must leave

↪ delay(walkFromConversationAreaToExit)
                            departMixer(this@Student)
                        } else {
                            // decide to leave or go to
                             ↪ recruiting
                            if (isLeaver) {

↪ delay(walkFromConversationAreaToExit)
```

```
                                departMixer(this@Student)
                            } else {

↪  delay(walkFromConversationAreaToRecruiting)
                                if (!isClosed) {
                                    // proceed with
                                     ↪  recruiting visit
                                    val firstRecruiter =
                                     ↪  decideRecruiter()
                                    if (firstRecruiter ==
                                     ↪  myJHBuntRecruiters) {

↪  use(myJHBuntRecruiters, delayDuration = myTalkWithJHBunt)

↪  use(myMalWartRecruiters, delayDuration =
↪  myTalkWithMalMart)
                                    } else {

↪  use(myMalWartRecruiters, delayDuration =
↪  myTalkWithMalMart)

↪  use(myJHBuntRecruiters, delayDuration = myTalkWithJHBunt)
                                    }
                                }
                                // either closed or they
                                 ↪  visited recruiting

↪  delay(walkFromRecruitingToExit)
                                departMixer(this@Student)
                            }
                        }
                    }
                }
```

```
            }
        }
```

The process description is lengthy but fairly self-explanatory. The key complicating factor is that after each delay (for walking, name tags, etc.), we need to check if the warning for the closing of the mixer has occurred. The long if-else constructs ensure that there is only one path through the process for those students that depart early or due to closing. Those students that depart are sent to the `departMixer()` method, which collects statistics. The process of visiting the recruiters is more complex than in the previous presentation. In this case, we use a function, `decideRecruiter()` that picks the first station to visit.

```
private fun decideRecruiter(): ResourceWithQ {
     // check the equal case first to show no preference
     val j = myJHBuntRecruiters.waitingQ.size +
      ↪  myJHBuntRecruiters.numBusy
     val m = myMalWartRecruiters.waitingQ.size +
      ↪  myMalWartRecruiters.numBusy
     if (j == m ){
         if (myDecideRecruiter.value.toBoolean()) {
             return myJHBuntRecruiters
         } else {
             return myMalWartRecruiters
         }
     } else if (j < m) {
         return myJHBuntRecruiters
     } else  {
         // MalWart must be smaller
         return myMalWartRecruiters
     }
 }
```

The `decideRecruiter()` function compares the recruiting stations based on the number waiting and in service. If they are the same, we randomly pick between them; otherwise, we pick the recruiter that has less activity. To show no preference to either recruiter, it is important to first check the equality case. For example, if we were to select the recruiter with the least activity first, then whichever one we listed would be checked in the order of the if statement and thus a preference established. The process for the students that only visit the recruiting area is less complicated.

```
        val recruitingOnlyStudentProcess = process {
            myNumInSystem.increment()
            delay(walkToNameTags)
            // at name tag station
            if (isClosed) {
                // mixer closed during walking
                delay(walkFromNameTagsToExit)
                departMixer(this@Student)
            } else {
                delay(myNameTagTimeRV)
                if (isClosed) {
                    // mixer closed during name tag
                    delay(walkFromNameTagsToExit)
                    departMixer(this@Student)
                } else {
                    // proceed to recruiting
                    delay(walkFromNameTagsToRecruiting)
                    if (!isClosed) {
                        // proceed with recruiting visit
                        val firstRecruiter =
                         ↪  decideRecruiter()
                        if (firstRecruiter ==
                         ↪  myJHBuntRecruiters) {
                            use(myJHBuntRecruiters,
↪  delayDuration = myTalkWithJHBunt)
```

```
                            use(myMalWartRecruiters,
↪  delayDuration = myTalkWithMalMart)
                        } else {
                            use(myMalWartRecruiters,
↪  delayDuration = myTalkWithMalMart)
                            use(myJHBuntRecruiters,
↪  delayDuration = myTalkWithJHBunt)
                        }
                    }
                    // either closed or they visited
                     ↪  recruiting
                    delay(walkFromRecruitingToExit)
                    departMixer(this@Student)
                }
            }
```

As previously mentioned, the `departMixer()` function captures the statistics. We use the `isMixer` and `isLeaver` attributes to ensure the capture of system time for these types of students.

```
        private fun departMixer(departingStudent: Student) {
            myNumInSystem.decrement()
            val st = time - departingStudent.createTime
            myOverallSystemTime.value = st
            if (isMixer) {
                myMixingStudentSystemTime.value = st
                if (isLeaver) {
                    myMixingAndLeavingSystemTime.value = st
                }
            } else {
                myRecruitingOnlySystemTime.value = st
            }
```

```
        }
```

Now we are ready to run the base case of the mixer model before investigating the staffing and scheduling of the recruiters.

7.4.5. Running the Revised STEM Mixer Model

The base case assumes that the recruiters have the same capacity for the entire 6 hours of operation. The output from this model is quite lengthy. Notice that the response schedule collects statistics on the average during the interval for both the `Response` and `TWResponse` variables and the value of the variable at the start of the interval for `TWResponse` variables. The naming conventions for these automatically created responses are summarized in Table 7.7. There are a few items to note:

- The overall system time is quite high.
- The number of students at the recruiting area is high when the warning occurs, especially at the JHBunt recruiters with more than 35 students on average.
- The utilization of the recruiters over the entire time frame is very reasonable, 0.7838 and 0.5908 for the JHBunt and MalWart recruiters, respectively.
- The number of busy recruiters gets excessively high during the peak period, with the utilization averaging 99% for both recruiters.
- We see that the mixer ends substantially after the closing time.

The non-stationary aspects of this situation definitely have an effect on the performance. The hourly performance is especially helpful to understand this situation. If we only had the average performance over the entire time, we might be fooled into thinking that the capacity of the recruiters was not a problem.

Base Case Statistical Summary Report

Name	Count	Average	Half-Width
OverallSystem-Time	400	65.9	1.495
Recruitin-gOnlySystem-Time	400	58.287	1.486
P(Recruiting Only < 30 minutes)	400	0.341	0.011
MixingStu-dentSystemTime	400	77.295	1.586
MixingStudent-ThatLeavesSys-temTime	400	19.738	0.167
NumInSystem	400	27.092	0.569
NumInConversa-tionArea	400	3.033	0.034
NumAtJHBun-tAtClosing	400	35.618	1.232
NumAtMal-WartAtClosing	400	8.057	0.827
NumAtConver-sationAtClosing	400	0.635	0.077
NumInSystem-AtClosing	400	44.615	1.601
JHBuntR:In-stantaneousUtil	400	0.785	0.004
JHBuntR:Num-BusyUnits	400	2.356	0.011
JHBuntR:Sched-uledUtil	400	0.785	0.004
JHBuntR:WIP	400	15.739	0.422

Name	Count	Average	Half-Width
JHBuntR:Q:NumInQ	400	13.384	0.417
JHBuntR:Q:TimeInQ	400	34.2	1.164
MalWartR:InstantaneousUtil	400	0.595	0.006
MalWartR:NumBusyUnits	400	1.191	0.011
MalWartR:ScheduledUtil	400	0.595	0.006
MalWartR:WIP	400	7.72	0.257
MalWartR:Q:NumInQ	400	6.529	0.251
MalWartR:Q:TimeInQ	400	16.474	0.614
StudentsAtRecruiters	400	23.459	0.571
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.455	0.028
JHBuntR:NumBusyUnits:ValueAtStart:Hourly:01:[0_0,60_0]	400	0	0
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:02:[60_0,120_0]	400	1.396	0.046

Name	Count	Average	Half-Width
JHBuntR:NumBusyUnits:ValueAtStart:Hourly:02:[60_0,120_0]	400	0.818	0.083
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:03:[120_0,180_0]	400	2.716	0.029
JHBuntR:NumBusyUnits:ValueAtStart:Hourly:03:[120_0,180_0]	400	2.165	0.097
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:04:[180_0,240_0]	400	2.996	0.002
JHBuntR:NumBusyUnits:ValueAtStart:Hourly:04:[180_0,240_0]	400	2.983	0.015
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:05:[240_0,300_0]	400	3	0
JHBuntR:NumBusyUnits:ValueAtStart:Hourly:05:[240_0,300_0]	400	3	0
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:06:[300_0,360_0]	400	2.998	0.002
JHBuntR:NumBusyUnits:ValueAtStart:Hourly:06:[300_0,360_0]	400	3	0

Name	Count	Average	Half-Width
MalWartR:NumBusyUnits:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.25	0.014
MalWartR:NumBusyUnits:ValueAtStart:Hourly:01:[0_0,60_0]	400	0	0
MalWartR:NumBusyUnits:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.755	0.026
MalWartR:NumBusyUnits:ValueAtStart:Hourly:02:[60_0,120_0]	400	0.465	0.064
MalWartR:NumBusyUnits:IntervalAvg:Hourly:03:[120_0,180_0]	400	1.638	0.025
MalWartR:NumBusyUnits:ValueAtStart:Hourly:03:[120_0,180_0]	400	1.152	0.077
MalWartR:NumBusyUnits:IntervalAvg:Hourly:04:[180_0,240_0]	400	1.976	0.006
MalWartR:NumBusyUnits:ValueAtStart:Hourly:04:[180_0,240_0]	400	1.92	0.032
MalWartR:NumBusyUnits:IntervalAvg:Hourly:05:[240_0,300_0]	400	1.997	0.003

Name	Count	Average	Half-Width
MalWartR:NumBusyUnits:ValueAtStart:Hourly:05:[240_0,300_0]	400	1.995	0.007
MalWartR:NumBusyUnits:IntervalAvg:Hourly:06:[300_0,360_0]	400	1.834	0.03
MalWartR:NumBusyUnits:ValueAtStart:Hourly:06:[300_0,360_0]	400	1.982	0.016
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:01:[0_0,60_0]	399	0.04	0.023
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.557	0.098
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:03:[120_0,180_0]	400	5.856	0.376
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:04:[180_0,240_0]	400	20.507	0.776
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:05:[240_0,300_0]	400	38.733	1.04

Name	Count	Average	Half-Width
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:06:[300_0,360_0]	400	56.158	1.319
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:01:[0_0,60_0]	399	0.03	0.013
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.324	0.046
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:03:[120_0,180_0]	400	3.524	0.247
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:04:[180_0,240_0]	400	12.886	0.572
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:05:[240_0,300_0]	400	26.028	0.843
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:06:[300_0,360_0]	400	31.864	1.39
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.152	0.009

Name	Count	Average	Half-Width
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:01:[0_0,60_0]	400	0	0
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.465	0.015
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:02:[60_0,120_0]	400	0.272	0.028
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:03:[120_0,180_0]	400	0.905	0.01
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:03:[120_0,180_0]	400	0.722	0.032
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:04:[180_0,240_0]	400	0.999	0.001
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:04:[180_0,240_0]	400	0.994	0.005
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:05:[240_0,300_0]	400	1	0

Name	Count	Average	Half-Width
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:05:[240_0,300_0]	400	1	0
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:06:[300_0,360_0]	400	0.999	0.001
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:06:[300_0,360_0]	400	1	0
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.125	0.007
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:01:[0_0,60_0]	400	0	0
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.378	0.013
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:02:[60_0,120_0]	400	0.232	0.032
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:03:[120_0,180_0]	400	0.819	0.012

Name	Count	Average	Half-Width
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:03:[120_0,180_0]	400	0.576	0.039
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:04:[180_0,240_0]	400	0.988	0.003
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:04:[180_0,240_0]	400	0.96	0.016
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:05:[240_0,300_0]	400	0.998	0.001
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:05:[240_0,300_0]	400	0.998	0.003
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:06:[300_0,360_0]	400	0.917	0.015
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:06:[300_0,360_0]	400	0.991	0.008
StudentsAtRecruiters:IntervalAvg:PeakPeriod:[150_0,270_0]	400	30.891	0.809

Name	Count	Average	Half-Width
StudentsAtRecruiters:ValueAtStart:PeakPeriod:[150_0,270_0]	400	9.905	0.494
JHBuntR:InstantaneousUtil:IntervalAvg:PeakPeriod:[150_0,270_0]	400	0.992	0.002
JHBuntR:InstantaneousUtil:ValueAtStart:PeakPeriod:[150_0,270_0]	400	0.94	0.017
MalWartR:InstantaneousUtil:IntervalAvg:PeakPeriod:[150_0,270_0]	400	0.971	0.004
MalWartR:InstantaneousUtil:ValueAtStart:PeakPeriod:[150_0,270_0]	400	0.85	0.029
Mixer Ending Time	400	430.291	3.258
Prob(EndTime>360_0)	400	0.998	0.005
JHBuntR:SeizeCount	400	169.798	1.25

MalWartR:Seize-Count	400	169.798	1.25

To further investigate this situation, the same model was run but with the capacity of the resources set at infinity. This will cause no queueing to occur, but will indicate the number of busy resources by hour of the day. This can help in specifying the required staffing. In the following results, only the critical performance measures are presented.

Infinite Capacity Statistical Summary Report

Name	Count	Average	Half-Width
OverallSystem-Time	400	17.389	0.082
JHBuntR:Num-BusyUnits:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.458	0.029
JHBuntR:Num-BusyUnits:IntervalAvg:Hourly:02:[60_0,120_0]	400	1.456	0.051
JHBuntR:Num-BusyUnits:IntervalAvg:Hourly:03:[120_0,180_0]	400	3.693	0.08
JHBuntR:Num-BusyUnits:IntervalAvg:Hourly:04:[180_0,240_0]	400	5.115	0.092
JHBuntR:Num-BusyUnits:IntervalAvg:Hourly:05:[240_0,300_0]	400	4.973	0.095
JHBuntR:Num-BusyUnits:IntervalAvg:Hourly:06:[300_0,360_0]	400	1.62	0.064

Name	Count	Average	Half-Width
MalWartR:NumBusyUnits:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.252	0.015
MalWartR:NumBusyUnits:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.773	0.027
MalWartR:NumBusyUnits:IntervalAvg:Hourly:03:[120_0,180_0]	400	1.961	0.044
MalWartR:NumBusyUnits:IntervalAvg:Hourly:04:[180_0,240_0]	400	2.618	0.047
MalWartR:NumBusyUnits:IntervalAvg:Hourly:05:[240_0,300_0]	400	2.407	0.046
MalWartR:NumBusyUnits:IntervalAvg:Hourly:06:[300_0,360_0]	400	0.716	0.034
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:01:[0_0,60_0]	400	0	0
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:02:[60_0,120_0]	400	0	0
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:03:[120_0,180_0]	400	0	0

Name	Count	Average	Half-Width
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:04:[180_0,240_0]	400	0	0
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:05:[240_0,300_0]	400	0	0
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:06:[300_0,360_0]	400	0	0
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:01:[0_0,60_0]	400	0	0
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:02:[60_0,120_0]	400	0	0
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:03:[120_0,180_0]	400	0	0
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:04:[180_0,240_0]	400	0	0
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:05:[240_0,300_0]	400	0	0

Name	Count	Average	Half-Width
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:06:[300_0,360_0]	400	0	0
Mixer Ending Time	400	363.773	0.699
Prob(EndTime>360_0)	400	0.427	0.049

The results of running the non-stationary arrival schedule with infinite capacity for the recruiting resources indicate the effect on the resources. Notice how the estimated number busy units for the resources track the arrival pattern. Now considering the JHBunt recruiting station during the fourth hour of the day, we see an average number of recruiters busy of 5.128. Thus, if we set the resource capacity of the JHBunt recruiting station to 6 recruiters during the fourth hour of the day, we would expect to achieve a utilization of about 88%, ($5.128/6 = 0.8547$). With this thinking in mind we can determine the required capacity for each hour of the day and the expected utilization.

With these results in mind, we can develop a capacity schedule for this situation. The following code illustrates the creation of two schedules, one for controlling the JHBunt resource and one for controlling the MalMart resource. The capacity increases each hour until the last hour of the mixer.

```
    private val myJHBuntSchedule : CapacitySchedule =
     ↪ CapacitySchedule(this, 0.0)
    private val myMalWartSchedule : CapacitySchedule =
     ↪ CapacitySchedule(this, 0.0)
```

```
    init {

        myJHBuntSchedule.addItem(capacity = 1, duration =
↪   60.0)
        myJHBuntSchedule.addItem(capacity = 2, duration =
↪   60.0)
        myJHBuntSchedule.addItem(capacity = 4, duration =
↪   60.0)
        myJHBuntSchedule.addItem(capacity = 7, duration =
↪   60.0)
        myJHBuntSchedule.addItem(capacity = 7, duration =
↪   60.0)
        myJHBuntSchedule.addItem(capacity = 3, duration =
↪   60.0)
        myJHBuntRecruiters.useSchedule(myJHBuntSchedule,
↪   CapacityChangeRule.WAIT)

        myMalWartSchedule.addItem(capacity = 1, duration =
↪   60.0)
        myMalWartSchedule.addItem(capacity = 2, duration =
↪   60.0)
        myMalWartSchedule.addItem(capacity = 4, duration =
↪   60.0)
        myMalWartSchedule.addItem(capacity = 6, duration =
↪   60.0)
        myMalWartSchedule.addItem(capacity = 6, duration =
↪   60.0)
        myMalWartSchedule.addItem(capacity = 3, duration =
↪   60.0)
        myMalWartRecruiters.useSchedule(myMalWartSchedule,
↪   CapacityChangeRule.WAIT)

    }
```

A portion of the results are shown here. Notice that when using a capacity schedule that additional statistics on the time spent in the busy, idle, and inactive states are collected. The hourly responses for number busy and utilization are more realistic and the utilization during the peak is very much within a reasonable range of operation. However, the chance that the mixer goes over the planned time is still quite high. The reader is asked to suggest ways to address this situation within the exercises.

Scheduled Capacity Statistical Summary Report

Name	Count	Average	Half-Width
OverallSystem-Time	400	19.939	0.241
Recruitin-gOnlySystem-Time	400	13.179	0.25
P(Recruiting Only < 30 minutes)	400	0.958	0.005
MixingStu-dentSystemTime	400	30.054	0.24
MixingStudent-ThatLeavesSys-temTime	400	19.766	0.166
NumInSystem	400	9.696	0.155
NumInConversa-tionArea	400	3.558	0.042
NumAtJHBun-tAtClosing	400	1.137	0.173
NumAtMal-WartAtClosing	400	0.357	0.057
NumAtConver-sationAtClosing	400	0.678	0.081

Name	Count	Average	Half-Width
NumInSystem-AtClosing	400	2.315	0.206
JHBuntR:PTi-meInactive	400	0	0
JHBuntR:PTi-meIdle	400	0.224	0.005
JHBuntR:PTi-meBusy	400	0.776	0.005
JHBuntR:In-stantaneousUtil	400	0.621	0.007
JHBuntR:Nu-mActiveUnits	400	4.008	0.002
JHBuntR:Num-BusyUnits	400	2.773	0.028
JHBuntR:Sched-uledUtil	400	0.692	0.007
JHBuntR:WIP	400	3.974	0.127
JHBuntR:Q:Nu-mInQ	400	1.201	0.107
JH-BuntR:Q:TimeInQ	400	2.53	0.215
MalWartR:PTi-meInactive	400	0	0
MalWartR:PTi-meIdle	400	0.369	0.004
MalWartR:PTi-meBusy	400	0.631	0.004
MalWartR:In-stantaneousUtil	400	0.348	0.004
MalWartR:Nu-mActiveUnits	400	3.661	0.001

Name	Count	Average	Half-Width
MalWartR:NumBusyUnits	400	1.396	0.014
MalWartR:ScheduledUtil	400	0.381	0.004
MalWartR:WIP	400	1.46	0.017
MalWartR:Q:NumInQ	400	0.064	0.006
MalWartR:Q:TimeInQ	400	0.135	0.012
StudentsAtRecruiters	400	5.434	0.135
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.388	0.02
JHBuntR:NumBusyUnits:ValueAtStart:Hourly:01:[0_0,60_0]	400	0	0
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:02:[60_0,120_0]	400	1.339	0.037
JHBuntR:NumBusyUnits:ValueAtStart:Hourly:02:[60_0,120_0]	400	1.072	0.085
JHBuntR:NumBusyUnits:IntervalAvg:Hourly:03:[120_0,180_0]	400	3.395	0.049

Name	Count	Average	Half-Width
JHBuntR:Num-BusyUnits:Val-ueAt-Start:Hourly:03:[120_0,180_0]	400	2.983	0.122
JHBuntR:Num-BusyUnits:Inter-valAvg:Hourly:04:[180_0,240_0]	400	5.52	0.091
JHBuntR:Num-BusyUnits:Val-ueAt-Start:Hourly:04:[180_0,240_0]	400	5.92	0.171
JHBuntR:Num-BusyUnits:Inter-valAvg:Hourly:05:[240_0,300_0]	400	5.069	0.091
JHBuntR:Num-BusyUnits:Val-ueAt-Start:Hourly:05:[240_0,300_0]	400	5.303	0.177
JHBuntR:Num-BusyUnits:Inter-valAvg:Hourly:06:[300_0,360_0]	400	1.884	0.068
JHBuntR:Num-BusyUnits:Val-ueAt-Start:Hourly:06:[300_0,360_0]	400	3.555	0.182
MalWartR:Num-BusyUnits:Inter-valAvg:Hourly:01:[0_0,60_0]	400	0.241	0.013
MalWartR:Num-BusyUnits:Val-ueAt-Start:Hourly:01:[0_0,60_0]	400	0	0

Name	Count	Average	Half-Width
MalWartR:NumBusyUnits:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.776	0.027
MalWartR:NumBusyUnits:ValueAtStart:Hourly:02:[60_0,120_0]	400	0.603	0.074
MalWartR:NumBusyUnits:IntervalAvg:Hourly:03:[120_0,180_0]	400	1.979	0.043
MalWartR:NumBusyUnits:ValueAtStart:Hourly:03:[120_0,180_0]	400	1.48	0.122
MalWartR:NumBusyUnits:IntervalAvg:Hourly:04:[180_0,240_0]	400	2.626	0.048
MalWartR:NumBusyUnits:ValueAtStart:Hourly:04:[180_0,240_0]	400	2.67	0.166
MalWartR:NumBusyUnits:IntervalAvg:Hourly:05:[240_0,300_0]	400	2.403	0.046
MalWartR:NumBusyUnits:ValueAtStart:Hourly:05:[240_0,300_0]	400	2.692	0.144
MalWartR:NumBusyUnits:IntervalAvg:Hourly:06:[300_0,360_0]	400	0.848	0.041

Name	Count	Average	Half-Width
MalWartR:NumBusyUnits:ValueAtStart:Hourly:06:[300_0,360_0]	400	1.452	0.111
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:01:[0_0,60_0]	399	2.221	0.291
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:02:[60_0,120_0]	400	3.317	0.361
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:03:[120_0,180_0]	400	4.105	0.368
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:04:[180_0,240_0]	400	2.196	0.305
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:05:[240_0,300_0]	400	1.327	0.237
JHBuntR:Q:TimeInQ:IntervalAvg:Hourly:06:[300_0,360_0]	399	1.396	0.385
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:01:[0_0,60_0]	399	0.604	0.102

Name	Count	Average	Half-Width
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.377	0.048
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:03:[120_0,180_0]	400	0.195	0.032
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:04:[180_0,240_0]	400	0.034	0.009
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:05:[240_0,300_0]	400	0.022	0.007
MalWartR:Q:TimeInQ:IntervalAvg:Hourly:06:[300_0,360_0]	398	0.03	0.012
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.38	0.02
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:01:[0_0,60_0]	400	0	0
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.696	0.017

Name	Count	Average	Half-Width
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:02:[60_0,120_0]	400	0.536	0.042
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:03:[120_0,180_0]	400	0.852	0.012
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:03:[120_0,180_0]	400	0.746	0.031
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:04:[180_0,240_0]	400	0.789	0.013
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:04:[180_0,240_0]	400	0.846	0.024
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:05:[240_0,300_0]	400	0.72	0.013
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:05:[240_0,300_0]	400	0.757	0.025
JHBuntR:InstantaneousUtil:IntervalAvg:Hourly:06:[300_0,360_0]	400	0.566	0.019

Name	Count	Average	Half-Width
JHBuntR:InstantaneousUtil:ValueAtStart:Hourly:06:[300_0,360_0]	400	0.724	0.024
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:01:[0_0,60_0]	400	0.235	0.013
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:01:[0_0,60_0]	400	0	0
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:02:[60_0,120_0]	400	0.41	0.013
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:02:[60_0,120_0]	400	0.301	0.037
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:03:[120_0,180_0]	400	0.5	0.011
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:03:[120_0,180_0]	400	0.37	0.031
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:04:[180_0,240_0]	400	0.439	0.008

Name	Count	Average	Half-Width
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:04:[180_0,240_0]	400	0.445	0.028
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:05:[240_0,300_0]	400	0.396	0.008
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:05:[240_0,300_0]	400	0.449	0.024
MalWartR:InstantaneousUtil:IntervalAvg:Hourly:06:[300_0,360_0]	400	0.277	0.013
MalWartR:InstantaneousUtil:ValueAtStart:Hourly:06:[300_0,360_0]	400	0.452	0.031
StudentsAtRecruiters:IntervalAvg:PeakPeriod:[150_0,270_0]	400	9.985	0.3
StudentsAtRecruiters:ValueAtStart:PeakPeriod:[150_0,270_0]	400	7.843	0.416

Name	Count	Average	Half-Width
JHBuntR:InstantaneousUtil:IntervalAvg:PeakPeriod:[150_0,270_0]	400	0.818	0.009
JHBuntR:InstantaneousUtil:ValueAtStart:PeakPeriod:[150_0,270_0]	400	0.867	0.022
MalWartR:InstantaneousUtil:IntervalAvg:PeakPeriod:[150_0,270_0]	400	0.476	0.006
MalWartR:InstantaneousUtil:ValueAtStart:PeakPeriod:[150_0,270_0]	400	0.481	0.03
Mixer Ending Time	400	363.422	0.487
Prob(EndTime>360_0)	400	0.505	0.049
JHBuntR:SeizeCount	400	168.56	1.244
MalWartR:SeizeCount	400	168.56	1.244

Non-stationary arrival patterns are a fact of life in many systems that handle people. The natural daily processes of waking up, working, etc.

all contribute to processes that depend on time. This section illustrated how to model non-stationary arrival processes and illustrated some of the concepts necessary when developing staffing plans for such systems. Incorporating these modeling issues into your simulation models allows for more realistic analysis; however, this also necessitates much more complex statistical analysis of the input distributions and requires careful capture of meaningful statistics. Capturing statistics by time periods is especially important because the statistical results that do not account for the time varying nature of the performance measures may mask what may be actually happening within the model. The overall average across the entire simulation time horizon may be significantly different than what occurs during individual periods of time that correspond to non-stationary situations. A good design must reflect these variations due to time.

In the next section, we continue the exploration of advanced process modeling techniques by illustrating how to model the balking and reneging of entities within queues.

7.5. Examples of Advanced Event Models

This section will illustrate situations where combining event based modeling and process modeling is useful. In addition, we will exam how to simulate an inventory system using the event view.

7.5.1. Modeling Balking and Reneging

This situation has multiple types of customers with different priorities, different service times, and in the case of one type of customer, the desire (or ability) to renege from the system. You can find the completed model file for this section in the chapter files under the name, `WalkInHealthClinic.`

Example 7.8 (Walk-in Health Care Clinic)**.** A walk-in care clinic has analyzed their operations and they have found that they can classify their walk-in patients into three categories: high priority (urgent need of medical attention), medium priority (need standard medical attention), and low priority (non-urgent need of medical attention). On a typical day during the period of interest there are about 15 arrivals per hour with (25% being high priority, 60% being medium priority, and the remaining being low priority). The clinic is interested in understanding the waiting time for patients at the clinic. Upon arrival to the clinic the patients are triaged by a nurse into one of the three types of patients. This takes only 2-3 minutes uniformly distributed. Then, the patients wait in the waiting room based on their priority. Patients with higher priority are placed at the front of the line. Patients with the same priority are ordered based on a first-come first served basis. The service time distributions of the customers are given as follows.

Priority	Service Time Distribution (in minutes)
High	Lognormal($\mu = 38$, $\sigma = 8$)
Medium	Triangular(16, 22, 28)
Low	Lognormal($\mu = 12$, $\sigma = 2$)

The clinic has 4 doctors on staff to attend to the patients during the period of interest. They have found through a survey that if there are more than 10 people waiting for service an arriving low priority patient will exit before being triaged. Finally, they have found that the non-urgent (low priority) patients may depart if they have to wait longer than 15 ± 5 minutes after triage. That is, a non-urgent patient may enter the clinic and begin waiting for a doctor, but if they have to wait more than 15 ± 5 minutes (uniformly distributed) they will decide to renege and leave the clinic without getting service. The clinic would like to estimate the following:

1. the average system time of each type of patient
2. the probability that low priority patients balk

3. the probability that low priority patients renege

Solution to Example 7.8

For this problem, the system is the walk-in clinic, which includes doctors and a triage nurse who serve three types of patients. The system must know how the patients arrive (time between arrival distribution), how to determine the type of the patient, the triage time, the service time by type of patient, and the amount of time that a low priority patient is willing to wait. The following code defines the random variables needed for the modeling as well as the resources. Notice that this code explicitly defines the queue to hold the requests for the doctor. This is done to permit direct access to the reference for the queue during the implementation of reneging.

```
class WalkInHealthClinic(parent: ModelElement, name: String?
 ↪  = null) : ProcessModel(parent, name) {

    private val myTBArrivals: RVariableIfc =
     ↪  ExponentialRV(6.0, 1)

    private val triageRV = RandomVariable(this,
     ↪  UniformRV(2.0, 3.0, 2))
    private val highRV = RandomVariable(this,
     ↪  LognormalRV(38.0, 8.0 * 8.0, 3))
    private val mediumRV = RandomVariable(this,
     ↪  TriangularRV(16.0, 22.0, 28.0, 4))
    private val lowRV = RandomVariable(this,
     ↪  LognormalRV(12.0, 2.0 * 2.0, 5))

    private val renegeTimeRV = RandomVariable(this,
     ↪  UniformRV(10.0, 20.0, 6))
```

```
    private val doctorQ: RequestQ = RequestQ(this,
     ↪  "DoctorQ", discipline = Queue.Discipline.RANKED)
    private val doctorR: ResourceWithQ = ResourceWithQ(this,
     ↪  capacity = 5, queue = doctorQ, name = "Doctors")

    private val triageNurseR: ResourceWithQ =
     ↪  ResourceWithQ(this, capacity = 1, name =
     ↪  "TriageNurse")

    var balkCriteria = 10
        set(value) {
            require(value > 0) { "The balk criteria must be >
↪  0" }
            field = value
        }
```

In the following code fragment, we use a `DEmpiricalList` to hold the service distribution for the types of patients. In addition, a map is defined to model the relationship between the type of patient and the priority.

```
    // set up the service distribution and the random
     ↪  selection
    private val distributions = listOf(highRV, mediumRV,
     ↪  lowRV)
    private val distributionCDF = doubleArrayOf(0.25, 0.85,
     ↪  1.0)
    private val serviceRV = REmpiricalList(this,
     ↪  distributions, distributionCDF)
    private val typeMap = mapOf(highRV to 1, mediumRV to 2,
     ↪  lowRV to 3)
```

We will use these constructs within the patient processes. In the following

code, we use the `serviceRV` to randomly assign the service time distribution for the patient and then use the map to assign the patient's priority based on the service setting. The `clinicProcess` starts with a test to see if the low priority patient that arrives will balk if the doctor's waiting queue has too many people. If so, we collect statistics on the balking and exit the process.

```
    private inner class Patient : Entity() {
        private val service = serviceRV.element

        init {
            priority = typeMap[service]!!
        }

        val clinicProcess = process(isDefaultProcess = true)
         ↪ {
            if ((priority == 3) && (doctorQ.size >=
             ↪ balkCriteria)) {
                // record balking
                balkingProb.value = 1.0
                numBalked.increment()
                return@process
            }
            if (priority == 3){
                balkingProb.value = 0.0
            }
            use(triageNurseR, delayDuration = triageRV)
            // only low priority renege
            if (priority == 3) {
                schedule(this@Patient::renegeAction,
↪  renegeTimeRV, message = this@Patient)
            }
            val a = seize(doctorR)
            delay(service)
```

```
            release(a)
            if (priority == 3){
                renegingProb.value = 0.0
            }
            val st = time - this@Patient.createTime
            timeInSystem.value = st
            sysTimeMap[priority]?.value = st
            numServed.increment()
        }
```

Patients that get past the balking logic complete the rest of the process and thus do not balk. A response variable is used to collect this statistic. Then, the patient uses the triage nurse. After using the triage nurse, the low priority patient might need to renege. Thus, the low priority patient schedules the reneging action. All patients proceed to used the doctor. After using the doctor, if the patient is a low priority patient, then the patient must not have reneged and statistics are collected. Finally, statistics are collected upon departure from the process. Let's take a closer look at the reneging action logic.

```
        private fun renegeAction(event: KSLEvent<Patient>) {
            val request: ProcessModel.Entity.Request? =
             ↪ doctorQ.find { it.entity == event.message }
            if (request != null) {
                doctorQ.removeAndTerminate(request)
                // reneging statistics
                renegingProb.value = 1.0
                numReneged.increment()
            }
        }
```

The low priority patient schedules its possible reneging. One can think of this as the patient, setting an alarm clock (the event scheduled by the

entity) for when to renege. This must be done because once the low priority patient seizes the doctor it may be suspended and cannot proceed (by itself) to exit the queue after a period of time. Thus, an event is scheduled at the time of the reneging (essentially when the patient's patience runs out). The above code is the event action. Here, the doctor queue (`doctorQ`) is searched to find the same entity that scheduled the event. If that entity's request is found then the request is removed from the doctor queue and reneging statistics are collected. If a request is not found, there is nothing to do because the patient has already started seeing the doctor.

Notice that the removal of the entity from the queue uses the `removeAndTerminate()` method of the `RequestQ` class. Since the entity is suspended within the process and is being held in the queue, we must remove the entity's request from the queue and *terminate* its process. For more complex termination situations, the user can supply a function to be invoked after the process is terminated via a parameter of the `removeAndTerminate()` method. For example, such a function could allow for the sending of the removed entity to further processing.

The results from simulating the clinic for 30 days of operation are as follows.

Walk-in Clinic Statistical Summary Report

Name	Count	Average	Half-Width
DoctorQ:NumInQ	30	0.93	0.232
DoctorQ:TimeInQ	30	4.79	1.078
Doctors:InstantaneousUtil	30	0.779	0.026
Doctors:NumBusyUnits	30	3.897	0.129
Doctors:ScheduledUtil	30	0.779	0.026
Doctors:WIP	30	4.828	0.339
TriageNurse:InstantaneousUtil	30	0.421	0.016
TriageNurse:NumBusyUnits	30	0.421	0.016
TriageNurse:ScheduledUtil	30	0.421	0.016
TriageNurse:WIP	30	0.575	0.037

Name	Count	Average	Half-Width
TriageNurse:Q:NumInQ	30	0.154	0.022
TriageNurse:Q:TimeInQ	30	0.889	0.107
Walk-In Clinic:TimeInSystem	30	33.165	1.445
Walk-In Clinic:TimeInSystemHigh	30	43.77	0.851
Walk-In Clinic:TimeInSystemMedium	30	31.47	1.626
Walk-In Clinic:TimeInSystemLow	30	17.188	0.653
Walk-In Clinic:ProbBalking	30	0	0
Walk-In Clinic:ProbReneging	30	0.293	0.075
Doctors:SeizeCount	30	94.733	2.442
TriageNurse:SeizeCount	30	101.233	3.791
Walk-In Clinic:NumServed	30	90.633	2.273
Walk-In Clinic:NumBalked	30	0	0
Walk-In Clinic:NumReneged	30	4.367	1.325

Notice that a fairly high (28%) of the low priority patients renege before seeing the doctor. This may or may not be acceptable in light of the other performance measures for the system. The reader is asked to further explore this model in the exercises. While some analytical work has been done for queueing systems involving balking and reneging, simulation allows for the modeling of more realistic types of queueing situations as well as even more complicated systems. The next subsection will explore such a situation.

7.5.2. Modeling a Reorder Point, Reorder Quantity Inventory Policy

In an inventory system, there are units of an item (e.g. computer printers, etc.) for which customers make demands. If the item is available (in stock) then the customer can receive the item and depart. If the item is not on hand when a demand from a customer arrives then the customer may

depart without the item (i.e. lost sales) or the customer may be placed on a list for when the item becomes available (i.e. back ordered). In a sense, the item is like a resource that is consumed by the customer. Unlike the previous notions of a resource, inventory can be replenished. The proper control of the replenishment process is the key to providing adequate customer service. There are two basic questions that must be addressed when controlling the inventory replenishment process: 1) When to order? and 2) How much to order?. If the system does not order enough or does not order at the right time, the system not be able to fill customer demand in a timely manner. Figure 7.12 illustrates a simple inventory system.

There are a number of different ways to manage the replenishment process for an inventory item. These different methods are called *inventory control policies*. An inventory control policy must determine (at the very least) when to place a replenishment order and how much to order. This section examines the use of a reorder point (r), reorder quantity (Q) inventory policy. This is often denoted as an (r, Q) inventory policy. The modeling of a number of other inventory control policies will be explored as exercises. After developing a basic understanding of how to model an (r, Q) inventory system, the modeling can be expanded to study supply chains. A supply chain can be thought of as a network of locations that hold inventory in order to satisfy end customer demand.

The topic of inventory systems has been well studied. A full exposition of the topic of inventory systems is beyond the scope of this text, but the reader can consult a number of texts within the area, such as (Hadley and Whitin 1963), (Axsäter 2006), Silver et al. (1998), or (Zipkin 2000) for more details. The reader interested in supply chain modeling might consult (Nahmias 2001), (Askin and Goldberg 2002), (Chopra and Meindl 2007), or (Ballou 2004).

Within this section we will develop a model of a continuous review (r, Q) inventory system with back-ordering. In a continuous review (r, Q) inventory control system, demand arrives according to some stochastic process. When a demand (customer order) occurs, the amount of the demand is

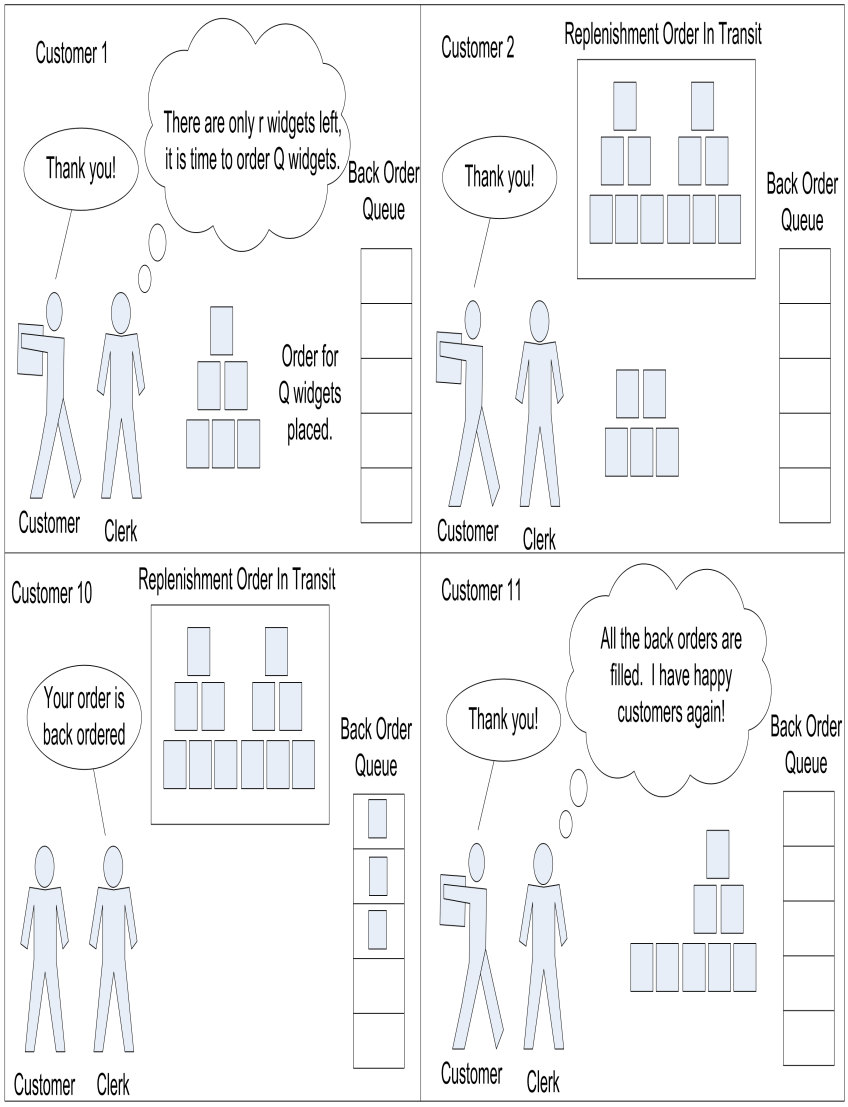


Figure 7.12.: A Simple Reorder Point, Reorder Quantity Inventory System

determined, and then the system checks for the availability of stock. If the stock on-hand is enough for the order, the demand is filled and the quantity on-hand is decreased. On the other hand, if the stock on-hand is not enough to fill the order, the entire order is back-ordered. The back-orders are accumulated in a queue and they will be filled after the arrival of a replenishment order. Assume for simplicity that the back-orders are filled on a first come first served basis. The inventory position (inventory on-hand plus on-order minus back-orders) is checked each time after a regular customer demand and the occurrence of a back-order. If the inventory position reaches or falls under the reorder point, a replenishment order is placed. The replenishment order will take a possibly random amount of time to arrive and fill any back-orders and increase the on-hand inventory. The time from when a replenishment order is placed until the time that it arrives to fill any back-orders is often called the lead time for the item.

There are three key state variables that are required to model this situation. Let $I(t)$, $IO(t)$, and $BO(t)$ be the amount of inventory on hand, on order, and back-ordered, respectively at time t. The net inventory, $IN(t) = I(t) - BO(t)$, represents the amount of inventory (positive or negative). Notice that if $I(t) > 0$, then $BO(t) = 0$, and that if $BO(t) > 0$, then $I(t) = 0$. These variables compose the inventory position, which is defined as:

$$IP(t) = I(t) + IO(t) - BO(t)$$

The inventory position represents both the current amount on hand, $I(t)$, the amounted back ordered, $BO(t)$, and the amount previously ordered, $IO(t)$. Thus, when placing an order, the inventory position can be used to determine whether or not a replenishment order needs to be placed. Since, $IO(t)$, is included in $IP(t)$, the system will only order, when outstanding orders are not enough to get $IP(t)$ above the reorder point.

In the continuous review (r, Q) policy, the inventory position must be checked against the reorder point as demands arrive to be filled. After

filling (or back-ordering) a demand, either $I(t)$ or $BO(t)$ will have changed (and thus $IP(t)$ will change). If $IP(t)$ changes, it must be checked against the reorder point. If $IP(t) \leq r$, then an order for the amount Q is placed.

The key performance measures for this type of system are the average amount of inventory on hand, the average amount of inventory back-ordered, the percentage of time that the system is out of stock, and the average number of orders made per unit time.

Let's discuss these performance measures before indicating how to collect them within a simulation. The average inventory on hand and the average amount of inventory back-ordered can be defined as follows:

$$\bar{I} = \frac{1}{T}\int_0^T I(t)\mathrm{d}t$$
$$\overline{BO} = \frac{1}{T}\int_0^T BO(t)\mathrm{d}t$$

As can be seen from the definitions, both $I(t)$ and $BO(t)$ are time-persistent variables and their averages are time averages. Under certain conditions as T goes to infinity, these time averages will converge to the steady state performance for the (r, Q) inventory model. The percentage of time that the system is out of stock can be defined based on $I(t)$ as follows:

$$SO(t) = \begin{cases} 1 & I(t) = 0 \\ 0 & I(t) > 0 \end{cases}$$

$$\overline{SO} = \frac{1}{T}\int_0^T SO(t)\mathrm{d}t$$

Thus, the variable $SO(t)$ indicates whether or not there is no stock on hand at any time. A time average value for this variable can also be defined and interpreted as the percentage of time that the system is out of stock. One

minus $\overline{SO}$ can be interpreted as the proportion of time that the system has stock on hand. Under certain conditions (but not always), this can also be interpreted as the fill rate of the system (i.e. the fraction of demands that can be filled immediately). Let Y_i be an indicator variable that indicates 1 if the i^{th} demand is immediately filled (without back ordering) and 0 if the i^{th} demand is back ordered upon arrival. Then, the fill rate is defined as follows.

$$\overline{FR} = \frac{1}{n} \sum_{i=1}^{n} Y_i$$

Thus, the fill rate is just the average number of demands that are directly satisfied from on hand inventory. The variables $\overline{SO}$ and $\overline{FR}$ are measures of customer service.

To understand the cost of operating the inventory policy, the average number of replenishment orders made per unit time or the *average order frequency* needs to be measured. Let $N(t)$ be the number of replenishment orders placed in $(0, t]$, then the average order frequency over the period $(0, t]$ can be defined as:

$$\overline{OF} = \frac{N(T)}{T}$$

Notice that the average order frequency is a rate (units/time).

In order to determine the best settings of the reorder point and reorder quantity, we need an objective function that trades-off the key performance measures within the system. This can be achieved by developing a total cost equation on a per time period basis. Let h be the holding cost for the item in terms of \$/unit/time. That is, for every unit of inventory held, we accrue h dollars per time period. Let b be the back order cost for the item in terms of \$/unit/time. That is, for every unit of inventory back ordered, we accrue b dollars per time period. finally, let k represent the cost in

dollars per order whenever an order is placed. The settings of the reorder point and reorder quantity depend on these cost factors. For example, if the cost per order is very high, then we should not want to order very often. However, this means that we would need to carry a lot of inventory to prevent a high chance of stocking out. If we carry more inventory, then the inventory holding cost is high.

The average total cost per time period can be formulated as follows:

$$\overline{TC} = k\overline{OF} + h\bar{I} + b\overline{BO}$$

A discussion of the technical issues and analytical results related to these variables can be found in Chapter 6 of (Zipkin 2000). Let's take a look at an example that illustrates an inventory situation and simulates it using the KSL.

Example 7.9 (Reorder Point, Reorder Quantity Inventory Model)**.** An inventory manager is interested in understanding the cost and service trade-offs related to the inventory management of computer printers. Suppose customer demand occurs according to a Poisson process at a rate of 3.6 units per month and the lead time is 0.5 months. The manager has estimated that the holding cost for the item is approximately \$0.25 per unit per month. In addition, when a back-order occurs, the estimate cost will be \$1.75 per unit per month. Every time that an order is placed, it costs approximately \$0.15 to prepare and process the order. The inventory manager has set the reorder point to 1 units and the reorder quantity to 2 units. Develop a simulation model that estimates the following quantities:

1. Average inventory on hand and back ordered
2. Average frequency at which orders are placed

3. Probability that an arriving customer does not have their demand immediately filled.

The examples associated with this chapter provide for a framework to model this kind of situation as well as to expand to model other inventory policies. We start the modeling by first defining an interface to represent something that can fill demand.

```
interface InventoryFillerIfc {
    /**
     * Represents an arrival of demand to be provided by the
 ↪  filler
     *
     * @param demand
     */
    fun fillInventory(demand: Int)
}
```

Then, we implement an abstract base class to represent different kinds of inventory situations. The class `Inventory` represents the key state variables of on-hand, on-order, and amount back ordered as time weighted response variables. The class requires the specification of the initial amount of inventory on-hand and a reference to an instance of an class that implements the `InventoryFillerIfc` interface. This instance is responsible for resupplying the inventory when it places a replenishment order. The initial amount on hand can also be changed through the `initialOnHand` property.

```
abstract class Inventory(
    parent: ModelElement,
```

```
    initialOnHand: Int = 1,
    replenisher: InventoryFillerIfc,
    name: String?
) : ModelElement(parent, name), InventoryFillerIfc {

    init {
        require(initialOnHand >= 0) { "The initial on-hand
↪   inventory must be >= 0" }
    }

    protected val myOnHand = TWResponse(this, initialValue =
↪    initialOnHand.toDouble(),
        name = "${this.name}:OnHand")

    val onHand: Int
        get() = myOnHand.value.toInt()

    val onHandResponse: TWResponseCIfc
        get() = myOnHand

    val probOfStockOnHandResponse = TWResponseFunction({ x ->
↪    (x > 0).toDouble() },
        myOnHand, name = "${this.name}:PTimeWithStockOnHand")

    @set:KSLControl(
        controlType = ControlType.INTEGER,
        lowerBound = 0.0
    )
    var initialOnHand: Int
        get() = myOnHand.initialValue.toInt()
        set(amount) {
            require(model.isNotRunning) { "The initial
↪   on-hand cannot be changed while the model is running."}
```

```
            require(amount >= 0) { "The initial amount on
↪  hand must be >= 0" }
            myOnHand.initialValue = amount.toDouble()
```

The base class also defines some additional responses and a queue to hold demands that must be back ordered because they cannot be immediately filled from stock on-hand. Then, it defines three abstract methods that must be implemented by sub-classes. There must be a way to define the initial policy parameters of the inventory control policy, a method for placing replenishment orders, and a method to check the inventory position.

```
    val probOfStockOnHandResponse = TWResponseFunction({ x ->
    ↪  (x > 0).toDouble() },
        myOnHand, name = "${this.name}:PTimeWithStockOnHand")

    @set:KSLControl(
        controlType = ControlType.INTEGER,
        lowerBound = 0.0
    )
    var initialOnHand: Int
        get() = myOnHand.initialValue.toInt()
        set(amount) {
            require(model.isNotRunning) { "The initial
↪  on-hand cannot be changed while the model is running."}
            require(amount >= 0) { "The initial amount on
↪  hand must be >= 0" }
            myOnHand.initialValue = amount.toDouble()
        }

    protected val myAmountBackOrdered = TWResponse(this,
    ↪  "${this.name}:AmountBackOrdered")
```

```
    val amountBackOrdered: Int
        get() = myAmountBackOrdered.value.toInt()

    val amountBackOrderedResponse: TWResponseCIfc
        get() = myAmountBackOrdered

    protected val myOnOrder = TWResponse(this,
     ↪  "${this.name}:OnOrder")

    val onOrder: Int
        get() = myOnOrder.value.toInt()

    val onOrderedResponse: TWResponseCIfc
        get() = myOnOrder

    protected val myNumReplenishmentOrders: Counter =
     ↪  Counter(this, "${this.name}:NumReplenishmentOrders")

    val numReplenishmentOrders: CounterCIfc
        get() = myNumReplenishmentOrders

    val inventoryPosition: Int
        get() = onHand + onOrder - amountBackOrdered

    protected val myBackOrderQ: Queue<Demand> = Queue(this,
     ↪  "${this.name}:BackOrderQ")

    val backOrderQTimeResponse: ResponseIfc
        get() = myBackOrderQ.timeInQ

    val backOrderNumInQResponse: TWResponseCIfc
        get() = myBackOrderQ.numInQ
```

```
    protected val myFirstFillRate = Response(this,
     ↪  "${this.name}:FillRate")

    val fillRateResponse: ResponseIfc
        get() = myFirstFillRate

    var replenishmentFiller: InventoryFillerIfc = replenisher

    inner class Demand(val originalAmount: Int = 1, var
     ↪  amountNeeded: Int) : QObject()

    abstract fun setInitialPolicyParameters(param:
     ↪  DoubleArray)

    abstract fun replenishmentArrival(orderAmount: Int)

    protected abstract fun checkInventoryPosition()
```

Notice that the `probOfStockOnHandResponse` property uses a `TWResponseFunction` to observe the on-hand response and to record whether stock is on hand. Its time average is the fraction of time that stock is on hand (the *ready rate*), which is reported as `PTimeWithStockOnHand`.

The base class defines what must happen to fill a demand, how to back order demand, and how to fill waiting back orders. The following code presents the default implementations. The fill demand method implements what happens when demand is fully filled. The back order demand method indicates what should be done for the amount that must be back ordered.

```
    protected open fun fillDemand(demand: Int) {
        myFirstFillRate.value = 1.0
        myOnHand.decrement(demand.toDouble())
    }
```

```
    protected open fun backOrderDemand(demand: Int) {
        myFirstFillRate.value = 0.0
        // determine amount to be back ordered
        val amtBO: Int = demand - onHand
        // determine the amount to give
        val amtFilled: Int = onHand
        // give all that can be given
        myOnHand.decrement(amtFilled.toDouble())
        myAmountBackOrdered.increment(amtBO.toDouble())
        // create a demand for the back order queue
        val d = Demand(demand, amtBO)
        myBackOrderQ.enqueue(d)
    }
```

The `fillBackOrders()` method can be called after a replenishment order comes in to fill requests that are waiting for inventory. In the default implementation, the amount to fill is determined and it is allocated to the waiting demand. If the demand is filled in full, it is removed from the queue; otherwise, its amount needed is reduced and it continues to wait.

```
    protected open fun fillBackOrders() {
        var amtToFill: Int = minOf(amountBackOrdered, onHand)
        myAmountBackOrdered.decrement(amtToFill.toDouble())
        myOnHand.decrement(amtToFill.toDouble())
        // now we have to give the amount to those waiting in
        ↪ the backlog queue
        // we assume filling is from first waiting until all
        ↪ of amtToFill is used
        while (myBackOrderQ.isNotEmpty) {
            val d = myBackOrderQ.peekNext()!!
            if (amtToFill >= d.amountNeeded) {
                amtToFill = amtToFill - d.amountNeeded
```

```
                d.amountNeeded = 0
                myBackOrderQ.removeNext()
            } else {
                d.amountNeeded = d.amountNeeded - amtToFill
                amtToFill = 0
            }
            if (amtToFill == 0) {
                break
            }
        }
    }

    protected open fun requestReplenishment(orderAmt: Int) {
        myOnOrder.increment(orderAmt.toDouble())
        myNumReplenishmentOrders.increment()
        replenishmentFiller.fillInventory(orderAmt)
```

Finally, the `requestReplenishment()` method increments the amount on order and calls the registered replenishment filler to fill the order. Thus, an instance of `Inventory` acts like a filler of inventory via its implementation of the `InventoryFillerIfc` interface and the fact that it uses an instance of such a class to meet its own replenishment requirements. Now, we are ready to explore the concrete implementation for the reorder point, reorder quantity inventory policy situation. This is implemented within the `RQInventory` class.

```
class RQInventory(
    parent: ModelElement,
    reorderPt: Int = 1,
    reorderQty: Int = 1,
    initialOnHand: Int = reorderPt + reorderQty,
    replenisher: InventoryFillerIfc,
    name: String?
```

```
) : Inventory(parent, initialOnHand, replenisher, name) {

    init {
        require(reorderQty > 0) { "The reorder quantity must
↪   be > 0" }
        require(reorderPt >= -reorderQty) { "reorder point
↪   ($reorderPt) must be >= - reorder quantity ($reorderQty)"
↪   }
    }

    private var myInitialReorderPt: Int = reorderPt
    private var myInitialReorderQty: Int = reorderQty
    private var myReorderPt = myInitialReorderPt
    private var myReorderQty = myInitialReorderQty

    @set:KSLControl(
        controlType = ControlType.INTEGER,
        lowerBound = 0.0
    )
    var initialReorderPoint: Int
        get() = myInitialReorderPt
        set(value) {
            setInitialPolicyParameters(value,
↪   myInitialReorderQty)
        }

    @set:KSLControl(
        controlType = ControlType.INTEGER,
        lowerBound = 1.0
    )
    var initialReorderQty: Int
        get() = myInitialReorderQty
        set(value) {
```

```
            setInitialPolicyParameters(myInitialReorderPt,
↪   value)
        }
```

The class also defines properties for the cost per order and the unit holding and back order costs (each 1.0 by default), along with responses that collect the ordering frequency and the components of the average total cost, $\overline{TC}$.

```
    private val myOrderingFrequency = Response(this,
     ↪  "${this.name}:OrderingFrequency")

    val orderingFrequency: ResponseCIfc
        get() = myOrderingFrequency

    @set:KSLControl(
        controlType = ControlType.DOUBLE,
        lowerBound = 0.0,
        comment = "$/order"
    )
    var costPerOrder: Double = 1.0
        set(value) {
            require(value >= 0.0) { "The ordering cost must
↪   be >= 0.0" }
            field = value
        }

    @set:KSLControl(
        controlType = ControlType.DOUBLE,
        lowerBound = 0.0,
        comment = "$/unit/time"
    )
    var unitHoldingCost: Double = 1.0
```

```
        set(value) {
            require(value >= 0.0) { "The holding cost must be
↪   >= 0.0" }
            field = value
        }

    @set:KSLControl(
        controlType = ControlType.DOUBLE,
        lowerBound = 0.0,
        comment = "$/unit/time"
    )
    var unitBackOrderCost: Double = 1.0
        set(value) {
            require(value >= 0.0) { "The backorder cost must
↪   be >= 0.0" }
            field = value
        }

    private val myTotalCost = Response(this,
     ↪  "${this.name}:TotalCost")
    val totalCost: ResponseCIfc
        get() = myTotalCost

    private val myOrderingCost = Response(this,
     ↪  "${this.name}:OrderingCost")
    val orderingCost: ResponseCIfc
        get() = myOrderingCost

    private val myHoldingCost = Response(this,
     ↪  "${this.name}:HoldingCost")
    val holdingCost: ResponseCIfc
        get() = myHoldingCost
```

```
    private val myBackorderCost = Response(this,
     ↪  "${this.name}:BackorderCost")
    val backOrderCost: ResponseCIfc
        get() = myBackorderCost

    private val myOrderingAndHoldingCost = Response(this,
     ↪  "${this.name}:OrderingAndHoldingCost")
    val orderingAndHoldingCost: ResponseCIfc
        get() = myOrderingAndHoldingCost
```

The policy parameters can also be set through the `setInitialPolicyParameters()` functions.

```
    fun setInitialPolicyParameters(reorderPt: Int,
     ↪  reorderQty: Int) {
        require(model.isNotRunning) { "The initial policy
↪  parameters cannot be changed while the model is
↪  running."}
        require(reorderQty > 0) { "reorder quantity must be >
↪  0" }
        require(reorderPt >= -reorderQty) { "reorder point
↪  must be >= - reorder quantity" }
        myInitialReorderPt = reorderPt
        myInitialReorderQty = reorderQty
    }

    override fun setInitialPolicyParameters(param:
     ↪  DoubleArray) {
        require(param.size == 2) { "There must be 2
↪  parameters" }
        setInitialPolicyParameters(param[0].toInt(),
↪  param[1].toInt())
```

Notice that the constructor for the class requires the reorder point and reorder quantity parameters and provides for the setting of the parameters. The class is a subclass of `Inventory`, which is a subclass of `ModelElement`. We need to have an `initialize()` method. Notice that the initialization resets the reorder point and reorder quantity and checks the inventory position. The check of the inventory position is required because the inventory on-hand at the start of the replication might trigger the need for a replenishment at time 0.0.

```
    override fun initialize() {
        super.initialize()
        myReorderPt = myInitialReorderPt
        myReorderQty = myInitialReorderQty
        checkInventoryPosition()
    }
```

To fill the inventory, we simply need to check if the amount on hand is sufficient, if so, the amount is filled, if not the amount to back order is determined. Because the filling of demand or the back ordering of demand may cause the change of the inventory position, the inventory position must be checked to see if a replenishment is required.

```
    override fun fillInventory(demand: Int) {
        require(demand > 0) { "arriving demand must be > 0" }
        if (onHand >= demand) { // fully filled
            fillDemand(demand)
        } else { // some is back ordered
            backOrderDemand(demand)
        }
        checkInventoryPosition()
    }
```

The check of the inventory position for a (r, q) inventory policy checks to see if the inventory position has fallen to or below the reorder point, r,

and if so, an order must be placed. Because the amount below the reorder point may be large due to random demand, batch multiples of the reorder quantity may be required in order to get above the reorder point. Let IP represent the inventory position. The number of batches ordered is the fewest that lifts the inventory position strictly above the reorder point, $\lfloor (r - IP)/Q \rfloor + 1$.

```
    override fun checkInventoryPosition() {
        if (inventoryPosition <= myReorderPt) {
            // Order the fewest whole batches that lift the
             ↪  position strictly above r:
            // floor((r - IP)/Q) + 1. The difference is
             ↪  non-negative here, so integer division is
            // the floor. A ceiling instead leaves the
             ↪  position exactly at r whenever r - IP is a
            // multiple of Q, one batch short.
            val n = (myReorderPt - inventoryPosition) /
             ↪  myReorderQty + 1
            requestReplenishment(n * myReorderQty)
```

The `requestReplenishment` method is used to place the order. When the replenishment order arrives, we must increase the amount on-hand, decrease the amount ordered, and process any back ordered demand.

```
    override fun replenishmentArrival(orderAmount: Int) {
        myOnOrder.decrement(orderAmount.toDouble())
        myOnHand.increment(orderAmount.toDouble())
        // need to fill any back orders
        if (amountBackOrdered > 0) { // back orders to fill
            fillBackOrders()
        }
        checkInventoryPosition()
```

At the end of each replication, the components of the average total cost are computed from the time averages of the on-hand and back ordered responses and from the ordering frequency, following the equation for $\overline{TC}$.

```
    override fun replicationEnded() {
        myHoldingCost.value = unitHoldingCost *
↪   myOnHand.withinReplicationStatistic.weightedAverage
        myBackorderCost.value = unitBackOrderCost *
↪   myAmountBackOrdered.withinReplicationStatistic.
↪   weightedAverage
        myOrderingFrequency.value =
↪   myNumReplenishmentOrders.value/(time -
↪   myNumReplenishmentOrders.timeOfWarmUp)
        myOrderingCost.value = costPerOrder *
↪   myOrderingFrequency.value
        myOrderingAndHoldingCost.value = myOrderingCost.value
↪   + myHoldingCost.value
        myTotalCost.value = myOrderingCost.value +
↪   myHoldingCost.value + myBackorderCost.value
```

Now we are ready to model the inventory situation described in the problem. In the following code, we encapsulate the problem within a class called `RQInventorySystem`. The key aspects of this model are the use of an event generator to generate the demand and the use of an inner class to provide a warehouse to fill the replenishment orders.

```
class RQInventorySystem(
    parent: ModelElement,
    reorderPt: Int = 1,
    reorderQty: Int = 1,
    name: String? = null
) : ModelElement(parent, name) {
```

```
private var demandAmountRV = RandomVariable(
    this, ConstantRV(1.0),
    name = "${this.name}:DemandAmountRV"
)

val demandAmount: RandomVariableCIfc
    get() = demandAmountRV

private var leadTimeRV = RandomVariable(
    this, ConstantRV(10.0),
    name = "${this.name}:LeadTimeRV"
)

var unitHoldingCost: Double
    get() = inventory.unitHoldingCost
    set(value) {
        inventory.unitHoldingCost = value
    }

var unitBackorderCost: Double
    get() = inventory.unitBackOrderCost
    set(value) {
        inventory.unitBackOrderCost = value
    }

var costPerOrder: Double
    get() = inventory.costPerOrder
    set(value) {
        inventory.costPerOrder = value
    }

val leadTime: RandomVariableCIfc
```

```
        get() = leadTimeRV

    private var timeBetweenDemandRV = ExponentialRV(365.0 /
     ↪  14.0, 1)

    private val myDemandGenerator = EventGenerator(this,
     ↪  this::sendDemand,
        timeBetweenDemandRV, timeBetweenDemandRV)
    val demandGenerator: EventGeneratorRVCIfc
        get() = myDemandGenerator

    private val inventory: RQInventory = RQInventory(
        this, reorderPt, reorderQty, replenisher =
         ↪  Warehouse(), name = "${this.name}:Item"
    )

    var initialOnHand: Int
        get() = inventory.initialOnHand
        set(amount) {
            require(amount >= 0) { "The initial amount on
↪   hand must be >= 0" }
            inventory.initialOnHand = amount
        }

    var initialReorderPoint: Int
        get() = inventory.initialReorderPoint
        set(value) {
            require(model.isNotRunning) { "The initial
↪   reorder point must be set before the simulation starts"}
            inventory.initialReorderPoint = value
        }

    var initialReorderQty: Int
```

```
        get() = inventory.initialReorderQty
        set(value) {
            inventory.initialReorderQty = value
        }

    fun setInitialPolicyParameters(reorderPt: Int,
     ↪  reorderQty: Int) {
        inventory.setInitialPolicyParameters(reorderPt,
↪   reorderQty)
    }

    private fun sendDemand(generator: EventGeneratorIfc) {
        inventory.fillInventory(demandAmountRV.value.toInt())
    }

    inner class Warehouse : InventoryFillerIfc {
        override fun fillInventory(demand: Int) {
            schedule(this::endLeadTimeAction, leadTimeRV,
↪   message = demand)
        }

        private fun endLeadTimeAction(event: KSLEvent<Int>) {
            inventory.replenishmentArrival(event.message!!)
```

The implementation of the warehouse simply schedules a delay to represent the lead time and then calls the inventory with the amount of the replenishment after the lead time interval. In this implementation, the amount of each demand is governed by the `demandAmount` random variable, which is a constant 1 by default. Thus, we assume that the demand occurs in units of one. The system class also passes the initial amount on hand and the three costs through to the inventory as properties. The exercises will ask the reader to generalize on this situation. Finally, we create an instance that represents the problem and perform the simulation.

```
fun main() {
    val m = Model()
    val reorderPoint = 1
    val reorderQty = 2
    val rqModel = RQInventorySystem(m, reorderPoint,
     ↪  reorderQty, "RQ Inventory Model")
    rqModel.initialOnHand = 0
    rqModel.demandGenerator.setInitialEventTimeProcesses(
 ↪  ExponentialRV(1.0 / 3.6, streamNum = 2))
    rqModel.leadTime.initialRandomSource = ConstantRV(0.5)
    rqModel.costPerOrder = 0.15
    rqModel.unitHoldingCost = 0.25
    rqModel.unitBackorderCost = 1.75

    m.lengthOfReplication = 110000.0
    m.lengthOfReplicationWarmUp = 10000.0
    m.numberOfReplications = 40
    m.simulate()
    m.print()
    val r = m.simulationReporter
    val out = m.outputDirectory.createPrintWriter("R-Q
     ↪  Inventory Results.md")
    r.writeHalfWidthSummaryReportAsMarkDown(out, df =
 ↪  MarkDown.D3FORMAT)
}
```

The results of the simulation can be checked against the analytical results for this situation. With unit Poisson demand, the inventory position of an (r, Q) policy is uniformly distributed on $\{r+1, \ldots, r+Q\}$, which is $\{2, 3\}$ for this example. Let D represent the demand during the lead time, which is Poisson with mean $3.6 \times 0.5 = 1.8$. A demand is filled immediately if the inventory position exceeds the lead time demand. Thus, the fill rate is the average of $P(D \leq 1)$ and $P(D \leq 2)$, which is $(0.4628 + 0.7306)/2 = 0.5967$.

The simulated fill rate of 0.597 agrees. Because the demand process is Poisson, an arriving customer sees the time average state of the system. Thus, the fraction of time with stock on hand (0.597) also equals the fill rate. The cost results reflect the example's costs: an ordering cost of $0.15 \times 1.8 = 0.27$, a holding cost of $0.25 \times 0.993 \approx 0.248$, and a back order cost of $1.75 \times 0.294 \approx 0.514$, for a total cost of about 1.032 per month.

Inventory Model Statistical Summary Report

Name	Count	Average	Half-Width
RQ Inventory Model:Item:OnHand	40	0.993	0.001
RQ Inventory Model:Item:PTimeWithStockOnHand	40	0.597	0
RQ Inventory Model:Item:AmountBackOrdered	40	0.294	0
RQ Inventory Model:Item:OnOrder	40	1.8	0.001
RQ Inventory Model:Item:BackOrderQ:NumInQ	40	0.294	0
RQ Inventory Model:Item:BackOrderQ:TimeInQ	40	0.202	0
RQ Inventory Model:Item:FillRate	40	0.597	0
RQ Inventory Model:Item:OrderingFrequency	40	1.8	0.001
RQ Inventory Model:Item:TotalCost	40	1.032	0.001
RQ Inventory Model:Item:OrderingCost	40	0.27	0
RQ Inventory Model:Item:HoldingCost	40	0.248	0
RQ Inventory Model:Item:BackorderCost	40	0.514	0.001
RQ Inventory Model:Item:OrderingAndHoldingCost	40	0.518	0
RQ Inventory Model:Item:NumReplenishmentOrders	40	180032.15	97.616

The answer to the third question of Example 7.9 is the probability that an arriving customer does not have their demand immediately filled. That is, $1 - 0.597 = 0.403$. Thus, about 40 percent of the customers must wait for their demand to be filled, which suggests that the manager should consider a different policy.

This example should give you a solid basis for developing more sophisticated inventory models. While analytical results are available for this example,

small changes in the assumptions necessitate the need for simulation. For example, what if the lead times are stochastic or the demand is not in units of 1. In the latter case, the filling of the back-order queue should be done in different ways. For example, suppose customers wanting 5 and 3 items respectively were waiting in the queue. Now suppose a replenishment of 4 items comes in. Do you give 4 units of the item to the customer wanting 5 units (partial fill) or do you choose the customer wanting 3 units and fill their entire back-order and give only 1 unit to the customer wanting 5 units. The more realism needed, the less analytical models can be applied, and the more simulation becomes useful.

7.6. Applying Queueing Theory Results to Verify and Validate a Simulation

In this section, we overview some verification and validation methods to apply when developing and testing a simulation model.

Verification is about checking if the model meets the desired output. In other words, does the model work correctly as designed? In verifying a model, you select a sample of test cases with known results and show that the model produces the expected output across the test cases. It is important to select the test cases in an unbiased manner and that the test cases encompass a large portion of the space that will be relevant when using the model. The test cases form the standard that the model must meet and thus are critical in convincing others that the verification process was of high quality.

Validation is whether or not the model meets it intended purpose. When validating a model, we are trying to build credibility that the model can be used as intended. In other words, that the model adequately represents the system for its intended purpose. Obviously, verification is a necessary but not sufficient step in validation. Validation is predicated on how the model is intended to be used and what it is supposed to represent.

In the following, we will illustrate a basic approach to verifying and validating the results of a simulation model by using some analytical approximations based on queueing theory and common sense. Let's start with an example system to be simulated.

Example 7.10 (Small Manufacturing System)**.** A small manufacturing system produces parts. The parts arrive from an upstream Poisson process with a rate of arrival of 1 part every 5 minutes. All parts that enter the system must go through a preparation station where there are 2 preparation workers. A part requires only 1 of the 2 workers during preparation. The preparation time is exponentially distributed with means of 8 minutes. If all the workers at the preparation station are busy, then the part waits for the next available preparation worker.

After preparation, the parts are processed on two different production lines. There is a 30% chance that the parts are built on line 1 and a 70% chance that they go to line 2. Line 1 has a build station staffed by 2 workers. Line 2 has a build station staffed by 3 workers. The time to build a part on line 1 is triangularly distributed with a (min = 2, mode = 6, max = 8) minutes. The time to build a part on line 2 is triangularly distributed with a (min = 3, mode = 6, max = 7) minutes. The build operation on the part only requires 1 of the workers available at the station. If all the workers at the station are busy, then the part waits for the next available worker.

After the parts are built, they go to a packaging station. The packaging station is staffed by 2 workers. Only 1 of the 2 workers is needed during packaging. If all the workers at the packaging station are busy, then the part waits for the next available packaging worker. The time to individually wrap a part is exponential with a mean of 2 minutes. After each individual part is wrapped, the worker fills a box with packing peanuts, and places the part into a box for shipping. The time to fill the box with peanuts and seal the box is uniformly distribution between 1 and 2 minutes. After the

packaging, the box is placed on a pallet building machine. Once 10 boxes accumulate, the pallet machine wraps the pallet. The wrapping time takes 2 minutes. After the wrapping is completed, the pallet leaves the system.

The production team believes that a batch run of production over 30 days of operation is the best way to produce these parts. Thus, at the beginning of a month there are currently no parts in production. Simulate the system for 100 replications of one month (30 days) of operation. The base time unit for the simulation should be in minutes. We want to simulate this system in order to measure the following performance metrics with some high degree of confidence.

Performance Measures
System time for parts regardless of which build line prior to palletizer
Prob. that the system time before palletizing of a part is less than 60 minutes
Utilization of preparation workers
Utilization of packaging workers
Utilization of line 1 build workers
Utilization of line 2 build workers
Utilization of the palletizer
Total number of parts produced in 30 days
Total number of pallets completed in 30 days

Suppose you build a simulation model for this situation and get the following results. The building of the model is left as an exercise for the reader.

Performance Measures	**Average**	**Half-wi**
System time for parts regardless of which build line prior to palletizer	31.73	0.44
Prob. that the system time before palletizing of a part is less than 60 minutes	0.89	0.01
Utilization of preparation workers	0.80	0.00

Performance Measures	Average	Half-wi
Utilization of packaging workers	0.35	0.00
Utilization of line 1 build workers	0.16	0.00
Utilization of line 2 build workers	0.25	0.00
Utilization of the palletizer	0.039	0.00
Total number of parts produced in 30 days	8623.36	17.75
Total number of pallets completed in 30 days	861.85	1.77

How do you know if these results seem reasonable? You can use basic queueing analysis and some simple math to help verify and validate your results.

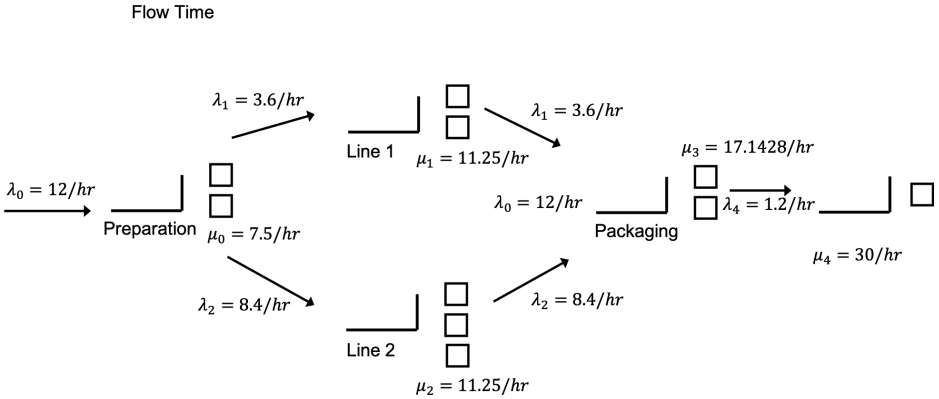


Figure 7.13.: Small Manufacturing System with Flow Rates

By examining Figure 7.13 and using a little flow analysis, we can get a pretty good approximation of the results. In the figure, we see each station (preparation, line 1, line 2, packaging, and the palletizer). The flow rate of the parts to each of the stations has also been noted. In what follows each of the stations will be analyzed (individually) and then the overall system will be analyzed using some basic queueing analysis using the results of Appendix C.

The approach is based on determining the rate of flow between the stations and using simple queueing formulas. Let's start with the preparation station.

7.6.1. Analyzing the Preparation Station

Let λ_0 be the arrival rate to the preparation station. Let μ_0 be the service rate at the preparation station. This is the rate at which an individual preparation worker works. Let MTBA_0 be the mean time between arrivals for the preparation station and let $E[\text{ST}_0]$ be the mean service time at the preparation station. Clearly from the problem description, we have:

$$\lambda_0 = \frac{1}{\text{MTBA}_0} = \frac{1}{5 \text{ min}} \times \frac{60 \text{ min}}{1 \text{ hour}} = 12/\text{hour}$$

$$\mu_0 = \frac{1}{E[\text{ST}_0]} = \frac{1}{8 \text{ min}} \times \frac{60 \text{ min}}{1 \text{ hour}} = 7.5/\text{hour}$$

Based on queueing theory, we have that the utilization, ρ, is:

$$\rho = \frac{\lambda}{c * \mu} = \frac{E[\text{ST}]}{c * \text{MTBA}}$$

where c is the number of servers at the station. For the preparation station, we have:

$$\rho_0 = \frac{\lambda_0}{c\mu_0} = \frac{12}{2 \times 7.5} = 0.8$$

From the simulation output results, we see that the utilization of the simulation is exactly that as predicted from the basic flow analysis. Let's examine the build lines.

7.6.2. Analyzing the Build Lines

In this case, we need to determine the arrival rate to each of the lines. First, we assume that the rate of arrivals out of the preparation station will be the same as the rate of arrivals into the preparation station. Since there is a probabilistic split of the flow to the lines, we can split the rate of parts flowing to line 1 and line 2 based on the probability of selecting the lines. Thus, we have that each line experiences the input rates as follows.

- Let λ_1 be the arrival rate to the line 1 station and let λ_2 be the arrival rate to the line 2 station.
- Let p_1 be the probability that line 1 is selected and let p_2 be the probability that line 2 is selected.

Thus, $\lambda_1 = p_1 \times \lambda_0 = 0.3 \times 12 = 3.6$ per hour and $\lambda_2 = p_2 \times \lambda_0 = 0.7 \times 12 = 8.4$ per hour. Now, we need to get the mean service time for the build lines. Because the service time distribution is triangular, we have that the mean of the triangular random variable, X, is:

$$E[X] = \frac{(\min + \text{ mode} + \max)}{3}$$

Therefore, we can compute the service rate for each of the lines, where ST_1 and ST_2 represent the service time random variable for line 1 and 2, respectively, as follows:

$$\mu_1 = \frac{1}{E[\text{ST}_1]} = \frac{1}{\frac{(2+6+8)}{3}} = \frac{1}{5.3\overline{3}\ \text{min}} \times \frac{60\ \text{min}}{1\ \text{hour}} = 11.25/\text{hour}$$

$$\mu_2 = \frac{1}{E[\text{ST}_2]} = \frac{1}{\frac{(3+6+7)}{3}} = \frac{1}{5.3\overline{3}\ \text{min}} \times \frac{60\ \text{min}}{1\ \text{hour}} = 11.25/\text{hour}$$

And, then we can compute the utilization for each of the lines.

$$\rho_1 = \frac{\lambda_1}{c\mu_1} = \frac{3.6}{2 \times 11.25} = 0.16$$

$$\rho_2 = \frac{\lambda_2}{c\mu_2} = \frac{8.4}{3 \times 11.25} = 0.24\overline{8}$$

Again, these results, match very closely the results from the simulation model. We can also approximate the results for packaging in the same manner.

7.6.3. Analyzing the Packaging Station

Again, we need to compute the incoming flow rate to the packaging station in order to compute the utilization at the station. Let λ_3 be the arrival rate to the packaging station. If we assume that everything that arrives to each of the two build lines eventually gets to packaging, we can see that $\lambda_3 = \lambda_1 + \lambda_2 = \lambda_0 = 12$ per hour.

The mean service time at the packaging station is the sum of the mean wrapping process time and the mean box filling time. The wrapping process has an exponential distribution with a mean of 2 minutes while the filling process has a continuous uniform distribution on the range of 1 to 2 minutes. Thus, using the expected values for these respective distributions, we have:

$$E\left[ST_w\right] = 2 \text{ min}$$

$$E\left[ST_f\right] = 1.5 \text{ min}$$

Let ST_3 be the service time at the packaging station. Because the time in service at the packaging station include both processing times, we have a utilization calculation as follows:

$$E\left[ST_3\right] = \ E\left[ST_w\right] + E\left[ST_f\right] = 3.5 \text{ min}$$

$$\mu_3 = \frac{1}{E\left[\text{ST}_3\right]} = \frac{1}{3.5 \text{ min}} \times \frac{60 \text{ min}}{1 \text{ hour}} = 17.1428 \text{ per hour}$$

$$\rho_3 = \frac{\lambda_3}{c\mu_3} = \frac{12}{2 \times 17.1428} = 0.35$$

Again, the results match very closely to the simulation results. With some additional thinking, we can even approximate the utilization of the palletizer.

7.6.4. Analyzing the Palletizing Station

Assuming that all parts that arrive to packaging also arrive to the palletizer, we have a rate of arrivals to the palletizer of 12 per hour. However, we have to wait until there are 10 parts before they are sent to the palletizer. The rate of 12 per hour is a mean time between arrivals of 1 part every 5 minutes. To get 10 parts, it will take on average 50 minutes. Thus, the mean time between arrivals to the palletizer is 50 minutes. Let λ_4 be the arrival rate to the palletizer. Thus, the arrival rate to the palletizer is

$$\lambda_4 = \frac{1}{\text{MTBA}_4} = \frac{1}{50 \text{ min}}$$

Thus, the utilization for the palletizer should be:

$$\rho_4 = \frac{\lambda_4}{c\mu_4} = \frac{E[ST_4]}{c * \text{MTBA}_4} = \frac{2}{1 \times 50} = 0.04$$

Again, this matches quite well with the simulation results. In general, as long as $\lambda < c\mu$ the approximation of utilization will be quite accurate.

7.6.5. Analyzing the Total System Time

We can go even further and approximate the total time spent in the system. If there is no waiting in any of the queues, the approximation is quite simple. We just need to total the time spent in service for the parts. Let $E[ST]$ be the total time spent in service. To compute this value, we can simply add up all of the mean service times for the stations visited. In the case of line 1 and line 2, we can take the weighted average based on the probability that the line is selected as follows:

$$E\left[\mathrm{ST}\right] = E\left[\mathrm{ST}_0\right] + p_1 E\left[\mathrm{ST}_1\right] + p_2 E\left[\mathrm{ST}_2\right] + E\left[\mathrm{ST}_3\right] + E[ST_4]$$

$$E\left[\mathrm{ST}\right] = 8 + 0.3 \times 5.3\overline{3} + 0.7 \times 5.3\overline{3} + 3.5 + 2.0 = 18.8\overline{3} \text{ minutes}$$

This can be easily tested within the simulation model by setting all the resource capacities to "infinite". If you do this, the results will indicate that the expected time in the system is 18.8951 ± 0.06 with 95% confidence. Clearly, these results are close enough to provide even more credibility to the simulation results.

By using additional queueing theory, we can approximate total time spent at each station by approximating the time spent within the queues and then adding up the total time. This will require additional approximations. For a random variable, X, the squared coefficient of variation is given by:

$$c_X^2 = \frac{\mathrm{Var}(X)}{\left(E\left[X\right]\right)^2}$$

Whitt (1983) suggests the following approximation for the queueing time in a GI/G/c queue, where GI is some general independent inter-arrival distribution, G is some general service distribution and c is the number of servers. The approximation has the following form:

$$W_q(\text{GI}/G/c) \cong \left(\frac{c_a^2 + c_s^2}{2}\right) W_q(M/M/c)$$

Where c_a^2 and c_s^2 represent the squared coefficient of variation of the inter-arrival time distribution and the service time distribution respectively and $W_q(M/M/c)$ is the expected waiting time in a M/M/c queue.

The main difficulty with applying this approximation is determining the variance of the inter-arrival time distribution. In what follows, we will simplify the approach even further and assume that the inter-arrival process at each station is a Poisson process. Thus, the inter-arrival distribution will be assumed to be exponential. Since the variance of an exponential distribution is the square of its mean, the squared coefficient of variation for all the arrival processes will be $c_a^2 = 1$. We will just need to compute the squared coefficient of variation for each of the service time distributions associated with each station. To determine the waiting time in queue, we will need the waiting time in queue equations for the M/M/c queueing model. The following table summarizes the formulas for the cases of 1, 2, and 3 servers.

Table 7.11.: Formulas for Expected Waiting Time for M/M/c Queue $\rho = \lambda/c\mu$

c = # servers	W_q =Expected Waiting time in Queue
1	$\frac{\rho^2}{\lambda(1-\rho)}$
2	$\frac{2\rho^3}{\lambda(1-\rho^2)}$
3	$\frac{9\rho^4}{\lambda(2+2\rho-\rho^2-3\rho^3)}$

We can now tabulate the waiting time in the queue for each of the stations. By using the variance associated with each service time distribution, as well as the mean, we can compute the squared coefficient of variation for the

service time distribution for each station. The following table summarizes the results. The calculations are left to the reader.

Table 7.12.: Summary Queueing Results for Each Station

Station	c	$\rho = \frac{\lambda}{c\mu}$	$W_q(M/M/c)$	c_a^2	c_s^2	$\frac{(c_a^2+c_s^2)}{2}$	$W_q(\mathrm{GI}/G/c)$
Preparation	2	0.8	$14.\overline{22}$	1	1	1	$14.\overline{22}$
Line 1 Build	2	0.16	0.0420	1	0.0547	0.5274	0.0221
Line 2 Build	3	$0.24\overline{8}$	0.0722	1	0.0254	0.5127	0.0370
Packaging	2	0.35	0.4886	1	0.3333	0.6666	0.3259
Palletizing	1	0.04	0.0722	1	0	0.5	0.0361

From these results, we can tabulate an approximation for the total time in the system. Let $E[TW_q]$ be the total expected waiting time of a part and $E[T]$ be the expected total time spent in the system for a part. Clearly, we have that, $E[T] = E[TW_q] + E[\mathrm{ST}]$, where $E[ST]$ is the total time spent in service for a part. Using the results of the table and taking a weighted average across the waiting time for the two build lines, we can see that the total expected waiting time in queue is:

$$E\left[TW_q\right] = 14.22+(0.3)*(0.420)+(0.7)*(0.0722)+0.4886+0.0722 = 14.957$$

Thus, the total expected time spent in the system should be approximately, $E[T] = E[TW_q] + E[\mathrm{ST}] = 18.83 + 14.957 = 33.79$ minutes. We can compare this value to the simulation result (31.73 ± 0.44) with 95% confidence.

Based on this comparison, we can see that the approximation is very good. Again, this should add a great deal of credibility to the reported results. You might think that, in general, simulation models will be "too complex" for such validation methods. Well, sure, I agree that there can be very complex simulation models, but that does not mean that parts of the model

would not lend themselves to a simple analysis as demonstrated here. If the individual parts of a simulation model can be validated. Then, as long as we believe that when putting the parts together to form the entire model, it is still valid, then why wouldn't the overall results not be valid? By verifying and validating individual components of the model, we can build up the credibility in the overall model.

7.6.6. Other Issues for Verification and Validation

As previously noted, verification is about checking if the model or algorithm produces its desired output. Verification is an issue of software quality control. Since computer simulation relies on representing a system within software, we must ensure that we produce software (code) that is error-free. While it is beyond the scope of this book to fully discuss quality control issues within software development, we will not two major approaches that should be applied to simulation model development: 1) testing and 2) debugging.

The test cases should be designed from simple tests to more complicated tests. Test cases can be designed to test individual components of the model. If each component passes its test, then testing all components together should occur. For a stochastic simulation, testing can be performed using statistical tests over the set of test cases that indicate with a high level of confidence that the model produces results that are not significantly different than the expected output. In addition, you might show that the model converges to the expected result. Here, error and relative error, their statistical properties, etc. are important metrics to use to provide overall confidence in the verification process.

Testing a simulation model begins by developing the model *in stages*. You should first identify the key modeling issues and develop pseudo-code to address those issues. Then, you should map out a logical development process by which the components of the model can be constructed as

independently of each other as possible. For example, for the small manufacturing system, the first stage of the model building process would be to simulate the preparation station. We can dispose of those parts that leave the preparation station and analyze whether or not the preparation station is working correctly. You should identify parts of the system that can be analyzes with very little dependence on the other components. If we only want to check if the logic of the component is working correctly, we can adjust the arrival process to have a simpler deterministic arrival pattern and step through the model logic. For example, create a single entity and send it through the model. Does it do what you intended? Force the entity to take a particular path through the system by setting appropriate attributes or changing appropriate flow of control statements. Does the entity do what it is supposed to do for the test? As noted in Example 7.10 another approach is to set the capacity of all resources to infinity and run the model. Then, you can check if the basic flow time is as expected without any queueing. A staged development process that starts with pseudo-code and tests each stage is critical to ensuring that the model works as intended. Start small. Build small. Test small. And, then integrate the components.

The second recommended method of verification is debugging the model. Use the debugging capability of your software and step through the model. Check edge cases and debug the model. Run the model with watches on variables that stop the model if an invalid condition occurs, trace output through the debugger or by printing out data or logging the entity flow. Document what you did for the debugging. State what you did and what you found. The fact that you performed these verification steps is the essential first step in validating your model. Validation is whether or not the model meets its intended purpose. Validation cannot be accomplished unless verification steps have occurred.

Validation cannot be done without convincing the user that the key conceptual elements of the system have been adequately represented in the model. This includes the modeling of the inputs to the system and how the conceptual model has been translated to a computerized representation.

This is why documenting the input models and the conceptual model is so important. The old axiom, garbage in equals garbage out is pertinent here. The user must believe that the inputs have been well-modeled. The user must understand what is being modeled.

It is also important that everyone agrees on how the model is intended to be used. If there is no agreement on the conceptual representation or on the model's use, then validation will be unsuccessful. Validation is predicated on the model representing the system. Thus, validation often involves comparing the output of the model to the output of the real system. If the user of the model cannot tell the difference between model output and real system output, then the model's representational power is quite high. If you have data from the real system, especially system output performance measures (e.g. throughput, system time, etc.), then you can compare the results of the simulation model to the actual results from the system. You can perform statistical tests to show that the results from the simulation model are not statistically different from the actual system. Again, a staged approach can be used. Do not feel that you have to validate the whole model against the whole system. Perhaps you can collect data on some critical sub-component of the system being modeled. You can then compare the results of your simulation of that critical sub-component to the actual results from that sub-component. Validating each sub-component goes a long way to validating the overall model. It may be easier to collect data from the actual system in stages or in these smaller components. Besides checking if the output matches the actual system output, you should check if the simulation model behaves as expected. The output of the model should react in the same manner as that expected for the real system or the conceptual understanding of how the system works. If you increase the arrival rate, does congestion go up? If not, it is likely something may be misrepresented within your model.

In many situations, the real system does not exist yet. Thus, you will not have real data for validation purposes. This is often the case when simulation is being used to design a system. In such situations, you may be able to approximate the expected performance via "hand" calculations.

In this context, you can and should approximation the performance of the simulation model using analytical methods, such as queueing theory. This can provide bounds on the expected performance and add to the credibility of the results. In addition, even if the whole system may not yet exist, perhaps sub-components of the real system exist. If so, you can test output from the sub-components to the associated sub-components of your model. Do not think like you have validate the whole model! Think about how you can validate parts of the model, with the validation of each part contributing the validation of the whole.

Carefully documenting your conceptual model and your assumptions is also an important task for validation. If the user of your model does not understand it through its careful documentation, then they are less likely to believe the results from the model. Performing sensitivity analysis on a set of representative test cases can build credibility in the usage of the model. Validation is ultimately about believability. Does the user believe the results of the model? Validation is whether or not the model/algorithm meets it intended purpose. Verification is about checking if the model/algorithm meets the desired output. You must perform these functions when building a simulation model.

7.7. Summary

This chapter provided a discussion of miscellaneous modeling constructs that can improve your modeling. The modeling of non-stationary arrivals was discussed and it motivated the exploration of advanced resource modeling constructs. The advanced resource modeling constructs allow for resources to be subjected to scheduled capacity changes. Finally, a few interesting and useful miscellaneous modeling constructs were presented.

The following new KSL constructs were discussed in this chapter:

`ResourcePool`: A class that represents a set of resources from which a resource can be selected and seized.

`ResourcePoolWithQ`: A class that represents a set of resource from which a resource can be selected and seized with a specified queue to hold entities that are not immediately allocated a resource.

`ResourceSelectionRuleIfc`: An interface that defines how a resource can be selected for allocation from a resource pool.

`NHPPEventGenerator`: A class that models the time between events from a non-homogeneous Poisson process.

`NHPPTimeBtwEventRV`: A specialized random variable that models the time between events according to a non-homogeneous Poisson process.

`CapacitySchedule`: A class that specifies that amount of capacity and the duration of availability for a resource.

`ResponseInterval`: A class that represents an interval of time over which statistical collection should be performed.

`ResponseSchedule`: A class that permits the specification of many response intervals and allows the repeating of the intervals over time.

`TimeSeriesResponse`: A class that divides the simulated time horizon into sequential periods of equal length and records the response for each period, for each replication, producing a time series that is suitable for plotting and further analysis.

With these concepts in your simulation toolkit, you are prepared to model more advanced situations. The next chapter explores how to model the movement of entities between locations, including movement that is constrained by resources, by vehicles, and by conveyors.

7.8. Exercises

Exercise 7.1. Consider the testing and repair shop. Suppose instead of increasing the overall arrival rate of jobs to the system, the new contract will introduce a new type of component into the system that will require a new test plan sequence. The following two tables represent the specifics associated with the new testing plan.

Test Plan	% of parts	Sequence
1	20%	2,3,2,1
2	12.5%	3,1
3	37.5%	1,3,1
4	20%	2,3
5	10%	2,1,3

Test Plan	Testing Time Parameters	Repair Time Parameters
1	(20,4.1), (12,4.2), (18,4.3), (16,4.0)	(30,60,80)
2	(12,4), (15,4)	(45,55,70)
3	(18,4.2), (14,4.4), (12,4.3)	(30,40,60)
4	(24,4), (30,4)	(35,65,75)
5	(20,4.1), (15,4), (12,4.2)	(20,30,40)

Management is interested in understanding where the potential bottlenecks are in the system and in developing alternatives to mitigate those bottlenecks so that they can still handle the contract. The new contract stipulates that 80% of the time the testing and repairs should be completed within 480 minutes. The company runs 2 shifts each day for each 5 day work week. Any jobs not completed at the end of the second shift are carried over to first shift of the next working day. Assume that the contract is going to last for 1 year (52 weeks). Build a simulation model that can assist the company in assessing the risks associated with the new contract.

Use the following stream allocations:

- Stream 1, part arrival process
- Stream 2, diagnostic time
- Stream 3, test plan distribution
- Streams 4-8, test plan 1 testing times and repair
- Streams 9-11, test plan 2 testing times and repair
- Streams 12-15, test plan 3 testing times and repair
- Streams 16-18, test plan 4 testing times and repair
- Streams 19-22, test plan 5 testing times and repair
- Stream 23, transfer time between stations

Exercise 7.2. Parts arrive at a 4 workstation system according to an exponential inter-arrival distribution with a mean of 10 minutes (stream 1). The workstation A has 2 machines. The three workstations (B, C, D) each have a single machine. There are 3 part types, each with an equal probability of arriving (stream 2). The process plan for the part types are given below. The entries are for exponential distributions with the mean processing time (MPT) parameter given.

Workstation, MPT	Workstation, MPT	Workstation, MPT
A, 9.5	C, 14.1	B, 15
A, 13.5	B, 15	C, 8.5
A, 12.6	B, 11.4	D, 9.0

Assume that the transfer time between arrival and the first station, between all stations, and between the last station and the system exit is exponentially distributed with a mean of 3 minutes (stream 3). Simulate the system for 30000 minutes and discuss the potential bottlenecks in the system.

Use the following stream allocations for the process plans:

- Streams 4-6, Plan 1

- Streams 7-9, Plan 2
- Streams 10-12, Plan 3

Exercise 7.3. A small manufacturing system produces three types of parts. There is a 30% chance of getting a Type 1 part, a 50% chance of getting a Type 2 part and a 20% chance of getting a Type 3 part. The parts arrive from an upstream process such that the time between arrivals is exponentially distributed with a mean of 3 minutes. All parts that enter the system must go through a preparation station where there are 2 preparation workers. The preparation time is exponentially distributed with means 3, 5, and 7 for part types 1, 2, and 3, respectively.

There is only space for 6 parts in the preparation queue. Any parts that that arrive to the system when there are 6 or more parts in the preparation queue cannot enter the system. These parts are shunted to a re-circulating conveyor, which takes 10 minutes to re-circulate the parts before they can try again to enter the preparation queue. Hint: Model the re-circulating conveyor as a simple deterministic delay.

After preparation, the parts are processed on two different production lines. A production line is dedicated to type 1 parts and a production line is dedicated to type 2 and 3 parts. Part types 2 and 3 are built one at a time on their line by 1 of 4 operators assigned to the build station. The time to build a part type 2 or 3 part is triangularly distributed with a (min = 5, mode = 10, max = 15) minutes. After the parts are built they leave the system.

Part type 1 has a more complicated process because of some special tooling that is required during the build process. In addition, the build process is separated into two different operations. Before starting operation 1, the part must have 1 of 10 special tooling fixtures. It takes between 1 and 2 minutes uniformly distributed to place the part in the tooling

fixture. An automated machine places the part in the tooling so that the operator at operation 1 does not have to handle the part. There is a single operator at operation 1 which takes 3 minutes on average exponentially distributed. The part remains in the tooling fixture after the first operation and proceeds to the second operation. There is 1 operator at the second operation which takes between 3 and 6 minutes uniformly distributed. After the second operation is complete, the part must be removed from the tooling fixture. An automated machine removes the part from the tooling so that the operator at operation 2 does not have to handle the part. It takes between 20 and 30 seconds uniformly distributed to remove the part from the tooling fixture. After the part is built, it leaves the system.

In this problem, the steady state performance of this system is required in order to identify potential long-term bottlenecks in this process. For this analysis, collect statistics on the following quantities:

- Queue statistics for all stations. Utilization statistics for all resources.
- The system time of parts by part type. The system time should not include the time spent on the re-circulating conveyor.
- The average number of parts on the conveyor.

Perform a warm up analysis on the system time of a part regardless of type.

Use the following stream allocations:

- Stream 1, part type distribution
- Stream 2, arrival process
- Streams 3-5, preparation time for types 1, 2, 3
- Stream 6, part type 2 or 3 build time
- Stream 7, part type 1 tooling placement time
- Stream 8, part type 1 first operation time
- Stream 9, part type 1 second operation time
- Stream 10, part type 1, tooling removal time

Exercise 7.4. Reconsider Exercise 7.3. A process change is being recommended for the build station for part type 2 and 3. In particular, a machine change will cause the processing time to be log-normally distributed with a mean of 10 and a standard deviation of 2 minutes. Compare the system time of the old configuration and the new configuration based on 30 replications of length 1000 hours with a warm up of 200 hours. Which configuration would you recommend?

Exercise 7.5. A patient arrives at the Emergency Room about every 20 $\pm$ 10 minutes (stream 1). The notation X $\pm$ Y means uniformly distributed with minimum $X-Y$ and maximum $X+Y$. They will be treated by either of two doctors.

Twenty percent of the patients are classified as NIA (need immediate attention) and the rest as CW (can wait), using stream 6. NIA patients have the highest priority for their first treatment. They see a doctor as soon as possible for 40 $\pm$ 37 minutes (stream 2). After it, they wait for their final treatment at a middle priority until a doctor is free again, when they receive further treatment for 30 $\pm$ 25 minutes (stream 3) and are discharged.

CW patients have the lowest priority for their first treatment and are treated (when their turn comes) for 15 $\pm$ 14 minutes (stream 4). After it, they wait for their final treatment at the middle priority until a doctor is free, receive 10 $\pm$ 8 minutes (stream 5) of final treatment, and are discharged.

An important aspect of this system is that patients that have already seen the doctor once compete with newly arriving patients that need a doctor. Patients waiting for their final treatment share one priority

level, below newly arriving NIA patients and above newly arriving CW patients. Thus, there is one shared queue for the first treatment activity and the final treatment activity. In addition, we assume that the doctors are interchangeable. That is, it does not matter which of the 2 doctors performs the first or final treatment. *Hint:* in the KSL, a smaller priority value means a higher priority.

Simulate for 20 days of continuous operation, 24 hours per day. Precede this by a 2-day initialization period to load the system with patients.

Measure the average queue length of NIA patients from arrival to first seeing a doctor. What percent have to wait to see the doctor for the first treatment? Report statistics on the initial waiting time for NIA patients. What percent wait less than 5 minutes before seeing a doctor for the first time? Report statistics on the time in system for the patients. Report statistics on the remaining time in system from after the first treatment to discharge, for all patients. Discuss what changes to your model are necessary if the doctors are not interchangeable. That is, suppose there are two doctors: Dr. House and Dr. Wilson. The patient must get the same doctor for their final treatment as for their first treatment. For example, if a patient gets Dr. House for their first treatment, they must see Dr. House for their final treatment. You do not have to implement the changes.

Exercise 7.6. Consider the M/G/1 queue with the following variation. The server works continuously as long as there is at least one customer in the system. The customers are processed FIFO. When the server finishes serving a customer and finds the system empty, the server goes away for a length of time called a vacation. At the end of the vacation the server returns and begins to serve the customers, if any, who have arrived during the vacation. If the server finds no customers waiting at the end of a vacation, it immediately takes another vacation, and continues in this manner until it finds at least one waiting customer upon return from a

vacation. Assume that the time between customer arrivals is exponentially distributed with mean of 3 minutes (stream 1). The service distribution for each customer is a gamma distribution with a mean of 4.5 seconds and a variance of 3.375 (stream 2). The length of a vacation is a random variable uniformly distributed between 8 and 12 minutes (stream 3).

Run the simulation long enough to adequately develop a 95% confidence interval on the expected wait time in the queue for a arbitrary customer arriving in steady state. In addition, develop an empirical distribution for the number of customers waiting upon the return of the server from vacation. In other words, estimate the probability that $j = 0, 1, 2$, etc, where j is the number of waiting customers in the queue upon the return of the server from a vacation. This queue has many applications, for example, consider how a bus stop operates.

Exercise 7.7. An airline ticket office has two ticket agents answering incoming phone calls for flight reservations. In addition, six callers can be put on hold until one of the agents is available to take the call. If all eight phone lines (both agent lines and the hold lines) are busy, a potential customer gets a busy signal, and it is assumed that the call goes to another ticket office and that the business is lost. The calls and attempted calls occur randomly (i.e. according to Poisson process) at a mean rate of 15 per hour (stream 1). The length of a telephone conversation has an exponential distribution with a mean of 4 minutes (stream 2).

In addition, the ticket office has instituted an automated caller identification system that automatically places First Class Business (FCB) passengers at the head of the queue, waiting for an agent. Of the original 15 calls per hour, they estimate that roughly one-third of these come from FCB customers (stream 3). They have also noticed that FCB customers take approximately 3 minutes on average for the phone call, still exponentially distributed (stream 4). Regular customers still take on average 4 minutes, exponentially

distributed (stream 2). Simulate this system with and without the new prioritization scheme and compare the average waiting time for the two types of customers.

Exercise 7.8. Consider a system having 6 machines tended by 2 operators. In this system, there are two types of stoppages. Type 1 stoppage occurs after a fixed constant amount of machine running time, $1/\lambda_1 = 30$ minutes, and has a constant value of $1/\mu_1 = 10$ minutes as the service time for the stoppage. Type 2 stoppages occur after random intervals of time, negatively exponentially distributed, with a mean of $1/\lambda_2 = 10$ minutes (stream 1). Service times for type 2 stoppages are negative exponentially distributed with a mean of $1/\mu_2 = 4$ minutes (stream 2). Both of the operators have the same skills and can handle either type of stoppage. The machines wait for service from the operators in a first come first served queue with no priority given to either type of stoppage. Simulate this system for 10000 minutes to estimate the average number of waiting machines by type of stoppage, the average utilization of the operator, the average utilization of the machines, and the average waiting time of the machines by type of stoppage.

Exercise 7.9. Suppose a service facility consists of two stations in series (tandem), each with its own FIFO queue. Each station consists of a queue and a single server. A customer completing service at station 1 proceeds to station 2, while a customer completing service at station 2 leaves the facility. Assume that the inter-arrival times of customers to station 1 are IID exponential random variables with a mean of 1 minute (stream 1). Service times of customers at station 1 are exponential random variables with a mean of 0.7 minute (stream 2), and at station 2 are exponential

random variables with mean 0.9 minute (stream 3). Develop a model for this system.

a. Run the simulation for exactly 20000 minutes and estimate for each station the expected average delay in queue for the customer, the expected time-average number of customers in queue, and the expected utilization. In addition, estimate the average number of customers in the system and the average time spent in the system.

b. Use the results of queueing theory to verify and validate your results for part (a)

c. Suppose now there is a travel time from the exit of station 1 to the arrival to station 2. Assume that this travel time is distributed uniformly between 0 and 2 minutes. Modify your simulation and rerun it under the same conditions as in part (a).

Exercise 7.10. Parts arrive to a machine shop according to an exponential distribution with a mean of 10 minutes. Before the parts go into the main machining area they must be prepped. The preparation area has two preparation machines that are tended by 2 operators. Upon arrival parts are assigned to either of the two preparation machines with equal probability. Except for processing times the two preparation machines operate in the same manner. When a part enters a preparation machine area, it requires the attention of an operator to setup the part on the machine. After the machine is setup, the machine can process without the operator. Upon completion of the processing, the operator is required to remove the part from the machine. Two operators, shared by both preparation machines, do all the setups and part removals, first come, first served. The machine is needed only during processing. The times for the preparation machines are given in the table below according to a triangular distribution with the provided parameters:

Prep Machine	Setup Time	Process Time	Removal Time
1	(8,11,16)	(15,20,23)	(7,9,12)
2	(6,8,14)	(11,15,20)	(4,6,8)

After preparation the parts must visit the machine shop. There are 4 machines in the machine shop. The parts follow a specific sequence of machines within the shop. This is determined after they have been prepped. The percentage for each sequence is given in the table below. The #,(min, mode, max) provides the machine number, #, and the parameters for the processing times for a triangular distribution in minutes.

Sequence	%	#,(min, mode, max)	#,(min, mode, max)	#,(min, mode, max)	#,(min, mode,
1	12	1,(10.5,11.9,13.2)	2, (7.1,8.5,9.8)	3,(6.7,8,10)	4, (1,8.9,10.
2	14	1,(7.3,8.6,10.1)	3,(5.4,7.2, 11.3)	2,(9.6, 11.4, 15.3)	
3	31	2,(8.7,9.9,12)	4,(8.6,10.3,12.8)	1,(10.3, 12.4, 14.8)	3,(8.4,9.7,1
4	24	3,(7.9,9.3, 10.9)	4,(7.6,8.9,10.3)	3,(6.5,8.3,9.7)	2,(6.7,7.8,9.
5	19	2,(5.6,7.1,8.8)	1,(8.1, 9.4, 11.7)	4,(9.1, 10.7, 12.8)	

The transfer time between the prep area and the first machine in the sequence, between all machines, and between the last machine and the system exit follows a triangular distribution with parameters 2, 4, 6 minutes.

a. Run the model for 200,000 minutes. Report average and 95% half-width statistics on the utilization of the preparation operators, the utilization of the preparation machines, the utilization of the job shop machines 1-4, number of parts in the system, and time spent in the system (in minutes).

b. Recommend a warm up period for the total time spent in the system. Show your work to justify your recommendation.

c. Where is the bottleneck for this system? What would you recommend to improve the performance of this system?

Use the following stream allocations:

- Stream 1, part arrival process
- Stream 2, preparation machine routing
- Streams 3-5, setup, preparation, removal time for prep machine 1
- Streams 6-8, setup, preparation, removal time for prep machine 2
- Stream 9, sequence determination
- Stream 10-13, sequence 1 processing times
- Stream 14-16, sequence 2 processing times
- Stream 17-20, sequence 3 processing times
- Stream 21-24, sequence 4 processing times
- Stream 25-27, sequence 5 processing times
- Stream 28, transfer time

Exercise 7.11. Consider the simple three-workstation flow line. Parts entering the system are placed at a staging area for transfer to the first workstation. The staging area can be thought as the place where the parts enter the system prior to going to the first workstation. No processing takes place at the staging area, other than preparation to be directed to the appropriate stations. After the parts have completed processing at the first workstation, they are transferred to a paint station manned by a second worker, and then to a packaging station where they are packed by a third worker, and then to a second staging area where they exit the system.

The time between part arrivals at the system is exponentially distributed with a mean of 28 minutes (stream 1). The processing time at the first workstation is uniformly distributed between 21 and 25 minutes (stream 2). The paint time is log-normally distributed with a mean of 22 minutes and a standard deviation of 4 (stream 3). The packing time follows a triangular distribution with a minimum of 20, mode of 22, and a maximum of 26 (stream 4). The transfers are unconstrained, in that they do not

require a vehicle or resource, but all transfer times are exponential with a mean of 2 or 3 minutes (stream 5). Transfer times from the staging to the workstation and from pack to exit are 3 minutes. Transfer times from the workstation to paint and from paint to pack are 2 minutes. The performance measures of interest are the utilization and Work-In-Progress (WIP) at the workstation, paint and packaging operations. Figure 7.14 provides an illustration of the system.

(This problem is based on an example on page 209 and continues on page 217 of (Pegden et al. 1995). Used with permission)

Suppose that statistics on the part flow time, i.e. the total time a part spends in the system need to be collected. However, before simulating this process, it is discovered that a new part needs to be considered. This part will be painted a different color. Because the new part replaces a portion of the sales of the first part, the arrival process remains the same, but 30 percent of the arriving parts are randomly designated as the new type of part (stream 10). The remaining parts (70% of the total) are produced in the same manner as described above. However, the new part requires the addition of a different station with a painting time that is log-normally distributed with a mean of 49 minutes and a standard deviation of 7 minutes (stream 6). Assume that an additional worker is available at the new station. The existing station paints only the old type of part and the new station paints only the new parts. After the painting operation, the new part is transferred to the existing packing station and requires a packing time that follows a triangular distribution with a minimum value of 21, a mode of 23, and a maximum of 26 (stream 7). Run the model for 600,000 minutes with a 50,000 minute warm up period. If you were to select a resource to add capacity, which would it be?

Exercise 7.12. Suppose a service facility consists of two stations in series (tandem), each with its own FIFO queue. Each station consists of a queue

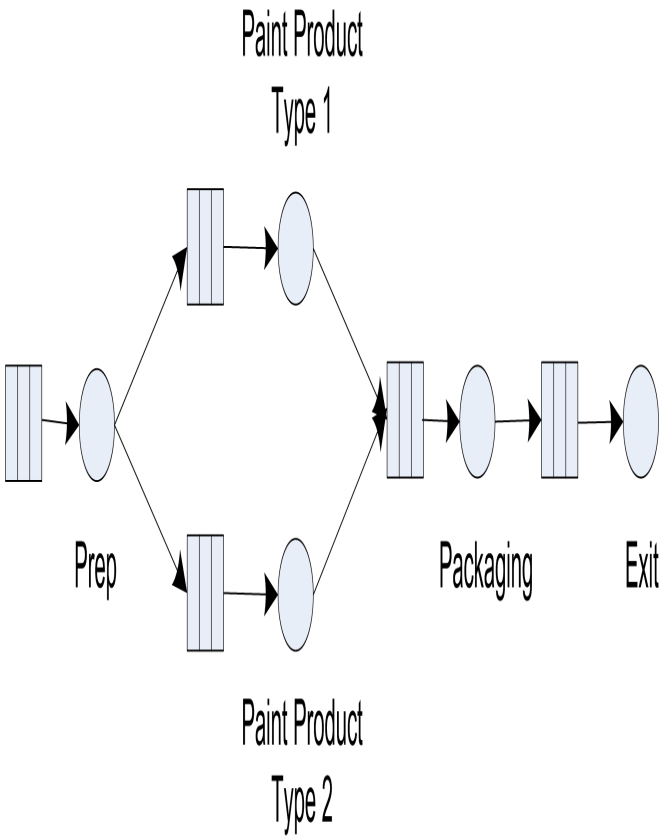


Figure 7.14.: Simple Painting Flow Line

and a single server. A customer completing service at station 1 proceeds to station 2, while a customer completing service at station 2 leaves the facility. Assume that the inter-arrival times of customers to station 1 are IID exponential random variables with a mean of 1.25 minutes (stream 1). Service times of customers at station 1 are exponential random variables with a mean of 0.7 minute (stream 2), and at station 2 are exponential random variables with mean 0.9 minute (stream 3).

Suppose that there is limited space at the second station. In particular, there is room for 1 customer to be in service at the second station and room for only 1 customer to wait at the second station. A customer completing service at the first station will not leave the service area at the first station unless there is a space available for it to wait at the second station. In other words, the customer will not release the server at the first station unless it can move to the second station.

Develop a model for this system. Run the simulation for exactly 20000 minutes with a warm up period of 2000 minutes for 20 replications and estimate for each station the expected average delay in queue for the customer, the expected time-average number of customers in queue, and the expected utilization. In addition, estimate the average number of customers in the system and the average time spent in the system. Is the buffer size between the stations sufficient for this situation?

Exercise 7.13. A particular stock keeping unit (SKU) has demand that averages 14 units per year and is Poisson distributed. That is, the time between demands is exponentially distributed with a mean of 1/14 years (stream 1). Assume that 1 year = 360 days. The inventory is managed according to a (r, Q) inventory control policy with $r = 3$ and $Q = 4$. The SKU costs \$150. An inventory carrying charge of 0.20 is used and the annual holding cost for each unit has been set at 0.2 * \$150 = \$30 per unit per year. The SKU is purchased from an outside supplier and it is

estimated that the cost of time and materials required to place a purchase order is about $15. It takes 45 days to receive a replenishment order. The cost of back-ordering is very difficult to estimate, but a guess has been made that the annualized cost of a back-order is about $25 per unit per year.

a. Use simulation to evaluate the performance of this system using $Q = 4$ and $r = 3$. Report the average inventory on hand, the cost for operating the policy, the average number of back-orders, and the probability of a stock out for your model.

b. Now suppose that the lead-time is stochastic and governed by a log-normal distribution with a mean of 45 days and a standard deviation of 7 days (stream 2). What assumptions do you have to make to simulate this situation? Simulate this situation and compare/contrast the results with part (a).

Exercise 7.14. The Super Ready Auto Club has been serving customers for many years. The Super Ready Auto Club provides club, travel, and financial services to its members. One of its most popular services includes auto touring and emergency road service. Travel provides cruise packages, airline travel, and other vacation packages with special discounts for members. Finally, the financial services issues credit cards for members at special interest rates.

Super Ready Auto Club has regional call centers that handle incoming calls from members within a given region of the country. Table 7.13 presents the mean of the number of calls, recorded on an hourly basis over a period of 7 days, aggregated over a three state region.

The three types of service calls occur with 50% for auto service, 35% for credit card services, and 15% for travel services during the 8 am to 8 pm time frame. For the other hours of the day, the proportion changes to 90%

for auto service, 10% for credit card services, and 0% for travel services. A sample of call service times was recorded for each of the types as shown in Table 7.14, Table 7.15, and Table 7.16. You can find this data in the chapter files that accompany this chapter in the spreadsheet called *Super Ready Auto Club Data.xlsx*. Use stream 1 for road side service calls, stream 2 for vacation service calls, and stream 3 for credit card service calls. Use stream 4 for the call arrivals and stream 5 for the type of call.

Call center operators cost $18 per hour including fringe benefits. Management requires that, over the week, the probability that a call waits more than 3 minutes for an operator be less than 10%. Find a minimum cost staffing plan, the number of operators on duty in each hour of each day of the week, that meets this requirement. Measure the waiting time for a call, the average number of calls waiting, and the utilization of the operators. In addition, measure the waiting time of the calls by type. *Hint:* when the number of operators on duty drops at the end of an hour, use `CapacityChangeRule.IGNORE` so that the schedule does not wait for busy operators to finish their calls.

a. What happens to the waiting times if road-side assistance calls are given priority over the credit card and travel service calls?

b. Consider how you would handle the following work rules. In each 4 hours of work, an operator gets one 10 minute break and one 30 minute food/beverage break. Discuss how you would staff the center under these conditions ensuring that there is always someone present (i.e. they are not all on break at the same time).

c. For your plan, estimate the probability that a call waits more than 3 minutes for the calls arriving in each hour of the week. Is the requirement met in every hour? Explain what you find, and estimate what it would cost to meet the 10% requirement in every hour.

Table 7.13.: Mean Number of Calls Received for Each Hour

Hour	Mon	Tue	Wed	Thur	Fri	Sat	Sun
1	6.7	6.1	8.7	8.9	5.6	6.2	9.4
2	9.7	9.4	9.3	7.1	1.4	7.0	8.8
3	8.5	6.0	70.4	8.6	7.9	7.8	13.4
4	9.0	10.0	6.8	8.3	6.4	6.3	6.7
5	6.8	9.2	10.9	44.9	8.2	6.5	7.5
6	8.5	4.2	1.4	6.3	7.5	7.8	8.0
7	6.7	6.9	9.0	8.9	6.7	46.2	9.8
8	38.2	35.0	75.8	57.8	37.2	82.5	78.8
9	20.0	77.6	48.7	25.9	67.1	28.1	99.2
10	76.2	75.4	71.1	51.3	86.2	106.9	42.0
11	92.5	105.4	28.2	100.2	90.9	92.4	43.6
12	76.4	37.3	37.3	77.8	60.3	37.9	68.9
13	79.1	64.2	25.4	76.8	65.1	77.9	116.6
14	75.2	102.2	64.1	48.7	93.3	64.5	39.5
15	117.9	26.8	43.8	80.1	86.4	96.8	52.1
16	100.2	85.9	54.3	144.7	70.9	167.9	153.5
17	52.7	64.7	94.7	94.6	152.2	98.0	63.7
18	65.3	114.9	124.9	213.9	91.5	135.7	79.8
19	126.1	69.6	89.4	43.7	62.7	111.9	180.5
20	116.6	70.2	116.8	99.6	113.0	84.3	64.1
21	38.6	20.5	7.6	3.8	58.6	50.8	68.5
22	8.5	17.1	8.8	13.5	8.2	68.1	7.8
23	9.8	48.9	8.9	95.4	40.8	58.5	44.1
24	9.6	1.3	28.5	9.6	32.2	74.5	46.7

Table 7.14.: Road Side Service Call Times in Minutes

33.6	34.0	33.2	31.3	32.3	32.1	39.0	35.2	37.6	37.4
32.3	37.2	39.3	32.1	34.7	35.1	33.6	32.8	32.8	40.0

37.8	38.0	42.4	37.4	36.6	36.6	33.3	34.7	30.2	33.1
36.8	34.4	36.4	35.9	32.6	37.6	36.7	32.3	40.0	37.0
36.0	34.6	33.9	31.7	33.8	39.3	37.8	33.7	35.2	38.2
34.6	33.5	36.3	38.9	35.5	35.2	35.0	33.8	35.8	35.8
38.2	38.8	35.7	38.7	30.0	33.6	33.4	34.7	35.1	35.5
34.0	33.2	33.7	32.5	28.9	34.4	34.2	31.4	38.7	35.3
35.5	39.4	32.6	36.2	33.2	39.3	41.1	34.3	38.6	33.0
35.3	38.1	33.5	34.0	36.4	33.6	43.1	37.2	35.6	36.0

Table 7.15.: Vacation Package Service Call Times in Minutes

52.2	23.7	33.9	30.5	18.2	45.2	38.4	49.8	53.9	38.8
33.4	44.2	28.4	49.0	24.9	46.4	35.8	40.3	16.3	41.4
63.1	37.5	33.5	48.0	27.6	38.2	28.6	35.2	24.5	42.9
38.9	19.8	32.7	41.4	42.7	27.4	38.7	30.1	39.6	53.5
28.8	42.2	42.4	29.2	22.7	50.9	34.2	53.1	18.5	26.5
40.9	20.7	36.6	33.7	26.4	32.8	25.1	39.7	30.9	34.2
35.5	31.8	27.2	28.6	26.9	30.6	26.1	23.2	33.3	35.0
29.6	31.6	29.9	28.5	29.3	30.6	31.2	26.7	25.7	41.2
27.5	52.0	27.3	69.0	34.2	31.4	21.9	29.8	31.4	46.1
23.8	35.9	41.5	51.0	17.9	33.9	32.2	26.5	25.0	54.6

Table 7.16.: Credit Card Service Call Times in Minutes

13.5	10.1	14.4	13.9	14.2	11.4	11.0	14.4	12.0	14.4
14.4	13.5	13.4	14.5	14.8	14.9	13.2	11.0	14.9	15.0
14.7	14.3	12.3	14.8	12.4	14.9	15.0	14.0	14.6	14.6
15.0	10.3	11.3	15.0	15.0	15.0	14.9	14.8	11.5	11.9
13.5	14.9	11.0	10.8	15.0	13.8	13.8	14.5	14.3	14.0
14.9	10.9	13.9	13.5	15.0	14.4	13.2	15.0	14.5	14.4
15.0	14.5	14.5	14.9	11.6	13.6	12.4	11.9	11.6	15.0

11.1	13.7	13.9	14.6	11.5	14.4	15.0	10.7	10.4	14.5
12.3	13.8	14.7	13.8	14.7	14.8	14.9	14.2	12.9	14.3
14.5	11.5	12.1	14.3	13.6	14.1	12.6	11.7	14.7	13.4

Exercise 7.15. Reconsider the STEM Career Fair Mixer Example. The results indicated that the utilization of the JHBunt recruiters was still high, around 82%, and the MalWart recruiters was low, around 50% after making the resource schedule changes. Design a schedule that gets both recruiting stations to about 80% during the peak period.

Exercise 7.16. Suppose a service facility consists of two stations in series (tandem), each with its own FIFO queue. Each station consists of a queue and a single server. A customer completing service at station 1 proceeds to station 2, while a customer completing service at station 2 leaves the facility. Assume that the inter-arrival times of customers to station 1 are IID exponential random variables with a mean of 1 minute (stream 1). Service times of customers at station 1 are exponential random variables with a mean of 0.7 minute (stream 2), and at station 2 are exponential random variables with mean 0.9 minute (stream 3). Assume that the travel time between the two stations must be modeled. The travel time is distributed according to a triangular distribution with parameters (1, 2, 4) minutes (stream 4).

a. Model the system assuming that the worker from the first station moves the parts to the second station. The movement of the part should be given priority if there is another part waiting to be processed at the first station.

b. Model the system with dedicated workers to move parts between the stations. Justify your recommended number of workers.

From your models, estimate the total system time for the parts, the utilization of the workers, and the average number of parts waiting for the workers. Run the simulation for exactly 20000 minutes with a warm up period of 5000 minutes, using 10 replications.

Exercise 7.17. Reconsider part (b) of Exercise 7.16. Instead of immediately moving the part, the transport worker waits until a batch of 5 parts has been produced at the first station. When this occurs, the worker moves the batch of parts to the second station. The parts are still processed individually at the second station. From your model, estimate the total system time for the parts, the utilization of the workers, and the average number of parts waiting for the workers. Run the simulation for exactly 20000 minutes with a warm up period of 5000 minutes, using 10 replications.

Exercise 7.18. Redo Exercise 7.1 using resource constrained transfer. Assume that there are 2 workers at the diagnostic station, 1 worker per testing station, and 3 workers at the repair station. Thus, there are a total of 8 workers in the system. Furthermore, assume that any of these 8 workers are capable of moving parts between the stations. For example, when a part completes its operation at the diagnostic station, any worker in the system can carry the part to the next station. When a part requires movement, it will wait for the next available idle worker to complete the movement. Assume that parts waiting for processing at a station will be given priority over parts that require movement between stations. Build a simulation model that can assist the company in assessing the risks associated with the new contract under this resource constrained situation.

Exercise 7.19. Redo Exercise 7.2 assuming that there is a pool of 3 workers that perform the transport between the stations. Assume that the transport time is triangularly distributed with parameters (2, 4, 6) all in minutes (stream 13). Make an assessment for the company for the appropriate number of workers to have in the transport worker pool. Use 10 replications.

Exercise 7.20. Redo Exercise 7.10 assuming that there is a pool of 3 workers that perform the transport between the stations. Assume that the transport time is triangularly distributed with parameters (2, 4, 6) all in minutes (stream 28). Make an assessment for the company for the appropriate number of workers to have in the transport worker pool. Use 10 replications.

Exercise 7.21. Consider the drive through pharmacy model of Section 4.4.4. Add a `ResponseSchedule` that collects statistics on the number of busy pharmacists, the time spent waiting in the queue, and the number of customers served, using four intervals of 120 minutes each. Run the model for 30 replications of 480 minutes.

a. Which interval has the highest average waiting time? Is the difference between that interval and the others statistically significant?
b. Repeat the analysis using a `TimeSeriesResponse` that is configured with the same period length and number of periods. Verify that the two mechanisms report the same averages.

Exercise 7.22. Repeat Exercise 7.21 with a mean time between arrivals of 20 minutes and 32 intervals of 15 minutes, so that some intervals contain no observations of the waiting time response.

a. Examine the number of observations that is reported for each interval by the `ResponseSchedule`. Explain why the counts differ across the intervals.
b. Examine the same periods within the raw data that is recorded by the `TimeSeriesResponse`. How does this mechanism indicate that a period had no observations?
c. Which of the two representations would you prefer if you intended to plot the results as a series over time? Justify your answer.

Exercise 7.23. Consider the STEM Career Fair Mixer model of Section 7.4. Define a standalone `ResponseInterval` that covers the final hour of operation and add the total number of students at the recruiters to it, requesting the empty interval statistic.

a. Report the estimated probability that the interval is empty. What does this quantity tell you about the reliability of the reported interval average?
b. The response that is created for a time-persistent variable includes a value at start statistic. Report it for this interval and explain what it contributes to the interpretation of the interval average.

---

8. Modeling Entity Movement

Learning Objectives

- To be able to model constrained entity transfer with resources
- To be able to model constrained entity transfer with movable resources
- To be able to model systems involving conveyors
- To be able to model systems involving automatic guided vehicles

Section 7.4.2 of Chapter 7 introduced the concept entity movement by computing a movement time from a velocity and a known distance. The entities can be imagined as "having little feet" that allow them to move from one location to another. That is, the entity is able to move itself. For example, in the test and repair example of Section 7.1.3, the parts moved between stations with a time delay. But how did they physically move?

One way to think about this is that there were workers always available to move the parts between the stations. If there is *always* a worker available, then it is as if there is an infinite supply of workers. Thus, whenever a part must be moved from one location to another the part uses one of the workers to make the movement. Since there is an infinite supply of workers, this is the same as the part moving itself (i.e. having little feet) and only the time delay for moving between the locations is relevant. If the entity is a person (e.g. a patient in a clinic), this analogy makes sense; however, if the entity is a part in a manufacturing system, this analogy begins to lose its appeal.

In many situations, modeling transfers with a delay is perfectly reasonable, especially if you are not interested in how the entities moved (only that they moved). However, in many situations, the movement of entities can become constrained by the lack of availability of the transfer mechanism. For example, the movement of parts may require that the parts be placed on a pallet and that a fork lift be used to move the parts. At any point in time, there may not be enough fork lifts available and thus the parts may have to wait for a fork lift to become available. When the potential for waiting for transport is significant, including the details the "how" behind the entity transfer becomes important. In addition, since movement can be a significant part of an operation, the design of the material movement system may be the main focus of the simulation modeling effort.

This chapter explores the various KSL constructs available to facilitate the modeling of the physical movement of entities between locations. The chapter begins by describing how to model transfers using resources. In this case, the transfer delay is accompanied by the use of a resource. Then, Section 8.2 presents how the KSL facilitates resource constrained movement using resources that can move. Since not all movement is as freely moving through space as people walking or fork trucks moving, the KSL provides constructs for modeling entity movement when the space between the locations becomes an important aspect of the modeling. Section 8.4 indicates how conveyors can be represented within the KSL and how they represent the space between locations. Then, in Section 8.5, transport is revisited to understand how to model the situation where the transporters compete for space while moving. This involves the modeling of the space between stations as a fixed path (i.e. like a road network with intersections). As usual, these concepts will be illustrated with example models.

> **Note**
>
> This chapter provides a series of example Kotlin code that illustrates the use of KSL constructs for implementing the modeling of entity movement. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

8.1. Resource Constrained Transfer

When an entity requires the use of something to complete the movement between locations, the KSL's `Resource` class can be used to model the situation. In this case, the physical (e.g. distance) aspects of the movement are not of interest; however, the time that may be constrained by the availability of the transport mechanism should be part of the modeling. For this situation, it is sufficient to model the transport mechanism with a resource (or resource pool). We will illustrate the approach by enhancing the test and repair example from Section 7.1.3.

8.1.1. Test and Repair with Resource Constrained Transfer

Recall that in the test and repair shop, parts follow 1 of 4 different test plans through the shop. Each part first goes to the diagnostic station where it determines the sequence of stations that it will visit via an assignment of a test plan. After being diagnosed, it then proceeds to the first station in its test plan. In the example, it was assumed that a worker (from somewhere) was *always* available to move the part to the next station and that the transfer time took between 2-4 minutes uniformly distributed. The diagnostic station had two diagnostic machines and each test station had 1

testing machine. Finally, the repair station had 3 workers that performed the necessary repairs on the parts after testing.

An *implicit* assumption in the model was that there was a worker staffing each of the two diagnostic machines (1 worker for each machine) and that there was a worker assigned to each test station. The modeling of the workers was not a key component of the modeling because the machines cannot operate without the worker. Thus, seizing the machine is essentially the same as seizing the worker.

In this section, the use of the workers to move the entities and to staff the stations will be explicitly modeled. Assume that there are 2 workers at the diagnostic station, 1 worker per testing station, and 3 workers at the repair station. Thus, there are a total of 8 workers in the system. For simplicity, assume that any of these 8 workers are capable of moving parts between the stations. For example, when a part completes its operation at the diagnostic station, any worker in the system can carry the part to the next station. In reality, it may be useful to assign certain workers to certain transfers (e.g. diagnostic workers move parts to the part's first station); however, for simplicity these issues will be ignored and any worker will be allowed to do any transport in this example.

This approach also requires that any worker is capable of noticing that a part needs movement. For example, perhaps the part is put in a basket and a light goes on indicating that the part needs movement. When a part requires movement, it will wait for the next available idle worker to complete the movement. In this situation, a worker may be busy tending to a part in process at a station or the worker may be busy moving a part between stations. Figure 8.1 illustrates the new situation for the test and repair shop involving the use of workers.

Since workers are required for the processing of parts at the stations and they might have to perform the movement of parts between stations, the workers must be shared between two activities. Thus, when a worker completes one activity a mechanism is needed to indicate which activity should proceed next. A simple mechanism is to assign a priority to one

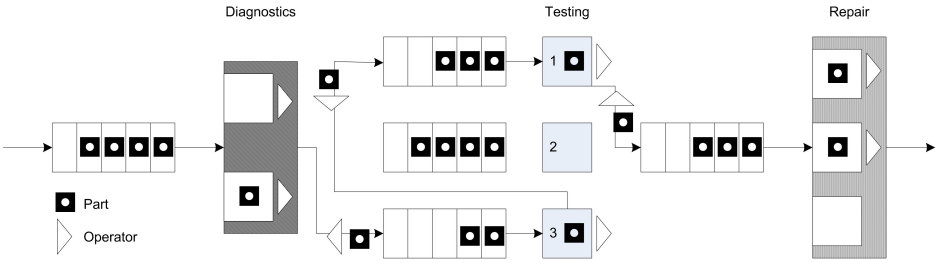


Figure 8.1.: Test and Repair Shop with Workers Providing the Movement

of the tasks. Thus, it seems reasonable to assume that parts waiting for processing at a station are given priority over parts that require movement between stations.

Figure 8.2 illustrates an activity diagram for the situation where the workers are called for the transport. In the figure, each worker is indicated individually. For example, DW1 refers to worker 1 at the diagnostic station. In the figure, the visitation of each part to a different test station is illustrated with a loop back to the transfer time after the testing delay. Thus, the figure represents all three test stations with the test station queue, testing delay, and test machine combination. Unfortunately, this does not explicitly indicate that a test worker is assigned to each test station individually. In particular, the other resources marked TW2 and TW3 should technically have seize and release arrows associated with them for the testing activity at a particular station.

It should be clear from the figure that three pools of resources will be required in this model. A resource pool should be defined for the diagnostic workers with two members. A resource pool should be defined for the three repair workers. Finally, a resource pool should be defined to hold each of the workers DW1, DW2, TW1, TW2, TW3, RW1, RW2, RW3 that are available for transporting parts. Resource pools are discussed in Section 7.1.2.

When a part requires diagnostics, it will seize one of the workers in the

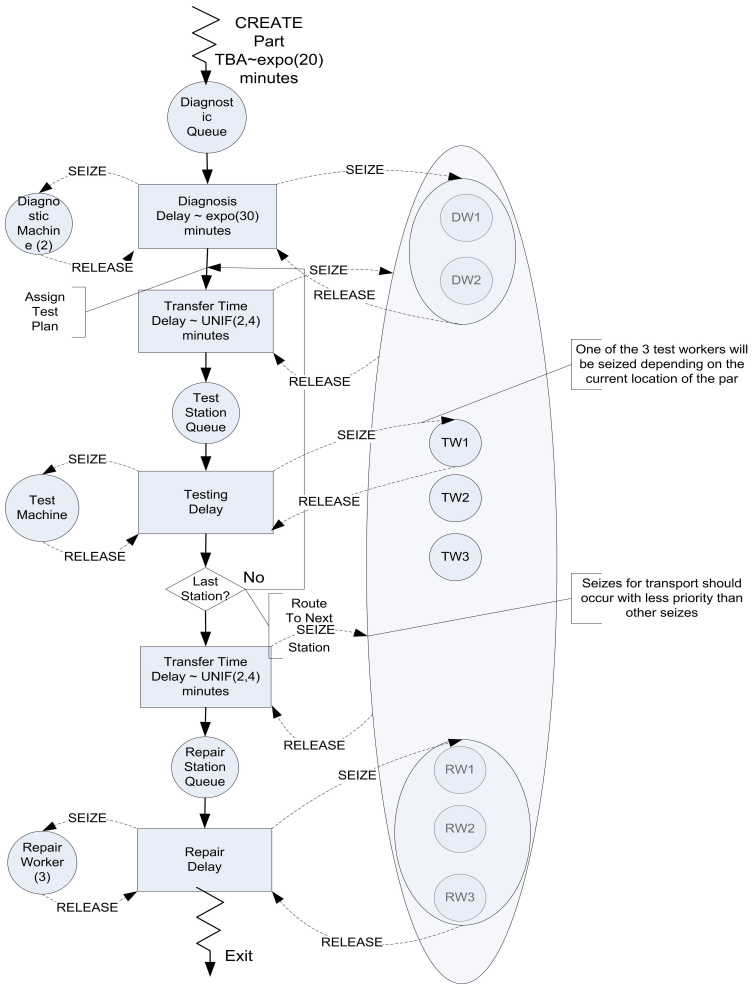


Figure 8.2.: Activity Diagram for Revised Test and Repair Situation

diagnostic workers pool. When a part requires testing, it will seize the appropriate test worker for its current station. Finally, at the repair station, the part will seize one of the repair workers. In order to receive transport, the part will seize from the entire pool of transport workers.

8.1.1.1. Implementing Test and Repair Resource Constrained Transfer

Based on Figure 8.2, the basic process for resource constrained transfer should be something like that shown in the following pseudo-code.

```
SEIZE 1 diagnostic machine
SEIZE 1 diagnostic worker from diagnostic worker set
DELAY for diagnostic time
RELEASE diagnostic machine
RELEASE diagnostic worker

SEIZE 1 worker from worker set
DELAY for transfer time
RELEASE worker

SEIZE appropriate test machine
SEIZE appropriate test worker
DELAY for testing time
RELEASE test machine
RELEASE test worker

SEIZE 1 worker from worker set
IF not at last station
    DELAY for transfer time to next test station
ELSE
    DELAY for transfer time by to Repair
ENDIF
```

```
RELEASE worker
SEIZE repair worker
DELAY for repair time
RELEASE repair worker
```

This logic assumes that each worker has been placed in resource pools. At the diagnostic station both the machine and a diagnostic worker are required. When the part completes the processing at the diagnostic station, the part seizes a worker from the overall set of workers and routes with a transfer delay to the appropriate testing station.

After arriving at the test station, the part releases the worker that performed the transport and proceeds with the seizing of the test machine and worker. Again, prior to the routing to the next station, a worker from the overall set of workers is seized. Notice also that the worker is released after the part is transferred to the repair station.

Now let's take a look at how to modify the test and repair model from Section 7.1.3 to use these new concepts. The key changes involve the definition of the resource pools that involve the workers at each station. Resource pools are discussed in Section 7.1.2. The diagnostic workers can be defined as follows:

```
    private val dw1 = Resource(this, name =
     ↪  "DiagnosticsWorker1")
    private val dw2 = Resource(this, name =
     ↪  "DiagnosticsWorker2")
    private val diagnosticWorkers: ResourcePoolWithQ =
     ↪  ResourcePoolWithQ(
        this,
        listOf(dw1, dw2), name = "DiagnosticWorkersPool"
    )
```

First, the individual resources representing the diagnostic workers are constructed.
Then, the resource pool to hold the diagnostic workers is created by supplying the resources in a list. The testing workers and the repair workers can be defined in a similar manner:

```
    private val tw1 = ResourceWithQ(this, name =
     ↪  "TestWorker1")
    private val myTest1: ResourceWithQ = ResourceWithQ(this,
     ↪  "Test1", capacity = 1)
    private val tw2 = ResourceWithQ(this, name =
     ↪  "TestWorker2")
    private val myTest2: ResourceWithQ = ResourceWithQ(this,
     ↪  "Test2", capacity = 1)
    private val tw3 = ResourceWithQ(this, name =
     ↪  "TestWorker3")
    private val myTest3: ResourceWithQ = ResourceWithQ(this,
     ↪  "Test3", capacity = 1)

    private val rw1 = Resource(this, name = "RepairWorker1")
    private val rw2 = Resource(this, name = "RepairWorker2")
    private val rw3 = Resource(this, name = "RepairWorker3")
    private val repairWorkers: ResourcePoolWithQ =
     ↪  ResourcePoolWithQ(
        this,
        listOf(rw1, rw2, rw3), name = "RepairWorkersPool"
    )

    private val transportWorkers: ResourcePoolWithQ =
     ↪  ResourcePoolWithQ(
        this, listOf(tw1, tw2, tw3, dw1, dw2, rw1, rw2,
         ↪  rw3), name = "TransportWorkersPool"
    )
```

Finally, the resource pool to represent the transport workers can be defined based on the previously declared resources (test workers, diagnostic workers, and repair workers). The remaining declarations for the model are the same as in Section 7.1.3.

Now, the process routine needs to be updated to reflect the use of the pools and the use of transport workers for moving the parts.

```
    private inner class Part : Entity() {
        val testAndRepairProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            wip.increment()
            timeStamp = time
            //every part goes to diagnostics
            val dd1 = seize(diagnosticWorkers) // worker is
             ↪  needed to use the machine
            val d1 = seize(myDiagnosticMachines)
            delay(diagnosticTime)
            release(d1) // release machine first, because
↪  worker is needed to release
            release(dd1)
            val twa = seize(transportWorkers, seizePriority
             ↪  = KSLEvent.MEDIUM_PRIORITY)
            delay(moveTime)
            release(twa)
            // determine the test plan
            val plan: List<TestPlanStep> =
             ↪  planList.randomElement
            // get the iterator
            val itr = plan.iterator()
            // iterate through the plan
            while (itr.hasNext()) {
                val tp = itr.next()
                // visit tester
```

```
                val tt1 = seize(tp.tester)// tester needed
                 ↪ first to work on machine
                val t1 = seize(tp.testMachine)
                delay(tp.processTime)
                release(t1)
                release(tt1)
                val twa1 = seize(transportWorkers,
                 ↪ seizePriority = KSLEvent.MEDIUM_PRIORITY)
                delay(moveTime)
                release(twa1)
            }
            // visit repair
            val rw = seize(repairWorkers)
            delay(repairTimes[plan]!!)
            release(rw)
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

Notice how the workers are seized before the machine at each station.

```
            val dd1 = seize(diagnosticWorkers) // worker is
             ↪ needed to use the machine
            val d1 = seize(myDiagnosticMachines)
            delay(diagnosticTime)
            release(d1) // release machine first, because
↪  worker is needed to release
            release(dd1)
```

Also notice the specification of the use of the transport workers when moving to the next location for testing. The seize request for the transport

workers has medium priority (as compared to the default high priority for normal seizing of resources.

```
val twa = seize(transportWorkers, seizePriority
 ↪  = KSLEvent.MEDIUM_PRIORITY)
delay(moveTime)
release(twa1)
```

Therefore, resource constrained movement can be easily modeled by seizing the required resource for the move; however, the time to move is specified as a delay. The reader is asked to explore the effect of different resource allocation rules on system performance in the exercises. When the modeling of the distance to move is important, then we can use movable resources.

8.2. Constrained Transfer with Movable Resources

A *movable resource* refers to one or more identical resources that can be allocated to an entity for the purpose of moving entities between locations. The travel time between locations depends on distance and speed.

The standard *delay()* function assumes no resource constrained transfer. As indicated in the last section, the situation of a general resource being used for the transfer can be easily modeled. In that situation, the time of the transfer was specified. In the case of movable resources, resource constrained transfer is still being modeled, but in addition, the physical distance of the transfer is explicitly modeled. In essence, rather than specifying a time to transfer, you specify a velocity and a distance for the transfer. From the velocity and distance associated with the transfer, the time of the transfer can be computed. In these situations, the physical device and its movement (with or without the entity) through the system is of key interest within the modeling.

The modeling of a resource that moves needs to include the following aspects:

- movement of the resource (empty) to the origin location for pick up
- movement of the resource (not empty) to the destination location for drop off

Besides these two aspects, there are natural modeling enhancements that might be considered:

- time to load the entity for the pick up
- time to unload the entity for the drop off
- possible movement of the resource to a home location after drop off

The KSL models this situation using movable resources with a spatial model.

8.2.1. Spatial Models

A spatial model represents the physical space associated with the environment. For the purposes of movable resources, a spatial model provides the distance between locations. The KSL supports a variety of spatial models. The simplest spatial model is probably the `DistancesModel` class. Figure 8.3 illustrates the main classes for spatial models. A spatial model holds spatial elements. Spatial elements are objects that implement the `SpatialElementIfc` interface.

The `SpatialModel` class is an abstract base class. The key abstract concepts that need to be implemented by sub-classes are:

```
    /**
     *  The default initial location.
     */
    abstract var defaultLocation: LocationIfc
```

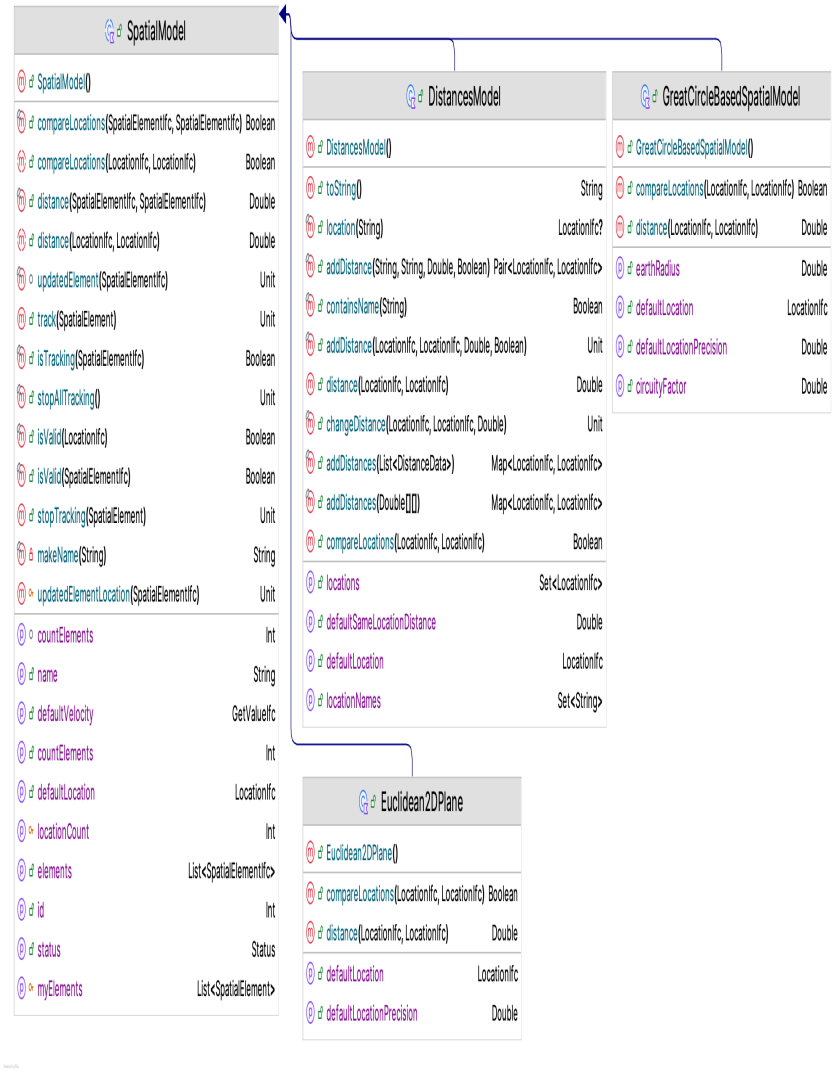


Figure 8.3.: Spatial Models

```
    /**
     * Computes the distance between [fromLocation] and
↪   [toLocation] based on
     * the spatial model's distance metric
     * @return the distance between the two locations
     */
    abstract fun distance(fromLocation: LocationIfc,
     ↪  toLocation: LocationIfc): Double

    /**
     * Returns true if [firstLocation] is the same as
↪   [secondLocation]
     * within the underlying spatial model. This is not
↪   object reference
     * equality, but rather whether the locations within the
↪   underlying
     * spatial model can be considered spatially (equivalent)
↪   according to the model.
     * This may or may not imply that the distance between
↪   the locations is zero.
     * No assumptions about distance are implied by true.
     *
     * Requirement: The locations must be valid within the
↪   spatial model.
     */
    abstract fun compareLocations(firstLocation: LocationIfc,
     ↪  secondLocation: LocationIfc): Boolean
```

The KSL provides basic spatial model implementations for the following situations.

- Distances: The distance between locations is provided based on an

origin-destination matrix.

- Great Circle: A great circle spatial model represents the distance between two coordinates on the earth based on latitude and longitude. It provides an approximate distance of traveling along the great circle between the coordinates. The model implemented within the KSL allows for the adjustment of the distance based on circuity factor. The circuity factor will adjust the distance based on the mode of transport road or rail.
- Euclidean 2D Plane: The distance is based on computing the distance between two points in the Euclidean 2D plane.
- Rectangular Grid: The 2D plane is divided into a grid with the upper left most corner point representing (0,0). The grid is specified based on the width and height divided into a number of rows and columns. This facilitates the moving between the cells of the grid.

The main abstraction for a spatial model is computing the distance between two locations.

A location has a name and an identity. In addition, a location is associated with a spatial model. From the spatial model, a location can compute the distance to other locations within the spatial model. Spatial models are responsible for creating locations and for computing the distance between locations. A spatial model is responsible for defining what constitutes a location. For example, a great circle spatial model uses GPS coordinates to define locations.

As previously mentioned, a basic spatial model is a distances model, which is implemented via the `DistancesModel` class.

In essence, a distance model is a matrix that holds the distances between origins and destinations. Notice from Figure 8.5 that the inner class `Location` implements the `LocationIfc` interface for the distance model. The user is responsible for providing the data associated with the locations. The specification can be in the form of matrix or via the data class

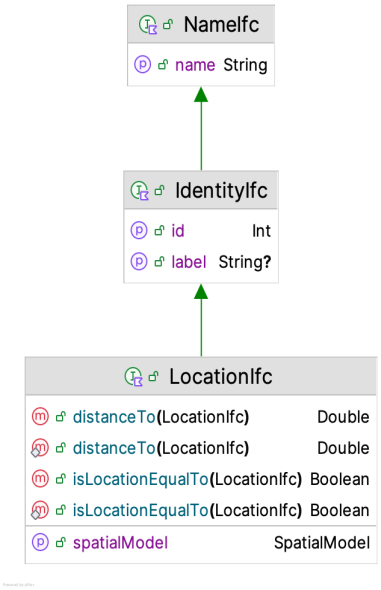


Figure 8.4.: Location Interface

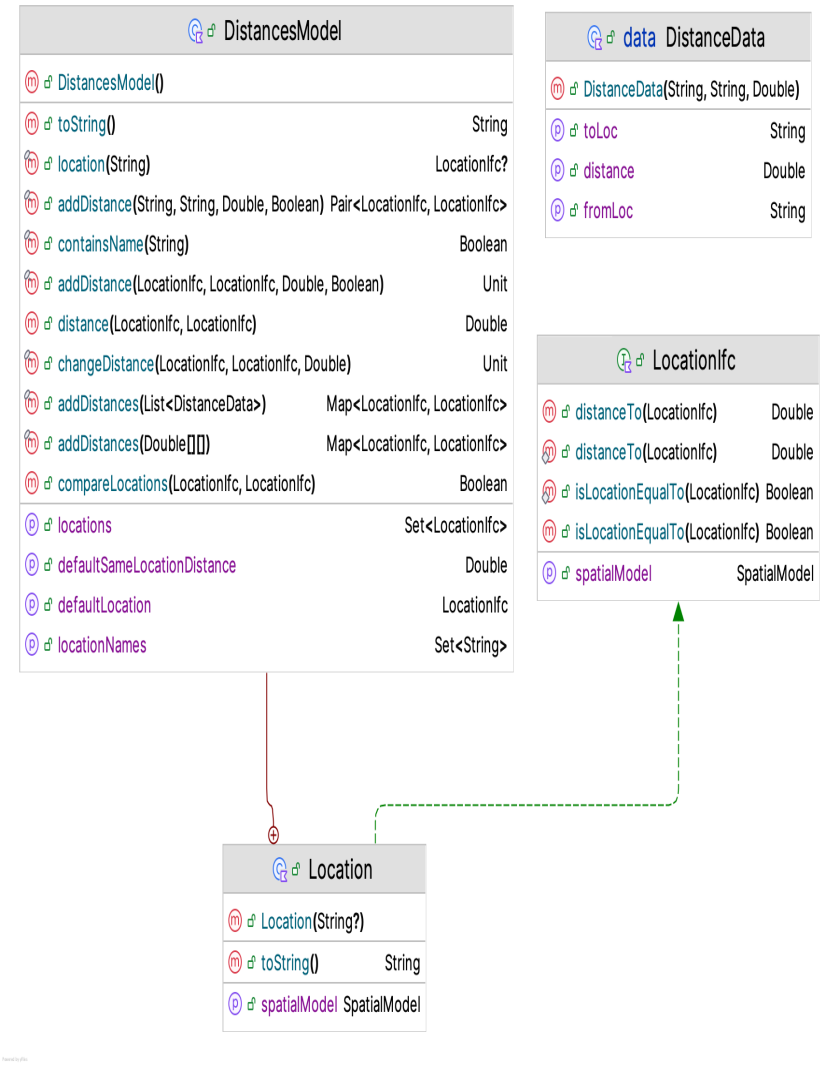


Figure 8.5.: Distances Model

`DistanceData` or individually specified via the `addDistance()` function. The following code illustrates the concepts.

```
    private val dm = DistancesModel()
    private val enter = dm.Location("Enter")
    private val station1 = dm.Location("Station1")
    private val station2 = dm.Location("Station2")
    private val exit = dm.Location("Exit")

    init {
        // distance is in feet
        dm.addDistance(enter, station1, 60.0, symmetric =
↪   true)
        dm.addDistance(station1, station2, 30.0, symmetric =
↪   true)
        dm.addDistance(station2, exit, 60.0, symmetric =
↪   true)
        dm.addDistance(station2, enter, 90.0, symmetric =
↪   true)
        dm.addDistance(exit, station1, 90.0, symmetric =
↪   true)
        dm.addDistance(exit, enter, 150.0, symmetric = true)
        dm.defaultVelocity = myWalkingSpeedRV
        spatialModel = dm
    }
```

In the code, a distances model is created, used to define locations, and then used to specify the distance between locations. As shown in Figure 8.3, a distances model also includes a default velocity because it is a sub-class of the `SpatialModel` class. Now we are ready to discuss movable resources.

8.2.2. Movable Resources

As mentioned, a movable resource is a resource that moves. The class `MovableResource` shown in Figure 8.6 is a sub-class of the `Resource` class, while implementing the `SpatialElementIfc` and `VelocityIfc` interfaces. A spatial element is something that can be within a spatial model. Notice that the `SpatialElementIfc` interface uses locations to specify the initial, previous, and current location of the spatial element.

To use movable resources within a process model, we need to define some new suspending functions and discuss how entities keep track of their location. The `Entity` class implements the `SpatialElementIfc` interface. Some of the key properties that assist with the use of entities within a spatial model include:

- `initialLocation` The location of the entity when it is created specified as a `LocationIfc` interface instance.
- `previousLocation` The previous location of the entity after it has moved to its current location as specified as a `LocationIfc` interface instance.
- `currentLocation` The current location of the entity within the spatial model as specified as a `LocationIfc` interface instance.
- `isMoving` A boolean property that indicates if the entity is experiencing a movement.
- `velocity` A property that implements the `GetValueIfc` interface that reports the velocity to use when the entity is moving between locations.

To have entities use movable resources within a process, we need to introduce new suspending functions. To move an entity from one location to another (without a resource), we can use the following variations of the `move()` function:

- `move(fromLoc: LocationIfc, toLoc: Location, velocity: Double, movePriority: Int, suspensionName: String?)`

Figure 8.6.: Movable Resource

- `move(fromLoc: LocationIfc, toLoc: Location, velocity: GetValueIfc, movePriority: Int, suspensionName: String?)`

To move something that implements the `SpatialElementIfc` interface use the following `move()` function. Note that the entity experiences the delay while the spatial element is moved to the supplied location.

- `move(spatialElement: SpatialElementIfc, toLoc: Location, velocity: Double, movePriority: Int, suspensionName: String?)`

This function uses the spatial element's current location as the origin. Since movable resources are also spatial elements, we can move them. Again, the entity experiences the time delay associated with the movement of the resource to the supplied location.

- `move(movableResource: MovableResource, toLoc: Location, velocity: Double, movePriority: Int, suspensionName: String?)`
- `move(movableResourceWithQ: MovableResourceWithQ, toLoc: Location, velocity: Double, movePriority: Int, suspensionName: String?)`

In addition, we can cause both the entity and a spatial model to move together using variations of the `moveWith()` function.

- `moveWith(spatialElement: SpatialElementIfc, toLoc: Location, velocity: Double, movePriority: Int, suspensionName: String?)`
- `moveWith(movableResource: MovableResource, toLoc: Location, velocity: Double, movePriority: Int, suspensionName: String?)`
- `moveWith(movableResourceWithQ: MovableResourceWithQ, toLoc: Location, velocity: Double, movePriority: Int, suspensionName: String?)`

In order to use these functions, the entity and the spatial element must be at the *same* location. Thus, the standard approach to using a movable resource to move an entity involves first seizing the resource, then moving the resource to the entity's current location, and finally moving with the resource to the destination. Here is a simple pattern:

```
val a = seize(movableResource)
move(movableResource, entity.currentLocation, emptyVelocity,
 ↪  emptyMovePriority)
delay(loadingDelay, loadingPriority)
moveWith(movableResource, toLoc, transportVelocity,
 ↪  transportPriority)
delay(unLoadingDelay, unLoadingPriority)
release(a)
```

In fact, this pattern is so common, there is a suspending function that encapsulates it into one function.

```
   /**
     *  Causes transport of the entity via the movable
 ↪  resource from the entity's current location to the
 ↪  specified location at
     *  the supplied velocities.
     *  If not specified, the default velocity of the movable
 ↪  resource is used for the movement.
     *  @param movableResource, the spatial element that will
 ↪  be moved
     *  @param toLoc the location to which the entity is
 ↪  supposed to move
     *  @param emptyVelocity the velocity associated with the
 ↪  movement to the entity's location
     *  @param transportVelocity the velocity associated with
 ↪  the movement to the desired location
```

```
     *  @param transportQ the queue that the entity waits in
↪  if the resource is busy
     *  @param requestPriority, a priority can be used to
↪  determine the order of events for
     *  requests for transport
     *  @param emptyMovePriority, since the move is
↪  scheduled, a priority can be used to determine the order
↪  of events for
     *  moves that might be scheduled to complete at the same
↪  time.
     *  @param transportPriority, since the move is
↪  scheduled, a priority can be used to determine the order
↪  of events for
     *  moves that might be scheduled to complete at the same
↪  time.
     */
    suspend fun transportWith(
        movableResource: MovableResource,
        toLoc: LocationIfc,
        emptyVelocity: Double =
         ↪  movableResource.velocity.value,
        transportVelocity: Double =
         ↪  movableResource.velocity.value,
        transportQ: RequestQ,
        loadingDelay: GetValueIfc = ConstantRV.ZERO,
        unLoadingDelay: GetValueIfc = ConstantRV.ZERO,
        requestPriority: Int = TRANSPORT_REQUEST_PRIORITY,
        emptyMovePriority: Int = MOVE_PRIORITY,
        loadingPriority: Int = DELAY_PRIORITY,
        transportPriority: Int = MOVE_PRIORITY,
        unLoadingPriority: Int = DELAY_PRIORITY
    )
```

> **Note**
>
> The `transportWith()` function is meant to be a convenience function to cover a standard pattern of usage; however, if your situation varies from the standard pattern, then you can always use the basic `move()` and `moveWith()` functions to model a wide variety of situations.

We are now ready to illustrate entity movement and movement using movable resources.

8.2.3. Tandem Queue Model With Movement

Recall the tandem queue model in Example 7.1. For this section, we are going to enhance the example with movement. In this situation, parts will enter the system at a loading dock that is 60 feet from the station staffed by the first worker. For simplicity, let's assume that there are always plenty of workers available to move the part from the loading dock to the first station. After the part completes processing at station 1, it is moved to station 2. Again, assume that there are plenty of workers available to move the part from station 1 to station 2. The distance from station 1 to station 2 is 30 feet. After completing processing at station 2, the part is moved to a storage area, where it exits the system. The distance from station 2 to the storage area is 60 feet. We can assume that the walking speed of the workers that move the parts is a triangular distributed random variable with a minimum of 88, a mode of 176, and a maximum of 264, all in feet per minute.

To model this situation, we will use the distances spatial model. To implement this situation, we will:

- Use the `Location` class within the `DistancesModel` to define locations
- Use the `addDistance()` function to specify the distance between two locations

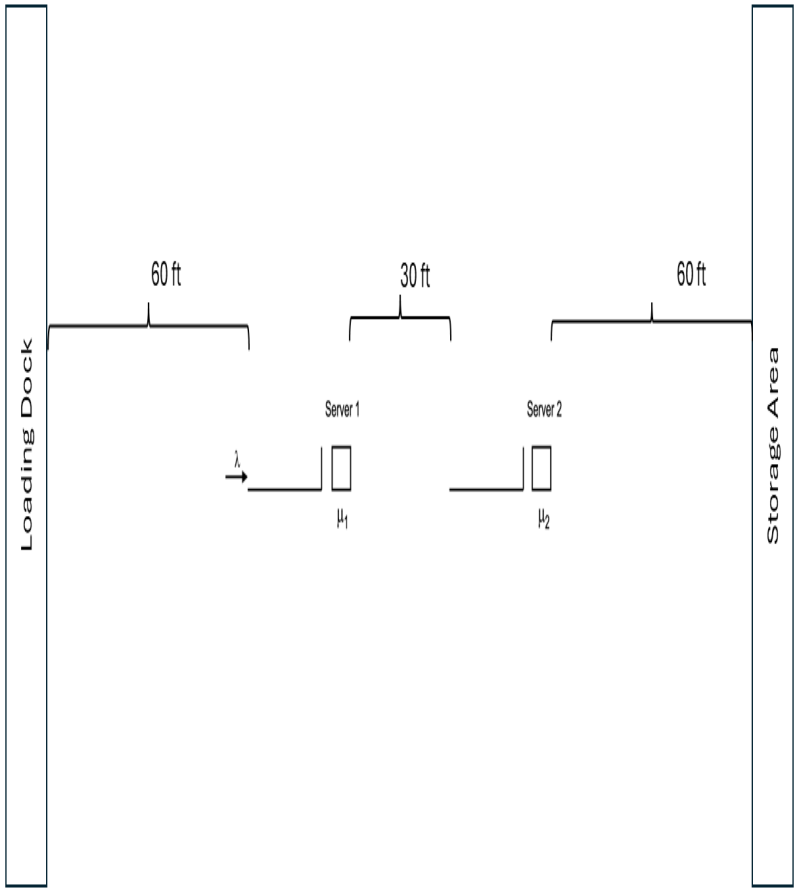


Figure 8.7.: Tandem Queue with Distances

- Use the `spatialModel` property to specify the spatial model used by the `ProcessModel`

Let's start by defining the distances via the `DistancesModel` class. In the following code, the `DistancesModel` instance is created and then used to create the four locations within the spatial model's context. Then, the `init` block is used to add the distance data to the model.

```
class TandemQueueWithUnConstrainedMovement(
  parent: ModelElement,
  name: String? = null
) : ProcessModel(parent, name) {

    // velocity is in feet/min
    private val myWalkingSpeedRV = TriangularRV(88.0, 176.0,
     ↪  264.0)
    private val dm = DistancesModel()
    private val enter = dm.Location("Enter")
    private val station1 = dm.Location("Station1")
    private val station2 = dm.Location("Station2")
    private val exit = dm.Location("Exit")

    init {
        // distance is in feet
        dm.addDistance(enter, station1, 60.0, symmetric =
↪  true)
        dm.addDistance(station1, station2, 30.0, symmetric =
↪  true)
        dm.addDistance(station2, exit, 60.0, symmetric =
↪  true)
        dm.defaultVelocity = myWalkingSpeedRV
        spatialModel = dm
    }
```

Notice that the distances are assumed to be symmetric. That is the distance from the `enter` location to the `station1` location is the same (60 feet) in both directions. The line `dm.defaultVelocity = myWalkingSpeedRV` indicates that whenever a velocity is needed for a movement, the walking speed distribution is used. Finally, the line `spatialModel = dm` causes the process model to use the distances model for its spatial model. This ensures that the entities created within the process model will use the distances model as their spatial model context. Once distances have been defined the tandem queue system movement can be easily handled within the process description.

```
    private inner class Customer : Entity() {
        val tandemQProcess: KSLProcess =
         ↪ process(isDefaultProcess = true) {
            currentLocation = enter
            wip.increment()
            timeStamp = time
            moveTo(station1)
            use(worker1, delayDuration = st1)
            moveTo(station2)
            use(worker2, delayDuration = st2)
            moveTo(exit)
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

Notice the setting of the entity's `currentLocation` in the first line of the process. This line ensures that the entity is located at the `enter` location when it starts the process. The current location property will be updated by the `moveTo()` functions so that after the movement, the entity's location is the destination of the movement. The results of running the model with the additional movement indicate that the movement affects the time spent in the system.

Name	Count	Average	Half-Width
worker1:InstantaneousUtil	30	0.6982	0.0023
worker1:NumBusyUnits	30	0.6982	0.0023
worker1:ScheduledUtil	30	0.6982	0.0023
worker1:WIP	30	2.2966	0.0388
worker1:Q:NumInQ	30	1.5983	0.0371
worker1:Q:TimeInQ	30	1.6001	0.0356
worker2:InstantaneousUtil	30	0.8984	0.0037
worker2:NumBusyUnits	30	0.8984	0.0037
worker2:ScheduledUtil	30	0.8984	0.0037
worker2:WIP	30	8.5321	0.4317
worker2:Q:NumInQ	30	7.6337	0.4289
worker2:Q:TimeInQ	30	7.6378	0.4190
TandemQModel:NumInSystem	30	11.7191	0.4414
TandemQModel:TimeInSystem	30	11.7288	0.4250
worker1:SeizeCount	30	14981.8667	40.6211
worker2:SeizeCount	30	14979.4667	41.3926

In the previous example, we assumed that there was an unlimited number of workers available to move the parts. This results in only a simple delay time to move from one location to another. Now, let's make the more realistic assumption that in order to move a finite number of workers are available such that when a part needs to move between the stations one of the workers dedicated to the transport task must be available to move the part; otherwise the part must wait until a transport worker is available. For simplicity, we are going to assume that their are 3 transport workers, with one dedicated to moving parts to station 1, another dedicated to moving parts from station 1 to station 2, and the third worker dedicated to moving parts from the third workstation to storage. We are also going to assume that the transport worker stays at the drop off location until requested for its next movement.

Because the transport workers stay at the drop off location, we may need to consider two-way travel. Also, we will need to consider where the transport workers are located at the start of the simulation. We are going to assume that the transport workers all start at the `enter` location. These assumptions will require an update to the distances. Why? Because we now need to account for the distance to move empty.

```
    private val myWalkingSpeedRV = TriangularRV(88.0, 176.0,
     ↪  264.0)
    private val dm = DistancesModel()
    private val enter = dm.Location("Enter")
    private val station1 = dm.Location("Station1")
    private val station2 = dm.Location("Station2")
    private val exit = dm.Location("Exit")

    init {
        // distance is in feet
        dm.addDistance(enter, station1, 60.0, symmetric =
↪   true)
        dm.addDistance(station1, station2, 30.0, symmetric =
↪   true)
        dm.addDistance(station2, exit, 60.0, symmetric =
↪   true)
        dm.addDistance(station2, enter, 90.0, symmetric =
↪   true)
        dm.addDistance(exit, station1, 90.0, symmetric =
↪   true)
        dm.addDistance(exit, enter, 150.0, symmetric = true)
        dm.defaultVelocity = myWalkingSpeedRV
        spatialModel = dm
    }
```

Notice that we need to specify the distance from the `station2` location to the `enter` location. This is because the movable resource will be at

the `enter` location at the start of the simulation and will need to move to the `station2` location for a pick up. This movement (as well as others) occur because the resource must move to the pick up location (empty) and because the resource stays at the location of the drop off. Now, we can define the movable resources.

```
private val mover1: MovableResourceWithQ =
 ↪  MovableResourceWithQ(this, enter, myWalkingSpeedRV,
 ↪  "Mover1")
private val mover2: MovableResourceWithQ =
 ↪  MovableResourceWithQ(this, enter, myWalkingSpeedRV,
 ↪  "Mover2")
private val mover3: MovableResourceWithQ =
 ↪  MovableResourceWithQ(this, enter, myWalkingSpeedRV,
 ↪  "Mover3")
```

Notice that we define the resources to have a queue. This queue will hold requests for the resource when more than one part requires a move. The process description can now be updated to use the movable resources. The following code explicitly seizes the movable resource, moves the resource to the location of the part, and then moves with the resource before then releasing the resource.

```
    private inner class Customer : Entity() {
        val tandemQProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            currentLocation = enter
            wip.increment()
            timeStamp = time
            val a1 = seize(mover1)
            move(mover1, toLoc = enter)
            moveWith(mover1, toLoc = station1)
            release(a1)
```

```
            use(worker1, delayDuration = st1)
            val a2 = seize(mover2)
            move(mover2, toLoc = station1)
            moveWith(mover2, toLoc = station2)
            release(a2)
            use(worker2, delayDuration = st2)
            val a3 = seize(mover3)
            move(mover3, toLoc = station2)
            moveWith(mover3, toLoc = exit)
            release(a3)
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

The following code shortens the process by using the `transportWith()` and `use()` functions.

```
    private inner class Customer : Entity() {
        val tandemQProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            currentLocation = enter
            wip.increment()
            timeStamp = time
            transportWith(mover1, station1)
            use(worker1, delayDuration = st1)
            transportWith(mover2, station2)
            use(worker2, delayDuration = st2)
            transportWith(mover3, exit)
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

Suppose that there is a time required to load and unload the part. Let's assume that the loading time is uniformly distributed between 0.5 and 0.8 minutes and the unloading time is uniformly distributed between 0.25 and 0.5 minutes. In addition, suppose that the workers the perform the movement are not dedicated to a particular station, but instead, they are in a pool of workers that may perform the pick up and delivery of the parts.

For this situation, we can use an instance of the `MovableResourcePoolWithQ` class. To define the pool the following code can be used.

```
private val mover1 = MovableResource(this, enter,
 ↪  myWalkingSpeedRV, "Mover1")
private val mover2 = MovableResource(this, enter,
 ↪  myWalkingSpeedRV, "Mover2")
private val mover3 = MovableResource(this, enter,
 ↪  myWalkingSpeedRV, "Mover3")
private val moverList = listOf(mover1, mover2, mover3)
private val movers = MovableResourcePoolWithQ(this,
 ↪  moverList, myWalkingSpeedRV, name = "Movers")
```

By using the `transportWith()` function we can easily add the loading and unloading time delays as follows:

```
private val myLoadingTime = RandomVariable(this,
 ↪  UniformRV(0.5, 0.8))
val loadingTimeRV: RandomSourceCIfc
    get() = myLoadingTime
private val myUnLoadingTime = RandomVariable(this,
 ↪  UniformRV(0.25, 0.5))
val unloadingTimeRV: RandomSourceCIfc
    get() = myUnLoadingTime
```

```
    private inner class Customer : Entity() {
        val tandemQProcess: KSLProcess =
            process(isDefaultProcess = true) {
                currentLocation = enter
                wip.increment()
                timeStamp = time
                transportWith(movers, station1, loadingDelay =
    myLoadingTime, unLoadingDelay = myUnLoadingTime)
                use(worker1, delayDuration = st1)
                transportWith(movers, station2, loadingDelay =
    myLoadingTime, unLoadingDelay = myUnLoadingTime)
                use(worker2, delayDuration = st2)
                transportWith(movers, exit, loadingDelay =
    myLoadingTime, unLoadingDelay = myUnLoadingTime)
                timeInSystem.value = time - timeStamp
                wip.decrement()
        }
    }
```

A pool of movable resources is essentially a "motor pool" or fleet. An importance aspect of modeling a pool of movable resources is how to dispatch the next movable resource. The concept of pooled resources was discussed in Section 7.1.2. In that section, the need for a resource selection rule was briefly mentioned. Within a spatial context, how the next resource is selected to respond to a request may become important. Recall that a resource selection rule determines the list of resources that have enough capacity available to meet the request. In the context of movable resources, the requested capacity is always one unit. Thus, the resource selection rule returns a list of available resources (not seized). Then, from this list a resource must be selected for allocation. The construction of the list of movable resources for possible allocation is governed by the `MovableResourceSelectionRuleIfc` interface.

```
/**
 * Provides for a method to select movable resources from a
 ↪  list such that
 * the returned list will contain movable resources that can
 ↪  satisfy the request
 * or the list will be empty.
 */
fun interface MovableResourceSelectionRuleIfc {
    /**
     * @param list of resources to consider selecting from
     * @return the selected list of resources. It may be
 ↪  empty
     */
    fun selectMovableResources(list: List<MovableResource>):
     ↪  MutableList<MovableResource>
}
```

By default, the movable resource selection rule is provided by the `MovableResourceSelectionRule` class, which simply returns all available units for possible allocation. Since the allocation rules selects from this list, it is likely that the default selection rule will work for the vast majority of situations.

Within a spatial context, the resource selected for allocation may need additional consideration. For example, it may make sense to allocate the resource that is closest to the requesting entity. The KSL supports a variety of movable resource allocation rules via the `MovableResourceAllocationRuleIfc` interface.

```
/**
 *  Function to determine which movable resource should be
 ↪  allocated to
 *  a request. The function provides the location of the
 ↪  request to allow
```

```
 *  distance based criteria to be used.
 */
fun interface MovableResourceAllocationRuleIfc {

    /** The method assumes that the provided list of
     ↪  resources has
     *  enough units available to satisfy the needs of the
 ↪  request.
     *
     * @param requestLocation the location associated with
 ↪  the request. This information can be
     * used to determine the allocation based on distances.
     * @param resourceList list of resources to be allocated
 ↪  from
     * @return the amount to allocate from each resource as a
 ↪  map
     */
    fun selectMovableResourceForAllocation(
        requestLocation: LocationIfc,
        resourceList: MutableList<MovableResource>
    ): MovableResource
}
```

The KSL provides the following movable resource allocation rules.

- `ClosestMovableResourceAllocationRule`: This rule determines which of the available movable resources is closest to the request's location. This is the default rule.
- `FurthestMovableResourceAllocationRule`: This rule determines which of the available movable resources is furthers from the request's location. This rule may be useful to return movable resources from the outskirts of the spatial model to more central activities.

- `RandomMovableResourceAllocationRule`: This rule randomly selects from the available movable resources. This rule may be useful for a fairer distribution of activity across the resources.
- `MovableResourceAllocateInOrderListedRule` This rule returns the first available resource from the selected resources.
- `MovableResourceAllocationRule`: This rule allows a user to determine the order of allocation based on a user defined comparator. The following comparison based rules are based on this approach.
  - `LeastUtilizedMovableResourceAllocationRule`: Allocates to the movable resource that is currently the least utilized (based on the utilization statistics). This can be useful if fairness is a criteria.
  - `LeastSeizedMovableResourceAllocationRule`: Allocates to the movable resource that has been seized (allocated) the least. This can be useful if fairness is a criteria.

The reader will be asked to explore the use of these different rules within the exercises.

8.3. Modeling the Test and Repair System with Movable Resources

The results of the constrained resource analysis in Section 8.1.1 indicated that modeling constrained transfer for the parts in the test and repair system may have a significant effect on the system's ability to meet the contract requirements. This may be due to having the workers share the roles of tending the machines and transporting the parts. The following example investigates whether or not a set of dedicated workers would make sense for this system. In particular, the number of workers to dedicate to the transport task needs to be determined. Since there is a lot of walking involved, the model needs to be more precise in the physical modeling of the situation. This could also allow different layout configurations to be

simulated if the relocation of the stations would make a difference in the efficiency of the system.

To model this situation using movable resources, the distance between the stations and a way to model the velocity of transport are required. Since the workers have a natural variability in the speed of their walking, a model for human walking speed is needed. Based on some time study data, the velocity of a worker walking in the facility has been determined to be distributed according to a triangular distribution with a minimum of 22.86, a mode of 45.72, and a maximum of 52.5, all in meters per minute. Since this distribution will be used in many locations in the model, a random variable should be defined to represent the velocity.

Based on measuring the distance between the stations, the approximate distance between the stations has been determined as given in Table 8.1. Recall that both the loaded and unloaded distances for the movement should be specified. For example, even though no parts are routed from repair to diagnostics, the distance from repair to diagnostics should be given because the worker (transporter) may be at the repair station when something needs to be moved from the diagnostic station. Thus, the worker must walk from the repair station to the diagnostic station (unloaded) in order to pick up the part. Notice also that the distances do not have to be symmetric (i.e. the distance from test 1 to test 2 does not have to be the same as the distance from test 2 to test 1).

Table 8.1.: Transporter Distances between Stations

Station	Diagnostics	Test 1	Test 2	Test 3	Repair
Diagnostics	–	40	70	90	100
Test 1	43	–	10	60	80
Test 2	70	15	–	65	20
Test 3	90	80	60	–	25
Repair	110	85	25	30	–

Starting with the finished model from Example 7.4 of Section 7.1.3 we can make small changes in order to utilize movable resources. The first step is to define the locations and walking speed for the transport workers.

```
    // velocity is in meters/min
    private val myWalkingSpeedRV = RandomVariable(this,
     ↪  TriangularRV(22.86, 45.72, 52.5))
    private val dm = DistancesModel()
    private val diagnosticStation =
     ↪  dm.Location("DiagnosticStation")
    private val testStation1 = dm.Location("TestStation1")
    private val testStation2 = dm.Location("TestStation2")
    private val testStation3 = dm.Location("TestStation3")
    private val repairStation = dm.Location("RepairStation")
```

Since the transport workers need to walk without carrying a part, we need to define the distances to and from the locations as shown in the following code. Notice that the walking speed is used for the default velocity of movement within the distance model.

```
    init {
        // distance is in meters
        dm.addDistance(diagnosticStation, testStation1, 40.0)
        dm.addDistance(diagnosticStation, testStation2, 70.0)
        dm.addDistance(diagnosticStation, testStation3, 90.0)
        dm.addDistance(diagnosticStation, repairStation,
↪   100.0)

        dm.addDistance(testStation1, diagnosticStation, 43.0)
        dm.addDistance(testStation1, testStation2, 10.0)
        dm.addDistance(testStation1, testStation3, 60.0)
        dm.addDistance(testStation1, repairStation, 80.0)
```

```
        dm.addDistance(testStation2, diagnosticStation, 70.0)
        dm.addDistance(testStation2, testStation1, 15.0)
        dm.addDistance(testStation2, testStation3, 65.0)
        dm.addDistance(testStation2, repairStation, 20.0)

        dm.addDistance(testStation3, diagnosticStation, 90.0)
        dm.addDistance(testStation3, testStation1, 80.0)
        dm.addDistance(testStation3, testStation2, 60.0)
        dm.addDistance(testStation3, repairStation, 25.0)

        dm.addDistance(repairStation, diagnosticStation,
↪   110.0)
        dm.addDistance(repairStation, testStation1, 85.0)
        dm.addDistance(repairStation, testStation2, 25.0)
        dm.addDistance(repairStation, testStation3, 30.0)

        dm.defaultVelocity = myWalkingSpeedRV
        spatialModel = dm
    }
```

Now, we can define the transport workers as a pool of movable resources. We will specify that there will be three workers dedicated to the transport task. In this case, the workers will start active at the diagnostics station. For the purposes of this example, we will rely on the default movable resource selection and allocation rules.

```
    private val transportWorkers = MovableResourcePoolWithQ(
        this, 3, diagnosticStation, myWalkingSpeedRV, name =
↪       "TransportWorkerPool")
```

Since this situation involves traveling to and from the stations, we need to make a slight change to the test plan definitions given in Example 7.4

of Section 7.1.3. We need to add the destination location to the test plan sequences.

```
    // define steps to represent a plan, include location
     ↪  information
    inner class TestPlanStep(
        val testMachine: ResourceWithQ,
        val processTime: RandomIfc,
        val testStation: LocationIfc)

    // make all the plans
    private val testPlan1 = listOf(
        TestPlanStep(myTest2, t11, testStation2),
         ↪  TestPlanStep(myTest3, t12, testStation3),
        TestPlanStep(myTest2, t13, testStation2),
↪   TestPlanStep(myTest1, t14, testStation1)
    )
```

Notice that the property `testStation` has been added to the `TestPlanStep` class. This property will hold the destination information for transport from the entity's current location to its destination. The code illustrates how the information for test plan 1 has been updated. Recall from Example 7.4 that the test machine visitation sequence for test plan 1 is $2-3-2-1$. This information has been added to the test plan steps for test plan 1. The other test plans are updated in a similar manner. The process description for the test and repair activities becomes straightforward.

```
    private inner class Part : Entity() {
        val plan: List<TestPlanStep> = planList.randomElement

        val testAndRepairProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
```

```
            currentLocation = diagnosticStation
            wip.increment()
            timeStamp = time
            //every part goes to diagnostics
            use(diagnosticWorkers, delayDuration =
↪   diagnosticTime)
            // get the iterator
            val itr = plan.iterator()
            // iterate through the plan
            while (itr.hasNext()) {
                val tp = itr.next()
                // goto the location
                transportWith(transportWorkers, toLoc =
↪   tp.testStation)
                // use the tester
                use(tp.testMachine, delayDuration =
↪   tp.processTime)
            }
            // visit repair
            transportWith(transportWorkers, toLoc =
↪   repairStation)
            use(repairWorkers, delayDuration =
↪   repairTimes[plan]!! )
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

Notice that the property `plan` is randomly determined when the part is created. The `testAndRepairProcess` has the entity starting at the diagnostic station by setting the part's `currenLocation` property to the diagnostic station location. Then, the part uses the diagnostic workers. Because the test plan has the destination of the next station, the trans-

port is placed at the top of the iteration through the test plan sequence. The `transportWith()` function is used to transport the part using the `transportWorkers` movable resource pool. Then, the appropriate test machine is used for the appropriate processing time for the test plan step. After visiting all the test machines on its sequence the part then transports to the repair station, where it uses the repair workers.

The following provides the results from running the model under the same settings as in Example 7.4.

Statistical Summary Report

Name	Count	Average	Half-Width
DiagnosticWorkers:InstantaneousUtil	10	0.753	0.007
DiagnosticWorkers:NumBusyUnits	10	1.506	0.014
DiagnosticWorkers:ScheduledUtil	10	0.753	0.007
DiagnosticWorkers:Q:NumInQ	10	2.006	0.129
DiagnosticWorkers:Q:TimeInQ	10	39.92	2.475
DiagnosticWorkers:WIP	10	3.512	0.142
Test1:InstantaneousUtil	10	0.858	0.007
Test1:NumBusyUnits	10	0.858	0.007
Test1:ScheduledUtil	10	0.858	0.007
Test1:Q:NumInQ	10	3.007	0.276
Test1:Q:TimeInQ	10	53.326	4.489
Test1:WIP	10	3.866	0.283
Test2:InstantaneousUtil	10	0.781	0.006
Test2:NumBusyUnits	10	0.781	0.006
Test2:ScheduledUtil	10	0.781	0.006
Test2:Q:NumInQ	10	1.651	0.065
Test2:Q:TimeInQ	10	43.608	1.468
Test2:WIP	10	2.432	0.07
Test3:InstantaneousUtil	10	0.866	0.006
Test3:NumBusyUnits	10	0.866	0.006
Test3:ScheduledUtil	10	0.866	0.006

Name	Count	Average	Half-Width
Test3:Q:NumInQ	10	2.613	0.188
Test3:Q:TimeInQ	10	52.011	3.535
Test3:WIP	10	3.48	0.193
RepairWorkers:InstantaneousUtil	10	0.872	0.004
RepairWorkers:NumBusyUnits	10	2.616	0.012
RepairWorkers:ScheduledUtil	10	0.872	0.004
RepairWorkers:Q:NumInQ	10	1.283	0.092
RepairWorkers:Q:TimeInQ	10	25.551	1.724
RepairWorkers:WIP	10	3.899	0.102
TransportWorkerPool:R1:InstantaneousUtil	10	0.224	0.001
TransportWorkerPool:R1:NumBusyUnits	10	0.224	0.001
TransportWorkerPool:R1:ScheduledUtil	10	0.224	0.001
TransportWorkerPool:R1:FracTimeMoving	10	0.224	0.001
TransportWorkerPool:R1:FracTimeTransporting	10	0.147	0.001
TransportWorkerPool:R1:FracTimeMovingEmpty	10	0.077	0.001
TransportWorkerPool:R2:InstantaneousUtil	10	0.159	0.001
TransportWorkerPool:R2:NumBusyUnits	10	0.159	0.001
TransportWorkerPool:R2:ScheduledUtil	10	0.159	0.001
TransportWorkerPool:R2:FracTimeMoving	10	0.159	0.001
TransportWorkerPool:R2:FracTimeTransporting	10	0.104	0.001
TransportWorkerPool:R2:FracTimeMovingEmpty	10	0.054	0
TransportWorkerPool:R3:InstantaneousUtil	10	0.086	0.001
TransportWorkerPool:R3:NumBusyUnits	10	0.086	0.001
TransportWorkerPool:R3:ScheduledUtil	10	0.086	0.001
TransportWorkerPool:R3:FracTimeMoving	10	0.086	0.001
TransportWorkerPool:R3:FracTimeTransporting	10	0.057	0.001
TransportWorkerPool:R3:FracTimeMovingEmpty	10	0.029	0
TransportWorkerPool:NumBusy	10	0.468	0.003
TransportWorkerPool:FractionBusy	10	0.156	0.001
TransportWorkerPool:Q:NumInQ	10	0.001	0
TransportWorkerPool:Q:TimeInQ	10	0.005	0
NumInSystem	10	17.658	0.543

Name	Count	Average	Half-Width
TimeInSystem	10	351.557	9.033
ProbWithinLimit	10	0.819	0.018
DiagnosticWorkers:SeizeCount	10	12540.1	71.465
Test1:SeizeCount	10	14058	116.688
Test2:SeizeCount	10	9442.7	63.431
Test3:SeizeCount	10	12532.6	70.454
RepairWorkers:SeizeCount	10	12527.2	68.882
TransportWorkerPool:R1:SeizeCount	10	23114.5	129.863
TransportWorkerPool:R2:SeizeCount	10	16478.3	88.372
TransportWorkerPool:R3:SeizeCount	10	8977	124.473

As can be seen in the results, the utilization of the three transporters is very low (0.156). Less than three workers are probably needed for the transport task. The reader is asked to explore this issue as an exercise. These results match very closely with the results based on a commercial simulation package discussed in this book..

8.4. Modeling Systems with Conveyors

A conveyor is a track, belt, or some other device that provides movement over a fixed path. Typically, conveyors are used to transport items that have high volumes over short to medium range distances. The speeds of conveyance typically range from 20-80 feet per minute to as fast as 500 feet per minute. Conveyors can be gravity based or powered. The modeling of conveyors can be roughly classified as follows:

Accumulating Items on the conveyor continue to move forward when there is a blockage on the conveyor.

Non-accumulating Items on the conveyor stop when the conveyor stops

Fixed spacing Items on the conveyor have a fixed space between them or the items ride in a bucket or bin.

Random spacing Items are placed on the conveyor in no particular position and take up space on the conveyor.

You have experienced conveyors in some form. For example, the belt conveyor at a grocery store is like an accumulating conveyor. An escalator is like a fixed spaced non-accumulating conveyor (the steps are like a bucket for the person to ride in. People movers in airports are like non-accumulating, random spacing conveyors. When designing systems with conveyors there are a number of performance measures to consider:

Throughput capacity The number of loads processed per time

Delivery time The time taken to move the item from origin to destination

Queue lengths The queues to get on the conveyor and for blockages when the conveyor accumulates

Number of carriers used or the space utilized The number of spaces on the conveyor used

The modeling of conveyors does not necessarily require specialized simulation constructs. For example, gravity based conveyors can be modeled with a deterministic delay. The delay is set to model the time that it takes the entity to fall or slide from one location to another. One simple way to model multiple items moving on a conveyor is to use a resource to model the front of the conveyor with a small delay to load the entity on the conveyor. This allows for spacing between the items on the conveyor. Once on the conveyor the entity releases the front of the conveyor and delays for its move time to the end of the conveyor. At the end of the conveyor, there could be another resource and a delay to unload the conveyor. As long as the delays are deterministic, the entities will not pass each other on the conveyor. If you are not interested in modeling the space taken by the conveyor, then this type of modeling is very reasonable. (Henriksen

and Schriber 1986) discuss a number of ways to approach the simulation modeling of conveyors. When space becomes an important element of the modeling, then the simulation language constructs for conveyors become useful. For example, if there actually is a conveyor between two stations and the space allocated for the length of the conveyor is important to the system operation, you might want to use conveyor related constructs.

A conveyor is a material-handling device for transferring or moving entities along a pre-determined path having fixed pre-defined loading and discharge points. Each entity to be conveyed must wait for sufficient space on the conveyor before it can gain entry and begin its transfer. In essence, simulation modeling constructs for conveyors model the travel path via a mapping of space/distance to resources. Space along the path is divided up into units of resources called cells. The conveyor is then essentially a set of moving cells of equal length. Whenever an entity reaches a conveyor entry point it must wait for a predefined amount of unoccupied and available consecutive cells in order to get on the conveyor.

Figure 8.8 illustrates the idea of modeling a conveyor as a set of contiguous cells representing the space on the conveyor. One way to think of this is like an escalator with each cell being a step.

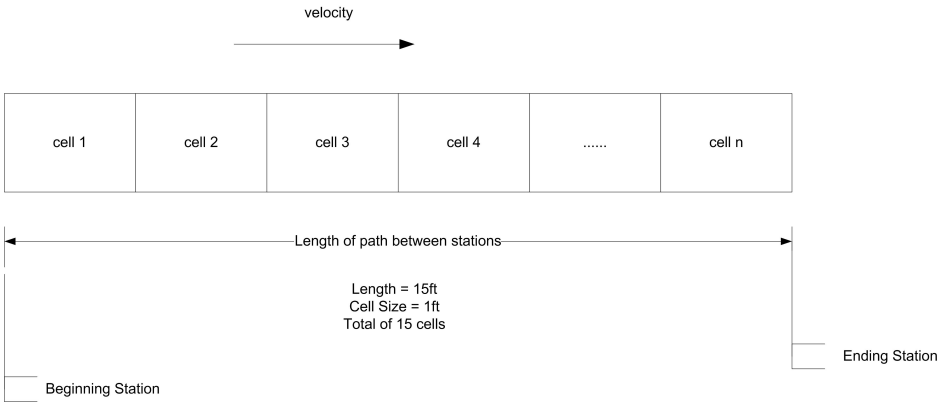


Figure 8.8.: A Conveyor Conceptualized as a Set of Contiguous Cells

In the figure, if each cell represents 1 foot and the total length of the conveyor is 15 feet, then there will be 15 cells. The cell size of the conveyor is the smallest portion of a conveyor that an entity can occupy. In modeling with conveyors, the size of the entity also matters. For example, think of people riding an escalator. Some people have a suitcase and take up two steps and others do not have a suitcase an only take up one step. An entity must acquire enough contiguous cells to hold their physical size in order to be conveyed.

Conveyors travel in a single direction. For non-accumulating conveyors, the spacing remains the same between the entities. When an entity is placed on the conveyor, the entire conveyor is disengaged or stopped until instructions are given to transfer the entity to its destination. When the entity reaches its destination, the entire conveyor is again disengaged until instructions are given to remove the entity from the conveyor, at which time it is engaged or started. Thus, all entities on a non-accumulating conveyor experience the loading and unloading delays of all the other entities.

As previously mentioned, the conveyor is divided into a number of cells. To get on the conveyor and begin moving, the entity must have its required number of contiguous cells. For example, in Figure 8.9, the circle needed 1 cell to get on the conveyor. Suppose the hexagon was trying to get on the conveyor. As cell 2 became available, the hexagon would seize it. Then it would wait for the next cell (cell 1) to become available and seize it. After having both required cells, it is "on" the conveyor. But moving on a conveyor is a bit more continuous than this. Conceptually, you can think of the hexagon, starting to move on the conveyor when it gets cell 2. When one cell moves, all cells must move, in lock step. Since entities are mapped on to the cells by their size, when a cell moves the entity moves. Reversing this analogy becomes useful. Suppose the cells are fixed, but the entities can move. In order for an entity to "take a step" it needs the next cell. As it crosses over to the next cell it releases the previous cell that it occupied. It is as if the entity's "step size" is 1 cell at a time.

Figure 8.10 illustrates an *approximate* activity flow diagram for an entity

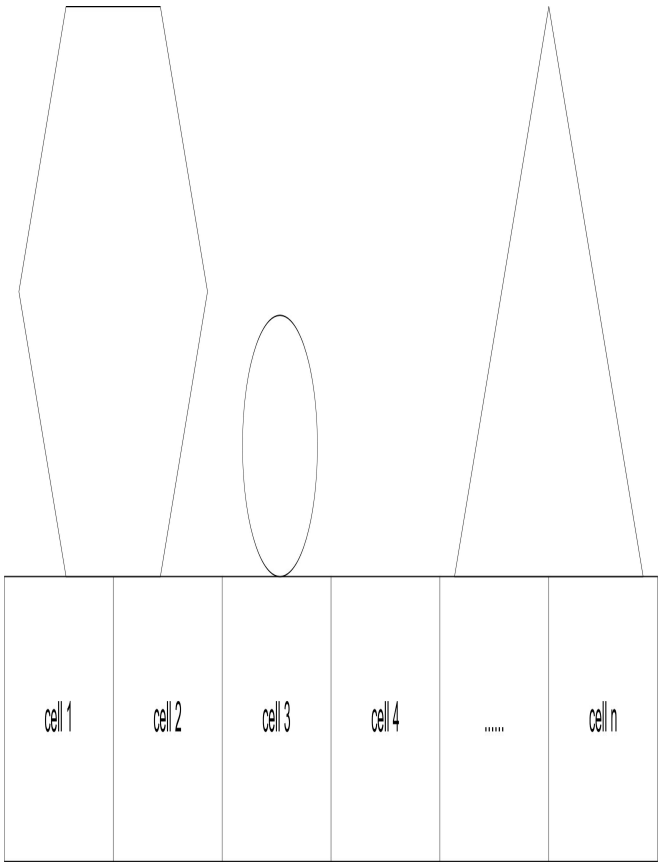


Figure 8.9.: Different Entity Sizes on a Conveyor

using a non-accumulating conveyor. Suppose the entity size is 2 feet and each cell is 1 foot on the conveyor. There is a queue for the entities waiting to access the conveyor. The cells of the conveyor act as resources modeling the space on the conveyor. The movements of the entity through the space of each cell are activities. For the 2 foot entity, it first seizes the first cell and then moves through the distance of the cell. After the movement it seizes the next cell and moves through its length. Because it is 2 feet (cells) long it seizes the 3rd cell and then releases the first cell before delaying for the move time for the 3rd cell. As cells become released other entities can seize them and continue their movement. The repeated pattern of overlapping `seize()` and `release()` function calls that underlie conveyor modeling clearly depends upon the number of cells and the size of the entity. Thus, the activity diagram would actually vary by the size of the entity (number of cells required).

The larger the number of cells to model a segment of a conveyor the more slowly the model will execute; however, a larger number of cells allows for a more "continuous" representation of how entities actually exit and leave the conveyor. Larger cell sizes force entities to delay longer to move and thus waiting entities must delay for available space longer. A smaller mapping of cells to distance allows then entity to"creep" onto the conveyor in a more continuous fashion. For example, suppose the conveyor was 6 feet long and space was modeled in inches. That is, the length of the conveyor is 72 inches. Now suppose the entity required 1 foot of space while on the conveyor. If the conveyor is modeled with 72 cells, the entity starts to get on the conveyor after only 1 inch (cell) and requires 12 cells (inches) when riding on the conveyor. If the mapping of distance was 1 cell equals 1 foot, the entity would have to wait until it got a whole cell of 1 foot before it moved onto the conveyor.

An accumulating conveyor can be thought of as always running. When an entity stops moving on the conveyor (e.g. to unload), other entities are still allowed on the conveyor since the conveyor continues moving. When entities on the conveyor meet up with the stopped entity, they stop, and a

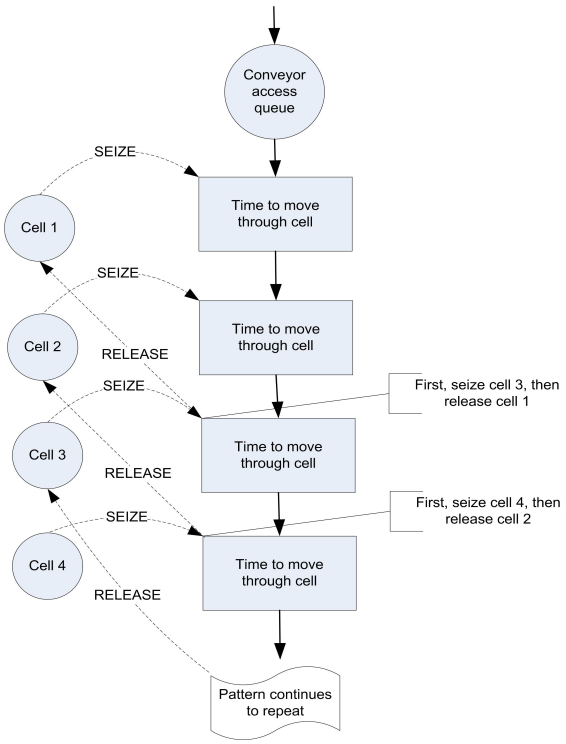


Figure 8.10.: Activity Diagram for Non-Accumulating Conveyor

queue (on the conveyor) accumulates behind the stopped entity until the original stopped entity is removed or transferred to its destination.

As an analogy, imagine wearing a pair of roller blades and being on a people mover in an airport. Don't ask how you got your roller blades through security! While on the people mover you aren't skating (you are standing still, resting, but still moving with the people mover). Up ahead of you, some seriously deranged simulation book author places a bar across the people mover. When you reach the bar, you grab on. Since you are on roller blades, you remain stationary at the bar, while your roller blades are going like mad underneath you.

Now imagine all the people on the mover having roller blades. The people following you will continue approaching the bar until they bump into you. Everyone will pile up behind the bar until the bar is removed. You are on an accumulating conveyor! Notice that while you were on the conveyor and there was no blockage, the spacing between the people remained the same, but that when the blockage occurred the spacing between the entities decreased until they bump up against each other. To summarize:

- Accumulating conveyors are always moving.
- If an entity stops to exit or receives processing on the conveyor, other entities behind it are blocked and begin to queue up on the conveyor.
- Entities in front of the blockage continue moving.
- When a blockage ends, blocked entities, may continue, but must first wait for the entities immediately in front of them to move forward.

In modeling accumulating conveyors, the main differences occur in the actions of how the entity accesses and exits the conveyor. Instead of disengaging the conveyor as with non-accumulating conveyors the conveyor continues to run. The conveyor allocates the required number of cells to any waiting entities as space becomes available. Any entities that are being conveyed continue until they reach a blockage. If the blocking entity is

removed or conveyed, the accumulated entities only start to move when the required number of cells becomes available.

8.4.1. KSL Conveyor Constructs

The KSL provides suspending functions and classes to support the modeling of conveyors.

A conveyor consists of a series of segments. A segment has a starting location (origin) and an ending location (destination) and is associated with a conveyor. The start and end of each segment represent locations along the conveyor where entities can enter and exit. A conveyor does not utilize a spatial model. The locations associated with its segments are instances of the `String` class. Thus, movement on the conveyor is disassociated with any spatial model associated with the entity using the conveyor. Since a spatial model using instances of the `LocationIfc` interface and `LocationIfc` extends the `IdentityIfc.` the locations associated with segments can be the same as those associated with spatial models because the `IdentityIfc` has a `String` property called `name;` however, the entity's location attributes expect instances of the `LocationIfc` interface. The modeler is responsible for making the update to the entity location, if applicable.

A conveyor has a cell size, which represents the length of each cell on all segments of the conveyor. A conveyor also has a maximum permitted number of cells that can be occupied by an item riding on the conveyor. A conveyor has an initial velocity. Each segment moves at the same velocity.

Each segment has a specified length that is divided into a number of equally sized contiguous cells. The length of any segment must be an integer multiple of the conveyor's cell size so that each segment will have an integer number of cells to represent its length. If a segment consists of 12 cells and the length of the conveyor is 12 feet then each cell represents 1 foot. If a segment consists of 4 cells and the length of the segment is 12

feet, then each cell of the segment represents 3 feet. Thus, a cell represents a generalized unit of distance along the segment. Segments may have a different number of cells because they may have different lengths.

The cells on a conveyor are numbered from 1 to n, with 1 at the entry of the first segment, and n at the exit of the last segment, where n is the number of cells for the conveyor. Items that ride on the conveyor must be allocated cells and then occupy the cells while moving on the conveyor. Items can occupy more than one cell while riding on the conveyor. For example, if the conveyor has 5 cells (1, 2, 3, 4, 5) and the item needs 2 cells and is occupying cells 2 and 3, then the front cell associated with the item is cell 3 and the rear cell associated with the item is cell 2.

An entity trying to access the conveyor at an entry cell of the conveyor, waits until it can block the entry cell. A request for entry on the conveyor will wait for the entry cell if the entry cell is unavailable (blocked or occupied) or if another item is positioned to use the entry cell. Once the entity has control of the entry cell, this creates a blockage on the conveyor, which may restrict conveyor movement. For a non-accumulating conveyor the blockage stops all movement on the conveyor. For accumulating conveyors, the blockage restricts movement behind the blocked cell. Gaining control of the entry cell does not position the entity to ride on the conveyor. The entity simply controls the entry cell and causes a blockage. The entity is not on the conveyor during the blockage.

If the entity decides to ride on the conveyor, the entity will be allocated cells based on its request and occupy those cells while moving on the conveyor. First, the entity's request is removed from the request queue and positioned to enter the conveyor at the entry cell. To occupy a cell, the entity must move the distance represented by the cell (essentially covering the cell). Thus, an entering entity takes the time needed to move through its required cells before being fully on the conveyor. An entity occupies a cell during the time it traverses the cell's length. Thus, assuming a single item, the time to move from the start of a segment to the end of

the segment is the time that it takes to travel through all the cells of the segment, including the entry cell and the exit cell.

When an entity riding on the conveyor reaches its destination (an exit cell), the entity causes a blockage on the conveyor. For a non-accumulating conveyor, the entire conveyor stops. For an accumulating conveyor, the blockage restricts movement behind the blockage causing items behind the blockage to continue to move until they cannot move forward. When an entity exits the conveyor, there will be a delay for the entity to move through the cells that it occupies before the entity can be considered completely off of the conveyor. Thus, exiting the conveyor is not assumed to be instantaneous. This implementation assumes that the entity must move through the occupied cells to get off the conveyor. Any delay to unload the item from the conveyor will be in addition to the exiting delay. Thus, a modeling situation in which the entity is picked up off the conveyor may need to account for the "extra" exiting delay. This implementation basically assumes that the item is pushed through the occupying cells at the end of the conveyor in order to exit. Scenarios where the item is picked up would not necessarily require the time to move through the cells.

A conveyor is considered circular if the entry location of the first segment is the same as the exit location of the last segment. The user specifies a conveyor by providing the information associated with the segments. The specification of a conveyor can be performed by using the `ConveyorSegments` class or and the `Segment` class, or via a builder companion object of the `Conveyor` class. Figure 8.11 illustrates the `ConveyorSegments` class and the `Segment` class.

The `ConveyorSegments` class provides the data necessary to specify the segments of the conveyor. While the `Segment` class is a data class that can be used to build up the segments. The constructor of the `Conveyor` class takes in an instance of the `ConveyorSegments` class. Essentially, the `ConveyorSegments` class is a specialized collection that holds instances of the `Segment` class.

Figure 8.11.: ConveyorSegments and Segment Classes

```
class Conveyor(
    parent: ModelElement,
    segmentData: ConveyorSegments,
    val conveyorType: Type = Type.ACCUMULATING,
    velocity: Double = 1.0,
    val maxEntityCellsAllowed: Int = 1,
    name: String? = null
) : ModelElement(parent, name)
```

Notice from Figure 8.11 that the conveyor segments provide information about the total length of the conveyor, the first location (entry) and the last location (exit). Also available is a list of the exit locations and whether the conveyor is circular. A conveyor is circular if the exit location is connected to the entry location.

While the conveyor can be specified directly via the conveyor segments, a builder can be used to construct the conveyor. An example of builder code to make a circular conveyor is as follows:

```
        loopConveyor = Conveyor.builder(this, "LoopConveyor")
            .conveyorType(Conveyor.Type.NON_ACCUMULATING)
            .velocity(30.0)
            .cellSize(10)
            .maxCellsAllowed(2)
            .firstSegment(myDrillingResource.name,
             ↪  myMillingResource.name, 70)
            .nextSegment(myPlaningResource.name, 90)
            .nextSegment(myGrindingResource.name, 50)
            .nextSegment(myInspectionResource.name, 180)
            .nextSegment(myDrillingResource.name, 250)
            .build()
```

Notice that the `myDrillingResource` location is both the first and last

location in the conveyor. Also, notice that the *name* of the model element is being used to define the segments because this provides a unique string.

Another important interface that is involved with conveyor modeling is the `ConveyorRequestIfc` interface as shown in Figure 8.12.

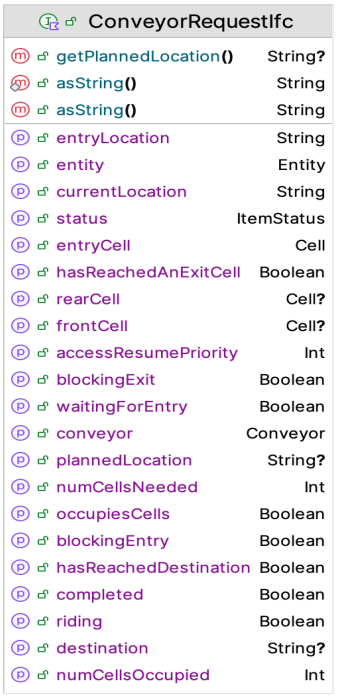


Figure 8.12.: Conveyor Requests

Similar to the `Allocation` class when using resources, an entity having control of cells on a conveyor needs an instance of the `ConveyorRequestIfc` interface. Besides providing information about the request and cells associated with using a conveyor, instances of this interface essentially act as a 'ticket' to ride on the conveyor and are necessary when exiting the conveyor.

The suspending functions associated with using a conveyor are as follows.

- `requestConveyor(conveyor: Conveyor, entryLocation: String, numCellsNeeded: Int)`: This function returns an instance of the `ConveyorRequestIfc` interface. This suspending function requests the number of cells indicated at the entry location of the conveyor. If the number of cells are not immediately available the process is suspended until the number of cells can be allocated (in full). The request for the cells will wait for the allocation in the queue associated with the start of the segment associated with the entry location of the conveyor. After this suspending function returns, the entity holds the cells in the returned cell allocation, but the entity is not on the conveyor. The entity can then decide to ride on the conveyor using the cell allocation or release the cell allocation by exiting the conveyor without riding. The behavior of the conveyor during access is governed by the type of conveyor. A blockage occurs at the entry point of the segment while the entity has the allocated cells and before exiting or riding.
- `rideConveyor(conveyorRequest: ConveyorRequestIfc, destination: String)`: This suspending function causes the entity to be associated with an item that occupies the allocated cells on the conveyor. The item will move on the conveyor until it reaches the supplied destination. After this suspending function returns, the item associated with the entity will be occupying the cells it requires at the exit location of the segment associated with the destination. The item will remain on the conveyor until the entity indicates that the cells are to be released by using the exit function. The behavior of the conveyor during the ride and when the item reaches its destination is governed by the type of conveyor. A blockage occurs at the destination location of the segment while the entity occupies the final cells before exiting or riding again. If the destination implements the `LocationIfc` interface then the current location property of the entity will be updated to this value at the completion

of the ride.

- `exitConveyor(conveyorRequest: ConveyorRequestIfc)`: This suspending function causes the item associated with the allocated cells to exit the conveyor. If there is no item associated with the allocated cells, the cells are immediately released without a time delay. If there is an item occupying the associated cells there will be a delay while the item moves through the deallocated cells and then the cells are deallocated. After exiting the conveyor, the cell allocation is deallocated and cannot be used for further interaction with the conveyor.
- `convey(conveyor: Conveyor, entryLocation: String, destination: String, numCellsNeeded: Int)`: This suspending function combines the `requestConveyor()`, `rideConveyor()`, and `exit()` functions into one suspending function.
- `transferTo(conveyorRequest: ConveyorRequestIfc, nextConveyor: Conveyor, entryLocation: String,)` This suspending function causes the entity to transfer from one conveyor to another. The entity will suspend while accessing the required cells at the next conveyor. Once the desired cells are obtained the entity will exit its current conveyor and be positioned to ride on the next conveyor.

Now we are ready to put these concepts into action.

8.4.2. Tandem Queue System with Conveyors

This section revisits the tandem queue system of previous sections and explores the use of conveyors to transport the parts. For this section, we will assume the distances used in Section 8.2.2 as still being relevant. Figure 8.13 illustrates the situation.

For this situation, we will assume that the conveyor speed is 30 feet per minute. There is one conveyor with three segments. The first segment will

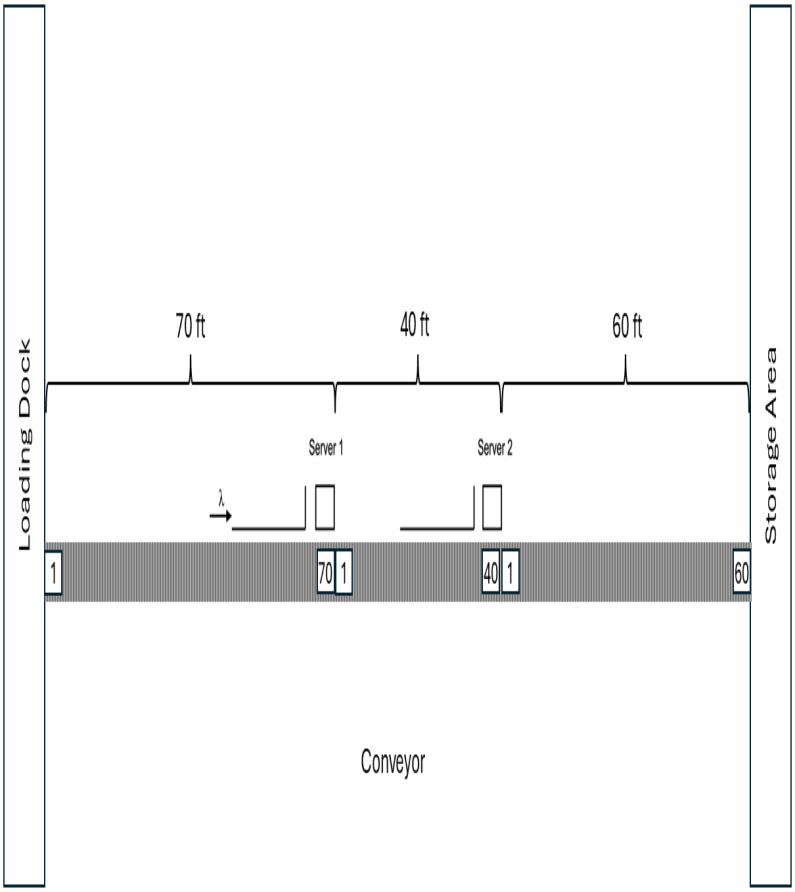


Figure 8.13.: Tandem Queue with Conveyors

be 70 feet, the second segment will be 40 feet, and the third segment 60 feet. The beginning of the first segment starts at the loading dock and ends at the resource of the first station. The second segment starts at the end of the firs segment and goes to the resource of the second station. Finally, the third segment, goes from the resource at the second station to the store area. Initially, we will assume that the part is removed from the conveyor for the work to occur at the station. The time to remove the part from the conveyor and to place it back on the conveyor after processing is considered to be negligible.

We are going to show multiple models of this situation to compare and contrast different ways to model with conveyors.

- Model the system with simple delays acting as the conveyor.
- Model the system with the part exiting the conveyor for processing at each station using both accumulating and non-accumulating conveyors.
- Model the system with the part staying on the conveyor for processing at each station using both accumulating and non-accumulating conveyors.

8.4.2.1. Tandem Queue System with Conveyors and Deterministic Delays

To model the system with simple delays to represent the conveyor, we can use deterministic delays to represent the time to travel on the conveyor from location to location.

```
    private inner class Part : Entity() {
        val tandemQProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            wip.increment()
            timeStamp = time
```

```
            delay(delayDuration = 70.0/30.0) // 30 fpm for 70
↪ ft
            use(resource = worker1, delayDuration = st1)
            delay(delayDuration = 40.0/30.0) // 30 fpm for 40
↪ ft
            use(resource = worker2, delayDuration = st2)
            delay(delayDuration = 60.0/30.0) // 30 fpm for 60
↪ ft
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

Using a simple deterministic delay works reasonably well to represent the time of the part in the system. This approximation is reasonable if we do not care about the utilization of the conveyor in terms of space. Because the delay is deterministic, the sequence of parts that enter the conveyor is the same that exits at the stations. The results are useful for a baseline for more detailed modeling.

Conveyor via Deterministic Delay: Statistical Summary Report

Name	Count	Average	Half-Width
worker1:InstantaneousUtil	30	0.698	0.002
worker1:NumBusyUnits	30	0.698	0.002
worker1:ScheduledUtil	30	0.698	0.002
worker1:Q:NumInQ	30	1.598	0.037
worker1:Q:TimeInQ	30	1.6	0.035
worker1:WIP	30	2.296	0.039
worker2:InstantaneousUtil	30	0.901	0.004
worker2:NumBusyUnits	30	0.901	0.004
worker2:ScheduledUtil	30	0.901	0.004
worker2:Q:NumInQ	30	7.791	0.454

Name	Count	Average	Half-Width
worker2:Q:TimeInQ	30	7.795	0.44
worker2:WIP	30	8.692	0.457
TandemQueueWithConveyor:NumInSystem	30	16.648	0.474
TandemQueueWithConveyor:TimeInSystem	30	16.662	0.444
worker1:SeizeCount	30	14982.233	40.422
worker2:SeizeCount	30	14978.633	40.03

8.4.2.2. Tandem Queue System with Conveyors Work Performed Off the Conveyor

Now, we will model the system using accumulating and non-accumulating conveyors and have the parts exit the conveyor and request the conveyor at each location. Let's start by defining the conveyor. The basic tandem queueing system was modified to define the locations and conveyor with its segments.

```
class TandemQueueWithConveyors(
    parent: ModelElement,
    conveyorType: Conveyor.Type =
     ↪  Conveyor.Type.NON_ACCUMULATING,
    name: String? = null
) : ProcessModel(parent, name) {

    private val enter = "Enter"
    private val station1 = "Station1"
    private val station2 = "Station2"
    private val exit = "Exit"

    // velocity is in feet per min
    private val conveyor = Conveyor.builder(this, "Conveyor")
        .conveyorType(conveyorType)
```

```
        .velocity(30.0)
        .cellSize(1)
        .maxCellsAllowed(1)
        .firstSegment(enter, station1, 70)
        .nextSegment(station2, 40)
        .nextSegment(exit, 60)
        .build()
```

The velocity of the conveyor is 30 feet per minute. The basic cell size is 1 unit and the maximum number of cells that any part can occupy is one. The conveyor has three segments of 70, 40, and 60 feet, respectively. The `conveyorType` will be specified as accumulating or non-accumulating. The process description is a straight-forward use of the previously mentioned conveyor suspending functions.

```
    private inner class Part : Entity() {
        val tandemQProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            wip.increment()
            timeStamp = time
            val conveyorRequest = requestConveyor(conveyor =
             ↪  conveyor, entryLocation = enter,
             ↪  numCellsNeeded = 1)
            rideConveyor(conveyorRequest = conveyorRequest,
↪  destination = station1)
            exitConveyor(conveyorRequest)
            use(resource = worker1, delayDuration = st1)
            val conveyorRequest2 = requestConveyor(conveyor
             ↪  = conveyor, entryLocation = station1,
             ↪  numCellsNeeded = 1)
            rideConveyor(conveyorRequest = conveyorRequest2,
↪  destination = station2)
            exitConveyor(conveyorRequest = conveyorRequest2)
```

```
            use(resource = worker2, delayDuration = st2)
            val conveyorRequest3 = requestConveyor(conveyor
             ↪  = conveyor, entryLocation = station2,
             ↪  numCellsNeeded = 1)
            rideConveyor(conveyorRequest = conveyorRequest3,
↪  destination = exit)
            exitConveyor(conveyorRequest = conveyorRequest3)
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

Notice how the conveyor request is captured and used within the ride and exit functions. If there is no need to use the conveyor request instances, then by using the `convey()` function, this process description can be significantly shortened.

```
        val tandemQProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            wip.increment()
            timeStamp = time
            convey(conveyor = conveyor, entryLocation =
↪  enter, destination = station1)
            use(resource = worker1, delayDuration = st1)
            convey(conveyor = conveyor, entryLocation =
↪  station1, destination = station2)
            use(resource = worker2, delayDuration = st2)
            convey(conveyor = conveyor, entryLocation =
↪  station2, destination = exit)
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
```

To specify that the conveyor is non-accumulating use `Conveyor.Type.NON_ACCUMULATING` and for accumulating use `Conveyor.Type.ACCUMULATING` when constructing the conveyor. The results for the non-accumulating case are as follows.

Non-Accumulating Working Off Conveyor: Statistical Summary Report

Name	Count	Average	Half-Width
Conveyor:Enter:AccessQ:NumInQ	30	0	0
Conveyor:Enter:AccessQ:TimeInQ	30	0	0
Conveyor:Station1:AccessQ:NumInQ	30	0	0
Conveyor:Station1:AccessQ:TimeInQ	30	0	0
Conveyor:Station2:AccessQ:NumInQ	30	0	0
Conveyor:Station2:AccessQ:TimeInQ	30	0	0
Conveyor:NumOccupiedCells	30	5.659	0.015
Conveyor:CellUtilization	30	0.033	0
worker1:InstantaneousUtil	30	0.698	0.002
worker1:NumBusyUnits	30	0.698	0.002
worker1:ScheduledUtil	30	0.698	0.002
worker1:Q:NumInQ	30	1.597	0.037
worker1:Q:TimeInQ	30	1.599	0.035
worker1:WIP	30	2.296	0.039
worker2:InstantaneousUtil	30	0.901	0.004
worker2:NumBusyUnits	30	0.901	0.004
worker2:ScheduledUtil	30	0.901	0.004
worker2:Q:NumInQ	30	7.79	0.454
worker2:Q:TimeInQ	30	7.794	0.44
worker2:WIP	30	8.692	0.457
TandemQueueWithConveyor:NumInSystem	30	16.699	0.474
TandemQueueWithConveyor:TimeInSystem	30	16.713	0.444
worker1:SeizeCount	30	14982.2	40.432
worker2:SeizeCount	30	14978.6	40.033

The first thing to notice is that there are automatic statistics reported for the queues associated with the entry points of the conveyor. In addition, statistics are reported for the number of occupied cells and utilization of the cells for each segment. Finally, the conveyor's overall cell utilization and number of occupied cells is reported. The results reported here are not very interesting. We see little waiting to get on the conveyor at the enter, station 1, and station 2 entry points. The utilization of the workers and queueing at the stations is almost the same as in the previous results. We notice that the time spent in the system is about 1 minute higher. This is likely due to the time to go through the cells to enter and exit the conveyor segments.

The same model can be executed with an accumulating conveyor by changing the conveyor type property of the conveyor. The results are presented here.

Accumulating Working Off the Conveyor: Statistical Summary Report

Name	Count	Average	Half-Width
Conveyor:Enter:AccessQ:NumInQ	30	0	0
Conveyor:Enter:AccessQ:TimeInQ	30	0	0
Conveyor:Station1:AccessQ:NumInQ	30	0	0
Conveyor:Station1:AccessQ:TimeInQ	30	0	0
Conveyor:Station2:AccessQ:NumInQ	30	0	0
Conveyor:Station2:AccessQ:TimeInQ	30	0	0
Conveyor:NumOccupiedCells	30	5.659	0.015
Conveyor:CellUtilization	30	0.033	0
worker1:InstantaneousUtil	30	0.698	0.002
worker1:NumBusyUnits	30	0.698	0.002
worker1:ScheduledUtil	30	0.698	0.002
worker1:Q:NumInQ	30	1.597	0.037
worker1:Q:TimeInQ	30	1.599	0.035
worker1:WIP	30	2.296	0.039

Name	Count	Average	Half-Width
worker2:InstantaneousUtil	30	0.901	0.004
worker2:NumBusyUnits	30	0.901	0.004
worker2:ScheduledUtil	30	0.901	0.004
worker2:Q:NumInQ	30	7.79	0.454
worker2:Q:TimeInQ	30	7.794	0.44
worker2:WIP	30	8.692	0.457
TandemQueueWithConveyor:NumInSystem	30	16.699	0.474
TandemQueueWithConveyor:TimeInSystem	30	16.713	0.444
worker1:SeizeCount	30	14982.2	40.432
worker2:SeizeCount	30	14978.6	40.033

The results for the accumulating and non-accumulating conveyors are essentially the same. Each part is on the conveyor only while it travels between stations, and the conveyor is lightly used (about 3% of its cells are occupied). Thus, the brief stops of the non-accumulating conveyor while a part enters or exits rarely delay another part. The total time in the system is not statistically different from the deterministic delay case.

8.4.2.3. Tandem Queue System with Conveyors Work Performed On the Conveyor

In many systems, the work associated with a station can often be performed while the part is on the conveyor. As we will see, more care is needed in synchronizing the movement on the conveyor with the work content. This section illustrates how to change the process when the work is performed on the conveyor. Figure 8.14 illustrates that the worker is working on the part while it is on the last cells of the segments (cell 70 of segment 1 and cell 40 of segment 2).

To model this situation, we simply do not exit the conveyor at each station. The configuration of the conveyor remains the same. Notice that in the

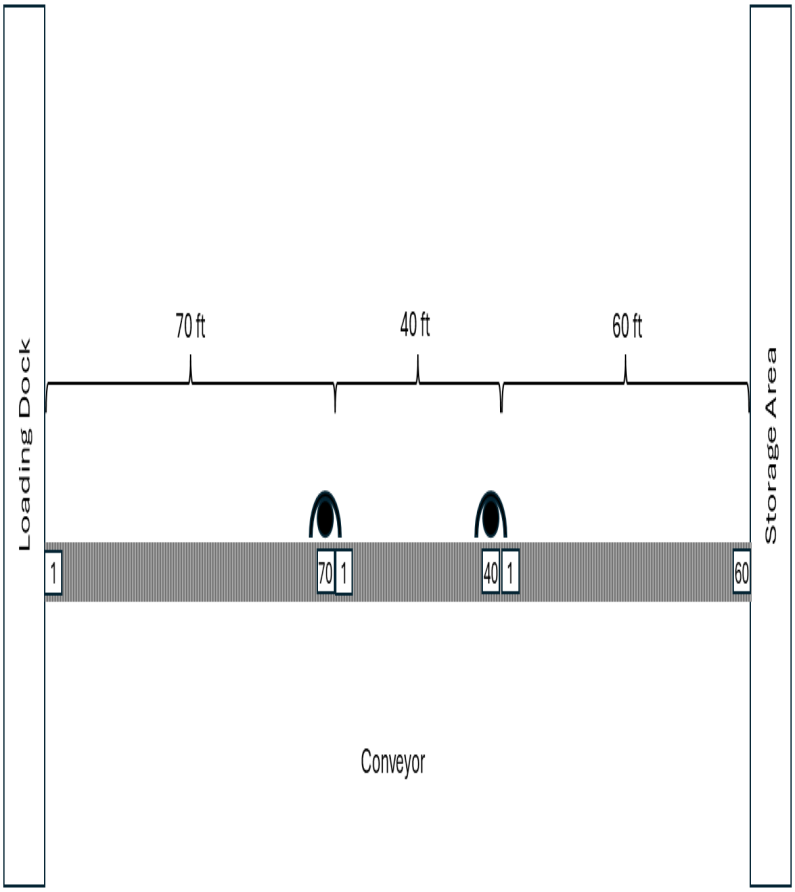


Figure 8.14.: Tandem Queue with Conveyors

following code, we request the conveyor and ride it between the stations. Finally, we only exit the conveyor at the storage location (exit).

```
    private inner class Part : Entity() {
        val tandemQProcess: KSLProcess =
            process(isDefaultProcess = true) {
                wip.increment()
                timeStamp = time
                val conveyorRequest = requestConveyor(conveyor =
                    conveyor, entryLocation = enter,
                    numCellsNeeded = 1)
                rideConveyor(conveyorRequest = conveyorRequest,
    destination = station1)
                use(resource = worker1, delayDuration = st1)
                rideConveyor(conveyorRequest = conveyorRequest,
    destination = station2)
                use(resource = worker2, delayDuration = st2)
                rideConveyor(conveyorRequest = conveyorRequest,
    destination = exit)
                exitConveyor(conveyorRequest = conveyorRequest)
                timeInSystem.value = time - timeStamp
                wip.decrement()
            }
    }
```

Also, notice that we cannot use the `convey()` function because it assumes that the `exit()` function is used for each conveyance. The `rideConveyor()` function causes the conveyor to block until it is called again or until the `exit()` function is called. Thus, the `use()` functions are conceptually happening while the part is still on the conveyor. During this time, for an accumulating conveyor, the parts will pile up behind the blockage, essentially queueing up on the conveyor. Recall the analogy of the seriously deranged simulation professor in the airport. As we can see from the

following results, working on the conveyor has some interesting effects on the statistics.

Accumulating Conveyor with Work On Conveyor: Statistical Summary Report

Name	Count	Average	Half-Width
Conveyor:Enter:AccessQ:NumInQ	30	0	0
Conveyor:Enter:AccessQ:TimeInQ	30	0	0
Conveyor:Station1:AccessQ:NumInQ	30	0	0
Conveyor:Station1:AccessQ:TimeInQ	0	NaN	NaN
Conveyor:Station2:AccessQ:NumInQ	30	0	0
Conveyor:Station2:AccessQ:TimeInQ	0	NaN	NaN
Conveyor:NumOccupiedCells	30	18.369	0.643
Conveyor:CellUtilization	30	0.108	0.004
worker1:InstantaneousUtil	30	0.698	0.002
worker1:NumBusyUnits	30	0.698	0.002
worker1:ScheduledUtil	30	0.698	0.002
worker1:Q:NumInQ	30	0	0
worker1:Q:TimeInQ	30	0	0
worker1:WIP	30	0.698	0.002
worker2:InstantaneousUtil	30	0.901	0.004
worker2:NumBusyUnits	30	0.901	0.004
worker2:ScheduledUtil	30	0.901	0.004
worker2:Q:NumInQ	30	0	0
worker2:Q:TimeInQ	30	0	0
worker2:WIP	30	0.901	0.004
TandemQueueWithConveyor:NumInSystem	30	18.386	0.643
TandemQueueWithConveyor:TimeInSystem	30	18.401	0.61
worker1:SeizeCount	30	14982.167	40.405
worker2:SeizeCount	30	14977.633	40.149

Notice that the queues for the workers are zero. This is because the entity

that seizes the worker blocks all the entities behind it on the conveyor. They never get a chance to seize the worker until the worker is available. Notice that the conveyor now holds about 18.4 occupied cells on average, compared with about 5.7 when the parts exited the conveyor at each station. Essentially, the entities are waiting on the conveyor. Notice also that the queues for entry onto the conveyor at station 1 and 2 are zero. This is because no parts exit at those stations and thereby need to wait to get back on the conveyor. The conveyor can be configured not to report these unnecessary statistics.

Now let's see the effect of using a non-accumulating conveyor in the situation.

Non-accumulating Conveyor with Work On Conveyor:Statistical Summary Report

Name	Count	Average	Half-Width
Conveyor:Enter:AccessQ:NumInQ	30	2360.298	42.602
Conveyor:Enter:AccessQ:TimeInQ	30	2360.777	40.291
Conveyor:Station1:AccessQ:NumInQ	30	0	0
Conveyor:Station1:AccessQ:TimeInQ	0	NaN	NaN
Conveyor:Station2:AccessQ:NumInQ	30	0	0
Conveyor:Station2:AccessQ:TimeInQ	0	NaN	NaN
Conveyor:NumOccupiedCells	30	170	0
Conveyor:CellUtilization	30	1	0
worker1:InstantaneousUtil	30	0.564	0.002
worker1:NumBusyUnits	30	0.564	0.002
worker1:ScheduledUtil	30	0.564	0.002
worker1:Q:NumInQ	30	0	0
worker1:Q:TimeInQ	30	0	0
worker1:WIP	30	0.564	0.002
worker2:InstantaneousUtil	30	0.728	0.001
worker2:NumBusyUnits	30	0.728	0.001
worker2:ScheduledUtil	30	0.728	0.001

Name	Count	Average	Half-Width
worker2:Q:NumInQ	30	0	0
worker2:Q:TimeInQ	30	0	0
worker2:WIP	30	0.728	0.001
TandemQueueWithConveyor:NumInSystem	30	2530.325	42.602
TandemQueueWithConveyor:TimeInSystem	30	2531.064	39.97
worker1:SeizeCount	30	12081.533	31.257
worker2:SeizeCount	30	12081.533	31.257

These results are dramatically different. Why? We again see that the queues for the resources are zero; however, the queue for accessing the conveyor at the enter location is out of control. Why might this be the expected behavior? Recall that for a non-accumulating conveyor, the segments disengage when a part is entering or has reached its destination. The `rideConveyor()` function moves the part to its destination on the conveyor. When the part reaches its destination, the conveyor disengages again. The cell that the part occupies on the conveyor remains occupied while the part is being processed. Thus, when the part is using the worker at station 1 (or station 2) the conveyor is essentially **stopped**. No parts are moving on the conveyor. Therefore, the parts trying to enter at the beginning of the conveyor must wait for the service of **every** part on their segment of the conveyor. This results in a huge bottleneck. Obviously, this is not how we would design such a system. The specification of a non-accumulating conveyor is not appropriate for this situation; however, the usage of an accumulating conveyor is very appropriate. It may be possible to use non-accumulating conveyors when the work is performed on the conveyor; however, the work needs to be perfectly balanced. In the next section, we will see how to model the test and repair system with a circular conveyor.

8.4.3. Test and Repair via Conveyors

For simplicity, the test and repair shop will be used for illustrating the use of conveyors. Figure 8.15 shows the test and repair shop with the conveyors and the distance between the stations. From the figure, the distances from each station going in clockwise order are as follows:

- Diagnostic station to test station 1, 20 feet
- Test station 1 to test station 2, 20 feet
- Test station 2 to repair, 15 feet
- Repair to test station 3, 45 feet
- Test station 3 to diagnostics, 30 feet

Assume that the conveyor's velocity is 10 feet per minute. These figures have been provided in this example; however, in modeling a real system, you will have to tabulate this information. To illustrate the use of entity sizes, assume the following concerning the parts following the four test plans:

- Test plan 1 and 2 parts require 1 foot of space while riding on the conveyor
- Test plan 3 and 4 parts require 2 feet of space while riding on the conveyor

For this example, we will assume that the conveyor is an accumulating conveyor. Now, we are ready to define the conveyor and its segments.

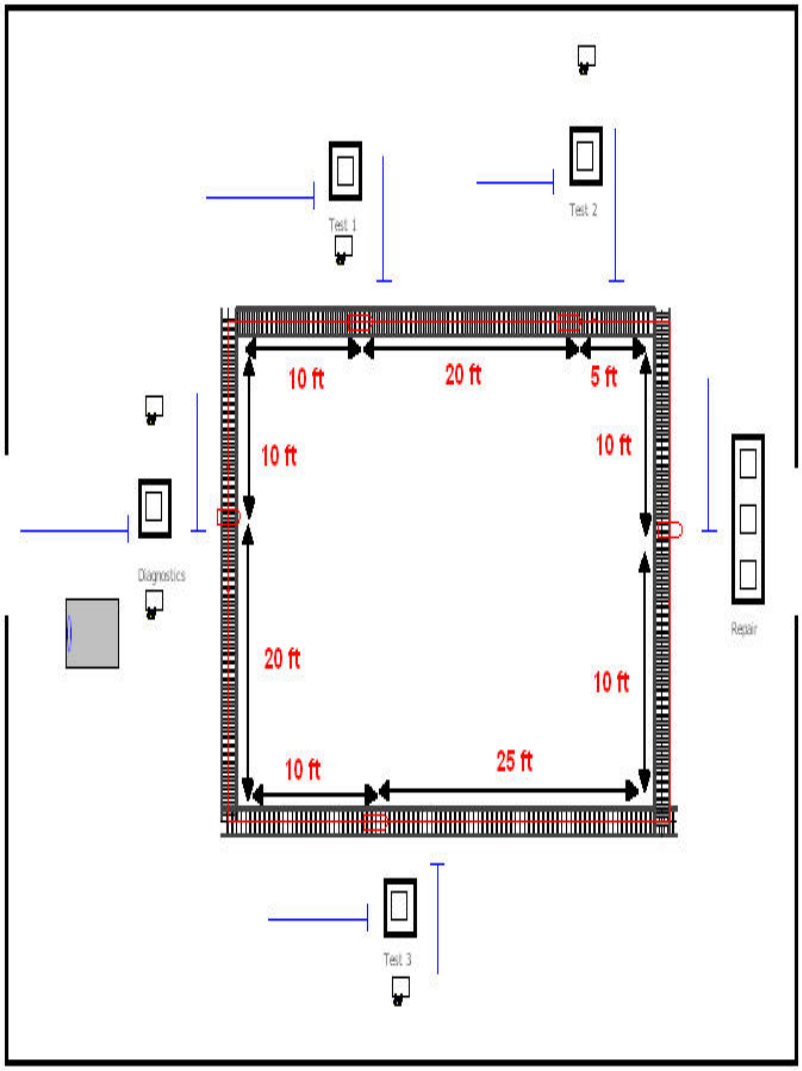


Figure 8.15.: Test and Repair Shop with Conveyors

8.4.3.1. Test and Repair Conveyor Definitions

To define the conveyor within code, we can use the builder functionality provided by the `Conveyor` class's companion object. Because we have parts that require different amounts of space on the conveyor, we need to set the maximum number of cells allowed by any entity. The following KSL code provides the conveyor definition. Also, note the code turns off the reporting of access queues associated with the repair station. This is done because no parts need to access the conveyor at the repair station. For this illustrative example, the code also prints out the conveyor configuration, which we will review.

```
    private val loopConveyor: Conveyor =
     ↪  Conveyor.builder(this, "LoopConveyor")
            .conveyorType(Conveyor.Type.ACCUMULATING)
            .velocity(10.0)
            .cellSize(1)
            .maxCellsAllowed(2)
            .firstSegment(myDiagnostics.name, myTest1.name, 20)
            .nextSegment(myTest2.name, 20)
            .nextSegment(myRepair.name, 15)
            .nextSegment(myTest3.name, 45)
            .nextSegment(myDiagnostics.name, 30)
            .build()

    init {
        loopConveyor.accessQueueAt(myRepair.name).
↪   defaultReportingOption = false
        println(loopConveyor)
    }
```

The configuration of the conveyor can be seen via its `toString()` function. As we can see from the output, we have an accumulating conveyor with

a velocity of 10 feet per minute. The total length of the conveyor is 130 feet. Since each cell equates to 1 foot, there are 130 cells. There are five segments. Since the conveyor's last location is the same as its first location, it is considered a circular conveyor. The conveyor also determines which locations are downstream, relative to the first location. This ensures that the movement request from one location to another location is feasible relative to the path defined by the conveyor. As noted, the segments consist of a sequence of cells. For example, the first segment is between diagnostics and the test 1 station. The entry location for diagnostics is cell 1 and the exit location for test 1 is cell 20. Note that every location has both an entry cell and an exit cell.

```
Conveyor : LoopConveyor
type = ACCUMULATING
is circular = true
velocity = 10.0
cellSize = 1
max number cells allowed to occupy = 2
cell Travel Time = 0.1
Segments:
first location = Diagnostics
last location = Diagnostics
Segment: 1 = (start = Diagnostics --> end = Test1 : length = 20)
Segment: 2 = (start = Test1 --> end = Test2 : length = 20)
Segment: 3 = (start = Test2 --> end = Repair : length = 15)
Segment: 4 = (start = Repair --> end = Test3 : length = 45)
Segment: 5 = (start = Test3 --> end = Diagnostics : length = 30)
total length = 130
Downstream locations:
Diagnostics : [Test1 -> Test2 -> Repair -> Test3 -> Diagnostics]
Test1 : [Test2 -> Repair -> Test3 -> Diagnostics]
Test2 : [Repair -> Test3 -> Diagnostics]
Repair : [Test3 -> Diagnostics]
```

```
Test3 : [Diagnostics]
Segment Cells:
Locations[Diagnostics->Test1] = Cells[1..20]
Locations[Test1->Test2] = Cells[21..40]
Locations[Test2->Repair] = Cells[41..55]
Locations[Repair->Test3] = Cells[56..100]
Locations[Test3->Diagnostics] = Cells[101..130]
```

Now, we are ready to update the process description for the test and repair system to use the conveyor.

8.4.3.2. Test and Repair Conveyor Processing

The test plans for the problem remain the same as in the previous examples; however, the parts that follow different test plans require different cell sizes. To facilitate the specification of cell size requirements for each test plan, we can use a map to hold the requirements.

```
    private val cellSizes = mapOf(testPlan1 to 1, testPlan2
     ↪  to 1, testPlan3 to 2, testPlan4 to 2)
```

We can use the `cellSizes` map to determine the number of cells need by the part when using the conveyor. By using the `convey()` function, the process description becomes very compact.

```
    private inner class Part : Entity() {

        // determine the test plan
        val plan: List<TestPlanStep> = planList.randomElement
        val cellsNeeded = cellSizes[plan]!!

        val testAndRepairProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
```

```
            wip.increment()
            timeStamp = time
            //every part goes to diagnostics
            use(resource = myDiagnostics, delayDuration =
↪ diagnosticTime)
            // get the iterator
            val itr = plan.iterator()
            // iterate through the plan
            var entryLocation = myDiagnostics.name
            while (itr.hasNext()) {
                val tp = itr.next()
                convey(
                    conveyor = loopConveyor,
                    entryLocation = entryLocation,
                    destination = tp.resource.name,
                    numCellsNeeded = cellsNeeded
                )
                use(resource = tp.resource, delayDuration =
↪ tp.processTime)
                entryLocation = tp.resource.name
            }
            timeInSystem.value = time - timeStamp
            wip.decrement()
        }
    }
```

When a part is created, its test plan is randomly determined and assigned. Then, the `cellSizes` map is used to look up the assigned number of cells needed when riding on the conveyor. The main challenge associated with the process code is ensuring that the entry location is correctly specified when using the conveyor. Note that before the iterator loop, we specify the entry location, essentially as the station that we are leaving. This is the location that we need to access to get on the conveyor. This variable

is updated again after using the resource. It may seem strange to specify the location via the stored references of the resources to visit. However, since the locations associated with a conveyor need only be instances of classes that implement the `IdentityIfc` interface, and all model elements implement this interface, model elements such as resources can be used as locations via the model element's `name` property.

The results of executing the model for 30 replications of 20,000 time units (minutes) with a warm up period of 5000 minutes is as follows.

Statistical Summary Report

Name	Count	Average	Half-Width
Diagnostics:InstantaneousUtil	10	0.753	0.007
Diagnostics:NumBusyUnits	10	1.506	0.014
Diagnostics:ScheduledUtil	10	0.753	0.007
Diagnostics:Q:NumInQ	10	2.006	0.129
Diagnostics:Q:TimeInQ	10	39.92	2.475
Diagnostics:WIP	10	3.512	0.142
Test1:InstantaneousUtil	10	0.858	0.007
Test1:NumBusyUnits	10	0.858	0.007
Test1:ScheduledUtil	10	0.858	0.007
Test1:Q:NumInQ	10	2.983	0.272
Test1:Q:TimeInQ	10	52.904	4.43
Test1:WIP	10	3.842	0.279
Test2:InstantaneousUtil	10	0.781	0.006
Test2:NumBusyUnits	10	0.781	0.006
Test2:ScheduledUtil	10	0.781	0.006
Test2:Q:NumInQ	10	1.636	0.07
Test2:Q:TimeInQ	10	43.208	1.587
Test2:WIP	10	2.417	0.074
Test3:InstantaneousUtil	10	0.866	0.006
Test3:NumBusyUnits	10	0.866	0.006
Test3:ScheduledUtil	10	0.866	0.006

Name	Count	Average	Half-Width
Test3:Q:NumInQ	10	2.611	0.19
Test3:Q:TimeInQ	10	51.965	3.559
Test3:WIP	10	3.477	0.194
Repair:InstantaneousUtil	10	0.872	0.004
Repair:NumBusyUnits	10	2.616	0.012
Repair:ScheduledUtil	10	0.872	0.004
Repair:Q:NumInQ	10	1.282	0.091
Repair:Q:TimeInQ	10	25.524	1.695
Repair:WIP	10	3.898	0.101
TestAndRepairWithConveyor:NumInSystem	10	18.261	0.54
TestAndRepairWithConveyor:TimeInSystem	10	363.571	8.902
ProbWithinLimit	10	0.803	0.018
LoopConveyor:Diagnostics:AccessQ:NumInQ	10	0	0
LoopConveyor:Diagnostics:AccessQ:TimeInQ	10	0.001	0
LoopConveyor:Test1:AccessQ:NumInQ	10	0	0
LoopConveyor:Test1:AccessQ:TimeInQ	10	0.001	0
LoopConveyor:Test2:AccessQ:NumInQ	10	0	0
LoopConveyor:Test2:AccessQ:TimeInQ	10	0.001	0
LoopConveyor:Test3:AccessQ:NumInQ	10	0	0
LoopConveyor:Test3:AccessQ:TimeInQ	10	0.001	0
LoopConveyor:NumOccupiedCells	10	1.667	0.01
LoopConveyor:CellUtilization	10	0.013	0
Diagnostics:SeizeCount	10	12540.1	71.465
Test1:SeizeCount	10	14057.6	116.559
Test2:SeizeCount	10	9442.7	63.431
Test3:SeizeCount	10	12532.4	70.688
Repair:SeizeCount	10	12526.6	69.078

The results produced by the KSL are within statistical variation of the same system modeled with a commercial simulation language discussed in this book.

8.4.4. Miscellaneous Concepts in Conveyor Modeling

This section presents some common conveyor modeling situations that can be modeled using the KSL: conveyors that merge, conveyors that diverge, and conveyors that recirculate.

Figure 8.16 illustrates the concept of merging. In the figure, there are three conveyors. The first conveyor goes from area A to the sorting area. The second conveyor goes from area B to the sorting area. The third conveyor goes from the sorting area to processing at area C. The most important aspect of modeling this situation is to note that there are three conveyors. Thus, there would be three instances of the KSL `Conveyor` class that need to be specified. Generally, the velocities of the conveyors may be the same, but not necessarily. To model this situation, we must be able to transfer an entity on one conveyor to another conveyor. The `transferTo()` suspending function provides this functionality.

The `transferTo()` suspending function causes the entity to transfer from its current conveyor to another conveyor. The entity will suspend while accessing the required cells at the next conveyor. Once the desired cells are obtained the entity will exit its current conveyor and be positioned to ride on the next conveyor. After executing this function, the entity will be blocking the entry cell of the next conveyor and must then execute the `rideConveyor()` function to move on the new conveyor. The transfer will include the time needed for the entity to move through the exit cells of the current conveyor, any waiting to access the next conveyor, and be positioned to ride on the next conveyor. There are some important conditions that must be met to utilize this function. First, the entity must be on a conveyor and located at an exit cell/location. Obviously, the entity must transfer to a different conveyor than the one that it currently occupies. Finally, the entity's exit cell's location must correspond to the same location on the conveyor to which it is transferring. That is, the location of transfer must be an exit location on the current conveyor and an entry location on the subsequent conveyor. In Figure 8.16, the cell denoted

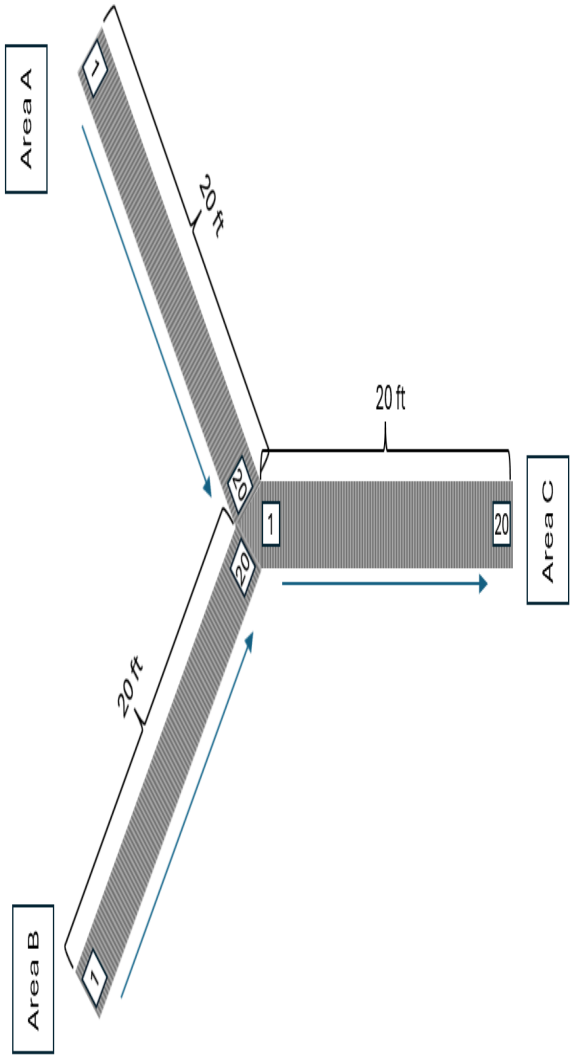


Figure 8.16.: Two Conveyors Merge into One

as 20 on the area A and B conveyors are exit cells. The cell denoted 1 on the conveyor heading towards area C is the entry cell. Parts ride the two input conveyors until they reach the end of the conveyor (cell 20) and then transfer to cell 1 of the subsequent conveyor.

The situation illustrated in Figure 8.16 can be modeled with code similar to the following. First we need to define the three conveyors.

```
class ConveyorMerging(
    parent: ModelElement,
    name: String? = null
) : ProcessModel(parent, name) {

    private val areaA = "AreaA"
    private val areaB = "AreaB"
    private val sorting = "Sorting"
    private val areaC = "AreaC"

    private val conveyor1 = Conveyor.builder(this,
     ↪  "Conveyor1")
        .conveyorType(Conveyor.Type.ACCUMULATING)
        .velocity(1.0)
        .cellSize(1)
        .maxCellsAllowed(1)
        .firstSegment(areaA, sorting, 20)
        .build()

    private val conveyor2 = Conveyor.builder(this,
     ↪  "Conveyor2")
        .conveyorType(Conveyor.Type.ACCUMULATING)
        .velocity(1.0)
        .cellSize(1)
        .maxCellsAllowed(1)
        .firstSegment(areaB, sorting, 20)
```

```
        .build()

    private val conveyor3 = Conveyor.builder(this,
     ↪  "Conveyor3")
        .conveyorType(Conveyor.Type.ACCUMULATING)
        .velocity(1.0)
        .cellSize(1)
        .maxCellsAllowed(1)
        .firstSegment(sorting, areaC, 20)
        .build()
```

Now, we can define processes for the entities that use the conveyors. There could be a variety of ways to handle this situation; however, for simplicity, we will assume that there are two types of entities, one coming from area 1 and the other coming from area 2. The process code for a part starting at area 1 would be something like this:

```
    private val tba1 = ExponentialRV(10.0, 1)
    private val generateAreaAParts =
     ↪  EntityGenerator(this::AreaAPart, tba1, tba1)

    private val tba2 = TriangularRV(10.0, 16.0, 18.0, 2)
    private val generateAreaBParts =
     ↪  EntityGenerator(this::AreaBPart, tba2, tba2)

    private val myAreaASystemTime = Response(this,
     ↪  "AreaASystemTime")
    private val myAreaBSystemTime = Response(this,
     ↪  "AreaBSystemTime")

    private inner class AreaAPart : Entity() {
        val mergeProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
```

```
            val cr = requestConveyor(conveyor = conveyor1,
             ↪  entryLocation = areaA, numCellsNeeded = 1)
            rideConveyor(conveyorRequest = cr, destination =
↪  sorting)
            val tr = transferTo(conveyorRequest = cr,
             ↪  nextConveyor = conveyor3, entryLocation =
             ↪  sorting)
            rideConveyor(conveyorRequest = tr, destination =
↪  areaC)
            exitConveyor(conveyorRequest = tr)
            myAreaASystemTime.value = time -
↪  entity.createTime
        }
    }

    private inner class AreaBPart : Entity() {
        val mergeProcess: KSLProcess =
         ↪  process(isDefaultProcess = true) {
            val cr = requestConveyor(conveyor = conveyor2,
             ↪  entryLocation = areaB, numCellsNeeded = 1)
            rideConveyor(conveyorRequest = cr, destination =
↪  sorting)
            val tr = transferTo(conveyorRequest = cr,
             ↪  nextConveyor = conveyor3, entryLocation =
             ↪  sorting)
            rideConveyor(conveyorRequest = tr, destination =
↪  areaC)
            exitConveyor(conveyorRequest = tr)
            myAreaBSystemTime.value = time -
↪  entity.createTime
        }
    }
```

Notice that the part requests the cells on the conveyor at area A, and then rides to sorting. Then, the part transfers from conveyor 1 to conveyor 3. Notice that the returned conveyor request must be supplied to the `transferTo()` function. Also, notice that the sorting location is both an exit location for conveyor 1 and an entry location for conveyor 3. The `transferTo()` function returns a new conveyor request that represents the fact that the entity is now on the next conveyor. This new conveyor request is used to ride conveyor 3 to the processing at area C and to exit at the processing area. The logic for the parts traveling from area B to processing is very similar. They start at area B and transfer to conveyor 3 at the sorting area. It is important to note that the parts from the two feeder conveyors will compete for the entry cell of conveyor 3 at the sorting location. The parts traveling on conveyors 1 and 2 do not immediately exit at the transfer point. They first receive the necessary cells on conveyor 3 before exiting their current conveyor. Thus, they will block their current conveyor while waiting to access the next conveyor. Thus, we can see why it may be important to model this situation with accumulating conveyors.

Figure 8.17 also illustrates the concept of merging. In this case, there are two conveyors, with one conveyor connecting to the mid-point of the other conveyor. Again, the `transferTo()` function for use with conveyors can be used to address this situation.

The code for this situation would be very similar to the previous example. The important thing to note is that there are only two conveyors. The main conveyor has two segments. The feeder conveyor has one segment. The exit cell/location of the feeder conveyor matches the entry cell/location of the second segment of the main conveyor at cell 6. For example, the main conveyor goes from area A location to the midpoint location to the end point location, with lengths 5 feet respectively. The second conveyor is a single conveyor from the area B location to the midpoint location with a length of 10 feet. In other words, they *share* a common location positioned at cell 6.

The entities arrive at the area A and area B locations. Entities that

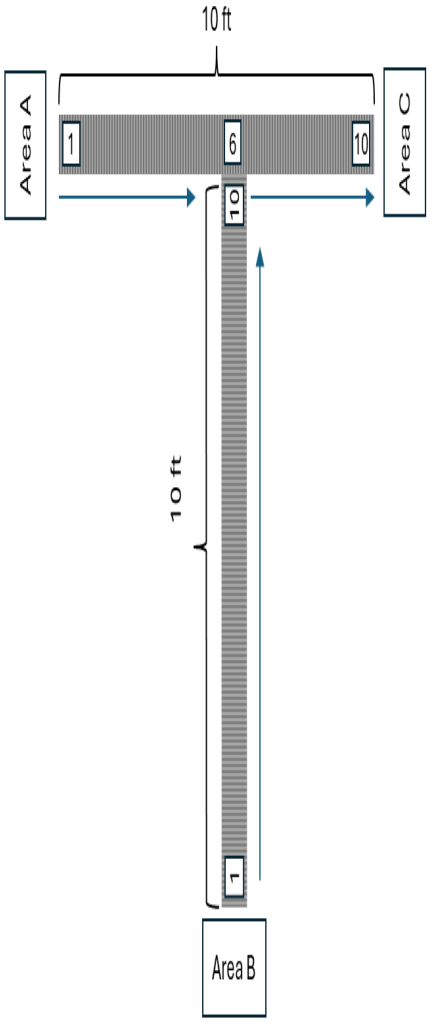


Figure 8.17.: One Conveyor Merges into Another

arrive at the area A location first convey to the midpoint location and then immediately convey to the end point location They do not exit the conveyor at the midpoint location. Entities from the area B location are conveyed to the midpoint location. Once at the midpoint location they first request cells from the main conveyor and then exit their conveyor. This would be modeled using the `transferTo()` function. This causes entities coming from area B to potentially wait for space on the main conveyor. This basic idea can be expanded to any number of feeder conveyors along a longer main conveyor. The end of each feeder conveyor would be mapped to the entry cell of a segment along the main conveyor.

Figure 8.18 illustrates the concept of a diverging conveyor. This situation is the reverse Figure 8.16 with 3 conveyors. One conveyor from area A to sorting point, connects to two other conveyors, one conveyor from sorting to area B and the other conveyor from sorting to area C.

Diverging conveyors are often used in systems that sort items. The items come in on one main conveyor and are transferred to any number of other conveyors based on their attributes (e.g. destination). In this simple illustration, the parts arrive to the arrival area A and access a conveyor to the sorting area. Once an entity reaches the sorting station, the entity would need to determine which of the two conveyors to access. This could be easily accomplished with an if statement or other decision logic. Then the entity would transfer to the subsequent conveyor, ride and then exit. The exercises ask the reader to implement this situation.

With a little imagination a more sophisticated sorting location can be implemented. For example, a number of conveyors that handle the parts by destination might be used in the system. When the parts are created they are given an attribute indicating their destination station. When they get to the sorting location, a Kotlin `when` statement can be used to pick the correct conveyor handling their destination.

The final conveyor modeling situation to be described is that of a recirculating conveyor. We saw in Section 8.4.3 for modeling the test and repair system, that we can create and use a circular conveyor. That is, a conveyor

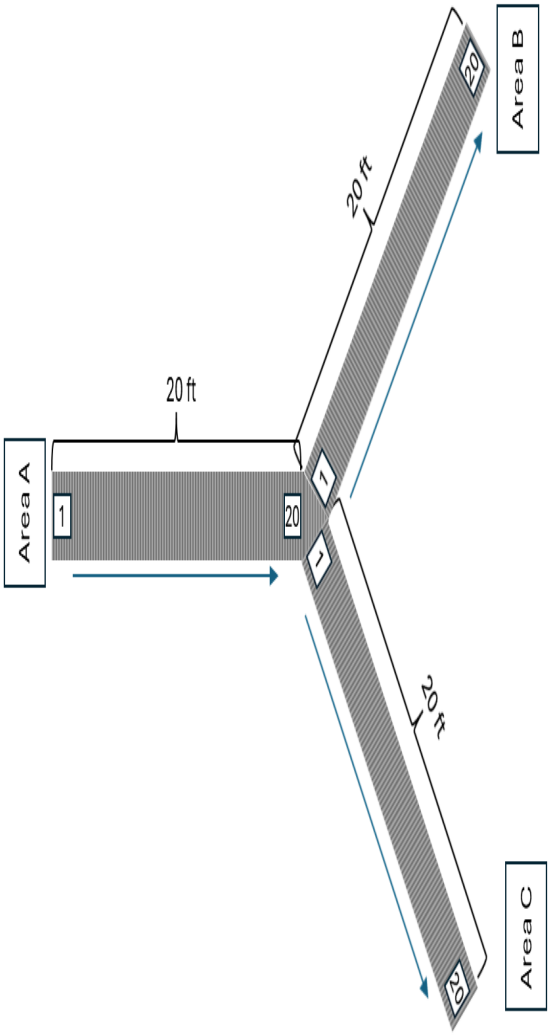


Figure 8.18.: One Conveyor Diverges into Two Conveyors

for which its ending location also corresponds to its entry location. A recirculating conveyor is a circular conveyor in which the entities may ride on the conveyor until there is adequate space at their desired location. In essence the conveyor is used as space to hold the entities that cannot get off due to inadequate space at the necessary station. For example, suppose that in the test and repair shop there was only space for one waiting part at test station two. Thus, the size of test station two's queue can be at most 1. In such a situation, when a part designated for work at test station 2 reaches the location, it needs to check if the available space (queue at test station 2) is available. If there is space available in front of the testing machine, then the part can exit the conveyor. Now, what should we do if there is inadequate space at the test station?

We could wait on the conveyor at test station 2 until there is space, but this will block the entire conveyor while the processing at test station 2 proceeds and its queue reduces. Alternatively, we would not exit the conveyor at test station 2 and instead ride the full conveyor's length *back* to test station 2. By the time that the entity has ridden around the conveyor, there may now be space available at test station 2. As previously noted, in essence, the entire conveyor is being used as waiting space. This kind of situation can be easily modeled with if statements that check if the there is space and if not cause the entity to continue riding. The exercises ask the reader to implement this situation.

8.5. Modeling Guided Path Transporters

This section presents the KSL's modeling constructs for automated guided vehicle (AGV) systems. An AGV is an autonomous battery powered vehicle that can be programmed to move between locations along paths. A complete discussion of AGV systems is beyond the scope of this text. The interested reader should refer to standard texts on material handling or manufacturing systems design for a more complete introduction to the technology. For example, the texts by (Askin and Standridge 1993) and

(Singh 1996) have complete chapters dedicated to the modeling of AGV systems. For the purposes of this section, the discussion will be limited to AGV systems that follow a prescribed path (e.g. tape, embedded wires, etc.) to make things simpler. There are newer AGVs that do not rely on following a path, but rather are capable of navigating via sensors within buildings, see for example (Rossetti and Seldanari 2001).

When modeling with guided transporters, the most important issue to understand is that the transporters can now compete with each other for the space along their paths. Recall from Section 8.2.2 that a movable resource travels from one location to another over a distance supplied by a spatial model. Two movable resources can be in the same place at the same time, in the same way that two workers can walk down the same corridor. An AGV on a guide path cannot do this. The space along the path must be explicitly modeled (like it was for conveyors) and the size of the transporter matters. Conceptually, a *guided transporter* is a mobile resource that moves along a fixed path and competes with other mobile resources for space along the path.

The difference is not a small correction. Table 8.2 compares the same delivery system modeled both ways. Parts arrive every 4 minutes to an entry station and are carried 204 feet to an exit station at the end of a dead end aisle, at 10 feet per minute, over 2000 minutes. The only difference between the two models is whether the carts are movable resources over a distances model or guided transporters on a guide path. The results come from the KSL's automated tests, which check this comparison as part of the library's test suite. Because the arrivals are deterministic, there is no sampling error in these numbers.

Table 8.2.: Free Path and Guided Path Models of the Same Delivery System

Carts	Free path completions	Guided completions	Free path time in system	Guided time in system
1	64	64	878.4	878.4
2	128	128	752.4	754.6
4	255	236	498.5	538.6
6	380	236	248.4	538.6
8	492	236	31.0	538.6

Notice that with one or two carts the two models agree. In that range, a free path model is a fair approximation. Beyond four carts, the guided model stops improving because the aisle to the exit admits one cart at a time, and no number of carts can put two of them down it. The free path model has no notion of an aisle, so it keeps rewarding every cart that is added. Thus, a study that sized this fleet from the free path model would buy eight carts, expect about 31 minutes in the system, and get about 539.

> **!** Important
>
> The guided path constructs (the `ksl.modeling.guidedpath` package) are new in KSL release R1.7 and are marked as *experimental*. The basic constructs presented in this section have been checked against results from a commercial guided path simulation package; however, details of the API may change in later releases. The KSL guided path implementation does not model vehicle acceleration, deceleration, or turning. The time to traverse a zone is the zone's length divided by the vehicle's velocity (adjusted by the zone's velocity factor).

The rest of this section proceeds as follows. First, we discuss the concepts of guided path networks: links, zones, intersections, and stations. Then, we

present the KSL classes that represent these concepts and the suspending functions that entities use to request and ride a guided transporter. A simple AGV system illustrates the constructs, and then the issues of deadlock, spurs, and idle vehicles are discussed. Finally, the test and repair system is revisited with its transport workers moving on a guide path.

8.5.1. Guided Path Networks

When using a guided transporter, the travel path is divided into a network of links and intersections. This is called the vehicle's *path network*. Figure 8.19 illustrates the major concepts in guided vehicle modeling: networks, links, intersections, zones, and stations.

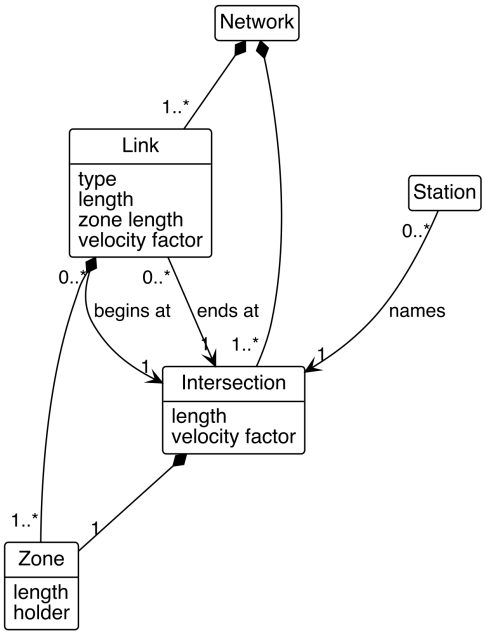


Figure 8.19.: Concepts of a Guided Path Network

A *link* is a connection between two points in a network. Every link has a beginning intersection and an ending intersection. The link represents the space between two intersections. The space on the link is divided into an integer number of zones. Each *zone* has a given length as specified in a consistent unit of measure. All zones on a link have the same length. That is, the length of a link must be an integer multiple of its zone length. For example, a 48 foot link with a zone length of 12 feet consists of 4 zones.

The link can also be of a certain type (unidirectional, bidirectional, or spur). A unidirectional link restricts traffic to one direction, from its beginning intersection to its ending intersection. A bidirectional link implies that the transporter is allowed to traverse in both directions along the link, but only one direction at a time. A spur is used to model "dead ends". A velocity change factor can also be associated with a link to indicate a change in the basic velocity of the vehicle while traversing the link. For example, for a link going through an area with many people present, you may want the velocity of the transporter to automatically be reduced. Figure 8.20 illustrates a simple three link network.

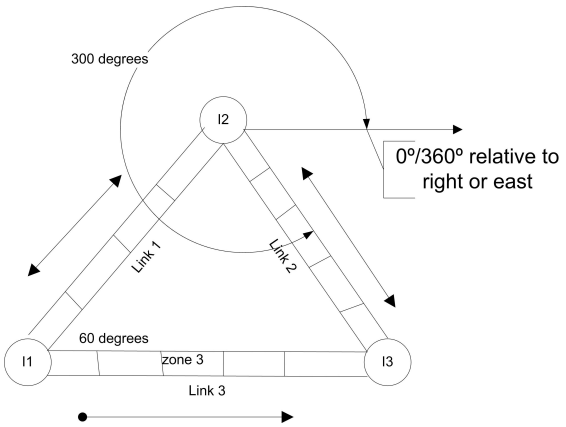


Figure 8.20.: Example Three Link Network

In the figure, Link 1 is bidirectional with a beginning intersection labeled

`I1` and an ending intersection labeled `I2`. Link 1 consists of 4 zones and Links 2 and 3 each consist of 5 zones. The figure also shows a direction of travel for each link, measured in degrees with zero (or 360) degrees to the east (right). Thus, the beginning direction for Link 1 in the figure is 60 degrees, and the beginning direction for Link 2 is 300 degrees. In a simulation package that models the acceleration, deceleration, and turning of vehicles, these directions determine how much a vehicle must turn when moving from one link to the next. The KSL records a beginning direction for each link; however, the direction is only used for drawing the network and for animation. The KSL never uses the direction (or the drawing coordinates of the intersections) to compute travel. Movement is based entirely on the declared lengths of the links.

In general, an *intersection* is simply a point in the network; however, the representation of intersections can be more detailed. An intersection also models space: the space between two or more links. Thus, as indicated in Figure 8.19, an intersection has a length (zero by default) and a velocity change factor. In the KSL, an intersection is itself a zone. That is, an intersection can hold at most one vehicle at a time, just like a zone on a link. When there is more than one path between two intersections, the network chooses a path using a route selection rule. By default, the shortest path (based on link lengths) is used.

Clearly, a fixed path network consists of a set of links and the associated intersections. Also from Figure 8.19, a *station* may be associated with an intersection within the network and an intersection may be associated with many stations. In the KSL, a station is simply a name (an alias) for an intersection, which allows the modeler to refer to the place where work occurs (e.g. `"EntryStation"`) rather than to the intersection's label (e.g. `"I1"`). Transporters within a guided path network can be sent to specific stations or intersections.

8.5.2. KSL Guided Path Constructs

The KSL represents guided path networks and guided transporters with the classes in the `ksl.modeling.guidedpath` package. Figure 8.21 illustrates the main classes and some of their key properties.

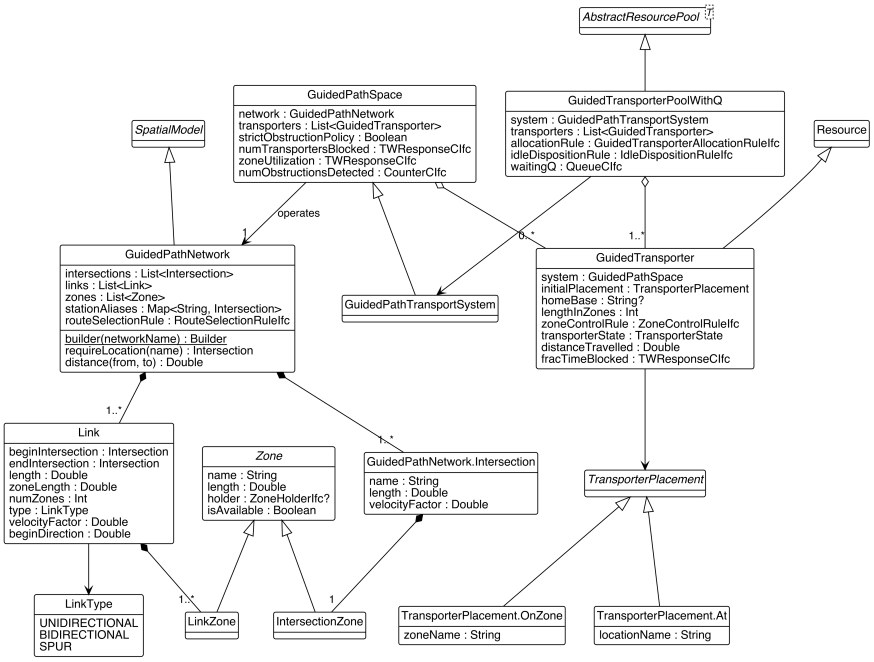


Figure 8.21.: Guided Path Classes

There are four main classes:

- `GuidedPathNetwork` represents the fixed geometry of the network: the intersections, the links, and the zones of the links. A network is built once and does not change during the simulation. Notice from Figure 8.21 that `GuidedPathNetwork` is a sub-class of `SpatialModel`. Thus, a network can serve as the spatial

model of a process model (see Section 8.2), and its intersections (`GuidedPathNetwork.Intersection`) are locations that implement the `LocationIfc` interface. The distance between two intersections is the shortest path distance along the links of the network.

- `GuidedPathTransportSystem` is a model element that operates a network during the simulation. It keeps track of which transporter holds which zone, resets the zones between replications, and collects statistics on blocking and congestion. Its super-class, `GuidedPathSpace`, provides the zone bookkeeping.
- `GuidedTransporter` represents a vehicle. Notice that it is a sub-class of `Resource` with a capacity of one. Thus, a guided transporter is seized and released in the same manner as any other resource.
- `GuidedTransporterPoolWithQ` represents a group of guided transporters (a fleet) with a queue that holds the requests for transport. This class is similar to the `MovableResourcePoolWithQ` class of Section 8.3.

Why are there separate network and system objects? The network represents what is fixed (the geometry) and the system represents what changes during the simulation (the occupancy of the zones). A network should be used by only one transport system. The KSL will throw an exception if a second system is attached to the same network.

8.5.2.1. Building a Network

A network is constructed by using a builder that is accessed from the `GuidedPathNetwork` class. The builder provides functions for declaring intersections, links, and stations. The following code illustrates how to build the three link network of Figure 8.20, assuming that each zone is 12 feet long.

```
val network = GuidedPathNetwork.builder("ThreeLinkNetwork")
    .link("Link1", "I1", "I2", length = 48.0, zoneLength =
    ↪ 12.0,
```

```
        type = LinkType.BIDIRECTIONAL, beginDirection = 60.0)
    .link("Link2", "I2", "I3", length = 60.0, zoneLength =
    ↪  12.0,
        type = LinkType.BIDIRECTIONAL, beginDirection =
↪  300.0)
    .link("Link3", "I1", "I3", length = 60.0, zoneLength =
    ↪  12.0,
        type = LinkType.BIDIRECTIONAL, beginDirection = 0.0)
    .station("Shipping", "I3")
    .build()
```

The `link()` function requires the name of the link, the name of its beginning intersection, the name of its ending intersection, the length of the link, and the zone length. Intersections that are named by a link are created automatically. The type of link is specified with the `LinkType` enumeration, which has values `UNIDIRECTIONAL` (the default), `BIDIRECTIONAL`, and `SPUR`. The link's `velocityFactor` (default 1.0) and `beginDirection` (default 0.0) can also be supplied. A variation of the `link()` function accepts the number of zones (`numZones`) rather than the zone length. Notice that the zone length is specified for each link. Thus, different links within the same network can have different zone sizes. The `station()` function associates a station name with an intersection. Finally, the `build()` function checks the specification and creates the network.

The `intersection()` function of the builder can be used to explicitly declare an intersection along with its length, velocity factor, and drawing coordinates (`x`, `y`). If no intersection is given coordinates, the KSL lays out the network for drawing based on the lengths of the links.

When the network is built, the KSL logs a warning for each bidirectional link because bidirectional links are the usual source of deadlock (see Section 8.5.4). It also warns if some intersection cannot be reached from another intersection. For example, on a network of only unidirectional links, there may be no path back to where a vehicle started.

8.5.2.2. Guided Transporters and Pools

The constructor of the `GuidedTransporter` class is as follows:

```
class GuidedTransporter @JvmOverloads constructor(
    val system: GuidedPathSpace,
    initialPlacement: TransporterPlacement,
    velocity: RVariableIfc,
    override val lengthInZones: Int = 1,
    val zoneControlRule: ZoneControlRuleIfc =
     ↪  EndOfZoneControl(),
    name: String? = null,
    override val physicalLength: Double? = null,
    override val loadCapacity: Int = 1
)
```

A guided transporter belongs to a transport system. The initial placement of the transporter is specified with an instance of the `TransporterPlacement` class: `TransporterPlacement.At(name)` places the transporter at a named intersection or station and `TransporterPlacement.OnZone(name)` places it on a named zone of a link. The transporter is placed at its initial position at the start of every replication. Because transporters take space on the path, the choice of initial position can be important. Two transporters cannot be placed in the same zone.

The velocity of the transporter is specified by a random variable. By default, the velocity is sampled for each move. The `lengthInZones` property specifies how many zones the vehicle covers. Thus, a vehicle that is two zones long covers two zones of a link while it moves. The `zoneControlRule` governs when the transporter releases the zone behind it, which will be discussed shortly. The optional `physicalLength` property allows the length of a vehicle that is shorter than a zone to be specified in the units of the network; however, most models can ignore this property and use

`lengthInZones`. Finally, each transporter also has a `homeBase` property, which names an intersection or station to which the transporter can return when it is idle.

Guided path transporters move from zone to zone on the links. The size of the zone, the zone control, and the size of the transporter govern how close the transporters can get to each other while moving. Zones are similar to the cell concept for conveyors. The selection of the size of the zones is really governed by the physical characteristics of the actual system. A zone represents the unit of space that the control system of the real system reserves for a vehicle. In addition, each zone traversal is one event in the simulation. Thus, halving the zone size will double the number of events needed to model the same movement.

Zone control governs how the transporter moves from zone to zone. Figure 8.22 illustrates two zone control rules. The KSL provides three rules:

- `StartOfZoneControl` specifies that the transporter will release its backward most zone when it starts into its next zone. This allows following vehicles to immediately begin to move into the released zone.
- `EndOfZoneControl` specifies that the transporter will release its backward most zone after it has moved into the next zone (i.e. reached the end of the next zone). This prevents a following vehicle from moving into the transporter's current zone until it is fully out of the zone. This is the default rule.
- `DistanceIntoZoneControl(d)` specifies that the transporter releases its backward-most zone after traveling `d` distance units into the next zone. The start and end rules are the two extremes of this rule.

In the figure, the vehicle is contained in only one zone. In general, a vehicle could cover many zones. The release at end form of control allows for more separation between the vehicles when they are moving. These rules also apply to how the vehicles move through intersections and onto links because an intersection is a zone. For example, under the end rule, a

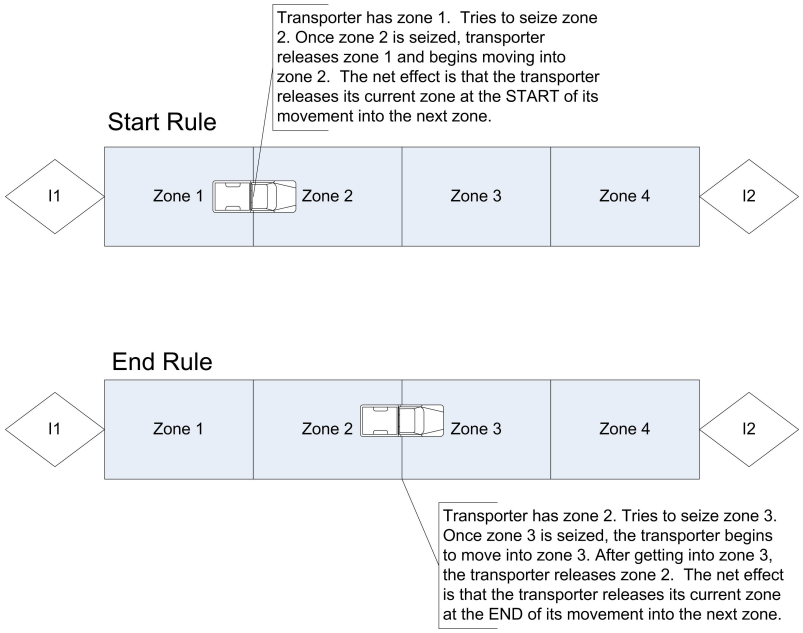


Figure 8.22.: Illustrating Zone Control

vehicle crossing an intersection holds the intersection until it has traveled through the first zone of the link that it is entering. The zone control rule is a modeling parameter. The control system of a real installation implements one of these rules, and which one it implements can change the throughput of the system.

The constructor of the `GuidedTransporterPoolWithQ` class is as follows:

```
open class GuidedTransporterPoolWithQ @JvmOverloads
 ↪  constructor(
    parent: ModelElement,
    val system: GuidedPathTransportSystem,
    transporters: List<GuidedTransporter>,
    allocationRule: GuidedTransporterAllocationRuleIfc =
     ↪  ClosestByNetworkDistanceRule(),
    override var idleDispositionRule: IdleDispositionRuleIfc
     ↪  = ParkInPlaceRule(),
    name: String? = null
)
```

The pool holds a list of transporters that all belong to the same transport system. Two rules govern the pool. The *allocation rule* selects which idle transporter will be sent to a pick up location. The default rule, `ClosestByNetworkDistanceRule`, selects the idle transporter that is closest to the pick up location. Notice that the distance is the distance along the guide path. On a one-way loop, a transporter that is just past the pick up location may have to travel all the way around the loop. Other rules provided by the KSL include `FurthestByNetworkDistanceRule`, `LeastUsedTransporterRule`, `CyclicalTransporterRule`, and `RandomTransporterRule`.

The *idle disposition rule* determines what a transporter does when it is released and there are no requests waiting. The KSL provides three rules:

- `ParkInPlaceRule` leaves the transporter where it was released. This is the default.
- `ReturnToHomeBaseRule` sends the transporter to its `homeBase` location.
- `MoveToStagingAreaRule(locationName)` sends the transporter to the named staging location.

As we will see in Section 8.5.4, the choice of where idle transporters wait is one of the most important decisions when modeling a guided path system.

8.5.2.3. Suspending Functions for Guided Transport

Entities use guided transporters through suspending functions that are similar to the `transportWith()` function of Section 8.2.2. The `guidedTransport()` function encapsulates the full pattern: request a transporter, wait for it to arrive at the pick up location, load, ride to the destination, unload, and release the transporter.

```
suspend fun guidedTransport(
    pool: GuidedTransporterPoolWithQ,
    destination: String,
    pickupLocation: String = entity.currentLocation.name,
    loadingDelay: GetValueIfc = ConstantRV.ZERO,
    unLoadingDelay: GetValueIfc = ConstantRV.ZERO,
    requestPriority: Int = TRANSPORT_REQUEST_PRIORITY,
    loadingPriority: Int = DELAY_PRIORITY,
    unLoadingPriority: Int = DELAY_PRIORITY,
    suspensionName: String? = null
): GuidedTransportResult
```

Notice that the destination and pick up location are names of stations or intersections. By default, the pick up location is the name of the entity's current location. The function returns an instance of the

`GuidedTransportResult` class, which reports what the journey cost: the time waiting for the transporter to arrive (`approachTime`), the time riding (`rideTime`), the time that the transporter was blocked by other vehicles during the journey (`blockedTime`), the distance traveled (`routeLength`), and the number of zones traversed (`zonesTraversed`).

The `guidedTransport()` function is built from three suspending functions, which can be used directly when something needs to happen between being picked up and being dropped off:

```
val request = requestGuidedTransporter(carts, pickupLocation
 ↪  = "EntryStation")
delay(1.0) // load by hand
val result = transportBy(request, destination =
 ↪  "ExitStation")
delay(2.0) // unload
releaseGuidedTransporter(request, carts)
```

The `requestGuidedTransporter()` function requests a transporter from the pool and suspends the entity until the allocated transporter has arrived at the pick up location. The returned `GuidedTransportRequest` represents the entity's claim on the transporter. The `transportBy()` function carries the entity to the destination, with optional loading and unloading delays. Finally, the `releaseGuidedTransporter()` function returns the transporter to the pool. After the release, the request cannot be used again.

The transport system automatically collects statistics. These include the time-weighted number of transporters moving, blocked, and idle (`NumTransportersMoving`, `NumTransportersBlocked`, `NumTransportersIdle`), the fraction of zones that are occupied (`ZoneUtilization`), and the per transport responses (`ApproachTime`, `RideTime`, `TransportBlockedTime`, `RouteLengthPerTransport`, `ZonesTraversedPerTransport`). For each transporter, the fraction

of time moving, transporting, moving empty, and blocked are collected (`FracTimeMoving`, `FracTimeTransporting`, `FracTimeMovingEmpty`, `FracTimeBlocked`). Statistics for each link and each zone can be turned on with the `collectLinkStatistics` and `collectZoneStatistics` properties of the transport system. They are off by default because a network with many zones would otherwise produce a very large report.

We are now ready to illustrate these constructs with an example.

8.5.3. A Simple AGV System

In this example, parts arrive to an entry station according to a Poisson process with a mean time between arrivals of 20 minutes, where they wait for one of two available AGVs. Once loaded onto the AGV (taking 0.5 minutes) they are transported to the exit station. At the exit station, the part is unloaded, again taking 0.5 minutes. The basic layout of the system is shown in Figure 8.23. There are seven intersections and seven links in this example. The distances between the intersections are shown in the figure.

In modeling this situation, you need to make a number of decisions regarding transporter characteristics, the division of space along the links, the directions of the links, and where the vehicles wait. Let's assume that the item being transported is rather large. Because of this, the transporter is a cart that is 6 feet in length and only travels 10 feet per minute. There are two carts available for transport.

In this example, assume that the carts should not get too close to each other during movement (perhaps because the big part overhangs the cart). Let's choose a zone size of 12 feet for the links of the main loop. Thus, while the cart is in a zone it essentially takes up half of the zone. Each cart is one zone long, and under the default end of zone control, a following cart remains a full zone behind the cart in front of it. With a zone length of 12

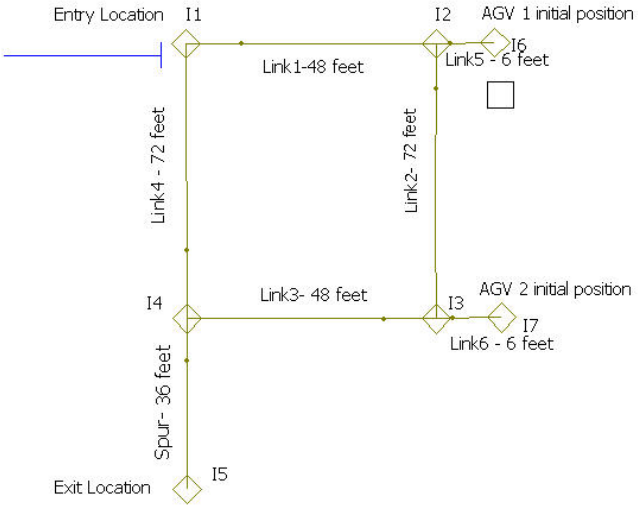


Figure 8.23.: Layout for Simple AGV Example

feet, Link 1 consists of 4 zones of 12 feet in length. Thus, the maximum number of vehicles that could be on Link 1 is four.

Now, you need to decide on the direction of travel permitted on the links. In this system, let's assume that the AGVs will travel clockwise around the main loop on unidirectional links. The link from intersection `I4` to the exit station at `I5` is a spur. Since AGVs are battery powered, they typically have a charging station. Because of this, most systems will be designed with a set of spurs to act as a "home base" for the vehicles. For this simple system, spurs to intersections `I6` and `I7` will be used as the home bases and initial positions of the two carts. These spurs are only 6 feet long (one cart length), so they are given a zone length of 6 feet. Since the zone length is specified for each link, the home spurs can have a different zone size than the main loop.

The following code builds the network for this example. The full source code is in the file `SimpleAGVExample.kt` within the

`ksl.examples.general.guidedpath` package of the `KSLExamples` project.

```
    private val entryStation = "EntryStation"
    private val exitStation = "ExitStation"
    private val agv1Home = "I6"
    private val agv2Home = "I7"
    private val loopZoneLength = 12.0
    private val homeSpurZoneLength = 6.0

    private fun buildNetwork(): GuidedPathNetwork =
        GuidedPathNetwork.builder("SimpleAgvNetwork")
            .intersection("I1", x = 0.0, y = 72.0)
            .intersection("I2", x = 48.0, y = 72.0)
            .intersection("I3", x = 48.0, y = 0.0)
            .intersection("I4", x = 0.0, y = 0.0)
            .intersection("I5", x = 0.0, y = -36.0)
            .intersection("I6", x = 54.0, y = 72.0)
            .intersection("I7", x = 54.0, y = 0.0)
            .link("Link1", "I1", "I2", length = 48.0,
             ↪  zoneLength = loopZoneLength, beginDirection =
             ↪  0.0)
            .link("Link2", "I2", "I3", length = 72.0,
             ↪  zoneLength = loopZoneLength, beginDirection =
             ↪  270.0)
            .link("Link3", "I3", "I4", length = 48.0,
             ↪  zoneLength = loopZoneLength, beginDirection =
             ↪  180.0)
            .link("Link4", "I4", "I1", length = 72.0,
             ↪  zoneLength = loopZoneLength, beginDirection =
             ↪  90.0)
            .link(
                "Spur", "I4", "I5", length = 36.0, zoneLength
                 ↪  = loopZoneLength,
```

```
            type = LinkType.SPUR, beginDirection = 270.0
        )
        .link(
            "Link5", "I2", "I6", length = 6.0, zoneLength
             ↪  = homeSpurZoneLength,
            type = LinkType.SPUR, beginDirection = 0.0
        )
        .link(
            "Link6", "I3", "I7", length = 6.0, zoneLength
             ↪  = homeSpurZoneLength,
            type = LinkType.SPUR, beginDirection = 0.0
        )
        .station(entryStation, "I1")
        .station(exitStation, "I5")
        .build()
```

Notice that the intersections are declared with coordinates so that the network can be drawn and animated. The four links of the main loop are unidirectional (the default type). The beginning directions of 0, 270, 180, and 90 degrees indicate the clockwise travel around the loop, but recall that the KSL does not use them to compute movement. The spur to the exit station and the two home spurs are declared with `LinkType.SPUR`. Finally, the entry station is associated with intersection `I1` and the exit station is associated with intersection `I5`.

Because the loop is one way, the distances in the network are not symmetric. The trip from the entry station to the exit station must go the long way around the loop: $48 + 72 + 48 + 36 = 204$ feet. The return trip from the exit station to the entry station is only $36 + 72 = 108$ feet because the spur can be traveled in both directions and Link 4 runs directly from `I4` to `I1`.

The following code defines the transport system, the two carts, and the pool of carts.

```
    private val network: GuidedPathNetwork = buildNetwork()

    init {
        spatialModel = network
    }

    val system = GuidedPathTransportSystem(this, network,
     ↪  name = "AgvSystem")

    private val cart1 = GuidedTransporter(
        system, TransporterPlacement.At(agv1Home),
↪   ConstantRV(10.0), 1, EndOfZoneControl(), "Cart1"
    ).apply { homeBase = agv1Home }

    private val cart2 = GuidedTransporter(
        system, TransporterPlacement.At(agv2Home),
↪   ConstantRV(10.0), 1, EndOfZoneControl(), "Cart2"
    ).apply { homeBase = agv2Home }

    private val carts = GuidedTransporterPoolWithQ(
        this, system, listOf(cart1, cart2),
        ClosestByNetworkDistanceRule(),
↪   ReturnToHomeBaseRule(), "Carts"
    )
```

Notice that the network is assigned as the spatial model of the process model. The parts travel on the guide path, so the network is their spatial model too. Each cart is placed at its home intersection, travels at a constant 10 feet per minute, is one zone long, and uses the end of zone control rule. The `homeBase` property of each cart is set to its starting intersection. The pool uses the closest by network distance allocation rule and returns idle carts to their home bases.

The process for the parts is very simple.

```
    private inner class Part : Entity("Part") {
        val delivery = process(isDefaultProcess = true) {
            val arrived = time
            currentLocation =
↪   network.requireLocation(entryStation)
            guidedTransport(
                carts,
                destination = exitStation,
                pickupLocation = entryStation,
                loadingDelay = myLoadingTime,
                unLoadingDelay = myUnLoadingTime
            )
            myTimeInSystem.value = time - arrived
            myCompleted.increment()
        }
    }
```

The part records its arrival time and sets its current location to the entry station. Then, the `guidedTransport()` function is used to request a cart from the pool, have the cart pick up the part at the entry station, and carry it to the exit station. The loading and unloading times are random variables (`myLoadingTime` and `myUnLoadingTime`) defined as constants of 0.5 minutes. After the part is unloaded and the cart is released, the part's time in the system is collected and the count of delivered parts is incremented.

Before running the model, let's think about what we should expect. Let d represent the distance traveled from the entry station to the exit station and let v represent the velocity of a cart. A loaded cart that is never blocked by the other cart rides for $d/v = 204/10 = 20.4$ minutes. Counting the zones along the route, the cart enters 4 zones on Link 1, 6 zones on Link 2, 4 zones on Link 3, 3 zones on the spur, and the 4 intersections `I2`, `I3`, `I4`, and `I5`, for a total of 21 zones. The model was run for 10 replications

of 8000 minutes with a warm up period of 1000 minutes. Table 8.3 presents selected results.

Table 8.3.: Selected Results for the Simple AGV System

Response	Average	Half-Width
AgvSystem:RideTime	20.579	0.026
AgvSystem:ApproachTime	11.234	0.131
AgvSystem:TransportBlockedTime	0.206	0.030
AgvSystem:ZonesTraversedPerTransport	21.000	0.000
AgvSystem:RouteLengthPerTransport	204.000	0.000
AgvSystem:NumTransportersBlocked	0.011	0.001
Carts:FractionBusy	0.820	0.019
Carts:Q:TimeInQ	29.062	4.378
Cart1:FracTimeBlocked	0.005	0.001
Cart2:FracTimeBlocked	0.005	0.002
TimeInSystem	61.761	4.340
PartsDelivered	349.900	8.777

Notice that the route length (204 feet) and the number of zones traversed (21) match the hand calculation exactly. The average ride time of 20.579 minutes is slightly more than the 20.4 minutes of unobstructed travel. The difference is caused by the cart occasionally being blocked by the other cart, which is a delay that a free path model cannot represent. The carts are busy about 82% of the time, and a part waits on average about 29 minutes for a cart to be allocated. The approach time (about 11.2 minutes) is the time from the cart's allocation until the part is aboard, excluding the loading time. Thus, the bulk of a part's 61.8 minutes in the system is spent waiting for a cart and riding on a cart.

8.5.4. Deadlock, Spurs, and Idle Vehicles

To understand why the direction of travel and spurs are important, you need to understand the concept of deadlock. As indicated in the zone control discussion, guided path transporters move by seizing and releasing zones. Now consider the possibility of a bidirectional link with two transporters moving in opposite directions. What will happen when the transporters meet? They will crash! No, not really, but they will cause the system to get in a state of deadlock. *Deadlock* occurs when one vehicle requires a zone currently under the control of a second vehicle, and the second vehicle requires the zone held by the first vehicle. More generally, deadlock is a circular wait among two or more vehicles. The KSL checks for a circular wait whenever a vehicle is blocked. If one is detected, the simulation stops by throwing a `GuidedPathDeadlockException`. The exception includes a report that names each vehicle in the cycle, the zone that it holds, and the zone that it is waiting for.

There is a related situation that is not a circular wait. Suppose a station was attached to `I4` in the simple AGV example and a transporter (`T1`) was sent directly to that station for some processing. Now, suppose that transporter (`T2`) has just dropped off something at the exit point, `I5`, and that a new part has just come in at `I1`. Transporter (`T2`) will want to go through `I4` on its way to `I1` to pick up the new part. Transporter (`T2`) will wait until transporter (`T1`) finishes its processing at `I4` and if transporter (`T1`) *eventually* moves there will be no problem. However, suppose transporter (`T1`) is released at `I4`, it is left where it stands, and no other parts arrive for processing. Transporter (`T1`) will become idle at the station attached to `I4` and will continue to occupy the space associated with `I4`. Since no arriving parts will ever cause transporter (`T1`) to move, transporter (`T2`) will be stuck. There will be the one part waiting for transporter (`T2`), but transporter (`T2`) will never get there to pick it up.

This situation is not a circular wait, because it ends the moment that something dispatches transporter (`T1`). The KSL calls this an *idle transporter*

obstruction. When a transporter becomes blocked by an idle transporter that has nothing scheduled to move it, the KSL logs a warning, counts the occurrence in the `NumObstructionsDetected` response, and tracks the time-weighted number of transporters waiting behind idle vehicles in the `NumBlockedByIdleVehicle` response. The simulation is not stopped. If you want the simulation to stop when an obstruction occurs, set the `strictObstructionPolicy` property of the transport system to true. Here is an example of the warning:

```
WARN  k.m.guidedpath.GuidedPathSpace - At time 306.2407379572777, transporter (Cart1
is blocked on zone (I4), which is held by idle transporter (Cart2). Nothing is sched
to move the idle transporter, so this wait lasts until something dispatches it, and
lasts to the end of the replication if nothing does. This is not a circular wait.
Give idle transporters a home base or a staging area so that they leave the path.
```

The point is: care must be taken when designing guided path networks so that deadlock is prevented by design. For novice modelers, deadlock can seem troublesome, especially if you don't think about where to place the vehicles and if you have bidirectional links. In most real systems, you should not have to worry about deadlock because you should design the system not to have it in the first place! In the simple example, all unidirectional links were used along with spurs. This prevents many possible deadlock situations. To illustrate what is meant by design, consider the obstruction situation. A simple solution to prevent it would be to provide a spur for transporter (`T1`) to receive the processing that was occurring at the station associated with `I4`, and instead associate the processing station with the intersection at the end of the new spur. Thus, during the processing the vehicle will be out of the way. If you were designing a real system, wouldn't you do this common sense thing anyway? Another simple solution would be to always send idle vehicles to a staging area or home base that gets them out of the way of the main path network. In the KSL, this is the purpose of the `ReturnToHomeBaseRule` and `MoveToStagingAreaRule` idle disposition rules.

Let's briefly discuss the operation of spurs and bidirectional links. Notice that the link from `I4` to `I5` is a spur in the simple AGV example. A spur is a special type of link designed to help prevent some deadlock situations. A spur models a "dead end" and is bidirectional. If a vehicle is sent to the intersection at the end of the spur (or to a station attached to the end intersection), then the vehicle reserves the spur so that it can get back out. Any other vehicle that is sent to the end of the spur is not permitted to enter the intersection at the entrance of the spur until the first vehicle has left the spur, including the intersection at its dead end. Thus, the second vehicle waits on the main path rather than following the first vehicle into the spur, where the two would face each other with neither able to move. This logic only applies to vehicles that are sent to the end of the spur. Any vehicles sent to or through the entering intersection of the spur are still permitted to gain access to the spur's entering intersection. This allows traffic to continue on the main path while vehicles are down the spur. Unfortunately, if a vehicle stops at the entering intersection and then never moves, the obstruction situation described previously occurs.

Bidirectional links work similarly to unidirectional links, but a vehicle moving on a bidirectional link must have control of both the entering zone on the link and the direction of travel. If another vehicle tries to move on the bidirectional link it must be moving in the same direction as any vehicles currently on the link; otherwise, it will wait. Unfortunately, if a vehicle (wanting to go down a bidirectional link) gains control of the intersection that a vehicle traveling on the bidirectional link needs to exit the link, deadlock may be possible. Because it is so easy to get in deadlock situations with bidirectional links, you should think carefully about their use when using them within your models. Where the physical aisle is wide enough for two vehicles to pass, it should be modeled as a pair of unidirectional links (one in each direction) rather than as one bidirectional link.

The simple AGV example can illustrate the effect of where idle vehicles wait. The `SimpleAGVExample` class has a control (`sendCartsHome`) that switches the idle disposition rule of the pool between `ReturnToHomeBaseRule` and

`ParkInPlaceRule`. The `main()` function of the example runs both configurations as two scenarios using the same random number streams, and then computes the paired difference of selected responses (carts sent home minus carts left in place). The following output presents the results.

```
Where an idle cart waits: CartsSentHome minus CartsLeftInPlace, paired by replicatio
(10 replications, 95% intervals)
  response                                  difference   half-
width  detectable?
  PartsDelivered                               0.1000       0.5278           no
  TimeInSystem                                -1.2009       1.0846          yes
  AgvSystem:NumObstructionsDetected          -40.5000       5.3453          yes
  AgvSystem:NumBlockedByIdleVehicle           -0.1166       0.0104          yes
  AgvSystem:NumTransportersBlocked            -0.1380       0.0126          yes
```

When the carts are left in place, about 40.5 obstructions occur per replication. Every one of them is at `I4`, the intersection at the entrance of the exit spur: one cart stands at `I4` with nothing to do and the other cart needs `I4` in order to continue. None of these waits lasts. Parts keep arriving, each arrival dispatches the parked cart, and both configurations deliver the same number of parts because the throughput of this system is limited by the arrivals. Thus, the cost of leaving the carts in place shows up as about 1.2 minutes of additional time in the system and an increase in the number of transporters blocked. In a system where arrivals stop or become infrequent, the same obstruction can last until the end of the replication, with the rest of the fleet stuck behind it. Thus, you should give the vehicles a place to wait that is off the main path.

8.5.5. Test and Repair with Guided Transporters

This section revisits the test and repair shop of Section 8.3. In that section, three transport workers were modeled as a pool of movable resources over

a distances model. Here, the transport workers travel on a guide path. Everything about the work is identical: the same test plans, processing times, repair times, arrival process, stations, number of transport workers, and walking speed distribution. Only the representation of the space changes.

Figure 8.24 presents the guide path for the shop. The stations are connected by a single one-way aisle that forms a loop: diagnostics, test station 1, test station 2, test station 3, repair, and back to diagnostics. The lengths of the aisles are the distances from the distances model of Section 8.3 along this cycle. Thus, the two models agree on how far apart the stations are; however, on the guide path a worker can only travel one way around the loop and can be held up by another worker in front of it. Each transport worker also has its own 5 meter parking spur off the diagnostic station.

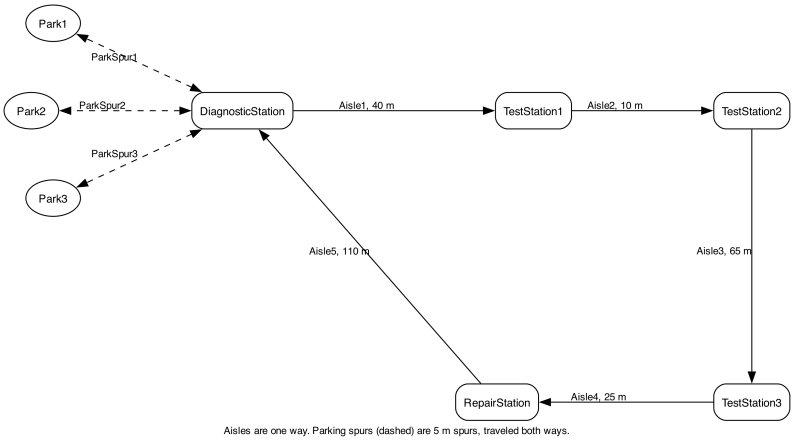


Figure 8.24.: Guide Path for the Test and Repair Shop

The following code creates the network. The full source code is in the file `TestAndRepairShopWithGuidedTransporters.kt` within the `ksl.examples.book.chapter8` package of the `KSLExamples` project.

```
    private val diagnosticStation = "DiagnosticStation"
    private val testStation1 = "TestStation1"
    private val testStation2 = "TestStation2"
    private val testStation3 = "TestStation3"
    private val repairStation = "RepairStation"
    private val zoneLength = 5.0

    private fun createNetwork(numSpurs: Int, networkName:
     ↪  String = "ShopAisle"): GuidedPathNetwork {
        var b = GuidedPathNetwork.builder(networkName)
            .intersection(diagnosticStation, x = 0.0, y =
             ↪  0.0)
            .intersection(testStation1, x = 40.0, y = 0.0)
            .intersection(testStation2, x = 50.0, y = 0.0)
            .intersection(testStation3, x = 50.0, y = -65.0)
            .intersection(repairStation, x = 25.0, y = -65.0)
            .link("Aisle1", diagnosticStation, testStation1,
             ↪  length = 40.0, zoneLength = zoneLength,
             ↪  beginDirection = 0.0)
            .link("Aisle2", testStation1, testStation2,
             ↪  length = 10.0, zoneLength = zoneLength,
             ↪  beginDirection = 0.0)
            .link("Aisle3", testStation2, testStation3,
             ↪  length = 65.0, zoneLength = zoneLength,
             ↪  beginDirection = 270.0)
            .link("Aisle4", testStation3, repairStation,
             ↪  length = 25.0, zoneLength = zoneLength,
             ↪  beginDirection = 180.0)
            .link("Aisle5", repairStation,
             ↪  diagnosticStation, length = 110.0, zoneLength
             ↪  = zoneLength, beginDirection = 90.0)
        for (i in 1..numSpurs) {
            b = b.intersection("Park$i", x = -10.0, y = -10.0
↪  * i)
```

```
            b = b.link(
                "ParkSpur$i", diagnosticStation, "Park$i",
                ↪ length = zoneLength, zoneLength =
                ↪ zoneLength,
                type = LinkType.SPUR, beginDirection = 180.0
            )
        }
        return b.build()
    }
```

Notice that in this network, the intersections are named after the stations, so no station aliases are needed. The aisle is divided into 5 meter zones, which divides every aisle length exactly. A loop adds one parking spur for each transport worker. The coordinates are only used for drawing. For example, the declared length of `Aisle5` is 110 meters, even though the straight line distance between the drawing coordinates of the repair station and the diagnostic station is about 70 meters. A guide path does not need to be drawn to scale.

The transport system, the workers, and the pool are defined as follows. The workers walk with the same triangular distributed speed (in meters per minute) as in Section 8.3.

```
    val network: GuidedPathNetwork = aisleNetwork ?:
    ↪ createNetwork(numTransporters)

    init {
        spatialModel = network
    }

    val transportSystem = GuidedPathTransportSystem(this,
    ↪ network, name = "ShopTransport")
```

```
    private val homes: List<String> =
        transporterHomes ?: (1..numTransporters).map {
↪   "Park$it" }

    private val carts: List<GuidedTransporter> =
     ↪  (1..numTransporters).map { i ->
        GuidedTransporter(
            transportSystem, TransporterPlacement.At(homes[i
↪   - 1]),
            transporterVelocity ?: myWalkingSpeedRV, 1,
↪   zoneControlRule, name = "Worker$i",
            physicalLength = transporterPhysicalLength
        ).apply { homeBase = homes[i - 1] }
    }

    val transportWorkers = GuidedTransporterPoolWithQ(
        this, transportSystem, carts,
        ClosestByNetworkDistanceRule(), idleDispositionRule,
↪   "TransportWorkerPool"
    )
```

The class's constructor has parameters for the number of transporters (default 3), the zone control rule (default `EndOfZoneControl()`), and the idle disposition rule (default `ReturnToHomeBaseRule()`). It also allows a different network, velocity, and set of home locations to be supplied, which makes it convenient to experiment with alternative layouts. Each worker starts at, and returns to, its own parking spur.

The process for the parts follows the same structure as in Section 8.3. The key difference is that the guided transport needs the name of the pick up station, so the part keeps track of the station where it currently is (`at`).

```
        val testAndRepairProcess: KSLProcess =
            process(isDefaultProcess = true) {
                var at = diagnosticStation
                var carried = 0.0
                currentLocation =
network.requireLocation(diagnosticStation)
                wip.increment()
                myNumberIn.increment()
                timeStamp = time
                use(diagnosticWorkers, delayDuration =
diagnosticTime)
                for (tp in plan) {
                    val leg = guidedTransport(
                        transportWorkers, destination =
tp.testStation, pickupLocation = at
                    )
                    carried += leg.approachTime + leg.rideTime
                    at = tp.testStation
                    use(tp.testMachine, delayDuration =
tp.processTime)
                }
                val lastLeg = guidedTransport(transportWorkers,
                    destination = repairStation, pickupLocation
                    = at)
                carried += lastLeg.approachTime +
lastLeg.rideTime
                use(repairWorkers, delayDuration =
repairTimes[plan]!!)
                myTransferTime.value = carried
                timeInSystem.value = time - timeStamp
                myNumberOut.increment()
                wip.decrement()
            }
```

Notice that the `testStation` property of each test plan step now holds the name of the station on the guide path. For each step of the test plan, the part uses `guidedTransport()` to be carried from its current station to the next test station. The `GuidedTransportResult` returned from each transport is used to accumulate the time that the part spent with a worker (the approach time plus the ride time), which is collected in the `TransferTime` response. After the test plan is complete, the part is carried to the repair station.

Both models were run for 10 replications of 249,600 minutes (52 weeks, 5 days per week, two 480 minute shifts per day). Table 8.4 compares selected results from the movable resource model of Section 8.3 and the guided path model, both run with the same version of the KSL.

Table 8.4.: Test and Repair Shop with Movable Resources and with Guided Transporters

Response	Movable resources	Guided path
Test1:InstantaneousUtil	0.858 ± 0.007	0.858 ± 0.007
Test3:InstantaneousUtil	0.866 ± 0.006	0.866 ± 0.006
RepairWorkers:InstantaneousUtil	0.872 ± 0.004	0.872 ± 0.004
TransportWorkerPool:FractionBusy	0.156 ± 0.001	0.271 ± 0.002
TransportWorkerPool:Q:TimeInQ	0.005 ± 0.000	0.016 ± 0.001
NumInSystem	17.658 ± 0.543	17.968 ± 0.540
TimeInSystem	351.557 ± 9.033	357.728 ± 8.916
ProbWithinLimit	0.819 ± 0.018	0.810 ± 0.018

Notice that the utilizations of the test and repair stations are the same in both models. They have to be. The processing times are drawn from

the same random number streams by the same parts in the same order, regardless of how the parts move between the stations. The transport workers, however, are busy about 1.7 times as much of the time on the guide path (0.271 versus 0.156). Because the aisle is one way, a worker that is allocated to pick up a part must travel forward around the loop to reach it, and the trip from one station to another can be much longer than the direct distance. For example, test plan 1 requires a part to move from test station 3 back to test station 2. On the guide path, the part is carried through the repair station, the diagnostic station, and test station 1, a distance of $25 + 110 + 40 + 10 = 185$ meters. The guided path results also report how much the workers interfered with each other. Each worker was blocked by another worker for less than 1% of the time (`FracTimeBlocked` ranges from 0.005 to 0.008). The net effect is about 6.2 more minutes in the system for the average part. This difference is smaller than the half-widths of the two estimates, so more replications (or a paired comparison using common random numbers) would be needed to determine whether the difference is significant.

There is one subtle aspect of these results to be careful about. The individual worker utilizations from the guided path model are quite different: about 0.36 for `Worker1`, 0.28 for `Worker2`, and 0.17 for `Worker3`. This is not a finding about the layout. All three workers return to parking spurs of the same length off the diagnostic station, so their network distances to any pick up location are the same when they are idle. The `ClosestByNetworkDistanceRule` then breaks the tie in the order that the workers were declared. Using the `LeastUsedTransporterRule` allocation rule would spread the work more evenly, although a more even split need not reduce the time that parts spend in the system. The exercises ask you to explore these issues.

8.5.6. Further Topics in Guided Path Modeling

This section has provided a basic introduction to using guided path transporters in models. There are many more issues related to the use of these constructs that have not been covered:

1. Closing zones for a period of time (e.g. a spill or a maintenance activity in an aisle) and modeling places where pedestrians cross the aisles of the vehicles.
2. Statistics for individual links and zones, which help identify where congestion occurs within a network.
3. Specifying the allocation, idle disposition, and zone contention rules by name, so that the rules can be varied within scenarios and experiments.
4. Multi-floor layouts, in which a lift is modeled as a link with a single zone.
5. The *active* AGV constructs of the `ksl.modeling.agv` package, in which the vehicles are agents that decide what to do next and are directed by a dispatcher, rather than passive resources that are requested by entities.

More details on these topics can be found in the KSL guided path guide (`docs/guides/ksl-guidedpath.md` in the KSL repository) and in the examples within the `ksl.examples.general.guidedpath` package of the `KSLExamples` project. Advanced discussions of guided path modeling are also given in Banks et al. (1995) and Pegden et al. (1995), including within Pegden et al. (1995) the possible use of guided path constructs for modeling automated storage and retrieval systems (AS/RS) and overhead cranes.

8.6. Summary

The modeling of entity movement can be achieved with a simple delay. If distance and velocity are available, then we can compute the time of the delay. The KSL provides spatial models and suspending functions that facilitate the movement of entities. The `Entity` class provides basic tracking of the location of the entity when using movement constructs (the `initialLocation`, `previousLocation`, `currentLocation`, `isMoving`, and `velocity` properties). Movable resources are resources that can move within a spatial model. In such situations, the distance between origins and destinations becomes an essential component of the modeling.

In addition to movable resources, this chapter presented the KSL constructs for modeling conveyors. The simplest approach to modeling conveyors is to simply model the transport time with deterministic delays. However, if the space associated with the conveyors is an important modeling issue, then the KSL `Conveyor` class can be used.

Finally, this chapter presented the KSL constructs for modeling guided path transporters, such as automated guided vehicles. When transporters compete for the space along a fixed path, a free path model of their movement can be very optimistic. The KSL represents the path as a network of links and intersections whose space is divided into zones, and each zone can be held by at most one vehicle. The modeling of guided path systems requires careful thought about the design of the network. One-way links, spurs, and home bases for idle vehicles prevent many deadlock and obstruction situations.

The following new KSL constructs were discussed in this chapter:

`SpatialModel`: A representation of space that can track elements in the space, their movement, and govern how they move.

`DistancesModel`: A special case of a spatial model that holds the distance between named locations.

`LocationIfc`: An interface that describes a location within a spatial model from which distances can be computed.

`MovableResource`: A resource that can be moved and used for pickup and delivery.

`MovableResourceWithQ`: A movable resource that has a queue to hold requests for pickup and delivery.

`MovableResourcePool`: A pool of movable resources that can be used to transport entities between locations.

`MovableResourcePoolWithQ`: A pool of movable resources with a queue to hold requests for pickup and delivery.

`MovableResourceSelectionRuleIfc`: An interface for rules that select movable resources within pools.

`MovableResourceAllocationRuleIfc`: An interface for rules that determine which movable resource will receive the allocation.

`move()`: A suspending function that moves a spatial element from its location to another location.

`moveWith()`: A suspending function used with movable resources to move both the resource and the entity to a location.

`transportWith()`: A suspending function that moves a movable resource to the entity's location and then moves both the entity and the resource to the specified destination.

`Conveyor`: A class that models a conveyor as a series of segments, each divided into cells, on which entities ride between locations.

`ConveyorRequestIfc`: An interface for the request (the "ticket") that gives an entity control of cells on a conveyor.

`requestConveyor()`: A suspending function that requests cells (space) on the conveyor.

`rideConveyor()`: A suspending function that causes the entity to move through cells on the conveyor.

`exitConveyor()`: A suspending function that causes the entity to exit the cells that it occupies on the conveyor.

`convey()`: A suspending function that combines requesting, riding, and exiting the conveyor.

`transferTo()`: A suspending function that causes an entity to transfer from one conveyor to another.

`GuidedPathNetwork`: The fixed network of intersections, links, zones, and stations, which is also a spatial model.

`GuidedPathTransportSystem`: The model element that operates a network, tracks the occupancy of its zones, and collects congestion statistics.

`GuidedTransporter`: A capacity one resource that moves on the network by claiming the zone ahead of it.

`GuidedTransporterPoolWithQ`: A pool of guided transporters with a queue for requests, governed by an allocation rule and an idle disposition rule.

`LinkType`: The type of a link: unidirectional, bidirectional, or spur.

`StartOfZoneControl`, `EndOfZoneControl`, `DistanceIntoZoneControl`: Rules for when a transporter releases the zone behind it.

`ReturnToHomeBaseRule`, `ParkInPlaceRule`, `MoveToStagingAreaRule`: Rules for where idle transporters wait.

`guidedTransport()`: A suspending function that requests a guided transporter, rides it to a destination, and releases it.

`requestGuidedTransporter()`: A suspending function that requests a guided transporter and waits for it to arrive at the pick up location.

`transportBy()`: A suspending function that rides an allocated guided transporter to a destination.

`releaseGuidedTransporter()`: A suspending function that returns the guided transporter to its pool.

The next chapter returns to Monte Carlo methods and presents advanced techniques, including bootstrapping, variance reduction, and the generation of multi-variate and correlated random variates.

8.7. Exercises

Exercise 8.1. Consider the STEM Career Fair Mixer of Example 7.7. Change the model so that the implementation uses the `DistancesModel` class to represent the travel distances. Use the `moveTo()` function to model the movement of the students within the STEM Career Fair. Compare your results to the implementation in Chapter 7.

Exercise 8.2. Consider the implementation of the tandem queue with constrained movement in Section 8.2.3 that uses a pool of three movers. With the loading and unloading times, the movers cannot keep up with the work, so remove the loading and unloading times. Change the model so that the implementation uses the following movable resource allocation rules.

- `MovableResourceAllocateInOrderListedRule`
- `FurthestMovableResourceAllocationRule`
- `LeastUtilizedMovableResourceAllocationRule`
- `RandomMovableResourceAllocationRule` (with stream 4)

Compare the results of the four rules. Which rule do you recommend and why?

Exercise 8.3. Consider the implementation of the Test and Repair Shop of Section 8.3. Change the model so that the implementation uses the following movable resource allocation rules.

- `MovableResourceAllocateInOrderListedRule`
- `FurthestMovableResourceAllocationRule`
- `LeastUtilizedMovableResourceAllocationRule`
- `RandomMovableResourceAllocationRule` (with stream 4)

Compare your results to the results of Section 8.3. Which rule do you recommend and why?

Exercise 8.4. Consider the implementation of the Test and Repair Shop of Section 8.3. Re-analyze this situation and recommend an appropriate number of transport workers.

Exercise 8.5. Reconsider Exercise 7.19. The distances between the four stations (in feet) are given in the following table. After the parts finish the processing at the last station of their sequence they are transported to an exit station, where they begin their shipping process. Parts arrive directly at station A, where every sequence starts, and the fork trucks carry the parts between the stations and to the exit.

Station	A	B	C	D	Exit
A	–	50	60	80	90
B	55	–	25	55	75
C	60	20	–	70	30
D	80	60	65	–	35

Exit	100	80	35	40	–

There are 2 fork trucks in the system. The velocity of the fork trucks varies within the facility due to various random congestion effects. Assume that the fork truck's velocity between the stations is triangularly distributed with parameters (3, 4, 5) miles per hour. The fork trucks start at station A. Simulate the system for 30000 minutes using 10 replications and discuss the potential bottlenecks in the system.

Exercise 8.6. Reconsider the Test and Repair example from Section 8.3 In this problem, the workers have a home base that they return to whenever there are no more waiting requests rather than idling at their last drop off point. The home base is located at the center of the shop. The distances from each station to the home base are given in as follows.

Station	Diagnostics	Test 1	Test 2	Test 3	Repair	Home base
Diagnostics	–	40	70	90	100	20
Test 1	43	–	10	60	80	20
Test 2	70	15	–	65	20	15
Test 3	90	80	60	–	25	15
Repair	110	85	25	30	–	25
Home base	20	20	15	15	25	–

Rerun the model using the same run parameters as the chapter example and report the average number of requests waiting for transport. Give an assessment of the risk associated with meeting the contract specifications based on this new design.

Exercise 8.7. *(This problem is based on an example on page 223 of (Pegden et al. 1995). Used with permission)* Reconsider Exercise 7.11. Assume that all parts are transferred by using two fork trucks that travel at an average speed of 150 feet per minute. The distances (in feet) between the stations are provided in the table below.

	Enter	Workstation	Paint	New Paint	Pack	Exit
Enter	–	325	445	445	565	815
Workstation		–	120	130	240	490
Paint			–	250	120	370
New Paint				–	130	380
Pack					–	250
Exit						–

The distances are symmetric. Both the drop-off and pickup points at a station are at the same physical location. Once the truck reaches the pickup/drop-off station, it takes 2 minutes to load the part at the pickup station and 2 minutes to unload it at the drop-off station. The fork trucks start at the Enter station.

Analyze this system to determine any potential bottleneck operations. Report on the average flow times of the parts as a whole and individually. Also obtain statistics representing the average number of parts waiting for allocation of a transporter and the number of busy transporters. Run the model for 600,000 minutes with a 50,000 minute warm up period, using 10 replications.

Exercise 8.8. Three independent conveyors deliver 1 foot parts to a warehouse. Once inside the warehouse, the conveyors merge onto one main conveyor to take the parts to shipping. Parts arriving on conveyor

1 follow a Poisson process with a rate of 3 parts per minute. Parts arriving on conveyor 2 follow a Poisson process with a rate of 5 parts per minute. Finally, parts arriving on conveyor 3 have an inter-arrival time uniformly distributed between (0.2 and 0.4) minutes. Conveyors 1 and 2 are accumulating conveyors and are 15 and 20 feet long respectively. They both run at a velocity of 12 feet per minute. Conveyor 3 is an accumulating conveyor and is 10 feet long. It runs at a velocity of 20 feet per minute. The main conveyor is 25 feet long and is an accumulating conveyor operating at a speed of 25 feet per minute. Consider the parts as being disposed after they reach the end of the main conveyor. Simulate this system for 480 minutes and estimate the access times, the time waiting to merge onto the main conveyor, the time in the system, and the average number of parts in the system.

Exercise 8.9. Consider the scenario discussed in Section 8.4.4 concerning recirculating conveyors regarding the test and repair system. Assume that each test station only has room for two parts (the part being processed and one waiting). Implement the conveyor system such that parts that arrive to a full testing station recirculate until they can eventually get processed. Run the model for the same model settings as used in the textbook example. What is the effect of recirculating on the conveyor utilization? How does the finite buffers and recirculating conveyor affect the total system time for the parts?

Exercise 8.10. Consider the scenario discussed in Section 8.4.4 concerning two conveyors with one conveyor merging into the middle of the second conveyor. Entities arrive at the midpoint station from both entry points (A and B). Those arriving from Station A continue on their conveyor.

Those arriving from Station B exit the merge belt conveyor and access the main belt conveyor. Type A entities arrive to station A according to a Poisson process with a rate of 1 arrival every 3 minutes. Similarly, Type B entities arrive according to a Poisson process with a rate of 1 arrival every 3 minutes to station B.

The main belt conveyor has two segments each of length 5 feet. The first segment goes from station A to the middle (merge point) and the second segment goes from the merge point to the exit station. The merging conveyor has one segment of length 10 feet, which goes from station B to the merge point. Both conveyors have velocity of 1 foot per minute with cell size of 1 and the maximum cells occupied as one. Both conveyors are also accumulating.

Develop a KSL model for this situation using base time units of minutes. Report the results for system time and throughput for the parts based on 480 minutes of operation.

Exercise 8.11. Consider the scenario discussed in Section 8.4.4 concerning a diverging conveyor. Entities arrive at the arrival station according to a Poisson process with a rate of 1 part every 12 minutes (stream 1). The parts are conveyed on the main conveyor to a sorting area. The length of the main conveyor is 20 feet. The main conveyor has a velocity of 1 foot per minute and uses a cell size of 1. Assume the maximum cell size is also one. All conveyors in this exercise are accumulating conveyors.

At the sorting area, the parts are shunted to either of two processing areas: area 1 and area 2. Thus, there is a conveyor from the sorting area to the first processing area. Call this conveyor the line 1 conveyor. There is also a conveyor from the sorting area to the second processing area. Call this conveyor the line 2 conveyor. Both conveyors have one segment of 20 feet and use velocity of 1 foot per minute and single unit cell sizes with maximum cell size of 1.

The logic at the sorting area is simple. Each arriving part is shunted to area 1 or 2 in an alternating fashion. The first arriving part goes to area 1. At the sorting area there is a small delay of 3 seconds as the part is scanned and then sent to the appropriate area.

Develop a KSL model for this situation using base time units of minutes. Report the results for system time and throughput for the parts based on 10 replications of 120 minutes of operation.

Exercise 8.12. Answer the following questions concerning guided path networks in the KSL.

a. True or False: In the KSL, the beginning direction of a link (in degrees) affects the time required for a guided transporter to traverse the link.

b. The length of a link must be an integer multiple of its _____.

c. A transporter is sent to the intersection at the dead end of a spur while another transporter is already at the dead end. Where does the second transporter wait?

d. What is the difference between a deadlock and an idle transporter obstruction? Which one stops the simulation by default?

e. True or False: Under the `EndOfZoneControl` rule, a following transporter can begin moving into a zone as soon as the transporter in front of it starts moving out of the zone.

Exercise 8.13. Consider the simple AGV system of Section 8.5.3 with the same arrival process, loading and unloading times, layout, and run settings (10 replications of 8000 minutes with a warm up period of 1000 minutes).

a. Change the zone control rule of both carts to `StartOfZoneControl`. Compare the time in the system of the parts and the fraction of time each cart is blocked to the results in Table 8.3.

b. Using the original zone control rule, change the mean time between arrivals from 20 minutes to 15 minutes. What happens to the queue of parts waiting for a cart? Add a third cart with its own 6 foot home spur off intersection `I1`, and recommend the number of carts for the system.

Exercise 8.14. Consider the test and repair shop with guided transporters of Section 8.5.5. Use the same run settings as in the section (10 replications of 249,600 minutes).

a. Run the model with 1, 2, and 3 transport workers. Recommend the number of transport workers needed for the shop and justify your recommendation in terms of the time in the system of the parts and the queueing for the workers.

b. Using 3 transport workers, change the allocation rule of the pool to `LeastUsedTransporterRule`. How does the rule affect the utilization of the individual workers and the time in the system of the parts?

c. Using 3 transport workers, change the idle disposition rule to `ParkInPlaceRule`. Report the number of obstructions detected and discuss the effect on the system.

Exercise 8.15. This problem considers the use of AGVs for the test and repair system of Section 8.3. The layout of the proposed system is given in Figure 8.25 with all measurements in meters.

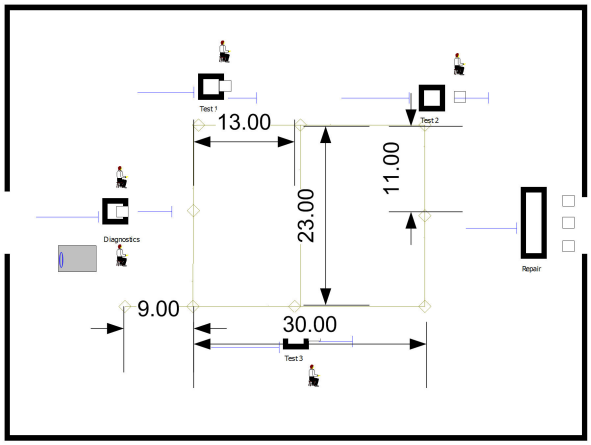


Figure 8.25.: Proposed Layout for the AGV Test and Repair System

There is only 1 AGV in this system. It is 1 meter in length and moves at a velocity of 30 meters per minute. Its home base is at the dead end of the 9 meter spur. Since there will be only one AGV all links are bidirectional. The design associates the stations with the nearest intersection on the path network. The intersection nearest Diagnostics on the left side of the loop is level with the intersection nearest Repair, 11 meters below the top of the loop. There is no time to load or unload the AGV.

a. Simulate the system for 10 replications of 249,600 minutes (4160 hours). Estimate the chance that the contract specifications are met and the average system time of the jobs. In addition, assess the queueing for and the utilization of the AGV.

b. Consider the possibility of having a 2 AGV system. What changes to your design do you recommend? Be specific enough that you could simulate your design.

Exercise 8.16. *(This problem is based on an example on page 381 of (Pegden et al. 1995). Used with permission)* This problem is Exercise 7.15 of the online edition of (Rossetti 2015), adapted for the KSL. Reconsider Exercise 8.7 with the use of AGVs. There are now 3 AGVs that travel at a speed of 100 feet per minute to transport the parts within the system. As in Exercise 8.7, it takes 2 minutes to load a part and 2 minutes to unload it.

To avoid deadlock situations the guided path network has been designed for one way travel, except for Link 15, which is a two-way spur. In addition, to prevent an idle vehicle from blocking vehicles performing transports, a staging area (Intersection 12) has been placed on the guided path network. Whenever a vehicle completes a transport, a check should be made of the number of requests pending for the transporters. If there are no requests pending, the AGV should be sent to the staging area. If there are multiple vehicles idle, they should wait on Link 11. Link 15 is a spur from intersection 11 to intersection 6. The stations for the operation on the parts should be assigned to the intersections as shown in the figure. The links should all be divided into zones of 10 feet each. The initial position of the vehicles should be along Link 11. Each vehicle is 10 feet in length or 1 zone. The release at start form of zone control should be used.

The guided path network for the AGVs is given in Figure 8.26. Note that two links in the figure are labeled Link 4. Treat the 40 foot link from intersection 7 to intersection 4 as Link 3 and the 80 foot link from intersection 4 to intersection 8 as Link 4. In addition, the figure shows Link 16 (from intersection 11 to intersection 1) as 90 feet. Use 80 feet (8 zones) for Link 16.

Analyze this system to determine any potential bottleneck operations. Report on the average flow times of the parts as a whole and individually. Also obtain statistics representing the average number of parts waiting for allocation of a transporter and the number of busy transporters. Run the model for 600,000 minutes with a 50,000 minute warm up period, using 10 replications.

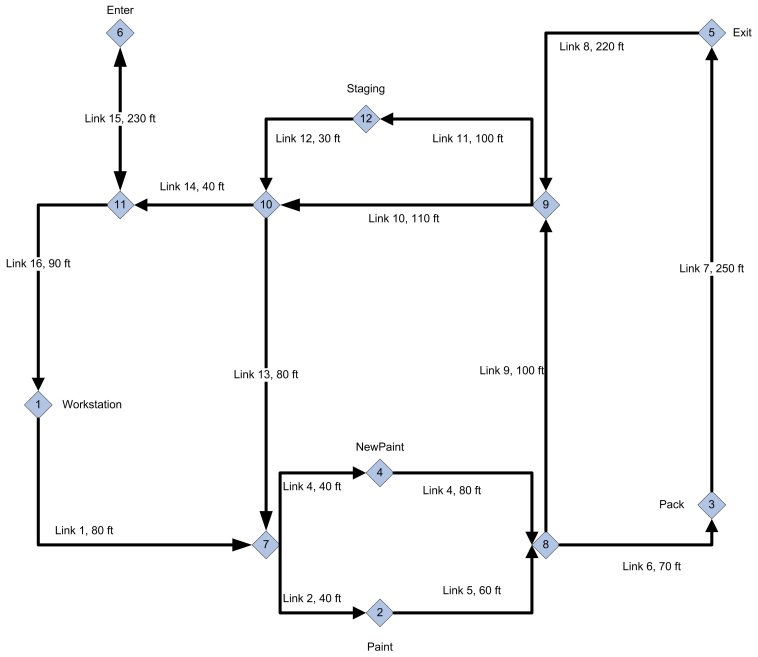


Figure 8.26.: AGV System Layout for the Painting Flow Line

Exercise 8.17. A single automatic guided vehicle (AGV) is used to pick up finished parts from three machines and drop them off at a store room. The AGV is designed to carry 5 totes. Each tote can hold up to 10 parts. The machines produce individual parts according to a Poisson process with the rates indicated in the table below. The machines are designed to directly drop the parts into a tote. When a tote is full, it is released down a gravity conveyor to the AGV loading area. It takes 2 seconds for the tote to move along the conveyor to the machine's loading area.

Station	Production Rate	Tote Loading Time	Travel distance to next station
Machine 1	2 per minute	uniform(3,5) seconds	40 meters
Machine 2	1 per minute	uniform(2,5) seconds	28 meters
Machine 3	3 per minute	uniform(4,6) seconds	55 meters

The AGV moves from station to station in the sequence (Machine 1, Machine 2, Machine 3, store room). If there are totes waiting at the machine's loading area, the AGV loads the totes up to its capacity. After the loading is completed, the AGV moves to the next station. If there are no totes waiting at the station, the AGV delays for 5 seconds, if any totes arrive during the delay they are loaded; otherwise, the AGV proceeds to the next station. After visiting the third machine's station, the AGV proceeds to the store room. At the storeroom any totes that the AGV is carrying are dropped off. The AGV is designed to release all totes at once, which takes between 8 and 12 seconds uniformly distributed. After completing its visit to the store room, it proceeds to the first machine. The distance from the storeroom to the first station is 30 meters. The AGV travels at a velocity of 30 meters per minute. The AGV starts at the store room. Use streams 1, 2, and 3 for the parts produced by machines 1, 2, and 3, streams 4, 5, and 6 for their tote loading times, and stream 7 for

the time to release the totes at the store room. Simulate the performance of this system over a period of 10,000 minutes, using 10 replications.

Model the route of the AGV as a one-way guided path loop through the four stations. Hint: Represent the AGV's route with a single entity that requests the guided transporter once, keeps it for the entire simulation, and uses the `transportBy()` function to move from each station to the next. The totes can be entities that wait at the loading areas until the AGV picks them up.

a. Estimate the average system time for a part. What is the average number of totes carried by the AGV on each trip? What is the average route time? That is, what is the average time that it takes for the AGV to complete one circuit of its route?

b. Suppose the sizing of the loading area for the totes is of interest. What is the required size of the loading area (in terms of the number of totes) such that the probability that all totes can wait is virtually 100%.

---

9. Advanced Monte Carlo Methods

Learning Objectives

- To be able to explain and apply the basic principles of bootstrap sampling
- To be able to apply variance reduction techniques to simulation experiments
- To be able to apply the main techniques recommended for generating multi-variate random variables
- To be able to apply Markov Chain Monte Carlo techniques to the generation of random variables

This chapter builds on the methods presented in Appendix A, Chapter 2, and Chapter 3 to present advanced Monte Carlo techniques that are often applied in practice. The chapter begins with a discussion of bootstrap sampling methods. Bootstrap sampling was popularized by (Efron and Tibshirani 1994) and remains a practical and useful technique that every simulation practitioner should be able understand and apply. Then, the chapter presents more details on the application of variance reduction techniques (VRTs). While some of the facilities of the KSL for the application of two variance reduction techniques (common random numbers and antithetic variates) have already been presented, this chapter will provide more details on the theory of those and other variance reduction techniques. In addition, the application of the techniques will be illustrated via some simple examples. As discussed in Chapter 2, the KSL has exceptional functionality for the generation from uni-variate distributions. In this chapter, the capabilities for generating from multi-variate distributions

will be reviewed and illustrated via some simple examples. Finally, a brief discussion of the important topic of Markov Chain Monte Carlo (MCMC) will be provided. Then, the KSL's framework for performing MCMC will be illustrated. This chapter assumes that the reader is familiar with the general concepts presented in Appendix A, Chapter 2, and Chapter 3.

> **Note**
>
> This chapter provides example code of using the KSL to implement advanced Monte Carlo techniques. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

9.1. Bootstrap Methods

Bootstrapping is a statistical procedure that resamples from a sample to create many simulated samples. Although there are some parametric bootstrapping approaches, bootstrapping is generally considered a non-parametric Monte Carlo method that estimates characteristics of a population by resampling. The samples from the original sample are used to make inferences about the population and about the statistical properties computed from the original sample. The intuitions associated with bootstrapping come from a deep understanding of what it means to sample.

The concepts of estimation theory presented in Section 3.3 of Chapter 3 are based on having a random sample. Let x_i represent the i^{th} observations in a sample, $x_1, x_2, \cdots, x_n$ of size n. Represent the *random variables* in the sample as $X_1, X_2, \cdots, X_n$. The random variables form a **random sample**, if 1) the X_i are independent random variables and 2) every X_i has the same probability distribution. We assume that the random sample comes from some *population*. In statistical estimation theory, a population is

the complete set of items from which we desire to make inferences. We assume that the population represents a set of things that have one or more characteristics in common. A population may be finite or infinite. In the case of a finite population, the size of the population may be so large or it may be too time consuming or costly to enumerate all elements of the population. However, conceptually, for a finite population, we can assign an integer to each element and thus count the elements and the final number of elements will be finite number. In the case of an infinite population, the definition of the underlying characteristics of the items makes enumeration impossible. That is, we cannot represent the number of elements in the population with a finite integer.

The concept of a finite population and an infinite population is sometimes confused with the concepts of discrete and continuous random variables. A continuous random variable has (by the underlying characteristics of its elements) an infinite population. For example, consider a random variable representing the time of arrival, A_i. Since the underlying quantity is *time*, we note that time (conceptually) is a continuous thing, it is infinitely divisible and has no limit (at least based on our current understand of physics). A discrete random variable can have either a finite or an infinite population. Consider for example, the number of heads within 100 flips of a coin. This random variable is governed by a binomial distribution, and can take on any of the values $0, 1, 2, \cdots, n$ where $n = 100$, the (finite) number of trials. In this case, the population (set of possible values) is finite. However, now consider a geometric random variable. The set of possible values for a geometric random variable $0, 1, 2, \cdots,$. That is, we could (in theory, however unlikely), wait an "infinite" amount of time until the first success occurs. The population of a geometric random variable (although discrete) is infinite.

Sampling theory and statistical inference are applied to situations involving both finite and infinite populations. In many of the cases of finite populations, it is impractical to enumerate the items. In the case of an infinite population, it is *impossible* to enumerate the items. Thus, we are forced to sample from the population. That is, select through some mechanism, a

subset of the elements of the population. If we select the elements in the sample in such a manner that each element in the sample is equally likely to have come from the population, then we have a random sample.

In essence, bootstrapping takes this notion one step further. In bootstrapping, we **assume** that some original random sample is *essentially* the population. That is, we assume that a random sample, if large enough, can act as the actual population. Thus, sampling from the sample is, in essence, sampling from the population. This sampling and resampling process can provide information about the properties of statistical quantities computed from the original sample. Bootstrapping assumes that if the original sample is representative of the true population, then statistics generated from samples of the original sample of the population provide observations of the *sampling* distribution.

Let's formalize these concepts with some notation and an informal overview of some of the theory associated with bootstrapping. The interested reader should refer to (Efron and Tibshirani 1986), (Efron and Tibshirani 1994), and (Davison AC 1997) for more detailed treatments. This presentation follows somewhat the tutorial (Wehrens and Buydens 2000). Let the cumulative distribution function of some random variable, X, be denoted as F. Let $x_1, x_2, \cdots, x_n$ be a set of observed values of X from F. Suppose that there is some unknown parameter θ that represents a population parameter of interest. For example, θ might be the mean, median, correlation, etc. Suppose that θ can be expressed or computed by some function $t(\cdot)$ applied to the distribution function F. That is, we can write $\theta = t(F)$. For example, in the case of the population mean, $E[X] = \theta$, $t(\cdot)$ is the expectation function such that $\theta = \int x dF(x)$.

In general, because the underlying probabilistic description of the population may not be known, we usually do not know F. However, we are able to sample (through some mechanism) observations $\vec{x} = (x_1, x_2, \cdots, x_n)$ from F via the random variable X. This sample represents the empirical distribution, $F_n(x)$, of F. That is, $F_n(x)$ is an estimate of F.

We can either use $F_n(x)$ to estimate θ or use some function $u(\cdot)$ applied to the sample to estimate θ. That is, we can formulate an estimator, $\widehat{\theta} = t(F_n) = u(\vec{x})$. For example in the case of the population mean, $E[X] = \theta$, $u(\cdot)$ is the sample mean, $\bar{X} = \frac{1}{n}\sum_i^n X_i$. In bootstrapping, this equivalence is called the *plug-in principle*. As we will see, bootstrapping makes a more explicit use of the empirical distribution, F_n.

Since we have that $(x_1, x_2, \cdots, x_n)$ is a set of observed values of X from F, let's define $\vec{x}^* = (x_1^*, x_2^*, \cdots, x_n^*)$ as a set of observations from F_n. That is, define X^* to be a possible observation selected at random from $(x_1, x_2, \cdots, x_n)$. Thus, $F_n(x) = P(X^* = x_i) = 1/n$ for $i = 1, 2, \cdots, n$. Let $\vec{X}^* = (X_1^*, X_2^*, \cdots, X_n^*)$ be a random sample of size n from $P(X^* = x_i)$. Then, the random variables $(X_1^*, X_2^*, ...X_n^*)$ are independent and uniformly distributed over the set $(x_1, x_2, \cdots, x_n)$. Notice that values within $(x_1^*, x_2^*, \cdots, x_n^*)$ may be repeated because the sampling is done with replacement (to ensure that they are uniformly likely). Note also that we can form as many samples $(x_1^*, x_2^*, \cdots, x_n^*)$ as we want by repeating the sampling procedure from F_n.

The empirical distribution, $F_n(x)$, is an estimator for $F(x)$ and it can be shown that F_n is a sufficient statistic for F. To summarize, F yields random variables X of which we have a sample, which yields F_n. When we sample from F_n, we get random variables X^*. These samples from F_n also have an empirical cumulative distribution function (ECDF). Let's denote the ECDF of F_n as F_n^*. Thus, a statistic formed from $(x_1^*, x_2^*, \cdots, x_n^*)$ will be a *possible* statistic computed on $(X_1, X_2, \cdots, X_n)$. Since a statistic computed from $(x_1^*, x_2^*, \cdots, x_n^*)$ yields a *possible* statistic computed on $(X_1, X_2, \cdots, X_n)$, these are possible observations from the sampling distribution of the estimator. This allows for the estimation of some parameter θ of $F(x)$ and the variability of the estimator by resampling. This is the basis of the bootstrapping principle.

We call the samples $(x_1^*, x_2^*, \cdots, x_n^*)$ bootstrap samples. Let $x^{*(b)}$ be the b^{th} bootstrap sample from $F_n(x)$, with $b = 1, 2, \cdots, B$, where B is the number of bootstrap samples. Let $\widehat{\theta}$ be some estimator of θ and let $\widehat{\theta}_b^*$ be an

observation of $\hat{\theta}$ based on the b^{th} bootstrap sample, $x^{*(b)}$. The quantities, $\hat{\theta}_b^*$ for $b = 1, 2, \cdots, B$ are called the bootstrap replicates (of the estimator).

The bootstrap estimate of the sampling distribution of $\hat{\theta}_b^*$, $F_{\hat{\theta}_b^*}(\cdot)$ is the empirical distribution of $(\theta_1^*, \theta_2^*, \cdots, \theta_n^*)$. Thus, $F_{\hat{\theta}_b^*}(\cdot)$ provides information on the sampling distribution of $\hat{\theta}$. The quality of the information depends entirely on the quality of the original sample $(x_1, x_2, \cdots, x_n)$ of X from F. Therefore, $\hat{\theta}$ is a statistic computed from the original sample, and $\hat{\theta}^*$ is a statistic computed from the resample. The bootstrap principle assumes that $F^* \approx F$ and the variation in $\hat{\theta}$ is well approximated by the variation in $\hat{\theta}^*$.

If the bootstrap principle holds, then we can use the bootstrapping process to develop an estimate of the standard error of our estimator. We have an estimator, $\hat{\theta}$. It has some variance, $var(\hat{\theta})$ and standard deviation, $\sqrt{var(\hat{\theta})}$. This quantity is called the standard error of the estimator. That is, $se(\hat{\theta}) = \sqrt{var(\hat{\theta})}$. The bootstrap estimator of the standard error of the estimator,$\hat{\theta}$, is:

$$\widehat{se(\hat{\theta})} = \sqrt{\frac{1}{B-1}\sum_{b=1}^{B}\left(\hat{\theta}_b^* - \bar{\hat{\theta}^*}\right)^2}$$

where the quantity, $\bar{\hat{\theta}^*}$, is the average over the computed bootstrap estimates of the estimator:

$$\bar{\hat{\theta}^*} = \frac{1}{B}\sum_{b=1}^{B}\hat{\theta}_b^*$$

As you can see, the bootstrap estimator for the standard error of the estimator, $\widehat{se(\hat{\theta})}$, is simply the sample standard deviation of the bootstrap replicates.

The bootstrapping process not only provides information about the standard error of the estimator, $\hat{\theta}$, it can also provide information about the bias of the estimator. If $\hat{\theta}$ is an unbiased estimator of θ then $E[\hat{\theta}] = \theta$. Thus, the bias of an estimator, $\hat{\theta}$, for θ is:

$$bias(\hat{\theta}) = E[\hat{\theta} - \theta] = E[\hat{\theta}] - \theta$$

To estimate the bias, the bootstep process uses the bootstrap replicates of $\hat{\theta}_b^*$ to estimate the sampling distribution of $\hat{\theta}$ to get:

$$\widehat{bias}(\hat{\theta}) = \bar{\hat{\theta}}^* - \hat{\theta}$$

where $\hat{\theta}$ is the estimate of θ computed from the original sample,$(x_1, x_2, . \cdots, x_n)$. The bias and standard error of the estimator provide information about the accuracy and precision of the estimator. We can also provide confidence intervals for the estimator based on the bootstrapping process.

To derive confidence intervals from the bootstrapping process consider the probability distribution associated with $\hat{\theta} - \theta$. Suppose γ_α denotes the α-percentile of the distribution of $\hat{\theta} - \theta$, then a confidence interval for θ is based on the probability statement:

$$P\left(\gamma_{\alpha/2} \leq \hat{\theta} - \theta \leq \gamma_{1-\alpha/2}\right) = 1 - \alpha$$

After rewriting this statement, we have an interval of the form:

$$\hat{\theta} - \gamma_{1-\alpha/2} \leq \theta \leq \hat{\theta} - \gamma_{\alpha/2}$$

Within the bootstrap literature a number of approaches have been developed that provide *approximate* confidence intervals based on the previously noted form. These include:

- standard normal bootstrap confidence interval (SNBCI)

- basic bootstrap confidence interval (BBCI)
- bootstrap percentile confidence interval (BPCI)
- bootstrap bias corrected and acceleration adjusted (BCa)
- bootstrap-t confidence interval (BTCI), sometimes called the percentile-t confidence interval

The standard normal bootstrap confidence interval is useful to consider first because of its simplicity; however, the quality of the confidence intervals can be poor unless its assumptions hold. Based on the central limit theorem, we know that if the estimator, $\hat{\theta}$, is the sample average, then for large sample sizes, we have:

$$Z = \frac{\hat{\theta} - E[\hat{\theta}]}{se(\hat{\theta})}$$

with $Z \approx N(0,1)$. Therefore if $\hat{\theta}$ is unbiased, an approximate $100 \times (1-\alpha)\%$ confidence interval will have the form:

$$\hat{\theta} - z_{1-\alpha/2} se(\hat{\theta}) \leq \theta \leq \hat{\theta} + z_{1-\alpha/2} se(\hat{\theta})$$

To apply bootstrapping to compute this interval, we simply estimate $se(\hat{\theta})$ with the bootstrap estimator of the standard error of the estimator, $\widehat{se(\hat{\theta})}$.

The basic bootstrap confidence interval (BBCI), is also relatively simple. This confidence interval assumes that the distribution of $\hat{\theta} - \theta$ is *approximately* the same as $\hat{\theta} - \theta^*$. Suppose that $\hat{\theta}$ is an estimator for θ.

$$
\begin{aligned}
1-\alpha &\approx P\left(\hat{\theta}^*_{\alpha/2} \leq \hat{\theta}^* \leq \hat{\theta}^*_{1-\alpha/2}\right) \\
&= P\left(\hat{\theta}^*_{\alpha/2} - \hat{\theta} \leq \hat{\theta}^* - \hat{\theta} \leq \hat{\theta}^*_{1-\alpha/2} - \hat{\theta}\right) \\
&= P\left(\hat{\theta} - \hat{\theta}^*_{\alpha/2} \geq \hat{\theta} - \hat{\theta}^* \geq \hat{\theta} - \hat{\theta}^*_{1-\alpha/2}\right) \\
&\approx P\left(\hat{\theta} - \hat{\theta}^*_{\alpha/2} \geq \theta - \hat{\theta} \geq \hat{\theta} - \hat{\theta}^*_{1-\alpha/2}\right) \\
&= P\left(2\hat{\theta} - \hat{\theta}^*_{\alpha/2} \geq \theta \geq 2\hat{\theta} - \hat{\theta}^*_{1-\alpha/2}\right)
\end{aligned}
$$

Thus, we have the lower limit as $L = 2\hat{\theta} - \hat{\theta}^*_{1-\alpha/2}$ and the upper limit as $U = 2\hat{\theta} - \hat{\theta}^*_{\alpha/2}$. We can estimate $\hat{\theta}^*_{\alpha/2}$ and $\hat{\theta}^*_{1-\alpha/2}$ by the percentiles of the bootstrap replicates distribution, which yields a final BBCI approximate $100 \times (1-\alpha)\%$ confidence interval as:

$$2\hat{\theta} - \hat{\theta}^*_{1-\alpha/2} \leq \theta \leq 2\hat{\theta} - \hat{\theta}^*_{\alpha/2}$$

where θ^*_p is the p^{th} sample quantile from the empirical distribution of the bootstrap replicates.

The percentile bootstrap confidence interval (PBCI) uses the empirical distribution of the bootstrap replicates to estimate the quantiles for the sampling distribution of $\hat{\theta}$. Suppose $(\theta^*_1, \theta^*_2, \cdots, \theta^*_n)$ are the bootstrap replicates of $\hat{\theta}$. From the empirical distribution of $(\theta^*_1, \theta^*_2, \cdots, \theta^*_n)$ compute the $\alpha/2$ quantile, say $\theta^*_{\alpha/2}$ and the $1-\alpha/2$ quantile, $\theta^*_{1-\alpha/2}$. Then, approximate $100 \times (1-\alpha)\%$ PBCI confidence interval is:

$$(\hat{\theta}^*_{1-\alpha/2}, \hat{\theta}^*_{\alpha/2})$$

The bootstrap bias corrected and acceleration adjusted (BCa) interval modifies the percentile bootstrap confidence interval by correcting for bias and adjusting for possible skewness in the sampling distribution of the estimator. The theory and quality of this confidence interval is discussed in (Efron and Tibshirani 1994). An approximate $100 \times (1-\alpha)\%$ BCa confidence interval is $(\hat{\theta}^*_{\alpha_1}, \hat{\theta}^*_{\alpha_2})$ where α_1 and α_2 are defined as follows:

$$\alpha_1 = \Phi\left(\hat{z}_0 + \frac{\hat{z}_0 + z_{\alpha/2}}{1 - \hat{a}(\hat{z}_0 + z_{\alpha/2})}\right)$$

$$\alpha_2 = \Phi\left(\hat{z}_0 + \frac{\hat{z}_0 + z_{1-\alpha/2}}{1 - \hat{a}(\hat{z}_0 + z_{1-\alpha/2})}\right)$$

where $z_p = \Phi^{-1}(p)$. The bias correction factor, z_0, is given as follows.

$$z_0 = \Phi^{-1}\left(\frac{1}{B}\sum_{b=1}^{B} I(\hat{\theta}^*_b < \hat{\theta})\right)$$

where $I(\cdot)$ is an indicator function. The acceleration adjustment factor, $\hat{a}$, is given as follows.

$$\hat{a} = \frac{\sum_{i=1}^{n} \left(\bar{\theta}_{(\cdot)} - \theta_{(i)}\right)^3}{6\sum_{i=1}^{n}\left(\left(\bar{\theta}_{(\cdot)} - \theta_{(i)}\right)^2\right)^{3/2}}$$

The quantity $\bar{\theta}_{(\cdot)}$ is the mean of the estimates of the leave one out jackknife samples, with:

$$\bar{\theta}_{(\cdot)} = \frac{1}{n}\sum_{i=1}^{n}\hat{\theta}_{(i)}$$

and $\hat{\theta}_{(i)}$ is the i^{th} jackknife replicate. The jackknife replicates, $\hat{\theta}_{(i)}$ for $i = 1, 2, \cdots, n$ are computed by applying the estimator, $\hat{\theta} = T_n(x)$ on the jackknife samples. A jackknife sample, $\vec{x}_{(i)}$, is a subset of $(x_1, x_2, \cdots, x_n)$ that leaves out the i^{th} observation such that:

$$\vec{x}_{(i)} = (x_1, \cdots, x_{i-1}, x_{i+1}, \cdots, x_n)$$

Thus, $\hat{\theta}_{(i)} = T_{n-1}(\vec{x}_{(i)})$ for $i = 1, 2, \cdots, n$. Jackknifing is another form of resampling that is often applied to reduce the bias in an estimator. We will not discuss the theory of jackknifing in this text, but we will point out that the KSL has support for the approach.

The bootstrap-t (BTCI) provides an approximate confidence interval by considering the distribution of

$$\frac{\hat{\theta} - \theta}{se(\hat{\theta})}$$

As we know from basic statistical theory, if $\hat{\theta}$, is the sample average and the $se(\hat{\theta})$ is estimated by the sample error when the observations are IID normal, then this ratio should have a student-t distribution. In the PTCI approach, does not assume a student-t distribution, but rather uses the bootstrap observations of:

$$t_b^* = \frac{\hat{\theta}_b^* - \hat{\theta}}{\widehat{se}(\hat{\theta}_b^*)}$$

Notice that $\widehat{se}(\hat{\theta}_b^*)$ is the standard error of the b^{th} *bootstrap sample* and that it may not have a closed form. If $\hat{\theta}$, is the sample average, then

we can use the sample standard error of the b^{th} bootstrap sample, $\vec{x}_b^* = (x_1^*, x_2^*, ...x_n^*)$.

In general for this approach, the reference distribution is formed from resamples from the bootstrap sample. As usual, suppose that $(x_1, x_2, ...x_n)$ is the original sample. Then, the $100 \times (1-\alpha)\%$ bootstrap t-interval is:

$$(\hat{\theta} - t^*_{1-\alpha/2}\widehat{se}(\hat{\theta}^*), \hat{\theta} - t^*_{\alpha/2}\widehat{se}(\hat{\theta}^*))$$

We compute $\widehat{se}(\hat{\theta}^*)$, $t^*_{\alpha/2}$, and $t^*_{1-\alpha/2}$ as follows:

1. Compute $\hat{\theta}$ from $\vec{x} = (x_1, x_2, ...x_n)$
2. For $b = 1$ to B

a. Sample from $\vec{x}_b^*$ with replacement from $\vec{x}$.
b. Compute θ_b^* from $\vec{x}_b^*$.
c. Estimate the standard error $\widehat{se}(\hat{\theta}_b^*)$ from sampling from $\vec{x}_b^*$. In the case of $\hat{\theta}$ being the mean, then the standard error of the bootstrap sample can be directly computed, without sampling.
d. Compute

$$t_b^* = \frac{\theta_b^* - \hat{\theta}}{\widehat{se}(\hat{\theta}_b^*)}$$

3. The sample $(t_1^*, t_2^*, \cdots, t_B^*)$ is the reference distribution for the bootstrap t-statistic. From these observations, find $t^*_{\alpha/2}$, and $t^*_{1-\alpha/2}$.
4. Compute $\widehat{se}(\hat{\theta}^*)$ as the sample standard deviation of the θ_b^* for $b = 1, 2, \cdots, B$
5. Compute the confidence intervals

$$(\hat{\theta} - t^*_{1-\alpha/2}\widehat{se}(\hat{\theta}^*), \hat{\theta} - t^*_{\alpha/2}\widehat{se}(\hat{\theta}^*))$$

(Efron and Tibshirani 1986) discuss many of the characteristics of the different bootstrap confidence intervals. Of the five presented here, the

bootstrap percentile confidence interval is often utilized; however, because the BCa essentially corrects for bias and skewness issues for the BPCI, we would recommend utilizing the BCa. The next recommended confidence interval would be the bootstrap-t; however, because it often requires an extra bootstrapping step to estimate $\widehat{se}(\hat{\theta}_b^*)$ it can require a substantial amount of extra computation. Thus, the bootstrap-t has not found wide acceptance in practice, except in the case of when the estimator is the sample average.

9.1.1. Bootstrapping Using the KSL

In this section, we will explore how the KSL supports the application of bootstrapping. First, we will illustrate how to do bootstrapping using basic KSL constructs. This is intended to illustrate the overall bootstrapping process. Then we will discuss KSL constructs that encapsulate the bootstrapping process within classes. In essence, bootstrapping is just sampling again and again from some discrete population. Thus, the KSL `DPopulation` class can serve as a starting point. Alternatively, the KSL `EmpiricalRV` class can be used. Let's start with a simple example that has an original sample size of 10 elements. For example, assume through some sampling process we have selected 10 students (at random) and counted the number of books in their backpack. The following code illustrates placing the data in an array and computing some basic statistics related to the sample.

Example 9.1 (Bootstraping Example 1)**.** This example defines a population of values within an array. Then, the example uses the `DPopulation` class to sample from the population. Statistics on the samples are captured and used to form a confidence interval.

```
// make a population for illustrating bootstrapping
val mainSample = doubleArrayOf(6.0, 7.0, 5.0, 1.0, 0.0, 4.0,
 ↪  6.0, 0.0, 6.0, 1.0)
```

```
println("Sample of size 10 from original population")
println(mainSample.joinToString())
println()
// compute statistics on main sample
val mainSampleStats = Statistic(mainSample)
println("Main Sample")
println("average = ${mainSampleStats.average}")
println("90% CI =
 ↪  ${mainSampleStats.confidenceInterval(.90)}")
println()
```

Running this code results in the following:

```
Sample of size 10 from original population
6.0, 7.0, 5.0, 1.0, 0.0, 4.0, 6.0, 0.0, 6.0, 1.0

Main Sample
average = 3.5999999999999996
90% CI = [1.978733917471777, 5.221266082528222]
```

In the notation that we have been using $\bar{x} = \hat{\theta} = 3.5\bar{9}$. The sample size is small and it is very unlikely that the distribution of the number books in a student's backpack would be normally distributed. Therefore, the reported confidence interval is certainly approximate. The KSL `Statistic` class reports its confidence interval assuming a Student-T distribution. We would like to use bootstrapping to produce a confidence interval. We can get started by treating our original sample as if it is the population and sampling from it. Here is some KSL code to illustrate this process.

```
// make the sample our pseudo-population
val samplePopulation = DPopulation(mainSample)
val bootStrapAverages = mutableListOf<Double>()
```

```
// illustrate 10 bootstrap samples
println("BootStrap Samples:")
for (i in 1..10){
    val bootStrapSample = samplePopulation.sample(10)
    val avg = bootStrapSample.average()
    println("sample_$i  = (${bootStrapSample.joinToString()})
 ↪  with average = $avg")
   bootStrapAverages.add(avg)
}
println()
val lcl =
 ↪  Statistic.percentile(bootStrapAverages.toDoubleArray(),
 ↪  0.05)
val ucl =
 ↪  Statistic.percentile(bootStrapAverages.toDoubleArray(),
 ↪  0.95)
val ci = Interval(lcl, ucl)
println("Percentile based 90% ci = $ci")
```

In this code, we first create an instance of `DPopulation` using the array of observations of the book count data. Then, a list is created to hold the averages of each bootstrap sample. The loop creates 10 bootstrap samples and captures the average of each sample. Notice that the `DPopulation` is used to sample *with replacement* a sample of size 10. It is important that the sample size, 10, be the same as the size of the original sample because our confidence statements are about the original sample (with size 10). Given the bootstrap averages are captured in a list, `bootStrapAverages`, we can then use the percentile method to get the lower and upper limits for the confidence interval based on the bootstrap reference distribution.

```
BootStrap Samples:
sample_1  = (7.0, 1.0, 1.0, 6.0, 5.0, 4.0, 0.0, 1.0, 7.0, 0.0) with average = 3.2
sample_2  = (4.0, 0.0, 1.0, 5.0, 6.0, 1.0, 5.0, 6.0, 1.0, 7.0) with average = 3.6
```

```
sample_3  = (0.0, 7.0, 6.0, 5.0, 6.0, 7.0, 4.0, 5.0, 6.0, 0.0) with average = 4.6
sample_4  = (6.0, 6.0, 6.0, 0.0, 4.0, 6.0, 6.0, 1.0, 1.0, 0.0) with average = 3.6
sample_5  = (4.0, 6.0, 6.0, 1.0, 1.0, 4.0, 0.0, 6.0, 7.0, 4.0) with average = 3.9
sample_6  = (6.0, 0.0, 5.0, 7.0, 0.0, 6.0, 7.0, 6.0, 0.0, 6.0) with average = 4.3
sample_7  = (0.0, 6.0, 6.0, 1.0, 1.0, 0.0, 0.0, 5.0, 6.0, 0.0) with average = 2.5
sample_8  = (6.0, 6.0, 1.0, 0.0, 0.0, 6.0, 6.0, 0.0, 4.0, 6.0) with average = 3.5
sample_9  = (6.0, 4.0, 4.0, 1.0, 6.0, 7.0, 4.0, 6.0, 6.0, 7.0) with average = 5.1
sample_10  = (7.0, 1.0, 7.0, 5.0, 6.0, 0.0, 1.0, 6.0, 1.0, 0.0) with average = 3.4

Bootstrap sample averages
3.2, 3.6, 4.6, 3.6, 3.9, 4.3, 2.5, 3.5, 5.1, 3.4

Percentile based 90% ci = [2.5, 5.1]
```

The results show 10 bootstrap samples. Notice that due to sampling with replacement, we can have values in the bootstrap samples that repeat and that it is clearly possible that some observations within the original sample, do not appear in the bootstrap samples. It is important to note that this bootstrap confidence interval is for our original estimate, $\bar{x} = \hat{\theta} = 3.5\bar{9}$. The bootstrap sample averages, $(3.2, 3.6, 4.6, 3.6, 3.9, 4.3, 2.5, 3.5, 5.1, 3.4)$ represent the bootstrap estimate of the sampling distribution of $\hat{\theta}^*_b$, $F_{\hat{\theta}^*_b}(\cdot)$, which we are using as an approximation for the sampling distribution of $\hat{\theta}$.

The idea of repeatedly sampling from a given population has been implemented into a set of classes within the KSL that will perform the bootstrapping process and compute a variety of quantities (including confidence intervals) based on the bootstrap samples. Figure 9.1 presents the `BootstrapEstimateIfc` interface, the main implementation of this interface (`Bootstrap`), and a data class, `BootstrapEstimate,` that is useful when building approaches that require bootstrapping.

The `Bootstrap` class requires an array of doubles that represents the original data from which resampling will occur. While implementing the

Figure 9.1.: KSL Bootstrap Classes

`BootstrapEstimateIfc` interface, the `Bootstrap` class also implements the `RNStreamControlIfc`, and `RNStreamChangeIfc` interfaces, which allow for the control of the underlying random numbers used within the sampling. It is useful to review the constructor of the `Bootstrap` class.

```
open class Bootstrap(
    originalData: DoubleArray,
    val estimator: BSEstimatorIfc = BSEstimatorIfc.Average(),
    streamNumber: Int = 0,
    val streamProvider: RNStreamProviderIfc =
     ↪ KSLRandom.DefaultRNStreamProvider,
    name: String? = null
) : IdentityIfc by Identity(name), RNStreamControlIfc,
 ↪ BootstrapEstimateIfc, StreamNumberIfc
```

As noted in the constructor, the class requires an array holding the original data, an instance of the `BSEstimatorIfc` interface, the stream for the random sampling and an optional name. The `BSEstimatorIfc` interface is a functional interface with an `estimate()` function. Thus, this implementation assumes that the estimate can be computed from a uni-variate array of data and produces a single estimated quantity. The KSL also supports multiple estimated quantities from a uni-variate array of data.

As can be noted from the default parameter for the estimator, the `BSEstimatorIfc` interface provides some basic statistical estimators (average, variance, median, minimum, maximum). Of course, the user should be able to implement others as needed.

```
fun interface BSEstimatorIfc {
    fun estimate(data: DoubleArray): Double
}
```

According to Figure 9.1, the `Bootstrap` class also will compute the following:

- the bootstrap replicates, $\hat{\theta}_b^*$
- the bootstrap differences, $\hat{\theta}_b^* - \hat{\theta}$
- across bootstrap statistics, statistics on the replicates
- statistics for each bootstrap sample
- boot strap bias estimate
- boot strap standard error estimate
- normal bootstrap confidence interval
- basic bootstrap confidence interval
- percentile bootstrap confidence interval
- BCa bootstrap confidence interval
- bootstrap-t confidence interval

Once constructing an instance of the `Bootstrap` class is complete, you can call the `generateSamples()` function. The following code shows the `generateSamples()` function. The function requires the number of bootstrap samples to generate, and optionally, you can have the bootstrap samples saved, and indicate if the bootstrap-t samples are required.

```
    fun generateSamples(numBootstrapSamples: Int,
     ↪  saveBootstrapSamples: Boolean = false,
        numBootstrapTSamples: Int = 0
    ) {
        require(numBootstrapSamples > 1) { "The number of
↪   bootstrap samples must be greater than 1" }
        if (numBootstrapTSamples > 0) {
            require(numBootstrapTSamples > 1) { "The number
↪   of bootstrap-t samples must be greater than 1" }
        }
        this.numBootstrapSamples = numBootstrapSamples
        myAcrossBSStat.reset()
        myBSEstimates.clearData()
        myStudentizedTValues.clearData()
        for (s in myBSArrayList) {
            s.clearData()
```

```
        }
        myBSArrayList.clear()
        originalDataEstimate =
↪   estimator.estimate(myOriginalData)
        for (i in 0 until numBootstrapSamples) {
            val sample: DoubleArray =
             ↪  myOriginalPop.sample(myOriginalPop.size())
            val x = estimator.estimate(sample)
            myAcrossBSStat.collect(x)
            myBSEstimates.save(x)
            if (saveBootstrapSamples) {
                val das = DoubleArraySaver()
                das.save(sample)
                myBSArrayList.add(das)
            }
            if (numBootstrapTSamples > 1) {
                bootstrapTSampling(numBootstrapTSamples, x,
↪   sample)
            }
            innerBoot(x, sample)
        }
    }
```

This code is rather straightforward, having a loop to implement the sampling. In essence, it is not much different conceptually than the previously code using the `DPopulation` class.

Example 9.2 (Bootstraping Example 2)**.** To setup and run bootstrapping using the `Bootstrap` class, we have the following code:

```
    // make a population for illustrating bootstrapping
    val mainSample = doubleArrayOf(6.0, 7.0, 5.0, 1.0, 0.0,
     ↪  4.0, 6.0, 0.0, 6.0, 1.0)
```

```
    println("Sample of size 10 from original population")
    println(mainSample.joinToString())
    println()
    // compute statistics on main sample
    val mainSampleStats = Statistic(mainSample)
    println("Main Sample")
    println("average = ${mainSampleStats.average}")
    println("90% CI =
↪   ${mainSampleStats.confidenceInterval(.90)}")
    println()
    // now to the bootstrapping
    val bs = Bootstrap(mainSample, estimator =
     ↪  BSEstimatorIfc.Average(), streamNumber = 3)
    bs.generateSamples(400)
    println(bs)
```

Running this code results in the following:

```
Sample of size 10 from original population
6.0, 7.0, 5.0, 1.0, 0.0, 4.0, 6.0, 0.0, 6.0, 1.0

Main Sample
average = 3.5999999999999996
90% CI = [1.978733917471777, 5.221266082528222]

-------------------------------------------------------
Bootstrap statistical results:
label = ID_5
-------------------------------------------------------
statistic name = ID_5
number of bootstrap samples = 400
size of original sample = 10
original estimate = 3.5999999999999996
```

```
bias estimate = 0.03775000000000084
across bootstrap average = 3.6377500000000005
bootstrap std. err. estimate = 0.8329071215825669
default c.i. level = 0.95
norm c.i. = [1.967532037915134, 5.232467962084865]
basic c.i. = [1.8999999999999995, 5.1]
percentile c.i. = [2.1, 5.3]
BCa c.i. = [1.9, 5.0]
-----------------------------------------------------
```

The example leaves the `numBootstrapTSamples` parameter at its default of 0, so the bootstrap-t interval is not computed and the output reports the other four intervals. The four intervals differ because each rests on a different approximation. The BCa interval, [1.9, 5.0], adjusts the percentile interval, [2.1, 5.3], for bias and skewness. The main question that comes up when bootstrapping is what value should be used for the number of bootstrap samples, B. (Efron and Tibshirani 1994) provide some guidance indicating a range at least starting in the hundreds. There has been some work that suggests the quality of the process is a function of n, the size of the original sample. The work suggests that larger n requires larger B. The relationship $B \approx 40n$ has been suggested.

As previously mentioned, the KSL also supports the estimation of multiple quantities from the sample. This functionality is used extensively within the distribution modeling functionality of the KSL to provide confidence intervals on the parameters of distributions. Figure 9.2 presents the `BootstrapSampler` class, which will compute multiple estimators on a sample using the `MVBSEstimatorIfc` interface.

As shown in the following code, the `MVBSEstimatorIfc` interface defines a function that takes in a sample and returns many estimated quantities.

```
/**
 *  Given some data, produce multiple estimated statistics
```

Figure 9.2.: KSL BootstrapSampler Classes

```
 *  from the data and store the estimated quantities in
 *  the returned array. It is up to the user to interpret
 *  the array values appropriately.
 */
interface MVBSEstimatorIfc {
    /**
     * The name to associate with each dimension of the
     * array that is returned by estimate(). The names
     * should be unique. The order of the list of names
 ↪  should
     * match the order of elements in the returned array.
     */
    val names: List<String>

    fun estimate(data: DoubleArray): DoubleArray
}
```

Example 9.3 (Illustrating the BootstrapSampler Class)**.** The following code illustrates the use of the `BootstrapSampler` class. This provides bootstrap estimates for basic statistics.

```
val ed = ExponentialRV(10.0)
val data = ed.sample(50)
val stat = Statistic(data)
println(stat)
println()
val bss = BootstrapSampler(data, BasicStatistics())
val estimates = bss.bootStrapEstimates(300)
for(e in estimates){
  println(e.asString())
}
```

This code samples from an exponential distribution to get the initial sample

and then uses the `BootstrapSampler` class to provide bootstrap estimates for the basic statistics. The `BasicStatistics` class is just a simple implementation of the `MVBSEstimatorIfc` interface that computes the average, variance, minimum, maximum, skewness, kurtosis, lag-1 correlation, and lag-1 covariance.

```
@Suppress("unused")
class BasicStatistics(aName: String? = null) :
 ↪  MVBSEstimatorIfc, IdentityIfc by Identity(aName){

    private val stat = Statistic()

    override val names: List<String> = listOf(
        "average", "variance", "min", "max", "skewness",
         ↪  "kurtosis",
        "lag1Correlation", "lag1Covariance")

    override fun estimate(data: DoubleArray): DoubleArray {
        stat.reset()
        stat.collect(data)
        val array = DoubleArray(8)
        array[0] = stat.average
        array[1] = stat.variance
        array[2] = stat.min
        array[3] = stat.max
        array[4] = stat.skewness
        array[5] = stat.kurtosis
        array[6] = stat.lag1Correlation
        array[7] = stat.lag1Covariance
        return array
    }
}
```

This same approach could be used to add additional statistics. The output

from the sampling can be quite extensive and thus is not provided here. For a final discussion of bootstrapping, we discuss the KSL capabilities for bootstrapping cases and illustrate its application to bootstrapping a simple regression model.

The previous KSL bootstrapping functionality focused on uni-variate data. The same concepts can also be applied to multi-variate data. Since the data structures to hold multi-variate data can be complex, we focus on representing the population of items via identifiers. That is, we assume that each item in the population can be assigned a unique label and that the sampling and resampling process takes place on the set of identifiers. The sampled identifiers then can be used to index into some complex data structure to select the sampled items. Then from each item, a set of values are observed. Thus, each sampled item, i, results in a set of observations $\vec{x_i} = x_1, x_2, \cdots, x_m$, where m is the number of attributes observed about each item. In some bootstrapping literature, the set of sampled items is called the cases, with each item called a case. Thus, in this situation, the bootstrapping process is sometimes called case based bootstrapping.

Within a programming context, we can represent each item with an object, which has some attributes. The attributes represent the data observed about the object. Placing the objects into a collection, permits sampling from the collection. For example, the cases could be the rows in a regression data set. The data could be stored in arrays, in data frames, database tables, etc. As long as each item the population can be uniquely labeled, we can sample the labels, and then select the items based on their identifiers.

Figure 9.3 presents the `CaseBootstrapSampler` class, which will compute multiple estimators using case based sample via the `CaseBootEstimatorIfc` interface. This sampler has similar functionality as the `BootstrapSampler` class. As in the `BootstrapSampler` situation, we assume that the estimation process may result in one or more estimated quantities; however, in this situation, the estimation process requires a set of items from which the quantities will be computed.

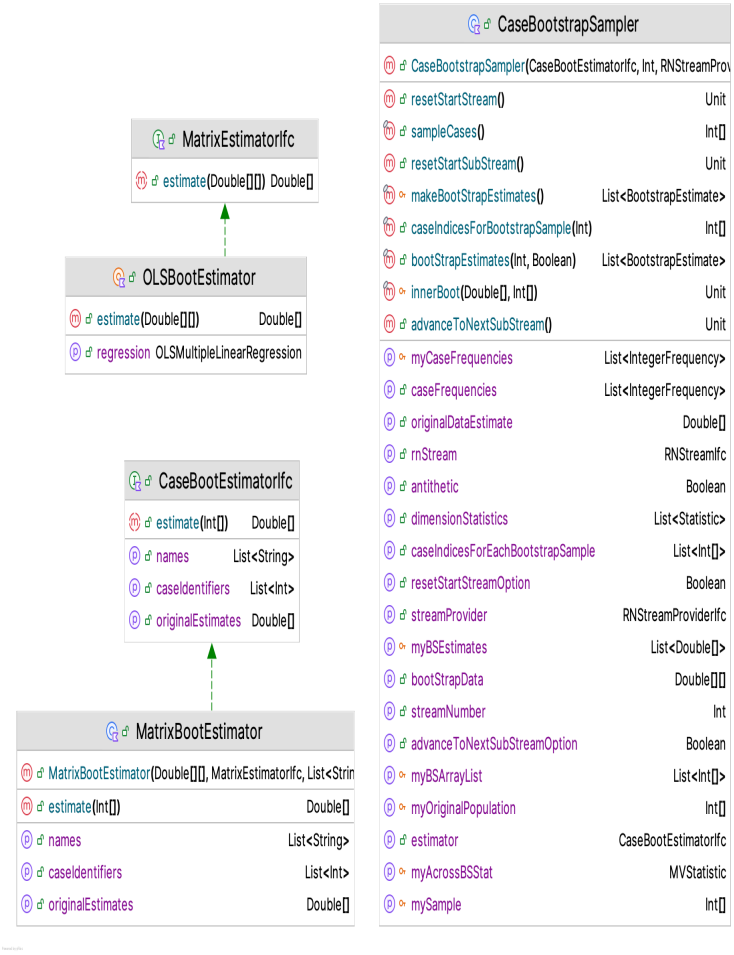


Figure 9.3.: KSL CaseBootstrapSampler Classes

It should be instructive to review the associated interfaces in Figure 9.3. The `CaseBootstrapSampler` class requires an instance of the `CaseBootEstimatorIfc` interface. The `CaseBootEstimatorIfc` interface does not define the data structure associated with the sampling process. Instead, it defines a list of identifiers that represent the finite sample from which bootstrap sampling occurs.

```
interface CaseBootEstimatorIfc {

    /**
     * The name to associate with each dimension of the
     * array that is returned by estimate(). The names
     * should be unique. The order of the list of names
↪   should
     * match the order of elements in the returned array.
↪   This
     * list is used to label the elements that are estimated.
     */
    val names: List<String>

    /**
     *  The estimates from the estimator based on the
↪   original (not resampled) data.
     */
    val originalEstimates: DoubleArray

    /**
     *  The set of case identifiers. This set must hold
↪   unique
     *  integers that serve as the sampling population.
↪   Elements (cases)
     *  are sampled with replacement from this set to specify
     *  the data that will be used in the estimation process.
     */
```

```
    val caseIdentifiers: List<Int>

    /**
     *  The [caseIndices] array contains the case identifiers
↪   that
     *  should be used to select the data on which the
↪   estimation
     *  process should be executed. The function produces an
     *  array of estimates during the estimation process,
↪   which
     *  are associated with the labels in the names list. The
     *  case indices array may have repeated case identifiers
     *  due to sampling with replacement.
     */
    fun estimate(caseIndices: IntArray): DoubleArray
}
```

As we can see from the previous Kotlin code, the `names` and `originalEstimates` properties have a similar functionality as in the `BootstrapSampler` class. The names are the labels for the things estimated from the sample.

The property, `caseIdentifiers` is a list holding the identifiers for the cases. For example, if the cases are rows in a regression dataset, then the list might hold the values $(0, 1, 2, \cdots, n-1)$, where n is the number of rows. That is, the list holds the *indices* of the rows. The sampling process will sample from the identifier population and use the identifiers to access the actual items. This approach necessitates a different `estimate()` function. Here, the estimation process, requires an array of integers that represent the identifiers of the cases selected to be included in the estimation process. Note that because the sampling is with replacement, the same item can be selected multiple times (or not at all) during the bootstrap sampling. Conceptually, this is no different from what we have previously illustrated,

but now our population is simply a set of identifiers.

The class `MatrixBootEstimator` is simply an implementation of the `CaseBootEstimatorIfc` interface designed to work with a rectangular array (i.e. a matrix) of data, where the rows are the cases. The interface `MatrixEstimatorIfc` is a convenience interface for directly computing the statistical quantities from a supplied matrix. The selection process for `MatrixBootEstimator` is simply to select the rows as shown in the following code.

```
    override fun estimate(caseIndices: IntArray): DoubleArray
     ↪  {
        // select the rows of the matrix from the supplied
         ↪  indices
        val m = Array(matrix.size) { matrix[caseIndices[it]]
         ↪  }
        return matrixEstimator.estimate(m)
    }
```

The `CaseBootstrapSampler` class samples the indices (`mySample`) in the following code, which is then supplied to the `estimate(caseIndices: IntArray)` function.

```
    fun sampleCases(): IntArray {
        for (i in myOriginalPopulation.indices) {
            val index = rnStream.randInt(0,
             ↪  myOriginalPopulation.size - 1)
            mySample[i] = myOriginalPopulation[index]
        }
        return mySample
    }
```

Finally, the object `OLSBootEstimator,` in Figure 9.3, is an object that implements the `MatrixEstimatorIfc` interface an computes the regression

coefficients by assuming that the first column of the matrix is the regressor, Y, and the remaining columns represent the predictor matrix X. Because of the fact that we use the ordinary least squares estimation routines from the Hipparchus library, the X matrix does not need to include the column for estimating the intercept term.

Putting all of these concepts together, we can illustrate the bootstrapping of a regression estimation process. As shown in the following code, we generate some regression data. For simplicity, we assume normally distributed predictors and error.

Example 9.4 (Illustrating Case Based Bootstrapping)**.** This example illustrates case based bootstrapping for a linear regression model.

```
// first make some data for the example
val n1 = NormalRV(10.0, 3.0)
val n2 = NormalRV(5.0, 1.5)
val e = NormalRV()
// make a simple linear model with some error
val data = Array<DoubleArray>(100) {
    val x1 = n1.value
    val x2 = n2.value
    val y = 10.0 + 2.0*x1 + 5.0*x2 + e.value
    doubleArrayOf(y, x1, x2)
}
//data.write()
// apply bootstrapping to get bootstrap confidence
 ↪  intervals on regression parameters
val cbs = CaseBootstrapSampler(MatrixBootEstimator(data,
 ↪  OLSBootEstimator))
val estimates = cbs.bootStrapEstimates(399)
// print out the bootstrap estimates
for(be in estimates){
    println(be)
```

```
    }
```

The results of the bootstrapping process are as follows. As we can see, the confidence intervals for the known regression coefficients ($\beta_0 = 10.0, \beta_1 = 2.0, \beta_2 = 5.0$) do a reasonable job of covering the known quantities.

```
-----------------------------------------------------
Bootstrap statistical results:
-----------------------------------------------------
statistic name = b0
number of bootstrap samples = 399
size of original sample = 100
original estimate = 10.497046113278433
bias estimate = -0.030842849345942724
across bootstrap average = 10.46620326393249
bootstrap std. err. estimate = 0.6542295748425296
default c.i. level = 0.95
norm c.i. = [9.214779707932347, 11.779312518624518]
basic c.i. = [9.235300221696459, 11.756239973214825]
percentile c.i. = [9.23785225334204, 11.758792004860407]

-----------------------------------------------------
Bootstrap statistical results:
-----------------------------------------------------
statistic name = b1
number of bootstrap samples = 399
size of original sample = 100
original estimate = 1.905041830235976
bias estimate = 0.003230000187099291
across bootstrap average = 1.9082718304230752
bootstrap std. err. estimate = 0.05470066041236848
default c.i. level = 0.95
```

```
norm c.i. = [1.797830505810743, 2.012253154661209]
basic c.i. = [1.7863888875926772, 2.0016950545762637]
percentile c.i. = [1.8083886058956882, 2.0236947728792747]

-----------------------------------------------------
Bootstrap statistical results:
-----------------------------------------------------
statistic name = b2
number of bootstrap samples = 399
size of original sample = 100
original estimate = 5.125969379754494
bias estimate = -1.7595189984920268E-4
across bootstrap average = 5.125793427854645
bootstrap std. err. estimate = 0.07213142347485296
default c.i. level = 0.95
norm c.i. = [4.984594387476197, 5.267344372032791]
basic c.i. = [4.99197903898926, 5.278594489440505]
percentile c.i. = [4.973344270068483, 5.259959720519728]
```

For more of the theory and methods of bootstrapping within the context of regression, we refer the interested reader to the text, (Fox and Weisberg 2018). Since the purpose of this section was to illustrate how the KSL facilitates bootstrapping, the interested reader may want to explore more of the theory. From that standpoint, the books by (Efron and Tibshirani 1994) and (Davison AC 1997) are good starting points. See also the paper (Efron and Tibshirani 1986).

9.2. Variance Reduction Techniques

A variance reduction technique (VRT) represents a sampling strategy that has the overall goal of finding an estimator that will have lower variance than the typical estimator based on straightforward random sampling. The

objective of a VRT is to reduce the theoretical population variance of the **sampling** distribution associated with an estimator $\hat{\theta}$, of some population parameter θ. For example, in many instances, we are interested in the population mean, θ, and thus the standard estimator that is proposed is the sample mean $\hat{\theta} = \bar{Y}$. Suppose we have the following IID observations $(Y_1, Y_2, \cdots, Y_n)$ with $E[Y_i] = \theta$ and $\text{Var}[Y_i] = \sigma_Y^2$. Then the standard (or so called crude) estimator for θ is $\bar{Y}$ and the variance of the estimator is

$$\text{Var}[\bar{Y}] = \text{Var}\left[\frac{1}{n}\sum_{i=1}^{n} Y_i\right] = \frac{1}{n^2}\sum_{i=1}^{n}\text{Var}[Y_i] = \frac{n\,\text{Var}[Y_i]}{n^2} = \frac{\text{Var}[Y_i]}{n} = \frac{\sigma_Y^2}{n}$$

When applying a VRT, the goal is to do better than $\text{Var}[\bar{Y}]$. While it may be obvious, it is worth stating that the simplest variance reduction technique is to use a larger sample. Consider estimating the population mean, θ based on a random sample using:

$$\bar{Y}(n) = \frac{1}{n}\sum_{i=1}^{n} Y_i$$

Here, we have emphasized that the estimator, $\bar{Y}(n)$ is a function of the sample size, n. So consider two estimators, $\hat{\theta}_1 = \bar{Y}(n_1)$ and $\hat{\theta}_2 = \bar{Y}(n_2)$ where $n_1 < n_2$. Thus, we will have that:

$$\text{Var}[\bar{Y}(n_1)] = \text{Var}[\hat{\theta}_1] = \frac{\sigma_Y^2}{n_1}$$

and,

$$\text{Var}[\bar{Y}(n_2)] = \text{Var}[\hat{\theta}_2] = \frac{\sigma_Y^2}{n_2}$$

and thus because $n_1 < n_2$, we have that $\text{Var}[\bar{Y}(n_2)] < \text{Var}[\bar{Y}(n_1)]$. Thus, we have two estimators $\hat{\theta}_1$ and $\hat{\theta}_2$ where the variance of one estimator is

smaller than the variance of another. In this particular case, both of the estimators are unbiased, and not considering the extra time that may be required to sample the larger sample, we would prefer the estimator $\widehat{\theta}_2$ over the estimator $\widehat{\theta}_1$ because $\widehat{\theta}_2$ has lower variance. That is, the variance of its **sampling** distribution is lower. As noted, we have to ignore the possible extra computation time needed for $\widehat{\theta}_2$. In what follows, we will generally ignore computational issues, and in most of the VRT methods that will be presented, the assumption will be very reasonable. However, there is also the effort needed to implement a more "complicated" sampling method than straightforward random sampling. In some modeling situations, this extra implementation effort should not be ignored when considering the possible use of a VRT.

The important thing to remember is that the variance reduction refers to reducing the population variance of the *estimator* of θ. The parameter of interest does not have to be the mean. The parameter θ can be any parameter of the underlying population. For example, we might try to estimate the median, quantile, or the population variance σ_Y^2. One important point to remember, variance reduction methods do not affect the variability of the process (population). That is σ_Y^2 does not change. When applying a VRT, we are changing the variance associated with the *sampling distribution* of the estimator, $\widehat{\theta}$.

The major types of variance reduction techniques to be discussed in this section, include:

1. Manipulating randomness
    a. Common Random Numbers (CRN)
    b. Antithetic Variates (AV)
2. Exploiting process knowledge
    a. Indirect Estimation (IE)
    b. Control Variates (CV)
    c. Stratified and Post Stratified Sampling (PS)

d. Conditional Expectation (CE)
e. Importance Sampling (IS)

Manipulating randomness refers to the judicious use of the random number streams. Exploiting process knowledge refers to the general intuition that more information reduces uncertainty.

9.2.1. Common Random Numbers (CRN)

In Section 5.7.1.2, we already discussed the use of common random numbers when comparing two systems. Here we will review the key results and mention a couple of implementation issues.

CRN exploits the idea that if each alternative experiences the same randomness, then the difference between the two alternatives will be because of a true underlying difference, rather than because of the randomness. Define $\theta = \theta_1 - \theta_2$ as the parameter of interest, and define $D_i = X_i - Y_i$ as the difference between the i^{th} pairs of performance observations, for $i = 1, 2, \cdots, n$, where n is the number of observations. The point estimator is $\hat{\theta} = \bar{D} = \bar{X} - \bar{Y}$, with interval estimator:

$$\bar{D} \pm t_{1-\alpha/2,n-1} \frac{s_D}{\sqrt{n}}$$

By considering the variance of $\bar{D} = \bar{X} - \bar{Y}$, we have that,

$$\text{Var}[\bar{D}] = \frac{\text{Var}[D_i]}{n} = \frac{1}{n}\text{Var}[X_i - Y_i] = \frac{1}{n}\Big[\text{Var}[X_i] + \text{Var}[Y_i] - 2\text{cov}[X_i, Y_i]\Big]$$

And, therefore,

$$\text{Var}[\bar{D}] = \frac{\sigma_X^2}{n} + \frac{\sigma_Y^2}{n} - 2\rho_{XY}\sigma_X\sigma_Y$$

If the random variables X and Y are independent, then $\rho_{XY} = 0$ and:

$$V_{IND} = \text{Var}[\bar{D}] = \frac{\sigma_X^2}{n} + \frac{\sigma_Y^2}{n}$$

In analyzing the worth of a VRT, we will need to derive the variance of the estimator. Thus, deriving the variance for the CRN and the independent (IND) estimators, we have:

$$V_{CRN} = V_{IND} - 2\rho_{XY}\sigma_X\sigma_Y$$

$$V_{IND} - V_{CRN} = 2\rho_{XY}\sigma_X\sigma_Y$$

This implies that if there is positive correlation between X and Y, $\rho_{XY} \geq 0$, we will have:

$$V_{IND} - V_{CRN} \geq 0$$

$$V_{CRN} \leq V_{IND}$$

Therefore, the variance of the CRN estimator should be smaller than the variance of the crude estimator if there is positive correlation. If we can induce a positive correlation within the pairs (X_i, Y_i), then when we use

$$\hat{\theta} = \frac{1}{n}\sum_{i=1}^{n} D_i = \frac{1}{n}\sum_{i=1}^{n}(X_i - Y_i)$$

we will have a variance reduction (a more precise estimate). It is important to note that the sampling distribution of $\hat{\theta}$ is different *because* the pairs (X_i, Y_i) were sampled in a *dependent* manner.

So, CRN will work when we can induce a positive correlation within the pairs (X_i, Y_i) and it will backfire (cause an increase in the variance) if there is a negative correlation within the pairs (X_i, Y_i). In the case of simple Monte Carlo, it should be clear that we can induce positive correlation between X_i and Y_i by using the same random numbers during the generation process. Suppose $X_i \sim F_X(x)$ and $Y_i \sim F_Y(y)$ and we have the inverse transform functions for F_X and F_Y. Then, to generate X_i and Y_i we use:

$$\begin{aligned} U_i &\sim U(0,1) \\ X_i &= F_X^{-1}(U_i) \\ Y_i &= F_Y^{-1}(U_i) \end{aligned}$$

Thus, if U_i is large, then because the inverse CDF, $F^{-1}(p)$ is a monotone function, X_i will be large *and* Y_i will be large. In addition, if U_i is small, then X_i will be small *and* Y_i will be small. Thus, there will be positive correlation. In this simple case of $D_i = X_i - Y_i$ the monotone relationship will be maintained; however, in a general discrete event dynamic simulation modeling context, the situation is much more complex.

For example, suppose we are estimating the waiting time in bank open from 9 am to 5 pm, θ. Here we have an arrival distribution, $F_A(x)$, and a service distribution, $F_S(x)$, and the number of tellers that govern the underlying stochastic processes. Suppose we want to compare the performance (in terms of the average waiting time) for system 1, θ_1 to the average waiting time for system 2, θ_2, where the arrival rate of system 1, λ_1 is higher than the arrival rate for system 2, λ_2 and the service distributions remain the same, as well as the number of tellers.

Now, suspending our analytical thinking about which system actually should have the higher waiting time, we run simulation experiments to

compare, $\hat{\theta}_1$ to $\hat{\theta}_2$. Assuming that we run two experiments, each with r as the number of replications, and m_r^i as the number of observed customers in replication r for system i, where W_{kj}^i is the observed waiting time for system i for customer k in replication j, we have that our estimators of performance for $i = 1, 2$ to be:

$$\hat{\theta}_i = \frac{1}{r} \sum_{j=1}^{r} \bar{W}_j^i$$

where,

$$\bar{W}_j^i = \frac{1}{m_j^i} \sum_{k=1}^{m_j^i} W_{kj}^i$$

Notice that the number of observations for each system i, m_j^i for each replication j can vary by the system being considered and the number of observations within each replication j can be different. Thus, the differences are $D_j = \bar{W}_j^1 - \bar{W}_j^2$ for $j = 1, 2, \cdots, r$ and $\bar{D} = \hat{\theta}_1 - \hat{\theta}_2$ is our estimator of the difference between system 1 and system 2.

For CRN to work, we need to induce a positive correlation within the pairs $(\bar{W}_j^1, \bar{W}_j^2)$ for $j = 1, 2, \cdots, r$. How can we possibly do that using common random numbers? Note that we are not **directly** generating $\bar{W}_j^i$ using an inverse cumulative distribution function as in the previously discussed simple case of $D_i = X_i - Y_i$. The random variables $\bar{W}_j^i$ are obtained from averaging the observed waiting times, W_{kj}^i, for system i for the $k = 1, 2, \cdots, m_j^i$ customers in replication j. Also, note that we are not **directly** generating (via inverse transform functions) the waiting time observations for each customer, W_{kj}^i.

Focusing on a single replication with subscript j, let's consider how the random numbers are used within the replication. Let A_{kj} be the k^{th} inter-arrival time, where a_{1j} is the realized inter-arrival time of the first

customer, and so forth on the j^{th} replication. Let S_{kj} be the service time of the k^{th} customer, where s_{1j} is the realized service time of the first customer, and so forth on the j^{th} replication. Assume that we use the inverse transform technique for generating A_{kj} and S_{kj}, such that the k^{th} customer's inter-arrival and service times are governed by $a_{kj} = F_A^{-1}(u_{kj})$ and $s_{kj} = F_S^{-1}(v_{kj})$, where $U_{kj} \sim U(0,1)$ and $V_{kj} \sim U(0,1)$ resulting in realizations, u_{kj} and v_{kj}. Note that in what follows I have denoted m_j^1 with subscript j to denote thej^{th} replication and with superscript 1 to denote system 1.

$$
\begin{gathered}
u_{1j}, u_{2j}, u_{3j}, \cdots, u_{m_j^1 j} \\
a_{1j}, a_{2j}, a_{3j}, \cdots, a_{m_j^1 j} \\
v_{1j}, v_{2j}, v_{3j}, \cdots, v_{m_j^1 j} \\
s_{1j}, s_{2j}, s_{3j}, \cdots, s_{m_j^1 j} \\
\vdots \\
\downarrow \\
w_{1j}, w_{2j}, w_{3j}, \cdots, w_{m_j^1 j} \rightarrow \bar{w}_j^1
\end{gathered}
$$

Given values for a_{kj} and s_{kj}, we can execute the event logic and produce observations of w_{kj}^i of the waiting times for the m_j^i customers within each replication j for system 1. Notice that the pseudo random numbers for the first customer, (u_{1j}, v_{1j}) are used to produced the inter-arrival time and service time for the first customer, $(a_{1j} = F_{A^1}^{-1}(u_{1j}), s_{1j} = F_{S^1}^{-1}(v_{1j}))$, which then produces the waiting time for the first customer, w_{1j}. That is, $(u_{1j}, v_{1j}) \rightarrow (a_{1j}, s_{1j}) \rightarrow w_{1j}$. Here A^1 and S^1 denote the arrival and service time distributions for system 1.

Now, consider the simulation of system 2.

$$u_{1j}, u_{2j}, u_{3j}, \cdots, u_{m_j^2 j}$$
$$a_{1j}, a_{2j}, a_{3j}, \cdots, a_{m_j^2 j}$$
$$v_{1j}, v_{2j}, v_{3j}, \cdots, v_{m_j^2 j}$$
$$s_{1j}, s_{2j}, s_{3j}, \cdots, s_{m_j^2 j}$$
$$\vdots$$
$$\downarrow$$
$$w_{1j}, w_{2j}, w_{3j}, \cdots, w_{m_j^2 j} \rightarrow \bar{w}_j^2$$

When using common random numbers the same sequences of pseudo-random numbers $(u_{1j}, u_{2j}, u_{3j}, \cdots, u_{m_j^i j})$ and $(v_{1j}, v_{2j}, v_{3j}, \cdots, v_{m_j^i j})$ will be used for each replication of system i. This will **not** imply that the same inter-arrival and service time values, (a_{1j}, s_{1j}), will be used in the two experiments because inter-arrival distribution for system 1, F_{A^1}, is not the same as the inter-arrival distribution for system 2, F_{A^2}, because the arrival rate of system 1 (λ_1) is bigger than the arrival rate of system 2, (λ_2). However, it should be clear that the inter-arrival times for the two systems will be correlated because a large u_{1j} will produce a large a_{1j}^1 for system 1 and a large a_{1j}^2 for system 2. In addition, because the same $(v_{1j}, v_{2j}, v_{3j}, \cdots, v_{m_j^i j})$ will be used and the service time distributions are the same for system 1 and system 2, the same service times will be used for each customer. That is, each customer will experience exactly the same service time when executing the experiments for system 1 and system 2. These insights should suggest to you that correlation will be induced between pairs $(\bar{W}_j^1, \bar{W}_j^2)$ and thus a variance reduction is likely. Note that system 1 will use more pseudo random numbers $(u_{1j}, u_{2j}, u_{3j}, \cdots)$ than system 2 because system one's arrival rate is higher than system two's arrival rate, thereby, causing $m_j^1 > m_j^2$ for $j = 1, 2, \cdots, r$.

A best practice for inducing correlation is to try to *synchronize* the use of the pseudo-random numbers when executing the simulation experiments. Best practices for synchronization include:

- Utilize the inverse transform technique to generate random variates. The KSL does this by default. Methods such as acceptance/rejection do not have a one-to-one mapping between U_i and X_i. A one-to-one mapping occurs by default for the inverse transform technique.
- Dedicate a different stream to each underlying source of randomness within the model. The KSL does this automatically.
- Dedicate all the random variates to be used by an entity when the entity is created and store them in attributes for when they are needed. This may be impractical due to storage/memory issues or it may be impossible to know in advance if the attribute will be needed. In addition, in general, it takes extra effort to implement this approach in most simulation packages.
- If unable to fully synchronize, then you should try to ensure that the random variables that cannot be synchronized are independent (i.e. use different streams).
- Make sure that each replication (sample path) starts in the same place for each synchronized random number stream. The KSL does this automatically by advancing to the next sub-stream of the current stream for each random variable at the beginning of each replication. This ensures that at the beginning of each replication, the same pseudo-random numbers are used.

CRN should work if the performance measure of interest for the alternatives is monotonic (in the same direction for the systems) for the random number stream. A large value of the random variable that causes $\hat{\theta}_1$ to go up will also cause $\hat{\theta}_2$ to go up. Or, a large value of the random variable that causes $\hat{\theta}_1$ to go down will also cause $\hat{\theta}_2$ to go down. In addition, a small value of the random variable causes $\hat{\theta}_1$ to go up will also cause $\hat{\theta}_2$ to go up. Or, a small value of the random variable that causes $\hat{\theta}_1$ to go down will also cause $\hat{\theta}_2$ to go down. These rules are about inducing positive correlation. If negative correlation occurs, then the variance will be inflated. The good news is that you can always check for negative correlation and not utilize

CRN if negative correlation is being observed. Because you are likely to run some small pilot runs to plan the experiments (e.g. sample size), I recommend using those pilot runs to also check if CRN is working as intended. Almost all modern simulation languages facilitate simulation using common random numbers by allowing the user to specify streams.

9.2.2. Antithetic Variates (AV)

Antithetic variates is a class of variance reduction technique that utilizes the manipulation of the underlying random number streams to change the sampling distribution. In contrast to CRN, antithetic variates is applicable to the reduction of variance for estimators from a single simulation experiment (single system). In antithetic variates, we attempt to induce correlation between the (executions or runs) of the simulation. Suppose that we can generate observations of our performance measure $(Y_1, Y_2, \cdots, Y_n)$ where Y_i is the observed performance of on run i of the simulation. Here the observations $(Y_1, Y_2, \cdots, Y_n)$ are not assumed to be independent. Thus, we call them *runs* rather than *replications*.

Assume that we have a covariance stationary process with $(Y_1, Y_2, \cdots, Y_n)$ with $E[Y_i] = \theta$ and $var[Y_i] = \sigma_Y^2$. We can derive the variance of $\bar{Y}$ as follows:

$$\text{Var}\left[\bar{Y}\right] = \frac{\sigma_Y^2}{n}\left[1 + 2\sum_{j=1}^{n-1}\left(1 - \frac{j}{n}\right)\rho_j\right]$$

where

$$\rho_j = \frac{cov(Y_i, Y_{i+j})}{\sigma_Y^2}$$

Thus, if we can induce negative correlation, then the $\text{Var}\left[\bar{Y}\right]$ can be reduced. The trick is to try to induce negative correlation and still be able

to form valid confidence intervals. To do this we induce correlation within pairs (Y_i, Y_{i+1}) by manipulating the random number streams.

Let n be the sample size and ensure that n is even. Consider the sequence of pairs, (Y_{2j-1}, Y_{2j}) for $j = 1, 2, \cdots, (n/2)$ where the random variables within a pair are dependent but they are independent across the pairs. With $m = n/2$, define the AV estimator as:

$$\hat{Y} = \frac{1}{m}\sum_{j=1}^{m}\frac{Y_{2j-1} + Y_{2j}}{2} = \frac{1}{m}\sum_{j=1}^{m}\hat{Y}_j = \bar{Y}$$

where

$$\hat{Y}_j = \frac{Y_{2j-1} + Y_{2j}}{2}$$

The variance of $\hat{Y}$ is reduced by inducing a negative correlation within pairs (Y_i, Y_{i+1}). Confidence intervals can be computed by treating the sequence $(\hat{Y}_1, \hat{Y}_2, \cdots, \hat{Y}_m)$ as a random sample of size m and applying confidence interval methods.

Let's take a look at how AV works. Consider the $var[\hat{Y}]$

$$\text{Var}[\hat{Y}] = \frac{\text{Var}(\hat{Y}_j)}{m} = \frac{1}{m}\left[\text{Var}\left(\frac{Y_{2j-1} + Y_{2j}}{2}\right)\right] = \frac{1}{4m}\left[\text{Var}[Y_{2j-1}]+\text{Var}[Y_{2j}]+2cov[Y_{2j-1}, Y_{2j}]\right]$$

Since the process is covariance stationary $\text{Var}[Y_{2j-1}] = \text{Var}[Y_{2j}] = \sigma_Y^2$. Thus,

$$\text{Var}[\hat{Y}_j] = \frac{1}{4}\left[\sigma_Y^2 + \sigma_Y^2 + 2\sigma_Y^2\rho\right] = \frac{\sigma_Y^2}{2}(1+\rho)$$

where the correlation is defined as:

$$\rho = corr(Y_{2j-1}, Y_{2j}) = \frac{cov(Y_{2j-1}, Y_{2j})}{\sigma_Y^2}$$

Because

$$\text{Var}[\hat{Y}_j] = \frac{1}{4}\left[\sigma_Y^2 + \sigma_Y^2 + 2\sigma_Y^2\rho\right] = \frac{\sigma_Y^2}{2}(1+\rho)$$

we have

$$\text{Var}[\hat{Y}] = \frac{\sigma_Y^2}{2m}(1+\rho) = \frac{\sigma_Y^2}{n}(1+\rho)$$

Therefore, if $\rho < 0$, then $\text{Var}[\hat{Y}] < \frac{\sigma_Y^2}{n}$. This implies a variance reduction that is directly proportional to the amount of negative correlation that can be induced within the pairs.

To implement antithetic variates, we can consider the following recipe:

- Each simulation run (like a replication) use random number streams such that alternating runs use antithetic streams.
  - Run 1: $(u_1, u_2, \cdots, u_k)$
  - Run 2: $(1-u_1, 1-u_2, \cdots, 1-u_k)$
  - Run 3: $(u_{k+1}, u_{k+2}, \cdots, u_{2k})$
  - Run 4: $(1-u_{k+1}, 1-u_{k+2}, \cdots, 1-u_{2k})$, etc.
- Each stream is mapped to some distribution, $F(x)$ in the model, then, suppose,X_{2j-1} and X_{2j} are inputs on runs $2j-1$ and $2j$, where $j = 1, 2, \cdots, m$. Then, if the inverse transform method is used we have $X_{2j-1} = F^{-1}(u)$ and $X_{2j} = F^{-1}(1-u)$.

If the underlying random variables map to outputs monotonically, similar to the CRN situation, then the negative correlation may be preserved within the pairs for the *simulation* performance measures, which will cause a variance reduction. Just like in CRN, synchronization is important and the same best practices are recommended.

An example of how to perform an antithetic experiment within a Monte Carlo context was presented in Section 2.1.4. Within a simple Monte Carlo context the implementation of antithetic variates is relatively straightforward. Reviewing the implementation of the class `MC1DIntegration` within the `ksl.utilities.mcintegration` package is illustrative of the basic approach. The `MC1DIntegration` class provides for the use of antithetic variates when estimating the area of a 1-dimensional function. In the following Kotlin code, we see that in the `init` block if the `antitheticOption` is true an antithetic instance is created from the random variable used in the sampling.

```
class MC1DIntegration (
    function: FunctionIfc,
    sampler: RVariableIfc,
    antitheticOption: Boolean = true
) : MCExperiment() {
    protected val myFunction: FunctionIfc
    protected val mySampler: RVariableIfc
    protected var myAntitheticSampler: RVariableIfc? = null

    init {
        myFunction = function
        mySampler = sampler
        if (antitheticOption){
            myAntitheticSampler =
↪   sampler.antitheticInstance()
        }
        confidenceLevel = 0.99
```

```
    }
```

Recall that the `MCExperiment` class, see Section 3.8, requires the user to provide an implementation of the `MCReplicationIfc` interface. In the `MC1DIntegration` class we have the following implementation.

```
    override fun replication(j: Int): Double {
        return if (isAntitheticOptionOn) {
            val y1 = myFunction.f(mySampler.sample())
            val y2 =
             ↪ myFunction.f(myAntitheticSampler!!.sample())
            (y1 + y2) / 2.0
        } else {
            myFunction.f(mySampler.sample())
        }
    }
```

Note that if the antithetic variate option is turned on, then the `replication(j: Int)` function returns

$$\hat{Y}_j = \frac{Y_{2j-1} + Y_{2j}}{2}$$

by using the antithetic sampler for Y_{2j}. Thus, each replication is defined as a pair of runs (antithetic off, antithetic on). This implementation uses a separate antithetic sampler; however, as shown in Section 2.1.4 the same effect can be achieved by using one stream, resetting it and turning on the antithetic option for the stream.

Implementing antithetic variates within the context of DEDS is more complicated. Similar to the discussion associated with CRN, let's consider what happens for the pairs of runs $2j-1$ and $2j$.

For the $2j-1$ run we have:

$$\begin{gathered} u_1, u_2, u_3, \cdots, u_m \\ a_1, a_2, a_3, \cdots, a_m \\ v_1, v_2, v_3, \cdots, v_m \\ s_1, s_2, s_3, \cdots, s_m \\ \vdots \\ \downarrow \\ w_1, w_2, w_3, \cdots, w_m \rightarrow \bar{w}_{2j-1} \end{gathered}$$

For the $2j$ run we use the antithetic variates such that, $a_k = F_A^{-1}(1-u_k)$ and $s_k = F_S^{-1}(1-v_k)$:

$$\begin{gathered} 1-u_1, 1-u_2, 1-u_3, \cdots, 1-u_m \\ a_1, a_2, a_3, \cdots, a_m \\ 1-v_1, 1-v_2, 1-v_3, \cdots, 1-v_m \\ s_1, s_2, s_3, \cdots, s_m \\ \vdots \\ \downarrow \\ w_1, w_2, w_3, \cdots, w_m \rightarrow \bar{w}_{2j} \end{gathered}$$

This should cause the pairs $(\bar{W}_{2j-1}, \bar{W}_{2j})$ to have negative correlation. Again the actual realization of some variance reduction will depend upon synchronization strategies and responses, such as the waiting time, responding in a monotonic fashion to the antithetic sampling.

Within a DEDS context, the KSL permits the model to be executed and the streams controlled to implement antithetic sampling. The property `antitheticOption` of the `Model` class controls whether the streams use antithetic sampling. It may be instructive to see how this is implemented within the `Model` class.

```
    private fun handleAntitheticReplications() {
        // handle antithetic replications
        if (antitheticOption) {
            logger.info { "Executing
↪   handleAntitheticReplications() setup" }
            if (currentReplicationNumber % 2 == 0) {
                // even number replication
                // return to beginning of sub-stream
                resetStartSubStream()
                // turn on antithetic sampling
                antitheticOption(true)
            } else  // odd number replication
                if (currentReplicationNumber > 1) {
                    // turn off antithetic sampling
                    antitheticOption(false)
                    // advance to next sub-stream
                    advanceToNextSubStream()
                }
        }
    }
```

In the previous code, we see that if the replication number is even (a multiple of 2), the streams are reset to the beginning of their current sub-stream and the antithetic option is turned on for *every* random variable (instance of the `RandomVariable` class) used within the model. This causes the sampling performed during an even run to use antithetic pseudo-random numbers. Then, for odd replications, the antithetic option is turned off and the normal stream advancement occurs. Thus, within the KSL a simple change of a property permits antithetic sampling.

9.2.3. Indirect Estimation

While not as generalizable as CRN or AV, indirect estimation is interesting because it illustrates the general concept of replacing uncertainty with knowledge to reduce variance. The basic idea is to exploit an analytical relationship involving the parameter of interest with other random variables whose population values are known. The most useful application of indirect estimation has been the use of Little's formula or other conservation law relationships for simulations involving queueing.

Suppose we are interested in estimating W_q, W_s, L_q, and L_s in a GI/G/c queueing simulation. Little's formulat tells us that $L_s = \lambda W_s$,$L_q = \lambda W_q$, and $B = L_s - L_q = \lambda E[ST]$, where $E[ST]$ is the mean of the service time distribution and λ is the mean arrival rate. We also know that $W_s = W_q + E[ST]$. During the simulation, we can directly observe the waiting time W_{q_i} of each customer in the queue and estimate W_q with:

$$\hat{W}_q = \frac{1}{n} \sum_{i=1}^{n} W_{q_i}$$

Indirect estimation suggests that we estimate the other performance measures using the operational relationships.

- $\hat{W}_s = \hat{W}_q + E[ST]$
- $\hat{L}_q = \lambda \hat{W}_q$, and
- $\hat{L}_s = \lambda(\hat{W}_q + E[ST])$

Notice that known quantities $E[ST]$ and λ are used. This replaces variability with certainty causing a variance reduction. Experimental studies have shown that this approach will work well provided that the operational formulas are applicable. However, this approach requires the implementation of the analytical relationships and formulation of the estimators which is not standard in any simulation packages. For further references the interested reader should consult Chapter 11 of (Law 2007).

9.2.4. Control Variates (CV)

The idea is to exploit knowledge of the system by other variables for which the true mean is known and exploit any dependence that it might have with our parameter of interest. Let's consider a simplified example.

Example 9.5 (Simple Health Clinic)**.** Suppose that we are simulating a heart disease reduction program operating between 10am and Noon. Suppose that 30 patients per day are scheduled at 5 minute intervals. The clinic process is as follows. The patient first visits a clerk to gather their medical information. Then, the patient visits a medical technician where blood pressure and vitals are recorded and it is determined whether or not the patient should visit the nurse. If the patient visits the nurse practitioner, then they receive some medical service and return to a clerk for payment. If the patient does not need to see the nurse practitioner, then the patient goes directly to the clerk for checkout.

- Let Y_j be the utilization of the nurse on the j^{th} day. We are interested in estimating the utilization of the nurse on any given day.
- Let X_j be the number of patients who consulted with the nurse on the j^{th} day.
- Let p be the proportion of patients that see the nurse. Because there are 30 patients per day, we have $E[X_j] = 30p = \mu$, which is a known quantity.

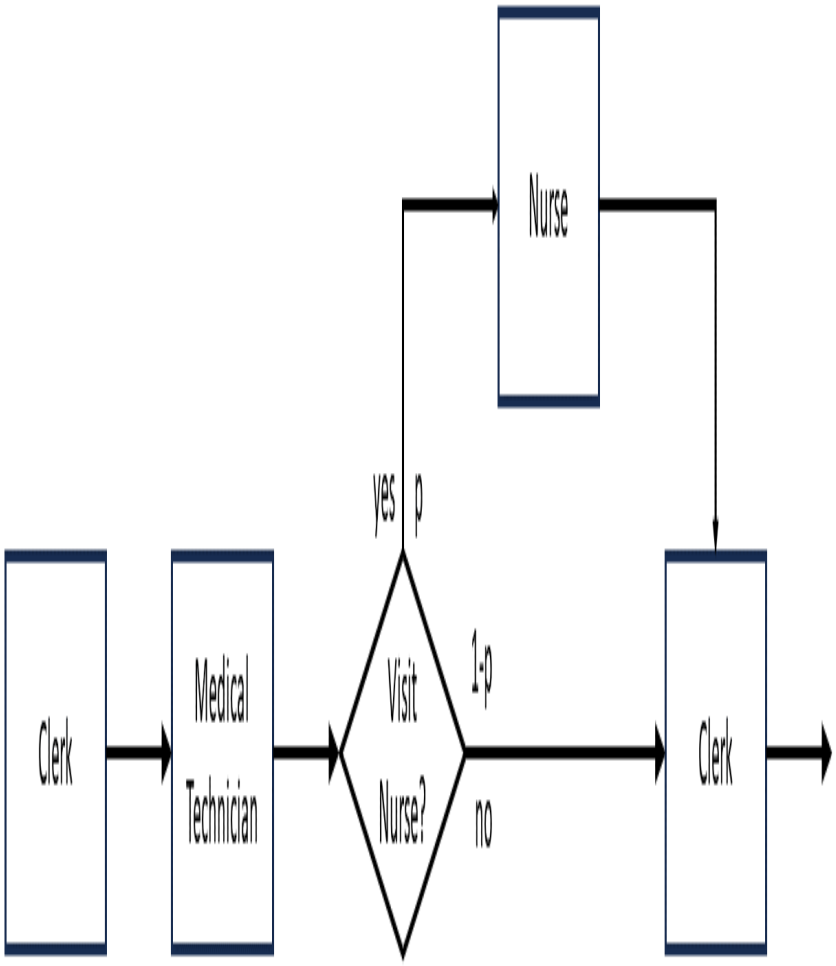


Figure 9.4.: Simple Health Clinic

To build up a control variate estimator, define the following random variable, $Z_j = Y_j + C(X_j - \mu)$, where C is assumed to be a known constant. Since, our parameter of interest is $E[Y_j]$ consider $E[Z_j]$

$$E[Z_j] = E[Y_j + C(X_j - \mu)] = E[Y_j] + C(E[X_j] - \mu) = E[Y_j]$$

Thus, we can estimate $E[Y_j]$ by observing Z_j, and computing

$$\bar{Z} = \frac{1}{n}\sum_{i=1}^{n} Z_i$$

The sample average, $\bar{Z}$, will be an unbiased estimator of $E[Y_j]$ if C is a constant known in advance. The random variable Z_j forms our control variate estimator and the quantity X_j with known mean μ is called the control variate. Intuitively, we can see that X_j should be related to the utilization of the nurse. For example, if X_j is high for a particular simulation run, then the nurse should see more patients and thus have a higher utilization. Similarly, if X_j is low for a particular simulation run, then the nurse should see less patients and thus have a lower utilization. Thus, we can conclude that X_j is correlated with our quantity of interest Y_j. The linear form $Z_j = Y_j + C(X_j - \mu)$ adjusts Y_j up or down based on what is observed for X_j. Why would we want to do this? That is why does using a control variate work?

Consider deriving the variance of our control variate estimator, $\text{Var}[Z_j]$. This results in:

$$\text{Var}[Z_j] = \text{Var}[Y_j + C(X_j - \mu)] = \text{Var}[Y_j] + C^2\text{Var}[X_j] + 2C\,\text{cov}(Y_j, X_j)$$

Rearranging this equation, we have:

$$\text{Var}[Z_j] = \text{Var}[Y_j] + C(C\text{Var}[X_j] + 2\,\text{cov}[Y_j, X_j])$$

Thus, there will be a variance reduction, $\text{Var}[Z_j] < \text{Var}[Y_j]$, if $C(C\text{Var}[X_j] + 2\,\text{cov}[Y_j, X_j]) < 0$. That is, we will get a variance reduction if this condition is true. There are two cases to consider. If the constant $C > 0$, then the value $C(C\text{Var}[X_j] + 2\,\text{cov}[Y_j, X_j]) < 0$ if

$$C < \frac{-2\text{cov}(Y_j, X_j)}{\text{Var}(X_j)}$$

However, if the constant $C < 0$, then the value $C(C\text{Var}[X_j] + 2\,\text{cov}[Y_j, X_j]) > 0$, if

$$C > \frac{-2\text{cov}(Y_j, X_j)}{\text{Var}(X_j)}$$

Thus, for a suitably chosen value for C, we can reduce the variance. How do we pick C? Well, it makes sense to pick the value of C that minimizes the variance of the estimator, $\text{Var}[Z_j]$, and thus pick the value of C that maximizes the variance reduction. To minimize $\text{Var}[Z_j]$ we can take the derivative with respect to C, and set the derivative equal to 0, and solve for the value of C.

$$\frac{d\,\text{Var}[Z_j]}{d\,C} = 2C\,\text{Var}(X_j) + 2\text{cov}(Y_j, X_j) = 0$$

$$C^* = \frac{-\text{cov}(Y_j, X_j)}{\text{Var}(X_j)}$$

Substituting C^* into $\text{Var}[Z_j]$ yields the following:

$$\text{Var}[Z_j] = \text{Var}[Y_j](1 - \rho_{XY}^2)$$

where ρ_{XY} is the correlation between X and Y, $\text{corr}(Y_j, X_j)$. Thus, the variance of our estimator, $\text{Var}[Z_j]$, depends directly on the correlation between Y_j and X_j, with more correlation (either positive or negative), the more variance reduction.

Unfortunately C^* is in general unknown and must be estimated from the data. Looking at the form for C^*, we need to estimate the numerator,$\text{cov}(Y_j, X_j)$, and the denominator, $\text{Var}(X_j)$. Consider estimating $\text{cov}(Y_j, X_j)$ with the following:

$$\widehat{cov(Y_j, X_j)} = \frac{\sum_{i=1}^{n}(Y_i - \bar{Y})(X_i - \bar{X})}{n-1}$$

And, consider estimating $\text{Var}(X_j)$ with the following:

$$\widehat{\text{Var}(X_j)} = \frac{\sum_{i=1}^{n}(X_i - \bar{X})^2}{n-1}$$

Substituting these estimators into the form for C^*, this implies an estimator for C^* of:

$$\hat{C}^* = \frac{\sum_{i=1}^{n}(Y_i - \bar{Y})(X_i - \bar{X})}{\sum_{i=1}^{n}(X_i - \bar{X})^2}$$

This formula should remind you of the estimator for the slope of a regression line. From the form for Z_j it should now be readily apparent that it is essentially assuming a linear relationship and that the optimal value of C would be the slope of the line (under the assumption that a linear relationship holds). This leads to a more general basis for the form of control variates.

Suppose we want to estimate $\theta = E[Y]$, and we can also observe some other random variable, X, whose distribution is known, with mean $\mu = E[X]$. Assume that $E[Y|X = x] \approx \beta_0 + \beta_1(x - \mu)$. That is, given we observe x, there is an (approximate) linear relationship between Y and X. This might be justified by thinking of $g(x) = E[Y|X = x]$ as a function of x and noting that a linear relationship could at least be possible within a small range of μ by expanding $g(x)$ as a Taylor series about μ.

$$g(x) \approx g(\mu) + \frac{d\, g(\mu)}{d\, x}(x - \mu) + \cdots$$

Recall that $E[Y] = E\Big[E[Y|X]\Big] = \beta_0 + \beta_1(X - \mu) = \beta_0 = \theta$. Thus, if we can find an estimator for β_0 *and* the linear relationship is true, then we will have an unbiased estimator for θ. But, this is exactly the same form of the previously derived control variate estimator. In what follows, we will assume that the linear form, $E[Y|X = x] = \beta_0 + \beta_1(x - \mu)$ holds.

Assuming that $E[Y|X = x] = \beta_0 + \beta_1(x - \mu)$ is true, then the control variate estimator for $\theta = E[Y]$ is $\hat{\beta}_0$ where:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(Y_i - \bar{Y})(X_i - \bar{X})}{\sum_{i=1}^{n}(X_i - \bar{X})^2}$$

And,

$$\hat{\beta}_0 = \bar{Y} - \hat{\beta}_1(\bar{X} - \mu)$$

If the linear form for $E[Y|X = x]$ is true, then it can be shown that $E[\hat{\beta}_0] = \beta_0$ and $E[\hat{\beta}_1] = \beta_1$. That is, the control variate estimator $\hat{\beta}_0$ will be unbiased, and as we have shown, there will be a variance reduction if there is some correlation between the control variate and the quantity of interest. For the technical details of when control variates work and when they do not, please see the following references (Nelson and Pei. 2021) and (Nelson 1990). As noted in those references, if the relationship between Y and X is bi-variate normal then the linear form for $E[Y|X = x]$ is true. In addition, the results can be extended to more than one control variate and if the relationship between the quantity of interest Y and the controls is multi-variate normal then control variates will again work. Since in many cases the observations of Y and the controls are averages of within replication data, it is likely that the assumption of normality will be

approximately true. In what follows, we indicate some results from (Nelson 1990) and refer the interested reader to that paper for the proofs.

First, if (Y, X) are bivariate normal then it can be shown that:

$$\text{Var}[\hat{\beta}_0] = (1 - \rho^2)\left(\frac{n-2}{n-3}\right)\frac{\sigma_Y^2}{n}$$

From this, we can see that $\text{Var}[\hat{\beta}_0] \leq \text{Var}[\bar{Y}]$ if and only if $\rho^2 \geq 1/(n-2)$. Suppose $n = 10$, then $\rho^2 \geq 1/8 = 0.125$. That is, $|\rho| \geq \sqrt{0.125} \approx 0.354$. Thus, a moderate amount of correlation is enough to get a variance reduction, and the required correlation decreases as n increases. These results generalize to more than one control variate. Let q be the number of control variates. We then formulate the control variate estimator from a linear model of the following form:

$$Y_i = \beta_0 + \beta_1(X_1 - \mu_1) + \beta_2(X_2 - \mu_2) + \cdots + \beta_q(X_q - \mu_q) + \epsilon_i$$

If the Y and X_j are multi-variate normally distributed, then

$$var[\hat{\beta}_0] = (1 - R_{XY}^2)\left(\frac{n-2}{n-q-2}\right)\frac{\sigma_Y^2}{n}$$

There will be a variance reduction if $R_{XY}^2 \geq \frac{q}{n-2}$, where R_{XY} is the multiple correlation of Y on X. The control variate estimator for $\theta = E[Y]$ will be $\hat{\beta}_0$, which is the intercept term for the regression. Thus, any standard regression program can be used to compute the estimate. Everything works if the linear relationship holds and the Y and X_j are multi-variate normal. If the assumptions do not hold, then the control variate estimator will, in general, be biased. It may have smaller variance, but it may not be unbiased. The recommendation is to use a small number of controls $1 \leq q \leq 10$ and batch the replications if necessary to make the data more normal. If n is the number of replications and $n \geq 100$ with $1 \leq q \leq 5$, then batch n into between $30 \leq k \leq 60$ batches.

Control variates is a general variance reduction technique. In general within a DEDS situation, there will be many possible controls because any of the input distributions with their known mean values are candidates for controls. With many possible controls, you should think about which input distributions might have more influence over the estimated performance measure as a method for selecting controls. In addition, there is little extra programming that needs to be implemented other than capturing the output of the Y and the controls X and their mean values. There is little possibility that control variates will backfire (increase the variance); however, you need to be aware of the previously mentioned possibility of bias. Thus, checking for the normality of the responses and controls is essential. As mentioned, batching the output might be useful. In addition, jackknifing the estimator may help in reducing possible bias. However, these fixes required advanced sophistication from the analyst and will likely need to be implemented.

The KSL has support for using control variates within the context of a DEDS simulation via the `ControlVariateDataCollector` class. The `ControlVariateDataCollector` class works in a similar fashion as the `ReplicationDataCollector` class that was discussed in Section 5.4.1. The following code illustrates its use.

Example 9.6 (Illustrating Control Variates)**.** This example illustrates how to define controls, collect the responses, and compute the control variate estimators using KSL regression constructs.

```
    val model = Model("CV Example")
    model.numberOfReplications = 100
    val palletWorkCenter = PalletWorkCenter(model)
    val cvCollector = ControlVariateDataCollector(model)
    cvCollector.addResponse(palletWorkCenter.
↪   totalProcessingTime, "TotalTime")
    cvCollector.addControlVariate(palletWorkCenter.
↪   processingTimeRV, (8.0+ 12.0+ 15.0)/3.0, "PalletTime")
```

```
    cvCollector.addControlVariate(palletWorkCenter.
↪   numPalletsRV, (100.0*0.8), "NumPallets")
    model.simulate()
    model.print()
    println(cvCollector)
    val regressionData =
     ↪  cvCollector.collectedData("TotalTime", 20)
    println(regressionData)
    println()
    val regressionResults =
     ↪  cvCollector.regressionResults(regressionData)
    println(regressionResults)
```

Notice the use of the method, `cvCollector.collectedData("TotalTime", 20)`. This method prepares the data for a regression analysis. The value 20 tells the method to form 20 batches from the 100 replications into 20 batches of size 5. The following output is the standard half-width report for the 100 replications.

```
Half-Width Statistical Summary Report - Confidence Level (95.000)%

Name                              Count         Average      Half-
Width
-------------------------------------------------------------------------------
NumBusyWorkers                    100            1.9154          0.0133
PalletQ:NumInQ                    100            6.9721          0.5609
PalletQ:TimeInQ                   100           42.1123          3.3255
Num Pallets at WC                 100            8.8875          0.5674
System Time                       100           53.7523          3.3254
Total Processing Time             100          497.7907          6.0995
P{total time > 480 minutes}       100            0.7300          0.0885
RandomVariable_4:CVResponse       100           11.6400          0.0299
```

```
RandomVariable_6:CVResponse     100           80.8000          0.7316
Num Processed                   100           81.8000          0.7316
------------------------------------------------------------------------
```

As you can see in the following output from the `ControlVariateDataCollector` class, the data is organized in a manner that permits the application of linear regression.

```
Control Variate Collector
Responses:
response: TotalTime

Controls:
control: PalletTime      mean = 11.666666666666666
control: NumPallets      mean = 80.0

Replication Data Collector
   Total Processing Time RandomVariable_4:CVResponse RandomVariable_6:CVResponse
 0            482.417201                   11.991551                         75.0
 1            461.544205                   11.681418                         74.0
 2            521.417293                   11.701470                         78.0
 3            476.025297                   11.699098                         80.0
 4            534.281150                   11.464543                         86.0
 5            485.735690                   11.475689                         82.0
 6            477.468018                   11.645124                         80.0
 7            482.557886                   11.579375                         79.0
 8            499.628817                   11.765892                         82.0
 9            471.007443                   11.603611                         78.0

Control Variate Data
    TotalTime PalletTime NumPallets
 0 482.417201   0.324884       -5.0
 1 461.544205   0.014752       -6.0
```

```
 2 521.417293   0.034803       -2.0
 3 476.025297   0.032431        0.0
 4 534.281150  -0.202123        6.0
 5 485.735690  -0.190977        2.0
 6 477.468018  -0.021543        0.0
 7 482.557886  -0.087292       -1.0
 8 499.628817   0.099225        2.0
 9 471.007443  -0.063056       -2.0
```

Thus, you can use your favorite linear regression analysis software to analyze the data and compute the control variate estimate. The KSL has support for performing ordinary least squares regression. In the previously provided code notice that the control variate collector is used to get the regression data via the lines:

```
val regressionData = cvCollector.collectedData("TotalTime", 20)
println(regressionData)
```

Then, the collector uses the regression data to estimate the regression coefficients.

```
val regressionResults = cvCollector.regressionResults(regressionData)
println(regressionResults)
```

Below is a snippet of the results of applying the control variate estimator via the estimated regression. The estimate of the intercept term is the control variate estimate.

```
Regression Results
------------------------------------------------------------------------------
Parameter Estimation Results
    Predictor  parameter parameterSE     TValue P-Values LowerLimit UpperLimit
```

```
 0  Intercept 494.631368    2.792054 177.156788 0.000000 488.740649 500.522088
 1 PalletTime  89.977513   44.683070   2.013682 0.060151  -
4.295525 184.250550
 2 NumPallets   6.943175    1.339181   5.184644 0.000075   4.117751   9.768599
-------------------------------------------------------------------------------
```

Notice that the estimated response is different than the standard estimate (497.7907) versus (494.631368). In addition, the half-width of the control variate estimator is smaller. Further analysis should be performed to check the normality assumptions and assess the quality of the estimated value. The KSL `RegressionResultsIfc` interface has the ability to check the normality of the residuals perform additional regression analysis operations. This analysis is left as an exercise for the reader.

9.2.5. Stratified and Post Stratified Sampling

The idea behind stratified sampling is to take advantage of known properties or characteristics within the underlying population that is being sampled. In stratified sampling, conditional expectation is used to reduce the variance by conditioning on another variable (similar to how control variates use a control variable). In stratified sampling, we sample within the different strata and then form an overall estimate from the estimated value from each strata.

For example, suppose that we were estimating the overall GPA of engineering majors (electrical, mechanical, industrial, computer, chemical, biological, etc.). We know that each major has a different number of students within the college of engineering. If we take a simple random sample from the entire population, we might not observe a particular major just due to the sampling. However, if we knew (or could estimate) the proportion of students within each major, then we could randomly sample from each major (strata) and form a combined GPA estimate from the weighted average of the strata averages. This is the basic idea of stratified

sampling. We can control the variability of the estimator by how many students we sample from each major.

Let's develop some notation to specify these concepts. Recall that $\theta = E[Y] = E\Big[E[Y|X]\Big] = \int E[Y|X = x]d\,F(x)$ where $F(x)$ is the CDF of some random variable X. Let the range of X be denoted by Ω. Divide Ω into k sub-ranges L_i, $i = 1, 2, \cdots, k$, such that the L_i are mutually exclusive and collectively exhaustive. The L_i are called the strata for X. Using conditional expectation, we have that:

$$\theta = \sum_{i=1}^{k} E[Y|x \in L_i]P(X \in L_i)$$

Defining $p_j = P(X \in L_i)$ and define $\hat{\theta}_j$ as the estimator of $E[Y|x \in L_j]$. Then, the stratified *estimator* is:

$$\hat{\theta} = \sum_{i=j}^{k} \widehat{E[Y|x \in L_j]}\, p_j = \sum_{j=1}^{k} \hat{\theta}_j\, p_j$$

In this situation, p_j are known constants, computed in advance. This method replaces uncertainty by computing p_j and estimating $\hat{\theta}_j$, where $\hat{\theta}_j$ is the sample average of the observations that fall within strata L_j.

$$p_j = P(X \in L_i) = \int_{b_{j-1}}^{b_j} f(x)dx$$

In stratified sampling, we can pre-determine how many observations to sample within each strata as part of the design of the sampling plan. There is another form of stratified sampling, that classifies each observation after sampling from the overall population as belonging to strata L_j. This approach is called post-stratified sampling.

In post stratified sampling, we observe the IID pairs $(Y_1, X_1), (Y_2, X_2), \cdots, (Y_n, X_n)$ and form an estimator for θ_j by computing the observed sample average for the strata. Let Y_{ij} be the i^{th} observation of Y when $X \in L_j$, for $j = 1, 2, \cdots, k$ and $i = 1, 2, \cdots, N_j$, where N_j is the number of observations of Y_i in the j^{th} strata. Note that N_j is a random variable. The estimator for θ_j is:

$$\hat{\theta_j} = \frac{1}{N_j} \sum_{i=1}^{N} Y_{ij}$$

The post stratified estimator is then

$$\hat{\theta} = \sum_{j=1}^{k} \hat{\theta_j} \, p_j$$

If $N_j > 0$ for all j, then $E[\hat{\theta_j}] = \theta_j$ and thus $E[\hat{\theta}] = \theta$. That is, it will be an unbiased estimator.

There are two main issues when applying stratified sampling: 1) how to form the strata, and 2) how many observations should be sampled in each strata. The formation of the strata is often problem dependent. Then, given the chosen strata a sampling plan (i.e. N_j for each strata can be analyzed). The key idea for picking the strata is to try to ensure that the means for each strata are different from the overall mean. Let $\sigma_j^2 = var[Y_j | X \in L_j]$ be the within strata variance. One can show that:

$$\text{Var}(\hat{\theta}) \cong \sum_{j=1}^{k} \frac{p_j \, \sigma_j^2}{n} + \sum_{j=1}^{k} \frac{(1 - p_j) \, \sigma_j^2}{n^2}$$

And, that

$$\text{Var}(\bar{Y}) = \text{Var}(\hat{\theta}) + \sum_{j=1}^{k} \frac{(\theta_j - \theta)^2 \, p_j}{n}$$

Thus, a large spread (difference) for the j^{th} strata mean from the overall mean θ will cause, $\text{Var}(\bar{Y}) \geq \text{Var}(\hat{\theta})$. You should also pick the number of strata k to be small such that you can try to guarantee that $n\,p_j \geq 5$ to ensure that $N_j > 0$.

In the case of stratified sampling, we can control N_j. That is, N_j is no longer random and under our control as n_j for $j = 1, 2, \cdots, k$. We pre-plan how much to sample in each L_j via n_j. One can show that in this case the optimal allocation of n that minimizes that variance is:

$$n_j = \frac{n\,p_j\,\sigma_j}{\sum_{i=1}^{k} p_i\,\sigma_i}$$

In essence, the minimal variance will occur if n_j is sampled proportional to $p_i\,\sigma_i$. For further details of these results, see (Rubinstein and Kroese. 2017) or (Ross 2023). By estimating the within strata variance $\sigma_j^2 = \text{Var}[Y_j | X \in L_j]$ via a pilot run a starting value for n_j can be obtained. Another (convenient) approach is to allocate the n_j proportionally as $n_j = n\,p_j$. As discussed in (Rubinstein and Kroese. 2017), the stratified sampling estimator will be unbiased and should have a variance reduction when compared to the crude Monte Carlo (CMC) estimator.

The stratified sampling method within a Monte Carlo context is relatively straight-forward to implement. The KSL utilities package for random sampling and statistical collection can be used in such an implementation. For the case of a DEDS situation, stratified sampling has less direct appeal. Although the post-stratified sampling method could be automated by capturing the response Y_j and generated X from the simulation (within replication data), it is not straight-forward to connect the generated X values with the responses. Thus, the KSL does not provide constructs to specifically implement this variance reduction technique.

9.2.6. Conditional Expectation (CE)

For completeness, this section discusses another less general form of variance reduction technique: conditional expectation. As we will see, this technique is very problem dependent. Thus, this section presents the concepts within the context of a particular example. For more of the theory behind this technique, we refer the interested reader to (Rubinstein and Kroese. 2017) or (Ross 2023). Like stratified sampling and control variates this method work on using $E[Y|X]$. Recall $\theta = E[Y] = E\Big[E[Y|X]\Big]$. It can be shown, see (Ross 2023), that $\text{Var}[E[Y|X]] \leq \text{Var}[\bar{Y}]$. Thus, if we *know* the function $E[Y|X = x]$ and have a sample $X_1, X_2, \cdots, X_n$, then we can use:

$$\hat{\theta} = \frac{1}{n} \sum_{i=1}^{n} E[Y|X_i]$$

This technique relies on being able to derive the function $E[Y|X = x]$ from the problem situation, which can be very problem dependent. We will demonstrate this via an example involving a stochastic activity network.

Let X_i be the time required to complete arc i. Assume that the X_i are independent. A path through an activity network is a sequence of arcs going from a source node to a sink node. For the example network, we have three paths.

- Path 1: $Y_1 = X_1 + X_4$
- Path 2: $Y_2 = X_1 + X_3 + X_5$
- Path 3: $Y_3 = X_2 + X_5$

Activity networks are often simulated in order to observe the longest path,e.g. $Y = max(Y_1, Y_2, Y_3)$ and estimate quantities such as $E[Y]$ and $P(Y \leq t)$. A crude Monte Carlo algorithm for estimating $E[Y]$ and $P(Y \leq t)$ is as follows.

0. Let P_j be the arcs on path j; choose t; let $c = 0$; $sum = 0$

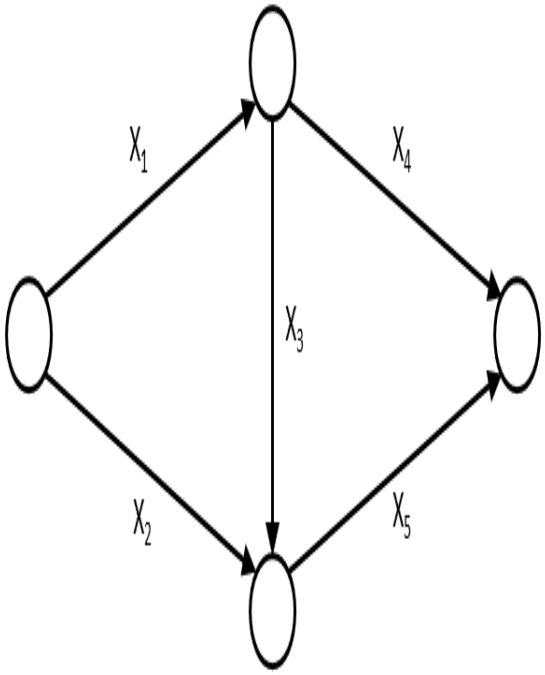


Figure 9.5.: Example Stochastic Activity Network

for $n = 1$ to k;

1. generate $X_i \sim F_i(x)$, $i = 1, 2, 3, 4, 5$
2. compute $Y_j = \sum_{i \in P_j} X_i$, for $j = 1, 2, 3$
3. let $Y = max(Y_1, Y_2, Y_3)$
4. if $(Y \leq t)$ then $c = c + 1$; $sum = sum + Y$

end for;

5. $\widehat{P(Y \leq t)} = c/k$
6. $\bar{Y} = sum/k$

In what follows, we will develop functional form for $E[Y|X = x]$ so that the conditional expectation variance reduction technique can be applied. We begin by developing the distribution for $P(Y \leq t)$. The distribution of Y the maximum path length is defined as:

$$P(Y \leq t) = P(max(Y_1, Y_2, Y_3) \leq t) = P(Y_1 \leq t, Y_2 \leq t, Y_3 \leq t)$$

Thus, the Y_i are not independent random variable because they share arcs. This dependence can be exploited for the application of CE. Another way to write $P(Y \leq t)$ can be useful. Define an indicator function $g(X_1, X_2, X_3, X_4, X_5)$ as follows:

$$g(X_1, X_2, X_3, X_4, X_5) = \begin{cases} 1 & \text{if } Y \leq t \\ 0 & \text{if } Y > t \end{cases}$$

Now, define $f_i(x_i)$ as the distribution function the length, X_i, of each arc i. Then, the indicator function $g(\cdot)$ can be used to define $P(Y \leq t)$ as follows:

$$P(Y \leq t) = \int_0^\infty \int_0^\infty \int_0^\infty \int_0^\infty \int_0^\infty g(X_1, \cdots, X_5)\, f_1(x_1) f_2(x_2) \cdots f_5(x_5)\, dx_1 \cdots dx_5$$

Thus, $P(Y \leq t) = E[g(X_1, \cdots, X_5)]$. This form $P(Y \leq t) = E[g(X_1, \cdots, X_5)]$ allows us to more easily see the conditional expectation possibilities.

Consider $E[g(X_1, \cdots, X_5)|X_1 = x_1, X_5 = x_5]$, which is the conditional expectation of $g(\cdot)$ given arcs one and five. Notice from Figure 9.5 that X_1 and X_5 occur in each of the three paths. That is, arcs one and five are common on the three paths.

Note that by conditional expectation, we can write $P(Y \leq t) = E_{X_1,X_5}\Big[E[g(X_1, \cdots, X_5)|X_1, X_5]\Big]$ and that $E[g(X_1, \cdots, X_5)|X_1 = x_1, X_5 = x_5]$ can be written as:

$$\begin{aligned}
E[g(X_1, \cdots, X_5)|X_1 = x_1, X_5 = x_5] &= P(Y \leq t|X_1 = x_1, X_5 = x_5) \\
&= P(Y_1 \leq t, Y_2 \leq t, Y_3 \leq t|X_1 = x_1, X_5 = x_5) \\
&= P(X_1 + X_4 \leq t, X_1 + X_3 + X_5 \leq t, X_2 + X_5 \leq t|X_1 = x_1, X_5 = \\
&= P(X_4 \leq t - x_1, X_3 \leq t - x_1 - x_5, X_2 \leq t - x_5) \\
&= P(X_4 \leq t - x_1)P(X_3 \leq t - x_1 - x_5)P(X_2 \leq t - x_5) \\
&= F_4(t - x_1)F_3(t - x_1 - x_5)F_2(t - x_5)
\end{aligned}$$

Notice that $E[g(X_1, \cdots, X_5)|X_1 = x_1, X_5 = x_5]$ is simply a function of x_1 and x_5. Thus after observing x_1 and x_5, we can average over this expression and compute $P(Y \leq t)$. A simple algorithm for this can be written as.

0. $sum = 0$

for $n = 1$ to k;

1. generate X_1 and X_5
2. sum = sum + $F_4(t - x_1)F_3(t - x_1 - x_5)F_2(t - x_5)$

end for;

4. $P(\widehat{Y \leq t}) = sum/k$

The conditional expectation estimator (if found) will guarantee a variance reduction. The amount of variance reduction is problem dependent.

Since this technique is problem dependent, the KSL does not provide general support for applying CE; however, as noted in the simple algorithms that were presented, the KSL support for generating random variates and collecting statistics would be essential in such an implementation. The difficulty of applying CE to static Monte Carlo situations is based on the problem situation but relatively straight-forward if the conditional expectation function can be derived; however, CE has received limited application in the more general DEDS situation.

9.2.7. Importance Sampling (IS)

The final variance reduction technique to be discussed is general in nature; however, although it has many applications its application is significant within a specific static Monte Carlo situation, specifically the estimation of the area of a function.

$$\theta = \int_a^b g(x)\mathrm{d}x$$

Recall that the crude or simple Monte Carlo evaluation of an integral utilizes the uniform distribution:

$$f_X(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

Therefore,

$$E_X[g(x)] = \int_a^b g(x) f_X(x) \mathrm{d}x = \int_a^b g(x) \frac{1}{b-a} \mathrm{d}x$$

Defining $Y = (b-a)\, g(X)$ and substituting, yields,

$$\begin{aligned} E[Y] &= E[(b-a)\, g(X)] = (b-a)\, E[g(X)] \\ &= (b-a) \int_a^b g(x) \frac{1}{b-a} \mathrm{d}x = \int_a^b g(x) \mathrm{d}x = \theta \end{aligned}$$

Therefore, by estimating the expected value of Y with the sample average of the Y_i values, we can estimate the desired integral. Importance sampling does this same thing but does not necessarily use the $U(a, b)$ distribution. Importance sampling rewrites the integration as follows:

$$\theta = \int_a^b g(x) \mathrm{d}x = \int_a^b \frac{g(x)}{w(x)} w(x) \mathrm{d}x$$

The function $w(x)$ is a probability density function defined over $[a, b]$, where $X \sim w(x)$. Instead of defining $Y = (b-a)\, g(X)$, we define Y as:

$$Y = \frac{g(X)}{w(X)}$$

Therefore:

$$E[Y] = E_X\left[\frac{g(X)}{w(X)}\right] = \int_a^b \frac{g(x)}{w(x)} w(x) \mathrm{d}x = \int_a^b g(x) \mathrm{d}x = \theta$$

To estimate θ, we use $\bar{Y}$, where $X_i \sim w(x)$ and:

$$\hat{\theta} = \bar{Y}(n) = \frac{1}{n}\sum_{i=1}^{n} Y_i = \frac{1}{n}\sum_{i=1}^{n} \frac{g(X_i)}{w(X_i)}$$

A judicious choice of $w(x)$ can reduce the variance of the estimator $\hat{\theta}$ allowing us to get more precision in the same amount of samples. While illustrated for 1-D integration, this same theory applies to higher dimensional integrals. However, higher dimensional integrals requires that $\vec{X} \sim w(\vec{X})$ be a multi-variate distribution.

In general, any function $g(x)$ can be factored into two functions such that $g(x) = h(x)f(x)$ where $f(x)$ is a probability density function. The optimal choice for $w(x)$ is the function $f(x)$. However, to find this function, we would in general need to know θ, which is the thing we are trying to estimate.

A good choice for $w(x)$ is one that will cause more samples when $g(x)$ is large and sample infrequently when $g(x)$ is small. That is, sample more in the more important areas. This is the genesis for the name importance sampling. Let's take a look at an example to illustrate these insights.

Suppose we want to integrate the function $g(x) = x^2$ over the range $0 \leq x \leq 1$. This is easy because we can compute the answer:

$$\theta = \int_0^1 g(x)\mathrm{d}x = \int_0^1 x^2\mathrm{d}x = \frac{x^3}{3}\Big|_0^1 = \frac{1}{3}$$

The crude MC estimator has the following factorization, $g(x) = h(x)\, f(x)$, where $h(x) = x^2$ and:

$$f_X(x) = \begin{cases} \frac{1}{1-0} & 0 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Thus, the crude MC estimator for $\theta = E_f[h(X)]$ is:

$$\bar{Y}(n) = \frac{1}{n}\sum_{i=1}^{n} h(X_i) = \frac{1}{n}\sum_{i=1}^{n} g(X_i)$$

Let's derive the variance of the crude MC estimator.

$$\text{Var}(\bar{Y}(n)) = \frac{1}{n}\text{Var}(g(X_i)) = \frac{1}{n}\Big(E[g^2(X)] - \theta^2\Big)$$

$$E[g^2(X)] = \int_0^1 g^2(x)dx = \int_0^1 x^4 dx = \frac{x^5}{5}\bigg|_0^1 = \frac{1}{5}$$

$$\text{Var}(\bar{Y}(n)) = \frac{1}{n}\left(\frac{1}{5} - \theta^2\right) = \frac{1}{n}\left(\frac{1}{5} - \frac{1}{9}\right) = \frac{1}{n}\left(\frac{4}{45}\right)$$

We must try to do better than this value using importance sampling. Also, note that in general, we would not know θ to do this calculation.

The effectiveness of importance sampling hinges on picking the proposal distribution $w(x)$. The goal is to pick a proposal density $w(x)$ that improves on $\text{Var}(\bar{Y}(n))$ of the crude MC estimator. Consider the following possibility:

$$w(x) = \begin{cases} 2x & 0 \le x \le 1 \\ 0 & \text{otherwise} \end{cases}$$

Clearly, this $w(x)$ is a probability density function since $\int_0^1 w(x)dx = \int_0^1 2x\,dx = \frac{2x^2}{2}\Big|_0^1 = 1$. The importance sampling estimator is based on the following:

$$\hat{\theta} = \bar{Y}(n) = \frac{1}{n}\sum_{i=1}^{n} Y_i = \frac{1}{n}\sum_{i=1}^{n} \frac{g(X_i)}{w(X_i)}$$

Let's derive the variance of this estimator based on the given function $w(x)$. Using the definition of variance, we have:

$$\begin{aligned}\text{Var}(\hat{\theta}) &= \frac{1}{n}\text{Var}\left(\frac{g(X_i)}{w(X_i)}\right) = \frac{1}{n}\left(E_w\left[\left(\frac{g(X_i)}{w(X_i)}\right)^2\right] - \theta^2\right) \\ &= \frac{1}{n}\left(\int_0^1 \frac{g^2(x)}{w^2(x)}\, w(x)\, dx - \theta^2\right)\end{aligned}$$

Now, we need to complete this derivation by deriving the second moment for the example problem.

$$\int_0^1 \frac{g^2(x)}{w(x)}\, dx = \int_0^1 \frac{x^4}{2x}\, dx = \int_0^1 \frac{x^3}{2}\, dx = \frac{x^4}{8}\Big|_0^1 = \frac{1}{8}$$

Therefore:

$$\text{Var}(\hat{\theta}) = \frac{1}{n}\left(\frac{1}{8} - \theta^2\right) = \frac{1}{n}\left(\frac{1}{8} - \frac{1}{9}\right) = \frac{1}{n}\left(\frac{1}{72}\right)$$

Let's look at the ratio of the variances of the two competing estimators:

$$VR = \frac{\text{Var}(\hat{\theta})}{\text{Var}(\bar{Y}(n))} = \frac{\frac{1}{n}\left(\frac{1}{72}\right)}{\frac{1}{n}\left(\frac{4}{45}\right)} = \frac{4}{3240} < 1$$

There will be a significant variance reduction with this choice of $w(x)$ over using the standard uniform distribution.

Figure 9.6 shows the two proposal functions $w_1(x)$ ($U(0,1)$) and $w_2(x) = 2x$ relative to $g(x)$. We can see that the closer the proposal distribution is to $g(x)$ as the intuition behind why there is such a dramatic variance reduction.

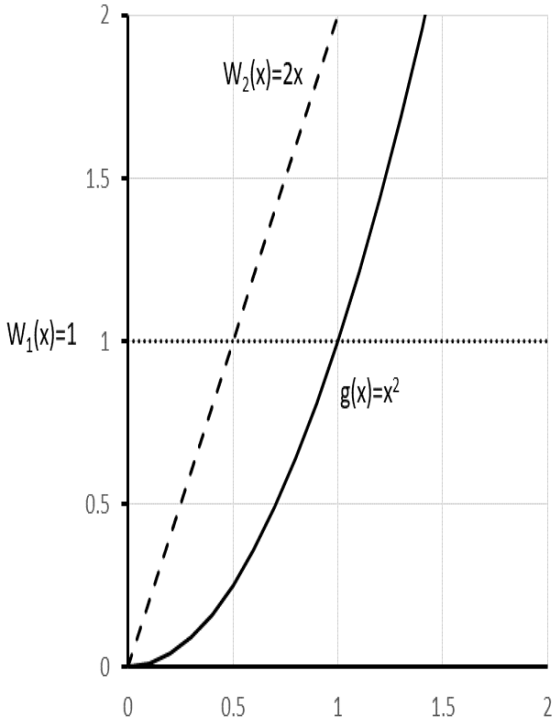


Figure 9.6.: Importance Sampling Based on w(x)

(Rubinstein and Kroese. 2017) devotes considerable time to the topic of importance sampling. The most interesting aspect of their treatment is the discussion on sequential importance sampling, which dynamically updates the proposal distribution as the sampling procedure proceeds in a manner that tries to converge towards the optimal proposal distribution. We encourage the interested reader to explore the reference for further details.

The KSL supports the application of importance sampling via the `MC1DIntegration` and `MCMultiVariateIntegration` classes. Section 3.8 described the use of the `MC1DIntegration`, which permits the application of importance sampling by allowing the user to provide $h(x)$, where

- $w(x)$ is the probability distribution for the random variable supplied by the sampler,
- $g(x)$ is the function that needs to be integrated, and
- $h(x)$ is a factorization of $g(x)$ such that $g(x) = h(x) * w(x)$, that is $h(x) = g(x)/w(x)$

The exercises will ask the reader to apply importance sampling to some of the problems already faced in Chaper Chapter 3. The KSL also supports the application of importance sampling to multi-dimensional integrals; however, since that topic requires the concepts of how to generate from multi-variate distributions, we will defer that discussion until after the next section.

9.3. Generating Multi-Variate and Correlated Random Variates

The purpose of this section is to present the KSL constructs that facilitate the sampling from multi-variate distributions. In order to do this, we will discuss some of the common methods for generating correlated random variables. We will focus on those techniques that have implementations

within the KSL. The presentation will be focused on the practical application of the methods rather than the theory. Because it will be instructive, we will start with generating from bi-variate distributions, specifically the bi-variate normal distribution.

9.3.1. Generating from a Bivariate Normal Distribution

A bivariate normal distribution is specified by the expected values, variances, and correlation between the random variables. For an excellent refresher on the mathematical basis for bivariate normal random variables, we refer the reader to (Pishro-Nik 2014) and specifically Section 5.3.2.

We specify a bivariate normal random variables as $(X_1, X_2) \sim BVN(\mu_1, \sigma_1^2, \mu_2, \sigma_2^2, \rho)$. The key result necessary for generating BVN random variables is the following procedure:

1. Let Z_1 and Z_2 be independent $N(0, 1)$ random variables.
2. Define $X_1 = \sigma_1 Z_1 + \mu_1$ and $X_2 = \sigma_2(\rho Z_1 + \sqrt{(1-\rho^2)} Z_2) + \mu_2$, where $-1 < \rho < 1$.
3. Then, $(X_1, X_2) \sim BVN(\mu_1, \sigma_1^2, \mu_1 2, \sigma_2^2, \rho)$

An outline for the proof of this result can be found in (Pishro-Nik 2014), Section 5.3.2. Thus, if we can generate independent standard normal random variables, we can generate bivariate normal random variables via simple algebra. As can be seen in the following code, the KSL `BivariateNormalRV` class uses this procedure to generate an array that contains the bivariate normal sample.

```
class BivariateNormalRV @JvmOverloads constructor(
    val mean1: Double = 0.0,
    val v1: Double = 1.0,
    val mean2: Double = 0.0,
    val v2: Double = 1.0,
    val corr: Double = 0.0,
```

```
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc =
     ↪ KSLRandom.DefaultRNStreamProvider,
    name: String? = null
) : MVRVariable(streamNum, streamProvider, name) {
.
.
.
    override fun generate(array: DoubleArray) {
        require(array.size == dimension) { "The size of the
 ↪ array to fill does not match the sampling dimension!" }
        val z0 = Normal.stdNormalInvCDF(rnStream.randU01())
        val z1 = Normal.stdNormalInvCDF(rnStream.randU01())
        val s1 = sqrt(v1)
        val s2 = sqrt(v2)
        array[0] = mean1 + s1 * z0
        array[1] = mean2 + s2 * (corr * z0 + sqrt(1.0 - corr
 ↪ * corr) * z1)
    }
```

Figure 9.7 presents the key classes and interfaces involved in the generation of a BVN sample. Notice that the `BivariateNormalRV` class sub-classes from the `MVRVariable` class, which in turn implements the `MVRVariableIfc` and the `MVSampleIfc` interfaces. The key is the `generate(array: DoubleArray)` method, in which the generation algorithm must be implemented.

The functionality of the `MVSampleIfc` interface is especially noteworthy, with the following methods:

- `sample(): DoubleArray`: generates an array that holds the sample
- `sample(array: DoubleArray)`: fills the supplied array with the generated values

Figure 9.7.: Key Classes for BVN Generation

- `sample(sampleSize: Int): List<DoubleArray>`: generates a list holding the randomly generated arrays. This can easily be converted to a 2-dimensional array that contains the samples as its rows.
- `sampleByColumn(sampleSize: Int): Array<DoubleArray>`: generates a 2-dimensional array (matrix) where the columns contain the samples.
- `sample(values: Array<DoubleArray>)`: fills the supplied array of arrays with randomly generated samples by rows.

The key property is the `dimension` property, which specifies the size of the array from the `sample()` method. In the case of the BVN random variable, the dimension is equal to 2. As you can see from Figure 9.7 the analogies with the one dimensional `SampleIfc` and `RVariableIfc` interfaces described in Section 2.2 should be clear. In addition, the `MVRVariableIfc`interface implements the `RNStreamControlIfc` and `RNStreamChangeIfc` interfaces. As you may recall, the `RNStreamControlIfc` interface allows you to control the underlying stream of random numbers. The multi-dimensional case works the same as in the single variate case. Thus, sampling multi-variate distributions should be a straightforward step as long as you realize that you are generating arrays of data.

There are only two directly implemented bi-variate distributions the bivariate normal and the bivariate lognormal distribution. The implementation of the bivariate lognormal is conceptually similar to that of the bivariate normal. Since it can be shown that for the previously discussed bivariate normal procedure that the generated X_1 and X_2 are individually normally distributed then the algorithm for the generating from the bivariate lognormal distribution is straightforward because of the relationship between the two random variables. That is, if $X \sim N(\mu, \sigma^2)$ then the random variable $Y = e^X$ will be lognormally distributed $LN(\mu_l, \sigma_l^2)$, where

$$E[Y] = \mu_l = e^{\mu+\sigma^2/2}$$

$$\text{Var}[Y] = \sigma_l^2 = e^{2\mu+\sigma^2}\left(e^{\sigma^2} - 1\right)$$

Thus, in implemented the bivariate lognormal distribution, we just need to generate BVN random variable and apply $Y = e^X$. This is shown in the `generate()` method of the `BivariateLogNormalRV` class.

```
    override fun generate(array: DoubleArray) {
        require(array.size == dimension) { "The size of the
↪   array to fill does not match the sampling dimension!" }
        myBVN.sample(array)
        // transform them to bi-variate lognormal
        array[0] = exp(array[0])
        array[1] = exp(array[1])
    }
```

Developing your own bivariate generation classes can readily be accomplished by following the pattern presented by the `BivariateNormalRV` and `BivariateLogNormalRV` classes and sub-classing from the `MVRVariable` abstract base class. In this section, we discussed the generation of bivariate normal random variables. The key concept to remember moving into the next section is that now we have a procedure for generating correlated random variables. While the generation of bivariate lognormal random variables used the fact that we can generate bivariate normal random variables, we will see that these ideas can be generalized.

9.3.2. Copulas and Multi-variate Generation Methods

In this section, we will discuss general approaches for generating multi-variate random variables. To reduce the burden of excessive notation, the presentation will focus on the 2-dimensional case. This should not limit the ideas because the concepts can be readily generalized to higher dimensions. One of the most useful concepts used in the generation of multi-variate random variates is that of a copula.

Definition 9.1 (Copula)**.** A d-dimensional *copula*, $C : [0,1]^d :\to [0,1]$ is a cumulative distribution function (CDF) with uniform marginals. That is, if we have a joint distribution of random variables, $(U_1, U_2, \cdots, U_d)$ where each $U_i \sim U(0,1)$, then the copula is the joint CDF:

$$C(u_1, u_2, \cdots, u_d) = P\{U_1 \leq u_1, U_2 \leq u_2, \cdots, U_d \leq u_d\}$$

Note that in the definition of a copula, the $(U_1, U_2, \cdots, U_d)$ are not assumed to be independent. In fact, the whole point is for them to have a dependence structure. A definitive reference text on copulas, their properties and their uses can be found in (Nelson 1999). The main theorem concerning copulas that is relevant to our discussion is due to (Sklar 1959), see (Nelson 1999) for a proof and further discussion.

Theorem 9.1 (Sklar's Theorem)**.** *For any random variables,* $(X_1, X_2, \cdots, X_d)$ *with joint CDF:*

$$F(x_1, x_2, \cdots, x_d) = P\{X_1 \leq x_1, X_2 \leq x_2, \cdots, X_d \leq x_d\}$$

and marginals $F_j(x) = P(X_j \leq x_j)$ *for* $j = 1, 2, \cdots, d$*, there exists a copula, such that:*

$$F(x_1, x_2, \cdots, x_d) = C(F_1(x_1), F_2(x_2), \cdots, F_d(x_d))$$

If each $F_j(x)$ *is continuous, then the copula,* $C(\cdot)$ *will be unique.*

Because of Theorem 9.1, we can express the joint CDF, $F(\cdot)$ in terms of a copula, $C(\cdot)$ and a set of marginal distribution functions, $F_j(x_j)$. This fact allows us to separate the marginals, $F_j(x_j)$ from the dependence structure implied by the copula, $C(\cdot)$. This is extremely useful because it allows us to generate dependent $(U_1, U_2, \cdots, U_d)$ and then use the inverse transform technique to generate from each marginal, $F_j(x_j)$ so that the resulting X_j have a dependence structure implied by the specified copula.

There are many possible choices for specifying copulas. One way to specify a copula is to extract it from a known joint distribution $F(\cdot)$ via an inverse transform operation:

$$C(u_1, u_2, \cdots, u_d) = F(F_1^{-1}(u_1), F_2^{-1}(u_2), \cdots, F_d^{-1}(u_d))$$

The resulting copula will then take on its name from the joint distribution, F, assuming that the marginals are continuous.

An example of this is the so called Gaussian copula. Let $\mathbf{X} \sim \text{MVN}_d(\vec{0}, \mathbf{P})$ be a multi-variate normal random variable, where $\mathbf{P}$ is the correlation matrix of $\mathbf{X}$. Then, the Gaussian copula is as follows:

$$C_{\mathbf{P}}^G(u_1, u_2, \cdots, u_d) = \Phi_{\mathbf{P}}(\Phi_1^{-1}(u_1), \Phi_2^{-1}(u_2), \cdots, \Phi_d^{-1}(u_d))$$

where $\Phi(\cdot)$ is the standard normal CDF and $\Phi_{\mathbf{P}}(\cdot)$ is the joint CDF of $\mathbf{X}$.

To make this concrete, let's consider the bivariate normal copula. Let (X_1, X_2) be from a bivariate normal distribution such that $\text{BVN}(\mu_1 = 0, \sigma_1^2 = 1, \mu_2 = 0, \sigma_2^2 = 1, \rho)$. Now to generate from the Gaussian copula, we have the following algorithm:

1. Generate $Z_1 \sim N(0,1)$ and $Z_2 \sim N(0,1)$
    a. Generate $V_1 \sim U(0,1)$ and $V_2 \sim U(0,1)$
    b. Let $z_1 = \Phi^{-1}(v_1)$ and $z_2 = \Phi^{-1}(v_2)$

2. Let $x_1 = z_1$ and $x_2 = \rho z_1 + z_2\sqrt{(1-\rho^2)}$.
3. Let $u_1 = \Phi(x_1)$ and $u_2 = \Phi(x_2)$
4. Return (u_1, u_2)

The bivariate random variate (u_1, u_2) returned from this algorithm will be from a bivariate Gaussian copula. The random variables from this copula, (U_1, U_2) will be correlated because the generated (x_1, x_2) will have correlation ρ. Thus, we can use the correlated random variables (U_1, U_2) to generate other random variables that have a dependence structure that is implied by the correlation between (U_1, U_2).

For example let $\vec{Y} = (Y_1, Y_2)$ be a random vector and let $F_{Y_1}(\cdot)$ and $F_{Y_2}(\cdot)$ be the marginal distributions associated with Y_1 and Y_2. Then, we can generate $\vec{Y}$ as follows:

$$Y_1 = F_{Y_1}^{-1}(u_1) = F_{Y_1}^{-1}(\Phi(x_1))$$
$$Y_2 = F_{Y_2}^{-1}(u_1) = F_{Y_2}^{-1}(\Phi(x_2))$$

where $x_1 = z_1$ with $Z_1 \sim N(0,1)$ and $x_2 = \rho z_1 + z_2\sqrt{(1-\rho^2)}$ with $Z_2 \sim N(0,1)$. This will cause the vector, $\vec{Y}$ to have the corresponding Gaussian BVN copula. The KSL `BVGaussianCopulaRV` class implements these ideas to generate bivariate correlated random variables. The user is required to supply the inverse CDF functions for the two marginals. Note that, in general, the two distributions, $F_{Y_1}(\cdot)$ and $F_{Y_2}(\cdot)$ do not have to be from the same family. In addition, as long as the inverse CDF function is available the distribution could be discrete or continuous.

```
class BVGaussianCopulaRV @JvmOverloads constructor(
    val marginal1: InverseCDFIfc,
    val marginal2: InverseCDFIfc,
    val bvnCorrelation: Double,
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc =
     ↪  KSLRandom.DefaultRNStreamProvider,
```

```
    name: String? = null
) : MVRVariable(streamNum, streamProvider, name) {

    private val bvGaussianCopula =
     ↪  BVGaussianCopula(bvnCorrelation, streamNum,
     ↪  streamProvider)

    override val dimension: Int
        get() = bvGaussianCopula.dimension

    override fun instance(streamNumber: Int,
     ↪  rnStreamProvider: RNStreamProviderIfc):
     ↪  MVRVariableIfc {
        return BVGaussianCopulaRV(marginal1, marginal2,
         ↪  bvnCorrelation, streamNumber, rnStreamProvider)
    }

    override fun generate(array: DoubleArray) {
        require(array.size == dimension) { "The length of the
 ↪  array was not the proper dimension" }
        // generate the uniforms
        val u = bvGaussianCopula.sample()
        // apply the inverse transform for each of the
         ↪  marginals
        array[0] = marginal1.invCDF(u[0])
        array[1] = marginal2.invCDF(u[1])
    }

}
```

This idea generalizes to the generation of random vectors of dimension, d, if we use the d-dimensional Gaussian copula, $C^G_{\mathbf{P}}(\cdot)$. Thus, because of Sklar's Theorem the joint distribution of $\vec{Y}$ can be written using the specified

Gaussian copula. This is due to the invariance of copulas to monotonic transformations, which is discussed in (Nelson 1999).

There are many different kinds of copulas that have been developed which can be used to generate correlated random variables. The general procedure is as follows:

1. Specify a copula and its parameters
2. Generate $\vec{U} = (u_1, u_2, \cdots, u_d)$ from the copula
3. Generate $\vec{Y} = (Y_1, Y_2, \cdots, Y_d)$, where $Y_i = F_i^{-1}(u_i)$

The generated vector $\vec{Y}$ will have a dependence structure that is determined by the dependence structure of the vector $\vec{U}$. It is important to realize that the dependence structures of these two random vectors will be related, but not necessarily the same. The key is to specify a copula that is easy to generate from and to understand how its dependence structure translates to the desired dependence structure. (Nelsen 2004) presents a survey of the properties of various copulas. In addition, a comprehensive compendium of copulas is available in (Nadarajah et al. 2017).

The KSL supports the generation of random vectors due to its implementation of the multi-variate normal distribution. Thus, the Gaussian copula is available to KSL users. In addition, the formulas in (Nadarajah et al. 2017) can be readily implemented for other copulas.

A d-dimensional multi-variate normal distribution is specified by a vector of means $\vec{\mu} = (\mu_1, \mu_2, \cdots, \mu_d)^T$ and a $d \times d$ covariance matrix, . The entries of are $\text{COV}(X_i, X_j)$ and is symmetric and positive definite. Define $\mathbf{C}$ a $d \times d$ matrix representing the Cholesky decomposition of , where $= \mathbf{C}\mathbf{C}^T$. Then the algorithm for generating MVN random variables is just an extension to the method for generating BVN random variables:

1. Generate $(Z_1, Z_2, \cdots, Z_d)$ as IID $N(0, 1)$ random variables
2. For $i = 1, 2, \cdots, d$, let $X_i = \mu_i + \sum_{j=1}^{i} c_{ij} Z_j$, where $c_i j$ is the (i, j)th element of $\mathbf{C}$, Return $(X_1, X_2, \cdots, X_d)$

Figure 9.8.: Key Classes for Multi-Variate Generation

The KSL `MVNormalRV` implements this algorithm.

```
class MVNormalRV  @JvmOverloads constructor(
    means: DoubleArray,
    covariances: Array<DoubleArray>,
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc =
     ↪  KSLRandom.DefaultRNStreamProvider,
    name: String? = null
) : MVRVariable(streamNum, streamProvider, name)
```

An example for generating from a MVN is shown in the following code.

```
    val cov = arrayOf(
        doubleArrayOf(1.0, 1.0, 1.0, 1.0, 1.0),
        doubleArrayOf(1.0, 2.0, 2.0, 2.0, 2.0),
        doubleArrayOf(1.0, 2.0, 3.0, 3.0, 3.0),
        doubleArrayOf(1.0, 2.0, 3.0, 4.0, 4.0),
        doubleArrayOf(1.0, 2.0, 3.0, 4.0, 5.0)
    )
    val means = doubleArrayOf(10.0, 10.0, 10.0, 10.0, 10.0)
    val rv = MVNormalRV(means, cov)
    for (i in 1..5) {
        val sample: DoubleArray = rv.sample()
        println(KSLArrays.toCSVString(sample))
    }
```

This code produces the following output.

```
10.608312205708812, 12.427876158938025, 15.095818342789086, 13.895399804504395, 15.8
12.357092448543806, 12.093733136888977, 12.89119464537901, 12.45554505056937, 13.885
9.351088769471737, 9.08925546297716, 8.826393183582276, 9.972369809797016, 10.159999
```

```
9.094163074400619, 8.983286878421879, 10.986502362541005, 10.212432226337382, 11.371
9.837869522611827, 9.038466912761681, 9.124068606401634, 9.590808967808215, 11.56139
```

To generate $(U_1, U_2, \cdots, U_d)$ using a Gaussian copula, you can use the `MVGaussianCopula` class. This class requires the specification of the correlation matrix.

```
class MVGaussianCopula @JvmOverloads constructor(
    correlation: Array<DoubleArray>,
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc =
     ↪ KSLRandom.DefaultRNStreamProvider,
    name: String? = null
) : MVRVariable(streamNum, streamProvider, name)
```

The following examples illustrates how to use the class.

```
    val cov = arrayOf(
        doubleArrayOf(1.0, 1.0, 1.0, 1.0, 1.0),
        doubleArrayOf(1.0, 2.0, 2.0, 2.0, 2.0),
        doubleArrayOf(1.0, 2.0, 3.0, 3.0, 3.0),
        doubleArrayOf(1.0, 2.0, 3.0, 4.0, 4.0),
        doubleArrayOf(1.0, 2.0, 3.0, 4.0, 5.0)
    )
    val rho = MVNormalRV.convertToCorrelation(cov)
    val rv = MVGaussianCopula(rho)
    for (i in 1..5) {
        val sample: DoubleArray = rv.sample()
        println(KSLArrays.toCSVString(sample))
    }
```

This produces the following output, which are all marginal uniform random variables.

```
0.7285097863655003, 0.9569891872203626, 0.9983698799092053, 0.9742745590896011, 0.99
0.9907906696144968, 0.9306291228202694, 0.9524642953735563, 0.8902338139058006, 0.95
0.2581978776398398, 0.2597897797244717, 0.24901831402144003, 0.49448874976678453, 0.
0.18251108664042276, 0.2360936451712804, 0.715511037152583, 0.542294556596819, 0.730
0.4356015531081042, 0.24828181183321235, 0.3065268847809944, 0.4189440794684452, 0.7
```

The `MVGaussianCopulaRV` class puts all these components together to generate random vectors using the d-dimensional Gaussian copula by allowing the user to specify the inverse CDF functions for the marginal distributions.

```
class MVGaussianCopulaRV @JvmOverloads constructor(
    val marginals: List<InverseCDFIfc>,
    correlation: Array<DoubleArray>,
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc =
     ↪ KSLRandom.DefaultRNStreamProvider,
    name: String? = null
) : MVRVariable(streamNum, streamProvider, name) {

    private val myCopula = MVGaussianCopula(correlation,
     ↪ stream)
.
.
    override fun generate(array: DoubleArray) {
        require(array.size == dimension) { "The length of the
 ↪ array was not the proper dimension" }
        // generate the uniforms
        val u = myCopula.sample()
        // apply the inverse transform for each of the
         ↪ marginals
        for (i in u.indices) {
            array[i] = marginals[i].invCDF(u[i])
```

```
        }
    }
```

Similar strategies can be implemented for other copula specifications.

9.3.3. Autocorrelated Generation

This section involves how to explicitly model dependence (autocorrelation) within random samples. An important application is to generate correlated values from simulation input models. In the fitting of input models, it was assumed and tested that the sample observations did not have correlation. But, what do you do if the data does have correlation? For example, let X_i be the service time of the i^{th} customer. What if the X_i have significant correlation? That is, the service times are correlated. Input processes might also correlated. That is, the time between arrivals might be correlated. Research has shown, see (Patuwo et al. 1993) and (Livny et al. 1993), that ignoring the correlation when it is in fact present can lead to gross underestimation of the actual performance estimates for the system.

The discussion here is based on the Normal-to-Anything Transformation as discussed in Banks et al. (2005), (Cario and Nelson 1998), (Cario and Nelson 1996), and (Biller and Nelson 2003). Suppose you have a $N(0,1)$ random variable, Z_i, and a way to compute the CDF, $\Phi(z)$, of the normal distribution. Then, from the discussion of the inverse transform technique, the random variable, $\Phi(Z_i)$ will have a U(0,1) distribution. Suppose that you wanted to generate a random variable X_i with CDF $F(x)$, then you can use $\Phi(Z_i)$ as the source of uniform random numbers in an inverse transform technique.

$$X_i = F^{-1}(U_i) = F^{-1}((\Phi(Z_i)))$$

This transform is called the normal-to-anything (NORTA) transformation. It can be shown that even if the Z_i are correlated (and thus so are the

$\Phi(Z_i)$ then the X_i will have the correct CDF and will also be correlated. Unfortunately, the correlation is not directly preserved in the transformation so that if the Z_i has correlation ρ_z the X_i will have ρ_x (not necessarily the same). Thus, in order to induce correlation in the X_i you must have a method to induce correlation in the Z_i.

One method to induce correlation in the Z_i is to generate the Z_i from an stationary autoregressive time-series model of order 1, i.e. $AR(1)$. A stationary $AR(1)$ model with $N(0,1)$ marginal distributions has the following form:

$$Z_i = \phi(Z_{i-1}) + \varepsilon_i$$

where $i = 1, 2, 3, ...$ and ε_i are IID $N(0, 1-\phi^2)$, $i = 1, 2, 3, ...$, $-1 < \phi < 1$, and $Z_1 \sim N(0,1)$. Thus, it is relatively straightforward to generate this process by first generating Z_1, then generating $\varepsilon_i \sim N(0, 1-\phi^2)$ and using $Z_i = \phi(Z_{i-1}) + \varepsilon_i$ to compute the next Z_i. It can be shown that this $AR(1)$ process will have lag-1 correlation:

$$\phi = \rho^1 = corr(Z_i, Z_{i+1})$$

Therefore, you can generate a random variable Z_i that has a desired correlation and through the NORTA transformation produce X_i that are correlated with the correlation being functionally related to ρ_z. By changing ϕ through trial and error one can get the correlation for the X_i that is desired. Procedures for accomplishing this are given in the previously mentioned references.

The implementation of this technique can be readily achieved in a general way via already available constructs. The `AR1CorrelatedRNStream` class implements these ideas in the form of a stream that can be supplied to random variables and other constructs that require a stream.

```
class AR1CorrelatedRNStream @JvmOverloads constructor(
    val lag1Corr: Double,
    private val stream: RNStreamIfc =
     ↪ KSLRandom.nextRNStream(),
) : RNStreamIfc by stream {
    private var myX: Double = 0.0
    private val errorVariance: Double

    init {
        require( (-1.0 < lag1Corr) && (lag1Corr < 1.0)){ "The
 ↪ correlation must be (-1,1)" }
        // generate the first value for the process N(mean,
         ↪ variance)
        myX = KSLRandom.rNormal(stream)
        errorVariance = (1.0 - lag1Corr * lag1Corr)
    }

    private var myPrevU: Double = Double.NaN

    override val previousU: Double
        get() = myPrevU

    override fun randU01(): Double {
        val e = KSLRandom.rNormal(0.0, errorVariance, stream)
        myX = lag1Corr * myX + e
        val u = Normal.stdNormalCDF(myX)
        myPrevU = u
        return u
    }
}
```

In the next section, we explore a general method for generating multi-variate samples using Markov Chain Monte Carlo methods.

9.3.4. Introduction to Markov Chain Monte Carlo Methods

In this section, we present the a brief overview of Markov Chain Monte Carlo (MCMC) methods, especially as they relate to the constructs available within the KSL. MCMC has a deep and important history of both theoretical and practical applications. This overview provides a minimal amount of theory and then presents the related KSL classes. For tutorials on MCMC the reader should consult the following references, (Brooks 1998), (Robert and Changye, n.d.), (Geyer 2011), (Van Ravenzwaaij and Brown 2018), and (Speagle 2020). For textbook treatments within the context of simulation, the reader might consult (Ross 2023), (Fishman 2006), or (Rubinstein and Kroese. 2017).

MCMC is a numerical simulation technique that facilitates the approximate generation of random variates from arbitrary distributions. The distributions need only be specified up to a multiplicative constant, which is very useful in Bayesian applications. In addition, the theory works for multi-variate distributions, which allows for the generation of random vectors from arbitrary joint probability distributions. Because MCMC facilitates the generation of samples from multi-variate distributions it often has applications in estimating complex multi-dimensional integrals and has applications in the optimization of multi-variate functions.

Recall that the application of the Monte Carlo method, basically comes down to solving problems of the form for $\vec{x} \in \mathbb{R}^d$ and integration limit set, S, such that $S \subseteq \mathbb{R}^d$:

$$\theta = \int_S g(\vec{x}) \mathrm{d}\vec{x}$$

We estimated θ by generating random samples of $\vec{X}_i$ according to some probability distribution $w(\vec{x})$ over the limiting set, S, and averaging over the Y_i, where

$$Y_i = \frac{g(\vec{X}_i)}{w(\vec{X}_i)}$$

which is justified because

$$E[Y] = E_{\vec{X}}\left[\frac{g(\vec{X})}{w(\vec{X})}\right] = \int_S g(\vec{x})\mathrm{d}\vec{x} = \theta$$

When we discussed importance sampling, we noted that $g(\vec{x})$ can be factorized as follows $g(\vec{x}) = h(\vec{x})w(\vec{x})$ where $w(\vec{x})$ is a probability distribution with $\vec{x} \in S \subseteq \mathbb{R}^d$. Thus, an estimate of θ is

$$\hat{\theta} = \bar{Y}(n) = \frac{1}{n}\sum_{i=1}^{n} Y_i = \frac{1}{n}\sum_{i=1}^{n} \frac{g(\vec{X}_i)}{w(\vec{X}_i)} = \frac{1}{n}\sum_{i=1}^{n} h(\vec{X}_i)$$

We know that independent $\vec{X}_i$ ensures that the law of large numbers has, $\hat{\theta} \to \theta$ as $n \to \infty$ with probability 1. But what happens if the $\vec{X}_i$ are dependent? Under some technical conditions, it can still be shown that $\hat{\theta} \to \theta$. Thus, sequences of dependent $\vec{X}_i$ could be used. One method for generating dependent $\vec{X}_i$ would be from a Markov chain that has $w(\vec{X})$ as its stationary limiting distribution. In the Markov Chain literature, $w(\vec{X})$ is often denoted as $\pi(\vec{X})$.

For simplicity in what follows, we will focus on the univariate case, with the understanding that the presentation can be extended to a multi-dimensional setting. Because it should make it easier to conceptualize the approach, we will first present some basic concepts using discrete Markov Chains.

A Markov Chain is a collection of random variables $\{X_1, X_2, \cdots\}$ where we interpret the X_n as the state of the process at time step n. For simplicity, suppose the state space is finite and labeled $\{1, 2, \cdots, N\}$. Let $P\{X_{n+1} = j | X_n = i, X_{n-1} = k, \cdots, X_0 = l\}$ be the probability that the

process transitions to state j at time $n+1$ given all previous state visits. If this probability only depends on the previous state, then we have a Markov Chain. Thus, if the following is true

$$P\{X_{n+1} = j | X_n = i, X_{n-1} = k, \cdots, X_0 = l\} = P\{X_{n+1} = j | X_n = i\}$$

We call $P\{X_{n+1} = j | X_n = i\}$ the single step transition probabilities, p_{ij}, and the process $\{X_n\}$ is a Markov Chain. Notice that the p_{ij} form a conditional probability distribution across the states $j = 1, 2, \cdots, N$ for a given state i. Thus, we have that

$$\sum_{j=1}^{N} p_{ij} = \sum_{j=1}^{N} P\{X_{n+1} = j | X_n = i\} = 1$$

Therefore the process must be in some state $j \in \{1, 2, \cdots, N\}$ after it transitions from state i. To simulate k transitions of a discrete state Markov Chain, we can perform the following algorithm:

0. initialize $X_0 = i$ and $n = 0$

for n = 1 to k;

1. generate $j \sim P\{X_{n+1} | X_n = i\}$
2. let $X_n = j$ and $i = j$

end for;

The KSL `DMarkovChain` class in the `ksl.utilities.random.markovchain` package facilitates the simulation of simple discrete state Markov Chains.

Example 9.7 (Simulating a Markov Chain)**.** An ergodic Markov Chain is setup and simulated with statistics collected on the probability of being within the states.

```
    val p = arrayOf(
        doubleArrayOf(0.3, 0.1, 0.6),
        doubleArrayOf(0.4, 0.4, 0.2),
        doubleArrayOf(0.1, 0.7, 0.2)
    )

    val mc = DMarkovChain(1, p)
    val f = IntegerFrequency()

    for (i in 1..100000) {
        f.collect(mc.nextState())
    }
    println("True Steady State Distribution")
    println("P{X=1} = " + 238.0 / 854.0)
    println("P{X=2} = " + 350.0 / 854.0)
    println("P{X=3} = " + 266.0 / 854.0)
    println()
    println("Observed Steady State Distribution")
    println(f)
```

This code results in the following output.

```
True Steady State Distribution
P{X=1} = 0.2786885245901639
P{X=2} = 0.4098360655737705
P{X=3} = 0.3114754098360656

Observed Steady State Distribution
Frequency Tabulation ID_2
---------------------------------------
Number of cells = 3
Lower limit = -2147483648
Upper limit = 2147483647
```

```
Under flow count = 0
Over flow count = 0
Total count = 100000.0
Minimum value observed = 1
Maximum value observed = 3
---------------------------------------
Value    Count   Proportion
1    27812.0     0.27812
2    41138.0     0.41138
3    31050.0     0.3105
---------------------------------------
```

As we can see from the output, the distribution of the observations from the chain are converging to the steady state limiting distribution.

There are many issues related to the analysis of Markov Chains that are discussed in some of the aforementioned textbooks. The main property of concern here is that a Markov Chain is said to be *irreducible* if for each pair of states i and j, there exists a probability of starting in state i that eventually allows state j to be visited. For an irreducible Markov Chain that is also aperiodic, there will exist a limiting distribution across the states, $\pi_j = \lim_{n\to\infty} P\{X_n = j\}$, which represents the steady state probability that the process is in state j or sometimes called the long run proportion of time that the process is in state j. Suppose now that we have a Markov Chain such that π_j exists, then it can be shown that for any function $h(\cdot)$ on X_n, that:

$$\lim_{n\to\infty} \frac{1}{n} \sum_{i=1}^{n} h(X_i) = \sum_{j=1}^{N} h(j)\pi_j \quad \text{w.p. 1}$$

This result is the motivation for MCMC.

In MCMC, we want to generate random variables, $X \sim P(X = j)$, for $j = 1, 2, \cdots, N$ and estimate $E[h(X)] = \sum_{j=1}^{N} h(j)p_j$. Thus, why not set

up a Markov Chain with limiting probability $\pi_j = p_j$, then X_n will have approximately the distribution of X after a sufficiently large number of steps n of the Markov Chain. Thus, we can estimate $E[h(X)]$ by $\frac{1}{n}\sum_{i=1}^{n} h(X_i)$ for large n. The key requirement is to be able to set up a Markov Chain such that $\pi_j = p_j$ is true. You might ask, if we have p_j, why not just use:

$$E[h(X)] = \sum_{j=1}^{N} h(j)p_j$$

In many cases, especially in the continuous case, p_j is not known or is known only up to a multiplicative constant. This is when MCMC become useful. In traditional Markov Chain theory, we are typically interested in determining the steady state distribution, π_j from a given transition matrix, $\mathbf{P}$, assuming that it is irreducible and aperiodic. The key MCMC result, turns this problem around: Given a desired π_j what $\mathbf{P}$ should we construct to get the desired π_j. The Metropolis-Hastings algorithm prescribes a method to address this problem.

The basic idea behind the Metropolis-Hastings algorithm is to use an arbitrary transition matrix $\mathbf{Q}$ and accept or reject proposed state changes in such a manner that the resulting observations will come from a transition matrix $\mathbf{P}$ that has the desired π_j. The steps are as follows:

1. When $X_n = i$, generate random variable $Y \sim P(Y = j) = q_{ij}$, $j \in S$
2. If $Y = j$, let $X_{n+1} = j$ with probability α_{ij} or i with probability $1 - \alpha_{ij}$, where $\alpha_{ij} = min\left[\frac{\pi_j q_{ji}}{\pi_i q_{ij}}, 1\right]$

The resulting Markov Chain, X_n will have one step transition probabilities:

$$p_{ij} = \begin{cases} q_{ij}\alpha_{ij} & \text{if } i \neq j \\ 1 - \sum_{k \neq i} q_{ik}\alpha_{ik} & \text{if } i = j \end{cases}$$

and the Markov Chain will have the desired stationary probabilities π_j. The proof of this result can be found in (Ross 2023). This seems like a lot of work given that we have to know π_j; however, close examination of α_{ij} indicates that we need only know π_j up to proportionality constant, e.g. $\pi_j = b_i/C$ because of the ratio form of α_{ij}.

The payoff is when we want to generate samples from some arbitrary multi-dimensional pdf, $f(\vec{x})$. The theory can be generalized to this case by generalizing from a discrete Markov Chain to a continuous Markov Chain. This is done by defining a probability transition function $q(\vec{x}, \vec{y})$ which is called the proposal function. The proposal function proposes the next state and is acting as the arbitrary transition matrix $\mathbf{Q}$. With that in mind, it is really a conditional probability density function $q(\vec{y}|\vec{x})$ which represents the probability of generating $\vec{Y}$ given we are in state $\vec{x}$. The α_{ij} is generalized to a function, $\alpha(\vec{x}, \vec{y})$, which is called the acceptance probability function. The general Metropolis-Hastings algorithm is:

Let $f(\vec{x})$ be the target distribution.

Let $q(\vec{y}|\vec{x})$ be the proposal distribution.

Let $\rho(\vec{x}, \vec{y})$ be the acceptance ratio, where

$$\rho(\vec{x}, \vec{y}) = \frac{q(\vec{x}|\vec{y}) \times f(\vec{y})}{q(\vec{y}|\vec{x}) \times f(\vec{x})}$$

Let $\alpha(\vec{x}, \vec{y}) = min[\rho(\vec{x}, \vec{y}), 1]$ be the acceptance function.

1. Set $t = 0$
2. Set X_0

for t = 1 to ?;

3. $Y \sim q(\vec{y}|\vec{x})$
4. $U \sim U(0, 1)$
5. if $U \leq \alpha(\vec{x}, \vec{y})$ then $X_{t+1} = Y$ else $X_{t+1} = X_t$

end for;

When applying the Metropolis-Hastings algorithm, you obviously need the distribution from which you intend to generate random variates, $f(\vec{x})$. The function $f(\vec{x})$ can be the distribution function specified up to a proportionality constant. The second key function is the proposal distribution (or proposal function), $q(\vec{x}, \vec{y}) = q(\vec{y}|\vec{x})$. There are many possible forms for this function and those forms specify the type of sampler for the MCMC.

Figure 9.9 presents the main classes and interfaces for the Metropolis-Hastings implementation within the KSL. The `MetropolisHastingsMV` class is supported by two interfaces: 1) `FunctionMVIfc` and 2) `ProposalFunctionMVIfc.` Let's examine these two classes before reviewing the Metropolis-Hastings implementation.

The purpose of the `FunctionMVIfc` interface is to represent the desired multi-variate distribution, $f(\vec{x})$. This is the probability density of the distribution from which you want the random vectors. This interface requires the dimension of the function and the method to represent the functional transformation.

```
interface FunctionMVIfc {

    /**
     *
     * the expected size of the array
     */
    val dimension: Int

    /**
     * Returns the value of the function for the specified
↪   variable value.
     * The implementor of f(x) should check if the array size
↪   is the
     * same as the dimension of the function
```

Figure 9.9.: Key Classes and Interfaces for MCMC

```
     */
    fun f(x: DoubleArray): Double
}
```

The second interface is the `ProposalFunctionMVIfc` interface which represents proposal density function $q(\vec{y}|\vec{x})$ with the `generateProposedGivenCurrent()` function and the proposal function's contribution to the computation of the acceptance ratio function, $\rho(\vec{x}, \vec{y})$ via the `proposalRatio()` function. The examples that follow will illustrate this function. The `generateProposedGivenCurrent()` function takes in the current state as its parameter and returns a possible next state.

```
interface ProposalFunctionMVIfc {

    /**
     *
     * the expected size of the array
     */
    val dimension: Int

    /** The ratio of g(y,x)/g(x,y).  The ratio of the
     ↪  proposal function
     * evaluated at x = current and y = proposed, where g()
 ↪  is some
     * proposal function of x and y. The implementor should
 ↪  ensure
     * that the returned ratio is a valid double
     *
     * @param current the x to evaluate
     * @param proposed the y to evaluate
     * @return the ratio of the proposal function
     */
    fun proposalRatio(current: DoubleArray, proposed:
     ↪  DoubleArray): Double
```

```
    /**
     *
     * @param current the current state value of the chain
 ↪  (i.e. x)
     * @return the generated possible state (i.e. y) which
 ↪  may or may not be accepted
     */
    fun generateProposedGivenCurrent(current: DoubleArray):
     ↪  DoubleArray
}
```

Let's consider a couple of common samplers. The first to consider is the *independence* sampler. For the independence sampler, the probability of going to $\vec{y}$ from state $\vec{x}$ is independent of $\vec{x}$. Thus, the proposal function, $q(\vec{y}|\vec{x})$ is simply a function of $\vec{y}$. That is, $q(\vec{y}|\vec{x}) = q(\vec{y})$. This causes the acceptance ratio to be:

$$\rho(\vec{x}, \vec{y}) = \frac{q(\vec{x}|\vec{y}) \times f(\vec{y})}{q(\vec{y}|\vec{x}) \times f(\vec{x})} = \frac{q(\vec{x}) \times f(\vec{y})}{q(\vec{y}) \times f(\vec{x})}$$

Similar in some respects as the acceptance-rejection method, good proposal functions $q(\vec{y})$ should have similar shape as $f(\vec{x})$. Even though the name of this sampler suggests independence, this independence is in the proposal function, not in the resulting Markov Chain and its state values. That is, the generated $\vec{X}_i$ will be dependent because they come from the underlying Markov Chain implied by the proposal function.

Another common sampler is the *random walk* sampler. For a random walk sampler, the proposal function is symmetric. Because of symmetry, $q(\vec{y}|\vec{x}) = q(\vec{x}|\vec{y})$. This causes the acceptance ratio to be:

$$\rho(\vec{x}, \vec{y}) = \frac{q(\vec{x}|\vec{y}) \times f(\vec{y})}{q(\vec{y}|\vec{x}) \times f(\vec{x})} = \frac{f(\vec{y})}{f(\vec{x})}$$

Typical implementations have, $\vec{Y} = \vec{x} + \vec{W}$, where $\vec{W} \sim G(\vec{W})$ and the distribution $G(\cdot)$ is symmetric, such as the multi-variate normal distribution, $\mathbf{MVN}(\vec{0},)$. Let's look at an example to put these ideas into practice.

Example 9.8 (MCMC Sampler Example)**.** Suppose that the random variable X_1 is the zinc content of an ore sample with a range of values in $[0.5, 1.5]$ and the random variable X_2 is the iron content of the ore with values in $[20.0, 35.0]$. The joint distribution density function for the random variables has been modeled as:

$$f(x_1, x_2) = \begin{cases} \frac{39}{400} - \frac{17(x_1-1)^2}{50} - \frac{(x_2-25)^2}{10000} & 0.5 \leq x_1 \leq 1.5;\ 20 \leq x_2 \leq 35 \\ 0.0 & \textit{otherwise} \end{cases}$$

a) Develop a MCMC independent sampler to generate (X_1, X_2) from this distribution.
b) Develop a MCMC random walk sampler to generate (X_1, X_2) from this distribution.

The key issue is developing a suitable proposal function. Recall that for an independence sampler, we have $q(\vec{y}|\vec{x}) = q(\vec{y})$. The function, $q(\vec{y})$ is a probability density function and it must be a function of $\vec{y} = (y_1, y_2)^T$. That is, $q(y_1, y_2)$ is a joint density function for random variables Y_1 and Y_2. Now, it might be more readily apparent as to the reason an independence sampler is common. Notice that the general proposal function $q(\vec{y}|\vec{x})$ would be a conditional distribution of the form $q(y_1, y_2|X_1 = x_1, X_2 = x_2)$. Thus, an independence sampler avoids having to develop a function that includes

$X_1 = x_1, X_2 = x_2$ in its functional form. So, we need a joint density function that will generate, (Y_1, Y_2).

The proposed random vector, $\vec{Y}$, needs to be within the set of possible values for $\vec{X}$. There is no requirement that the joint density have a correlation structure. Thus, it is acceptable to specify the distributions for Y_1 and Y_2 as independent marginals. Therefore to keep things simple for illustrative purposes, we will specify $Y_1 \sim beta(\alpha_1 = 2, \alpha_2 = 5, min = 0.5, max = 1.5)$ and $Y_2 \sim beta(\alpha_1 = 2, \alpha_2 = 5, min = 20.0, max = 35.0)$. Both of these distributions are not symmetric and cover the ranges for Y_1 and Y_2.

We make no claim that these distributions are better than any other distributions for the range of $\vec{X}$ for the purposes of MCMC. They simply meet the requirement. Thus, letting $g_{Y_1}(y_1)$ be the generalized beta PDF for Y_1 and $g_{Y_2}(y_2)$ be the generalized beta PDF for Y_2, we have that the joint density $q(y_1, y_2) = g_{Y_1}(y_1) \times g_{Y_2}(y_2)$ because of the assumption of independence between Y_1 and Y_2.

Now we can specify the acceptance ratio function:

$$\rho(\vec{x}, \vec{y}) = \frac{q(\vec{x}|\vec{y}) \times f(\vec{y})}{q(\vec{y}|\vec{x}) \times f(\vec{x})} = \frac{q(\vec{x}) \times f(\vec{y})}{q(\vec{y}) \times f(\vec{x})} = \frac{g_{Y_1}(x_1) \times g_{Y_2}(x_2) \times f(y_1, y_2)}{g_{Y_1}(y_1) \times g_{Y_2}(y_2) \times f(x_1, x_2)}$$

Notice that $\rho(\vec{x}, \vec{y})$ is simply a function of x_1, x_2, y_1, y_2. We are *evaluating* the probability density functions at the points x_1, x_2, y_1, y_2. Thus, $\rho(\vec{x}, \vec{y})$ is essentially a likelihood ratio. In implementing the Metropolis-Hastings algorithm it is useful to define the acceptance ratio, $\rho(\vec{x}, \vec{y})$, in terms of the proposal ratio, $p_r(\vec{x}, \vec{y})$, and the function ratio, $f_r(\vec{x}, \vec{y})$, where:

$$p_r(\vec{x}, \vec{y}) = \frac{q(\vec{x}|\vec{y})}{q(\vec{y}|\vec{x})}$$

and,

$$f_r(\vec{x}, \vec{y}) = \frac{f(\vec{y})}{f(\vec{x})}$$

which yields, $\rho(\vec{x}, \vec{y}) = p_r(\vec{x}, \vec{y}) \times f_r(\vec{x}, \vec{y})$. The `proposalRatio()` function of the `ProposalFunctionMVIfc` interface implements $p_r(\vec{x}, \vec{y})$.

To implement the Metropolis-Hastings algorithm for this sampler we need to implement the `FunctionMVIfc` interface to implement the distribution of interest. Here is the KSL code for the example problem.

```
object Function : FunctionMVIfc {
    override val dimension: Int
        get() = 2

    override fun f(x: DoubleArray): Double {
        val x0p = 17.0 * (x[0] - 1.0) * (x[0] - 1.0) / 50.0
        val x1p = (x[1] - 25.0) * (x[1] - 25.0) / 10000.0
        return (39.0 / 400.0) - x0p - x1p
    }
}
```

In addition, we need to implement the `ProposalFunctionMVIfc` interface, which requires the proposal ratio, $p_r(\vec{x}, \vec{y})$ and the proposal function $q(\vec{x}|\vec{y})$. The KSL code for the suggested independence sampler is as follows. The code used the `GeneralizedBeta` class from the `ksl.utilities.distributions` package because the distributions package contains the implementations for the probability density function.

```
object ExampleIndependencePF : ProposalFunctionMVIfc {

    private val myY1Dist = GeneralizedBeta(alphaShape = 2.0,
     ↪  betaShape = 5.0, minimum = 0.5, maximum = 1.5)
```

```
    private val myY2Dist = GeneralizedBeta(alphaShape = 2.0,
     ↪  betaShape = 5.0, minimum = 20.0, maximum = 35.0)
    private val myY1RV = myY1Dist.randomVariable()
    private val myY2RV = myY2Dist.randomVariable()

    override val dimension: Int
        get() = 2

    override fun proposalRatio(currentX: DoubleArray,
     ↪  proposedY: DoubleArray): Double {
        //g(y|x) = g(x,y)
        // proposal ratio = g(x|y)/g(y|x) = g(y,x)/g(x,y)
        // for independent sampler the proposal ratio is
         ↪  g(x)/g(y)
        val gx = myY1Dist.pdf(currentX[0]) *
         ↪  myY2Dist.pdf(currentX[1])
        val gy = myY1Dist.pdf(proposedY[0]) *
         ↪  myY2Dist.pdf(proposedY[1])
        return (gx / gy)
    }

    override fun generateProposedGivenCurrent(currentX:
     ↪  DoubleArray): DoubleArray {
        return doubleArrayOf(myY1RV.value, myY2RV.value)
    }
}
```

To run the simulation and generate the random vectors, we use the `MetropolisHastingsMV` class as illustrated in the following code. Because the function in the example is relatively simple, the expected values of the marginals can be readily computed. In this case, $E[X_1] = 1.0$ and $E[X_2] = 1751/64 \approx 27.36$. The chain starts at $(1.0, 27.5)$, a legal point near these values. The initial starting point can be some arbitrary state

as long as it is a legal point. This code warms the chain up by 10000 steps and attaches an observer to the generation process to capture the generated values to a file.

```
    val f = Function
    val q = ExampleIndependencePF
    val x0 = doubleArrayOf(1.0, 27.5)
    val m = MetropolisHastingsMV(x0, f, q)
    m.runWarmUpPeriod(10000)
    m.attachObserver(WriteData("IndData.csv"))
    m.runAll(10000)
    println(m)
```

The results are only illustrative of the type of default results that are available.

```
MetropolisHastingsMV
Initialized Flag = true
Burn In Flag = true
Initial X =[0.6364642774756969, 28.12096270621112]
Current X = [0.7659084790625525, 26.987110000370624]
Previous X = [0.8055044338305741, 30.436063687178418]
Last Proposed Y= [0.7659084790625525, 26.987110000370624]
Last Prob. of Acceptance = 0.138993174923458
Last f(Y) = 0.0784735337252082
Last f(X) = 0.08168322257064176
Acceptance Statistics
Statistic{name='Acceptance Statistics', n=10000.0, avg=0.36480000000000007, sd=0.481
BatchStatistic{name='X_1', n=39.0, avg=0.9604509592371925, sd=0.09229882862110408, c
1 corr=0.06320655074977856, Total number observed = 10000.0}
BatchStatistic{name='X_2', n=39.0, avg=26.295168067266985, sd=1.5629830939761131, ci
1 corr=0.02800560418008783, Total number observed = 10000.0}
```

The `MetropolisHastingsMV` class automatically estimates the probability of acceptance for the generation process. Generally, the higher the better for efficiency and other characteristics of the chain. In general, achieving a high probability of acceptance may be quite difficult. Probability of acceptance values in the range from 20-50% are not uncommon. The statistics collected on the marginal output show that this sampler does not work well. The confidence interval for X_1, [0.931, 0.990], does not contain $E[X_1] = 1.0$, and the confidence interval for X_2, [25.79, 26.80], does not contain $E[X_2] \approx 27.36$. The default results use the `BatchStatistic` class because of the correlation that is typically present in MCMC output. The correlation reported is for the batch means, not the state observations. The raw observation statistics based on the `Statistic` class can be obtained via the `observedStatistics()` method of the `MetropolisHastingsMV` class. Keep in mind that the confidence intervals reported from the `Statistic` class assume independent observations. Thus, they will have bias as discussed in Section 5.6.3.

We can see from some of the initial data that the state vector can repeat in the sampling. Long runs of repeated observations indicate that the Markov Chain is not mixing well. In this run, 6,352 of the 10,000 observations repeat the previous state, and the longest run repeats one state 286 times. This can be a common occurrence for independence samplers, and here the choice of proposal causes it. The beta(2, 5) density of each proposal is nearly 0 near the upper end of its range, where $f(\vec{x})$ is not small. Thus, the sampler rarely proposes states in that region, and when the chain does reach such a state, almost every proposal from it has a small acceptance probability, so the chain stays there. On balance, the region is under-represented in the sample, which is why the estimated means are too small. Exercise 9.19 asks you to design a better independence sampler.

```
0.5376005066766236, 26.352266492202997
0.8477251697039869, 23.90232944372732
0.7957188105835784, 21.02435324965421
0.9680440314244422, 21.682499716304555
```

```
1.0525995524200615, 26.843069769496953
1.0525995524200615, 26.843069769496953
1.0525995524200615, 26.843069769496953
1.0525995524200615, 26.843069769496953
1.0525995524200615, 26.843069769496953
1.0525995524200615, 26.843069769496953
1.0525995524200615, 26.843069769496953
0.8572452041877787, 30.282898990787338
```

We will reexamine the example, in order to illustrate a random walk sampler. For a random walk sampler, we need to have a symmetric distribution. The random variable X has a symmetric distribution if and only if there is a number c such that $P(X < c - k) = P(X > c + k)$, for all k. A partial list of symmetric distributions can be found on this Wikipedia page. The most commonly used distributions in random walk samplers are the uniform and the normal distributions.

The main challenge that we face in this example is that the proposal distribution, $q(\vec{x}|\vec{y})$ should have the same set of possible values as the multi-variate distribution function $f(\vec{x})$. In the example $f(\vec{x})$ is bounded within the limits, $0.5 \leq x_1 \leq 1.5$ and $20 \leq x_2 \leq 35$. We avoided this issue in the independence sampler implementation because of the choice of the beta distributions for the marginals of the sampler.

As previously noted, typical implementations have, $\vec{Y} = \vec{x} + \vec{W}$, where $\vec{W} \sim G(\vec{W})$ and $G(\cdot)$ is symmetric. Because the proposed value $\vec{x} + \vec{W}$ may "walk" outside the bounds, we need to handle this situation. One possible choice would be to use the truncated normal distribution $N_{[a,b]}(\mu = x, \sigma^2)$, where $[a, b]$ is specified by our limits on $\vec{X}$. Exercise 9.20 asks you to develop this sampler. In this example, we will use the uniform distribution $U(x - c, x + c)$ where $c > 0$ is some typically small positive constant. We will handle the boundary situation within the implementation.

The implementation for $f(\vec{x})$ remains the same. The proposal ratio and proposal function are implemented in the following code. Here we specify

the intervals for each of the dimensions, the small constants, c_i for each dimension, and the random variable, $e_i \sim U(-c_i, c_i)$ for each dimension. Because `genYGivenX()` discards any value outside the range and tries again, the proposed value y_i is uniformly distributed over the part of $[x_i - c_i, x_i + c_i]$ that lies within the range of X_i. Let $[a_i, b_i]$ represent the range of X_i, and let $L_i(x_i)$ represent the length of the part of $[x_i - c_i, x_i + c_i]$ within that range:

$$L_i(x_i) = \min(b_i, x_i + c_i) - \max(a_i, x_i - c_i)$$

Thus, $q(y_i|x_i) = 1/L_i(x_i)$. Away from the boundaries, $L_i(x_i) = 2c_i$ and the proposal is symmetric. Near a boundary, $L_i(x_i)$ is smaller, so $q(y_i|x_i)$ and $q(x_i|y_i)$ can differ, and the proposal ratio is:

$$p_r(\vec{x}, \vec{y}) = \frac{L_1(x_1)L_2(x_2)}{L_1(y_1)L_2(y_2)}$$

For X_1, $c_1 = 1$ covers the whole range from any x_1, so $L_1(x_1) = 1$ always. For X_2, the ratio matters whenever the current or proposed value is within 5 of 20 or 35. Treating the proposal as symmetric, with a proposal ratio of 1.0, would bias the chain near the boundaries.

```
object ExampleRandomWalkPF : ProposalFunctionMVIfc {
    private val y1Interval = Interval(0.5, 1.5)
    private val y2Interval = Interval(20.0, 35.0)
    private val y1c = 1.0
    private val y2c = 5.0
    private val e1rv = UniformRV(-y1c, y1c)
    private val e2rv = UniformRV(-y2c, y2c)

    override val dimension: Int
        get() = 2
```

```
    override fun proposalRatio(currentX: DoubleArray,
     ↪  proposedY: DoubleArray): Double {
        // g(y|x) = 1/L(x), where L(x) is the length of [x -
         ↪  c, x + c] within the range
        // proposal ratio = g(x|y)/g(y|x) = L(x)/L(y)
        val lx = windowLength(y1Interval, y1c, currentX[0])
         ↪  * windowLength(y2Interval, y2c, currentX[1])
        val ly = windowLength(y1Interval, y1c, proposedY[0])
         ↪  * windowLength(y2Interval, y2c, proposedY[1])
        return (lx / ly)
    }

    private fun windowLength(interval: Interval, c: Double,
     ↪  x: Double): Double {
        return minOf(interval.upperLimit, x + c) -
         ↪  maxOf(interval.lowerLimit, x - c)
    }

    private fun genYGivenX(interval: Interval, rv: UniformRV,
     ↪  x: Double): Double {
        var yp: Double
        do {
            val e = rv.value
            yp = x + e
        } while (!interval.contains(yp))
        return yp
    }

    override fun generateProposedGivenCurrent(currentX:
     ↪  DoubleArray): DoubleArray {
        val y1 = genYGivenX(y1Interval, e1rv, currentX[0])
        val y2 = genYGivenX(y2Interval, e2rv, currentX[1])
        return doubleArrayOf(y1, y2)
```

```
    }
}
```

A small private function `genYGivenX()` is used to sample a value $x_i + e_i$ until the values are within the desired intervals. The private function `windowLength()` computes $L_i(x_i)$ for the proposal ratio. To setup and run this case, we have the following code, which is similar to previous run simulation code, except for the use of the random walk sampler.

```
    val f = Function
    val q = ExampleRandomWalkPF
    val x0 = doubleArrayOf(1.0, 27.5)
    val m = MetropolisHastingsMV(x0, f, q)
    m.runWarmUpPeriod(10000)
    m.attachObserver(WriteData("RWData.csv"))
    m.runAll(10000)
    println(m)
```

These results are much better than the independence sampler results. The acceptance probability is about 76%, and the confidence intervals for X_1, [0.988, 1.000], and X_2, [27.03, 27.52], contain the expected values.

```
MetropolisHastingsMV
Initialized Flag = true
Burn In Flag = true
Initial X =[1.232898824952116, 21.66210904618112]
Current X = [0.9901386983588482, 34.56485919177585]
Previous X = [1.0010915655254986, 30.856035274473697]
Last Proposed Y= [0.9901386983588482, 34.56485919177585]
Last Prob. of Acceptance = 1.0
Last f(Y) = 0.08831828347233045
Last f(X) = 0.09407027997121117
```

```
Acceptance Statistics
Statistic{name='Acceptance Statistics', n=10000.0, avg=0.7649999999999971, sd=0.4240
BatchStatistic{name='X_1', n=39.0, avg=0.9942722556241099, sd=0.01908247292570086, c
1 corr=0.02188004287227909, Total number observed = 10000.0}
BatchStatistic{name='X_2', n=39.0, avg=27.274340246967373, sd=0.7516747812388619, ci
1 corr=0.06419291643105446, Total number observed = 10000.0}
```

As we can see from the sampled vectors, there appears to be more mixing.

```
1.069579952507243, 22.024550682660546
0.680904832582785, 22.609508333912288
1.2548563613121715, 20.082804816132036
1.1412489245140436, 21.174950928987187
1.1412489245140436, 21.174950928987187
1.1412489245140436, 21.174950928987187
1.1412489245140436, 21.174950928987187
1.1412489245140436, 21.174950928987187
1.1412489245140436, 21.174950928987187
1.1412489245140436, 21.174950928987187
0.9609462269302526, 22.30999636475328
0.8646527486506351, 24.00918525036553
```

This section provided an introduction to Markov Chain Monte Carlo methods. The topic of MCMC is vast and the proliferation of research and applications has been enormously active in the past 20 years. This introduction was meant to only illustrate some of the key concepts that you need to understand when applying the technique using the KSL. There are many additional issues that we did not address, including:

- warm up period determination - This has similar characteristics as to that discussed in Section 5.6.1 but because of the structure of Markov Chains, there are some additional theoretical results that could be explored.

- statistical analysis - As noted in the implementation discussion, the KSL automatically use batch statistics when reporting the MCMC results. There are other procedures, such as standardized time series and others that may be of interest for MCMC.
- design of efficient samplers - As the example noted, the design of samplers can be challenging work. We did not discuss the Gibbs sampler and I encourage the interested reader to review the suggested references for more details and other samplers.

9.4. Summary

This chapter presented advanced Monte Carlo techniques: bootstrap methods for estimating the sampling distribution of an estimator, variance reduction techniques for improving the precision of simulation estimates, methods for generating multi-variate and correlated random variates, and Markov Chain Monte Carlo methods. The KSL provides classes within the `ksl.utilities` packages that facilitate each of these techniques. For variance reduction, you should also review random number streams, their creation, and their usage (Section 2.1).

The following new KSL constructs were discussed in this chapter:

`Bootstrap`: A class that performs bootstrap sampling on a set of data in order to estimate the sampling distribution of a statistic and to form bootstrap confidence intervals.

`BootstrapSampler`: A class that repeatedly samples, with replacement, from the original data and uses an estimator to compute one or more bootstrap statistics from each sample.

`BSEstimatorIfc`: An interface that defines how a bootstrap estimate is computed from a sample of data.

`CaseBootstrapSampler`: A class that performs case based bootstrap sampling, in which whole cases (rows) of data are resampled.

`ControlVariateDataCollector`: A class that collects the replication values of responses and controls for the purpose of applying the method of control variates.

`MCMultiVariateIntegration`: A class that estimates the integral of a multi-dimensional function via Monte Carlo sampling.

`BivariateNormalRV`: A random variable that generates pairs of values from a bivariate normal distribution.

`MVGaussianCopulaRV`: A multi-variate random variable that uses a Gaussian copula to generate correlated values with specified marginal distributions.

`AR1CorrelatedRNStream`: A random number stream that generates autocorrelated uniform variates.

`DMarkovChain`: A class that randomly generates the states of a discrete Markov Chain from its transition probabilities.

`MetropolisHastingsMV`: A class that implements the multi-variate Metropolis-Hastings algorithm for generating samples from a target distribution.

`ProposalFunctionMVIfc`: An interface that defines the proposal function used within the Metropolis-Hastings algorithm.

`FunctionMVIfc`: An interface that defines a multi-variate function, such as the target density used within the Metropolis-Hastings algorithm.

As you can see, the KSL provides many immediately useful implementations for advanced Monte Carlo work and a library structure that facilitates the development of additional implementations. The next chapter turns from generating and analyzing random samples to using simulation models to analyze scenarios and to perform designed experiments.

9.5. Exercises

Exercise 9.1. Consider Exercise 9.11. Develop a bootstrap confidence interval for the median profit. Report your confidence interval at the 0.99 level. The profits of Exercise 9.11 come in antithetic pairs, which are not independent. Resample the pairs rather than the individual profits.

Exercise 9.2. Consider Exercise 9.11. Develop a bootstrap confidence interval for the minimum profit. Report your confidence interval at the 0.99 level. The profits of Exercise 9.11 come in antithetic pairs, which are not independent. Resample the pairs rather than the individual profits.

Exercise 9.3. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{8}{9}\left(\frac{5x}{16} + \frac{3x^3}{32} + \frac{5x^4}{256}\right) & 0 \leq x \leq 2 \\ 0 & \text{otherwise} \end{cases}$$

In all cases, implement and test your algorithms.

a) Develop an algorithm to generate random variables from the probability density function that uses both the inverse transform method and the composition method (together).

b) Develop an algorithm to generate random variables from the probability density function that uses acceptance rejection.

Exercise 9.4. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{2x}{25} & 0 \leq x \leq 5 \\ 0 & \text{otherwise} \end{cases}$$

Write a KSL program to generate observations from this distribution using MCMC.

Exercise 9.5. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{3x^2}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

Write a KSL program to generate observations from this distribution using MCMC.

Exercise 9.6. Use the Monte Carlo method and antithetic variates to estimate the following integral with 95% confidence to within ± 0.01.

$$\int_1^4 \left(\sqrt{x} + \frac{1}{2\sqrt{x}}\right) \mathrm{d}x$$

Exercise 9.7. Use the Monte Carlo method and antithetic variates to estimate the following integral with 99% confidence to within ±0.01.

$$\theta = \int_0^1 \int_0^1 (4x^2y + y^2)\,\mathrm{d}x\mathrm{d}y$$

Exercise 9.8. Use the Monte Carlo method and importance sampling to estimate the following integral with 95% confidence to within ±0.01.

$$\int_1^4 \left(\sqrt{x} + \frac{1}{2\sqrt{x}}\right)\mathrm{d}x$$

Exercise 9.9. Use the Monte Carlo method and importance sampling to estimate the following integral with 99% confidence to within ±0.01.

$$\theta = \int_0^1 \int_0^1 (4x^2y + y^2)\,\mathrm{d}x\mathrm{d}y$$

Exercise 9.10. A firm is trying to decide whether or not to invest in two proposals A and B that have the net cash flows shown in the following table, where $N(\mu, \sigma)$ represents that the cash flow value comes from a normal distribution with the provided mean and standard deviation.

End of Year	0	1	2	3	4
A	$N(-250, 10)$	$N(75, 10)$	$N(75, 10)$	$N(175, 20)$	$N(150, 40)$
B	$N(-250, 5)$	$N(150, 10)$	$N(150, 10)$	$N(75, 20)$	$N(75, 30)$

The interest rate has been varying recently and the firm is unsure of the rate for performing the analysis. To be safe, they have decided that the interest rate should be modeled as a beta random variable over the range from 2 to 7 percent with $\alpha_1 = 4.0$ and $\alpha_2 = 1.2$. Given all the uncertain elements in the situation, they have decided to perform a simulation analysis in order to assess the situation. Use the simulation to answer the following questions. Use stream 1 for the interest rate and streams 2 to 6 for the cash flows of years 0 to 4. Base your analysis on 1,000 samples.

Compare the expected present worth of the two alternatives using common random numbers. Estimate the probability that alternative A has a higher present worth than alternative B.

Determine the number of samples needed to be 95% confidence that you have estimated the $P[PW(A) > PW(B)]$ to within $\pm$ 0.10.

Exercise 9.11. A firm produces YBox gaming stations for the consumer market. Their profit function is:

$$\text{Profit} = (\text{unit price} - \text{unit cost}) \times (\text{quantity sold}) - \text{fixed costs}$$

Suppose that the unit price is \$200 per gaming station, and that the other variables have the following probability distributions:

Unit Cost	80	90	100	110
Probability	0.20	0.40	0.30	0.10
Quantity Sold	1000	2000	3000	
Probability	0.10	0.60	0.30	

The fixed cost has a triangular distribution with a minimum of $150,000, a mode of $200,000, and a maximum of $250,000.

Use a simulation model with 1,000 antithetic pairs to estimate the mean profit from your sample and compute a 95% confidence interval for the mean profit. Use stream 1 for the unit cost, stream 2 for the quantity sold, and stream 3 for the fixed cost. Estimate the probability that the profit will be positive.

Exercise 9.12. YBox video game players arrive according to a Poisson process with a rate of 8 per hour (stream 1) to a two-person station for testing. The inspection time per YBox set is EXPO(10) minutes (stream 2). On the average 82% of the sets pass inspection (stream 4). The remaining 18% are routed to an adjustment station with a single operator. Adjustment time per YBox is UNIF(7,14) minutes (stream 3). After adjustments are made, the units are routed back to the inspection station to be retested. Build a simulation model of this system. Use 30 replications of 30,000 minutes with a warm up period of 3,000 minutes.

Use control variates to estimate the total time a set spends in the system.

Exercise 9.13. Complete Exercise 5.7.

Exercise 9.14. Complete Exercise 5.8.

Exercise 9.15. Complete Exercise 5.9.

Exercise 9.16. Complete Exercise 5.10.

Exercise 9.17. Let X be a random variable with a lognormal distribution with mean 2.0, and variance 4.0. Let Y be a random variable with a Weibull distribution with shape 5.0 and scale 10.0. We are interested in generating the joint distribution of (X, Y) when X and Y have dependence specified with the following copulas:

a) Gausian copula with correlation 0.8. Generate 1000 pairs of (X, Y). Measure the Pearson, Kendall, and Spearman correlation between X and Y. Make a 2-D plot of (X, Y).

b) Clayton copula with θ, where U is the first coordinate and V is the second coordinate. Generate 1000 pairs of (U, V) from the copula. Measure the Pearson, Kendall, and Spearman correlation between U and V when $\theta = 2$. Generate 1000 pairs of (X, Y) using the copula with mapping U to X and V to Y. Measure the Pearson, Kendall, and Spearman correlation between X and Y when $\theta = 2$. Make a 2-D plot of (X, Y).

c) Frank copula with θ, where U is the first coordinate and V is the second coordinate. Generate 1000 pairs of (U, V) from the copula. Measure the Pearson, Kendall, and Spearman correlation between U and V when $\theta = 2$. Generate 1000 pairs of (X, Y) using the copula with mapping U to X and V to Y. Measure the Pearson, Kendall, and Spearman correlation between X and Y when $\theta = 2$. Make a 2-D plot of (X, Y).

Exercise 9.18. Suppose that the random variable X_1 is the zinc content of an ore sample with a range of values in $[0.5, 1.5]$ and the random variable X_2 is the iron content of the ore with values in $[20.0, 35.0]$. The joint distribution density function for the random variables has been modeled as:

$$f(x_1, x_2) = \begin{cases} \frac{39}{400} - \frac{17(x_1-1)^2}{50} - \frac{(x_2-25)^2}{10000} & 0.5 \leq x_1 \leq 1.5;\ 20 \leq x_2 \leq 35 \\ 0.0 & \textit{otherwise} \end{cases}$$

a) Derive the marginal distributions for X_1 and X_2. Using the marginal distributions compute $E[X_1]$, $\text{Var}[X_1]$, $E[X_2]$, $\text{Var}[X_2]$.

b) Derive and compute $\text{cov}(X_1, X_2)$ and $\text{corr}(X_1, X_2)$.

c) Derive the conditional distributions for $X_1|X_2$ and $X_2|X_1$.

Exercise 9.19. Suppose that the random variable X_1 is the zinc content of an ore sample with a range of values in $[0.5, 1.5]$ and the random variable X_2 is the iron content of the ore with values in $[20.0, 35.0]$. The joint

distribution density function for the random variables has been modeled as:

$$f(x_1, x_2) = \begin{cases} \frac{39}{400} - \frac{17(x_1-1)^2}{50} - \frac{(x_2-25)^2}{10000} & 0.5 \leq x_1 \leq 1.5;\ 20 \leq x_2 \leq 35 \\ 0.0 & \textit{otherwise} \end{cases}$$

Design an independence sampler that is better than the sampler in the example from Section 9.3.4.

a) Estimate the correlation of (X_1, X_2) from the example's implementation and for your sampler. Compare the estimated quantity to the theoretical value.

b) Report the acceptance probability estimate for your sampler.

c) Make a time series plot of the example's x_1 and x_2 series. Make an ACF plot for the time series.

d) Make a time series plot of your independence sampler x_1 and x_2 series. Make an ACF plot for the time series.

e) Compare the means and variances of X_1 and X_2 estimated from the example's implementation and from your sampler with the values derived in Exercise 9.18. Explain why the example's sampler mixes poorly and why your proposal does better.

Exercise 9.20. Reconsider the zinc and iron content example of Section 9.3.4. Develop a random walk sampler whose proposal for each dimension is a truncated normal distribution, $Y_i \sim N_{[a_i,b_i]}(\mu = x_i, \sigma_i^2)$, where $[a_i, b_i]$ is the range of X_i and σ_i is a standard deviation that you choose. The KSL `TruncatedNormal` class provides the density of this distribution.

a) The truncated normal proposal is not symmetric near the boundaries. Derive the proposal ratio $p_r(\vec{x}, \vec{y})$ for this sampler. *Hint:* the density of $N_{[a_i,b_i]}(\mu = x_i, \sigma_i^2)$ divides by $\Phi((b_i - x_i)/\sigma_i) - \Phi((a_i - x_i)/\sigma_i)$, where $\Phi(\cdot)$ is the standard normal CDF, and this normalizing constant depends on the current state.

b) Implement the sampler. Estimate the means and variances of X_1 and X_2, and compare them with the values derived in Exercise 9.18.

c) Try at least two values of each σ_i and report the acceptance probability for each. How does the choice of σ_i affect the acceptance probability and the mixing of the chain?

d) Make a time series plot and an ACF plot of your sampler's x_1 and x_2 series, and compare them with those of the random walk sampler in the example.

Exercise 9.21. Suppose that the random variable X_1 is the zinc content of an ore sample with a range of values in $[0.5, 1.5]$ and the random variable X_2 is the iron content of the ore with values in $[20.0, 35.0]$. The joint distribution density function for the random variables has been modeled as:

$$f(x_1, x_2) = \begin{cases} \frac{39}{400} - \frac{17(x_1-1)^2}{50} - \frac{(x_2-25)^2}{10000} & 0.5 \leq x_1 \leq 1.5;\ 20 \leq x_2 \leq 35 \\ 0.0 & \textit{otherwise} \end{cases}$$

Consider the following algorithm for generating a random vector $(X_1, X_2, \cdots, X_d)$ from a d-dimensional distribution $F_{\vec{X}}(\vec{x})$. Let $F_i(x_i|x_1, x_2, \cdots, x_{i-1})$ be the conditional distribution of X_i given that $X_j = x_j$ for $j = 1, 2, \cdots, x_{i-1}$. Let $F_{X_i}(x_i)$ be the marginal distribution for X_i, for $i = 1, 2, \cdots, d$. Then a algorithm for generating $\vec{X}$ with distribution $F_{\vec{X}}(\vec{x})$ is:

1. Generate X_1 with distribution $F_{X_1}(\cdot)$.
2. Generate X_2 with distribution $F_2(\cdot|X_1)$.
3. Generate X_3 with distribution $F_3(\cdot|X_1, X_2)$.
4. etc.

d. Generate X_d with distribution $F_d(\cdot|X_1, X_2, \cdots, X_{d-1})$ and return $\vec{X} = (X_1, X_2, \cdots, X_d)$.

a) Prove that the algorithm will result in $\vec{X} \sim F_{\vec{X}}(\vec{x})$ for the 2-dimensional case of (X_1, X_2).

b) Apply the algorithm to generate zinc and iron content, (X_1, X_2), from $f(x_1, x_2)$.

c) Estimate the correlation of (X_1, X_2) from your implementation. Compare the estimated quantity to the theoretical value.

d) Provide a statistical analysis of the marginal samples when compared to the theoretical marginal distributions.

Exercise 9.22. Consider the following joint density of two random variables x and y given by:

$$f(x, y) = \begin{cases} \frac{6}{5}(x + y^2) & 0 \leq x \leq 1; 0 \leq y \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

a) Develop an algorithm to generate from this joint probability distribution.

b) Illustrate a histogram of the marginal distributions using your algorithm

c) Estimate the correlation between x and y using the data from your algorithm

---

Exercise 9.23. Suppose that the random variable X_1 is the zinc content of an ore sample with a range of values in $[0.5, 1.5]$ and the random variable X_2 is the iron content of the ore with values in $[20.0, 35.0]$. The joint distribution density function for the random variables has been modeled as:

$$f(x_1, x_2) = \begin{cases} \frac{39}{400} - \frac{17(x_1-1)^2}{50} - \frac{(x_2-25)^2}{10000} & 0.5 \leq x_1 \leq 1.5;\ 20 \leq x_2 \leq 35 \\ 0.0 & otherwise \end{cases}$$

Develop a Gibbs sampler for generating from this distribution.

a) Estimate the correlation of (X_1, X_2) for your sampler. Compare the estimated quantity to the theoretical value.

b) Report the acceptance probability estimate for your sampler.

c) Make a time series plot of your Gibbs sampler x_1 and x_2 series. Make an ACF plot for the time series.

Exercise 9.24. Use Latin hyper-cube sampling to estimate the following integral with 99% confidence to within ± 0.01.

$$\theta = \int_0^1 \int_0^1 (4x^2 y + y^2)\, \mathrm{d}x\mathrm{d}y$$

Compare and contrast this method to the method you used in Exercise 9.7 or Exercise 9.9.

---

10. Simulating Many Scenarios and Performing Experiments

Learning Objectives

- To be able to explain how a KSL model can be instrumented for analysis.
- To be able to instrument a KSL model for scenario analysis and experimentation.
- To be able to setup and execute a set of model scenarios.
- To be able to explain and discuss the application of experimental design methods within a simulation modeling context.
- To be able to recognize how simulation experiments are different from standard application of experimental methods.
- To be able to apply experimental design methods within the KSL for simulation analysis.

This chapter builds on the methods presented in Chapter 5. Section 5.7 introduced how simulation models can be used to compare different system configurations. An important aspect of the comparison process was to select the best simulation configuration (or to screen out inferior configurations). Section 10.1 of this chapter illustrates how to simulate many configurations, while Section 10.2 discusses how to setup experiments. Both of these situations beg one particular question. *How to specify the configurations to be analyzed?*. To specify a configuration, we must instrument the model so that the parameters of interest can be controlled.

A simulation model has many parameters that can be varied. Each parameter setting represents a different system configuration. All possible parameter settings represent the *design* space of the problem. This chapter presents concepts for how to configure scenarios to explore the design space of a problem and the need for the use of systematic experimental design methods for this process. We explore the design space to make inferences about how the real system will behave under different parameter settings (i.e. for different design configurations).

The first part of this chapter discusses the underlying architecture provided by the KSL to instrument models. This architecture will be used to setup scenarios that analyze a simple situation in order to illustrate that the same process can be used for larger models.

Section 10.2 of this chapter presents methods that allow the analyst to systematically explore the design space using experimental design methods. Using experimental design methods allows an efficient allocation of simulation runs for the purpose of understanding the relationships between the inputs and outputs of the model. Within this context, the analyst must explicitly specify the framework for setting the parameters to obtain the desired analysis. That is, the analyst must specify an experimental design.

> **Note**
>
> This chapter provides example code of using the KSL to implement scenarios and experimental designs. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

10.1. Simulating Many Scenarios

As noted in the introduction, there is often a need to simulate many variations of the same or different models and capture the results for further analysis. This section provides an overview of the KSL constructs for running many scenarios and for instrumenting a model for experiments. The primary purpose of this section is to explain the built in constructs and illustrate their use within simple examples. In most realistic situations, the models will have more complex input and output mapping than illustrated in this section. The illustrated constructs can be readily scaled up to larger models and more complex situations; however, this section only presents basic use cases. We start by understanding controls and how to manage the parameters of random variables.

10.1.1. Control Annotations

Because simulation models may have many types of inputs variables that may need to be changed by the modeler, the KSL defines an access protocol based on Kotlin property variables that have been annotated to define them as a controllable variable. The annotations provide meta-data about the element that can be used to define an approach based on reflection for setting the values of the properties prior to running a simulation model. This functionality is implemented in the `ksl.controls` package.

The `KSLControl` annotation can be used on the setter method of properties within model elements to indicate that those properties can be used to control the execution of the simulation model. The annotation field type must be supplied and must be one of the valid control types as specified by the enum `ControlType.` The user is responsible for making sure that the type field matches (or is consistent with) the type of the property. Even though the optional annotation fields (`lowerBound` and `upperBound`) are specified as double values, they will be converted to an appropriate value for the specified type. Boolean properties are represented as a 1.0 (true)

and 0.0 (false) within the numerical conversion for the controls. If a control is BOOLEAN, then the user can supply a 1.0 to represent true and a 0.0 to represent false when setting the control, which will then be set to true or false, appropriately. Current control types include (Double, Int, Long, Short, Byte, Float) and Boolean. In essence, the numeric types are all represented as a Double with appropriate conversions occurring when the property is assigned.

```
@MustBeDocumented // flag inclusion in documentation
@Inherited // flag that it is inherited by subclasses
@Target( AnnotationTarget.PROPERTY_SETTER) // targets setters
 ↪  ONLY
annotation class KSLControl(
    /**
     * the type of the control
     */
    val controlType: ControlType,

    /**
     * the name of the control
     */
    val name: String = "",

    /** If this field is not specified, it will be translated
     ↪  to the smallest
     * negative value associated with the type specified by
 ↪  the field type.
     * For example, if the type is INTEGER then the default
 ↪  lower bound will
     * be Integer.MIN_VALUE. The user can supply more
 ↪  constraining values for
     * the bounds that are within the range of the associated
 ↪  type.
     */
```

```
    val lowerBound: Double = Double.NEGATIVE_INFINITY,

    /** If this field is not specified, it will be translated
     ↪  to the largest
     * positive value associated with the type specified by
↪  the field type.
     * For example, if the type is INTEGER then the default
↪  lower bound will
     * be Integer.MAX_VALUE. The user can supply more
↪  constraining values for
     * the bounds that are within the range of the associated
↪  type.
     */
    val upperBound: Double = Double.POSITIVE_INFINITY,

    /**
     *
     * comment associated with the annotation
     */
    val comment: String = "",

    /** Indicates whether to include in the controls for the
     ↪  model or not. Provides
     * a simple mechanism for turning off or not including
↪  specific controls.
     * The default is to include the control in the
↪  extraction process.
     */
    val include: Boolean = true
)
```

The `KSLControl` annotation becomes more useful if you provide carefully considered values for its parameters. The annotation `KSLControl` has been

defined with properties `controlType`, `name`, `lowerBound`, `upperBound`, `comment`, and `include`. The most important properties are `controlType` and the bounds. By default the bounds are positive and negative infinity.

The `PalletWorkCenter` model that was illustrated in Chapter 5 has an annotated property. In the following code, we see the Kotlin annotation syntax `@set:KSLControl` to annotate the `numWorkers` property for the `PalletWorkCenter` model.

```
    @set:KSLControl(
        controlType = ControlType.INTEGER,
        lowerBound = 1.0
    )
    var numWorkers = numWorkers
        set(value) {
            require(value >= 1) { "The number of workers must
↪   be >= 1" }
            require(!model.isRunning) { "Cannot change the
↪   number of workers while the model is running!" }
            field = value
        }
```

In the example code for the `numWorker` property, the lower bound has been specified as 1.0 to indicate that the value of this property cannot be less than 1.0. The control type of `ControlType.INTEGER` ensures that even though the control's value is specified as a `Double`, the supplied value will be converted to an `Int` when it is applied. Thus, this approach simplifies the specification of parameter values.

Model elements that are part of the KSL framework may already have properties that have been annotated as a `KSLControl`. For example, the initial value property for a `Variable` has been annotated as follows.

```
@set:KSLControl(
    controlType = ControlType.DOUBLE
)
override var initialValue: Double = theInitialValue
    set(value) {
        require(domain.contains(value)) { "The initial value,
 ↪  $value must be within the specified range for the
 ↪  variable: $domain" }
        if (model.isRunning) {
            Model.logger.info { "The user set the initial
 ↪  value during the replication. The next replication will
 ↪  use a different initial value" }
        }
        field = value
    }
```

Thus, when you use provided KSL model elements, you should note that you may already have pre-defined controls.

The `KSLControl` annotation covers numeric properties (Double, Int, Short, Byte, and Float), which are all converted to a Double when utilized within KSL constructs. It also covers the case of a Boolean control by mapping true to 1.0 and false to 0.0. A wide-range of situations can be handled simply by mapping numbers over to cases. For example, by using an integer control, we can condition based on the value of the integer and invoke different decision paths. Thus, properties annotated with `KSLControl` can be used to do more than just set the value of a variable.

For categorical variables, which do not naturally have an ordering, it can be useful to use the `StringControl` annotation.

```
@MustBeDocumented
@Inherited
```

```
@Target(AnnotationTarget.PROPERTY_SETTER)
annotation class KSLStringControl(
    val allowedValues: Array<String> = [],
    val name:          String        = "",
    val comment:       String        = "",
    val include:       Boolean       = true,
)
```

The annotation can be specified with a list of values that are permitted. Finally, there is the possibility of specifying properties as general JSON controls. JSON (JavaScript Object Notation) is a lightweight, text-based data format used to store and exchange information.

```
@MustBeDocumented
@Inherited
@Target(AnnotationTarget.PROPERTY_SETTER)
annotation class KSLJsonControl(
    val expectedTypeHint: String  = "",
    val name:             String  = "",
    val comment:          String  = "",
    val include:          Boolean = true,
)
```

JSON controls permit you to specify elements such as lists, arrays, maps as controls. The `ksl.controls` package guide provides additional details on how to specify string and JSON controls.

Annotations indicate in a generic manner which properties can be controllable. When you develop your own model elements, you can take advantage of the generic access protocol and build interesting ways to set the controls of your models. In what follows, we will illustrate how to use controls.

The `Model` class has a `controls()` function that returns all of the controls associated with every model element within a model. The following code

illustrates how to access the controls for a model and print out their key values.

Example 10.1 (Illustrating a Control)**.** The following example code illustrates how to access a control and change its value, resulting in the change of the associated property.

```
fun main() {
    val model = Model("Pallet Model MCB")
    // add the model element to the main model
    val palletWorkCenter = PalletWorkCenter(model, name
     ↪  ="PWC")
    println("Original value of property:")
    println("num workers = ${palletWorkCenter.numWorkers}")
    println()
    val controls =  model.controls()
    println("Control keys:")
    for (controlKey in controls.controlKeys()) {
        println(controlKey)
    }
    // find the control with the desired key
    val control = controls.control("PWC.numWorkers")!!
    // set the value of the control
    control.value = 3.0
    println()
    println("Current value of property:")
    println("num workers = ${palletWorkCenter.numWorkers}")
}
```

This results in the following output.

```
Original value of property:
num workers = 2
```

```
Control keys:
PWC.numWorkers
NumBusyWorkers.initialValue
PalletQ:NumInQ.initialValue
Num Pallets at WC.initialValue
Num Processed.initialCounterLimit
Num Processed.initialValue
P{total time > 480 minutes}.initialValue

Current value of property:
num workers = 3
```

The important aspect of this output to notice are the keys associated with the controls. The `numWorkers` property associated with the `PalletWorkCenter` class has the key `PWC.numWorkers`. The `PWC` portion of the key has been derived from the assigned name of the created instance of the `PalletWorkCenter` class. The name of the model element is unique and the Kotlin property name `numWorkers` must be unique. Thus, this combination is unique and can be used to look up the control associated with the annotated property.

The second aspect of this example is the fact that prior to setting the control, the underlying `numWorkers` property had a value of 2.0. Then, after accessing the control and changing its value, the `numWorkers` property value was updated to 3.0. Now, this is a bit of overkill for changing the value of a single property; however, this illustrates the basic mechanics of using controls. Because the key for a control is unique, as long as we know the associated key, we can change the value of the associated property via its associated control. A more useful use case would be to store the key-value pairs in a file and read in the new values of the properties from the file. Then, a generic procedure can be written to change all the controls. This is exactly the approach taken by many of the associated KSL applications.

10.1.2. Random Variable Parameters

Stochastic simulation models use random variables. Thus, it is common to need to change the values of the parameters associated with the random variables when running experiments. The architecture of the KSL with respect to random variables can be somewhat daunting. The key issue is that changing out the random variable or its parameters requires careful methods because random variables are immutable and changing their values must ensure that replications start with same settings (so that they are truly replicates). The second complicating factor is that the number of parameters associated with random variables varies widely by type of random variable. For example, an exponential random variable has one parameter, its mean, while a triangular random variable has three parameters (min, mode, max). Thus, the KSL provides a generic protocol for accessing and changing the parameter values of every random variable associated with a KSL model. This is accomplished via the `RVParameterSetter` class and its associated supporting classes within the `ksl.rvariable.parameters` package. This section will provide a brief overview of how to utilize this generic protocol for changing the parameters associated with the random variables of a KSL model.

Example 10.2 (Changing Random Variable Parameters)**.** The following example code illustrates how to access the parameters of a random variable and how to apply the change within the model.

```
fun main() {
    val model = Model("Pallet Model MCB")
    // add the model element to the main model
    val palletWorkCenter = PalletWorkCenter(model, name
     ↪  ="PWC")
    println(palletWorkCenter.processingTimeRV)
    val tmpSetter = RVParameterSetter(model)
    val map = tmpSetter.rvParameters
```

```
    println()
    println("Standard Map Representation:")
    for ((key, value) in map) {
        println("$key -> $value")
    }
    val rv  = map["ProcessingTimeRV"]!!
    rv.changeDoubleParameter("mode", 14.0)
    tmpSetter.applyParameterChanges(model)
    println()
    println(palletWorkCenter.processingTimeRV)
}
```

Similar to how controls function, the parameters associated with a model can be retrieved by creating an instance of the `RVParameterSetter` class. The `RVParameterSetter` class has many representations of the parameters associated with random variables within the model. An instance of the `RVParameterSetter` class holds a map that uses the name of the random variable from the model as the key and returns an instance of the `RVParameters` class. The `RVParameters` class holds information about the parameters of the random variable based on their types (Double, Int, DoubleArray). Currently the `RVParameterSetter` class only holds random variables that have integer or double type parameters.

In the code example, first the string representation of the random variable is printed. We can see from the output that its name is `ProcessingTimeRV` and its underlying source of randomness is a triangular distributed random variable with minimum 8, mode 12, and maximum 15, using stream 3. The returned map of random variable parameters is used to get the `RVParameters` associated with the `ProcessingTimeRV` and then the mode parameter is changed to the value of 14.

```
ModelElement{Class Name=RandomVariable, Id=4, Name='ProcessingTimeRV', Parent Name='
Initial random Source: TriangularRV(min=8.0, mode=12.0, max=15.0) with stream 3
```

```
Standard Map Representation:
Pallet_Model_MCB:DefaultUniformRV -> RV Type = Uniform
Double Parameters {min=0.0, max=1.0}
ProcessingTimeRV -> RV Type = Triangular
Double Parameters {min=8.0, mode=12.0, max=15.0}
TransportTimeRV -> RV Type = Exponential
Double Parameters {mean=5.0}
NumPalletsRV -> RV Type = Binomial
Double Parameters {probOfSuccess=0.8, numTrials=100.0}

ModelElement{Class Name=RandomVariable, Id=4, Name='ProcessingTimeRV', Parent Name='
Initial random Source: TriangularRV(min=8.0, mode=14.0, max=15.0) with stream 3
```

Then the code, applies the parameter change to the model. It is important to note that changing parameter value in the map **does not** change the parameter in the model. The change is only applied to the model when the `applyParameterChanges()` function is invoked. As we can see from the printed output for the associated random variable, the mode has been updated to 14. As was the case for controls, changing individual parameter values via this generic protocol is a bit of overkill for a single parameter. However, this protocol defines a general approach that will work as long as we know the name of the random variable within the model and the name of the parameter to change. The code prints out the map relationship from which we can easily make note of the associated names of the random variables and the names of their defined parameters.

The `RVParameterSetter` class also has a flattened structure for holding parameter values similar to how controls work. The name of the random variable is concatenated with its associated parameter name to form a unique key. The following code illustrates this representation.

```
    val flatMap = tmpSetter.flatParametersAsDoubles
    println("Flat Map Representation:")
    for ((key, value) in flatMap) {
        println("$key -> $value")
    }
```

The resulting print out of the map is show here.

```
Flat Map Representation:
Pallet_Model_MCB:DefaultUniformRV.min -> 0.0
Pallet_Model_MCB:DefaultUniformRV.max -> 1.0
ProcessingTimeRV.min -> 8.0
ProcessingTimeRV.mode -> 14.0
ProcessingTimeRV.max -> 15.0
TransportTimeRV.mean -> 5.0
NumPalletsRV.probOfSuccess -> 0.8
NumPalletsRV.numTrials -> 100.0
```

Notice that the name of the processing time random variable `ProcessingTimeRV` has been concatenated with its parameter names `min`, `mode`, and `max`. This representation is useful for storing controls and random variable parameters within the same file or data structure for generic processing.

10.1.3. Building Models with `ModelBuilderIfc`

Controls allow a model to be instrumented. In previous examples, the approach taken for constructing a model instance followed the same pattern:

- create the model instance
- create model elements for the model

- configure the model and model elements

This is illustrated in the following code that creates the drive through pharmacy model.

```
fun main(){
    //create the model instance
    val m = Model()
    // create model elements and attach them to the model
    val dtp = DriveThroughPharmacy(m, name = "DriveThrough")
    // configure model elements
    dtp.arrivalRV.initialRandomSource = ExponentialRV(6.0, 1)
    dtp.serviceRV.initialRandomSource = ExponentialRV(3.0, 2)
    // configure model run parameters
    m.numberOfReplications = 30
    m.lengthOfReplication = 20000.0
    m.lengthOfReplicationWarmUp = 5000.0
    // simulate
    m.simulate()
    // get results
    m.print()
}
```

To be able to utilize models in a more general manner (e.g. in experiments or within related KSL applications), we need to generalize the model building process. This is the purpose of the `ModelBuilderIfc` interface:

```
interface ModelBuilderIfc {

    fun build(
        modelConfiguration: Map<String, String>? = null,
        experimentRunParameters: ExperimentRunParametersIfc?
         ↪  = null
```

```
    ): Model

}
```

The `ModelBuilderIfc` interface provides a contract that defines an abstract method that is responsible for constructing `Model` object instances. The KSL framework may call this `build` function many times. Thus, it is important that the implementation be kept as simple as possible and as close to the aforementioned pattern as possible. Refer to the documentation for further details. An example implementation for the drive through pharmacy model example is shown in the following code:

```
object DriveThroughPharmacyWithQModelBuilder :
 ↪  ModelBuilderIfc {
    override fun build(
        modelConfiguration: Map<String, String>?,
        experimentRunParameters: ExperimentRunParametersIfc?
    ): Model {
        val model = Model("DriveThroughPharmacyWithQ",
         ↪  autoCSVReports = false)
        val sim = DriveThroughPharmacyWithQ(model, numServers
         ↪  = 1, name = "Pharmacy")
        model.numberOfReplications = 30
        model.lengthOfReplication = 20000.0
        model.lengthOfReplicationWarmUp = 5000.0
        return model
    }
}
```

The optional `modelConfiguration` parameter is available if there is some situation that cannot be handled via the controls framework to allow a model to be configured. The key of the map can be used to understand

how the associated (mapped) value string should be interpreted. Think of this parameter as a generalization of the `main(args: Array<String>)` function that is commonly used to supply arguments to a program. The second optional parameter, `experimentRunParameters` can be used if you want to allow different run parameters during the build, rather than essentially hard-coding the defaults as in the example code. Keep in mind that it is best practice to provide appropriate default parameters. If you do not, the number of replications will be 1. The length of the replication will be infinity and the warm up period will be 0.0. If you do not specify a finite value for the length of the replication, then you are responsible for ensuring that condition within the model will stop the replication; otherwise, your program will never end.

Providing a builder for your models is essential for taking advantage of KSL based applications. The next section will illustrate how to utilize models and controls to run scenarios.

10.1.4. Setting Up and Running Multiple Scenarios

This section puts the concepts presented in the last two sections into practice by illustrating how to construct scenarios and how to execute the scenarios.

The code found in Example 5.7 has a very common pattern.

- create a model and its elements
- set up the model
- simulate the model
- create another model or change the previous model's parameters
- simulate the model
- repeat for each experimental configuration

Within the KSL these concepts have been generalized by providing the `Scenario` class and the `ScenarioRunner` class. A scenario represents the

specification of a model to run, with some inputs. Each scenario will produce a simulation run (`SimulationRun`). The naming of a scenario is important. The name of the scenario should be unique within the context of running multiple scenarios with a `ScenarioRunner` instance. The name of the scenario is used to assign the name of the model's experiment. Thus, if the scenario names are unique, then the experiment names will be unique. In the context of running multiple scenarios, it is important that the experiment names be unique to permit automated storage within the associated KSL database.

The following code presents the constructor of a scenario. Notice that the key parameters of the constructor are a model *builder*, a map of inputs, and the name of the scenario. The input map represents (key, value) pairs specifying controls or random variable names (in flat map form) along with its assigned value.

```
class Scenario constructor(
    val modelBuilder: ModelBuilderIfc,
    name: String,
    inputs: Map<String, Double> = emptyMap(),
    stringInputs: Map<String, String> = emptyMap(),
    jsonInputs: Map<String, String> = emptyMap(),
    runParameters: ExperimentRunParameters,
    modelConfiguration: Map<String, String>? = null,
    val modelConstructionMode: ScenarioModelConstructionMode
     ↪  = ScenarioModelConstructionMode.MODEL_BUILDER
) : Identity(name) {
...
```

I encourage you to review the `Scenario` class's documentation for convenience constructors.

Once a scenario is defined, a list of scenarios can be provided to the `ScenarioRunner` class. The purpose of the `ScenarioRunner` class is to

run all the defined scenarios as a batch and the collection of the simulation results from the scenarios into a `KSLDatabase` instance. Let's take a look at how to use the `ScenarioRunner` class.

Example 10.3 (Using a ScenarioRunner)**.** In this code example, we see that an instance of the `ScenarioRunner` class can be used to run many scenarios. The loop after running the scenarios is simply for the purpose of displaying the results.

```
fun main() {
    val scenarioRunner = ScenarioRunner("Example5_10",
     ↪ buildScenarios())
    scenarioRunner.simulate()
    for (s in scenarioRunner.scenarioList) {
        val sr = s.simulationRun?.statisticalReporter()
        val r = sr?.halfWidthSummaryReport(title = s.name)
        println(r)
        println()
    }
}
```

The `buildScenarios()` function is simply a function that returns a list of scenarios. This is where the real magic happens. The first thing to note about the following code is that scenarios can be defined on the same or different models. It makes no difference to the `ScenarioRunner` class whether the scenarios are related to one or more models. The `ScenarioRunner` will simply execute all the scenarios that it is supplied. In the following code, the `PalletWorkCenter` model is again used in the illustration as one of the scenario models. Here, maps of inputs using the controls and random variable parameters, are used to configure the inputs to three different scenarios. For illustrative purposes only, a fourth scenario is created and configured using the drive through pharmacy model.

```
fun buildScenarios() : List<Scenario> {
    val model = Model("Pallet Model", autoCSVReports = true)
    // add the model element to the main model
    val palletWorkCenter = PalletWorkCenter(model, name =
     ↪  "PWC")
    // set up the model
    model.resetStartStreamOption = true
    model.numberOfReplications = 30

    val sim1Inputs = mapOf(
        "ProcessingTimeRV.mode" to 14.0,
        "PWC.numWorkers" to 1.0,
        )

    val sim2Inputs = mapOf(
        "ProcessingTimeRV.mode" to 14.0,
        "PWC.numWorkers" to 2.0,
    )

    val sim3Inputs = mapOf(
        "ProcessingTimeRV.mode" to 14.0,
        "PWC.numWorkers" to 3.0,
    )

    val dtpModel = Model("DTP Model", autoCSVReports = true)
    dtpModel.numberOfReplications = 30
    dtpModel.lengthOfReplication = 20000.0
    dtpModel.lengthOfReplicationWarmUp = 5000.0
    val dtp = DriveThroughPharmacyWithQ(dtpModel, name =
     ↪  "DTP")
    val sim4Inputs = mapOf(
        "DTP.numPharmacists" to 2.0,
    )
```

```
    val s1 = Scenario(model = model, inputs = sim1Inputs,
     ↪  name = "One Worker")
    val s2 = Scenario(model = model, inputs = sim2Inputs,
     ↪  name = "Two Worker")
    val s3 = Scenario(model = model, inputs = sim3Inputs,
     ↪  name = "Three Worker")
    val s4 = Scenario(model = dtpModel, inputs = sim4Inputs,
     ↪  name = "DTP_Experiment")

    return listOf(s1, s2, s3, s4)
}
```

> **Warning**
>
> When configuring the scenarios do not forget to specify the model's experimental run parameters. That is, make sure to provide the run length, number of replications, and the warm up period (if needed). This can be achieved by setting the parameters of the model or by providing the values when constructing a scenario.

The running of many scenarios will produce much output. The resulting output from running the code from Example 10.3 is shown in Figure 10.1.

We see from Figure 10.1 that an output directory is created having the name of the `ScenarioRunner` instance associated with it. Within that output directory is an output directory for each of the scenarios. In addition, for this example, a KSL database has been created, which holds all of the simulation results from the scenario runs. As we can see from the constructor of the `ScenarioRunner` class, a property is available to access the KSL database within code. As previously illustrated in Example 5.7 the KSL database reference could be used to create a `MultipleComparisonAnalyzer` instance and perform a MCB analysis. Or as illustrated in Section 5.4.1, the

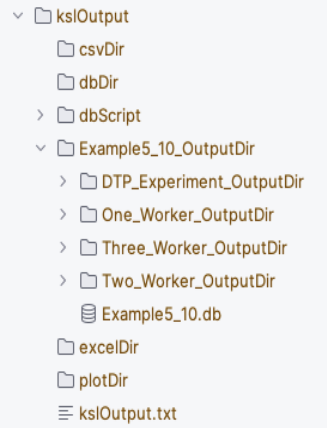


Figure 10.1.: Output from Running Scenarios

database can be used to access the statistical results and export results to CSV (or Excel) files.

```
class ScenarioRunner(
    name: String,
    scenarioList: List<Scenario> = emptyList(),
    val pathToOutputDirectory: Path =
     ↪ KSL.createSubDirectory(name.replace(" ", "_") +
     ↪ "_OutputDir"),
    val kslDb: KSLDatabase =
     ↪ KSLDatabase("${name}.db".replace(" ", "_"),
     ↪ pathToOutputDirectory)
) : Identity(name) {
```

Also, the properties `scenarioList` and `scenariosByName` allow access to the scenarios. The `scenarioList` property was used in illustrating the output from the scenarios as per the code in Example 10.3.

10.1.5. Executing Scenarios Concurrently: `ConcurrentScenarioRunner`

The `ScenarioRunner` illustrated in the previous section executes its scenarios one at a time. This is safe even when several scenarios share the same `Model` instance, precisely because each scenario's replications finish running before the next scenario begins, where nothing is ever happening at the same time. However, when scenarios are independent of one another, as they typically are, there is no real reason to insist on this sequential one-at-a-time discipline. Each scenario's replications could just as easily run on their own thread, in parallel with every other scenario's replications, cutting the wall-clock time of a large scenario analysis by roughly the number of processor cores available.

The KSL provides `ConcurrentScenarioRunner` for this purpose. It is built from the same three ingredients as `ScenarioRunner`: a name, a list of scenarios, and a `KSLDatabase` to hold the results. Its `simulate()` function plays the same role. The difference is entirely in how each scenario obtains the `Model` instance it will simulate.

Concurrent execution requires a model *builder* rather than a model. Running several scenarios at the same time means several coroutines may be executing replications simultaneously. If two of those coroutines were simulating the same shared `Model` instance, they would be racing to read and write the same event calendar, the same random-variable state, and the same statistics collectors, with unpredictable and incorrect results as the outcome. The `ConcurrentScenarioRunner` class avoids this by requiring every scenario submitted to it to be built from a instance of a `ModelBuilderIfc` instance (Section 10.1.3) rather than a single, already-built `Model` instance. Recall that a `ModelBuilderIfc` is a small, repeatable *recipe* for constructing a model: calling its `build()` function twice returns two completely independent `Model` instances. The KSL `Model` class is specifically designed to permit parallel execution at the *model* level. `ConcurrentScenarioRunner` calls `build()` once per scenario, inside that

scenario's own coroutine, so every scenario simulates its own private model and none of the models can interfere with one another.

> **Warning**
>
> A `Scenario` built from the backward-compatible `Scenario(model = ..., ...)` constructor wraps a single, shared `Model` instance. Such a scenario is safe to submit to a sequential `ScenarioRunner`, but it **must not** be submitted to `ConcurrentScenarioRunner` because doing so throws an exception immediately, before any simulation runs.

The following example builds the same "One Worker," "Two Worker," and "Three Worker" comparison from Example 5.7, this time using a `ModelBuilderIfc` for `PalletWorkCenter` so that the three scenarios can run concurrently.

Example 10.4 (Running Scenarios Concurrently)**.**

```
object BuildPalletWorkCenter : ModelBuilderIfc {
    override fun build(
        modelConfiguration: Map<String, String>?,
        experimentRunParameters: ExperimentRunParametersIfc?
    ): Model {
        val model = Model("Pallet Model")
        PalletWorkCenter(model, name = "PWC")
        model.numberOfReplications = 30
        return model
    }
}

fun main() = runBlocking {
    val s1 = Scenario(BuildPalletWorkCenter, "One Worker",
     ↪ mapOf("PWC.numWorkers" to 1.0),
        numberReplications = 30, lengthOfReplication =
 ↪ Double.POSITIVE_INFINITY)
```

```
    val s2 = Scenario(BuildPalletWorkCenter, "Two Worker",
     ↪  mapOf("PWC.numWorkers" to 2.0),
        numberReplications = 30, lengthOfReplication =
 ↪  Double.POSITIVE_INFINITY)
    val s3 = Scenario(BuildPalletWorkCenter, "Three Worker",
     ↪  mapOf("PWC.numWorkers" to 3.0),
        numberReplications = 30, lengthOfReplication =
 ↪  Double.POSITIVE_INFINITY)

    val runner = ConcurrentScenarioRunner("Pallet
     ↪  Comparison", listOf(s1, s2, s3))
    runner.simulate()
    runner.print()
}
```

Two things about this code are new. First, `main()` is wrapped in `runBlocking { }` and `simulate()` is a Kotlin suspend function. The simulation process has a two-phase design (simulate every scenario concurrently, then commit results to the database sequentially, so that two scenarios are never writing to the same database file at the same time). This only makes sense inside a coroutine, so a plain fun `main()` needs `runBlocking` to bridge into the use of coroutines. Second, notice that `ConcurrentScenarioRunner` class's constructor takes exactly the same arguments as the `ScenarioRunner` class's constructor. The runner itself does not need to know anything about how many scenarios there are or how they relate to one another; only how each individual `Scenario` was constructed matters.

Running scenarios concurrently does not, by itself, change how random numbers are assigned to them. By default, every scenario still starts from the beginning of its assigned streams, exactly as under sequential execution. Thus, concurrent execution reproduces the same common-random-number (CRN) comparison that `ScenarioRunner` would have produced:

the three worker configurations remain paired on the same underlying random numbers, which is exactly what we want when comparing alternatives (Section 5.7.2).

Sometimes CRN is not appropriate, for example, when the scenarios being batched together are not really alternatives being compared at all, or when a downstream procedure assumes independent samples. For these cases, call `useIndependentRandomStreams()` before simulating:

```
val runner = ConcurrentScenarioRunner("Pallet
 ↪  Comparison", listOf(s1, s2, s3))
runner.useIndependentRandomStreams()
runner.simulate()
```

This assigns each scenario a non-overlapping block of random number streams instead of letting all of them start from the same place. Deciding between the default (CRN) and `useIndependentRandomStreams()` is a modeling decision, not merely a performance switch: use CRN when comparing alternatives, and independent streams when the scenarios must be statistically independent of one another.

> **Note**
>
> `ConcurrentScenarioRunner` has additional capabilities not covered here: controlling whether each scenario writes to its own output subdirectory, canceling one running scenario without disturbing the others, and reporting per-replication progress to a graphical front end. See the KSL experiments guide for the full reference.

Whichever runner we use to execute a set of scenarios, the natural next question is which configuration is best. This is exactly the question `MultipleComparisonAnalyzer` was built to answer (Section 5.7.2), and no new machinery is needed to apply it here. Both `ScenarioRunner` and `ConcurrentScenarioRunner` expose the same `observationsAsMap()`

function and the same `kslDb` property, so a `MultipleComparisonAnalyzer` can be built directly from either one's results in the same way.

Example 10.5 (Comparing Scenario Results)**.**

```
// works identically whether runner is a ScenarioRunner
 ↪  or a ConcurrentScenarioRunner
val responseName =
 ↪  PalletWorkCenter(Model()).totalProcessingTime.name
val mca = runner.kslDb?.multipleComparisonAnalyzerFor(
    expNames = runner.scenarioList.map { it.name },
    responseName = responseName
)
println(mca)
```

The single throw-away `PalletWorkCenter(Model())` on the first line is not simulated; it exists only so that we can read off the response's name without hardcoding the string "Total Processing Time" ourselves, matching the practice used throughout Chapter 5. The screening output is identical in form to Example 5.7's: MCB intervals for each scenario, and a list of scenarios remaining after screening. Extending the comparison to more configurations, as in Exercise 10.7, requires nothing more than adding more scenarios to the list passed to the runner.

Using scenarios is best when you have a single model with some configuration changes or when you build different models to examine the same system (with different) configurations expressed within the code of the model. If you have a single model and it is instrumented for analysis, a good approach to exercising the model may be within the context of an experimental design, which is the topic of the next section.

10.2. Experimental Design Methods

As discussed in the previous sections, when analyzing a situation using simulation, we must often simulate a model many times. This initiates the need to be efficient and effective when running many scenarios. Experimental design concepts can assist with ensuring an efficient and effective approach to experimenting with a simulation model.

An **experiment** is a data collection procedure that occurs under controlled conditions to identify and understand the relationships between variables. A computer *simulation experiment* is an experiment executed (virtually) on a computer using a *simulation model*. The simulation model is a proxy representation for the actual physical system. When we run a simulation model, we are performing an experiment. Computer simulation **is** experimentation on a computer. An **experimental design** is a detailed plan for collecting and using data to understand the relationships between variables. An experimental design describes how to execute the experiment. For a computer simulation, an experimental design details the settings for the model's inputs and the specification of the desired outputs. Experimental design is an iterative process. Don't expect to get the plan perfect the first time or without some preliminary ad-hoc experimentation. The goal of experimental design is to be able to understand the relationships between the variables in the most efficient and effective manner possible.

Factors are the attributes of the model (parameters and/or input variables) that may be changed (or controlled) in a simulation experiment. Factors are the parameters/inputs that are of interest during experimentation. The *factor levels* are the set of possible values that factors are changed to (set at) for an experimental run. These are sometimes called treatments in physical experiments, but generally not so for computer experiments. For example, if the reorder quantity is a factor, the levels might be (5, 20, 50) units. The levels are the settings that can be used for the factor. A *design point* is the specification of an instance of factor settings that will result in observations of the response. An *experimental run* (or run) is the

execution of an experimental design point resulting in a single observation of the output variables. In experimental design, an experimental run is often called a *replicate*. In the simulation context, it is often referred to as a *replication*. An *experimental design* can be thought of as the set of design points and the specification of how many replications are required for each point.

10.2.1. Defining Factors

The KSL provides functionality to define and run experiments based on experimental design contexts. We start with being able to represent factors. Figure 10.2 presents the class diagram for the `Factor` class. Factors have a name and a specification of the levels for the factors.

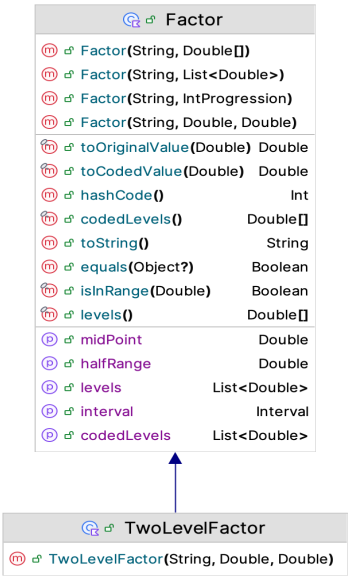


Figure 10.2.: KSL Factors

When setting up an experimental design, the regressors (factors) are often coded in such a manner to facilitate the interpretation of the model coefficients. That is, they are often scaled to a range of values between -1 and 1, or between 0 and 1. Within the KSL, the levels of the factors should be supplied via their uncoded values. The KSL `Factor` class can perform the translation to and from the coded and uncoded design space.

Suppose we have non-standardized factor p and denoted as w_p. Suppose w_p ranges between its lowest value, l_p and its highest value u_p. That is, w_p is in the interval, $w_p \in [l_p, u_p]$. For example, suppose w_p is the mean of the service time distribution and w_p has levels $[5, 10, 15, 20, 25]$. Thus, $l_p = 5$ and $u_p = 25$

- Let h_p be the half-range, where $h_p = (u_p - l_p)/2$. The half-range measures the spread of the factor.
- Let m_p be the mid-point of the range, where $m_p = (u_p + l_p)/2$.

A standard approach is to the code the factor as:

$$x_{ip} = \frac{w_{ip} - m_p}{h_p}$$

For example, suppose $w_{ip} \in \{5, 10, 15, 20, 25\}$. Thus, $h_p = (25 - 5)/2 = 10$ and $m_p = (25 + 5)/2 = 15$. Thus, we have $x_{ip} \in \{-1, \frac{-1}{2}, 0, \frac{1}{2}, 1\}$. Notice that for a factor with **two** levels this always results in a coding of ± 1. Low is coded to -1 and high is coded to $+1$.

The following illustrates the creation of two factors, one with 3 levels and another with 2 levels.

```
val f1 = Factor("A", doubleArrayOf(1.0, 2.0, 3.0, 4.0))
val f2 = Factor("B", doubleArrayOf(5.0, 9.0))
val factors = setOf(f1, f2)
```

10.2.2. Defining Experimental Designs

Once we have defined factors for the experiment, we can proceed with defining an experimental design. The KSL supports factorial designs, two level factorial designs, central composite designs, and general experimental designs. Figure 10.3 presents the major classes and interfaces related to KSL experiments.

The first important item to note is the use of the `ExperimentalDesignIfc` interface. The experimental designs supported by the KSL all implement this interface. The `ExperimentalDesignIfc` interface provides functionality that allows the design points in the design to be specified and iterated. To create an experimental design, we must specify the set of factors, or as in the case of a composite design, the starting factorial design.

The following code illustrates how to create a factorial design and will print out the design points.

```
val f1 = Factor("A", doubleArrayOf(1.0, 2.0, 3.0, 4.0))
val f2 = Factor("B", doubleArrayOf(5.0, 9.0))
val factors = setOf(f1, f2)
val fd = FactorialDesign(factors)
println(fd)
println()
println("Factorial Design as Data Frame")
println(fd.designPointsAsDataframe())
println()
println("Coded Factorial Design as Data Frame")
println(fd.designPointsAsDataframe(true))
println()
```

There are eight design points in this design because we have factor A with 4 levels and factor B with 2 levels, resulting in $4 \times 2 = 8$ design points.

Figure 10.3.: KSL Experimental Design Classes

```
FactorialDesign
name: ID_3
number of design points: 8
Factors
Factor: A
Levels = 1.0,2.0,3.0,4.0
halfRange = 1.5
midPoint = 2.5
Coded Levels = -1.0,-0.3333333333333333,0.3333333333333333,1.0

Factor: B
Levels = 5.0,9.0
halfRange = 2.0
midPoint = 7.0
Coded Levels = -1.0,1.0

Factorial Design as Data Frame
     A   B
 0 1.0 5.0
 1 1.0 9.0
 2 2.0 5.0
 3 2.0 9.0
 4 3.0 5.0
 5 3.0 9.0
 6 4.0 5.0
 7 4.0 9.0

 Coded Factorial Design as Data Frame
           A    B
 0 -1.000000 -1.0
 1 -1.000000  1.0
 2 -0.333333 -1.0
 3 -0.333333  1.0
 4  0.333333 -1.0
```

```
5  0.333333  1.0
6  1.000000 -1.0
7  1.000000  1.0
```

A two-level factorial design is a special case of the more general factorial design where factors can have more than two levels. For two-level factorial designs, the KSL provides the `TwoLevelFactorialDesign` class. The following code illustrates how to create a two-level factorial design.

```
    val design = TwoLevelFactorialDesign(
        setOf(
            TwoLevelFactor("A", 5.0, 15.0),
            TwoLevelFactor("B", 2.0, 11.0),
            TwoLevelFactor("C", 6.0, 10.0),
            TwoLevelFactor("D", 3.0, 9.0),
        )
    )
    val fdf = design.designPointsAsDataframe()
    println("Full design points")
    fdf.print(rowsLimit = 36)
    println("Coded full design points")
    design.designPointsAsDataframe(true).print(rowsLimit =
↪   36)
    println()
```

Note that for this design, we need to define the factors using the `TwoLevelFactor` class, which restricts the levels to only two values. Since there are only two levels, the coded values are (-1) and (+1).

Once we have a full two-level factorial design, we may want to only simulate a portion of the design. A fractional factorial design is an experimental design that specifies some fraction of the full factorial combination of design points. The most commonly specified fractional designs are one-half

fraction of 2^k designs. A $\frac{1}{2}$ fraction of a 2^k design has $2^{k-1} = \frac{1}{2}2^k$ runs and is called a 2^{k-1} fractional factorial design. In the context of 2^k designs, we may have larger fractions, i.e. 2^{k-p} designs, where $p \geq 1$. Running a fraction of the design points reduces the number of runs; however, it also limits what models can be estimated.

Using a $\frac{1}{2}$ fraction creates two sets of runs. Below the rows have been rearranged so that column (123) starts with +1 for the first half-fraction. A 2^{3-1} fractional factorial design has $2^2 = 4$ runs. Running the rows with +1 of column (123), would be running the first (positive) $\frac{1}{2}$ fraction of the design.

I	(1)	(2)	(3)	(12)	(13)	(23)	(123)	*Fraction*
+1	+1	−1	−1	−1	−1	+1	+1	*Positive*
+1	−1	+1	−1	−1	+1	−1	+1	*Positive*
+1	−1	−1	+1	+1	−1	−1	+1	*Positive*
+1	+1	+1	+1	+1	+1	+1	+1	*Positive*
+1	+1	+1	−1	+1	−1	−1	−1	*Negative*
+1	+1	−1	+1	−1	+1	−1	−1	*Negative*
+1	−1	+1	+1	−1	−1	+1	−1	*Negative*
+1	−1	−1	−1	+1	+1	+1	−1	*Negative*

We can specify fractional designs within the KSL by requesting an iterator from the full two-level design that contains the desired portion of the design. This can be accomplished with the `halfFractionIterator()` function or the `fractionalIterator()` function. The following code illustrates how to access these iterators.

```
// create the full design
val design = TwoLevelFactorialDesign(
    setOf(
        TwoLevelFactor("A", 5.0, 15.0),
        TwoLevelFactor("B", 2.0, 11.0),
```

```
            TwoLevelFactor("C", 6.0, 10.0),
            TwoLevelFactor("D", 3.0, 9.0),
            TwoLevelFactor("E", 4.0, 16.0)
        )
    )
    println("Positive half-fraction")
    val hitr = design.halfFractionIterator()
    // convert iterator to data frame for display
    hitr.asSequence().toList().toDataFrame(coded =
↪   true).print(rowsLimit = 36)
    println()
    // This is a resolution III 2^(5-2) design
    // This design can be found here:
     ↪  https://www.itl.nist.gov/div898/handbook/pri/section3/eqns/2to5m2.txt
    // Specify a design relation as a set of sets.
    // The sets are the columns of the design that define the
     ↪  associated generator
    val relation = setOf(setOf(1, 2, 4), setOf(1, 3, 5),
     ↪  setOf(2, 3, 4, 5))
    val itr = design.fractionalIterator(relation)
    println("number of factors = ${itr.numFactors}")
    println("number of points = ${itr.numPoints}")
    println("fraction (p) = ${itr.fraction}")
    val dPoints = itr.asSequence().toList()
    val df = dPoints.toDataFrame(coded = true)
    println("Fractional design points")
    df.print(rowsLimit = 36)
```

First the full factorial design is constructed. The `halfFractionIterator()` function has the following signature. The first parameter should be (+1.0) to get the positive half-fraction, or (-1.0) to get the negative half-fraction.

```
    /**
     *  @param half indicates the half-fraction to iterate.
↪  1.0 indicates the positive
     *  half-fraction and -1.0 the negative half-fraction.
↪  The default is 1.0
     *  @param numReps the number of replications for the
↪  design points.
     *  Must be greater or equal to 1. If null, then the
↪  current value for the
     *  number of replications of each design point is used.
↪  Null is the default.
     */
    fun halfFractionIterator(half: Double = 1.0, numReps:
     ↪  Int? = null): TwoLevelFractionalIterator
```

The results of the half-fraction code are as follows. Notice that there are only 16 (of the 32) design points.

```
Positive half-fraction
       A    B    C    D    E
  0 -1.0 -1.0 -1.0 -1.0  1.0
  1 -1.0 -1.0 -1.0  1.0 -1.0
  2 -1.0 -1.0  1.0 -1.0 -1.0
  3 -1.0 -1.0  1.0  1.0  1.0
  4 -1.0  1.0 -1.0 -1.0 -1.0
  5 -1.0  1.0 -1.0  1.0  1.0
  6 -1.0  1.0  1.0 -1.0  1.0
  7 -1.0  1.0  1.0  1.0 -1.0
  8  1.0 -1.0 -1.0 -1.0 -1.0
  9  1.0 -1.0 -1.0  1.0  1.0
 10  1.0 -1.0  1.0 -1.0  1.0
 11  1.0 -1.0  1.0  1.0 -1.0
```

```
12  1.0  1.0 -1.0 -1.0  1.0
13  1.0  1.0 -1.0  1.0 -1.0
14  1.0  1.0  1.0 -1.0 -1.0
15  1.0  1.0  1.0  1.0  1.0
```

The `fractionalIterator()` function is more complicated. It's signature is as follows.

```
fun fractionalIterator(
    relation: Set<Set<Int>>,
    numReps: Int? = null,
    sign: Double = 1.0
): TwoLevelFractionalIterator {
    return TwoLevelFractionalIterator(relation, numReps,
     ↪  sign)
}
```

This iterator should present each design point in the associated fractional design until all points in the fractional design have been presented. The iterator checks if the coded values of the design point are in the defining relation specified by the factor numbers stored in the relation set. Suppose the designing relation is $I = 124 = 135 = 2345$. Then relation specification is `setOf(setOf(1,2,4), setOf(1,3,5), setOf(2,3,4,5))`. The values in the words must be valid factor indices. That is, if a design has 5 factors, then the indices must be in the set (1,2,3,4,5), with 1 referencing the first factor, 2 the 2nd, etc.

To learn more about fractional designs and design generators, we refer the interested reader to the fractional factorial design section of the NIST Engineering Statistics Handbook.

The resulting design points for the 2^{5-2} resolution III design are as follows. Notice that there are only 8 design points in this resolution III design.

```
number of factors = 5
number of points = 8
fraction (p) = 2
Fractional design points
      A    B    C    D    E
 0 -1.0 -1.0 -1.0  1.0  1.0
 1 -1.0 -1.0  1.0  1.0 -1.0
 2 -1.0  1.0 -1.0 -1.0  1.0
 3 -1.0  1.0  1.0 -1.0 -1.0
 4  1.0 -1.0 -1.0 -1.0 -1.0
 5  1.0 -1.0  1.0 -1.0  1.0
 6  1.0  1.0 -1.0  1.0 -1.0
 7  1.0  1.0  1.0  1.0  1.0
```

As illustrated for the fractional design case, an iterator over design points is an essential component of how the KSL implements its experimental design functionality. Figure 10.4 illustrates the basic functionality of a design point iterator.

There are a couple of items of note. First, a design point knows the number of replications it will require and has access to the factors and to the associated design. The design point can provide coded and uncoded values. The design point iterator also has access to the factors, the design, and can move through the design points. This functionality is essential for how the `DesignedExperiment` class operates.

10.2.3. Executing a Designed Experiment

The purpose of the `DesignedExperiment` class is to simulate an experimental design. The `DesignedExperiment` class causes each design point in its supplied design to be simulated for the specified number of replications.

Figure 10.5 illustrates the KSL's `DesignedExperiment` class. The signature of its constructor is as follows.

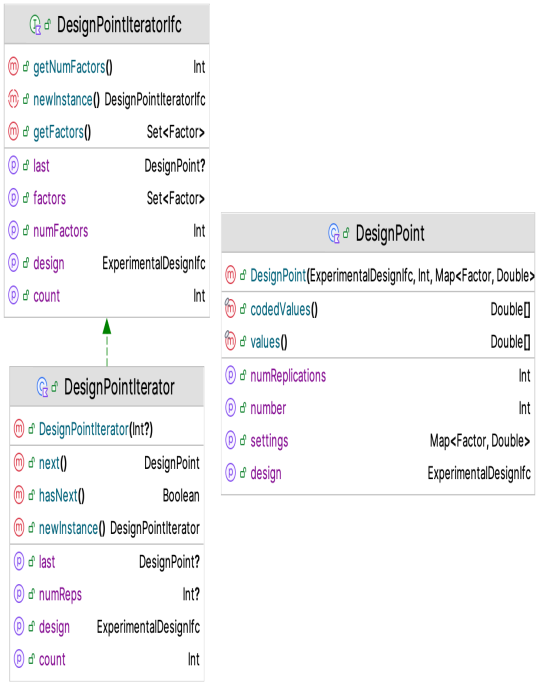


Figure 10.4.: KSL Design Point Iterator Classes

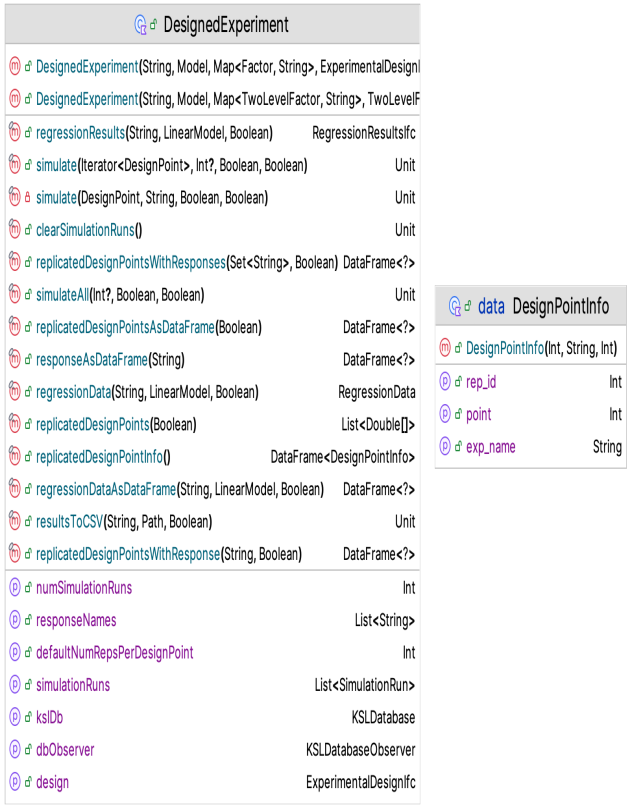


Figure 10.5.: KSL DesignedExperiment Class

```
class DesignedExperiment(
    name: String,
    private val model: Model,
    private val factorSettings: Map<Factor, String>,
    val design: ExperimentalDesignIfc,
    val kslDb: KSLDatabase? =
     ↪ KSLDatabase("${name}.db".replace(" ", "_"),
        model.outputDirectory.dbDir)
) : Identity(name) {
```

Notice that the constructor requires a name for the experiment, the model that will be exercised, the factorial settings, and the experimental design. The factor settings are specified by using controls or random variable parameter settings. The following code sets up a three factor, two level factorial design and simulates the design points.

```
    val fA = Factor("Server", doubleArrayOf(1.0, 2.0, 3.0))
    val fB = Factor("MeanST", doubleArrayOf(0.6, 0.7))
    val fC = Factor("MeanTBA", doubleArrayOf(1.0, 5.0))
    val factors = mapOf(
        fA to "MM1Q.numServers",
        fB to "MM1_Test:ServiceTime.mean",
        fC to "MM1_Test:TBA.mean"
    )
    val m = Model("MM1_Test")
    m.numberOfReplications = 15
    m.lengthOfReplication = 10000.0
    m.lengthOfReplicationWarmUp = 5000.0
    val ggc = GIGcQueue(m, 1, name = "MM1Q")

    val fd = FactorialDesign(factors.keys)
    val de = DesignedExperiment("FactorDesignTest", m,
     ↪ factors, fd)
```

```
println("Design points being simulated")
fd.designPointsAsDataframe().print(rowsLimit = 36)
println()
de.simulateAll(numRepsPerDesignPoint = 3)
println("Simulation of the design is completed")
de.resultsToCSV()
```

Notice that we can specify how many times each design point is replicated when simulating the design. This class works in a very similar manner as that described in Section 10.1.4 for running multiple scenarios. The `DesignedExperiment` class captures the results from the simulation runs to a database, which can be found in the `dbDir` directory associated with the model being simulated. The `DesignedExperiment` class also provides multiple ways to capture the design and its results, especially in the form of a data frame. The results can be easily exported to a CSV file for further analysis by your favorite statistical software.

```
println("Replicated design points")
de.replicatedDesignPointsAsDataFrame().print(rowsLimit =
↪ 36)
println()
println("Responses as a data frame")
de.responseAsDataFrame("System Time").print(rowsLimit =
↪ 36)
println()
de.replicatedDesignPointsWithResponse("System
↪ Time").print(rowsLimit = 36)
println()
de.replicatedDesignPointsWithResponses().print(rowsLimit
↪ = 36)
println()
de.replicatedDesignPointsWithResponses(coded =
↪ true).print(rowsLimit = 36)
```

The results for all responses, an individual response, and the design points (coded or uncoded) are readily available for further analysis.

The `DesignedExperiment` class simulates its design points one at a time, reusing the single model instance passed to its constructor. For larger designs with many factors, many levels, the execution time can become significant, even though the design points have nothing to do with one another: each one only needs its own replications simulated and recorded. Thus, the KSL provides the `ParallelDesignedExperiment` class, a drop-in counterpart that runs every design point concurrently.

The only real difference is how the model is supplied. Where the `DesignedExperiment` class takes a single, already-built `model: Model`, the `ParallelDesignedExperiment` class takes a `ModelBuilderIfc` (Section 10.1.3) in that same argument position, for the same reason a `Scenario` instance needs one for the `ConcurrentScenarioRunner` class. That is, every design point gets its own private `Model` instance, so none of them can interfere with one another while running at the same time.

```
object BuildMM1Q : ModelBuilderIfc {
    override fun build(
        modelConfiguration: Map<String, String>?,
        experimentRunParameters: ExperimentRunParametersIfc?
    ): Model {
        val m = Model("MM1_Test")
        m.numberOfReplications = 15
        m.lengthOfReplication = 10000.0
        m.lengthOfReplicationWarmUp = 5000.0
        GIGcQueue(m, 1, name = "MM1Q")
        return m
    }
}
```

```
fun main() = runBlocking {
    val de = ParallelDesignedExperiment("FactorDesignTest",
     ↪ BuildMM1Q, factors, fd)
    de.simulateAll(numRepsPerDesignPoint = 3)
    de.resultsToCSV()
}
```

Compare this to `DesignedExperiment("FactorDesignTest", m, factors, fd)` from Section 10.2.3: the name, `factorSettings` parameter, and `design` arguments are unchanged, and the second argument is still "the thing that supplies the model". In this case, it has just changed from a live `Model` instance to a `ModelBuilderIfc` builder instance that builds a model. Everything after construction, `simulateAll()`, `resultsToCSV()`, and every data-frame and regression method is called exactly as it would be for the `DesignedExperiment` class. Both classes share the same results API.

One default is worth mentioning because it is the *opposite* of the `ConcurrentScenarioRunner` class's approach. The `ParallelDesignedExperiment` class starts each design point on an independent block of random number streams by default, rather than common random numbers. This is the right default for fitting a regression model (Section 10.2.4) because it keeps the observations for each design point statistically independent, which is what ordinary least squares assumes. Call `useCommonRandomNumbers()` before simulating if every design point should instead share the same starting stream position.

> **Note**
>
> `ParallelDesignedExperiment` also supports canceling an individual in-flight design point and choosing whether each point writes to its own output subdirectory. See the KSL experiments guide for details.

10.2.4. Analyzing a Designed Experiment

This final example illustrates how the results from a designed experiment can be analyzed using linear regression. The following code creates a model for a reorder level, reorder quantity inventory model. Details of this model can be found in Section 7.5.2 of Chapter 7. The following code creates the KSL discrete-event simulation model.

```
    val m = Model("ResponseSurfaceDemo")
    val rqModel = RQInventorySystem(m, name = "RQInventory")
    rqModel.costPerOrder = 0.15 //$ per order
    rqModel.unitHoldingCost = 0.25 //$ per unit per month
    rqModel.unitBackorderCost = 1.75 //$ per unit per month
    rqModel.initialReorderPoint = 2
    rqModel.initialReorderQty = 3
    rqModel.initialOnHand = rqModel.initialReorderPoint +
↪   rqModel.initialReorderQty
    rqModel.demandGenerator.setInitialEventTimeProcesses(
↪   ExponentialRV(1.0 / 3.6))
    rqModel.leadTime.initialRandomSource = ConstantRV(0.5)
    m.lengthOfReplication = 72.0
    m.lengthOfReplicationWarmUp = 12.0
    m.numberOfReplications = 30
```

In the following code, we setup two factors, one for the reorder level and one for the reorder quantity. Then, we specify the factorial design.

```
    val r = TwoLevelFactor("ReorderLevel", low = 1.0, high =
↪    5.0)
    println(r)
    val q = TwoLevelFactor("ReorderQty", low = 1.0, high =
↪    7.0)
    println(q)
```

```
println()
val design = TwoLevelFactorialDesign(setOf(r, q))
println("Design points being simulated")
val df = design.designPointsAsDataframe()
df.print(rowsLimit = 36)
```

Finally, we specify the settings of the factors using controls. See Section 10.1.1 for more on controls. Then, we specify the designed experiment and simulate all of the design points. Each design point is replicated 20 times.

```
val settings = mapOf(
    r to "RQInventory:Item.initialReorderPoint",
    q to "RQInventory:Item.initialReorderQty",
)
val de = DesignedExperiment("R-Q Inventory Experiment",
 ↪  m, settings, design)
de.simulateAll(numRepsPerDesignPoint = 20)
println("Simulation of the design is completed")
```

After running the design, we can access the design points and the results. Then, we can run a linear regression based on the results stored in the data frame. Figure 10.6 presents the functions and properties of the KSL `LinearModel` class.

The KSL `LinearModel` class provides the ability to specify a linear model (for use in regression and design of experiments). That is, the linear model is specified only as a string. The terms are specified by the names of the factors. As an example, consider a model with three factors, "A", "B", "C", with full model = "A B C A*B A*C B*C A*B*C". The full regression model for this situation is:

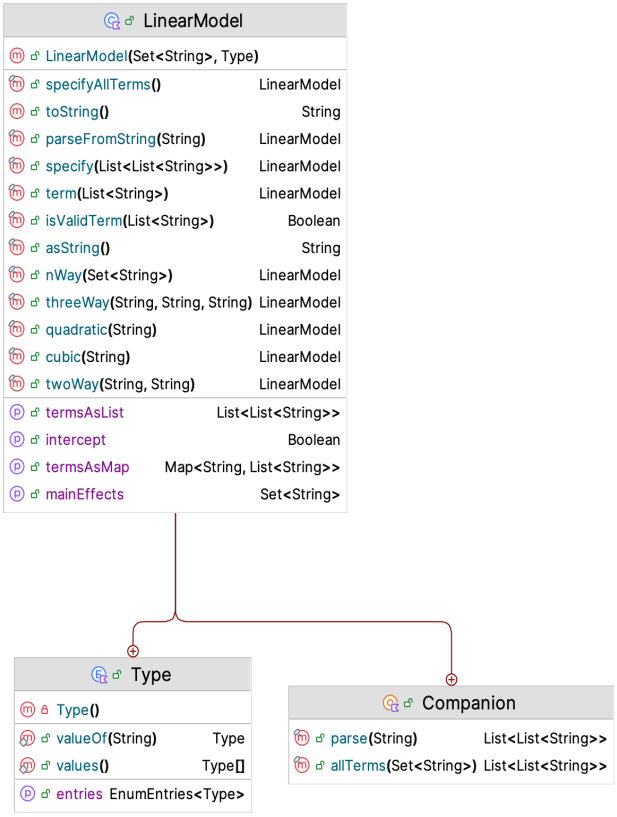


Figure 10.6.: KSL LinearModel Class

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_{12} x_1 x_2 + \beta_{13} x_1 x_3 + \beta_{23} x_2 x_3 + \beta_{123} x_1 x_2 x_3 + \epsilon$$

To specify this model using the KSL `LinearModel` class, we use the following.

```
val m3 = LinearModel(setOf("A", "B", "C"))
m3.specifyAllTerms()
```

The KSL `LinearModel` class has functions that allow you to build the terms of the model. In addition, you can parse a string representation, such as "A B C A*B A*C B*C A*B*C", to build the model. Let's take a look at the code for specifying the linear model for the reorder point, reorder quantity model.

```
    val resultsDf = de.replicatedDesignPointsWithResponse(
     ↪  "RQInventory:Item:TotalCost", coded = true)
    resultsDf.print(rowsLimit = 80)
    println()
    val lm = design.linearModel(type =
     ↪  LinearModel.Type.AllTerms)
    println(lm.asString())
    println()
    val lmDF = resultsDf.addColumnsFor(lm)
    lmDF.print(rowsLimit = 80)
    val regressionResults =
     ↪  de.regressionResults("RQInventory:Item:TotalCost",
     ↪  lm)
    println()
    println(regressionResults)
    regressionResults.showResultsInBrowser()
```

The design points and the responses are extracted from the designed experiment into a data frame with the following line of code.

```
val resultsDf = de.replicatedDesignPointsWithResponse(
 ↪  "RQInventory:Item:TotalCost", coded = true)
```

Then, an instance of the `LinearModel` class is created based on all the terms available in the model.

```
val lm = design.linearModel(type =
 ↪  LinearModel.Type.AllTerms)
println(lm.asString())
```

This results in the output representing the linear model.

```
ReorderQty ReorderLevel ReorderLevel*ReorderQty
```

This is essentially the following linear regression model.

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_{12} x_1 x_2 + \epsilon$$

To illustrate how to use the linear model to setup the data frame to perform the regression analysis we can use the following line of code.

```
val lmDF = resultsDf.addColumnsFor(lm)
```

In this case, the product term "ReorderLevel*ReorderQty" is added to the data frame to allow for the estimation of the interaction term. This line invokes an extension function for the `DataFrame` class to add the columns needed to the data frame that will allow the regression of the specified linear model.

Actually, this line is unnecessary within this example because the following line of code tells the designed experiment to perform the regression using the linear model and as part of that process, the associated data frame is created before the regression is performed to get the regression results.

```
    val regressionResults =
     ↪  de.regressionResults("RQInventory:Item:TotalCost",
     ↪  lm)
```

The regression results along with a browser representation (not shown here) are as follows. Notice that the regression analysis prepares an ANOVA table and estimates the parameters of the proposed linear model.

```
Regression Results
------------------------------------------------------------------------------
Analysis of Variance
Source               SumSq        DOF             MS            f_0         P(F>f0
Regression         6.98981          3       2.329937     520.883150       0.00000
Error             0.339952         76       0.004473
Total              7.32976         79
------------------------------------------------------------------------------
Error Variance (MSE) = 0.004473051454598468
Regression Standard Error = 0.06688087510341403
R-Squared = 0.9536203426372002
Adjusted R-Squared = 0.9517895666886685
------------------------------------------------------------------------------
Parameter Estimation Results
                Predictor parameter parameterSE   t0-
ratio 2*P(T>|t0|) LowerLimit UpperLimit
 0               Intercept  1.490275    0.007478 199.300931    0.000000   1.475382
 1            ReorderLevel  0.247156    0.007478  33.053257    0.000000   0.232263
 2              ReorderQty -0.017659    0.007478  -2.361671    0.020755  -
0.032552  -0.002767
```

```
 3 ReorderLevel*ReorderQty  0.161167    0.007478  21.553519    0.000000   0.146274
-------------------------------------------------------------------------------
```

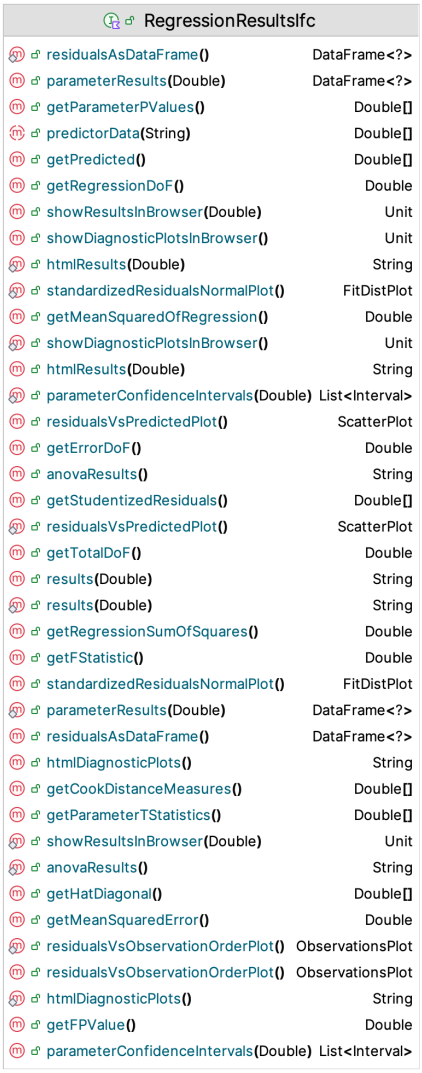
As presented in Figure 10.7, the KSL provides basic support for performing linear regression analysis.

The functionality includes support for performing the ANOVA analysis, estimating the parameters, displaying diagnostic plots, and for performing an analysis of the residuals. The most useful functionality is to use the `showResultsInBrowser()` and `showDiagnosticPlotsInBrowser()` functions.

The performing of an regression analysis is made possible by the `RegressionData` class as shown in Figure 10.8. The key purpose of this class is to organize the regression matrix and the response array for analysis. The predictor names and response names are also specified.

The `OLSRegression` class with the following signature performs the regression. Notice that there is a constructor that will use a data frame, which requires the specification of the column name of the response variable and the columns needed for the predictor (regressor) variables.

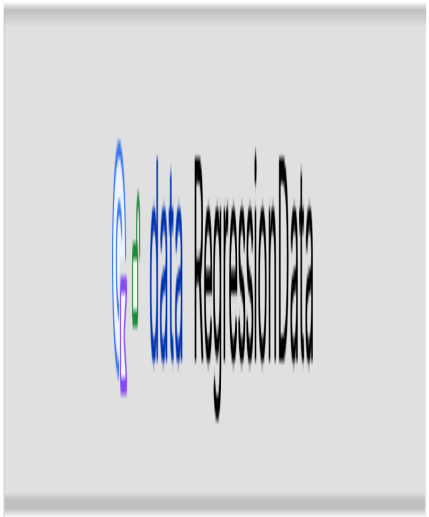
```
class OLSRegression(regressionData: RegressionData) :
↪  RegressionResultsIfc {

    /**
     *  Create the regression data from a data frame. The
↪  data frame
     *  must have a column with the response name
↪  [responseName] and
     *  columns with the names in the list [predictorNames].
↪  The
     *  data type of these columns must be Double.
↪  [hasIntercept] indicates
     *  if the regression should include an intercept term.
↪  The default is
```

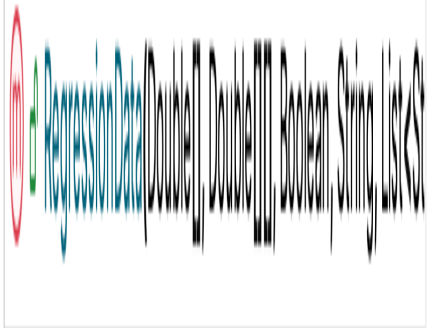


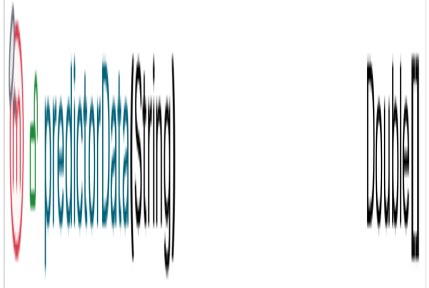
(a) Functions

(b) Properties

Figure 10.7.: KSL Linear Regression Support







```
     *  true. The data in the data frame does not need to
↪  have a column
     *  for estimating the intercept.
     */
    constructor(
        df: AnyFrame,
        responseName: String,
        predictorNames: List<String>,
        hasIntercept: Boolean = true
    ) : this(RegressionData.create(df, responseName,
     ↪  predictorNames, hasIntercept))
```

The KSL regression functionality is available with in the `ksl.utilities.statistics` package.

10.2.5. Central Composite Designs and Response Surface Methods

The two-level factorial designs used throughout this section support fitting only a *first-order model with interactions*: main effects and the interactions between them, as in the regression examples of Section 10.2.4. A first-order model with interactions can tell us that a factor matters, and whether two factors interact, but it cannot tell you whether the response *curves* as a factor changes, because every factor is only ever observed at two levels, and two points always fit a straight line perfectly, no matter what the true relationship looks like. This matters especially for queueing systems, where performance measures like waiting time are well known to respond *non-linearly* to their inputs: as a system's utilization approaches 1, waiting time does not simply increase, it accelerates. Thus, a first-order model with interactions, built from only the two corners of a studied region, cannot represent that acceleration. A *central composite design (CCD)* is what adds the ability to detect and estimate it, and estimating a curved response

over a region of the factors' settings is what the term *response surface* actually references.

A CCD starts from an ordinary two-level factorial design and augments it with two additional kinds of runs. *Center-point* replications, which represents the design's midpoint, run several times and reveal whether curvature is present. If the prediction of the first-order model with interactions at the center disagrees with what is actually observed there, the true response is curved. *Axial* (or "star") points, placed along each factor's own axis beyond its low and high corners, let us estimate each factor's individual quadratic effect. Together, the factorial, axial, and center points support fitting a full second-order model.

> ⚠ Warning
>
> A central composite design's axial points are, by construction, not restricted to the coded levels ± 1 used by the factorial corners, and they usually are not whole numbers. This makes a CCD a poor fit for a factor that is only meaningful at integer values. For example, the reorder point and reorder quantity of Section 10.2.3 are both `Int`-valued properties, so a CCD built directly on them would generate axial points like 5.83 units of inventory, which the control mechanism would have to round before applying. Worse, a *circumscribed* CCD's axial points may fall **outside** the low and high corners you specified (Section 10.2.2); depending on the ranges chosen, that can push a factor like reorder quantity into invalid territory entirely. For the ranges used in Section 10.2.3, the low axial point for `ReorderQty` would be *negative*. A CCD needs factors that are genuinely continuous over the whole region being studied, including somewhat beyond it. This section uses a different pair of factors for exactly that reason.

The `MeanST` and `MeanTBA` factors from Section 10.2.3 represent the mean service time and mean time between arrivals for the `GIGcQueue` model. They are exactly the kind of factor required. Like any random variable's

mean, they are continuous. Thus, a fractional axial point is perfectly natural rather than something that has to be rounded away.

Example 10.6 (Building a Central Composite Design)**.**

```
    val st = TwoLevelFactor("MeanST", low = 2.0, high = 4.0)
    val tba = TwoLevelFactor("MeanTBA", low = 6.0, high =
     ↪  10.0)

    val ccd = CentralCompositeDesign(
        factors = setOf(st, tba),
        numFactorialReps = 10,
        numAxialReps = 10,
        numCenterReps = 20,
        axialSpacing =
↪   CentralCompositeDesign.rotatableAxialSpacing(numFactors =
↪   2)
    )
```

`rotatableAxialSpacing()` computes a principled axial spacing rather than leaving you to guess one: for a two-factor design it works out to $\sqrt{2} \approx 1.414$ coded units. Decoded, this places the axial points at `MeanST` ≈ 1.59 and 4.41 (with `MeanTBA` held at its center value of 8.0), and at `MeanTBA` ≈ 5.17 and 10.83 (with `MeanST` held at its center value of 3.0). Every one of these points, along with the four factorial corners and the center point, keeps the ratio of `MeanST` to `MeanTBA`, the implied server utilization, comfortably below 1: the highest utilization anywhere in the design is at the factorial corner where `MeanST` is high and `MeanTBA` is low ($4.0/6.0 \approx 0.67$), not at any single axial point, since each axial point moves only one factor away from the center at a time. Checking this before simulating, the way this calculation just did, is exactly the discipline the warning expects.

The center point is given more replications (20) than each factorial or axial point (10), because it is the only point in the design that lets us directly estimate the underlying noise in the system.

Simulating the design uses `DesignedExperiment` exactly as in Section 10.2.3.

Example 10.7 (Fitting a Response Surface)**.**

```
val m = Model("MM1_Test")
m.numberOfReplications = 15
m.lengthOfReplication = 10000.0
m.lengthOfReplicationWarmUp = 5000.0
GIGcQueue(m, 1, name = "MM1Q")

val settings = mapOf<Factor, String>(
    st to "MM1_Test:ServiceTime.mean",
    tba to "MM1_Test:TBA.mean"
)
val de = DesignedExperiment("Queue Response Surface", m,
 ↪  settings, ccd)
de.simulateAll()
println("Simulation of the design is completed")

val firstOrderModel = ccd.linearModel(type =
 ↪  LinearModel.Type.FirstAndSecond)
val firstOrderResults = de.regressionResults("System
 ↪  Time", firstOrderModel)

val secondOrderModel = ccd.linearModel(type =
 ↪  LinearModel.Type.FirstAndSecond)
    .quadratic("MeanST")
    .quadratic("MeanTBA")
val secondOrderResults = de.regressionResults("System
 ↪  Time", secondOrderModel)
```

```
    println("First-order R-squared:
↪   ${firstOrderResults.rSquared}")
    println("Second-order R-squared:
↪   ${secondOrderResults.rSquared}")
    secondOrderResults.showResultsInBrowser()
```

Notice that `simulateAll()` is called here with *no* argument. Passing an explicit `numRepsPerDesignPoint`, as Section 10.2.3 did, would overwrite the replication count on every design point uniformly, including the center point and erasing the extra center-point replications the design was just given. Thus, the argument is left out, which tells `DesignedExperiment` to use the replication count already carried by each design point.

Two regression models are fit to the same data. `firstOrderModel` is the same kind of model used in Section 10.2.4 with main effects and their interaction and `secondOrderModel` adds the two quadratic terms that only a design with axial and center points can support. Neither `LinearModel.Type.FirstAndSecond` nor `.specifyAllTerms()` ever adds a squared term automatically. The `.quadratic(name)` function must be added explicitly for each factor whose curvature we want to estimate. Comparing `firstOrderResults.rSquared` to `secondOrderResults.rSquared` gives a quick, informal check for curvature: if the second-order model's R^2 is not meaningfully larger, system time is behaving close to linearly over the region studied, and the simpler first-order model with interactions is adequate. If it is meaningfully larger (and the parameter results shown by `secondOrderResults.showResultsInBrowser()` show one or both quadratic terms as statistically significant), system time really does accelerate faster than a straight line as utilization rises, and the second-order model gives a more honest picture of that acceleration *within the region actually studied*. As always with a fitted regression model, resist the temptation to trust its predictions outside that region, and remember that

a CCD's own axial points already push right to the edge of what can be trusted, per the warning above.

10.3. Summary

The KSL provides classes to construct, run, and analyze scenarios of interest to the modeler. In addition, analysts can setup experiments for a simulation model. The following new KSL constructs were discussed in this chapter for analyzing scenarios:

`KSLControl`: This annotation class allows properties to be annotated as a control for a simulation model. Controls can be used to change model inputs during experiments.

`RVParameterSetter`: This class facilitates changing the parameters of the random variables used within a KSL model.

`Scenario`: This class represents a KSL model and its inputs to be run by the `ScenarioRunner` class.

`ScenarioRunner`: This class will run a set of scenarios and capture the results within a KSL database.

The following new KSL constructs were discussed in this chapter for working with experiments:

`Factor`: This class provides a general approach to defining the factors used within an experiment and tying them to the simulation model. In addition, the coding of the factor at different levels is facilitated.

`ExperimentalDesignIfc`: This interface defines how an experiment behaves.

`ExperimentalDesign`: This class is a concrete implementation of the `ExperimentalDesignIfc` interface to specify a general experiment.

`FactorialDesign`: This class implements the `ExperimentalDesignIfc` interface and provides the ability to define an experiment that has many factors with many levels.

`TwoLevelFactorialDesign`: This class is a special case of `FactorialDesign` class and provides a representation for the use of the classic full 2^K factorial design and its fractions.

`CentralCompositeDesign`: This class facilitates the construction of a factorial design that uses central composite points.

`DesignedExperiment`: This class facilitates the running of an experimental design and the tabulation of the results in a database.

`LinearModel`: This class can be used to specify a linear model for the output of a designed experiment and prepare the output for use in a linear regression.

The KSL provides functionality for using simulation models to analyze specific scenarios of interest and to perform designed experiments. The next chapter builds on these capabilities to search for the best values of a model's inputs via simulation optimization.

10.4. Exercises

Exercise 10.1. For each of the following model properties, indicate (i) which KSL control mechanism would be used to expose it as a control (a numeric `@KSLControl` with its `ControlType`, a `@KSLStringControl`, a `@KSLJsonControl`, or a random variable parameter changed through `RVParameterSetter`) and (ii) give a plausible control key of the form `ElementName.propertyName`.

a. The mean of an exponential processing-time random variable, `ServiceTimeRV`
b. The number of pharmacists working at the element `DTP`
c. A dispatching rule for queue `TestQueue`, restricted to `"FIFO"`, `"LIFO"`, or `"SPT"`
d. A vector of five daily arrival-rate multipliers for element `ArrivalSchedule`
e. Whether balking is permitted at queue `TestQueue`
f. The reorder quantity for inventory element `RQInventory:Item`

For the string- and JSON-valued properties, also state what should happen if an invalid value is assigned (e.g., a rule name not in the allowed list).

Exercise 10.2. A queueing model has a service-rate factor with levels 4, 6, 8, 10, and 12 customers per hour, and a staffing factor with levels 2 and 3 servers.

a. Compute the half-range and midpoint for the service-rate factor and give its five coded values.
b. Give the coded values for the two-level staffing factor.
c. A third factor, buffer capacity, is added with levels 5, 10, 15, and 20. How many design points are in the full factorial design formed from all three factors?
d. Suppose instead you can only afford 8 of the design points from part (c). What fraction of the full design is this, and what would you call a design of this size?

Exercise 10.3. Reconsider the Computer Test and Repair Shop model of Section 7.1.3 (Chapter 7) that you built for Exercise 7.1. Choose two properties that a shop manager would plausibly want to adjust between runs

(for example, the number of repair technicians at the repair station and the mean of the diagnostic-time distribution) and expose them for generic access: annotate the technician count with `@set:KSLControl` and identify the KSL name of the diagnostic-time random variable's parameter. Write a program that (a) lists all of the model's control keys and random-variable parameter keys, (b) uses `Model.controls()` and `RVParameterSetter` to change both values, and (c) re-simulates and reports the change in average system time. Verify your change took effect by comparing the two configurations' printed output.

Exercise 10.4. Write an object that implements `ModelBuilderIfc` for the `PalletWorkCenter` model, such that each call to `build()` returns a completely independent `Model` instance configured with 2 workers and 30 replications. Demonstrate independence by building two model instances from your builder, changing the number of workers on one of them, and showing that the other is unaffected. In 3-4 sentences, explain why `Scenario` requires a `ModelBuilderIfc` rather than a single, already-built `Model` when it is going to be submitted to `ConcurrentScenarioRunner`.

Exercise 10.5. Reconsider Exercise 6.21, in which the Tie-Dye T-Shirt shop's order arrival rate was increased by 20, 40, 60, and 80 percent and the model was re-run for each case by hand. Instead of manually changing the arrival-rate parameter and re-running the model five times (baseline plus four increases), define the five configurations as a list of `Scenario` instances that override the appropriate arrival-process parameter key, and execute all five with a single `ScenarioRunner`. Reproduce the summary table from Exercise 6.21 (average time to produce an order and worker utilization, with half-widths) directly from the `ScenarioRunner`'s results and confirm your numbers match your earlier answer.

Exercise 10.6. Rebuild the five scenarios of Exercise 10.5 so that each uses a `ModelBuilderIfc` (see Exercise 10.4) rather than a shared `Model` instance, and run all five concurrently with `ConcurrentScenarioRunner`.

a. Run them first with the default stream assignment and confirm your results match Exercise 10.5 exactly.
b. Call `useIndependentRandomStreams()` and re-run.
c. Compare the half-widths of the average-time-to-produce-an-order estimates under the two stream policies, and explain which policy is more appropriate for this comparison and why.

Exercise 10.7. Reconsider Example 5.7 of Chapter 5, which compared the `PalletWorkCenter` model configured with one, two, and three workers by manually resetting `model.experimentName` and re-simulating.

a. Recreate the "One Worker," "Two Worker," and "Three Worker" configurations as `Scenario` instances using the `PWC.numWorkers` control key and execute them with `ScenarioRunner`. Confirm you reach the same MCB conclusion (three workers is best).
b. Extend the study by adding "Four Worker" and "Five Worker" scenarios. Using the runner's database and `multipleComparisonAnalyzerFor()` applied to `totalProcessingTime`, determine whether a fourth or fifth worker still meaningfully reduces total processing time, and recommend a staffing level, taking into account that each additional worker has a cost.

Exercise 10.8. Reconsider the 4-workstation system of Exercise 7.2, but give Workstation B two machines, since with one machine its queue grows without bound. Define two factors: the number of machines at Workstation A (levels 2 and 3) and the mean transfer time between stations (levels 2 and 4 minutes).

a. Build a `TwoLevelFactorialDesign` for these two factors and list its four design points, both coded and uncoded.
b. Map the two factors to the appropriate control keys in your Exercise 7.2 model and use `DesignedExperiment` to simulate all four design points with 20 replications each.
c. Using the resulting data frame, does adding a machine at Workstation A or reducing the transfer time have the larger effect on average system time? Is there evidence of interaction between the two factors?

Exercise 10.9. Reconsider the reorder-point, reorder-quantity inventory system of Exercise 7.13. Define two factors using `TwoLevelFactor`: `ReorderPoint` with low = 2 and high = 6, and `ReorderQty` with low = 2 and high = 8. Build a `TwoLevelFactorialDesign`, map the factors to the control keys of your Exercise 7.13 model, and use `DesignedExperiment` to simulate all four design points with 30 replications each (choose and justify a replication length that yields a reasonable number of demands per replication). Using `design.linearModel()` and `regressionResults()`, fit a model for total inventory-related cost (holding, ordering, and backorder cost) as a function of the two coded factors and their interaction. Which factor has the larger effect on cost? Is the interaction significant?

Exercise 10.10. Reconsider the queueing system used for the designed experiment example of Section 10.2.3, the single-server `GIGcQueue` model

with mean service time and mean time between arrivals exposed as controls `MM1_Test:ServiceTime.mean` and `MM1_Test:TBA.mean`.

a. Using the same `TwoLevelFactor` definitions given in Section 10.2.5 (`MeanST` with low = 2.0 and high = 4.0, and `MeanTBA` with low = 6.0 and high = 10.0), build the `CentralCompositeDesign` shown in that section, using `CentralCompositeDesign.rotatableAxialSpacing()` for the axial spacing, 10 replications for each factorial and axial point, and 20 replications for the center point. Print the design's coded and uncoded design points. How many *distinct* design points does this design have, and how many total simulation runs (across all replications) will `simulateAll()` execute?

b. For every distinct design point in part (a), compute the implied server utilization (mean service time divided by mean time between arrivals). Confirm that the highest utilization anywhere in the design occurs at one of the factorial corners, not at an axial point, and that every point stays below 1 (a stable queue).

c. Build the `GIGcQueue` model as described in Section 10.2.3, holding the number of servers fixed at 1, with 15 replications of length 10,000 minutes and a 5,000-minute warm-up. Using `DesignedExperiment`, simulate the design from part (a). Before you run it, explain what would go wrong if you passed `numRepsPerDesignPoint = 15` to the `simulateAll()` call instead of leaving it unspecified.

d. Fit two models to the `"System Time"` response: a first-order model with interactions (`LinearModel.Type.FirstAndSecond`, matching Section 10.2.4) and a second-order model that adds `.quadratic("MeanST")` and `.quadratic("MeanTBA")`. Report and compare the R^2 of the two models. Are the quadratic terms statistically significant? Based on your results, does system time in this system curve noticeably as utilization rises within the studied region, or is the first-order model with interactions adequate here?

e. Suppose instead you tried to build a `CentralCompositeDesign` directly on `ReorderPoint` and `ReorderQty` from Section 10.2.3, using the ranges shown there (`ReorderLevel` from 1 to 5, `ReorderQty` from 1 to 7). Compute, by hand, the coded and uncoded axial points for `ReorderQty`. What do you find, and why does this confirm the warning given at the start of Section 10.2.5?

Exercise 10.11. Reconsider the machine-shop system of Exercise 7.10. Define a four-factor, two-level design using the number of preparation machines, the number of units of one job-shop machine (for example, 1 or 2 units of machine 3), and two service-time levels of your choosing (state your low and high levels and justify them). Report the number of busy job-shop machines as one of the responses.

a. Build the full factorial design and state how many design points it has.
b. Using the run length from Exercise 7.10, estimate how long a sequential `DesignedExperiment` would take to simulate this design on your computer.
c. Rewrite the experiment using `ParallelDesignedExperiment` with a `ModelBuilderIfc` for the machine-shop model, and compare wall-clock time against part (b).
d. Which random-stream policy (`DesignPointRandomStreamPolicy.INDEPENDENT_RANDOM_STREAM` or `COMMON_RANDOM_NUMBERS`) is more appropriate for this design, and why?

Exercise 10.12. Reconsider Exercise 7.15, in which the JHBunt recruiting station at the STEM Career Fair Mixer was over-utilized (about 82%) and the MalWart station was under-utilized (about 50%), and you were asked

to design a schedule bringing both stations to about 80% utilization by trial and error. Instead, treat the number of recruiters scheduled at each station during the peak period as two factors (choose and justify reasonable low and high levels based on your Exercise 7.15 experience), and build a small factorial design. Using either `DesignedExperiment` or a hand-built list of `Scenario` objects run through `ScenarioRunner`, evaluate each staffing combination and recommend the one that comes closest to 80% utilization at both stations simultaneously. Discuss any combination(s) that meet the target and the labor-cost trade-offs among them.

Exercise 10.13. For each situation below, state whether you would use (i) a small number of individually specified `Scenario` objects run through `ScenarioRunner`/`ConcurrentScenarioRunner`, or (ii) a formal experimental design (`FactorialDesign`, `TwoLevelFactorialDesign`, or `CentralCompositeDesign`) run through `DesignedExperiment`/`ParallelDesignedExperiment`, and justify each choice in 1-2 sentences.

a. Comparing three specific, named vendor-proposed staffing plans for a call center.
b. Understanding how four continuous input parameters jointly affect a factory's throughput, including possible interactions and curvature.
c. Screening six candidate warehouse layouts, each requiring a substantively different model structure, to identify the best one or two for further study.
d. Determining the reorder point and reorder quantity that minimize cost for an inventory system, when you suspect the cost surface is curved near its minimum.

---

11. Simulation Optimization Methods

Learning Objectives

- To be able to explain how optimization methods can be applied within a simulation context
- To be able to discuss the challenges associated with applying optimization methods to simulation models
- To be able to recognize and explain some common simulation optimization algorithms
- To be able to apply KSL constructs for simulation optimization

Section 11.1 of the chapter presents an approach to exploring the design space that is driven by optimal search methods. Specifically, the section illustrates standard search methods such as stochastic hill climbing, simulated annealing, and the cross-entropy method. An important aspect of applying these approaches to "optimize" a simulation model is the fact that the output from the simulation model has uncertainty. The uncertainty of the objective response or in constraints will require additional understanding of the effect of uncertainty on the search process and on the interpretation of the results from the search. Unlike the approach based on experimental designs, a *simulation optimization* approach is driven via an algorithm that may or may not have guarantees on the quality of the solution produced.

> **Note**
>
> This chapter provides example code of using the KSL to implement experimental design and optimization techniques. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

11.1. Simulation Optimization Methods

The field of simulation optimization encompasses the use of computer simulation models as part of an optimization search process. As was discussed in Chapter 1, simulation models take on numerous forms including deterministic/stochastic, static/dynamic, and continuous/discrete models. Each of these forms may require specialized optimization methods in order to utilize the simulation model as part of the search process. Consider Figure 11.1 within the context of optimization, we can visualize how a simulation model (the box labeled "Model for Predicting System Performance" in the figure), can be used as part of an optimization process.

The dark-shaded area of Figure 11.1 represents an optimization model. In general form, an optimization model has the ability to generate candidate alternatives which are evaluated by the simulation model. The output from the simulation model is used by some kind of "evaluation model", typically an objective function, but the evaluation may in fact constitute multiple objectives. The optimization model is an iterative process that continues generating alternatives for evaluation and evaluating the alternatives until the desire solution quality is achieved. In Chapter 5 and in Section 10.2, the decision maker or analyst is the "optimization model". That is, it is up to the analyst to determine the set of alternatives to evaluate and to make conclusions about which are to be recommended. This section will describe

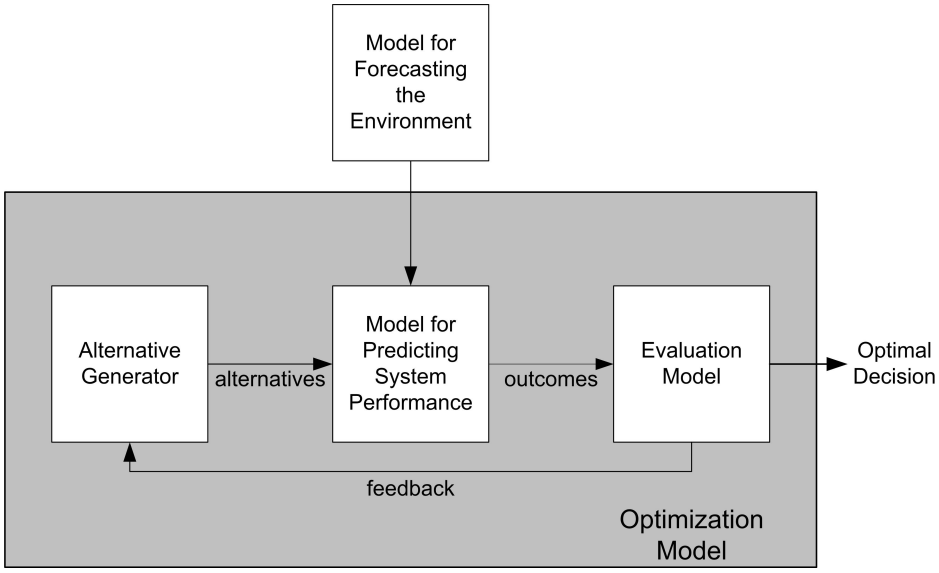


Figure 11.1.: Simulation as Part of an Optimization Process

processes for which the optimization model presented in Figure 11.1 is automated in some fashion via algorithms.

From this context, we can conclude that simulation optimization has substantial areas that can be extremely complex, theoretical, yet with many practical applications. Thus, for the purposes of this chapter, we will not attempt to present a discussion of the wide-range of algorithms and topics that underpin the area of simulation optimization. Excellent tutorials already exist on this extensive topic. We refer the interested reader to (Jian and Henderson 2015), (Pasupathy and Ghosh 2013), (Nelson 2014), and (Spall 2012) for introductions. In addition, the focus of this section is on the practical concepts that you need, as a user of such algorithms, to understand some of the trade-offs and issues that occur when using them.

While some algorithms offer theoretical proofs of convergence, that issue is by-passed within this presentation. The focus here is how to use the algorithm to obtain useful solutions within a reasonable time-frame. In this regard, the concepts presented in (Henderson 2021) are relevant because we are interested in algorithms that are well equipped to tackle problems with little structure and which provide solutions that improve an objective function over practical time-scales. In general, we desire algorithms that are general, relatively easy to use or setup, and can be applied in a variety of contexts.

With respect to this chapter, the general form of a simulation optimization problem can be written as:

$$\min_{x \in S} g(x)$$

where g is an objective function, x represents the decision variables, with constraints $x \in S$. The objective function can be linear or non-linear and the decision variables may be continuous or discrete. The objective function and/or constraints may involve randomness and one (or both) require a simulation model for evaluation. That is, the simulation model is like a

"black-box" to which we can provide inputs and then observe outputs for use in the optimization process.

Because our use case primarily involves systems that include randomness, a common method is to present the formulation in terms of expected values, where the objective function g and the constraint set S is written as:

$$g(x) = E[g(x, \xi_i)]$$
$$S = \{x : E[h(x, \xi_i)] \leq 0\}$$

where ξ_i represents the randomness associated with the i^{th} observation (sample) for the objective function, $g(\cdot)$ and $h(\cdot)$ represents a performance measure, both, estimated from the simulation. In addition, we must note that while the formulation does not indicate this explicitly, the inputs, x, are vector-valued and the responses $h(\cdot)$ may also be vectors. A wide variety of formulations are also seen in the literature. See (Jian and Henderson 2015) and (Pasupathy and Ghosh 2013) for further characterizations. Thus, the general formulation is quite challenging, and even more so because uncertainty (randomness) is involved.

The KSL provides a framework of classes that permit the construction of problem instances, which can be solved via different optimization algorithms as illustrated in Figure 11.1. The KSL's `ksl.simopt` package contains functionality to implement and use simulation optimization algorithms for KSL models. The package has the following sub-packages:

- `ksl.simopt.problem` This package has classes for defining optimization problem instances. The functionality includes specifying the objective function, the decision variables, and the constraints.
- `ksl.simopt.evaluator` This package has classes that provide a well-defined interaction protocol with a simulation model. That is, the package defines a general contract for what constitutes the inputs and outputs from the model. This allows responses to be translated to solutions for problem instances that can be used by algorithms.

- `ksl.simopt.solvers` This package provides the algorithms and general functionality for iterative optimization methods. The KSL denotes these simulation optimization algorithms as *solvers*.
- `ksl.simopt.cache` This package provides basic functionality to cache (in memory) executions of simulation models. Because the execution of a simulation model may be a long-running task and algorithms may visit design points multiple times during the search process, the caching of solutions may be useful. This package defines general interfaces for caching solutions and simulation runs.

The following sections present the KSL constructs found in the `ksl.simopt` packages for simulation optimization and illustrate their application.

11.2. Defining Simulation Optimization Problems

This section describes the `ksl.simopt.problem` package. The main purpose of this package is to define an optimization problem (decision variables, objective function, and constraints) and to connect the problem with the inputs and outputs of a simulation model. The most important class within this package is the `ProblemDefinition` class. The `ProblemDefinition` class requires an identifier for the model, a string that represents the name of the response variable in the model that represents the objective function, the names of any response variables used in defining response related constraints, the names of the model variables that represent the inputs (decision variables), and the type of optimization (minimize or maximize). Let I represent the set of input variables, let x_i represent the i^{th} input variable, and $\vec{x}$ represent a member of I. Let $G(\vec{x})$ be a random variable representing the objective function response from the simulation at input variable settings $\vec{x}$. Let L represent the set of deterministic linear constraints. Let F represent the set of deterministic functional constraints with members $f_i(\vec{x})$. Let R represent the set of constraints based on simulation responses with $R_i(\vec{x})$ representing a random response from the

simulation model as a function of the input variables. Finally, let l_i and u_i represent known bounds on input variable x_i.

An instance of the `ProblemDefinition` class represents the following mathematical program:

$$\begin{aligned}
&\min g(\vec{x}) = E[G(\vec{x})] \\
&\sum_{i=1}^{|I|} a_{ij} x_i \leq b_j \quad \text{for} \quad j = 1, \ldots, |L| \\
&f_i(\vec{x}) \leq c_i \quad \text{for} \quad i = 1, \ldots, |F| \\
&E[R_i(\vec{x})] \leq r_i \quad \text{for} \quad i = 1, \ldots, |R| \\
&l_i \leq x_i \leq u_i \quad \text{for} \quad i = 1, \ldots, |I|
\end{aligned}$$

Thus, the problems represented by the `ProblemDefinition` class may contain deterministic linear and functional constraints and constraints that are based on observed responses. We call the latter *response* constraints. The input variables must be specified to be limited within some *finite* bounds. For the purposes of the problem, the input range bounds provide the largest space of interest that define the search bounds. These bounds are finite to impose some limits on the search and to facilitate the generation of input values within the search space.

As part of building the problem definition, you specify the objective function. The value of the objective function, $G(\vec{x})$, should be a **response from the simulation model**. Thus, it is up to you to implement the simulation model so that it returns $G(\vec{x})$. Also, note that the mathematical formulation is in terms of a minimization problem. Thus, if the actual mathematical objective is for maximization: $\max g(\vec{x}) = E[G(\vec{x})]$, then you should specify the type of the problem as maximization. The problem definition class will translate the problem into a minimization problem such that objective function is the negative of $G(\vec{x})$.

$$\max_{x \in S} g(x) = \min_{x \in S} -g(x)$$

The constraints can be specified in terms of the type of inequality (less than or greater than). Figure 11.2 presents the constructor and properties for the `ProblemDefinition` class.

Figure 11.2.: Problem Definition Properties

Let's take a closer look at the constructor for the `ProblemDefinition` class.

```
class ProblemDefinition @JvmOverloads constructor(
    problemName: String? = null,
    val modelIdentifier: String,
    val objFnResponseName: String,
```

```
    inputNames: List<String>,
    responseNames: List<String> = emptyList(),
    val optimizationType: OptimizationType =
     ↪  OptimizationType.MINIMIZE,
    indifferenceZoneParameter: Double = 0.0,
    objFnGranularity: Double = 0.0
) : IdentityIfc by Identity(problemName)
```

The constructor requires a string `modelIdentifier` that represents a unique identifier for the model associated with the optimization problem. This identifier will be used to ensure that the simulation evaluation process connects the requests for simulation evaluations with the correct model.

We do not specify a functional form for the objective function, $G(\vec{x})$. Instead, the problem definition must specify a valid name for the simulation response variable that represents the objective function via the `objFnResponseName` property. Similarly, the problem definition requires the names of the response variables associated with the response constraints in the form of the `responseNames` list. The type of optimization problem must be specified. Finally, the list of strings, `inputNames` represents the named input (decision) variables related to the model. This list must not be empty. If the list has repeated elements, they are removed. We will discuss how to specify these names when an example is presented. The `indifferenceZoneParameter` parameter represents the smallest actual difference that is important to detect for the objective function response. This parameter can be used by solvers to determine if differences between solutions are considered practically significant. We will discuss the concept of granularity in Section 11.2.2.

Figure 11.3 presents the functions for the `ProblemDefinition` class. This functionality includes:

- adding/defining/validating input variables
- adding/defining linear constraints

- adding/defining functional constraints
- adding/defining response constraints
- randomly generating feasible points in the input space
- converting arrays of variables to valid input variables
- checking for different feasibility conditions and computing constraint violations
- specifying a penalty function for translating a constrained problem to an unconstrained problem
- representing neighborhoods for the problem

Figure 11.3.: Problem Definition Functions

11.2.1. Specifying Penalty Functions

Many algorithms utilize penalty functions to address constraint violations within the search process. Penalty functions are a variation on the Lagrangian approach of converting a constrained optimization into an unconstrained optimization problem. Recall the standardized minimization representation:

$$
\begin{aligned}
&\min g(\vec{x}) = E[G(\vec{x})] \\
&\sum_{i=1}^{|I|} a_{ij} x_i \leq b_j \quad \text{for} \quad j = 1, \ldots, |L| \\
&f_i(\vec{x}) \leq c_i \quad \text{for} \quad i = 1, \ldots, |F| \\
&E[R_i(\vec{x})] \leq r_i \quad \text{for} \quad i = 1, \ldots, |R| \\
&l_i \leq x_i \leq u_i \quad \text{for} \quad i = 1, \ldots, |I|
\end{aligned}
$$

We see that each constraint is presented as a less than or equal to constraint. Let's call the left-hand side of any constraint, LHS_k, and the right-hand side RHS_k, where k represents one of the linear, functional or response constraints. A violation of the constraint occurs if $LHS_k > RHS_k$. If this occurs, then we have a violation, and the amount of the violation is the difference, $LHS_k - RHS_k$. Let v_k represent the violation associated with constraint k, where $v_k = \max(0, LHS_k - RHS_k)$. Let K represent the total number of constraints. That is $K = |L| + |F| + |R|$. For the purposes of the penalty function, we ignore the lower and upper bound constraints. If you need to include a bound type constraint within the penalty function processing, then you can easily do that by using a linear constraint.

There are many possible approaches to defining a penalty function. To facilitate different kinds of penalty functions, the `ksl.simopt.problem` package provides an abstract `PenaltyFunction` class. A penalty function is *bound* to the one constraint it serves and computes its contribution directly from a solution:

```
abstract class PenaltyFunction(val constraint:
 ↪  PenalizableConstraint?) {

    /** Returns a copy of this penalty bound to the supplied
     ↪  constraint. */
    abstract fun boundTo(c: PenalizableConstraint):
     ↪  PenaltyFunction

    /** The penalty contribution for this penalty's
     ↪  constraint at the supplied solution. */
    abstract fun penalty(solution: Solution): Double
}
```

Both the `ConstraintIfc` interface implemented for (linear and functional constraints) and `ResponseConstraint` implement a small shared `PenalizableConstraint` capability, reporting a non-negative violation magnitude for a solution and a stable identity key, which is what lets a single `PenaltyFunction` serve either kind of constraint. A penalty function you write or configure is typically an *unbound template* (its `constraint` property is `null`); the `ProblemDefinition` class resolves each constraint's effective template and its own override if supplied; otherwise, the problem's type-level default is used as a bound copy the first time it is needed, and caches the result.

The package provides two kinds of built-in penalty functions, distinguished by whether they remember anything about past visits to a solution.

The `DynamicPolynomialPenalty` class represents a memory-less function and is the default for all three constraint categories:

$$P(v_k, t) = (C \cdot t^{\beta}) \cdot v_k^{\alpha}$$

where t is the solution's evaluation number and C (`basePenalty`), β (`iterationExponent`), and α (`violationExponent`) are constructor parameters, defaulting to $C = 100$, $\beta = 1$, and $\alpha = 1$. With these defaults, this is the "naive" penalty of (Park and Kim 2015): an increasing sequence multiplying the raw violation.

> **Warning**
>
> The violation exponent α defaults to 1, not 2. Squaring a violation that is naturally a fraction (a fill-rate gap of 0.02, say) makes the penalty *smaller*, not larger, which is rarely what you want. Raise α above 1 only for violations that are naturally on the order of 1 or larger.

The `ParkKimPenalty` class additionally remembers, for each solution and response constraint, how infeasible that solution has looked across every visit the solver has made to it. Each visit contributes a standardized measure of how far that visit's estimated response fell from the constraint's right-hand side; folding a new visit in updates a running average S across all of a solution's visits, positive when the solution looks infeasible on average. S in turn drives a penalty-sequence value λ: λ appreciates (multiplies by a factor $a > 1$) when S indicates infeasibility and depreciates (multiplies by a factor $d \in (0, 1)$) when S indicates feasibility. The penalty actually charged uses only the current S and λ:

$$P = \lambda \cdot [S]^{+}, \qquad [S]^{+} = \max(0, S)$$

(Park and Kim 2015) give the full derivation of S and prove that a sequence satisfying their Condition 1, implies diverging for an infeasible constraint and vanishing for a feasible one, makes the penalized objective converge to the true constrained optimum. The `AppreciateDepreciateSequence` class represents the KSL's implementation of their foundational appreciate/depreciate rule. A memoryful penalty is recognizable by overriding

`usesMemory` to return `true`, and by overriding a third hook, `foldVisit()`, that folds each new visit's observations into a constraint's running memory; the sequence rule itself is pluggable via the `PenaltySequence` interface, so more refined sequences from the literature could be added later without changing how penalties are computed or folded.

Because it needs a visit history, `ParkKimPenalty` falls back to a memory-less penalty (`DynamicPolynomialPenalty` by default) until a solution has been visited more than once. It is opt-in, not a default:

```
problemDefinition.defaultResponsePenalty = ParkKimPenalty(
    sequence =
 ↪  AppreciateDepreciateSequence(appreciationFactor = 2.0,
 ↪  depreciationFactor = 0.5)
)
```

The same `penaltyFunction` argument accepted by `responseConstraint()` can instead attach a `ParkKimPenalty` to one specific response constraint rather than every response constraint in the problem.

> **Warning**
>
> `ParkKimPenalty` only has something to remember if the solver actually revisits the same design point more than once. Many solvers evaluate each candidate point once and move on, in which case `ParkKimPenalty` behaves exactly like its memory-less fallback. It is most useful paired with a solver and replication policy that deliberately re-samples promising points with a growing number of replications.

`ProblemDefinition` exposes one default penalty function per constraint category, each independently settable:

```
var defaultLinearPenalty: PenaltyFunction =
 ↪  DynamicPolynomialPenalty.defaultPenaltyFunction
var defaultFunctionalPenalty: PenaltyFunction =
 ↪  DynamicPolynomialPenalty.defaultPenaltyFunction
var defaultResponsePenalty: PenaltyFunction =
 ↪  DynamicPolynomialPenalty.defaultPenaltyFunction
```

Any individual linear or response constraint can override its category's default by supplying a `penaltyFunction` argument when it is created:

```
fun linearConstraint(
    equation: Map<String, Double>,
    rhsValue: Double = 0.0,
    inequalityType: InequalityType =
     ↪  InequalityType.LESS_THAN,
    penaltyFunction: PenaltyFunction? = null
): LinearConstraint

fun responseConstraint(
    name: String,
    rhsValue: Double,
    inequalityType: InequalityType =
     ↪  InequalityType.LESS_THAN,
    target: Double = 0.0,
    tolerance: Double = 0.0,
    penaltyFunction: PenaltyFunction? = null
): ResponseConstraint
```

> **Warning**
>
> `functionalConstraint()` does not accept a `penaltyFunction` argument; every functional constraint in a problem uses

> `defaultFunctionalPenalty`. If a functional constraint needs a different penalty, change `defaultFunctionalPenalty` for the problem. Keep in mind that this changes the penalty for every functional constraint in that problem, not just one.

`ProblemDefinition.penaltyFncValue(solution: Solution)` sums the bound penalty functions' contributions across every constraint:

```
fun penaltyFncValue(solution: Solution): Double {
    var totalPenalty = 0.0
    val bound = boundPenalties()
    for (lc in linearConstraints) totalPenalty +=
     ↪  bound.getValue(lc).penalty(solution)
    for (fc in functionalConstraints) totalPenalty +=
     ↪  bound.getValue(fc).penalty(solution)
    for (rc in responseConstraints) totalPenalty +=
     ↪  bound.getValue(rc).penalty(solution)
    return totalPenalty
}
```

The returned value is always non-negative and is *not* adjusted for the direction of optimization. Internally, `ProblemDefinition` always works with a minimization-oriented objective where `penalizedObjFncValue = objFncValue + penaltyFncValue(solution)`. Thus, a constraint violation must always make the penalized value worse, whether the underlying problem minimizes or maximizes.

For a `ParkKimPenalty` bound constraint, `penalty(solution)` only *reads* the memory already carried on the solution. It does not update it. New observations are folded into a solution's memory exactly once, when the simulation oracle actually produces them: after each evaluation, the `Evaluator` calls `ProblemDefinition.foldPenaltyMemory()`, which asks every memoryful bound penalty to fold that visit's new observations in, and the result

is carried on the resulting `Solution`. Scoring the same solution again later, for example, when re-ranking solutions pulled from a cache, does not, by itself, add another visit.

`ProblemDefinition` also provides functions that compute raw violations directly, independent of any penalty function. These are useful for reporting, or for writing your own penalty logic:

- `linearConstraintViolations(inputs: Map<String, Double>): DoubleArray`
- `totalLinearConstrainViolations(inputs: Map<String, Double>): Double`
- `functionalConstraintViolations(inputs: Map<String, Double>): DoubleArray`
- `totalFunctionalConstraintViolations(inputs: Map<String, Double>): Double`
- `responseConstraintViolations(averages: Map<String, Double>): Map<String, Double>`
- `totalResponseConstraintViolations(averages: Map<String, Double>): Double`

If you need a different penalty structure altogether, you can implement `PenaltyFunction` directly and assign it as one of `ProblemDefinition`'s three category-level defaults, or as a `penaltyFunction` argument on an individual linear or response constraint.

11.2.2. Defining Input Variables

For optimization problems, the decision variables are the key quantities that need to be determined. For a `ProblemDefinition` instance, the `InputDefinition` class, illustrated in Figure 11.4 is used to define the decision variables.

The constructor for defining inputs is as follows:

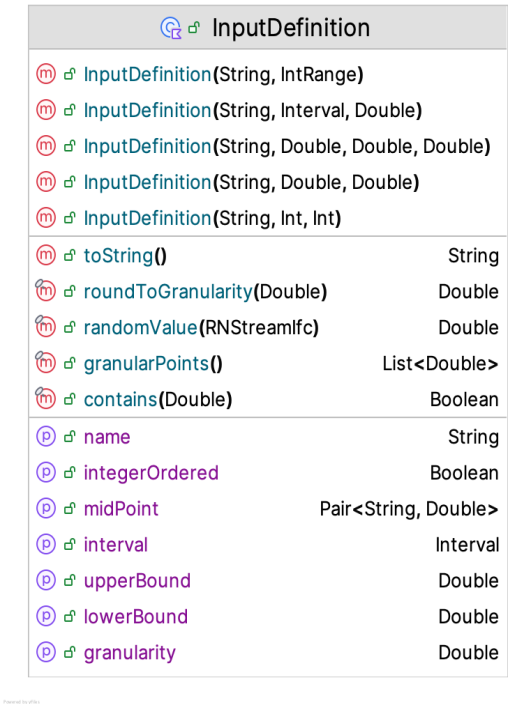


Figure 11.4.: The InputDefinition Class

```
class InputDefinition @JvmOverloads constructor(
    val name: String,
    val lowerBound: Double,
    val upperBound: Double,
    granularity: Double = 0.0
)
```

You provide the name of the input and its bounds. The name must match one of the input names within the problem definition instance.

Finally, there is the `granularity` parameter. The specified granularity indicates the acceptable precision for the variable's value with respect to decision-making. If the granularity is 0 then no rounding will be applied when evaluating the variable. Granularity defines the level of precision for an input variable to which the problem will be solved. Setting granularity to 0, the default, means that the solver will attempt to find a solution to the level of machine precision. For any positive granularity value, the solution will be found to some multiple of that granularity. For example, setting granularity to 1 implies an integer-ordered input variable. The specification of granularity reflects a reality for the decision maker that there is a level of precision beyond which it is not practical to implement a solution.

Granularity represents the finest division of the measurement scale. For example, a 12-inch rule that has inches divided into 4 quarters has a granularity of 1/4 or 0.25. The granularity parameter specifies the degree of this precision so that a function can be applied which rounds the supplied value to the nearest multiple of the granularity. Suppose, we had a quarter inch granularity, the rounding process would round 3.1459 to 3.25 because 3.25 is the nearest multiple of the granularity of (0.25). For the case of 3.0459 with a granularity of 0.25, the value of 3.0459 would be rounded to 3.0. Thus, with a quarter inch granularity we can only have values that are multiples of 0.25. That is, granularity permits a discretization of a continuous variable to points that are realistic with

respect to the measurement scale associated with the variable. Thus, continuous variables can be handled as discrete variables if necessary. This functionality recognizes that any variable represented on a digital computer is a discrete representation (even to machine precision). For many realistic problems, from a practical standpoint, we cannot implement decision variables beyond some level of granularity.

If an input variable is defined with a granularity of 1.0, then it is considered integer-ordered. If an optimization problem has only integer-ordered decision variables, then the problem is considered integer-ordered. Some solvers are constructed to specifically handle this kind of problem. The R-SPLINE solver discussed in (Wang et al. 2013) is an integer-ordered solver and has been implemented within the KSL.

11.2.3. Linear and Functional Constraints

The specification of constraints is similar to defining input variables. The `ProblemDefinition` class provides functions that allow the specification of each type of constraint: linear, functional, and response. The `LinearConstraint` class indicates how to compute a linear constraint. Figure 11.5 presents the `ConstraintIfc` interface with two implementations in the `LinearConstraint` and `FunctionalConstraint` classes.

The `LinearConstraint` class takes in a map that provides the coefficient a_i associated with each decision variable x_i. Because of the linear form of the equation, this representation is sufficient to define the linear equation. In addition, the right-hand side variable b_j and the form of the inequality is specified. The class will translate a greater than constraint to the mathematically equivalent less than constraint.

```
data class LinearConstraint(
    val equation: Map<String, Double>,
    override var rhsValue: Double = 0.0,
    override val inequalityType: InequalityType =
     ↪  InequalityType.LESS_THAN
```

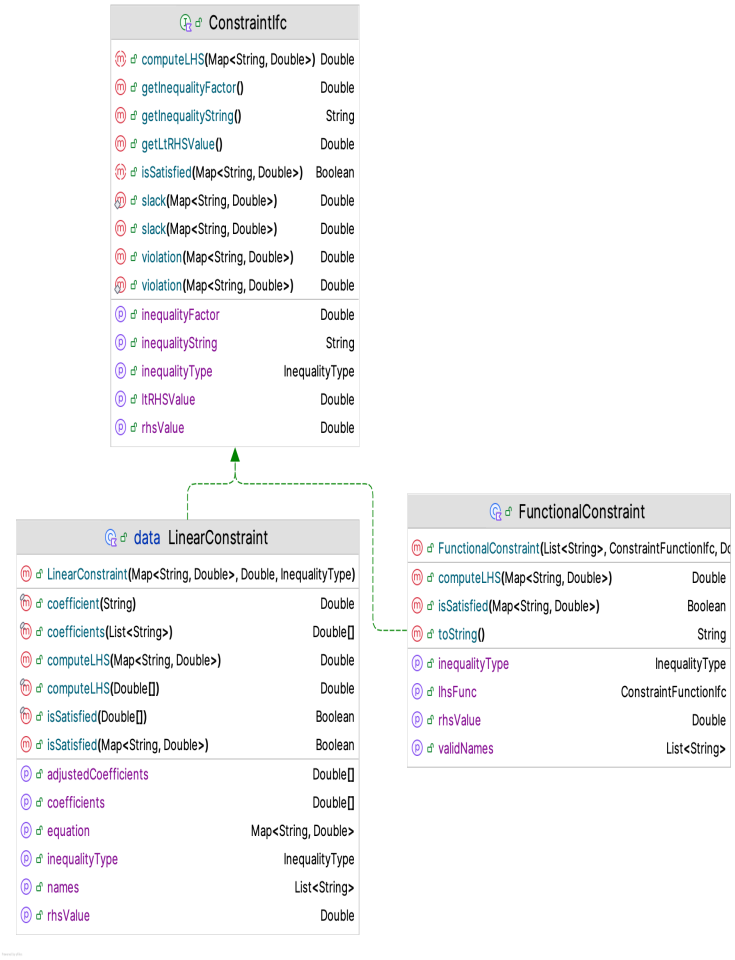


Figure 11.5.: Deterministic Constraints

```
) : ConstraintIfc
```

As noted in Figure 11.5, deterministic constraints can compute the left-hand side of the constraint and compare it with the right-hand side to determine if the constraint is satisfied, whether there is slack in the constraint, and the amount of the constraint's violation.

As shown in the `FunctionalConstraint` constructor, the constraint is defined as a function of the input parameters.

```
class FunctionalConstraint(
    validNames: List<String>,
    val lhsFunc: ConstraintFunctionIfc,
    override val rhsValue: Double = 0.0,
    override val inequalityType: InequalityType =
     ↪  InequalityType.LESS_THAN
) : ConstraintIfc {
```

The `ConstraintFunctionIfc` interface is a functional interface. It simply takes in a map containing the named input variables and their values and returns the left-hand side of the constraint. Thus, you can specify any functional form by implementing this interface.

```
fun interface ConstraintFunctionIfc {

    fun lhs(inputs: Map<String, Double>): Double

}
```

11.2.4. Response Constraints

The final type of constraint is the *response* constraint. Response constraints are different from deterministic constraints because these constraints rely on the execution of the simulation model for the observation of their values. These are very difficult constraints within a mathematical programming context. Figure 11.6 provides the KSL representation for modeling response constraints.

A response constraint requires the name of a valid simulation response variable, the right-hand side value, and the type of inequality. In addition, some solvers may utilize information about a constraint in the form of a target and a tolerance. The target parameter is used by some algorithms to serve as a cut-off point between desirable and unacceptable solutions with respect to the constraint. The tolerance parameter is used by some algorithms to specify how much the decision maker is willing to be off from the target and is conceptually similar to an indifference parameter.

```
class ResponseConstraint(
    val responseName: String,
    val rhsValue: Double,
    val inequalityType: InequalityType =
     ↪  InequalityType.LESS_THAN,
    val target: Double = 0.0,
    val tolerance: Double = 0.0
) {
```

Response constraints will also compute the slack and violation amounts. The default implementation is to not take into account randomness in these calculations. That is, the estimated response value is treated as "deterministic" when computing the violation. Recognizing that feasibility checking may be more complex with these stochastic constraints, the `ResponseConstraint` class permits you to supply an instance of the `FeasibilityCheckerIfc` interface.

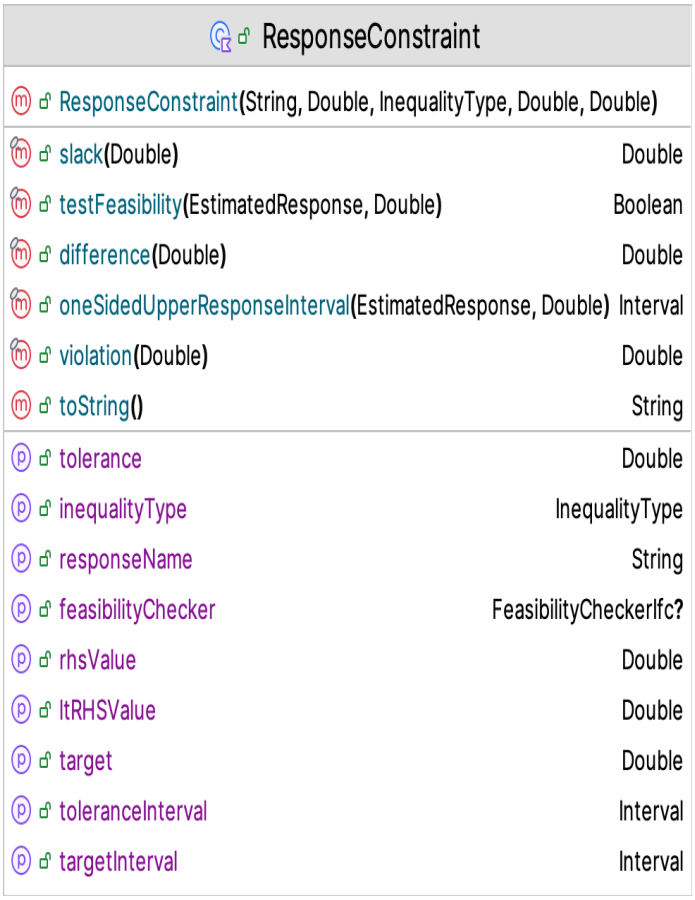


Figure 11.6.: Response Constraints

```
fun interface FeasibilityCheckerIfc {

    fun isFeasible(
        responseConstraint: ResponseConstraint,
        estimatedResponse: EstimatedResponse,
        confidenceLevel: Double
    ): Boolean
}
```

The `FeasibilityCheckerIfc` interface allows for the checking of feasibility based on some statistical confidence. If you supply an instance of this interface, then it will be used when testing for feasibility for a response constraint. Now, let's take a look at an example to illustrate how to specify a problem.

11.2.5. Setting Up a Problem Definition

Example 11.1 (Reorder Point, Reorder Quantity Inventory Model)**.** An inventory manager is interested in understanding the cost and service trade-offs related to the inventory management of computer printers. Suppose customer demand occurs according to a Poisson process at a rate of 3.6 units per month and the lead time is 0.5 months. The manager has estimated that the holding cost for the item is approximately $0.25 per unit per month. In addition, when a back-order occurs, the estimate cost will be $1.75 per unit per month. Every time that an order is placed, it costs approximately $0.15 to prepare and process the order. The inventory manager wants to find the optimal setting for the reorder point and the reorder quantity with respect to the total ordering and holding cost while ensuring a fill rate of at least 0.95. Set up a KSL simulation optimization problem for this situation.

This is Example 7.9 from Chapter 7. To prepare to use the implemented model from Example 7.9, we need to ensure that controls are defined for the input variables and to note the names of the responses involved in the problem. The two decision variables are the reorder point and the reorder quantity. Within the `RQInventory` class, we have the following defined controls:

```
    @set:KSLControl(
        controlType = ControlType.INTEGER,
        lowerBound = 0.0
    )
    var initialReorderPoint: Int
        get() = myInitialReorderPt
        set(value) {
            setInitialPolicyParameters(value,
↪   myInitialReorderQty)
        }

    @set:KSLControl(
        controlType = ControlType.INTEGER,
        lowerBound = 1.0
    )
    var initialReorderQty: Int
        get() = myInitialReorderQty
        set(value) {
            setInitialPolicyParameters(myInitialReorderPt,
↪   value)
        }
```

Recall from Section 10.1.1 of Chapter 10 that the name of the control will be the model element's name concatenated with the name of the property separated by a "dot". In the `RQInventorySystem` class definition, we see that the instances of inventory use the name supplied for the parent with the string "Item" as shown in the following code.

```
private val inventory: RQInventory = RQInventory(
    this, reorderPt, reorderQty, replenisher = Warehouse(),
     ↪  name = "${this.name}:Item"
)
```

Thus, the names of the two controls will be the strings:

- "Inventory:Item.initialReorderQty"
- "Inventory:Item.initialReorderPoint"

The fill rate and total ordering and holding cost responses have similar naming conventions resulting in these strings:

- "Inventory:Item:FillRate"
- "Inventory:Item:OrderingAndHoldingCost"

These names are essential for constructing the problem definition. Section 10.1.1 also describes how to get the names of all controls within a model.

Finally, we need to know the model identifier. By default, this will be the assigned name of the model; however, in general you can assign it. The basic code for building the reorder point, reorder quantity inventory model is as follows:

```
val reorderQty: Int = 2
val reorderPoint: Int = 1
val model = Model("RQInventoryModel")
val rqModel = RQInventorySystem(model, reorderPoint,
 ↪  reorderQty, "Inventory")
rqModel.initialOnHand = 0
rqModel.demandGenerator.setInitialEventTimeProcesses(
 ↪  ExponentialRV(1.0 / 3.6))
rqModel.costPerOrder = 0.15
```

```
rqModel.unitHoldingCost = 0.25
rqModel.unitBackorderCost = 1.75
rqModel.leadTime.initialRandomSource = ConstantRV(0.5)
model.lengthOfReplication = 20000.0
model.lengthOfReplicationWarmUp = 10000.0
model.numberOfReplications = 40
```

Thus, the model identifier will be the string "RQInventoryModel". Now, the code for constructing the problem definition can be completed.

```
val problemDefinition = ProblemDefinition(
    problemName = "InventoryProblem",
    modelIdentifier = "RQInventoryModel",
    objFnResponseName =
 ↪  "Inventory:Item:OrderingAndHoldingCost",
    inputNames = listOf("Inventory:Item.initialReorderQty",
 ↪  "Inventory:Item.initialReorderPoint"),
    responseNames = listOf("Inventory:Item:FillRate")
)
problemDefinition.inputVariable(
    name = "Inventory:Item.initialReorderQty",
    interval = Interval(1.0, 100.0),
    granularity = 1.0
)
problemDefinition.inputVariable(
    name = "Inventory:Item.initialReorderPoint",
    interval = Interval(1.0, 100.0),
    granularity = 1.0
)
problemDefinition.responseConstraint(
    name = "Inventory:Item:FillRate",
    rhsValue = 0.95,
    inequalityType = InequalityType.GREATER_THAN
```

```
)
```

The constructor uses a name that you assign for the problem, the model identifier, the name of the response for the objective function, the names of the input (decision) variables, and the names of the responses. Note that the type of optimization was not specified. Thus, the type is minimization by default. Then, the two input variables representing the initial reorder point and initial reorder quantity are defined over search intervals. Notice that the granularity is set to 1.0, which indicates that this is an integer-ordered (discrete) problem. Finally, the fill rate constraint is added to indicate that desired solutions need to have the fill-rate response greater than 95 percent. The problem definition instance will be required when setting up the solver and the simulation evaluation process.

11.3. Representing the Simulation Oracle

The simulation *oracle* is a term used to represent the concept of the simulation model as a "black-box" that accepts inputs and returns outputs as illustrated in Figure 11.1. To be able to define general problems and have them solved by general algorithms, we need to establish a protocol to represent the inputs, the model, and the outputs for generation, use, and consumption by optimization solvers. The purpose of the `ksl.simopt.evaluator` package is to provide classes and interfaces that establish this interaction protocol.

Figure 11.7 presents three major interfaces which support the general execution of KSL simulation models. The `SimulationOracleIfc` interface provides behavior that receives evaluation requests (inputs) and translates those requests to simulation results. An implementer of the `SimulationProviderIfc` interface is a simulation oracle that has additional functionality to validate evaluation requests. A simulation provider is associated with a single simulation model. The `SimulationServiceIfc`

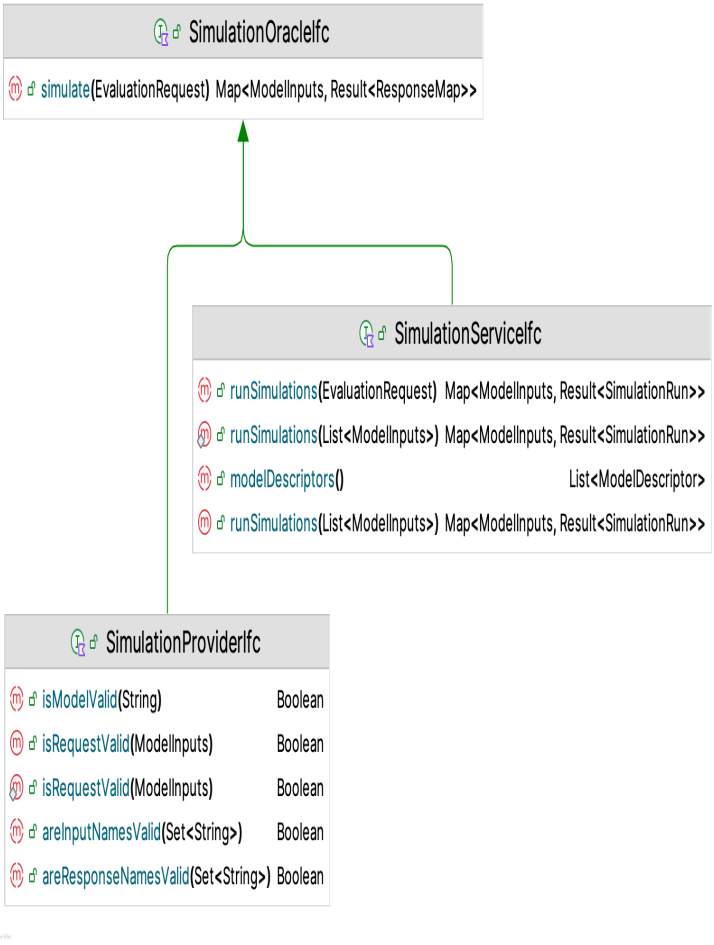


Figure 11.7.: Simulation Oracle Interfaces

interface also implements the `SimulationOracleIfc` interface and has the capability to execute simulation requests for multiple models.

11.3.1. Requesting Simulation Evaluations

We will focus on the simulation oracle and how it relates to the `EvaluatorIfc` interface shown in Figure 11.8.

As we will see in the section concerning solvers, solvers use implementations of the `EvaluatorIfc` interface to request simulation evaluations. As we can see in Figure 11.8, the constructor for the `Evaluator` class requires an instance of the `ProblemDefinition` class and an instance of an implementation of the `SimulationOracleIfc` interface. That is, implementations of the `EvaluatorIfc` interface translate simulation results into solutions that solvers can use. We will overview this process, starting with what it means to make a request for a simulation evaluation.

The process for requesting a simulation execution starts with an instance of the `EvaluationRequest` class. As shown in Figure 11.9, an evaluation request is a data class which holds model inputs and options for the simulation execution process.

An evaluation request represents many possible evaluations in the form of a list of model inputs. The evaluation request specifies the model via the model identifier, whether the evaluations should be run with common random numbers, and whether cached results can be used to satisfy the request. The option to run the evaluations with common random numbers is only valid if there are two or more requested evaluations (i.e. the list of model inputs has two or more elements). In addition, using cached results is not allowed if the request uses common random numbers. Thus, an evaluation request represents a set of simulation executions that will be handled by the simulation oracle. The evaluation requests are translated into a map of model inputs with the results within a map of responses. Let's take a closer examination of these two concepts.

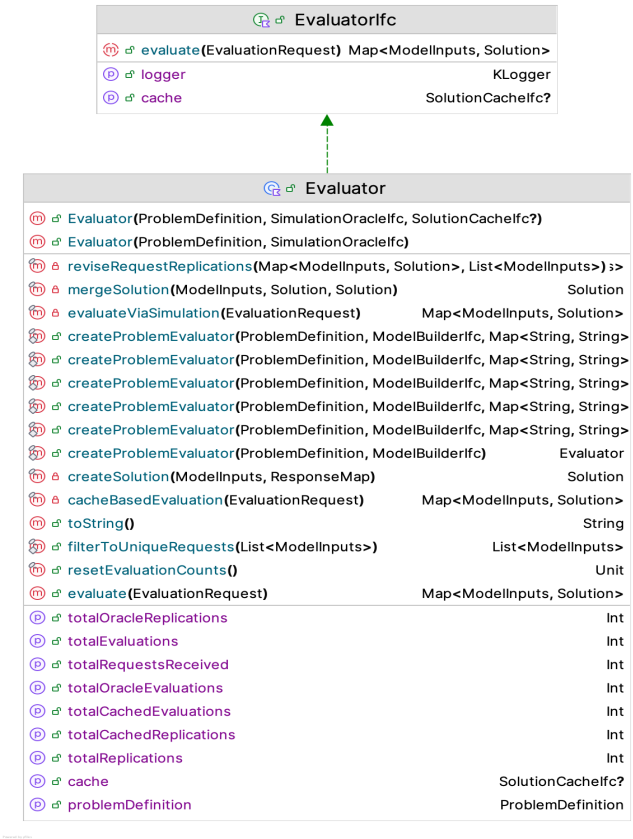


Figure 11.8.: Simulation Evaluator Interfaces

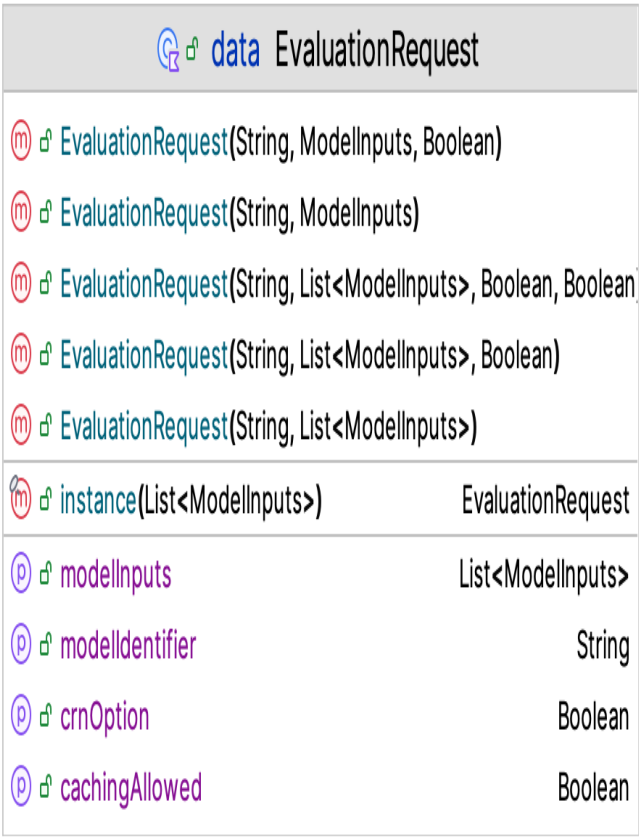


Figure 11.9.: Evaluation Requests

11.3.2. Representing Model Inputs

As noted in Figure 11.9, an evaluation request consists of a list of model inputs. A model input is a specification for the inputs that will be used for the simulation run and the outputs that are requested. Figure 11.10 shows the `ModelInputs` class.

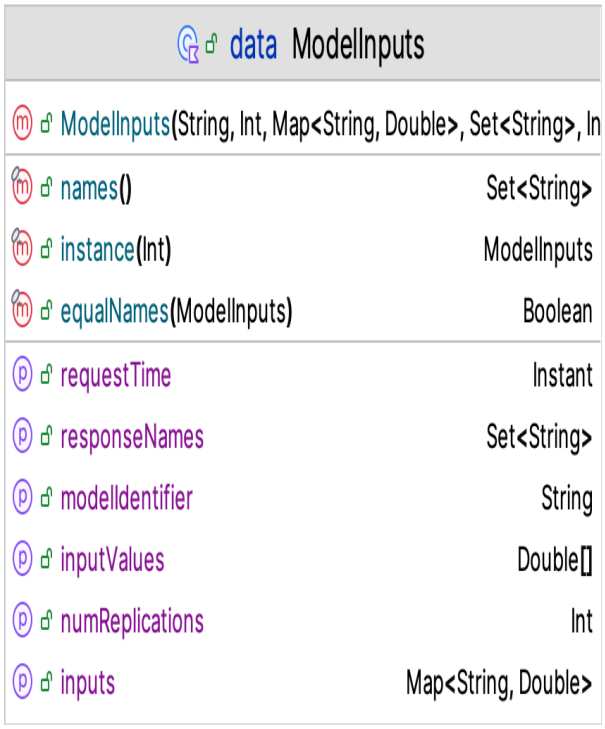


Figure 11.10.: Model Inputs

The `ModelInputs` class is also a Kotlin data class. As we can see in the

data class's constructor, model inputs require the model identifier, the number of replications for the simulation run, the inputs (specified as name, value pairs in a map), and the names of the responses required from the simulation run. If the inputs map is empty, the current settings of the simulation model are used for the simulation execution. If the set of response names is empty, then all responses from the simulation are returned.

```
data class ModelInputs(
    val modelIdentifier: String,
    val numReplications: Int,
    val inputs: Map<String, Double> = emptyMap(),
    val responseNames: Set<String> = emptySet(),
    val requestTime: Instant = Clock.System.now()
)
```

Model inputs have a very special representation for determining their equality. Two instances of the `ModelInputs` class are considered equal if:

1. the `modelIdentifier` properties are equal, and
2. the `responseNames` properties are equal (contain all the same response names), and
3. the `inputs` properties are equal (contain the same (key, value) pairs)

It is important to note that two instances will be considered equal even if their number of requested replications are **different**. For example, a request for 10 replications at a reorder point of 3 and a reorder quantity of 4 is equal to a request for 20 replications at the same inputs. This definition of equality allows results to be cached by model inputs.

So far, we have discussed the inputs to the `simulate()` function of the `SimulationOracleIfc` interface. Each model input in the evaluation request is paired with the requested responses in the form of a map. The `SimulationOracleIfc` interface prescribes that the responses are wrapped

within a Kotlin `Result` instance. A Kotlin `Result` indicates whether an outcome from a process was a success or a failure. Since simulation evaluations may be executed remotely via simulation services, there are many opportunities for failure to occur (e.g. network failure, service denial, etc.). A more likely scenario is that some execution error occurred during the simulation run. Thus, the user of the simulation oracle can react according to the success or failure of the request.

```
interface SimulationOracleIfc {

    fun simulate(evaluationRequest: EvaluationRequest):
     ↪  Map<ModelInputs, Result<ResponseMap>>

}
```

11.3.3. Representing Model Outputs

Now we can discuss the `ResponseMap` class. The `ResponseMap` class, illustrated in Figure 11.11, is also a Kotlin data class. The use of data classes for the mappings between inputs and outputs is an intentional part of the package design. The data classes are *serializable*, which allows them to be distributed via network calls.

The `ResponseMap` class is a map. In fact, it implements the `Map<String, EstimatedResponse>` interface. The string key for the map is the name of the response (as per the model and the `EvaluationRequest`) and the paired value is an instance of the `EstimatedResponse` class. The `EstimatedResponse` class, illustrated in Figure 11.12, holds basic summary statistics across the replications associated with the simulation run. The `ResponseMap` class has functionality that facilitates the reporting of statistics for its contained `EstimatedResponse` instances. In addition, the `ResponseMap` class supports the *merging* of the responses that it contains. This is important for the case of solvers that may revisit decision points

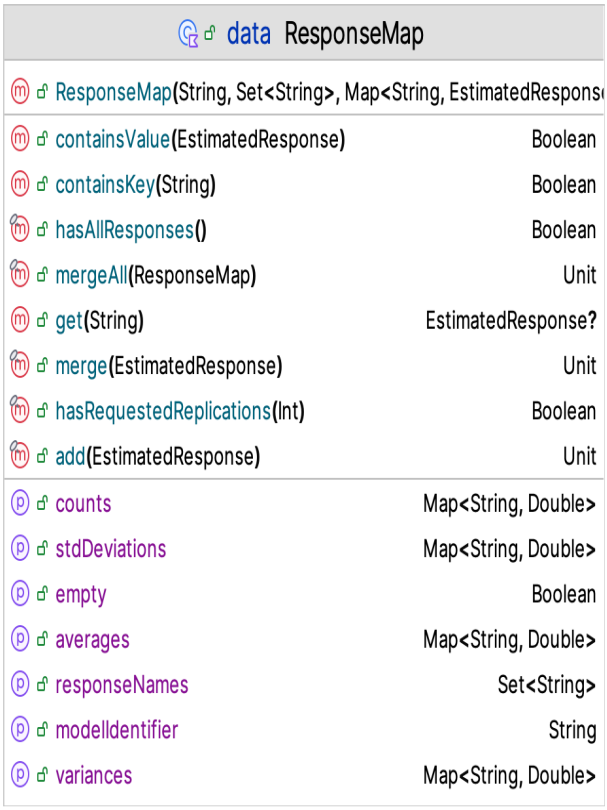


Figure 11.11.: The ResponseMap Class

with additional requests for replications. If caching is available, then the results from previous samples are merged with new samples so that the cached replications can partially fill the request.

Figure 11.12.: The EstimatedResponse Class

Suppose we have two estimates for the same response, with summary statistics $(\bar{x}_1,\ s_1^2,\ n_1)$ and $(\bar{x}_2,\ s_2^2,\ n_2)$. Merging of the two response estimates, creates a single estimated response $(\bar{x},\ s^2,\ n)$. We assume that the two estimates are from independent samples. The values for n and $\bar{x}$

are as follows:

$$n = n_1 + n_2$$
$$\bar{x} = \frac{n_1\bar{x}_1 + n_2\bar{x}_2}{n}$$

Thus, $\bar{x}$ is the weighted average of the estimates. The computation of s^2 follows the idea of a pooled variance. For the case where $n_1 > 2$ and $n_2 > 2$, we can compute the weighted average:

$$s^2 = \frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{n - 2}$$

The case where $n_1 = 1$ and $n_2 = 1$ can be handled easily by noting that this is the same as two independent observations. The cases where $n_1 = 1$ and $n_2 = 2$ or $n_1 = 2$ and $n_2 = 1$ can be handled by noting that the situation with $n_i = 1$ is just adding an additional observation to a statistical summary. The assumption that the estimates are independent is important when computing the variances. Thus, cached results are not used when common random numbers are used.

Within the context of an optimization problem, the algorithms require *solutions*. This brings up the final topic of this section: the translation of responses to solutions. As noted at the beginning of this section, solvers require the implementation of the `EvaluatorIfc` interface. As we can see from the definition of the interface, the interface provides a translation from evaluation requests to solutions. Every request to the simulation oracle for a set of inputs is translated to a map that contains the model inputs that resulted in the returned solution.

```
interface EvaluatorIfc {

    /**
     *  A possible cache to hold evaluated solutions
     */
```

```
    val cache: SolutionCacheIfc?

    /**
     *  Processes the supplied requests for solutions. The
↪   solutions may come from an associated
     *  solution cache (if present or allowed) or via
↪   evaluations by the simulation oracle.
     *  The CRN option is applied to the set of requests and
↪   does not permit
     *  cached solutions, even if caching is permitted.
     *
     *  @param evaluationRequest a request for evaluation
     *  @return a map containing the model inputs and
↪   resulting solutions as pairs
     */
    fun evaluate(evaluationRequest: EvaluationRequest):
     ↪  Map<ModelInputs, Solution>

}
```

Figure 11.13 presents the `Solution` class. Again, the `Solution` class is a Kotlin data class. It holds the inputs that caused the solution, the objective function, and a list of the estimated responses for the response constraints.

As we can see from the constructor, the `inputMap` property holds the inputs associated with the solution. The `estimatedObjFnc` property represents the estimated objective function value as an `EstimatedResponse` instance. The `responseEstimates` property hold the list of the response associated with the response constraints. The `evaluationNumber` is an integer that marks the current number of evaluations performed by the evaluator that produced the solution. In essence, calls for evaluation produce a sequence of solutions. The `isValid` property is used to indicate whether the solution

Figure 11.13.: The Solution Class

is valid. A solution will be invalid if there was some problem or exception that occurred during the evaluation process. Invalid solutions will also be constructed as *bad* solutions. Essentially, a bad solution is a solution that is infeasible and has the worst possible objective function value with respect to the problem instance.

```
data class Solution(
    val inputMap: InputMap,
    val estimatedObjFnc: EstimatedResponse,
    val responseEstimates: List<EstimatedResponse>,
    val evaluationNumber: Int,
    val isValid: Boolean = true
) : Comparable<Solution>, FeasibilityIfc by inputMap,
 ↪  EstimatedResponseIfc by estimatedObjFnc
```

As we can see from the constructor, solutions can be compared and their feasibility checked. Feasibility is established with respect to the problem instance. There are four types of feasibility modeled.

- Input range feasibility: If the input values are within the ranges defined for the input variable, then the solution is considered *input range* feasible.
- Linear constraint feasibility: If the input values satisfy all linear constraints, then the solution is considered feasible with respect to the linear constraints.
- Functional constraint feasibility: If the input values satisfy all functional constraints, then the solution is considered feasible with respect to the functional constraints.
- Input feasible: If the input values are input range feasible, linear constraint feasible, and functional constraint feasible, then the solution is considered *input feasible*.

Notice that feasibility ignores the issue of feasibility with respect to the response constraints. Essentially, input feasibility is checking the inputs.

It does not check the outputs (response). A solution implements the `EstimatedResponseIfc` interface based on the value of the estimated objective function response.

To summarize, the procedures defined in the `ksl.simopt.evaluator` package permit requests for simulation evaluations. The inputs to the simulation model are contained within the request. The simulation model is executed at the specified inputs and the requested results returned. For simulation optimization algorithms, the input-output mapping is encapsulated within instances of the `Solution` class. The `Solution` class contains information on the following:

- The inputs used to execute the simulation.
- The value of the objective function estimated via simulation, the penalized objective function value, and the constraint violations.
- The estimated responses involved in any response constraints.
- The ability to assess the feasibility of the solution with respect to the inputs and deterministic constraints.

The information from solutions is designed to be consumed by simulation optimization algorithms, which is the topic of the next section.

11.4. Characterizing Simulation Optimization Algorithms

The KSL `ksl.simopt.solvers` package provides a general framework for designing and implementing simulation optimization algorithms. For the purpose of this section, we call these algorithms, *solvers*. The `solver` package provides the `Solver` abstract base class that can be specialized to implement different algorithms. The KSL also provides a set of implemented algorithms for use on simulation optimization problems. Figure 11.14 illustrates the class hierarchy for existing solvers. The `Solver` class and the `StochasticSolver` classes are abstract base classes. Essentially, the

`Solver` class defines the basic properties and methods needed for solving an optimization problem. The `StochasticSolver` class assumes that the optimization process may incorporate random search and provides the solver access to random streams.

The constructors for the `Solver` class and the `StochasticSolver` class show that a solver must have a reference to the problem and an evaluator. In addition, solvers are conceptualized as iterative processes. Each iteration of a solver may evaluate solutions and move towards a "best" solution. Thus, solvers require a limit on the maximum number of iterations. As we will see, there will be more general ways to implement stopping conditions; however, every solver has a pre-specified maximum number of iterations permitted before it will stop.

Within the context of simulation optimization, the supplied evaluator promises to execute requests for evaluations of the simulation model at particular design points (as determined by the algorithm). In addition, because of the stochastic nature of the evaluation, the solver may request one or more replications for its evaluation requests. The number of replications may change dynamically. For example, a solver may use fewer replications early in the search and more replications as it approaches promising solutions. Thus, you need to supply a function to determine the number of replications per evaluation. Within the framework of the hooks for sub-classes, you could specify more complex procedures for determining the number of replications per evaluation. The constructor uses the `ReplicationPerEvaluationIfc` interface to specify the current number of replications for each evaluation request. The number of replications can vary as the solver proceeds. Solvers also *emit* solutions to permit tracking/tracing the solution process.

```
abstract class Solver(
    val problemDefinition: ProblemDefinition,
    evaluator: EvaluatorIfc,
    maximumIterations: Int,
```

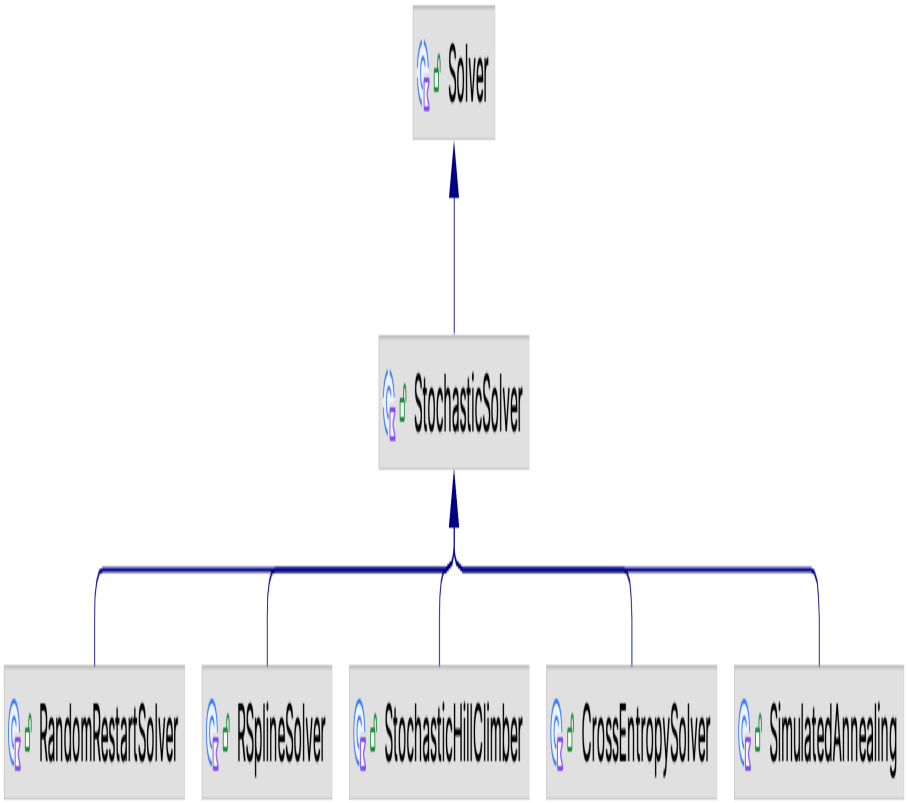


Figure 11.14.: The Solver Class Hierarchy

```
    var replicationsPerEvaluation:
     ↪  ReplicationPerEvaluationIfc,
    name: String? = null
) : IdentityIfc by Identity(name), Comparator<Solution>,
 ↪  SolverEmitterIfc by SolverEmitter() {
```

The `StochasticSolver` inherits from the `Solver` class and requires that a source of random streams be provided. It also allows stream control.

```
abstract class StochasticSolver(
    problemDefinition: ProblemDefinition,
    evaluator: EvaluatorIfc,
    maxIterations: Int,
    replicationsPerEvaluation: ReplicationPerEvaluationIfc,
    streamNum: Int = 0,
    val streamProvider: RNStreamProviderIfc =
     ↪  KSLRandom.DefaultRNStreamProvider,
    name: String? = null
) : Solver(problemDefinition, evaluator, maxIterations,
 ↪  replicationsPerEvaluation, name), RNStreamControlIfc {
```

11.4.1. Structure of the Iterative Process

A solver is an iterative algorithm that searches for the optimal solution for a defined problem. In the abstract `Solver` class, the algorithm is conceptualized as having a main iterative loop. The basic algorithm is structured conceptually as an iterative loop:

```
initializeIterations()
while(hasNextIteration()){
  mainIteration()
```

```
  iterationCounter++
  checkStoppingCondition()
}
endIterations()
```

First the algorithm is initialized. Then, while there are additional iterations or the stopping criteria has not been met the main iteration is executed. After all the iterations have ended, there is a clean up phase. The loop is the main loop that ultimately determines the convergence of the algorithm and recommended solution. Some algorithms have additional "inner" loops". In general, inner loops are often used to control localized search for solutions. If an algorithm has additional inner loops, these can be embedded within the main loop via the sub-classing process. Specialized implementations may have specific methods for determining stopping criteria; however, to avoid the execution of a large number of iterations, the iterative process has a specified maximum number of iterations.

The `Solver` class has three abstract protected functions that must be implemented.

- `protected abstract fun startingPoint(): InputMap`: The purpose of this function is to determine the starting point for the solver's process. In the default implementation, this function is called from the `initializeIterations()` function.
- `protected abstract fun nextPoint(): InputMap`: The purpose of this function is to determine the next point to be evaluated for the solver's process. Specific implementations will call this function within their implementations of the `mainIteration()` function.
- `protected abstract fun mainIteration()`: The purpose of this function is to contain the logic that iteratively executes until the maximum number of iterations is reached or until the stopping criteria is met.

The `Solver` class has these protected open functions that can be overridden with more specific behavior:

- `protected open fun initializeIterations()`: This function is called prior to the running of the main loop of iterations. The base implementation determines a starting point, requests an evaluation, and sets the initial current solution.
- `protected open fun isStoppingCriteriaSatisfied(): Boolean`: This function is called from `checkStoppingCondition()` to implement a stopping criteria that is based on the quality of the solution. You have the option to specify a function via the `solutionQualityEvaluator` property that will be used from this function. If no solution quality checker is supplied or this function is not overridden, then this function should report false. In that case, the main loop will only stop when the maximum number of iterations has been reached.
- `protected open fun beforeMainIteration()`: An empty hook function to permit logic to be inserted prior to the call to `mainIteration()`.
- `protected open fun afterMainIteration()`: An empty hook function to permit logic to be inserted after the call to `mainIteration()`.
- `protected open fun mainIterationsEnded()`: An empty hook function to permit logic to be inserted after the main iterative loop completes.
- `protected open fun generateNeighbor(currentPoint: InputMap,rnStream: RNStreamIfc): InputMap`: A basic function that can be used by sub-classes to generate a point that is a neighbor to the current point. The process can be random. You have the option of either overriding this function or supplying an instance of the `GenerateNeighborIfc` interface via the `neighborGenerator` property. If a `neighborGenerator` is not supplied, the base algorithm, randomly selects one of the coordinates from the current point and then randomly generates an input range feasible value for the

coordinate.

To better understand how these functions work together, let's review a specific implementation. A simple algorithm is based on stochastic hill climbing. The following code presents the implementations. The main iteration function is as follows:

```
override fun mainIteration() {
    // generate a random neighbor of the current solution
    val nextPoint = nextPoint()
    // evaluate the solution
    val nextSolution = requestEvaluation(nextPoint)
    if (compare(nextSolution, currentSolution) < 0) {
        currentSolution = nextSolution
    }
    // capture the last solution
    solutionChecker.captureSolution(currentSolution)
}
```

This algorithm searches for an optimal solution by iteratively evaluating and moving to neighborhood solutions, as determined by the evaluator and the problem definition. It uses a stochastic approach to explore the solution space by using a random number stream. Randomly generated points are evaluated and if the new point has a better solution than the current solution, the proposed point becomes the current solution. This process continues until the maximum number of iterations is met.

The implementations of `startingPoint()` and `nextPoint()` both inherited from `StochasticSolver` are as follows:

```
override fun startingPoint(): InputMap {
    return startingPointGenerator?.startingPoint(
     ↪ problemDefinition) ?:
     ↪ problemDefinition.startingPoint(rnStream)
```

```
    }

    override fun nextPoint(): InputMap {
        return generateNeighbor(currentPoint, rnStream)
    }
```

We see that `nextPoint()` uses the availability of the supplied `generateNeighbor()` function. We also see that the implementation of the `startingPoint()` function will use an instance of the `StartingPointIfc` interface via the `startingPointGenerator` property or rely on the problem definition to randomly generate an input feasible point.

To summarize, an implementation of a solver must at the very least specify:

- how to determine its starting point (`startingPoint()`),
- how to determine its next point for evaluation (`nextPoint()`), and
- how to search its the solution space (`mainIteration()`).

11.4.2. Solver Plug-in Behavior

As mentioned in the previous section, the base implementation allows you to either sub-class for adding specific behavior or to provide behavior through specific functional forms. This section overviews the capabilities for plugging in behavior. The following functional interfaces are important to this process.

- `ReplicationPerEvaluationIfc` interface: This functional interface can be used by solvers to determine the number of replications to request when asking the simulation oracle for evaluations. This permits strategies that start with fewer replications during the initial search and then increase replications when exploring higher

quality solutions. As shown in Figure 11.15, there are two concrete implementations: the `FixedReplicationsPerEvaluation` and `FixedGrowthRateReplicationSchedule` classes. The fixed schedule causes a constant number of replications to be used for every evaluation request. The fixed growth rate replication schedule models an increasing number of replications based on a growth rate factor that is a function of the number of iterations.

```
fun interface ReplicationPerEvaluationIfc {

    fun numReplicationsPerEvaluation(solver: Solver) : Int
}
```

- `GenerateNeighborIfc` interface: This functional interface can be used by solvers to specify how to generate a neighbor relative to the supplied input. The `ensureFeasibility` parameter should indicate if the generation method should ensure that the returned input map is feasible with respect the constraints of the problem.

```
fun interface GenerateNeighborIfc {

    fun generateNeighbor(inputMap: InputMap, solver: Solver,
     ↪  ensureFeasibility: Boolean) : InputMap

}
```

- `NeighborSelectorIfc` interface: This functional interface defines functions for selecting a neighbor (point) from a defined neighborhood represented as a set containing inputs. The supplied solver could be used to allow memory during the selection process.

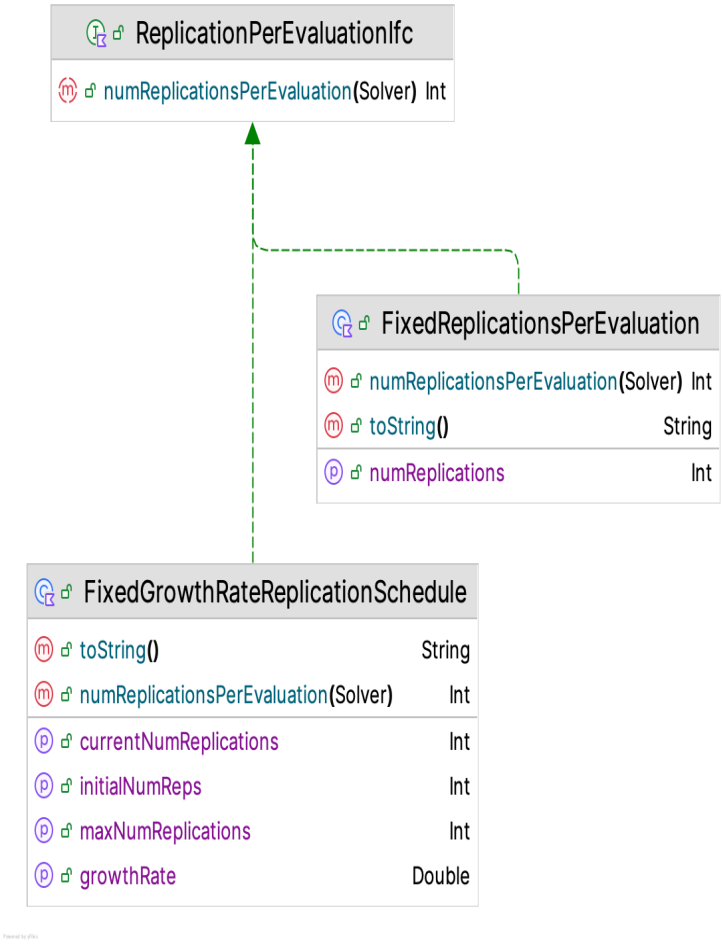


Figure 11.15.: Replications per Evaluation Hierarchy

```
fun interface NeighborSelectorIfc {

    fun select(neighborhood: Set<InputMap>, solver: Solver) :
     ↪  InputMap

}
```

- `SolutionQualityStoppingCriteriaIfc` interface: This functional interface represents functions that define when to stop the iterative process based on the quality of the solution. As previously noted, the `solutionQualityEvaluator` property will be used when the stopping criteria is evaluated within the main loop. Examples of different ways to stop the algorithms will be presented during the discussion of specific algorithms. The `solutionQualityEvaluator` property provides you with an approach to overriding the base behavior with different behavior rather than through sub-classing.

```
fun interface SolutionQualityEvaluatorIfc {

    fun isStoppingCriteriaReached(solver: Solver): Boolean

}
```

11.4.3. Miscellaneous Concepts in the `Solver` Class

This section presents some additional functionality within the `Solver` class that can be used when implementing specific algorithms. Figure 11.16 represents the properties associated with the `Solver` and `StochasticSolver` abstract base classes.

The primary functionality to understand is how the `Solver` class keeps track of the solutions that it produces. The most important property to

Figure 11.16.: Solver and StochasticSolver Properties

understand how to use is the `currentSolution` property. For this property, it is useful for us to see how the current solution is defined:

```
    var currentSolution: Solution =
     ↪  problemDefinition.badSolution()
        protected set(value) {
            // save the previous solution
            previousSolution = field
            // update the current solution
            field = value
            myBestSolutions.add(value)
        }
```

The current solution is initialized as a bad solution. Recall that a bad solution is a solution that is the worst possible solution for the problem. This can be retrieved via the `ProblemDefinition` class's `badSolution()` function. The implementation of the solver algorithm should set the current solution during its search process. Notice that the previous solution is set during the assignment. This property provides access to the previous solution visited by the solver. Some solvers may compare the current solution with the previous solution as part of the stopping criteria evaluation process.

Also note that the `myBestSolutions` protected property is used to capture the newly assigned solution. The protected `myBestSolutions` property is an instance of the `Solutions` class. Figure 11.17 presents the `SolutionsIfc` interface and the `Solutions` class.

The `SolutionsIfc` implements the `List<Solution>` interface, which allows instances of the `Solutions` class to be treated as a list. The implementation of the `Solutions` class provides an ordering to the solutions based on the order in which they are added to the list and based on the value of the penalized objective function. The constructor of the `Solutions` class is presented in the following code.

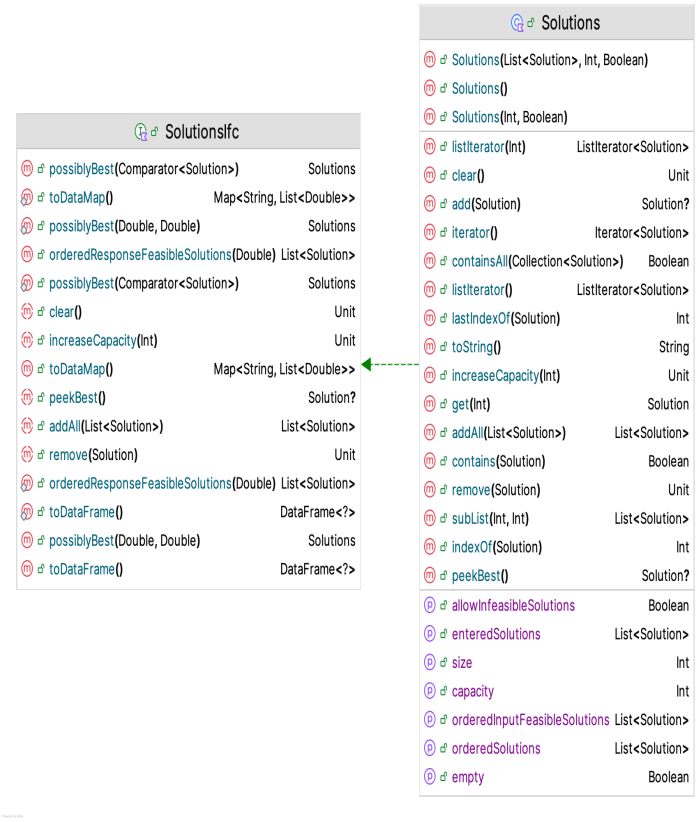


Figure 11.17.: The Solutions Class

```
class Solutions(
    capacity: Int = defaultCapacity,
    var allowInfeasibleSolutions: Boolean = false
) : SolutionsIfc
```

Conceptually, an instance of the `Solutions` class is like a cache, with a default capacity. It also controls whether it can store infeasible solutions. While the `SolutionsIfc` interface implements the `List<Solution>` interface, the implementation within the `Solutions` class allows mutability by providing specialized `add()`, `addAll()`, and `remove()` functions. Three functions facilitate extracting the best of the stored solutions.

- `peekBest(): Solution?`: This function returns the solution with the lowest penalized objective function value.
- `possiblyBest(comparator: Comparator<Solution>): Solutions`: This function returns a list of solutions that are possibly the best using the supplied comparator.
- `possiblyBest(level: Double = DEFAULT_CONFIDENCE_LEVEL,indifferenceZone: Double = 0.0): Solutions`: This function returns solutions that could be considered best using a penalized objective function confidence interval comparator. The basic procedure is to select the smallest or largest solution as the best depending on the direction of the objective. Then, this procedure uses the best solution as the standard and compares all the solutions with it in a pair-wise manner. Any solutions that are considered not statistically different from the best solution are returned. The confidence interval is for each individual comparison with the best. Thus, to control the overall confidence, you will want to adjust the individual confidence interval level such that the overall confidence in the process is controlled as discussed in Section 5.7.2.

The property `enteredSolutions` returns the solutions in the order in which they occurred (i.e. the order in which they were added to the solutions).

The properties `orderedInputFeasibleSolutions` and `orderedSolutions` return the solutions ordered by penalized objective function values.

In some situations, you may want to compare the effectiveness of different solvers for a problem. There are four properties of the `Solver` class that can help with this assessment process.

- `iterationCounter`: The `iterationCounter` property counts the number of iterations performed by a solver. Since some solvers may use additional stopping criteria, this property can be used to assess the total number of iterations performed.
- `numOracleCalls`: Each time a solver requests evaluations from the oracle, this property is incremented. It can be used to indicate how many simulation evaluations were performed. Since simulation evaluations can be computationally expensive, this property can help us understand how efficient a solver is with respect to simulation evaluations.
- `numReplicationsRequested`: Similar to the `numOracleCalls` property, this property totals the number of replications requested during the solver's execution. This property can help us understand how efficient a solver is in terms of its sampling.
- `iterativeProcess`: This property references the underlying iterative process via the `IterativeProcessStatusIfc` interface. Figure 11.18 presents this interface. From the perspective of evaluating a solver's efficiency, the `beginExecutionTime` and `endExecutionTime` properties can be used to compute the total time spent by the solver.

The final concept that we discuss involves classes that assist with checking for convergence. Recall that algorithms will select a candidate point and evaluate that point via the simulation oracle. For many algorithms, the selection of the point is driven by a process that may converge such that the same point (or nearly the same point) is repeatedly selected. We may choose to stop the algorithm if the same point is repeatedly selected. The `SolutionChecker` class is designed to hold and test if its contained

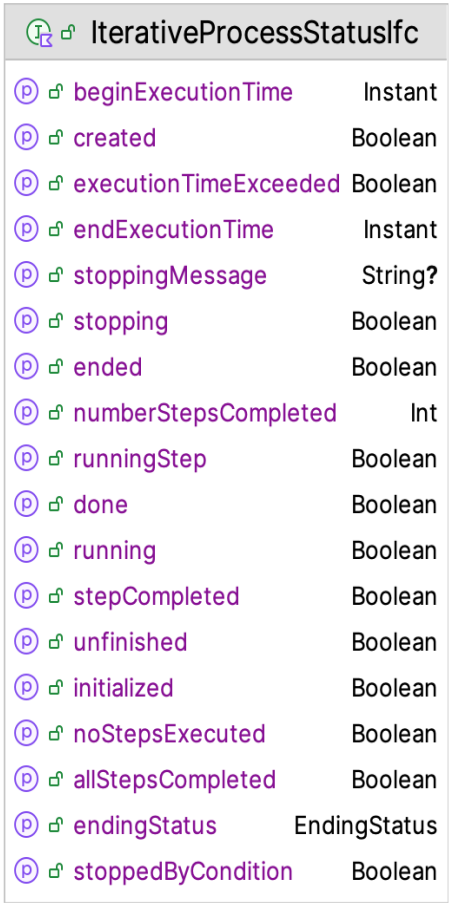


Figure 11.18.: IterativeProcessStatusIfc Interface

solutions are all the same. Figure 11.19 presents the `SolutionChecker` class and the `SolutionEqualityIfc` interfaces.

The purpose of the `SolutionEqualityIfc` interface is to provide a mechanism to check for the equality between two solutions. As we can see in Figure 11.19, there are a variety of ways to define equality.

- `ConfidenceIntervalEquality`: This class checks for equality between solutions based on whether the confidence interval on the difference contains the indifference zone parameter. This is based on an approximate confidence interval on the difference between the unpenalized objective function values.
- `PenalizedObjectiveFunctionEquality`: This class checks if the penalized objective function values are within some specified precision. The uncertainty of the values is not taken into consideration.
- `InputEquality`: This class indicates that two solutions are equal based only on the values of their inputs. If the solutions do not have the same inputs, then the penalized objective function is used to determine the ordering. Thus, two solutions are considered the same if they have the same input values, regardless of the value of the objective functions.
- `InputsAndConfidenceIntervalEquality`: This class checks for equality between solutions based on whether the confidence interval on the difference contains the indifference zone parameter and whether the input variable values are the same.

The `SolutionChecker` class uses an instance of the `SolutionEqualityIfc` interface to check if the elements that it holds all test as equal. The constructor requires a threshold value that represents the number of solutions to check. When the solution checker contains the specified number of solutions, you can call the `checkSolutions()` function, which will return true if all the contained solutions test as equal. The specified threshold limit also specifies the capacity. The checker will only hold the specified number of solutions.

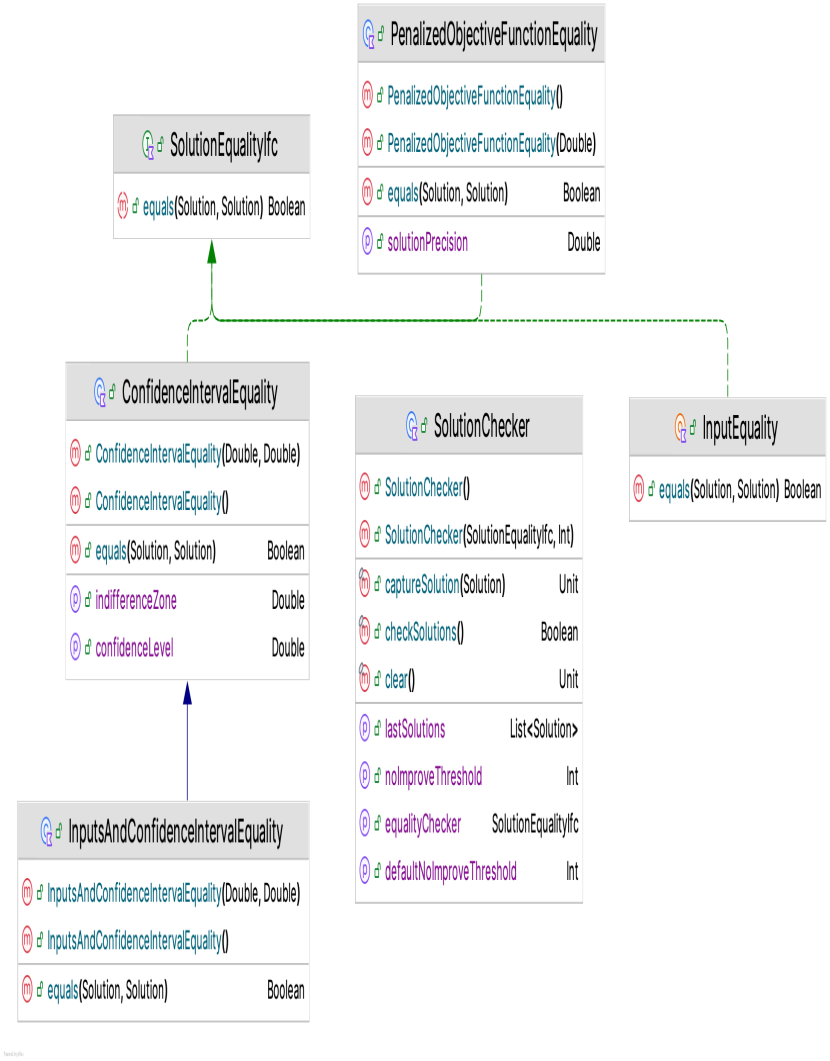


Figure 11.19.: The Solution Checker Class

```
class SolutionChecker(
    var equalityChecker: SolutionEqualityIfc = InputEquality,
    noImproveThreshold: Int = defaultNoImproveThreshold
)
```

An instance of the `SolutionChecker` class can be used as part of the logic for checking if the stopping criteria for the algorithm indicates if the algorithm should stop. Examples of this will be illustrated when covering specific algorithms.

11.5. Implemented Solvers

This section will overview the set of solvers provided within the KSL for use on simulation optimization problems. As shown in Figure 11.14, the following algorithms are available.

- `StochasticHillClimber`: A stochastic hill climbing solver implements the idea of randomly moving through the search space. As new solutions are found, the current solution is updated if the next solution is better than the previous solution. Variations of stochastic hill climbing involve how to select points. It is one of the simplest algorithms to implement and it is still useful because in the limit, the algorithm will eventually visit the optimal solution.
- `SimulatedAnnealing`: Simulated annealing is a stochastic optimization algorithm that models the optimization process based on an analogy with a metallurgical annealing process. Popularized by (Kirkpatrick et al. 1983), simulated annealing has been widely used as a general purpose optimization methodology. For further background on simulated annealing, the interested reader should consider this on-line book.

- `CrossEntropySolver`: The cross-entropy method is a population based algorithm that is based on minimizing the cross-entropy between a target distribution and a parametric distribution. (De Boer et al. 2005) present a detailed tutorial on the cross-entropy method. The cross-entropy method is a general purpose meta-heuristic. While designed for deterministic optimization problems, the cross-entropy method has also been applied to stochastic optimization problems.
- `RSplineSolver`: The R-SPLINE method is presented in (Wang et al. 2013). This method is designed for integer-ordered (discrete) optimization problems. The overall theory guarantees convergence to locally optimal solutions.
- `RandomRestartSolver`: Many solvers require an initial starting point. The algorithm's final solution may be sensitive to the choice of the initial starting point. A common strategy is to repeat the algorithm at different starting points and select the solution that is best across the starting points. This solver permits other solvers to be called repeatedly for randomly selected starting points.

The previously discussed framework should allow you to implement your own solvers. This section discusses some of the key features of the provided solvers, which may be of interest when using the solvers. Since the `StochasticHillClimber` solver was already discussed, it will not be discussed here. We will start with the classic *simulated annealing* algorithm.

11.5.1. Simulated Annealing

Simulated annealing is an optimization approach that is based on the concepts of Markov Chain Monte Carlo (MCMC) (see Section 9.3.4). As presented in Chapter 9, MCMC can be used to generate from arbitrarily specified multivariate distributions. Simulated annealing exploits the MCMC sampling process to find a mode of density function, $f(x)$. This mode represents a point where $f(x)$ is maximized. The process produces a sequence of density functions $f_T(x) \propto [f(x)]^{1/T}$, where T denotes the

temperature of the distribution. The sampling process produces a Markov chain such that the generated values x_k are from the distributions f_{T_k} for decreasing temperatures $T_1, T_2, \ldots$. As the temperatures decrease, the distributions converge to a highly peaked distribution, where the peak is at the maximum of the distribution, f. The point causing the peak will also be the solution to the optimization problem. A key factor in this process is the cooling schedule that determines the temperature sequence.

As we can see from the constructor, you must supply the problem definition, the evaluator and values for the key parameters of the annealing process:

- `temperatureConfiguration`: How the starting temperature is chosen: either a fixed value you supply (`TemperatureConfiguration.Fixed`), or automatically calibrated by the solver itself (`TemperatureConfiguration.AutoCalibrate`) via a short random walk over the objective function landscape before the search begins.
- `coolingSchedule`: An implementation of the `CoolingScheduleIfc` interface to determine the temperature at each iteration.
- `stoppingTemperature`: If the current temperature goes below this value, the algorithm will stop.

```
class SimulatedAnnealing @JvmOverloads constructor(
    problemDefinition: ProblemDefinition,
    evaluator: EvaluatorIfc,
    val temperatureConfiguration: TemperatureConfiguration =
     ↪  TemperatureConfiguration.Fixed(
     ↪  defaultInitialTemperature),
    var coolingSchedule: CoolingScheduleIfc =
     ↪  ExponentialCoolingSchedule(defaultInitialTemperature
     ↪  ),
    stoppingTemperature: Double = defaultStoppingTemperature,
    maximumIterations: Int = defaultMaxNumberIterations,
    replicationsPerEvaluation: ReplicationPerEvaluationIfc,
    streamNum: Int = 0,
```

```
    streamProvider: RNStreamProviderIfc = RNStreamProvider(),
    name: String? = null
) : StochasticSolver(problemDefinition, evaluator,
 ↪  maximumIterations, replicationsPerEvaluation, streamNum,
 ↪  streamProvider, name) {
```

`TemperatureConfiguration` is a sealed type with two variants. `Fixed(temperature)` uses a statically specified starting temperature. The `AutoCalibrate(targetProbability, sampleSize)` class option instead has the solver estimate a sensible starting temperature. During initialization it runs a short random walk of `sampleSize` steps over the problem, measures how much the objective function typically worsens on an uphill step, and picks a temperature that would accept such a step with roughly the value specified by `targetProbability`. This removes the guesswork of picking `initialTemperature` by hand, at the cost of some additional oracle calls before the real search begins.

Pseudo-code for the overall algorithm is as follows:

```
1. Initialize the solution and the temperature
2. While not converged
  2.1. Generate a possible solution based on some probabilistic process
  2.2. Calculate the change in the objective function
  2.3. If an improvement or acceptance probability is met
    2.3.1 accept the new solution
  2.4 Reduce the temparture according the cooling schedule
3. Return the best solution found
```

A unique aspect of the simulated annealing process is that a solution that is not an improvement over the current solution can be accepted as the current solution. That is, accepting a worse solution permits exploration of the solution space. As the temperature decreases it becomes less likely that non-improving solutions will be accepted.

Although the algorithm will stop when the maximum number of iterations is met, implementations often use approaches that stop when consecutive objective function values are less than some tolerance, when the temperature becomes too small, or when the objective function value has not changed after a number of iterations.

The `SimulatedAnnealing` class implements the `initializeIterations()`, `mainIteration()`, and the `isStoppingCriteriaSatisfied()` functions. The `mainIteration()` function follows closely the outlined pseudo-code:

```
    override fun mainIteration() {
        // generate a random neighbor of the current solution
        val nextPoint = nextPoint()
        // evaluate the point to get the next solution
        val nextSolution = requestEvaluation(nextPoint)
        // calculate the cost difference
        costDifference = nextSolution.penalizedObjFncValue -
↪   currentSolution.penalizedObjFncValue
        // if the cost difference is negative, it is
         ↪  automatically an acceptance
        if (costDifference < 0.0) {
            // no need to generate a random variate
            currentSolution = nextSolution
            lastAcceptanceProbability = 1.0
            logger.trace { "Solver: $name : solution improved
↪   to $nextSolution" }
        } else {
            // decide whether an increased cost should be
             ↪  accepted
            val u = rnStream.randU01()
            lastAcceptanceProbability =
↪   acceptanceProbability(costDifference, currentTemperature)
            if (u < lastAcceptanceProbability) {
                currentSolution = nextSolution
```

```
                logger.trace { "Solver: $name : non-improving
↪  solution was accepted: $nextSolution" }
            } else {
                // stay at the current solution
                logger.trace { "Solver: $name : rejected
↪  solution $nextSolution" }
            }
        }
        // update the current temperature based on the
         ↪  cooling schedule
        currentTemperature =
↪  coolingSchedule.nextTemperature(iterationCounter)
        // capture the last solution
        solutionChecker.captureSolution(currentSolution)
    }
```

The implementation uses the `nextPoint()` function from the `StochasticSolver` class to generate a neighbor of the current point. The new point is evaluated via the simulation oracle. Then, the objective function cost difference is computed. If there is an improvement, the next solution is automatically accepted as the current solution. If the proposed point caused an increase in the objective function, then the algorithm randomly generates a U(0,1) variate and compares it to the current acceptance probability. If the randomly generated value is less than the acceptance probability, the proposed point is accepted; otherwise, the current solution stays the same. The acceptance probability criteria is based on the Boltzmann distribution and is a function of the objective function difference and the current value of the temperature.

```
    fun acceptanceProbability(costDifference: Double,
     ↪  temperature: Double) : Double {
        require(temperature > 0.0) { "The temperature must be
↪  positive" }
```

```
        return if (costDifference <= 0.0) {
            1.0
        } else {
            exp(-costDifference / temperature)
        }
    }
```

The current temperature is computed by a function, which you can define, that implements the cooling schedule. The default implementation uses an exponential cooling schedule. The `ksl.simopt.solvers.algorithms` package also provides implementations for linear and logarithmic cooling schedules.

```
class ExponentialCoolingSchedule(
    initialTemperature: Double,
    val coolingRate: Double = defaultCoolingRate
) : CoolingSchedule (initialTemperature)  {
    init {
        require(coolingRate > 0.0) { "Cooling rate must be
 ↪  positive" }
        require(coolingRate < 1.0) { "Cooling rate must be
 ↪  less than 1.0" }
    }

    override fun nextTemperature(iteration: Int): Double {
        require(iteration > 0) { "The iteration must be
 ↪  positive. It was $iteration." }
        return initialTemperature *
         ↪  coolingRate.pow(iteration)
    }
}
```

The implementation of the `SimulatedAnnealing` class uses an instance of

the previously discussed `SolutionChecker` class as part of the stopping criteria to check if the last 5 solutions are the same or if the temperature has become too small.

```
override fun isStoppingCriteriaSatisfied(): Boolean {
    return solutionQualityEvaluator?.
     ↪  isStoppingCriteriaReached(this) ?:
        (checkTemperature() ||
         ↪  solutionChecker.checkSolutions())
}

private fun checkTemperature() : Boolean {
    return currentTemperature < stoppingTemperature
}
```

The parentheses around `checkTemperature() || solutionChecker.checkSolutions()` are essential: Kotlin's `?:` binds tighter than `||`, so without them the no-improvement fallback would silently apply even when a `solutionQualityEvaluator` is set, stopping the search earlier than intended.

11.5.2. Cross Entropy

Similar in some respect to simulated annealing, the cross-entropy method is a stochastic optimization procedure that can be applied to both deterministic and stochastic optimization problems. The basic procedure is described in (De Boer et al. 2005) and can be characterized as a population based approach because it uses a sample of the search space to define an elite population. Cross-entropy uses rare-event simulation techniques to cause the parameters of a sampling distribution to converge to the desired solution of the related optimization problem. The cross-entropy method can be applied to optimization problems involving both continuous and discrete variables.

As presented in (Rubinstein and Kroese. 2017) the basic cross-entropy algorithm is as follows. The value ρ is called the elite percentage. The algorithm presented here is in terms of minimizing the objective function.

1. Let $k = 0$ be the iteration counter. Initialize $\hat{v}_0$ as the initial parameters of some density function $f(\cdot; v_k)$. Let $N^e = \lceil \rho N \rceil$ be the size of the elite sample. Let N be the size of the cross-entropy population.
2. **Repeat**
    1. Generate a sample $\vec{X}_1, \vec{X}_2, ..., \vec{X}_N$ from $f(\cdot; v_{k-1})$.
    2. Calculate the performance $G(\vec{X}_1), G(\vec{X}_2), ..., G(\vec{X}_N)$.
    3. Order the performance from smallest to largest: $G_{(1)}, G_{(2)}, ..., G_{(N)}$
    4. Let $\hat{\gamma}_k = G_{(N^e)}$
    5. Use the same sample $\vec{X}_1, \vec{X}_2, ..., \vec{X}_N$ and solve a corresponding stochastic program, see (Rubinstein and Kroese. 2017) resulting in the estimated parameters of the sampling distribution $\tilde{v}_k$.
    6. Update the parameters according to: $\hat{v}_k = \alpha \tilde{v}_k + (1 - \alpha)\hat{v}_{k-1}$.
    7. k = k + 1
3. **Until** : a stopping criteria is met.

The stochastic program mentioned in step 2.5 of the algorithm is essentially the estimation of the parameters of $f(\cdot; v_k)$ via the maximum likelihood method based on the *elite sample* at each iteration. The elite sample consists of those $\vec{X}_i$ such that $G(\vec{X}_i) \geq \hat{\gamma}_k$. (Rubinstein and Kroese. 2017) recommend that the procedure be stopped if for some $k \geq d$, say $d = 5$, when $\hat{v}_k = \hat{v}_{k-1} = \cdots = \hat{v}_{k-d}$.

The choice of the density function $f(\cdot; v_k)$ is an important aspect of the algorithm because the parameters converge to the optimal solution of the associated optimization problem. Actually, similar to simulated annealing, the sequence of density functions $f(\cdot; \hat{v}_1), f(\cdot; \hat{v}_2), ...$ converges to a degenerate distribution assigning all probability to a single point x_k, for which the objective function value is greater than or equal to $\hat{\gamma}_k$. This point represents the solution to optimization problem. (De Boer et al.

2005) discuss different possibilities for the density function. The normal distribution is recommended for continuous optimization problems while the Bernoulli distribution has been used for discrete optimization problems. The critical aspect of this choice is the ability to easily compute the maximum likelihood estimates for the parameters of the distribution based on the elite sample.

The KSL implementation of the cross-entropy solver follows this general algorithm and builds from the `StochasticSolver` class. You must specify an instance of the `CESampler` class or accept the default of a sampler based on the multivariate normal distribution. The stopping criteria is based on providing an instance of the `InputsAndConfidenceIntervalEquality` class, which implements the `SolutionEqualityIfc` interface. This will test if $\hat{v}_k = \hat{v}_{k-1} = \cdots = \hat{v}_{k-d}$ is true based on equal values of inputs and a comparison of the solutions based on confidence interval comparisons with the best solution.

```
class CrossEntropySolver @JvmOverloads constructor(
    problemDefinition: ProblemDefinition,
    evaluator: EvaluatorIfc,
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc = RNStreamProvider(),
    ceSampler: CESampler =
     ↪  CENormalSampler(problemDefinition),
    maxIterations: Int = ceDefaultMaxIterations,
    replicationsPerEvaluation: ReplicationPerEvaluationIfc,
    solutionEqualityChecker: SolutionEqualityIfc =
     ↪  InputsAndConfidenceIntervalEquality(),
    name: String? = null
) : StochasticSolver(problemDefinition,
    evaluator, maxIterations,
    replicationsPerEvaluation, streamNum,
    streamProvider, name
)
```

`CESampler` is an abstract class and is a settable property, not fixed at construction. A sampler is self-sufficient (it owns its own stream provider so it can be built and exercised standalone, outside of any solver); handing one to a `CrossEntropySolver`, either at construction or by later assignment, *attaches* it, adopting it onto the solver's own stream provider on a stream distinct from the solver's base stream. A sampler cannot be reattached while the solver is running, and its dimension must match the problem's number of input variables.

The implementation generates the points $\vec{X}_1, \vec{X}_2, \ldots, \vec{X}_N$ via the function call `ceSampler.sample(sampleSize())` within the `mainIteration()` function. The function `sampleSize()` computes the size, N, of the cross-entropy population at each iteration. Then, the solver requests the evaluation of those points to produce the estimated performance of the objective function, $G(\vec{X}_1), G(\vec{X}_2), \ldots, G(\vec{X}_N)$.

```
    override fun mainIteration() {
        // Generate the cross-entropy sample population
        val points = ceSampler.sample(sampleSize())
        // Convert the points to be able to evaluate.
        val inputs =
         ↪  problemDefinition.convertPointsToInputs(points)
        // request evaluations for solutions
        val evaluations = requestEvaluations(inputs)
        val results = evaluations.values.toList()
        if (results.isEmpty()) {
            // Returning will cause no updating on this
             ↪  iteration.
            // New points will be generated for another try
             ↪  on the next iteration.
            return
        }
        // At least one result, so proceed with processing.
        // Process the results to find the elites. This
         ↪  should fill myEliteSolutions.
```

```
        myEliteSolutions = findEliteSolutions(results)
        // convert elite solutions to points
        val elitePoints =
         ↪  extractSolutionInputPoints(myEliteSolutions)
        // update the sampler's parameters
        ceSampler.updateParameters(elitePoints)
        // specify the current solution
        currentSolution = myEliteSolutions.first()
        // capture the last solution
        solutionChecker.captureSolution(currentSolution)
    }
```

The elite solutions are found via the call to `findEliteSolutions()` and the points associated with the elite solutions are used to update the parameters of the cross-entropy sampler. The best solution from the elite sample is used to represent the current solution. Just like the simulated annealing algorithm, the cross-entropy implementation uses an instance of the `SolutionChecker` class to remember the last d solutions and to stop when they have converged.

There are a number of key issues that must be implemented for the cross-entropy method.

- How large should the population (N) be?
- What should the elite percentage ρ be? Alternatively, what should be the size of the elite sample?
- What parametric family of distributions should be used for the sampling distribution?
  - How to initialize the parameters of the sampling distribution?
  - How to update the parameters of the sampling distribution?
  - What should the value of the smoothing parameter α be?

(De Boer et al. 2005) recommends that the size of the population, N, be related to the dimension of the problem. If we let K be the number

of parameters that need to be estimated for the sampling distribution, then (De Boer et al. 2005) recommends $N = cK$, where c is a number between 1 and 10. For example, if the sampling distribution family is the normal distribution, we must estimate two parameters for each input variable, that is (μ_i, σ_i) for $i = 1, 2, \ldots, |I|$. Thus, $K = 2 * |I|$. The KSL provides more flexibility with respect to determining the population size. As previously mentioned, the implementation provides the `sampleSize()` function, which can be customized by providing an implementation of the `SampleSizeFnIfc` functional interface.

```
fun interface SampleSizeFnIfc {
    fun sampleSize(ceSolver: CrossEntropySolver): Int
}
```

The base implementation will use a supplied sample size function or use the value of the `ceSampleSize` property.

```
    fun sampleSize(): Int {
        return sampleSizeFn?.sampleSize(this) ?: ceSampleSize
    }
```

The value of the `ceSampleSize` property is based on a default value that you can change or a recommended sample size based on the size of a sample needed to adequately estimate the quantile of a distribution.

```
    var ceSampleSize: Int = minOf(recommendCESampleSize(),
     ↪ defaultMaxCESampleSize)
        set(value) {
            require(value >= defaultMinCESampleSize) { "The
↪ CE sample size must be >= $defaultMinCESampleSize" }
            field = minOf(value, defaultMaxCESampleSize)
        }
```

The recommended cross-entropy sample size is based on the results presented in (Chen and Kelton 1999) and is implemented in the `recommendCESampleSize()` companion function.

```
        fun recommendCESampleSize(
            elitePct: Double = defaultElitePct,
            ceQuantileConfidenceLevel: Double =
             ↪ defaultCEQuantileConfidenceLevel,
            maxProportionHalfWidth: Double =
             ↪ defaultMaxProportionHalfWidth,
        ): Int
```

The computed sample size may be sufficient to adequately estimate the quantile associated with the desired elite percentage using a desired quantile confidence level and an adjusted half-width bound for the associated quantile proportion. Defaults are provided for each of these parameters.

Lastly, the KSL cross-entropy implementation permits customization of the size of the elite sample. In some situations, you may want to vary the size of the elite sample as the algorithm progresses to improve estimation of the parameters of the sampling distribution. The KSL implementation provides the `eliteSize()` function for this purpose.

```
    fun eliteSize(): Int {
        return eliteSizeFn?.eliteSize(this) ?:
         ↪ maxOf(ceil(elitePct * ceSampleSize).toInt(),
            defaultMinEliteSize)
    }
```

Note that you can supply an instance of the `EliteSizeIfc` functional interface, which simply promises to compute the size of the elite sample.

```
fun interface EliteSizeIfc {

    fun eliteSize(ceSolver: CrossEntropySolver): Int

}
```

The implementation of the `eliteSize()` function will use a supplied function specified by the `eliteSizeFn` property or determine the size based on the specified elite percentage.

The questions associated with the sampling distribution and the updating of its parameters should be addressed by the choice of the implementation of the `CESampler` class chosen for the algorithm.

Figure 11.20 presents the `CESampler` abstract base class and the `CENormalSampler` class, which implements the sampling process based on the normal distribution. Note that the `CESampler` class implements the `MVSampleIfc` and `RNStreamControlIfc` interfaces. Thus, it can act as a multivariate sampler and can control the underlying random number stream. The four most relevant functions are:

- `updateParameters(elites: List<DoubleArray>)`: This function should update the parameters based on the points from the elite sample (held in a list of double arrays).
- `parameters(): DoubleArray`: This function returns the current parameter values.
- `initializeParameters(values: DoubleArray)`: This function is used to specify the initial values of the parameters.
- `hasConverged(): Boolean`: This function should return true if the parameters of the underlying sampling distribution are considered converged.

The initialization of the parameters of the cross-entropy sampling distribution is interesting because we do not have any knowledge from an elite

Figure 11.20.: CESampler Class

sample when the algorithm starts. That is, initializing the parameters is like determining the starting point of the optimization problem. For the `CENormalSampler` class, the implementation uses a randomly generated point in the solution space as the specification of the means of the multivariate normal distribution. Then, the input range of each point is used to approximate the initial values of the standard deviations for the distributions. The standard approximation for the standard deviation as a function of the range (R) is used: $\sigma = R/4$. Thus, input variables that have a wider range of possible values start off with a higher variance parameter. The means associated with the normal distribution will converge to the recommended solution, while the standard deviations will continue to get closer and closer to zero as the process converges to a degenerate distribution.

As noted in Figure 11.20, the `CENormalSampler` class requires the specification of the smoothing constant, α, for both the mean and the standard deviation parameters. You also have the ability to specify more or less variability via the `initialVariabilityFactor` property, which will be used to multiply the approximated initial values for the standard deviation parameters. Its default value is 1.0. You must also specify the `cvThreshold` parameter value. This threshold is used to test if the parameters have converged. The default implementation computes the coefficient of variation of the parameters and if the coefficient of variation goes below this threshold, then the parameter is considered to have converged. If all the parameters have converged, then the `hasConverged()` function will return true. The default coefficient of variation threshold is set to 0.03.

Updating the parameters of the normal sampler is easy. We can just use the sample average and sample standard deviation of the elite points associated with the elite sample as the new estimates, and then smooth those estimates using the smoothing constants. Based on the recommendations from (De Boer et al. 2005), the default smoothing constants are set to 0.85. The KSL only supplies the `CENormalSampler` because testing has indicated that it performs okay for both discrete and continuous problems.

You can implement the defined interfaces if you require other parametric distributions.

11.5.3. R-SPLINE

This section will present an overview of the R-SPLINE algorithm discussed in (Wang et al. 2013). Since that paper has substantial details of the algorithm, the emphasis in this presentation will be on how the `ksl.simopt` package supports its implementation. As previously noted, the R-SPLINE algorithm is for discrete integer ordered optimization problems. If you attempt to solve a problem that is not integer ordered, the R-SPLINE implementation will report an illegal argument exception for the problem definition parameter.

As a `StochasticSolver`, the `RSplineSolver` class requires a specification of the replications per evaluation and an implementation of a stopping criteria. Similar to the cross-entropy method, the R-SPLINE implementation also uses the `InputsAndConfidenceIntervalEquality` class for checking if the algorithm should stop. However, this is different from the approaches recommended in (Wang et al. 2013). Unique to the R-SPLINE algorithm is that the algorithm does not use a fixed replication sampling mechanism but rather the number of replications per evaluation will grow as the number of iterations increases. This is an important aspect of the theory behind the algorithm to justify claims that in the limit the algorithm will converge to local neighborhood optimal solutions.

```
class RSplineSolver @JvmOverloads constructor(
    problemDefinition: ProblemDefinition,
    evaluator: EvaluatorIfc,
    maximumIterations: Int = defaultMaxNumberIterations,
    replicationsPerEvaluation:
     ↪ FixedGrowthRateReplicationSchedule,
    solutionEqualityChecker: SolutionEqualityIfc =
     ↪ InputsAndConfidenceIntervalEquality(),
```

```
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc = RNStreamProvider(),
    name: String? = null
) : StochasticSolver(problemDefinition, evaluator,
 ↪  maximumIterations,
    replicationsPerEvaluation, streamNum, streamProvider,
     ↪  name)
```

A second constructor is also available for cases where building a `FixedGrowthRateReplicationSchedule` by hand is not necessary: it accepts `initialNumReps`, `sampleSizeGrowthRate`, and `maxNumReplications` directly and constructs the schedule for you.

The R-SPLINE algorithm combines three ideas: 1) piece-wise linear interpolation search, 2) discrete neighborhood enumeration, and 3) retrospective solving of a sequence of sample path problems with an increasing sample size using the previous solution as the starting point for the next problem. (Wang et al. 2013) specifies algorithm 1 of R-SPLINE as

1. Set $X_0^* = x_0$
2. for $k = 1, 2, ...$
3. $X_k^* = SPLINE(X_{k-1}^*, m_k, b_k)$
4. end for

The sequences m_k and b_k specify the sample size for the k^{th} iteration and the limit on the number of simulation evaluation calls, respectively. As we can see, the `mainIteration()` function represents this algorithm. (Wang et al. 2013) recommends $m_{k+1} = \lceil 1.1 m_k \rceil$, which increases the sample size by 10 percent for each iteration.

```
    override fun mainIteration() {
        // The initial (current) solution is randomly
         ↪  selected or specified by the user.
```

```
        // It will be the current solution until beaten by
         ↪ the SPLINE search process.
        // Call SPLINE for the next solution using the
         ↪ current sample size (m_k) and
        // current SPLINE oracle call limit (b_k).
        val splineSolution = spline(
            currentSolution,
            rSPLINESampleSize, splineCallLimit
        )
        currentSolution = splineSolution
        // capture the last solution
        solutionChecker.captureSolution(currentSolution)
    }
```

The SPLINE function is fairly complex. Given a specification of the sample size for the iteration, m_k, the supplied solution may need additional replications evaluated, which will result in a new estimated solution. Then, an inner loop of the spline searching process commences. This involves two phases. First, a search of a piece-wise linear interpolation of the objective function and then a search of the neighborhood resulting from the piece-wise linear interpolation search. The details of this process can be found as algorithm 2 in (Wang et al. 2013).

```
    private fun spline(
        initSolution: Solution,
        sampleSize: Int,
        splineCallLimit: Int
    ): Solution {
        require(initSolution.isInputFeasible()) { "The
↪   initial solution to the SPLINE function must be input
↪   feasible!" }
        // use the initial solution as the new (starting)
         ↪ solution for the line search (SPLI)
```

```
        var newSolution = if (sampleSize >
         ↪  initSolution.count){
            requestEvaluation(initSolution.inputMap,
↪  sampleSize)
        } else {
            initSolution
        }
        // save the starting solution
        val startingSolution = newSolution
        // initialize the number of oracle calls
        var splineOracleCalls = 0
        for(i in 1..splineCallLimit){
            // perform the line search to get a solution
             ↪  based on the new solution
            val spliResults =
             ↪  searchPiecewiseLinearInterpolation(
             ↪  newSolution, sampleSize)
            // SPLI cannot cause harm
            val neStartingSolution = if
             ↪  (compare(spliResults.solution, newSolution)
             ↪  <= 0){
                spliResults.solution
            } else {
                newSolution
            }
            // search the neighborhood starting from the SPLI
             ↪  solution
            val neSearchResults =
             ↪  neighborhoodSearch(neStartingSolution,
             ↪  sampleSize)
            splineOracleCalls = splineOracleCalls +
↪  spliResults.numOracleCalls +
↪  neSearchResults.numOracleCalls
```

```
            newSolution = neSearchResults.solution
            // if the candidate solution and the NE search
             ↪  starting solution are the same, we can stop
            if ((neStartingSolution ==
             ↪  neSearchResults.solution) ||
             ↪  compare(neStartingSolution,
             ↪  neSearchResults.solution) == 0) {
                break
            }
        }
        // Check if the starting solution is better than the
         ↪  solution from the SPLINE search.
        return if (compare(startingSolution, newSolution) <
         ↪  0) {
            startingSolution
        } else {
            // The new solution might be better, but it might
             ↪  be input-infeasible.
            if (newSolution.isInputFeasible()) {
                newSolution
            } else {
                // not feasible, go back to the last feasible
                 ↪  solution
                startingSolution
            }
        }
    }
```

The piece-wise linear search process proceeds roughly as follows. Given a point, $x \in \mathbb{R}^d \setminus \mathbb{Z}^d$ in the space, consider the 2^d integer points that are the vertices of the hyper-cube that contains x. The algorithm chooses $d + 1$ vertices so that the convex hull of the points form a simplex that contains x. An interpolation of the objective function evaluated at the points of the

simplex provides a linear interpolation function from which an estimate of the gradient associated with the objective function can be computed. The gradient is used to determine a direction and then a linear line search is used to proceed in the specified direction.

This process continues until no improvement can be made in the direction of search at which point the piece-wise linear search process returns with the last point associated with its search. Then, a Von Neumann neighborhood is found around the returned point and the neighborhood is enumerated and the best solution from the enumeration process returned. If the enumeration solution is the same as the solution from the piece-wise linear search process then the SPLINE process stops; otherwise, another SPLINE cycle occurs. The cycles will stop when the number of spline calls is reached or the SPLINE search and the enumeration search are the same. An important aspect of the gradient estimation process is that common random numbers are used when evaluating the points that form the simplex around the proposed point. To implement these procedures the KSL provides the following functions.

- `searchPiecewiseLinearInterpolation(solution: Solution, sampleSize: Int): SPLIResults`: This is essentially algorithm 4 of (Wang et al. 2013).
- `piecewiseLinearInterpolation(solution: Solution,sampleSize: Int,perturbation: Double): PLIResults`: This is algorithm 3 of (Wang et al. 2013).
- `perturb(point: DoubleArray, perturbation: Double): DoubleArray`: This is the PERTURB function mentioned in (Wang et al. 2013).
- `neighborhoodSearch(solution: Solution,sampleSize: Int): NESearchResults`: This function implements the enumeration and search of the neighborhood defined in the SPLINE process.

11.5.4. Randomized Restart Solvers

In general, optimization algorithms rely on a starting point to initialize the search process. The performance of the algorithm may be sensitive to the choice of the starting point, which may cause the search process to converge to a local optimum or to limit the exploration of the search space. A random restart solver is a solver that starts another solver using different randomly generated starting points. `RandomRestartSolver` has two public constructors. The one shown here is the historical, sequential form: the supplied solver is reused, run to completion once per restart.

```
class RandomRestartSolver(
    restartingSolver: Solver,
    maxNumRestarts: Int = defaultMaxRestarts,
    streamNum: Int = 0,
    streamProvider: RNStreamProviderIfc = RNStreamProvider(),
    name: String? = null
)
```

A second constructor builds restarts from a `SolverFactoryIfc` instead of a single solver instance, and can run them **concurrently**: with `concurrentRestarts > 1`, up to that many restarts run at the same time, each on its own worker with its own solver instance and its own private evaluation resources, since restarts are statistically independent of one another. Each restart's starting point and simulation streams are pre-drawn before launch, so results are reproducible regardless of which worker finishes first. An optional confirmation stage can re-evaluate the best candidates from the completed restarts under common random numbers before reporting a final winner. This is the same screening idea from Section 5.7.2, applied here to pick among restarts rather than among designed-experiment scenarios. The mechanics of setting up concurrent restarts (the solver and evaluator factories, stream-block sizing,

the confirmation stage's parameters) are covered in the KSL developer guide.

The `mainIterations()` function is very straightforward. Since we want independent solutions, we clear a solution cache if it is being used. Then, a new starting point is generated and assigned as the starting point for the solver. The solver is executed for all of its iterations (or until its stopping criteria are met). The best solution from the execution of the solver is captured.

```
    override fun mainIteration() {
        if (isConcurrentMode) {
            concurrentMainIteration()
        } else {
            sequentialMainIteration()
        }
    }

    private fun sequentialMainIteration() {
        // clear the evaluator cache between randomized runs,
         ↪  but allow caching during the run itself
        // this will cause new replications to be generated
        if (clearCacheBetweenRuns) {
            evaluator.cache?.clear()
        }
        // Note: each restart's penalty ramp begins fresh
         ↪  because every solver run resets
        // the shared evaluator's evaluation clock in
         ↪  Solver.initializeIterations,
        // matching concurrent mode, where each restart's
         ↪  private evaluator starts its
        // own clock. The evaluator's cumulative statistics
         ↪  counters are unaffected.
        // The first run honors a user-supplied starting
         ↪  point (the documented contract,
```

```
        // matching concurrent mode's restart 0); every other
         ↪  restart draws a random
        // input-feasible point.
        val startPoint = if (iterationCounter == 1 &&
         ↪  startingPoint != null) {
            startingPoint!!
        } else {
            startingPoint()
        }
        restartingSolver.startingPoint = startPoint
        logger.debug { "Starting a new randomized run at
↪  point: ${startPoint.inputValues.joinToString()}" }
        // run the solver until it finds a solution
        restartingSolver.runAllIterations()
        numOracleCalls = numOracleCalls +
↪  restartingSolver.numOracleCalls
        numReplicationsRequested = numReplicationsRequested
↪  + restartingSolver.numReplicationsRequested
        // get the best solution from the solver run
        val bestSolution = restartingSolver.bestSolution
        logger.debug { "Best solution found from the solver
↪  run: ${bestSolution.asString()}" }
        currentSolution = bestSolution
        logger.debug { "Current best:
↪  ${currentSolution.asString()}" }
    }
```

Thus, repeated executions of the solver start with a different starting point and may produce different solutions. In the next section, we will illustrate how to set up solvers to solve an optimization problem.

11.6. Using a Solver on a Problem

Section 11.2.5 presented how to define a problem definition for a reorder point, reorder quantity inventory system. We will use that problem definition to set up a cross-entropy solver and a random restart solver for optimizing the problem. Recall that the problem is to minimize the inventory ordering and holding costs subject to a fill-rate constraint. To use a solver, we need at a minimum the following:

- an instance of the `ProblemDefinition` class
- an instance of something that implements the `ModelBuilderIfc` interface
- an instance of the `EvaluatorIfc` interface

The companion object of the `Solver` class provides factory functions for creating the available solvers with default configurations. The factory functions will create an evaluator for executing the model that is configured to run the simulations locally (within the same execution thread) and is configured to use a `MemorySolutionCache()` to store and reuse solutions requested by the solver.

11.6.1. Illustrating the Cross-Entropy Solver

This section illustrates how to setup and run the cross-entropy solver for the reorder point, reorder quantity inventory model. Let's start with a review of the factory function that creates a cross-entropy solver. The factory function requires a problem definition, something that will build the model, and the necessary parameters to construct the cross-entropy solver.

```
        fun createCrossEntropySolver(
            problemDefinition: ProblemDefinition,
            modelBuilder: ModelBuilderIfc,
```

```
            ceSampler: CESampler? = null,
            startingPoint: MutableMap<String, Double>? =
             ↪ null,
            maxIterations: Int =
             ↪ CrossEntropySolver.ceDefaultMaxIterations,
            replicationsPerEvaluation: Int =
             ↪ defaultReplicationsPerEvaluation,
            solutionCache: SolutionCacheIfc =
             ↪ MemorySolutionCache(),
            simulationRunCache: SimulationRunCacheIfc? =
             ↪ null,
            experimentRunParameters:
             ↪ ExperimentRunParametersIfc? = null,
            streamNum: Int = 0,
            streamProvider: RNStreamProviderIfc =
             ↪ RNStreamProvider(),
            name: String? = null,
            parallelOptions: ParallelEvaluationOptions =
             ↪ ParallelEvaluationOptions()
        ): CrossEntropySolver {
            val evaluator = Evaluator.createProblemEvaluator(
                problemDefinition = problemDefinition,
↪   modelBuilder = modelBuilder, solutionCache =
↪   solutionCache,
                simulationRunCache = simulationRunCache,
↪   experimentRunParameters = experimentRunParameters,
                parallelOptions = parallelOptions
            )
            // The default sampler owns its own provider;
             ↪ CrossEntropySolver attaches whatever sampler
             ↪ it
            // is given onto the solver's provider (on a
             ↪ stream distinct from the solver's base
             ↪ stream).
```

```
            val sampler = ceSampler ?:
             ↪  CENormalSampler(problemDefinition)
            val ce = CrossEntropySolver(
                problemDefinition = problemDefinition,
                evaluator = evaluator,
                streamNum = streamNum,
                streamProvider = streamProvider,
                ceSampler = sampler,
                maximumIterations = maxIterations,
                replicationsPerEvaluation =
↪  replicationsPerEvaluation,
                name = name
            )
            if (startingPoint != null) {
                ce.startingPoint =
↪  problemDefinition.toInputMap(startingPoint)
            }
            return ce
        }
```

Notice the factory function allows passing through key parameters for both the construction of the evaluator and the solver. The evaluator is setup with the problem definition and the model builder. The rest of the parameters for the creation of the evaluator use the defaults.

The following code defines an object that implements the `ModelBuilderIfc` interface. This code is very similar to the code used to create models throughout the textbook. For this simple example, there will not be a need to use the parameters passed to the `build()` function. One important item to note is that the settings of the run parameters (length of warm up and the length of the replications) will stay as specified in this function; however, these base settings could be changed during the building process. The solver architecture via the previously described `EvaluationRequest`

class only permits the changing of the number of replications. Thus, once a model is built and provided to the solver, its basic configuration cannot be changed except for inputs and number of replications.

```
object BuildRQModel : ModelBuilderIfc {
    override fun build(
        modelConfiguration: Map<String, String>?,
        experimentRunParameters: ExperimentRunParametersIfc?
    ): Model {
        val reorderQty: Int = 2
        val reorderPoint: Int = 1
        val model = Model("RQInventoryModel")
        val rqModel = RQInventorySystem(model, reorderPoint,
         ↪  reorderQty, "Inventory")
        rqModel.initialOnHand = 0
        rqModel.demandGenerator.setInitialEventTimeProcesses
 ↪  (ExponentialRV(1.0 / 3.6))
        rqModel.costPerOrder = 0.15
        rqModel.unitHoldingCost = 0.25
        rqModel.unitBackorderCost = 1.75
        rqModel.leadTime.initialRandomSource =
 ↪  ConstantRV(0.5)
        model.lengthOfReplication = 20000.0
        model.lengthOfReplicationWarmUp = 10000.0
        model.numberOfReplications = 40
        return model
    }
}
```

Now we can construct the solver and run it. The following code provides a function for constructing and running the iterations of a solver and for printing out results from the optimization process.

```
fun runCESolver(
    simulationRunCache: SimulationRunCacheIfc? = null,
    experimentRunParameters: ExperimentRunParametersIfc? =
     ↪ null
) {
    val problemDefinition =
     ↪ makeRQInventoryModelProblemDefinition()
    val modelBuilder = BuildRQModel

    val solver = Solver.Companion.createCrossEntropySolver(
        problemDefinition = problemDefinition,
        modelBuilder = modelBuilder,
        startingPoint = null,
        maxIterations = 100,
        replicationsPerEvaluation = 50,
        simulationRunCache = simulationRunCache,
        experimentRunParameters = experimentRunParameters
    )
    val tracker = ConsoleSolverStateTracker(solver)
    tracker.startTracking()
    solver.runAllIterations()
    println()
    println(solver)
    println()
    println("Solver Results Summary:")
    solver.printResults()
    println()
    println("Final (Best) Solution:")
    println(solver.bestSolution.asString())
    println()
    println("Approximate screening:")
    val solutions = solver.bestSolutions.possiblyBest()
    println(solutions)
```

```
    println("Dataframe")
    val df = solver.bestSolutions.toDataFrame()
    df.schema().print()
    df.print()
}
```

The required problem definition is supplied by the previously reviewed `makeRQInventoryModelProblemDefinition()` function, the model builder is supplied by the object that builds the model.

Rather than writing a custom function to react to a solver's progress, you can use one of the pre-built trackers in the `ksl.simopt.solvers.trackers` package. A tracker is a small object that attaches itself as an observer to a solver and reacts automatically to the two kinds of events that every `Solver` emits: an *iteration* event, carrying a `SolverStateSnapshot` (the iteration number, the cumulative number of oracle calls and replications requested so far, and the current and best solutions found so far), and a *life cycle* event, carrying a `SolverStatus` (`INITIALIZED`, `STARTED`, `COMPLETED`, or `ERROR`). All of the built-in trackers extend `AbstractSolverStateTracker`, which handles attaching to and detaching from these two emitters, so that a specific tracker only needs to say what it wants to do with each snapshot or status change.

The `ksl.simopt.solvers.trackers` package currently provides:

- `ConsoleSolverStateTracker`: prints each snapshot and life cycle transition to the console. This is the tracker used in the example.
- `CsvSolverStateTracker`: appends each snapshot to a CSV file, managing the file across multiple solver runs.
- `DataFrameSolverStateTracker`: accumulates snapshots in memory and compiles them into a Kotlin `DataFrame` for further analysis.

Some solvers, such as the `RandomRestartSolver` class, are really two solvers working together: an outer solver that manages the restarts

and an inner solver that performs the search between restarts. For these composite solvers, the package provides nested counterparts, the `NestedConsoleSolverStateTracker`, `NestedCsvSolverStateTracker`, and `NestedDataFrameSolverStateTracker` classes, each of which tracks the outer and inner solver together. The illustration of simulated annealing with random restarts later in this chapter uses a nested tracker. Consult the KSL developer guide for `ksl.simopt` for further details on all six trackers.

Using a tracker takes two lines. Construct the tracker around an already-constructed solver, and tell it to start tracking before running the solver's iterations:

```
val tracker = ConsoleSolverStateTracker(solver)
tracker.startTracking()
solver.runAllIterations()
```

The `startTracking()` function attaches the tracker to the solver's `iterationEmitter` and `lifeCycleEmitter`; calling it again on an already-tracking tracker has no effect. A matching `stopTracking()` function detaches the tracker if you want to silence it before the solver finishes. A solver only builds and emits a snapshot when something is listening, so attaching a tracker is also how you see intermediate iterations. Without one attached, only the final results are available once `runAllIterations()` returns. The frequency of iteration snapshots is controlled by the solver's `snapShotFrequency` property (default 1, i.e. every iteration).

The cross-entropy solver is constructed and all the iterations of the solver are executed. Note that even though the maximum number of iterations was specified as 100, the actual number of iterations may be less due to the stopping criteria. Solvers have a useful `toString()` method and also provide functions and properties to examine the solutions.

Let's take a look at some of the emitted solutions.

```
>>> SOLVER INITIALIZED: Baseline captured.
[Iter:    0 | Oracle:      0] EstObj: 1.79769E+308 | PenObj: 1.79769E+308 | Cur: (In
2147483646.000, -2147483646.000) | Best: (Inventory:Item.initialReorderQty, Inventor
2147483646.000, -2147483646.000)
>>> SOLVER STARTED: Beginning iterations...
[Iter:    1 | Oracle:     34] EstObj:      3.08984 | PenObj:      3.08984 | Cur: (In
[Iter:    2 | Oracle:     67] EstObj:      2.22964 | PenObj:      2.22964 | Cur: (In
[Iter:    3 | Oracle:    101] EstObj:      2.22964 | PenObj:      2.22964 | Cur: (In
[Iter:    4 | Oracle:    136] EstObj:      2.22964 | PenObj:      2.22964 | Cur: (In
[Iter:    5 | Oracle:    170] EstObj:      2.22964 | PenObj:      2.22964 | Cur: (In
[Iter:    6 | Oracle:    203] EstObj:      2.22964 | PenObj:      2.22964 | Cur: (In
[Iter:    7 | Oracle:    238] EstObj:      1.84026 | PenObj:      1.84026 | Cur: (In
```

We see that the solution is improving over the first few iterations. As shown for iterations 30 through 40, we see that the last 5 solutions are the same and thus the cross-entropy solver will stop.

```
[Iter:   30 | Oracle:    939] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   31 | Oracle:    964] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   32 | Oracle:    986] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   33 | Oracle:   1009] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   34 | Oracle:   1031] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   35 | Oracle:   1052] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   36 | Oracle:   1071] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   37 | Oracle:   1087] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   38 | Oracle:   1102] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   39 | Oracle:   1119] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
[Iter:   40 | Oracle:   1135] EstObj:      1.06429 | PenObj:      1.06429 | Cur: (In
```

The `toString()` function prints out the final results of the solver and its basic parameter settings. It took about 5 minutes of execution time for the solver to converge, with a total of 1135 design points evaluated and 56,750

replications requested, with 23,400 executed and 33,350 coming from the cache.

```
================================================================
SOLVER RESULTS
================================================================
Solver          : ID_49
Problem         : InventoryProblem
Termination     : Stopping Criteria Satisfied
Execution Time  : 317153ms

--- Algorithmic Budget ---
Total Iterations        : 40

--- Evaluator Usage ---
Evaluator Invocations   : 40
Design Points Evaluated : 1135

Total Replications Req. : 56750
    Executed by Oracle : 23400
    Satisfied by Cache : 33350

Cache Savings           : 58.8% of simulation budget

--- Trajectory Summary ---
[INITIAL POINT]
Not Applicable (Population-based or not provided)

[FINAL/CURRENT POINT]
Solution(id = 459, status = Valid, evaluation = 40)
  Objectives:
    Estimated = 1.0642933757429534
    Penalized = 1.0642933757429534
```

```
    Granular  = 1.0642933757429534
    Penalty   = 0.0
  Estimated Objective Function Details:
    Name      = Inventory:Item:OrderingAndHoldingCost
    Average   = 1.0642933757429534
    Variance  = 4.433980023127621E-6
    Count     = 50.0
    95%CI     = [1.0636949419163644, 1.0648918095695423]
  Inputs:
    Inventory:Item.initialReorderQty = 4.0
    Inventory:Item.initialReorderPoint = 3.0
  Response Constraints:
      [SATISFIED] Inventory:Item:FillRate | LHS:     0.9608 >= RHS:     0.9500 | Vio

[BEST POINT FOUND]
Solution(id = 459, status = Valid, evaluation = 23)
  Objectives:
    Estimated = 1.0642933757429534
    Penalized = 1.0642933757429534
    Granular  = 1.0642933757429534
    Penalty   = 0.0
  Estimated Objective Function Details:
    Name      = Inventory:Item:OrderingAndHoldingCost
    Average   = 1.0642933757429534
    Variance  = 4.433980023127621E-6
    Count     = 50.0
    95%CI     = [1.0636949419163644, 1.0648918095695423]
  Inputs:
    Inventory:Item.initialReorderQty = 4.0
    Inventory:Item.initialReorderPoint = 3.0
  Response Constraints:
      [SATISFIED] Inventory:Item:FillRate | LHS:     0.9608 >= RHS:     0.9500 | Vio
================================================================
```

The best solution found appears to be at a reorder point of 3 and a reorder quantity of 4, with an ordering and holding cost of 1.064 per month. In addition, the fill rate constraint is estimated as 0.9608 and its constraint is not violated. Notice that confidence intervals on the solutions are provided. The output from the code for screening indicates that the found solution is likely the best **observed** solution because the other solutions are screened out. We have no real guarantee that there is some point that was not observed that could be better. For this small problem, however, we can check. Recall from Section 7.5.2 that with unit Poisson demand the fill rate and the average on-hand inventory of an (r, Q) policy can be computed exactly. Evaluating every policy with $0 \leq r \leq 14$ and $1 \leq Q \leq 29$ shows that $r = 3$ and $Q = 4$ is the least cost policy that meets the fill rate constraint, with an exact ordering and holding cost of 1.0642 and an exact fill rate of 0.9605. Thus, the cross-entropy solver found the optimal policy.

```
    println("Approximate screening:")
    val solutions = solver.bestSolutions.possiblyBest()
    println(solutions)
```

```
Approximate screening:
Solutions:
totalSolutions = 1
capacity = 10
allowInfeasibleSolutions = false
orderedSolutions:
Solution(id = 459, status = Valid, evaluation = 23)
  Objectives:
    Estimated = 1.0642933757429534
    Penalized = 1.0642933757429534
    Granular  = 1.0642933757429534
    Penalty   = 0.0
  Estimated Objective Function Details:
```

```
  Name      = Inventory:Item:OrderingAndHoldingCost
  Average   = 1.0642933757429534
  Variance  = 4.433980023127621E-6
  Count     = 50.0
  95%CI     = [1.0636949419163644, 1.0648918095695423]
Inputs:
  Inventory:Item.initialReorderQty = 4.0
  Inventory:Item.initialReorderPoint = 3.0
Response Constraints:
    [SATISFIED] Inventory:Item:FillRate | LHS:     0.9608 >= RHS:     0.9500 | Vio
```

As noted in the specification of the problem definition, this is an integer ordered problem, where the reorder point and reorder quantity are limited to integer values. As we can note from the output of the optimization process, the evaluated points are limited to integer values. However, you might note that the cross-entropy method estimates the parameters of the sampling distribution (the means and standard deviations of the normal sampler) as continuous values.

Because of the specification of granularity equal to 1.0, the evaluation of these points are rounded to the specified granularity for the evaluation process. If you run the code, you will notice a distinct speed up once (4.0, 3.0) is found. This is due to the fact that the estimates for the distribution parameters via the maximum likelihood estimation process and the application of the smoothing constants have essentially converged to those points which are within the specified granularity.

In addition, the optimization process was run with a solution cache. Thus, every time that those points are requested for evaluation, we get back the same solution without additional simulation runs. That is, the cache avoids the time to execute the simulation model, which quickly causes 5 iterations to have the same solution, meeting the stopping criteria. The use of a solution cache limits the variability of the iterations. An exercise suggests running experiments to understand how caching affects the convergence of solvers.

11.6.2. Illustrating Simulated Annealing with Random Restarts

In this section, we solve the same reorder point, reorder quantity problem using a simulated annealing solver that is restarted at randomly generated points. Simulated annealing may be sensitive to the initial point. Thus, it makes sense to explore different starting points. There is a `createRandomRestart<Family>Solver` factory function for each of the solver families available in the KSL; let's review the one for simulated annealing.

```
fun createRandomRestartSimulatedAnnealingSolver(
    problemDefinition: ProblemDefinition,
    modelBuilder: ModelBuilderIfc,
    maxNumRestarts: Int = defaultMaxRestarts,
    startingPoint: MutableMap<String, Double>? =
     ↪ null,
    temperatureConfiguration:
     ↪ TemperatureConfiguration =
     ↪ TemperatureConfiguration.AutoCalibrate(),
    coolingSchedule: CoolingScheduleIfc =
     ↪ ExponentialCoolingSchedule(
     ↪ defaultInitialTemperature),
    stoppingTemperature: Double = SimulatedAnnealing
     ↪ .defaultStoppingTemperature,
    maxIterations: Int = defaultMaxNumberIterations,
    replicationsPerEvaluation: Int =
     ↪ defaultReplicationsPerEvaluation,
    solutionCache: SolutionCacheIfc =
     ↪ MemorySolutionCache(),
    simulationRunCache: SimulationRunCacheIfc? =
     ↪ null,
    experimentRunParameters:
     ↪ ExperimentRunParametersIfc? = null,
    streamNum: Int = 0,
```

```
            streamProvider: RNStreamProviderIfc =
             ↪ RNStreamProvider(),
            name: String? = null,
            parallelOptions: ParallelEvaluationOptions =
             ↪ ParallelEvaluationOptions(),
            concurrentRestarts: Int = 1,
            concurrentOptions: ConcurrentRunOptions =
             ↪ ConcurrentRunOptions()
        ): RandomRestartSolver {
            val evaluator = Evaluator.createProblemEvaluator(
                problemDefinition = problemDefinition,
↪ modelBuilder = modelBuilder, solutionCache =
↪ solutionCache,
                simulationRunCache = simulationRunCache,
↪ experimentRunParameters = experimentRunParameters,
                parallelOptions = parallelOptions
            )
            // Builds one inner SimulatedAnnealing instance
             ↪ per restart. With the default
            // AutoCalibrate configuration, each restart
             ↪ re-calibrates its own starting
            // temperature to the local landscape around its
             ↪ (randomly chosen) starting point,
            // rather than reusing one temperature across
             ↪ every restart.
            val innerSolverBuilder = { innerEvaluator:
             ↪ EvaluatorIfc, innerName: String? ->
                SimulatedAnnealing(
                    problemDefinition = problemDefinition,
                    evaluator = innerEvaluator,
                    temperatureConfiguration =
↪ temperatureConfiguration,
                    coolingSchedule = coolingSchedule,
```

```
                    stoppingTemperature =
↪  stoppingTemperature,
                    maxIterations = maxIterations,
                    replicationsPerEvaluation =
↪  replicationsPerEvaluation,
                    name = innerName
                )
            }
            // ... assembles a RandomRestartSolver around the
             ↪  inner-solver builder, choosing
            // sequential or concurrent restart execution
             ↪  based on concurrentRestarts.
        }
```

This factory function has a similar structure to the cross-entropy factory function. An evaluator is built from the problem definition and model builder, and an inner `SimulatedAnnealing` solver is built from the temperature, cooling, and stopping parameters. Notice the default `temperatureConfiguration` is `TemperatureConfiguration.AutoCalibrate()`, not a fixed value: each restart calibrates its own starting temperature to the landscape around its own (randomly chosen) starting point, rather than every restart sharing one hand-picked temperature. The `concurrentRestarts` parameter (default 1, meaning sequential, the behavior illustrated here) can be raised to run restarts at the same time, each with its own inner solver instance and private evaluation resources; that capability is covered in the KSL developer guide for the `ksl.simopt` package, along with the CRN-based confirmation stage that can pick a winner across the completed restarts.

Since we already have a model builder and problem definition, for this example we can reuse those functions from the previous example. We will accept the default settings, including the auto-calibrated starting

temperature, except for the maximum number of restarts and the per-restart iteration and replication limits.

```
fun runSimulatedAnnealingWithRestarts(
    simulationRunCache: SimulationRunCacheIfc? = null,
    experimentRunParameters: ExperimentRunParametersIfc? =
     ↪ null
) {
    val problemDefinition =
     ↪ makeRQInventoryModelProblemDefinition()
    val modelBuilder = BuildRQModel

    val solver =
     ↪ Solver.createRandomRestartSimulatedAnnealingSolver(
        problemDefinition = problemDefinition,
        modelBuilder = modelBuilder,
        maxIterations = 100,
        replicationsPerEvaluation = 50,
        simulationRunCache = simulationRunCache,
        experimentRunParameters = experimentRunParameters
    )
    val tracker = NestedConsoleSolverStateTracker(solver,
     ↪ solver.restartingSolver)
    tracker.startTracking()
    solver.runAllIterations()
    println()
    println(solver)
    println()
    println("Solver Results Summary:")
    solver.printResults()
    println()
    println("Final Solution:")
    println(solver.bestSolution.asString())
    println()
```

```
    println("Approximate screening:")
    val solutions = solver.bestSolutions.possiblyBest()
    println(solutions)
}
```

The `RandomRestartSolver` class is exactly the kind of composite solver the `ksl.simopt.solvers.trackers` package's nested trackers were built for: `solver` is the outer, restart-managing solver, and `solver.restartingSolver` is the inner `SimulatedAnnealing` instance that actually searches between restarts. The `NestedConsoleSolverStateTracker` instance attaches to both at once, printing one `MACRO` line per completed restart (its cumulative oracle calls and best point so far) with each of that restart's own iterations printed as an indented `MICRO` line beneath it, so you can see both the restart-level and the annealing-level trajectory in one trace.

The print trace of the 5 restart solver iterations is as follows. Within the trace, we can see the point where the simulated annealing solver started and the resulting solution. It should be clear from the output that the simulated annealing process improves upon the initial starting point's solution.

```
======================================================
>>> EXPERIMENT INITIALIZED: DefaultExperiment (Run 1)
[Run 1: DefaultExperiment | MACRO: 0] Total Oracle Calls: 1 | Best EstObj: 9.712 | B
>>> MACRO EXPERIMENT STARTED
    -> [Micro:    0] EstObj:      24.44 | PenObj:      24.44 | Cur: (Inventory:Item.
    -> [Micro:    1] EstObj:      24.44 | PenObj:      24.44 | Cur: (Inventory:Item.
    -> [Micro:    2] EstObj:      24.44 | PenObj:      24.44 | Cur: (Inventory:Item.
    -> [Micro:    3] EstObj:      24.44 | PenObj:      24.44 | Cur: (Inventory:Item.
...
[Run 1: DefaultExperiment | MACRO: 1] Total Oracle Calls: 132 | Best EstObj: 4.204 |
    -> [Micro:    0] EstObj:      16.07 | PenObj:      16.07 | Cur: (Inventory:Item.
```

```
    -> [Micro:    1] EstObj:      16.07 | PenObj:      16.07 | Cur: (Inventory:Item.
    -> [Micro:    2] EstObj:      16.07 | PenObj:      16.07 | Cur: (Inventory:Item.
    -> [Micro:    3] EstObj:      16.07 | PenObj:      16.07 | Cur: (Inventory:Item.
    -> [Micro:    4] EstObj:      16.07 | PenObj:      16.07 | Cur: (Inventory:Item.
....
[Run 1: DefaultExperiment | MACRO: 4] Total Oracle Calls: 535 | Best EstObj: 2.908 |
    -> [Micro:    0] EstObj:      17.44 | PenObj:      17.44 | Cur: (Inventory:Item.
    -> [Micro:    1] EstObj:      17.18 | PenObj:      17.18 | Cur: (Inventory:Item.
    -> [Micro:    2] EstObj:      17.18 | PenObj:      17.18 | Cur: (Inventory:Item.
```

The solver results indicate that the total number of simulation calls was 187 with 33600 total replications over the approximately 6 minutes of execution time. The random restart solver will remember the initial solutions and the solution found from the underlying solver. We see that a solution with reorder point 4 and reorder quantity of 9 is recommended and the fill-rate constraint is met. This solution is noticeably worse than the one found from the cross-entropy optimization process. Its ordering and holding cost is 1.860, about 75 percent above the optimal 1.064. Thus, five restarts of simulated annealing with these settings were not enough to find the optimal policy.

```
================================================================
SOLVER RESULTS
================================================================
Solver          : ID_49
Problem         : InventoryProblem
Termination     : Execution Halted/Error
Execution Time  : 381164ms

--- Algorithmic Budget ---
Total Iterations        : 5

--- Evaluator Usage ---
```

```
Evaluator Invocations   : 187
Design Points Evaluated : 672

Total Replications Req. : 33600
    Executed by Oracle : 33250
    Satisfied by Cache : 350

Cache Savings           : 1.0% of simulation budget

--- Trajectory Summary ---
[INITIAL POINT]
Solution(id = 6, status = Valid, evaluation = 1)
  Objectives:
    Estimated = 9.712011563243665
    Penalized = 9.712011563243665
    Granular  = 9.712011563243665
    Penalty   = 0.0
  Estimated Objective Function Details:
    Name      = Inventory:Item:OrderingAndHoldingCost
    Average   = 9.712011563243665
    Variance  = 2.6974995778540726E-5
    Count     = 50.0
    95%CI     = [9.71053551701061, 9.71348760947672]
  Inputs:
    Inventory:Item.initialReorderQty = 14.0
    Inventory:Item.initialReorderPoint = 33.0
  Response Constraints:
      [SATISFIED] Inventory:Item:FillRate | LHS:     1.0000 >= RHS:     0.9500 | Vio

[BEST POINT FOUND]
Solution(id = 671, status = Valid, evaluation = 35)
  Objectives:
    Estimated = 1.86020639262809
    Penalized = 1.86020639262809
```

```
    Granular  = 1.86020639262809
    Penalty   = 0.0
  Estimated Objective Function Details:
    Name      = Inventory:Item:OrderingAndHoldingCost
    Average   = 1.86020639262809
    Variance  = 1.9889495729616593E-5
    Count     = 50.0
    95%CI     = [1.858938941698555, 1.8614738435576252]
  Inputs:
    Inventory:Item.initialReorderPoint = 4.0
    Inventory:Item.initialReorderQty = 9.0
  Response Constraints:
      [SATISFIED] Inventory:Item:FillRate | LHS:     0.9945 >= RHS:     0.9500 | Vio
05, 95% HW=  0.0001)
================================================================
```

The screening results indicate that a reorder point of 4 and a reorder quantity of 9 is likely the best of those observed.

```
Approximate screening:
Solutions:
totalSolutions = 1
capacity = 10
allowInfeasibleSolutions = false
orderedSolutions:
Solution(id = 671, status = Valid, evaluation = 35)
  Objectives:
    Estimated = 1.86020639262809
    Penalized = 1.86020639262809
    Granular  = 1.86020639262809
    Penalty   = 0.0
  Estimated Objective Function Details:
    Name      = Inventory:Item:OrderingAndHoldingCost
```

```
    Average   = 1.86020639262809
    Variance  = 1.9889495729616593E-5
    Count     = 50.0
    95%CI     = [1.858938941698555, 1.8614738435576252]
  Inputs:
    Inventory:Item.initialReorderPoint = 4.0
    Inventory:Item.initialReorderQty = 9.0
  Response Constraints:
      [SATISFIED] Inventory:Item:FillRate | LHS:     0.9945 >= RHS:     0.9500 | Vio
05, 95% HW=  0.0001)
```

11.7. Additional Solvers

The chapter has illustrated four solvers in depth: stochastic hill climbing, simulated annealing, cross-entropy, and R-SPLINE, along with how to restart any of them from multiple starting points. The KSL provides four more solver families, each built on a search principle not yet covered, plus a way to run several algorithms against each other instead of committing to one. This section motivates each conceptually and points to its literature; the full parameter surface, with every constructor argument, every pluggable strategy, every default, is documented in the KSL developer guide for the `ksl.simopt` package.

- **Genetic Algorithm.** Like cross-entropy, a genetic algorithm searches with a population rather than a single point, but where cross-entropy's population exists only to re-fit one shared sampling distribution each iteration, a genetic algorithm's population *is* the search: individual solutions persist across generations, the best are selected as parents, pairs of parents are recombined (crossover) to produce offspring, and each offspring is independently perturbed (mutation). This direct manipulation of solution encodings, borrowed from biological evolution (Holland 1975), is a natural fit when the decision space is combinatorial with orderings, subsets, assignments,

where "blending" a shared continuous distribution has no natural meaning. The `Solver.createGeneticAlgorithmSolver(...)` factory function builds one.

- **Particle Swarm Optimization.** Also population-based, but the population is a swarm of particles moving through the continuous search space: each particle's velocity is pulled toward both its own best-seen position and the swarm's best-seen position, damped by an inertia term that favors broad exploration early and local refinement late. The swarm's positions are evaluated as one batched request every iteration, which is why particle swarm is the one solver family in the KSL that defaults to evaluating its candidates in parallel rather than one at a time. The method is due to (Kennedy and Eberhart 1995). The `Solver.createParticleSwarmSolver(...)` factory function builds one.
- **Bayesian Optimization.** Every solver covered so far is a *direct* search: generate candidate points, simulate them, use the results to generate more candidates. Bayesian optimization instead spends most of its effort reasoning about a statistical surrogate which is typically a Gaussian process fit to every point observed so far, and simulates only the single point its surrogate judges most promising to evaluate next (Jones et al. 1998). This approach has some similarities with the cross-entropy method. Because the real (expensive) objective is touched once per iteration rather than once per population member, this is the solver of choice when a single simulation run is costly enough that only a few dozen to a few hundred total evaluations are affordable. This is the opposite regime from the population-based methods, which assume many cheap evaluations per generation. The `Solver.createBayesianOptimizationSolver(...)` factory function builds one.
- **Industrial Strength COMPASS (ISC).** R-SPLINE, covered earlier, converges to a locally optimal solution on integer-ordered problems. ISC extends this idea in two directions at once and is the only solver family in the KSL that can back its recommendation with a formal, statistically defensible guarantee rather than just a good-

looking answer. It runs in three phases: a global phase (a niching genetic algorithm) that identifies several promising regions instead of collapsing onto one, a local phase that runs a COMPASS search (Hong and Nelson 2006) within each region, and a clean-up phase that applies a ranking-and-selection procedure to certify at a chosen confidence level that the returned solution is the best, or acceptably close to the best, among the regions examined. The full three-phase procedure is due to (Xu et al. 2010). The formal guarantee is opt-in: it applies once a meaningful indifference-zone parameter is set on the problem, and runs in a faster, non-guaranteeing mode by default. The `Solver.createISCSolver(...)` factory function builds one.

With eight solver families now available, the `SolverPortfolio` class offers a different kind of choice: rather than committing to one algorithm, race several of them at once on the same problem, each with its own private evaluator so the runs are genuinely independent, and let the results decide. An optional confirmation stage can then re-evaluate the top finishers under common random numbers before declaring a winner. This is the same screening idea already used to compare designed-experiment scenarios in Chapter 10 and to pick among random restarts earlier in this chapter, applied here across different algorithms rather than different starting points or scenarios. Constructing and running a portfolio is covered in the KSL developer guide for the `ksl.simopt` package.

11.8. Summary

The KSL provides a set of classes for developing and using simulation optimization algorithms. The optimization problem can contain both deterministic and stochastic constraints and can include both continuous and discrete variables within the search space.

The following new KSL constructs were discussed in this chapter:

`ProblemDefinition`: This class defines an optimization problem involving an objective function, deterministic linear constraints, deterministic functional constraints, and response constraints.

`PenaltyFunction`: This abstract class defines the interface for penalty functions that can be used to handle constraint violations within an optimization problem, bound to the specific constraint each instance serves.

`InputDefinition`: This class defines the input variables for an optimization problem.

`ConstraintIfc`: This interface defines the behavior of deterministic constraints for an optimization problem. There are two types of deterministic constraints specified by the `LinearConstraint` and `FunctionalConstraint` classes.

`ResponseConstraint`: This class models constraints in a simulation optimization problem that are based on responses from the execution of the simulation model.

`SimulationOracleIfc`: This interface provides a general protocol for translating evaluation requests into evaluation results.

`SimulationProviderIfc`: This interface implements the `SimulationOracleIfc` interface and provides functionality to validate evaluation requests.

`SimulationServiceIfc`: This interface implements the `SimulationOracleIfc` interface to enable the requesting of simulation evaluations from many models.

`EvaluatorIfc`: This interface defines a general protocol for translating evaluation requests into solutions that are needed by simulation optimization solvers.

`Evaluator`: This class is a concrete implementation of the `EvaluatorIfc` interface that executes a model in a local execution.

`EvaluationRequest`: A key class for communicating between solvers and evaluators. It represents the inputs associated with the evaluation, the

outputs needed for the evaluation, and options for running common random numbers and using solution caching.

`ModelInputs`: This class is an intermediary between input variables within a problem definition and the input variables of a simulation model.

`ResponseMap`: This class represents the named responses from the simulation and their estimated value.

`EstimatedResponseIfc`: This interface encapsulates summary statistics for the simulation responses across the replications.

`EstimatedResponse`: This class is a concrete implementation of the `EstimatedResponseIfc` interface.

`Solution`: This fundamental class encapsulates the inputs, outputs, and constraints associated with an evaluation request placed by a solver.

`Solver`: This is an abstract base class that represents the concepts needed by all simulation optimization algorithms.

`StochasticSolver`: This is an abstract class that represents solvers that use randomness in the search process. It has concrete implementations of `StochasticHillClimber`, `SimulatedAnnealing`, `CrossEntropySolver`, `RSplineSolver`, and `RandomRestartSolver`.

`ReplicationPerEvaluationIfc`: This interface allows algorithms to change the number of replications during the search process. It has concrete implementations of `FixedReplicationsPerEvaluation` and `FixedGrowthRateReplicationSchedule`.

`SolutionEqualityIfc`: This interface and its variations allow for different processes to compare or check for solution equality, which is essential in implementing stopping criteria for algorithms.

The KSL provides functionality for using simulation models within the context of simulation optimization. This chapter completes the main body of the book. The appendices that follow provide background material on random number and random variate generation, input distribution

modeling, queueing theory, common probability distributions, and the KSL's utility classes.

11.9. Exercises

Exercise 11.1. Use an RSPLINE solver to solve the reorder point, reorder quantity problem discussed in the chapter. Compare its results to the results from Section 11.6.

Exercise 11.2. Use a simulated annealing solver for the reorder point, reorder quantity problem discussed in the chapter. Run a set of experiments to evaluate the effect of the initial temperature on the performance of the algorithm. Use different starting points to understand if there is an interaction between the starting point and the initial temperature in terms of execution time to achieve a solution.

Exercise 11.3. Suppose a manufacturing system contains 5 machines, each subject to randomly occurring breakdowns. A machine runs for an amount of time that is an exponential random variable with a mean of 10 hours before breaking down. At present there are 2 operators to fix the broken machines. The amount of time that an operator takes to service the machines is exponential with a mean of 4 hours. An operator repairs only 1 machine at a time. If more machines are broken down than the current number of operators, the machines must wait for the next available operator for repair. They form a FIFO queue to wait for the next available

operator. The number of operators required to tend to the machines in order to minimize down time in a cost effective manner is desired. Assume that it costs the system 60 dollars per hour for each machine that is broken down. Each operator is paid 15 dollars per hour regardless of whether they are repairing a machine or not.

a. Formulate an optimization problem that minimizes the total expected cost per hour of operating this system in terms of the number of operators.
b. Build a discrete-event dynamic simulation model to estimate the total expected cost per hour as a function of the number of operators.
c. Apply a simulated annealing solver to find the optimal number of operators to this problem. How confident are you in your result?
d. Did you need simulation optimization to solve this problem?

Exercise 11.4. Consider Exercise 7.13 of Chapter 7. After building a simulation model for the situation, apply the following solvers to find the optimal reorder point and reorder quantity that minimizes the total cost in terms of ordering, holding, and back order costs.

a. Stochastic hill climbing
b. Cross entropy
c. Simulated annealing
d. RSPLINE

Compare the performance of the solvers in terms of execution time, number of oracle calls, number of replications, and solution quality.

Exercise 11.5. A key issue for the simulated annealing algorithm is the specification of the initial temperature. The KSL's `SimulatedAnnealing` solver can auto-calibrate its own starting temperature via `TemperatureConfiguration.AutoCalibrate`, which runs a short random walk over the problem before the search begins. An alternative approach is discussed in this stack exchange discussion, which references (Ben-Ameur 2004). Read the paper, and compare its recommended initial temperature to the KSL's auto-calibrated value for the reorder point, reorder quantity problem. Do the two approaches agree? Does the choice affect the solver's execution time or the quality of the solution found?

Exercise 11.6. Consider the application of the cross-entropy solver to the reorder point, reorder quantity inventory problem in Section 11.6.1. Run the solver with and without using a solution cache. Compare the performance of the cases in terms of execution time, number of oracle calls, number of replications, and solution quality.

Exercise 11.7. In a continuous review (s, S) inventory control system, the system operates essentially the same as an (r, Q) control policy except that when the reorder point, s, is reached, the system instead orders enough inventory to reach a maximum inventory level of S. Thus, the amount of the order will vary with each order. Repeat the analysis of Exercise 11.4 assuming that the SKU is controlled with a (s, S) policy. Assume that s = 4 and S = 7.

a. Develop a KSL simulation model for this system and simulate this system and compare the cost results to the results found in Exercise 7.13.

b. Apply the solvers suggested in Exercise 11.4 and evaluate their performance on this problem.

Exercise 11.8. Consider a shop that produces and stocks items. The items are produced in lots. The demand process for each item is Poisson with the rate given below. Back ordering is permitted. The pertinent data for each of the items is given in the following table. The carrying charge on each item is 0.20. The lead time is considered constant with the value provided in the table. Assume that there are 360 days in each year.

Item	1	2	3	4	5	6
Demand Rate (units per year)	1000	500	2000	100	50	250
Unit cost (dollars per unit)	20	100	50	500	1000	200
Lead time mean (days)	2	5	5	6	14	8

Suppose that management desires to set the optimal (r, Q) policy variables for each of the items so as to minimize the total holding cost and ordering costs. Assume that it costs approximately $5 to prepare and process each order. In addition, they desire to have the system fill rate to be no lower than 80% and no individual item's fill rate to be below 60%. They also desire to have the average customer wait time (for any part) to be less than or equal to 3 days. Simulate this system and use simulation optimization to find an optimal policy recommendation for the items. You might start by analyzing the parts individually to have initial starting values for the optimization search.

Exercise 11.9. As illustrated in Figure 11.21, a multi-echelon system consists of an external supplier, a warehouse, and a set of retailers. Assume that there is only 1 item type stocked at each location within the system. The warehouse supplies each of the four retailers when they make a replenishment request.

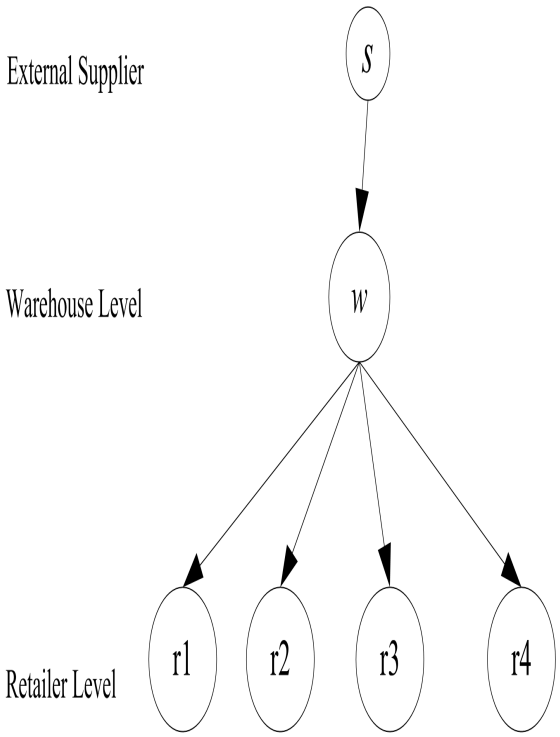


Figure 11.21.: Multi-Echelon Inventory System

Suppose that the retail locations do not allow back ordering. That is, customers who arrive when there is not enough to fill their demand are lost. In addition, suppose that the amount demanded for each customer is governed by a shifted geometric random variable with parameter p = 0.9. The warehouse and the retailers control the SKU with (s, S) policies as

shown in the following table.

	Policy (s, S)	Demand Poisson (mean rate per day)
Retailer 1	(1, 3)	(0.2)
Retailer 2	(1, 5)	(0.1)
Retailer 3	(2, 6)	(0.3)
Retailer 4	(1, 4)	(0.4)
Warehouse	(4, 16)	

The overall rate to the retailers is 1 demand per day, but individually they experience the rates as shown in the table. In other words, the retailers are not identical in terms of the policy or the demand that they experience. The lead time is 5 days for the warehouse and 1 day for each retailer. The warehouse allows back ordering.

a. Develop an KSL model for this situation. Estimate the expected number of lost sales per year for each retailer, the average amount of inventory on hand at the retailers and at the warehouse, the fill rate for each retailer and for the warehouse, and the average number of items back ordered at the warehouse. Run the model for 10 replications of 3960 days with a warm up of 360 days.
b. The unit cost of the item is 10 dollars and the holding charge at each retailer is (10, 15, 20, 25) percent per unit per year for the four retailers, respectively. Suppose the holding charge at the warehouse is 15 percent per unit per year. Assume that the ordering cost at each location is approximately 2 dollars for each order placed. Develop an optimization model that will determine the optimal inventory policy parameters for each location that minimizes the overall total cost of inventory ordering and holding costs subject to at least 96 percent fill rates at each of the retailer locations and at least a 90 percent fill rate at the warehouse.

A. Generating Pseudo-Random Numbers and Random Variates

Learning Objectives

- To be able to describe and use linear congruential pseudo-random number generation methods
- To be able to describe current state of the art pseudo-random number generation methods
- To be able to define and use key terms in pseudo-random number generation methods such as streams, seeds, period, etc.
- To be able to explain the key issues in pseudo-random number testing
- To be able to derive and implement an inverse cumulative distribution function based random variate generation algorithm
- To be able to explain and implement the convolution algorithm for random variate generation
- To be able to explain and implement the acceptance rejection algorithm for random variate generation

Randomness in simulation is often modeled by using random variables and probability distributions. Thus, simulation languages require the ability to generate random variates. A random variate is an instance (or realization) of a random variable. In this section, you will learn how simulation languages allow for the generation of randomness. Generating

randomness requires algorithms that permit sequences of numbers to act as the underlying source of randomness within the model. Then, given a good source of randomness, techniques have been established that permit the sequences to be transformed so that they can represent a wide variety of random variables (e.g. normal, Poisson, etc.). The algorithms that govern these procedures are described in the second part of this chapter.

A.1. Pseudo Random Numbers

This section indicates how uniformly distributed random numbers over the range from 0 to 1 are obtained within simulation programs. While commercial simulation packages provide substantial capabilities for generating random numbers, we still need to understand how this process works for the following reasons:

1. The random numbers within a simulation experiment might need to be controlled in order to take advantage of them to improve decision making.
2. In some situations, the commercial package does not have ready made functions for generating the desired random variables. In these situations, you will have to implement an algorithm to generate the random variates.

In addition, simulation is much broader than just using a commercial package. You can perform simulation in any computer language and spreadsheets. The informed modeler should know how the key inputs to simulation models are generated.

In simulation, large amount of cheap (easily computed) random numbers are required. In general, consider how random numbers might be obtained:

1. Dice, coins, colored balls
2. Specially designed electronic equipment

3. Algorithms

Clearly, within the context of computer simulation, it might be best to rely on algorithms; however, if an algorithm is used to generate the random numbers then they will not be truly random. For this reason, the random numbers that are used in computer simulation are called *pseudo random*.

Definition A.1 (Pseudo-Random Numbers)**.** A sequence of pseudo-random numbers, U_i, is a deterministic sequence of numbers in $(0, 1)$ having the same relevant statistical properties as a sequence of truly random $U(0, 1)$ numbers.((Ripley 1987))

A set of statistical tests are performed on the pseudo-random numbers generated from algorithms in order to indicate that their properties are not significantly different from a true set of $U(0, 1)$ random numbers. The *algorithms* that produce pseudo-random numbers are called *random number generators*. In addition to passing a battery of statistical tests, the random number generators need to be fast and they need to be able to reproduce a sequence of numbers if and when necessary.

The following section discusses random number generation methods. The approach will be practical, with just enough theory to motivate future study of this area and to allow you to understand the important implications of random number generation. A more rigorous treatment of random number and random variable generation can be found such texts as (Fishman 2006) and (Devroye 1986).

A.1.1. Random Number Generators

Over the history of scientific computing, there have been a wide variety of techniques and algorithms proposed and used for generating pseudo-random numbers. A common technique that has been used (and is still in use) within a number of simulation environments is discussed in this text. Some new types of generators that have been recently adopted within many simulation environments, especially the one used within the KSL and Arena, will also be briefly discussed.

A linear congruential generator (LCG) is a recursive algorithm for producing a sequence of pseudo random numbers. Each new pseudo random number from the algorithm depends on the previous pseudo random number. Thus, a starting value called the *seed* is required. Given the value of the seed, the rest of the sequence of pseudo random numbers can be completely determined by the algorithm. The basic definition of an LCG is as follows

---

Definition A.2 (Linear Congruential Generator)**.** A LCG defines a sequence of integers, $R_0, R_1, ...$ between 0 and $m-1$ according to the following recursive relationship:

$$R_{i+1} = (aR_i + c) \bmod m$$

where R_0 is called the seed of the sequence, a is called the constant multiplier, c is called the increment, and m is called the modulus. (m, a, c, R_0) are integers with $a > 0$, $c \geq 0$, $m > a$, $m > c$, $m > R_0$, and $0 \leq R_i \leq m-1$.

To compute a corresponding pseudo-random uniform number, we use

$$U_i = \frac{R_i}{m}$$

Notice that an LCG defines a sequence of integers and subsequently a sequence of real (rational) numbers that can be considered pseudo random numbers. Remember that pseudo random numbers are those that can "fool" a battery of statistical tests. The choice of the seed, constant multiplier, increment, and modulus, i.e. the parameters of the LCG, will determine the properties of the sequences produced by the generator. With properly chosen parameters, an LCG can be made to produce pseudo random numbers. To make this concrete, let's look at a simple example of an LCG.

Example A.1 (Simple LCG Example)**.** Consider an LCG with the following parameters ($m = 8$, $a = 5$, $c = 1$, $R_0 = 5$). Compute the first nine values for R_i and U_i from the defined sequence.

Let's first remember how to compute using the mod operator. The mod operator is defined as:

$$z = y \bmod m = y - m \left\lfloor \frac{y}{m} \right\rfloor$$

where $\lfloor x \rfloor$ is the floor operator, which returns the greatest integer that is less than or equal to x. For example,

$$\begin{aligned} z &= 17 \bmod 3 \\ &= 17 - 3\left\lfloor \frac{17}{3} \right\rfloor \\ &= 17 - 3\lfloor 5.\overline{66} \rfloor \\ &= 17 - 3 \times 5 = 2 \end{aligned}$$

Thus, the mod operator returns the integer remainder (including zero) when $y \geq m$ and y when $y < m$. For example, $z = 6 \bmod 9 = 6 - 9\left\lfloor \frac{6}{9} \right\rfloor = 6 - 9 \times 0 = 6$.

Using the parameters of the LCG, the pseudo-random numbers are:

$$\begin{aligned} R_1 &= (5R_0 + 1) \bmod 8 = 26 \bmod 8 = 2 \Rightarrow U_1 = 0.25 \\ R_2 &= (5R_1 + 1) \bmod 8 = 11 \bmod 8 = 3 \Rightarrow U_2 = 0.375 \\ R_3 &= (5R_2 + 1) \bmod 8 = 16 \bmod 8 = 0 \Rightarrow U_3 = 0.0 \\ R_4 &= (5R_3 + 1) \bmod 8 = 1 \bmod 8 = 1 \Rightarrow U_4 = 0.125 \\ R_5 &= 6 \Rightarrow U_5 = 0.75 \\ R_6 &= 7 \Rightarrow U_6 = 0.875 \\ R_7 &= 4 \Rightarrow U_7 = 0.5 \\ R_8 &= 5 \Rightarrow U_8 = 0.625 \\ R_9 &= 2 \Rightarrow U_9 = 0.25 \end{aligned}$$

In the previous example, the U_i are simple fractions involving $m = 8$. Certainly, this sequence does not appear very random. The U_i can only take on rational values in the range, $0, \frac{1}{m}, \frac{2}{m}, \frac{3}{m}, \ldots, \frac{(m-1)}{m}$ since $0 \leq R_i \leq m-1$. This implies that if m is small there will be gaps on the interval $[0, 1)$, and if m is large then the U_i will be more densely distributed on $[0, 1)$.

Notice that if a sequence generates the same value as a previously generated value then the sequence will repeat or cycle. An important property of

a LCG is that it has a long cycle, as close to length m as possible. The length of the cycle is called the *period* of the LCG. Ideally the period of the LCG is equal to m. If this occurs, the LCG is said to achieve its full period. As can be seen in the example, the LCG is full period. Until recently, most computers were 32 bit machines and thus a common value for m is $2^{31} - 1 = 2,147,483,647$, which represents the largest integer number on a 32 bit computer using 2's complement integer arithmetic. This choice of m also happens to be a prime number, which leads to special properties.

A proper choice of the parameters of the LCG will allow desirable pseudo random number properties to be obtained. The following result due to (Hull and Dobell 1962), see also (Law 2007), indicates how to check if a LCG will have the largest possible cycle.

Theorem A.1 (LCG Theorem)**.** *An LCG has full period if and only if the following three conditions hold: (1) The only positive integer that (exactly) divides both m and c is 1 (i.e. c and m have no common factors other than 1), (2) If q is a prime number that divides m then q should divide $(a-1)$. (i.e. $(a-1)$ is a multiple of every prime number that divides m), and (3) If 4 divides m, then 4 should divide $(a-1)$. (i.e. $(a-1)$ is a multiple of 4 if m is a multiple of 4)*

Now, let's apply this theorem to the example LCG and check whether or not it should obtain full period. To apply the theorem, you must check if each of the three conditions holds for the generator.

- Condition 1: c and m have no common factors other than 1.
  The factors of $m = 8$ are $(1, 2, 4, 8)$, since $c = 1$ (with factor 1) condition 1 is true.

- Condition 2: $(a-1)$ is a multiple of every prime number that divides m. The first few prime numbers are (1, 2, 3, 5, 7). The prime numbers, q, that divide $m = 8$ are $(q = 1, 2)$. Since $a = 5$ and $(a-1) = 4$, clearly $q = 1$ divides 4 and $q = 2$ divides 4. Thus, condition 2 is true.
- Condition 3: If 4 divides m, then 4 should divide $(a-1)$.
  Since $m = 8$, clearly 4 divides m. Also, 4 divides $(a-1) = 4$. Thus, condition 3 holds.

Since all three conditions hold, the LCG achieves full period.

> **Note**
>
> The theorem only tells us when a specification of (a, c, m) will obtain full period. In case (3), it does not tell us anything about what happens if 4 does not divide m. If 4 divides m, and we pick a so that 4 divides $(a-1)$, then the condition will be met. If 4 does not divide m, then the theorem really does not tell us one way or the other whether the LCG obtains full period.

There are some simplifying conditions, see Banks et al. (2005), which allow for easier application of the theorem. For $m = 2^b$, (m a power of 2) and c not equal to 0, the longest possible period is m and can be achieved provided that c is chosen so that the greatest common factor of c and m is 1 and $a = 4k + 1$ where k is an integer. The previous example LCG satisfies this situation.

For $m = 2^b$ and $c = 0$, the longest possible period is $(m/4)$ and can be achieved provided that the initial seed, R_0 is odd and $a = 8k + 3$ or $a = 8k + 5$ where $k = 0, 1, 2, \cdots$.

The case of m a prime number and $c = 0$, defines a special case of the LCG called a *prime modulus multiplicative linear congruential generator* (PMMLCG). For this case, the longest possible period is $m - 1$ and can

be achieved if the smallest integer, k, such that $a^k - 1$ is divisible by m is $m - 1$.

Thirty-Two bit computers have been very common for over 20 years. In addition, $2^{31} - 1 = 2,147,483,647$ is a prime number. Because of this, $2^{31} - 1$ has been the choice for m with $c = 0$. Two common values for the multiplier, a, have been:

$$a = 630,360,016$$
$$a = 16,807$$

The latter of which was used within many simulation packages for a number of years. Notice that for PMMLCG's the full period cannot be achieved (because $c = 0$), but with the proper selection of the multiplier, the next best period length of $m - 1$ can be obtained. In addition, for this case $R_0 \in \{1, 2, \ldots, m - 1\}$ and thus $U_i \in (0, 1)$. The limitation of $U_i \in (0, 1)$ is very useful when generating random variables from various probability distributions, since 0 cannot be realized. When using an LCG, you must supply a starting seed as an initial value for the algorithm. This seed determines the sequence that will come out of the generator when it is called within software. Since generators cycle, you can think of the sequence as a big circular list as indicated in Figure A.1.

Starting with seed $R_0 = 5$, you get a sequence $\{2, 3, 0, 1, 6, 7, 4, 5\}$. Starting with seed, $R_0 = 1$, you get the sequence $\{6, 7, 4, 5, 2, 3, 0, 1\}$. Notice that these two sequences overlap with each other, but that the first half $\{2, 3, 0, 1\}$ and the second half $\{6, 7, 4, 5\}$ of the sequence do not overlap. If you only use 4 random numbers from each of these two *subsequences* then the numbers will not overlap. This leads to the definition of a stream:

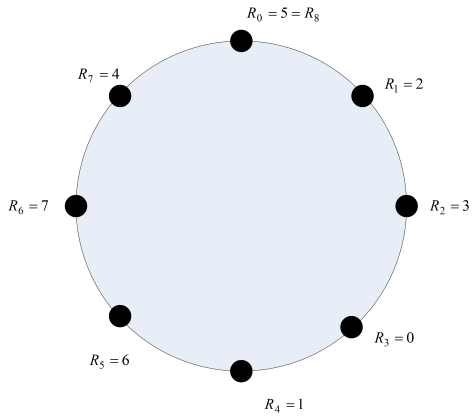


Figure A.1.: Sequence for Simple LCG Example

Definition A.3 (Stream)**.** The subsequence of random numbers generated from a given seed is called a random number stream.

You can take the sequence produced by the random number generator and divide it up into subsequences by associating certain seeds with streams. You can call the first subsequence stream 1 and the second subsequence stream 2, and so forth. Each stream can be further divided into subsequences or sub-streams of non-overlapping random numbers.

In this simple example, it is easy to remember that stream 1 is defined by seed, $R_0 = 5$, but when m is large, the seeds will be large integer numbers, e.g. $R_0 = 123098345$. It is difficult to remember such large numbers. Rather than remember this huge integer, an assignment of stream numbers to seeds is made. Then, the sequence can be reference by its stream number. Naturally, if you are going to associate seeds with streams you would want to divide the entire sequence so that the number of non-overlapping random numbers in each stream is quite large. This

ensures that as a particular stream is used that there is very little chance of continuing into the next stream. Clearly, you want m to be as large as possible and to have many streams that contain as large as possible number of non-overlapping random numbers. With today's modern computers even m is $2^{31} - 1 = 2,147,483,647$ is not very big. For large simulations, you can easily run through all these random numbers.

Random number generators in computer simulation languages come with a default set of streams that divide the "circle" up into independent sets of random numbers. The streams are only independent if you do not use up all the random numbers within the subsequence. These streams allow the randomness associated with a simulation to be controlled. During the simulation, you can associate a specific stream with specific random processes in the model. This has the advantage of allowing you to check if the random numbers are causing significant differences in the outputs. In addition, this allows the random numbers used across alternative simulations to be better synchronized.

Now a common question for beginners using random number generators can be answered. That is, *If the simulation is using random numbers, why to I get the same results each time I run my program?* The corollary to this question is, *If I want to get different random results each time I run my program, how do I do it?* The answer to the first question is that the underlying random number generator is starting with the same seed each time you run your program. Thus, your program will use the same pseudo random numbers today as it did yesterday and the day before, etc. The answer to the corollary question is that you must tell the random number generator to use a different seed (or alternatively a different stream) if you want different invocations of the program to produce different results. The latter is not necessarily a desirable goal. For example, when developing your simulation programs, it is desirable to have repeatable results so that you can know that your program is working correctly. Unfortunately, many novices have heard about using the computer clock to "randomly" set the seed for a simulation program. This is a *bad* idea and very much not recommended in our context. This idea is more appropriate within

a gaming simulation, in order to allow the human gamer to experience different random sequences.

Given current computing power, the previously discussed PMMLCGs are insufficient since it is likely that all the 2 billion or so of the random numbers would be used in performing serious simulation studies. Thus, a new generation of random number generators was developed that have extremely long periods. The random number generator described in L'Ecuyer et al. (2002) is one example of such a generator. It is based on the combination of two multiple recursive generators resulting in a period of approximately 3.1×10^{57}. This is the same generator that is now used in many commercial simulation packages. The generator as defined in (Law 2007) is:

$$
\begin{aligned}
R_{1,i} &= (1,403,580R_{1,i-2} - 810,728R_{1,i-3})[\mathrm{mod}(2^{32} - 209)] \\
R_{2,i} &= (527,612R_{2,i-1} - 1,370,589R_{2,i-3})[\mathrm{mod}(2^{32} - 22,853)] \\
Y_i &= (R_{1,i} - R_{2,i})[\mathrm{mod}(2^{32} - 209)] \\
U_i &= \frac{Y_i}{2^{32} - 209}
\end{aligned}
$$

The generator takes as its initial seed a vector of six initial values $(R_{1,0}, R_{1,1}, R_{1,2}, R_{2,0}, R_{2,1}, R_{2,2})$. The first initially generated value, U_i, will start at index 3. To produce five pseudo random numbers using this generator we need an initial seed vector, such as:

$$\{R_{1,0}, R_{1,1}, R_{1,2}, R_{2,0}, R_{2,1}, R_{2,2}\} = \{12345, 12345, 12345, 12345, 12345, 12345\}$$

Using the recursive equations, the resulting random numbers are as follows:

	i=3	i=4	i=5	i=6	i=7
$Z_{1,i-3} =$	12345	12345	12345	3023790853	3023790853
$Z_{1,i-2} =$	12345	12345	3023790853	3023790853	3385359573

	i=3	i=4	i=5	i=6	i=7
$Z_{1,i-1} =$	12345	3023790853	3023790853	3385359573	1322208174
$Z_{2,i-3} =$	12345	12345	12345	2478282264	1655725443
$Z_{2,i-2} =$	12345	12345	2478282264	1655725443	2057415812
$Z_{2,i-1} =$	12345	2478282264	1655725443	2057415812	2070190165
$Z_{1,i} =$	3023790853	3023790853	3385359573	1322208174	2930192941
$Z_{2,i} =$	2478282264	1655725443	2057415812	2070190165	1978299747
$Y_i =$	545508589	1368065410	1327943761	3546985096	951893194
$U_i =$	0.127011122076	0.318527565471	0.309186015655	0.82584686312	0.221629915834

While it is beyond the scope of this text to explore the theoretical underpinnings of this generator, it is important to note that the use of this new generator is conceptually similar to that which has already been described. The generator allows multiple independent streams to be defined along with sub-streams.

> **Note**
>
> A random number stream is a sub-sequence of pseudo-random numbers that start at particular place with a larger sequence of pseudo-random numbers. The starting point of a sequence of pseudo-random numbers is called the *seed*. A seed allows us to pick a particular stream. Having multiple streams is useful to assign different streams to different sources of randomness within a model. Streams can be further divided into sub-streams. This facilitates the control of the use of pseudo-random numbers when performing experiments.

The fantastic thing about this generator is the sheer size of the period. Based on their analysis, L'Ecuyer et al. (2002) state that it will be "approximately 219 years into the future before average desktop computers will have the capability to exhaust the cycle of the (generator) in a year of continuous computing." In addition to the period length, the generator

has an enormous number of streams, approximately 1.8×10^{19} with stream lengths of 1.7×10^{38} and sub-streams of length 7.6×10^{22} numbering at 2.3×10^{15} per stream. Clearly, with these properties, you do not have to worry about overlapping random numbers when performing simulation experiments. The generator was subjected to a rigorous battery of statistical tests and is known to have excellent statistical properties. The subject of modeling and testing different distributions is deferred to a separate part of this book.

A.2. Generating Random Variates from Distributions

In simulation, pseudo random numbers serve as the foundation for generating samples from probability distribution models. We will now assume that the random number generator has been rigorously tested and that it produces sequences of $U_i \sim U(0,1)$ numbers. We now want to take the $U_i \sim U(0,1)$ and utilize them to generate from probability distributions.

The realized value from a probability distribution is called a random variate. Simulations use many different probability distributions as inputs. Thus, methods for generating random variates from distributions are required. Different distributions may require different algorithms due to the challenges of efficiently producing the random variables. Therefore, we need to know how to generate samples from probability distributions. In generating random variates the goal is to produce samples X_i from a distribution $F(x)$ given a source of random numbers, $U_i \sim U(0,1)$. There are four basic strategies or methods for producing random variates:

1. Inverse transform or inverse cumulative distribution function (CDF) method
2. Convolution
3. Acceptance/Rejection
4. Mixture and Truncated Distributions

The following sections discuss each of these methods.

A.2.1. Inverse Transform Method

The inverse transform method is the preferred method for generating random variates provided that the inverse transform of the cumulative distribution function can be easily derived or computed numerically. The key advantage for the inverse transform method is that for every U_i use a corresponding X_i will be generated. That is, there is a one-to-one mapping between the pseudo-random number u_i and the generated variate x_i.

The inverse transform technique utilizes the inverse of the cumulative distribution function as illustrated in Figure A.2, will illustrates simple cumulative distribution function. First, generate a number, u_i between 0 and 1 (along the U axis), then find the corresponding x_i coordinate by using $F^{-1}(\cdot)$. For various values of u_i, the x_i will be properly 'distributed' along the x-axis. The beauty of this method is that there is a one to one mapping between u_i and x_i. In other words, for each u_i there is a unique x_i because of the monotone property of the CDF.

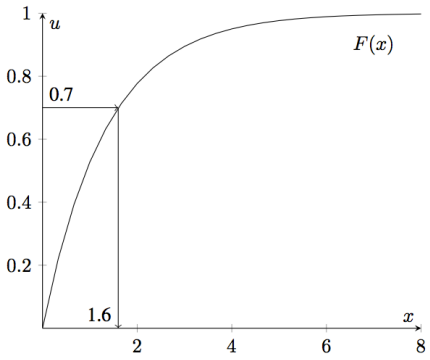


Figure A.2.: Inverse Transform Method

The idea illustrated in Figure A.2 is based on the following theorem.

Theorem A.2 (Inverse Transform)**.** *Let X be a random variable with $X \sim F(x)$. Define another random variable Y such that $Y = F(X)$. That is, Y is determined by evaluating the function $F(\cdot)$ at the value X. If Y is defined in this manner, then $Y \sim U(0, 1)$.*

The proof utilizes the definition of the cumulative distribution function to derive the CDF for Y.

$$\begin{aligned} F(y) &= P\{Y \leq y\} \\ &= P\{F(X) \leq y\} \text{ substitute for } Y \\ &= P\left\{F^{-1}(F(X)) \leq F^{-1}(y)\right\} \text{ apply inverse} \\ &= P\left\{X \leq F^{-1}(y)\right\} \text{ definition of inverse} \\ &= F(F^{-1}(y)) \text{ definition of CDF} \\ &= y \text{ definition of inverse} \end{aligned}$$

Since $P(Y \leq y) = y$ defines a $U(0, 1)$ random variable, the proof is complete.

This result also works in reverse if you start with a uniformly distributed random variable then you can get a random variable with the distribution of $F(x)$. The idea is to generate $U_i \sim U(0, 1)$ and then to use the inverse cumulative distribution function to transform the random number to the appropriately distributed random variate.

Let's assume that we have a function, *randU01()*, that will provide pseudo-random numbers on the range (0,1). Then, the following presents the pseudo-code for the inverse transform algorithm.

1. $u = rand01()$
2. $x = F^{-1}(u)$
3. return x

Line 1 generates a uniform number. Line 2 takes the inverse of u and line 3 returns the random variate. The following example illustrates the inverse transform method for the exponential distribution.

The exponential distribution is often used to model the time until and event (e.g. time until failure, time until an arrival etc.) and has the following probability density function:

$$f(x) = \begin{cases} 0.0 & \text{if}\{x < 0\} \\ \lambda e^{-\lambda x} & \text{if}\{x \geq 0\} \end{cases}$$

with

$$E[X] = \frac{1}{\lambda}$$
$$Var[X] = \frac{1}{\lambda^2}$$

Example A.2 (Generating Exponential Random Variates)**.** Consider a random variable, X, that represents the time until failure for a machine tool. Suppose X is exponentially distributed with an expected value of $1.\overline{33}$. Generate a random variate for the time until the first failure using a uniformly distributed value of $u = 0.7$.

Solution for Example A.2 In order to solve this problem, we must first compute the CDF for the exponential distribution. For any value, $b < 0$, we have by definition:

$$F(b) = P\{X \leq b\} = \int_{-\infty}^{b} f(x)\, dx = \int_{-\infty}^{b} 0 dx = 0$$

For any value $b \geq 0$,

$$\begin{aligned} F(b) = P\{X \leq b\} &= \int_{-\infty}^{b} f(x)dx \\ &= \int_{-\infty}^{0} f(x)dx + \int_{0}^{b} f(x)dx \\ &= \int_{0}^{b} \lambda e^{\lambda x} dx = -\int_{0}^{b} e^{-\lambda x}(-\lambda)dx \\ &= -e^{-\lambda x}\Big|_{0}^{b} = -e^{-\lambda b} - (-e^{0}) = 1 - e^{-\lambda b} \end{aligned}$$

Thus, the CDF of the exponential distribution is:

$$F(x) = \begin{cases} 0 & \text{if}\{x < 0\} \\ 1 - e^{-\lambda x} & \text{if}\{x \geq 0\} \end{cases}$$

Now the inverse of the CDF can be derived by setting $u = F(x)$ and solving for $x = F^{-1}(u)$.

$$\begin{aligned} u &= 1 - e^{-\lambda x} \\ x &= \frac{-1}{\lambda} \ln(1 - u) = F^{-1}(u) \end{aligned}$$

For Example A.2, we have that $E[X] = 1.\overline{33}$. Since $E[X] = 1/\lambda$ for the exponential distribution, we have that $\lambda = 0.75$. Since $u = 0.7$, then the generated random variate, x, would be:

$$x = \frac{-1}{0.75} \ln(1 - 0.7) = 1.6053$$

Thus, if we let $\theta = E[X]$, the formula for generating an exponential random variate is simply:

$$x = \frac{-1}{\lambda} \ln(1 - u) = -\theta \ln(1 - u) \tag{A.1}$$

In the following pseudo-code, we assume that *randU01()* is a function that returns a uniformly distributed random number over the range (0,1).

1. $u = randU01())$
2. $x = \frac{-1}{\lambda} \ln(1 - u)$
3. return x

Thus, the key to applying the inverse transform technique for generating random variates is to be able to first derive the cumulative distribution function (CDF) and then to derive its inverse function. It turns out that for many common distributions, the CDF and inverse CDF well known.

The uniform distribution over an interval (a, b) is often used to model situations where the analyst does not have much information and can only assume that the outcome is equally likely over a range of values. The uniform distribution has the following characteristics:

$$X \sim \text{Uniform}(a, b)$$

$$f(x) = \begin{cases} \frac{1}{b-a} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

$$E[X] = \frac{a+b}{2}$$
$$Var[X] = \frac{(b-a)^2}{12}$$

$$F(x) = \begin{cases} 0.0 & x < a \\ \frac{x-a}{b-a} & a \leq x \leq b \\ 1.0 & x > b \end{cases}$$

Example A.3 (Inverse CDF for Uniform Distribution)**.** Consider a random variable, X, that represents the amount of grass clippings in a mower bag in pounds. Suppose the random variable is uniformly distributed between 5 and 35 pounds. Generate a random variate for the weight using a pseudo-random number of $u = 0.25$.

Solution for Example A.3 To solve this problem, we must determine the inverse CDF algorithm for the $U(a, b)$ distribution. The inverse of the CDF can be derived by setting $u = F(x)$ and solving for $x = F^{-1}(u)$.

$$\begin{aligned} u &= \frac{x-a}{b-a} \\ u(b-a) &= x-a \\ x &= a + u(b-a) = F^{-1}(u) \end{aligned}$$

For the example, we have that $a = 5$ and $b = 35$ and $u = 0.25$, then the generated x would be:

$$F^{-1}(u) = x = 5 + 0.25 \times (35-5) = 5 + 7.5 = 12.5$$

Notice how the value of u is first scaled on the range $(0, b-a)$ and then shifted to the range (a, b). For the uniform distribution this transformation is linear because of the form of its $F(x)$.

1. $u = randU01()$
2. $x = a + u(b-a)$
3. return x

For the previous distributions a closed form representation of the cumulative distribution function was available. If the cumulative distribution function can be inverted, then the inverse transform method can be easily used to generate random variates from the distribution. If no closed form analytical formula is available for the inverse cumulative distribution function, then often we can resort to numerical methods to implement the function. For example, the normal distribution is an extremely useful distribution and numerical methods have been devised to provide its inverse cumulative distribution function.

The inverse CDF method also works for discrete distributions. For a discrete random variable, X, with possible values $x_1, x_2, \ldots, x_n$ (n may be infinite), the probability distribution is called the probability mass function (PMF) and denoted:

$$f(x_i) = P(X = x_i)$$

where $f(x_i) \geq 0$ for all x_i and

$$\sum_{i=1}^{n} f(x_i) = 1$$

The cumulative distribution function is

$$F(x) = P\left(X \leq x\right) = \sum_{x_i \leq x} f\left(x_i\right)$$

and satisfies, $0 \leq F\left(x\right) \leq 1$, and if $x \leq y$ then $F(x) \leq F(y)$.

In order to apply the inverse transform method to discrete distributions, the cumulative distribution function can be searched to find the value of x associated with the given u. This process is illustrated in the following example.

Example A.4 (Discrete Empirical Distribution)**.** Suppose you have a random variable, X, with the following discrete probability mass function and cumulative distribution function.

x_i	1	2	3	4
$f(x_i)$	0.4	0.3	0.2	0.1

Plot the probability mass function and cumulative distribution function for this random variable. Then, develop an inverse cumulative distribution function for generating from this distribution. Finally, given $u_1 = 0.934$ and $u_2 = 0.1582$ are pseudo-random numbers, generate the two corresponding random variates from this PMF.

Solution for Example A.4 To solve this example, we must understand the functional form of the PMF and CDF, which are given as follows:

$$P\{X = x\} = \begin{cases} 0.4 & \text{x} = 1 \\ 0.3 & \text{x} = 2 \\ 0.2 & \text{x} = 3 \\ 0.1 & \text{x} = 4 \end{cases}$$

$$F(x) = \begin{cases} 0.0 & \text{if } x < 1 \\ 0.4 & \text{if } 1 \leq x < 2 \\ 0.7 & \text{if } 2 \leq x < 3 \\ 0.9 & \text{if } 3 \leq x < 4 \\ 1.0 & \text{if } x \geq 4 \end{cases}$$

Figure A.3 illustrates the CDF for this discrete distribution.

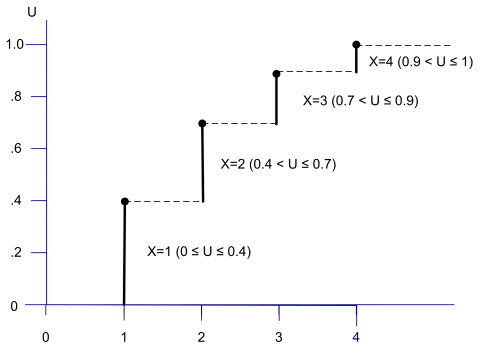


Figure A.3.: Example Empirical CDF

Examining Figure A.3 indicates that for any value of u_i in the interval, $(0.4, 0.7]$ you get an x_i of 2. Thus, generating random numbers from this distribution can be accomplished by using the inverse of the cumulative distribution function.

$$F^{-1}(u) = \begin{cases} 1 & \text{if } 0.0 \leq u \leq 0.4 \\ 2 & \text{if } 0.4 < u \leq 0.7 \\ 3 & \text{if } 0.7 < u \leq 0.9 \\ 4 & \text{if } 0.9 < u \leq 1.0 \end{cases}$$

Suppose $u_1 = 0.934$, then by $F^{-1}(u)$, $x = 4$. If $u_2 = 0.1582$, then $x = 1$. Thus, we use the inverse transform function to look up the appropriate x for a given u.

For a discrete distribution, given a value for u, pick x_i, such that $F(x_{i-1}) < u \leq F(x_i)$ provides the inverse CDF function. Thus, for any given value of u the generation process amounts to a table lookup of the corresponding value for x_i. This simply involves searching until the condition $F(x_{i-1}) < u \leq F(x_i)$ is true. Since $F(x)$ is an increasing function in x, only the upper limit needs to be checked. The following presents these ideas in the form of an algorithm.

1. u = randU01()
2. i = 1
3. x = x_i
4. WHILE F(x) u
5. i=i+1
6. x = x_i
7. END WHILE
8. RETURN x

In the algorithm, if the test $F(x) \leq u$ is true, the while loop moves to the next interval. If the test failed, $u > F(x_i)$ must be true. The while loop stops and x is the last value checked, which is returned. Thus, only the upper limit in the next interval needs to be tested. Other more complicated and possibly more efficient methods for performing this process are discussed in (Fishman 2006) and (Ripley 1987).

Using the inverse transform method, for discrete random variables, a Bernoulli random variate can be easily generated as shown in the following.

1. u = randu01()
2. IF ($u \leq p$) THEN
3. x=1
4. ELSE
5. x=0
6. END IF
7. RETURN x

To generate *discrete uniform* random variables, the inverse transform method yields the following algorithm:

1. $u = randU01()$
2. $x = a + \lfloor (b - a + 1)u \rfloor$
3. return x

Notice how the discrete uniform distribution inverse transform algorithm is different from the case of the continuous uniform distribution associated with Example A.4. The inverse transform method also works for generating geometric random variables. Unfortunately, the geometric distribution has multiple definitions. Let X be the number of Bernoulli trials needed to get one success. Thus, X has range $1, 2, ...$ and distribution:

$$P\{X = k\} = (1 - p)^{k-1} p$$

We will call this the shifted geometric in this text. The algorithm to generate a shifted geometric random variables is as follows:

1. $u = randU01()$
2. $x = 1 + \left\lfloor \frac{\ln(1-u)}{\ln(1-p)} \right\rfloor$
3. return x

Notice the use of the floor operator $\lfloor \cdot \rfloor$. If the geometric is defined as the number of failures $Y = X - 1$ before the first success, then Y has range $0, 1, 2, ...$ and probability distribution:

$$P\{Y = k\} = (1-p)^k p$$

We will call this distribution the geometric distribution in this text. The algorithm to generate a geometric random variables is as follows:

1. $u = randU01()$
2. $y = \left\lfloor \frac{\ln(1-u)}{\ln(1-p)} \right\rfloor$
3. return y

The Poisson distribution is often used to model the number of occurrences within an interval time, space, etc. For example,

- the number of phone calls to a doctor's office in an hour
- the number of persons arriving to a bank in a day
- the number of cars arriving to an intersection in an hour
- the number of defects that occur within a length of item
- the number of typos in a book
- the number of pot holes in a mile of road

Assume we have an interval of real numbers, and that incidents occur at random throughout the interval. If the interval can be partitioned into sub-intervals of small enough length such that:

- The probability of more than one incident in a sub-intervals is zero
- The probability of one incident in a sub-intervals is the same for all intervals and proportional to the length of the sub-intervals, and
- The number of incidents in each sub-intervals is independent of other sub-intervals

Then, we have a Poisson distribution. Let the probability of an incident falling into a subinterval be p. Let there be n subintervals. An incident either falls in a subinterval or it does not. This can be considered a Bernoulli trial. Suppose there are n subintervals, then the number of incidents that fall in the large interval is a binomial random variable with expectation $n * p$. Let $\lambda = np$ be a constant and keep dividing the main interval into smaller and smaller subintervals such that λ remains constant. To keep λ constant, increase n, and decrease p. What is the chance that x incidents occur in the n subintervals?

$$\binom{n}{x}\left(\frac{\lambda}{n}\right)^{x}\left(1-\frac{\lambda}{n}\right)^{n-x}$$

Take the limit as n goes to infinity

$$\lim_{n\to\infty}\binom{n}{x}\left(\frac{\lambda}{n}\right)^{x}\left(1-\frac{\lambda}{n}\right)^{n-x}$$

and we get the Poisson distribution:

$$P\{X=x\}=\frac{e^{-\lambda}\lambda^{x}}{x!}\quad \lambda>0,\quad x=0,1,...$$

where $E[X]=\lambda$ and $Var[X]=\lambda$. If a Poisson random variable represents the number of incidents in some interval, then the mean of the random variable must equal the expected number of incidents in the same length of interval. In other words, the units must match. When examining the

number of incidents in a unit of time, the Poisson distribution is often written as:

$$P\{X(t) = x\} = \frac{e^{-\lambda t}(\lambda t)^x}{x!}$$

where $X(t)$ is number of events that occur in t time units. This leads to an important relationship with the exponential distribution.

Let $X(t)$ be a Poisson random variable that represents the number of arrivals in t time units with $E[X(t)] = \lambda t$. What is the probability of having no events in the interval from 0 to t?

$$P\{X(t) = 0\} = \frac{e^{-\lambda t}(\lambda t)^0}{0!} = e^{-\lambda t}$$

This is the probability that no one arrives in the interval $(0, t)$. Let T represent the time until an arrival from any starting point in time. What is the probability that $T > t$? That is, what is the probability that the time of the arrival is sometime after t? For T to be bigger than t, we must not have anybody arrive before t. Thus, these two events are the same: $\{T > t\} = \{X(t) = 0\}$. Thus, $P\{T > t\} = P\{X(t) = 0\} = e^{-\lambda t}$. What is the probability that $T \leq t$?

$$P\{T \leq t\} = 1 - P\{T > t\} = 1 - e^{-\lambda t}$$

This is the CDF of T, which is an exponential distribution. Thus, if T is a random variable that represents the time between events and T is exponential with mean $1/\lambda$, then, the number of events in t will be a Poisson process with $E[X(t)] = \lambda t$. Therefore, a method for generating Poisson random variates with mean λ can be derived by counting the number of events that occur before t when the time between events is exponential with mean $1/\lambda$.

Example A.5 (Generate Poisson Random Variates)**.** Let $X(t)$ represent the number of customers that arrive to a bank in an interval of length t, where t is measured in hours. Suppose $X(t)$ has a Poisson distribution with mean rate $\lambda = 4$ per hour. Use the following pseudo-random number (0.971, 0.687, 0.314, 0.752, 0.830) to generate a value of $X(2)$. That is, generate the number of arrivals in 2 hours.

Solution for Example A.5 Because of the relationship between the Poisson distribution and the exponential distribution, the time between events T will have an exponential distribution with mean $0.25 = 1/\lambda$. Thus, we have:

$$T_i = \frac{-1}{\lambda} \ln(1 - u_i) = -0.25 \ln(1 - u_i)$$

$$A_i = \sum_{k=1}^{i} T_k$$

where T_i represents the time between the $i - 1$ and i arrivals and A_i represents the time of the i^{th} arrival. Using the provided u_i, we can compute T_i (via the inverse transform method for the exponential distribution) and A_i until A_i goes over 2 hours.

i	u_i	T_i	A_i
1	0.971	0.881	0.881
2	0.687	0.290	1.171
3	0.314	0.094	1.265
4	0.752	0.349	1.614
5	0.830	0.443	2.057

Since the arrival of the fifth customer occurs after time 2 hours, $X(2) = 4$. That is, there were 4 customers that arrived within the 2 hours. This example is meant to be illustrative of one method for generating Poisson random variates. There are much more efficient methods that have been developed.

The inverse transform technique is general and is used when $F^{-1}(\cdot)$ is closed form and easy to compute. It also has the advantage of using one $U(0, 1)$ for each X generated, which helps when applying certain techniques that are used to improve the estimation process in simulation experiments. Because of this advantage many simulation languages utilize the inverse transform technique even if a closed form solution to $F^{-1}(\cdot)$ does not exist by numerically inverting the function.

A.2.2. Convolution

Many random variables are related to each other through some functional relationship. One of the most common relationships is the convolution relationship. The distribution of the sum of two or more random variables is called the *convolution*. Let $Y_i \sim G(y)$ be independent and identically distributed random variables. Let $X = \sum_{i=1}^{n} Y_i$. Then the distribution of X is said to be the n-fold convolution of Y. Some common random variables that are related through the convolution operation are:

- A binomial random variable is the sum of Bernoulli random variables.
- A negative binomial random variable is the sum of geometric random variables.
- An Erlang random variable is the sum of exponential random variables.
- A Normal random variable is the sum of other normal random variables.

- A chi-squared random variable is the sum of squared normal random variables.

The basic convolution algorithm simply generates $Y_i \sim G(y)$ and then sums the generated random variables. Let's look at a couple of examples. By definition, a negative binomial distribution represents one of the following two random variables:

- The number of failures in sequence of Bernoulli trials before the r$^{\text{th}}$ success, has range $\{0, 1, 2, ...\}$.
- The number of trials in a sequence of Bernoulli trials until the r$^{\text{th}}$ success, it has range $\{r, r+1, r+2, ...\}$

The number of failures before the r$^{\text{th}}$ success, has range $\{0, 1, 2, ...\}$. This is the sum of geometric random variables with range $\{0, 1, 2, ...\}$ with the same success probability.

If $Y \sim NB(r,\ p)$ with range $\{0, 1, 2, \cdots\}$, then

$$Y = \sum_{i=1}^{r} X_i$$

when $X_i \sim Geometric(p)$ with range $\{0, 1, 2, ...\}$, and X_i can be generated via inverse transform with:

$$X_i = \left\lfloor \frac{ln(1-U_i)}{ln(1-p)} \right\rfloor$$

Note that $\lfloor \cdot \rfloor$ is the floor function.

If we have a negative binomial distribution that represents the number of trials until the r$^{\text{th}}$ success, it has range $\{r, r+1, r+2, ...\}$, in this text we call this a shifted negative binomial distribution.

A random variable from a "shifted" negative binomial distribution is the sum of shifted geometric random variables with range $\{1, 2, 3, ...\}$. with same success probability. In this text, we refer to this geometric distribution as the shifted geometric distribution.

If $T \sim NB(r, p)$ with range $\{r, r+1, r+2, ...\}$, then

$$T = \sum_{i=1}^{r} X_i$$

when $X_i \sim Shifted\ Geometric(p)$ with range $\{1, 2, 3, ...\}$, and X_i can be generated via inverse transform with:

$$X_i = 1 + \left\lfloor \frac{ln(1 - U_i)}{ln(1 - p)} \right\rfloor$$

Notice that the relationship between these random variables as follows:

Let Y be the number of failures in a sequence of Bernoulli trials before the r^{th} success.

Let T be the number of trials in a sequence of Bernoulli trials until the r^{th} success,

Then, clearly, $T = Y + r$. Notice that we can generate Y, via convolution, as previously explained and just add r to get T.

$$Y = \sum_{i=1}^{r} X_i$$

Where $X_i \sim Geometric(p)$ with range $\{0, 1, 2, ...\}$, and X_i can be generated via inverse transform with:

$$X_i = \left\lfloor \frac{ln(1 - U_i)}{ln(1 - p)} \right\rfloor \tag{A.2}$$

Example A.6 (Generate Negative Binomial Variates via Convolution)**.** Use the following pseudo-random numbers $u_1 = 0.35$, $u_2 = 0.64$, $u_3 = 0.14$, generate a random variate from Negative Binomial distribution having parameters $r = 3$ and $p = 0.3$. Also, using the same pseudo-random numbers, generate a random variate from a *shifted* Negative Binomial distribution having parameters $r = 3$ and $p = 0.3$.

Solution for Example A.6 To solve this example, we need to apply Equation A.2 to each provided u_i to compute the three values of X_i.

$$X_1 = \left\lfloor \frac{ln(1-0.35)}{ln(1-0.3)} \right\rfloor = 1$$

$$X_2 = \left\lfloor \frac{ln(1-0.64)}{ln(1-0.3)} \right\rfloor = 2$$

$$X_3 = \left\lfloor \frac{ln(1-0.14)}{ln(1-0.3)} \right\rfloor = 0$$

Thus, we have that $Y = X_1 + X_2 + X_3 = 1 + 2 + 0 = 3$. To generate T for the shifted negative binomial, we have that $T = Y + r = Y + 3 = 3 + 3 = 6$. Notice that this all can be easily done within a spreadsheet or within a computer program.

As another example of using convolution consider the requirement to generate random variables from an Erlang distribution. Suppose that $Y_i \sim \text{Exp}(E[Y_i] = 1/\lambda)$. That is, Y is exponentially distributed with rate parameter λ. Now, define X as $X = \sum_{i=1}^{r} Y_i$. One can show that X will have an Erlang distribution with parameters (r, λ), where $E[X] = r/\lambda$ and $Var[X] = r/\lambda^2$. Thus, an Erlang(r, λ) is an r-fold convolution of r exponentially distributed random variables with common mean $1/\lambda$.

Example A.7 (Generate Erlang Random Variates via Convolution)**.** Use the following pseudo-random numbers $u_1 = 0.35$, $u_2 = 0.64$, $u_2 = 0.14$, generate a random variate from an Erlang distribution having parameters $r = 3$ and $\lambda = 0.5$.

Solution for Example A.7 This requires generating 3 exponential distributed random variates each with $\lambda = 0.5$ and adding them up.

$$
\begin{aligned}
y_1 &= \frac{-1}{\lambda} \ln(1 - u_1) = \frac{-1}{0.5} \ln(1 - 0.35) = 0.8616 \\
y_2 &= \frac{-1}{\lambda} \ln(1 - u_2) = \frac{-1}{0.5} \ln(1 - 0.64) = 2.0433 \\
y_3 &= \frac{-1}{\lambda} \ln(1 - u_3) = \frac{-1}{0.5} \ln(1 - 0.14) = 0.3016 \\
x &= y_1 + y_2 + y_3 = 0.8616 + 2.0433 + 0.3016 = 3.2065
\end{aligned}
$$

Because of its simplicity, the convolution method is easy to implement; however, in a number of cases (in particular for a large value of n), there are more efficient algorithms available.

A.2.3. Acceptance/Rejection

In the acceptance-rejection method, the probability density function (PDF) $f(x)$, from which it is desired to obtain a sample is replaced by a proxy PDF, $w(x)$, that can be sampled from more easily. The following illustrates how $w(x)$ is defined such that the *selected* samples from $w(x)$ can be used directly to represent random variates from $f(x)$. The PDF $w(x)$ is based on

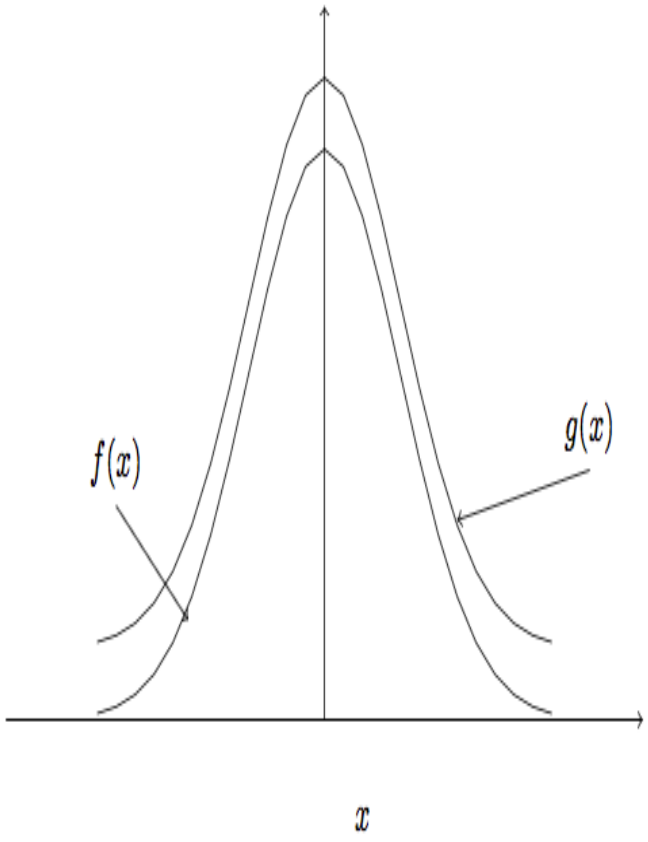


Figure A.4.: Concept of a Majorizing Function

the development of a majorizing function for $f(x)$. A majorizing function, $g(x)$, for $f(x)$, is a function such that $g(x) \geq f(x)$ for $-\infty < x < +\infty$.

Figure A.4 illustrates the concept of a majorizing function for $f(x)$, which simply means a function that is bigger than $f(x)$ everywhere.

In addition, to being a majorizing function for $f(x)$, $g(x)$ must have finite area. In other words,

$$c = \int_{-\infty}^{+\infty} g(x)dx$$

If $w(x)$ is defined as $w(x) = g(x)/c$ then $w(x)$ will be a probability density function. The acceptance-rejection method starts by obtaining a random variate W from $w(x)$. Recall that $w(x)$ should be chosen with the stipulation that it can be easily sampled, e.g. via the inverse transform method. Let $U \sim U(0, 1)$. The steps of the procedure are as provided in the following algorithm. The sampling of U and W continue until $U \times g(W) \leq f(W)$ and W is returned. If $U \times g(W) > f(W)$, then the loop repeats.

1. REPEAT
2. Generate $W \sim w(x)$
3. Generate $U \sim U(0, 1)$
4. UNTIL $(U \times g(W) \leq f(W))$
5. RETURN W

The validity of the procedure is based on deriving the cumulative distribution function of W given that the $W = w$ was accepted, $P\{W \leq x | W = w \text{ is accepted}\}$.

The efficiency of the acceptance-rejection method is enhanced as the probability of rejection is reduced. This probability depends directly on the choice of the majorizing function $g(x)$. The acceptance-rejection method

has a nice intuitive geometric connotation, which is best illustrated with an example.

Example A.8 (Acceptance-Rejection Example)**.** Consider the following PDF over the range $[-1, 1]$. Develop an acceptance/rejection based algorithm for $f(x)$.

$$f(x) = \begin{cases} \frac{3}{4}(1 - x^2) & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

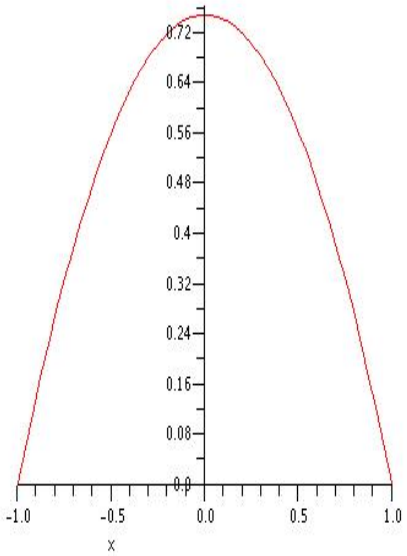


Figure A.5.: Plot of f(x)

Solution for Example A.8 The first step in deriving an acceptance/rejection algorithm for the $f(x)$ in Example A.8 is to select an appropriate majorizing function. A simple method to choose a majorizing function is to set $g(x)$ equal to the maximum value of $f(x)$. As can be seen from the plot of $f(x)$ the maximum value of $3/4$ occurs at x equal to 0. Thus, we can set $g(x) = 3/4$. In order to proceed, we need to construct the PDF associated with $g(x)$. Define $w(x) = g(x)/c$ as the PDF. To determine c, we need to determine the area under the majorizing function:

$$c = \int_{-1}^{1} g(x)dx = \int_{-1}^{1} \frac{3}{4}dx = \frac{3}{2}$$

Thus,

$$w(x) = \begin{cases} \frac{1}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

This implies that $w(x)$ is a uniform distribution over the range from $[-1, 1]$. Based on the discussion of the continuous uniform distribution, a uniform distribution over the range from a to b can be generated with $a + U(b - a)$. Thus, for this case ($a = -1$ and $b = 1$), with $b - a = 1 - -1 = 2$. The acceptance rejection algorithm is as follows:

1. Repeat
1.1 Generate $U_1 \sim U(0, 1)$
1.2 $W = -1 + 2U_1$
1.3 Generate $U_2 \sim U(0, 1)$
1.4 $f = \frac{3}{4}(1 - W^2)$
2. Until $(U_2 \times \frac{3}{4} \leq f)$
3. Return W

Steps 1.1 and 1.2 of generate a random variate, W, from $w(x)$. Note that W is in the range $[-1, 1]$ (the same as the range of X) so that step 1.4 is simply finding the height associated with W in terms of f. Step 1.3 generates U_2. What is the range of $U_2 \times \frac{3}{4}$? The range is $[0, \frac{3}{4}]$. Note that this range corresponds to the range of possible values for f. Thus, in step 2, a point between $[0, \frac{3}{4}]$ is being compared to the candidate point's height $f(W)$ along the vertical axis. If the point is under $f(W)$, the W is accepted; otherwise the W is rejected and another candidate point must be generated. In other words, if a point "under the curve" is generated it will be accepted.

As illustrated in the previous, the probability of acceptance is related to how close $g(x)$ is to $f(x)$. The ratio of the area under $f(x)$ to the area under $g(x)$ is the probability of accepting. Since the area under $f(x)$ is 1, the probability of acceptance, P_a, is:

$$P_a = \frac{1}{\int_{-\infty}^{+\infty} g(x)dx} = \frac{1}{c}$$

where c is the area under the majorizing function. For the example, the probability of acceptance is $P_a = 2/3$. Based on this example, it should be clear that the more $g(x)$ is similar to the PDF $f(x)$ the better (higher) the probability of acceptance. The key to efficiently generating random variates using the acceptance-rejection method is finding a suitable majorizing function over the same range as $f(x)$. In the example, this was easy because $f(x)$ had a finite range. It is more challenging to derive an acceptance rejection algorithm for a probability density function, $f(x)$, if it has an infinite range because it may be more challenging to find a good majorizing function.

A.2.4. Mixture Distributions, Truncated Distributions, and Shifted Random Variables

This section describes three random variate generation methods that build on the previously discussed methods. These methods allow for more flexibility in modeling the underlying randomness. First, let's consider the definition of a mixture distribution and then consider some examples.

Definition A.4 (Mixture Distribution)**.** The distribution of a random variable X is a *mixture distribution* if the CDF of X has the form:

$$F_X(x) = \sum_{i=1}^{k} \omega_i F_{X_i}(x)$$

where $0 < \omega_i < 1$, $\sum_{i=1}^{k} \omega_i = 1$, $k \geq$ and $F_{X_i}(x)$ is the CDF of a continuous or discrete random variable X_i, $i = 1, \ldots, k$.

Notice that the ω_i can be interpreted as a discrete probability distribution as follows. Let I be a random variable with range $I \in \{1, \ldots, k\}$ where $P\{I = i\} = \omega_i$ is the probability that the i^{th} distribution $F_{X_i}(x)$ is selected. Then, the procedure for generating from $F_X(x)$ is to randomly generate I from $g(i) = P\{I = i\} = \omega_i$ and then generate X from $F_{X_I}(x)$. The following algorithm presents this procedure.

1. Generate $I \sim g(i)$
2. Generate $X \sim F_{X_I}(x)$
3. return X

Because mixture distributions combine the characteristics of two or more distributions, they provide for more flexibility in modeling. For example, many of the standard distributions that are presented in introductory probability courses, such as the normal, Weibull, lognormal, etc., have a single mode. Mixture distributions are often utilized for the modeling of data sets that have more than one mode.

As an example of a mixture distribution, we will discuss the hyper-exponential distribution. The hyper-exponential is useful in modeling situations that have a high degree of variability. The coefficient of variation is defined as the ratio of the standard deviation to the expected value for a random variable X. The coefficient of variation is defined as $c_v = \sigma/\mu$, where $\sigma = \sqrt{Var[X]}$ and $\mu = E[X]$. For the hyper-exponential distribution $c_v > 1$. The hyper-exponential distribution is commonly used to model service times that have different (and mutually exclusive) phases. An example of this situation is paying with a credit card or cash at a checkout register. The following example illustrates how to generate from a hyper-exponential distribution.

Example A.9 (Hyper-Exponential Random Variate)**.** Suppose the time that it takes to pay with a credit card, X_1, is exponentially distributed with a mean of 1.5 minutes and the time that it takes to pay with cash, X_2, is exponentially distributed with a mean of 1.1 minutes. In addition, suppose that the chance that a person pays with credit is 70%. Then, the overall distribution representing the payment service time, X, has an hyper-exponential distribution with parameters $\omega_1 = 0.7$, $\omega_2 = 0.3$, $\lambda_1 = 1/1.5$, and $\lambda_2 = 1/1.1$.

$$
\begin{aligned}
F_X(x) &= \omega_1 F_{X_1}(x) + \omega_2 F_{X_2}(x) \\
F_{X_1}(x) &= 1 - \exp(-\lambda_1 x) \\
F_{X_2}(x) &= 1 - \exp(-\lambda_2 x)
\end{aligned}
$$

Derive an algorithm for this distribution. Assume that you have two pseudo-random numbers, $u_1 = 0.54$ and $u_2 = 0.12$, generate a random variate from $F_X(x)$.

Solution for Example A.9 In order to generate a payment service time, X, we can use the mixture distribution algorithm.

1. Generate $u \sim U(0,1)$
2. Generate $v \sim U(0,1)$
3. If $(u \leq 0.7)$
4. $X = F_{X_1}^{-1}(v) = -1.5 \ln(1-v)$
5. else
6. $X = F_{X_2}^{-1}(v) = -1.1 \ln(1-v)$
7. end if
8. return X

Using $u_1 = 0.54$, because $0.54 \leq 0.7$, we have that

$$X = F_{X_1}^{-1}(0.12) = -1.5 \ln(1-0.12) = 0.19175$$

In the previous example, generating X from $F_{X_i}(x)$ utilizes the inverse transform method for generating from the two exponential distribution functions. In general, $F_{X_i}(x)$ for a general mixture distribution might be any distribution. For example, we might have a mixture of a Gamma and a Lognormal distribution. To generate from the individual $F_{X_i}(x)$ one would use the most appropriate generation technique for that distribution. For example, $F_{X_1}(x)$ might use inverse transform, $F_{X_2}(x)$ might use acceptance/rejection, $F_{X_3}(x)$ might use convolution, etc. This provides great flexibility in modeling and in generation.

In general, we may have situations where we need to control the domain over which the random variates are generated. For example, when we are modeling situations that involve time (as is often the case within simulation), we need to ensure that we do not generate negative values. Or, for example, it may be physically impossible to perform a task in a time that is shorter than a particular value. The next two generation techniques assist with modeling these situations.

A *truncated distribution* is a distribution derived from another distribution for which the range of the random variable is restricted. Truncated distributions can be either discrete or continuous. The presentation here illustrates the continuous case. Suppose we have a random variable, X with PDF, $f(x)$ and CDF $F(x)$. Suppose that we want to constrain $f(x)$ over interval $[a, b]$, where $a < b$ and the interval $[a, b]$ is a subset of the original support of $f(x)$. Note that it is possible that $a = -\infty$ or $b = +\infty$. Constraining $f(x)$ in this manner will result in a new random variable, $X|a \leq X \leq b$. That is, the random variable X given that X is contained in $[a, b]$. Random variables have a probability distribution. The question is what is the probability distribution of this new random variable and how can we generate from it.

This new random variable is governed by the conditional distribution of X given that $a \leq X \leq b$ and has the following form:

$$f(x|a \leq X \leq b) = f^*(x) = \begin{cases} \frac{g(x)}{F(b)-F(a)} & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

where

$$g(x) = \begin{cases} f(x) & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

Note that $g(x)$ is not a probability density. To convert it to a density, we need to find its area and divide by its area. The area of $g(x)$ is:

$$\begin{aligned}\int_{-\infty}^{+\infty} g(x)dx &= \int_{-\infty}^{a} g(x)dx + \int_{a}^{b} g(x)dx + \int_{b}^{+\infty} g(x)dx \\ &= \int_{-\infty}^{a} 0dx + \int_{a}^{b} f(x)dx + \int_{b}^{+\infty} 0)dx \\ &= \int_{a}^{b} f(x)dx = F(b) - F(a)\end{aligned}$$

Thus, $f^*(x)$ is simply a "re-weighting" of $f(x)$. The CDF of $f^*(x)$ is:

$$F^*(x) = \begin{cases} 0 & \text{if } x < a \\ \frac{F(x)-F(a)}{F(b)-F(a)} & a \leq x \leq b \\ 0 & \text{if } b < x \end{cases}$$

This leads to a straight forward algorithm for generating from $f^*(x)$ as follows:

1. Generate $u \sim U(0, 1)$
2. $W = F(a) + (F(b) - F(a))u$
3. $X = F^{-1}(W)$
4. return X

Lines 1 and 2 of the algorithm generate a random variable W that is uniformly distributed on $(F(a), F(b))$. Then, that value is used within the original distribution's inverse CDF function, to generate a X given that $a \leq X \leq b$. Let's look at an example.

Example A.10 (Generating a Truncated Random Variate)**.** Suppose X represents the distance between two cracks in highway. Suppose that X has an exponential distribution with a mean of 10 meters. Generate a distance restricted between 3 and 6 meters using the pseudo-random number 0.23.

Solution for Example A.10 The CDF of the exponential distribution with mean 10 is:

$$F(x) = 1 - e^{-x/10}$$

Therefore $F(3) = 1 - \exp(-3/10) = 0.259$ and $F(6) = 0.451$. The exponential distribution has inverse cumulative distribution function:

$$F^{-1}(u) = \frac{-1}{\lambda} \ln(1-u)$$

First, we generate a random number uniformly distributed between $F(3)$ and $F(6)$ using $u = 0.23$:

$$W = 0.259 + (0.451 - 0.259) \times 0.23 = 0.3032$$

Therefore, in order to generate the distance we have:

$$X = -10 \times \ln(1 - 0.3032) = 3.612$$

Lastly, we discuss shifted distributions. Suppose X has a given distribution $f(x)$, then the distribution of $X + \delta$ is termed the shifted distribution and is specified by $g(x) = f(x - \delta)$. It is easy to generate from a shifted distribution, simply generate X according to $F(x)$ and then add δ.

Example A.11 (Generating a Shifted Weibull Random Variate Example)**.** Suppose X represents the time to setup a machine for production. From past time studies, we know that it cannot take any less than 5.5 minutes to prepare for the setup and that the time after the 5.5 minutes is random with a Weibull distribution with shape parameter $\alpha = 3$ and scale parameter $\beta = 5$. Using a pseudo-random number of $u = 0.73$ generate a value for the time to perform the setup.

Solution for Example A.11 The Weibull distribution has a closed form cumulative distribution function:

$$F(x) = 1 - e^{-(x/\beta)^{\alpha}}$$

Thus, the inverse CDF function is:

$$F^{-1}(u) = \beta \left[-\ln(1-u)\right]^{1/\alpha}$$

Therefore to generate the setup time we have:

$$5.5 + 5\left[-\ln(1-0.73)\right]^{1/3} = 5.5 + 5.47 = 10.97$$

Within this section, we illustrated the four primary methods for generating random variates 1) inverse transform, 2) convolution, 3) acceptance/rejection, and 4) mixture and truncated distributions. These are only a starting point for the study of random variate generation methods.

A.3. Summary

This section covered a number of important concepts used within simulation including:

- Generating pseudo-random numbers
- Generating random variates and processes

Section D.1 and Section D.2 summarize many of the properties of common discrete and continuous distributions. These topics provide a solid foundation for modeling random components within simulation models. Not only should you now understand how random numbers are generated you also know how to transform those numbers to allow the generation from a wide variety of probability distributions. To further your study of random variate generation, you should study the generation of multi-variate distributions.

A.4. Exercises

Exercise A.1. The sequence of random numbers generated from a given seed is called a random number (a)_____.

Exercise A.2. State four major methods of generating random variables from any distribution. (a)_____. (b)_____. (c)_____. (d)_____.

Exercise A.3. Consider the multiplicative congruential generator with ($a = 13$, $m = 64$, $c = 0$, and seeds $X_0 = 1,2,3,4$). a) Using Theorem A.1, does this generator achieve its maximum period for these parameters? b) Generate one period's worth of uniform random variables from each of the supplied seeds.

Exercise A.4. Consider the multiplicative congruential generator with ($a = 11$, $m = 64$, $c = 0$, and seeds $X_0 = 1,2,3,4$). a) Using Theorem A.1, does this generator achieve its maximum period for these parameters? b) Generate one period's worth of uniform random variables from each of the supplied seeds.

Exercise A.5. Consider the linear congruential generator with ($a = 11$, $m = 16$, $c = 5$, and seed $X_0 = 1$). a) Using Theorem A.1, does this generator achieve its maximum period for these parameters? b) Generate 2 pseudo-random uniform numbers for this generator.

Exercise A.6. Consider the linear congruential generator with ($a = 13$, $m = 16$, $c = 13$, and seed $X_0 = 37$). a) Using Theorem A.1, does this generator achieve its maximum period for these parameters? b) Generate 2 pseudo-random uniform numbers for this generator.

Exercise A.7. Consider the linear congruential generator with ($a = 8$, $m = 10$, $c = 1$, and seed $X_0 = 11$). a) Using Theorem A.1, does this generator achieve its maximum period for these parameters? b) Generate 2 pseudo-random uniform numbers for this generator.

Exercise A.8. Consider the following discrete distribution of the random variable X whose probability mass function is $p(x)$.

x	0	1	2	3	4
$p(x)$	0.3	0.2	0.2	0.1	0.2

a. Determine the CDF $F(x)$ for the random variable, X.
b. Create a graphical summary of the CDF. See Example A.4.
c. Create a look-up table that can be used to determine a sample from the discrete distribution, $p(x)$. See Example A.4.
d. Generate 3 values of X using the following pseudo-random numbers $u_1 = 0.943, u_2 = 0.398, u_3 = 0.372$

Exercise A.9. Consider the following uniformly distributed random numbers:

U_1	U_2	U_3	U_4	U_5	U_6	U_7	U_8
0.9396	0.1694	0.7487	0.3830	0.5137	0.0083	0.6028	0.8727

a. Generate an exponentially distributed random number with a mean of 10 using the 1st random number.

b. Generate a random variate from a (12, 22) discrete uniform distribution using the 2nd random number.

Exercise A.10. Consider the following uniformly distributed random numbers:

U_1	U_2	U_3	U_4	U_5	U_6	U_7	U_8
0.9559	0.5814	0.6534	0.5548	0.5330	0.5219	0.2839	0.3734

a. Generate a uniformly distributed random number with a minimum of 12 and a maximum of 22 using U_8.
b. Generate 1 random variate from an Erlang($r = 2$, $\beta = 3$) distribution using U_1 and U_2
c. The demand for magazines on a given day follows the following probability mass function:

x	40	50	60	70	80
$P(X = x)$	0.44	0.22	0.16	0.12	0.06

Using the supplied random numbers for this problem starting at U_1, generate 4 random variates from the probability mass function.

Exercise A.11. Suppose that customers arrive at an ATM via a Poisson process with mean 7 per hour. Determine the arrival time of the first 6 customers using the following pseudo-random numbers via the inverse transformation method. Start with the first row and read across the table.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.12. The demand, D, for parts at a repair bench per day can be described by the following discrete probability mass function:

D	0	1	2
$p(D)$	0.3	0.2	0.5

Generate the demand for the first 4 days using the following sequence of U(0,1) random numbers: 0.943, 0.398, 0.372, 0.943.

Exercise A.13. The service time for an automated storage and retrieval system has a shifted exponential distribution. It is known that it takes a minimum of 15 seconds for any retrieval. The time beyond the minimum is exponentially distributed with a mean of 45 seconds. Generate two service times for this distribution using the following sequence of U(0,1) random numbers: 0.943, 0.398, 0.372, 0.943.

Exercise A.14. The time to failure for a computer printer fan has a Weibull distribution with shape parameter $\alpha = 2$ and scale parameter $\beta = 3$. Testing has indicated that the distribution is limited to the range

from 1.5 to 4.5. Generate two random variates from this distribution using the following sequence of U(0,1) random numbers: 0.943, 0.398, 0.372, 0.943.

Exercise A.15. The interest rate for a capital project is unknown. An accountant has estimated that the interest rate will be between 2% and 5% within the next year. The accountant believes that any interest rate in this range is equally likely. You are tasked with generating interest rates for a cash flow analysis of the project. Generate two random variates from this distribution using the following sequence of U(0,1) random numbers: 0.943, 0.398, 0.372, 0.943.

Exercise A.16. Customers arrive at a service location according to a Poisson distribution with mean 10 per hour. The installation has two servers. Experience shows that 60% of the arriving customers prefer the first server. Start with the first row and read across the table determine the arrival times of the first three customers at each server.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.17. Consider the triangular distribution:

$$F(x) = \begin{cases} 0 & x < a \\ \dfrac{(x-a)^2}{(b-a)(c-a)} & a \leq x \leq c \\ 1 - \dfrac{(b-x)^2}{(b-a)(b-c)} & c < x \leq b \\ 1 & b < x \end{cases}$$

a. Derive an inverse transform algorithm for this distribution. b. Using 0.943, 0.398, 0.372, 0.943, 0.204 generate 5 random variates from the triangular distribution with $a = 2$, $c = 5$, $b = 10$.

Exercise A.18. Consider the following probability density function:

$$f(x) = \begin{cases} \dfrac{3x^2}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

a. Derive an inverse transform algorithm for this distribution. b. Using 0.943, 0.398 generate two random variates from this distribution.

Exercise A.19. Consider the following probability density function:

$$f(x) = \begin{cases} 0.5x - 1 & 2 \leq x \leq 4 \\ 0 & \text{otherwise} \end{cases}$$

a. Derive an inverse transform algorithm for this distribution. b. Using 0.943, 0.398 generate two random variates from this distribution.

Exercise A.20. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{2x}{25} & 0 \leq x \leq 5 \\ 0 & \text{otherwise} \end{cases}$$

a. Derive an inverse transform algorithm for this distribution. b. Using 0.943, 0.398 generate two random variates from this distribution.

Exercise A.21. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{2}{x^3} & x > 1 \\ 0 & x \leq 1 \end{cases}$$

a. Derive an inverse transform algorithm for this distribution. b. Using 0.943, 0.398 generate two random variates from this distribution.

Exercise A.22. The times to failure for an automated production process have been found to be randomly distributed according to a Rayleigh distribution:

$$f(x) = \begin{cases} 2\beta^{-2} x e^{(-(x/\beta)^2)} & x > 0 \\ 0 & \text{otherwise} \end{cases}$$

a. Derive an inverse transform algorithm for this distribution. b. Using 0.943, 0.398 generate two random variates from this distribution with $\beta = 2.0$.

Exercise A.23. Starting with the first row, first column and reading by rows use the random numbers from the following table to generate 2 random variates from the negative binomial distribution with parameters $(r = 4, p = 0.4)$ using the convolution method.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.24. Starting with the first row, first column and reading by rows use the random numbers from the following table to generate 2 random variates from the negative binomial distribution with parameters $(r = 4, p = 0.4)$ using a sequence of Bernoulli trials to get 4 successes.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.25. Suppose that the processing time for a job consists of two distributions. There is a 30% chance that the processing time is lognormally distributed with a mean of 20 minutes and a standard deviation of 2 minutes, and a 70% chance that the time is uniformly distributed between 10 and 20 minutes. Using the first row of random numbers from the following table generate two job processing times. Hint: $X \sim LN(\mu, \sigma^2)$ if and only if $\ln(X) \sim N(\mu, \sigma^2)$. Also, note that:

$$E[X] = e^{\mu+\sigma^2/2}$$
$$Var[X] = e^{2\mu+\sigma^2}\left(e^{\sigma^2}-1\right)$$

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.26. Suppose that the service time for a patient consists of two distributions. There is a 25% chance that the service time is uniformly distributed with minimum of 20 minutes and a maximum of 25 minutes, and a 75% chance that the time is distributed according to a Weibull distribution with shape of 2 and a scale of 4.5. Using the first row of random numbers from the following table generate the service time for two patients.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.27. If $Z \sim N(0,1)$, and $Y = \sum_{i=1}^{k} Z_i^2$ then $Y \sim \chi_k^2$, where χ_k^2 is a chi-squared random variable with k degrees of freedom. Using the first two rows of random numbers from the following table generate two χ_5^2 random variates.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.28. In the (a)_____ technique for generating random variates, you want the (b)_____ function to be as close as possible to the distribution function that you want to generate from in order to ensure that the (c)_____ is as high as possible, thereby improving the efficiency of the algorithm.

Exercise A.29. Prove that the acceptance-rejection method for continuous random variables is valid by showing that for any x,

$$P\{X \leq x\} = \int_{-\infty}^{x} f(y)dy$$

Hint: Let E be the event that the acceptance occurs and use conditional probability.

Exercise A.30. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{3x^2}{2} & -1 \leq x \leq 1 \\ 0 & \text{otherwise} \end{cases}$$

a. Derive an acceptance-rejection algorithm for this distribution. b. Using the first two rows of random numbers from the following table generate 2 random variates using your algorithm.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.31. This problem is based on (Cheng 1977), see also (Ahrens and Dieter 1972). Consider the gamma distribution:

$$f(x) = \beta^{-\alpha} x^{\alpha-1} \frac{e^{-x/\beta}}{\Gamma(\alpha)}$$

where x > 0 and $\alpha > 0$ is the shape parameter and $\beta > 0$ is the scale parameter. In the case where α is a positive integer, the distribution reduces to the Erlang distribution and $\alpha = 1$ produces the negative exponential distribution.

Acceptance-rejection techniques can be applied to the cases of $0 < \alpha < 1$ and $\alpha > 1$. For the case of $0 < \alpha < 1$ see Ahrens and Dieter (1972). For the case of $\alpha > 1$, Cheng (1977) proposed the following majorizing function:

$$g(x) = \left[\frac{4\alpha^{\alpha} e^{-\alpha}}{a\Gamma(\alpha)}\right] h(x)$$

where $a = \sqrt{(2\alpha - 1)}$, $b = \alpha^a$, and $h(x)$ is the resulting probability density function when converting $g(x)$ to a density function:

$$h(x) = ab\frac{x^{a-1}}{(b + x^a)^2} \quad \text{for } x > 0$$

a. Develop an inverse transform algorithm for generating from $h(x)$

b. Using the first two rows of random numbers from the following table, generate two random variates from a gamma distribution with parameters $\alpha = 2$ and $\beta = 10$ via the acceptance/rejection method.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.32. Parts arrive to a machine center with three drill presses according to a Poisson distribution with mean λ. The arriving customers are assigned to one of the three drill presses randomly according to the respective probabilities p_1, p_2, and p_3 where $p_1 + p_2 + p_3 = 1$ and $p_i > 0$ for $i = 1, 2, 3$. What is the distribution of the inter-arrival times to each drill press? Specify the parameters of the distribution. Suppose that p_1, p_2, and p_3 equal to 0.25, 0.45, and 0.3 respectively and that λ is equal to 12 per minute.

Using the first row of random numbers from the following table generate the time of the first arrival at each drill press.

0.943	0.398	0.372	0.943	0.204	0.794
0.498	0.528	0.272	0.899	0.294	0.156
0.102	0.057	0.409	0.398	0.400	0.997

Exercise A.33. Consider the following function:

$$f(x) = \begin{cases} cx^2 & a \leq x \leq b \\ 0 & \text{otherwise} \end{cases}$$

a. Determine the value of c that will turn $f(x)$ into a probability density function. The resulting probability density function is called a parabolic distribution.
b. Denote the probability density function found in part (a), $f(x)$. Let X be a random variable from $f(x)$. Derive the inverse cumulative distribution function for $f(x)$.

Exercise A.34. Consider the following probability density function:

$$f(x) = \begin{cases} \frac{3(c-x)^2}{c^3} & 0 \leq x \leq c \\ 0 & \text{otherwise} \end{cases}$$

Derive an inverse cumulative distribution algorithm for generating from $f(x)$.

B. Probability Distribution Modeling

Learning Objectives

- To be able to model discrete distributions based on data
- To be able to model continuous distributions based on data
- To be able to perform basic statistical tests on uniform pseudo-random numbers

When performing a simulation study, there is no substitution for actually observing the system and collecting the data required for the modeling effort. As outlined in Section 1.7, a good simulation methodology recognizes that modeling and data collection often occurs in parallel. That is, observing the system allows conceptual modeling which allows for an understanding of the input models that are needed for the simulation. The collection of the data for the input models allow further observation of the system and further refinement of the conceptual model, including the identification of additional input models. Eventually, this cycle converges to the point where the modeler has a well defined understanding of the input data requirements. The data for the input model must be collected and modeled.

Input modeling begins with data collection, probability, statistics, and analysis. There are many methods available for collecting data, including time study analysis, work sampling, historical records, and automatically collected data. Time study and work sampling methods are covered in a standard industrial engineering curriculum. Observing the time an operator takes to perform a task via a time study results in a set of observations of

the task times. Hopefully, there will be sufficient observations for applying the techniques discussed in this section.

Work sampling is useful for developing the percentage of time associated with various activities. This sort of study can be useful in identifying probabilities associated with performing tasks and for validating the output from the simulation models. Historical records and automatically collected data hold promise for allowing more data to be collected, but also pose difficulties related to the quality of the data collected. In any of the above mentioned methods, the input models will only be as good as the data and processes used to collect the data.

One especially important caveat for new simulation practitioners: do not rely on the people in the system you are modeling to correctly collect the data for you. If you do rely on them to collect the data, you must develop documents that clearly define what data is needed and how to collect the data. In addition, you should train them to collect the data using the methods that you have documented. Only through careful instruction and control of the data collection processes will you have confidence in your input modeling.

A typical input modeling process includes the following procedures:

1. Documenting the process being modeled: Describe the process being modeled and define the random variable to be collected. When collecting task times, you should pay careful attention to clearly defining when the task starts and when the task ends. You should also document what triggers the task.
2. Developing a plan for collecting the data and then collect the data: Develop a sampling plan, describe how to collect the data, perform a pilot run of your plan, and then collect the data.
3. Graphical and statistical analysis of the data: Using standard statistical analysis tools you should visually examine your data. This should

include such plots as a histogram, a time series plot, and an autocorrelation plot. Again, using statistical analysis tools you should summarize the basic statistical properties of the data, e.g. sample average, sample variance, minimum, maximum, quartiles, etc.

4. Hypothesizing distributions: Using what you have learned from steps 1 - 3, you should hypothesize possible distributions for the data.

5. Estimating parameters: Once you have possible distributions in mind you need to estimate the parameters of those distributions so that you can analyze whether the distribution provides a good model for the data. With current software this step, as well as steps 3, 4, and 6, have been largely automated.

6. Checking goodness of fit for hypothesized distributions: In this step, you should assess whether or not the hypothesized probability distributions provide a good fit for the data. This should be done both graphically (e.g. histograms, P-P plots and Q-Q plots) and via statistical tests (e.g. Chi-Squared test, Kolmogorov-Smirnov Test). As part of this step you should perform some sensitivity analysis on your fitted model.

During the input modeling process and after it is completed, you should document your process. This is important for two reasons. First, much can be learned about a system simply by collecting and analyzing data. Second, in order to have your simulation model accepted as useful by decision makers, they must *believe* in the input models. Even non-simulation savvy decision makers understand the old adage "Garbage In = Garbage Out". The documentation helps build credibility and allows you to illustrate how the data was collected.

The following section provides a review of probability and statistical concepts that are useful in distribution modeling.

B.1. Random Variables and Probability Distributions

This section discusses some concepts in probability and statistics that are especially relevant to simulation. These will serve you well as you model randomness in the inputs of your simulation models. When an input process for a simulation is stochastic, you must develop a probabilistic model to characterize the process's behavior over time. Suppose that you are modeling the service times in the pharmacy example. Let X_i be a random variable that represents the service time of the i^{th} customer. As shown in Figure B.1, a random variable is a function that assigns a real number to each outcome, s, in a random process that has a set of possible outcomes, S.

In this case, the process is the service times of the customers and the outcomes are the possible values that the service times can take on, i.e. the range of possible values for the service times.

The determination of the range of the random variable is part of the modeling process. For example, if the range of the service time random variable is the set of all possible positive real numbers, i.e. $X_i \in \Re^+$ or in other words, $X_i \geq 0$, then the service time should be modeled as a *continuous* random variable.

Suppose instead that the service time can only take on one of five discrete values 2, 5, 7, 8, 10, then the service time random variable should be modeled as a *discrete* random variable. Thus, the first decision in modeling a stochastic input process is to appropriately define a random variable and its possible range of values.

The next decision is to characterize the probability distribution for the random variable. As indicated in Figure B.2, a probability distribution for a random variable is a function that maps from the range of the random variable to a real number, $p \in [0, 1]$. The value of p should be interpreted as the probability associated with the event represented by the random variable.

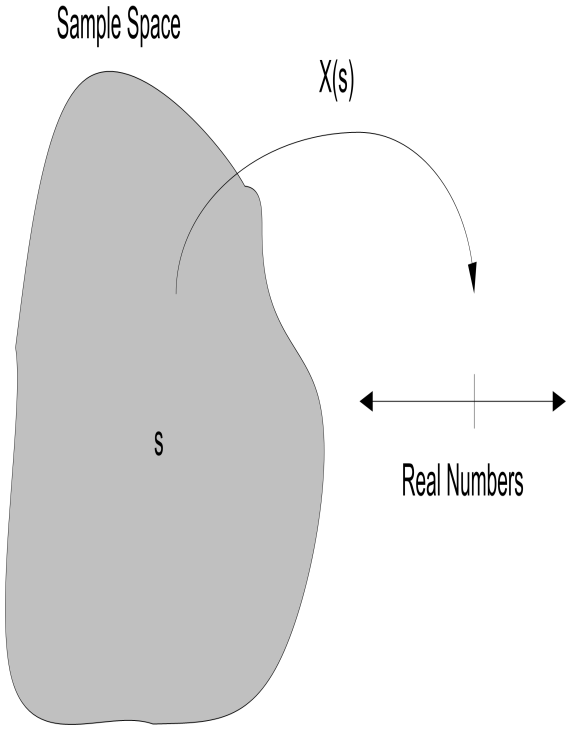


Figure B.1.: Random Variables Map Outcomes to Real Numbers

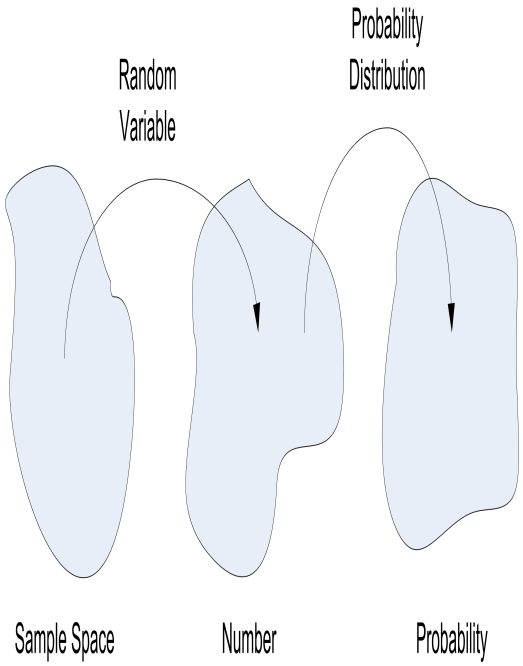


Figure B.2.: Probability Distributions Map Random Variables to Probabilities

For a discrete random variable, X, with possible values $x_1, x_2, ... x_n$ (n may be infinite), the function, $f(x)$ that assigned probabilities to each possible value of the random variable is called the probability mass function (PMF) and is denoted:

$$f(x_i) = P(X = x_i)$$

where $f(x_i) \geq 0$ for all x_i and $\sum_{i=1}^{n} f(x_i) = 1$. The probability mass function describes the probability value associated with each discrete value of the random variable.

For a continuous random variable, X, the mapping from real numbers to probability values is governed by a probability density function, $f(x)$ and has the properties:

1. $f(x) \geq 0$
2. $\int_{-\infty}^{\infty} f(x)dx = 1$ (The area must sum to 1.)
3. $P(a \leq x \leq b) = \int_{a}^{b} f(x)dx$ (The area under f(x) between a and b.)

The probability density function (PDF) describes the probability associated with a range of possible values for a continuous random variable.

A cumulative distribution function (CDF) for a discrete or continuous random variable can also be defined. For a discrete random variable, the cumulative distribution function is defined as

$$F(x) = P(X \leq x) = \sum_{x_i \leq x} f(x_i)$$

and satisfies, $0 \leq F(x) \leq 1$, and for $x \leq y$ then $F(x) \leq F(y)$.

The cumulative distribution function of a continuous random variable is

$$F(x) = P(X \leq x) = \int_{-\infty}^{x} f(u)du \text{ for } -\infty < x < \infty$$

Thus, when modeling the elements of a simulation model that have randomness, one must determine:

- Whether or not the randomness is discrete or continuous
- The form of the distribution function (i.e. the PMF or PDF)

To develop an understanding of the probability distribution for the random variable, it is useful to characterize various properties of the distribution such as the expected value and variance of the random variable.

The *expected value* of a discrete random variable X, is denoted by $E[X]$ and is defined as:

$$E[X] = \sum_{x} xf(x)$$

where the sum is defined through all possible values of x. The *variance* of X is denoted by $Var[X]$ and is defined as:

$$\begin{aligned} Var[X] &= E[(X - E[X])^2] \\ &= \sum_{x} (x - E[X])^2 f(x) \\ &= \sum_{x} x^2 f(x) - (E[X])^2 \\ &= E[X^2] - (E[X])^2 \end{aligned}$$

Suppose X is a continuous random variable with PDF, $f(x)$, then the expected value of X is

$$E[X] = \int_{-\infty}^{\infty} x f(x) dx$$

and the variance of X is

$$Var[X] = \int_{-\infty}^{\infty} (x - E[X])^2 f(x) dx = \int_{-\infty}^{\infty} x^2 f(x) dx - (E[X])^2$$

which is equivalent to $Var[X] = E[X^2] - (E[X])^2$ where

$$E[X^2] = \int_{-\infty}^{\infty} x^2 f(x)$$

Another parameter that is often useful is the coefficient of variation. The coefficient of variation is defined as:

$$c_v = \frac{\sqrt{Var[X]}}{E[X]}$$

The coefficient of variation measures the amount of variation relative to the mean value (provided that $E\{X\} \neq 0$).

To estimate $E\{X\}$, the sample average, $\bar{X}(n)$,

$$\bar{X}(n) = \frac{1}{n} \sum_{i=1}^{n} X_i$$

is often used. To estimate $Var[X]$, assuming that the data is independent, the sample variance, S^2,

$$S^2(n) = \frac{1}{n-1} \sum_{i=1}^{n} (X_i - \bar{X})^2$$

can be used. Thus, an estimator for the coefficient of variation is:

$$\hat{c}_v = \frac{s}{\bar{x}}$$

A number of other statistical quantities are also useful when trying to characterize the properties of a distribution:

- skewness - Measures the asymmetry of the distribution about its mean.
- kurtosis - Measures the degree of peakedness of the distribution
- order statistics - Used when comparing the sample data to the theoretical distribution via P-P plots or Q-Q plots.
- quantiles (1st quartile, median, 3rd quartile) - Summarizes the distribution of the data.
- minimum, maximum, and range - Indicates the range of possible values

Skewness can be estimated by:

$$\hat{\gamma}_1 = \frac{\frac{1}{n}\sum_{i=1}^{n}\left(X_i - \bar{X}\right)^3}{\left[S^2\right]^{3/2}}$$

For a unimodal distribution, negative skew indicates that the tail on the left side of the PDF is longer or fatter than the right side. Positive skew indicates that the tail on the right side is longer or fatter than the left side. A value of skewness near zero indicates symmetry.

Kurtosis can be estimated by:

$$\hat{\gamma}_2 = \frac{n-1}{(n-2)(n-3)}\left((n+1)g_2 + 6\right)$$

where, g_2 is:

$$g_2 = \frac{\frac{1}{n}\sum_{i=1}^{n}\left(X_i - \bar{X}\right)^4}{\left(\frac{1}{n}\sum_{i=1}^{n}\left(X_i - \bar{X}\right)^2\right)^2} - 3$$

Order statistics are just a fancy name for the sorted data. Let $(x_1, x_2, \ldots x_n)$ represent a sample of data. If the data is sorted from smallest to largest, then the i^{th} ordered element can be denoted as,$x_{(i)}$. For example, $x_{(1)}$ is the smallest element, and $x_{(n)}$ is the largest, so that $(x_{(1)}, x_{(2)}, \ldots x_{(n)})$ represents the ordered data and these values are called the order statistics. From the order statistics, a variety of other statistics can be computed:

1. minimum = $x_{(1)}$
2. maximum = $x_{(n)}$
3. range = $x_{(n)} - x_{(1)}$

The median, $\tilde{x}$, is a measure of central tendency such that one-half of the data is above it and one-half of the data is below it. The median can be estimated as follows:

$$\tilde{x} = \begin{cases} x_{((n+1)/2)} & n \text{ is odd} \\ \dfrac{x_{(n/2)} + x_{((n/2)+1)}}{2} & n \text{ is even} \end{cases}$$

For example, consider the following data:

$$x_{(1)} = 3, x_{(2)} = 5, x_{(3)} = 7, x_{(4)} = 7, x_{(5)} = 38$$

Because $n = 5$, we have:

$$\frac{n+1}{2} = \frac{5+1}{2} = 3$$

$$\tilde{x} = x_{(3)} = 7$$

Suppose we have the following data:

$$x_{(1)} = 3, x_{(2)} = 5, x_{(3)} = 7, x_{(4)} = 7$$

Because $n = 4$, we have:

$$\begin{aligned} x_{(n/2)} &= x_{(2)} \\ x_{((n/2)+1)} &= x_{(3)} \\ \tilde{x} &= \frac{x_{(2)} + x_{(3)}}{2} = \frac{5+7}{2} = 6 \end{aligned}$$

The first quartile is the first 25% of the data and can be thought of as the 'median' of the first half of the data. Similarly, the third quartile is the first 75% of the data or the 'median' of the second half of the data. Different methods are used to estimate these quantities in various software packages; however, their interpretation is the same, summarizing the distribution of the data.

As noted in this section, a key decision in distribution modeling is whether the underlying random variable is discrete or continuous. The next section discusses how to model discrete distributions.

B.2. Modeling with Discrete Distributions

There are a wide variety of discrete random variables that often occur in simulation modeling. Section D.1 summarizes the functions and characteristics some common discrete distributions. Table B.1 provides an overview of some modeling situations for common discrete distributions.

Table B.1.: Common Modeling Situations for Discrete Distributions

Distribution	Modeling Situations
Bernoulli(p)	independent trials with success probability p
Binomial(n,p)	sum of n Bernoulli trials with success probability p
Geometric(p)	number of Bernoulli trials until the first success
Negative Binomial(r,p)	number of Bernoulli trials until the r^{th} success
Discrete Uniform(a,b)	equally likely outcomes over range (a, b)
Discrete Uniform $v_1, \cdots, v_n$	equally likely over values v_i
Poisson(λ)	counts of occurrences in an interval, area, or volume

By understanding the modeling situations that produce data, you can hypothesize the appropriate distribution for the distribution fitting process.

B.3. Fitting Discrete Distributions

This section illustrates how to model and fit a discrete distribution to data. Although the steps in modeling discrete and continuous distributions are very similar, the processes and tools utilized vary somewhat. The first thing to truly understand is the difference between discrete and continuous random variables. Discrete distributions are used to model discrete random variables. Continuous distributions are used to model continuous random variables. This may seem obvious but it is a key source of confusion for novice modelers.

Discrete random variables take on any of a specified *countable* list of values. Continuous random variables take on any numerical value in an interval or collection of intervals. The source of confusion is when looking at a file of the data, you might not be able to tell the difference. The discrete values may have decimal places and then the modeler thinks that the data is continuous. The modeling starts with what is being collected and how it is being collected (not with looking at a file!).

B.3.1. Fitting a Poisson Distribution

Since the Poisson distribution is very important in simulation modeling, the discrete input modeling process will be illustrated by fitting a Poisson distribution. Example B.1 presents data collected from an arrival process. As noted in Table B.1, the Poisson distribution is a prime candidate for modeling this type of data.

Example B.1 (Fitting a Poisson Distribution)**.** Suppose that we are interested in modeling the demand for a computer laboratory during the morning hours between 9 am to 11 am on normal weekdays. During this time a team of undergraduate students has collected the number of students arriving to the computer lab during 15 minute intervals over a period of 4 weeks.

Since there are four 15 minute intervals in each hour for each two hour time period, there are 8 observations per day. Since there are 5 days per week, we have 40 observations per week for a total of $40 \times 4 = 160$ observations. A sample of observations per 15 minute interval are presented in Table B.2. The full data set is available with the chapter files. Check whether a Poisson distribution is an appropriate model for this data.

Table B.2.: Computer Laboratory Arrival Counts by Week, Period, and Day

Week	Period	M	T	W	TH	F
1	9:00-9:15 am	8	5	16	7	7
1	9:15-9:30 am	8	4	9	8	6
1	9:30-9:45 am	9	5	6	6	5
1	9:45-10:00 am	10	11	12	10	12

Week	Period	M	T	W	TH	F
1	10:00-10:15 am	6	7	14	9	3
1	10:15-10:30 am	11	8	7	7	11
1	10:30-10:45 am	12	7	8	3	6
1	10:45-11:00 am	8	9	9	8	6
2	9:00-9:15 am	10	13	7	7	7
3	9:00-9:15 am	5	7	14	8	8
4	9:00-9:15 am	7	11	8	5	4
4	10:45-11:00 am	8	9	7	9	6

The solution to Example B.1 involves the following steps:

1. Visualize the data.
2. Check if the week, period, or day of week influence the statistical properties of the count data.
3. Tabulate the frequency of the count data.
4. Estimate the mean rate parameter of the hypothesized Poisson distribution.
5. Perform goodness of fit tests to test the hypothesis that the number of arrivals per 15 minute interval has a Poisson distribution versus the alternative that it does not have a Poisson distribution.

B.3.2. Visualizing the Data

When analyzing a data set it is best to begin with visualizing the data. We will analyze this data utilizing the R statistical software package. Assuming that the data is in a comma separate value (csv) file called

PoissonCountData.csv in the R working directory, the following commands will read the data into the R environment, plot a histogram, plot a time series plot, and make an autocorrelation plot of the data.

```
p2 = read.csv("PoissonCountData.csv")
hist(p2$N, main="Computer Lab Arrivals", xlab = "Counts")
plot(p2$N,type="b",main="Computer Lab Arrivals", ylab = "Count", xlab = "Observation
acf(p2$N, main = "ACF Plot for Computer Lab Arrivals")
```

Table B.3 illustrates the layout of the data frame in R. The first column indicates the week, the 2nd column represents the period of the day, the 3rd column indicates the day of the week, and the last column indicated with the variable, N, represents the count for the week, period, day combination. Notice that each period of the day is labeled numerically. To access a particular column within the data frame you use the $ operator. Thus, the reference, $p\$N$ accesses all the counts across all of the week, period, day combinations. The variable, $p\$N$, is subsequently used in the *hist*, *plot*, and *acf* commands.

Table B.3.: Computer Lab Arrival Data.

Week	Period	Day	N
1	1	M	8
1	2	M	8
1	3	M	9
1	4	M	10
1	5	M	6
1	6	M	11
1	7	M	12
1	8	M	8
2	1	M	10
2	2	M	6
2	3	M	7

Table B.3.: Computer Lab Arrival Data.

Week	Period	Day	N
2	4	M	11
2	5	M	10
2	6	M	10
2	7	M	5
2	8	M	13
3	1	M	5
3	2	M	15
3	3	M	5
3	4	M	7

As can be seen in Figure B.3 the data has a somewhat symmetric shape with nothing unusual appearing in the figure. The shape of the histogram is consistent with possible shapes associated with the Poisson distribution.

The time series plot, shown in Figure B.4, illustrates no significant patterns. We see random looking data centered around a common mean value with no trends of increasing or decreasing data points and no cyclical patterns of up and down behavior.

An autocorrelation plot allows the dependence within the data to be quickly examined. An autocorrelation plot is a time series assessment tool that plots the lag-k correlation versus the lag number. In order to understand these terms, we need to provide some background on how to think about a time series. A time series is a sequence of observations ordered by observation number, $X_1, X_2, ...X_n$. A time series, $X_1, X_2, ...X_n$, is said to be *covariance stationary* if:

- The mean of X_i, $E[X_i]$, exists and is a constant with respect to time. That is, $\mu = E[X_i]$ for $i=1, 2, …$

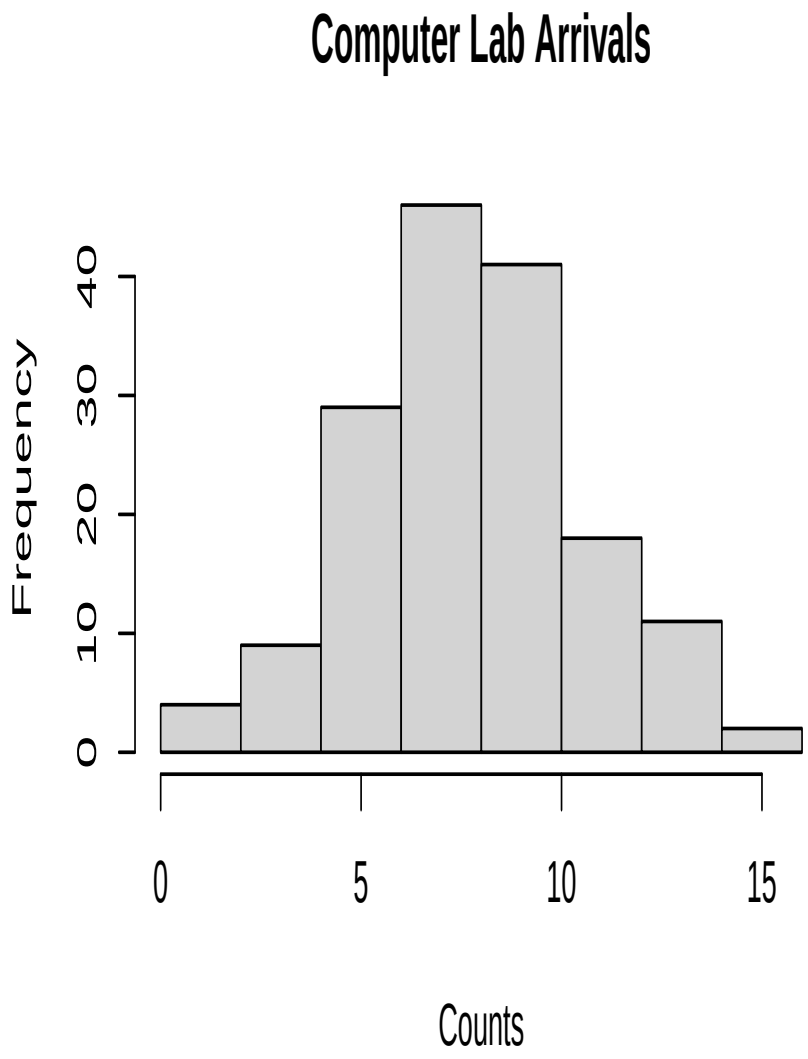


Figure B.3.: Histogram of Computer Lab Arrivals

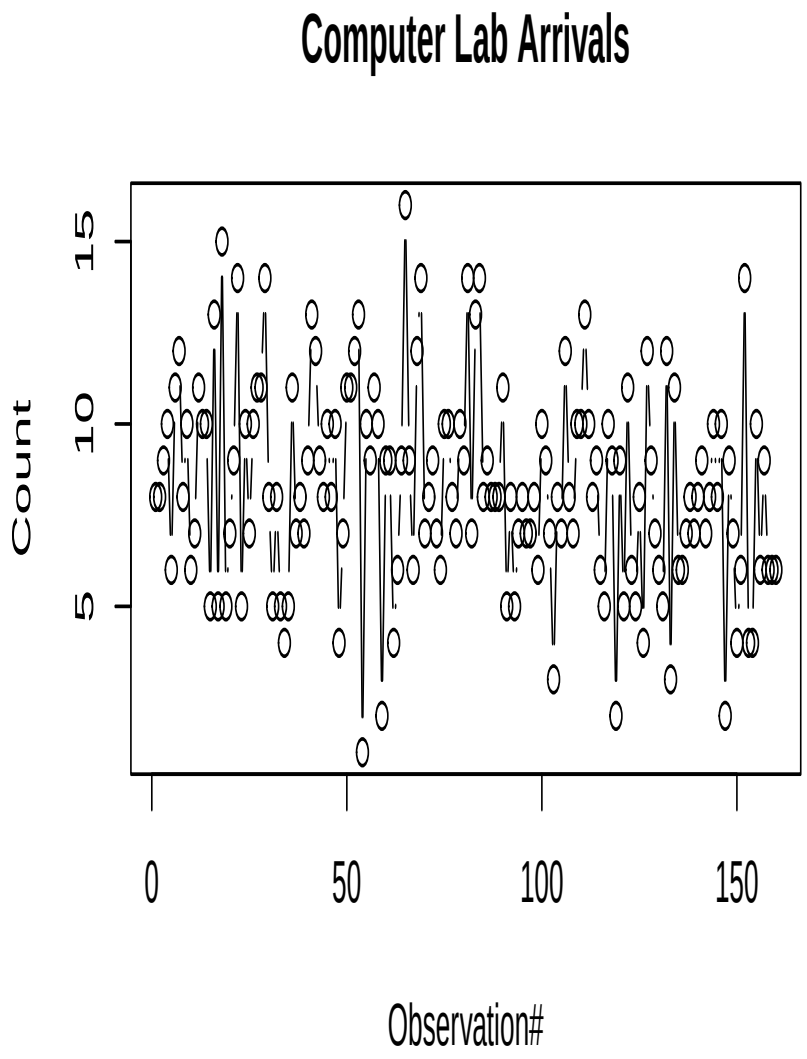


Figure B.4.: Time Series Plot of Computer Lab Arrivals

- The variance of X_i, $Var[X_i]$ exists and is constant with respect to time. That is, $\sigma^2 = Var[X_i]$ for $i=1, 2, …$
- The lag-k autocorrelation, $\rho_k = cor[X_i, X_{i+k}]$, is not a function of time i but is a function of the distance k between points. That is, the correlation between any two points in the series does not depend upon where the points are in the series, it depends only upon the distance between them in the series.

Recall that the correlation between two random variables is defined as:

$$cor[X, Y] = \frac{cov[X, Y]}{\sqrt{var[X]Var[Y]}}$$

where the covariance, $cov[X, Y]$ is defined by:

$$cov[X, Y] = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y]$$

The correlation between two random variables X and Y measures the strength of linear association. The correlation has no units of measure and has a range: $[-1, 1]$. If the correlation between the random variables is less than zero then the random variables are said to be *negatively correlated*. This implies that if X tends to be high then Y will tend to be low, or alternatively if X tends to be low then Y will tend to be high. If the correlation between the random variables is positive, the random variables are said to be *positively correlated*. This implies that if X tends to be high then Y will tend to be high, or alternatively if X tends to be low then Y will tend to be low. If the correlation is zero, then the random variables are said to be uncorrelated. If X and Y are independent random variables then the correlation between them will be zero. The converse of this is not necessarily true, but an assessment of the correlation should tell you something about the linear dependence between the random variables.

The autocorrelation between two random variables that are k time points apart in a covariance stationary time series is given by:

$$\begin{aligned}\rho_k = cor[X_i, X_{i+k}] &= \frac{cov[X_i, X_{i+k}]}{\sqrt{Var[X_i]Var[X_{i+k}]}} \\ &= \frac{cov[X_i, X_{i+k}]}{\sigma^2} \text{ for } k = 1, 2, ...\end{aligned}$$

A plot of ρ_k for increasing values of k is called an autocorrelation plot. The autocorrelation function as defined above is the theoretical function. When you have data, you must estimate the values of ρ_k from the actual times series. This involves forming an estimator for ρ_k. (Law 2007) suggests plotting:

$$\hat{\rho}_k = \frac{\hat{C}_k}{S^2(n)}$$

where

$$\hat{C}_k = \frac{1}{n-k}\sum_{i=1}^{n-k}\left(X_i - \bar{X}(n)\right)\left(X_{i+k} - \bar{X}(n)\right) \tag{B.1}$$

$$S^2(n) = \frac{1}{n-1}\sum_{i=1}^{n}\left(X_i - \bar{X}(n)\right)^2 \tag{B.2}$$

$$\bar{X}(n) = \frac{1}{n}\sum_{i}^{n} X_i \tag{B.3}$$

are the sample covariance, sample variance, and sample average respectively.

Some time series analysis books, see for examples Box et al. (1994), have a slightly different definition of the sample autocorrelation function:

$$r_k = \frac{c_k}{c_0} = \frac{\sum_{i=1}^{n-k} \left(X_i - \bar{X}(n)\right)\left(X_{i+k} - \bar{X}(n)\right)}{\sum_{i=1}^{n} \left(X_i - \bar{X}(n)\right)^2} \tag{B.4}$$

where

$$c_k = \frac{1}{n}\sum_{i=1}^{n-k} \left(X_i - \bar{X}(n)\right)\left(X_{i+k} - \bar{X}(n)\right) \tag{B.5}$$

Notice that the numerator in Equation B.4 has $n - k$ terms and the denominator has n terms. A plot of r_k versus k is called a correlogram or sample autocorrelation plot. For the data to be uncorrelated, r_k should be approximately zero for the values of k. Unfortunately, estimators of r_k are often highly variable, especially for large k; however, an autocorrelation plot should still provide you with a good idea of independence.

Formal tests can be performed using various time series techniques and their assumptions. A simple test can be performed based on the assumption that the series is white noise, $N(0, 1)$ with all $\rho_k = 0$. Box et al. (1994) indicate that for large n, $\text{Var}(r_k) \approx \frac{1}{n}$. Thus, a quick test of dependence is to check if sampled correlations fall within a reasonable confidence band around zero. For example, suppose $n = 100$, then $\text{Var}(r_k) \approx \frac{1}{100} = 0.01$. Then, the standard deviation is $\sqrt{0.01} = 0.1$. Assuming an approximate, 95% confidence level, yields a confidence band of $\pm 1.645 \times 0.1 = \pm 0.1645$ about zero. Therefore, as long as the plotted values for r_k do not fall outside of this confidence band, it is likely that the data is independent.

A sample autocorrelation plot can be easily developed once the autocorrelations have been computed. Generally, the maximum lag should be set to

no larger than one tenth of the size of the data set because the estimation of higher lags is generally unreliable.

As we can see from Figure B.5, there does not appear to be any significant correlation with respect to observation number for the computer lab arrival data. The autocorrelation is well-contained within the lag correlation limits denoted with the dashed lines within the plot.

Because arrival data often varies with time, it would be useful to examine whether or not the count data depends in some manner on when it was collected. For example, perhaps the computer lab is less busy on Fridays. In other words, the counts may depend on the day of the week. We can test for dependence on various factors by using a Chi-square based contingency table test. These tests are summarized in introductory statistics books. See (Montgomery and Runger 2006).

First, we can try to visualize any patterns based on the factors. We can do this easily with a scatter plot matrix within the *lattice* package of R. In addition, we can use the *xtabs* function to tabulate the data by week, period, and day. The *xtabs* function specifies a modeling relationship between the observations and factors. In the R listing, $N \sim Week + Period + Day$ indicates that we believe that the count column N in the data, depends on the $Week$, $Period$, and the Day. This builds a statistical object that can be summarized using the *summary* command.

```
library(lattice)
splom(p2)
mytable = xtabs(N~Week + Period + Day, data=p2)
summary(mytable)
Call: xtabs(formula = N ~ Week + Period + Day, data = p2)
Number of cases in table: 1324
Number of factors: 3
Test for independence of all factors:
    Chisq = 133.76, df = 145, p-value = 0.7384
```

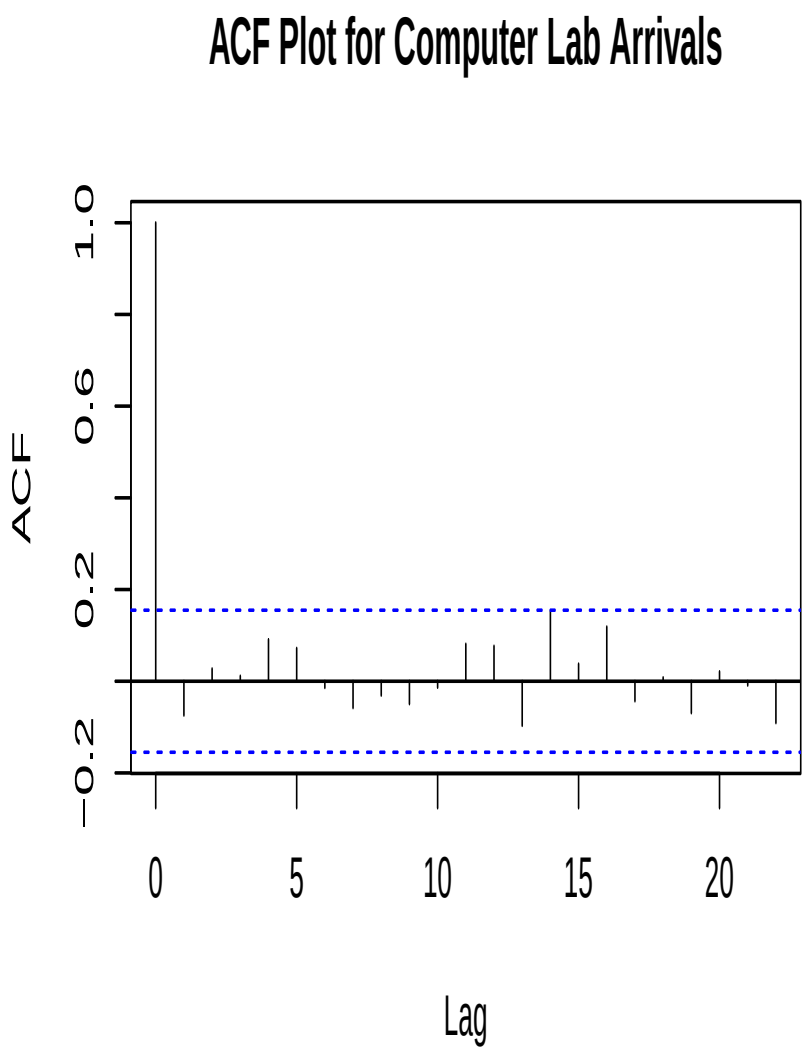


Figure B.5.: Autocorrelation Function Plot for Computer Lab Arrivals

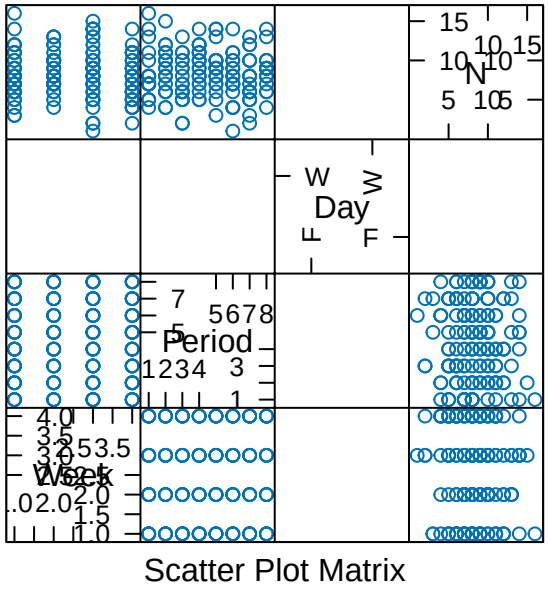


Figure B.6.: Scatter Plot Matrix from Lattice Package for Computer Lab Arrivals

A scatter plot matrix plots the variables in a matrix format that makes it easier to view pairwise relationships within the data. Figure B.6 presents the results of the scatter plot. Within the cells, the data looks reasonably 'uniform'. That is, there are no discernible patterns to be found. This provides evidence that there is not likely to be dependence between these factors. To formally test this hypothesis, we can use the multi-factor contingency table test provided by using the *summary* command on the output object, *myTable* of the *xtabs* command.

The results of using, *summary(mytable)* show that the chi-square test statistic has a very high p-value, 0.7384, when testing if N depends on $Week$, $Period$, and Day. The null hypothesis,H_0, of a contingency table test of this form states that the counts are independent of the factors versus the alternative, H_a, that the counts are dependent on the factors. Since the p-value is very high, we should not reject H_0. What does this all mean? In essence, we can now treat the arrival count observation as 160 independent observations. Thus, we can proceed with formally testing if the counts come from a Poisson distribution without worrying about time or factor dependencies within the data. If the results of the analysis indicated dependence on the factors, then we might need to fit separate distributions based on the dependence. For example, if we concluded that the days of the week were different (but the week and period did not matter), then we could try to fit a separate Poisson distribution for each day of the week. When the mean rate of occurrence depends on time, this situation warrants the investigation of using a non-homogeneous (non-stationary) Poisson process. The estimation of the parameters of a non-stationary Poisson process is beyond the scope of this text. The interested reader should refer to (Leemis 1991) and other such references.

B.3.3. Estimating the Rate Parameter for the Poisson Distribution

Testing if the Poisson distribution is a good model for this data can be accomplished using various statistics tests for a Poisson distribution. The basic approach is to compare the hypothesized distribution function in the form of the PDF, PMF, or the CDF to a fit of the data. This implies that you have hypothesized a distribution *and* estimated the parameters of the distribution in order to compare the hypothesized distribution to the data. For example, suppose that you hypothesize that the Poisson distribution will be a good model for the count data. Then, you need to estimate the rate parameter associated with the Poisson distribution.

There are two main methods for estimating the parameters of distribution functions 1) the method of moments and 2) the method of maximum likelihood. The method of moments matches the empirical moments to the theoretical moments of the distribution and attempts to solve the resulting system of equations. The method of maximum likelihood attempts to find the parameter values that maximize the joint probability distribution function of the sample. Estimation of the parameters from sample data is based on important statistical theory that requires the estimators for the parameters to satisfy statistical properties (e.g. unique, unbiased, invariant, and consistency). It is beyond the scope of this book to cover the properties of these techniques. The interested reader is referred to (Law 2007) or to (Casella and Berger 1990) for more details on the theory of these methods.

To make concrete the challenges associated with fitting the parameters of a hypothesized distribution, the maximum likelihood method for fitting the Poisson distribution will be used on the count data. Suppose that you hypothesize that the distribution of the count of the number of arrivals in the i^{th} 15 minute interval can be modeled with a Poisson distribution. Let N be the number of arrivals in a 15 minute interval. Note that the intervals do not overlap and that we have shown that they can be considered

independent of each other. We are hypothesizing that N has a Poisson distribution with rate parameter λ, where λ represents the expected number of arrivals per 15 minute interval.

$$f(n;\lambda) = P\{N = n\} = \frac{e^{-\lambda}\,(\lambda)^n}{n!}$$

Let N_i be the number of arrivals in the i^{th} interval of the $k = 160$ intervals. The $N_1, N_2, ..., N_k$ form a random sample of size k. Then, the joint probability mass function of the sample is:

$$L(\lambda) = g(n_1, n_2, ..., n_k; \lambda) = f(n_1;\lambda)f(n_2;\lambda)\,...\,f(n_k;\lambda) = \prod_{i=1}^{k} f(n_i;\lambda)$$

The $(n_1, n_2, ..., n_k)$ are observed (known values) and λ is unknown. The function $L(\lambda)$ is called the likelihood function. To estimate the value of λ by the method of maximum likelihood, we must find the value of λ that maximizes the function $L(\lambda)$. The interpretation is that we are finding the value of the parameter that is maximizing the likelihood that it came from this sample.

Substituting the definition of the Poisson distribution into the likelihood function yields:

$$\begin{aligned} L(\lambda) &= \prod_{i=1}^{k} \frac{e^{-\lambda}\,(\lambda)^{n_i}}{n_i!} \\ &= \frac{e^{-k\lambda}\lambda^{\sum_{i=1}^{k} n_i}}{\prod_{i=1}^{k} n_i!} \end{aligned}$$

It can be shown that maximizing $L(\lambda)$ is the same as maximizing $ln(L(\lambda))$. This is called the log-likelihood function. Thus,

$$ln(L(\lambda)) = -k\lambda + ln(\lambda)\sum_{i=1}^{k} n_i - \sum_{i=1}^{k} ln(n_i!)$$

Differentiating with respect to λ, yields,

$$\frac{dln(L(\lambda))}{d\lambda} = -k + \frac{\sum_{i=1}^{k} n_i}{\lambda}$$

When we set this equal to zero and solve for λ, we get

$$\begin{aligned} 0 &= -k + \frac{\sum_{i=1}^{k} n_i}{\lambda} \\ \hat{\lambda} &= \frac{\sum_{i=1}^{k} n_i}{k} \end{aligned} \tag{B.6}$$

If the second derivative, $\frac{d^2 lnL(\lambda)}{d\lambda^2} < 0$ then a maximum is obtained.

$$\frac{d^2 lnL(\lambda)}{d\lambda^2} = \frac{-\sum_{i=1}^{k} n_i}{\lambda^2}$$

because the n_i are positive and λ is positive the second derivative must be negative; therefore the maximum likelihood estimator for the parameter of the Poisson distribution is given by Equation B.6. Notice that this is the sample average of the interval counts, which can be easily computed using the *mean()* function within R.

While the Poisson distribution has an analytical form for the maximum likelihood estimator, not all distributions will have this property. Estimating the parameters of distributions will, in general, involve non-linear optimization. This motivates the use of software tools such as R when performing the analysis. Software tools will perform this estimation process

with little difficulty. Let's complete this example using R to fit and test whether or not the Poisson distribution is a good model for this data.

B.3.4. Chi-Squared Goodness of Fit Test for Poisson Distribution

In essence, a distributional test examines the hypothesis $H_0 : X_i \sim F_0$ versus the alternate hypothesis of $H_a : X_i \nsim F_0$. That is, the null hypothesis is that data come from distribution function, F_0 and the alternative hypothesis that the data is not distributed according to F_0.

As a reminder, when using a hypothesis testing framework, we can perform the test in two ways: 1) pre-specifying the Type 1 error and comparing to a critical value or 2) pre-specifying the Type 1 error and comparing to a p-value. For example, in the first case, suppose we let our Type 1 error $\alpha = 0.05$, then we look up a critical value appropriate for the test, compute the test statistic, and reject the null hypothesis H_0 if test statistic is too extreme (large or small depending on the test). In the second case, we compute the p-value for the test based on the computed test statistic, and then reject H_0 if the p-value is less than or equal to the specified Type 1 error value.

This text emphasizes the p-value approach to hypothesis testing because computing the p-value provides additional information about the significance of the result. The p-value for a statistical test is the smallest α level at which the observed test statistic is significant. This smallest α level is also called the observed level of significance for the test. The smaller the p-value, the more the result can be considered statistically significant. Thus, the p-value can be compared to the desired significance level, α. Thus, when using the p-value approach to hypothesis testing the testing criterion is:

- If the p-value $> \alpha$, then do not reject H_0
- If the p-value $\leq \alpha$, then reject H_0

An alternate interpretation for the p-value is that it is the probability assuming H_0 is true of obtaining a test statistic at least as *extreme* as the observed value. *Assuming* that H_0 is true, then the test statistic will follow a known distributional model. The p-value can be interpreted as the chance assuming that H_0 is true of obtaining a more extreme value of the test statistic than the value actually obtained. Computing a p-value for a hypothesis test allows for a different mechanism for accepting or rejecting the null hypothesis. Remember that a Type I error is

$$\alpha = P(\text{Type 1 error}) = P(\text{rejecting the null when it is in fact true})$$

This represents the chance you are willing to take to make a mistake in your conclusion. The p-value represents the observed chance associated with the test statistic being more extreme under the assumption that the null is true. A small p-value indicates that the observed result is rare under the assumption of H_0. If the p-value is small, it indicates that an outcome as extreme as observed is possible, but not probable under H_0. In other words, chance by itself may not adequately explain the result. So for a small p-value, you can reject H_0 as a plausible explanation of the observed outcome. If the p-value is large, this indicates that an outcome as extreme as that observed can occur with high probability. For example, suppose you observed a p-value of 0.001. This means that assuming H_0 is true, there was a 1 in 1000 chance that you would observe a test statistic value at least as extreme as what you observed. It comes down to whether or not you consider a 1 in 1000 occurrence a rare or non-rare event. In other words, do you consider yourself that lucky to have observed such a rare event. If you do not think of this as lucky but you still actually observed it, then you might be suspicious that something is not "right with the world". In other words, your assumption that H_0 was true is likely wrong. Therefore, you reject the null hypothesis, H_0, in favor of the alternative hypothesis, H_a. The risk of you making an incorrect decision here is what you consider a rare event, i.e. your α level.

For a discrete distribution, the most common distributional test is the Chi-Squared goodness of fit test, which is the subject of the next section.

B.3.5. Chi-Squared Goodness of Fit Test

The Chi-Square Test divides the range of the data into, k, intervals (or classes) and tests if the number of observations that fall in each interval (or class) is close the expected number that should fall in the interval (or class) given the hypothesized distribution is the correct model. In the case of discrete data, the intervals are called classes and they are mapped to groupings along the domain of the random variable. For example, in the case of a Poisson distribution, a possible set of $k = 7$ classes could be $\{0\}$, $\{1\}$, $\{2\}$, $\{3\}$, $\{4\}$, $\{5\}$, $\{6 \text{ or more}\}$. As a general rule of thumb, the classes should be chosen so that the expected number of observations in each class is at least 5.

Let c_j be the observed count of the observations contained in the j^{th} class. The Chi-Squared test statistic has the form:

$$\chi_0^2 = \sum_{j=1}^{k} \frac{\left(c_j - np_j\right)^2}{np_j} \tag{B.7}$$

The quantity np_j is the expected number of observations that should fall in the j^{th} class when there are n observations.

For large n, an approximate $1 - \alpha$ level hypothesis test can be performed based on a Chi-Squared test statistic that rejects the null hypothesis if the computed χ_0^2 is greater than $\chi_{\alpha,k-s-1}^2$, where s is the *number of estimated parameters* for the hypothesized distribution. $\chi_{\alpha,k-s-1}^2$ is the upper critical value for a Chi-squared random variable χ^2 such that $P\{\chi^2 > \chi_{\alpha,k-s-1}^2\} = \alpha$. The p-value, $P\{\chi_{k-s-1}^2 > \chi_0^2\}$, for this test can be computed in using the following formula:

$$\text{CHISQ.DIST.RT}(\chi_0^2, k-s-1)$$

In the statistical package R, the formula is:

$$\text{pchisq}(\chi_0^2, k-s-1, lower.tail = FALSE)$$

The null hypothesis is that the data come from the hypothesized distribution versus the alternative hypothesis that the data do not come from the hypothesized distribution.

Let's perform a Chi-squared goodness of fit test of computer lab data for the Poisson distribution. A good first step in analyzing discrete data is to summarize the observations in a table. We can do that easily with the R *table()* function.

```
# tabulate the counts
tCnts = table(p2$N)
tCnts
```

```
 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16
 1  3  2  7 13 16 21 25 20 21 11  7  5  6  1  1
```

From this analysis, we see that there were no week-period combinations that had 0 observations, 1 that had 1 count, 3 that had 2 counts, and so forth. Now, we will estimate the rate parameter of the hypothesized Poisson distribution using Equation B.6 and tabulate the expected number of observations for a proposed set of classes.

```
# get number of observations
n = length(p2$N)
n
```

```
[1] 160
```

```
# estimate the rate for Poisson from the data
lambda = mean(p2$N)
lambda
```

```
[1] 8.275
```

```
# setup vector of x's across domain of Poisson
x = 0:15
x
```

```
 [1]  0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

```
# compute the probability for Poisson
prob = dpois(x, lambda)
prob
```

```
 [1] 0.0002548081 0.0021085367 0.0087240706 0.0240638947 0.0497821822
 [6] 0.0823895116 0.1136288681 0.1343255548 0.1389429957 0.1277503655
[11] 0.1057134275 0.0795253284 0.0548393410 0.0349073498 0.0206327371
[16] 0.0113823933
```

```
# view the expected counts for these probabilities
prob*n
```

```
 [1]  0.04076929  0.33736587  1.39585130  3.85022316  7.96514916 13.18232186
 [7] 18.18061890 21.49208877 22.23087932 20.44005848 16.91414839 12.72405254
[13]  8.77429457  5.58517596  3.30123794  1.82118293
```

We computed the probability associated with the Poisson distribution with $\hat{\lambda} = 8.275$. Recall that the Possion distribution has the following form:

$$P\{X = x\} = \frac{e^{-\lambda}\lambda^x}{x!} \quad \lambda > 0, \quad x = 0, 1, ...$$

Then, by multiplying each probability, $P\{X = i\} = p_i$ by the number of observations $n = 160$, we can get the expected number, $n \times p_i$, that we should observed under the hypothesized distribution. Since the expected values for $x = 0, 1, 2, 3, 4$ are so small, we consolidate them in to one class to have the expected number to be at least 5. We will also consolidate the range $x \geq 13$ into a single class and the values of 11 and 12 into a class.

```
# compute the probability for the classes
# the vector is indexed starting at 1, prob[1] = P(X=0)
cProb = c(sum(prob[1:5]), prob[6:11], sum(prob[12:13]),
 ↪  ppois(12, lambda, lower.tail = FALSE))
cProb
```

```
[1] 0.08493349 0.08238951 0.11362887 0.13432555 0.13894300 0.12775037 0.10571343
[8] 0.13436467 0.07795112
```

```
# compute the expected counts for each of the classes
expected = cProb*n
expected
```

```
[1] 13.58936 13.18232 18.18062 21.49209 22.23088 20.44006 16.91415 21.49835
[9] 12.47218
```

Thus, we will use the following $k = 9$ classes: {0,1,2,3,4}, {5}, ···, {10}, {11,12}, and {13 or more }. Now, we need to summarize the observed frequencies for the proposed classes.

```
# transform tabulated counts to data frame
dfCnts = as.data.frame(tCnts)
# extract only frequencies
cnts = dfCnts$Freq
cnts
```

```
 [1]  1  3  2  7 13 16 21 25 20 21 11  7  5  6  1  1
```

```
# consolidate classes for observed
observed = c(sum(cnts[1:4]), cnts[5:10], sum(cnts[11:12]),
 ↪  sum(cnts[13:16]))
observed
```

```
[1] 13 13 16 21 25 20 21 18 13
```

Now, we have both the observed and expected tabulated and can proceed with the chi-squared goodness of fit test. We will compute the result directly using Equation B.7.

```
# compute the observed minus expected components
chisq = ((observed - expected)^2)/expected
# compute the chi-squared test statistic
sumchisq = sum(chisq)
# chi-squared test statistic
print(sumchisq)
```

```
[1] 2.233903
```

```
# set the degrees of freedom, with 1 estimated parameter s =
 ↪  1
df = length(expected) - 1 - 1
# compute the p-value
pvalue = 1 - pchisq(sumchisq, df)
# p-value
print(pvalue)
```

```
[1] 0.9457686
```

Based on such a high p-value, we would not reject H_0. Thus, we can conclude that there is not enough evidence to suggest that the lab arrival count data is some other distribution than the Poisson distribution. In this section, we did the Chi-squared test *by-hand*. R has a package that facilitates distribution fitting called, *fitdistrplus*. We will use this package to analyze the computer lab arrival data again in the next section.

B.3.6. Using the fitdistrplus R Package on Discrete Data

Fortunately, R has a very useful package for fitting a wide variety of discrete and continuous distributions call the *fitdistrplus* package. To install and load the package do the following:

```
install.packages('fitdistrplus')
library(fitdistrplus)
plotdist(p2$N, discrete = TRUE)
```

The *plotdist* command will plot the empirical probability mass function and the cumulative distribution function as illustrated in Figure B.7.

To perform the fitting and analysis, you use the *fitdist* and *gofstat* commands. In addition, plotting the output from the *fitdist* function will

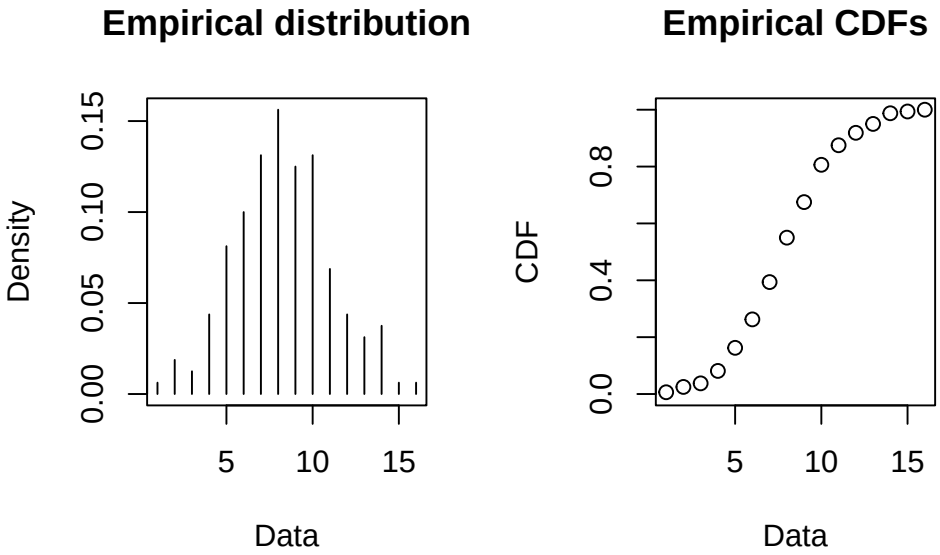


Figure B.7.: Plot of the Empirical PMF and CDF of the Computer Lab Arrivals

provide a comparison of the empirical distribution to the theoretical distribution.

```
fp = fitdist(p2$N, "pois")
summary(fp)
plot(fp)
gofstat(fp)
```

```
fp = fitdist(p2$N, "pois")
summary(fp)
```

```
Fitting of the distribution ' pois ' by maximum likelihood
Parameters :
       estimate Std. Error
lambda    8.275  0.2274176
Loglikelihood:  -393.7743   AIC:  789.5485   BIC:  792.6237
```

```
gofstat(fp)
```

```
Chi-squared statistic:  2.233903
Degree of freedom of the Chi-squared distribution:  7
Chi-squared p-value:  0.9457686
Chi-squared table:
      obscounts theocounts
<= 4   13.00000   13.58936
<= 5   13.00000   13.18232
<= 6   16.00000   18.18062
<= 7   21.00000   21.49209
<= 8   25.00000   22.23088
<= 9   20.00000   20.44006
<= 10  21.00000   16.91415
```

```
<= 12   18.00000   21.49835
> 12    13.00000   12.47218

Goodness-of-fit criteria
                                  1-mle-pois
Akaike's Information Criterion     789.5485
Bayesian Information Criterion     792.6237
```

The output of the *fitdist* and the *summary* commands provides the estimate of $\lambda = 8.275$. The result object, *fp*, returned by the *fitdist* can then subsequently be used to plot the fit, *plot* and perform a goodness of fit test, *gofstat*.

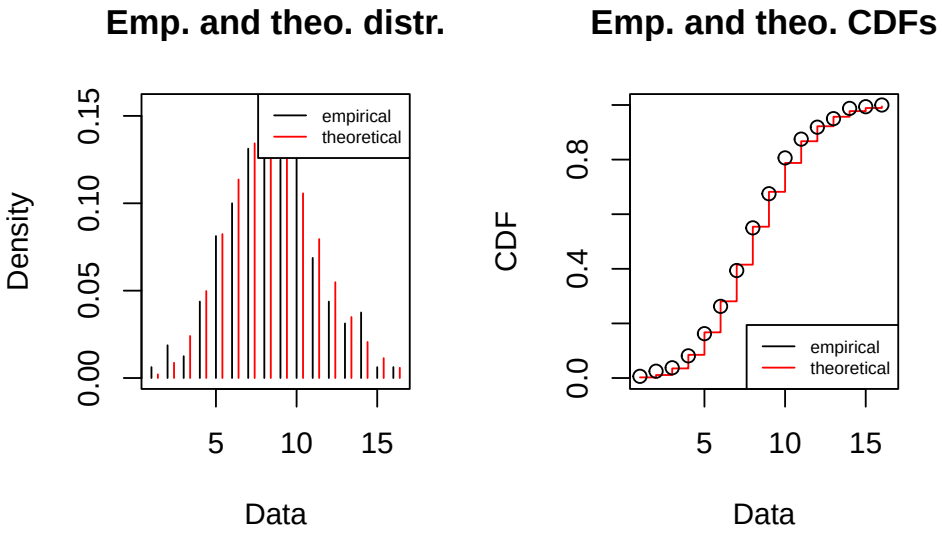


Figure B.8.: Plot of the Empirical and Theoretical CDF of the Computer Lab Arrivals

The *gofstat* command performs a chi-squared goodness of fit test and computes the chi-squared statistic value (here 2.233903) and the p-value (0.9457686). These are exactly the same results that we computed in the

previous section. Clearly, the chi-squared test statistic p-value is quite high. Again the null hypothesis is that the observations come from a Poisson distribution with the alternative that they do not. The high p-value suggests that we should not reject the null hypothesis and conclude that a Poisson distribution is a very reasonable model for the computer lab arrival counts.

B.3.7. Fitting a Discrete Empirical Distribution

We are interested in modeling the number of packages delivered on a small parcel truck to a hospital loading dock. A distribution for this random variable will be used in a loading dock simulation to understand the ability of the workers to unload the truck in a timely manner. Unfortunately, a limited sample of observations is available from 40 different truck shipments as shown in Table B.4

Table B.4.: Loading Dock Data

130	130	110	130	130	110	110	130	120	130
140	140	130	110	110	140	130	140	130	120
140	150	120	150	120	130	120	100	110	150
130	120	120	130	120	120	130	130	130	100

Developing a probability model will be a challenge for this data set because of the limited amount of data. Using the following *R* commands, we can get a good understanding of the data. Assuming that the data has been read into the variable, *packageCnt*, the *hist()*, *stripchart()*, and *table()* commands provide an initial analysis.

```
packageCnt = scan("data/AppDistFitting/TruckLoadingData.txt")
hist(packageCnt, main="Packages per Shipment", xlab="#packages")
stripchart(packageCnt,method="stack",pch="o")
table(packageCnt)
```

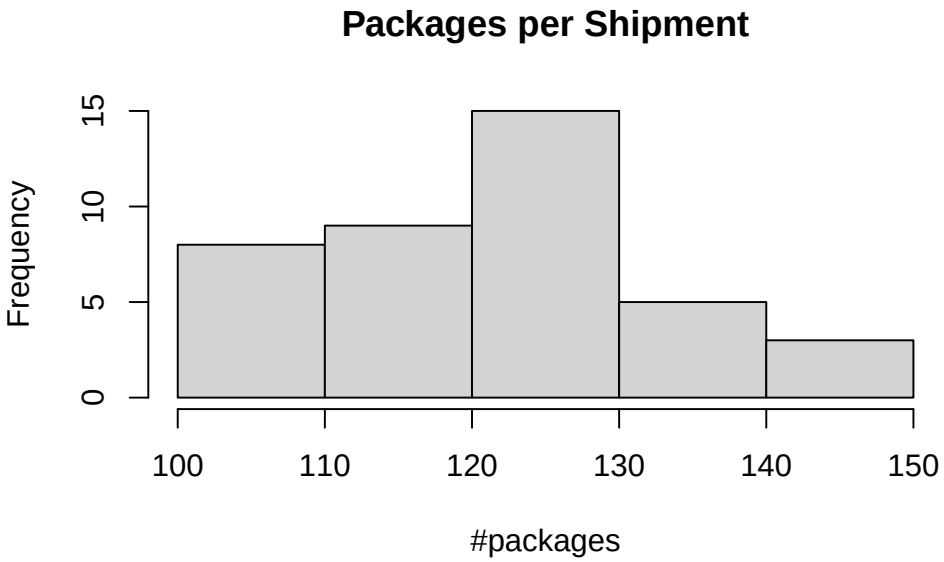


Figure B.9.: Histogram of Package Count per Shipment

As we can see from the *R* output, the range of the data varies between 100 and 150. The histogram, shown in Figure B.9, illustrates the shape of the data. It appears to be slightly positively skewed. Figure B.10 presents a dot plot of the observed counts. From this figure, we can clearly see that the only values obtained in the sample are 100, 110, 120, 130, 140, and 150. It looks as though the ordering comes in tens, starting at 100 units.

Because of the limited size of the sample and limited variety of data points, fitting a distribution will be problematic. However, we can fit a discrete empirical distribution to the proportions observed in the sample. Using the table() command, we can summarize the counts. Then, by dividing by 40 (the total number of observations), we can get the proportions as show in the *R* listing.

```
tp = table(packageCnt)
tp/length(packageCnt)
```

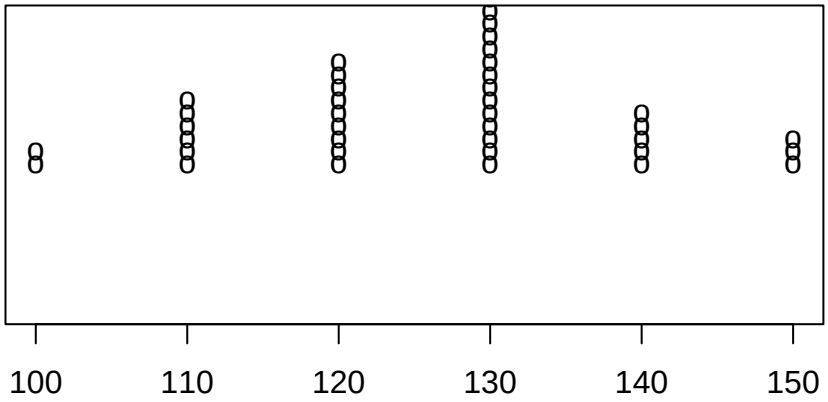


Figure B.10.: Dot Plot of Package Countper Shipment

```
packageCnt
  100   110   120   130   140   150
0.050 0.150 0.225 0.375 0.125 0.075
```

Thus, we can represent this situation with the following probability mass and cumulative distribution functions.

$$P\{X = x\} = \begin{cases} 0.05 & \text{x} = 100 \\ 0.15 & \text{x} = 110 \\ 0.225 & \text{x} = 120 \\ 0.375 & \text{x} = 130 \\ 0.125 & x = 140 \\ 0.075 & x = 150 \end{cases}$$

$$F(x) = \begin{cases} 0.0 & \text{if } x < 100 \\ 0.05 & \text{if } 100 \leq x < 110 \\ 0.20 & \text{if } 110 \leq x < 120 \\ 0.425 & \text{if } 120 \leq x < 130 \\ 0.80 & \text{if } 130 \leq x < 140 \\ 0.925 & \text{if } 140 \leq x < 150 \\ 1.0 & \text{if } x \geq 150 \end{cases}$$

The previous two examples illustrated the process for fitting a discrete distribution to data for use in a simulation model. Because discrete event dynamic simulation involves modeling the behavior of a system over time, the modeling of distributions that represent the time to perform a task is important. Since the domain of time is on the set of positive real numbers, it is a continuous variable. We will explore the modeling of continuous distributions in the next section.

B.4. Modeling with Continuous Distributions

Continuous distributions can be used to model situations where the set of possible values occurs in an interval or set of intervals. Within discrete event simulation, the most common use of continuous distributions is for the modeling of the time to perform a task. Section D.2 summarizes the properties of common continuous distributions.

The continuous uniform distribution can be used to model situations in which you have a lack of data and it is reasonable to assume that everything is equally likely within an interval. Alternatively, if you have no a priori knowledge that some events are more likely than others, then a uniform distribution seems like a reasonable starting point. The uniform distribution is also commonly used to model machine processing times that have very precise time intervals for completion. The expected value and variance of

a random variable with a continuous uniform distribution over the interval (a, b) is:

$$E[X] = \frac{a+b}{2}$$
$$Var[X] = \frac{(b-a)^2}{12}$$

Often the continuous uniform distribution is specified by indicating the $\pm$ around the expected value. For example, we can say that a continuous uniform over the range (5, 10) is the same as a uniform with 7.5 ± 2.5. The uniform distribution is symmetric over its defined interval.

The triangular distribution is also useful in situations with a lack of data if you can characterize a most likely value for the random variable in addition to its range (minimum and maximum). This makes the triangular distribution very useful when the only data that you might have on task times comes from interviewing people. It is relatively easy for someone to specify the most likely task time, a minimum task time, and a maximum task time. You can create a survey instrument that asks multiple people familiar with the task to provide these three estimates. From, the survey you can average the responses to develop an approximate model. This is only one possibility for how to combine the survey values.

If the most likely value is equal to one-half the range, then the triangular distribution is symmetric. In other words, fifty percent of the data is above and below the most likely value. If the most likely value is closer to the minimum value then the triangular distribution is right-skewed (more area to the right). If the most likely value is closer to the maximum value then the triangular distribution is left-skewed. The ability to control the skewness of the distribution in this manner also makes this distribution attractive.

The beta distribution can also be used to model situations where there is a lack data. It is a bounded continuous distribution over the range from (0,

1) but can take on a wide variety of shapes and skewness characteristics. The beta distribution has been used to model the task times on activity networks and for modeling uncertainty concerning the probability parameter of a discrete distribution, such as the binomial. The beta distribution is commonly shifted to be over a range of values (a, b).

The exponential distribution is commonly used to model the time between events. Often, when only a mean value is available (from the data or from a guess), the exponential distribution can be used. A random variable, X, with an exponential distribution rate parameter λ has:

$$E[X] = \frac{1}{\lambda}$$
$$Var[X] = \frac{1}{\lambda^2}$$

Notice that the variance is the square of the expected value. This is considered to be highly variable. The coefficient of variation for the exponential distribution is $c_v = 1$. Thus, if the coefficient of variation estimated from the data has a value near 1.0, then an exponential distribution may be possible choice for modeling the situation.

An important property of the exponential distribution is the lack of memory property. The lack of memory property of the exponential distribution states that given Δt is the time period that elapsed since the occurrence of the last event, the time t remaining until the occurrence of the next event is independent of Δt. This implies that, $P\{X > \Delta t + t | X > t\} = P\{X > t\}$. This property indicates that the probability of the occurrence of the next event is dependent upon the length of the interval since the last event, but not the absolute time of the last occurrence. It is the interval of elapsed time that matters. In a sense the process's clock resets at each event time and the past does not matter when predicting the future. Thus, it "forgets" the past. This property has some very important implications, especially when modeling the time to failure. In most situations, the history of the process does matter (such as wear and tear on the machine). In which case,

the exponential distribution may not be appropriate. Other distributions of the exponential family may be more useful in these situations such as the gamma and Weibull distributions. Why is the exponential distribution often used? Two reasons: 1) it often is a good model for many situations found in nature and 2) it has very convenient mathematical properties.

While the normal distribution is a mainstay of probability and statistics, you need to be careful when using it as a distribution for input models because it is defined over the entire range of real numbers. For example, within simulation the time to perform a task is often required; however, time must be a positive real number. Clearly, since a normal distribution can have negative values, using a normal distribution to model task times can be problematic. If you attempt to delay for negative time you will receive an error. Instead of using a normal distribution, you might use a truncated normal, see Section A.2.4. Alternatively, you can choose from any of the distributions that are defined on the range of positive real numbers, such as the lognormal, gamma, Weibull, and exponential distributions. The lognormal distribution is a convenient choice because it is also specified by two parameters: the mean and variance.

Table B.5 lists common modeling situations for various continuous distributions.

Table B.5.: Common Modeling Situations for Continuous Distribution

Distribution	Modeling Situations
Uniform	when you have no data, everything is equally likely to occur within an interval, machi
Normal	modeling errors, modeling measurements, length, etc., modeling the sum of a large nu
Exponential	time to perform a task, time between failures, distance between defects
Erlang	service times, multiple phases of service with each phase exponential
Weibull	time to failure, time to complete a task
Gamma	repair times, time to complete a task, replenishment lead time
Lognormal	time to perform a task, quantities that are the product of a large number of other qua
Triangular	rough model in the absence of data assume a minimum, a maximum, and a most likel

Distribution	Modeling Situations
Beta	useful for modeling task times on bounded range with little data, modeling probabilit

Once we have a good idea about the type of random variable (discrete or continuous) and some ideas about the distribution of the random variable, the next step is to fit a distributional model to the data. In the following sections, we will illustrate how to fit continuous distributions to data.

B.5. Fitting Continuous Distributions

Previously, we examined the modeling of discrete distributions. In this section, we will look at modeling a continuous distribution using the functionality available in R. This example starts with step 3 of the input modeling process. That is, the data has already been collected. Additional discussion of this topic can be found in Chapter 6 of (Law 2007).

Example B.2 (Fitting a Gamma Distribution)**.** Suppose that we are interested in modeling the time that it takes to perform a computer component repair task. The 100 observations are provide below in minutes. Fit an appropriate distribution to this data.

	1	2	3	4	5	6	7	8	9	10
1	15.3	10.0	12.6	19.7	9.4	11.7	22.6	13.8	15.8	17.2
2	12.4	3.0	6.3	7.8	1.3	8.9	10.2	5.4	5.7	28.9
3	16.5	15.6	13.4	12.0	8.2	12.4	6.6	19.7	13.7	17.2

	1	2	3	4	5	6	7	8	9	10
4	3.8	9.1	27.0	9.7	2.3	9.6	8.3	8.6	14.8	11.1
5	19.5	5.3	25.1	13.5	24.7	9.7	21.0	3.9	6.2	10.9
6	7.0	10.5	16.1	5.2	23.0	16.0	11.3	7.2	8.9	7.8
7	20.1	17.8	14.4	8.4	12.1	3.6	10.9	19.6	14.1	16.1
8	11.8	9.2	31.4	16.4	5.1	20.7	14.7	22.5	22.1	22.7
9	22.8	17.7	25.6	10.1	8.2	24.4	30.8	8.9	8.1	12.9
10	9.8	5.5	7.4	31.5	29.1	8.9	10.3	8.0	10.9	6.2

B.5.1. Visualizing the Data

The first steps are to visualize the data and check for independence. This can be readily accomplished using the *hist*, *plot*, and *acf* functions in R. Assume that the data is in a file called, *taskTimes.txt* within the R working directory.

```
y = scan(file="data/AppDistFitting/taskTimes.txt")
hist(y, main="Task Times", xlab = "minutes")
plot(y,type="b",main="Task Times", ylab = "minutes", xlab = "Observation#")
acf(y, main = "ACF Plot for Task Times")
```

As can be seen in Figure B.11, the histogram is slightly right skewed.

The time series plot, Figure B.12, illustrates no significant pattern (e.g. trends, etc.).

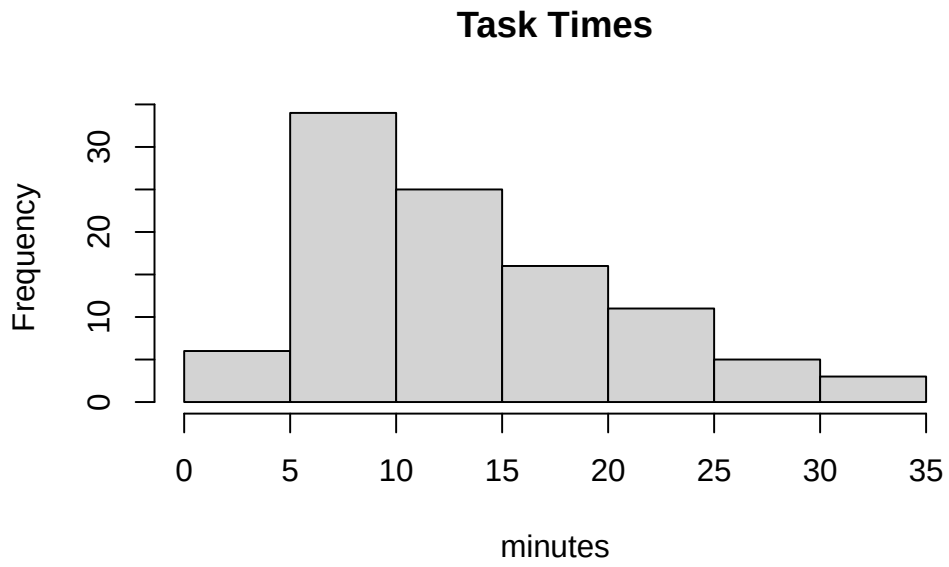


Figure B.11.: Histogram of Computer Repair Task Times

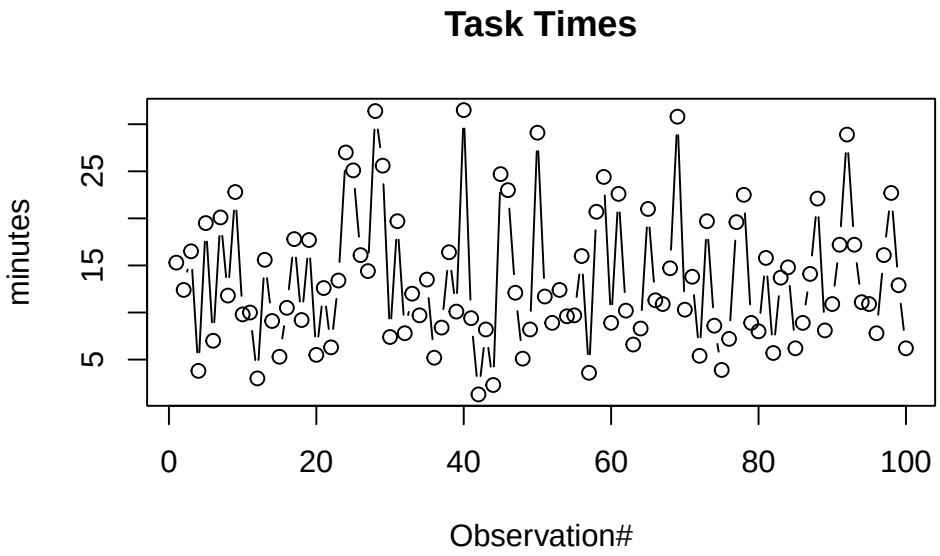


Figure B.12.: Time Series Plot of Computer Repair Task Times

Finally, the autocorrelation plot, Figure B.13, shows no significant correlation (the early lags are well within the confidence band) with respect to observation number.

Based on the visual analysis, we can conclude that the task times are likely to be independent and identically distributed.

B.5.2. Statistically Summarize the Data

An analysis of the statistical properties of the task times can be easily accomplished in R using the *summary*, *mean*, *var*, *sd*, and *t.test* functions. The *summary* command summarizes the distributional properties in terms of the minimum, maximum, median, and 1st and 3rd quartiles of the data.

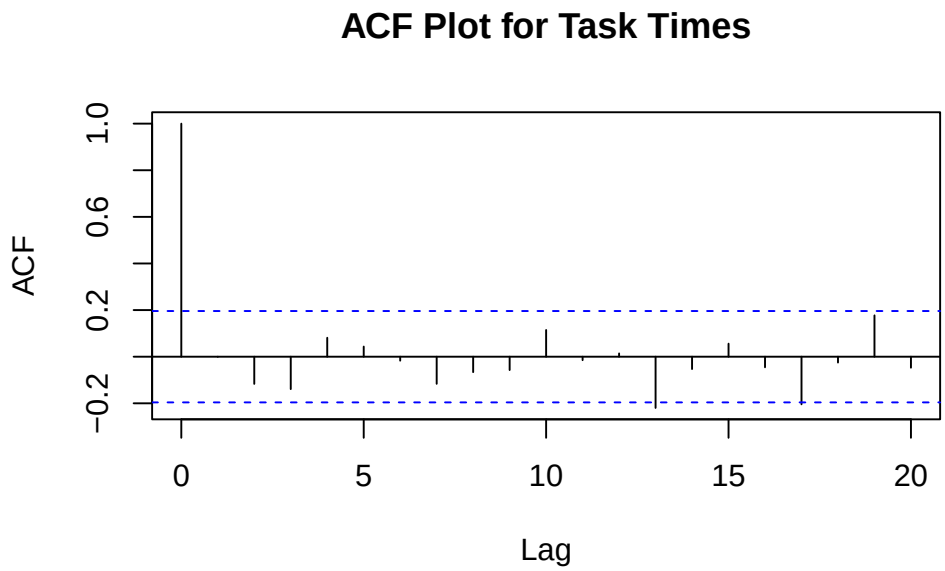


Figure B.13.: ACF Plot of Computer Repair Task Times

```
summary(y)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  1.300   8.275  11.750  13.412  17.325  31.500
```

The *mean*, *var*, *sd* commands compute the sample average, sample variance, and sample standard deviation of the data.

```
mean(y)
```

```
[1] 13.412
```

```
var(y)
```

```
[1] 50.44895
```

```
sd(y)
```

```
[1] 7.102742
```

Finally, the *t.test* command can be used to form a 95% confidence interval on the mean and test if the true mean is significantly different from zero.

```
t.test(y)
```

```
    One Sample t-test

data:  y
t = 18.883, df = 99, p-value < 2.2e-16
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 12.00266 14.82134
sample estimates:
mean of x
   13.412
```

The *descdist* command of the *fitdistrplus* package will also provide a description of the distribution's properties.

```
descdist(y, graph=FALSE)
```

```
summary statistics
------
min:  1.3   max:  31.5
median:  11.75
mean:  13.412
estimated sd:  7.102742
estimated skewness:  0.7433715
estimated kurtosis:  2.905865
```

The median is less than the mean and the skewness is less than 1.0. This confirms the visual conclusion that the data is slightly skewed to the right. Before continuing with the analysis, let's recap what has been learned so far:

- The data appears to be stationary. This conclusion is based on the time series plot where no discernible trend with respect to time is found in the data.
- The data appears to be independent. This conclusion is from the autocorrelation plot.
- The distribution of the data is positively (right) skewed and unimodal. This conclusion is based on the histogram and from the statistical summary.

B.5.3. Hypothesizing and Testing a Distribution

The next steps involve the model fitting processes of hypothesizing distributions, estimating the parameters, and checking for goodness of fit. Distributions such as the gamma, Weibull, and lognormal should be candidates for this situation based on the histogram. We will perform the analysis for the gamma distribution 'by hand' so that you can develop an understanding of the process. Then, the *fitdistrplus* package will be illustrated.

Here is what we are going to do:

1. Perform a chi-squared goodness of fit test.
2. Perform a K-S goodness of fit test.
3. Examine the P-P and Q-Q plots.

Recall that the Chi-Squared Test divides the range of the data, $(x_1, x_2, \ldots, x_n)$, into, k, intervals and tests if the number of observations that fall in each interval is close the expected number that should fall in the interval given the hypothesized distribution is the correct model.

Since a histogram tabulates the necessary counts for the intervals it is useful to begin the analysis by developing a histogram. Let

$b_0, b_1, \cdots, b_k$ be the breakpoints (end points) of the class intervals such that $(b_0, b_1], (b_1, b_2], \cdots, (b_{k-1}, b_k]$ form k disjoint and adjacent intervals. The intervals do not have to be of equal width. Also, b_0 can be equal to $-\infty$ resulting in interval $(-\infty, b_1]$ and b_k can be equal to $+\infty$ resulting in interval $(b_{k-1}, +\infty)$. Define $\Delta b_j = b_j - b_{j-1}$ and if all the intervals have the same width (except perhaps for the end intervals), $\Delta b = \Delta b_j$.

To count the number of observations that fall in each interval, define the following function:

$$c(\vec{x} \leq b) = \#\{x_i \leq b\}\ i = 1, \ldots, n \tag{B.8}$$

$c(\vec{x} \leq b)$ counts the number of observations less than or equal to x. Let c_j be the observed count of the x values contained in the j^{th} interval $\left(b_{j-1}, b_j\right]$. Then, we can determine c_j via the following equation:

$$c_j = c(\vec{x} \leq b_j) - c(\vec{x} \leq b_{j-1}) \tag{B.9}$$

Define $h_j = c_j/n$ as the relative frequency for the j^{th} interval. Note that $\sum_{j=1}^{k} h_j = 1$. A plot of the cumulative relative frequency, $\sum_{i=1}^{j} h_i$, for each j is called a cumulative distribution plot. A plot of h_j should resemble the true probability distribution in shape because according to the mean value theorem of calculus.

$$p_j = P\{b_{j-1} \leq X \leq b_j\} = \int_{b_{j-1}}^{b_j} f(x)\mathrm{d}x = \Delta b \times f(y) \text{ for } y \in \left(b_{j-1}, b_j\right)$$

Therefore, since h_j is an estimate for p_j, the shape of the distribution should be proportional to the relative frequency, i.e. $h_j \approx \Delta b \times f(y)$.

The number of intervals is a key decision parameter and will affect the visual quality of the histogram and ultimately the chi-squared test statistic calculations that are based on the tabulated counts from the histogram. In general, the visual display of the histogram is highly dependent upon the number of class intervals. If the widths of the intervals are too small, the histogram will tend to have a ragged shape. If the width of the intervals are too large, the resulting histogram will be very block like. Two common rules for setting the number of interval are:

1. Square root rule, choose the number of intervals, $k = \sqrt{n}$.
2. Sturges rule, choose the number of intervals, $k = \lfloor 1 + \log_2(n) \rfloor$.

A frequency diagram in R is very simple by using the *hist()* function. The *hist()* function provides the frequency version of histogram and *hist(x, freq=F)* provides the density version of the histogram. The *hist()* function will automatically determine breakpoints using the Sturges rule as its default. You can also provide your own breakpoints in a vector. The *hist()* function will automatically compute the counts associated with the intervals.

```
# make histogram with no plot
h = hist(y, plot = FALSE)
# show the histogram object components
h
```

```
$breaks
[1]  0  5 10 15 20 25 30 35

$counts
[1]  6 34 25 16 11  5  3

$density
[1] 0.012 0.068 0.050 0.032 0.022 0.010 0.006
```

```
$mids
[1]  2.5  7.5 12.5 17.5 22.5 27.5 32.5

$xname
[1] "y"

$equidist
[1] TRUE

attr(,"class")
[1] "histogram"
```

Notice how the *hist* command returns a result object. In the example, the result object is assigned to the variable h. By printing the result object, you can see all the tabulated results. For example the variable *h$counts* shows the tabulation of the counts based on the default breakpoints.

```
h$counts
```

```
[1]  6 34 25 16 11  5  3
```

The breakpoints are given by the variable *h$breaks*. Note that by default *hist* defines the intervals as right-closed, i.e. $(b_{k-1}, b_k]$, rather than left-closed, $[b_{k-1}, b_k)$. If you want left closed intervals, set the *hist* parameter, *right = FALSE*. The relative frequencies, h_j can be computed by dividing the counts by the number of observations, i.e. *h$counts/length(y)*.

The variable *h$density* holds the relative frequencies divided by the interval length. In terms of notation, this is, $f_j = h_j/\Delta b_j$. This is referred to as the density because it estimates the height of the probability density curve.

To define your own break points, put them in a vector using the collect command (for example: $b = c(0, 4, 8, 12, 16, 20, 24, 28, 32)$) and then specify

the vector with the *breaks* option of the *hist* command if you do not want to use the default breakpoints. The following listing illustrates how to do this.

```
# set up some new break points
b = c(0,4,8,12,16,20,24,28,32)
b
```

```
[1]  0  4  8 12 16 20 24 28 32
```

```
# make histogram with no plot for new breakpoints
hb = hist(y, breaks = b, plot = FALSE)
# show the histogram object components
hb
```

```
$breaks
[1]  0  4  8 12 16 20 24 28 32

$counts
[1]  6 16 30 17 12  9  5  5

$density
[1] 0.0150 0.0400 0.0750 0.0425 0.0300 0.0225 0.0125 0.0125

$mids
[1]  2  6 10 14 18 22 26 30

$xname
[1] "y"

$equidist
[1] TRUE
```

```
attr(,"class")
[1] "histogram"
```

You can also use the *cut()* function and the *table()* command to tabulate the counts by providing a vector of breaks and tabulate the counts using the *cut()* and the *table()* commands without using the *hist* command. The following listing illustrates how to do this.

```
#define the intervals
y.cut = cut(y, breaks=b)
# tabulate the counts in the intervals
table(y.cut)
```

```
y.cut
  (0,4]   (4,8]  (8,12] (12,16] (16,20] (20,24] (24,28] (28,32]
      6      16      30      17      12       9       5       5
```

By using the *hist* function in R, we have a method for tabulating the relative frequencies. In order to apply the chi-square test, we need to be able to compute the following test statistic:

$$\chi_0^2 = \sum_{j=1}^{k} \frac{\left(c_j - np_j\right)^2}{np_j} \tag{B.10}$$

where

$$p_j = P\{b_{j-1} \leq X \leq b_j\} = \int_{b_{j-1}}^{b_j} f(x)\mathrm{d}x = F(b_j) - F(b_{j-1}) \tag{B.11}$$

Notice that p_j depends on $F(x)$, the cumulative distribution function of the hypothesized distribution. Thus, we need to hypothesize a distribution and estimate the parameters of the distribution.

For this situation, we will hypothesize that the task times come from a gamma distribution. Therefore, we need to estimate the shape (α) and the scale (β) parameters. In order to do this we can use an estimation technique such as the method of moments or the maximum likelihood method. For simplicity and illustrative purposes, we will use the method of moments to estimate the parameters.

The method of moments is a technique for constructing estimators of the parameters that is based on matching the sample moments (e.g. sample average, sample variance, etc.) with the corresponding distribution moments. This method equates sample moments to population (theoretical) ones. Recall that the mean and variance of the gamma distribution are:

$$E[X] = \alpha\beta$$
$$Var[X] = \alpha\beta^2$$

Setting $\bar{X} = E[X]$ and $S^2 = Var[X]$ and solving for α and β yields,

$$\hat{\alpha} = \frac{(\bar{X})^2}{S^2}$$
$$\hat{\beta} = \frac{S^2}{\bar{X}}$$

Using the results, $\bar{X} = 13.412$ and $S^2 = 50.44895$, yields,

$$\hat{\alpha} = \frac{(\bar{X})^2}{S^2} = \frac{(13.412)^2}{50.44895} = 3.56562$$
$$\hat{\beta} = \frac{S^2}{\bar{X}} = \frac{50.44895}{13.412} = 3.761478$$

Then, you can compute the theoretical probability of falling in your intervals. Table B.6 illustrates the computations necessary to compute the chi-squared test statistic.

Table B.6.: Chi-Squared Goodness of Fit Calculations

j	b_{j-1}	b_j	c_j	$F(b_{j-1})$	$F(b_j)$	p_j	np_j	$\frac{(c_j - np_j)^2}{np_j}$
1	0.00	5.00	6.00	0.00	0.08	0.08	7.89	0.45
2	5.00	10.00	34.00	0.08	0.36	0.29	28.54	1.05
3	10.00	15.00	25.00	0.36	0.65	0.29	28.79	0.50
4	15.00	20.00	16.00	0.65	0.84	0.18	18.42	0.32
5	20.00	25.00	11.00	0.84	0.93	0.09	9.41	0.27
6	25.00	30.00	5.00	0.93	0.97	0.04	4.20	0.15
7	30.00	35.00	3.00	0.97	0.99	0.02	1.72	0.96
8	35.00	∞	0.00	0.99	1.00	0.01	1.03	1.03

Since the 7th and 8th intervals have less than 5 expected counts, we should combine them with the 6th interval. Computing the chi-square test statistic value over the 6 intervals yields:

$$\begin{aligned}
\chi_0^2 &= \sum_{j=1}^{6} \frac{\left(c_j - np_j\right)^2}{np_j} \\
&= \frac{(6.0 - 7.89)^2}{7.89} + \frac{(34 - 28.54)^2}{28.54} + \frac{(25 - 28.79)^2}{28.79} + \frac{(16 - 18.42)^2}{218.42} \\
&+ \frac{(11 - 9.41)^2}{9.41} + \frac{(8 - 6.95)^2}{6.95} \\
&= 2.74
\end{aligned}$$

Since two parameters of the gamma were estimated from the data, the degrees of freedom for the chi-square test is 3 (#intervals - #parameters

- 1 = 6-2-1). Computing the p-value yields $P\{\chi_3^2 > 2.74\} = 0.433$. Thus, given such a high p-value, we would not reject the hypothesis that the observed data is gamma distributed with $\alpha = 3.56562$ and $\beta = 3.761478$.

The following listing provides a script that will compute the chi-square test statistic and its p-value within R.

```
a = mean(y)*mean(y)/var(y) #estimate alpha
b = var(y)/mean(y) #estmate beta
hy = hist(y, plot=FALSE) # make histogram
LL = hy$breaks # set lower limit of intervals
UL = c(LL[-1],10000) # set upper limit of intervals
FLL = pgamma(LL,shape = a, scale = b) #compute F(LL)
FUL = pgamma(UL,shape = a, scale = b)  #compute F(UL)
pj = FUL - FLL # compute prob of being in interval
ej = length(y)*pj # compute expected number in interval
e = c(ej[1:5],sum(ej[6:8])) #combine last 3 intervals
cnts = c(hy$counts[1:5],sum(hy$counts[6:7])) #combine last 3
 ↪  intervals
chissq = ((cnts-e)^2)/e #compute chi sq values
sumchisq = sum(chissq) # compute test statistic
df = length(e)-2-1 #compute degrees of freedom
pvalue = 1 - pchisq(sumchisq, df) #compute p-value
print(sumchisq) # print test statistic
```

```
[1] 2.742749
```

```
print(pvalue) #print p-value
```

```
[1] 0.4330114
```

Notice that we computed the same χ^2 value and p-value as when doing the calculations by hand.

B.5.4. Kolmogorov-Smirnov Test

The Kolmogorov-Smirnov (K-S) Test compares the hypothesized distribution, $\widehat{F}(x)$, to the empirical distribution and does not depend on specifying intervals for tabulating the test statistic. The K-S test compares the theoretical cumulative distribution function (CDF) to the empirical CDF by checking the largest absolute deviation between the two over the range of the random variable. The K-S Test is described in detail in (Law 2007), which also includes a discussion of the advantages/disadvantages of the test. For example, (Law 2007) indicates that the K-S Test is more powerful than the Chi-Squared test and has the ability to be used on smaller sample sizes.

To apply the K-S Test, we must be able to compute the empirical distribution function. The empirical distribution is the proportion of the observations that are less than or equal to x. Recalling Equation B.8, we can define the empirical distribution as in Equation B.12.

$$\tilde{F}_n(x) = \frac{c(\vec{x} \leq x)}{n} \tag{B.12}$$

To formalize this definition, suppose we have a sample of data, x_i for $i = 1, 2, \cdots, n$ and we then sort this data to obtain $x_{(i)}$ for $i = 1, 2, \cdots, n$, where $x_{(1)}$ is the smallest, $x_{(2)}$ is the second smallest, and so forth. Thus, $x_{(n)}$ will be the largest value. These sorted numbers are called the *order statistics* for the sample and $x_{(i)}$ is the i^{th} order statistic.

Since the empirical distribution function is characterized by the proportion of the data values that are less than or equal to the i^{th} order statistic for each $i = 1, 2, \cdots, n$, Equation B.12 can be re-written as:

$$\tilde{F}_n\left(x_{(i)}\right) = \frac{i}{n} \tag{B.13}$$

The reason that this revised definition works is because for a given $x_{(i)}$ the number of data values less than or equal to $x_{(i)}$ will be i, by definition of the order statistic. For each order statistic, the empirical distribution can be easily computed as follows:

$$\begin{aligned}
\tilde{F}_n\left(x_{(1)}\right) &= \frac{1}{n} \\
\tilde{F}_n\left(x_{(2)}\right) &= \frac{2}{n} \\
&\vdots \\
\tilde{F}_n\left(x_{(i)}\right) &= \frac{i}{n} \\
&\vdots \\
\tilde{F}_n\left(x_{(n)}\right) &= \frac{n}{n} = 1
\end{aligned}$$

A continuity correction is often used when defining the empirical distribution as follows:

$$\tilde{F}_n\left(x_{(i)}\right) = \frac{i - 0.5}{n}$$

This enhances the testing of continuous distributions. The sorting and computing of the empirical distribution is easy to accomplish in a spreadsheet program or in the statistical package R.

The K-S Test statistic, D_n is defined as $D_n = \max\{D_n^+, D_n^-\}$ where:

$$\begin{aligned}
D_n^+ &= \max_{1 \le i \le n} \left\{ \tilde{F}_n\left(x_{(i)}\right) - \hat{F}(x_{(i)}) \right\} \\
&= \max_{1 \le i \le n} \left\{ \frac{i}{n} - \hat{F}(x_{(i)}) \right\}
\end{aligned}$$

$$D_n^- = \max_{1 \le i \le n} \left\{ \hat{F}(x_{(i)}) - \tilde{F}_n\left(x_{(i-1)}\right) \right\}$$
$$= \max_{1 \le i \le n} \left\{ \hat{F}(x_{(i)}) - \frac{i-1}{n} \right\}$$

The K-S Test statistic, D_n, represents the largest vertical distance between the hypothesized distribution and the empirical distribution over the range of the distribution. Table F.5 contains critical values for the K-S test, where you reject the null hypothesis if D_n is greater than the critical value D_α, where α is the Type 1 significance level.

Intuitively, a large value for the K-S test statistic indicates a poor fit between the empirical and the hypothesized distributions. The null hypothesis is that the data comes from the hypothesized distribution. While the K-S Test can also be applied to discrete data, special tables must be used for getting the critical values. Additionally, the K-S Test in its original form assumes that the parameters of the hypothesized distribution are known, i.e. given without estimating from the data. Research on the effect of using the K-S Test with estimated parameters has indicated that it will be conservative in the sense that the actual Type I error will be less than specified.

The following R listing, illustrates how to compute the K-S statistic by hand (which is quite unnecessary) because you can simply use the *ks.test* command as illustrated.

```
j = 1:length(y) # make a vector to count y's
yj = sort(y) # sort the y's
Fj = pgamma(yj, shape = a, scale = b)  #compute F(yj)
n = length(y)
D = max(max((j/n)-Fj),max(Fj - ((j-1)/n))) # compute K-S test
 ↪  statistic
print(D)
```

```
[1] 0.05265431
```

```
ks.test(y, 'pgamma', shape=a, scale =b) # compute k-s test
```

```
    Asymptotic one-sample Kolmogorov-Smirnov test

data:  y
D = 0.052654, p-value = 0.9444
alternative hypothesis: two-sided
```

Based on the very high p-value of 0.9444, we should not reject the hypothesis that the observed data is gamma distributed with $\alpha = 3.56562$ and $\beta = 3.761478$.

We have now completed the chi-squared goodness of fit test as well as the K-S test. The Chi-Squared test has more general applicability than the K-S Test. Specifically, the Chi-Squared test applies to both continuous and discrete data; however, it suffers from depending on the interval specification. In addition, it has a number of other shortcomings which are discussed in (Law 2007). While the K-S Test can also be applied to discrete data, special tables must be used for getting the critical values. Additionally, the K-S Test in its original form assumes that the parameters of the hypothesized distribution are known, i.e. given without estimating from the data. Research on the effect of using the K-S Test with estimated parameters has indicated that it will be conservative in the sense that the actual Type I error will be less than specified. Additional advantage and disadvantage of the K-S Test are given in (Law 2007). There are other statistical tests that have been devised for testing the goodness of fit for distributions. One such test is Anderson-Darling Test. (Law 2007) describes this test. This test detects tail differences and has a higher power than the K-S Test for many popular distributions. It can be found as standard output in commercial distribution fitting software.

B.5.5. Visualizing the Fit

Another valuable diagnostic tool is to make probability-probability (P-P) plots and quantile-quantile (Q-Q) plots. A P-P Plot plots the empirical distribution function versus the theoretical distribution evaluated at each order statistic value. Recall that the empirical distribution is defined as:

$$\tilde{F}_n(x_{(i)}) = \frac{i}{n}$$

Alternative definitions are also used in many software packages to account for continuous data. As previously mentioned

$$\tilde{F}_n(x_{(i)}) = \frac{i - 0.5}{n}$$

is very common, as well as,

$$\tilde{F}_n(x_{(i)}) = \frac{i - 0.375}{n + 0.25}$$

To make a P-P Plot, perform the following steps:

1. Sort the data to obtain the order statistics: $(x_{(1)}, x_{(2)}, ... x_{(n)})$
2. Compute $\tilde{F}_n(x_{(i)}) = \frac{i - 0.5}{n} = q_i$ for i= 1, 2, ... n
3. Compute $\hat{F}(x_{(i)})$ for i= 1, 2, ... n where $\hat{F}$ is the CDF of the hypothesized distribution
4. Plot $\hat{F}(x_{(i)})$ versus $\tilde{F}_n(x_{(i)})$ for i= 1, 2, ... n

The Q-Q Plot is similar in spirit to the P-P Plot. For the Q-Q Plot, the quantiles of the empirical distribution (which are simply the order statistics) are plotted versus the quantiles from the hypothesized distribution. Let $0 \leq q \leq 1$ so that the q^{th} quantile of the distribution is denoted by x_q and is defined by:

$$q = P(X \leq x_q) = F(x_q) = \int_{-\infty}^{x_q} f(u)du$$

As shown in Figure B.14, x_q is that value on the measurement axis such that 100q% of the area under the graph of $f(x)$ lies to the left of x_q and 100(1-q)% of the area lies to the right. This is the same as the inverse cumulative distribution function.

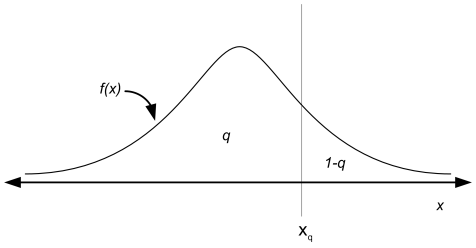


Figure B.14.: The Quantile of a Distribution

For example, the z-values for the standard normal distribution tables are the quantiles of that distribution. The quantiles of a distribution are readily available if the inverse CDF of the distribution is available. Thus, the quantile can be defined as:

$$x_q = F^{-1}(q)$$

where F^{-1} represents the inverse of the cumulative distribution function (not the reciprocal). For example, if the hypothesized distribution is N(0,1) then $1.96 = \Phi^{-1}(0.975)$ so that $x_{0.975} = 1.96$ where $\Phi(z)$ is the CDF of the

standard normal distribution. When you give a probability to the inverse of the cumulative distribution function, you get back the corresponding ordinate value that is associated with the area under the curve, e.g. the quantile.

To make a Q-Q Plot, perform the following steps:

1. Sort the data to obtain the order statistics: $(x_{(1}, x_{(2)}, ... x_{(n)})$
2. Compute $q_i = \dfrac{i - 0.5}{n}$ for i= 1, 2, ... n
3. Compute $x_{q_i} = \hat{F}^{-1}(q_i)$ for where i = 1, 2, ... n is the $\hat{F}^{-1}$ inverse CDF of the hypothesized distribution
4. Plot x_{q_i} versus $x_{(i)}$ for i = 1, 2, ... n

Thus, in order to make a P-P Plot, the CDF of the hypothesized distribution must be available and in order to make a Q-Q Plot, the inverse CDF of the hypothesized distribution must be available. When the inverse CDF is not readily available there are other methods to making Q-Q plots for many distributions. These methods are outlined in (Law 2007). The following example will illustrate how to make and interpret the P-P plot and Q-Q plot for the hypothesized gamma distribution for the task times.

The following R listing will make the P-P and Q-Q plots for this situation.

```
plot(Fj,ppoints(length(y))) # make P-P plot
abline(0,1) # add a reference line to the plot
qqplot(y, qgamma(ppoints(length(y)), shape = a, scale = b)) # make Q-
Q Plot
abline(0,1) # add a reference line to the plot
```

The function *ppoints()* in R will generate $\tilde{F}_n(x_{(i)})$. Then you can easily use the distribution function (with the "p", as in pgamma()) to compute the theoretical probabilities. In R, the quantile function can be found by

appending a "q" to the name of the available distributions. We have already seen qt() for the student-t distribution. For the normal, we use qnorm() and for the gamma, we use qgamma(). Search the R help for 'distributions' to find the other common distributions. The function *abline()* will add a reference line between 0 and 1 to the plot. Figure B.15 illustrates the P-P plot.

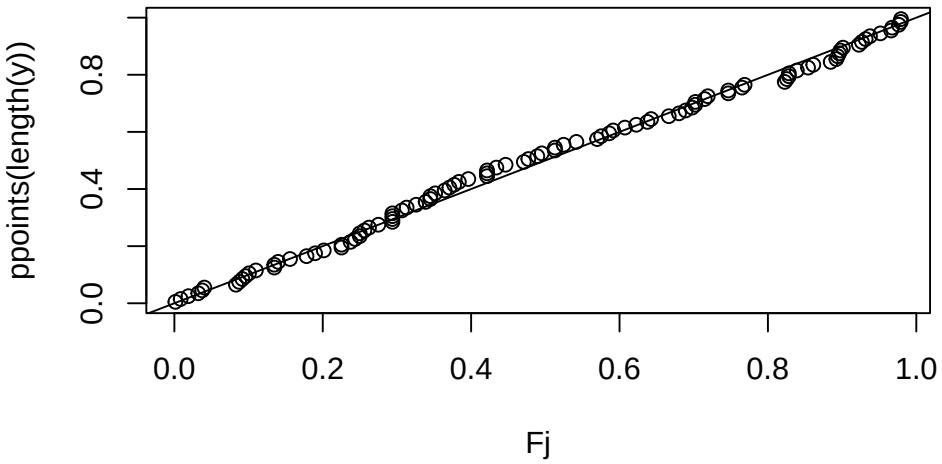


Figure B.15.: The P-P Plot for the Task Times with Gamma(shape = 3.56, scale = 3.76)

The Q-Q plot should appear approximately linear with intercept zero and slope 1, i.e. a 45 degree line, if there is a good fit to the data. In addition, curvature at the ends implies too long or too short tails, convex or concave curvature implies asymmetry, and stragglers at either ends may be outliers. The P-P Plot should also appear linear with intercept 0 and slope 1. The abline() function was used to add the reference line to the plots. Figure B.16 illustrates the Q-Q plot. As can be seen in the figures, both plots do not appear to show any significant departure from a straight line. Notice that the Q-Q plot is a little off in the right tail.

Now, that we have seen how to do the analysis 'by hand', let's see how easy

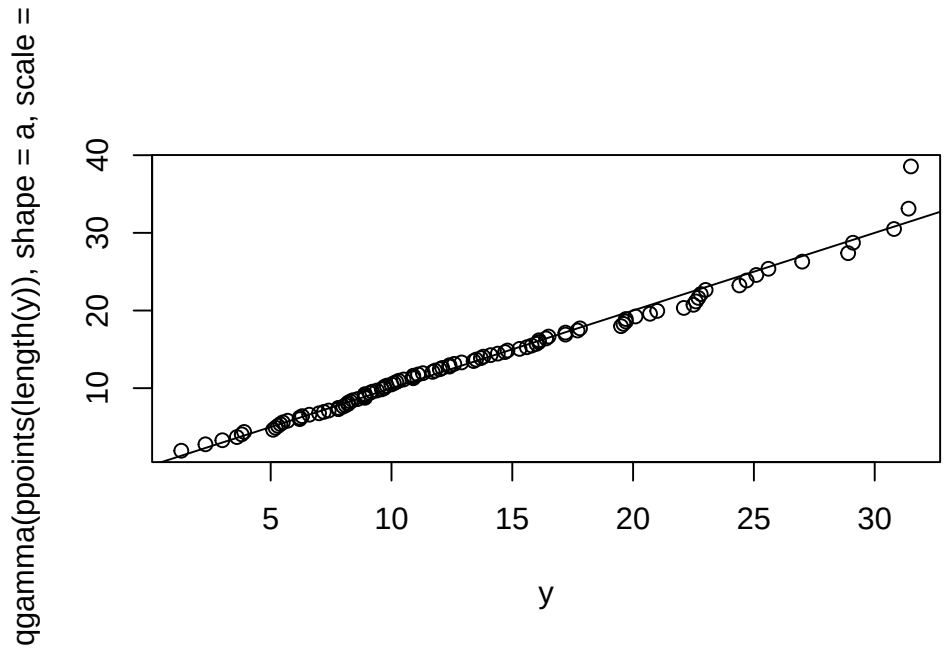


Figure B.16.: The Q-Q Plot for the Task Times with Gamma(shape = 3.56, scale = 3.76)

it can be using the *fitdistrplus* package. Notice that the *fitdist* command will fit the parameters of the distribution.

```
library(fitdistrplus)
fy = fitdist(y, "gamma")
print(fy)
```

```
Fitting of the distribution ' gamma ' by maximum likelihood
Parameters:
       estimate Std. Error
shape 3.4102879 0.46061917
rate  0.2542498 0.03699707
```

Then, the *gofstat* function does all the work to compute the chi-square goodness of fit, K-S test statistic, as well as other goodness of fit criteria. The results lead to the same conclusion that we had before: the gamma distribution is a good model for this data.

```
gfy = gofstat(fy)
print(gfy)
```

```
Goodness-of-fit statistics
                              1-mle-gamma
Kolmogorov-Smirnov statistic  0.04929185
Cramer-von Mises statistic    0.03756130
Anderson-Darling statistic    0.25487215

Goodness-of-fit criteria
                                1-mle-gamma
Akaike's Information Criterion    663.3157
Bayesian Information Criterion    668.5260
```

```
print(gfy$chisq) # chi-squared test statistic
```

```
[1] 3.544628
```

```
print(gfy$chisqpvalue) # chi-squared p-value
```

```
[1] 0.8956986
```

```
print(gfy$chisqdf)  # chi-squared degrees of freedom
```

```
[1] 8
```

Plotting the object returned from the *fitdist* command via (e.g. plot(fy)), produces a plot (Figure B.17) of the empirical and theoretical distributions, as well as the P-P and Q-Q plots.

Figure B.18 illustrates the P-P and Q-Q plots if we were to hypothesize a uniform distribution. Clearly, the plots in Figure B.18 illustrate that a uniform distribution is not a good model for the task times.

B.6. Testing Uniform (0,1) Pseudo-Random Numbers

Now that we have seen the general process for fitting continuous distributions, this section will discuss the special case of testing for uniform (0,1) random variables. The reason that this is important is because these methods serve the basis for testing if pseudo-random numbers can reasonably be expected to perform as if they are U(0,1) random variates. Thus, this section provides an overview of what is involved in testing the statistical properties of random number generators. Essentially a random number

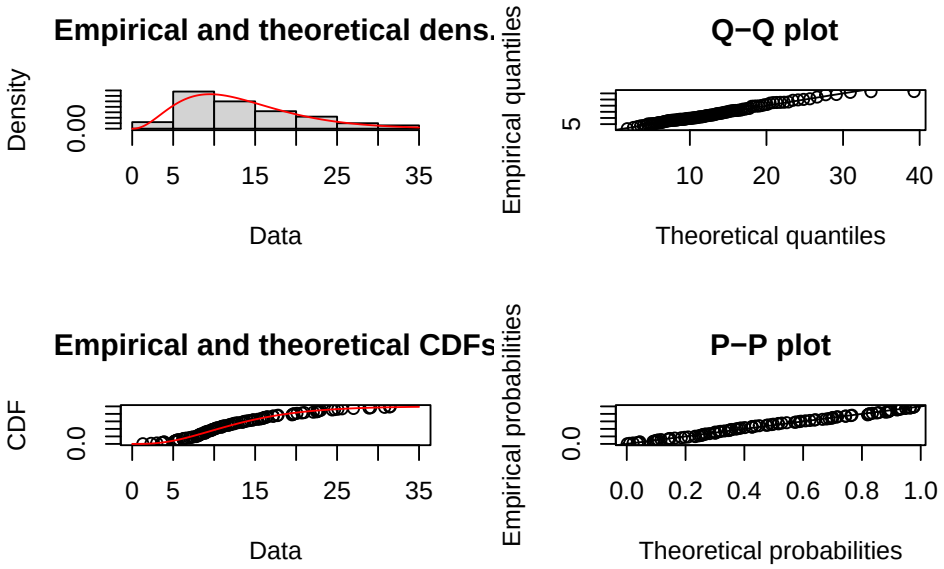


Figure B.17.: Distribution Plot from fitdistrplus for Gamma Distribution Fit of Computer Repair Times

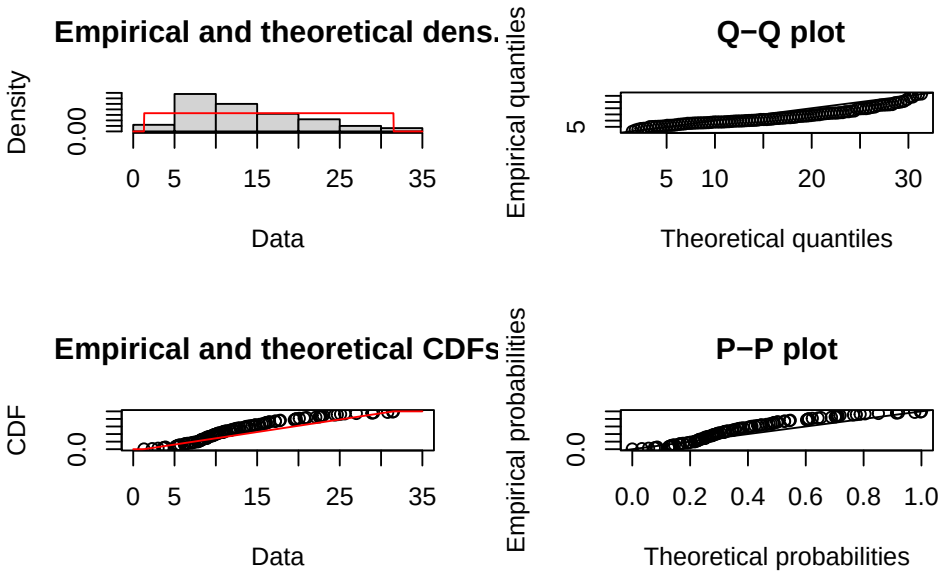


Figure B.18.: Distribution Plot from fitdistrplus for Uniform Distribution Fit of Computer Repair Times

generator is supposed to produce sequences of numbers that appear to be independent and identically distributed (IID) $U(0,1)$ random variables. The hypothesis that a sample from the generator is IID $U(0,1)$ must be made. Then, the hypothesis is subjected to various statistical tests. There are standard batteries of test, see for example (Soto 1999), that are utilized. Typical tests examine:

- Distributional properties: e.g. Chi-Squared and Kolmogorov-Smirnov test
- Independence: e.g. Correlation tests, runs tests
- Patterns: e.g. Poker test, gap test

When considering the quality of random number generators, the higher dimensional properties of the sequence also need to be considered. For example, the serial test is a higher dimensional Chi-Squared test. Just like for general continuous distributions, the two major tests that are utilized to examine the distributional properties of sequences of pseudo-random numbers are the Chi-Squared Goodness of Fit test and the Kolmogorov-Smirnov test. In the case of testing for $U(0,1)$ random variates some simplifications can be made in computing the test statistics.

B.6.1. Chi-Squared Goodness of Fit Tests for Pseudo-Random Numbers

When applying the Chi-Squared goodness of fit to test if the data is $U(0,1)$, the following is the typical procedure:

1. Divide the interval $(0,1)$ into k equally spaced classes so that $\Delta b = b_j - b_{j-1}$ resulting in $p_j = \frac{1}{k}$ for $j = 1, 2, \cdots, k$. This results in the expected number in each interval being $np_j = n \times \frac{1}{k} = \frac{n}{k}$

2. As a practical rule, the expected number in each interval np_j should be at least 5. Thus, in this case $\frac{n}{k} \geq 5$ or $n \geq 5k$ or $k \leq \frac{n}{5}$. Thus, for a given value of n, you should choose the number of intervals $k \leq \frac{n}{5}$,
3. Since the parameters of the distribution are known $a = 0$ and $b = 1$, then $s = 0$. Therefore, we reject $H_0 : U_i \sim U(0, 1)$ if $\chi_0^2 > \chi_{\alpha,k-1}^2$ or if the p-value is less than α

If p_j values are chosen as $\frac{1}{k}$, then Equation B.14 can be rewritten as:

$$\chi_0^2 = \frac{k}{n} \sum_{j=1}^{k} \left(c_j - \frac{n}{k} \right)^2 \tag{B.14}$$

Let's apply these concepts to a small example.

Example B.3 (Testing 100 Pseudo-Random Numbers)**.** Suppose we have 100 observations from a pseudo-random number generator. Perform a χ^2 test that the numbers are distributed $U(0, 1)$.

0.971	0.668	0.742	0.171	0.350	0.931	0.803	0.848	0.160	0.085
0.687	0.799	0.530	0.933	0.105	0.783	0.828	0.177	0.535	0.601
0.314	0.345	0.034	0.472	0.607	0.501	0.818	0.506	0.407	0.675
0.752	0.771	0.006	0.749	0.116	0.849	0.016	0.605	0.920	0.856
0.830	0.746	0.531	0.686	0.254	0.139	0.911	0.493	0.684	0.938
0.040	0.798	0.845	0.461	0.385	0.099	0.724	0.636	0.846	0.897
0.468	0.339	0.079	0.902	0.866	0.054	0.265	0.586	0.638	0.869
0.951	0.842	0.241	0.251	0.548	0.952	0.017	0.544	0.316	0.710
0.074	0.730	0.285	0.940	0.214	0.679	0.087	0.700	0.332	0.610
0.061	0.164	0.775	0.015	0.224	0.474	0.521	0.777	0.764	0.144

Since we have $n = 100$ observations, the number of intervals should be less than or equal to 20. Let's choose $k = 10$. This means that $p_j = 0.1$ for all j. The following table summarizes the computations for each interval for computing the chi-squared test statistic.

j	b_{j-1}	b_j	p_j	$c(\vec{x} \leq b_{j-1})$	$c(\vec{x} \leq b_j)$	c_j	np_j	$\frac{(c_j - np_j)^2}{np_j}$
1	0	0.1	0.1	0	13	13	10	0.9
2	0.1	0.2	0.1	13	21	8	10	0.4
3	0.2	0.3	0.1	21	28	7	10	0.9
4	0.3	0.4	0.1	28	35	7	10	0.9
5	0.4	0.5	0.1	35	41	6	10	1.6
6	0.5	0.6	0.1	41	50	9	10	0.1
7	0.6	0.7	0.1	50	63	13	10	0.9
8	0.7	0.8	0.1	63	77	14	10	1.6
9	0.8	0.9	0.1	77	90	13	10	0.9
10	0.9	1	0.1	90	100	10	10	0

Summing the last column yields:

$$\chi_0^2 = \sum_{j=1}^{k} \frac{\left(c_j - np_j\right)^2}{np_j} = 8.2$$

Computing the p-value for $k - s - 1 = 10 - 0 - 1 = 9$ degrees of freedom, yields $P\{\chi_9^2 > 8.2\} = 0.514$. Thus, given such a high p-value, we would not reject the hypothesis that the observed data is $U(0, 1)$. This process can be readily implemented within a spreadsheet or performed using R. Assuming that the file, u01data.txt, contains the PRNs for this example,

then the following R commands will perform the test:

```
data = scan(file="data/AppDistFitting/u01data.txt") # read in
 ↪  the file
b = seq(0,1, by = 0.1) # set up the break points
h = hist(data, b, right = FALSE, plot = FALSE) # tabulate the
 ↪  counts
chisq.test(h$counts) # perform the test
```

```
    Chi-squared test for given probabilities

data:  h$counts
X-squared = 8.2, df = 9, p-value = 0.5141
```

Because we are assuming that the data is $U(0,1)$, the chisq.test() function within R is simplified because its default is to assume that all data is equally likely. Notice that we get exactly the same result as we computed when doing the calculations manually.

B.6.2. Higher Dimensional Chi-Squared Test

The pseudo-random numbers should not only be uniformly distributed on the interval $(0,1)$ they should also be uniformly distributed within within the unit square, $\{(x,y) : x \in (0,1), y \in (0,1)\}$, the unit cube, and so forth for higher number of dimensions d.

The serial test, described in (Law 2007) can be used to assess the quality of the higher dimensional properties of pseudo-random numbers. Suppose the sequence of pseudo-random numbers can be formed into non-overlapping vectors each of size d. The vectors should be independent and identically

distributed random vectors uniformly distributed on the d-dimensional unit hyper-cube. This motivates the development of the serial test for uniformity in higher dimensions. To perform the serial test:

1. Divide (0,1) into k sub-intervals of equal size.
2. Generate n vectors of pseudo-random numbers each of size d,

$$\begin{aligned}\vec{U}_1 &= (U_1, U_2, \cdots, U_d)\\ \vec{U}_2 &= (U_{d+1}, U_{d+2}, \cdots, U_{2d})\\ &\vdots\\ \vec{U}_n &= (U_{(n-1)d+1}, U_{(n-1)d+2}, \cdots, U_{nd})\end{aligned}$$

3. Let $c_{j_1,j_2,\cdots,j_d}$ be the count of the number of $\vec{U}_i$'s having the first component in subinterval j_1, second component in subinterval j_2 etc.
4. Compute

$$\chi_0^2(d) = \frac{k^d}{n}\sum_{j_1=1}^{k}\sum_{j_2=1}^{k}\cdots\sum_{j_d=1}^{k}\left(c_{j_1,j_2,\cdots,j_d} - \frac{n}{k^d}\right)^2 \tag{B.15}$$

5. Reject the hypothesis that the $\vec{U}_i$'s are uniformly distributed in the d-dimensional unit hyper-cube if $\chi_0^2(d) > \chi_{\alpha,k^d-1}^2$ or if the p-value $P\{\chi_{k^d-1}^2 > \chi_0^2(d)\}$ is less than α.

(Law 2007) provides an algorithm for computing $c_{j_1,j_2,\cdots,j_d}$. This test examines how uniformly the random numbers fill-up the multi-dimensional space. This is a very important property when applying simulation to the evaluation of multi-dimensional integrals as is often found in the physical sciences.

Example B.4 (2-D Chi-Squared Test in R for U(0,1))**.** Using the same data as in the previous example, perform a 2-Dimensional χ^2 Test using the statistical package R. Use 4 intervals for each of the dimensions.

The following code listing is liberally commented for understanding the commands and essentially utilizes some of the classification and tabulation functionality in R to compute Equation B.15. The code displayed here is available in the files associated with the chapter in file, 2dchisq.R.

```
nd = 100 #number of data points
data <- read.table("u01data.txt") # read in the data
d = 2 # dimensions to test
n = nd/d # number of vectors
m = t(matrix(data$V1,nrow=d)) # convert to matrix and transpose
b = seq(0,1, by = 0.25) # setup the cut points
xg = cut(m[,1],b,right=FALSE) # classify the x dimension
yg = cut(m[,2],b,right=FALSE) # classify the y dimension
xy = table(xg,yg) # tabulate the classifications
k = length(b) - 1 # the number of intervals
en = n/(k^d) # the expected number in an interval
vxy = c(xy) # convert matrix to vector for easier summing
vxymen = vxy-en # substract expected number from each element
vxymen2 = vxymen*vxymen # square each element
schi = sum(vxymen2) # compute sum of squares
chi = schi/en # compute the chi-square test statistic
dof = (k^d) - 1 # compute the degrees of freedom
pv = pchisq(chi,dof, lower.tail=FALSE) # compute the p-value
# print out the results
cat("#observations = ", nd,"\n")
cat("#vectors = ", n, "\n")
cat("size of vectors, d = ", d, "\n")
```

```
cat("#intervals =", k, "\n")
cat("cut points = ", b, "\n")
cat("expected # in each interval, n/k^d = ", en, "\n")
cat("interval tabulation = \n")
print(xy)
cat("\n")
cat("chisq value =", chi,"\n")
cat("dof =", dof,"\n")
cat("p-value = ",pv,"\n")
```

The results shown in the following listing are for demonstration purposes only since the expected number in the intervals is less than 5. However, you can see from the interval tabulation, that the counts for the 2-D intervals are close to the expected. The p-value suggests that we would not reject the hypothesis that there is non-uniformity in the pairs of the psuedo-random numbers. The R code can be generalized for a larger sample or for performing a higher dimensional test. The reader is encouraged to try to run the code for a larger data set.

```
Output:
#observations =  100
#vectors =  50
size of vectors, d =  2
#intervals = 4
cut points =  0 0.25 0.5 0.75 1
expected # in each interval, n/k^d =  3.125
interval tabulation =
            yg
xg           [0,0.25) [0.25,0.5) [0.5,0.75) [0.75,1)
  [0,0.25)          5          0          3        2
  [0.25,0.5)        2          1          2        7
  [0.5,0.75)        3          3          3        7
  [0.75,1)          4          1          4        3
```

```
chisq value = 18.48
dof = 15
p-value =  0.2382715
```

B.6.3. Kolmogorov-Smirnov Test for Pseudo-Random Numbers

When applying the K-S test to testing pseudo-random numbers, the hypothesized distribution is the uniform distribution on 0 to 1. The CDF for a uniform distribution on the interval (a, b) is given by:

$$F(x) = P\{X \leq x\} = \frac{x-a}{b-a} \text{ for } a < x < b$$

Thus, for $a = 0$ and $b = 1$, we have that $F(x) = x$ for the $U(0, 1)$ distribution. This simplifies the calculation of D_n^+ and D_n^- to the following:

$$\begin{aligned} D_n^+ &= \max_{1 \leq i \leq n} \left\{ \frac{i}{n} - \hat{F}(x_{(i)}) \right\} \\ &= \max_{1 \leq i \leq n} \left\{ \frac{i}{n} - x_{(i)} \right\} \end{aligned}$$

$$\begin{aligned} D_n^- &= \max_{1 \leq i \leq n} \left\{ \hat{F}(x_{(i)}) - \frac{i-1}{n} \right\} \\ &= \max_{1 \leq i \leq n} \left\{ x_{(i)} - \frac{i-1}{n} \right\} \end{aligned}$$

Example B.5 (K-S Test for U(0,1))**.** Given the data from Example B.3 test the hypothesis that the data appears $U(0, 1)$ versus that it is not $U(0, 1)$ using the Kolmogorov-Smirnov goodness of fit test at the $\alpha = 0.05$ significance level.

A good way to organize the computations is in a tabular form, which also facilitates the use of a spreadsheet. The second column is constructed by sorting the data. Recall that because we are testing the $U(0,1)$ distribution, $F(x) = x$, and thus the third column is simply $F(x_{(i)}) = x_{(i)}$. The rest of the columns follow accordingly.

i	$x_{(i)}$	i/n	$\frac{i-1}{n}$	$F(x_{(i)})$	$\frac{i}{n} - F(x_{(i)})$	$F(x_{(i)}) - \frac{i-1}{n}$
1	0.006	0.010	0.000	0.006	0.004	0.006
2	0.015	0.020	0.010	0.015	0.005	0.005
3	0.016	0.030	0.020	0.016	0.014	-0.004
4	0.017	0.040	0.030	0.017	0.023	-0.013
5	0.034	0.050	0.040	0.034	0.016	-0.006
95	0.933	0.950	0.940	0.933	0.017	-0.007
96	0.938	0.960	0.950	0.938	0.022	-0.012
97	0.940	0.970	0.960	0.940	0.030	-0.020
98	0.951	0.980	0.970	0.951	0.029	-0.019
99	0.952	0.990	0.980	0.952	0.038	-0.028
100	0.971	1.000	0.990	0.971	0.029	-0.019

Computing D_n^+ and D_n^- yields

$$\begin{aligned} D_n^+ &= \max_{1 \le i \le n} \left\{ \frac{i}{n} - x_{(i)} \right\} \\ &= 0.038 \\ D_n^- &= \max_{1 \le i \le n} \left\{ x_{(i)} - \frac{i-1}{n} \right\} \\ &= 0.108 \end{aligned}$$

Thus, we have that

$$D_n = \max\{D_n^+, D_n^-\} = \max\{0.038, 0.108\} = 0.108$$

Referring to Table F.5 and using the approximation for sample sizes greater than 35, we have that $D_{0.05} \approx 1.36/\sqrt{n}$. Thus, $D_{0.05} \approx 1.36/\sqrt{100} = 0.136$. Since $D_n < D_{0.05}$, we would not reject the hypothesis that the data is uniformly distribution over the range from 0 to 1.

The K-S test performed in the solution to Example B.5 can also be readily performed using the statistical software *R*. Assuming that the file, *u01data.txt*, contains the data for this example, then the following R commands will perform the test:

```
data = scan(file="data/AppDistFitting/u01data.txt") # read in
 ↪  the file
ks.test(data,"punif",0,1) # perform the test
```

```
    Asymptotic one-sample Kolmogorov-Smirnov test

data:  data
D = 0.10809, p-value = 0.1932
alternative hypothesis: two-sided
```

Since the p-value for the test is greater than $\alpha = 0.05$, we would not reject the hypothesis that the data is $U(0, 1)$.

B.6.4. Testing for Independence and Patterns in Pseudo-Random Numbers

A full treatment for testing for independence and patterns within sequences of pseudo-random numbers is beyond the scope of this text. However, we will illustrate some of the basic concepts in this section.

As previously noted an analysis of the autocorrelation structure associated with the sequence of pseudo-random numbers forms a basis for testing dependence. A sample autocorrelation plot can be easily developed once the autocorrelation values have been estimated Generally, the maximum lag should be set to no larger than one tenth of the size of the data set because the estimation of higher lags is generally unreliable.

An autocorrelation plot can be easily performed using the statistical software R for the data in Example B.3. Using the *acf()$ function in R makes it easy to get estimates of the autocorrelation estimates. The resulting plot is shown in Figure B.19. The r_0 value represents the estimated variance. As can be seen in the figure, the acf() function of R automatically places the confidence band within the plot. In this instance, since none of the $r_k, k \geq 1$ are outside the confidence band, we can conclude that the data is likely independent observations.

```
rho
```

```
Autocorrelations of series 'data', by lag

     0      1      2      3      4      5      6      7      8      9
 1.000  0.015 -0.068  0.008  0.179 -0.197 -0.187  0.066 -
0.095 -0.120
```

The autocorrelation test examines serial dependence; however, sequences that do not have other kinds of patterns are also desired. For example, the

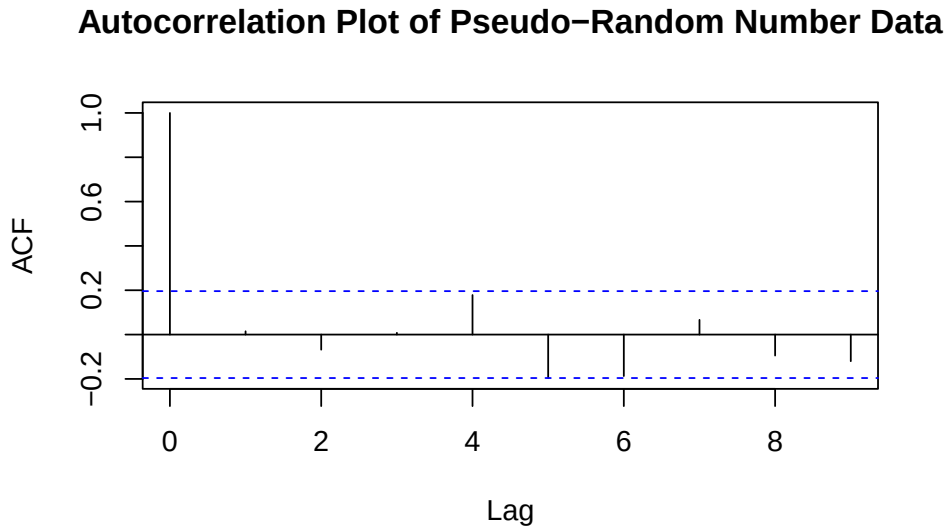


Figure B.19.: Autocorrelation Plot for u01data.txt Data Set

runs test attempts to test 'upward' or 'downward' patterns. The runs up and down test count the number of runs up, down or sometimes just total number of runs versus expected number of runs. A run is a succession of similar events preceded and followed by a different event. The length of a run is the number of events in the run. The Table B.7 illustrates how to compute runs up and runs down for a simple sequence of numbers. In the table, a sequence of '-' indicates a run down and a sequence of '+' indicates a run up. In the table, there are a 8 runs (4 runs up and 4 runs down).

Table B.7.: Example Runs Up and Runs Down

0.90	0.13	0.27	0.41	0.71	0.28	0.18	0.22	0.26	0.19	0.61	0.87	0.95	0.21	0.79
	-	+	+	+	-	-	-	+	-	+	+	+	-	+

Digit patterns can also examined. The gap test counts the number of

digits that appear between repetitions of a particular digit and uses a K-S test statistic to compare with the expected number of gaps. The poker test examines for independence based on the frequency with which certain digits are repeated in a series of numbers, (e.g. pairs, three of a kind, etc.). Banks et al. (2005) discusses how to perform these tests.

Does all this testing really matter? Yes! You should always know what pseudo-random number generator you are using within your simulation models and you should always know if the generator has passed a battery of statistical tests. The development and testing of random number generators is serious business. You should stick with well researched generators and reliable well tested software.

B.7. Additional Distribution Modeling Concepts

This section wraps up the discussion of input modeling by covering some additional topics that often come up during the modeling process.

Most of the examples in this appendix model a continuous random variable. But what do you do if you are modeling a discrete random variable? The steps in the modeling process are essentially the same. Commercial software will have options for fitting some of the common discrete distributions. The fitting process for a discrete distribution is simplified in one way because the bins for the frequency diagram are naturally determined by the range of the random variable. For example, if you are fitting a geometric distribution, you need only tabulate the frequency of occurrence for each of the possible values of the random variable 1, 2, 3, 4, etc.. Occasionally, you may have to group bins together to get an appropriate number of observations per bin. The fitting process for the discrete case primarily centers on a straightforward application of the Chi-Squared goodness of fit test, which was outlined in this chapter, and is also covered in many introductory probability and statistics textbooks.

If you consider the data set as a finite set of values, then why can't you just reuse the data? In other words, why should you go through all the trouble of fitting a theoretical distribution when you can simply reuse the observed data, say for example by reading in the data from a file. There are a number of problems with this approach. The first difficulty is that the observations in the data set are a *sample*. That is, they do not necessarily cover all the possible values associated with the random variable. For example, suppose that only a sample of 10 observations was available for the service time problem. Furthermore, assume that the sample was as follows:

1	2	3	4	5	6	7	8	9	10
36.84	38.5	46.23	46.23	46.23	48.3	52.2	53.46	53.46	56.11

Clearly, this sample does not contain the high service times that were in the 100 sample case. The point is, if you resample from only these values, you will never get any values less than 36.84 or bigger than 56.11. The second difficulty is that it will be more difficult to experimentally vary this distribution within the simulation model. If you fit a theoretical distribution to the data, you can vary the parameters of the theoretical distribution with relative ease in any experiments that you perform. Thus, it is worthwhile to attempt to fit and use a reasonable input distribution.

But what if you cannot find a reasonable input model either because you have very limited data or because no model fits the data very well? In this situation, it is useful to try to use the data in the form of the empirical distribution. Essentially, you treat each observation in the sample as equally likely and randomly draw observations from the sample. In many situations, there are repeated observations within the sample (as above) and you can form a discrete empirical distribution over the values. If this is done for the sample of 10 data points, a discrete empirical distribution can be formed as shown in Table B.8. Again, this limits us to only the values observed in the sample.

Table B.8.: Simple Empirical Distribution

X	PMF	CDF
36.84	0.1	0.1
38.5	0.1	0.2
46.23	0.3	0.5
48.3	0.1	0.6
53.46	0.2	0.8
55.33	0.1	0.9
56.11	0.1	1

One can also use the continuous empirical distribution, which interpolates between the distribution values.

What do you do if the analysis indicates that the data is dependent or that the data is non-stationary? Either of these situations can invalidate the basic assumptions behind the standard distribution fitting process. First, suppose that the data shows some correlation. The first thing that you should do is to verify that the data was correctly collected. The sampling plan for the data collection effort should attempt to ensure the collection of a random sample. If the data was from an automatic collection procedure then it is quite likely that there may be correlation in the observations. This is one of the hazards of using automatic data collection mechanisms. You then need to decide whether modeling the correlation is important or not to the study at hand. Thus, one alternative is to simply ignore the correlation and to continue with the model fitting process. This can be problematic for two reasons. First, the statistical tests within the model fitting process will be suspect, and second, the correlation may be an important part of the input modeling. For example, it has been shown that correlated arrivals and correlated service times in a simple queueing model can have significant effects on the values of the queue's performance measures. If you have a large enough data set, a basic approach is to form a random sample from the data set itself in order to break up the correlation.

Then, you can proceed with fitting a distribution to the random sample; however, you should still model the dependence by trying to incorporate it into the random generation process. There are some techniques for incorporating correlation into the random variable generation process. An introduction to the topic is provided in (Banks et al. 2005).

If the data show non-stationary behavior, then you can attempt to model the dependence on time using time series models or other non-stationary models. Suffice to say, that these advanced techniques are beyond the scope of this text; however, the next section will discuss the modeling of a special non-stationary model, the non-homogeneous Poisson process, which is very useful for modeling time dependent arrival processes. For additional information on these methods, the interested reader is referred to (Law 2007) and (Leemis and Park 2006) or the references therein.

Finally, all of the above assumes that you have data from which you can perform an analysis. In many situations, you might have no data whatsoever either because it is too costly to collect or because the system that you are modeling does not exist. In the latter case, you can look at similar systems and see how their inputs were modeled, perhaps adopting some of those input models for the current situation. In either case, you might also rely on expert opinion. In this situation, you can ask an expert in the process to describe the characteristics of a probability distribution that might model the situation. This is where the uniform and the triangular distributions can be very useful, since it is relatively easy to get an expert to indicate a minimum possible value, a maximum possible value, and even a most likely value.

Alternatively, you can ask the expert to assist in making an empirical distribution based on providing the chance that the random variable falls within various intervals. The breakpoints near the extremes are especially important to get. Table B.9 presents a distribution for the service times based on this method.

Table B.9.: Breakpoint Based Empirical Distribution

X	PMF	CDF
(36, 100]	0.1	0.1
(100, 200]	0.1	0.2
(200, 400]	0.4	0.6
(400, 600]	0.2	0.8
(600, ∞)	0.2	1.0

Whether you have lots of data, little data, or no data, the key final step in the input modeling process is *sensitivity analysis*. Your ultimate goal is to use the input models to drive your larger simulation model of the system under study. You can spend significant time and energy collecting and analyzing data for an input model that has no significant effect on the output measures of interest to your study. You should start out with simple input models and incorporate them into your simulation model. Then, you can vary the parameters and characteristics of those models in an experimental design to assess how sensitive your output is to the changes in the inputs. If you find that the output is very sensitive to particular input models, then you can plan, collect, and develop better models for those situations. The amount of sensitivity is entirely modeler dependent. Remember that in this whole process, you are in charge, not the software. The software is only there to support your decision making process. Use the software to justify your art.

B.8. Summary

In this chapter you learned how to analyze data in order to model the input distributions for a simulation model. The input distributions drive the stochastic behavior of the simulation model.

The modeling of the input distributions requires:

- Understanding how the system being modeled works. This understanding improves overall model construction in an iterative fashion: model the system, observe some data, model the data, model the system, etc.
- Carefully collecting the data using well thought out collection and sampling plans
- Analyzing the data using appropriate statistical techniques
- Hypothesizing and testing appropriate probability distributions
- Incorporating the models in to your simulations

Properly modeling the inputs to the simulation model form a critical foundation to increasing the validity of the simulation model and subsequently, the credibility of the simulation outputs.

B.9. Exercises

The files referenced in the exercises are available in the files associated with this chapter.

Exercise B.1. The observations available in the text file, *problem1.txt*, represent the count of the number of failures on a windmill turbine farm per year. Using the techniques discussed in the chapter recommend an input distribution model for this situation.

Exercise B.2. The observations available in the text file, *problem2.txt*, represent the time that it takes to repair a windmill turbine on each occurrence in minutes. Using the techniques discussed in the chapter recommend an input distribution model for this situation.

Exercise B.3. The observations available in the text file, *problem3.txt*, represent the time in minutes that it takes a repair person to drive to the windmill farm to repair a failed turbine. Using the techniques discussed in the chapter recommend an input distribution model for this situation.

Exercise B.4. The observations available in the text file, *problem4.txt*, represent the time in seconds that it takes to service a customer at a movie theater counter. Using the techniques discussed in the chapter recommend an input distribution model for this situation.

Exercise B.5. The observations available in the text file, *problem5.txt*, represent the time in hours between failures of a critical piece of computer testing equipment. Using the techniques discussed in the chapter recommend an input distribution model for this situation.

Exercise B.6. The observations available in the text file, *problem6.txt*, represent the time in minutes associated with performing a lube, oil and maintenance check at the local Quick Oil Change Shop. Using the techniques discussed in the chapter recommend an input distribution model for this situation.

Exercise B.7. If $Z \sim N(0,1)$, and $Y = \sum_{i=1}^{k} Z_i^2$ then $Y \sim \chi_k^2$, where χ_k^2 is a chi-squared random variable with k degrees of freedom. Set up a model to generate $n = 32$, 64, 128, 256, 1024 observations of Y with $k = 5$. For each sample, fit a distribution to the sample.

Exercise B.8. Consider the following sample (also found in *problem8.txt*) that represents the time (in seconds) that it takes a hematology cell counter to complete a test on a blood sample.

23.79	75.51	29.89	2.47	32.37
29.72	84.69	45.66	61.46	67.23
94.96	22.68	86.99	90.84	56.49
30.45	69.64	17.09	33.87	98.04
12.46	8.42	65.57	96.72	33.56
35.25	80.75	94.62	95.83	38.07
14.89	54.80	95.37	93.76	83.64
50.95	40.47	90.58	37.95	62.42
51.95	65.45	11.17	32.58	85.89
65.36	34.27	66.53	78.64	58.24

a. Test the hypothesis that this data is drawn from a uniform distribution at a 95% confidence level assuming that the interval is between 0 and 100.

b. The interval of the distribution is between a and b, where a and b are unknown parameters estimated from the data.

c. Consider the following output for fitting a uniform distribution to a data set. Would you reject or not reject the hypothesis that the data is uniformly distributed.

```
Distribution Summary
Distribution:   Uniform(36, 783)

Chi Square Test
  Number of intervals   = 7
  Degrees of freedom    = 6
  Test Statistic        = 164
  Corresponding p-value < 0.005

Kolmogorov-Smirnov Test
  Test Statistic    = 0.495
  Corresponding p-value < 0.01

Data Summary
Number of Data Points   = 100
Min Data Value          = 36.8
Max Data Value          = 782
Sample Mean             = 183
Sample Std Dev          = 142

Histogram Summary
Histogram Range     = 36 to 783
Number of Intervals = 10
```

Exercise B.9. Consider the following frequency data on the number of orders received per day by a warehouse.

j	x_j	c_j	np_j	$\frac{(c_j - np_j)^2}{np_j}$
1	0	10		
2	1	42		

j	x_j	c_j	np_j	$\frac{(c_j - np_j)^2}{np_j}$
3	2	27		
4	3	12		
5	4	6		
6	5 or more	3		
	Totals	100		

a. Compute the sample mean for this data.
b. Perform a χ^2 goodness of fit test to test the hypothesis (use a 95% confidence level) that the data is Poisson distributed. Complete the provided table to show your calculations.

Exercise B.10. The number of electrical outlets in a prefabricated house varies between 10 and 22 outlets. Since the time to perform the install depends on the number of outlets, data was collected to develop a probability distribution for this variable. The data set is given below and also found in file *problem10.txt*:

14	16	14	16	12	14	12	13	12	12
12	12	14	22	13	16	15	15	15	21
18	12	17	14	13	11	12	14	10	10
16	12	13	11	13	11	11	12	13	16
15	12	11	14	11	12	11	11	13	17

Fit a probability model to the number of electrical outlets per prefabricated house.

Exercise B.11. Test the following fact by generating instances of $Y = \sum_{i=1}^{r} X_i$, where $X_i \sim expo(\beta)$ and $\beta = E[X_i]$. Using $r = 5$ and $\beta = 2$. Be careful to specify the correct parameters for the exponential distribution function. The exponential distribution takes in the mean of the distribution as its parameter. Generate 10, 100, 1000 instances of Y. Perform hypothesis tests to check $H_0 : \mu = \mu_0$ versus $H_1 : \mu \neq \mu_0$ where μ is the true mean of Y for each of the sample sizes generated. Use the same samples to fit a distribution using the `PDFModeler` class. Properly interpret the statistical results supplied by the `PDFModeler`.

Exercise B.12. Suppose that we are interested in modeling the arrivals to Sly's BBQ Food Truck during 11:30 am to 1:30 pm in the downtown square. During this time a team of undergraduate students has collected the number of customers arriving to the truck for 10 different periods of the 2 hour time frame for each day of the week. The data is given below and also found in file *problem12.csv*

Obs#	M	T	W	R	F	S	SU
1	13	4	4	3	8	8	9
2	6	5	7	7	5	6	8
3	7	14	10	5	5	5	10
4	12	6	10	5	12	7	4
5	6	8	8	5	4	11	9
6	10	6	9	3	3	6	4
7	9	5	5	5	7	5	4
8	7	10	11	9	7	10	13
9	8	4	2	7	6	5	7
10	9	2	6	8	7	4	9

1. Visualize the data.

2. Check if the day of the week influences the statistical properties of the count data.
3. Tabulate the frequency of the count data.
4. Estimate the mean rate parameter of the hypothesized Poisson distribution.
5. Perform goodness of fit tests to test the hypothesis that the number of arrivals for the interval 11:30 am to 1:30 pm has a Poisson distribution versus the alternative that it does not have a Poisson distribution.

Exercise B.13. A copy center has one fast copier and one slow copier. The copy time per page for the fast copier is thought to be lognormally distributed with a mean of 1.6 seconds and a standard deviation of 0.3 seconds. A co-op Industrial Engineering student has collected some time study data on the time to copy a page for the slow copier. The times, in seconds, are given in the data file *problem13.txt*. Test whether the time to copy a page on the slow copier can be modeled with a lognormal distribution.

C. Queueing Theory

Learning Objectives

- To be able to interpret basic queueing theory notation
- To be able to compute queueing results for single queue situations
- To be able to identify and apply standard queueing models

Many real-life situations involve the possible waiting of entities (e.g. customers, parts, etc.) for resources (e.g. bank tellers, machines, etc.). Systems that involve waiting lines are called queueing systems. This appendix introduces analytical (formula-based) approaches to modeling the performance of these systems.

Once the performance of the system is modeled, the design of the system to meet operational requirements becomes an important issue. For example, in the simple situation of modeling a drive through pharmacy, you might want to determine the number of waiting spaces that should be available so that arriving customers can have a high chance of entering the line. In these situations, having more of the resource (pharmacist) available at any time will assist in meeting the design criteria; however, an increase in a resource typically comes at some cost. Thus, design questions within queueing systems involve a fundamental trade-off between customer service and the cost of providing that service.

To begin the analysis of these systems, a brief analytical treatment of the key modeling issues is presented. Analytical results are available only for simplified situations; however, the analytical treatment will serve two

purposes. First, it will provide an introduction to the key modeling issues, and second, it can provide approximate models for more complicated situations. The purpose of this appendix is not to provide a comprehensive treatment of queueing theory for which there are many excellent books already. The purpose of this appendix is to provide an introduction to this topic for persons new to simulation so that they can better understand their modeling and analysis of queueing systems. This appendix also serves as a reference for the formulas associated with these models to facilitate their use in verifying and validating simulation models.

C.1. Single Line Queueing Stations

Chapter 4 presented the pharmacy model and analyzed it with a single server, single queue queueing system called the M/M/1. This section shows how the formulas for the M/M/1 model are derived and discusses the key notation and assumptions of analytical models for systems with a single queue. In addition, you will also learn how to simulate variations of these models.

In a queueing system, there are customers that compete for resources by moving through processes. The competition for resources causes waiting lines (queues) to form and delays to occur within the customer's process. In these systems, the arrivals and/or service processes are often stochastic. Queueing theory is a branch of mathematical analysis of systems that involve waiting lines in order to predict (and control) their behavior over time. The basic models within queueing theory involve a single line that is served by a set of servers. Figure C.1 illustrates the major components of a queueing system with a single queue feeding into a set of servers.

In queueing theory the term customer is used as a generic term to describe the entities that flow and receive service. A resource is a generic term used to describe the components of the system that are required by a customer as the customer moves through the system. The individual units of the

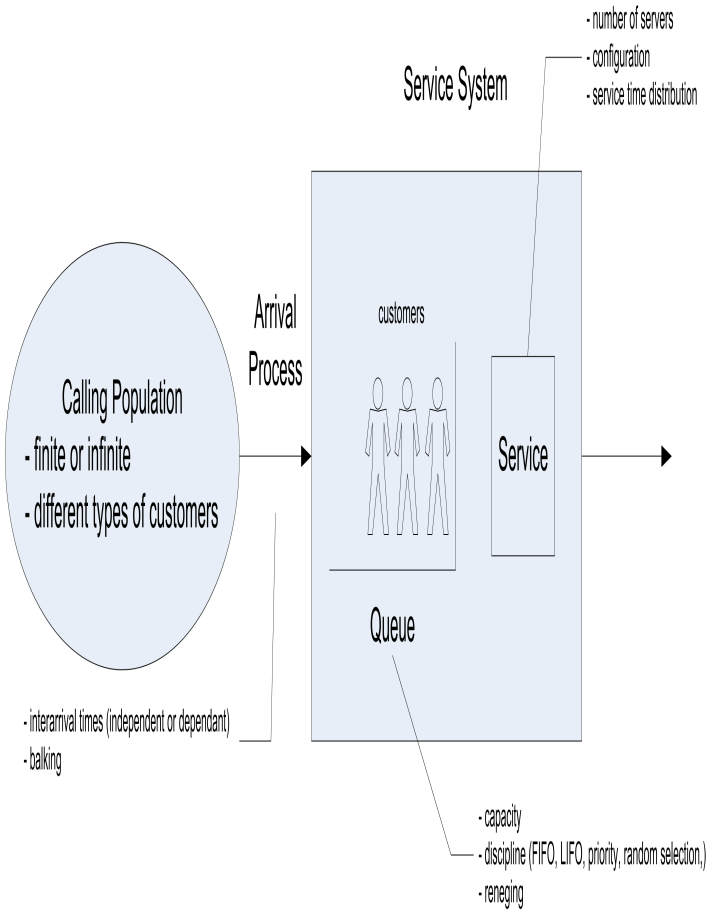


Figure C.1.: Example Single Queue System

resource are often called servers. In Figure C.1, the potential customers can be described as coming from a calling population. The term calling population comes from the historical use of queueing models in the analysis of phone calls to telephone trunk lines. Customers within the calling population may arrive to the system according to an arrival process. In the finite population case, the arrival rate that the system experiences will quite naturally decrease as customers arrive since fewer customers are available to arrive if they are within the system.

In the infinite calling population case, the rate of arrivals to the system does not depend upon on how many customers have already arrived. In other words, there are so many potential customers in the population that the arrival rate to the system is not affected by the current number of customers within the system. Besides characterizing the arrival process by the rate of arrivals, it is useful to think in terms of the inter-arrival times and in particular the inter-arrival time distribution. In general, the calling population may also have different types of customers that arrive at different rates. In the analytical analysis presented here, there will only be one type of customer.

The queue is that portion of the system that holds waiting customers. The two main characteristics for the queue are its size or capacity, and its discipline. If the queue has a finite capacity, this indicates that there is only enough space in the queue for a certain number of customers to be waiting at any given time. The queue discipline refers to the rule that will be used to decide the order of the customers within the queue. A first-come, first-served (FCFS) queue discipline orders the queue by the order of arrival, with the most recent arrival always joining the end of the queue. A last-in, first out (LIFO) queue discipline has the most recent arrival joining the beginning of the queue. A last-in, first out queue discipline acts like a stack of dishes. The first dish is at the bottom of the stack, and the last dish added to the stack is at the top of the stack. Thus, when a dish is needed, the next dish to be used is the last one added to the stack. This type of discipline often appears in manufacturing settings when the items

are placed in bins, with newly arriving items being placed on top of items that have previously arrived.

Other disciplines include random and priority. You can think of a random discipline modeling the situation of a server randomly picking the next part to work on (e.g. reaches into a shallow bin and picks the next part). A priority discipline allows the customers to be ordered within the queue by a specified priority or characteristic. For example, the waiting items may be arranged by due-date for a customer order.

The resource is that portion of the system that holds customers that are receiving service. Each customer that arrives to the system may require a particular number of units of the resource. The resource component of this system can have 1 or more servers. In the analytical treatment, each customer will required only one server and the service time will be governed by a probability distribution called the service time distribution. In addition, for the analytical analysis the service time distribution will be the same for all customers. Thus, the servers are all identical in how they operate and there is no reason to distinguish between them. Queueing systems that contain servers that operate in this fashion are often referred to as parallel server systems. When a customer arrives to the system, the customer will either be placed in the queue or placed in service. After waiting in line, the customer must select one of the (identical) servers to receive service. The following analytical analysis assumes that the only way for the customer to depart the system is to receive service.

C.1.1. Queueing Notation

The specification of how the major components of the system operate gives a basic system configuration. To help in classifying and identifying the appropriate modeling situations, Kendall's notation ((Kendall 1953)) can be used. The basic format is:

arrival process / service process / number of servers / system capacity / size of the calling population / queue discipline

For example, the notation M/M/1 specifies that the arrival process is Markovian (M) (exponential time between arrivals), the service process is Markovian (M) (exponentially distributed service times, and that there is 1 server. When the system capacity is not specified it is assumed to be infinite. Thus, in this case, there is a single queue that can hold any customer that arrives. The calling population is also assumed to be infinite if not explicitly specified. Unless otherwise noted the queue discipline is assumed to be FCFS.

Traditionally, the first letter(s) of the appropriate distribution is used to denote the arrival and service processes. Thus, the case LN/D/2 represents a queue with 2 servers having a lognormally (LN) distributed time between arrivals and deterministic (D) service times. Unless otherwise specified it is typically assumed that the arrival process is a renewal process, i.e. that the time between arrivals are independent and identically distributed and that the service times are also independent and identically distributed.

Also, the standard models assume that the arrival process is independent of the service process and vice a versa. To denote any distribution, the letter G for general (any) distribution is used. The notation (GI) is often used to indicate a general distribution in which the random variables are independent. Thus, the G/G/5 represents a queue with an arrival process having any general distribution, a general distribution for the service times, and 5 servers. Figure C.2 illustrates some of the common queueing systems as denoted by Kendall's notation.

There are a number of quantities that form the basis for measuring the performance of queueing systems: - The time that a customer spends waiting in the queue: T_q

- The time that a customer spends in the system (queue time plus service time): T

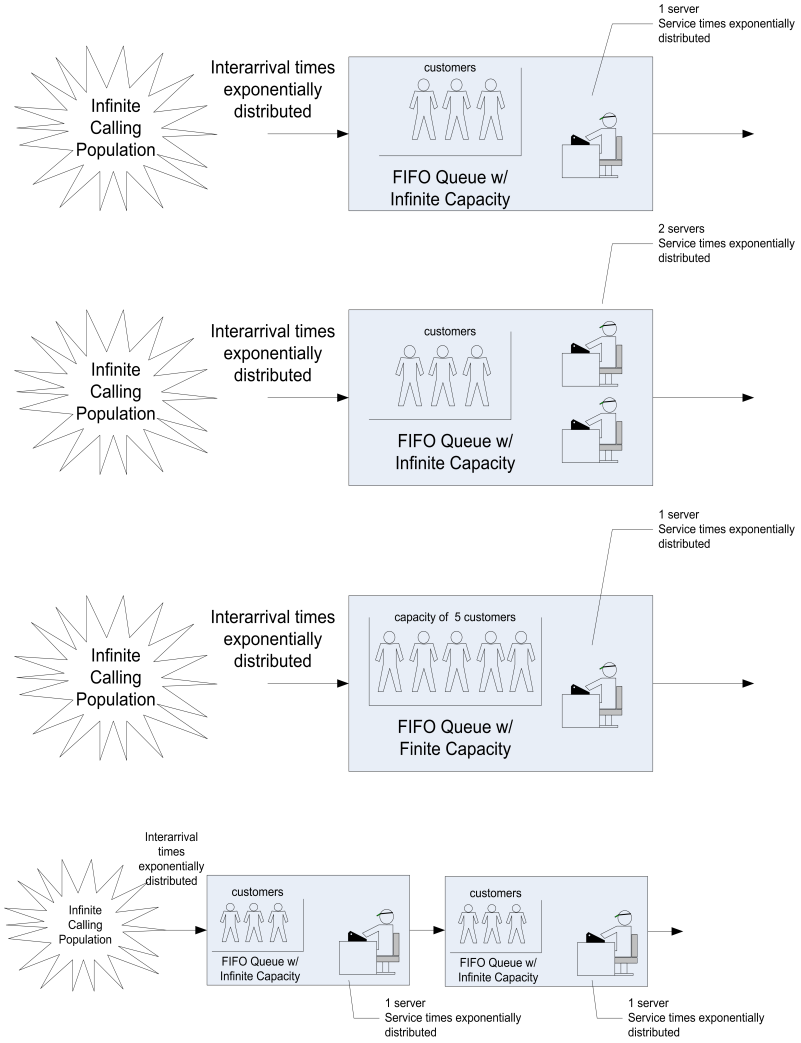


Figure C.2.: Illustrating Some Common Queueing Situations

- The number of customers in the queue at time t: $N_q(t)$
- The number of customers that are in the system at time t: $N(t)$
- The number of customer in service at time t: $N_b(t)$

These quantities will be random variables when the queueing system has stochastic elements. We will assume that the system is work conserving, i.e. that the customers do not exit without received all of their required service and that a customer uses one and only one server to receive service. For a work conserving queue, the following is true:

$$N(t) = N_q(t) + N_b(t)$$

This indicates that the number of customers in the system must be equal to the number of customers in queue plus the number of customers in service. Since the number of servers is known, the number of busy servers can be determined from the number of customers in the system. Under the assumption that each customer requires 1 server (1 unit of the resource), then $N_b(t)$ is also the current number of busy servers. For example, if the number of servers is 3 and the number of customers in the system is 2, then there must be 2 customers in service (two servers that are busy). Therefore, knowledge of $N(t)$ is sufficient to describe the state of the system. Because $N(t) = N_q(t) + N_b(t)$ is true, the following relationship between expected values is also true:

$$E[N(t)] = E[N_q(t)] + E[N_b(t)]$$

If ST is the service time of an arbitrary customer, then it should be clear that:

$$E[T] = E[T_q] + E[ST]$$

That is, the expected system time is equal to the expected waiting time in the queue plus the expected time spent in service.

C.1.2. Little's Formula

Chapter 4 presented time persistent data and computed time-averages. For queueing systems, one can show that relationships exist between such quantities as the expected number in the queue and the expected waiting time in the queue. To understand these relationships, consider Figure C.3 which illustrates the sample path for the number of customers in the queue over a period of time.

Let $A_i; i = 1 \ldots n$ represent the time that the i^{th} customer enters the queue, $D_i; i = 1 \ldots n$ represent the time that the i^{th} customer exits the queue, and $T_{q_i} = D_i$ - A_i for $i = 1 \ldots n$ represent the time that the i^{th} customer spends in the queue. Recall that the average time spent in the queue was:

$$\bar{T}_q = \frac{\sum_{i=1}^{n} T_{q_i}}{n} = \frac{0 + 11 + 8 + 7 + 2 + 5 + 6 + 0}{8} = \frac{39}{8} = 4.875$$

The average number of customers in the queue was:

$$\begin{aligned}\bar{L}_q &= \frac{0(2-0) + 1(7-2) + 2(10-7) + 3(13-10) + 2(15-13) + 1(16-15)}{25} \\ &+ \frac{4(17-16) + 3(18-17) + 2(21-18) + 1(22-21) + 0(25-22)}{25} \\ &= \frac{39}{25} = 1.56\end{aligned}$$

By considering the waiting time lengths within the figure, the area under the sample path curve can be computed as:

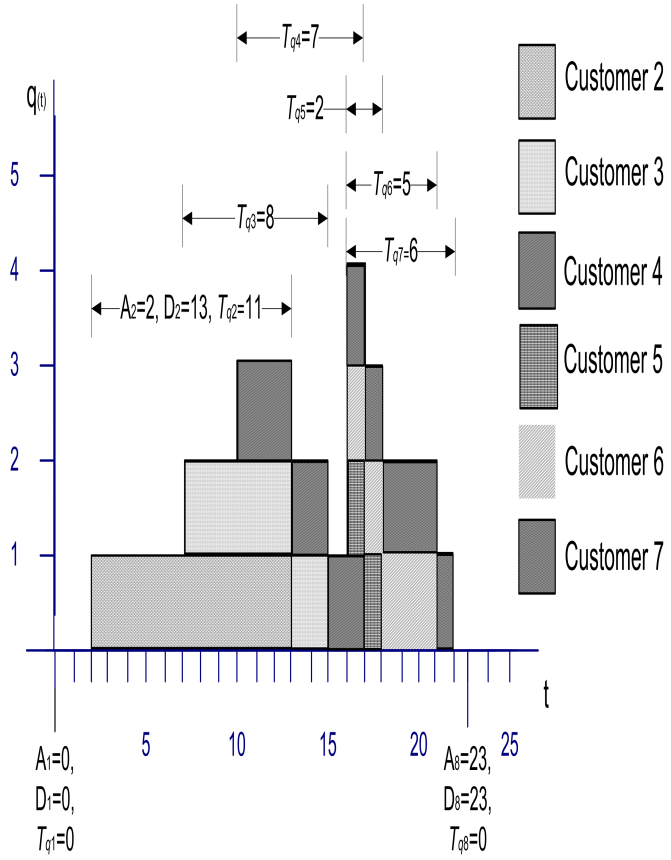


Figure C.3.: Sample Path for the Number of Customers in a Queue

$$\sum_{i=1}^{n} T_{q_i} = 39$$

But, by definition the area should also be:

$$\int_{t_0}^{t_n} q(t)\mathrm{d}t$$

Thus, it is no coincidence that the computed value for the numerators in $\bar{T}_q$ and $\bar{L}_q$ for the example is 39. Operationally, this *must* be the case.

Define $\bar{R}$ as the average rate that customers exit the queue. The average rate of customer exiting the queue can be estimated by counting the number of customers that exit the queue over a period of time. That is,

$$\bar{R} = \frac{n}{t_n - t_0}$$

where n is the number of customers that departed the system during the time $t_n - t_0$. This quantity is often called the average throughput rate. For this example, $\bar{R} = 8/25$. By combining these equations, it becomes clear that the following relationship holds:

$$\bar{L}_q = \frac{\int_{t_0}^{t_n} q(t)\mathrm{d}t}{t_n - t_0} = \frac{n}{t_n - t_0} \times \frac{\sum_{i=1}^{n} T_{q_i}}{n} = \bar{R} \times \bar{T}_q$$

This relationship is a conservation law and can also be applied to other portions of the queueing system as well. In words the relationship states that:

Average number in queue = average throughput rate × average waiting time in queue

When the service portion of the system is considered, then the relationship can be translated as:

Average number in service = average throughput rate × average time in service

When the entire queueing system is considered, the relationship yields:

Average number in the system = average throughput rate × average time in the system

These relationships hold operationally for these statistical quantities as well as for the expected values of the random variables that underlie the stochastic processes. This relationship is called Little's formula after the queueing theorist who first formalized the technical conditions of its applicability to the stochastic processes within queues of this nature. The interested reader is referred to (Little 1961) and (Glynn and Whitt 1989) for more on these relationships. In particular, Little's formula states a relationship between the steady state expected values for these processes.

To develop these formulas, define N, N_q, N_b as random variables that represent the number of customers in the system, in the queue, and in service at an arbitrary point in time in steady state. Also, let λ be the expected arrival rate so that $1/\lambda$ is the mean of the inter-arrival time distribution and let $\mu = 1/E[ST]$ so that $E[ST] = 1/\mu$ is the mean of the service time distribution. The expected values of the quantities of interest can be defined as:

$$\begin{aligned} L &\equiv E[N] \\ L_q &\equiv E[N_q] \\ B &\equiv E[N_b] \\ W &\equiv E[T] \\ W_q &\equiv E[T_q] \end{aligned}$$

Thus, it should be clear that

$$L = L_q + B$$
$$W = W_q + E[ST]$$

In steady state, the mean arrival rate to the system should also be equal to the mean through put rate. Thus, from Little's relationship the following are true:

$$L = \lambda W$$
$$L_q = \lambda W_q$$
$$B = \lambda E[ST] = \frac{\lambda}{\mu}$$

To gain an intuitive understanding of Little's formulas in this situation, consider that in steady state the mean rate that customers exit the queue must also be equal to the mean rate that customers enter the queue. Suppose that you are a customer that is departing the queue and you look behind yourself to see how many customers are left in the queue. This quantity should be L_q on average. If it took you on average W_q to get through the queue, how many customers would have arrived on average during this time? If the customers arrive at rate λ then $\lambda \times W_q$ is the number of customers (on average) that would have arrived during your time in the queue, but these are the customers that you would see (on average) when looking behind you. Thus, $L_q = \lambda W_q$.

Notice that λ and μ must be given and therefore B is known. The quantity B represents the expected number of customers in service in steady state, but since a customer uses only 1 server while in service, B also represents the expected number of busy servers in steady state. If there are c identical servers in the resource, then the quantity, B/c represents the fraction of the servers that are busy. This quantity can be interpreted as the utilization of the resource as a whole or the average utilization of a server, since they are all identical. This quantity is defined as:

$$\rho = \frac{B}{c} = \frac{\lambda}{c\mu}$$

The quantity, $c\mu$, represents the maximum rate at which the system can perform work on average. Because of this $c\mu$ can be interpreted as the mean capacity of the system. One of the technical conditions that is required for Little's formula to be applicable is that $\rho < 1$ or $\lambda < c\mu$. That is, the mean arrival rate to the system must be less than mean capacity of the system. This also implies that the utilization of the resource must be less than 100%.

The queueing system can also be characterized in terms of the *offered load*. The offered load is a dimensionless quantity that gives the average amount of work offered per time unit to the c servers. The offered load is defined as: $r = \lambda/\mu$. Notice that this can be interpreted as each customer arriving with $1/\mu$ average units of work to be performed. The steady state conditions thus indicate that $r < c$. In other words, the arriving amount of work to the queue cannot exceed the number of servers. These conditions make sense for steady state results to be applicable, since if the mean arrival rate was greater than the mean capacity of the system, the waiting line would continue to grow over time.

C.1.3. Deriving Formulas for Markovian Single Queue Systems

Notice that with $L = \lambda W$ and the other relationships, all of the major performance measures for the queue can be computed if a formula for one of the major performance measures (e.g. L, L_q, W, or W_q) are available. In order to derive formulas for these performance measures, the arrival and service processes must be specified.

This section shows that for the case of exponential time between arrivals and exponential service times, the necessary formulas can be derived. It is useful to go through the basic derivations in order to better understand

the interpretation of the various performance measures, the implications of the assumptions, and the concept of steady state.

To motivate the development, let's consider a simple example. Suppose you want to model an old style telephone booth, which can hold only one person while the person uses the phone. Also assume that any people that arrive while the booth is in use immediately leave. In other words, nobody waits to use the booth.

For this system, it is important to understand the behavior of the stochastic process $N(t); t \geq 0$, where $N(t)$ represents the number of people that are in the phone booth at any time t. Clearly, the possible values of $N(t)$ are 0 and 1, i.e. $N(t) \in \{0, 1\}$. Developing formulas for the probability that there are 0 or 1 customers in the booth at any time t, i.e. $P_i(t) = P\{N(t) = i\}$ will be the key to modeling this situation.

Let λ be the mean arrival rate of customers to the booth and let $ST = 1/\mu$ be the expected length of a telephone call. For example, if the mean time between arrivals is 12 minutes, then $\lambda = 5$/hr, and if the mean length of a call is 10 minutes, then $\mu = 6$/hr. The following reasonable assumptions will be made:

1. The probability of a customer arriving in a small interval of time, Δt, is roughly proportional to the length of the interval, with the proportionality constant equal to the mean rate of arrival, λ.
2. The probability of a customer completing an ongoing phone call during a small interval of time is roughly proportional to the length of the interval and the proportionality constant is equal to the mean service rate, μ.
3. The probability of more than one arrival in an arbitrarily small interval, Δt, is negligible. In other words, Δt, can be made small enough so that only one arrival can occur in the interval.

4. The probability of more than one service completion in an arbitrarily small interval, Δt, is negligible. In other words, Δt, can be made small enough so that only one service can occur in the interval.

Let $P_0(t)$ and $P_1(t)$ represent the probability that there is 0 or 1 customer using the booth, respectively. Suppose that you observe the system at time t and you want to derive the probability for 0 customers in the system at some future time, $t + \Delta t$. Thus, you want, $P_0(t + \Delta t)$.

For there to be zero customers in the booth at time $t + \Delta t$, there are two possible situations that could occur. First, there could have been no customers in the system at time t and no arrivals during the interval Δt, or there could have been one customer in the system at time t and the customer completed service during Δt. Thus, the following relationship should hold:

$$\begin{aligned}\{N(t+\Delta t)=0\} =& \{\{N(t)=0\} \cap \{\text{no arrivals during } \Delta t\}\} \cup \\ & \{\{N(t)=1\} \cap \{\text{service completed during } \Delta t\}\}\end{aligned}$$

It follows that:

$$P_0(t+\Delta t) = P_0(t)P\{\text{no arrivals during}\Delta t\} + P_1(t)P\{\text{service completed during}\Delta t\}$$

In addition, at time $t + \Delta t$, there might be a customer using the booth, $P_1(t+\Delta t)$. For there to be one customer in the booth at time $t+\Delta t$, there are two possible situations that could occur. First, there could have been no customers in the system at time t and one arrival during the interval Δt, or there could have been one customer in the system at time t and the customer did not complete service during Δt. Thus, the following holds:

$$P_1(t+\Delta t) = P_0(t)P\{1 \text{ arrival during } \Delta t\} + P_1(t)P\{\text{no service completed during } \Delta t\}$$

Because of assumptions (1) and (2), the following probability statements can be used:

$$\begin{aligned} P\{\text{1 arrival during } \Delta t\} &\cong \lambda \Delta t \\ P\{\text{no arrivals during } \Delta t\} &\cong 1 - \lambda \Delta t \\ P\{\text{service completed during } \Delta t\} &\cong \mu \Delta t \\ P\{\text{no service completed during } \Delta t\} &\cong 1 - \mu \Delta t \end{aligned}$$

This results in the following:

$$\begin{aligned} P_0(t + \Delta t) &= P_0(t)[1 - \lambda \Delta t] + P_1(t)[\mu \Delta t] \\ P_1(t + \Delta t) &= P_0(t)[\lambda \Delta t] + P_1(t)[1 - \mu \Delta t] \end{aligned}$$

Collecting the terms in the equations, rearranging, dividing by Δt, and taking the limit as goes Δt to zero, yields the following set of differential equations:

$$\begin{aligned} \frac{dP_0(t)}{dt} &= \lim_{\Delta t \to 0} \frac{P_0(t + \Delta t) - P_0(t)}{\Delta t} = -\lambda P_0(t) + \mu P_1(t) \\ \frac{dP_1(t)}{dt} &= \lim_{\Delta t \to 0} \frac{P_1(t + \Delta t) - P_1(t)}{\Delta t} = \lambda P_0(t) - \mu P_1(t) \end{aligned}$$

It is also true that $P_0(t) + P_1(t) = 1$. Assuming that $P_0(0) = 1$ and $P_1(0) = 0$ as the initial conditions, the solutions to these differential equations are:

$$P_0(t) = \left(\frac{\mu}{\lambda + \mu}\right) + \left(\frac{\lambda}{\lambda + \mu}\right) e^{-(\lambda + \mu)t} \tag{C.1}$$

$$P_1(t) = \left(\frac{\lambda}{\lambda + \mu}\right) - \left(\frac{\lambda}{\lambda + \mu}\right) e^{-(\lambda + \mu)t} \tag{C.2}$$

These equations represent the probability of having either 0 or 1 customer in the booth at any time. If the limit as t goes to infinity is considered, the *steady state probabilities*, P_0 and P_1 can be determined:

$$P_0 = \lim_{t\to\infty} P_0(t) = \frac{\mu}{\lambda+\mu} \tag{C.3}$$

$$P_1 = \lim_{t\to\infty} P_1(t) = \frac{\lambda}{\lambda+\mu} \tag{C.4}$$

These probabilities can be interpreted as the chance that an arbitrary customer finds the booth either empty or busy after an infinitely long period of time has elapsed.

If only the steady state probabilities are desired, there is an easier method to perform the derivation, both from a conceptual and a mathematical standpoint. The assumptions (1-4) that were made ensure that the arrival and service processes will be Markovian. In other words, that the time between arrivals of the customer is exponentially distributed and that the service times are exponentially distributed. In addition, the concept of steady state can be used.

Consider the differential equations. These equations govern the rate of change of the *probabilities* over time. Consider the analogy of water to probability and think of a dam or container that holds an amount of water. The rate of change of the level of water in the container can be thought of as:

Rate of change of level = rate into container – rate out of the container

In steady state, the level of the water should not change, thus the rate into the container must equal the rate out of the container. Using this analogy,

$$\frac{dP_i(t)}{dt} = \text{rate in - rate out}$$

and for steady state: *rate in = rate out* with the probability *flowing* between the states. Figure C.4 illustrates this concept via a state transition diagram. If N represents the steady state number of customers in the system (booth), the two possible states that the system can be in are 0 and 1. The rate of transition from state 0 to state 1 is the rate that an arrival occurs and the state of transition from state 1 to state 0 is the service rate.

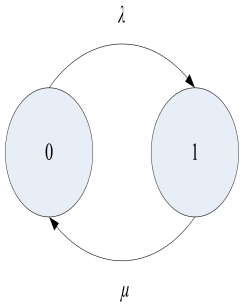


Figure C.4.: Two State Rate Transition Diagram

The rate of transition into state 0 can be thought of as the rate that probability flows from state 1 to state 0 times the chance of being in state 1, i.e. μP_1. The rate of transition out of state 0 can be thought of as the rate from state 0 to state 1 times the chance of being in state 0, i.e. λP_0. Using these ideas yields:

State	*rate in*	=	*rate out*
0	$\mu\ P_1$	=	$\lambda\ P_0$
1	$\lambda\ P_0$	=	$\mu\ P_1$

Notice that these are identical equations, but with the fact that $P_0 + P_1 = 1$, we will have two equations and two unknowns (P_0, P_1). Thus, the equations can be easily solved to yield the same results as in Equation C.3 and

Equation C.4. Sets of equations derived in this manner are called *steady state* equations.

Now, more general situations can be examined. Consider a general queueing system with some given number of servers. An arrival to the system represents an increase in the number of customers and a departure from the system represents a decrease in the number of customers in the system.

Figure C.5 illustrates a general state transition diagram for this system. Let N be the number of customers in the system in steady state and define:

$$P_n = P\{N = n\} = \lim_{t \to \infty} P\{N(t) = n\}$$

as the steady state probability that there are n customers in the system. Let λ_n be the mean arrival rate of customers entering the system when there are n customers in the system, $\lambda_n \geq 0$. Let μ_n be the mean service rate for the overall system when there are n customers in the system. This is the rate, at which customers depart when there are n customers in the system. In this situation, the number of customers may be infinite, i.e. $N \in \{0, 1, 2, ...\}$. The steady state equations for this situation are as follows:

State	*rate in*	=	*rate out*
0	$\mu_1 P_1$	=	$\lambda_0 P_0$
1	$\lambda_0 P_0 + \mu_2 P_2$	=	$\mu_1 P_1 + \lambda_1 P_1$
2	$\lambda_1 P_1 + \mu_3 P_3$	=	$\mu_2 P_2 + \lambda_2 P_2$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	$\lambda_{n-1} P_{n-1} + \mu_{n+1} P_{n+1}$	=	$\mu_n P_n + \lambda_n P_n$

These equations can be solved recursively starting with state 0. This yields:

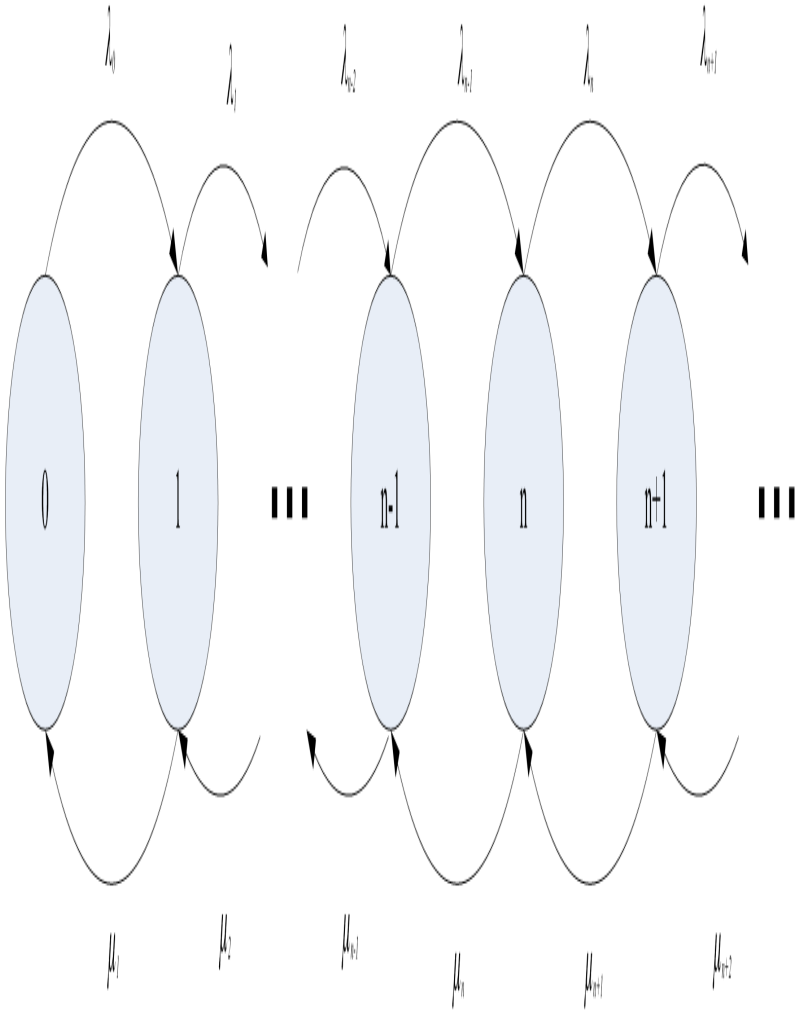


Figure C.5.: General Rate Transition Diagram for Any Number of States

$$P_1 = \frac{\lambda_0}{\mu_1} P_0$$
$$P_2 = \frac{\lambda_1 \lambda_0}{\mu_2 \mu_1} P_0$$
$$\vdots$$
$$P_n = \frac{\lambda_{n-1} \lambda_{n-2} \cdots \lambda_0}{\mu_n \mu_{n-1} \cdots \mu_1} P_0 = \prod_{j+1}^{n-1} \left(\frac{\lambda_j}{\mu_{j+1}} \right) P_0$$

for $n = 1, 2, 3, ...$ Provided that $\sum_{n=0}^{\infty} P_n = 1$, P_0 can be computed as:

$$P_0 = \left[\sum_{n=0}^{\infty} \prod_{j=0}^{n-1} \left(\frac{\lambda_j}{\mu_{j+1}} \right) \right]^{-1}$$

Therefore, for any given set of λ_n and μ_n, one can compute P_n. The P_n represent the steady state probabilities of having n customers in the system. Because P_n is a probability distribution, the expected value of this distribution can be computed. What is the expected value for the P_n distribution? The expected number of customers in the system in steady state. This is L. The expected number of customers in the system and the queue are given by:

$$L = \sum_{n=0}^{\infty} n P_n$$
$$L_q = \sum_{n=c}^{\infty} (n - c) P_n$$

where c is the number of servers.

There is one additional formula that is needed before Little's formula can be applied with other known relationships. For certain systems, e.g. finite

system size, not all customers that arrive will enter the queue. Little's formula is true for the customers that enter the system. Thus, the effective arrival rate must be defined. The effective arrival rate is the mean rate of arrivals that actually enter the system. This is given by computing the expected arrival rate across the states. For infinite system size, we have:

$$\lambda_e = \sum_{n=0}^{\infty} \lambda_n P_n$$

For finite system size, k, we have:

$$\lambda_e = \sum_{n=0}^{k-1} \lambda_n P_n$$

since $\lambda_n = 0$ for $n \geq k$. This is because nobody can enter when the system is full. All these relationships yield:

$$\begin{aligned} L &= \lambda_e W \\ L_q &= \lambda_e W_q \\ B &= \frac{\lambda_e}{\mu} \\ \rho &= \frac{\lambda_e}{c\mu} \\ L &= L_q + B \\ W &= W_q + \frac{1}{\mu} \end{aligned}$$

Section C.4 presents the results of applying the general solution for P_n to different queueing system configurations. Table C.1 presents specific results for the M/M/c queueing system for $c = 1, 2, 3$. Using these results and those in Section C.4, the analysis of a variety of different queueing situations is possible.

Table C.1.: Results M/M/c $\rho = \lambda/c\mu$

c	P_0	L_q
1	$1-\rho$	$\frac{\rho^2}{1-\rho}$
2	$\frac{1-\rho}{1+\rho}$	$\frac{2\rho^3}{1-\rho^2}$
3	$\frac{2(1-\rho)}{2+4\rho+3\rho^2}$	$\frac{9\rho^4}{2+2\rho-\rho^2-3\rho^3}$

This section has only scratched the surface of queueing theory. A vast amount of literature is available on queueing theory and its application. You should examine (Gross and Harris 1998), (Cooper 1990), and (Kleinrock 1975) for a more in depth theoretical development of the topic. There are also a number of free on-line resources available on the topic. The interested reader should search on "Queueing Theory Books On Line". The next section presents some simple examples to illustrate the use of the formulas.

C.2. Examples and Applications of Queueing Analysis

The derivations and formulas in the previous section certainly appear to be intimidating and they can be tedious to apply. Fortunately, there is readily available software that can be used to do the calculations. On line resources for software for queueing analysis can be found at:

- List of queueing theory software resources
- QTSPlus Excel based queueing theory software that accompanies Gross et al. (2008)

The most important part of performing a queueing analysis is to identify the most appropriate queueing model for a given situation. Then, software tools can be used to analyze the situation. This section provides a number of examples and discusses the differences between the systems so that you can better apply the results of the previous sections. The solutions to these types of problems involve the following steps:

1. Identify the arrival and service processes
2. Identify the size of the arriving population and the size of the system
3. Specify the appropriate queueing model and its input parameters
4. Identify the desired performance measures
5. Compute the required performance measures

C.2.1. Infinite Queue Examples

In this section, we will explore two queueing systems (M/M/1 and M/M/c) that have an infinite population of arrivals and an infinite size queue. The examples illustrate some of the common questions related to these types of queueing systems.

Example C.1. Customers arrive at a one window drive through pharmacy according to a Poisson distribution with a mean of 10 per hour. The service time per customer is exponential with a mean of 5 minutes. There are 3 spaces in front of the window, including that for the car being served. Other arriving cars can wait outside these 3 spaces. The pharmacy is interested in answering the following questions:

1. What is the probability that an arriving customer can enter one of the 3 spaces in front of the window?

2. What is the probability that an arriving customer will have to wait outside the 3 spaces?
3. What is the probability that an arriving customer has to wait?
4. How long is an arriving customer expected to wait before starting service?
5. How many car spaces should be provided in front of the window so that an arriving customer has a $\gamma = 40\%$ chance of being able to wait in one of the provided spaces?

Solution to Example C.1

The customers arrive according to a Poisson process, which implies that the time between arrivals is exponentially distributed. Thus, the arrival process is Markovian (M). The service process is stated as exponential. Thus, the service process is Markovian (M). There is only 1 window and customers wait in front of this window to receive service. Thus, the number of servers is $c = 1$. The problem states that customers that arrive when the three spaces are filled, still wait for service outside the 3 spaces. Thus, there does not appear to be a restriction on the size of the waiting line. Therefore, this situation can be considered an infinite size system. The arrival rate is specified for any likely customer and there is no information given concerning the total population of the customers. Thus, it appears that an infinite population of customers can be assumed. We can conclude that this is a M/M/1 queueing situation with an arrival rate $\lambda = 10/hr$ and a service rate of $\mu = 12/hr$. Notice the input parameters have been converted to a common unit of measure (customers/hour).

The equations for the M/M/1 can be readily applied. From Section C.4 the following formulas can be applied:

$$
\begin{aligned}
c &= 1 \\
\lambda_n &= \lambda = 10/hr \\
\lambda_e &= \lambda \\
\mu_n &= \mu = 12/hr \\
\rho &= \frac{\lambda}{c\mu} = r = 10/12 = 5/6 \\
P_0 &= 1 - \frac{\lambda}{\mu} = 1 - r = 1/6 \\
P_n &= P_0 r^n = \frac{1}{6}\left(\frac{5}{6}\right)^n \\
L &= \frac{r}{1-r} = \frac{(5/6)}{1-(5/6)} = 5 \\
L_q &= \frac{r^2}{1-r} = \frac{(5/6)^2}{1-(5/6)} = 4.1\bar{6} \\
W_q &= \frac{L_q}{\lambda} = \frac{r}{\mu(1-r)} = 0.41\bar{6} \text{ hours} = 25.02 \text{ minutes} \\
W &= \frac{L}{\lambda} = 5/10 = 0.5 \text{ hours} = 30 \text{ minutes}
\end{aligned}
$$

Let's consider each question from the problem in turn:

1. Probability statements of this form are related to the underlying state variable for the system. In this case, let N represent the number of customers in the system. To find the probability that an arriving customer can enter one of the 3 spaces in front of the window, you should consider the question: *When can a customer enter one of the 3 spaces?* A customer can enter one of the three spaces when there are 0 or 1 or 2 customers in the system. This is not $N = 0$, 1, 2, or 3 because if there are 3 customers in the system, then the 3^{rd} space is

taken. Therefore, $P\{N \leq 2\}$ needs to be computed.
To compute $P[N \leq 2]$ note that:

$$P[N \geq n] = \sum_{j=n}^{\infty} P_0 r^j = (1-r) \sum_{j=n}^{\infty} r^{j-n} = (1-r)\frac{r^n}{1-r} = r^n$$

Therefore, $P[N \leq n]$ is:

$$P[N \leq n] = 1 - P[N > n] = 1 - P[N \geq n+1] = 1 - r^{n+1}$$

Thus, we have that $P[N \leq 2] = 1 - r^3 \cong 0.42$

2. An arriving customer will have to wait outside of the 3 spaces, when there are more than 2 (3 or more) customers in the system. Thus, $P[N > 2]$ needs to be computed to answer question (b). This is the complement event for part (a), $P[N > 2] = 1 - P[N \leq 2] \cong 0.58$

3. An arriving customer has to wait when there are 1 or more customers already at the pharmacy. This is $P[N \geq 1] = 1 - P[N < 1] = 1 - P_0$.

$$P[N \geq 1] = 1 - P[N < 1] = 1 - P_0 = 1 - (1-r) = \rho = \frac{5}{6}$$

4. The waiting time that does not include service is the queueing time. Thus, W_q needs to be computed to answer question (d).

$$W_q = 0.41\bar{6} \text{ hours} = 25.02 \text{ minutes}$$

5. This is a design question for which the probability of waiting in one of the provided spaces is used to determine the number of spaces to provide. Suppose that there are m spaces. An arriving customer can waiting in one of the spaces if there are $m-1$ or less customers in the system. Thus, m needs to be chosen such that $P\{N \leq m-1\} = 0.4$.

$$P[N \leq m-1] = \gamma$$
$$1 - r^m = \gamma$$
$$r^m = 1 - \gamma$$
$$m = \frac{\ln(1-\gamma)}{\ln r} = \frac{\ln(1-0.4)}{\ln(\frac{5}{6})} = 2.8 \cong 3 \text{ spaces}$$

Rounding up guarantees $P[N \leq m-1] \geq 0.4)$.

The following example models self-service copiers.

Example C.2. The Student Union Copy center is considering the installation of self-service copiers. They predict that the arrivals will be Poisson with a rate of 30 per hour and that the time spent copying is exponentially distributed with a mean of 1.75 minutes. They would like the chance that 4 or more people in the copy center to be less than 5%. How many copiers should they install?

Solution to Example C.2

The Poisson arrivals and exponential service times make this situation an M/M/c where c is the number of copiers to install and $\lambda = 0.5$ and $\mu = 1/1.75$ per minute. To meet the design criteria, $P[N \geq 4] = 1 - P[N \leq 3]$ needs to be computed for systems with $c = 1, 2, ...$ until $P[N \geq 4]$ is less than 5%. This can be readily achieved with the provided formulas or by utilizing the aforementioned software. In what follows, the QTSPlus software was used.

The software is very self-explanatory. It is important to remember that when applying the queueing formulas make sure that you keep your time units consistent. Setting up the QTSPlus spreadsheet as shown in Figure C.6 with $c = 3$ yields the results shown in Figure C.7. By changing the number of servers, one can find that $c = 3$ meets the probability requirement as shown in Table C.2.

Table C.2.: Results for Example C.2, $c = 1, 2, 3, 4$

c	$P[N \geq 4] = 1 - P[N \leq 3]$
1	0.586182
2	0.050972
3	0.019034
4	0.013022

C.2.1.1. Square Root Staffing Rule

Often in the case of systems with multiple servers such as the M/M/c, you want to determine the best value of c, as in Example C.2. Another common design situation is to determine the value c of such that there is an acceptable probability that an arriving customer will have to wait. For the case of Poisson arrivals, this is the same as the steady state probability that there are more than c customers in the system. For the M/M/c model, this is called the Erlang delay probability:

$$P_w = P[N \geq c] = \sum_{n=c}^{\infty} P_n = \frac{\dfrac{r^c}{c!}}{\dfrac{r^c}{c!} + (1-\rho)\sum_{j=0}^{c-1} \dfrac{r^j}{j!}}$$

Even though this is a relatively easy formula to use (especially in view of available spreadsheet software), an interesting and useful approximation

M/M/c: POISSON ARRIVALS TO MULTIPLE EXPONENTIAL SERVERS

Input Parameters:	
Arrival rate (λ)	0.5
Mean service time ($1/\mu$)	1.75
Number of servers in the system (c)	3
Plot Parameters:	
Maximum size for probability chart	15
Total time horizon for probability plotting	2.
Results:	
Mean interarrival time ($1/\lambda$)	2.
Service rate (μ)	0.571429
Average # arrivals in mean service time (r)	0.875
Server utilization (ρ)	29.17%
Fraction of time all servers are idle (p_0)	0.414003
Mean number of customers in the system (L)	0.901871
Mean number of customers in the queue (Lq)	0.026871
Mean wait time (W)	1.803743
Mean wait time in the queue (Wq)	0.053743

Figure C.6.: QTSPlus M/M/c Spreadsheet Results

	A	B	C	D
1	Customer Size Distribution			
2	n	prob(n)	CDF(n)	1-CDF(n)
3	0	0.414003	0.414003	0.585997
4	1	0.362253	0.776256	0.223744
5	2	0.158486	0.934741	0.065259
6	3	0.046225	0.980966	0.019034
7	4	0.013482	0.994448	0.005552
8	5	0.003932	0.998381	0.001619
9	6	0.001147	0.999528	0.000472
10	7	0.000335	0.999862	0.000138
11	8	0.000098	0.999960	0.000040
12	9	0.000028	0.999988	0.000012
13	10	0.000008	0.999997	0.000003
14	11	0.000002	0.999999	0.000001
15	12	0.000001	1.000000	0.000000
16	13	0.000000	1.000000	0.000000
17	14	0.000000	1.000000	0.000000
18	15	0.000000	1.000000	0.000000

Figure C.7.: QTSPlus M/M/c State Probability Results

has been developed called the square root staffing rule. The derivation of the square root staffing rule is given in (Tijms 2003). In what follows, the usefulness of the rule is discussed.

The *square root staffing rule* states that the least number of servers, c^*, required to meet the criteria, $P_w \leq \alpha$ is given by: $c^* \cong r + \gamma_\alpha \sqrt{r}$, where the factor γ_α is the solution to the equation:

$$\frac{\gamma \Phi(\gamma)}{\varphi(\gamma)} = \frac{1-\alpha}{\alpha}$$

The functions, $\Phi(\cdot)$ and $\varphi(\cdot)$ are the cumulative distribution function (CDF) and the probability density function (PDF) of a standard normal random variable. Therefore, given a design criteria, α, in the form a probability tolerance, you can find the number of servers that will result in the probability of waiting being less than α. This has very useful application in the area of call service centers and help support lines.

Example C.3. The Student Union Copy center is considering the installation of self-service copiers. They predict that the arrivals will be Poisson with a rate of 30 per hour and that the time spent copying is exponentially distributed with a mean of 1.75 minutes. Find the least number of servers such that the probability that an arriving customer waits is less than or equal to 0.10.

Solution to Example C.3

For $\lambda = 0.5$ and $\mu = 1/1.75$ per minute, you have that $r = 0.875$. Figure C.8 illustrates the use of the *SquareRootStaffingRule.xls* spreadsheet

that accompanies this chapter. The spreadsheet has text that explains the required inputs. Enter the offered load r in cell B3 and the delay criteria of 0.1 in cell B4. An initial search value is required in cell B5. The value of 1.0 will always work. The spreadsheet uses the goal seek functionality to solve for γ_α.

For this problem, this results in about 3 servers being needed to ensure that the probability of wait will be less than 10%. The square root staffing rule has been shown to be quite a robust approximation and can be useful in many design settings involving staffing.

The previous examples have had infinite system capacity. The next section presents examples of finite capacity queueing systems.

C.2.2. Finite Queue Examples

In this section, we will explore three queueing systems that are finite in some manner, either in space or in population. The examples illustrate some of the common questions related to these types of systems.

Example C.4. A single machine is connected to a conveyor system. The conveyor causes parts to arrive to the machine at a rate of 1 part per minute according to a Poisson distribution. There is a finite buffer of size 5 in front of the machine. The machine's processing time is considered to be exponentially distributed with a mean rate of 1.2 parts per minute. Any parts that arrive on the conveyor when the buffer is full are carried to other machines that are not part of this analysis. What are the expected system time and the expected number of parts at the machining center?

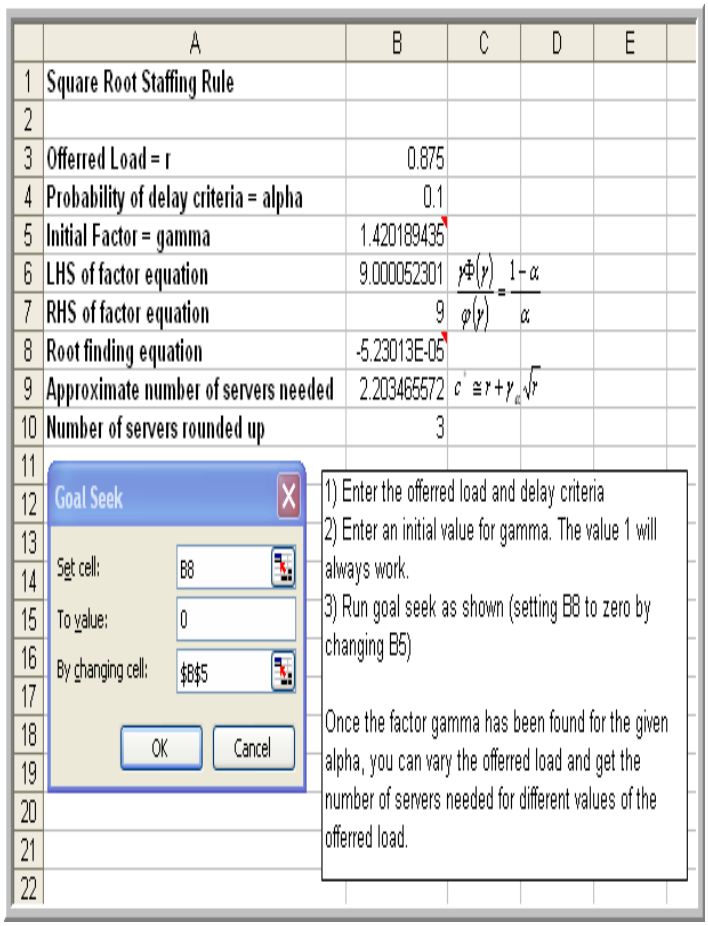


Figure C.8.: Spreadsheet for Square Root Staffing Rule

Solution to Example C.4

The finite buffer, Poisson arrivals, and exponential service times make this situation an M/M/1/6 where $k = 6$ is the size of the system (5 in buffer + 1 in service) and $\lambda = 1$ and $\mu = 1.2$ per minute. The desired performance measures are W and L. Figure C.9 presents the results using *QTSPlus*. Notice that in this case, the effective arrival rate must be computed:

$$\lambda_e = \sum_{n=0}^{k-1} \lambda_n P_n = \sum_{n=0}^{k-1} \lambda P_n = \lambda \sum_{n=0}^{k-1} P_n = \lambda(1 - P_k)$$

Rearranging this formula, yields, $\lambda = \lambda_e + \lambda_f$ where $\lambda_f = \lambda P_k$ equals the mean number of customers that are turned away from the system because it is full. According to Figure C.9, the expected number of turned away because the system is full is about 0.077 per minute (or about 4.62 per hour).

Let's take a look at modeling a situation that we experience everyday, parking lots.

Example C.5. The university has a row of 10 parking meter based spaces across from the engineering school. During the peak hours students arrive to the parking lot at a rate of 40 per hour according to a Poisson distribution and the students use the parking space for approximately 60 minutes exponentially distributed. If all the parking spaces are taken, it can be assumed that an arriving student does not wait (goes somewhere else to park). Suppose that the meters cost $w = \$0.03$ per minute, i.e. \$2 per hour. How much income does the university potentially lose during peak hours on average because the parking spaces are full?

M/M/1/K: POISSON ARRIVALS TO A SPACE-LIMITED SINGLE EXPONENTIAL SERVER

Input Parameters:

Arrival rate (λ)	1.
Mean service time ($1/\mu$)	.833333
Maximum capacity of system ($K > 1$)	6

Plot Parameters:

Total time horizon for probability plotting [ALL PROBABILITIES ARE PLOTTED!]	15.

Results:

Mean interarrival time ($1/\lambda$)	1
Effective arrival rate (λ_{eff})	0.92257607
Service rate (μ)	1.2
Traffic intensity (ρ) [NEED NOT BE < 1]	0.83333333
Server utilization (ρ_{eff}) [MUST BE < 100%]	76.88%
Fraction of time the server is idle (p_0)	0.23118661
Probability that the system is full (p_K)	0.07742393
Expected number turned away/unit time	0.07742393
Mean number of customers in the system (L)	2.29016258
Mean number of customers in the queue (Lq)	1.52134918
Mean waiting time (W)	2.48235635
Mean waiting time in the queue (Wq)	1.64902302

Figure C.9.: Results for Finite Buffer M/M/1/6 Queue

Solution to Example C.5

In this system, the parking spaces are the servers of the system. There are 10 parking spaces so that $c = 10$. In addition, there is no waiting for one of the meters and thus no queue forms. Therefore, the system size, k, is also 10. Because of the Poisson arrival process and exponential service times, this can be considered an M/M/10/10 queueing system. In other words, the size of the system is same as the number of servers, $c = k = 10$.

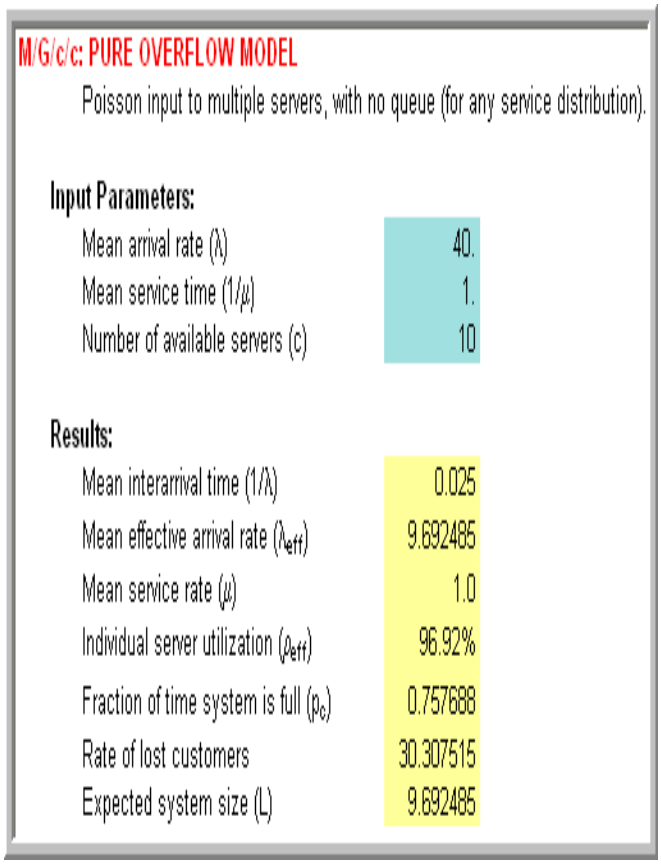
In the case of a M/M/c/c queueing system, P_c represents the probability that all the servers in the system are busy. Thus, it also represents the probability that an arriving customer will be turned away. The formula for the probability of a lost customer is called the Erlang loss formula:

$$P_c = \frac{\frac{r^c}{c!}}{\sum_{n=0}^{c} \frac{r^n}{n!}}$$

Customers arrive at the rate $\lambda = 20/hr$ whether the system is full or not. Thus, the expected number of lost customers per hour is λP_c. Since each customer brings $1/\mu$ service time charged at $w = \$2$ per hour, each arriving customer brings w/μ of income on average; however, not all arriving customers can park. Thus, the university loses $w \times 1/\mu \times \lambda P_c$ of income per hour. Figure C.10 illustrates the use of the *QTSPlus* software. In the figure the arrival rate of 40 per hour and the mean service time of 1 hours is entered. The number of parking spaces, 10, is the size of the system.

Thus according to Figure C.10, the university is losing about $\$2 \times 30.3 = \$60.6$ per hour because the metered lot is full during peak hours.

For this example, the service times are exponentially distributed. It turns out that for the case of $c = k$, the results for the M/M/c/c model are the same for the M/G/c/c model. In other words, the form of the service time distribution does not matter. The mean of the service distribution is the critical input parameter to this analysis.



M/G/c/c: PURE OVERFLOW MODEL

Poisson input to multiple servers, with no queue (for any service distribution).

Input Parameters:

Mean arrival rate (λ)	40.
Mean service time ($1/\mu$)	1.
Number of available servers (c)	10

Results:

Mean interarrival time ($1/\lambda$)	0.025
Mean effective arrival rate (λ_{eff})	9.692485
Mean service rate (μ)	1.0
Individual server utilization (ρ_{eff})	96.92%
Fraction of time system is full (p_c)	0.757688
Rate of lost customers	30.307515
Expected system size (L)	9.692485

Figure C.10.: Parking Lot Results for M/M/10/10 System

In the next example, a *finite population* of customers that can arrive, depart, and then return is considered. A classic and important example of this type of system is the machine interference or operator tending problem. A detailed discussion of the analysis and application of this model can be found in (Stecke 1992).

In this situation, a set of machines are tended by 1 or more operators. The operators must tend to stoppages (breakdowns) of the machines. As the machines breakdown, they may have to wait for the operator to complete the service of other machines. Thus, the machine stoppages cause the machines to *interfere* with the productivity of the set of machines because of their dependence on a common resource, the operator.

The situation that we will examine and for which analytical formulas can be derived is called the M/M/c/k/k queueing model where c is the number of servers (operators) and k is the number of machines (size of the population). Notice that the size of the system is the same as the size of the calling population in this particular model. For this system, the arrival rate of an *individual machine*, λ, is specified. This is not the arrival rate of the population as has been previously utilized. The service rate of μ for each operator is also necessary. The arrival and service rates for this system are:

$$\lambda_n = \begin{cases} (k-n)\lambda & n = 0, 1, 2, \ldots k \\ 0 & n \geq k \end{cases}$$

$$\mu_n = \begin{cases} n\mu & n = 1, 2, \ldots c \\ c\mu & n \geq c \end{cases}$$

These rates are illustrated in the state diagram of Figure C.11 for the case of 2 operators and 5 machines. Thus, for this system, the arrival rate to the system decreases as more machines breakdown. Notice that in the figure the arrival rate from state 0 to state 1 is 5λ. This is because there

are 5 machines that are not broken down, each with individual rate, λ. Thus, the total rate of arrivals to the system is 5λ. This rate goes down as more machines breakdown. When all the machines are broken down, the arrival rate to the system will be zero.

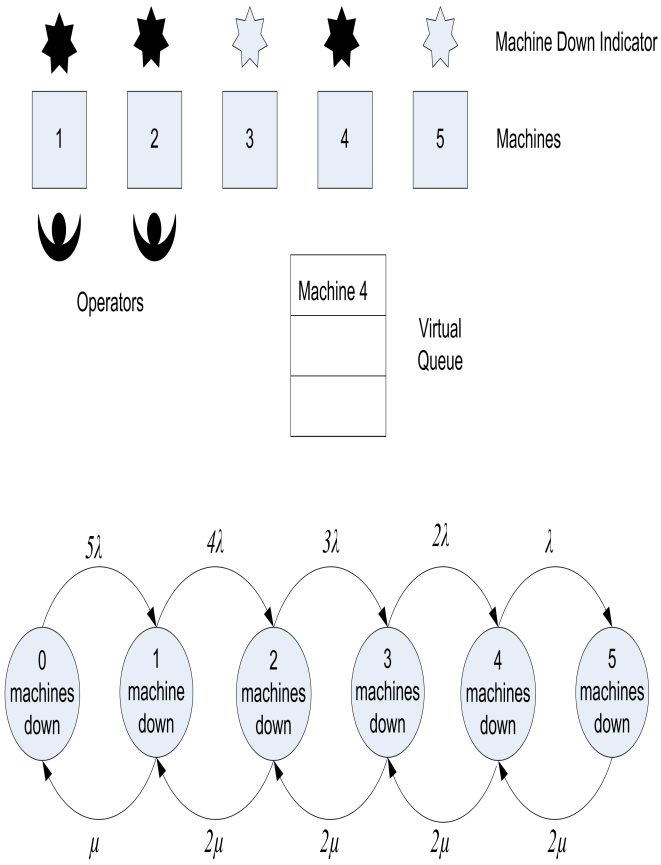


Figure C.11.: System Involving Machine Interference

Notice also that the service rate from state 1 to state 0 is μ. When one machine is in the system, the operator works at rate μ. Note also that the rate from state 2 to state 1 is 2μ. This is because when there are 2 broken

down machines the 2 operators are working each at rate μ. Thus, the rate of leaving from state 2 to 1 is 2μ. Because there are only 2 operators in this illustration, the maximum rate is 2μ for state 2-5.

Notice that the customer in this system is the machine. Even though the machines do not actually move to line up in a queue, they form a virtual queue for the operators to receive their repair. Let's consider an example of this system.

Example C.6. Suppose a manufacturing system contains 5 machines, each subject to randomly occurring breakdowns. A machine runs for an amount of time that is an exponential random variable with a mean of 10 hours before breaking down. At present there are 2 operators to fix the broken machines. The amount of time that an operator takes to service the machines is exponential with a mean of 4 hours. An operator repairs only 1 machine at a time. If more machines are broken down than the current number of operators, the machines must wait for the next available operator for repair. They form a FIFO queue to wait for the next available operator. The number of operators required to tend to the machines in order to minimize down time in a cost effective manner is desired. Assume that it costs the system \$60 per hour for each machine that is broken down. Each operator is paid \$15 per hour regardless of whether they are repairing a machine or not.

Solution to Example C.6

The arrival rate of an *individual machine* is $\lambda = 1/10$ per hour and the service rate of $\mu = 1/4$ per hour for each operator. In order to decide the most appropriate number of operators to tend the machines, a service

criteria or a way to measure the cost of the system is required. Since costs are given, let's formulate how much a given system configuration costs.

The easiest way to formulate a cost is to consider what a given system configuration costs on a per time basis, e.g. the cost per hour. Clearly, the system costs $15 \times c$ (\$/hour) to employ c operators. The problem also states that it costs \$60/hour for each machine that is broken down. A machine is broken down if it is waiting for an operator or if it is being repaired by an operator. Therefore a machine is broken down if it is in the queueing system.

In terms of queueing performance measures, L machines can be expected to be broken down at any time in steady state. Thus, the expected steady state cost of the broken down machines is $60 \times L$ per hour. The total expected cost per hour, $E[TC]$, of operating a system configuration in steady state is thus:

$$E[TC] = 60 \times L + 15 \times c$$

Thus, the total expected cost, $E[TC]$, can be evaluated for various values of c and the system that has the lowest cost determined.

Using the *QTSPlus* software, the necessary performance measures can be calculated and tabulated. Within the *QTSPlus* software you must choose the model category *Multiple Servers* and then choose the *Markov Multi-Server Finite Source Queue without Spares* model. Figure C.12 illustrates the inputs for the case of 1 server with 5 machines. Note that the software requests the time between arrivals, $1/lambda$ and the mean service time, $1/\mu$, rather than λ and μ.

Table C.3.: Tabulated Results for Example C.6

λ	0.10	0.10	0.10	0.10	0.10
μ	0.25	0.25	0.25	0.25	0.25
c	1	2	3	4	5

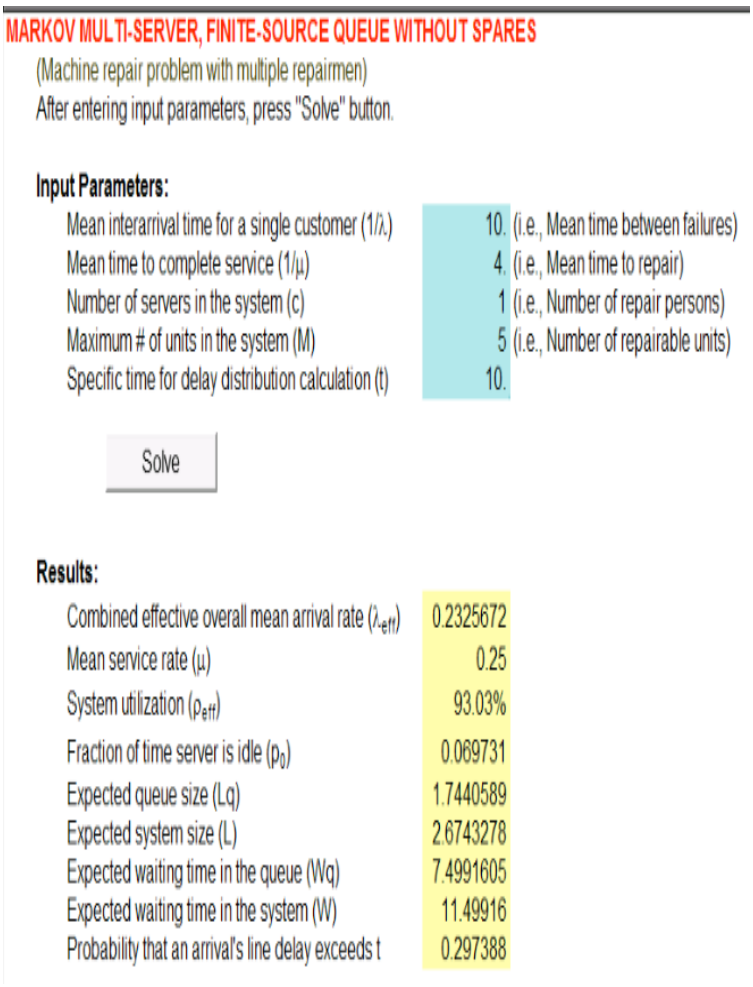
N	5	5	5	5	5
K	5	5	5	5	5
System	M/M/1/5/5	M/M/2/5/5	M/M/3/5/5	M/M/4/5/5	M/M/5/5/5
L	2.674	1.661	1.457	1.430	1.429
L_q	1.744	0.325	0.040	0.002	0.000
B	0.930	1.336	1.417	1.428	1.429
B/c	0.930	0.668	0.472	0.357	0.286
$E[TC]$	\$175.460	\$129.655	\$132.419	\$145.816	\$160.714
$\overline{MU}$	0.465	0.668	0.709	0.714	0.714

Table C.3 shows the results of the analysis. The results indicate that as the number of operators increase, the expected cost reaches its minimum at $c = 2$. As the number of operators is increased, the machine utilization increases but levels off. The machine utilization is the expected number of machines that are not broken down divided by the number of machines:

$$\text{Machine Utilization} = \overline{MU} = \frac{k - L}{k} = 1 - \frac{L}{k}$$

C.3. Non-Markovian Queues and Approximations

So far, the queueing models that have been analyzed all assume a Poisson arrival process and exponential service times. For other arrival and service processes only limited or approximate results are readily available. There are two cases worth mentioning here. The first case is the M/G/1 queue and the second case is an approximation for the GI/G/c queueing system. Recall that G represents any general distribution. In other words the results will hold regardless of the distribution. Also, GI refers to an arrival process which has the time between arrivals as independent and identically distributed random variables with any distribution. Table C.4 presents the basic results for the M/G/1 and M/D/1 queueing systems.



MARKOV MULTI-SERVER, FINITE-SOURCE QUEUE WITHOUT SPARES

(Machine repair problem with multiple repairmen)

After entering input parameters, press "Solve" button.

Input Parameters:

Mean interarrival time for a single customer ($1/\lambda$)	10.	(i.e., Mean time between failures)
Mean time to complete service ($1/\mu$)	4.	(i.e., Mean time to repair)
Number of servers in the system (c)	1	(i.e., Number of repair persons)
Maximum # of units in the system (M)	5	(i.e., Number of repairable units)
Specific time for delay distribution calculation (t)	10.	

Solve

Results:

Combined effective overall mean arrival rate (λ_{eff})	0.2325672
Mean service rate (μ)	0.25
System utilization (ρ_{eff})	93.03%
Fraction of time server is idle (p_0)	0.069731
Expected queue size (Lq)	1.7440589
Expected system size (L)	2.6743278
Expected waiting time in the queue (Wq)	7.4991605
Expected waiting time in the system (W)	11.49916
Probability that an arrival's line delay exceeds t	0.297388

Figure C.12.: Modeling the Machine Interference Problem in QTSPlus

Table C.4.: Results M/G/1 and M/D/1

Model	Parameters	L_q
M/G/1	$E[ST] = \frac{1}{\mu}$; $Var[ST] = \sigma^2$; $r = \lambda/\mu$	$L_q = \frac{\lambda^2\sigma^2 + r^2}{2(1-r)}$
M/D/1	$E[ST] = \frac{1}{\mu}$; $Var[ST] = 0$; $r = \lambda/\mu$	$L_q = \frac{r^2}{2(1-r)}$

For the M/G/1 model with a service distribution having a mean $E[ST] = 1/\mu$ and variance σ^2, the expected number in the system is:

$$L = \frac{\lambda^2\sigma^2 + r^2}{2(1-r)} + r$$

From this formula for the expected number in the system, the other performance measures can be obtained via Little's formula. Notice that only the mean and the variance of the service time distribution are necessary in this case.

For the case of the GI/G/c queue a number of approximations have been influenced by an approximation for the GI/G/1 queue that first appeared in (Kingman 1964). His single-server approximation is shown below:

$$W_q(GI/G/1) \approx \left(\frac{c_a^2 + c_s^2}{2}\right) W_q(M/M/1)$$

In this equation, W_q(M/M/1) denotes the expected waiting time in the queue for the M/M/1 model c_a^2 and c_s^2 and represent the squared coefficient of variation for the inter-arrival time and service time distributions. Recall that for a random variable, X, the squared coefficient of variation is given by $c_X^2 = Var[X]/(E[X])^2$. (Whitt 1983) used a very similar approximation

for the GI/G/c queue to compute the traffic congestion at each node in a queueing network for his Queueing Network Analyzer:

$$W_q(GI/G/c) \approx \left(\frac{c_a^2 + c_s^2}{2}\right) W_q(M/M/c)$$

A discussion of queueing approximations of this form as well as additional references can be found in (Whitt 1993).

Thus, to approximate the performance of a GI/G/c queue, you need only the first two moments of the inter-arrival and service time distributions and a way to compute the waiting time in the queue for a M/M/c queueing system. These results are useful when trying to verify and validate a simulation model of a queueing system, especially in the case of a system that consists of more than one queueing system organized into a network. Before examining that more complicated case, some of the issues related to using to simulate single queue systems should be examined.

The following section summarizes the formulas for the previously mentioned queueing systems. The appendix is finished off with some example exercises that can be used to test your understanding of the application of these formulas.

C.4. Summary of Queueing Formulas

This section provides the formulas for basic single queue stations and is meant simply as a resource where the formulas are readily available for potential application. The following notation is used within this section.

Let N represent the steady state number of customers in the system, where $N \in \{0, 1, 2, ..., k\}$ where k is the maximum number of customers in the system and may be infinite (∞).

Let λ_n be the arrival rate when there are $N = n$ customers in the system.

Let μ_n be the service rate when there are $N = n$ customers in the system.

Let $P_n = P[N = n]$ be the probability that there are n customers in the system in steady state.

When λ_n is constant for all n, we write $\lambda_n = \lambda$.

When μ_n is constant for all n, we write $\mu_n = \mu$.

Let λ_e be the effective arrival rate for the system, where

$$\lambda_e = \sum_{n=0}^{\infty} \lambda_n P_n$$

Since $\lambda_n = 0$ for $n \geq k$ for a finite system size, k, we have:

$$\lambda_e = \sum_{n=0}^{k-1} \lambda_n P_n$$

Let $\rho = \frac{\lambda}{c\mu}$ be the utilization.

Let $r = \frac{\lambda}{\mu}$ be the offered load.

C.4.1. M/M/1 Queue

$$\begin{aligned} \lambda_n &= \lambda \\ \mu_n &= \mu \\ r &= \lambda/\mu \end{aligned}$$

$$\begin{aligned} P_0 &= 1 - r \\ P_n &= P_0 r^n \\ L_q &= \frac{r^2}{1-r} \end{aligned}$$

Table C.5.: Results M/G/1 and M/D/1

Model	Parameters	L_q
M/G/1	$E[ST] = \frac{1}{\mu}$; $Var[ST] = \sigma^2$; $r = \lambda/\mu$	$L_q = \frac{\lambda^2\sigma^2 + r^2}{2(1-r)}$
M/D/1	$E[ST] = \frac{1}{\mu}$; $Var[ST] = 0$; $r = \lambda/\mu$	$L_q = \frac{r^2}{2(1-r)}$

C.4.2. M/M/c Queue

$$\lambda_n = \lambda$$
$$\mu_n = \begin{cases} n\mu & 0 \leq n < c \\ c\mu & n \geq c \end{cases}$$
$$\rho = \lambda/c\mu \quad r = \lambda/\mu$$

$$P_0 = \left[\sum_{n=0}^{c-1} \frac{r^n}{n!} + \frac{r^c}{c!(1-\rho)}\right]^{-1}$$
$$L_q = \left(\frac{r^c\rho}{c!(1-\rho)^2}\right)P_0$$

$$P_n = \begin{cases} \frac{(r^n)^2}{n!}P_0 & 1 \leq n < c \\ \frac{r^n}{c!c^{n-c}}P_0 & n \geq c \end{cases}$$

Table C.6.: Results M/M/c $\rho = \lambda/c\mu$

c	P_0	L_q
1	$1-\rho$	$\frac{\rho^2}{1-\rho}$
2	$\frac{1-\rho}{1+\rho}$	$\frac{2\rho^3}{1-\rho^2}$
3	$\frac{2(1-\rho)}{2+4\rho+3\rho^2}$	$\frac{9\rho^4}{2+2\rho-\rho^2-3\rho^3}$

C.4.3. M/M/c/k Queue

$$\lambda_n = \begin{cases} \lambda & n < k \\ 0 & n \geq k \end{cases}$$

$$\mu_n = \begin{cases} n\mu & 0 \leq n < c \\ c\mu & c \leq n \leq k \end{cases}$$

$$\rho = \lambda/c\mu \quad r = \lambda/\mu$$

$$\lambda_e = \lambda(1 - P_k)$$

$$P_0 = \begin{cases} \left[\sum_{n=0}^{c-1} \frac{r^n}{n!} + \frac{r^c}{c!}\frac{1-\rho^{k-c+1}}{1-\rho}\right]^{-1} & \rho \neq 1 \\ \left[\sum_{n=0}^{c-1} \frac{r^n}{n!} + \frac{r^c}{c!}(k-c+1)\right]^{-1} & \rho = 1 \end{cases}$$

$$P_n = \begin{cases} \frac{r^n}{n!}P_0 & 1 \leq n < c \\ \frac{r^n}{c!c^{n-c}}P_0 & c \leq n \leq k \end{cases}$$

C.4.4. M/G/c/c Queue

$$\lambda_n = \begin{cases} \lambda & n < c \\ 0 & n \geq c \end{cases}$$

$$\mu_n = \begin{cases} n\mu & 0 \leq n \leq c \\ 0 & n > c \end{cases}$$

$$\rho = \lambda / c\mu \quad r = \lambda / \mu$$

$$\lambda_e = \lambda(1 - P_k)$$

$$P_0 = \left[\sum_{n=0}^{c} \frac{r^n}{n!} \right]^{-1}$$

$$P_n = \frac{r^n}{n!} P_0$$

$$0 \leq n \leq c$$

$$L_q = 0$$

C.4.5. M/M/1/k Queue

$$\lambda_n = \begin{cases} (k-n)\lambda & 0 \leq n < k \\ 0 & n \geq k \end{cases}$$

$$\mu_n = \begin{cases} (k-n)\lambda & 0 \leq n \leq k \\ 0 & n > k \end{cases}$$

$$r = \lambda / \mu \quad \lambda_e = \lambda(k - L)$$

$$P_0 = \left[\sum_{n=0}^{k} \prod_{j=0}^{n-1} \left(\frac{\lambda_j}{\mu_{j+1}} \right) \right]^{-1}$$

$$P_n = \binom{k}{n} n! r^n P_0$$

$$L_q = \begin{cases} \frac{\rho}{1-\rho} - \frac{\rho(k\rho^k+1)}{1-\rho^{k+1}} & \rho \neq 1 \\ \frac{k(k-1)}{2(k+1)} & \rho = 1 \end{cases}$$

C.4.6. M/M/c/k Queue

$$\lambda_n = \begin{cases} (k-n)\lambda & 0 \leq n < k \\ 0 & n \geq k \end{cases}$$

$$\mu_n = \begin{cases} n\mu & 0 \leq n < c \\ c\mu & n \geq c \end{cases}$$

$$r = \lambda/\mu \quad \lambda_e = \lambda(k-L)$$

$$P_0 = \left[\sum_{n=0}^{k}\prod_{j=0}^{n-1}(\frac{\lambda_j}{\mu_{j+1}})\right]^{-1}$$

$$P_n = \begin{cases} \binom{k}{n} r^n P_0 & 1 \leq n < c \\ \binom{k}{n} \frac{n!}{c^{n-c}c!} r^n P_0 & c \leq n \leq k \end{cases}$$

$$L_q = \begin{cases} \frac{P_0 r^c \rho}{c!(1-\rho)^2}[1-\rho^{k-c}-(k-c)\rho^{k-c}(1-\rho)] & \rho < 1 \\ \frac{r^c(k-c)(k-c+1)}{2c!}P_0 & \rho = 1 \end{cases}$$

C.4.7. M/M/1/k/k Queue

$$\lambda_n = \begin{cases} (k-n)\lambda & 0 \le n < k \\ 0 & n \ge k \end{cases}$$

$$\mu_n = \begin{cases} (k-n)\lambda & 0 \le n \le k \\ 0 & n > k \end{cases}$$

$$r = \lambda/\mu \quad \lambda_e = \lambda(k - L)$$

$$P_0 = \left[\sum_{n=0}^{k}\prod_{j=0}^{n-1}\left(\frac{\lambda_j}{\mu_{j+1}}\right)\right]^{-1}$$

$$P_n = \binom{k}{n} n! r^n P_0$$

$$0 \le n \le k$$

$$L_q = k - \left(\frac{\lambda + \mu}{\lambda}\right)(1 - P_0)$$

C.4.8. M/M/c/k/k Queue

$$\lambda_n = \begin{cases} (k-n)\lambda & 0 \le n < k \\ 0 & n \ge k \end{cases}$$

$$\mu_n = \begin{cases} n\mu & 0 \le n < c \\ c\mu & n \ge c \end{cases}$$

$$r = \lambda/\mu \quad \lambda_e = \lambda(k - L)$$

$$P_0 = \left[\sum_{n=0}^{k}\prod_{j=0}^{n-1}\left(\frac{\lambda_j}{\mu_{j+1}}\right)\right]^{-1}$$

$$P_n = \begin{cases} \binom{k}{n} r^n P_0 & 1 \leq n < c \\ \binom{k}{n} \frac{n!}{c^{n-c} c!} r^n P_0 & c \leq n \leq k \end{cases}$$

$$L_q = \sum_{n=c}^{k} (n-c) P_n$$

C.5. Exercises

For the exercises in this section, first start with specifying the appropriate queueing models needed to solve the exercise using Kendall's notation. Then, specify the parameters of the model, e.g. λ_e, μ, c, size of the population, size of the system, etc. Specify how and what you would compute to solve the problem. Be as specific as possible by specifying the equations needed. Then, compute the quantities if requested. You might also try to use to solve the problems via simulation.

Exercise C.1. *True* or *False*: In a queueing system with random arrivals and random service times, the performance will be best if the arrival rate is equal to the service rate because then there will not be any queueing.

Exercise C.2. The Burger Joint in the UA food court uses an average of 10,000 pounds of potatoes per week. The average number of pounds of potatoes on hand is 5,000. On average, how long do potatoes stay in the restaurant before being used? What queueing concept is use to solve this problem?

Exercise C.3. Consider a single pump gas station where the arrival process is Poisson with a mean time between arrivals of 10 minutes. The service time is exponentially distributed with a mean of 6 minutes. Specify the appropriate queueing model needed to solve the problem using Kendall's notation. Specify the parameters of the model and what you would compute to solve the problem. Be as specific as possible by specifying the equation needed. Then, compute the desired quantities.

a. What is the probability that you have to wait for service?

b. What is the mean number of customer at the station?

c. What is the expected time waiting in the line to get a pump?

Exercise C.4. Suppose an operator has been assigned to the responsibility of maintaining 3 machines. For each machine the probability distribution of the running time before a breakdown is exponentially distributed with a mean of 9 hours. The repair time also has an exponential distribution with a mean of 2 hours. Specify the appropriate queueing model needed to solve the problem using Kendall's notation. Specify the parameters of the model and what you would compute to solve the problem. Be as specific as possible by specifying the equation needed. Then, compute the desired quantities.

a. What is the probability that the operator is idle?

b. What is the expected number of machines that are running?

c. What is the expected number of machines that are not running?

Exercise C.5. SuperFastCopy wants to install self-service copiers, but cannot decide whether to put in one or two machines. They predict that arrivals will be Poisson with a rate of 30 per hour, and the time spent copying is exponentially distributed with a mean of 1.75 minutes. Because the shop is small they want the probability of 5 or more customers in the shop to be small, say less than 7%. Make a recommendation based on queueing theory to SuperFastCopy.

Exercise C.6. Each airline passenger and his or her carry-on baggage must be checked at the security checkpoint. Suppose XNA averages 10 passengers per minute with exponential inter-arrival times. To screen passengers, the airport must have a metal detector and baggage X-ray machines. Whenever a checkpoint is in operation, two employees are required (one operates the metal detector, one operates the X-ray machine). The passenger goes through the metal detector and simultaneously their bag goes through the X-ray machine. A checkpoint can check an average of 12 passengers per minute according to an exponential distribution.

What is the probability that a passenger will have to wait before being screened? On average, how many passengers are waiting in line to enter the checkpoint? On average, how long will a passenger spend at the checkpoint?

Exercise C.7. Two machines are being considered for processing a job within a factory. Jobs arrive at the machine according to a Poisson process with a rate of 4.5 jobs per hour. The first machine has an exponentially distributed processing time with a mean of 10 minutes. For the second machine the vendor has indicated that the mean processing time is 10 minutes but with a standard deviation of 6 minutes. Using queueing theory, which machine is better in terms of the average waiting time of the jobs?

Exercise C.8. Customers arrive at a one-window drive in bank according to a Poisson distribution with a mean of 10 per hour. The service time for each customer is exponentially distributed with a mean of 5 minutes. There are 3 spaces in front of the window including that for the car being served. Other arriving cars can wait outside these 3 spaces. Specify the appropriate queueing model needed to solve the problem using Kendall's notation. Specify the parameters of the model and what you would compute to solve the problem. Be as specific as possible by specifying the equation needed. Then, compute the desired quantities.

a. What is the probability that an arriving customer can enter one of the 3 spaces in front of the window?

b. What is the probability that an arriving customer will have to wait outside the 3 spaces?

c. How long is an arriving customer expected to wait before starting service?

d. How many spaces should be provided in front of the window so that an arriving customer can wait in front of the window at least 20% of the time? In other words, the probability of at least one open space must be greater than 20%.

Exercise C.9. Joe Rose is a student at Big State U. He does odd jobs to supplement his income. Job requests come every 5 days on the average, but the time between requests is exponentially distributed. The time for completing a job is also exponentially distributed with a mean of 4 days.

a. What would you compute to find the chance that Joe will not have any jobs to work on?

b. What would you compute to find the average value of the waiting jobs if Joe gets about $25 per job?

Exercise C.10. The manager of a bank must determine how many tellers should be available. For every minute a customer stands in line, the manager believes that a delay cost of 5 cents is incurred. An average of 15 customers per hour arrive at the bank. On the average, it takes a teller 6 minutes to complete the customer's transaction. It costs the bank $9 per hour to have a teller available. Inter-arrival and service times can be assumed to be exponentially distributed.

What is the minimum number of tellers that should be available in order for the system to be stable (i.e. not have an infinite queue)? If the system has 3 tellers, what is the probability that there will be no one in the bank? What is the expected total cost of the system per hour, when there are 2 tellers?

Exercise C.11. You have been hired to analyze the needs for loading dock facilities at a trucking terminal. The present terminal has 4 docks on the main building. Any trucks that arrive when all docks are full are assigned to a secondary terminal, which is a short distance away from the main terminal. Assume that the arrival process is Poisson with a rate of 5 trucks each hour. There is no available space at the main terminal for trucks to wait for a dock. At the present time nearly two-thirds of the arriving trucks are diverted to the secondary terminal. The average service time per truck is two hours on the main terminal and 3 hours on the secondary terminal, both exponentially distributed. Two proposals are being considered. The first proposal is to expand the main terminal by adding docks so that at least 80% of the arriving trucks can be served

there with the remainder being diverted to the secondary terminal. The second proposal is to expand the space that can accommodate up to 8 trucks. Then, only when the holding area is full will the trucks be diverted to secondary terminal.

What queueing model should you use to analyze the first proposal? State the model and its parameters. State what you would do to determine the required number of docks so that at least 80% of the arriving trucks can be served for the first proposal. Note you do not have to compute anything. What model should you use to analyze the 2nd proposal? State the model and its parameters.

Exercise C.12. Sly's convenience store operates a two-pump gas station. The lane leading to the pumps can house at most five cars, including those being serviced. Arriving cars go elsewhere if the lane is full. The distribution of the arriving cars is Poisson with a mean of 20 per hour. The time to fill up and pay for the purchase is exponentially distributed with a mean of 6 minutes.

a. Specify using queueing notation, exactly what you would compute to find the percentage of cars that will seek business elsewhere?

b. Specify using queueing notation, exactly what you would compute to find the utilization of the pumps?

Exercise C.13. An airline ticket office has two ticket agents answering incoming phone calls for flight reservations. In addition, two callers can be put on hold until one of the agents is available to take the call. If all four phone lines (both agent lines and the hold lines) are busy, a potential customer gets a busy signal, and it is assumed that the call goes to another

ticket office and that the business is lost. The calls and attempted calls occur randomly (i.e. according to Poisson process) at a mean rate of 15 per hour. The length of a telephone conversation has an exponential distribution with a mean of 4 minutes.

a. Specify using queueing notation, exactly what you would compute to find the probability of losing a potential customer?

b. What would you compute to find the probability that an arriving phone call will not start service immediately but will be able to wait on a hold line?

Exercise C.14. SuperFastCopy has three identical copying machines. When a machine is being used, the time until it breaks down has an exponential distribution with a mean of 2 weeks. A repair person is kept on call to repair the machines. The repair time for a machine has an exponential distribution with a mean of 0.5 week. The downtime cost for each copying machine is $100 per week.

a. Let the state of the system be the number of machines not working, Construct a state transition diagram for this queueing system.

b. Write an expression using queueing performance measures to compute the expected downtime cost per week.

Exercise C.15. NWH Cardiac Care Unit (CCU) has 5 beds, which are virtually always occupied by patients who have just undergone major heart surgery. Two registered nurses (RNs) are on duty in the CCU in each of the three 8 hour shifts. Each patient requires a nurse's attention about every two hours, following an exponential distribution. The RN will then spend

an average of 30 minutes (exponentially distributed) assisting the patient and updating medical records regarding the problem and care provided.

a. What would you compute to find the average number of patients being attended by the nurses?

b. What would you compute to fine the average time that a patient spends waiting for one of the nurses to arrive?

Exercise C.16. HJ Bunt, Transport Company maintains a large fleet of refrigerated trailers. For the purposes of this problem assume that the number of refrigerated trailers is conceptually infinite. The trailers require service on an irregular basis in the company owned and operated service shop. Assume that the arrival of trailers to the shop is approximated by a Poisson distribution with a mean rate of 3 per week. The length of time needed for servicing a trailer varies according to an exponential distribution with a mean service time of one-half week per trailer. The current policy is to utilize a centralized contracted outsourced service center whenever more than two trailers are in the company shop, so that, at most one trailer is allowed to wait. Assume that there is currently one 1 mechanic in the company shop.

Specify using Kendall's notation the correct queueing model for this situation including the appropriate parameters. What would you compute to determine the expected number of repairs that are outsourced per week?

Exercise C.17. Rick is a manager of a small barber shop at Big State U. He hires one barber. Rick is also a barber and he works only when he has more than one customer in the shop. Customers arrive randomly at a rate of 3 per hour. Rick takes 15 minutes on the average for a hair cut, but his

employee takes 10 minutes. Assume that the cutting time distributions are exponentially distributed. Assume that there are only 2 chairs available with no waiting room in the shop.

a. Let the state of the system be the number of customers in the shop, Construct a state transition diagram for this queueing system.

b. What is the probability that a customer is turned away?

c. What is the probability that the barber shop is idle?

d. What is the steady-state mean number of customers in the shop?

Exercise C.18. Using the supplied data set, draw the sample path for the state variable, $N(t)$. Give a formula for estimating the time average number in the system, $N(t)$, and then use the data to compute the time average number in the system over the range from 0 to 25. Assume that the value of $N(t$ is the value of the state variable just after time t.

t	0	2	4	5	7	10	12	15	20
$N(t)$	0	1	0	1	2	3	2	1	0

a. Give a formula for estimating the time average number in the system, $N(t)$, and then use the data to compute the time average number in the system over the range from 0 to 25.

b. Give a formula for estimating the mean rate of arrivals over the interval from 0 to 25 and then use the data to estimate the mean arrival rate.

c. Estimate the average time in the system (waiting and in service) for the customers indicated in the diagram.

d. What queueing formula relationship is used in this problem?

D. Distributions

D.1. Discrete Distrbutions

Bernoulli	$Ber(p)$
Parameters:	$0 < p < 1$, probability of success
PMF:	$P[X = 1] = p,\ P[X = 0] = 1 - p$
Inverse CDF:	$F^{-1}(u) =$ if $(u < p), 1$ else 0
Expected Value:	$E[X] = p$
Variance:	$Var[X] = p(1 - p)$
Generation:	BernoulliRV(probOfSuccess:Double, stream:RNStreamIf)
Spreadsheet Generation:	= IF(RAND()< p, 1, 0)
Modeling:	the number of successes in one trial

Binomial	$Binom(n, p)$
Parameters:	$0 < p < 1$, probability of success, n, number of trials
PMF:	$P[X = x] = \binom{n}{x} p^x (1 - p)^{n-x} \quad x = 0, 1, \ldots, n$
Inverse CDF:	no closed form available
Expected Value:	$E[X] = np$
Variance:	$Var[X] = np(1 - p)$
Generation:	BinomialRV(pSuccess:Double, numTrials:Int, stream:RNStreamIf)
Spreadsheet Generation:	= BINOM.INV(n,p,RAND())
Modeling:	the number of successes in n trials

Shifted Geometric	Shifted Geo(p)
Parameters:	$0 < p < 1$, probability of success
PMF:	$P[X = x] = p(1-p)^{x-1} \quad x = 1, 2, \ldots,$
Inverse CDF	$F^{-1}(u) = 1 + \left\lfloor \frac{ln(1-u)}{ln(1-p)} \right\rfloor$
Expected Value:	$E[X] = 1/p$
Variance:	$Var[X] = (1-p)/p^2$
Generation:	ShiftedGeometricRV(probOfSuccess:Double, stream:RNStreamIf)
Spreadsheet Generation:	= 1 + INT(LN(1-RAND())/LN(1-p))
Modeling:	the number of trials until the first success

Negative Binomial Defn. 1	NB1(r, p)
Parameters:	$0 < p < 1$, probability of success, r^{th} success
PMF:	$P[X = x] = \binom{x-1}{r-1} p^r (1-p)^{x-r} \quad x = r, r+1 \ldots,$
Inverse CDF:	no closed form available
Expected Value:	$E[X] = r/p$
Variance:	$Var[X] = r(1-p)/p^2$
Generation:	NegativeBinomialRV(probOfSuccess:Double, numSuccess: Int, strea
Spreadsheet Generation:	use convolution of shifted geometric
Modeling:	the number of trials until the r^{th} success

Negative Binomial Defn. 2	NB2(r, p)
Parameters:	$0 < p < 1$, probability of success, r^{th} success
PMF:	$P[Y = y] = \binom{y+r-1}{r-1} p^r (1-p)^y \quad y = 0, 1, \ldots$
Inverse CDF:	no closed form available
Expected Value:	$E[Y] = r(1-p)/p$
Variance:	$Var[Y] = r(1-p)/p^2$
Generation:	value of NegativeBinomialRV minus 1
Spreadsheet Generation:	use convolution of geometric
Modeling:	the number of failures prior to the r^{th} success

Poisson	Pois(λ)
Parameters:	$\lambda > 0$
PMF:	$P[X = x] = \frac{e^{-\lambda}\lambda^x}{x!} \quad x = 0, 1, ...$
Inverse CDF:	no closed form available
Expected Value:	$E[X] = \lambda$
Variance:	$Var[X] = \lambda$
Generation:	PoissonRV(mean: Double, stream:RNStreamIfc)
Spreadsheet Generation:	not available, approximate with lookup table approach
Modeling:	the number of occurrences during a period of time

Discrete Uniform	DU(a, b)
Parameters:	$a \leq b$
PMF:	$P[X = x] = \frac{1}{b-a+1} \quad x = a, a+1, ..., b$
Inverse CDF:	$F^{-1}(u) = a + \lfloor (b - a + 1)u \rfloor$
Expected Value:	$E[X] = (b + a)/2$
Variance:	$Var[X] = \left((b - a + 1)^2 - 1\right)/12$
Generation:	DUniformRV(min: Int, max:Int, stream:RNStreamIfc)
Spreadsheet Generation:	=RANDBETWEEN(a,b)
Modeling:	equal occurrence over a range of integers

D.2. Continuous Distrbutions

Uniform	$U(a, b)$
Parameters:	a = minimum, b = maximum, $-\infty < a < b < \infty$
PDF:	$f(x) = \frac{1}{b-a}$ for $a \leq x \leq b$
CDF:	$F(x) = \frac{x-a}{b-a}$ if $a \leq x \leq b$
Inverse CDF:	$F^{-1}(p) = a + p(b - a)$ if $0 < p < 1$
Expected Value:	$E[X] = \frac{a+b}{2}$

Uniform	$U(a, b)$
Variance:	$V[X] = \frac{(b-a)^2}{12}$
Generation:	UniformRV(min: Double, max:Double, stream:RNStreamIfc)
Spreadsheet Generation:	= a + RAND()*(b-a)
Modeling:	assumes equally likely across the range, when you have lack of data, task times

Normal	$N(\mu, \sigma^2)$
Parameters:	$-\infty < \mu < +\infty$ (mean), $\sigma^2 > 0$ (variance)
CDF:	No closed form
Inverse CDF:	No closed form
Expected Value:	$E[X] = \mu$
Variance:	$Var[X] = \sigma^2$
Generation:	NormalRV(mean:Double, variance:Double, stream:RNStreamIfc)
Spreadsheet Generation:	= NORM.INV(RAND(), μ, σ)
Modeling:	task times, errors

Exponential	EXPO($1/\lambda$)
Parameters:	$\lambda > 0$
PDF:	$f(x) = \lambda e^{-\lambda x}$ if $x \geq 0$
CDF:	$F(x) = 1 - e^{-\lambda x}$ if $x \geq 0$
Inverse CDF:	$F^{-1}(p) = (-1/\lambda) \ln (1 - p)$ if $0 < p < 1$
Expected Value:	$E[X] = \theta = 1/\lambda$
Variance:	$Var[X] = 1/\lambda^2$
Generation:	ExponentialRV(mean:Double, stream:RNStreamIfc)
Spreadsheet Generation:	= $(-1/\lambda)$LN(1-RAND())
Modeling:	time between arrivals, time to failure highly variable task time

Weibull	WEIB(β, α)
Parameters:	$\beta > 0$ (scale), $\alpha > 0$ (shape)
CDF:	$F(x) = 1 - e^{-(x/\beta)^\alpha}$ if $x \geq 0$
Inverse CDF:	$F^{-1}(p) = \beta\left[-\ln(1-p)\right]^{1/\alpha}$ if $0 < p < 1$
Expected Value:	$E[X] = \left(\frac{\beta}{\alpha}\right)\Gamma\left(\frac{1}{\alpha}\right)$
Variance:	$Var[X] = \left(\frac{\beta^2}{\alpha}\right)\left\{2\Gamma\left(\frac{2}{\alpha}\right) - \left(\frac{1}{\alpha}\right)\left(\Gamma\left(\frac{1}{\alpha}\right)\right)^2\right\}$
Generation:	WeibullRV(shape:Double, scale:Double, stream:RNStreamIfc)
Spreadsheet Generation:	$= (\beta)(-\text{LN}(1 - \text{RAND}()) \wedge (1/\alpha)$
Modeling:	task times, time to failure

Erlang	Erlang(r,β)
Parameters:	$r > 0$, integer, $\beta > 0$ (scale)
CDF:	$F(x) = 1 - e^{(-x/\beta)} \sum_{j=0}^{r-1} \frac{(x/\beta)^j}{j!}$ if $x \geq 0$
Inverse CDF:	No closed form
Expected Value:	$E[X] = r\beta$
Variance:	$Var[X] = r\beta^2$
Generation:	use GammaRV class
Spreadsheet Generation:	= GAMMA.INV(RAND(), r, β)
Modeling:	task times, lead time, time to failure,

Gamma	Gamma(α,β)
Parameters:	$\alpha > 0$, shape, $\beta > 0$ (scale)
CDF:	No closed form
Inverse CDF:	No closed form
Expected Value:	$E[X] = \alpha\beta$
Variance:	$Var[X] = \alpha\beta^2$
Generation:	GammaRV(shape:Double, scale:Double, stream:RNStreamIfc)

Gamma	Gamma(α,β)
Spreadsheet Generation:	= GAMMA.INV(RAND(), α, β)
Modeling:	task times, lead time, time to failure,

Beta	BETA(α_1,α_2)
Parameters:	shape parameters $\alpha_1 > 0$, $\alpha_2 > 0$
CDF:	No closed form
Inverse CDF:	No closed form
Expected Value:	$E[X] = \dfrac{\alpha_1}{\alpha_1 + \alpha_2}$
Variance:	$Var[X] = \dfrac{\alpha_1 \alpha_2}{(\alpha_1 + \alpha_2)^2(\alpha_1 + \alpha_2 + 1)}$
Generation:	BetaRV(alpha1:Double, alpha2:Double, stream:RNStreamIfc)
Spreadsheet Generation:	BETA.INV(RAND(), α_1, α_2)
Modeling:	activity time when data is limited, probabilities

Lognormal	LOGN(μ_l, σ_l)
Parameters:	$\mu = \ln\left(\mu_l^2 / \sqrt{\sigma_l^2 + \mu_l^2}\right) \quad \sigma^2 = \ln\left((\sigma_l^2/\mu_l^2) + 1\right)$
CDF:	No closed form
Inverse CDF:	No closed form
Expected Value:	$E[X] = \mu_l = e^{\mu + \sigma^2/2}$
Variance:	$Var[X] = \sigma_l^2 = e^{2\mu+\sigma^2}\left(e^{\sigma^2} - 1\right)$
Generation:	LognormalRV(mean:Double, variance:Double, stream:RNStreamIfc)
Spreadsheet Generation:	LOGNORM.INV(RAND(), μ, σ)
Modeling:	task times, time to failure

Triangular	TRIA(a, m, b)
Parameters:	a = minimum, m = mode, b = maximum

Triangular	TRIA(a, m, b)
CDF:	$F(x) = \frac{(x-a)^2}{(b-a)(m-a)}$ for $a \le x \le m$
	$F(x) = 1 - \frac{(b-x)^2}{(b-a)(b-m)}$ for $m < x \le b$
Inverse CDF:	$F^{-1}(u) = a + \sqrt{(b-a)(m-a)u}$ for $0 < u < \frac{m-a}{b-a}$
	$F^{-1}(u) = b - \sqrt{(b-a)(b-m)(1-u)}$ for $\frac{m-a}{b-a} \le u$
Expected Value:	$E[X] = (a+m+b)/3$
Variance:	$Var[X] = \frac{a^2+b^2+m^2-ab-am-bm}{18}$
Generation:	TriangularRV(min:Double, mode:Double, max:Double, stream:RNStreamI
Spreadsheet Generation:	implement $F^{-1}(u)$ as VBA function
Modeling:	task times, activity time when data is limited

E. KSL Utility Packages

Learning Objectives

- To be able to use the classes and functionality associated with KSL utilities
- To be able to use the KSL file input and output related utilities for CSV, Excel, data frames, and databases
- To be able to use the KSL array related utilities
- To be able to use the KSL plotting functionality
- To be able to use the KSL functionality for performing designed experiments

The purpose of this appendix is to describe the general utilities available within the KSL. The utilities are meant to assist programmers in using Kotlin and especially the use of the simulation constructs available within the KSL. The package `ksl.utilities` hold a wide variety of classes that support both the programmer and the developer of simulation models. The important items to be discussed in this appendix include:

- The `ksl.utilities.io` package, which provides support mostly for file input and output some of which, e.g. the `MarkDown` and `StatisticReporter` classes have already been mentioned.
- The `KSLArrays` and `RArrays` classes provide support for working with 1-D and 2-D arrays, including a number of extension functions that are useful for computational work and simulation related activities.

- Miscellaneous utilities involving observers, math, and useful interfaces and classes that facilitate working within the KSL

> **Note**
>
> This chapter provides illustrative examples of some of the functionality of the `ksl.utilities` package. The full source code of the examples can be found in the accompanying `KSLExamples` project associated with the KSL repository. The files for each example of this chapter can be found here.

We will start by looking at the utilities for input and output.

E.1. The `OutputDirectory` Class and `KSL` Object

We often need to get data into a program or to write data out. Because of the plethora of file formats and differing methods for creating and using files, the KSL provides some utilities for working with comma separated value (CSV), Excel, Markdown, text, and database files. The most relevant classes include `OutputDirectory` class, the `KSL` *object* and the `KSLFileUtil` class. Figure E.1 illustrates the functions and properties of the `KSL` object and the `OutputDirectory` class.

The `OutputDirectory` class is an abstraction for a file directory to store output. When working with a particular simulation model, it is useful to store all of the results and files generated by the model in one directory. The `OutputDirectory` class facilitates this common use case. An instance of `OutputDirectory` requires a path to the file directory and then forms some standard sub-directories (`excelDir`, `dbDir`, `csvDir`, `outDir`) to hold various files that may be generated for these common types of files. The `KSL` object is, in essence, a default directory to hold all KSL output. Notice that `OutputDirectory` provides functions for creating files and directories within it. The most useful of the functions is the

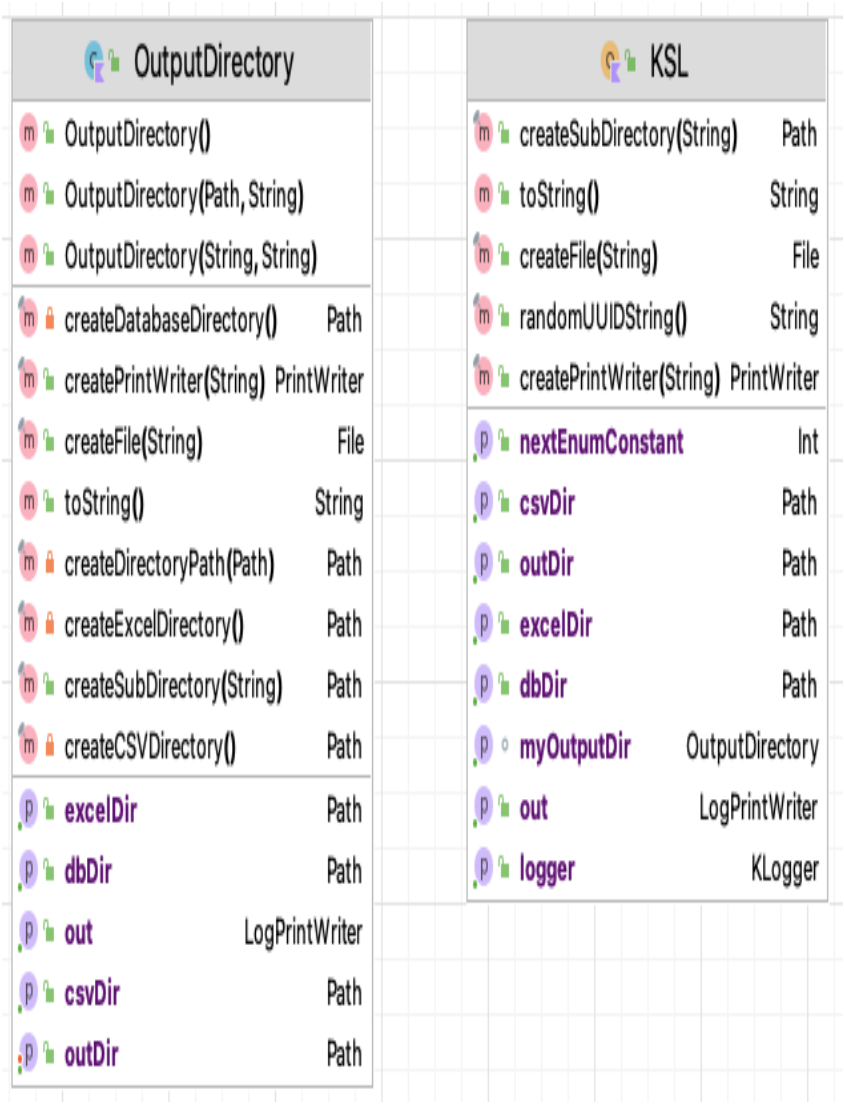


Figure E.1.: OutputDirectory and KSL

`createPrintWriter(name: String)` function which takes in the name of the file and creates a `PrintWriter` set up to write to the file. Also, notice the `createSubDirectory()` function. The directories created by this function are relative to the defined output directory. This alleviates the burden of fully specifying path strings and working with paths.

Here is some sample code that uses the `OutputDirectory` class.

```
    // get the working directory
    val path = Paths.get("").toAbsolutePath()
    println("Working Directory = $path")
    // creates a directory called TestOutputDir in the
     ↪  current working directory
    // Creates subdirectories: csvDir, dbDir, excelDir and
     ↪  file out.txt
    val outDir =
     ↪  OutputDirectory(path.resolve("TestOutputDir"))
    // write to the default file
    outDir.out.println("Use out property to write out text to
↪   a file.")
    // Creates a PrintWriter (and file) to write to within
     ↪  TestOutputDir
    val pw = outDir.createPrintWriter("PW_File.txt")
    pw.println("Hello, World")
    val subDir = outDir.createSubDirectory("SubDir")
    println(outDir)
```

The `KSL` object is essentially a predefined instance of `OutputDirectory` that creates a default directory called `kslOutput` within the current working directory. The property `out` is an instance of a `LogPrintWriter`, which is a class that wraps a `PrintWriter` but also has a property called `OUTPUT_ON`, which has type boolean, where true indicates that output will be written and false turns off the output. You can use this to stop excessive print

messages globally. I find this useful for simple debugging messages. Besides this field, `KSL` has a standard logger for logging program messages.

```
    // use KSL like you use OutputDirectory but with some
     ↪  added functionality
    // The PW_File.txt file with be within the kslOutput
     ↪  directory within the working directory
    val pw = KSL.createPrintWriter("PW_File.txt")
    pw.println("Hello, World!")
    // Creates subdirectory SubDir within the kslOutput
     ↪  directory
    KSL.createSubDirectory("SubDir")
    // use KSL.out to write to kslOutput.txt
    KSL.out.println("Information written into kslOutput.txt")
    println(KSL)
    // KSL also has logger. This logs to logs/ksl.log
    KSL.logger.info { "This is an informational log comment!"
↪   }
```

E.2. Logging Options

Logging is controlled by a logback XML file for configuring the loggers. The `logback.xml` file can be found in the resources folder within the source code repository. The more interesting classes that have default loggers written to the logs directory include:

- `KSL` useful for general purpose logging and written to `ksl.log` with default level DEBUG.
- `DatabaseIfc` captures database interaction, written to `kslDbLog.log` with default level INFO.
- `ExcelUtil` captures Excel file interactions, written to `kslExcelLog.log` with default level INFO.

- `KSLFileUtil` captures file interactions, written to `ksl_io.log` with default level INFO.
- `Model` captures key model actions as the model is processed, written to `kslSim.log` with default level of INFO.
- `ModelElement` captures detailed actions related to model element processing, written to `kslModelElement.log` with default level INFO.
- `ProcessModel` captures detailed actions related to entity processing, written to `kslEntity.log` with default level INFO. Very detailed entity tracing can be achieved by setting the log level to TRACE.
- `RNStreamProvider` captures random number stream assignments, written to `kslStreams.log` with default level INFO.
- `Controls` captures simulation control assignment, written to `controlsFile.log` with default level INFO.

For diagnosing issues related to these classes, you can change the debug level within the `logback.xml` file. Be careful with setting the `ProcessModel` logger to trace because the generated files will be large and the execution time of the model will be longer because of the extra IO.

E.3. The `KSLFileUtil` Object

The `KSLFileUtil` Object supports the creation/deletion of files, directories, and working with file extensions. It also facilitates the reading and writing of 1-D and 2-D arrays. Figure E.2 illustrates the functions and properties of the `KSLFileUtil` object

- `copyDirectory()` and `copyFile()` facilitate path and file based copying
- `createFile()`, `createDirectories()`, `createPrintWriter()`, all create according to their function names
- `deleteDirectory()` will delete a directory based on a `File` or a `Path`

Figure E.2.: KSLFileUtil Class

- `createCSVFileName()`, `createTxtFileName()` will make a string that has the appropriate extension
- `isCSVFile()`, `isTeXFile()`, `isTextFile()` check for the appropriate extension

As previously noted, there is a logger available for logging file interactions. A useful property is the `programLaunchDirectory`, which provides the path to the directory in which the program is executing.

The `KSLFileUtil` object also helps with array IO.

- `write(array, out: PrintWriter)` has versions for working with arrays of primitives: `Array<DoubleArray>`, `Array<IntArray>`, `DoubleArray`, `IntArray`. The functions that write double values also have an optional argument to control formatting, `df: DecimalFormat?`. There are equivalent extension functions for `Array<DoubleArray>`, `Array<IntArray>`, `DoubleArray`, `IntArray` for writing to files.
- `scanToArray(path: Path)` will read the values file associated with the path into an `DoubleArray`.
- `toCSVString(array: DoubleArray, df: DecimalFormat?)` will format a line representing the array of data as a comma separated value string.

These functions are used in a number of other packages when working with data and files.

E.4. CSV, Excel, and Tabular Data Files

The KSL also has simple utilities to work with comma separated value (CSV) files and Excel files. The underlying CSV file processing library used by the KSL is Commons CSV. The KSL provision of working with CSV files is not meant to replace the functionality of those libraries. Instead, the purpose is to provide a simple facade so that users can do some simple

processing without worrying about the complexities of a full featured CSV library. Figure E.3 illustrates the functions and properties of the `CSVUtil` object

Figure E.3.: CSVUtil Class

Assuming that the CSV data is organized with first row as a header and each subsequent row as the data for each column as follows:

```
"x", "y"
1.1, 2.0
4.3, 6.4
```

The KSL class `CSVUtil` has the following functions:

- `readRows() : List<Array<String>>` Reads all rows into list holding the rows within an array of strings
- `readRows() : Array<DoubleArray>` Reads all rows into array holding the rows within an array of doubles
- `readToColumns() : Array<DoubleArray>` reads all of the rows and transposes them into the columns
- `writeArrayToCSVFile(array: Array<DoubleArray>)` will write a 2-D array to a file with the CSV format, allowing an optional header and whether quotes are added to strings.
- `csvIterator(): Iterator<Array<String>>` will iterate through a CSV file.

The following code illustrates the use of the `CSVUtil` class. An instance of a `NormalRV` random variable is used to make a 5 row, 4 column matrix of normal $\mu = 0$ and $\sigma = 1$ random variates.

```
    val n = NormalRV()
    val matrix = n.sampleAsColumns(sampleSize = 5, nCols = 4)
    for(i in matrix.indices){
        println(matrix[i].contentToString())
    }
    val h = listOf("col1", "col2", "col3", "col4")
    val p = KSL.csvDir.resolve("data.csv")
    CSVUtil.writeArrayToCSVFile(matrix, header =
↪   h.toMutableList(), pathToFile = p)
    println()
    val dataAsList: List<Array<String>> =
     ↪  CSVUtil.readRowsToListOfStringArrays(p, skipLines =
     ↪  1)
    val m = KSLArrays.parseTo2DArray(dataAsList)
    for(i in matrix.indices){
        println(m[i].joinToString(prefix = "[", postfix =
↪   "]"))
```

```
    }
```

The contents of the matrix are printed to console. The header is made for the CSV file and then the matrix is written to the file using the `CSVUtil` function to write an array to a CSV file. Then, the `CSVUtil` object is used to read the data into list of strings, which are parsed to double values. You can use the Commons CSV functionality to read and write values via its API. The approach illustrated here is only meant for simple file processing.

Excel file operations are available within the KSL through the Apache POI library. The KSL provision of working with Excel files is not meant to replace the functionality of the POI library. Instead, the purpose is to provide a simple facade so that users can do some simple processing without worrying about the complexities of a full featured POI library. Figure E.4 illustrates the functions and properties of the `ExcelUtil` object.

The KSL class `ExcelUtil` provides the basic functions primarily concerned with reading and writing tabular data:

- `createSheet(workbook: Workbook, sheetName:String)` Creates a sheet within the workbook with the name.
- `writeSheetToCSV()` Treats the columns as fields in a csv file, writes each row as a separate csv row in the resulting csv file
- `writeCell()` Writes the object to the Excel cell
- `readCellAsObject() : Any?` the data in the form of an object
- `readCellAsString() : String` the data in the form of a String
- `readRowAsObjectList() : List<Any?>` an list of strings representing the contents of the cells
- `readRowAsStringArray() : Array<String?>` an array of strings representing the contents of the cells
- `readRowAsStringList() : List<String?>` a list of strings representing the contents of the cells

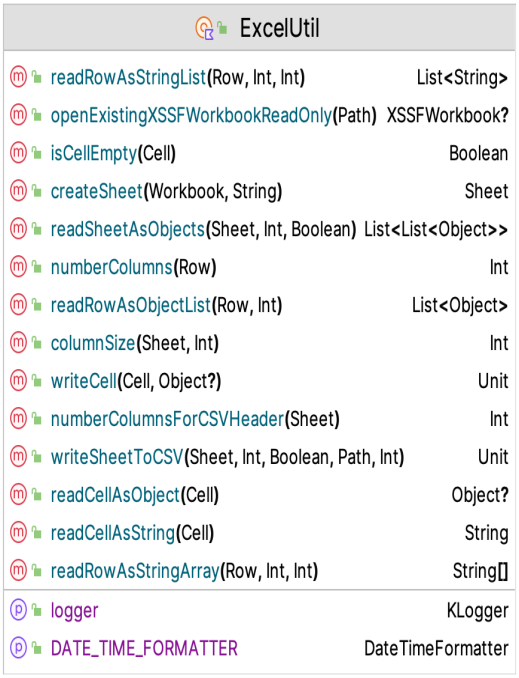


Figure E.4.: ExcelUtil Class

- `readSheetAsObjects() : List<List<Any?>>` a list of lists of the objects representing each cell of each row of the sheet
- `columnSize()` number of rows that have data in a particular column as defined by not having a null cell.

To use the `ExcelUtil` object, it is useful to know how the Apache POI library works because you need to know how to create workbooks. The functionality of the POI library is also exposed as part of the KSL API. The main use of the object `ExcelUtil` within the KSL is for importing and exporting database tables and record sets to Excel.

Comma separated value (CSV) files and Excel files are commonly used to store data in a tabular (row/column) format. The KSL also provides some basic functionality for reading and writing tabular files.

A common programming necessity is to write or read data that is organized in rows and columns. While libraries that support CSV file processing are readily available, it is sometimes useful to have a quick and easy way to write (or read) tabular data that is not as sophisticated as a database. While Excel files also provide this ability, using the Apache POI may be more than is required. To make this type of file processing easier, the KSL provides a general interface for representing tabular data as a data class via the `TabularData` class. In addition, the `TabularFile`, `TabularOutputFile,` and `TabularInputFile` classes permit simple row oriented reading and writing of tabular data.

Figure E.5 illustrates the functions and properties of the `TabularData` class. The basic usage of this class is to subclass a data class from it to define the contents of a row of data. This can be used for database processing as well as for working with the tabular file classes.

Figure E.6 illustrates the functions and properties of the classes for working with tabular data stored in files.

The following code illustrates how to make a tabular file with 5 columns. The first three columns hold numeric data. The fourth column holds text,

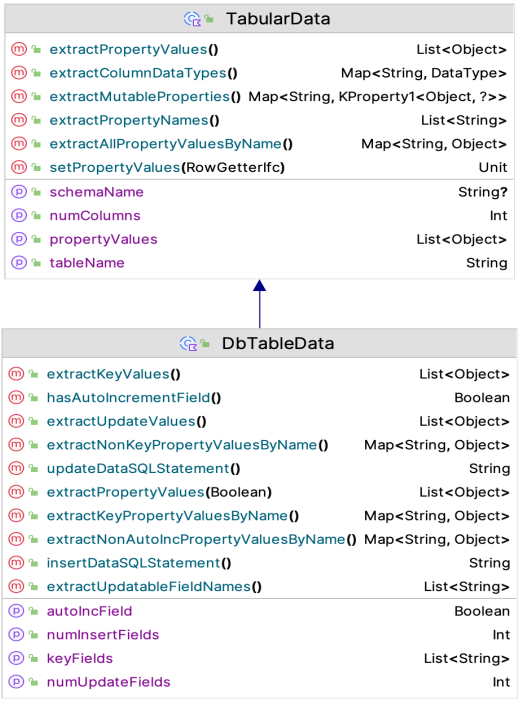


Figure E.5.: TabularData Class

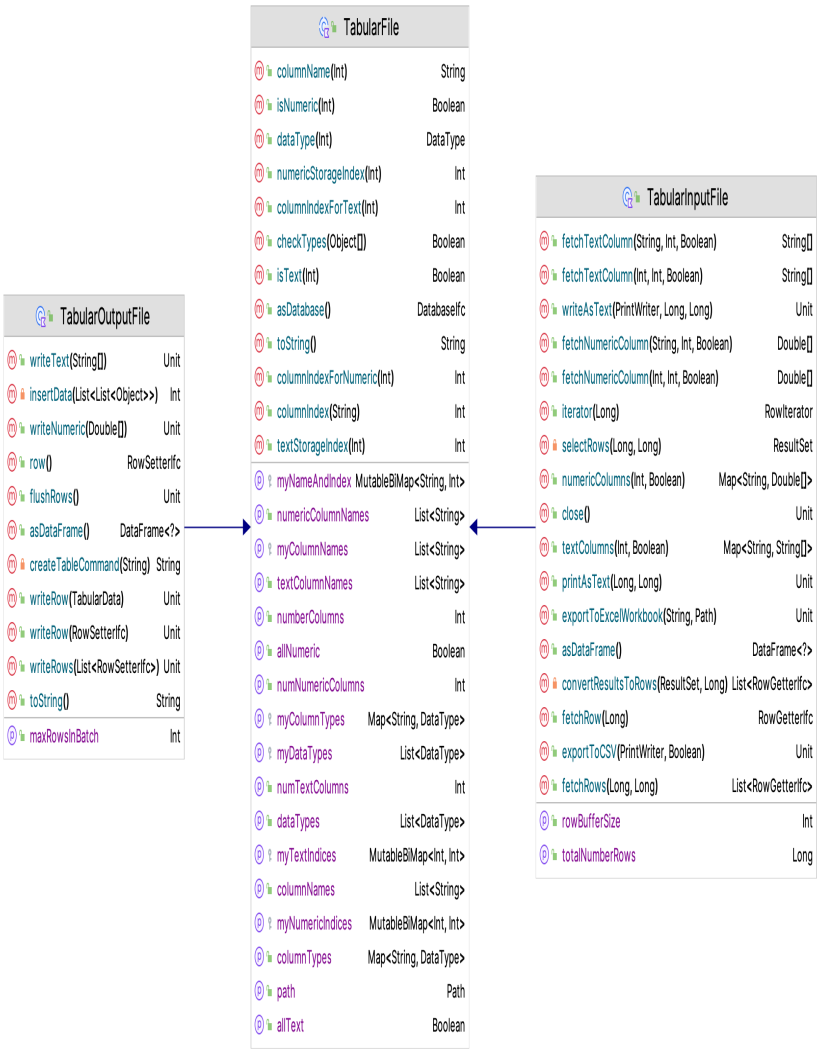


Figure E.6.: Tabular File Classes

and the fifth column holds numeric data. The enum `DataType` defines two types of data to be held in columns: numeric or text. This covers a vast majority of simple computing needs.

```
val path: Path = KSL.outDir.resolve("demoFile")
// configure the columns
val columns: MutableMap<String, DataType> =
 ↪  TabularFile.columns(3,
 ↪  DataType.NUMERIC).toMutableMap()
columns["c4"] = DataType.TEXT
columns["c5"] = DataType.NUMERIC
// make the file
val tif = TabularOutputFile(columns, path)
println(tif)
```

The `columns()` function is used to define the columns as a map of column names and the data types. This describes a type of column within the tabular file.

The numeric type should be used for numeric data (float, double, long, int, etc.). In addition, use the numeric type for boolean values, which are stored 1.0 = true, 0.0 = false). The text type should be used for strings and date/time data. Date and time data is saved as ISO8601 strings ("YYYY-MM-DD HH:MM:SS.SSS"). If you need more type complexity, you should use a database or some other more advanced serialization method such as JSON, Parquet, or Arrow.

The following code creates some random data and writes rows of data to the file. The `RowSetterIfc` interface is use in that it allows for the writing of any numeric columns and any text columns as separate row operations.

```
// needed for some random data
val n = NormalRV(10.0, 1.0)
```

```
val k = 15
// get a row
val row: RowSetterIfc = tif.row()
// write some data to each row
println("Writing rows...")
for (i in 1..k) {
    // reuse the same row, many times
    // can fill all numeric columns
    row.setNumeric(n.sample(5))
    // can set specific columns
    row.setText(3, "text data $i")
    // need to write the row to the buffer
    tif.writeRow(row)
}
// don't forget to flush the buffer
tif.flushRows()
println("Done writing rows!")
```

The tabular file package also allows for the definition of the data schema via data class. The type of the properties of the data class are translated to either text or numeric columns. The following code illustrates this functionality. Here a data class is used to define a text field and a numeric field, which are then written to the file.

```
val path: Path = KSL.outDir.resolve("TabularDataFile")
data class SomeData(var someText: String = "", var
 ↪  someData: Double = 0.0): TabularData("SomeData")
val rowData = SomeData()
// use the data class instance to define the columns and
 ↪  their types
val tof = TabularOutputFile(rowData, path)
println(tof)
```

```
    // needed for some random data
    val n = NormalRV(10.0, 1.0)
    val k = 15
    // write some data to each row
    println("Writing rows...")
    for (i in 1..k) {
        // reuse the same row, many times
        // can fill all numeric columns
        rowData.someData = n.value
        rowData.someText = "text data $i"
        // need to write the row to the buffer
        tof.writeRow(rowData)
    }
    // don't forget to flush the buffer
    tof.flushRows()
    println("Done writing rows!")
```

As long as the data class is compatible with the rows of the file, then the same approach can be used to read in the data as illustrated in the next code snippet.

```
    val tif = TabularInputFile(path)
    println(tif)

    // TabularInputFile is Iterable and foreach construct
     ↪  works across rows
    println("Printing all rows from an iterator using data
↪   class")
    for (row in tif.iterator()) {
        rowData.setPropertyValues(row)
        println(rowData)
    }
    println()
```

Reading data from the file has a bit more functionality through some useful iterators. The following code opens a tabular file and iterates the rows.

```
val path: Path = KSL.outDir.resolve("demoFile")
val tif = TabularInputFile(path)
println(tif)

// TabularInputFile is Iterable and foreach construct
 ↪  works across rows
println("Printing all rows from an iterator")
for (row in tif.iterator()) {
    println(row)
}
println()
```

You can fetch specific rows or a subset of rows. Multiple iterators can be active at the same time.

```
// You can fetch rows as a list
println("Printing a subset of rows")
val rows: List<RowGetterIfc> = tif.fetchRows(1, 5)
for (row in rows) {
    println(row)
}
println()
```

You can start the iterator at a particular row.

```
println("Print starting at row 9")
val iterator: TabularInputFile.RowIterator =
 ↪  tif.iterator(9)
while (iterator.hasNext()) {
```

```
        println(iterator.next())
    }
    println()
```

You can grab various columns as arrays.

```
    println("Printing column 0")
    val numericColumn: DoubleArray =
     ↪  tif.fetchNumericColumn(0, 10, true)
    for (v in numericColumn) {
        println(v)
    }
```

You can write the data to an Excel workbook.

```
    try {
        tif.exportToExcelWorkbook("demoData.xlsx",
↪   KSL.excelDir)
    } catch (e: IOException) {
        e.printStackTrace()
    }
```

You can pretty print rows of the data and export the data to a CSV file.

```
    tif.printAsText(1, 5)
    val printWriter: PrintWriter =
     ↪  KSL.createPrintWriter("data.csv")
    tif.exportToCSV(printWriter, true)
    printWriter.close()
```

You can even convert the tabular data to an SQLLite database or turn the data into a data frame.

```
    try {
        val database: DatabaseIfc = tif.asDatabase()
    } catch (e: IOException) {
        e.printStackTrace()
    }
    println()
    println("Printing a data frame version")
    val df = tif.asDataFrame()
    println(df)
```

Much of this functionality is used within the implementation of the KSL database implementation, which will be discussed in a subsequent section. Since we just mentioned data frames, we discuss that functionality in the next section.

E.5. The `DataFrameUtil` Object

A data frame is an in-memory data structure that holds tabular data. That is, data having rows and columns. Kotlin has a library to support this type of functionality and the `DataFrameUtil` object has been designed to facilitate the use of data frame within the `KSL`. Documentation, examples, and the basic functionality of Kotlin data frames can be found at this repository. Kotlin data frames provide similar functionality as that found in other data frame libraries such as R. Figure E.7 illustrates the functions and properties of the `DataFrameUtil` object.

The main functionality added by `DataFrameUtil` is sampling from rows and columns of the data frame and computing some basic statistics. The functions for sampling without replacement return a new data frame with the sampled rows. The functions for permutation return a new data frame with the permuted rows. The functions for randomly selection will select an element from the a column of the data frame or for selecting an entire

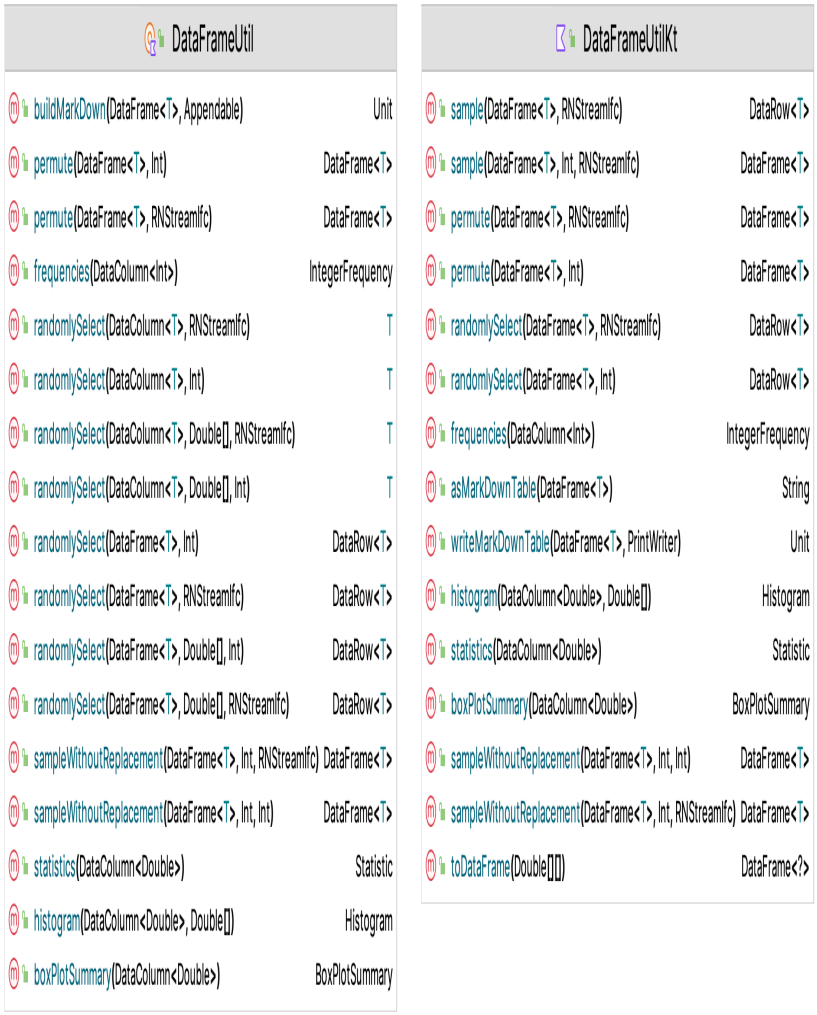


Figure E.7.: The DataFrameUtil Object

row. The element or row can be selected with equal probability or via an empirical distribution over the elements (by row). `KSL` statistics, histogram, box plot statistics, and frequencies can all be computed over the columns.

- sampleWithoutReplacement(DataFrame< T >, Int, RNStreamIfc) DataFrame< T >
- sampleWithoutReplacement(DataFrame< T > Int, Int) DataFrame< T >
- permute(DataFrame< T >, Int) DataFrame< T >
- permute(DataFrame< T >, RNStreamIfc) DataFrame< T >
- randomlySelect(DataColumn< T >, RNStreamIfc) T
- randomlySelect(DataColumn< T >, Int) T
- randomlySelect(DataColumn< T >, Double[], RNStreamIfc) T
- randomlySelect(DataColumn< T >, Double[], Int) T
- randomlySelect(DataFrame< T >, Int) DataRow< T >
- randomlySelect(DataFrame< T >, RNStreamIfc) DataRow< T >
- randomlySelect(DataFrame< T >, Double[], Int) DataRow< T >
- randomlySelect(DataFrame< T >, Double[], RNStreamIfc) DataRow< T >
- buildMarkDown(DataFrame< T >, Appendable) Unit
- histogram(DataColumn< *Double* >, Double[]) Histogram
- statistics(DataColumn< *Double* >) Statistic
- frequencies(DataColumn< Int >) IntegerFrequency
- boxPlotSummary(DataColumn< *Double* >) BoxPlotSummary

Extension functions are also available for a data frame and its columns. The Kotlin data frame library has been included in the KSL as part of the API. Thus, clients also have access to the full features associated with the library. The main usage within the KSL is in the capturing of simulation output data. The easiest way to do this is by using the `KSLDatabase` class. Data frame instances can be requested as part of the database functionality of the KSL.

E.6. KSL Database Utilities

Some of the database functionality for use when accessing simulation results has already been discussed in Chapter 5. This section presents some of the more general database utilities available within the KSL framework. These utilities basically exist to help with implementing the KSL database functionality. However, users may find some of this functionality useful for other purpose. However, these utilities are not meant as a substitute for more advanced database frameworks such as JOOQ, Exposed, and KTorm. We refer the interested reader to those libraries for more advanced work for database processing using Kotlin.

Figure E.8 illustrates the properties of the interfaces, `DatabaseIOIfc` and `DatabaseIfc` which represent the main functionality for working with databases.

The `DatabaseIOIfc` interface is responsible for defining the functionality related to input and output. As shown in Figure E.9 the capabilities are easily discerned from the names of the methods. Exporting tables and views to Excel or CSV files are probably the most useful functions. For small tables, capturing the table as a Markdown table can be useful.

The `DatabaseIfc` interface defines what a database can do and the `Database` class provides a concrete implementation for working with a database. The functionality of the `DatabaseIfc` interface is extensive. Besides the IO related functions, we note a few useful methods here. The `execute` methods allow the execution of an SQL string, list of SQL commands, or script file. It is the user's responsibility for forming an appropriate SQL string.

- executeCommand(String) Boolean
- executeCommand(String) Boolean
- executeCommands(List<String>) Boolean
- executeCommands(List<String>) Boolean
- executeScript(Path) Boolean

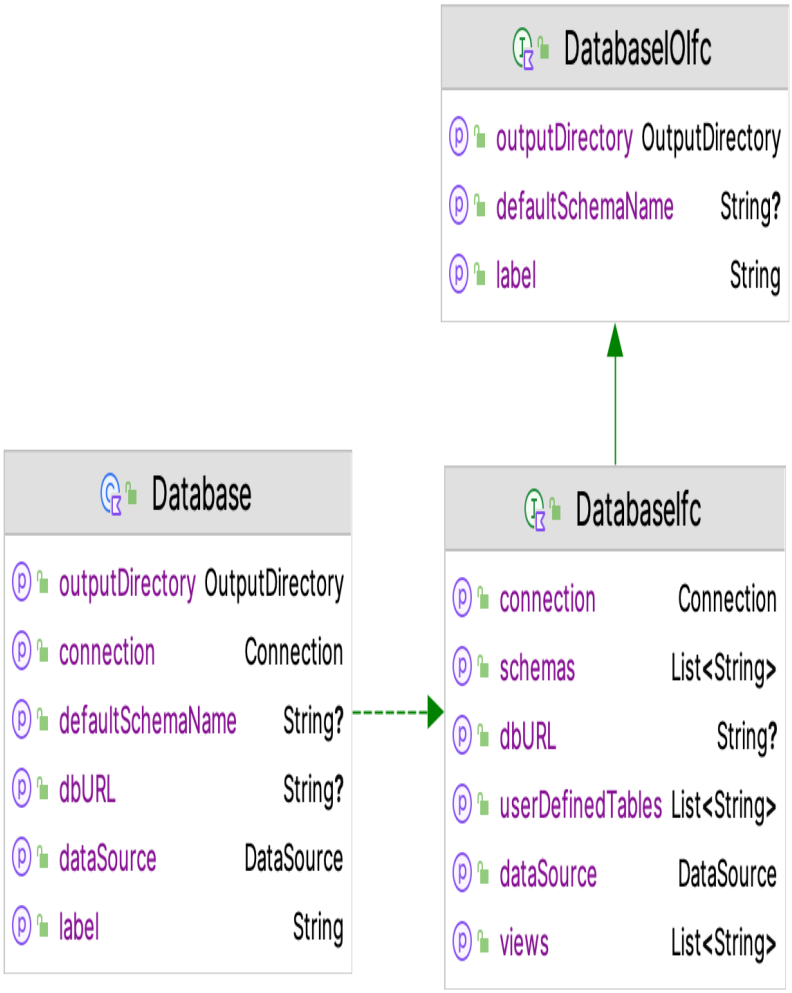


Figure E.8.: Properties of the Main Database Classes

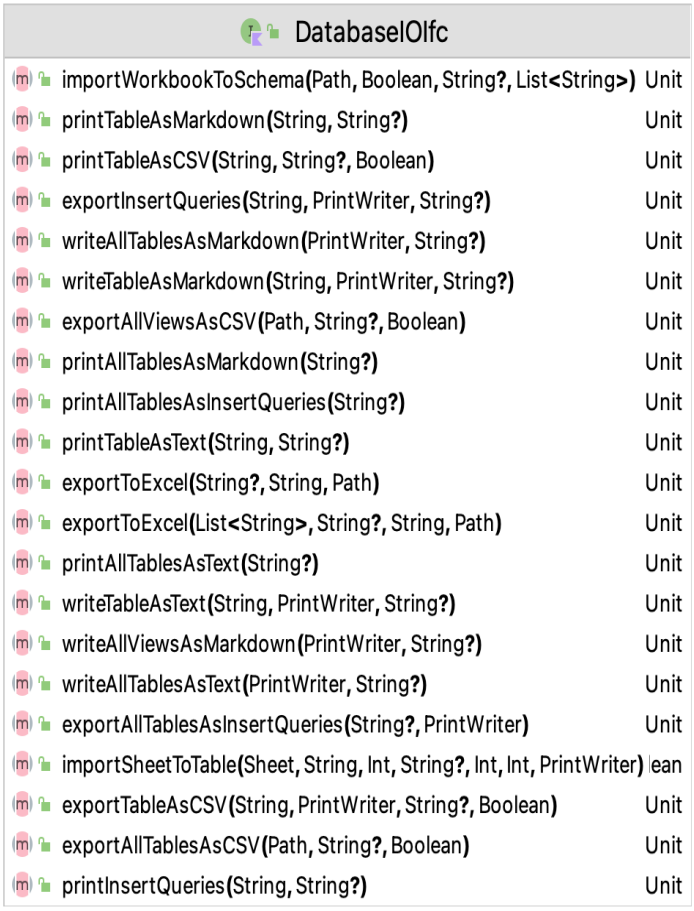


Figure E.9.: Methods of the DatabaseIOIfc Interface

- executeScript(Path) Boolean

There are methods for checking if the database contains specific named tables or views.

- containsSchema(String) Boolean
- containsSchema(String) Boolean
- containsTable(String) Boolean
- containsTable(String, String) Boolean
- containsTable(String, String) Boolean
- containsTable(String) Boolean
- containsView(String) Boolean
- containsView(String) Boolean

A set of methods for fetching data from the database facilitate the execution of SQL select statements. The select all methods will select all the data from a named table or view.

- fetchCachedRowSet(String) CachedRowSet?
- fetchCachedRowSet(String) CachedRowSet?
- fetchOpenResultSet(String) ResultSet?
- fetchOpenResultSet(String) ResultSet?
- selectAll(String, String?) CachedRowSet?
- selectAll(String, String?) CachedRowSet?
- selectAllIntoOpenResultSet(String, String?) ResultSet?
- selectAllIntoOpenResultSet(String, String?) ResultSet?

The interface also provides some basic methods for inserting and updating data within the database by using Kotlin data classes, specifically subclasses of the `DbTableData` class, which extends the `TabularData` class and allows the user to supply the data for a table in a data class instance similar to how it was done for working with tabular files. However, in this case more general mapping of data types to database types is permitted besides just numeric or text.

- insertDbDataIntoTable(List<String>, String, String?) Int
- insertDbDataIntoTable(T, String, String?) Int
- insertDbDataIntoTable(List<String>, String, String?) Int
- insertDbDataIntoTable(T, String, String?) Int
- updateDbDataInTable(List<String>, String, String?) Int
- updateDbDataInTable(T, String, String?) Int
- updateDbDataInTable(List<String>, String, String?) Int
- updateDbDataInTable(T, String, String?) Int

When working with the database it is useful to get a connection, access table meta data, check if tables have data, check if tables or views exist.

- getConnection() Connection
- getSchemas() List<String>
- getUserDefinedTables() List~<String>
- getViews() List<String>
- hasData(String?) Boolean
- hasData(String?) Boolean
- hasTables(String) Boolean
- hasTables(String) Boolean
- isTableEmpty(String, String?) Boolean
- isTableEmpty(String, String?) Boolean
- numRows(String, String?) Long
- numRows(String, String?) Long
- tableMetaData(String, String?) List<ColumnMetaData>
- tableMetaData(String, String?) List<ColumnMetaData>
- tableNames(String) List<String>
- tableNames(String) List<String>
- viewNames(String) List<String>
- viewNames(String) List<String>

We refer the interested reader to the KSL KDoc documentation for further details about using these methods.

The `DatabaseIfc` functionality defines basic capabilities for working with any database implementation. The KSL provides functionality to create SQLite, Derby , and DuckDb embedded databases. In addition, the KSL facilitates the creation of a database on a Postgres database server.

Figure E.10 illustrates the functionality of the `EmbeddedDbIfc` interface, and its SQLite, Derby, and DuckDb implementations. Readers interested in other databases can review these implementations for how to structure code for other databases. The `PostgresDb` class provides similar functionality. The main purpose is to be able to supply a data source to the `Database` class.

Figure E.11 provides the functionality for the `KSLDatabase` class. Although most of this functionality has been mentioned within Chapter 5, it is useful to note that since the `KSLDatabase` class is a database, it also has all of the previously mentioned database functionality.

The main notable methods involve the export, printing, or writing of the underlying data from the tables or views storing the simulation statistical results. In addition, there are a number of properties that will provide data frame representations of the simulation output data. For example, the data class `WithinRepStatTableData` represents the statistics collected within each replication and is used to by the `withinReplicationResponseStatistics` property to create a data frame that holds the statistical data. Figure E.12 provides the data elements held for within replication data.

The within replication statistical data represents the summary statistics of the data collected during a replication. The most relevant properties are the `average`, `minimum`, and `maximum`. To compute across replication statistics we can use the `average` for each replication. The statistical quantities are captured within the underlying database as discussed in Chapter 5. The user can post-process any of this data using commonly available database technology and SQL.

E. KSL Utility Packages

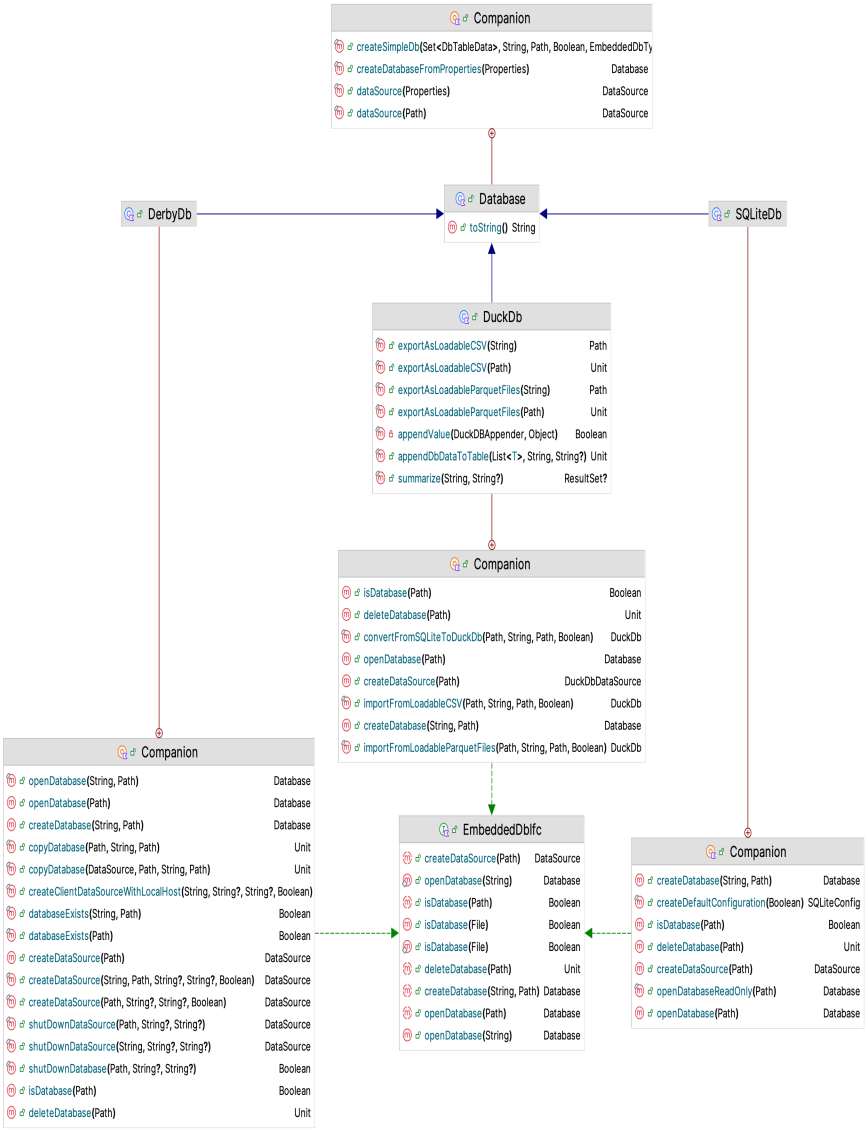


Figure E.10.: Working with Embedded Databases

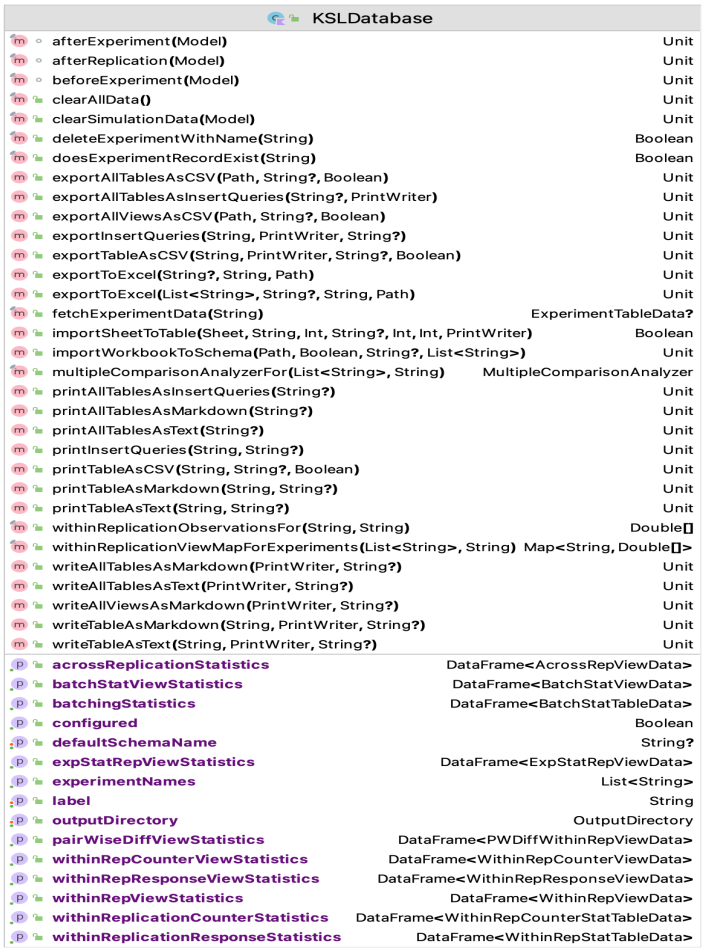


Figure E.11.: The KSLDatabase Class

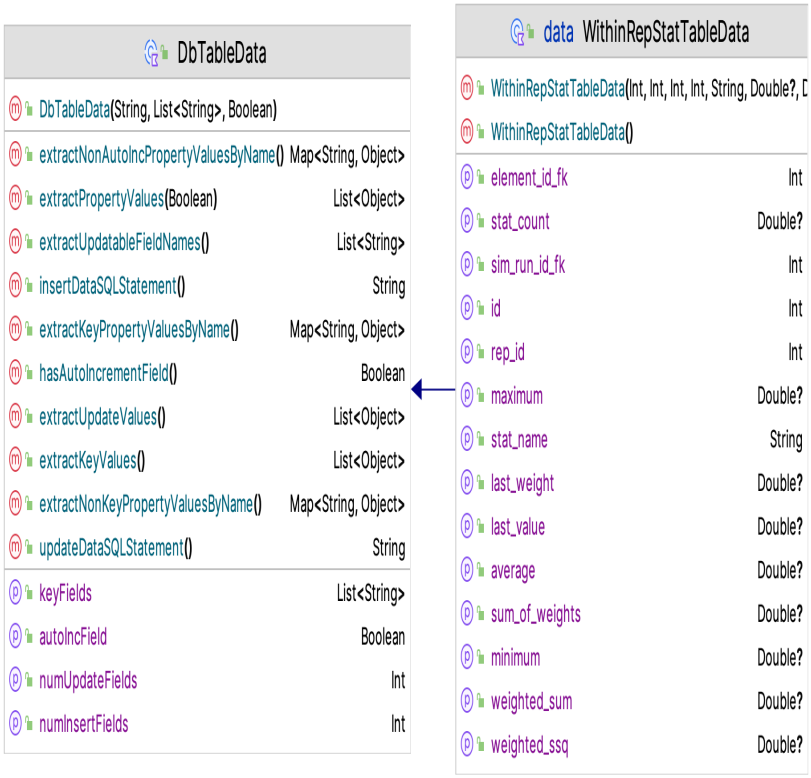


Figure E.12.: Within Replication Statistical Data

The database utilities also offers the ability to quickly create simple databases via the `Database` companion object's `createSimpleDb()` function. The purpose of this function is to allow the creation of a quick and dirty database solution based on the `DbTableData` data classes. By defining data classes that are sub-classes of `DbTableData`, a `CREATE TABLE` specification can be obtained and the database created. Then, the database can be used to insert data from instances of the `DbTableData` sub-classes. The following code illustrates this possibility.

First, we define two data classes that extend the `DbTableData` class. In this code, we define a `Person` and a `City` class, each with a property `id` that will act as the primary key of the table. The primary key must be specified; however, the use of an auto-generated (surrogate) key is not supported at this time.

```
data class Person(
    var id: Int,
    var name: String,
    var age: Int
) : DbTableData("Persons", listOf("id"))

data class City(
    var id: Int,
    var name: String,
    var population: Int
) : DbTableData("Cities", listOf("id"))
```

Then, by providing a simple instance of these classes to the `SimpleDb` class constructor, we will cause a database to be created that is setup to hold the data from the supplied data classes. If you want to see the `CREATE TABLE` statements used to define the tables, then uncomment the two print statements in the following code.

```
    val p = Person(1, "manuel", age = 10)
//    println(p.createTableSQLStatement())
    val c = City(1, "London", population = 1000)
//    println(c.createTableSQLStatement())
    val db = Database.createSimpleDb(setOf(p, c),
     ↪  "TestSimpleDb")
    db.insertDbDataIntoTable(p)
    db.insertDbDataIntoTable(c)
```

You can then used the full functionality provided by the KSL database utilities because the `createSimpleDb()` function returns an instance of the `DatabaseIfc` and `DatabaseIOIfc` interfaces. This functionality is useful for setting up small databases to hold generated data. It does not facilitate more advanced data definition or object-relational mappings. Please see more advanced database frameworks such as JOOQ, Exposed, and KTorm for more robust solutions.

E.7. Array Utilities

The KSL provides a wide-variety of functions and extensions for working with `Double`, `Int`, and `Long` arrays. Most of the functionality is focused on working with one-dimensional `Double` arrays. Figure E.13 illustrates the many methods for working with arrays via the `KSLArrays` object. These methods are also available as extension functions.

There are a number of functions that facilitate conversions: to primitive types, to strings, conversion between types, and parsing.

- toPrimitives(Double[], Double) Double[]
- toPrimitives(List<Double>, Double) Double[]
- toPrimitives(Int[], Int) Int[]
- toPrimitives(List<Int>, Int) Int[]

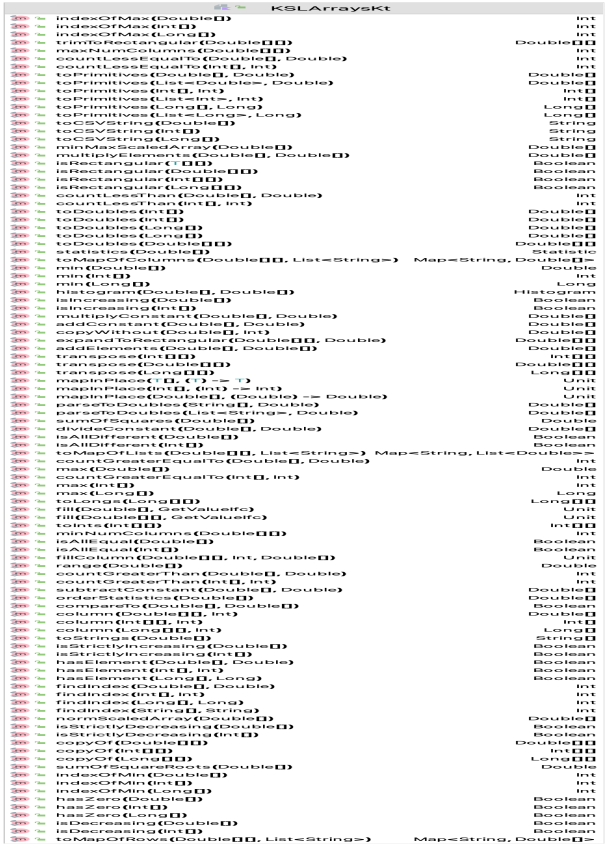


Figure E.13.: Array Utilities

- toPrimitives(Long[], Long) Long[]
- toPrimitives(List<Long>, Long) Long[]
- toCSVString(Double[]) String
- toCSVString(Int[]) String
- toCSVString(Long[]) String
- toDoubles(Int[]) Double[]
- toDoubles(Int[]) Double[]
- toDoubles(Long[]) Double[]
- toDoubles(Long[]) Double[]
- toDoubles(Double[][]) Double[][]
- parseToDoubles(String[], Double) Double[]
- parseToDoubles(List<String>, Double) Double[]
- toStrings(Double[]) String[]
- toInts(Int[][]) Int[][]
- toLongs(Long[][]) Long[][]

Then, there are useful functions for performing basic statistical computations such as finding the indices of the minimum or maximum, counting elements, finding the minimum, finding the maximum, and computing statistics.

- indexOfMax(Double[]) Int
- indexOfMax(Int[]) Int
- indexOfMax(Long[]) Int
- indexOfMin(Double[]) Int
- indexOfMin(Int[]) Int
- indexOfMin(Long[]) Int
- countLessEqualTo(Double[], Double) Int
- countLessEqualTo(Int[], Int) Int
- countLessThan(Double[], Double) Int
- countLessThan(Int[], Int) Int
- statistics(Double[]) Statistic
- min(Double[]) Double
- min(Int[]) Int

- min(Long[]) Long
- histogram(Double[], Double[]) Histogram
- countGreaterEqualTo(Double[], Double) Int
- max(Double[]) Double
- countGreaterEqualTo(Int[], Int) Int
- max(Int[]) Int
- max(Long[]) Long
- range(Double[]) Double
- countGreaterThan(Double[], Double) Int
- countGreaterThan(Int[], Int) Int
- subtractConstant(Double[], Double) Double[]
- orderStatistics(Double[]) Double[]
- sumOfSquares(Double[]) Double
- sumOfSquareRoots(Double[]) Double

In addition, there are methods for checking the arrays. For example, methods exist to check if the 2-D array is rectangular in shape. That is, the rows all have the same number of elements. You can also check if the elements are all different, are strictly increasing or decreasing, and contain a zero value.

- isRectangular(T[][]) Boolean
- isRectangular(Double[][]) Boolean
- isRectangular(Int[][]) Boolean
- isRectangular(Long[][]) Boolean
- isIncreasing(Double[]) Boolean
- isIncreasing(Int[]) Boolean
- isAllDifferent(Double[]) Boolean
- isAllDifferent(Int[]) Boolean
- isStrictlyIncreasing(Double[]) Boolean
- isStrictlyIncreasing(Int[]) Boolean
- hasElement(Double[], Double) Boolean
- hasElement(Int[], Int) Boolean
- hasElement(Long[], Long) Boolean

- findIndex(Double[], Double) Int
- findIndex(Int[], Int) Int
- findIndex(Long[], Long) Int
- findIndex(String[], String) Int
- isStrictlyDecreasing(Double[]) Boolean
- isStrictlyDecreasing(Int[]) Boolean
- hasZero(Double[]) Boolean
- hasZero(Int[]) Boolean
- hasZero(Long[]) Boolean
- isDecreasing(Double[]) Boolean
- isDecreasing(Int[]) Boolean

Then, there are methods for operating on the entire array. First, there are functions to reshape the arrays such as trimming non-rectangular arrays to rectangular, transposing the arrays, and expanding non-rectangular arrays to be rectangular. Then, there are functions to apply to each element such as multiplying the elements by a constant, adding a constant, and dividing by a constant. You can also apply a function to the elements in-place. That is transform each element without creating a new array.

- trimToRectangular(Double[][]) Double[][]
- minMaxScaledArray(Double[]) Double[]
- multiplyElements(Double[], Double[]) Double[]
- multiplyConstant(Double[], Double) Double[]
- addConstant(Double[], Double) Double[]
- copyWithout(Double[], Int) Double[]
- expandToRectangular(Double[][], Double) Double[][]
- transpose(Int[][]) Int[][]
- transpose(Double[][]) Double[][]
- transpose(Long[][]) Long[][]
- divideConstant(Double[], Double) Double[]
- normScaledArray(Double[]) Double[]
- toMapOfLists(Double[][], List<String>) Map<String, List<Double>>
- toMapOfColumns(Double[][], List<String>) Map<String, Double[]>

- toMapOfRows(Double[][], List<String>) Map<String, Double[]>
- mapInPlace(T[], (T) -> T) Unit
- mapInPlace(Int[], (Int) -> Int) Unit
- mapInPlace(Double[], (Double) -> Double) Unit
- fillColumn(Double[][], Int, Double[]) Unit
- fill(Double[], GetValueIfc) Unit
- fill(Double[][], GetValueIfc) Unit

Figure E.14 illustrates the many methods for permuting and sampling arrays.

Much of the functionality described here can (and has been) used to implement more complex operations used within the KSL. There are other libraries that are under active develop for working with arrays with design goals that focus on efficiency and memory. The interested reader might look at kmath and the multi-dimensional arrays library, multik.

E.8. KSL Plotting Utilities

This file demonstrates how to create many of the plots that are available within the ksl.utilities.io.plotting package. The KSL uses the lets-plot library as its underlying plotting platform.

The KSL wraps the lets-plot functionality into a set of classes to create commonly used plots within simulation. The KSL has classes that facilitate the construction of the following plots.

- `ACFPlot`: The autocorrelation function (ACF) for an array of data plots the correlation between X_i and X_{i+k} for different values of k.
- `BoxPlot`: The box plot summary for an array of data plots the summary statistics representing the first quartile, median, and third quartile of the data along with outliers.
- `MultiBoxPlot` A multi-box plot presents multiple box plots on the same plotting frame.

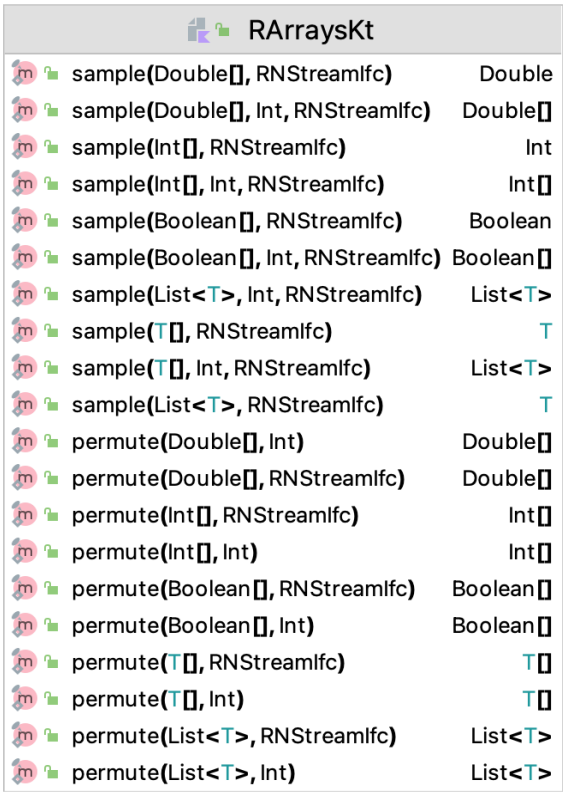


Figure E.14.: Random Array Utilities

- `CDFDiffPlot`: CDF difference plot, as defined in Chapter 6 of (Law 2007), plots the difference between the theoretical distribution $F(x)$ and the empirical distribution $F_n(x)$ over the range of the data.
- `ConfidenceIntervalsPlot`: A confidence intervals plot presents a set of confidence interval on one plotting frame.
- `DensityPlot`: A density plot presents the density of the data relative to a possible probability distribution.
- `DiscreteCDFPlot`: A discrete CDF plot presents the cumulative distribution function for a discrete empirical random variable with a step-function representation.
- `DotPlot`: A dot plot present the distribution of the data as a sequence of dots.
- `ECDFPlot`: An empirical CDF plot presents the distribution of the data as a cumulative estimate of the CDF.
- `FitDistPlot`: A fit distribution plot presents four plots (ECDFPlot, PPPlot, QQPlot, HistogramPlot) of data for distribution fitting diagnostics.
- `FunctionPlot`: A function plot facilitates the plotting of a univariate function.
- `HistogramPlot`: A histogram plot summarizes the distribution of the data over well-defined bins.
- `HistogramDensityPlot`: A histogram plot summarizes the distribution of the data over well-defined bins and super imposes a density representation over the histogram.
- `IntegerFrequencyPlot`: An integer frequency plot presents a bar graph representation of integer frequency tabulated data.
- `StateFrequencyPlot`: A state frequency plot presents a bar graph representation of integer frequency data tabulated across state observations.
- `ObservationsPlot`: An observations plot presents a time-ordered plot of the data.
- `PMFComparisonsPlot`: A PMF comparison plot presents plots the theoretical PMF versus an empirical PMF.
- `PMFPlot`: A PMF plot presents a dot bar graph representation of a

probability mass function.

- `PPPlot`: A P-P plot presents the model probability $\hat{F}(x_{(i)})$ versus the empirical probability $\tilde{F}_n(x_{(i)})$ based on the sample for i= 1, 2,... n
- `QQPlot` A Q-Q plot presents the order statistics versus the theoretical order statistics, x_{q_i} versus $x_{(i)}$ for i = 1, 2, ... n.
- `ScatterPlot`: A scatter plot is a simple x-y plot of two data arrays.
- `StateVariablePlot`: A state variable plot presents a step-function representation for time-persistent variables as per Figure 4.4.
- `WelchPlot`: A Welch plot presents the cumulative sum plot of Welch data as illustrate in Figure 5.15.
- `PartialSumsPlot`: A partial sums plot presents the partial sums process for diagnosing whether initialization bias is present.

In what follows, we present some example code for creating a few of these plots. The full code can be found in the files associated with this appendix.

In the following code, a bivariate normal random variable is created and a sample array is made containing the X and Y data. Then a scatter plot is made from the data.

```
    val bvn = BivariateNormalRV(0.0, 1.0, 0.0, 1.0, 0.8)
    val data = bvn.sampleByColumn(1000)
    val plot = ScatterPlot(data[0], data[1])
    plot.showInBrowser(plotTitle =  "Scatter Plot of
↪   Bivariate Normal RV")
    plot.saveToFile("ScatterPlotDemo", plotTitle = "Scatter
↪   Plot of Bivariate Normal RV")
```

Figure E.15 presents the plot as generated by the `saveToFile()` function.

The returned `plot` variable implements the `PlotIfc` interface. The `PlotIfc` interface is shown in Figure E.16. The function `buildPlot()`

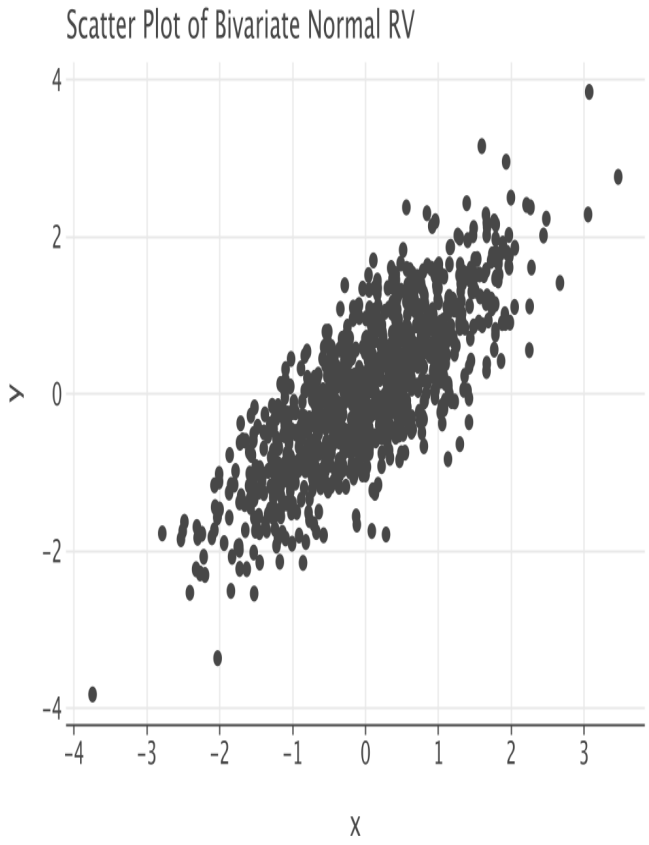


Figure E.15.: Example Scatter Plot

returns an instance of the `Plot` class from the lets-plot library. This instance can be further customized using the concepts of that library. The `toHTML()` function generates an HTML file that can be opened by a browser. This is what the `showInBrowser()` function does.

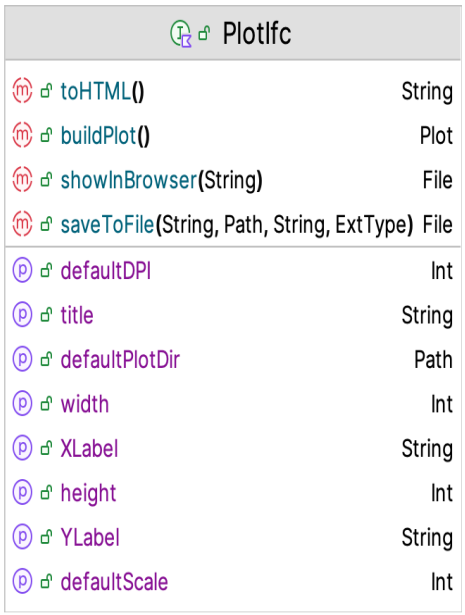


Figure E.16.: PlotIfc Interface

The following code illustrates how to display multiple box plots.

```
val n = NormalRV()
val m = mutableMapOf<String, BoxPlotSummary>()
for (i in 1..5) {
    val bps = BoxPlotSummary(n.sample(200), "BPS$i")
    m[bps.name] = bps
}
val plot = MultiBoxPlot(m)
```

```
    plot.showInBrowser(plotTitle =  "Box Plots")
    plot.saveToFile("BoxPlotDemo", plotTitle = "Box Plots")
```

The code makes a map of `BoxPlot` instances indexed by their names and then displays them together on a single plot axis.

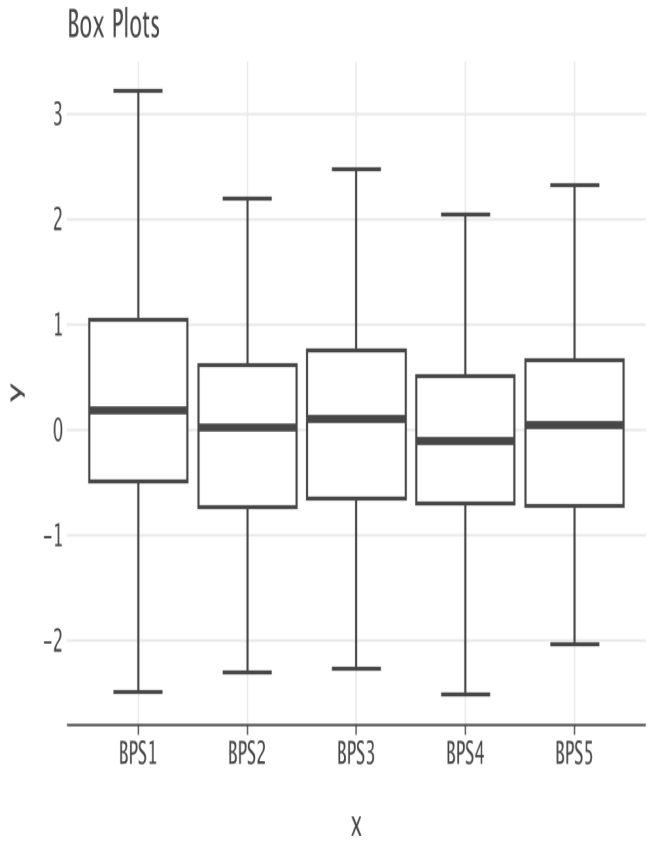


Figure E.17.: Example Multiple Box Plots

The following code illustrates how to display multiple confidence intervals

on the same plot.

```
    val n = NormalRV()
    val m = mutableMapOf<String, Interval>()
    for (i in 1..5) {
        val s = Statistic(n.sample(200))
        m[s.name] = s.confidenceInterval
    }
    val plot = ConfidenceIntervalsPlot(m, referencePoint =
     ↪  0.0)
    plot.showInBrowser(plotTitle = "Confidence Intervals")
    plot.saveToFile("ConfidenceIntervalsPlot", plotTitle =
↪   "Confidence Intervals")
```

The code makes a map of `Interval` instances indexed by the names of their associated statistics and then displays them together on a single plot axis.

Finally, the following code illustrates how to create a state variable plot. In this case, two arrays are required. The first arrays hold the times at which the state variable changes. The second array holds the value of the state variable for the associated times.

```
    val t = doubleArrayOf(0.0, 2.0, 5.0, 11.0, 14.0, 17.0,
     ↪  22.0, 26.0, 28.0, 31.0, 35.0, 36.0)
    val n = doubleArrayOf(0.0, 1.0, 0.0, 1.0, 2.0, 3.0, 4.0,
     ↪  3.0, 2.0, 1.0, 0.0, 0.0)
    val plot = StateVariablePlot(n, t, "Response")
    plot.showInBrowser()
    plot.saveToFile("StateVariableDemo", plotTitle = "State
↪   Variable Plot")
```

Section 5.4.1 of Chapter 5 presented how to make a `ResponseTrace` of a variable from a simulation model. In reviewing, Figure E.19, we see that

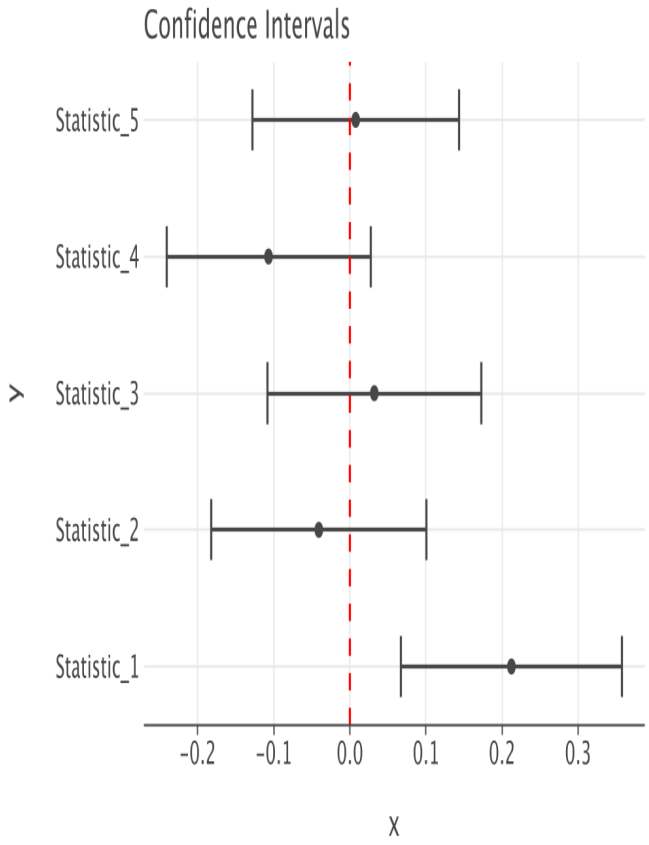


Figure E.18.: Example Multiple Confidence Interval Plots

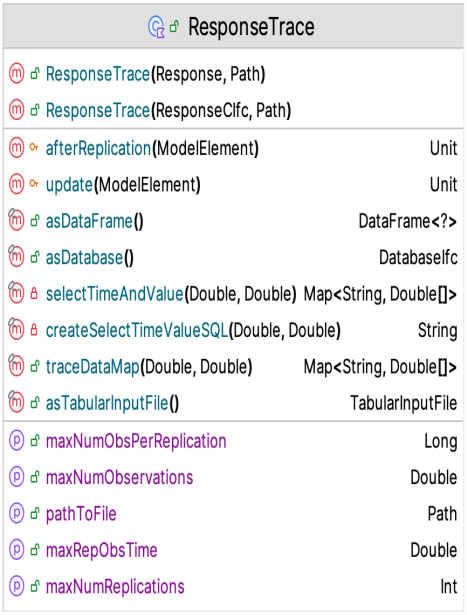


Figure E.19.: ResponseTrace Class

the `traceDataMap()` function will allow you to grab the data for a state variable plot. The function returns a map that has the time and values for the variable for a particular replication less than or equal to the provided `time` value.

```
fun traceDataMap(repNum: Double, time: Double =
 ↪  Double.MAX_VALUE) : Map<String, DoubleArray>{
```

Thus, you can easily create a state variable plot for a response trace when needed.

E.9. Miscellaneous Utilities

The KSL also provides a number of other useful class within the utilities package. The `KSLMath` class provides for working with finer levels of numerical precision and has some useful functions for computing factorials and binomial coefficients. In addition, there are functions for converting doubles to other numeric values.

- `double getDefaultNumericalPrecision()`: returns the default numerical precision that can be expected on the machine
- `boolean equal(double a, double b)`: returns true if the two doubles are equal with respect to the default numerical precision
- `boolean equal(double a, double b, double precision)`: returns true if the two doubles are equal with respect to the specified precision
- `boolean within(double a, double b, double precision)`: returns true if the absolute difference between the double is within the specified precision
- `double factorial(int n)`: returns a numerically stable computed value of the factorial

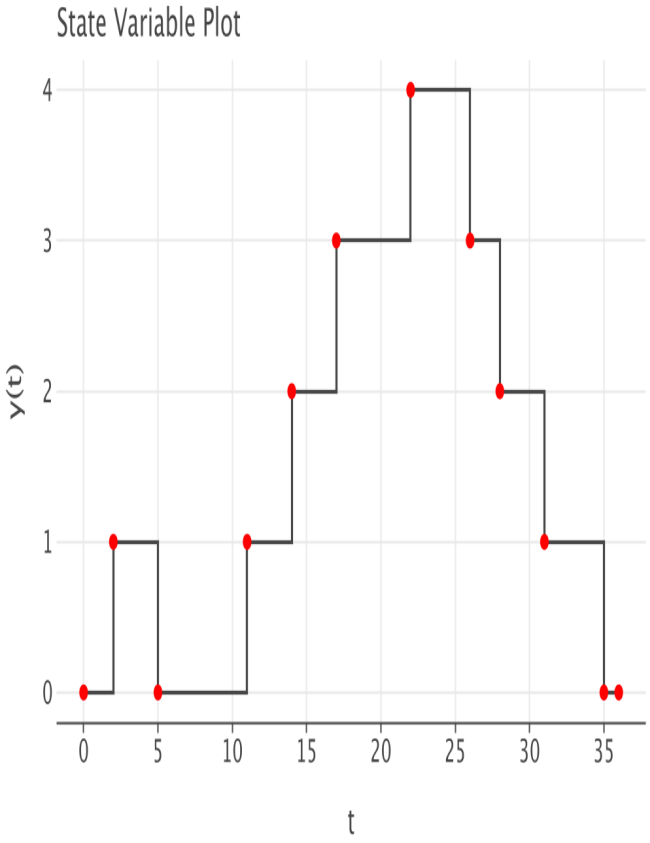


Figure E.20.: Example State Variable Plot

- `double binomialCoefficient(int n, int k)`: returns a numerically stable computed value of the binomial coefficient
- `double logFactorial(int n)`: returns the natural logarithm of the factorial

The following list may be of interest and lead the reader to further exploration of the details.

- `GetValueIfc` defines a general function and property to return a value
- `IdentityIfc` a general interface to ensure that an implementer have an identity and a name.
- `Interval` is useful to representing an inclusive real-valued interval such as a confidence interval.
- `PreviousValueIfc` allows an implementer to remember and return the previous value.
- `KSLMaps` hold utility functions for making maps from arrays, flattening maps of maps, unflattening maps, and converting maps to JSON.
- `utilities.rootfinding` This package has an implementation for finding the root of a function via binary search
- `utilities.observers` has base classes for implementing and using components that implement the observer pattern.

F. Statistical Tables

Table F.1.: Cumulative standard normal distribution, z = -3.49 to 0

	-0.09	-0.08	-0.07	-0.06	-0.05	-0.04	-0.03	-0.02	-0.01	0
-3.40	0.0002	0.0003	0.0003	0.0003	0.0003	0.0003	0.0003	0.0003	0.0003	0.0003
-3.30	0.0003	0.0004	0.0004	0.0004	0.0004	0.0004	0.0004	0.0005	0.0005	0.0005
-3.20	0.0005	0.0005	0.0005	0.0006	0.0006	0.0006	0.0006	0.0006	0.0007	0.0007
-3.10	0.0007	0.0007	0.0008	0.0008	0.0008	0.0008	0.0009	0.0009	0.0009	0.0010
-3.00	0.0010	0.0010	0.0011	0.0011	0.0011	0.0012	0.0012	0.0013	0.0013	0.0013
-2.90	0.0014	0.0014	0.0015	0.0015	0.0016	0.0016	0.0017	0.0018	0.0018	0.0019
-2.80	0.0019	0.0020	0.0021	0.0021	0.0022	0.0023	0.0023	0.0024	0.0025	0.0026
-2.70	0.0026	0.0027	0.0028	0.0029	0.0030	0.0031	0.0032	0.0033	0.0034	0.0035
-2.60	0.0036	0.0037	0.0038	0.0039	0.0040	0.0041	0.0043	0.0044	0.0045	0.0047
-2.50	0.0048	0.0049	0.0051	0.0052	0.0054	0.0055	0.0057	0.0059	0.0060	0.0062
-2.40	0.0064	0.0066	0.0068	0.0069	0.0071	0.0073	0.0075	0.0078	0.0080	0.0082
-2.30	0.0084	0.0087	0.0089	0.0091	0.0094	0.0096	0.0099	0.0102	0.0104	0.0107
-2.20	0.0110	0.0113	0.0116	0.0119	0.0122	0.0125	0.0129	0.0132	0.0136	0.0139
-2.10	0.0143	0.0146	0.0150	0.0154	0.0158	0.0162	0.0166	0.0170	0.0174	0.0179
-2.00	0.0183	0.0188	0.0192	0.0197	0.0202	0.0207	0.0212	0.0217	0.0222	0.0228
-1.90	0.0233	0.0239	0.0244	0.0250	0.0256	0.0262	0.0268	0.0274	0.0281	0.0287
-1.80	0.0294	0.0301	0.0307	0.0314	0.0322	0.0329	0.0336	0.0344	0.0351	0.0359
-1.70	0.0367	0.0375	0.0384	0.0392	0.0401	0.0409	0.0418	0.0427	0.0436	0.0446
-1.60	0.0455	0.0465	0.0475	0.0485	0.0495	0.0505	0.0516	0.0526	0.0537	0.0548
-1.50	0.0559	0.0571	0.0582	0.0594	0.0606	0.0618	0.0630	0.0643	0.0655	0.0668
-1.40	0.0681	0.0694	0.0708	0.0721	0.0735	0.0749	0.0764	0.0778	0.0793	0.0808
-1.30	0.0823	0.0838	0.0853	0.0869	0.0885	0.0901	0.0918	0.0934	0.0951	0.0968

-1.20	0.0985	0.1003	0.1020	0.1038	0.1056	0.1075	0.1093	0.1112	0.1131	0.1151
-1.10	0.1170	0.1190	0.1210	0.1230	0.1251	0.1271	0.1292	0.1314	0.1335	0.1357
-1.00	0.1379	0.1401	0.1423	0.1446	0.1469	0.1492	0.1515	0.1539	0.1562	0.1587
-0.90	0.1611	0.1635	0.1660	0.1685	0.1711	0.1736	0.1762	0.1788	0.1814	0.1841
-0.80	0.1867	0.1894	0.1922	0.1949	0.1977	0.2005	0.2033	0.2061	0.2090	0.2119
-0.70	0.2148	0.2177	0.2206	0.2236	0.2266	0.2296	0.2327	0.2358	0.2389	0.2420
-0.60	0.2451	0.2483	0.2514	0.2546	0.2578	0.2611	0.2643	0.2676	0.2709	0.2743
-0.50	0.2776	0.2810	0.2843	0.2877	0.2912	0.2946	0.2981	0.3015	0.3050	0.3085
-0.40	0.3121	0.3156	0.3192	0.3228	0.3264	0.3300	0.3336	0.3372	0.3409	0.3446
-0.30	0.3483	0.3520	0.3557	0.3594	0.3632	0.3669	0.3707	0.3745	0.3783	0.3821
-0.20	0.3859	0.3897	0.3936	0.3974	0.4013	0.4052	0.4090	0.4129	0.4168	0.4207
-0.10	0.4247	0.4286	0.4325	0.4364	0.4404	0.4443	0.4483	0.4522	0.4562	0.4602
-0.00	0.4641	0.4681	0.4721	0.4761	0.4801	0.4840	0.4880	0.4920	0.4960	0.5000

Table F.2.: Cumulative standard normal distribution, z = 0.0 to 3.49

	0	0.01	0.02	0.03	0.04	0.05	0.06	0.07	0.08	0.09
0.00	0.5000	0.5040	0.5080	0.5120	0.5160	0.5199	0.5239	0.5279	0.5319	0.5359
0.10	0.5398	0.5438	0.5478	0.5517	0.5557	0.5596	0.5636	0.5675	0.5714	0.5753
0.20	0.5793	0.5832	0.5871	0.5910	0.5948	0.5987	0.6026	0.6064	0.6103	0.6141
0.30	0.6179	0.6217	0.6255	0.6293	0.6331	0.6368	0.6406	0.6443	0.6480	0.6517
0.40	0.6554	0.6591	0.6628	0.6664	0.6700	0.6736	0.6772	0.6808	0.6844	0.6879
0.50	0.6915	0.6950	0.6985	0.7019	0.7054	0.7088	0.7123	0.7157	0.7190	0.7224
0.60	0.7257	0.7291	0.7324	0.7357	0.7389	0.7422	0.7454	0.7486	0.7517	0.7549
0.70	0.7580	0.7611	0.7642	0.7673	0.7704	0.7734	0.7764	0.7794	0.7823	0.7852
0.80	0.7881	0.7910	0.7939	0.7967	0.7995	0.8023	0.8051	0.8078	0.8106	0.8133
0.90	0.8159	0.8186	0.8212	0.8238	0.8264	0.8289	0.8315	0.8340	0.8365	0.8389
1.00	0.8413	0.8438	0.8461	0.8485	0.8508	0.8531	0.8554	0.8577	0.8599	0.8621
1.10	0.8643	0.8665	0.8686	0.8708	0.8729	0.8749	0.8770	0.8790	0.8810	0.8830
1.20	0.8849	0.8869	0.8888	0.8907	0.8925	0.8944	0.8962	0.8980	0.8997	0.9015
1.30	0.9032	0.9049	0.9066	0.9082	0.9099	0.9115	0.9131	0.9147	0.9162	0.9177
1.40	0.9192	0.9207	0.9222	0.9236	0.9251	0.9265	0.9279	0.9292	0.9306	0.9319
1.50	0.9332	0.9345	0.9357	0.9370	0.9382	0.9394	0.9406	0.9418	0.9429	0.9441

1.60	0.9452	0.9463	0.9474	0.9484	0.9495	0.9505	0.9515	0.9525	0.9535	0.9545
1.70	0.9554	0.9564	0.9573	0.9582	0.9591	0.9599	0.9608	0.9616	0.9625	0.9633
1.80	0.9641	0.9649	0.9656	0.9664	0.9671	0.9678	0.9686	0.9693	0.9699	0.9706
1.90	0.9713	0.9719	0.9726	0.9732	0.9738	0.9744	0.9750	0.9756	0.9761	0.9767
2.00	0.9772	0.9778	0.9783	0.9788	0.9793	0.9798	0.9803	0.9808	0.9812	0.9817
2.10	0.9821	0.9826	0.9830	0.9834	0.9838	0.9842	0.9846	0.9850	0.9854	0.9857
2.20	0.9861	0.9864	0.9868	0.9871	0.9875	0.9878	0.9881	0.9884	0.9887	0.9890
2.30	0.9893	0.9896	0.9898	0.9901	0.9904	0.9906	0.9909	0.9911	0.9913	0.9916
2.40	0.9918	0.9920	0.9922	0.9925	0.9927	0.9929	0.9931	0.9932	0.9934	0.9936
2.50	0.9938	0.9940	0.9941	0.9943	0.9945	0.9946	0.9948	0.9949	0.9951	0.9952
2.60	0.9953	0.9955	0.9956	0.9957	0.9959	0.9960	0.9961	0.9962	0.9963	0.9964
2.70	0.9965	0.9966	0.9967	0.9968	0.9969	0.9970	0.9971	0.9972	0.9973	0.9974
2.80	0.9974	0.9975	0.9976	0.9977	0.9977	0.9978	0.9979	0.9979	0.9980	0.9981
2.90	0.9981	0.9982	0.9982	0.9983	0.9984	0.9984	0.9985	0.9985	0.9986	0.9986
3.00	0.9987	0.9987	0.9987	0.9988	0.9988	0.9989	0.9989	0.9989	0.9990	0.9990
3.10	0.9990	0.9991	0.9991	0.9991	0.9992	0.9992	0.9992	0.9992	0.9993	0.9993
3.20	0.9993	0.9993	0.9994	0.9994	0.9994	0.9994	0.9994	0.9995	0.9995	0.9995
3.30	0.9995	0.9995	0.9995	0.9996	0.9996	0.9996	0.9996	0.9996	0.9996	0.9997
3.40	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9997	0.9998

Table F.3.: Percentage Points $t_{\nu,\alpha}$ of the Student-t Distribution

$\nu \backslash \alpha$	0.1	0.05	0.025	0.01	0.005	0.0025	0.001	0.0005
1	3.078	6.314	12.706	31.821	63.657	127.321	318.309	636.619
2	1.886	2.920	4.303	6.965	9.925	14.089	22.327	31.599
3	1.638	2.353	3.182	4.541	5.841	7.453	10.215	12.924
4	1.533	2.132	2.776	3.747	4.604	5.598	7.173	8.610
5	1.476	2.015	2.571	3.365	4.032	4.773	5.893	6.869
6	1.440	1.943	2.447	3.143	3.707	4.317	5.208	5.959
7	1.415	1.895	2.365	2.998	3.499	4.029	4.785	5.408
8	1.397	1.860	2.306	2.896	3.355	3.833	4.501	5.041
9	1.383	1.833	2.262	2.821	3.250	3.690	4.297	4.781
10	1.372	1.812	2.228	2.764	3.169	3.581	4.144	4.587

$\nu\backslash\alpha$	0.1	0.05	0.025	0.01	0.005	0.0025	0.001	0.0005
11	1.363	1.796	2.201	2.718	3.106	3.497	4.025	4.437
12	1.356	1.782	2.179	2.681	3.055	3.428	3.930	4.318
13	1.350	1.771	2.160	2.650	3.012	3.372	3.852	4.221
14	1.345	1.761	2.145	2.624	2.977	3.326	3.787	4.140
15	1.341	1.753	2.131	2.602	2.947	3.286	3.733	4.073
16	1.337	1.746	2.120	2.583	2.921	3.252	3.686	4.015
17	1.333	1.740	2.110	2.567	2.898	3.222	3.646	3.965
18	1.330	1.734	2.101	2.552	2.878	3.197	3.610	3.922
19	1.328	1.729	2.093	2.539	2.861	3.174	3.579	3.883
20	1.325	1.725	2.086	2.528	2.845	3.153	3.552	3.850
21	1.323	1.721	2.080	2.518	2.831	3.135	3.527	3.819
22	1.321	1.717	2.074	2.508	2.819	3.119	3.505	3.792
23	1.319	1.714	2.069	2.500	2.807	3.104	3.485	3.768
24	1.318	1.711	2.064	2.492	2.797	3.091	3.467	3.745
25	1.316	1.708	2.060	2.485	2.787	3.078	3.450	3.725
26	1.315	1.706	2.056	2.479	2.779	3.067	3.435	3.707
27	1.314	1.703	2.052	2.473	2.771	3.057	3.421	3.690
28	1.313	1.701	2.048	2.467	2.763	3.047	3.408	3.674
29	1.311	1.699	2.045	2.462	2.756	3.038	3.396	3.659
30	1.310	1.697	2.042	2.457	2.750	3.030	3.385	3.646
31	1.309	1.696	2.040	2.453	2.744	3.022	3.375	3.633
32	1.309	1.694	2.037	2.449	2.738	3.015	3.365	3.622
33	1.308	1.692	2.035	2.445	2.733	3.008	3.356	3.611
34	1.307	1.691	2.032	2.441	2.728	3.002	3.348	3.601
35	1.306	1.690	2.030	2.438	2.724	2.996	3.340	3.591
40	1.303	1.684	2.021	2.423	2.704	2.971	3.307	3.551
45	1.301	1.679	2.014	2.412	2.690	2.952	3.281	3.520
50	1.299	1.676	2.009	2.403	2.678	2.937	3.261	3.496
∞	1.282	1.645	1.960	2.326	2.576	2.807	3.090	3.291

Table F.4.: Percentage Points $\chi^2_{\nu,\alpha}$ of the Chi-Square Distribution (ν) Degrees of Freedom

$\nu\backslash\alpha$	0.1	0.05	0.025	0.01	0.005	0.0025	0.001	0.0005
1	2.706	3.841	5.024	6.635	7.879	9.141	10.828	12.116
2	4.605	5.991	7.378	9.210	10.597	11.983	13.816	15.202
3	6.251	7.815	9.348	11.345	12.838	14.320	16.266	17.730
4	7.779	9.488	11.143	13.277	14.860	16.424	18.467	19.997
5	9.236	11.070	12.833	15.086	16.750	18.386	20.515	22.105
6	10.645	12.592	14.449	16.812	18.548	20.249	22.458	24.103
7	12.017	14.067	16.013	18.475	20.278	22.040	24.322	26.018
8	13.362	15.507	17.535	20.090	21.955	23.774	26.124	27.868
9	14.684	16.919	19.023	21.666	23.589	25.462	27.877	29.666
10	15.987	18.307	20.483	23.209	25.188	27.112	29.588	31.420
11	17.275	19.675	21.920	24.725	26.757	28.729	31.264	33.137
12	18.549	21.026	23.337	26.217	28.300	30.318	32.909	34.821
13	19.812	22.362	24.736	27.688	29.819	31.883	34.528	36.478
14	21.064	23.685	26.119	29.141	31.319	33.426	36.123	38.109
15	22.307	24.996	27.488	30.578	32.801	34.950	37.697	39.719
16	23.542	26.296	28.845	32.000	34.267	36.456	39.252	41.308
17	24.769	27.587	30.191	33.409	35.718	37.946	40.790	42.879
18	25.989	28.869	31.526	34.805	37.156	39.422	42.312	44.434
19	27.204	30.144	32.852	36.191	38.582	40.885	43.820	45.973
20	28.412	31.410	34.170	37.566	39.997	42.336	45.315	47.498
21	29.615	32.671	35.479	38.932	41.401	43.775	46.797	49.011
22	30.813	33.924	36.781	40.289	42.796	45.204	48.268	50.511
23	32.007	35.172	38.076	41.638	44.181	46.623	49.728	52.000
24	33.196	36.415	39.364	42.980	45.559	48.034	51.179	53.479
25	34.382	37.652	40.646	44.314	46.928	49.435	52.620	54.947
26	35.563	38.885	41.923	45.642	48.290	50.829	54.052	56.407
27	36.741	40.113	43.195	46.963	49.645	52.215	55.476	57.858
28	37.916	41.337	44.461	48.278	50.993	53.594	56.892	59.300
29	39.087	42.557	45.722	49.588	52.336	54.967	58.301	60.735

$\nu\backslash\alpha$	0.1	0.05	0.025	0.01	0.005	0.0025	0.001	0.0005
30	40.256	43.773	46.979	50.892	53.672	56.332	59.703	62.162
31	41.422	44.985	48.232	52.191	55.003	57.692	61.098	63.582
32	42.585	46.194	49.480	53.486	56.328	59.046	62.487	64.995
33	43.745	47.400	50.725	54.776	57.648	60.395	63.870	66.403
34	44.903	48.602	51.966	56.061	58.964	61.738	65.247	67.803
35	46.059	49.802	53.203	57.342	60.275	63.076	66.619	69.199
40	51.805	55.758	59.342	63.691	66.766	69.699	73.402	76.095
50	63.167	67.505	71.420	76.154	79.490	82.664	86.661	89.561
60	74.397	79.082	83.298	88.379	91.952	95.344	99.607	102.695
70	85.527	90.531	95.023	100.425	104.215	107.808	112.317	115.578
80	96.578	101.879	106.629	112.329	116.321	120.102	124.839	128.261
90	107.565	113.145	118.136	124.116	128.299	132.256	137.208	140.782
100	118.498	124.342	129.561	135.807	140.169	144.293	149.449	153.167

Table F.5.: Kolmogorov-Smirnov Test Critical Values

n	$D_{0.1}$	$D_{0.05}$	$D_{0.01}$
10	0.36866	0.40925	0.48893
11	0.35242	0.39122	0.46770
12	0.33815	0.37543	0.44905
13	0.32549	0.36143	0.43247
14	0.31417	0.34890	0.41762
15	0.30397	0.33760	0.40420
16	0.29472	0.32733	0.39201
17	0.28627	0.31796	0.38086
18	0.27851	0.30936	0.37062
19	0.27136	0.30143	0.36117
20	0.26473	0.29408	0.35241
25	0.23768	0.26404	0.31657
30	0.21756	0.24170	0.28987
35	0.20185	0.22425	0.26897

n	$D_{0.1}$	$D_{0.05}$	$D_{0.01}$
over 35	$1.22/\sqrt{n}$	$1.36/\sqrt{n}$	$1.63/\sqrt{n}$

See (Miller 1956). Additional values can be computed at:

`http://www.ciphersbyritter.com/JAVASCRP/NORMCHIK.HTM#KolSmir`

References

Ahrens, J., and V. Dieter. 1972. "Computer Methods for Sampling from the Exponential and Normal Distributions." *Communications of the Association for Computing Machinery* 15: 873–82.

Akaike, H. 1974. "A New Look at the Statistical Model Identification." *IEEE Transactions on Automatic Control* 19 (6): 716–23. https://doi.org/10.1109/TAC.1974.1100705.

Alexopoulos, C., and A. F. Seila. 1998. "Output Data Analysis." In *Handbook of Simulation*, edited by J. Banks. John Wiley & Sons, New York.

Askin, R. G., and J. B. Goldberg. 2002. *Design and Analysis of Lean Production Systems*. John Wiley & Sons.

Askin, R. G., and C. R. Standridge. 1993. *Modeling and Analysis of Manufacturing Systems*. John Wiley & Sons.

Axsäter, S. 2006. *Inventory Control*. Springer Science + Business Media.

Balci, O. 1997. "Principles of Simulation Model Validation, Verification, and Testing." *Transactions of the Society for Computer Simulation International*.

Balci, O. 1998. "Verification, Validation, and Testing." In *The Handbook of Simulation*. John Wiley & Sons.

Ballou, R. H. 2004. *Business Logistics/Supply Chain Management: Planning, Organizing, and Controlling the Supply Chain*. 5th ed. Prentice-Hall.

Banks, J. 1998. *Handbook of Simulation: Principles, Methodology, Advances, Applications, and Practice*. Edited by J. Banks. John Wiley & Sons.

Banks, J., B. Burnette, H. Kozloski, and J. Rose. 1995. *Introduction to SIMAN v and CINEMA v*. John Wiley & Sons.

Banks, J., J. Carson, B. Nelson, and D. Nicol. 2005. *Discrete-Event System Simulation*. 4th ed. Prentice Hall.

Ben-Ameur, Walid. 2004. "Computing the Initial Temperature of Simulated Annealing." *Computational Optimization and Applications* 29 (3): 369–85. https://doi.org/10.1023/B:COAP.0000044187.23143.bd.

Biller, B., and B. L. Nelson. 2003. "Modeling and Generating Multivarate Time- Series Input Processes Using a Vector Autogressive Technique." *Assoc. Comput. Mach. Trans. Modeling and Comput. Simul.* 13: 211–37.

Blanchard, B. S., and W. J Fabrycky. 1990. *Systems Engineering and Analysis*. Prentice- Hall.

Box, G. E. P., G. M. Jenkins, and G. C. Time Series Analysis Reinsel. 1994. *Forecasting and Control*. 3rd ed. Prentice Hall.

Box, G. E. P., and M. F. Muller. 1958. "Note on the Generation of Random Normal Deviates." *Annals of Mathematical Statistics* 29: 610–11.

Brooks, Stephen. 1998. "Markov Chain Monte Carlo Method and Its Application." *Journal of the Royal Statistical Society: Series D (The*

Statistician) 47 (1): 69–100.

Cario, M. C., and B. L. Nelson. 1996. "Autoregressive to Anything: Time Series Input Processes for Simulation." *Operations Research Letters* 19: 51–58.

Cario, M. C., and B. L. Nelson. 1998. "Numerical Methods for Fitting and Simulating Autoregressive-to-Anything Processes." *INFORMS Journal of Computing* 10: 72–81.

Casella, G., and R. Berger. 1990. *Statistical Inference*. Wadsworth & Brooks/Cole.

Cash, C. R., D. G. Dippold, J. M. Long, B. L. Nelson, and W. P. Pollard. 1992. "Evaluation of Tests for Initial Conditions Bias." In *Proceedings of the 1992 Winter Simulation Conference*, edited by J. J. Swain, D. Goldsman, R. C. Crain, and J. R. Wilson.

Chen, E. Jack, and W. David Kelton. 1999. "Simulation-Based Estimation of Quantiles." *Proceedings of the 31st Conference on Winter Simulation Simulation—a Bridge to the Future - WSC '99* (Phoenix, Arizona, United States) 1: 428–34. https://doi.org/10.1145/324138.324272.

Cheng, R. C. 1977. "The Generation of Gamma Variables with Nonintegral Shape Parameters." *Applied Statistics* 26 (1): 71–75.

Chopra, S., and Meindl. 2007. *Supply Chain Management: Strategy, Planning, and Operations*. 3rd ed. Prentice-Hall.

Chung, C. A. 2004. *Simulation Modeling Handbook: A Practical Approach*. CRC Press.

Cody, W. J. 1969. "Rational Chebyshev Approximations for the Error Function." *Mathematics of Computation* 23 (107): 631–37.

Command, Air Force Systems. 1991. *ASD Directorate of Systems Engineering and DSMC Technical Management Department*.

Cooper, R. B. 1990. *Introduction to Queueing Theory*. 3rd ed. CEEPress Books.

Davison AC, Hinkley DV. 1997. *Bootstrap Methods and Their Application*. Cambridge University Press.

De Boer, Pieter-Tjerk, Dirk P. Kroese, Shie Mannor, and Reuven Y. Rubinstein. 2005. "A Tutorial on the Cross-Entropy Method." *Annals of Operations Research* 134 (1): 19–67. https://doi.org/10.1007/s10479-005-5724-z.

Devroye, L. 1986. *Non-Uniform Random Variate Generation*. Springer-Verlag.

Efron, B., and R. Tibshirani. 1986. "Bootstrap Methods for Standard Errors, Confidence Intervals, and Other Measures of Statistical Accuracy." *Statistical Science* 1 (1): 54–75.

Efron, B., and R. Tibshirani. 1994. *An Introduction to the Bootstrap*. CRC Press.

Fishman, G. S. 2001. *Discrete-Event Simulation: Modeling, Programming, and Analysis*. Springer.

Fishman, G. S., and L. S. Yarberry. 1997. "An Implementation of the Batch Means Method." *INFORMS Journal on Computing* 9.

Fishman, George. 2006. *A First Course in Monte Carlo*. Thomson Brooks/Cole.

Fox, J., and S. Weisberg. 2018. *An r Companion to Applied Regression*.

SAGE Publications.

Gan, F., and K. J. Koehler. 1990. "Goodness-of-Fit Tests Based on p-p Probability Plots." *Technometrics* 32 (3): 289–303.

Geyer, Charles. 2011. *Handbook of Markov Chain Monte Carlo*. Edited by Galin Jones Steve Brooks Andrew Gelman and Xiao-Li Meng. Vol. 20116022. CRC Handbooks of Modern Statistical Methods. Chapman & Hall.

Glynn, P. W., and W. Whitt. 1989. "Extensions of the Queueing Relation and $L = \lambda W$ and $H = \lambda G$." *Operations Research* 37: 634–44.

Goldsman, D., and B. L. Nelson. 1998. "Comparing Systems via Simulation." In *Handbook of Simulation*, edited by J. Banks. John Wiley & Sons.

Gross, D., and C. M. Harris. 1998. *Fundamentals of Queueing Theory*. 3rd ed. John Wiley & Sons.

Gross, D., J. F. Shortle, J. M. Thompson, and C. M. Harris. 2008. *Fundamentals of Queueing Theory*. John Wiley & Sons.

Hadley, G., and T. M. Whitin. 1963. *Analysis of Inventory Systems*. Prentice Hall.

Henderson, Shane G. 2021. "Reflections On Simulation Optimization." *2021 Winter Simulation Conference (WSC)* (Phoenix, AZ, USA), December, 1–15. https://doi.org/10.1109/WSC52266.2021.9715361.

Henriksen, J., and T. Schriber. 1986. "Simplified Approaches to Modeling Accumulating and Non-Accumulating Conveyor Systems." In *Proceedings of the 1986 Winter Simulation Conference*. Institute of Electrical; Electronic Engineers.

Holland, John H. 1975. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. University of Michigan Press.

Hong, L. Jeff, and Barry L. Nelson. 2006. "Discrete Optimization via Simulation Using COMPASS." *Operations Research* 54 (1): 115–29. https://doi.org/10.1287/opre.1050.0237.

Hull, T. E., and A. R. Dobell. 1962. "Random Number Generators." *SIAM Review* 4: 230–54.

Jian, Nanjing, and Shane G. Henderson. 2015. "An Introduction to Simulation Optimization." *Proceedings of the 2015 Winter Simulation Conference* (Huntington Beach, California), WSC '15, 1780–94.

Jones, Donald R., Matthias Schonlau, and William J. Welch. 1998. "Efficient Global Optimization of Expensive Black-Box Functions." *Journal of Global Optimization* 13 (4): 455–92. https://doi.org/10.1023/A:1008306431147.

Kelton, W. D., R. P. Sadowski, and D. T. Sturrock. 2004. *Simulation with Arena*. 3rd ed. McGraw-Hill.

Kendall, D. G. 1953. "Stochastic Processes Occurring in the Theory of Queues and Their Analysis by the Method of Imbedded Markov Chains." *Annals of Mathematical Statistics* 24: 338–54.

Kennedy, James, and Russell Eberhart. 1995. "Particle Swarm Optimization." *Proceedings of ICNN'95 - International Conference on Neural Networks* (Perth, WA, Australia) 4: 1942–48. https://doi.org/10.1109/ICNN.1995.488968.

Kingman, J. F. C. 1964. *The Heavy Traffic Approximation in the Theory of Queues*. Proceedings of the Symposium On Congestion Theory.

Kirkpatrick, S, C D Gelatt, and M P Vecchi. 1983. "Optimization by Simulated Annealing." *Science* 220 (4598): 671–80. https://doi.org/10.1126/science.220.4598.6.

Kleinrock, L. 1975. *Queueing Systems*. Vol. 1. John Wiley & Sons.

L'Ecuyer, P., R. Simard, and W. D. Kelton. 2002. "An Object-Oriented Random Number Package with Many Long Streams and Substreams." *Operations Research* 50: 1073–75.

Lada, E. K., J. R. Wilson, and N. M. Steiger. 2003. *A Wavelet-Based Spectral Method for Steady-State Simulation Analysis*. Proceedings of the 2003 Winter Simulation Conference.

Law, A. 2007. *Simulation Modeling and Analysis*. 4th ed. McGraw-Hill.

Leemis, L. 1991. "Nonparametric Estimation of the Cumulative Intensity Function for a Nonhomogeneous Poisson Process." *Management Science* 37 (7): 886–900.

Leemis, L. M., and S. K. Park. 2006. *Discrete-Event Simulation: A First Course*. Prentice-Hall.

Levina, E., and P. Bickel. 2001. "The Earth Mover's Distance Is the Mallows Distance: Some Insights from Statistics." *Proceedings Eighth IEEE International Conference on Computer Vision. ICCV 2001* 2: 251–256 vol.2. https://doi.org/10.1109/ICCV.2001.937632.

Little, J. D. C. 1961. "A Proof for the Queuing Formula $L = \lambda W$." *Operations Research* 9: 383–87.

Litton, J. R., and C. H. Harmonosky. 2002. *A Comparison of Selective Initialization Bias Elimination Methods*. Proceeding of the 2002 Winter Simulation Conference.

Livny, M., B. Melamed, and A. K. Tsiolis. 1993. "The Impact of Autocorrelation on Queueing Systems." *Management Science* 39 (3): 322–39.

Matejcik, F. J., and B. L. Nelson. 1995. "Two-Stage Multiple Comparisons with the Best for Computer Simulation." *Operations Research* 43 (4): 633–40.

Miller, L. H. 1956. "Table of Percentage Points of Kolmogorov Statistics." *Journal of the American Statistical Association*, 111–21.

Montgomery, D. C., and G. C. Runger. 2006. *Applied Statistics and Probability for Engineers*. 4th ed. John Wiley & Sons.

Muckstadt, J. A., and A. Sapras. 2010. *Principles of Inventory Management: When You Are down to Four, Order More*. Springer.

Nadarajah, Saralees, Emmanuel Afuecheta, and Stephen Chan. 2017. "A Compendium of Copulas." *Statistica* 77 (4): 279–328. https://doi.org/10.6092/issn.1973-2201/7202.

Nahmias, S. 2001. *Production and Operations Analysis*. 4th ed. McGraw-Hill.

Nelsen, Roger B. 2004. "Properties and Applications of Copulas : A Brief Survey." https://api.semanticscholar.org/CorpusID:2508363.

Nelson, B. L. 1990. "Control Variate Remedies." *Operations Research* 38 (6): 974–92.

Nelson, B. L. 1995. *Stochastic Modeling Analysis and Simulation*. McGraw-Hill.

Nelson, B. L., and L. Pei. 2021. *Foundations and Methods of Stochastic Simulation: A First Course*. Vol. 316. International Series in

Operations Research & Management Science. Springer International Publishing.

Nelson, B. L., J. Swan, D. Goldsman, and W. Song. 2001. "Simple Procedures for Selecting the Best Simulated System When the Number of Alternatives Is Large." *Operations Research* 49 (6): 950–63.

Nelson, Barry L. 2014. "Optimization via Simulation Over Discrete Decision Variables." *TutORials in Operations Research. Models, Methods, and Applications for Innovative Decision Making*, October, 193–207. https://doi.org/10.1287/educ.1100.0069.

Nelson, Roger B. 1999. *An Introduction to Copulas*. ISBN o-387-98623-5. Springer-Verlag.

Park, Chuljin, and Seong-Hee Kim. 2015. "Penalty Function with Memory for Discrete Optimization via Simulation with Stochastic Constraints." *Operations Research* 63 (5): 1195–212. https://doi.org/10.1287/opre.2015.1417.

Pasupathy, Raghu, and Soumyadip Ghosh. 2013. "Simulation Optimization: A Concise Overview and Implementation Guide." In *Theory Driven by Influential Applications*, edited by Huseyin Topaloglu, J. Cole Smith, and Harvey J. Greenberg. INFORMS. https://doi.org/10.1287/educ.2013.0118.

Patuwo, B. E., R. L. Disney, and D. C. Mcnickle. 1993. "The Effect of Correlated Arrivals on Queues." *IIE Transactions* 25 (3): 105–10.

Pegden, C. D., R. E. Shannon, and R. P. Sadowski. 1995. *Introduction to Simulation Using SIMAN*. 2nd ed. McGraw-Hill.

Pishro-Nik, H. 2014. *Introduction to Probability, Statistics, and Random Processes*. Kappa Research LLC.

Ravindran, A., D. Phillips, and J Solberg. 1987. *Operations Research Principles and Practice*. 2nd ed. John Wiley & Sons.

Ripley, B. D. 1987. *Stochastic Simulation*. John Wiley & Sons Inc.

Robert, Christian P., and Wu Changye. n.d. "Markov Chain Monte Carlo Methods, a Survey with Some Frequent Misunderstandings." http://arxiv.org/abs/2001.06249.

Robinson, S. 2005. "Automated Analysis of Simulation Output Data." *Proceedings of the 2005 Winter Simulation Conference* (Piscataway, New Jersey) 763-770.

Ross, S. 1997. *Introduction to Probability Models*. 6th ed. Academic Press.

Ross, Sheldon M. 2023. *Simulation (6th Edition)*. Elsevier.

Rossetti, M. D. 2008. "JSL: An Open-Source Object-Oriented Framework for Discrete-Event Simulation in Java." *International Journal of Simulation and Process Modeling* 4 (1): 69–87.

Rossetti, M. D. 2015. *Simulation Modeling and Arena*. John Wiley & Sons.

Rossetti, M. D., and P. J. Delaney. 1995. *Control of Initialization Bias in Queueing Simulations Using Queueing Approximations*. Institute of Electrical; Electronics Engineers.

Rossetti, M. D., and Z. Li. 2005. "Exploring Exponentially Weighted Moving Average Control Charts to Determine the Warm-up Period." In *Proceedings of the 2005 Winter Simulation Conference*, edited by M. E. Kuhl, N. M. Steiger, F. B. Armstrong, and J. A. Joines. Institute of Electrical; Electronics Engineers.

Rossetti, M. D., and F. Seldanari. 2001. "Multi-Objective Analysis of

Hospital Delivery Systems." *Computers and Industrial Engineering* 41: 309–33.

Rubinstein, R., and D. Kroese. 2017. *Simulation and the Monte Carlo Method*. John Wiley & Sons Inc.

Rumbaugh, J. R., M. R. Blaha, L. Lorenson, F. Eddy, and W. Premerlani. 1991. *Object-Oriented Modeling and Design*. Prentice Hall.

Schmeiser, B. W. 1982. "Batch Size Effects in the Analysis of Simulation Output." *Operations Research* 30: 556–68.

Schwarz, Gideon. 1978. "Estimating the Dimension of a Model." *The Annals of Statistics* 6 (2): 461–64. https://doi.org/10.1214/aos/1176344136.

Silver, E. A., D. F. Pyke, and R. Peterson. 1998. *Inventory Management and Production Planning and Scheduling*. 3rd ed. John Wiley & Sons.

Singh, N. 1996. *Systems Approach to Computer-Integrated Design and Manufacturing*. John Wiley & Sons.

Sklar, A. 1959. "Fonctions de r´Epartition a´ n Dimensions Et Leurs Marges." *Publ. Inst. Statist. Univ. Paris* 8: 229–31.

Soto, J. 1999. "Statistical Testing of Random Numbers." *Proceedings of the 22nd National Information Systems Security Conference*.

Spall, James C. 2012. "Stochastic Optimization." In *Handbook of Computational Statistics*, edited by James E. Gentle, Wolfgang Karl Härdle, and Yuichi Mori. Springer Berlin Heidelberg. https://doi.org/10.1007/978-3-642-21551-3_7.

Speagle, Joshua S. 2020. "A Conceptual Introduction to Markov Chain

Monte Carlo Methods." http://arxiv.org/abs/1909.12313.

Stecke, K. E. 1992. "Machine Interference: Assignment of Machines to Operators." Chap. 57 in *Handbook of Industrial Engineering*, edited by G. Salvendy. John-Wiley & Sons.

Steiger, N. M., and J. R. Wilson. 2002. "An Improved Batch Means Procedure for Simulation Output Analysis." *Management Science* 48.

Tijms, H. C. 2003. *A First Course in Stochastic Models*. John-Wiley & Sons.

Van Ravenzwaaij, Pete Cassey, Don, and Scott D. Brown. 2018. "A Simple Introduction to Markov Chain Monte–Carlo Sampling." *Psychonomic Bulletin & Review* 25 (1): 143–54.

Vose, David. 2010. "Fitting Distributions to Data." https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=19631811c7fd08cac567a4ee886acae6d82e8f3f.

Wang, Honggang, Raghu Pasupathy, and Bruce W. Schmeiser. 2013. "Integer-Ordered Simulation Optimization Using R-SPLINE: Retrospective Search with Piecewise-Linear Interpolation and Neighborhood Enumeration." *ACM Transactions on Modeling and Computer Simulation (TOMACS)* 23 (3): 17–24. https://doi.org/10.1145/2499913.2499916.

Wehrens, Hein Putter, Ron, and Lutgarde M. C Buydens. 2000. "The Bootstrap: A Tutorial." *Chemometrics and Intelligent Laboratory Systems* 54 (1): 35–52.

Welch, P. D. 1983. "A Graphical Approach to the Initial Transient Problem in Steady State Simulation." *10th IMACS World Congress on System Simulation and Scientific Computation*, 219–21.

White, K. P., M. J. Cobb, and S. C. Spratt. 2000. *A Comparison of Five Steady-State Truncation Heuristics for Simulation.* Proceeding of the 2000 Winter Simulation Conference.

Whitt, W. 1983. "The Queueing Network Analyzer." *The Bell System Technical Journal* 62 (9): 2779–815.

Whitt, W. 1989. "Planning Queueing Simulations." *Management Science* 35 (11): 1341–66.

Whitt, W. 1993. "Approximations for the GI/G/m Queue." *Productions and Operations Management* 2 (2): 114–61.

Williams, C. Arthur. 1950. "On the Choice of the Number and Width of Classes for the Chi-Square Test of Goodness of Fit." *Journal of the American Statistical Association* 45 (249): 77–86.

Wilson, J. R., and A. A. B. Pritsker. 1978. "A Survey of Re-Search on the Simulation Startup Problem." *Simulation.*

Xu, Jie, Barry L. Nelson, and L. Jeff Hong. 2010. "Industrial Strength COMPASS: A Comprehensive Algorithm and Software for Optimization via Simulation." *ACM Transactions on Modeling and Computer Simulation* 20 (1): 1–29. https://doi.org/10.1145/1667072.1667075.

Zipkin, P. H. 2000. *Foundations of Inventory Management.* McGraw-Hill.